

Contents

Front matter	3
OpenUSD Development from Zero	
How to Use This Book	4
The NCP-USD Exam	7
The 14-Day Study Plan	11
Setting Up Your Lab	13
Part I — OpenUSD Fundamentals	17
Chapter 1Chapter 1 — What Is OpenUSD?..	17
Chapter 2Chapter 2 — The Stage	28
Chapter 3Chapter 3 — Layers	36
Chapter 4Chapter 4 — Prims and Prim Paths	46
Chapter 5Chapter 5 — Properties: Attributes, Relationships, Metadata	55
Chapter 6Chapter 6 — Schemas and Model Kinds	62
Chapter 7Chapter 7 — USD File Formats and Reading USDA	70
Part II — Data Modeling	77
Chapter 8Chapter 8 — Usd vs. Sdf: Two Ways to See Data	77
Chapter 9Chapter 9 — Value Types	84
Chapter 10Chapter 10 — Time Samples and Animation Data	94
Chapter 11Chapter 11 — Primvars	104
Chapter 12Chapter 12 — Metadata in Depth	113
Chapter 13Chapter 13 — Built-in Schemas, Xforms, and Extents	120
Part III — Composition	127
Chapter 14Chapter 14 — Composition Fundamentals	127
Chapter 15Chapter 15 — Sublayers, Layer Stacks, and Edit Targets	151
Chapter 16Chapter 16 — References	170
Chapter 17Chapter 17 — Payloads	192
Chapter 18Chapter 18 — Variant Sets	208
Chapter 19Chapter 19 — Inherits, Classes, and Specializes	230
Chapter 20Chapter 20 — Overs and Relocates	247
Chapter 21Chapter 21 — LIVERPS and Value Resolution	263
Chapter 22Chapter 22 — Composition Design and Debugging	282
Part IV — Content Aggregation	312
Chapter 23Chapter 23 — Asset Structure and Model Hierarchy	312
Chapter 24Chapter 24 — Native Scenegraph Instancing	332
Chapter 25Chapter 25 — Point Instancing	348
Chapter 26Chapter 26 — Large Scene Optimization	366

Part V — Data Exchange	385
Chapter 27Chapter 27 — Data Exchange Concepts	385
Chapter 28Chapter 28 — Units, Axes, and Naming	405
Chapter 29Chapter 29 — Writing Exporters, Converters, and Importers	420
Chapter 30Chapter 30 — Validating USD Assets	437
Chapter 31Chapter 31 — File Formats in Depth	448
Part VI — Pipeline Development	458
Chapter 32Chapter 32 — Pipeline Architecture and Asset Management	458
Chapter 33Chapter 33 — Asset Resolution	478
Chapter 34Chapter 34 — Exporter Hooks, Flattening, and Change Processing	495
Chapter 35Chapter 35 — Build Configurations and DCC Integration	510
Part VII — Customizing USD	521
Chapter 36Chapter 36 — The Plugin System	521
Chapter 37Chapter 37 — Custom Schemas	532
Chapter 38Chapter 38 — Kinds, Variant Fallbacks, File Formats, Resolvers, Scene Indices	546
Part VIII — Visualization	559
Chapter 39Chapter 39 — UsdGeom: Geometry, Transforms, Cameras	559
Chapter 40Chapter 40 — UsdShade: Materials and Shaders	580
Chapter 41Chapter 41 — UsdLux and Rendering	603
Part IX — Debugging and Performance	617
Chapter 42Chapter 42 — Stage Introspection and Composition Debugging	617
Chapter 43Chapter 43 — Diagnostics and Profiling Tools	643
Chapter 44Chapter 44 — Performance: Load and Render Times	658
Part X — Python Labs	675
Lab 01Lab 01 — Install and verify usd-core	675
Lab 02Lab 02 — Your first stage: Xform + Sphere, save as USDA	675
Lab 03Lab 03 — Open and traverse a stage	676
Lab 04Lab 04 — Root layer, session layer, export to string	677
Lab 05Lab 05 — Build a sublayer stack and see who wins	678

Lab 06Lab 06 — Attributes and value types	679	Lab 35Lab 35 — UsdLux lights, LightAPI, intensity × 2^exposure	706
Lab 07Lab 07 — Relationships and targets	680	Lab 36Lab 36 — Debugging toolkit: delegate, Trace, MallocTag	707
Lab 08Lab 08 — Metadata, customData, assetInfo, stage metadata.....	680	Lab 37Lab 37 — Capstone: publish a component, assemble, validate, inventory .	708
Lab 09Lab 09 — Authoring with Sdf specs + Sdf.ChangeBlock	681	USDA Reading Workbook	710
Lab 10Lab 10 — Time samples, Get() vs Get(t), linear vs held	682	USDA reading exercises	710
Lab 11Lab 11 — Mesh topology, subdivisionScheme, and extent.....	683	Question bank.....	719
Lab 12Lab 12 — Indexed vertex displayColor primvar	684	Question Bank — Fundamentals (FUN)	719
Lab 13Lab 13 — defaultPrim, explicit paths, UnresolvedPrimPath, internal refs	685	Question Bank — Composition (COMP)	722
Lab 14Lab 14 — Time-offset references (Sdf.LayerOffset)	686	Question Bank — Data Exchange (DE).....	738
Lab 15Lab 15 — Payloads, LoadNone, and OpenMasked	687	Question Bank — Pipeline Development (PD)	746
Lab 16Lab 16 — Variant sets and GetVariantEditContext	688	Question Bank — Data Modeling (DM)	753
Lab 17Lab 17 — Classes, inherits, and a local override	688	Question Bank — Debugging and Troubleshooting (DBG)	759
Lab 18Lab 18 — Specializes vs inherits vs references	689	Question Bank — Content Aggregation (CA)	764
Lab 19Lab 19 — Edit targets, EditContext, and offset mapping	690	Question Bank — Visualization (VIS).....	769
Lab 20Lab 20 — LIVERPS laboratory.....	691	Question Bank — Customizing USD (CUST)	773
Lab 21Lab 21 — Composition introspection toolkit.....	692	Mock exams	776
Lab 22Lab 22 — Component, assembly, group, and a broken model chain	693	Mock Exam 1 — NCP-OUSD Practice Paper	776
Lab 23Lab 23 — Native instancing: prototypes, proxies, de-instance	694	Mock Exam 1 — Answer Key	789
Lab 24Lab 24 — PointInstancer: append a prototype, hide one instance	695	Mock Exam 2 — NCP-OUSD Practice Paper	794
Lab 25Lab 25 — JSON/OBJ → USD converter	696	Mock Exam 2 — Answer Key	805
Lab 26Lab 26 — Units and up-axis: USD does not convert.....	697	Mock Exam 3 — NCP-OUSD Practice Paper	810
Lab 27Lab 27 — Asset validator (UsdValidation)	698	Mock Exam 3 — Answer Key	820
Lab 28Lab 28 — Dependencies, Flatten vs FlattenLayerStack, USDZ.....	699	Flashcards	824
Lab 29Lab 29 — Asset resolution: anchor, search path, stage context.....	700	Flashcards	824
Lab 30Lab 30 — ObjectsChanged, ChangeBlock, and the DefinePrim trap	701	Cheat sheets.....	831
Lab 31Lab 31 — Codeless custom schema (schema.usda vs registry)	702	Cheat sheet — LIVERPS & value resolution	831
Lab 32Lab 32 — Kind registry and an unregistered custom kind.....	703	Cheat sheet — USDA syntax.....	833
Lab 33Lab 33 — Xform ops, XformCache, and Gf.Camera	704	Cheat sheet — Python API (usd-core 26.8)	836
Lab 34Lab 34 — PreviewSurface and MaterialBindingAPI.....	705	Cheat sheet — List editing	839
		Cheat sheet — Primvars & interpolation	841
		Cheat sheet — Instancing decisions	843
		Cheat sheet — Debugging toolkit	845
		Cheat sheet — File formats & CLI tools	847
		Cheat sheet — Schemas & kinds	849
		Cheat sheet — NCP-OUSD domains (one-pager)	851
		Final certification checklist	853
		Back matter	856
		Glossary.....	856
		Objective coverage matrix.....	863
		References and further reading	865

Front matter

OpenUSD Development from Zero

A 14-Day Study Book for the NVIDIA-Certified Professional: OpenUSD Development (NCP-OUSD) Exam

Edition: 1.0 (draft) · **Prepared:** October 2026

Verified against: OpenUSD **26.08** (usd-core 26.8, Python 3.12)

Exam blueprint verified: 7 October 2026, against the official NVIDIA certification page and the *NCP-OUSD Exam Study Guide* v1.1.0 (October 2025)

Disclaimer

This is an **independent study book**. It is not written, reviewed, endorsed, or sponsored by NVIDIA, Pixar Animation Studios, or the Alliance for OpenUSD.

KEY

****All practice questions, USDA exercises, flashcards, and mock exams in this book are original material written for study purposes. None of them are actual NVIDIA exam questions, and none reproduce exam content.**** They are designed to test the same official objectives in a similar style.

- Exam facts (format, price, domain weights) were correct when verified on the date above. NVIDIA can change them at any time. Check the official certification page before you book.
- Code examples were run against the OpenUSD version shown above. Other versions may behave differently; the book marks known differences with **VERSION** boxes.
- OpenUSD, USD, and related names belong to their respective owners. NVIDIA and NCP-OUSD are trademarks of NVIDIA Corporation. All other trademarks belong to their owners.

About this book

You do not need any OpenUSD experience to start. You do need basic Python: variables, lists, functions, for loops, and running a script from a terminal. Everything else is taught from the beginning.

The book works **offline**. Every chapter contains its own explanations, examples, and answers. URLs appear only in “Further reading” lists and in the References section, for readers who want more depth.

How to Use This Book

What is inside

Part	Content	Exam weight
Front matter	Exam overview, 14-day plan, lab setup	—
I	OpenUSD fundamentals (Ch 1-7)	Not scored directly; everything depends on it
II	Data modeling (Ch 8-13)	13%
III	Composition (Ch 14-22)	23%
IV	Content aggregation (Ch 23-26)	10%
V	Data exchange (Ch 27-31)	15%
VI	Pipeline development (Ch 32-35)	14%
VII	Customizing USD (Ch 36-38)	6%
VIII	Visualization (Ch 39-41)	8%
IX	Debugging and performance (Ch 42-44)	11%
X	Python labs, USDA workbook, question bank, mock exams, flashcards, cheat sheets, final checklist	—
Back matter	Glossary, references, objective coverage matrix	—

Read Parts I-IX in order. Later chapters assume earlier ones. Part X is used throughout the 14 days according to the schedule.

How every concept is taught

Each concept section follows the same 13 steps, always in this order. Once you know the pattern, you can find what you need quickly, for example by skipping straight to “Exam traps” during revision.

Step	Heading	What you get
1	What is it?	A one- or two-sentence definition
2	Why do we need it?	The problem it solves
3	Beginner explanation	An everyday analogy, and where the analogy breaks
4	Technical explanation	Precise rules and the OpenUSD classes involved
5	Mental model	One picture or sentence to remember in the exam
6	Simple example	The smallest possible case
7	USDA example	The concept written in USD’s text format
8	Python example	A complete runnable script and its real output
9	Real-world use case	How studios and companies use it
10	Common mistakes	Errors people make, and the fix
11	Exam traps	Wording and look-alike answers to watch for
12	Practice questions	2-3 questions with answers directly after
13	Exam takeaways	3-5 points to memorize

Each chapter starts with learning goals and key terms. It ends with a summary, a “If you see... → think...” table, 8-15 review questions, and their answers.

Boxes used in this book

Boxes are told apart by their **label and border style**, not by color, so they work in black-and-white print.

EXAM TIP

Advice about what the exam emphasizes or how to approach a question type.

TRAP

A common way exam questions mislead you.

MISTAKE

An error people often make in real code or files, and how to fix it.

VERSION

Behavior that differs between OpenUSD versions. The book is verified on USD 26.08.

KEY

A fact you must memorize.

NOTE

Optional extra depth. Safe to skip on a first read.

Code conventions

- **Python** examples are complete scripts: imports included, nothing hidden. Each runnable example is followed by **Expected output**, the real output from USD 26.08.
- **USDA** examples are complete text layers starting with `#usda 1.0`. When an example uses several files, each starts with a caption such as *File: chair.usda*.
- A code block labelled **not run automatically** needs something extra, such as command-line arguments, a C++ compiler, or a tool not included in `usd-core`. The text explains why.
- OpenUSD's documentation is written in C++ terms. The Python name is usually the same without the prefix: C++ `UsdStage::Open()` is Python `Usd.Stage.Open()`, and C++ `SdfPath` is Python `Sdf.Path`.

C++ name	Python name
<code>UsdStage</code>	<code>Usd.Stage</code>
<code>UsdPrim</code>	<code>Usd.Prim</code>
<code>SdfLayer</code>	<code>Sdf.Layer</code>
<code>SdfPath</code>	<code>Sdf.Path</code>
<code>UsdGeomMesh</code>	<code>UsdGeom.Mesh</code>
<code>GfVec3f</code>	<code>Gf.Vec3f</code>
<code>VtArray<int></code>	<code>Vt.IntArray</code>
<code>TfToken</code>	a plain Python <code>str</code>

Lab difficulty

☆☆☆ guided, short · ★★★ some problem-solving · ★★★★★ open-ended, combines several chapters

How to study with this book

1. **Read actively.** Before reading a “Python example”, predict its output. Then compare.
2. **Type the code.** Typing the labs yourself teaches far more than reading them.
3. **Answer before you look.** Cover the answers to practice questions with a sheet of paper.
4. **Track weak areas.** After every checkpoint and mock exam, write your per-domain scores in the 14-day tracker (F4). Re-read the chapters for any domain under 70%.
5. **Use the cheat sheets last.** They are for revision, not first learning.

EXAM TIP

Composition is 23% of the exam, and it shows up again inside debugging, pipeline, and content-aggregation questions. If you have extra time anywhere, spend it on Part III.

The NCP-OUUSD Exam

Exam facts

Verified on 7 October 2026 against the official NVIDIA certification page. Check again before booking.

Fact	Value
Full name	NVIDIA-Certified Professional: OpenUSD Development (NCP-OUUSD)
Level	Professional (NVIDIA describes it as intermediate-level)
Number of questions	60–70
Time limit	120 minutes
Delivery	Online, remotely proctored. You need a Certiverse account to access the exam
Price	US \$200
Language	English
Validity	2 years from issue. Recertify by retaking the exam
Credential	Digital badge and optional certificate
Passing score	Not published on the certification page

Who it is for. NVIDIA describes the candidate as a developer who builds, maintains, and optimizes 3D content pipelines with OpenUSD. The stated prerequisite is two to three years of OpenUSD plus Python or C++ experience, **or** completion of the study-guide materials. This book is designed to cover that study-guide material from zero.

The blueprint

The exam has eight scored domains. The weight is the share of questions from that domain.

Domain	Weight	≈ Questions (of 65)	Chapters
Composition	23%	15	14–22
Data Exchange	15%	10	27–31
Pipeline Development	14%	9	32–35
Data Modeling	13%	8	8–13
Debugging and Troubleshooting	11%	7	42–44
Content Aggregation	10%	7	23–26
Visualization	8%	5	39–41
Customizing USD	6%	4	36–38

KEY

Composition (23%) is almost a quarter of the exam. Data Exchange and Pipeline Development together add another 29%.

Share of exam (each # = 1%)

Composition	#####	23%
Data Exchange	#####	15%
Pipeline Development	#####	14%
Data Modeling	#####	13%
Debugging	#####	11%
Content Aggregation	#####	10%

Visualization	#####	8%
Customizing USD	#####	6%

Official objectives

NVIDIA's study guide lists specific tasks under each domain. They are summarized below, with the chapters that teach them. Numbers follow NVIDIA's numbering (domains in the guide's alphabetical order). Some objectives appear under two domains; the book teaches them once and cross-references them.

1. Composition (23%)

Obj	You should be able to...	Ch
1.1	Change the strength of an opinion	14, 19, 21
1.2	Choose an instancing style for data at different scales	22, 26
1.3	Compare references, payloads, and sublayers, and know when to use each	3, 15-17
1.4	Reuse the same animation on several elements at different time offsets	10, 16
1.5	Design layering strategies for multi-user workflows	15, 22
1.6	Explain LIVERPS in simple terms	14, 21
1.7	Know when variants are, and are not, the right way to structure assets	18
1.8	Find why an opinion from one layer does not take effect	20-22
1.9	Prepare an internal asset for delivery to external parties	22, 34
1.10	Remove properties from instanced component prims in an assembly	22, 24
1.11	Split a monolithic asset into collaborative workstreams	22

2. Content Aggregation (10%)

Obj	You should be able to...	Ch
2.1	Add a new prototype to a PointInstancer	25
2.2	Change the color of an instance-proxy mesh without breaking instancing	24
2.3	Hide PointInstancer instances efficiently	25
2.4	Use instances for efficient reuse in large scenes	23, 24, 26
2.5	Remove properties from instanced component prims in an assembly	24

3. Customizing USD (6%)

Obj	You should be able to...	Ch
3.1	Build a USD plugin against a given USD version	35, 36
3.2	Build USD from source with custom dependencies	35
3.3	Create custom model kinds when appropriate	6, 38
3.4	Create custom schemas for proprietary data	6, 37
3.5	Integrate a custom asset resolver	33, 38
3.6	Use schemas for nonstandard data during import/export	37
3.7	Write a Hydra scene index plugin that generates renderable geometry	38
3.8	Write an asset resolver that generates in-memory renderable prims	38

4. Data Exchange (15%)

Obj	You should be able to...	Ch
4.1	Convert USD assets to common 3D formats (such as glTF) with fidelity	27
4.2		27

Obj	You should be able to...	Ch
	Document conceptual mappings between USD and another data model (such as MaterialX)	
4.3	Explain USDC vs. USDA trade-offs	7, 31
4.4	Implement a round-trip pipeline between a DCC and USD	27, 35
4.5	Use schemas for nonstandard data during import/export	29, 37
4.6	Write a validator for assets exported from a DCC	30
4.7	Write an exporter or converter to USD	28, 29
4.8	Write or extend a USD importer in a DCC	29, 35

5. Data Modeling (13%)

Obj	You should be able to...	Ch
5.1	Add a primvar to a mesh	11
5.2	Choose the right value types for attribute data	9, 10
5.3	Represent custom metadata	5, 12
5.4	Retrieve the properties of a prim	2, 4, 5, 8
5.5	Understand what causes unexpected visual results	13
5.6	Update a mesh's extent after changing its points	13

6. Debugging and Troubleshooting (11%)

Obj	You should be able to...	Ch
6.1	Know when SdfChangeBlock relieves performance bottlenecks	8, 34, 44
6.2	Find why an opinion from one layer does not take effect	20, 42
6.3	Resolve asset-management issues	33, 42
6.4	Understand what causes unexpected visual results	13, 42, 44
6.5	Use diagnostics and profiling tools (TfDebug, diagnostic delegates, Trace, TfMallocTag)	43

7. Pipeline Development (14%)

Obj	You should be able to...	Ch
7.1	Convert USD assets to common 3D formats with fidelity	27
7.2	Document asset structure guidelines	23, 32
7.3	Explain USDC vs. USDA trade-offs	7, 31
7.4	Implement round-trip pipelines between a DCC and USD	35
7.5	Integrate custom resolvers to manage asset paths	33
7.6	Represent custom metadata	12
7.7	Validate that asset paths are formatted correctly	33
7.8	Write or extend a USD importer in a DCC	35

8. Visualization (8%)

Obj	You should be able to...	Ch
8.1	Add a primvar to a mesh	11, 39
8.2	Assign UsdPreviewSurface materials	40
8.3	Bind materials to a mesh	40
8.4	Build a UsdPreviewSurface network that reads diffuse color from a primvar	40

What the questions look like

NVIDIA's study guide includes sample questions. They show these styles, which this book's practice material copies in **format only**:

- **Single answer:** choose one of four options (A-D).
- **Multiple select:** "Select two options" or "Select three options". The question always tells you how many to pick.
- **USDA reading:** one or more short .usda files are shown, and you must work out the composed result, such as "What is the final value of this attribute?" or "Why doesn't this variant selection take effect?"
- **Scenario:** a pipeline or debugging situation where you choose the best design or explanation.

TRAP

In USDA-reading questions, check **every** file shown, including which file is the root layer and the order of arcs inside `prepend references = [...]`. One overlooked local opinion usually decides the answer.

Exam-day strategy

- **Pace:** 120 minutes for 60–70 questions is about **1 min 45 s per question**. Aim to finish a first pass by minute 90.
- **First pass:** answer what you know. Flag anything that needs long USDA tracing and come back to it.
- **USDA questions:** write the layer stack and arcs on your scratch area (if the proctoring rules allow it) and apply LIVERPS step by step (Chapter 21).
- **Multiple select:** pick exactly the number asked. Each option should be true *on its own*.
- **Eliminate:** distractors often name real APIs used in the wrong place, or swap "stronger" and "weaker". Rule those out first.
- **Logistics:** read NVIDIA's and the proctoring provider's rules before exam day: ID, system check, a quiet room with a clear desk, and a stable internet connection. Requirements can change, so follow the current official instructions.

EXAM TIP

Several objectives appear in two domains (for example "why doesn't an opinion take effect" is 1.8 and 6.2). Mastering one skill earns marks in both.

The 14-Day Study Plan

How the plan works

- **About 4 hours per day:** roughly 2.5 hours reading, 1 hour on labs, and 30 minutes on questions and flashcards.
- **Days 1-4** build the foundations everything else depends on.
- **Days 5-7** are all composition, the largest domain (23%).
- **Days 8-12** cover the remaining domains, roughly in proportion to their weight.
- **Days 12-14** each include a full 120-minute mock exam.

If you have less time per day, stretch the plan rather than skipping labs. Labs are where the concepts stick.

Daily routine

1. **Warm-up (10 min):** review yesterday's flashcards.
2. **Read (about 2.5 h):** chapters for the day. Predict every Python output before reading it.
3. **Labs (about 1 h):** type and run the day's labs.
4. **Practice (about 30 min):** the day's question set, then mark your answers.
5. **Log (5 min):** write scores and weak topics in the tracker below.

The plan

Day	Focus	Read	Labs	Practice
1	Orientation, setup, stage and layers	Front matter, Ch 1-3	01-05	Fundamentals Q 1-15 · fundamentals flashcards
2	Prims, properties, schemas, file formats	Ch 4-7	06-07	Fundamentals Q 16-30 · USDA exercises 1-4
3	Data modeling I: Usd vs. Sdf, value types, time samples	Ch 8-10	08-10	Data Modeling Q 1-26
4	Data modeling II: primvars, metadata, extents	Ch 11-13	11-12	Data Modeling Q 27-52 · USDA ex. 5-8 · checkpoint
5	Composition I: fundamentals, sublayers, references, payloads	Ch 14-17	13-15, 19	Composition Q 1-30
6	Composition II: variants, inherits, specializes, overs, relocates	Ch 18-20	16-18	Composition Q 31-60 · USDA ex. 9-14
7	Composition III: LIVERPS, value resolution, design and debugging	Ch 21-22	20-21	Composition Q 61-92 · LIVERPS cheat sheet · week-1 review
8	Content aggregation	Ch 23-26	22-24	Content Aggregation Q 1-40 · USDA ex. 15-18
9	Data exchange	Ch 27-31	25-28	Data Exchange Q 1-60
10	Pipeline development	Ch 32-35	29-30	Pipeline Q 1-56 · USDA ex. 19-22
11	Customizing USD and visualization	Ch 36-41	31-35	Customizing Q 1-24 · Visualization Q 1-32
12	Debugging and performance · Mock Exam 1	Ch 42-44	36	Debugging Q 1-44 · Mock 1 (120 min) and review

Day	Focus	Read	Labs	Practice
13	Weak-area repair · Mock Exam 2	Your weak chapters	37 (capstone)	Mock 2 and review · USDA ex. 23-30 · all flashcards
14	Final revision · Mock Exam 3	Cheat sheets	—	Mock 3 and review · final checklist · light review in the evening

EXAM TIP

Day 7's week-1 review matters. Re-do every composition question you got wrong on days 5-7 *before* moving on. Later domains build on composition.

Progress tracker

Fill this in by hand as you go.

Day	Done	Hours	Question score	Weak topics
1	☐		___ / ___	
2	☐		___ / ___	
3	☐		___ / ___	
4	☐		___ / ___	
5	☐		___ / ___	
6	☐		___ / ___	
7	☐		___ / ___	
8	☐		___ / ___	
9	☐		___ / ___	
10	☐		___ / ___	
11	☐		___ / ___	
12	☐		___ / ___	
13	☐		___ / ___	
14	☐		___ / ___	

Mock exam scores

Domain	Questions per mock	Mock 1	Mock 2	Mock 3
Composition	15	___	___	___
Data Exchange	10	___	___	___
Pipeline Development	9	___	___	___
Data Modeling	8	___	___	___
Debugging and Troubleshooting	7	___	___	___
Content Aggregation	7	___	___	___
Visualization	5	___	___	___
Customizing USD	4	___	___	___
Total	65	___	___	___

Readiness target: at least 80% overall on Mock 3 and at least 70% in every domain.

This is the book's own target; NVIDIA does not publish a passing score on its certification page. If you miss it, add review days for your weakest domains before booking the exam.

Setting Up Your Lab

You will run every example in this book yourself. This section takes you from an empty computer to a working OpenUSD Python setup in about 15 minutes.

What you need

Item	Why
Python 3 (64-bit)	OpenUSD's Python API runs in it
usd-core package	Pixar's official OpenUSD build for Python, published on PyPI
A text editor	To read and edit .usda and .py files (any editor works)
A terminal	To run scripts

usd-core supports specific Python versions. Check the "Requires" information on its PyPI page before installing. This book was verified with Python 3.12 and usd-core 26.8.

Step 1: Create a project folder and a virtual environment

A **virtual environment** is a private Python installation for one project, so packages you install don't affect anything else.

Linux and macOS:

```
mkdir usd-study
cd usd-study
python3 -m venv .venv
source .venv/bin/activate
```

Windows (PowerShell):

```
mkdir usd-study
cd usd-study
py -m venv .venv
.venv\Scripts\Activate.ps1
```

After activation your prompt shows (.venv). From now on, python means the Python inside this environment.

Step 2: Install OpenUSD

```
python -m pip install usd-core
```

To install exactly the version this book was verified with:

```
python -m pip install "usd-core==26.8"
```

VERSION

`usd-core` version numbers drop the leading zero of the month: package 26.8 is OpenUSD release 26.08. Inside Python, `Usd.GetVersion()` returns `(0, 26, 8)`.

Step 3: Check the installation

Save as check_usd.py and run python check_usd.py:

```
from pxr import Usd
print("USD version:", Usd.GetVersion())
```

Expected output

```
USD version: (0, 26, 8)
```

All of OpenUSD's Python modules live in the pxr package. You will mostly use Usd, Sdf, UsdGeom, UsdShade, UsdLux, Gf, Vt, Tf, and Ar. Chapter 1 explains what each one does.

Step 4: Your first USD file

Save as `hello.py` and run it:

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateNew("hello.usda")
world = UsdGeom.Xform.Define(stage, "/World")
ball = UsdGeom.Sphere.Define(stage, "/World/Ball")
ball.GetRadiusAttr().Set(2.0)
stage.SetDefaultPrim(world.GetPrim())
stage.GetRootLayer().Save()

print(stage.GetRootLayer().ExportToString())
```

Expected output

```
#usda 1.0
(
  defaultPrim = "World"
)

def Xform "World"
{
  def Sphere "Ball"
  {
    double radius = 2
  }
}
```

The script created a file called `hello.usda` next to it. Open it in your text editor: it contains exactly the text printed above. You have just written a **scene description**: a transform called `World` holding a sphere called `Ball` with a radius of 2. Every term here (stage, prim, attribute, default prim) is explained in Chapters 2–5.

Command-line tools and their Python stand-ins

A full OpenUSD build includes command-line tools. You will meet them in the exam and in real pipelines:

Tool	What it does	Chapter
<code>usdcat</code>	Prints a USD file as text, converts between formats, can flatten	7, 31
<code>usdtree</code>	Prints the prim hierarchy	2
<code>usdchecker</code>	Validates a file against OpenUSD rules	30
<code>usdzip</code>	Creates or lists <code>.usdz</code> packages	31
<code>usdresolve</code>	Shows how an asset path resolves	33
<code>usddiff</code>	Compares two USD files	31
<code>usdedit</code>	Opens any USD file as text in an editor	7
<code>usdview</code>	Interactive 3D viewer and inspector	41, 42

KEY

The pip package ``usd-core`` contains the Python modules but **none** of these command-line tools, and no ``usdview``. Everything they do can also be done from Python, which is what this book does.

Typical command-line usage, for reference (**not run automatically**: needs a full OpenUSD build):

```
usdcat hello.usda          # print as text
usdcat hello.usda -o hello.usdc  # convert to binary crate format
usdcat --flatten shot.usda -o shot_flat.usda
usdtree hello.usda
```

```
usdchecker hello.usda
usdzip hello.usdz hello.usda
```

The three small scripts below do the same jobs with `usd-core`. They are **not run automatically** because they need command-line arguments; they were run by hand against USD 26.08 with the outputs shown.

usdcat_lite.py: print or convert a layer:

```
import sys
from pxr import Sdf

# Usage: python usdcat_lite.py input.usd [output.usda]
src = Sdf.Layer.FindOrOpen(sys.argv[1])
if len(sys.argv) > 2:
    src.Export(sys.argv[2])          # format chosen by the output extension
    print("wrote", sys.argv[2])
else:
    print(src.ExportToString())
```

`python usdcat_lite.py hello.usda hello.usdc` prints `wrote hello.usdc`. The new file is binary. `python usdcat_lite.py hello.usdc` prints it back as readable text, because `ExportToString()` always produces USDA text.

usdtree_lite.py: print the prim hierarchy:

```
import sys
from pxr import Usd

# Usage: python usdtree_lite.py input.usd
stage = Usd.Stage.Open(sys.argv[1])
for prim in stage.Traverse():
    depth = len(prim.GetPath().GetPrefixes()) - 1
    print("  " * depth + f"{prim.GetName()} ({prim.GetTypeName()}")
```

`python usdtree_lite.py hello.usda` prints:

```
World (Xform)
  Ball (Sphere)
```

usdzip_lite.py: package a file and its dependencies as `.usdz`:

```
import sys
from pxr import UsdUtils

# Usage: python usdzip_lite.py input.usd output.usdz
ok = UsdUtils.CreateNewUsdzPackage(sys.argv[1], sys.argv[2])
print("packaged:", ok)
```

`python usdzip_lite.py hello.usda hello.usdz` prints `packaged: True`.

Validation (the `usdchecker` replacement) needs more background. It is covered with the `UsdValidation` framework in Chapter 30.

Getting usdview (optional)

`usdview` is not required for anything in this book, and every lab works without it. It is still worth knowing because exam questions may mention it, especially its **Layer Stack** and **Composition** inspection panels (Chapter 42).

Ways to get it:

1. **NVIDIA prebuilt binaries (Windows and Linux)**. NVIDIA's OpenUSD developer page (developer.nvidia.com/openusd, checked 2026-10-08) offers **USD 25.08, Python 3.12** packages for **Windows** and **Linux**. Those archives include `usdview` and the rest of the USD toolset. After extracting:
 - **Windows**: run `.\scripts\usdview_gui.bat` from the extracted folder (File → Open to load a stage). `.\scripts\usdview.bat` your `.usda` requires a file argument.

- **Linux:** run `./scripts/usdview_gui.sh`. On Ubuntu, install the X11 packages listed in NVIDIA's Learn OpenUSD usdview install guide first (`libxkbcommon-x11-0` and related `libxcb-*` packages).

The same page archives **USD 25.05, Python 3.11** for Windows and Linux. The prebuilt USD version **lags** this book's verified **26.08**; `pip install usd-core` never includes `usdview`.

1. **macOS:** NVIDIA does not currently ship a macOS prebuilt. Build OpenUSD from source with `build_usd.py` from the OpenUSD GitHub repository (Chapter 35). NVIDIA's Learn OpenUSD install notes say the same: `usdview` runs on macOS after you build it yourself.
2. **Build from source** on any platform if you need a viewer that matches this book's Python API (USD 26.08).

VERSION

Verified 2026-10-08: NVIDIA prebuilts are USD 25.08 / Python 3.12 for Windows and Linux and include ``usdview``. There is no NVIDIA macOS prebuilt. This book's labs use ``usd-core`` 26.8 and do not need ``usdview``.

Working folder for the labs

Keep one folder per lab so generated files don't collide:

```
usd-study/
├── .venv/
├── check_usd.py
├── hello.py
├── labs/
│   ├── lab01/
│   ├── lab02/
│   └── ...
```

Troubleshooting

Symptom	Cause	Fix
<code>ModuleNotFoundError: No module named 'pxr'</code>	The virtual environment isn't active, or <code>usd-core</code> was installed into a different Python	Activate <code>.venv</code> and run <code>python -m pip install usd-core</code> again
<code>pip</code> reports no matching distribution	Your Python version is not supported by <code>usd-core</code> , or there is no network	Check the supported versions on PyPI and install a supported Python
<code>Usd.Stage.CreateNew</code> raises "A layer already exists with identifier..."	That file is already open as a layer in the same Python session (common in interactive shells and notebooks)	Restart Python, or use <code>Usd.Stage.Open</code> to work with the open layer
Your earlier content in a file disappeared	In a new Python session, <code>CreateNew</code> on an existing file starts an empty layer, and <code>Save()</code> replaces the file	Use <code>Usd.Stage.Open</code> to edit existing files; use <code>CreateNew</code> only for new ones
Script runs but no file appears	Paths are relative to the folder you ran the script from	<code>cd</code> into the script's folder first

MISTAKE

Running ``pip install usd-core`` with one Python and the script with another. Using ``python -m pip ...`` always installs into the same Python that ``python`` runs.

Part I — OpenUSD Fundamentals

CHAPTER 1

Chapter 1 — What Is OpenUSD?

Exam domain: Foundations for every domain · **Objectives:** none directly (prepares all of them) **Study day:** 1 · **Est. time:** 60 min **Prerequisites:** Front matter F5 (you can run a Python script)

Learning goals

- Explain in one sentence what OpenUSD is, and what it is not.
- Distinguish **scene description**, **file format**, and **runtime library**.
- Name where OpenUSD is used and why those industries adopted it.
- Map the main OpenUSD software modules (Tf, Gf, Vt, Ar, Sdf, Pcp, Usd, schema domains, Hydra) to what each does.
- Import `pxr` modules and print the installed USD version.

Key terms

Term	One-line definition
OpenUSD / USD	An open-source system for describing, composing, and exchanging 3D scenes
Scene description	The data model: objects (prims) with properties, stored in layers
Layer	One document of scene description (often one file)
Stage	The composed scene you work with at runtime
Composition	The process of combining layers into one stage by fixed strength rules
<code>pxr</code>	The Python package that contains all OpenUSD modules

1.1 Scene description vs. file format vs. runtime

What is it? OpenUSD (Open Universal Scene Description) is an open-source system for describing 3D scenes, combining contributions from many files and people, and handing that scene to tools and renderers. People often shorten it to **USD**.

Why do we need it? A 3D project uses many tools: a modeler, an animation package, a lighting tool, a game engine, a CAD program, a renderer. Each tool historically had its own scene format. Moving data between them meant lossy exports, duplicated files, and broken work whenever someone upstream changed something.

Beginner explanation. Think of a shared recipe book in a large restaurant kitchen. The original recipe stays untouched. The pastry chef clips a note on top saying “use less sugar”. The head chef clips another saying “serve on a blue plate”. Anyone reading the recipe sees the original plus all the notes, combined in a known order.

USD works the same way: the original data stays intact, and later contributions are **layered on top**.

Where the analogy breaks: in USD the “notes” follow strict, predictable rules about which note wins when two disagree. Learning those rules (Part III, Composition) is the largest single part of the exam.

Technical explanation. USD is three things that are easy to confuse. Exam questions often mix them.

Part	What it is	Example
Scene description	A data model: prims (scene objects) with properties, arranged in a hierarchy, stored in layers	"There is a sphere at /World/Ball with radius 2"
File formats	Ways to save a layer to disk	.usda (text), .usdc (binary crate), .usd (either), .usdz (zip package)
Runtime libraries	C++ libraries with Python bindings that read, compose, edit, and serve scene data	pxr.Usd, pxr.Sdf, pxr.UsdGeom

The most important runtime idea is **composition**: USD reads several layers and combines them into one **composed** scene according to fixed strength rules. Applications see the composed result unless they ask for the authored details.

NOTE

"USD" and "OpenUSD" refer to the same technology. The open-source project is called OpenUSD. The libraries, file extensions, and Python package (`pxr`) still say "usd". This book uses ****OpenUSD**** for the project and ****USD**** in running text.

Mental model.

files on disk	runtime	what you see
.usda / .usdc / .usdz	--> compose layers	--> one Stage
(layers)	(Pcp + Usd)	(prims)
		--> tools / renderer

USD = a shared scene, built from stacked layers, combined by fixed rules, readable by every tool.

Simple example. A furniture company has a chair model. Design owns `chair_geo.usda` (the shape). Marketing wants a red version for a catalogue. Instead of copying the file, they create a small layer that says only “the chair’s color is red” and stack it on top. If design improves the shape tomorrow, the red catalogue version picks up the new shape automatically.

USDA example. `.usda` is USD’s human-readable text format. This complete layer describes one sphere:

```
#usda 1.0

def Xform "World"
{
  def Sphere "Ball"
  {
    double radius = 2
  }
}
```

How to read it:

- `#usda 1.0` is required. It marks the file as USDA text.
- `def` means “define a new prim here”.
- `Xform` and `Sphere` are **types** (schemas). An `Xform` is a transform group; a `Sphere` is a geometric sphere.
- `"World"` and `"Ball"` are **names**. Names join into a **prim path**: `/World/Ball`.
- `double radius = 2` is an **attribute** on the sphere.

You do not need to memorize every schema yet. You do need to recognize this as scene description, not “just a 3D file format”.

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
```

```
world = UsdGeom.Xform.Define(stage, "/World")
ball = UsdGeom.Sphere.Define(stage, "/World/Ball")
ball.GetRadiusAttr().Set(2.0)
print(stage.GetRootLayer().ExportToString())
```

Expected output

```
#usda 1.0

def Xform "World"
{
    def Sphere "Ball"
    {
        double radius = 2
    }
}
```

NOTE

`stage.GetRootLayer().ExportToString()` prints what that layer authored.
`stage.ExportToString()` prints a flattened composed view and may add a generated `doc` line. You will use both views in Chapter 8.

Real-world use case. A visual-effects shot is not one file. Layout publishes a camera and set. Animation publishes character poses. Lighting publishes lights. USD lets each department publish its own layer. The lighting department sees the composed shot without copying the animation files.

Common mistakes.

MISTAKE

Treating USD as "just another file format like OBJ or FBX". OBJ holds one mesh. USD holds a whole composed scene, with many files contributing opinions.

MISTAKE

Thinking "if I don't see it in this `.usda` file, it isn't in the scene". A prim can come from a referenced file that this layer only points to. Chapter 16.

Exam traps.

TRAP

A question that says "USD is a file format" as the *complete* answer. USD **includes** file formats, but the important part is the scene description and the composition runtime.

TRAP

Mixing up "layer" (a document of opinions) with "stage" (the composed result). Chapter 2 and Chapter 3 separate them.

Exam takeaways.

KEY

- OpenUSD = scene description + file formats + composition runtime. - A `.usda` file is one **layer**, not automatically the whole scene. - Composition is the feature that makes USD different from "just an interchange file".

1.2 History: Pixar, open source, AOUSD

What is it? USD started at **Pixar Animation Studios** as an internal scene-description system. Pixar open-sourced it in **2016**. In **2023**, companies formed the **Alliance for OpenUSD (AOUSD)** to evolve the specification in the open.

Why do we need it? Knowing the history helps you read docs. Older material says “Pixar USD”. Current material says “OpenUSD”. The technology is the same line of work. AOUSD is why the exam talks about a *standard*, not a single vendor’s tool.

Beginner explanation. A studio built a tool that solved their movie-pipeline problems so well that they published it, and other companies joined a group to keep it from fragmenting into twenty incompatible dialects.

Where the analogy breaks: OpenUSD is not “finished”. Releases still add features (relocates, UsdValidation, splines). This book is verified against **USD 26.08**.

Technical explanation. - **Pixar** still leads much of the open-source implementation (the GitHub repository `PixarAnimationStudios/OpenUSD`). - The Python package on PyPI is **usd-core**. Version 26.8 is OpenUSD release 26.08. - **AOUSD** publishes specifications (for example `UsdPreviewSurface`, `USDZ`) so implementations can interoperate. - NVIDIA’s certification tests the **OpenUSD developer** skill set (Python/C++ APIs, composition, pipelines), not a single DCC’s menus.

Mental model.

```
Pixar films --> open source (2016) --> AOUSD (2023) --> many industries
```

Simple example. If two applications both say they “support OpenUSD”, they should agree on what a Mesh, a Material, and a reference mean. That agreement is the point of a standard.

USDA example. There is no unique “history” syntax. The layer header you already saw is part of the spec:

```
#usda 1.0
(
    defaultPrim = "World"
    upAxis = "Y"
    metersPerUnit = 0.01
)

def Xform "World"
{
}
```

Those three metadata fields (`defaultPrim`, `upAxis`, `metersPerUnit`) exist because the spec needs a shared meaning for “which prim is the asset”, “which way is up”, and “how big is one unit”.

Python example.

```
from pxr import Usd
print("USD version:", Usd.GetVersion())
```

Expected output

```
USD version: (0, 26, 8)
```

The tuple is `(0, major, minor): 26.08`. The leading `0` is a historical major-version slot.

Real-world use case. A car maker, a game studio, and a VFX house can all sit on the AOUSD working groups and argue for schema changes. That is why you will see domains such as `UsdPhysics` and `UsdLux` in the same runtime as film-oriented `UsdGeom`.

Common mistakes.

MISTAKE

Assuming “NVIDIA USD” is a different scene description. NVIDIA contributes tools, training, and Omniverse applications. The scene description you study for NCP-OUUSD is OpenUSD.

MISTAKE

Pinning your career knowledge to one USD version without reading VERSION notes. APIs do get renamed (Chapter 24: ``GetMaster`` became ``GetPrototype``).

Exam traps.**TRAP**

"Pixar stopped USD when they open-sourced it." They did not. The implementation continues.

TRAP

Treating AOUSD as a replacement for the runtime. AOUSD governs the standard; you still call Pixar's (and others') libraries.

Exam takeaways.**KEY**

- Open-sourced by Pixar in 2016; AOUSD formed in 2023. - This book and its labs are verified on USD **26.08**. - The exam is about OpenUSD development, not one vendor's GUI.

1.3 Where OpenUSD is used

What is it? OpenUSD is used anywhere many 3D tools and teams must share a scene: film and VFX, games, manufacturing and digital twins, architecture/engineering/construction (AEC), robotics, and virtual production.

Why do we need it? The exam's job description spans those industries. Pipeline questions are written in industry-neutral language ("DCC", "asset", "assembly") so they apply to a factory digital twin as much as to a feature film.

Beginner explanation. If more than one program needs to look at the same 3D world, and more than one person needs to edit that world without overwriting each other, USD is a candidate.

Where the analogy breaks: a single artist exporting one statue to a 3D printer does not need composition. USD still helps as an interchange format, but its real power shows up at pipeline scale.

Technical explanation. Typical patterns:

Industry	What USD holds	Why composition matters
Film / VFX	Shots: cameras, characters, sets, lights, FX	Departments publish layers; lighting does not copy animation
Games	Packed environments, instanced props	Native and point instancing keep memory down (Part IV)
Manufacturing	Product variants, CAD-derived meshes	Variants select trim levels without duplicating geometry
AEC	Buildings, coordinated trades	References assemble a site from many consultants' models
Robotics / simulation	Robots, sensors, environments	One stage feeds both visualization and physics

Mental model. Same data model; different **assets** and **pipelines**. A "shot" and a "factory cell" are both composed stages.

Simple example. A robot-arm manufacturer publishes one component asset per arm model. A systems integrator references twenty arms into a cell assembly and overrides each arm's pose in a stronger layer. The original arm files stay untouched.

USDA example. A tiny “cell” that references an arm (the arm file would exist beside this layer in a real project):

```
#usda 1.0
(
    defaultPrim = "Cell"
)

def Xform "Cell" (
    kind = "assembly"
)
{
    def Xform "Arm_01" (
        prepend references = @./arm.usda@
    )
    {
        double3 xformOp:translate = (100, 0, 0)
        uniform token[] xformOpOrder = ["xformOp:translate"]
    }
}
```

You will author references for real in Chapter 16. For now, notice the idea: the cell **points at** the arm rather than embedding it.

Python example.

```
from pxr import Usd, UsdGeom, Kind

stage = Usd.Stage.CreateInMemory()
cell = UsdGeom.Xform.Define(stage, "/Cell")
Usd.ModelAPI(cell.GetPrim()).SetKind(Kind.Tokens.assembly)
arm = UsdGeom.Xform.Define(stage, "/Cell/Arm_01")
print("cell kind:", Usd.ModelAPI(cell.GetPrim()).GetKind())
print("paths:", [str(p.GetPath()) for p in stage.Traverse()])
```

Expected output

```
cell kind: assembly
paths: ['/Cell', '/Cell/Arm_01']
```

Real-world use case. Virtual production on a LED stage consumes a USD scene in real time: cameras, set pieces, and lights. Layout can still replace a referenced building overnight without the playback operator copying files by hand.

Common mistakes.

MISTAKE

Assuming USD is "only for movies". The exam's data-exchange and pipeline domains are industry-neutral.

MISTAKE

Copying assets between departments "to be safe". Copies go stale. Layers and references are the USD way.

Exam traps.

TRAP

A scenario set in manufacturing. The composition rules are the same as in VFX. Do not invent a second set of rules.

Exam takeaways.

KEY

- USD is used across film, games, manufacturing, AEC, robotics, and virtual production. - The exam talks about assets, assemblies, DCCs, and pipelines — not one industry's jargon.

1.4 The OpenUSD software stack

What is it? OpenUSD is a **stack of libraries**, not one module. Each library has a job. You import the ones you need from `pxr`.

Why do we need it? Exam questions name these libraries (`Sdf`, `Pcp`, `UsdGeom`, `Ar`, `Tf`). You must know which layer of the stack a problem lives on: “is this a path-resolution issue (`Ar`), an authored-spec issue (`Sdf`), or a composed-value issue (`Usd`)?”

Beginner explanation. Think of a newspaper:

- **Tf / Gf / Vt** are the printing press and the paper (utilities, math, arrays).
- **Ar** is the mail room (finding files from paths).
- **Sdf** is the stack of original manuscripts (what was actually written).
- **Pcp** is the editor who combines manuscripts by house rules (composition).
- **Usd** is the newspaper you read (the composed scene).
- **UsdGeom / UsdShade / UsdLux / ...** are the sections (sports, business) — **schemas** for kinds of data.
- **Hydra** is the newsstand display (imaging).

Where the analogy breaks: you often write through `Usd` and it authors into `Sdf`. You do not usually call `Pcp` directly.

Technical explanation. Bottom to top:

Module	Role	You will use it for
Tf	Foundation: errors, tokens, debug flags, plugins wiring	Diagnostics, <code>TfDebug</code> (Ch 43)
Gf	Graphics-friendly math: vectors, matrices, quaternions	<code>Gf.Vec3f</code> , <code>Gf.Matrix4d</code> (Ch 9)
Vt	Value types and arrays	<code>Vt.Vec3fArray</code> for mesh points (Ch 9)
Ar	Asset resolution	Turning <code>@./chair.usda@</code> into a real file (Ch 33)
Sdf	Scene Description Foundations: layers, specs, paths, value types	Authoring and inspecting what was written (Ch 3, 8)
Pcp	PrimCache Population: composition engine	Why a prim exists and which opinions won (Ch 21–22). Rarely called directly
Usd	Composed scene graph: Stage, Prim, Attribute, Relationship	Everyday authoring and querying (Ch 2–5)
Kind	Model-kind registry	<code>component</code> , <code>assembly</code> , <code>group</code> (Ch 6)
Plug	Plugin registry	Custom schemas and file formats (Ch 36)
UsdGeom / UsdShade / UsdLux / UsdPhysics / UsdSkel	Schema domains — typed APIs on top of <code>Usd</code>	Meshes, materials, lights, physics, skeletons
UsdUtils	Pipeline helpers	Flatten, USDZ, diagnostics, dependencies
UsdValidation	Validation framework (current)	Replaces the old <code>ComplianceChecker</code> (Ch 30)
Hydra		

Module	Role	You will use it for
	Imaging architecture (render delegates, scene indices)	usdview, renderers (Ch 41). Not in <code>usd-core</code>

VERSION

Verified on USD 26.08. ``UsdUtils.ComplianceChecker`` is **gone**. Use ``pxr.UsdValidation``. ``Trace`` (profiling) and ``Ts`` (splines) also ship in this build.

Mental model.

```
Hydra (imaging)          <-- not in usd-core
-----
UsdGeom UsdShade UsdLux  <-- schemas (typed opinions)
Usd                      <-- composed prims / attributes
Pcp                      <-- composition (LIVERPS)
Sdf                      <-- layers and specs
Ar                       <-- find the files
Tf  Gf  Vt              <-- utilities, math, arrays
```

Simple example. “Why is this prim’s radius 5?”

- Ask **Usd** for the composed value: `attr.Get()`.
- Ask **Sdf** which layer authored 5: look at the attribute spec.
- Ask **Pcp** (through `Usd.PrimCompositionQuery`) which composition arc brought that layer in.

USDA example. You do not import modules in USDA. You *see* which stack level a construct belongs to:

```
#usda 1.0
(
    defaultPrim = "World"
    metersPerUnit = 0.01
    upAxis = "Y"
)

def Xform "World"
{
    def Sphere "Ball"
    {
        double radius = 2
        rel material:binding = </World/Looks/Red>
    }
}
```

- The file itself is an **Sdf** layer.
- `def Sphere` is a **UsdGeom** schema.
- `rel material:binding` is a **UsdShade** relationship (once `MaterialBindingAPI` is applied; Ch 40).
- Opening this file creates a **Usd** stage.

Python example.

```
from pxr import Usd, Sdf, UsdGeom, Gf, Vt, Ar, Kind, Tf

print("Usd", Usd.GetVersion())
print("Sdf.Path", Sdf.Path("/World/Ball"))
print("Gf.Vec3f", Gf.Vec3f(1, 2, 3))
print("Vt.IntArray", Vt.IntArray([1, 2, 3]))
print("Ar default resolver:", type(Ar.GetResolver()).__name__)
print("Kind component is a model:", Kind.Registry.IsA("component", "model"))
print("Tf.Debug present:", hasattr(Tf, "Debug"))
print("UsdGeom.Sphere:", UsdGeom.Sphere)
```

Expected output


```

Usd (0, 26, 8)
Sdf.Path /World/Ball
Gf.Vec3f (1, 2, 3)
Vt.IntArray [1, 2, 3]
Ar default resolver: Resolver
Kind component is a model: True
Tf.Debug present: True
UsdGeom.Sphere: <class 'pxr.UsdGeom.Sphere'>

```

Real-world use case. A pipeline TD debugs a missing texture. The path string lives on an Sdf attribute. Ar fails to resolve it because the studio resolver does not search that folder. That is an **Ar** problem, not a Mesh schema problem. Choosing the right layer of the stack saves hours.

Common mistakes.

MISTAKE

Importing only `Usd` and then wondering why `Gf.Vec3f` is missing. Import each module you use: `from pxr import Usd, Gf`.

MISTAKE

Calling Hydra APIs in `usd-core`. Imaging is not in this pip package. Teach and test Hydra conceptually (Ch 41).

Exam traps.

TRAP

"Pcp is where you author a sphere." You author with Usd/Sdf. Pcp **composes**.

TRAP

"Sdf and Usd are two names for the same API." They are two views of the data (Ch 8).

Exam takeaways.

KEY

- Remember the stack: Tf/Gf/Vt → Ar → Sdf → Pcp → Usd → schema domains → Hydra. - Sdf = authored. Usd = composed. Pcp = the rules in between. - Schema domains (UsdGeom, UsdShade, UsdLux) are how you talk about meshes, materials, and lights.

1.5 The `pxr` Python package

What is it? `pxr` is the Python package that wraps OpenUSD. Every script in this book starts with `from pxr import ...`.

Why do we need it? The exam is a development exam. You must be fluent in the Python API names and how they map from the C++ docs.

Beginner explanation. `pxr` is a namespace (the letters come from PiXaR). Inside it are modules that match the stack in §1.4.

Where the analogy breaks: not every C++ library has a Python module in `usd-core`. Imaging (UsdImaging) is absent here.

Technical explanation. Verified on this book's environment (`usd-core` 26.8). Modules present include:

```

Ar, Gf, Kind, Pcp, Plug, Sdf, Sdr, Tf, Trace, Ts, Usd, UsdGeom, UsdHydra, UsdLod, UsdLux,
UsdMedia, UsdPhysics, UsdProc, UsdProfiles, UsdRender, UsdRi, UsdSemantics, UsdShade,
UsdShaders, UsdSkel, UsdUI, UsdUtils, UsdValidation, UsdVol, Vt, Work.

```

Naming map (C++ → Python), repeated from F2 because you will live in it:

C++	Python
UsdStage::Open	Usd.Stage.Open
UsdPrim	Usd.Prim
SdfLayer	Sdf.Layer
SdfPath	Sdf.Path
UsdGeomMesh	UsdGeom.Mesh
GfVec3f	Gf.Vec3f
VtArray<int>	Vt.IntArray

Nested classes become attributes of the module: `Usd.Stage`, `Usd.Prim`, `Usd.Attribute`.

Mental model. `from pxr import Usd, Sdf, UsdGeom` is the default starter import for most labs.

Simple example. Wrong: `import usd`. Right: `from pxr import Usd`.

USDA example. USDA is not Python. Keep them straight: USDA is the text form of a **layer**. Python is how you **create and query** stages.

```
#usda 1.0
def Xform "World" {}
```

Python example.

```
import pkgutil
import pxr
from pxr import Usd

names = sorted(m.name for m in pkgutil.iter_modules(pxr.__path__))
print("version", Usd.GetVersion())
print("has UsdValidation", "UsdValidation" in names)
print("has UsdImaging", "UsdImaging" in names)
print("has UsdMtlx", "UsdMtlx" in names)
```

Expected output

```
version (0, 26, 8)
has UsdValidation True
has UsdImaging False
has UsdMtlx False
```

Real-world use case. A studio's CI runs `usd-core` in a slim container to validate and convert assets. Artists use a full build with `usdview`. Both speak the same `pxr` API for the modules they share.

Common mistakes.

MISTAKE

``pip install usd`` or ``import usd``. The package is ``usd-core``; the import is ``pxr``.

MISTAKE

Mixing two Pythons: ``pip install`` with system Python, then running scripts with ``.venv``. Always ``python -m pip`` inside the venv (F5).

Exam traps.

TRAP

C++ snippet ``UsdGeomXformable`` vs Python ``UsdGeom.Xformable``. Same type, different punctuation.

Exam takeaways.

KEY

- Import from ``pxr``. Package name on pip: ``usd-core``. - C++ ``UsdFooBar`` → Python ``Usd.FooBar`` or ``UsdGeom.FooBar``. - ``usd-core`` has no `usdview` and no ``UsdImaging``.

Chapter lab(s)

Lab 01 (install and verify) and **Lab 02** (first stage) live in `python-labs/` and are written in Phase 11. Until then, run the Python examples in this chapter and in F5. That is enough for Day 1.

USDA reading exercise

USDA-01. Read this layer. What is the prim path of the sphere, and what is its radius?

```
#usda 1.0

def Xform "World"
{
    def Sphere "Ball"
    {
        double radius = 2
    }
}
```

Answer: Path `/World/Ball`. Radius 2 (a double). The Xform `/World` has no extra attributes.

Chapter review

Summary

- OpenUSD is scene description + formats + a composition runtime, not “just a file”.
- Layers are documents of opinions; a stage is the composed result (Ch 2–3).
- Industries share the same rules.
- Know the stack: Tf/Gf/Vt, Ar, Sdf, Pcp, Usd, schemas, Hydra.
- Python lives in `pxr`; this book uses USD 26.08.

If you see... → think...

If you see...	Think...
“USD is a file format” as a complete answer	Incomplete — also scene description + runtime
A prim not in the file you opened	Composition pulled it from another layer
Pcp in a question	Composition engine, not authoring API
<code>from usd import</code>	Wrong; <code>from pxr import Usd</code>
Hydra / usdview in a <code>usd-core</code> lab	Not in this pip package; answer conceptually

CHAPTER 2

Chapter 2 — The Stage

Exam domain: Data Modeling (13%) and foundations · **Objectives:** 5.4 (retrieve properties — you need a stage first) **Study day:** 1 · **Est. time:** 75 min **Prerequisites:** Chapter 1, F5

Learning goals

- Explain what a **stage** is and how it differs from a layer.
- Create, open, save, and export stages with `CreateInMemory`, `CreateNew`, `Open`, `Save`, and `Export`.
- Traverse prims with `Traverse` and look up a prim with `GetPrimAtPath`.
- Read and set stage metadata: `defaultPrim`, `upAxis`, `metersPerUnit`, time codes.

Key terms

Term	One-line definition
Stage	The runtime composed scene (<code>Usd.Stage</code>)
Root layer	The layer you opened or created; the stage's main document
Session layer	An anonymous scratch layer, stronger than the root, not saved with the file
Traversal	Walking the prim hierarchy on the stage
<code>defaultPrim</code>	Metadata naming the prim used when this layer is referenced without a prim path
<code>upAxis</code>	Which axis is “up” in this stage (Y or Z)
<code>metersPerUnit</code>	How many meters one scene unit represents

2.1 What a stage is

What is it? A **stage** is the live, composed 3D scene you work with in code. In Python it is a `Usd.Stage`.

Why do we need it? A layer is a document. A renderer, a validator, and your scripts need one combined view of every contributing document. That view is the stage.

Beginner explanation. If layers are transparent sheets, the stage is what you see on the projector after all the sheets are stacked.

Where the analogy breaks: the stage is not a screenshot. It is a live query API. Changing an opinion on a layer updates what the stage returns.

Technical explanation. - `Usd.Stage.Open(path)` composes the layer at `path` (and everything it depends on) into a stage. - Every stage has a **root layer** (`GetRootLayer()`) and a **session layer** (`GetSessionLayer()`). - The session layer is strongest and anonymous. Edits you put there vanish when the process ends unless you export them yourself. - `GetLayerStack()` lists the layers that currently contribute, **session layer first** (strongest), then the root, then sublayers in order.

Mental model.

```
session layer (scratch, strongest, not the file on disk)
root layer   (the file you opened)
sublayer 0
sublayer 1   (weaker as you go down)
\_____ compose _____/ --> Usd.Stage
```

Simple example. You open `shot.usda`. That file lists two sublayers, `anim.usda` and `lgt.usda`. You receive one `Usd.Stage`. Asking for `/World/Hero.xformOp:translate` returns the composed translation, not “the value in `shot.usda` only”.

USDA example. A stage is not stored as “a stage”. You store layers. This layer is enough to open as a stage:

```
#usda 1.0
(
    defaultPrim = "World"
)

def Xform "World"
{
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
print("root is anonymous:", stage.GetRootLayer().anonymous)
print("session is anonymous:", stage.GetSessionLayer().anonymous)
print("prims on empty stage:", [str(p.GetPath()) for p in stage.Traverse()])
UsdGeom.Xform.Define(stage, "/World")
print("after Define:", [str(p.GetPath()) for p in stage.Traverse()])
```

Expected output

```
root is anonymous: True
session is anonymous: True
prims on empty stage: []
after Define: ['/World']
```

Real-world use case. `usdview` opens a stage, not “just a file”. Its Layer Stack panel lists the same layers `GetLayerStack()` would return.

Common mistakes.

MISTAKE

Calling the `.usda` file “the stage”. The file is a layer. Opening it produces a stage.

MISTAKE

Editing the session layer and wondering why the file on disk did not change. Session edits are not the root layer.

Exam traps.

TRAP

“The stage is the root layer.” No. The stage *has* a root layer, plus a session layer, plus everything composition pulls in.

Exam takeaways.

KEY

- Stage = composed runtime scene. Layer = document of opinions.
- Every stage has a root layer and a session layer.
- The session layer is strongest and is not your saved file.

2.2 Creating, opening, saving

What is it? The common constructors and savers on `Usd.Stage`: `CreateInMemory`, `CreateNew`, `Open`, `Save`, `Export`.

Why do we need it? You cannot query or author anything without first having a stage. Choosing the wrong constructor loses files or fails in notebooks.

Beginner explanation. - **CreateInMemory** — scratch paper. Nothing hits disk until you export. - **CreateNew** — start a new file. - **Open** — open an existing file (and compose it). - **Save** — write the layers that are dirty back to their files. - **Export** — write the *composed* result to a new file (flatten-like; details in Ch 34).

Where the analogy breaks: Save saves **layers**, not “the stage as one magic blob”. If the root has sublayers, those files save separately.

Technical explanation. | Method | Use | |-----|-----| | `Usd.Stage.CreateInMemory()` | Tests, examples, scratch. Root and session are anonymous layers | | `Usd.Stage.CreateNew("path.usda")` | New identifier on disk. In a **new** Python process, if the file already exists, this starts an **empty** layer and `Save()` will replace the file | | `Usd.Stage.Open("path.usda")` | Compose an existing layer | | `stage.GetRootLayer().Save()` | Write the root layer | | `stage.Save()` | Save all dirty saveable layers | | `stage.Export("out.usda")` / `stage.ExportToString()` | Dump the composed scene |

MISTAKE

In the ****same**** Python session, ``CreateNew`` on an identifier that is already open raises ``A layer already exists with identifier...``. Restart the interpreter, or ``Open`` the existing layer. In a ****fresh**** session, ``CreateNew`` on an existing ****file**** does not error — it starts empty and can overwrite on Save. Use ``Open`` to edit existing files.

`Open` can take `Usd.Stage.InitialLoadSet` to start with payloads loaded or unloaded (Ch 17).

Mental model. `CreateInMemory` = RAM. `CreateNew` = new file. `Open` = existing file. `Save` = write layers. `Export` = snapshot of the composed stage.

Simple example. Lab scripts in this book almost always `CreateInMemory()` so they do not leave files behind. Pipeline tools almost always `Open` a published asset.

USDA example. After `CreateNew("hello.usda")` and defining `/World`, the file looks like:

```
#usda 1.0

def Xform "World"
{
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateNew("ch02_hello.usda")
UsdGeom.Xform.Define(stage, "/World")
stage.GetRootLayer().Save()

opened = Usd.Stage.Open("ch02_hello.usda")
print("opened prims:", [str(p.GetPath()) for p in opened.Traverse()])
print("identifier ends with:", opened.GetRootLayer().identifier.split("/")[-1])
print(opened.GetRootLayer().ExportToString())
```

Expected output

```
opened prims: ['/World']
identifier ends with: ch02_hello.usda
#usda 1.0

def Xform "World"
{
}
```

Real-world use case. A publisher tool `Opens` the working layer, authors `assetInfo`, then `Saves`. A delivery tool `Opens` the same asset and `Exports` a flattened package for a vendor who cannot resolve your internal references (Ch 34).

Common mistakes.

MISTAKE

Using `CreateNew`` every time you run a script on the same path. You wipe previous content when you Save.

MISTAKE

Forgetting `Save()`. In-memory and even `CreateNew` stages do not write until you save or export.

Exam traps.**TRAP**

"Export and Save are the same." Save writes authored layers. Export writes the composed result to a new location.

Exam takeaways.**KEY**

- `Open`` to edit. `CreateNew`` for a new file. `CreateInMemory`` for tests. - `Save`` $\neq$ `Export``. - `CreateNew`` in a fresh process can silently replace an existing file on Save.

2.3 Stage traversal

What is it? **Traversal** walks the prims on the stage. `GetPrimAtPath` jumps to one prim.

Why do we need it? `Obj 5.4` (retrieve properties of a prim) starts with finding the prim. Validators, converters, and debug tools all traverse.

Beginner explanation. The stage is a tree. Traversal is walking the tree from the root down.

Where the analogy breaks: inactive prims are skipped by `Traverse()` but still exist. Use `TraverseAll()` when you need them (see Ch 4).

Technical explanation. - `stage.Traverse()` yields `Usd.Prim` in depth-first, namespace order, skipping inactive and (by default) abstract prims. - `stage.GetPrimAtPath("/World/Ball")` returns a `Usd.Prim`. If nothing is there, you still get a handle; **check `IsValid()`**. - `stage.GetPseudoRoot()` is the hidden `/` prim that parents every root prim.

Mental model.

```

/                (pseudo-root, you rarely author it)
├── World
│   └── Ball

```

`Traverse()` visits `/World` then `/World/Ball`.

Simple example. "Print every prim type on this asset" is a ten-line traverse.

USDA example.

```

#usda 1.0

def Xform "World"
{
    def Sphere "Ball"
    {
        double radius = 2
    }
    def Cube "Box"
    {
    }
}

```

Traversal order: `/World`, `/World/Ball`, `/World/Box`.

Python example.

```

from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
UsdGeom.Xform.Define(stage, "/World")
UsdGeom.Sphere.Define(stage, "/World/Ball")
UsdGeom.Cube.Define(stage, "/World/Box")

print("Traverse:")
for prim in stage.Traverse():
    print(f"    {prim.GetPath()} type={prim.GetTypeName()}")

ball = stage.GetPrimAtPath("/World/Ball")
missing = stage.GetPrimAtPath("/Nope")
print("Ball valid:", ball.IsValid(), "name:", ball.GetName())
print("Nope valid:", missing.IsValid())

```

Expected output

```

Traverse:
  /World type=Xform
  /World/Ball type=Sphere
  /World/Box type=Cube
Ball valid: True name: Ball
Nope valid: False

```

Real-world use case. A converter traverses the stage, and for every `UsdGeom.Mesh` prim exports a glTF mesh (Ch 27). Skipping `IsValid()` checks is how importers crash on empty paths.

Common mistakes.**MISTAKE**

Using the result of `GetPrimAtPath` without `IsValid()`. Invalid prims look like objects; they just do not represent anything on the stage.

MISTAKE

Expecting `Traverse()` to visit inactive prims. It will not. Ch 4.

Exam traps.**TRAP**

"`GetPrimAtPath` raises if the prim is missing." It does not. It returns an invalid prim.

Exam takeaways.**KEY**

- `Traverse()` walks defined, active prims in namespace order. - Always `IsValid()` after `GetPrimAtPath`.

2.4 Stage metadata

What is it? **Stage metadata** is data about the whole scene, stored on the **root layer**: `defaultPrim`, `upAxis`, `metersPerUnit`, `timeCodesPerSecond`, `startTimeCode`, `endTimeCode`, and others.

Why do we need it? Without `defaultPrim`, referencing this file without a prim path fails. Without agreed `upAxis` and `metersPerUnit`, a chair from one DCC appears on its side or 100× too small in another (Obj 4.7, Ch 28). Time metadata tells animation tools the intended frame range.

Beginner explanation. Metadata is the label on the moving box: which way is up, how big is a unit, which object inside is “the thing”, and which frames the shot covers.

Where the analogy breaks: USD does **not** automatically rescale a referenced file to match the stage’s units. That is a famous exam fact (Ch 28).

Technical explanation. Verified defaults on USD 26.08 for a new in-memory stage:

Field	Default	API
upAxis	Y	UsdGeom.GetStageUpAxis / SetStageUpAxis
metersPerUnit	0.01 (centimetres)	UsdGeom.GetStageMetersPerUnit / SetStageMetersPerUnit
timeCodesPerSecond	24.0	stage.GetTimeCodesPerSecond() / SetTimeCodesPerSecond
framesPerSecond	24.0	GetFramesPerSecond / SetFramesPerSecond
startTimeCode / endTimeCode	0.0 / 0.0	GetStartTimeCode / SetStartTimeCode (same for end)
defaultPrim	unset	stage.SetDefaultPrim(prim) / GetDefaultPrim

UsdGeom.LinearUnits.meters is 1.0; UsdGeom.LinearUnits.centimeters is 0.01.

Tokens: UsdGeom.Tokens.y, UsdGeom.Tokens.z (and x).

Mental model. Stage metadata lives in the USDA header, inside the first parentheses after #usda 1.0.

Simple example. A camera-tracking department works Z-up in metres. They set upAxis = "Z" and metersPerUnit = 1. A DCC that defaults to Y-up centimetres must be taught to write those fields or the shot will not match.

USDA example.

```
#usda 1.0
(
    defaultPrim = "World"
    metersPerUnit = 0.01
    upAxis = "Z"
    timeCodesPerSecond = 24
    startTimeCode = 1001
    endTimeCode = 1100
)

def Xform "World"
{
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
print("default upAxis:", UsdGeom.GetStageUpAxis(stage))
print("default metersPerUnit:", UsdGeom.GetStageMetersPerUnit(stage))
print("default tps/fps:", stage.GetTimeCodesPerSecond(), stage.GetFramesPerSecond())

UsdGeom.SetStageUpAxis(stage, UsdGeom.Tokens.z)
UsdGeom.SetStageMetersPerUnit(stage, UsdGeom.LinearUnits.meters)
world = UsdGeom.Xform.Define(stage, "/World")
stage.SetDefaultPrim(world.GetPrim())
stage.SetStartTimeCode(1001)
stage.SetEndTimeCode(1100)

print("upAxis:", UsdGeom.GetStageUpAxis(stage))
print("metersPerUnit:", UsdGeom.GetStageMetersPerUnit(stage))
print("defaultPrim:", stage.GetDefaultPrim().GetName())
print("range:", stage.GetStartTimeCode(), stage.GetEndTimeCode())
print(stage.GetRootLayer().ExportToString())
```

Expected output

```
default upAxis: Y
default metersPerUnit: 0.01
default tps/fps: 24.0 24.0
upAxis: Z
```

```

metersPerUnit: 1.0
defaultPrim: World
range: 1001.0 1100.0
#usda 1.0
(
    defaultPrim = "World"
    endTimeCode = 1100
    metersPerUnit = 1
    startTimeCode = 1001
    upAxis = "Z"
)

def Xform "World"
{
}

```

Real-world use case. A data-exchange validator fails an export that has no `defaultPrim`, because downstream references = `@asset.usda@` (no prim path) cannot pick an entry point (Ch 16, Obj 4.6).

Common mistakes.

MISTAKE

Assuming USD converts units when you reference a metres file into a centimetres stage. It does not. The geometry numbers stay as authored (Ch 28).

MISTAKE

Setting ``upAxis`` as an attribute on ``/World``. It is **stage/layer metadata**, not a prim property.

Exam traps.

TRAP

Default ``metersPerUnit`` is ``0.01`` in this runtime, not ``1``. Do not memorize "USD is always metres".

TRAP

``defaultPrim`` value in USDA is the **prim name** (``"World"``), not the path ``"/World"``.

Exam takeaways.

KEY

- Always author ``defaultPrim``, ``upAxis``, and ``metersPerUnit`` on publishable assets. - Defaults on USD 26.08: upAxis Y, metersPerUnit 0.01, 24 fps. - USD does not auto-convert units across references.

Chapter lab(s)

Day 1 labs 02-03 (first stage; open and traverse) correspond to this chapter. Until Phase 11 writes the lab files, run this chapter's Python examples in your venv.

USDA reading exercise

USDA-02. From the layer in §2.4's USDA example: what is the default prim name, which way is up, and what is the first intended time code?

Answer: World, Z-up, 1001.

Chapter review

Summary

- A stage is the composed runtime scene.
- CreateInMemory / CreateNew / Open / Save / Export do different jobs.

- Traverse and GetPrimAtPath are how you find prims; check IsValid.
- Stage metadata lives in the layer header.

If you see... → think...

If you see...	Think...
"Stage vs layer"	Stage = composed; layer = document
Missing prim after GetPrimAtPath	Call IsValid()
Asset referenced with no prim path	defaultPrim
Object 100× too small after reference	metersPerUnit, no auto-convert
Edits disappeared after restart	You authored on the session layer

CHAPTER 3

Chapter 3 — Layers

Exam domain: Composition (23%) · **Objectives:** 1.3 (sublayers vs references vs payloads), 1.5 (multi-user layering) **Study day:** 1 · **Est. time:** 90 min **Prerequisites:** Chapter 2

Learning goals

- Explain what a **layer** is (`Sdf.Layer`) and how it differs from a stage.
- Identify the **root layer** and the **session layer**.
- Author **sublayers** and predict which opinion wins in a layer stack.
- Use an **anonymous layer** for scratch work.
- State the first rule of strength: in one layer stack, **stronger layers win**.

Key terms

Term	One-line definition
Layer	One document of scene description (<code>Sdf.Layer</code>)
Opinion	An authored value (or other authored decision) in a layer
Root layer	The layer the stage was opened from
Session layer	Strongest anonymous scratch layer on the stage
Sublayer	A layer included by another layer's <code>subLayers</code> list
Layer stack	The ordered list of layers that contribute locally (session + root + sublayers)
Anonymous layer	A layer with no file on disk (<code>anonymous = True</code>)

3.1 What a layer is

What is it? A **layer** is one document of USD scene description. On disk it is often a `.usda` / `.usdc` / `.usd` file. In code it is an `Sdf.Layer`.

Why do we need it? You cannot collaborate if everyone writes into one giant file. Layers let departments, tools, and versions each own a document. Composition combines them.

Beginner explanation. A layer is one transparent sheet. You can draw a whole picture on it, or only a red circle in the corner.

Where the analogy breaks: a layer can also *point at* other sheets (sublayers, references, payloads). Those pointers are themselves opinions.

Technical explanation. - Create with `Sdf.Layer.CreateNew(path)`, `Sdf.Layer.FindOrOpen(path)`, `Sdf.Layer.CreateAnonymous(".usda")`. - `layer.ExportToString()` / `Export(path)` / `ImportFromString(text)`. - `layer.anonymous` is `True` for in-memory layers. - The **identifier** is how USD names the layer (usually a path, or `anon:0x...:tmp.usda`). - You usually author through `Usd` on a stage (which writes into some layer). You use `Sdf` when you care about the document itself.

Mental model. Layer = file-like document of opinions. Stage = projector that stacks documents.

Simple example. `chair_geo.usda` contains the mesh. `chair_look.usda` contains only material bindings. Both are layers. Neither is “the stage” until something opens them.

USDA example.

```
#usda 1.0
(
  defaultPrim = "Chair"
)
```

```
def Xform "Chair"
{
    def Mesh "Geo"
    {
    }
}
```

This entire file is **one layer**.

Python example.

```
from pxr import Sdf

layer = Sdf.Layer.CreateAnonymous(".usda")
print("anonymous:", layer.anonymous)
print("format:", layer.GetFileFormat().formatId)
ok = layer.ImportFromString(
    '#usda 1.0\n'
    'def Xform "Chair" {\n'
    '}\n'
)
print("imported:", ok)
print("prim at /Chair:", bool(layer.GetPrimAtPath("/Chair")))
print(layer.ExportToString())
```

Expected output

```
anonymous: True
format: usda
imported: True
prim at /Chair: True
#usda 1.0

def Xform "Chair"
{
}
```

Real-world use case. A lighting TD's working file is a small layer that sublayers or overrides the shot. The lighting file is what they check into version control that day — not a copy of the whole show.

Common mistakes.

MISTAKE

Opening a layer with `Sdf.Layer.FindOrOpen`` and expecting composed references to be resolved. `Sdf`` sees `**this file**`. Composition happens on a `Usd.Stage``.

Exam traps.

TRAP

Using "layer" and "prim" as synonyms. A layer contains many prim `*specs*`. A prim on the stage may be assembled from many layers.

Exam takeaways.

KEY

- Layer = document (`Sdf``). Stage = composed view (`Usd``). - Anonymous layers have no file until you export them.

3.2 The root layer

What is it? The **root layer** is the layer you passed to `Open / CreateNew / CreateInMemory`. `stage.GetRootLayer()` returns it.

Why do we need it? Every composition has to start somewhere. The root layer is that starting document. Its `subLayers` list, its references, and its metadata (`defaultPrim`, `units`) set the stage's context.

Beginner explanation. If the stage is a book you are reading, the root layer is the book you pulled off the shelf. It may tell you to also read other chapters (sublayers) or other books (references).

Technical explanation. - Edits go to the **edit target**, which defaults to the root layer (Ch 15). - `GetLayerStack(includeSessionLayers=False)` starts with the root, then sublayers. - Flattening (`Usd.Stage.Flatten`, Ch 34) bakes the composed scene into a new root.

Mental model. Root layer = the handle of the stage.

Simple example. `usdview shot.usda` → root layer is `shot.usda`.

USDA example. A root layer that only lists sublayers (no prims of its own):

```
#usda 1.0
(
    defaultPrim = "World"
    subLayers = [
        @./anim.usda@,
        @./layout.usda@
    ]
)
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory("myroot.usda")
UsdGeom.Xform.Define(stage, "/World")
root = stage.GetRootLayer()
print("root anonymous:", root.anonymous)
print("root is edit target:", stage.GetEditTarget().GetLayer() == root)
print("prims authored on root:", list(root.rootPrims.keys()))
```

Expected output

```
root anonymous: True
root is edit target: True
prims authored on root: ['World']
```

Real-world use case. A shot's root layer may be almost empty except `subLayers` and a few overrides. That is a feature: the root is the “assembly sheet” for the shot.

Common mistakes.

MISTAKE

Assuming the root layer contains every prim you see on the stage. References bring prims from elsewhere (Ch 16).

Exam traps.

TRAP

"The root layer is always the strongest layer." The **session** layer is stronger than the root.

Exam takeaways.

KEY

- Root layer = the file you opened. - Session is stronger than root. - Default edit target is the root layer.

3.3 The session layer

What is it? The **session layer** is an anonymous layer USD creates on every stage. It is the **strongest** layer in the local layer stack. `stage.GetSessionLayer()` returns it.

Why do we need it? Viewers and tools need a place to put temporary overrides (hide this prim, mute that layer, try a variant) without dirtying the artist’s file.

Beginner explanation. Session layer = sticky notes on the projector glass. They sit on top of every sheet. Throw them away when you leave the room.

Where the analogy breaks: you *can* export the session layer if you want to keep the notes. USD will not do that for you on `stage.Save()`.

Technical explanation. - Always anonymous. - First in `GetLayerStack()` (default `includeSessionLayers=True`). - Author into it with `Usd.EditContext(stage, stage.GetSessionLayer())` (Ch 15). - `Save()` does not write it to the root file.

Mental model.

```
[session]  strongest, scratch
[root]
[sublayers]
```

Simple example. usdview’s display purposes and “hide” often live as session opinions so closing the viewer leaves the file clean.

USDA example. A session opinion looks like any other overlay. USD stores it in memory, not in your shot file:

```
#usda 1.0

over "World"
{
    over "Ball"
    {
        double radius = 99
    }
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
UsdGeom.Xform.Define(stage, "/World")
ball = UsdGeom.Sphere.Define(stage, "/World/Ball")
ball.GetRadiusAttr().Set(1.0)

with Usd.EditContext(stage, stage.GetSessionLayer()):
    ball.GetRadiusAttr().Set(99.0)

print("composed radius:", ball.GetRadiusAttr().Get())
print("root USDA:")
print(stage.GetRootLayer().ExportToString())
print("session USDA:")
print(stage.GetSessionLayer().ExportToString())
```

Expected output

```
composed radius: 99.0
root USDA:
#usda 1.0

def Xform "World"
{
    def Sphere "Ball"
    {
        double radius = 1
    }
}

session USDA:
#usda 1.0

over "World"
```

```
{
  over "Ball"
  {
    double radius = 99
  }
}
```

Real-world use case. A debug tool solo's one character by deactivating everyone else on the session layer. The shot file is unchanged.

Common mistakes.

MISTAKE

Authoring "just to try it" without an `EditContext`, then saving — you just wrote the experiment into the root layer.

Exam traps.

TRAP

"Session layer is weaker because it is temporary." Temporary $\neq$ weak. It is the strongest local layer.

Exam takeaways.

KEY

- Session layer: strongest, anonymous, not saved with the root. - Use it for temporary overrides.

3.4 Sublayers and the layer stack

What is it? A **sublayer** is a layer listed in another layer's `subLayers` metadata. Together, the session layer, the root, and those sublayers form the **local layer stack**.

Why do we need it? Obj 1.3 and 1.5: sublayers are how one *workstream* (layout, anim, lighting) owns a file **in the same namespace** as the shot. Many users edit the same `/World/Hero` without copying the prim.

Beginner explanation. Sublayers are extra transparent sheets **in the same pile** as the root. They describe the **same prim paths**.

Where the analogy breaks: **references** (Ch 16) bring in another pile *under a prim*. Sublayers *are* this pile. That distinction is an exam favourite.

Technical explanation. - USDA: `subLayers = [ @strong.usda@, @weak.usda@ ]` - **List order: stronger first.** The first sublayer is stronger than the second. - The root is stronger than all of its sublayers. - The session layer is stronger than the root. - Full local order: **session** $\rightarrow$ **root** $\rightarrow$ **sublayer[0]** $\rightarrow$ **sublayer[1]** $\rightarrow$... - Sublayers compose in the **same namespace** (no path prefix). - Layer offsets on sublayers shift time (Ch 10, 15).

Verified on USD 26.08: with `subLayers = [strong.usda, weak.usda]`, a radius of 5 in strong and 1 in weak composes to 5.

Mental model.

session	(strongest)
root.usda	subLayers = [@strong@, @weak@]
strong.usda	
weak.usda	(weakest)

Simple example. Shot root lists `lgt.usda` then `anim.usda` then `layout.usda`. Lighting can override the hero's visibility without editing animation's file.

USDA example. File: *weak.usda*

```
#usda 1.0

def Xform "World"
```



```
{
    def Sphere "Ball"
    {
        double radius = 1
    }
}
```

File: strong.usda

```
#usda 1.0

over "World"
{
    over "Ball"
    {
        double radius = 5
    }
}
```

File: root.usda

```
#usda 1.0
(
    subLayers = [
        @strong.usda@,
        @weak.usda@
    ]
)
```

Composed radius of /World/Ball is 5.

Python example.

```
from pxr import Usd, Sdf, UsdGeom

weak = Sdf.Layer.CreateAnonymous("weak.usda")
strong = Sdf.Layer.CreateAnonymous("strong.usda")
root = Sdf.Layer.CreateAnonymous("root.usda")
weak.ImportFromString(
    '#usda 1.0\n'
    'def Xform "World" {\n'
    '    def Sphere "Ball" {\n'
    '        double radius = 1\n'
    '    }\n'
    '}\n'
)
strong.ImportFromString(
    '#usda 1.0\n'
    'over "World" {\n'
    '    over "Ball" {\n'
    '        double radius = 5\n'
    '    }\n'
    '}\n'
)
root.subLayerPaths = [strong.identifier, weak.identifier]
stage = Usd.Stage.Open(root)
radius = UsdGeom.Sphere(stage.GetPrimAtPath("/World/Ball")).GetRadiusAttr().Get()
print("composed radius:", radius)
print("stack (no session):")
for layer in stage.GetLayerStack(includeSessionLayers=False):
    print(" ", layer.identifier.split(":")[-1])
```

Expected output

```
composed radius: 5.0
stack (no session):
  root.usda
  strong.usda
  weak.usda
```

Real-world use case. Multi-user shots: each department gets a sublayer in a known order. This is Obj 1.5. References are the wrong tool when everyone needs to edit the *same* /World/Hero path.

Common mistakes.**MISTAKE**

Writing `subLayers = [@weak@, @strong@]` and expecting the last file to win. **First** listed sublayer is stronger.

MISTAKE

Using a sublayer when you meant a reference: sublayers do not remap namespace and do not create a reusable asset boundary.

Exam traps.**TRAP**

"Sublayers, references, and payloads are three names for the same arc." They are not. Compare table in the takeaways; full treatment in Ch 16–17.

TRAP

Reading USDA `subLayers` bottom-up like some DCC GUIs. In the **file**, first = stronger.

Exam takeaways.**KEY**

- Sublayer order in the file: **first = stronger**. - Local stack: session → root → sublayers. - Sublayers share namespace; references (Ch 16) nest an asset under a prim; payloads (Ch 17) are unloadable references.

3.5 Anonymous layers

What is it? An **anonymous layer** exists only in memory. `layer.anonymous` is `True`. Identifiers look like `anon:0x...:name.usda`.

Why do we need it? Tests, session layers, procedural content, and “try this override” all need documents that are not files yet.

Beginner explanation. A scratch page. If you never export it, it disappears with the process.

Technical explanation. - `Sdf.Layer.CreateAnonymous(".usda")` or `CreateAnonymous("hint.usda")`. - `Usd.Stage.CreateInMemory()` makes anonymous root + session. - You may still `Export("disk.usda")` to materialize it. - Anonymous layers can be sublayers of each other (as in §3.4’s Python).

Mental model. Anonymous = no path on disk. Still a real layer.

Simple example. This book’s tests use anonymous layers so they do not clutter your project folder.

USDA example. There is no “anonymous” keyword in USDA. Anonymity is a runtime property. The contents still look like any layer:

```
#usda 1.0
def Sphere "Scratch" {}
```

Python example.

```
from pxr import Sdf

layer = Sdf.Layer.CreateAnonymous("scratch.usda")
print("anonymous:", layer.anonymous)
print("identifier contains 'anon:':", layer.identifier.startswith("anon:"))
print("identifier ends with:", layer.identifier.split(":")[-1])
```

Expected output

```
anonymous: True
identifier contains 'anon:': True
identifier ends with: scratch.usda
```

Real-world use case. A DCC exporter builds an anonymous layer, validates it, then exports to the publish path only if checks pass.

Common mistakes.

MISTAKE

Calling ``Save()`` on a stage whose root is anonymous and expecting a file. Export it, or `CreateNew` with a path.

Exam traps.

TRAP

"Anonymous layers cannot contribute opinions." They can. Session layers do it all day.

Exam takeaways.

KEY

- Anonymous layers are real layers without a disk path. - Session layers are anonymous.

3.6 Opinions and “strongest wins” (first look)

What is it? An **opinion** is an authored decision in a layer: a value, a specifier, a metadata field, a list of references. When two opinions disagree, **the stronger one wins**. In one layer stack, strength is stack order.

Why do we need it? This is the seed of LIVERPS (Ch 21). If you only remember one composition sentence for Day 1, remember: **stronger opinions win; weaker ones remain as backups**.

Beginner explanation. Two sticky notes on the same recipe step: the one on top is what the kitchen does. The lower note is still there if you peel the top one off (unmute, delete the override).

Where the analogy breaks: “stronger” is not “later in time”. A weaker file can be newer on disk and still lose.

Technical explanation. - For a scalar attribute in the **same** layer stack, the strongest authored value is the composed value. - A layer with **no opinion** on that attribute does not block weaker opinions. - `over` vs `def` is about *defining namespace*, not about value strength (Ch 4, Ch 20). An `over` in a stronger layer still wins the value. - Across composition **arcs** (references, inherits, ...) strength follows LIVERPS, not only layer-stack order. That is Part III.

Mental model. Ask the stack from the top: “Do you have a value for `radius`?” The first yes wins.

Simple example. Root has no radius. Strong sublayer says 5. Weak sublayer says 1. Result: 5. If you delete the 5, result becomes 1 — the weak opinion was waiting.

USDA example. See §3.4. The `over` in `strong.usda` wins over the `def` in `weak.usda` because of **stack position**, not because `over` is magically stronger than `def`.

Python example.

```
from pxr import Usd, Sdf, UsdGeom

weak = Sdf.Layer.CreateAnonymous("weak.usda")
root = Sdf.Layer.CreateAnonymous("root.usda")
weak.ImportFromString(
    '#usda 1.0\n'
    'def Sphere "Ball" {\n'
```

```

    '    double radius = 1\n'
    '}\n'
)
root.subLayerPaths = [weak.identifier]
stage = Usd.Stage.Open(root)
ball = UsdGeom.Sphere(stage.GetPrimAtPath("/Ball"))
print("from weak only:", ball.GetRadiusAttr().Get())
with Usd.EditContext(stage, stage.GetSessionLayer()):
    ball.GetRadiusAttr().Set(7)
print("after session override:", ball.GetRadiusAttr().Get())

```

Expected output

```

from weak only: 1.0
after session override: 7.0

```

Real-world use case. Animation publishes translation. Layout had placed the character. Animation's sublayer is stronger, so the animated translation wins. Layout's value is still in `layout.usda` if animation's curve is deleted.

Common mistakes.**MISTAKE**

Believing "the file I saved last wins". Strength is stack order, not mtime.

MISTAKE

Believing ``def`` beats ``over`` for values. Specifiers are not value strength.

Exam traps.**TRAP**

Combining two numbers (1 and 5 become 6). USD does not add opinions. One wins.

TRAP

Forgetting the session layer when predicting a `usdview` override.

Exam takeaways.**KEY**

- Strongest authored opinion wins. Weaker opinions remain underneath. - Stack order $\neq$ file modification time. - Specifier (``def``/``over``) $\neq$ value strength. - LIVERPS generalizes this idea across arcs (Ch 21).

Chapter lab(s)

Labs 04–05 (root/session; sublayer stack) belong here. Run this chapter's scripts until those lab files exist.

USDA reading exercises

USDA-03. Root `subLayers = [A@, B@]`. A has no radius. B has `radius = 3`. Composed radius?

Answer: 3. No opinion in A lets B show through.

USDA-04. Same stack, plus session `radius = 9`. Composed radius?

Answer: 9.

Chapter review**Summary**

- Layers are documents; stages compose them.
- Session > root > sublayers (first listed sublayer stronger than the next).
- Sublayers share namespace — that is why they are the multi-user shot pattern.

- Strongest opinion wins; absence is not a blocker.

If you see... → think...

If you see...	Think...
subLayers list	First = stronger; same namespace
Reference vs sublayer	Reuse under a prim vs collaborate on the same paths
usdview hide that didn't save	Session layer
Two numbers combined	They don't; one wins

CHAPTER 4

Chapter 4 — Prims and Prim Paths

Exam domain: Data Modeling (13%) · **Objectives:** 5.4 **Study day:** 2 · **Est. time:** 80 min **Prerequisites:** Chapters 2–3

Learning goals

- Explain what a **prim** is and how **prim paths** address them.
- Choose among specifiers **def**, **over**, and **class**.
- Distinguish typed and typeless prims.
- Activate and deactivate prims, and know what `Traverse()` hides.
- Read a namespace hierarchy.

Key terms

Term	One-line definition
Prim	A named object on the stage (<code>Usd.Prim</code>)
Prim path	The address of a prim, like <code>/World/Ball</code> (<code>Sdf.Path</code>)
Specifier	<code>def</code> , <code>over</code> , or <code>class</code> — how this layer introduces the prim
Typed prim	A prim with a schema type name (<code>Sphere</code> , <code>Xform</code> , <code>Mesh</code> , ...)
Typeless prim	A <code>def</code> or <code>over</code> with no type name
Active	Whether the prim participates in default traversal and imaging
Namespace	The parent/child tree of prim paths

4.1 Prims

What is it? A **prim** (primitive) is a named container on the stage. Almost everything you care about — a group, a mesh, a camera, a material — is a prim.

Why do we need it? Scenes need a unit of identity. Properties hang off prims. Composition arcs target prims. Paths point at prims.

Beginner explanation. If a stage is a company org chart, each named box is a prim.

Where the analogy breaks: two files can contribute to the *same* prim. `/World/Ball` is one prim on the stage, assembled from many layers.

Technical explanation. - `UsdGeom.Xform.Define(stage, "/World")` (and other `Define` methods) create a typed `def`. - `stage.DefinePrim("/A")` creates a typeless `def`. - `stage.DefinePrim("/B", "Xform")` creates a typed `def`. - `prim.GetName()`, `GetPath()`, `GetTypeName()`, `GetPrimStack()` (debug, Ch 22). - A prim handle can be invalid (`IsValid()`), defined or not (`IsDefined()`), abstract (`IsAbstract()`).

Mental model. Prim = named node. Path = its address. Type = optional schema.

Simple example. `/World` is an `Xform` prim. `/World/Ball` is a `Sphere` prim. `/World/Ball` cannot exist on the composed stage without some opinion on `/World`.

USDA example.

```
#usda 1.0

def Xform "World"
{
    def Sphere "Ball"
    {
    }
}
```

Python example.

```

from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
world = UsdGeom.Xform.Define(stage, "/World")
ball = UsdGeom.Sphere.Define(stage, "/World/Ball")
prim = ball.GetPrim()
print("name:", prim.GetName())
print("path:", prim.GetPath())
print("type:", prim.GetTypeName())
print("valid:", prim.IsValid(), "defined:", prim.IsDefined())
print("parent:", prim.GetParent().GetName())

```

Expected output

```

name: Ball
path: /World/Ball
type: Sphere
valid: True defined: True
parent: World

```

Real-world use case. A validator reports “missing prim `/World/Hero/Geo`”. That path is the identity everyone in the pipeline shares, even if twenty layers contribute to it.

Common mistakes.**MISTAKE**

Creating `/World/Ball` without ever defining `/World`. `Define` on a child will create parent specs as needed, but they may be typeless overs/defs you did not intend. Prefer to define parents explicitly.

Exam traps.**TRAP**

"A prim is a mesh." Meshes are one type of prim. Materials, cameras, and empty groups are prims too.

Exam takeaways.**KEY**

- Prim = named node on the stage. - One composed prim, many contributing layers.

4.2 Prim paths

What is it? A **prim path** is the prim’s address in the namespace tree. In Python it is `Sdf.Path`. In USDA it looks like `/World/Ball`.

Why do we need it? Every API that finds, binds, or references something uses a path. Wrong paths are the number-one authoring bug.

Beginner explanation. A path is like `/home/you/docs` on a filesystem, except the names are prims, not folders. The last name is this prim; everything before it is ancestors.

Where the analogy breaks: property paths exist too: `/World/Ball.radius` (a dot, not a slash).

Technical explanation. - Absolute prim path: `/World/Ball`. Always starts with `/`. - Property path: `/World/Ball.radius`. - `path.name` or `path.GetName()` — last element. - `path.GetParentPath()`, `GetPrefixes()`, `AppendChild("Ball")`, `AppendProperty("radius")`. - `IsPrimPath()`, `IsPropertyPath()`, `IsAbsolutePath()`. - Pseudo-root path is `/`.

Invalid characters in names are a data-exchange topic (`Tf.MakeValidIdentifier`, Ch 28).

Mental model.

```
/World/Ball.radius
 \  \  \
prim name property
```

Simple example. Relationship targets and material bindings are paths. If you bind `</Looks/Red>` but the material is `/World/Looks/Red`, the binding is broken (exam data-exchange trap in later chapters).

USDA example.

```
#usda 1.0

def Xform "World"
{
  def Sphere "Ball"
  {
    rel proxyPrim = </World/BallProxy>
  }
  def Cube "BallProxy"
  {
  }
}
```

`</World/BallProxy>` is a path in angle brackets — USDA’s way of writing an `Sdf.Path` as a value.

Python example.

```
from pxr import Sdf

prim = Sdf.Path("/World/Ball")
prop = prim.AppendProperty("radius")
print("prim:", prim, "IsPrimPath:", prim.IsPrimPath())
print("prop:", prop, "IsPropertyPath:", prop.IsPropertyPath())
print("prop's prim path:", prop.GetPrimPath())
print("parent:", prim.GetParentPath())
print("prefixes:", [str(p) for p in prim.GetPrefixes()])
print("child:", Sdf.Path("/World").AppendChild("Ball"))
```

Expected output

```
prim: /World/Ball IsPrimPath: True
prop: /World/Ball.radius IsPropertyPath: True
prop's prim path: /World/Ball
parent: /World
prefixes: ['/World', '/World/Ball']
child: /World/Ball
```

Real-world use case. A compositor’s “pick this asset” tool stores `/World/Set/Building_12`, not “the twelfth building”, so every department agrees.

Common mistakes.

MISTAKE

Writing `/World/Ball/radius` for an attribute. Attributes use a `**dot**`: `/World/Ball.radius`.

Exam traps.

TRAP

``defaultPrim`` in the header is ``"World"`` (a name), not ``"/World"`` (a path).

Exam takeaways.

KEY

- Prim paths use ``/``. Property paths use ``.`` after the prim. - ``Sdf.Path`` is the type; always prefer it over raw strings in APIs that accept paths.

4.3 Specifiers: def, over, class

What is it? A **specifier** is how a layer introduces a prim: **def** (define), **over** (overlay), **class** (abstract class prim).

Why do we need it? Composition needs a way to add new objects (**def**), to add opinions on objects that already exist (**over**), and to store shared templates (**class**). The FAQ question “over vs typeless def” is on NVIDIA’s reading list.

Beginner explanation. - **def** — “this object exists here.” - **over** — “if this object exists, here are extra notes.” An **over** alone does **not** make a defined prim on the stage. - **class** — “this is a template”, skipped by default traversal (`IsAbstract()`).

Where the analogy breaks: a stronger **over** still wins **values** over a weaker **def**. Specifiers are about *existence in namespace*, not about value strength (Ch 3.6).

Technical explanation. Verified on USD 26.08:

Specifier	USDA	Python	IsDefined() if nothing else defines it	In Traverse() ?
def	<code>def Xform "W"</code>	<code>Define / DefinePrim</code>	True	Yes (if active)
over	<code>over "W"</code>	<code>OverridePrim</code>	False	No
class	<code>class "C"</code>	<code>CreateClassPrim</code>	True, and <code>IsAbstract()</code> True	No

`OverridePrim("/C/D")` also authors **over** "C" for the parent.

Mental model. **def** plants a flag. **over** writes on a flag that should already be there. **class** plants a flag in the “templates” drawer.

Simple example. A shot layer should **over** the referenced character to set a pose, not **def** a second character at the same path (unless you really mean to define it locally).

USDA example. File: *defs.usda* — a **def** and a **class** in one layer

```
#usda 1.0

def Xform "World"
{
}

class "_class_Ball"
{
}
```

File: *overlay.usda* — **over** belongs in a different layer; two specs for the same prim in one layer are illegal

```
#usda 1.0

over "World"
{
}
```

Python example.

```
from pxr import Usd

stage = Usd.Stage.CreateInMemory()
defined = stage.DefinePrim("/A", "Xform")
overlay = stage.OverridePrim("/C/D")
klass = stage.CreateClassPrim("/_Class")
print("A specifier/defined/abstract:", defined.GetSpecifier(), defined.IsDefined(), defined.IsAbstract())
print("D specifier/defined:", overlay.GetSpecifier(), overlay.IsDefined())
print("C specifier/defined:", stage.GetPrimAtPath("/C").GetSpecifier(), stage.GetPrimAtPath("/C").IsDefined())
print("class specifier/defined/abstract:", klass.GetSpecifier(), klass.IsDefined(), klass.IsAbstract())
```

```
print("Traverse:", [str(p.GetPath()) for p in stage.Traverse()])
print("TraverseAll:", [str(p.GetPath()) for p in stage.TraverseAll()])
```

Expected output

```
A specifier/defined/abstract: Sdf.SpecifierDef True False
D specifier/defined: Sdf.SpecifierOver False
C specifier/defined: Sdf.SpecifierOver False
class specifier/defined/abstract: Sdf.SpecifierClass True True
Traverse: ['/A']
TraverseAll: ['/A', '/C', '/C/D', '/_Class']
```

Real-world use case. Inherits (Ch 19) point at class prims. The class holds default looks; instances def their own prims and inherit the class.

Common mistakes.

MISTAKE

Authoring only `over "Hero"` in a layer that is opened **by itself**. The prim is not defined; usdview looks empty. Someone must `def` it (often via a reference).

MISTAKE

Using typeless `def "Hero"` as a "soft" overlay. A typeless `def` ***does*** define the prim. If you meant overlay, use `over`.

Exam traps.

TRAP

"over is weaker than def." Not for values. Strength is stack/arcs. Specifier is existence.

TRAP

Expecting `class` prims in `Traverse()`. They are abstract.

Exam takeaways.

KEY

- `def` defines. `over` overlays (not defined by itself). `class` is abstract. - Specifier $\neq$ value strength. - Prefer `over` in shot layers that override referenced assets.

4.4 Typed vs. typeless prims

What is it? A **typed** prim has a schema type name (Xform, Sphere, Mesh). A **typeless** prim has an empty type name.

Why do we need it? Schemas give you attributes and APIs (GetRadiusAttr()). Typeless prims are still valid grouping nodes or placeholders.

Beginner explanation. Typed = labeled box ("this is a Sphere"). Typeless = unlabeled box (you can still put things in it).

Technical explanation. - GetTypeNames() returns "" when typeless. -

IsA(UsdGeom.Sphere) is False for typeless. - A typeless def still IsDefined(). - You can set a type later (prim.SetTypeName("Xform")). - Composition can provide the type from a weaker layer and opinions from a stronger typeless over.

Mental model. Type is just another opinion (a special one). It can come from a different layer than the properties.

Simple example. Root: over "Ball" { double radius = 2 }. Weaker sublayer: def Sphere "Ball" {}. Composed: a Sphere with radius 2.

USDA example.

```
#usda 1.0
```

```
def "Group"
{
    def Sphere "Ball"
    {
    }
}
```

Group is typeless; Ball is typed.

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
group = stage.DefinePrim("/Group")
ball = stage.DefinePrim("/Group/Ball", "Sphere")
print("Group type:", repr(group.GetTypeName()), "IsA Xform:", group.IsA(UsdGeom.Xform))
print("Ball type:", ball.GetTypeName(), "IsA Sphere:", ball.IsA(UsdGeom.Sphere))
```

Expected output

```
Group type: '' IsA Xform: False
Ball type: Sphere IsA Sphere: True
```

Real-world use case. Some pipelines `def Xform` for every group so tools can transform them. Others leave scope-only groups as `Scope` or typeless. Be consistent (Ch 32).

Common mistakes.

MISTAKE

Calling ``UsdGeom.Sphere(typelessPrim).GetRadiusAttr().Set(2)`` and assuming it is a sphere. Check ``IsA`` (or Define through the schema class).

Exam traps.

TRAP

"Typeless prims are illegal." They are legal. The FAQ contrast is `over` vs `*typeless def*` — both exist, different existence rules.

Exam takeaways.

KEY

- Type name is optional; schemas need it. - Typeless ``def`` still defines. Typeless ``over`` still does not.

4.5 Active and inactive prims

What is it? A prim is **active** by default. **Deactivating** it (`active = false`) removes it from default traversal and typical imaging, without deleting authored data.

Why do we need it? Hide a character from a shot, or prune a branch, without destroying the layers that define it. Unlike visibility (Ch 39), inactivity is a namespace-level switch.

Beginner explanation. Active = on the org chart. Inactive = still in the filing cabinet, not invited to the meeting.

Where the analogy breaks: children of an inactive prim are not visited by `Traverse()` even if they are themselves active.

Technical explanation. - `prim.IsActive()`, `prim.SetActive(False)`. - USDA: `def Sphere "Ball" ( active = false ) { }` - `Traverse()` skips inactive prims. `TraverseAll()` includes them. - Deactivation is an opinion — a stronger layer can reactivate.

Mental model. Inactive = pruned from the working set. Data remains.

Simple example. Layout deactivates `/World/TempBlockout` in a stronger layer so lighting never sees it.

USDA example.

```
#usda 1.0

def Xform "World"
{
    def Sphere "Ball" (
        active = false
    )
    {
        double radius = 2
    }
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
UsdGeom.Xform.Define(stage, "/World")
ball = UsdGeom.Sphere.Define(stage, "/World/Ball")
print("active:", ball.GetPrim().IsActive())
print("Traverse:", [str(p.GetPath()) for p in stage.Traverse()])
ball.GetPrim().SetActive(False)
print("after inactive, Traverse:", [str(p.GetPath()) for p in stage.Traverse()])
print("TraverseAll:", [str(p.GetPath()) for p in stage.TraverseAll()])
```

Expected output

```
active: True
Traverse: ['/World', '/World/Ball']
after inactive, Traverse: ['/World']
TraverseAll: ['/World', '/World/Ball']
```

Real-world use case. A variant (Ch 18) might deactivate whole geometry branches for a low-LOD choice.

Common mistakes.**MISTAKE**

Confusing `active = false` with `visibility = "invisible"`. Visibility is an imageable property; the prim still exists in traversal. Inactivity removes it from `Traverse()`.

Exam traps.**TRAP**

"Deactivating deletes the prim from layers." It authors an opinion. Weaker defs remain.

Exam takeaways.**KEY**

- Default: active. `Traverse()` skips inactive; `TraverseAll()` does not. - Inactivity $\neq$ visibility $\neq$ deletion.

4.6 Namespace hierarchy

What is it? **Namespace** is the tree formed by prim paths. Parents own children. Composition can add children from references under a parent prim.

Why do we need it? Transform inheritance, primvar inheritance (Ch 11), material binding (Ch 40), and model hierarchy (Ch 6) all walk ancestors.

Beginner explanation. Folders in a file browser, but each folder is a prim that can also have properties.

Where the analogy breaks: a referenced asset brings its *internal* tree in under the referencing prim, possibly renaming the root via `defaultPrim`.

Technical explanation. - `prim.GetChildren()`, `GetParent()`, `GetAllChildren()`. - Sibling order is the order they were authored (stable enough for tests if you define in order). - Do not encode variant identity in path names if you can use variant sets (Ch 18); paths that churn break bindings.

Mental model.

```
/World      Xform
/Lights     Xform or Scope
/Key        DistantLight
/Sets
/Building
/Chars
/Hero
```

Simple example. Putting materials under `/World/Looks` and meshes under `/World/Geo` is a namespace convention, not a USD requirement — but pipelines depend on conventions (Ch 23).

USDA example.

```
#usda 1.0
(
    defaultPrim = "World"
)

def Xform "World"
{
    def Scope "Geo"
    {
        def Mesh "Body" {}
    }
    def Scope "Looks"
    {
        def Material "Default" {}
    }
}
```

Python example.

```
from pxr import Usd, UsdGeom, UsdShade

stage = Usd.Stage.CreateInMemory()
UsdGeom.Xform.Define(stage, "/World")
UsdGeom.Scope.Define(stage, "/World/Geo")
UsdGeom.Mesh.Define(stage, "/World/Geo/Body")
UsdGeom.Scope.Define(stage, "/World/Looks")
UsdShade.Material.Define(stage, "/World/Looks/Default")
world = stage.GetPrimAtPath("/World")
print("children:", [c.GetName() for c in world.GetChildren()])
print("all paths:", [str(p.GetPath()) for p in stage.Traverse()])
```

Expected output

```
children: ['Geo', 'Looks']
all paths: ['/World', '/World/Geo', '/World/Geo/Body', '/World/Looks', '/World/Looks/Default']
```

Real-world use case. ASWF-style assets put geometry under `geo` and materials under `mtl` (Ch 23). Bindings use paths inside that tree.

Common mistakes.

MISTAKE

Deeply nesting every `gprim` under extra empty groups "for organization" until model hierarchy becomes expensive (Ch 23, 26).

Exam traps.

TRAP

Assuming USD requires `/World` as the root name. It is a convention. `defaultPrim` is the real entry point.

Exam takeaways.**KEY**

- Namespace = path tree. Children, parents, ancestors. - Conventions (`/World`, `geo`, `mtl`) are pipeline design, not core syntax. - `defaultPrim` marks the asset's entry prim.

Chapter lab(s)

Labs 06–07 start properties; prim/path practice is this chapter's examples plus USDA-05.

USDA reading exercise

USDA-05. In this layer, which prims does `Traverse()` visit, and why is `_Class` missing?

```
#usda 1.0

def Xform "World" {}
class "_Class" {}
over "Ghost" {}
```

Answer: Only `/World`. `_Class` is abstract. `/Ghost` is an `over` and not defined.

Chapter review**If you see... → think...**

If you see...	Think...
<code>/A/B/C</code> vs <code>/A/B.c</code>	Prim path vs property path
Empty usdview on a shot overlay	You used <code>over</code> without a <code>def</code> /reference
Prim missing from <code>Traverse</code>	Inactive, abstract, or undefined
<code>def "Group"</code>	Typeless defined prim

CHAPTER 5

Chapter 5 — Properties: Attributes, Relationships, Metadata

Exam domain: Data Modeling (13%) · **Objectives:** 5.3, 5.4 **Study day:** 2 · **Est. time:** 90 min **Prerequisites:** Chapter 4

Learning goals

- Retrieve a prim's properties (Obj 5.4).
- Author **attributes** (value, type, variability).
- Author **relationships** and list their targets.
- Read and write **metadata** on prims, properties, and layers (Obj 5.3 intro).
- Recognize property **namespaces** (primvars:, xformOp:, inputs:).

Key terms

Term	One-line definition
Property	Either an attribute or a relationship on a prim
Attribute	A typed value (possibly time-sampled)
Relationship	A pointer to other prims or properties (targets)
Metadata	Named data that is not a property (on layer, prim, or property)
Variability	varying (can have time samples) vs uniform (one value)
Namespace	The prefix before : in a property name

5.1 Properties

What is it? A **property** is either an **attribute** or a **relationship**. Both live on a prim and have a name.

Why do we need it? Obj 5.4: you must retrieve “the properties of a prim” — not only attributes, not only relationships.

Beginner explanation. If a prim is a form, properties are the fields. Some fields hold numbers (attributes). Some fields hold arrows to other forms (relationships).

Technical explanation. - `prim.GetPropertyNames()`, `GetProperties()`, `GetAttributes()`, `GetRelationships()`. - `prim.GetProperty("points")` returns a `Usd.Attribute` or `Usd.Relationship` subclass of `Usd.Property`. - `isinstance(prop, Usd.Attribute) / Usd.Relationship`. - Schema properties appear even before you author values (`HasAuthoredValue()` may be `False`).

Mental model.

```

Usd.Property
├── Usd.Attribute    (values)
└── Usd.Relationship (targets)

```

Simple example. A Mesh prim already *has* a `points` attribute from its schema. Until you set it, it has no authored value.

USDA example.

```

#usda 1.0

def Mesh "Body"
{
    point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
}

```

```
rel material:binding = </Looks/Red>
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/Body")
prim = mesh.GetPrim()
names = prim.GetPropertyNames()
print("has points:", "points" in names)
print("has proxyPrim rel:", "proxyPrim" in names)
print("attribute count:", len(prim.GetAttributes()))
print("relationship count:", len(prim.GetRelationships()))
prop = prim.GetProperty("points")
print("points is Attribute:", isinstance(prop, Usd.Attribute))
print("points HasAuthoredValue:", mesh.GetPointsAttr().HasAuthoredValue())
```

Expected output

```
has points: True
has proxyPrim rel: True
attribute count: 24
relationship count: 1
points is Attribute: True
points HasAuthoredValue: False
```

Real-world use case. A generic inspector panel lists `GetPropertyNames()` and then branches: if attribute, show value; if relationship, show targets.

Common mistakes.

MISTAKE

Using only ``GetAttributes()`` when the question says "properties". You will miss relationships.

Exam traps.

TRAP

"Metadata is a property." Metadata is a separate channel (section 5.4).

Exam takeaways.

KEY

- Property = attribute or relationship. - Schema properties exist before you author values.

5.2 Attributes

What is it? An **attribute** holds a typed value: a float, a `point3f[]`, a token, and so on. It may have a **default** value and/or **time samples** (Ch 10).

Why do we need it? Almost all numeric and string scene data is attributes: radius, points, visibility, shader inputs.

Beginner explanation. An attribute is a labeled cell with a type. `double radius = 2` means "a double named radius whose default is 2".

Technical explanation. - `attr.Get()` / `Set(value)` for the default (or at a time code, Ch 10). - `GetTypeName()` returns an `Sdf.ValueTypeName` (`double`, `point3f[]`, ...). - `GetVariability()`: `Sdf.VariabilityVarying` or `Sdf.VariabilityUniform`. - `IsCustom()`: `True` for properties you created that are not from a schema. - `CreateAttribute(name, Sdf.ValueTypeNames.Float)` on a prim. - Uniform attributes cannot (meaningfully) time-sample; they are "one value for all time".

Mental model. Attribute = typed value slot. Default vs time samples = Ch 10.

Simple example. `double radius = 2` on a Sphere is a schema attribute, not custom.

USDA example.

```
#usda 1.0

def Sphere "Ball"
{
    double radius = 2
    custom bool extraFlag = 1
    uniform token purpose = "default"
}
```

`custom` marks a non-schema attribute. `uniform` marks variability.

Python example.

```
from pxr import Usd, UsdGeom, Sdf

stage = Usd.Stage.CreateInMemory()
ball = UsdGeom.Sphere.Define(stage, "/Ball")
radius = ball.GetRadiusAttr()
print("type:", radius.GetTypeName())
print("variability:", radius.GetVariability())
print("custom:", radius.IsCustom(), "HasAuthoredValue:", radius.HasAuthoredValue())
radius.Set(2.0)
print("value:", radius.Get())
flag = ball.GetPrim().CreateAttribute("extraFlag", Sdf.ValueTypeNames.Bool)
flag.Set(True)
print("extraFlag custom:", flag.IsCustom(), "value:", flag.Get())
```

Expected output

```
type: double
variability: Sdf.VariabilityVarying
custom: False HasAuthoredValue: False
value: 2.0
extraFlag custom: True value: True
```

Real-world use case. Choosing `float` vs `double` vs `float3` vs `color3f` is Obj 5.2 (Ch 9). Wrong types break shaders and extents.

Common mistakes.**MISTAKE**

Calling ``Get()`` without ``Set()`` and assuming you always get a useful default. Some schema attributes have fallbacks (``radius`` does); others return ``None`` until authored (``points``).

Exam traps.**TRAP**

"All attributes are custom." Schema attributes are not custom.

Exam takeaways.**KEY**

- Attributes are typed; variability is uniform or varying. - ``HasAuthoredValue`` $\neq$ "Get returned a number".

5.3 Relationships

What is it? A **relationship** stores **targets**: paths to other prims or properties. It does not store a numeric value.

Why do we need it? Material bindings, proxy prims, connections, collections — anything that *points* uses a relationship.

Beginner explanation. A sticky arrow from this prim to that prim.

Where the analogy breaks: a relationship can have **several** targets, and list-editing (prepend/append/delete, Ch 14) applies.

Technical explanation. - `prim.CreateRelationship("look:material")` - `rel.AddTarget(path)`, `SetTargets(list)`, `GetTargets()`. - USDA: `rel material:binding = </Looks/Red>` - Targets are `Sdf.Paths`. They may be invalid (broken bindings) — USD will still store them. - `MaterialBindingAPI` must be **applied** before official material binding (Ch 40). You can still create a raw relationship here to learn the type.

Mental model. Attribute = value. Relationship = list of paths.

Simple example. `proxyPrim` on imageable prims points at a lighter proxy.

USDA example.

```
#usda 1.0

def Mesh "Body"
{
    rel proxyPrim = </BodyProxy>
}

def Mesh "BodyProxy"
{
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
body = UsdGeom.Mesh.Define(stage, "/Body")
proxy = UsdGeom.Mesh.Define(stage, "/BodyProxy")
rel = body.GetPrim().CreateRelationship("look:dest")
rel.AddTarget(proxy.GetPath())
print("is Relationship:", isinstance(rel, Usd.Relationship))
print("targets:", [str(t) for t in rel.GetTargets()])
print("namespace:", rel.GetNamespace(), "baseName:", rel.GetBaseName())
```

Expected output

```
is Relationship: True
targets: ['/BodyProxy']
namespace: look baseName: dest
```

Real-world use case. A broken `material:binding` that points outside `defaultPrim` is a classic export error (data exchange). The relationship exists; the target is just not part of the asset.

Common mistakes.

MISTAKE

Setting a relationship with ``Set(1.0)`` like an attribute. Use targets.

Exam traps.

TRAP

"Relationships store the material's color." They store ****paths****. Colors live on shader attributes.

Exam takeaways.

KEY

- Relationships point. Attributes value. - Always inspect targets when a binding "does nothing".

5.4 Metadata

What is it? **Metadata** is named data on a layer, prim, or property that is **not** a property. Examples: `kind`, `documentation`, `active`, `instanceable`, `defaultPrim`, `hidden`, `customData`.

Why do we need it? Obj 5.3 / 7.6: pipelines store non-schema info (shot codes, asset IDs) without pretending it is geometry.

Beginner explanation. Properties are columns in the table. Metadata is the sticky note on the table, the folder, or the whole file.

Where the analogy breaks: some things that *look* like properties in USDA (active = false in parentheses) are metadata.

Technical explanation. - Prim metadata sits in parentheses in USDA: `def Xform "W" ( kind = "component" )`. - Layer metadata sits in the header block. - `prim.GetMetadata("kind"), SetMetadata("documentation", "...")`. - **Registered** metadata has a known name and type. **Custom** extra dictionaries: `customData` and `assetInfo`. - `SetCustomDataByKey("shot", "s001") / GetCustomData()`. - `Usd.ModelAPI(prim).SetKind(...)` is the kind-aware API (Ch 6).

VERSION

Schema prims may already contain documentation inside ``customData`` (``userDocBrief``). Merging your keys keeps those entries. Verified on USD 26.08 for ``UsdGeom.Mesh``.

Mental model. Header / parentheses = metadata. Body with `type name = value` = properties.

Simple example. `kind = "component"` does not render. It tells asset tools “this is a publishable asset”.

USDA example.

```
#usda 1.0
(
    defaultPrim = "Chair"
    doc = "Hero chair v12"
)

def Xform "Chair" (
    kind = "component"
    customData = {
        string shot = "s001"
    }
)
{
}
```

Python example.

```
from pxr import Usd, UsdGeom, Kind

stage = Usd.Stage.CreateInMemory()
prim = UsdGeom.Xform.Define(stage, "/Chair").GetPrim()
prim.SetMetadata("documentation", "Hero chair")
Usd.ModelAPI(prim).SetKind(Kind.Tokens.component)
prim.SetCustomDataByKey("shot", "s001")
print("documentation:", prim.GetMetadata("documentation"))
print("kind:", Usd.ModelAPI(prim).GetKind())
print("shot key:", prim.GetCustomDataByKey("shot"))
print("IsModel:", prim.IsModel())
```

Expected output

```
documentation: Hero chair
kind: component
shot key: s001
IsModel: True
```

Real-world use case. A publisher writes `assetInfo` (identifier, version) so a resolver and a database can find the asset (Ch 12, 33).

Common mistakes.

MISTAKE

Creating an attribute named `kind` instead of setting kind metadata. Tools looking at `ModelAPI` will not see it.

Exam traps.**TRAP**

"customData is a relationship." It is a metadata dictionary.

Exam takeaways.**KEY**

- Metadata $\neq$ properties. Look in USDA parentheses and headers. - `customData` / `assetInfo` are the usual extension dictionaries.

5.5 Property namespaces

What is it? A **namespace** is the prefix of a property name before a colon:

`primvars:displayColor`, `xformOp:translate`, `inputs:intensity`, `material:binding`.

Why do we need it? Namespaces group properties, prevent collisions, and tell APIs which properties they own. Obj 5.1 (primvars) and visualization domains depend on this.

Beginner explanation. Like `primvars`, folders inside the prim, but the folder name is part of the property name.

Where the analogy breaks: the colon is the official separator. Nested namespaces exist (`primvars:ri:interpolation` in some pipelines).

Technical explanation. - `prop.GetName()` → full name. `GetNamespace()` → prefix. `GetBaseName()` → last piece. - `primvars`: — geometric primitive variables (Ch 11). - `xformOp`: — transform ops; order lives in `xformOpOrder` (Ch 39). Note: `xformOpOrder` itself has an **empty** namespace. - `inputs` / `outputs`: — shader and light connectable attributes (UsdLux `inputs:intensity` since USD 21.02). - `material:binding` — relationship namespace `material`, base name `binding`.

Mental model. `namespace:baseName` — always split on the last colon for the base name; `GetNamespace()` returns the rest.

Simple example. `primvars:displayColor` is not a mysterious second color system; it is a primvar named `displayColor`.

USDA example.

```
#usda 1.0

def Mesh "Body"
{
    color3f[] primvars:displayColor = [(1, 0, 0)]
    float3 xformOp:translate = (0, 10, 0)
    uniform token[] xformOpOrder = ["xformOp:translate"]
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/Body")
color = mesh.GetPrim().GetProperty("primvars:displayColor")
order = mesh.GetPrim().GetProperty("xformOpOrder")
print("displayColor name:", color.GetName())
print("displayColor namespace:", color.GetNamespace(), "base:", color.GetBaseName())
print("xformOpOrder namespace empty:", order.GetNamespace() == "")
print("xformOpOrder base:", order.GetBaseName())
```

Expected output

```
displayColor name: primvars:displayColor
displayColor namespace: primvars base: displayColor
xformOpOrder namespace empty: True
xformOpOrder base: xformOpOrder
```

Real-world use case. UsdLux lights use `inputs:intensity` so the intensity can be connected in a shading graph. Old files used a bare `intensity` attribute (VERSION note, Ch 41).

Common mistakes.

MISTAKE

Creating ``displayColor`` without the ``primvars:`` prefix and wondering why Hydra ignores it.

Exam traps.

TRAP

"xformOpOrder is namespaced ``xformOp``." Its name is literally ``xformOpOrder`` with no colon.

Exam takeaways.

KEY

- Colons are namespaces. ``GetNamespace()`` / ``GetBaseName()``. - Remember ``primvars:``, ``xformOp:``, ``inputs:``, ``material:``. - ``xformOpOrder`` has no namespace.

Chapter lab(s)

Labs 06–08 (attributes, relationships, metadata) map here.

USDA reading exercise

USDA-06. How many properties are authored on `Body`, and which is a relationship?

```
#usda 1.0
def Mesh "Body" {
  point3f[] points = [(0, 0, 0)]
  rel material:binding = </Looks/Red>
}
```

Answer: Two authored properties in the layer: `points` (attribute) and `material:binding` (relationship). The schema still *has* many other unauthored properties.

Chapter review

If you see... → think...

If you see...	Think...
"Retrieve properties"	Attributes and relationships
Parentheses in USDA	Metadata
<code>rel</code> keyword	Relationship / targets
A colon in a property name	Namespace

CHAPTER 6

Chapter 6 — Schemas and Model Kinds

Exam domain: Customizing USD (6%) and Data Modeling · **Objectives:** 3.3, 3.4 (intro; custom schemas in Ch 37) **Study day:** 2 · **Est. time:** 75 min **Prerequisites:** Chapter 5

Learning goals

- Explain what a **schema** is and why USD uses them.
- Distinguish **typed (IsA) schemas** from **API schemas**.
- Distinguish **single-apply** vs **multiple-apply** API schemas.
- Name the built-in schema domains you must know for the exam.
- Use **model kinds**: model, group, assembly, component, subcomponent.

Key terms

Term	One-line definition
Schema	A contract: prim type or API that defines properties and meaning
Typed / IsA schema	Gives the prim its type name (Mesh, Sphere); <code>prim.IsA(...)</code>
API schema	Extra bundle of properties applied onto a prim (HasAPI)
Single-apply	At most one application per prim (MaterialBindingAPI)
Multiple-apply	Named instances (CollectionAPI:include)
Kind	Extensible category for the model hierarchy (component, assembly, ...)

6.1 What a schema is

What is it? A **schema** is a named contract that says “this prim has these properties and this meaning.” `UsdGeom.Mesh` is a schema. So is `UsdShade.MaterialBindingAPI`.

Why do we need it? Without schemas, every tool would invent its own `points` vs `P` vs `vertices`. Schemas are how OpenUSD stays a *standard* (Obj 3.4, 3.6).

Beginner explanation. A schema is a form template. “Mesh form” always has `points` and `faceVertexIndices`. Tools can fill the form without a meeting.

Where the analogy breaks: applying an API schema is like stapling a second form onto the first (material binding on a mesh).

Technical explanation. - Schemas are generated from a `schema.usda` (Ch 37) into C++/Python classes. - **IsA / typed schemas** inherit `UsdTyped`. They set `typeName`. - **API schemas** inherit `UsdAPISchemaBase`. They do not change `typeName`; they add `apiSchemas` metadata. - Fallback values, doc strings, and property types all come from the schema.

Mental model. Type name = which IsA schema. `apiSchemas` list = extra API schemas.

Simple example. `def Mesh "Body" (` means IsA schema `Mesh`. Adding `prepend apiSchemas = ["MaterialBindingAPI"]` applies the binding API.

USDA example.

```
#usda 1.0

def Mesh "Body" (
    prepend apiSchemas = ["MaterialBindingAPI"]
)
{
    rel material:binding = </Looks/Red>
}
```

Python example.

```

from pxr import Usd, UsdGeom, UsdShade

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/Body")
prim = mesh.GetPrim()
print("typeName:", prim.GetTypeName())
print("IsA Mesh:", prim.IsA(UsdGeom.Mesh))
print("HasAPI before:", prim.HasAPI(UsdShade.MaterialBindingAPI))
UsdShade.MaterialBindingAPI.Apply(prim)
print("HasAPI after:", prim.HasAPI(UsdShade.MaterialBindingAPI))
print("applied:", prim.GetAppliedSchemas())

```

Expected output

```

typeName: Mesh
IsA Mesh: True
HasAPI before: False
HasAPI after: True
applied: ['MaterialBindingAPI']

```

Real-world use case. A CAD importer can put extra bolts-and-threads data on an API schema instead of inventing a rival Mesh type (Obj 3.6).

Common mistakes.

MISTAKE

Authoring ``rel material:binding`` without applying ``MaterialBindingAPI``. Validators and ``ComputeBoundMaterial`` expect the API applied (Ch 40).

Exam traps.

TRAP

"API schemas change the prim's typeName to MaterialBindingAPI." They do not. Type stays ``Mesh``.

Exam takeaways.

KEY

- Schema = contract. IsA changes typeName; API schemas append to ``apiSchemas``.

6.2 Typed (IsA) schemas

What is it? A **typed schema** (IsA schema) is the prim's type: Xform, Mesh, Camera, Material. Query with `prim.IsA(UsdGeom.Mesh)`.

Why do we need it? Traversal filters ("all meshes") and schema APIs (`UsdGeom.Mesh(prim).GetPointsAttr()`) depend on IsA, including **inherited** types (Mesh is a Gprim is a Boundable is an Xformable is Imageable).

Beginner explanation. IsA is the "this is a mesh" test, and it understands parent classes: a Mesh is a Gprim.

Technical explanation. Verified: `UsdGeom.Mesh IsA Gprim, Boundable, Xformable, Imageable`.

Python: `prim.IsA(UsdGeom.Gprim)`.

You cannot apply two typed schemas as the type name; a prim has one type (composition may still provide it from a weaker layer).

Mental model.

```

Imageable
  Xformable
    Boundable
      Gprim
        Mesh, Sphere, Cube, ...

```

```
Camera (xformable, not a gprim)
Scope (imageable, not xformable in the same way – see Ch 13)
```

(Exact inheritance is Ch 13. Remember Mesh → Gprim → Boundable for extents.)

Simple example. A bounding-box tool asks `IsA(UsdGeom.Boundable)` rather than listing every gprim type.

USDA example.

```
#usda 1.0

def Mesh "Body" {}
def Sphere "Ball" {}
def Camera "ShotCam" {}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/Body").GetPrim()
print("Mesh:", mesh.IsA(UsdGeom.Mesh))
print("Gprim:", mesh.IsA(UsdGeom.Gprim))
print("Boundable:", mesh.IsA(UsdGeom.Boundable))
print("Xformable:", mesh.IsA(UsdGeom.Xformable))
print("Imageable:", mesh.IsA(UsdGeom.Imageable))
print("Sphere?:", mesh.IsA(UsdGeom.Sphere))
```

Expected output

```
Mesh: True
Gprim: True
Boundable: True
Xformable: True
Imageable: True
Sphere?: False
```

Real-world use case. Extent computation (Obj 5.6) only makes sense on Boundables (Ch 13).

Common mistakes.

MISTAKE

Wrapping ``UsdGeom.Sphere(prim)`` without ``IsA`` — silent nonsense on the wrong type.

Exam traps.

TRAP

"Camera is a Gprim." Cameras are Xformable/Imageable, not polygonal Gprims.

Exam takeaways.

KEY

- ``IsA`` walks the typed-schema inheritance. - One type name per prim.

6.3 API schemas: single-apply vs multiple-apply

What is it? API schemas add optional capabilities. **Single-apply:** one per prim (MaterialBindingAPI). **Multiple-apply:** many, each with an instance name (CollectionAPI:include).

Why do we need it? Obj 3.4: custom APIs are how you extend prims. A typical “add physics-like properties to existing geometry” schema subclasses `UsdAPISchemaBase`, not `UsdTyped`.

Beginner explanation. Single-apply = one “Material Binding” stamp on the mesh. Multiple-apply = many named “Collection” stamps (include, exclude).

Technical explanation. - Apply: `UsdShade.MaterialBindingAPI.Apply(prim) → apiSchemas` includes `MaterialBindingAPI`. - Multiple-apply: `Usd.CollectionAPI.Apply(prim, "include") → CollectionAPI:include`. - Query: `prim.HasAPI(UsdShade.MaterialBindingAPI), GetAppliedSchemas()`. - Custom APIs you write later inherit `UsdAPISchemaBase` (Ch 37). Typed custom schemas inherit `UsdTyped`.

Mental model.

```
typeName: Mesh
apiSchemas: [MaterialBindingAPI, CollectionAPI:include]
```

Simple example. Collections for light linking use multiple-apply `CollectionAPI` with names per collection.

USDA example.

```
#usda 1.0

def Mesh "Body" (
    prepend apiSchemas = ["MaterialBindingAPI", "CollectionAPI:include"]
)
{
}
```

Python example.

```
from pxr import Usd, UsdGeom, UsdShade

stage = Usd.Stage.CreateInMemory()
prim = UsdGeom.Mesh.Define(stage, "/Body").GetPrim()
UsdShade.MaterialBindingAPI.Apply(prim)
Usd.CollectionAPI.Apply(prim, "include")
print(prim.GetAppliedSchemas())
print("Has MaterialBindingAPI:", prim.HasAPI(UsdShade.MaterialBindingAPI))
```

Expected output

```
['MaterialBindingAPI', 'CollectionAPI:include']
Has MaterialBindingAPI: True
```

Real-world use case. A studio's `StudioPublishAPI` (single-apply) adds `studio:assetId`. Multiple shots still share `Mesh`.

Common mistakes.

MISTAKE

Applying `CollectionAPI` without an instance name. Multiple-apply requires the name.

Exam traps.

TRAP

Custom `*physics` properties on existing geometry* → API schema (``UsdAPISchemaBase``), not a new typed `Mesh` subclass. (Original question; same idea as the study-guide sample, not a copy.)

Exam takeaways.

KEY

- Single-apply vs multiple-apply (named instances). - Extra properties on existing types → API schema. - New stand-alone prim types → typed / `IsA` schema.

6.4 Built-in schema domains

What is it? Schema domains are modules of related schemas: `UsdGeom`, `UsdShade`, `UsdLux`, `UsdPhysics`, `UsdSkel`, plus others (`UsdMedia`, `UsdVol`, `UsdUI`, ...).

Why do we need it? Visualization (8%) tests UsdGeom, UsdShade, UsdLux. Physics appears in the study-guide reading list. You must know which domain to import.

Beginner explanation. Geom = shape and transform. Shade = materials. Lux = lights. Physics = simulation. Skel = skeletons / blend shapes.

Technical explanation. | Domain | Typical types | Exam | |-----|-----|-----| |
 UsdGeom | Xform, Scope, Mesh, Sphere, Camera, PointInstancer, PrimvarsAPI | Heavy
 (Ch 39, 25) | | UsdShade | Material, Shader, NodeGraph, MaterialBindingAPI | Ch 40 | |
 UsdLux | DistantLight, RectLight, DomeLight, LightAPI | Ch 41 | | UsdPhysics | Rigid
 bodies, joints (overview) | Pipeline reading list | | UsdSkel | Skeleton, BlendShape |
 Awareness |

usd-core 26.08 includes these Python modules (no UsdMtlx, no UsdImaging).

Mental model. Import the domain you mean: `from pxr import UsdGeom, UsdShade, UsdLux`.

Simple example. A light is not a Mesh. `UsdLux.DistantLight.Define(stage, "/Lights/Key")`.

USDA example.

```
#usda 1.0

def DistantLight "Key"
{
}
```

Python example.

```
from pxr import Usd, UsdGeom, UsdShade, UsdLux, UsdPhysics, UsdSkel

stage = Usd.Stage.CreateInMemory()
print("Mesh", UsdGeom.Mesh.Define(stage, "/M").GetPrim().GetTypeName())
print("Material", UsdShade.Material.Define(stage, "/Mat").GetPrim().GetTypeName())
print("DistantLight", UsdLux.DistantLight.Define(stage, "/L").GetPrim().GetTypeName())
print("modules", UsdPhysics is not None, UsdSkel is not None)
```

Expected output

```
Mesh Mesh
Material Material
DistantLight DistantLight
modules True True
```

Real-world use case. A game exporter writes UsdGeom + UsdShade + a custom physics API, not a private “GameMesh” type, so usdview can still draw it.

Common mistakes.

MISTAKE

Putting lights under `UsdGeom` because they are “in the scene”. Use UsdLux.

Exam traps.

TRAP

Visualization questions naming UsdGeom/UsdShade/UsdLux specifically — do not answer with Sdf layer APIs.

Exam takeaways.

KEY

- Geom / Shade / Lux are the visualization trio. - Physics and Skel exist; know they are domains.

6.5 Model kinds

What is it? **Kind** is metadata that places a prim in the **model hierarchy**: a shallow table of contents for the stage. Core kinds: `model`, `group`, `assembly`, `component`, plus `subcomponent` (not a model).

Why do we need it? Obj 3.3: custom kinds when appropriate. Obj 2.4 / 7.2: asset structure. Traversal and instancing tools prune at **component** boundaries.

Beginner explanation. If the prim tree is every screw, the model hierarchy is the furniture: this is a *chair component*, this is a *room assembly*.

Where the analogy breaks: `subcomponent` is a kind for pieces **inside** a component (a lid, a wheel). It is **not** a model. Verified: `Kind.Registry.IsA("subcomponent", "model")` is `False`.

Technical explanation. Verified on USD 26.08 (`Kind.Tokens`): `model`, `group`, `assembly`, `component`, `subcomponent`.

Kind	IsA group?	IsA model?	Role
model		yes	Base category
group	yes	yes	Organizational model
assembly	yes (hence model)	yes	Aggregates other models
component		yes	Leaf publishable asset ("the chair")
subcomponent		no	Piece inside a component

```
Python: Usd.ModelAPI(prim).SetKind(Kind.Tokens.component), prim.IsModel(),
prim.IsGroup(), Kind.Registry.IsA("component", "model").
```

Rule of thumb from asset-structure docs: keep the model hierarchy **shallow**. A Gprim tagged `component` is a smell (too deep).

Mental model.

```
assembly (set / shot)
  group
    component (chair)    <-- prune point
      subcomponent (seat)
        Mesh             <-- not a model
```

Simple example. Publish `Chair` as `component`. The shot references many chairs into an assembly named `Set`.

USDA example.

```
#usda 1.0
(
  defaultPrim = "Chair"
)

def Xform "Chair" (
  kind = "component"
)
{
  def Mesh "Seat" (
    kind = "subcomponent"
  )
  {
  }
}
```

Python example.

```
from pxr import Usd, UsdGeom, Kind

stage = Usd.Stage.CreateInMemory()
chair = UsdGeom.Xform.Define(stage, "/Chair").GetPrim()
Usd.ModelAPI(chair).SetKind(Kind.Tokens.component)
seat = UsdGeom.Mesh.Define(stage, "/Chair/Seat").GetPrim()
```

```

Usd.ModelAPI(seat).SetKind(Kind.Tokens.subcomponent)
print("component is model:", Kind.Registry.IsA("component", "model"))
print("assembly is group:", Kind.Registry.IsA("assembly", "group"))
print("subcomponent is model:", Kind.Registry.IsA("subcomponent", "model"))
print("chair IsModel/IsGroup:", chair.IsModel(), chair.IsGroup())
print("seat IsModel:", seat.IsModel())

```

Expected output

```

component is model: True
assembly is group: True
subcomponent is model: False
chair IsModel/IsGroup: True False
seat IsModel: False

```

Real-world use case. Payload unloading (Ch 17) often stops at component models so a shot can open as a “table of contents” in under a second (performance guide).

Common mistakes.

MISTAKE

Marking every Mesh a `component`. That deepens the model hierarchy and costs composition time.

MISTAKE

Using `group` as the published asset's root kind. ASWF guidance: groups organize insides of assemblies; published roots are usually `component` or `assembly`.

Exam traps.

TRAP

"subcomponent is a model." It is a kind, but not in the model hierarchy.

TRAP

Kind vs typeName: `kind = "component"` does not make `GetTypeName()` return `component`. Type stays `Xform`.

Exam takeaways.

KEY

- Kind is metadata for the model hierarchy, not the prim type. - component / assembly / group are models. subcomponent is not. - Keep the hierarchy shallow; components are the prune point.

Chapter lab(s)

Lab 31–32 (schemas, kinds) come later. Run this chapter's scripts now.

USDA reading exercise

USDA-07. What is `/Chair`'s typeName vs kind?

```

#usda 1.0
def Xform "Chair" ( kind = "component" ) {}

```

Answer: typeName Xform, kind component. Both can be true at once.

Chapter review

If you see... → think...

If you see...	Think...
IsA vs HasAPI	Typed schema vs API schema
CollectionAPI:foo	Multiple-apply instance foo

If you see...	Think...
Physics fields on Mesh	API schema, not a new Mesh type
subcomponent	Inside a component, not a model
Kind vs type	Metadata vs typeName

CHAPTER 7

Chapter 7 — USD File Formats and Reading USDA

Exam domain: Data Exchange (15%) and Pipeline (14%) · **Objectives:** 4.3, 7.3 **Study day:** 2 · **Est. time:** 60 min **Prerequisites:** Chapters 1–6

Learning goals

- Explain **USDA**, **USDC** (crate), **USD** (either), and **USDZ**.
- State trade-offs: performance, readability, archival, diff/merge (Obj 4.3 / 7.3).
- Read a USDA file line by line.
- Convert formats with `Sdf.Layer.Export` (the `usdcat` equivalent in `usd-core`).

Key terms

Term	One-line definition
USDA	Human-readable text format (<code>.usda</code>)
USDC / crate	Binary format (<code>.usdc</code>); magic bytes <code>PXR-USDC</code>
USD	Extension <code>.usd</code> that may be crate <i>or</i> text; default new files are crate
USDZ	Uncompressed zip package of USD + images + audio
Crate	Pixar's name for the USDC binary layout

7.1 USDA (text)

What is it? **USDA** is the text encoding of a layer. Files usually end in `.usda`. The first line is `#usda 1.0`.

Why do we need it? Humans read, diff, and merge text. Asset *interfaces* (tiny files that only declare `defaultPrim`, variants, and a payload) are often USDA.

Beginner explanation. USDA is to USD what ASCII FBX is *trying* to be: a readable document. Unlike OBJ, it can express composition, not just a mesh.

Technical explanation. - Format id: `usda` (`layer.GetFileFormat().formatId`). - Larger on disk for heavy geometry; slow to parse relative to crate. - Excellent for git diffs of small layers. - `ExportToString()` always produces USDA text, even if the layer is crate.

Mental model. USDA = source code of a layer. USDC = compiled object file.

Simple example. `chair.usda` with twenty lines of interface metadata; `chair_geo.usdc` with a million points.

USDA example.

```
#usda 1.0
(
    defaultPrim = "World"
)

def Xform "World"
{
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateNew("ch07.usda")
UsdGeom.Xform.Define(stage, "/World")
stage.GetRootLayer().Save()
print("format:", stage.GetRootLayer().GetFileFormat().formatId)
print("starts with:", open("ch07.usda", "rb").read(9).decode("ascii"))
```

Expected output

```
format: usda
starts with: #usda 1.0
```

Real-world use case. Code review of a shot's root layer: reviewers read USDA sublayer lists in git.

Common mistakes.**MISTAKE**

Checking a 5 GB USDA "so we can see it". Open it in usdcat (or ExportToString on a subset). Prefer crate for caches.

Exam traps.**TRAP**

"USDZ cannot store meshes." It can. You just should not want to at scale.

Exam takeaways.**KEY**

- USDA = text, readable, good for small interface layers.

7.2 USDC (crate binary)

What is it? USDC is the binary **crate** format (.usdc). On disk it starts with magic bytes PXR-USDC.

Why do we need it? Obj 4.3: performance. Opening a stage keeps layers open; crate is smaller in memory and faster than text for "big data" (points, time samples).

Beginner explanation. Crate is the packed binary of the same layer. Same scene description, different encoding.

Where the analogy breaks: crate is **not** encrypted and not zip. USDZ is zip.

Technical explanation. - Format id: usdc. - Pixar's performance guide: prefer binary for geometry and shading caches; keep USDA for small interface files. - mmap-friendly; plays well with multithreading (especially vs Alembic Ogawa's file-descriptor story). - Tiny scenes can be *larger* as crate than as USDA because of headers (verified: a one-prim Xform was 30 bytes USDA vs ~583 bytes USDC). The win appears when arrays grow.

Mental model. Heavy data → crate. Light structure → USDA.

Simple example. Animation caches, packed crowds, hero meshes: .usdc.

USDA example. You do not write crate by hand. You **export** to it. This is the text equivalent of what crate stores:

```
#usda 1.0

def Xform "World"
{
}
```

Python example.

```
from pxr import Usd, UsdGeom, Sdf

stage = Usd.Stage.CreateNew("ch07.usda")
UsdGeom.Xform.Define(stage, "/World")
stage.GetRootLayer().Save()
stage.GetRootLayer().Export("ch07.usdc")
layer = Sdf.Layer.FindOrOpen("ch07.usdc")
print("format:", layer.GetFormat().formatId)
print("magic:", open("ch07.usdc", "rb").read(8))
print("text view:", layer.ExportToString().splitlines()[0])
```

Expected output

```
format: usdc
magic: b'PXR-USDC'
text view: #usda 1.0
```

Real-world use case. A farm converts Alembic caches to crate (usdc in a full build) so USD does not pay an extra translation tax (performance guide).

Common mistakes.**MISTAKE**

Committing huge crate files to git as if they were USDA. Use artifact storage.

Exam traps.**TRAP**

"USDC is a zip of USDA." No. Zip-of-files is USDZ. USDC is a native binary layer format.

Exam takeaways.**KEY**

- USDC = crate = binary layer. Magic `PXR-USDC`. - Prefer it for geometry caches.

7.3 The .usd extension

What is it? `.usd` is an extension that can hold **either** crate or USDA. USD detects the actual format from the contents (FAQ: "What file format is my `.usd` file?").

Why do we need it? Pipelines can say "always use `.usd`" and still switch encodings. Tools should not assume `.usd` is text.

Beginner explanation. `.usd` is a labeled box that might contain text or crate. Look inside (magic bytes or `GetFileFormat()`).

Technical explanation. Verified on USD 26.08: `Usd.Stage.CreateNew("file.usd")` creates **crate** contents (PXR-USDC) with format id `usd` (not `usdc`). The bytes are still crate. Detection: if the file starts with `#usda` it is text; if `PXR-USDC` it is crate.

Mental model. Extension $\neq$ encoding. `.usda` / `.usdc` are honest. `.usd` is "either".

Simple example. A publisher writes `.usd` so internal tools never bikeshed extensions; encoding is crate unless someone exports USDA into that path.

USDA example. If a `.usd` file is text, it still begins:

```
#usda 1.0

def Xform "World" {}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateNew("ch07.usd")
UsdGeom.Xform.Define(stage, "/World")
stage.GetRootLayer().Save()
print("format id:", stage.GetRootLayer().GetFileFormat().formatId)
print("magic:", open("ch07.usd", "rb").read(8))
```

Expected output

```
format id: usd
magic: b'PXR-USDC'
```

Real-world use case. `usdc in file.usd` (full build) prints text regardless of encoding — same as `ExportToString()`.

Common mistakes.

MISTAKE

Opening `.usd`` in a text editor and seeing binary garbage. It is crate. Use `usdcat / ExportToString`.

Exam traps.**TRAP**

"`.usd`` means USDA." Not necessarily. Default new `.usd`` in this runtime is crate.

Exam takeaways.**KEY**

- `.usd`` = either encoding. Detect contents. - Default `CreateNew("*.usd")`` here is crate.

7.4 USDZ (package)

What is it? USDZ is an **uncompressed, unencrypted zip** that packages USD files plus allowed media (png, jpeg, exr, avif, audio). Spec version 1.3 (OpenUSD 26.08 docs).

Why do we need it? Content delivery: one object, no unpacking required, possible streaming. Apple's AR and many viewers consume USDZ.

Beginner explanation. USDZ is a zip folder of a USD scene and its textures, with extra rules (64-byte alignment, no compression).

Where the analogy breaks: you cannot freely zip anything. Alembic inside USDZ is not allowed; files must be mmap-friendly.

Technical explanation. - First USD file in the package is the **default layer** (like `defaultPrim`, but for files). - `usdzip` in a full toolset; Python:

`UsdUtils.CreateNewUsdzPackage(src, dest)` (F5). - Editing: unpack, edit, repack. The package is read-only. - `--arkitAsset` enforces a stricter subset for iOS.

Mental model. USDZ = box. USDA/USDC = papers in the box. Textures = photos in the box.

Simple example. Marketing wants one file to drop into a web viewer: `chair.usdz`.

USDA example. USDZ is binary zip. The **default layer inside** might be:

```
#usda 1.0
(
    defaultPrim = "Chair"
)

def Xform "Chair"
{
}
```

Python example.

```
from pxr import Usd, UsdGeom, UsdUtils, Sdf

stage = Usd.Stage.CreateNew("pack_me.usda")
UsdGeom.Xform.Define(stage, "/World")
stage.SetDefaultPrim(stage.GetPrimAtPath("/World"))
stage.GetRootLayer().Save()
ok = UsdUtils.CreateNewUsdzPackage("pack_me.usda", "pack_me.usdz")
print("packaged:", ok)
print("usdz format id:", Sdf.FileFormat.FindByExtension("usdz").formatId)
```

Expected output

```
packaged: True
usdz format id: usdz
```

Real-world use case. Vendor delivery (Obj 1.9): localize textures, flatten or package, hand over one `.usdz`.

Common mistakes.**MISTAKE**

Compressing the zip. The spec requires ****zero compression**** so USD can mmap contents.

Exam traps.**TRAP**

"USDZ is just renamed USDC." USDC is one layer. USDZ is a package of files.

Exam takeaways.**KEY**

- USDZ = uncompressed zip package; first USD file is the default layer. - Not the same as USDC.

7.5 Reading USDA syntax

What is it? A line-by-line reading of the syntax you have been using since Chapter 1.

Why do we need it? Exam questions paste USDA. You must parse them under time pressure.

Beginner explanation. Treat USDA like a strict language: header, metadata, then nested prims.

Technical explanation. Anatomy:

```
#usda 1.0                                required magic
(                                          optional layer metadata
    defaultPrim = "World"
    subLayers = [@a.usda@, @b.usda@]
)
def Xform "World" (                      specifier type "name" (prim metadata)
    kind = "component"
)
{                                          prim body: properties and children
    double radius = 2                    attribute
    rel material:binding = </M>          relationship
    over "Child" { }                     child spec
}
```

Asset paths use @path@. Prim paths as values use </Path>.

Mental model. Header → metadata in () → tree of specs.

Simple example. See the combined example below.

USD example.

```
#usda 1.0
(
    defaultPrim = "World"
    metersPerUnit = 0.01
    upAxis = "Y"
)

def Xform "World" (
    kind = "component"
)
{
    def Sphere "Ball"
    {
        double radius = 2
    }
}
```

Python example.

```
from pxr import Sdf
```

```

text = '''#usda 1.0
(
    defaultPrim = "World"
)
def Xform "World" {
    def Sphere "Ball" {
        double radius = 2
    }
}
...
layer = Sdf.Layer.CreateAnonymous(".usda")
print("parse:", layer.ImportFromString(text))
print("defaultPrim:", layer.defaultPrim)
print("has /World/Ball:", bool(layer.GetPrimAtPath("/World/Ball")))

```

Expected output

```

parse: True
defaultPrim: World
has /World/Ball: True

```

Real-world use case. Debugging: if parse fails, the error is `Expected }` at `...` pointing at a missing newline or brace — as this book’s labs discovered with inline nested specs.

Common mistakes.**MISTAKE**

Putting two ``def "World"``` in one layer. Duplicate prim specs in one layer error.

Exam traps.**TRAP**

Skipping the header metadata when the question's answer lives in ``subLayers`` order.

Exam takeaways.**KEY**

- ``#usda 1.0`` required. ``@asset@`` vs ``.``. - Layer metadata in the header ``( )``; prim metadata in spec ``( )``.

I need to remove the “Wait” note from the chapter - that’s my internal note. Let me fix when editing.

Also the python example with nested braces on separate lines should work.

7.6 Converting formats

What is it? Changing a layer’s encoding: USDA ↔ USDC, packing USDZ, flattening (Ch 34).

Why do we need it? Obj 4.3: pick the right encoding. Pipelines convert on publish.

Beginner explanation. Same document, different save-as.

Technical explanation. Full toolset: `usdcat in.usda -o out.usdc`, `usdzip`, `usdcat --flatten`.

usd-core equivalents:

- `layer.Export("out.usdc")` — format from extension
- `layer.ExportToString()` — always USDA text
- `UsdUtils.CreateNewUsdzPackage`

`CreateNew("x.usd")` default crate, as above.

Mental model. Export chooses encoding by file extension.

Simple example. Publish: USDA interface stays; geo layer exported to USDC.

USDA example. Conversion does not change this meaning, only packing:

```
#usda 1.0
def Xform "World" {}
```

Python example.

```
from pxr import Usd, UsdGeom, Sdf

stage = Usd.Stage.CreateNew("conv.usda")
UsdGeom.Xform.Define(stage, "/World")
stage.GetRootLayer().Save()
stage.GetRootLayer().Export("conv.usdc")
print("usda id:", Sdf.Layer.FindOrOpen("conv.usda").GetFileFormat().formatId)
print("usdc id:", Sdf.Layer.FindOrOpen("conv.usdc").GetFileFormat().formatId)
```

Expected output

```
usda id: usda
usdc id: usdc
```

Real-world use case. A CI job fails if a published component’s payload is USDA over a size limit — encoding policy as documentation (Obj 7.2).

Common mistakes.

MISTAKE

Renaming `file.usda` to `file.usdc` without exporting. The bytes are still text; tools will fail.

Exam traps.

TRAP

"usdcats always flattens." Only with `--flatten`. Default is "show this layer as text".

Exam takeaways.

KEY

- Convert with Export / usdcats, not by renaming. - Extension selects format on Export.

Chapter lab(s)

Lab 01 leftover + format conversion in Labs 25–28. Run this chapter’s scripts.

USDA reading exercise

USDA-08. Is this file crate or text if its first bytes are `#usda 1.0` but its name is `hero.usd`?

Answer: Text encoding inside a `.usd` wrapper. Name does not force crate.

Chapter review

Obj 4.3 / 7.3 cheat box

Format	Readability	Heavy geometry	Typical use
USDA	Best	Poor	Interfaces, shot roots, reviews
USDC	Via usdcats	Best	Caches, payloads
.usd	Depends	Depends	"Either"; default new = crate here
USDZ	N/A (package)	Payload files inside	Delivery, AR, single-file share

Part II — Data Modeling

CHAPTER 8

Chapter 8 — Usd vs. Sdf: Two Ways to See Data

Exam domain: Data Modeling (13%) · **Objectives:** 5.4, 6.1 **Study day:** 3 · **Est. time:** 80 min **Prerequisites:** Chapters 2–5

Learning goals

- Contrast the **composed view** (Usd) with the **authored view** (Sdf).
- Inspect `PrimSpec`, `AttributeSpec`, and `RelationshipSpec`.
- Use `Sdf.Path` operations when talking to Sdf APIs.
- Author with Sdf for speed.
- Know when `Sdf.ChangeBlock` removes composition-notification overhead (Obj 6.1).

Key terms

Term	One-line definition
Composed view	What <code>Usd.Stage / Usd.Prim</code> return after composition
Authored view	What this <i>one layer</i> actually contains (Sdf)
Spec	An Sdf object for a prim, attribute, or relationship in one layer
Change block	Batches Sdf edits so USD recomposes once (Obj 6.1)

8.1 Composed view vs authored view

What is it? **Usd** answers “what is true on the stage after composition?” **Sdf** answers “what did this layer write?”

Why do we need it? Debugging (Obj 1.8, 6.2) is almost always “the composed value is X; which layer authored it?” You cannot answer that with `attr.Get()` alone.

Beginner explanation. Usd is the finished newspaper. Sdf is one reporter’s notebook.

Where the analogy breaks: you often *write* through Usd, which then fills in an Sdf spec on the edit-target layer.

Technical explanation. | Question | API | |-----|----| | Composed radius? |
`UsdGeom.Sphere(prim).GetRadiusAttr().Get()` | | Did *this* layer author radius? |
`layer.GetAttributeAtPath("/Ball.radius")` or `primSpec.attributes.get("radius")` | |
 Which specs contribute? | `prim.GetPrimStack()`, `attr.GetPropertyStack()` (Ch 22) |

`stage.GetPrimAtPath` may return a valid composed prim even when the **root layer** has only an `over` — the `def` lives in a sublayer or reference.

Mental model.

```
Usd.Prim / Usd.Attribute  -->  composed
Sdf.PrimSpec / AttributeSpec --> one layer
```

Simple example. Session says `radius=99`. Root says 1. `Get()` → 99. Root’s `AttributeSpec.default` → 1.

USDA example. *Root layer (no radius):*

```
#usda 1.0
(
  subLayers = [
    @geo.usda@
  ]
)
```

geo.usda:

```
#usda 1.0

def Sphere "Ball"
{
    double radius = 2
}
```

Usd: radius 2. Sdf on the root: no `/Ball` spec at all.

Python example.

```
from pxr import Usd, Sdf, UsdGeom

weak = Sdf.Layer.CreateAnonymous("geo.usda")
root = Sdf.Layer.CreateAnonymous("root.usda")
weak.ImportFromString(
    '#usda 1.0\n'
    'def Sphere "Ball" {\n'
    '    double radius = 2\n'
    '}\n'
)
root.subLayerPaths = [weak.identifier]
stage = Usd.Stage.Open(root)
prim = stage.GetPrimAtPath("/Ball")
print("composed radius:", prim.GetAttribute("radius").Get())
print("root has /Ball spec:", bool(root.GetPrimAtPath("/Ball")))
print("weak has /Ball spec:", bool(weak.GetPrimAtPath("/Ball")))
print("weak default:", weak.GetAttributeAtPath("/Ball.radius").default)
```

Expected output

```
composed radius: 2.0
root has /Ball spec: False
weak has /Ball spec: True
weak default: 2.0
```

Real-world use case. A lighting TD sees a red tint on a hero. Composed `displayColor` is red. Sdf on the lighting layer has no such opinion — the color comes from a referenced look file.

Common mistakes.

MISTAKE

Calling `rootLayer.ExportToString()` and concluding a prim "does not exist" because the root is empty. Open the composed stage.

Exam traps.

TRAP

"Usd and Sdf are two names for the same API." Different questions, different objects.

Exam takeaways.

KEY

- Usd = composed. Sdf = this layer. - Empty root $\neq$ empty stage.

8.2 Specs: `PrimSpec`, `AttributeSpec`, `RelationshipSpec`

What is it? A **spec** is the Sdf object for one prim or property **in one layer**.

Why do we need it? Authoring at Sdf speed, and debugging "is this opinion even in this file?"

Beginner explanation. If the layer is a document, a spec is one paragraph: one prim or one property as written there.

Technical explanation. - `Sdf.PrimSpec`, `Sdf.AttributeSpec`, `Sdf.RelationshipSpec`. - `layer.GetPrimAtPath("/World/Ball")` → `PrimSpec` or `None`. - `primSpec.attributes["radius"].default` - `prim.GetPrimStack()` → list of `PrimSpecs`, strongest first. - Specifiers, `typeName`, `nameChildren` live on `PrimSpec`.

Mental model. Stage `prim` → stack of `PrimSpecs` (one per contributing layer).

Simple example. `GetPrimStack()` on `/Ball` with a session override: two specs (session over, then def).

USDA example. The spec for `radius` is this line inside this file only:

```
#usda 1.0

def Sphere "Ball"
{
    double radius = 2
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
UsdGeom.Sphere.Define(stage, "/Ball").GetRadiusAttr().Set(2.0)
prim = stage.GetPrimAtPath("/Ball")
specs = prim.GetPrimStack()
print("stack count:", len(specs), "type:", type(specs[0]).__name__)
spec = specs[0]
print("typeName:", spec.typeName, "specifier:", spec.specifier)
print("radius default:", spec.attributes["radius"].default)
```

Expected output

```
stack count: 1 type: PrimSpec
typeName: Sphere specifier: Sdf.SpecifierDef
radius default: 2.0
```

Real-world use case. A “strip unpublished opinions” tool walks specs on the working layer and clears properties not on the allow-list.

Common mistakes.

MISTAKE

Holding a `PrimSpec` after the layer is reloaded — specs are layer objects; reload invalidates them.

Exam traps.

TRAP

``GetPrimAtPath`` on a **layer** vs a **stage**. Layer: spec or `None`. Stage: `Usd.Prim` (maybe invalid).

Exam takeaways.

KEY

- Specs are per-layer. ``GetPrimStack`` lists them. - Layer `GetPrimAtPath` ≠ Stage `GetPrimAtPath`.

8.3 Sdf.Path operations

What is it? `Sdf.Path` is the shared address type for Usd and Sdf. Sdf APIs often *require* it.

Why do we need it? String paths work in many Usd APIs. Sdf is pickier. Property paths must use `.`.

Beginner explanation. Same addresses as Chapter 4, now used to index a layer like a dictionary.

Technical explanation. - `Sdf.Path("/Ball.radius")` for `GetAttributeAtPath`. - `path.IsPrimPath()`, `IsPropertyPath()`, `AppendProperty`, `GetParentPath`. - Relative paths exist (`../Other`) — more important with references (Ch 16).

Mental model. Path in, spec out.

Simple example. `layer.GetAttributeAtPath("/Ball.radius")` — note the dot.

USDA example.

```
#usda 1.0

def Sphere "Ball"
{
    double radius = 2
}
```

The attribute path is `/Ball.radius`.

Python example.

```
from pxr import Sdf

p = Sdf.Path("/World/Ball")
a = p.AppendProperty("radius")
print(a.IsPropertyPath(), a.GetPrimPath(), a.name)
print("join child:", Sdf.Path("/World").AppendChild("Ball"))
```

Expected output

```
True /World/Ball radius
join child: /World/Ball
```

Real-world use case. A binding validator collects relationship targets as `Sdf.Paths` and checks each `IsPrimPath()`.

Common mistakes.

MISTAKE

``GetAttributeAtPath("/Ball/radius")`` — slash is a child prim, not an attribute.

Exam traps.

TRAP

Mixing ``Sdf.Path("/World")`` with the string `"World"` for ``defaultPrim``. `defaultPrim` is a `**name**`.

Exam takeaways.

KEY

- Property paths use a dot. Sdf lookups use ``Sdf.Path``.

8.4 Authoring with Sdf for speed

What is it? Creating `PrimSpecs` / `AttributeSpecs` directly on a layer instead of `UsdGeom.Xform.Define` in a loop.

Why do we need it? Massive procedural scenes: USD must recompose on each Usd edit unless you batch. Sdf authoring plus a change block is the usual fast path (Obj 6.1).

Beginner explanation. `Usd.Define` is “type this in the UI”. Sdf specs are “write the database rows, then refresh once”.

Technical explanation. - `Sdf.PrimSpec(layer, name, specifier, typeName)` for a **root** prim. - Nested: `Sdf.PrimSpec(parentSpec, name, specifier, typeName)`. - Then `Sdf.AttributeSpec(primSpec, "radius", Sdf.ValueTypeNames.Double)` and set `.default`. - After Sdf edits, the stage sees them; with `ChangeBlock`, notification is deferred.

Verified: 200 Xforms via `UsdGeom.Xform.Define` vs `Sdf.PrimSpec` + `ChangeBlock` — Sdf path was several times faster in this environment (micro-benchmark, not a promise of a specific ratio).

Mental model. Loop in Sdf → one composition update.

Simple example. A point-cloud importer creates 50,000 parent Xforms: use Sdf.

USDA example. The result still looks like ordinary USDA:

```
#usda 1.0

def Xform "Y0" {}
def Xform "Y1" {}
```

Python example.

```
from pxr import Usd, Sdf

stage = Usd.Stage.CreateInMemory()
layer = stage.GetRootLayer()
with Sdf.ChangeBlock():
    for i in range(3):
        Sdf.PrimSpec(layer, f"Y{i}", Sdf.SpecifierDef, "Xform")
print([str(p.GetPath()) for p in stage.Traverse()])
```

Expected output

```
['/Y0', '/Y1', '/Y2']
```

Real-world use case. DCC exporters that previously took minutes drop to seconds after moving the inner loop to Sdf + `ChangeBlock` (Obj 6.1).

Common mistakes.

MISTAKE

Mixing `Usd.Define` and Sdf edits on the same prims without understanding edit targets — you can double-author or miss the layer you meant.

Exam traps.

TRAP

"Sdf authoring skips composition rules." It authors into a layer; composition still applies when you query `Usd`.

Exam takeaways.

KEY

- Heavy procedural authoring: Sdf specs + `ChangeBlock`. - Query still through `Usd` when you need composed values.

8.5 Sdf.ChangeBlock (intro)

What is it? `Sdf.ChangeBlock` is a context manager (and C++ RAII object) that **batches** scene-description changes so listeners (the stage) update once.

Why do we need it? Obj 6.1: identify when `ChangeBlocks` alleviate bottlenecks. Answer: **many small Sdf/Usd edits** that would otherwise each cause indexing work.

Beginner explanation. Instead of ringing the doorbell for every grocery bag, carry all the bags in, then ring once.

Where the analogy breaks: nested `ChangeBlocks` are reference-counted; the flush happens when the outermost block ends.

Technical explanation.

```
with Sdf.ChangeBlock():
    # many spec edits
    pass
```

- Use around Sdf authoring loops and some Usd loops.
- Do not do slow I/O or user interaction inside the block while the stage is “stale”.
- Notices (`Usd.Notice`, Ch 34) fire after the block.

Mental model. `ChangeBlock` = pause notifications; resume and catch up at the end.

Simple example. See §8.4’s loop.

USDA example. `ChangeBlock` does not appear in USDA. It is runtime only.

```
#usda 1.0
def Xform "Hint" {}
```

Python example.

```
from pxr import Usd, Sdf

stage = Usd.Stage.CreateInMemory()
layer = stage.GetRootLayer()
with Sdf.ChangeBlock():
    Sdf.PrimSpec(layer, "A", Sdf.SpecifierDef, "Xform")
    Sdf.PrimSpec(layer, "B", Sdf.SpecifierDef, "Xform")
print("after block:", [str(p.GetName()) for p in stage.Traverse()])
```

Expected output

```
after block: ['A', 'B']
```

Real-world use case. A variant switcher that writes hundreds of overs: wrap in `ChangeBlock` or the UI freezes (Ch 44).

Common mistakes.

MISTAKE

Querying the stage in the middle of a huge block and assuming it already shows every spec. Wait until the block exits for a consistent view.

Exam traps.

TRAP

"ChangeBlock makes disk writes faster." It batches ****in-memory composition notifications****, not fsync.

Exam takeaways.

KEY

- `ChangeBlock` = batch edits, one update (Obj 6.1). - It does not change composition strength rules.

Chapter lab(s)

Lab 09 (Sdf specs & `ChangeBlock`) matches this chapter.

USDA reading exercise

USDA-09. Root has no prims, only `subLayers = [geo.usda@]. geo.usda defs /Ball`. Does `root.GetPrimAtPath("/Ball")` succeed? Does `stage.GetPrimAtPath("/Ball").IsValid()`?

Answer: Root Sdf lookup fails (None/false). Stage prim is valid.

Chapter review

If you see... → think...

If you see...	Think...
Get() vs layer text disagree	Composed vs authored
Slow Define loop	ChangeBlock + Sdf
GetPrimAtPath	Which object — stage or layer?

CHAPTER 9

Chapter 9 — Value Types

Exam domain: Data Modeling (13%) · **Objectives:** 5.2 **Study day:** 3 · **Est. time:** 90 min **Prerequisites:** Chapter 5

Learning goals

- Choose `Sdf.ValueTypeNames` for attribute data (Obj 5.2).
- Distinguish tokens vs strings, roles (`point3f` vs `float3` vs `color3f`), arrays vs scalars.
- Use `Gf` and `Vt` types in Python.
- Author asset paths as `Sdf.AssetPath`.

Key terms

Term	One-line definition
Value type name	The USDA type tag (<code>double</code> , <code>color3f</code> , <code>point3f[]</code>)
Role	Extra meaning on a vector (<code>Point</code> , <code>Color</code> , <code>Normal</code> , <code>TexCoord</code>)
Token	An interned identifier string (<code>TfToken</code> / Python <code>str</code> with type token)
<code>Vt</code> array	USD's typed array (<code>Vt.Vec3fArray</code> , <code>Vt.IntArray</code>)
<code>Gf</code>	Graphics math: <code>Gf.Vec3f</code> , <code>Gf.Matrix4d</code> , <code>Gf.Quatf</code>
Asset path	A resolvable path value (<code>Sdf.AssetPath</code> , USDA <code>@./x.png@</code>)

USD 26.08 exposes **111** names on `Sdf.ValueTypeNames`. You do not memorize all of them. You memorize the families and the roles.

9.1 Sdf.ValueTypeNames

What is it? `Sdf.ValueTypeNames` is the catalog of legal attribute types.

Why do we need it? `CreateAttribute` requires a type name. USDA declares it on every attribute.

Beginner explanation. It is the menu of types: pick `Float`, `Point3f`, `Color3fArray`, `Token`, `Asset`, `Matrix4d`, ...

Technical explanation. Python: `Sdf.ValueTypeNames.Color3f`. USDA writes `color3f`. Arrays append `Array` in Python (`Point3fArray`) and `[]` in USDA (`point3f[]`).

`typeName.role` is nonempty for role types (`Point`, `Color`).

Mental model. Python enum-like catalog ↔ USDA type tag.

Simple example. Mesh points: `Sdf.ValueTypeNames.Point3fArray` / `point3f[]`. Display color: `Color3fArray` / `color3f[]`.

USDA example.

```
#usda 1.0

def Mesh "Body"
{
    point3f[] points = [(0, 0, 0)]
    color3f[] primvars:displayColor = [(1, 0, 0)]
    token purpose = "default"
}
```

Python example.

```
from pxr import Sdf
```

```
print("Point3f:", Sdf.ValueTypeNames.Point3f, "role:", Sdf.ValueTypeNames.Point3f.role)
print("Float3:", Sdf.ValueTypeNames.Float3, "role:", repr(Sdf.ValueTypeNames.Float3.role))
print("Color3f:", Sdf.ValueTypeNames.Color3f, "role:", Sdf.ValueTypeNames.Color3f.role)
print("Point3fArray:", Sdf.ValueTypeNames.Point3fArray)
```

Expected output

```
Point3f: point3f role: Point
Float3: float3 role: ''
Color3f: color3f role: Color
Point3fArray: point3f[]
```

Real-world use case. An importer maps glTF `COLOR_0` to `color3f[]` primvars, not `float3[]`, so Hydra treats them as color.

Common mistakes.

MISTAKE

Using `Float3` for mesh points. Points have role `Point` (`point3f`) so transforms treat them as positions.

Exam traps.

TRAP

"float3 and point3f are identical in meaning." Same storage, different **role**.

Exam takeaways.

KEY

- Pick the type name (and role) that matches the meaning, not just the number of floats.

9.2 Scalars

What is it? Scalars are single values: `bool`, `int`, `int64`, `float`, `double`, `half`, `string`, `token`, `uchar`, ...

Why do we need it? Radius is `double` on Sphere. Flags are `bool`. Counts are `int`.

Beginner explanation. One cell, one value.

Technical explanation. - Sphere radius is **double** (not float) — verified. - `half` is 16-bit; rare in Python pipelines; more common in GPU-facing data. - `string` vs `token`: next section. - Bool in USD: `1 / 0` or `true / false` depending on writer; Python uses `True/False`.

Mental model. If it is not an array (`[]`) and not a vector (`3f`), it is a scalar.

Simple example. `double radius = 2 · bool doubleSided = 1`

USD example.

```
#usda 1.0

def Sphere "Ball"
{
    double radius = 2
    bool doubleSided = 1
    custom int count = 4
}
```

Python example.

```
from pxr import Usd, UsdGeom, Sdf

stage = Usd.Stage.CreateInMemory()
ball = UsdGeom.Sphere.Define(stage, "/Ball")
print("radius type:", ball.GetRadiusAttr().GetTypeName())
ball.GetRadiusAttr().Set(2.5)
print("radius:", ball.GetRadiusAttr().Get())
flag = ball.GetPrim().CreateAttribute("count", Sdf.ValueTypeNames.Int)
```

```
flag.Set(4)
print("count:", flag.Get(), flag.GetTypeName())
```

Expected output

```
radius type: double
radius: 2.5
count: 4 int
```

Real-world use case. Do not store a unique identifier that must survive as a human string in `token` if you need spaces and punctuation — use `string` (next section).

Common mistakes.**MISTAKE**

Setting a float Python value on a ``int`` attribute and assuming it truncates silently in every version. Match types.

Exam traps.**TRAP**

"All numeric USD attributes are float." Sphere radius is double; matrices are double; many shader inputs are float.

Exam takeaways.**KEY**

- Match the schema's scalar type. Radius is double.

9.3 Tokens vs strings

What is it? A **token** is an interned identifier (`purpose = "render"`). A **string** is general text (`documentation`, file labels).

Why do we need it? Tokens are compared by identity and used in enums (`purpose`, `visibility`, `interpolation`). Strings hold prose.

Beginner explanation. Token = keyword. String = sentence.

Where the analogy breaks: in Python, `Get()` on a token attribute still returns a Python `str`. The distinction is the **USD type**, not the Python class.

Technical explanation. - USDA: `token purpose = "default"` vs `string documentation = "Hero chair"`. - Tokens should be short, stable vocabulary. - `Vt.TokenArray / token[]` for lists (`xformOpOrder`).

Mental model. Closed vocabulary → token. Open text → string.

Simple example. `uniform token[] xformOpOrder = ["xformOp:translate"]`

USDA example.

```
#usda 1.0

def Xform "World"
{
    uniform token[] xformOpOrder = ["xformOp:translate"]
    custom string note = "layout v3"
}
```

Python example.

```
from pxr import Usd, Sdf

stage = Usd.Stage.CreateInMemory()
prim = stage.DefinePrim("/P")
tok = prim.CreateAttribute("purposeCopy", Sdf.ValueTypeNames.Token)
tok.Set("render")
s = prim.CreateAttribute("note", Sdf.ValueTypeNames.String)
```

```
s.Set("layout v3")
print("token value:", tok.Get(), "pytype:", type(tok.Get()).__name__, "usd:", tok.GetTypeName())
print("string value:", s.Get(), "usd:", s.GetTypeName())
```

Expected output

```
token value: render pytype: str usd: token
string value: layout v3 usd: string
```

Real-world use case. purpose tokens are default, render, proxy, guide (Ch 39). A free-form string would break filters.

Common mistakes.

MISTAKE

Storing asset paths in `string` when they must resolve — use `asset` (section 9.8).

Exam traps.

TRAP

"In Python tokens are a special class, so `==` fails against str." They come back as `str`. The USDA **type** still matters.

Exam takeaways.

KEY

- token = vocabulary. string = text. Python both look like str.

9.4 Vectors and roles

What is it? Vector types (float3, double3, half3, ...) plus **roles**: point3f, normal3f, vector3f, color3f, texCoord2f.

Why do we need it? Obj 5.2 and visualization: points transform as positions, normals as normals, colors as colors. Wrong role → “unexpected visual results” (Obj 5.5).

Beginner explanation. Three floats are not enough information. *What they mean* is the role.

Technical explanation. Verified roles: Point3f.role is Point; Color3f.role is Color; Float3.role is empty.

USDA	Meaning
point3f	Position (affected by translate/scale/rotate as a point)
vector3f	Direction/displacement
normal3f	Surface normal (inverse-transpose)
color3f / color4f	Color (and optional alpha)
texCoord2f	UV
float3	Three floats, no role

Arrays: point3f[] for mesh points.

Mental model. Role rides along with the three numbers.

Simple example. points is point3f[]. normals is normal3f[]. primvars:displayColor is color3f[].

USDA example.

```
#usda 1.0

def Mesh "Body"
{
    point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
```

```

normal3f[] normals = [(0, 1, 0), (0, 1, 0), (0, 1, 0)]
color3f[] primvars:displayColor = [(1, 0, 0)]
texCoord2f[] primvars:st = [(0, 0), (1, 0), (0, 1)]
}

```

Python example.

```

from pxr import Usd, Sdf, Gf

stage = Usd.Stage.CreateInMemory()
prim = stage.DefinePrim("/P")
c = prim.CreateAttribute("c", Sdf.ValueTypeNames.Color3f)
c.Set(Gf.Vec3f(1, 0, 0))
p = prim.CreateAttribute("p", Sdf.ValueTypeNames.Point3f)
p.Set(Gf.Vec3f(0, 10, 0))
print("color", c.Get(), c.GetTypeName())
print("point", p.Get(), p.GetTypeName())
print("roles", Sdf.ValueTypeNames.Color3f.role, Sdf.ValueTypeNames.Point3f.role)

```

Expected output

```

color (1, 0, 0) color3f
point (0, 10, 0) point3f
roles Color Point

```

Real-world use case. A converter that copies normals into `point3f[]` will transform them like positions — lighting breaks (Obj 5.5 / 6.4).

Common mistakes.**MISTAKE**

UVs as ``float2[]`` instead of ``texCoord2f[]`` — some renderers still work; PreviewSurface workflows expect texCoord roles (Ch 40).

Exam traps.**TRAP**

"color3f is just float3." Role ``Color`` is the point of the type.

Exam takeaways.**KEY**

- Roles: Point, Normal, Color, TexCoord. Empty role = plain floatN.

9.5 Matrices and quaternions

What is it? `matrix4d` (4×4 double matrix) and `quatf` / `quaterd` / `quath` store transforms and orientations.

Why do we need it? Xformable prims can use a transform matrix; PointInstancer uses quaternions for instance orientations (Ch 25).

Beginner explanation. A 4×4 matrix is “the whole pose in one blob”. A quaternion is “rotation only, compact”.

Technical explanation. - `Gf.Matrix4d`, `Gf.Quatf`. - USDA: `matrix4d xformOp:transform = ( (1,0,0,0), (0,1,0,0), (0,0,1,0), (0,0,0,1) )` - Prefer `xformOps` (translate/rotate/scale) for editable pipelines (Ch 39); matrices appear in caches and instancers.

Mental model. Matrix = full linear pose. Quat = rotation.

Simple example. Identity matrix = no transform.

USDA example.

```

#usda 1.0

def Xform "World"
{

```



```
    custom quatf spin = (1, 0, 0, 0)
}
```

Python example.

```
from pxr import Gf, Sdf, Usd

stage = Usd.Stage.CreateInMemory()
prim = stage.DefinePrim("/P")
m = prim.CreateAttribute("m", Sdf.ValueTypeNames.Matrix4d)
m.Set(Gf.Matrix4d(1.0))
q = prim.CreateAttribute("q", Sdf.ValueTypeNames.Quatf)
q.Set(Gf.Quatf(1, 0, 0, 0))
print("matrix[0][0]:", m.Get()[0][0])
print("quat:", q.Get())
```

Expected output

```
matrix[0][0]: 1.0
quat: (1, 0, 0, 0)
```

Real-world use case. `PointInstancer` orientations is `quath[]` (half quats) for memory (Ch 25).

Common mistakes.

MISTAKE

Assuming row-vector vs column-vector conventions without checking ``Gf.Matrix4d`` docs. Be consistent inside one pipeline.

Exam traps.

TRAP

"matrix4d is 4 floats." It is 16 doubles.

Exam takeaways.

KEY

- ``matrix4d`` and ``quat*`` are first-class types. Prefer `xformOps` when humans will edit.

9.6 Arrays and Vt

What is it? **Arrays** are homogeneous lists: `int[]`, `point3f[]`. In Python they are **Vt arrays**.

Why do we need it? Meshes, curves, primvars, instancer prototype indices — almost all bulk geometry is arrays.

Beginner explanation. A column of values of one type.

Technical explanation. - `Vt.IntArray`, `Vt.Vec3fArray`, `Vt.TokenArray`, ... - `attr.Set([(0,0,0),(1,0,0)])` often converts to the right Vt array. - Length must match the schema's rules (e.g. `faceVertexIndices` length = sum of `faceVertexCounts`).

Mental model. `[]` in USDA = Vt array in Python.

Simple example. Three corners of a triangle: `point3f[]` points length 3.

USDA example.

```
#usda 1.0

def Mesh "Tri"
{
    int[] faceVertexCounts = [3]
    int[] faceVertexIndices = [0, 1, 2]
    point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
}
```

Python example.

```
from pxr import Vt

pts = Vt.Vec3fArray([(0, 0, 0), (1, 0, 0), (0, 1, 0)])
idx = Vt.IntArray([0, 1, 2])
print(pts)
print(list(idx), len(idx))
```

Expected output

```
[(0, 0, 0), (1, 0, 0), (0, 1, 0)]
[0, 1, 2] 3
```

Real-world use case. NVIDIA's study-guide sample on `faceVertexIndices` length for 500 triangles (1500 indices) is **not copied** here. Original: 10 triangles need 30 indices if all are tris.

Common mistakes.**MISTAKE**

Setting Python lists of lists with the wrong inner length (2 vs 3) for ``Vec3fArray``.

Exam traps.**TRAP**

"faceVertexIndices length equals vertex count." It equals ****corners referenced****, not unique vertices.

Exam takeaways.**KEY**

- Bulk data is ``[]`` / Vt arrays. - Index array length follows topology, not unique vertex count.

9.7 Gf math types

What is it? Gf (Graphics Foundations) provides `Gf.Vec2f/3f/4f`, `Gf.Vec3d`, `Gf.Matrix4d`, `Gf.Quatf`, `Gf.Range3d`, ...

Why do we need it? Python attribute `Set/Get` for vectors and matrices uses Gf (and Vt for arrays of them).

Beginner explanation. Gf is the math kit. Vt is the array kit. Sdf names the type. Usd stores the value.

Technical explanation. `Gf.Vec3f(1, 2, 3)` prints `(1, 2, 3)`. Matrices are 4×4 doubles by default for xforms.

Mental model. Gf = one math object. Vt = many.

Simple example. `attr.Set(Gf.Vec3f(0, 10, 0))` for a point3f.

USDA example.

```
#usda 1.0

def Xform "W"
{
    custom float3 v = (0, 10, 0)
}
```

Python example.

```
from pxr import Gf

v = Gf.Vec3f(1, 2, 3)
print(v, v[1])
print(Gf.Matrix4d(1.0)[3][3])
```

Expected output

```
(1, 2, 3) 2.0
1.0
```

Real-world use case. Extent boxes use `Gf.Vec3f` min/max pairs in `float3[] extent` (Ch 13).

Common mistakes.

MISTAKE

Mixing ``Vec3f`` and ``Vec3d`` in the same attribute. Match the type name's precision (``3f`` vs ``3d``).

Exam traps.

TRAP

"Gf is only for cameras." It is used everywhere vectors exist.

Exam takeaways.

KEY

- Gf = math objects. Pair them with the matching `ValueTypeName`.

9.8 Asset paths

What is it? An **asset** value is a path USD will **resolve** (`Sdf.AssetPath`). USDA writes `@./tex.png@`.

Why do we need it? Textures, referenced layers, audio — anything that lives in another file. Obj 7.7: validate asset paths.

Beginner explanation. Not just a string: a ticket the resolver can turn into a real file (Ch 33).

Technical explanation. - Type: `Sdf.ValueTypeNames.Asset` / `asset`. - Python: `Sdf.AssetPath("./tex.png"); prints @./tex.png@`. - Anchored paths (`./`, `../`) resolve relative to the layer. - Search paths depend on `Ar` configuration.

Mental model. `@...@` = asset. `</...>` = prim path.

Simple example. `asset inputs:file = @./textures/albedo.png@` on a texture shader.

USDA example.

```
#usda 1.0

def Shader "Tex"
{
    asset inputs:file = @./albedo.png@
}
```

Python example.

```
from pxr import Sdf, Usd

stage = Usd.Stage.CreateInMemory()
prim = stage.DefinePrim("/Tex")
attr = prim.CreateAttribute("file", Sdf.ValueTypeNames.Asset)
attr.Set(Sdf.AssetPath("./albedo.png"))
print(attr.Get(), type(attr.Get()).__name__, attr.GetTypeName())
```

Expected output

```
@./albedo.png@ AssetPath asset
```

Real-world use case. A validator lists every `asset` attribute and checks `Ar.GetResolver().Resolve`.

Common mistakes.

MISTAKE

Windows backslashes in asset paths. Use forward slashes.

Exam traps.**TRAP**

Confusing ``@file@`` (asset) with `` `` (target path).

Exam takeaways.**KEY**

- Asset paths: type ``asset``, Python ``Sdf.AssetPath``, USDA ``@path@``.

9.9 Choosing the right type (Obj 5.2)

What is it? A decision table for exam questions that ask “which type should you use?”

Why do we need it? This is objective 5.2.

Beginner explanation. Ask: is it text, a keyword, a filename, a position, a color, a list, a pose?

Technical explanation. | If you mean... | Use | |-----|-----| | Keyword / enum | token or token[] | | Prose | string | | File to resolve | asset | | Position | point3f / point3f[] | | Normal | normal3f[] | | Color | color3f / color3f[] | | UV | texCoord2f[] | | Generic 3 floats | float3 (no role) | | Whole pose | matrix4d | | Bulk topology | int[] | | Sphere radius | double (schema) |

Mental model. Meaning first, then precision (f vs d), then array or not.

Simple example. Display color on a mesh: `color3f[] primvars:displayColor`, not `float[]`.

USDA example.

```
#usda 1.0

def Mesh "Body"
{
    point3f[] points = [(0, 0, 0)]
    color3f[] primvars:displayColor = [(0.8, 0.1, 0.1)]
    custom asset sourceFile = @./body.obj@
}
```

Python example.

```
from pxr import Sdf

choices = {
    "positions": str(Sdf.ValueTypeNames.Point3fArray),
    "color": str(Sdf.ValueTypeNames.Color3fArray),
    "filename": str(Sdf.ValueTypeNames.Asset),
    "purpose": str(Sdf.ValueTypeNames.Token),
}
print(choices)
```

Expected output

```
{'positions': 'point3f[]', 'color': 'color3f[]', 'filename': 'asset', 'purpose': 'token'}
```

Real-world use case. Data mapping documents (Obj 4.2) include a table exactly like this between DCC types and USD types.

Common mistakes.**MISTAKE**

One ``float[]`` dumping every channel. You lose roles, interpolation, and tooling.

Exam traps.

TRAP
Questions that offer both ``float3[]`` and ``point3f[]`` — pick the **role**.

Exam takeaways.

KEY
- Obj 5.2 = pick type **and role**. - asset vs token vs string vs point vs color is the usual four-way trap.

Chapter lab(s)

Lab 06 (attributes and value types) maps here.

USDA reading exercise

USDA-10. What is wrong with `float3[] points` on a Mesh if you care about transforms?

Answer: Missing Point role. Use `point3f[]`.

Chapter review

If you see... → think...

If you see...	Think...
<code>float3</code> vs <code>point3f</code>	Role
<code>token</code> vs <code>string</code> vs <code>asset</code>	Keyword vs prose vs file
<code>faceVertexIndices</code> length	Corners, not unique verts
<code>@path@</code>	Asset

CHAPTER 10

Chapter 10 — Time Samples and Animation Data

Exam domain: Data Modeling (13%) and Composition · **Objectives:** 1.4, 5.2 **Study day:** 3 · **Est. time:** 90 min **Prerequisites:** Chapters 2, 5, 9

Learning goals

- Distinguish a **default value** from **time samples**.
- Query with `Usd.TimeCode` and `Usd.TimeCode.Default()`.
- Read and set stage time metadata (`timeCodesPerSecond`, frame range).
- Predict **held** vs **linear** interpolation.
- Explain **layer offsets** well enough for Obj 1.4 (same clip, different times).
- Know what **value clips** and **splines** are, without treating them as the exam's main animation encoding.

Key terms

Term	One-line definition
Default value	The attribute value at <code>TimeCode.Default()</code> — not a frame
Time sample	A value authored at a numeric time code
TimeCode	USD's time type (<code>Usd.TimeCode</code>)
Interpolation	How values between samples are computed: linear (default) or held
Layer offset	<code>offset</code> and <code>scale</code> that remap a layer's time into the referencing/sublayering stage
Value clips	Metadata that streams animation from a sequence of files
Spline	Newer curve encoding on attributes (USD 24.03+; not the exam's core)

10.1 Default value vs. time samples

What is it? An attribute can hold a **default** (one value, “unvarying in time”) and/or **time samples** (a dictionary of time → value).

Why do we need it? Static assets use defaults. Animation uses samples. Mixing them wrongly is a classic “why doesn't my pose move?” bug.

Beginner explanation. Default = the still photograph. Time samples = the flip-book. If you ask “what does this look like as a still?” you get the photo. If you ask “what about frame 10?” you get the flip-book.

Where the analogy breaks: numeric time queries **do not use the default**. The default is only for `Default()` time. Frames before the first sample use the **first sample**, not the default. Verified on USD 26.08.

Technical explanation. - USDA default: `double radius = 2` - USDA samples: `double radius.timeSamples = { 1: 1, 11: 5, }` - `attr.Set(2.0)` authors the default. `attr.Set(5.0, 10)` authors a sample at time 10. - `attr.Get()` with no time = `Get(Usd.TimeCode.Default())`. - If there is **no authored default**, `Get()` returns the schema **fallback** (Sphere radius → 1.0) or `None` (no fallback, e.g. a translate op). - `GetNumTimeSamples()`, `GetTimeSamples()`.

Mental model.

<code>Get(Default)</code>	--> authored default, else schema fallback, else <code>None</code>
<code>Get(frame)</code>	--> time samples only (ignore default)

Simple example. Radius default 1, sample at 10 of 5: `Get()` is 1; `Get(10)` is 5; `Get(1)` is 5 (first sample, because 1 is before 10).

USDA example.

```
#usda 1.0

def Sphere "Ball"
{
    double radius = 1
    double radius.timeSamples = {
        10: 5,
        20: 9,
    }
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
ball = UsdGeom.Sphere.Define(stage, "/Ball")
radius = ball.GetRadiusAttr()
radius.Set(1.0)
radius.Set(5.0, 10)
radius.Set(9.0, 20)
print("samples:", radius.GetTimeSamples())
print("Get() / Default:", radius.Get(), radius.Get(Usd.TimeCode.Default()))
print("Get(0):", radius.Get(0))
print("Get(10):", radius.Get(10))
print("Get(15):", radius.Get(15))
print("Get(20):", radius.Get(20))
print("Get(100):", radius.Get(100))
```

Expected output

```
samples: [10.0, 20.0]
Get() / Default: 1.0 1.0
Get(0): 5.0
Get(10): 5.0
Get(15): 7.0
Get(20): 9.0
Get(100): 9.0
```

`Get(15)` is 7 because the stage default interpolation is **linear**.

Real-world use case. A character's rest pose is the default. Shot animation is samples. Tools that forget to pass a time code show the rest pose even though curves exist.

Common mistakes.

MISTAKE

Calling ``Get()`` in a playback loop and wondering why the value never changes. Pass the frame: ``Get(timecode)``.

MISTAKE

Expecting the default to fill times before the first sample. It does not.

Exam traps.

TRAP

"Default is frame 0." Default is a **special time**, not zero. ``Usd.TimeCode.Default()`` prints ``DEFAULT``.

Exam takeaways.

KEY

- Default $\neq$ frame 0. - ``Get()`` without a time is Default time. - Playback must pass a numeric TimeCode.

10.2 Usd.TimeCode and Default()

What is it? `Usd.TimeCode` is USD's time value. Two special values: **Default** and **EarliestTime**.

Why do we need it? Every animated `Get/Set/Compute` takes a `TimeCode`. Mixing "frame 1" with `Default` is how extents and bounds go stale (Obj 5.6, Ch 13).

Beginner explanation. A `TimeCode` is a bookmark in the shot's timeline — unless it is `Default`, which means "the still / non-animated value."

Technical explanation. - `Usd.TimeCode(10)` — numeric. - `Usd.TimeCode.Default()` — the default-value time. `IsDefault()` is `True`. - `Usd.TimeCode.EarliestTime()` — lower than any numeric time; used when resolving "the first opinion in time." - Many APIs accept a raw float/int as time; it is converted to a `TimeCode`. - `TimeCode` is **not** a Python float (`float(Usd.TimeCode(1))` `TypeError`s). Use `.GetValue()` if you need the number.

Mental model.

```
EARLIEST ... 1 2 3 ... N    and separately: DEFAULT
```

Simple example. `bboxCache = UsdGeom.BBoxCache(Usd.TimeCode(101), ...)` computes bounds at the shot frame, not at `Default`.

USDA example. `TimeCodes` appear as keys in `.timeSamples` blocks (they are just numbers in the file):

```
#usda 1.0

def Sphere "Ball"
{
    double radius.timeSamples = {
        1001: 1,
        1100: 2,
    }
}
```

Python example.

```
from pxr import Usd

d = Usd.TimeCode.Default()
t = Usd.TimeCode(1001)
print("Default:", d, "IsDefault:", d.IsDefault())
print("numeric:", t, "IsDefault:", t.IsDefault(), "GetValue:", t.GetValue())
print("Earliest:", Usd.TimeCode.EarliestTime())
```

Expected output

```
Default: DEFAULT IsDefault: True
numeric: 1001 IsDefault: False GetValue: 1001.0
Earliest: EARLIEST
```

Real-world use case. Shot start 1001, end 1100. All animation APIs in that show pass 1001-1100, never `Default`, during playback.

Common mistakes.

MISTAKE

``float(timeCode)`` — use ``timeCode.GetValue()``.

Exam traps.

TRAP

"EarliestTime is frame 0." It is a sentinel, weaker/earlier than any number.

Exam takeaways.

KEY

- Default is a sentinel. Numeric TimeCodes are frames (in whatever unit `timeCodesPerSecond` defines).

10.3 Stage time metadata

What is it? The root layer stores `timeCodesPerSecond`, `framesPerSecond`, `startTimeCode`, `endTimeCode`.

Why do we need it? So every tool agrees how to map TimeCodes to seconds and which range the shot covers.

Beginner explanation. The slate on the film can: 24 fps, frames 1001-1100.

Technical explanation. Verified defaults on a new stage in USD 26.08: **tps = 24, fps = 24, start = 0, end = 0.**

- `timeCodesPerSecond` converts TimeCodes to seconds: `seconds = timeCode / tps`.
- `framesPerSecond` is a hint for UIs; it can differ from tps in some pipelines (not common).
- These fields do **not** by themselves play animation. They label the stage.

Mental model. Header metadata = intended playback; samples = actual motion.

Simple example. A 48 fps capture might set `tps=48`. Layer offsets (next) still work in TimeCodes, not seconds.

USDA example.

```
#usda 1.0
(
    startTimeCode = 1001
    endTimeCode = 1100
    timeCodesPerSecond = 24
    framesPerSecond = 24
)

def Xform "World"
{
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
print("defaults tps/fps/start/end:",
      stage.GetTimeCodesPerSecond(),
      stage.GetFramesPerSecond(),
      stage.GetStartTimeCode(),
      stage.GetEndTimeCode())
stage.SetTimeCodesPerSecond(24)
stage.SetFramesPerSecond(24)
stage.SetStartTimeCode(1001)
stage.SetEndTimeCode(1100)
UsdGeom.Xform.Define(stage, "/World")
print("range:", stage.GetStartTimeCode(), stage.GetEndTimeCode())
```

Expected output

```
defaults tps/fps/start/end: 24.0 24.0 0.0 0.0
range: 1001.0 1100.0
```

Real-world use case. A conform tool refuses to publish a shot whose last animation sample is past `endTimeCode` — pipeline policy, not a USD error.

Common mistakes.

MISTAKE

Setting only fps and assuming samples are resampled. USD does not resample because you changed metadata.

Exam traps.**TRAP**

"startTimeCode defaults to 1001." It defaults to 0 on a new stage.

Exam takeaways.**KEY**

- Defaults: 24 tps / 24 fps / start 0 / end 0. - Metadata labels time; it does not rewrite samples.

10.4 Interpolation: held vs linear

What is it? **Interpolation** fills in values *between* samples. A stage has one interpolation type: **linear** (default) or **held**.

Why do we need it? Stepped animation (visibility, integer IDs, held poses) vs smooth motion. Obj 5.2-adjacent: the *type* of the attribute also matters (you do not linearly interpolate tokens).

Beginner explanation. Linear = draw a straight line between keys. Held = keep the previous key until the next one.

Where the analogy breaks: after the last sample, both modes keep the last value. Before the first sample, both use the first sample.

Technical explanation. - `stage.GetInterpolationType()` / `SetInterpolationType`. - Tokens: `Usd.InterpolationTypeLinear`, `Usd.InterpolationTypeHeld`. - Verified: samples at 1→(0,0,0) and 11→(10,0,0). Time 6 linear → (5,0,0). Time 6 held → (0,0,0). Time 20 (after last) → (10,0,0). - This is **stage-level**, not per-attribute, in the core API.

Mental model.

```
1: 0 ----- 11: 10 ===== 20

linear t=6: 5
held   t=6: 0
both   t=20: 10
```

Simple example. Visibility should be held. Translation of a camera is usually linear (or a spline, §10.7).

USDA example. Interpolation is **not** stored in the layer. It is a runtime stage setting. The samples themselves are:

```
#usda 1.0

def Xform "Ball"
{
    double3 xformOp:translate.timeSamples = {
        1: (0, 0, 0),
        11: (10, 0, 0),
    }
    uniform token[] xformOpOrder = ["xformOp:translate"]
}
```

Python example.

```
from pxr import Usd, UsdGeom, Gf

stage = Usd.Stage.CreateInMemory()
xf = UsdGeom.Xform.Define(stage, "/Ball")
attr = xf.AddTranslateOp().GetAttr()
```

```
attr.Set(Gf.Vec3d(0, 0, 0), 1)
attr.Set(Gf.Vec3d(10, 0, 0), 11)
print("default interp:", stage.GetInterpolationType())
print("linear t=6:", attr.Get(6))
stage.SetInterpolationType(Usd.InterpolationTypeHeld)
print("held t=6:", attr.Get(6))
print("t=20:", attr.Get(20))
```

Expected output

```
default interp: Usd.InterpolationTypeLinear
linear t=6: (5, 0, 0)
held t=6: (0, 0, 0)
t=20: (10, 0, 0)
```

Real-world use case. A layout pop-through (object teleports) is authored as held keys. If a tool forces linear, you see a slide nobody wanted.

Common mistakes.

MISTAKE

Authoring two samples on a `token` attribute and expecting a "blend." Tokens do not blend; you get held-like behavior.

Exam traps.

TRAP

"Held uses the **next** key." It uses the ***previous*** (the sample at or before the time).

Exam takeaways.

KEY

- Stage default interpolation is linear. - Held = previous sample. After last / before first = edge samples.

10.5 Layer offsets (Obj 1.4)

What is it? A **layer offset** remaps time when a layer is brought in by a **reference** or **sublayer**: `composedTime = sourceTime * scale + offset`.

Why do we need it? Obj 1.4: several characters reuse **the same animation layer** starting at different shot times. You do not copy the clip.

Beginner explanation. The dance is stored from 1–11. Hero A starts that dance at shot 1 (offset = 0). Hero B starts it at shot 11 (offset = 10). Same file, two clocks.

Where the analogy breaks: offset is in **TimeCodes**, not seconds. `scale` can speed up or reverse (negative scale).

Technical explanation. - Python: `Sdf.LayerOffset(offset=10, scale=1)` (scale 1 is the default; USDA may omit it). - `prim.GetReferences().AddReference(id, primPath=..., layerOffset=off)` - USDA: `prepend references = @anim.usda@ (offset = 10)` - Verified: source samples at 1 and 11 with offset 10 compose as samples at **11 and 21**. At composed time 11 you see the source's time-1 value.

Works on sublayers too (subLayers entries can carry offsets — Ch 15).

Mental model.

```
source:      1 ----- 11
offset 10:           11 ----- 21
```

Simple example. Two walkers, one clip, offsets 0 and 48.

USDA example. File: *anim.usda*

```
#usda 1.0
(
```

```

    defaultPrim = "Ball"
)

def Xform "Ball"
{
    double3 xformOp:translate.timeSamples = {
        1: (0, 0, 0),
        11: (10, 0, 0),
    }
    uniform token[] xformOpOrder = ["xformOp:translate"]
}

```

File: shot.usda

```

#usda 1.0

def Xform "HeroA" (
    prepend references = @./anim.usda@
)
{
}

def Xform "HeroB" (
    prepend references = @./anim.usda@ (
        offset = 10
    )
)
{
}

```

HeroB's translation samples sit 10 TimeCodes later than HeroA's.

Python example.

```

from pxr import Usd, Sdf

anim = Sdf.Layer.CreateAnonymous("anim.usda")
anim.ImportFromString(
    '#usda 1.0\n'
    '(\n'
    '    defaultPrim = "Ball"\n'
    ')\n'
    'def Xform "Ball" {\n'
    '    double3 xformOp:translate.timeSamples = {\n'
    '        1: (0, 0, 0),\n'
    '        11: (10, 0, 0),\n'
    '    }\n'
    '    uniform token[] xformOpOrder = ["xformOp:translate"]\n'
    '}\n'
)

root = Sdf.Layer.CreateAnonymous("shot.usda")
root.ImportFromString('#usda 1.0\ndef Xform "HeroB" {}\n')
stage = Usd.Stage.Open(root)
hero = stage.GetPrimAtPath("/HeroB")
hero.GetReferences().AddReference(
    anim.identifier,
    primPath=Sdf.Path("/Ball"),
    layerOffset=Sdf.LayerOffset(offset=10),
)

attr = hero.GetAttribute("xformOp:translate")
print("composed samples:", attr.GetTimeSamples())
print("t=11:", attr.Get(11))
print("t=21:", attr.Get(21))

```

Expected output

```

composed samples: [11.0, 21.0]
t=11: (0, 0, 0)
t=21: (10, 0, 0)

```

Real-world use case. Crowd shots instance one walk cycle with many offsets so the crowd does not march in lockstep (Ch 24-25).

Common mistakes.

MISTAKE

Offsetting in seconds (`offset = 10` meaning 10 seconds at 24 fps). USD's offset is in TimeCodes: 10 frames if tps=24, not 10 seconds.

Exam traps.**TRAP**

"Layer offset changes the file on disk." It remaps time **on that arc** only. `anim.usda` still has samples at 1 and 11.

Exam takeaways.**KEY**

- `composed = source \* scale + offset`. - Obj 1.4 = reuse one animation at different offsets; do not copy the clip.

10.6 Value clips (overview)

What is it? **Value clips** stream animation from a **sequence of files** (often one file per frame or chunk) instead of packing every sample into one layer.

Why do we need it? Caches that would make a single USDC enormous. The study guide lists value clips in the composition reading list.

Beginner explanation. Instead of one 200,000-sample file, you have `cache.1001.usdc`, `cache.1002.usdc`, ... and metadata that says which file is active at which time.

Where the analogy breaks: clips are **not** payloads. Payloads load topology. Clips stitch **values** over time.

Technical explanation. - API: `Usd.ClipsAPI(prim)`. - Metadata on the prim: clip asset paths, clip times, active times, optional template (`clipTemplateAssetPath`, `stride`, `start`, `end`). - `GenerateClipManifest` builds a manifest of which attributes are in the clips. - Clips contribute as a value-resolution source (Ch 21). Strength vs local opinions is a composition topic.

You will not author a full clip set in this chapter's lab. Know the name, the API class, and why pipelines use them.

Mental model. Clips = a playlist of cache files mapped onto the timeline.

Simple example. FX sim dumped per frame as USD; the character prim points at the playlist.

USDA example. Template-style metadata (shape only; paths are illustrative):

```
#usda 1.0

def Xform "Sim" (
  clips = {
    dictionary default = {
      string templateAssetPath = "./caches/sim.###.usdc"
      double templateStartTime = 1001
      double templateEndTime = 1100
      double templateStride = 1
    }
  }
)
{
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
prim = UsdGeom.Xform.Define(stage, "/Sim").GetPrim()
```

```
clips = Usd.ClipsAPI(prim)
print("schema:", type(clips).__name__)
print("has GetClipAssetPaths:", hasattr(clips, "GetClipAssetPaths"))
print("has GetClipTemplateAssetPath:", hasattr(clips, "GetClipTemplateAssetPath"))
```

Expected output

```
schema: ClipsAPI
has GetClipAssetPaths: True
has GetClipTemplateAssetPath: True
```

Real-world use case. A 1500-frame ocean sim: one crate per frame, one set of clip metadata on `/World/Ocean`.

Common mistakes.

MISTAKE

Putting clip files behind a payload and expecting clip metadata on an unloaded prim to still stream. Working-set (Ch 17) and clips interact; keep the clip metadata on a loaded prim.

Exam traps.

TRAP

"Value clips are the same as time samples in one layer." Clips **feed** values; they are a different encoding and file layout.

Exam takeaways.

KEY

- Clips = playlist of cache layers over time (``Usd.ClipsAPI``). - Time samples in one layer remain the basic encoding you must master.

10.7 Splines (version note)

What is it? **Splines** store a curve (tangents, interpolation segments) on an attribute instead of (or in addition to) dense samples. Python module `Ts`; `Usd.Attribute.GetSpline` exists in USD 26.08.

Why do we need it? The study guide lists the spline proposal. You should recognize the feature and the VERSION story, not design a spline editor for the exam.

Beginner explanation. Samples = a polyline through keys. Splines = Bezier/Hermite-style animation curves.

Technical explanation.

VERSION

Splines (`Ts`) appeared 24.03; attribute value resolution gained spline support later (25.05+); ``GetSpline`/`SetSpline`` APIs 25.08; clips+splines 26.08. This book is verified on ****26.08****, where ``GetSpline`` exists. ****Time samples remain the encoding to master for NCP-OUUSD.****

Mental model. If the question says "timeSamples dictionary," think samples. If it says "spline/tangents/`Ts`," think the newer curve object.

Simple example. A DCC may export Bezier camera motion as a spline in a 26.x pipeline, or bake it to samples for compatibility.

USDA example. You will meet `.timeSamples` far more often. A spline-backed attribute is still an ordinary typed attribute; the spline is extra encoded data (not required to paste a full spline USDA here).

```
#usda 1.0

def Sphere "Ball"
{
    double radius.timeSamples = {
```

```

    1: 1,
    10: 2,
  }
}

```

Python example.

```

from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
attr = UsdGeom.Sphere.Define(stage, "/Ball").GetRadiusAttr()
print("GetSpline exists:", hasattr(attr, "GetSpline"))
print("HasSpline:", attr.HasSpline())

```

Expected output

```

GetSpline exists: True
HasSpline: False

```

Real-world use case. A studio on USD 23.x cannot read spline-authored files from a 26.x vendor. Bake samples for interchange.

Common mistakes.**MISTAKE**

Assuming every USD file with animation has splines. Most caches are samples or clips.

Exam traps.**TRAP**

A question that shows ``timeSamples`` — the answer is samples, not splines.

Exam takeaways.**KEY**

- Splines exist in 26.08 (``GetSpline``). Samples are still the foundation. - Mark spline interchange as version-sensitive.

Chapter lab(s)

Lab 10 (time samples, interpolation, time codes) and Lab 14 (time-offset references) belong here.

USDA reading exercise

USDA-11. Clip samples at 1 and 11. Reference with `offset = 5`. At composed time 6, which source time is sampled? What composed sample times do you expect?

Answer: Source time $6 - 5 = 1$. Composed samples at 6 and 16.

Chapter review**If you see... → think...**

If you see...	Think...
Get() never animates	You queried Default time
Default vs frame 0	Different things
Two heroes, one walk file	Layer offset (Obj 1.4)
Held vs linear at mid-frame	Previous key vs lerp
Huge per-frame caches	Value clips

CHAPTER 11

Chapter 11 — Primvars

Exam domain: Data Modeling (13%) and Visualization (8%) · **Objectives:** 5.1, 8.1
Study day: 4 · **Est. time:** 80 min **Prerequisites:** Chapters 5, 9, 6

Learning goals

- Explain what a **primvar** is and add one to a mesh (Obj 5.1 / 8.1).
- Use `UsdGeom.PrimvarsAPI`.
- Choose **interpolation**: constant, uniform, varying, vertex, faceVarying.
- Author **indexed** primvars and `elementSize`.
- Predict **inheritance** of primvars down namespace.
- Use `primvars:displayColor` and `displayOpacity`.

Key terms

Term	One-line definition
Primvar	A geometric primitive variable: extra data on a gprim, often for shading
Interpolation	How one primvar array maps onto vertices / faces / the whole prim
Indexed primvar	Values plus an index array (like a palette)
<code>elementSize</code>	How many array entries make one logical element
Inheritance	A primvar on an ancestor applies to descendants unless overridden

11.1 What a primvar is

What is it? A **primvar** is data attached to a geometric prim, stored as an attribute in the `primvars: namespace` (and optional sibling `indices` attributes).

Why do we need it? Meshes need more than points: UVs, colors, IDs, custom shading data. Obj 5.1 / 8.1: add a primvar to a mesh. Obj 8.4: a shader reads a primvar for diffuse color (Ch 40).

Beginner explanation. Points say where the corners are. Primvars say what else each corner (or face, or the whole object) carries: color, UV, `st`, `id`.

Where the analogy breaks: a primvar can live on an **ancestor Xform** and still affect the child mesh (inheritance).

Technical explanation. - Attribute name: `primvars:<name>`. - API class: `UsdGeom.Primvar` wrapping the attribute; factory: `UsdGeom.PrimvarsAPI`. - `displayColor` / `displayOpacity` are built-in primvars on Gprims. - Roles matter: `color3f[]`, `texCoord2f[]` (Ch 9).

Mental model. `primvars:foo` = extra channel named `foo` on this geometry (or inherited).

Simple example. A whole mesh is red: one `color3f` with interpolation `constant`.

USDA example.

```
#usda 1.0

def Mesh "Body"
{
    color3f[] primvars:displayColor = [(1, 0, 0)] (
        interpolation = "constant"
    )
}
```

Python example.

```
from pxr import Usd, UsdGeom, Sdf, Gf
```



```

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/Body")
api = UsdGeom.PrimvarsAPI(mesh)
pv = api.CreatePrimvar(
    "displayColor",
    Sdf.ValueTypeNames.Color3fArray,
    UsdGeom.Tokens.constant,
)
pv.Set([Gf.Vec3f(1, 0, 0)])
print("attr:", pv.GetAttr().GetName())
print("interpolation:", pv.GetInterpolation())
print("HasPrimvar:", api.HasPrimvar("displayColor"))

```

Expected output

```

attr: primvars:displayColor
interpolation: constant
HasPrimvar: True

```

Real-world use case. Lookdev paints `primvars:st` UVs in one layer; lighting never touches `points`.

Common mistakes.

MISTAKE

Creating ``displayColor`` without the ``primvars:`` prefix. Hydra looks for the primvar namespace.

Exam traps.

TRAP

"Primvars are relationships." They are attributes (plus optional index attributes).

Exam takeaways.

KEY

- Primvar = ``primvars:`` attribute (+ optional indices). - Obj 5.1 / 8.1 = add one to a mesh via PrimvarsAPI or USDA.

11.2 UsdGeom.PrimvarsAPI

What is it? **PrimvarsAPI** is the schema API for creating, finding, and inheriting primvars.

Why do we need it? Direct `CreateAttribute("primvars:x", ...)` works, but you will forget interpolation metadata and indices. Use the API.

Beginner explanation. PrimvarsAPI is the toolbox labeled “channels on this mesh.”

Technical explanation. Verified on USD 26.08:

- `CreatePrimvar(name, typeName, interpolation)`
- `GetPrimvar(name)`, `HasPrimvar(name)`, `GetPrimvars()`
- `FindPrimvarWithInheritance(name)`, `FindPrimvarsWithInheritance()` — not `FindPrimvar` (that name does not exist).
- `UsdGeom.Primvar`: `GetInterpolation`, `SetInterpolation`, `GetElementSize`, `SetIndices`, `IsIndexed`, `GetAttr`.

`GetPrimvars()` on a Mesh also lists schema primvars such as `primvars:displayOpacity` even if you never created them.

Mental model. Create / Get = local. Find...WithInheritance = walk ancestors.

Simple example. Lookdev tool: `api.CreatePrimvar("st", TexCoord2fArray, vertex)`.

USDA example.

```
#usda 1.0

def Mesh "Body"
{
    texCoord2f[] primvars:st = [(0, 0), (1, 0), (0, 1)] (
        interpolation = "vertex"
    )
}
```

Python example.

```
from pxr import Usd, UsdGeom, Sdf

stage = Usd.Stage.CreateInMemory()
api = UsdGeom.PrimvarsAPI(UsdGeom.Mesh.Define(stage, "/Body"))
api.CreatePrimvar("st", Sdf.ValueTypeNames.TexCoord2fArray, UsdGeom.Tokens.vertex)
print("Has st:", api.HasPrimvar("st"))
print("names:", [p.GetName() for p in api.GetPrimvars()])
```

Expected output

```
Has st: True
names: ['primvars:displayColor', 'primvars:displayOpacity', 'primvars:st']
```

Real-world use case. A validator requires `st` with interpolation `vertex` or `faceVarying` before shading.

Common mistakes.**MISTAKE**

Calling ``FindPrimvar`` — it is ``GetPrimvar`` locally and ``FindPrimvarWithInheritance`` for ancestors.

Exam traps.**TRAP**

"CreatePrimvar replaces Mesh typeName." No; it is an API on the existing prim.

Exam takeaways.**KEY**

- ``PrimvarsAPI.CreatePrimvar`` / ``GetPrimvar`` / ``FindPrimvarWithInheritance``.

11.3 Interpolation

What is it? **Interpolation** says how the primvar array maps onto the mesh: `constant`, `uniform`, `varying`, `vertex`, `faceVarying`.

Why do we need it? Wrong interpolation = wrong colors / UVs (Obj 5.5 unexpected visuals). PreviewSurface reading a primvar (Obj 8.4) needs the interpolation the shader expects.

Beginner explanation. - **constant** — one value for the whole prim. - **uniform** — one value per face. - **vertex** — one value per point (indexed by `faceVertexIndices`). - **faceVarying** — one value per face-corner (so UVs can seam). - **varying** — like vertex for subdivision / interpolatable data (smooth across the surface).

Where the analogy breaks: `varying` vs `vertex` is subtle on a plain polygonal mesh; `faceVarying` vs `vertex` is the UV-seam question.

Technical explanation. Tokens: `UsdGeom.Tokens.constant` (and `uniform`, `varying`, `vertex`, `faceVarying`).

Lengths (triangle mesh, V vertices, F faces, C corners = sum of `faceVertexCounts`):

Interpolation	Typical array length
constant	1 (or <code>elementSize</code>)

Interpolation	Typical array length
uniform	F
vertex / varying	V
faceVarying	C (e.g. 3F for all tris)

Authored in USDA in the attribute's metadata block: `interpolation = "vertex"`.

Mental model. constant = object. uniform = face. vertex = point. faceVarying = corner.

Simple example. Hard-edged vertex colors that differ on a shared point must be **faceVarying**, not vertex.

USDA example.

```
#usda 1.0

def Mesh "Tri"
{
    int[] faceVertexCounts = [3]
    int[] faceVertexIndices = [0, 1, 2]
    point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
    color3f[] primvars:displayColor = [(1, 0, 0), (0, 1, 0), (0, 0, 1)] (
        interpolation = "vertex"
    )
}
```

Python example.

```
from pxr import UsdGeom

print(
    UsdGeom.Tokens.constant,
    UsdGeom.Tokens.uniform,
    UsdGeom.Tokens.varying,
    UsdGeom.Tokens.vertex,
    UsdGeom.Tokens.faceVarying,
)
```

Expected output

```
constant uniform varying vertex faceVarying
```

Real-world use case. UDIM UVs are faceVarying (or vertex if welded). Face colors for IDs are uniform.

Common mistakes.

MISTAKE

vertex-length array with faceVarying interpolation (or the reverse). Length must match the interpolation.

Exam traps.

TRAP

"constant means one float per vertex." Constant is **one** value for the prim.

Exam takeaways.

KEY

- Five interpolations. Match array length to topology. - Seams → faceVarying. Whole object → constant. Per face → uniform.

11.4 Indexed primvars

What is it? An **indexed primvar** stores a **values** array plus an **indices** array, like a palette.

Why do we need it? A million corners sharing 8 colors: store 8 colors and a million small ints, not a million colors.

Beginner explanation. Paint-by-number: the palette is the values; the numbers on the canvas are the indices.

Technical explanation. - `pv.SetIndices(Vt.IntArray([...])) → IsIndexed() True.` - USDA: `primvars:foo:indices = [0, 1, 0, 1]` beside `primvars:foo.` - Indices follow the interpolation (they replace the “long” array). Values are the unique set ($\times$ `elementSize`).

Mental model. `values[indices[i]]` is the data for element `i`.

Simple example. Values `[(1,0,0), (0,1,0)]`, indices `[0,1,0,1]` — red, green, red, green.

USDA example.

```
#usda 1.0

def Mesh "Body"
{
    color3f[] primvars:displayColor = [(1, 0, 0), (0, 1, 0)] (
        interpolation = "vertex"
    )
    int[] primvars:displayColor:indices = [0, 1, 0, 1]
}
```

Python example.

```
from pxr import Usd, UsdGeom, Sdf, Vt

stage = Usd.Stage.CreateInMemory()
api = UsdGeom.PrimvarsAPI(UsdGeom.Mesh.Define(stage, "/Body"))
pv = api.CreatePrimvar("id", Sdf.ValueTypeNames.IntArray, UsdGeom.Tokens.vertex)
pv.Set([10, 20])
pv.SetIndices(Vt.IntArray([0, 1, 0, 1]))
print("indexed:", pv.IsIndexed())
print("indices:", list(pv.GetIndices()))
print("values:", list(pv.Get()))
```

Expected output

```
indexed: True
indices: [0, 1, 0, 1]
values: [10, 20]
```

Real-world use case. Packed USD from DCC often indexes UVs and colors. Flattening (Ch 34) may expand them.

Common mistakes.

MISTAKE

Index out of range of the values array — some renderers skip the primvar silently (unexpected visuals, Obj 5.5).

Exam traps.

TRAP

"Indexed means interpolation = vertex." Indexing is independent of interpolation.

Exam takeaways.

KEY

- Indexed = palette + indices. Interpolation still applies to the indices.

11.5 elementSize

What is it? `elementSize` is how many array entries make **one** interpolated element. Default 1.

Why do we need it? A primvar of 3 floats per vertex stored as `float[]` with `elementSize = 3` instead of `float3[]`. Some DCC dumps use this.

Beginner explanation. If each vertex wants a 3-tuple but the array is flat floats, `elementSize=3` groups them.

Where the analogy breaks: prefer a vector type (`float3[]`, `color3f[]`) when you can. `elementSize` is the escape hatch.

Technical explanation. - `pv.GetElementSize()` / `SetElementSize(n)`. - USDA metadata: `elementSize = 3`. - Length rules multiply: vertex interpolation needs `V * elementSize` scalars.

Mental model. `logical_length = array_length / elementSize`.

Simple example. `float[] primvars:foo` length 9, `elementSize 3`, vertex on a 3-point mesh.

USDA example.

```
#usda 1.0

def Mesh "Body"
{
    float[] primvars:foo = [1, 0, 0, 0, 1, 0, 0, 0, 1] (
        interpolation = "vertex"
        elementSize = 3
    )
}
```

Python example.

```
from pxr import Usd, UsdGeom, Sdf

stage = Usd.Stage.CreateInMemory()
pv = UsdGeom.PrimvarsAPI(UsdGeom.Mesh.Define(stage, "/Body")).CreatePrimvar(
    "foo", Sdf.ValueTypeNames.FloatArray, UsdGeom.Tokens.vertex
)
print("default elementSize:", pv.GetElementSize())
pv.SetElementSize(3)
print("after:", pv.GetElementSize())
```

Expected output

```
default elementSize: 1
after: 3
```

Real-world use case. An old importer stored normals as flat floats with `elementSize 3`. A modern importer uses `normal3f[]`.

Common mistakes.

MISTAKE

`elementSize 3` on `color3f[]` (already 3-vectors) — you just tripled the meaning. Use 1 with vector types.

Exam traps.

TRAP

"`elementSize` is the number of faces." No.

Exam takeaways.

KEY

- Default elementSize is 1. Prefer vector types over packing with elementSize.

11.6 Primvar inheritance

What is it? A primvar authored on an **ancestor** applies to **descendants** that do not override it.

Why do we need it? Set `displayColor` once on `/Char` and every mesh under it tints, unless a mesh has its own primvar.

Beginner explanation. A team color on the jersey folder, not on every stitch.

Where the analogy breaks: inheritance is **namespace**, not composition-arc inherits (Ch 19). Different word, related idea.

Technical explanation. - `FindPrimvarsWithInheritance()` / `FindPrimvarWithInheritance("displayColor")`. - Verified: primvar on `/X`, child mesh `/X/C` finds `[(0, 1, 0)]`. - Local opinion on the child wins for that name.

Mental model. Walk up parents until you find that primvar name.

Simple example. Assembly tints all referenced components green; one component overrides to blue.

USDA example.

```
#usda 1.0

def Xform "Char"
{
    color3f[] primvars:displayColor = [(0, 1, 0)] (
        interpolation = "constant"
    )
    def Mesh "Body"
    {
    }
}
```

Python example.

```
from pxr import Usd, UsdGeom, Sdf

stage = Usd.Stage.CreateInMemory()
parent = UsdGeom.Xform.Define(stage, "/X")
child = UsdGeom.Mesh.Define(stage, "/X/C")
UsdGeom.PrimvarsAPI(parent).CreatePrimvar(
    "displayColor", Sdf.ValueTypeNames.Color3fArray, UsdGeom.Tokens.constant
).Set([(0, 1, 0)])
found = UsdGeom.PrimvarsAPI(child).FindPrimvarWithInheritance("displayColor")
print("inherited:", found.Get())
print("names:", [p.GetName() for p in UsdGeom.PrimvarsAPI(child).FindPrimvarsWithInheritance()])
```

Expected output

```
inherited: [(0, 1, 0)]
names: ['primvars:displayColor']
```

Real-world use case. A shot overlay sets `primvars:displayColor` on `/World/Hero` to mark the character in dailies without editing the asset.

Common mistakes.

MISTAKE

Expecting inheritance to jump *out* of a reference to the referencing prim's siblings. It walks ***ancestors of this prim*** on the composed stage.

Exam traps.

TRAP

Mixing primvar inheritance with the ****inherits**** composition arc. Different mechanisms.

Exam takeaways.**KEY**

- Primvars inherit down the namespace until overridden. - Use **Find*WithInheritance**, not only **GetPrimvar**.

11.7 displayColor and displayOpacity

What is it? **displayColor** and **displayOpacity** are the standard preview primvars on Gprims. usdview and many GL delegates draw them without a full material.

Why do we need it? Fast visualization and debugging. Obj 8.1 often uses **displayColor** as the primvar you add. Obj 8.4 then connects a *shader* to a primvar (which might still be **displayColor**).

Beginner explanation. The cheap crayon color on the mesh. Materials are the real paint (Ch 40).

Technical explanation. - Names: `primvars:displayColor (color3f[])`, `primvars:displayOpacity (float[])`. - Schema-provided: `GetPrimvars()` lists them on Mesh even before you Set. - Interpolation is yours to author (constant for a solid tint is common).

Mental model. **displayColor** = viewport crayon. **UsdPreviewSurface** = lookdev.

Simple example. Block out a set in gray constant **displayColor**; later bind materials.

USDA example.

```
#usda 1.0

def Mesh "Body"
{
    color3f[] primvars:displayColor = [(0.8, 0.8, 0.8)] (
        interpolation = "constant"
    )
    float[] primvars:displayOpacity = [1] (
        interpolation = "constant"
    )
}
```

Python example.

```
from pxr import Usd, UsdGeom, Sdf, Gf

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/Body")
api = UsdGeom.PrimvarsAPI(mesh)
color = api.CreatePrimvar(
    "displayColor", Sdf.ValueTypeNames.Color3fArray, UsdGeom.Tokens.constant
)
color.Set([Gf.Vec3f(0.8, 0.8, 0.8)])
print(api.GetPrimvar("displayColor").Get())
print("opacity exists:", api.HasPrimvar("displayOpacity"))
```

Expected output

```
[(0.8, 0.8, 0.8)]
opacity exists: True
```

Real-world use case. Instance-proxy color edits (Obj 2.2) often set **displayColor** on the instance proxy without breaking instancing (Ch 24).

Common mistakes.

MISTAKE

Assuming `displayColor` is a `UsdPreviewSurface`. It is a primvar. Binding a material is separate (Ch 40).

Exam traps.**TRAP**

"To color a mesh you must write a new schema." `displayColor` already exists.

Exam takeaways.**KEY**

- `displayColor` / `displayOpacity` = built-in preview primvars. - Adding them to a mesh satisfies Obj 5.1 / 8.1.

Chapter lab(s)

Lab 12 (primvars: interpolation and indexed). Lab 34 uses a primvar-driven `PreviewSurface`.

USDA reading exercise

USDA-12. A mesh has 2 faces, 4 vertices, each face a quad (8 corners). What length is `faceVarying st` without indexing?

Answer: 8.

Chapter review**If you see... → think...**

If you see...	Think...
Add a primvar to a mesh	PrimvarsAPI + interpolation
UV seams	<code>faceVarying</code>
Per-face IDs	uniform
Palette	indexed primvar
Color on the group, not the mesh	inheritance

CHAPTER 12

Chapter 12 — Metadata in Depth

Exam domain: Data Modeling (13%) and Pipeline (14%) · **Objectives:** 5.3, 7.6 **Study day:** 4 · **Est. time:** 60 min **Prerequisites:** Chapter 5

Learning goals

- Distinguish **registered** metadata from **custom** dictionaries.
- Represent pipeline data in `customData` and `assetInfo` (Obj 5.3, 7.6).
- Use `documentation`, `hidden`, and `kind`.
- List the important **layer** metadata fields.
- Choose metadata vs attributes vs custom schemas.

Key terms

Term	One-line definition
Registered metadata	Built-in named fields (<code>kind</code> , <code>hidden</code> , <code>defaultPrim</code> , ...)
<code>customData</code>	Dictionary for your keys; not interpreted by core USD
<code>assetInfo</code>	Dictionary intended for asset identity (identifier, version, ...)
<code>hidden</code>	Metadata hint that UIs should hide this prim
Layer metadata	Header fields on the layer / stage

12.1 Registered vs. custom metadata

What is it? **Registered** metadata has a known name and type in USD. **Custom** extra data usually goes in the `customData` or `assetInfo` dictionaries.

Why do we need it? Obj 5.3: represent custom metadata. If you invent a new top-level metadata field, many tools will ignore it unless you register it (advanced). Dictionaries always work.

Beginner explanation. Registered = printed fields on the government form. `customData` = the blank “notes” box.

Technical explanation. - Read/write: `prim.GetMetadata("kind")`, `SetMetadata("documentation", "...")`. - `HasMetadata`, `HasAuthoredMetadata`. - Unregistered names may fail or be ignored; use `customData` instead. - Specifiers' parentheses in USDA hold prim metadata.

Mental model. Known name → registered API. Anything else → `customData/assetInfo`.

Simple example. `kind` is registered. `pipelineShot` is not — put it in `customData`.

USDA example.

```
#usda 1.0

def Xform "Chair" (
    kind = "component"
    documentation = "Hero chair"
    customData = {
        string shot = "s001"
    }
)
{
}
```

Python example.

```
from pxr import Usd, UsdGeom, Kind
```

```
stage = Usd.Stage.CreateInMemory()
prim = UsdGeom.Xform.Define(stage, "/Chair").GetPrim()
Usd.ModelAPI(prim).SetKind(Kind.Tokens.component)
prim.SetMetadata("documentation", "Hero chair")
print("kind:", prim.GetMetadata("kind"))
print("documentation:", prim.GetMetadata("documentation"))
print("HasAuthored kind:", prim.HasAuthoredMetadata("kind"))
```

Expected output

```
kind: component
documentation: Hero chair
HasAuthored kind: True
```

Real-world use case. A studio schema might later *register* `studio:show`, but until then `customData` keeps exporters simple (Obj 7.6).

Common mistakes.

MISTAKE

Creating an attribute named ``documentation`` instead of metadata. Inspectors looking at metadata miss it.

Exam traps.

TRAP

"All metadata is `customData`." `kind`, `hidden`, `instanceable`, `apiSchemas` are registered.

Exam takeaways.

KEY

- Registered names vs dictionaries. - Obj 5.3: put unknown keys in `customData/assetInfo`, not random attributes.

12.2 customData and assetInfo

What is it? Two dictionary metadata fields: **customData** (general) and **assetInfo** (asset identity).

Why do we need it? Obj 5.3 / 7.6. Resolvers, databases, and publishers look in `assetInfo`. Arbitrary pipeline notes go in `customData`.

Beginner explanation. `assetInfo` = the library card (ISBN, edition). `customData` = sticky notes.

Where the analogy breaks: USD does not enforce that split. Pipelines should.

Technical explanation. - `SetCustomDataByKey("shot", "s001") / GetCustomData() / GetCustomDataByKey`. - `SetAssetInfoByKey("identifier", "chair") / GetAssetInfo()`. - Nested dictionaries are allowed. - Verified: schema prims may already contain `userDocBrief` inside `customData`. Read a **key**, do not assume the dict has only your keys.

Mental model.

```
assetInfo: identifier, name, version, payloadAssetDependencies, ...
customData: anything else
```

Simple example. Publisher writes `assetInfo.identifier = "props/chair"`. Shot tools never parse the file path for identity.

USDA example.

```
#usda 1.0

def Xform "Chair" (
    kind = "component"
    assetInfo = {
        string identifier = "props/chair"
```

```

    string name = "chair"
  }
  customData = {
    string shot = "s001"
    string department = "model"
  }
}
{
}

```

Python example.

```

from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
prim = UsdGeom.Xform.Define(stage, "/Chair").GetPrim()
prim.SetAssetInfoByKey("identifier", "props/chair")
prim.SetCustomDataByKey("shot", "s001")
print("assetInfo:", prim.GetAssetInfo())
print("shot:", prim.GetCustomDataByKey("shot"))

```

Expected output

```

assetInfo: {'identifier': 'props/chair'}
shot: s001

```

Real-world use case. An asset resolver (Ch 33) keys off `assetInfo.identifier` rather than the current filename so moves on disk do not break identity.

Common mistakes.

MISTAKE

Dumping the entire dict and asserting equality in tests — schema ``userDocBrief`` will surprise you (Ch 5).

Exam traps.

TRAP

"assetInfo is a relationship to the USDZ." It is metadata on the prim.

Exam takeaways.

KEY

- `assetInfo` = identity. `customData` = other pipeline keys. - Read by key.

12.3 documentation, hidden, kind

What is it? Three registered prim (or property) metadata fields you will see constantly.

Why do we need it? documentation for humans; hidden for UIs; kind for the model hierarchy (Ch 6).

Beginner explanation. documentation = tooltip. hidden = do not show in the outliner. kind = what kind of *asset* this is.

Technical explanation. - documentation — string. Layer also has `doc` in the header (stage / layer documentation). - hidden — bool. Hiding is **not** deactivating and **not** visibility=invisible. - kind — via `Usd.ModelAPI`.

Mental model. hidden ≠ inactive ≠ invisible. Three different switches.

Simple example. A rig's utility bones: hidden = true, still active, still computed.

USDA example.

```

#usda 1.0

def Xform "RigControls" (
  hidden = true
  documentation = "Do not show to lighting"

```

```
)
{
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
prim = UsdGeom.Xform.Define(stage, "/RigControls").GetPrim()
prim.SetMetadata("hidden", True)
prim.SetMetadata("documentation", "Do not show to lighting")
print("hidden:", prim.GetMetadata("hidden"))
print("IsActive:", prim.IsActive())
print("documentation:", prim.GetMetadata("documentation"))
```

Expected output

```
hidden: True
IsActive: True
documentation: Do not show to lighting
```

Real-world use case. usdview can hide hidden prims in the hierarchy while they still contribute.

Common mistakes.**MISTAKE**

Using `hidden` to skip expensive geometry. Use payloads, purpose, or deactivation (Ch 17, 39, 4).

Exam traps.**TRAP**

"hidden prims are skipped by Traverse()." Traverse skips **inactive** and abstract, not hidden.

Exam takeaways.**KEY**

- hidden $\neq$ active $\neq$ visibility. - kind is ModelAPI metadata, not typeName.

12.4 Layer metadata

What is it? The USDA header (...) fields: defaultPrim, upAxis, metersPerUnit, time range, subLayers, doc, startTimeCode, ...

Why do we need it? Stage-wide contract. You already set these in Ch 2 and 10. This section is the checklist.

Beginner explanation. The cover of the book: title, units, which chapter is the start (defaultPrim), table of contents (subLayers).

Technical explanation. | Field | Why it matters | |-----|-----| | defaultPrim | Referencing without a path (Ch 16) | | upAxis | Y vs Z | | metersPerUnit | No auto-convert | | timeCodesPerSecond / framesPerSecond | Time mapping | | startTimeCode / endTimeCode | Shot range | | subLayers | Local stack (Ch 3) | | doc | Layer documentation |

stage.GetMetadata("upAxis") reads authored stage/root metadata.

Mental model. Header = layer metadata. Parentheses on a prim = prim metadata.

Simple example. Every published component header should set defaultPrim, upAxis, metersPerUnit.

USDA example.

```
#usda 1.0
(
```

```

    defaultPrim = "Chair"
    doc = "Hero chair v12"
    metersPerUnit = 0.01
    upAxis = "Y"
    startTimeCode = 1
    endTimeCode = 1
)

def Xform "Chair"
{
}

```

Python example.

```

from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
UsdGeom.SetStageUpAxis(stage, UsdGeom.Tokens.y)
UsdGeom.SetStageMetersPerUnit(stage, 0.01)
world = UsdGeom.Xform.Define(stage, "/Chair")
stage.SetDefaultPrim(world.GetPrim())
print("defaultPrim:", stage.GetDefaultPrim().GetName())
print("upAxis:", UsdGeom.GetStageUpAxis(stage))
print("metersPerUnit:", UsdGeom.GetStageMetersPerUnit(stage))

```

Expected output

```

defaultPrim: Chair
upAxis: Y
metersPerUnit: 0.01

```

Real-world use case. A publish linter fails assets missing `defaultPrim` (Obj 4.6).

Common mistakes.

MISTAKE

Setting ``upAxis`` as an attribute on ``/World``.

Exam traps.

TRAP

``defaultPrim = "/Chair"`` with a slash — it is a `**name**`, ``"Chair"``.

Exam takeaways.

KEY

- Publishable layers: `defaultPrim`, `upAxis`, `metersPerUnit`, and a sensible time range.

12.5 Representing pipeline metadata (Obj 5.3, 7.6)

What is it? A decision guide: attribute vs metadata vs API schema vs custom typed schema.

Why do we need it? The exam asks you to “represent custom metadata” and to know when a **schema** is the right upgrade (Obj 3.4, 3.6).

Beginner explanation. Start with `customData`. If many tools need typed fields, graduate to an API schema.

Technical explanation. | Need | Put it here | |-----|-----| | Shot name, notes, one-off flags | `customData` | | Stable asset id / version | `assetInfo` | | Kind / hidden / docs | registered metadata | | Values a renderer must see | attributes / primvars | | Shared typed fields on many prims | API schema (Ch 37) | | A new kind of object | typed schema |

Do not store geometry in `customData`. Do not store the studio’s asset id as `primvars:id` unless shading needs it.

Mental model. Visibility to renderer → property. Visibility to pipeline → metadata.

Simple example. `studio:task = "model"` → `customData.points` → attribute.

USDA example.

```
#usda 1.0

def Xform "Chair" (
  kind = "component"
  assetInfo = {
    string identifier = "props/chair"
  }
  customData = {
    string department = "model"
    int version = 12
  }
)
{
}
```

Python example.

```
from pxr import Usd, UsdGeom, Kind

stage = Usd.Stage.CreateInMemory()
prim = UsdGeom.Xform.Define(stage, "/Chair").GetPrim()
Usd.ModelAPI(prim).SetKind(Kind.Tokens.component)
prim.SetAssetInfoByKey("identifier", "props/chair")
prim.SetCustomDataByKey("department", "model")
prim.SetCustomDataByKey("version", 12)
print(prim.GetAssetInfo()["identifier"], prim.GetCustomDataByKey("department"), prim.GetCustomDataByKey("version"))
```

Expected output

```
props/chair model 12
```

Real-world use case. A DCC exporter writes `customData` on the way out and a USD importer reads it back for round-trip (Obj 4.4) without polluting Mesh schemas.

Common mistakes.

MISTAKE

Encoding version in the prim `**name**` (``Chair_v12``). Names that churn break references. Use `assetInfo` + resolver (Ch 33).

Exam traps.

TRAP

"Custom metadata requires a C++ plugin." Dictionaries do not. Plugins are for **registered** new fields and schemas.

Exam takeaways.

KEY

- Pipeline strings → `customData` / `assetInfo`. - Geometry → attributes. Shared typed contracts → schemas.

Chapter lab(s)

Lab 08 (metadata, `customData`, `assetInfo`, stage metadata).

USDA reading exercise

USDA-13. Is `hidden = true` on a defined Mesh enough to remove it from `Traverse()`?

Answer: No. `Traverse` still visits it unless it is inactive or abstract.

Chapter review

If you see... → think...

If you see...	Think...
Custom metadata	customData / assetInfo
Asset identity	assetInfo
hidden vs active vs visibility	three switches
defaultPrim	layer metadata, a name

CHAPTER 13

Chapter 13 — Built-in Schemas, Xforms, and Extents

Exam domain: Data Modeling (13%) and Debugging (11%) · **Objectives:** 5.5, 5.6, 6.4
Study day: 4 · **Est. time:** 75 min **Prerequisites:** Chapters 6, 9, 11

Learning goals

- Place Mesh/Cube/Xform in the **Imageable / Xformable / Boundable / Gprim** chain.
- Explain **extent** and why it must stay in sync with `points` (Obj 5.6).
- Call `ComputeExtentFromPlugins` and set the extent attribute.
- Use `UsdGeom.BBoxCache` for world bounds.
- List common causes of **unexpected visual results** (Obj 5.5 / 6.4).

Key terms

Term	One-line definition
Imageable	Can be visualized; has purpose and visibility
Xformable	Can hold xformOps
Boundable	Has an <code>extent</code> (local-space AABB)
Gprim	Geometric primitive (Mesh, Cube, Sphere, ...)
extent	Two <code>float3</code> s: min and max corner in local space
BBoxCache	Cached computer of world/local bounds

13.1 Imageable, Xformable, Boundable, Gprim

What is it? UsdGeom’s typed-schema ladder: most drawables are **Gprims**, which are **Boundable**, **Xformable**, and **Imageable**.

Why do we need it? Obj 8.x / 5.5: visibility and purpose live on Imageable. Transforms on Xformable. Extent on Boundable. Knowing IsA tells you which APIs are legal.

Beginner explanation. Imageable = can be seen (or hidden). Xformable = can be moved. Boundable = has a box. Gprim = is a shape.

Where the analogy breaks: **Camera** is Xformable/Imageable but not a Gprim. **Scope** groups without a local transform stack like Xform. **Xform** is Xformable but not Boundable (it has no extent of its own).

Technical explanation. Verified: `UsdGeom.Mesh` and `UsdGeom.Cube` are Gprim, Boundable, Xformable, Imageable.

Imageable attributes:

- `visibility` fallback **inherited**
- `purpose` fallback **default** (also `render`, `proxy`, `guide`)

Xformable: `xformOp:*` and `xformOpOrder` (Ch 39).

Boundable: `extent` (`float3[]` of length 2).

Mental model.

```

Imageable
├── Xformable
│   └── Boundable → Gprim → Mesh, Cube, Sphere, ...

```



```

└─ Camera
└─ (Scope is imageable grouping)

```

Simple example. A bounding-box tool should `IsA(UsdGeom.Boundable)`, not hard-code `Mesh`.

USDA example.

```

#usda 1.0

def Cube "Box"
{
    double size = 2
    token purpose = "default"
    token visibility = "inherited"
}

```

Python example.

```

from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
cube = UsdGeom.Cube.Define(stage, "/Box").GetPrim()
print("Gprim", cube.IsA(UsdGeom.Gprim))
print("Boundable", cube.IsA(UsdGeom.Boundable))
print("Xformable", cube.IsA(UsdGeom.Xformable))
print("Imageable", cube.IsA(UsdGeom.Imageable))
img = UsdGeom.Imageable(cube)
print("visibility", img.GetVisibilityAttr().Get())
print("purpose", img.GetPurposeAttr().Get())

```

Expected output

```

Gprim True
Boundable True
Xformable True
Imageable True
visibility inherited
purpose default

```

Real-world use case. A “hide guides” button sets purpose filters on `BBoxCache` / `Hydra`, not deletion.

Common mistakes.

MISTAKE

Calling ``GetExtentAttr()`` on an Xform group. Groups are not `Boundable`; use `BBoxCache` to union children.

Exam traps.

TRAP

"Camera is a Gprim." It is not.

Exam takeaways.

KEY

- `Mesh/Cube`: `Imageable` + `Xformable` + `Boundable` + `Gprim`. - visibility fallback ``inherited``; purpose fallback ``default``.

13.2 The extent attribute (Obj 5.6)

What is it? `extent` is a `float3[]` of two corners — min and max of the prim’s geometry in **local** space.

Why do we need it? `Hydra`, instancing, draw modes, and frustum tools use `extent` (and `extentsHint`) as a **fast box**. If you move `points` and leave `extent` stale, you get clipped or missing draws (Obj 5.5, 5.6).

Beginner explanation. extent is the cardboard box around the object, measured before parent transforms.

Where the analogy breaks: Cube/Sphere schemas can **fallback** a correct extent from size/radius even when extent is not authored. A **Mesh** does not: `Get()` is `None` until you author it. Verified.

Technical explanation. - Cube size fallback 2.0 → extent fallback `[(-1,-1,-1), (1,1,1)]` even with `HasAuthoredValue()` `False`. - Mesh with points set: `GetExtentAttr().Get()` is **None** until `Set`. - After updating points you **must** recompute and `Set` extent (Obj 5.6). - Type: `float3[]`, length 2, not `point3f[]` (common type trap).

Mental model.

```
extent[0] = min xyz
extent[1] = max xyz
local space, axis-aligned
```

Simple example. Triangle points (0,0,0), (2,0,0), (0,3,0) → extent [(0,0,0), (2,3,0)].

USDA example.

```
#usda 1.0

def Mesh "Tri"
{
    int[] faceVertexCounts = [3]
    int[] faceVertexIndices = [0, 1, 2]
    point3f[] points = [(0, 0, 0), (2, 0, 0), (0, 3, 0)]
    float3[] extent = [(0, 0, 0), (2, 3, 0)]
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/Tri")
mesh.GetPointsAttr().Set([(0, 0, 0), (2, 0, 0), (0, 3, 0)])
mesh.GetFaceVertexCountsAttr().Set([3])
mesh.GetFaceVertexIndicesAttr().Set([0, 1, 2])
print("authored before:", mesh.GetExtentAttr().HasAuthoredValue(), mesh.GetExtentAttr().Get())
computed = UsdGeom.Boundable.ComputeExtentFromPlugins(
    UsdGeom.Boundable(mesh.GetPrim()), Usd.TimeCode.Default()
)
print("computed:", computed)
mesh.GetExtentAttr().Set(computed)
print("authored after:", mesh.GetExtentAttr().Get())
```

Expected output

```
authored before: False None
computed: [(0, 0, 0), (2, 3, 0)]
authored after: [(0, 0, 0), (2, 3, 0)]
```

Real-world use case. A deformer writes new points at each frame and must write extent time samples too, or the bounding box used for loading/culling is wrong.

Common mistakes.

MISTAKE

Updating points in a loop and never touching extent. Obj 5.6 is written for this.

Exam traps.

TRAP

"extent is in world space." Local. Parent xforms are applied by `BBoxCache`.

Exam takeaways.

KEY

- Mesh extent is not automatic. Compute and author it after points change. - Cube/Sphere may fallback; do not generalize that to Mesh.

13.3 ComputeExtent and ComputeExtentFromPlugins

What is it? APIs that **calculate** a bounding box from schema data.

`ComputeExtentFromPlugins` dispatches to the right Gprim plugin (Mesh, Cube, Sphere, ...).

Why do we need it? You should not hand-write min/max loops in every exporter. Obj 5.6: after editing points, compute then set.

Beginner explanation. Ask the schema: “given these points (or this size), what box do you occupy?”

Technical explanation. - `UsdGeom.Boundable.ComputeExtentFromPlugins(boundable, timeCode)` → `Vt.Vec3fArray` of two vectors, or empty/failure if it cannot. - There is also `ComputeExtent` on some concrete schemas. - Pass the **TimeCode** that matches the points you care about (Default vs frame). - `UsdGeom.ModelAPI.GetExtentsHint` — a model-level hint for unloaded payloads (Ch 26).

Mental model. Compute → then `GetExtentAttr().Set(...)`. Computing does not author by itself.

Simple example. Cube: plugins return `[(-1, -1, -1), (1, 1, 1)]` for default size 2.

USDA example. Computation is runtime. The authored result is ordinary USDA `extent`.

```
#usda 1.0

def Cube "Box"
{
    double size = 2
    float3[] extent = [(-1, -1, -1), (1, 1, 1)]
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
cube = UsdGeom.Cube.Define(stage, "/Box")
print("size fallback:", cube.GetSizeAttr().Get())
print(
    "plugins:",
    UsdGeom.Boundable.ComputeExtentFromPlugins(
        UsdGeom.Boundable(cube.GetPrim()), Usd.TimeCode.Default()
    ),
)
```

Expected output

```
size fallback: 2.0
plugins: [(-1, -1, -1), (1, 1, 1)]
```

Real-world use case. A USD exporter’s last step: traverse Boundables, compute extent at Default and at sample times, author them.

Common mistakes.**MISTAKE**

Computing at Default while points only exist as time samples — you get an empty/wrong box. Pass the frame.

Exam traps.

TRAP

"ComputeExtentFromPlugins writes the attribute." It returns values. You `Set`.

Exam takeaways.**KEY**

- Compute at the correct TimeCode, then Set. - Plugins know Cube vs Mesh vs Sphere.

13.4 UsdGeom.BBoxCache

What is it? **BBoxCache** computes **world** (and local) bounds, applying transforms, purpose, visibility, and child unioning, with caching.

Why do we need it? extent is local. Layout needs a box in the stage's world. Instancers and cameras use world bounds.

Beginner explanation. extent = box in object space. BBoxCache = box after every parent transform.

Technical explanation.

```
cache = UsdGeom.BBoxCache(timeCode, [UsdGeom.Tokens.default_])
cache.ComputeWorldBound(prim).GetRange()
```

- Second argument is the list of **purposes** to include.
- Reuse one cache when traversing many prims.
- Verified: triangle mesh world range `[(0,0,0)...(2,3,0)]` with identity xform.

Mental model. BBoxCache = extent + xform hierarchy + purpose filter.

Simple example. Frame-all-in-camera: union world bounds of selected prims.

USDA example. No USDA for the cache. The mesh it bounds:

```
#usda 1.0

def Mesh "Tri"
{
    point3f[] points = [(0, 0, 0), (2, 0, 0), (0, 3, 0)]
    int[] faceVertexCounts = [3]
    int[] faceVertexIndices = [0, 1, 2]
    float3[] extent = [(0, 0, 0), (2, 3, 0)]
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/Tri")
mesh.GetPointsAttr().Set([(0, 0, 0), (2, 0, 0), (0, 3, 0)])
mesh.GetFaceVertexCountsAttr().Set([3])
mesh.GetFaceVertexIndicesAttr().Set([0, 1, 2])
computed = UsdGeom.Boundable.ComputeExtentFromPlugins(
    UsdGeom.Boundable(mesh.GetPrim()), Usd.TimeCode.Default()
)
mesh.GetExtentAttr().Set(computed)
cache = UsdGeom.BBoxCache(Usd.TimeCode.Default(), ["default"])
print(cache.ComputeWorldBound(mesh.GetPrim()).GetRange())
```

Expected output

```
[(0, 0, 0)...(2, 3, 0)]
```

Real-world use case. A set dresser snaps objects using world bounds, not local extent.

Common mistakes.

MISTAKE

Creating a new BBoxCache per prim in a 100k loop. Reuse it.

Exam traps.**TRAP**

Passing purposes ``["render"]`` and wondering why ``purpose=proxy`` geometry is missing from the box.

Exam takeaways.**KEY**

- Local box = extent. World box = BBoxCache. - Include the purposes you actually draw.

13.5 Unexpected visual results (Obj 5.5, 6.4)

What is it? A checklist of authored-data bugs that look like “USD is broken” in the viewer.

Why do we need it? Obj 5.5 / 6.4: understand what causes unexpected visuals. Many are data-model mistakes from this part of the book.

Beginner explanation. If it looks wrong, ask: units, up axis, extent, primvar interpolation, winding, purpose/visibility, material binding, composition (later).

Technical explanation. | Symptom | Likely cause | |-----|-----| | Object 100× too small/large | metersPerUnit mismatch; no auto-convert (Ch 2, 28) | | Object on its side | upAxis Y vs Z | | Mesh missing or clipped | Stale/missing **extent**; wrong purpose; inactive; payload unloaded | | Black / wrong colors | displayColor interpolation; missing primvar; shader vs crayon | | Inside-out mesh | orientation / winding / left-handed points | | Invisible but prim exists | visibility=invisible vs inactive vs purpose | | Transform ignored | Missing xformOpOrder (Ch 39) | | Material does not show | Binding API not applied; target outside defaultPrim (Ch 40) | | Instance color edits all copies | Wrote on prototype, not proxy (Ch 24) |

This chapter owns the **extent / primvar / units / purpose** rows. Composition rows expand in Ch 42.

Mental model. Wrong picture → check data, then composition, then renderer.

Simple example. Artist moves vertices in DCC, exporter writes points but not extent → usdview culls the mesh.

USDA example. A mesh that will look “empty” to bounds-based tools (no extent, no points):

```
#usda 1.0

def Mesh "Broken"
{
    int[] faceVertexCounts = [3]
    int[] faceVertexIndices = [0, 1, 2]
}
```

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
broken = UsdGeom.Mesh.Define(stage, "/Broken")
print("points:", broken.GetPointsAttr().Get())
print("extent:", broken.GetExtentAttr().Get())
print(
    "plugins:",
    UsdGeom.Boundable.ComputeExtentFromPlugins(
        UsdGeom.Boundable(broken.GetPrim()), Usd.TimeCode.Default()
```

```
),
)
```

Expected output

```
points: None
extent: None
plugins: None
```

Real-world use case. A ticket “hero disappeared at frame 50”: points have samples, extent only has a Default box from rest pose. At frame 50 the character has walked out of the rest box.

Common mistakes.

MISTAKE

Blaming Hydra first. Check extent and purpose before render-delegate settings.

Exam traps.

TRAP

A question that shows updated points and an old extent — the answer is Obj 5.6, not "USD cannot deform."

Exam takeaways.

KEY

- Obj 5.6: points and extent stay together (including time samples). - Obj 5.5: units, axis, extent, interpolation, purpose/visibility, bindings. - Mesh extent is not free; Cube/Sphere fallbacks can hide the issue in tests.

Chapter lab(s)

Lab 11 (mesh from scratch + compute extent). Lab 12 primvars. Combined checkpoint: Day 4.

USDA reading exercise

USDA-14. Cube with `size = 2` and no authored extent. What box do bounds APIs still get at rest, and why might a Mesh with authored points but no extent differ?

Answer: Cube can fallback to `[(-1, -1, -1), (1, 1, 1)]`. `Mesh Get()` is None until authored; plugins can still compute if points exist, but many systems read the **attribute**.

Chapter review

If you see... → think...

If you see...	Think...
Update points	Update extent (Obj 5.6)
World box needed	BBoxCache
Missing mesh in viewer	extent / purpose / payload / visibility
100× scale	metersPerUnit
Cube extent not authored	schema fallback — don't assume Mesh does that

Part III — Composition

CHAPTER 14

Chapter 14 — Composition Fundamentals

Exam domain: Composition (23%) · **Objectives:** 1.1, 1.6 · **Study day:** 5 · **Est. time:** 120 min **Prerequisites:** Ch 3 (Layers), Ch 4 (Prims and Prim Paths), Ch 5 (Properties), Ch 8 (Usd vs. Sdf)

Part III starts here. Composition is the largest exam domain, and it also shows up inside the debugging, pipeline, and content-aggregation questions. This chapter gives you the vocabulary and the core rules. Chapters 15–22 then study each composition arc in depth.

Learning goals

- Explain what composition does and why OpenUSD composes layers instead of merging them.
- Define an opinion and its strength, and name three ways to change which opinion wins (Obj 1.1).
- List the seven composition arcs and recite LIVERPS in strength order (Obj 1.6).
- Predict the result of list editing with `prepend`, `append`, `delete`, and explicit lists.
- Describe a layer stack, a prim index, and a prim stack, and inspect them from Python.
- Tell which fields are replaced by the strongest opinion and which fields combine.

Key terms

Term	One-line definition
Composition	Combining opinions from many layers, connected by arcs, into one stage view
Composition arc	A link that pulls opinions from another layer or another prim path into a prim
Opinion	One authored value for one field (attribute value, metadata) at one path in one layer
Strength	The rank of an opinion; the strongest opinion for a field usually wins
LIVERPS	Strength order of arcs: Local, Inherits, VariantSets, rElocates, References, Payloads, Specializes
Layer stack	A root layer plus all its sublayers (recursively), in strength order
Local opinion	An opinion that comes from the stage's own layer stack (root, sublayers, session)
List op	A field that stores edits (<code>prepend</code> , <code>append</code> , <code>delete</code>) or an explicit list
List editing	Changing a weaker layer's list from a stronger layer without retyping it
Pcp	"Prim cache population", the OpenUSD library that computes composition
Prim index	Pcp's per-prim graph of nodes (sources of opinions), ordered by strength
Node / site	One source in the prim index; a site is a layer stack plus a prim path
Prim stack	The flat list of prim specs that contribute to a prim, strongest first
Flatten	Bake a composed stage into one layer with no arcs left

14.1 What composition is

What is it? Composition is the process that combines the contents of many layers into the single scene you see on a stage. Layers are connected by **composition arcs** (sublayers, references, variants, and others). OpenUSD follows those arcs, collects every **opinion** (authored value) about each prim, and decides which one wins.

Why do we need it? Real scenes are built by many people and tools. A modeler publishes a chair. A set dresser places fifty chairs in a room. A lighter tweaks a lamp. If all of that lived in one file, people would overwrite each other, every copy of the chair would be duplicated data, and nobody could update the chair once and see it change everywhere. Composition lets each piece stay in its own file and be reused, while the stage shows the combined result.

Beginner explanation. Think of **transparent sheets on an overhead projector**. Each sheet (layer) has some drawings on it. You stack them. Where two sheets draw on the same spot, the top sheet wins. Where only one sheet draws, you see that drawing. A reference is like **linking a reusable part from a catalog**: instead of drawing the chair again, the sheet says “put catalog item *chair* here”.

Where the analogy breaks: sheets on a projector only stack. Composition also has arcs that copy a whole subtree to a new path (references), switch between alternatives (variants), and share edits across many prims (inherits). The “top wins” rule also has exceptions: some fields, such as lists and dictionaries, combine instead of replacing.

Technical explanation. - Composition is computed by the **Pcp** library (“prim cache population”). The `Usd.Stage` asks Pcp to build a **prim index** for each prim. The prim index lists every source of opinions for that prim, in strength order (see 14.5). - Composition happens **per prim and per field**. For each field (an attribute’s default value, a metadata key such as `kind`, a relationship’s targets) OpenUSD looks at the opinions in strength order. For most fields the strongest opinion wins. This step is called **value resolution** (Chapter 21). - Composition is **non-destructive**. It never changes the source layers. If you edit a layer, the stage recomposes and shows the new result. - The `Usd` API (`Usd.Stage`, `Usd.Prim`, `Usd.Attribute`) shows you the **composed** result. The `Sdf` API (`Sdf.Layer`, `Sdf.PrimSpec`) shows you **what one layer authored** (Chapter 8). - `prim.GetPrimStack()` returns the `Sdf.PrimSpec` objects that contribute to a prim, strongest first. This is your first debugging tool for “where did this come from?”.

Mental model.

layers on disk	composition (Pcp)	what you see
+-----+		+-----+
room.usda ---+		Usd.Stage
+-----+		/Room
dressing ---+----->	follow arcs, collect	/Room/Chair
+-----+	opinions per prim, ----->	color=walnut
chair.usda ---+	order them by strength	height=0.9
+-----+	(files never change)	+-----+

Simple example. | Layer | Opinion about `/Room/Chair.color` |
 |-----|-----| | `dressing.usda` (sublayer of the room) | “walnut” | |
 | `chair.usda` (referenced asset) | “oak” | | **Composed result** | “walnut”, because local
 opinions beat referenced ones |

The height attribute is only authored in `chair.usda`, so the stage shows the chair’s own value `0.9`.

USDA example. File: `room.usda`

```
#usda 1.0
(
  subLayers = [@dressing.usda@]
)
```



```
def Xform "Room"
{
    def "Chair" (
        prepend references = @chair.usda@
    )
    {
    }
}
```

- `subLayers = [@dressing.usda@]` — adds a sublayer. Its opinions join this file's layer stack (Chapter 15).
- `def "Chair" (...)` — defines the prim `/Room/Chair` in this layer.
- `prepend references = @chair.usda@` — a reference arc. It pulls in the default prim of `chair.usda` (Chapter 16). `prepend` is a list-editing operation (14.4).
- The empty `{ }` body means this layer authors no values of its own on the chair.

Python example. The script writes three small files, opens the room, prints the composed values, lists the prim stack, and shows that `chair.usda` itself was not changed.

```
from pxr import Sdf, Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("chair.usda", """"#usda 1.0
(
    defaultPrim = "Chair"
)
def Xform "Chair"
{
    string color = "oak"
    double height = 0.9
}
""")
write("dressing.usda", """"#usda 1.0
over "Room"
{
    over "Chair"
    {
        string color = "walnut"
    }
}
""")
write("room.usda", """"#usda 1.0
(
    subLayers = [@dressing.usda@]
)
def Xform "Room"
{
    def "Chair" (
        prepend references = @chair.usda@
    )
    {
    }
}
""")

stage = Usd.Stage.Open("room.usda")
chair = stage.GetPrimAtPath("/Room/Chair")
print("composed color :", chair.GetAttribute("color").Get())
print("composed height:", chair.GetAttribute("height").Get())
for spec in chair.GetPrimStack():
    print("prim spec in", spec.layer.GetDisplayName(), "at", spec.path)

chair_layer = Sdf.Layer.FindOrOpen("chair.usda")
print("chair.usda still says:",
      chair_layer.GetAttributeAtPath("/Chair.color").default)
```

Expected output

```
composed color : walnut
composed height: 0.9
prim spec in room.usda at /Room/Chair
prim spec in dressing.usda at /Room/Chair
prim spec in chair.usda at /Chair
chair.usda still says: oak
```

Notice that the referenced spec lives at `/Chair` in its own file, but it contributes to `/Room/Chair` on the stage. Arcs can remap paths.

Real-world use case. In a film shot, the environment, the characters, the animation, and the lighting are separate files owned by separate departments. The shot file only contains arcs and a few overrides. When the modeling department republishes the chair, every shot that references it shows the new chair the next time it is opened. The same idea drives digital twins: a factory layout references one robot asset many times, and a fix to that asset reaches every station.

Common mistakes.**MISTAKE**

Editing the asset file to fix one shot. The fix then leaks into every shot that uses the asset. Author the override in the shot's own layer (a stronger, local opinion) instead.

MISTAKE

Reading a layer with ``Sdf`` and expecting composed values.
``Sdf.Layer.GetAttributeAtPath(...).default`` shows only what that one layer authored. Use ``Usd.Stage`` and ``Usd.Attribute.Get()`` for the composed answer.

MISTAKE

Thinking the stage is "one big file". A stage is a live view over many layers. Saving the stage saves the edited layers, not a merged copy.

Exam traps.**TRAP**

"Composition copies the referenced file into the scene" — false. Nothing is copied into your layer; opinions are read from the referenced layer at compose time.

TRAP

"The last layer loaded wins" — there is no time order. Strength comes from the arc type and the position of the layer, never from when a file was opened or saved.

TRAP

An answer that says composition happens "per file". It happens per prim (and value resolution per field). Two prims in the same file can have completely different sources.

Exam takeaways.**KEY**

- Composition = follow arcs, collect opinions per prim, rank them by strength. - It is computed by Pcp and is non-destructive: source layers never change. - ``Usd`` shows composed results; ``Sdf`` shows what one layer authored. - ``prim.GetPrimStack()`` answers "which layers contribute to this prim?".

14.2 Opinions and strength

What is it? An **opinion** is one authored value for one field at one path in one layer, for example `"/Ball.radius default is 2 in fix.usda"`. **Strength** is the rank of that opinion compared to other opinions about the same field. For most fields, the **strongest opinion wins**.

Why do we need it? Several layers often say different things about the same attribute. USD needs a deterministic rule to pick one, so every application that opens the scene shows the same result. Strength lets a downstream department override an upstream one on purpose, without editing the upstream file.

Beginner explanation. Use the **ladder of opinions**: picture a ladder where each rung is one opinion. USD asks the top rung first. If that rung has an answer, the question is settled. If the rung is silent about this field, USD asks the next rung down.

Where the analogy breaks: the ladder is built separately for each field. A rung can be silent about `radius` but have an answer for `color`. Also, a few fields (lists, dictionaries) collect answers from several rungs instead of taking just the top one (14.4, 14.6).

Technical explanation. Strength is decided in two steps:

1. **Within one layer stack**, stronger layers come first: the session layer, then the root layer, then sublayers in listed order (Chapter 15).
2. **Across arcs**, the arc type decides: opinions in the local layer stack beat inherits, which beat variants, and so on down LIVERPS (14.3, Chapter 21). Arcs nest: inside a referenced asset, the same rules apply again to that asset's own arcs.

An opinion only exists where something is authored. An `over` with no attributes contributes no attribute opinions. A schema's fallback value (for example `radius = 1` for `UsdGeom.Sphere`) is **not** an opinion; it is used only when no layer has an opinion at all.

Changing the strength of an opinion (Obj 1.1). You cannot set a "priority number" on an opinion. To make a value win, you move it or restructure the arcs:

Technique	How
Author in a stronger layer	Put the override in the root layer, a stronger sublayer, or the session layer
Reorder sublayers	Change the order of <code>subLayers</code> (first is strongest)
Use a stronger arc	A local opinion beats a referenced one; an inherited class beats a reference
Reorder an arc list	<code>prepend</code> puts an arc first (strongest) in its list; <code>append</code> puts it last
Remove the competitor	Delete the stronger opinion (<code>attr.Clear()</code> / <code>ClearAtTime()</code>) or the arc providing it

Introspection: `attr.GetPropertyStack()` returns the attribute specs that hold opinions, strongest first. `attr.GetResolveInfo()` tells you the source of the winning value (Chapter 21).

Mental model.

ask strongest first ----->				
[session]	->	[root]	->	[sublayer 1] -> [sublayer 2] -> [arcs: I V E R P S]
silent		silent		radius=2 radius=1
				^
				first answer wins: radius = 2

Simple example. `shot.usda` has `subLayers = [@fix.usda@, @model.usda@]`. `fix.usda` says `radius 2`, `model.usda` says `radius 1`. The composed radius is 2. Swap the order to `[@model.usda@, @fix.usda@]` and it becomes 1. Neither file changed.

USDA example. *File: fix.usda*

```
#usda 1.0

over "Ball"
{
    double radius = 2
}
```

- `over "Ball"` — “I have opinions about `/Ball`, but I am not defining it”. The definition (`def Sphere "Ball"`) lives in a weaker layer.
- `double radius = 2` — one opinion: the default value of `radius`. This layer is silent about every other field of `/Ball`.

Python example. Three ways to change which opinion wins, without editing the opinions themselves.

```
from pxr import Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("model.usda", '#usda 1.0\ndef Sphere "Ball"\n{\n    double radius = 1\n}\n')
write("fix.usda", '#usda 1.0\nover "Ball"\n{\n    double radius = 2\n}\n')
write("shot.usda", '#usda 1.0\n(\n    subLayers = [@fix.usda, @model.usda]\n)\n')

stage = Usd.Stage.Open("shot.usda")
radius = stage.GetPrimAtPath("/Ball").GetAttribute("radius")
stack = [spec.layer.GetDisplayName() for spec in radius.GetPropertyStack()]
print("1 start          :", radius.Get(), stack)

stage.GetRootLayer().subLayerPaths = ["model.usda", "fix.usda"]
print("2 reorder        :", radius.Get())

with Usd.EditContext(stage, stage.GetRootLayer()):
    radius.Set(3.0)
print("3 root layer     :", radius.Get())

with Usd.EditContext(stage, stage.GetSessionLayer()):
    radius.Set(4.0)
print("4 session layer   :", radius.Get())
stack = [spec.layer.GetDisplayName() for spec in radius.GetPropertyStack()]
print(" property stack  :", stack)
```

Expected output

```
1 start          : 2.0 ['fix.usda', 'model.usda']
2 reorder        : 1.0
3 root layer     : 3.0
4 session layer   : 4.0
 property stack  : ['shot-session.usda', 'shot.usda', 'model.usda', 'fix.usda']
```

`Usd.EditContext` temporarily sets the **edit target**, the layer that receives new edits (Chapter 15). The session layer is an in-memory layer that is stronger than the root layer (Chapter 3).

Real-world use case. In games and film, a lighting artist needs a lamp brighter in one shot only. The lamp asset says `inputs:intensity = 10`. The lighter authors `inputs:intensity = 40` in the shot’s lighting layer, which is stronger than the referenced asset. The asset stays untouched for all other shots. In AEC review sessions, a reviewer can hide a wall in the session layer without changing any saved file.

Common mistakes.

MISTAKE

Authoring an override into a layer that is weaker than the current winner, then wondering why nothing changed. Check `attr.GetPropertyStack()` first and author into a layer above the winning one.

MISTAKE

Treating a schema fallback as an opinion. A sphere with no authored `radius` reports 1.0, but no layer says so. `attr.HasAuthoredValue()` is `False` in that case.

MISTAKE

Expecting the most recently saved file to win. File timestamps play no role in strength.

Exam traps.**TRAP**

"Change the strength of an opinion" questions want a structural answer: move the opinion to a stronger layer, reorder sublayers, or use a stronger arc. There is no per-opinion priority field.

TRAP

A weaker layer that authors **more** attributes does not become stronger. Strength is per field: the weaker layer still wins for fields the stronger layer leaves silent.

TRAP

Session-layer edits win over the root layer, but they are not saved with `stage.Save()` (`stage.Save()` skips the session layer; `stage.SaveSessionLayers()` exists for that).

Exam takeaways.**KEY**

- Opinion = one authored value for one field, at one path, in one layer. - Strongest opinion wins, decided per field: layer order inside a layer stack, then arc type (LIVERPS). - To change strength: move the opinion, reorder sublayers or arc lists, use a stronger arc, or remove the competitor. - Schema fallbacks are not opinions. `GetPropertyStack()` shows who has an opinion.

14.3 The seven arcs at a glance

What is it? OpenUSD has seven **composition arcs**, the ways a prim can receive opinions from somewhere else: **sublayers**, **inherits**, **variant sets**, **relocates**, **references**, **payloads**, and **specializes**. Their strength order is remembered with the acronym **LIVERPS**.

Why do we need it? Each arc solves a different problem: stacking work (sublayers), reusing assets (references), deferring heavy data (payloads), switching alternatives (variants), broadcasting shared edits (inherits), providing overridable defaults (specializes), and renaming or moving prims that came from somewhere else (relocates). Knowing the order lets you predict which arc wins when two of them disagree.

Beginner explanation. | Arc | Analogy (used across this book) |

|----|-----| | Sublayers | Transparent sheets on a projector; top sheet wins | | Inherits | A style sheet every instance follows, even later edits | | Variant sets | A switch with named positions | | Relocates | A forwarding address: the prim now lives at a new path | | References | Linking a reusable part from a catalog | | Payloads | A reference you can choose not to load (a box in storage) | | Specializes | A default that anything else can override |

Where the analogy breaks: the arcs are not independent tools. They nest (a referenced asset can contain variants that contain references), and the strength rules apply again at every level of nesting.

Technical explanation. LIVERPS, strongest to weakest:

Letter	Arc	USDA keyword	Where it is authored	Chapter
L	Local opinions (the layer stack built by sublayers)	subLayers	layer metadata	15
I	Inherits	inherits	prim metadata	19
V	VariantSets	variantSets, variants, variantSet	prim metadata and body	18
E	rElocates	relocates	layer metadata	20
R	References	references	prim metadata	16
P	Payloads	payload	prim metadata	17
S	Specializes	specializes	prim metadata	19

- “Local” is not a separate arc keyword. It means all opinions from the stage’s own layer stack: the root layer, its sublayers, and the session layer. Sublayers are the arc that builds that stack.
- Inherits and specializes point at a prim path (often a `class` prim). References and payloads point at a layer (and optionally a prim path in it). Variants select one of several named bodies inside the same prim.
- **Relocates** are authored as layer metadata: `relocates = { </Old/Path>: </New/Path> }`. In Python you set them with `Sdf.Layer.relocates` (Chapter 20).
- `Usd.PrimCompositionQuery(prim).GetCompositionArcs()` lists the arcs on a prim in strength order. Each `Usd.CompositionArc` reports `GetArcType()` (a `Pcp.ArcType` value), `GetTargetLayer()`, and `GetTargetPrimPath()`.

VERSION

Verified on USD 26.08. The OpenUSD glossary spells the acronym ****LIVERPS**** (Local, Inherits, VariantSets, rElocates, References, Payloads, Specializes). Older books, talks, and the NVIDIA study material may say ****LIVRPS****, which simply leaves out relocates; the relative order of the other six arcs is the same. Relocates authoring and composition arrived in the USD 24.x releases (Chapter 20).

NOTE

The Python enum ``Pcp.ArcType`` lists its values as Root, Inherit, Relocate, Variant, Reference, Payload, Specialize. That enum order is not a strength chart; use LIVERPS.

Mental model.

STRONGEST				WEAKEST		
L ----->	I ----->	V ----->	E ----->	R ----->	P ----->	S
local	inherits	variants	relocates	references	payloads	specializes
(layer	(classes)	(switch)	(moved	(catalog	(lazy	(overridable
stack)			prims)	parts)	parts)	defaults)
"LIVERPS": the closer to L, the louder the voice.						

Simple example. A prim `/Bot` gets `team` from an inherited class, `scale` from its selected variant, `triangles` from a payload, and `paint` from a specialized default. If the class and the specialized prim both authored `paint`, the class (Inherits, I) would win over Specializes (S).

USDA example. File: *scene.usda* (one prim using every arc; `robot.usda`, `heavy.usda`, and `layout.usda` are separate files)

```
#usda 1.0
(
    subLayers = [@layout.usda@]
    relocates = {
        </Bot/Rig/Arm>: </Bot/Arm>
    }
)

class "_Base"
{
    string team = "blue"
}

def "_Defaults"
{
    string paint = "grey"
}

def Xform "Bot" (
    inherits = </_Base>
    specializes = </_Defaults>
    prepend references = @robot.usda@
    prepend payload = @heavy.usda@
    variantSets = "size"
    variants = {
        string size = "big"
    }
)
{
    variantSet "size" = {
        "big" {
            double scale = 2
        }
        "small" {
            double scale = 0.5
        }
    }
}
```

- `subLayers` (L) and `relocates` (E) are **layer** metadata in the parentheses at the top of the file.
- `inherits`, `specializes`, `references`, `payload`, `variantSets` are **prim** metadata in the parentheses after `def Xform "Bot"`.
- `variants = { string size = "big" }` is the variant **selection**; the `variantSet "size" = { ... }` block holds the variant bodies.
- The `relocate` moves `Arm` (which arrives from `robot.usda` under `Rig`) to `/Bot/Arm`.

Python example.

```
from pxr import Pcp, Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("robot.usda", """>#usda 1.0
(
    defaultPrim = "Robot"
)
def Xform "Robot"
{
    def Xform "Rig"
    {
        def Xform "Arm"
        {
```

```

        double length = 1
    }
}
"""
write("heavy.usda", ""#usda 1.0
(
    defaultPrim = "Geo"
)
def Xform "Geo"
{
    int triangles = 50000
}
"""
write("layout.usda", '#usda 1.0\nover "Bot"\n{\n    string fromLayout = "yes"\n}\n')
write("scene.usda", ""#usda 1.0
(
    subLayers = [@layout.usda@]
    relocates = {
        </Bot/Rig/Arm>: </Bot/Arm>
    }
)
class "_Base"
{
    string team = "blue"
}
def "_Defaults"
{
    string paint = "grey"
}
def Xform "Bot" (
    inherits = </_Base>
    specializes = </_Defaults>
    prepend references = @robot.usda@
    prepend payload = @heavy.usda@
    variantSets = "size"
    variants = {
        string size = "big"
    }
)
{
    variantSet "size" = {
        "big" {
            double scale = 2
        }
        "small" {
            double scale = 0.5
        }
    }
}
"""

stage = Usd.Stage.Open("scene.usda")
bot = stage.GetPrimAtPath("/Bot")
for arc in Usd.PrimCompositionQuery(bot).GetCompositionArcs():
    print(f"{str(arc.GetArcType()):24}",
          arc.GetTargetLayer().GetDisplayName(), arc.GetTargetPrimPath())

for name in ["fromLayout", "team", "scale", "triangles", "paint"]:
    print(name, "=", bot.GetAttribute(name).Get())

print("children of /Bot:", bot.GetChildrenNames())
arm = stage.GetPrimAtPath("/Bot/Arm")
for arc in Usd.PrimCompositionQuery(arm).GetCompositionArcs():
    if arc.GetArcType() == Pcp.ArcTypeRelocate:
        print("/Bot/Arm is relocated from", arc.GetTargetPrimPath())

```

Expected output

Pcp.ArcTypeRoot	scene.usda /Bot
Pcp.ArcTypeInherit	scene.usda /_Base
Pcp.ArcTypeVariant	scene.usda /Bot{size=big}
Pcp.ArcTypeReference	robot.usda /Robot
Pcp.ArcTypePayload	heavy.usda /Geo


```
Pcp.ArcTypeSpecialize    scene.usda /_Defaults
fromLayout = yes
team = blue
scale = 2.0
triangles = 50000
paint = grey
children of /Bot: ['Rig', 'Arm']
/Bot/Arm is relocated from /Bot/Rig/Arm
```

The query lists arcs in LIVERPS order. “Root” is the local layer stack (the L), which includes `layout.usda`. `Usd.Stage.Open` loads payloads by default, so `triangles` is visible (Chapter 17). The `Rig` prim stays, but `Arm` has moved out of it.

Real-world use case. A game studio’s level file sublayers a layout layer (L), references a prop catalog (R), loads heavy terrain through payloads (P), offers `lod` variants (V), puts shared “team color” settings on a class that all enemy prims inherit (I), and uses specializes for material defaults that any asset may override (S). Relocates let a character pipeline rename a rig joint hierarchy coming from a referenced rig without editing the rig file.

Common mistakes.

MISTAKE

Writing ``relocates`` in a prim's metadata. Relocates are `**layer**` metadata, at the top of the file, mapping old paths to new paths.

MISTAKE

Spelling the payload keyword ``payloads`` in USDA. The keyword is ``payload`` (for example ``prepend payload = @heavy.usda@``), even though the arc is called “payloads” in prose and the Python API is ``prim.GetPayloads()``.

MISTAKE

Thinking “Local” means “this one file”. Local means the whole local layer stack, including every sublayer and the session layer.

Exam traps.

TRAP

References are stronger than payloads, but that is a strength statement. It says nothing about loading: payloads can be unloaded, references cannot.

TRAP

Inherits (I) is near the top and Specializes (S) is at the bottom, even though both point at a class-like prim. A question pairing them is testing exactly that.

TRAP

“Variants are the weakest because they are inside the prim” — false. VariantSets (V) are stronger than references, payloads, and specializes.

Exam takeaways.

KEY

- Seven arcs: sublayers, inherits, variant sets, relocates, references, payloads, specializes. - LIVERPS = Local, Inherits, VariantSets, rElocates, References, Payloads, Specializes (strongest first). Older material says LIVRPS. - L = the local layer stack (root, sublayers, session), not one file. - Relocates and sublayers are layer metadata; the other arcs are prim metadata. - ``Usd.PrimCompositionQuery`` lists a prim's arcs in strength order.

14.4 List editing: `prepend`, `append`, `delete`, `explicit`

What is it? Some fields hold a **list**, for example the list of references on a prim, its inherit paths, its applied API schemas, or a relationship’s targets. These fields are stored as **list ops** (list operations). A stronger layer can **edit** a weaker layer’s list (add to the front, add to the back, remove items) instead of replacing it. This is **list editing**.

Why do we need it? Without list editing, a shot that wanted one extra reference on a prim would have to retype the full list from the asset. If the asset later added another reference, the shot’s copy would silently hide it. List editing lets each layer say only what it changes, so upstream additions still flow through.

Beginner explanation. Think of a shared shopping list on the fridge. Instead of rewriting the whole list, each family member leaves a sticky note: “add milk at the top”, “add bread at the bottom”, “cross out sugar”. Someone reading the list applies the notes from the most important person last, so their notes have the final say. If someone writes a brand-new list and says “ignore everything else”, that is an **explicit** list.

Where the analogy breaks: in USD the order of the final list matters. For arcs, an item near the front of the list is **stronger** than an item near the back.

Technical explanation. A list op (`Sdf.ReferenceListOp`, `Sdf.PathListOp`, `Sdf.TokenListOp`, ...) is either **explicit** or a set of **edits**:

USDA form	Meaning
<code>references = [@a@, @b@]</code>	Explicit: this is the whole list; ignore all weaker opinions
<code>prepend references = @c@</code>	Put <code>c</code> at the front of the list (strongest position)
<code>append references = @d@</code>	Put <code>d</code> at the back of the list (weakest position)
<code>delete references = @a@</code>	Remove <code>a</code> if a weaker layer added it

Rules verified on USD 26.08:

- Edits are applied weakest layer first, strongest layer last, so the strongest layer has the final word.
- Within one layer’s edits, `delete` is applied first, then `prepend`, then `append`.
- Prepending an item that is already in the list **moves** it to the front. Appending an existing item moves it to the back.
- Deleting an item that is not in the list does nothing (no error).
- An explicit list in a stronger layer discards all weaker opinions. An explicit list in a weaker layer is simply the starting list that stronger edits modify.
- One spec cannot be explicit and have edits at the same time. If a USDA prim writes both `references = ...` and `prepend references = ...`, the parser keeps only the last statement.
- `references = None` is an explicit **empty** list: it removes every weaker reference.

Python authoring: `prim.GetReferences().AddReference(...)` writes `prepend` by default (`position=Usd.ListPositionFrontOfPrependList`). Other positions are `Usd.ListPositionBackOfPrependList`, `Usd.ListPositionFrontOfAppendList`, `Usd.ListPositionBackOfAppendList`. `RemoveReference` writes a `delete` (or removes the item from an explicit list); `SetReferences([...])` writes an explicit list; `ClearReferences()` removes this layer’s opinion. The same pattern exists on `GetInherits()`, `GetSpecializes()`, `GetPayloads()`, and relationships (`AddTarget`, `RemoveTarget`, `SetTargets`).

NOTE

Older files may contain ``add`` and ``reorder`` list operations. They are legacy forms; new content should use ``prepend``, ``append``, ``delete``, or an explicit list.

Mental model.

weakest layer :	prepend [a, b]	-> list = [a, b]
stronger layer :	delete [b]	-> list = [a]
	append [c]	-> list = [a, c]
strongest layer:	prepend [n]	-> list = [n, a, c]
		^ front = strongest arc
strongest layer:	explicit [x] (instead)	-> list = [x] (all else ignored)

Simple example. The asset layer says `prepend references = [@a.usda@, @b.usda@]`. The shot layer says `delete references = @b.usda@` and `append references = @c.usda@`. The composed reference list is `[a, c]`: `b` is gone, `c` joined at the back, and `a` is the strongest reference.

USDA example. *File: strong.usda*

```
#usda 1.0

over "Thing" (
    delete references = @b.usda@
    append references = @c.usda@
)
{
}
```

- `over "Thing"` — this layer edits `/Thing`, which is defined in a weaker layer.
- `delete references = @b.usda@` — removes the `b.usda` reference that a weaker layer added.
- `append references = @c.usda@` — adds `c.usda` at the back, the weakest position.
- Neither line restates `a.usda`; it survives because this layer does not mention it.

Python example. First, the pure list rules with `Sdf.TokenListOp` (no stage needed):

```
from pxr import Sdf

start = ["M", "N"]

edits = Sdf.TokenListOp()
edits.deletedItems = ["M"]
edits.prependedItems = ["A"]
edits.appendedItems = ["Z"]
print("edits      :", edits.ApplyOperations(start))

move = Sdf.TokenListOp()
move.prependedItems = ["N"]
print("prepend existing:", move.ApplyOperations(start))

explicit = Sdf.TokenListOp.CreateExplicit(["X"])
print("explicit      :", explicit.ApplyOperations(start), explicit.isExplicit())

missing = Sdf.TokenListOp()
missing.deletedItems = ["Q"]
print("delete missing :", missing.ApplyOperations(start))
```

Expected output

```
edits      : ['A', 'N', 'Z']
prepend existing: ['N', 'M']
explicit    : ['X'] True
delete missing : ['M', 'N']
```

Now the same idea with real references across two sublayers, plus what Python writes when you author list edits:

```
from pxr import Sdf, Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)
```

```

for src in ["a", "b", "c"]:
    write(f"{src}.usda", f""#usda 1.0
    (
        defaultPrim = "P"
    )
def "P"
{{
    string src = "{src}"
}}
""")
write("weak.usda", ""#usda 1.0
def "Thing" (
    prepend references = [@a.usda@, @b.usda@]
)
{
}
""")
write("strong.usda", ""#usda 1.0
over "Thing" (
    delete references = @b.usda@
    append references = @c.usda@
)
{
}
""")
write("shot.usda", "#usda 1.0\n\n    subLayers = [@strong.usda@, @weak.usda@]\n\n")

stage = Usd.Stage.Open("shot.usda")
thing = stage.GetPrimAtPath("/Thing")
query = Usd.PrimCompositionQuery.GetDirectReferences(thing)
names = [arc.GetTargetLayer().GetDisplayName() for arc in query.GetCompositionArcs()]
print("composed references:", names)
print("src resolves to    :", thing.GetAttribute("src").Get())

mem = Usd.Stage.CreateInMemory()
x = mem.DefinePrim("/X")
x.GetReferences().AddReference("a.usda")
x.GetReferences().AddReference("c.usda", position=Usd.ListPositionBackOfAppendList)
ops = mem.GetRootLayer().GetPrimAtPath("/X").referenceList
print("authored prepend  :", [r.assetPath for r in ops.prependedItems])
print("authored append   :", [r.assetPath for r in ops.appendedItems])

```

Expected output

```

composed references: ['a.usda', 'c.usda']
src resolves to    : a
authored prepend   : ['a.usda']
authored append    : ['c.usda']

```

`a.usda` is first in the composed list, so its `src` opinion beats `c.usda`'s. The last two lines read the raw list op stored in the in-memory layer: `AddReference` wrote a `prepend` by default and an `append` when asked.

NOTE

`thing.GetMetadata("references")` returns only the strongest layer's list op (here the delete/append edits), not the composed list. To see the composed result, use Usd.PrimCompositionQuery` (Chapter 22).`

Real-world use case. A character asset applies `prepend apiSchemas = ["MaterialBindingAPI"]`. A shot adds a physics schema with its own `prepend apiSchemas = [...]`; both survive because list ops combine. In a factory digital twin, a station layer uses `delete references = @old_gripper.usda@` and `prepend references = @new_gripper.usda@` to swap a tool without retyping the robot's other references.

Common mistakes.

MISTAKE

Writing ``references = @c.usda@`` (explicit) when you meant "add one more". The explicit list wipes every weaker reference. Use ``prepend`` or ``append``.

MISTAKE

Using ``append`` for an override reference that must win. Appended items are the weakest. Use ``prepend`` when the new arc should be strongest.

MISTAKE

Writing ``prepend subLayers = [...].`subLayers`` is a plain ordered list, not a list op; that line does not parse. Just edit the list (Chapter 15).

Exam traps.**TRAP**

"``delete`` errors if the item is missing" — false; it is silently ignored.

TRAP

In an arc list, **`**front = strongest**`**. A question that asks which of two prepended references wins wants the one listed first in the strongest layer's edit.

TRAP

``references = None`` does not mean "no opinion". It is an explicit empty list that removes all weaker references.

Exam takeaways.**KEY**

- List ops: explicit (replace everything weaker) or edits (``prepend`` front, ``append`` back, ``delete`` remove). - Edits apply weakest first, strongest last; front of an arc list = strongest arc. - Prepending an existing item moves it; deleting a missing item is a no-op. - ``AddReference`` → ``prepend``; ``SetReferences`` → explicit; ``subLayers`` is not a list op.

14.5 Layer stacks and the prim index

What is it? A **layer stack** is a root layer plus all its sublayers (recursively), in strength order. A stage's **local layer stack** also includes the session layer on top. The **prim index** is the graph Pcp builds for each prim: one **node** per source of opinions (the local layer stack, each inherited class, each variant, each reference...), ordered by strength. Flattening the prim index into a list of prim specs gives the **prim stack**.

Why do we need it? These are the data structures behind every "why did this value win?" answer. The exam's debugging questions (Obj 1.8, Obj 6.2) expect you to reason about which layer stack and which node an opinion came from. Inspecting the prim index turns guessing into reading.

Beginner explanation. Picture a family tree of opinion sources. The prim itself is at the top. Under it hang its sources: "my local layers", "the class I inherit", "the asset I reference". Each source is itself a little stack of sheets (a layer stack). To answer a question, USD walks the tree in a fixed order (LIVERPS) and reads each stack of sheets top-down.

Where the analogy breaks: a family tree has one parent per person. A prim index can contain the same layer more than once (for example an inherited class in the same file as the prim), and sources can be implied across arcs (Chapter 19).

Technical explanation. - **Layer stack** (`Pcp.LayerStack`): root layer + sublayers, recursively depth-first. Each referenced or payloaded file brings **its own** layer stack (its root plus its sublayers). `stage.GetLayerStack()` returns the local one, session layers first. - **Site**: a layer stack plus a prim path, for example “the layer stack of `robot.usda` at `/Robot`”. - **Node** (`Pcp.NodeRef`): one site in the prim index plus how it got there: `node.arcType`, `node.layerStack`, `node.path`, `node.children`, `node.parent`. The **root node** is the prim’s own path in the local layer stack. - **Prim index** (`Pcp.PrimIndex`, from `prim.GetPrimIndex()`): the tree of nodes. Strength order is a depth-first walk: a node, then its children in LIVERPS order. Each child’s own children (for example the arcs inside a referenced asset) are visited before moving on to the next sibling. - **Prim stack**: `prim.GetPrimStack()` (or `primIndex.primStack`) returns the `Sdf.PrimSpec` objects in that order. Value resolution walks the same order (Chapter 21). - `primIndex.DumpToString()` prints a full debug dump. It contains memory addresses of anonymous layers, so this book walks the nodes itself to keep output stable. - Composition **errors** (for example an unresolvable reference) are recorded on the prim index (`primIndex.localErrors`) and reported as warnings; the stage still opens (Chapter 43).

Mental model.

prim index for <code>/Bot</code>		prim stack (strongest first)
-----		-----
Root [scene-session, scene] <code>/Bot</code>	-->	<code>scene.usda</code> <code>/Bot</code>
-- Inherit [scene...] <code>/_Base</code>	-->	<code>scene.usda</code> <code>/_Base</code>
`-- Reference [robot, robot_geo] <code>/Robot</code>	-->	<code>robot.usda</code> <code>/Robot</code>
	-->	<code>robot_geo.usda</code> <code>/Robot</code>

Simple example. `/Bot` in `scene.usda` inherits `/_Base` and references `robot.usda`. `robot.usda` sublayers `robot_geo.usda`. The prim index has three nodes (root, inherit, reference). The reference node’s layer stack has two layers, so it contributes two prim specs.

USDA example. *File: `robot.usda`* (the referenced asset has its own layer stack)

```
#usda 1.0
(
    defaultPrim = "Robot"
    subLayers = [@robot_geo.usda@]
)

def Xform "Robot"
{
    string status = "published"
}
```

- `subLayers = [@robot_geo.usda@]` — `robot.usda` and `robot_geo.usda` form one layer stack. When `scene.usda` references `robot.usda`, that whole stack becomes one reference node.
- `defaultPrim = "Robot"` — the prim a reference uses when it names no prim path.

Python example.

```
from pxr import Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("robot_geo.usda", '#usda 1.0\nover "Robot"\n{    int faces = 12\n}\n')
write("robot.usda", ""#usda 1.0
(
    defaultPrim = "Robot"
    subLayers = [@robot_geo.usda@]
)
```

```
def Xform "Robot"
{
    string status = "published"
}
"""
write("scene.usda", ""#usda 1.0
class "_Base"
{
    string team = "blue"
}
def Xform "Bot" (
    inherits = </_Base>
    prepend references = @robot.usda@
)
{
}
"""

stage = Usd.Stage.Open("scene.usda")
print("local layer stack:", [l.GetDisplayName() for l in stage.GetLayerStack()])
bot = stage.GetPrimAtPath("/Bot")

def walk(node, depth=0):
    layers = [l.GetDisplayName() for l in node.layerStack.layers]
    arc = str(node.arcType).replace("Pcp.ArcType", "")
    print("  " * depth + f"{arc:9} {node.path} {layers}")
    for child in node.children:
        walk(child, depth + 1)

walk(bot.GetPrimIndex().rootNode)
for spec in bot.GetPrimStack():
    print("spec:", spec.layer.GetDisplayName(), spec.path)
```

Expected output

```
local layer stack: ['scene-session.usda', 'scene.usda']
Root      /Bot ['scene-session.usda', 'scene.usda']
  Inherit  /_Base ['scene-session.usda', 'scene.usda']
  Reference /Robot ['robot.usda', 'robot_geo.usda']
spec: scene.usda /Bot
spec: scene.usda /_Base
spec: robot.usda /Robot
spec: robot_geo.usda /Robot
```

The session layer is part of every local-layer-stack node, but it has no specs here, so it is absent from the prim stack.

Real-world use case. A pipeline technical director (TD) debugging “the robot is the wrong color in shot 42” walks the prim index: is the winning opinion in the shot’s local layer stack, in a class, in a variant, or deep inside the referenced asset’s sublayers? Studio debugging tools and usdview’s composition view display this same tree.

Common mistakes.

MISTAKE

Assuming `stage.GetLayerStack()` lists every layer in the scene. It lists only the **local** layer stack. Layers reached through references and payloads are listed by `stage.GetUsedLayers()`.

MISTAKE

Printing `DumpToString()` in tests or docs and expecting stable output. It includes memory addresses; walk `rootNode` and its `children` instead.

MISTAKE

Looking only at the referencing file when a value is wrong. The referenced asset's own sublayers are part of the reference node and may hold the winning opinion.

Exam traps.**TRAP**

"A referenced file's sublayers are ignored" — false. A reference brings the target's whole layer stack.

TRAP

Prim stack vs. layer stack: the layer stack is a list of **layers** for a stage or asset; the prim stack is a list of **prim specs** for one prim.

Exam takeaways.**KEY**

- Layer stack = root + sublayers (recursive); the local one also has the session layer on top.
 - Prim index = tree of nodes (sites) in LIVERPS order; prim stack = its prim specs, strongest first. - A reference or payload node carries the target's whole layer stack. -
``prim.GetPrimIndex()``, ``prim.GetPrimStack()``, ``stage.GetLayerStack()``,
``stage.GetUsedLayers()``.

14.6 Composition is not merging

What is it? Composition does **not** merge files into one, and it does not blend values. Every layer stays separate. For each field, USD **picks** the strongest opinion. Only a few kinds of fields **combine** across layers: list ops (14.4), dictionaries such as `customData`, and the set of child prims and properties. Turning a composed stage into one merged layer is a separate, explicit step called **flattening**.

Why do we need it? Because layers stay separate, you can change, swap, mute, or reorder any of them and the stage updates. A merged file would freeze everyone's work together and lose the information about who authored what. Knowing which fields are picked and which combine prevents surprises, such as an array that you expected to "add to" but that was replaced.

Beginner explanation. Back to the projector sheets: if the top sheet draws a red circle and the bottom sheet draws a blue circle in the same spot, you see red, not purple. Composition never mixes the two. A photocopy of the projected image would be "flattening": you get one sheet, but you can no longer lift off the top layer.

Where the analogy breaks: some fields behave like a list or a labeled form instead of a drawing. A dictionary (like `customData`) is combined key by key, so a stronger layer can change one key and keep the weaker layer's other keys.

Technical explanation. | Field kind | Across layers | Example |
 |-----|-----|-----| | Attribute value (scalar **or** array) | Strongest wins; arrays are never concatenated | `int[] ids` | | Most metadata (`kind`, `active`, `documentation`, ...) | Strongest wins | `kind = "component"` | | List ops (`references`, `inherits`, `apiSchemas`, `relationship targets`, ...) | Edits combine (14.4) | `prepend apiSchemas` | | Dictionaries (`customData`, `assetInfo`) | Combined key by key; strongest wins per key | `customData` | | Child prims and properties | Union of names from all contributing specs | children of a prim | | Specifier (`def/over/class`) | A stronger `over` never undoes a weaker `def` | `over "Box" over def "Box"` |

Flattening:

- `stage.Flatten()` returns a new anonymous `Sdf.Layer` with all composition baked in: no sublayers, references, variants, or classes remain, only resolved specs. It is used for delivery and debugging (Chapter 22, Chapter 34).
- `UsdUtils.FlattenLayerStack(stage)` flattens only the local layer stack and keeps other arcs (Chapter 34).

- `stage.Export("file.usda")` writes the flattened result to a file.

Mental model.

COMPOSITION (live)		FLATTEN (one-time bake)
+-----+ +-----+		+-----+
strong weak	Flatten()	one layer, values picked
ids=[9] ids=[1,	----->	ids=[9] label="weak"
2,3]		no arcs, no sublayers
+-----+ +-----+		+-----+
files stay separate; pick per field		history of "who said what" lost

Simple example. `weak.usda` sets `ids = [1, 2, 3]` and `label = "weak"`. `strong.usda` sets `ids = [9]`. The composed `ids` is `[9]`, not `[9, 1, 2, 3]`. The composed `label` is `"weak"`, because the strong layer is silent about it.

USDA example. File: *weak.usda*

```
#usda 1.0

def "Box" (
  customData = {
    string owner = "modeling"
    int version = 1
  }
  prepend apiSchemas = ["CollectionAPI:a"]
)
{
  int[] ids = [1, 2, 3]
  string label = "weak"
}
```

- `customData = {...}` — a dictionary. A stronger layer can override `version` alone and keep `owner`.
- `prepend apiSchemas = [...]` — a token list op; stronger layers can add more schemas.
- `int[] ids = [1, 2, 3]` — an array value; a stronger `ids` replaces it entirely.

Python example.

```
from pxr import Sdf, Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("weak.usda", """"#usda 1.0
def "Box" (
  customData = {
    string owner = "modeling"
    int version = 1
  }
  prepend apiSchemas = ["CollectionAPI:a"]
)
{
  int[] ids = [1, 2, 3]
  string label = "weak"
}
""")
write("strong.usda", """"#usda 1.0
over "Box" (
  customData = {
    int version = 7
  }
  prepend apiSchemas = ["CollectionAPI:b"]
)
{
  int[] ids = [9]
}
```

```

}
"""
write("shot.usda", "#usda 1.0\n(\n    subLayers = [@strong.usda@, @weak.usda@]\n)\n")

stage = Usd.Stage.Open("shot.usda")
box = stage.GetPrimAtPath("/Box")
print("ids      :", box.GetAttribute("ids").Get())
print("label     :", box.GetAttribute("label").Get())
print("customData :", sorted(box.GetCustomData().items()))
print("apiSchemas :", box.GetAppliedSchemas())

flat = stage.Flatten()
print("flat subLayers:", list(flat.subLayerPaths))
print("flat ids      :", flat.GetAttributeAtPath("/Box.ids").default)
weak = Sdf.Layer.FindOrOpen("weak.usda")
print("weak.usda ids :", weak.GetAttributeAtPath("/Box.ids").default)

```

Expected output

```

ids      : [9]
label     : weak
customData : [('owner', 'modeling'), ('version', 7)]
apiSchemas : ['CollectionAPI:b', 'CollectionAPI:a']
flat subLayers: []
flat ids      : [9]
weak.usda ids : [1, 2, 3]

```

The array was replaced, the dictionary combined per key, and the list op combined. Flattening produced one layer with no sublayers, while `weak.usda` kept its own value.

Real-world use case. Studios keep composition live during production so departments can iterate, then flatten for delivery to a vendor or a game engine that should not depend on the studio's file structure (Obj 1.9, Chapter 22). Pipelines often write `customData` or `assetInfo` from several layers (for example the modeling department writes `owner`, the publishing tool writes `version`) and rely on the per-key combination.

Common mistakes.

MISTAKE

Authoring ``points = [new points]`` in a shot layer expecting to "add" points to a mesh. Arrays are replaced; author the full array.

MISTAKE

Flattening a production scene to "fix" a composition problem. You lose the arcs that made it editable. Debug the composition instead (Chapter 22).

MISTAKE

Overriding a whole ``customData`` dictionary to change one key, thinking you must restate all keys. You only need to author the keys you change.

Exam traps.

TRAP

"Composition merges the layers into the root layer" — false. Merging is what ``Flatten()`` does, on request, into a new layer.

TRAP

A question showing two array opinions and offering a concatenated answer. Arrays never concatenate across layers.

TRAP

A question asking which field kinds combine. Correct choices are list ops and dictionaries (and the set of children/properties), not attribute values.

Exam takeaways.**KEY**

- Composition picks the strongest opinion per field; it never blends or concatenates values. - Combining fields: list ops, dictionaries (`customData`, `assetInfo`), and the set of children/properties. - Flattening (`stage.Flatten()`, `stage.Export()`) is an explicit bake into one layer; arcs are lost. - Source layers are never modified by composition.

Chapter lab(s)

This chapter has no lab of its own. Practice it with **Lab 05 — Build a sublayer stack and see who wins** (Chapters 3 and 15): you build a three-layer stack, predict the winning opinion, then reorder layers and check your prediction. Later, **Lab 20 — LIVERPS laboratory: predict, then verify** (Chapter 21) combines every arc from 14.3 into one puzzle scene.

USDA reading exercises

Exercise 14-A. `shot.usda` contains `subLayers = [@strong.usda@, @weak.usda@]`. Each of `base.usda`, `old.usda`, and `new.usda` has a default prim with `string src` set to its own name.

File: weak.usda

```
#usda 1.0

def "Lamp" (
    prepend references = [@base.usda@, @old.usda@]
)
{
}
```

File: strong.usda

```
#usda 1.0

over "Lamp" (
    delete references = @old.usda@
    prepend references = @new.usda@
)
{
}
```

What is the composed reference list on `/Lamp`, and what does `/Lamp.src` resolve to?

Exercise 14-B. `shot2.usda` contains `subLayers = [@s2.usda@, @w2.usda@]`.

File: w2.usda

```
#usda 1.0

def "Rock" (
    customData = {
        string a = "w"
        string b = "w"
    }
)
{
    float[] weights = [1, 2]
    float size = 3
}
```

File: s2.usda

```
#usda 1.0

over "Rock" (
    customData = {
        string b = "s"
    }
}
```

```

)
{
    float[] weights = [5]
}

```

What are the composed `weights`, `size`, and `customData` of `/Rock`? Is `/Rock` defined?

Chapter review

Summary

- Composition combines opinions from many layers, linked by arcs, into one stage view. `Pcp` computes it; source layers never change.
- An opinion is one authored value for one field in one layer. Strength decides the winner, per field.
- Change strength structurally: move the opinion to a stronger layer, reorder sublayers or arc lists, use a stronger arc, or remove the competitor (Obj 1.1).
- Seven arcs; LIVERPS order: Local, Inherits, VariantSets, `rElocates`, References, Payloads, Specializes (Obj 1.6). Older material: LIVRPS.
- Relocates and sublayers are layer metadata; the other arcs are prim metadata.
- List ops: explicit lists replace weaker opinions; `prepend/append/delete` edit them. Front of an arc list is strongest.
- A layer stack is a root plus sublayers; each reference brings its target's whole layer stack.
- The prim index is a tree of nodes in strength order; the prim stack is its specs, strongest first.
- Values are picked, never blended. Only list ops, dictionaries, and `child/property` name sets combine.
- Flattening is an explicit bake into one layer; arcs are lost.

If you see... → think...

If you see...	Think...
"Which opinion wins?"	Walk LIVERPS, then layer order inside each layer stack
"Change the strength of an opinion"	Move it to a stronger layer, reorder, or use a stronger arc
<code>prepend</code> vs <code>append</code> on an arc	<code>Prepend</code> = front = strongest; <code>append</code> = back = weakest
<code>references = [...]</code> without a keyword	Explicit list: weaker reference opinions are discarded
<code>relocates = {...}</code>	Layer metadata; the E in LIVERPS, between variants and references
Two array values in two layers	Strongest array wins whole; no concatenation
<code>customData</code> in two layers	Combined per key
"Where did this value come from?"	<code>GetPropertyStack()</code> , <code>GetPrimStack()</code> , <code>PrimCompositionQuery</code>
"Merge the scene into one file"	<code>stage.Flatten()</code> / <code>stage.Export()</code> ; arcs are lost

Review questions

14-R1 · Obj 1.6 · Single choice In LIVERPS, which letter stands for an arc that is authored in layer metadata and remaps prim paths? A. L B. E C. R D. S

14-R2 · Obj 1.6 · Select two. Which **two** arcs are weaker than References? A. Inherits B. Payloads C. Specializes D. VariantSets

14-R3 · Obj 1.1 · Single choice A shot sublayers `[@anim.usda@, @layout.usda@]`. Layout wants its `/Car.xformOp:translate` to win over animation in this shot only, without touching `anim.usda`. What should be done? A. Re-save `layout.usda` so it is newer. B. Reorder the shot's sublayers to `[@layout.usda@, @anim.usda@]`. C. Convert `layout.usda` to `.usdc`. D. Add `prepend` before the attribute in `layout.usda`.

14-R4 · Obj 1.1 · USDA reading A weaker layer has `prepend references = [@a.usda@, @b.usda@]`. A stronger layer has `append references = @a.usda@`. What is the composed list, strongest first? A. `[a, b]` B. `[b, a]` C. `[a, b, a]` D. `[a]`

14-R5 · Obj 1.6 · Single choice A prim inherits `/_Class` (which sets `size = 2`) and specializes `/_Defaults` (which sets `size = 5`). Neither the prim's layer stack nor any other arc sets `size`. What is `size`? A. 5 B. 2 C. 7 D. The schema fallback

14-R6 · Single choice What does `prim.GetPrimStack()` return? A. Layers of the local layer stack B. Prim specs that contribute to the prim, strongest first C. Child prims in namespace order D. The variant sets on the prim

14-R7 · Select two. Which **two** fields combine across layers instead of taking only the strongest opinion? A. `customData` B. `double radius` C. `apiSchemas` D. `kind`

14-R8 · Obj 1.1 · Python reading What does this print?

```
op = Sdf.TokenListOp()
op.deletedItems = ["x"]
op.prependedItems = ["y"]
print(op.ApplyOperations(["x", "z"]))
```

A. `['y', 'z']` B. `['x', 'y', 'z']` C. `['z', 'y']` D. `['y', 'x', 'z']`

14-R9 · Single choice Which statement about composition is correct? A. It writes the resolved values into the root layer when the stage opens. B. It is recomputed when a contributing layer changes, and it never edits the layers. C. It concatenates array attributes from all layers. D. It only considers the root layer and its direct sublayers.

14-R10 · Single choice `scene.usda` references `asset.usda`, which sublayers `asset_shading.usda`. A wrong material comes from `asset_shading.usda`. In the prim index of the referencing prim, where does that opinion live? A. In the root node, because sublayers are local B. In the reference node, whose layer stack includes `asset_shading.usda` C. Nowhere; sublayers of referenced files are ignored D. In a specializes node

14-R11 · Single choice A stronger layer authors `inherits = None`. A weaker layer authored `prepend inherits = </_Base>`. What happens? A. The prim still inherits `/_Base`. B. The prim inherits nothing: `None` is an explicit empty list. C. A parse error occurs. D. `/_Base` becomes a specializes arc.

14-R12 · Obj 1.6 · Single choice Which pair is in the correct strength order (stronger first)? A. Payloads, then References B. Specializes, then Inherits C. VariantSets, then References D. References, then Local

The code in 14-R8 is a fragment (it omits the import), so it is not run automatically.

Answers

Exercise 14-A — Answer. The weak layer gives `[base, old]`. The strong layer deletes `old` (`[base]`) and prepends `new` (`[new, base]`). `new.usda` is the strongest reference, so `Lamp.src` is `"new"`. Verified by composing the files: the composed reference list is `['new.usda', 'base.usda']` and `src` resolves to `new`.

Exercise 14-B — Answer. `weights = [5]` (the strong array replaces the weak one), `size = 3` (only the weak layer has an opinion), `customData = {a: "w", b: "s"}` (dictionaries combine per key). `/Rock` is defined: the strong layer only has an `over`, but the weak layer's `def` is the defining specifier. Verified by composing the files.

- **14-R1: B.** `E = rElocates`, authored as `relocates = {...}` layer metadata. Review: §14.3.
- **14-R2: B and C.** Payloads (P) and Specializes (S) come after References (R). Inherits and VariantSets are stronger. Review: §14.3.
- **14-R3: B.** Reordering sublayers in the shot changes strength for this shot only. Save time and file format do not affect strength; `prepend` is not an attribute keyword. Review: §14.2.
- **14-R4: B.** Appending an item that is already in the list moves it to the back: `[b, a]`. Review: §14.4.
- **14-R5: B.** Inherits (I) is stronger than Specializes (S). The values are not added. Review: §14.3.
- **14-R6: B.** The `prim` stack is a list of `Sdf.PrimSpecs`. Review: §14.5.
- **14-R7: A and C.** Dictionaries and list ops combine; `radius` and `kind` take the strongest opinion. Review: §14.6.
- **14-R8: A.** Delete `x` first (`['z']`), then prepend `y` (`['y', 'z']`). Review: §14.4.
- **14-R9: B.** Composition is live and non-destructive. Review: §14.1, §14.6.
- **14-R10: B.** A reference node carries the target's whole layer stack. Review: §14.5.
- **14-R11: B.** `None` authors an explicit empty list in the stronger layer. Review: §14.4.
- **14-R12: C.** `V` comes before `R` in LIVERPS. Review: §14.3.

CHAPTER 15

Chapter 15 — Sublayers, Layer Stacks, and Edit Targets

Exam domain: Composition (23%) · **Objectives:** 1.3, 1.5 (also 1.1) · **Study day:** 5 · **Est. time:** 110 min **Prerequisites:** Ch 3 (Layers), Ch 10 (Time Samples), Ch 14 (Composition Fundamentals)

Chapter 3 introduced sublayers and the session layer. Chapter 14 showed that the local layer stack is the “L” in LIVERPS, the strongest source of opinions. This chapter covers how to control that stack: its order, its timing, where your edits go, how to switch layers off temporarily, and how teams share one scene without overwriting each other.

Learning goals

- Predict the strength order of a layer stack with nested sublayers and a session layer.
- Apply a layer offset (`offset`, `scale`) to a sublayer and compute the stage time of a time sample.
- Direct edits to a chosen layer or variant with `Usd.EditTarget`, `SetEditTarget`, and `Usd.EditContext`.
- Mute and unmute layers to isolate problems, and explain what muting does not do.
- Design a sublayer-based layering strategy for several departments working on one shot (Obj 1.5).
- Compare sublayers with references and payloads for a given use case (Obj 1.3).

Key terms

Term	One-line definition
Sublayer	A layer listed in another layer’s <code>subLayers</code> metadata; its opinions join the same layer stack
Layer stack	A root layer plus all its sublayers, recursively, in strength order
Session layer	An in-memory layer that is stronger than the root layer and is not saved by <code>stage.Save()</code>
Layer offset	A time mapping (<code>offset</code> , <code>scale</code>) applied to a sublayer or reference: $\text{stage time} = \text{layer time} \times \text{scale} + \text{offset}$
<code>Sdf.LayerOffset</code>	The Python class that stores a layer offset
Edit target	The layer (and optional path mapping) that receives new edits made through the Usd API
<code>Usd.EditContext</code>	A <code>with</code> block that sets an edit target temporarily and restores the previous one
Variant edit target	An edit target that writes inside a chosen variant of a variant set
Layer muting	Temporarily removing a layer from composition on one stage, without editing any file
Department layer	A sublayer owned by one team (layout, animation, lighting) in a shared shot

15.1 Sublayer ordering

What is it? A layer's `subLayers` metadata is an **ordered list** of other layers. The layer that lists them and all its sublayers form a **layer stack**. In that stack, the order is the strength order: the layer itself first, then its sublayers from first to last, with each sublayer's own sublayers inserted right after it.

Why do we need it? Sublayers let many people contribute to the **same namespace** (the same prim paths) from separate files. A shot file can stack a layout layer, an animation layer, and a lighting layer. Their order decides who wins when two of them author the same attribute. Getting the order wrong is one of the most common reasons an edit “doesn't show up” (Obj 1.8).

Beginner explanation. This is the **transparent sheets on a projector** analogy from Chapter 3: the top sheet wins. The `subLayers` list is read from the top sheet down. If one of those sheets has its own little pile of sheets clipped to it, that pile goes right under it, before the next sheet in the list.

Where the analogy breaks: a sheet that is silent about a field lets weaker sheets “show through” for that field only. Also, the session layer is a sheet that sits above the root layer but is never saved to disk.

Technical explanation. - Strength order of a stage's local layer stack: **session layer** (and its sublayers) → **root layer** → root's `subLayers[0]` and its sublayers (depth-first) → `subLayers[1]` → ... (verified on USD 26.08). - `stage.GetLayerStack(includeSessionLayers=True)` returns the layers in that order. `stage.GetRootLayer().subLayerPaths` is the editable list of the root's direct sublayers. - `subLayers` is a plain ordered list, not a list op: there is no `prepend subLayers`. To change order, edit the list (`layer.subLayerPaths = [...]`, `.insert(0, path)`, `.append(path)`, `.remove(path)`). The stage recomposes immediately. - A layer can appear in a stack only once. Sublayer paths are resolved relative to the layer that lists them (anchored paths, Chapter 33). - All layers in one layer stack share one namespace: `/World/Car` in every sublayer is the same prim. That is the key difference from a reference, which grafts another file's prim to a new path (Chapter 16).

Sublayers vs. references vs. payloads (Obj 1.3)

Question	Sublayer	Reference	Payload
Strength position	L (strongest, local)	R	P (below R)
Namespace	Same paths as the root	Target prim is mapped to the referencing prim's path	Same as reference
Granularity	Whole layer	One prim (subtree)	One prim (subtree)
Can be unloaded?	No (but can be muted)	No	Yes (<code>stage.Unload</code> , Chapter 17)
Typical use	Department/work layers in a shot	Reusing an asset many times	Heavy geometry you may not need

Mental model.

shot.usda	subLayers = [a, b, d]; b.usda: subLayers = [c]
STRONGEST	[shot-session] -> [shot] -> [a] -> [b] -> [c] -> [d] WEAKEST
	____/ b's own sublayer comes right after b, before d

Simple example. | Layer (strongest first) | `/Ball.radius` | |-----|-----| |
shot.usda (root) | — | | a.usda | 1 | | b.usda | 2 | | c.usda (sublayer of b) | 3 | | d.usda | 4 |
| **Composed** | 1 |

Reorder the root's list to [d, a, b] and the composed radius becomes 4.

USDA example. *File: shot.usda*

```
#usda 1.0
(
    subLayers = [
        @a.usda@,
        @b.usda@,
        @d.usda@
    ]
)
```

- `subLayers = [...]` — layer metadata at the top of the file. The first entry, `a.usda`, is the strongest sublayer.
- `@...@` — asset-path syntax. Relative paths are resolved next to `shot.usda`.
- The root has no prims of its own; it only assembles the stack. Its own opinions, if any, would beat all sublayers.

Python example.

```
from pxr import Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("c.usda", '#usda 1.0\ndef Sphere "Ball"\n{\n    double radius = 3\n}\n')
write("b.usda", ""'#usda 1.0
(
    subLayers = [@c.usda@]
)
over "Ball"
{
    double radius = 2
}
""')
write("a.usda", '#usda 1.0\nover "Ball"\n{\n    double radius = 1\n}\n')
write("d.usda", '#usda 1.0\nover "Ball"\n{\n    double radius = 4\n}\n')
write("shot.usda", "#usda 1.0\n(\n    subLayers = [@a.usda@, @b.usda@, @d.usda@]\n)\n")

stage = Usd.Stage.Open("shot.usda")
print("layer stack:", [l.GetDisplayName() for l in stage.GetLayerStack()])
radius = stage.GetPrimAtPath("/Ball").GetAttribute("radius")
print("radius:", radius.Get())
print("opinions:", [s.layer.GetDisplayName() for s in radius.GetPropertyStack()])

stage.GetRootLayer().subLayerPaths = ["d.usda", "a.usda", "b.usda"]
print("after reorder:", radius.Get())

stage.SetEditTarget(stage.GetSessionLayer())
radius.Set(9.0)
print("after session edit:", radius.Get())
print("opinions:", [s.layer.GetDisplayName() for s in radius.GetPropertyStack()])
```

Expected output

```
layer stack: ['shot-session.usda', 'shot.usda', 'a.usda', 'b.usda', 'c.usda', 'd.usda']
radius: 1.0
opinions: ['a.usda', 'b.usda', 'c.usda', 'd.usda']
after reorder: 4.0
after session edit: 9.0
opinions: ['shot-session.usda', 'd.usda', 'a.usda', 'b.usda', 'c.usda']
```

`c.usda` sits between `b.usda` and `d.usda`: nested sublayers are depth-first. The session layer beats everything.

Real-world use case. A visual-effects shot file typically contains only `subLayers`: `shot_fx.usda`, `shot_lighting.usda`, `shot_anim.usda`, `shot_layout.usda`, and a sequence-wide layer at the bottom. Sequence-level fixes in the bottom layer reach every shot of the sequence, and any shot can still override them in its own, stronger layers. Architecture firms use the same pattern: a base building layer, then a stronger “design option” layer, then a stronger “review markup” layer.

Common mistakes.

MISTAKE

Assuming the last sublayer in the list is the strongest (“the last one written wins”). The **first** entry is the strongest.

MISTAKE

Writing ``prepend subLayers = [...]'``. That does not parse; ``subLayers`` is a plain list. Edit the list order instead.

MISTAKE

Using a sublayer to bring in one asset at a new location. Sublayers cannot move prims to a new path; use a reference (Chapter 16).

Exam traps.

TRAP

Nested sublayers: a question may show ``b.usda`` with its own ``subLayers``. Its children rank right after ``b``, **before** the next entry of the parent's list.

TRAP

The root layer's own opinions beat all of its sublayers. Only the session layer is stronger than the root.

TRAP

“Sublayers can be unloaded like payloads” — false. Sublayers can be muted (15.4), but loading and unloading is a payload feature.

Exam takeaways.

KEY

- Order: session → root → ``subLayers[0]`` (+ its sublayers, depth-first) → ``subLayers[1]`` → ...
- First entry of ``subLayers`` = strongest sublayer. The root's own opinions beat all sublayers.
- ``subLayers`` is a plain list (edit ``subLayerPaths``); no ``prepend``/``append`` keywords.
- Sublayers share one namespace; references graft a prim to a new path; payloads can be unloaded.

15.2 Layer offsets on sublayers

What is it? A **layer offset** shifts and/or scales the time of everything in a sublayer. It has two numbers: `offset` (a shift in time codes) and `scale` (a speed factor). The rule is **stage time = layer time × scale + offset**. In Python it is an `Sdf.LayerOffset`.

Why do we need it? An animation layer may have been authored starting at frame 0, but the shot starts at frame 1001. Or a layer must play twice as slowly. Without offsets you would have to rewrite every time sample in the file. With an offset, the file stays untouched and the stage maps its times.

Beginner explanation. Imagine a song file and a playlist. The playlist entry can say “start this song 20 seconds in” and “play it at half speed”. The song itself is not edited; only the playlist entry changes how it plays.

Where the analogy breaks: the offset does not crop anything. Samples before the shifted start still exist (USD holds the first sample's value before it). And default values (non-time-sampled values) have no time, so offsets do not affect them.

Technical explanation. - USDA syntax:

`subLayers = [@anim.usda@ (offset = 20; scale = 2)]`. Either field may be left out (defaults `offset = 0`, `scale = 1`). - Python: `layer.subLayerOffsets` is a list of `Sdf.LayerOffset`, parallel to `layer.subLayerPaths`. Assign an element to change it: `layer.subLayerOffsets[0] = Sdf.LayerOffset(20, 2)`. The stage recomposes immediately.

- `Sdf.LayerOffset(offset, scale)` maps a time: `offset_obj * layer_time` returns the stage time. `GetInverse()` maps back. `IsIdentity()` is true for (0, 1). - **Nested offsets compose.** If `top.usda` sublayers `mid.usda` with offset 5, and `mid.usda` sublayers

`anim.usda` with offset 10, a sample at time 0 in `anim.usda` appears at time 15. `Sdf.LayerOffset` objects can be multiplied: `A * B` means “apply B, then A”. -

timeCodesPerSecond (TCPS) mismatch. If a sublayer's `timeCodesPerSecond` differs

from the root's, USD adds a scale automatically. A 24-TCPS layer under a 48-TCPS root has its sample at time 24 appear at time 48 (verified on USD 26.08). -

`attr.GetTimeSamples()` on the stage reports times already mapped to stage time. - Offsets also exist on references and payloads (Chapter 16, Obj 1.4) and appear in value resolution (Chapter 21).

Mental model.

<code>anim.usda (layer time)</code>	0	-----	10	
<code>x scale (2)</code>	0	-----	20	
<code>+ offset (20)</code>				----- 40
<code>stage time</code>			20	----- 40
			20	----- 40
 <code>stage = layer * scale + offset</code> <code>layer = (stage - offset) / scale</code>				

Simple example. `anim.usda` has `tx = 0` at time 0 and 100 at time 10. Sublayered with (`offset = 20; scale = 2`), the stage sees samples at times 20 and 40. At stage time 30 (layer time 5) the linear interpolation gives 50.

USDA example. *File: shot.usda*

```
#usda 1.0
(
    endTimeCode = 40
    startTimeCode = 0
    subLayers = [
        @anim.usda@ (offset = 20; scale = 2)
    ]
)
```

- `@anim.usda@ (offset = 20; scale = 2)` — the layer offset is written in parentheses right after the asset path, with fields separated by `;`.
- `startTimeCode / endTimeCode` — the stage's playback range (Chapter 10). They are not changed by the offset; you set them to cover the shifted animation.

Python example.

```
from pxr import Sdf, Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("anim.usda", """#usda 1.0
def "Ball"
{
    double tx.timeSamples = {
```

```

        0: 0,
        10: 100,
    }
}
"""
write("shot.usda", ""#usda 1.0
(
    subLayers = [@anim.usda@ (offset = 20; scale = 2)]
)
"""

stage = Usd.Stage.Open("shot.usda")
tx = stage.GetPrimAtPath("/Ball").GetAttribute("tx")
print("stage samples:", tx.GetTimeSamples())
for t in (10, 20, 30, 40):
    print(f"tx at stage time {t}: {tx.Get(t)}")

off = stage.GetRootLayer().subLayerOffsets[0]
print("offset, scale:", off.offset, off.scale)
print("layer 10 -> stage", off * 10.0)
print("stage 30 -> layer", off.GetInverse() * 30.0)

stage.GetRootLayer().subLayerOffsets[0] = Sdf.LayerOffset(offset=100)
print("new offset samples:", tx.GetTimeSamples())

write("mid.usda", "#usda 1.0\n(\n    subLayers = [@anim.usda@ (offset = 10)]\n)\n")
write("top.usda", "#usda 1.0\n(\n    subLayers = [@mid.usda@ (offset = 5)]\n)\n")
nested = Usd.Stage.Open("top.usda")
print("nested:", nested.GetPrimAtPath("/Ball").GetAttribute("tx").GetTimeSamples())
combined = Sdf.LayerOffset(5) * Sdf.LayerOffset(10)
print("A * B :", combined.offset, combined.scale)

```

Expected output

```

stage samples: [20.0, 40.0]
tx at stage time 10: 0.0
tx at stage time 20: 0.0
tx at stage time 30: 50.0
tx at stage time 40: 100.0
offset, scale: 20.0 2.0
layer 10 -> stage 40.0
stage 30 -> layer 5.0
new offset samples: [100.0, 110.0]
nested: [15.0, 25.0]
A * B : 15.0 1.0

```

Before the first sample (stage time 10), USD holds the first sample's value, 0.0.

Real-world use case. A crowd shot reuses one walk-cycle layer for a background group, shifted so it starts at frame 1001. A sports broadcast digital twin plays a recorded motion layer at half speed for a replay (`scale = 2`). An animation authored at 24 time codes per second can be sublayered into a 48-per-second project, and USD rescales the timing automatically.

Common mistakes.

MISTAKE

Computing the inverse: "offset 20 means a sample at stage 20 came from layer 20". No: layer time = (stage time – offset) / scale. Stage 20 is layer 0.

MISTAKE

Expecting an offset to change a default value. Defaults have no time; only time samples (and other time-based data such as value clips) move.

MISTAKE

Forgetting to extend ``endTimeCode`` after shifting animation later. The samples are there, but playback stops before them.

Exam traps.**TRAP**

Order of operations: scale first, then offset. With `(offset = 20; scale = 2)`, layer time 10 maps to 40, not 60.

TRAP

Nested sublayer offsets add up (and scales multiply). A question with two levels of offsets wants the combined mapping.

TRAP

Different `timeCodesPerSecond` values between root and sublayer are not an error. USD scales the sublayer's times automatically.

Exam takeaways.**KEY**

- Stage time = layer time × scale + offset; inverse: (stage − offset) / scale. - USDA: `@file.usda@ (offset = N; scale = S)`; Python: `layer.subLayerOffsets[i] = Sdf.LayerOffset(N, S)` . - Nested offsets compose; different `timeCodesPerSecond` values add an automatic scale. - Offsets move time samples, never default values.

15.3 Edit targets (Usd.EditTarget, SetEditTarget)

What is it? The **edit target** is the place where the Usd API writes your edits. When you call `attr.Set(...)` or `stage.DefinePrim(...)`, the change goes into the edit target's layer. By default the edit target is the stage's **root layer**. You change it with `stage.SetEditTarget(...)` or temporarily with `Usd.EditContext`.

Why do we need it? The Usd API shows a composed view, but every edit must land in exactly one layer. If you are the lighter, your edits must go into `lighting.usda`, not into the shot's root layer or the layout layer. Edit targets let you work with the convenient composed API while choosing precisely which file receives the change. Without them, all edits pile into the root layer (Obj 1.1, Obj 1.5).

Beginner explanation. You are looking at the projector image made of many sheets, holding a marker. The edit target is the answer to "which sheet am I drawing on right now?". You see the combined picture, but your ink only goes onto the sheet you picked.

Where the analogy breaks: drawing on a lower sheet does not guarantee the new drawing is visible. If a stronger sheet already has an opinion for that field, your edit is stored but loses.

Technical explanation. - `stage.GetEditTarget()` returns a `Usd.EditTarget`. `edit_target.GetLayer()` is the receiving layer. - `stage.SetEditTarget(layer)` accepts an `Sdf.Layer` or a `Usd.EditTarget`. The layer must be in the stage's **local layer stack** (session layer, root layer, or one of their sublayers). Otherwise USD raises `Tf.ErrorException` ("...is not in the local LayerStack..."). A muted layer is not in the layer stack, so it cannot be a target. - with `Usd.EditContext(stage, target)`: sets the target for the block, then restores the previous one, even if an exception occurs. Prefer it to calling `SetEditTarget` twice. - **Time mapping.** `stage.GetEditTargetForLocalLayer(layer)` builds a target that includes the sublayer's layer offset, so `attr.Set(value, stage_time)` stores the sample at the matching **layer** time. A bare `Usd.EditTarget(layer)` has no offset mapping and stores the stage time unchanged (verified on USD 26.08). - **Variant edit targets.** `variant_set.GetVariantEditTarget()` returns a target that writes inside the currently selected variant; `variant_set.GetVariantEditContext()` is the with-block form (Chapter 18). Select the variant first with `SetVariantSelection`. - You cannot target a layer that arrives only through a reference or payload from this stage. To edit an asset,

open the asset itself (`Usd.Stage.Open("asset.usda")` or `Sdf.Layer.FindOrOpen`) and edit it there. - Reading is never affected by the edit target: `Get()` always returns the composed value.

Mental model.

```

    composed view (read)  <---  all layers
        |
    attr.Set(v) --+--> EDIT TARGET --> exactly one layer (or one variant in it)
        |
    must be in the local layer stack:
    [session] [root] [sublayer] [sublayer] ...    (not muted)

```

Simple example. The shot sublayers `anim.usda` with `(offset = 20; scale = 2)`. You set the edit target with `GetEditTargetForLocalLayer(anim)` and call `tx.Set(1.0, 30)`. The file `anim.usda` gets a sample at time 5, because $(30 - 20) / 2 = 5$. With `Usd.EditTarget(anim)` instead, the sample is stored at time 30.

USDA example. File: `chair.usda` (after authoring through a variant edit target)

```

#usda 1.0

def "Chair" (
    variants = {
        string color = "red"
    }
    prepend variantSets = "color"
)
{
    variantSet "color" = {
        "blue" {
        }
        "red" {
            custom string paint = "red!"
        }
    }
}

```

- `prepend variantSets = "color"` and `variants = {...}` — written by `AddVariantSet` and `SetVariantSelection`.
- `custom string paint = "red!"` sits **inside** `"red" { }` because the edit target was the variant, not the prim itself.

Python example.

```

from pxr import Sdf, Tf, Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("anim.usda", '#usda 1.0\ndef "Ball"\n{\n}\n')
write("shot.usda", ""'#usda 1.0
(
    subLayers = [@anim.usda@ (offset = 20; scale = 2)]
)
""")

stage = Usd.Stage.Open("shot.usda")
anim = Sdf.Layer.Find("anim.usda")
print("default target:", stage.GetEditTarget().GetLayer().GetDisplayName())

stage.SetEditTarget(stage.GetEditTargetForLocalLayer(anim))
tx = stage.GetPrimAtPath("/Ball").CreateAttribute("tx", Sdf.ValueTypeNames.Double)
tx.Set(1.0, 30)
stage.SetEditTarget(Usd.EditTarget(anim))
tx.Set(2.0, 30)
print("anim.usda samples:", anim.GetAttributeAtPath("/Ball.tx").GetInfo("timeSamples"))

```

```

stage.SetEditTarget(stage.GetRootLayer())
with Usd.EditContext(stage, stage.GetSessionLayer()):
    stage.GetPrimAtPath("/Ball").SetMetadata("comment", "try bigger")
print("restored target:", stage.GetEditTarget().GetLayer().GetDisplayName())
print("in session layer:", stage.GetSessionLayer().GetPrimAtPath("/Ball").comment)

try:
    stage.SetEditTarget(Sdf.Layer.CreateAnonymous("elsewhere"))
except Tf.ErrorException as err:
    print("error, not in local stack:", "not in the local LayerStack" in str(err))

chair = Usd.Stage.CreateInMemory()
prim = chair.DefinePrim("/Chair")
color = prim.GetVariantSets().AddVariantSet("color")
color.AddVariant("red")
color.AddVariant("blue")
color.SetVariantSelection("red")
with color.GetVariantEditContext():
    prim.CreateAttribute("paint", Sdf.ValueTypeNames.String).Set("red!")
spec = chair.GetRootLayer().GetObjectAtPath("/Chair{color=red}.paint")
print("variant spec:", spec.path, spec.default)

```

Expected output

```

default target: shot.usda
anim.usda samples: {5.0: 1.0, 30.0: 2.0}
restored target: shot.usda
in session layer: try bigger
error, not in local stack: True
variant spec: /Chair{color=red}.paint red!

```

The first `Set` went through the offset mapping (stored at layer time 5). The second used a bare edit target (stored at 30). The `Usd.EditContext` block restored the root layer as the target afterwards.

Real-world use case. A layout tool opens the shot, sets the edit target to `layout.usda`, and moves props; a lighting tool opens the same shot with the target on `lighting.usda`. Asset tools use variant edit targets to author each look variant (`red`, `blue`) inside one asset file. Review tools use the session layer as the edit target, so reviewers can try changes without touching saved files.

Common mistakes.

MISTAKE

Forgetting to set the edit target, so every edit lands in the root (shot) layer. Use ``with Usd.EditContext(stage, layer):`` around each department's edits.

MISTAKE

Authoring into a weaker layer and expecting the value to change on screen. The edit is stored, but a stronger opinion still wins. Check ``attr.GetPropertyStack()``.

MISTAKE

Using ``Usd.EditTarget(layer)`` for animation in an offset sublayer. Samples end up at the wrong time. Use ``stage.GetEditTargetForLocalLayer(layer)``.

MISTAKE

Authoring in a variant without selecting it first. The variant edit target writes into the ****selected**** variant; call ``SetVariantSelection`` first.

Exam traps.

TRAP

The default edit target is the ****root layer****, not the session layer and not the strongest sublayer.

TRAP

"Set the edit target to the referenced asset's layer" — not possible from the referencing stage; only local layer stack layers are valid targets.

TRAP

The edit target changes where writes go, never what reads return. A question claiming ``Get()`` reads only from the edit target is wrong.

Exam takeaways.**KEY**

- Edit target = where ``Usd`` API writes go. Default = root layer. - ``stage.SetEditTarget(layer)`` / ``with Usd.EditContext(stage, layer):`` — layer must be in the local layer stack. - ``GetEditTargetForLocalLayer`` maps time through sublayer offsets; bare ``Usd.EditTarget(layer)`` does not. - ``GetVariantEditContext()`` writes inside the selected variant. - Editing a weaker layer stores the opinion but may not change the composed value.

15.4 Layer muting (MuteLayer)

What is it? Muting a layer tells one stage to compose as if that layer did not exist. You call `stage.MuteLayer(layer_identifier)`. Nothing in any file changes, and you can bring it back with `stage.UnmuteLayer(...)`.

Why do we need it? Debugging: "Is the lighting layer causing this?" Mute it and look. Performance: mute a heavy layer you do not need for your current task. Review: compare a scene with and without one department's work. All of this without editing `subLayers` in a file that others share.

Beginner explanation. On the projector, muting is like lifting one sheet off for a moment. The sheet still exists on your desk, unchanged. Put it back and the picture returns.

Where the analogy breaks: lifting a sheet affects only your projector. Another stage that opens the same files is not affected, and the mute is not saved anywhere.

Technical explanation. - `stage.MuteLayer(identifier)`, `stage.UnmuteLayer(identifier)`, `stage.IsLayerMuted(identifier)`, `stage.GetMutedLayers()`, and `stage.MuteAndUnmuteLayers(mute_list, unmute_list)` (one recomposition for many changes). They take layer **identifiers** (strings), for example `layer.identifier`. - Muting works on any layer the stage uses: sublayers, and also layers reached through references or payloads. Muting a referenced asset's layer produces a composition warning ("...was muted for reference...") and the reference contributes nothing. - The **root layer cannot be muted** (USD raises an error: "Cannot mute cache's root layer"). - A muted layer is removed from `stage.GetLayerStack()`, but `root.subLayerPaths` still lists it: muting is stage state, not file data. - A muted layer cannot be the edit target (it is no longer in the local layer stack). - After muting, USD may release the muted layer. Keep the identifier string, not an old `Sdf.Layer` Python object, to unmute it later. - If the muted layer held the only `def` of a prim, the prim may remain as an undefined `over` from other layers (`prim.IsDefined()` is `False`) or disappear entirely. - Muting is per stage and is not saved. To make the change permanent, edit `subLayers` instead.

Mental model.

```
files (unchanged):  shot.usda  subLayers = [fx.usda, model.usda]

stage A (fx muted):  [shot] ..... [model]      radius = 1
stage B (normal):    [shot] [fx]   [model]      radius = 5

mute = per stage, in memory, reversible, never saved
```


Simple example. `fx.usda` sets `radius = 5`; `model.usda` defines the ball with `radius = 1`. With both, radius is 5. Mute `fx.usda`: radius is 1. Unmute: 5 again. Opening the shot on a second stage gives 5 even while the first stage has it muted.

USDA example. File: `fx.usda`

```
#usda 1.0

over "Ball"
{
    double radius = 5
}
```

- This is the layer we will mute. It only holds an `over`, so muting it removes its opinion and nothing else.
- Note that the file has no “muted” flag: muting can only be requested on a stage, never authored in USDA.

Python example.

```
from pxr import Sdf, Tf, Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("model.usda", '#usda 1.0\ndef Sphere "Ball"\n{\n    double radius = 1\n}\n')
write("fx.usda", '#usda 1.0\nover "Ball"\n{\n    double radius = 5\n}\n')
write("shot.usda", '#usda 1.0\n(\n    subLayers = [@fx.usda@, @model.usda@]\n)\n')

stage = Usd.Stage.Open("shot.usda")
radius = stage.GetPrimAtPath("/Ball").GetAttribute("radius")
fx_id = Sdf.Layer.Find("fx.usda").identifier
print("before mute:", radius.Get())

stage.MuteLayer(fx_id)
print("muted:", radius.Get(), stage.IsLayerMuted(fx_id))
print("muted layers:", [p.split("/")[-1] for p in stage.GetMutedLayers()])
print("layer stack:", [l.GetDisplayName() for l in stage.GetLayerStack(False)])
print("file still lists:", list(stage.GetRootLayer().subLayerPaths))

other = Usd.Stage.Open("shot.usda")
print("second stage:", other.GetPrimAtPath("/Ball").GetAttribute("radius").Get())

stage.UnmuteLayer(fx_id)
print("unmuted:", radius.Get())

try:
    stage.MuteLayer(stage.GetRootLayer().identifier)
except Tf.ErrorException as err:
    print("mute root fails:", "Cannot mute" in str(err))

stage.MuteLayer(Sdf.Layer.Find("model.usda").identifier)
ball = stage.GetPrimAtPath("/Ball")
print("model muted -> defined:", ball.IsDefined(), "type:", repr(ball.GetTypeName()))
```

Expected output

```
before mute: 5.0
muted: 1.0 True
muted layers: ['fx.usda']
layer stack: ['shot.usda', 'model.usda']
file still lists: ['fx.usda', 'model.usda']
second stage: 5.0
unmuted: 5.0
mute root fails: True
model muted -> defined: False type: ''
```

The second stage shares the same layers but not the mute. With `model.usda` muted, `/Ball` exists only as an `over` from `fx.usda`: it is not defined and has no type.

Real-world use case. A lighter’s scene renders slowly. They mute the `fx.usda` layer to check whether the effects are responsible, without editing the shared shot file. A supervisor in review mutes the latest animation layer to compare with the previous layout. In a factory digital twin, an engineer mutes a “proposed changes” layer to see the current factory floor.

Common mistakes.

MISTAKE

Expecting ``stage.Save()`` to remember the mute. Muting is not stored in any layer. Remove the sublayer from ``subLayers`` to make it permanent.

MISTAKE

Passing an ``Sdf.Layer`` object to ``MuteLayer``. It takes the identifier string: ``stage.MuteLayer(layer.identifier)``.

MISTAKE

Keeping an old ``Sdf.Layer`` Python handle for a muted layer and using it later. It may have expired; look the layer up again by identifier after unmuteing.

Exam traps.

TRAP

"Muting deletes the layer's opinions from disk" — false. Muting is in memory, per stage, reversible.

TRAP

You cannot mute the root layer. A question offering "mute the root layer to test without it" is offering an impossible action.

TRAP

Muting vs. deactivating: muting removes a whole `**layer**`; ``prim.SetActive(False)`` removes one `**prim**` subtree and is an authored opinion that is saved (Chapter 4).

Exam takeaways.

KEY

- ``MuteLayer``, ``UnmuteLayer``, ``IsLayerMuted``, ``GetMutedLayers``, ``MuteAndUnmuteLayers`` take identifiers. - Per stage, in memory, reversible, never saved; files are untouched. - The root layer cannot be muted; a muted layer cannot be the edit target. - Great for debugging "which layer causes this?" (Obj 1.8, Chapter 42).

15.5 Multi-user layering strategies

What is it? A **layering strategy** is a plan for which layers exist in a shared scene, who owns each one, and in what order they are stacked. The usual building block is a **department layer**: a sublayer owned by one team, edited only by that team through its edit target.

Why do we need it? USD has no built-in file locking or merging. If two artists edit and save the same layer, one overwrites the other. A layering strategy avoids this by giving each workstream its own file, and it makes strength intentional: the team that should win a conflict sits higher in the stack (Obj 1.5).

Beginner explanation. Think of a group project where every student writes on their own transparent sheet, and the teacher stacks the sheets in an agreed order. Nobody erases anyone else’s work, and the order says whose drawing wins where they overlap.

Where the analogy breaks: in real productions the “order” is a pipeline decision, and some teams also need to edit assets (through references), not only the shot. Layering is one tool; asset structure (Chapter 23) is the other.

Technical explanation. Common patterns (conventions, not rules enforced by USD):

Pattern	How it works	Why
Department sublayers	<code>shot.usda</code> sublayers <code>lighting</code> , <code>fx</code> , <code>anim</code> , <code>layout</code>	One writer per file; clear ownership
Downstream is stronger	Later pipeline steps are higher in the list	Lighting can adjust what animation left; not the other way
Shared base at the bottom	<code>sequence.usda</code> or <code>set.usda</code> sublayered last	Fixes reach every shot; shots can override
Thin root layer	Root holds only <code>subLayers</code> (and stage metadata)	Nobody needs to write the root
Session layer for personal tweaks	Reviewer or artist edits not meant to be saved	Never pollutes shared files
Per-user work layer (optional)	A temporary strongest sublayer per artist, merged later	Try changes without blocking others

Rules of thumb:

- Each tool sets its edit target to its department’s layer before writing (15.3).
- Assets (characters, props) come in through **references or payloads**, not sublayers, so they can be reused and placed at any path (Chapter 16, 17). Shot-wide changes to many prims belong in **sublayers**. This is the core of Obj 1.3.
- `stage.Save()` saves every dirty layer in the local layer stack except the session layer. A careful tool saves only its own layer with `layer.Save()`.
- `usdview`’s Layer Stack view, `stage.GetLayerStack()`, and `attr.GetPropertyStack()` show which department’s layer holds the winning opinion. For splitting a monolithic asset into workstreams, see Chapter 22 (Obj 1.11).

Mental model.

```
shot.usda (thin root: subLayers only)
|
+-- shot_lighting.usda   owner: lighting   STRONGEST (downstream)
+-- shot_anim.usda       owner: animation
+-- shot_layout.usda     owner: layout
+-- sequence.usda        owner: sequence   WEAKEST (shared base)
|
|-- assets arrive by references/payloads, not sublayers
```

Simple example. Layout places `/Car` at `x = 0` in `shot_layout.usda`. Animation authors `x = 5` in `shot_anim.usda`. Lighting adds a lamp in `shot_lighting.usda`. Each department saves only its own file. The composed car is at `x = 5`, because the animation layer is stronger than layout.

USDA example. File: `shot.usda`

```
#usda 1.0
(
    defaultPrim = "World"
    endTimeCode = 1100
    metersPerUnit = 0.01
    startTimeCode = 1001
    subLayers = [
```

```

        @shot_lighting.usda@,
        @shot_anim.usda@,
        @shot_layout.usda@,
        @sequence.usda@
    ]
    upAxis = "Y"
)

```

- A thin root: only stage metadata and the ordered department layers.
- `shot_lighting.usda` is first (strongest); `sequence.usda` is last (weakest, shared).
- Stage metadata such as `upAxis` and `metersPerUnit` is read from the root layer (Chapter 2), so it lives here, not in department layers.

Python example. Three departments author into their own layers through edit contexts; then we save and check what each file contains.

```

from pxr import Sdf, Usd, UsdGeom

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

depts = ["shot_lighting.usda", "shot_anim.usda", "shot_layout.usda"]
for name in depts:
    write(name, "#usda 1.0\n")
paths = ", ".join(f"@{name}@" for name in depts)
write("shot.usda", f"#usda 1.0\n\n    subLayers = [{paths}]\n\n")

stage = Usd.Stage.Open("shot.usda")
layer = {name: Sdf.Layer.Find(name) for name in depts}

with Usd.EditContext(stage, layer["shot_layout.usda"]):
    car = UsdGeom.Xform.Define(stage, "/World/Car")
    car.AddTranslateOp().Set((0.0, 0.0, 0.0))
with Usd.EditContext(stage, layer["shot_anim.usda"]):
    op = car.GetPrim().GetAttribute("xformOp:translate")
    op.Set((5.0, 0.0, 0.0))
with Usd.EditContext(stage, layer["shot_lighting.usda"]):
    stage.DefinePrim("/World/Lamp", "SphereLight")
with Usd.EditContext(stage, stage.GetSessionLayer()):
    stage.GetPrimAtPath("/World/Lamp").SetMetadata("comment", "reviewer note")

def dirty_layers():
    return [l.GetDisplayName() for l in stage.GetLayerStack() if l.dirty]

print("dirty before save:", len(dirty_layers()), "layers")
stage.Save()
print("dirty after save :", dirty_layers())
print("car translate:", op.Get())
print("opinions:", [s.layer.GetDisplayName() for s in op.GetPropertyStack()])
for name in depts:
    paths = []
    layer[name].Traverse("/", paths.append)
    print(name, "holds", sorted(str(p) for p in paths if p.IsPrimPath()))

```

Expected output

```

dirty before save: 4 layers
dirty after save : ['shot-session.usda']
car translate: (5, 0, 0)
opinions: ['shot_anim.usda', 'shot_layout.usda']
shot_lighting.usda holds ['/World', '/World/Lamp']
shot_anim.usda holds ['/World', '/World/Car']
shot_layout.usda holds ['/World', '/World/Car']

```

`stage.Save()` wrote the three department layers and left the session layer (with the reviewer note) unsaved. Each department file holds only the prims it touched; the overs for `/World` were created automatically so the edits have a parent path.

Real-world use case. Feature-animation studios run shots this way: layout, animation, character effects, effects, and lighting each publish one layer per shot, and the shot file stacks them in pipeline order. Game studios split a level into terrain, gameplay, audio, and lighting layers so designers and artists can work at the same time. In AEC, the structural, mechanical, and electrical teams each own a layer over a shared base model, and a coordination layer on top records agreed changes.

Common mistakes.

MISTAKE

Letting every tool write into the root shot layer. Merges become impossible. Keep the root thin and set the edit target per department.

MISTAKE

Sublayering a character asset into a shot. The character is then fixed at its authored paths and cannot be placed twice. Reference it instead.

MISTAKE

Putting an upstream department above a downstream one, so the lighter's tweaks are overridden by animation. Order the stack in pipeline order: downstream stronger.

MISTAKE

Calling ``stage.Save()`` from a department tool and accidentally saving another department's half-finished edits. Save only your own layer with ``layer.Save()``.

Exam traps.

TRAP

"USD locks a layer while someone edits it" — false. Avoiding conflicts is the job of the layering strategy (one writer per layer).

TRAP

A multi-user question that offers "use variants for each department" is offering the wrong tool. Variants are alternatives for one asset (Chapter 18); departments need separate layers.

TRAP

Sublayers vs. references: shot-wide work on shared paths → sublayers; reusable assets placed at new paths → references; heavy data you may defer → payloads.

Exam takeaways.

KEY

- One writer per layer: department sublayers under a thin shot root. - Downstream departments are stronger; shared bases are weakest. - Each tool sets its edit target to its own layer; reviewers use the session layer. - Assets come in by reference/payload; shot-wide edits go in sublayers (Obj 1.3, 1.5). - ``stage.Save()`` saves dirty local layers except the session layer.

Chapter lab(s)

- **Lab 05 — Build a sublayer stack and see who wins** (Chapters 3, 15; Obj 1.1). You build a three-layer stack with a nested sublayer, predict each composed value, then reorder and mute layers to check your predictions.

- **Lab 19 — Edit targets: author into layers and variants** (Chapter 15; Obj 1.1, 1.5). You write department edits through `Usd.EditContext`, author a look variant through a variant edit target, and use `GetEditTargetForLocalLayer` on an offset animation layer.

USDA reading exercises

Exercise 15-A. Consider these files.

File: shot.usda

```
#usda 1.0
(
  subLayers = [
    @fix.usda@,
    @base.usda@ (offset = 100)
  ]
)
```

File: base.usda

```
#usda 1.0
(
  subLayers = [@older.usda@]
)

def Sphere "Ball"
{
  double radius.timeSamples = {
    0: 1,
    10: 2,
  }
}
```

File: fix.usda

```
#usda 1.0

over "Ball"
{
  color3f[] primvars:displayColor = [(1, 0, 0)]
}
```

`older.usda` contains `over "Ball" { double radius = 7 }`.

1. List the local layer stack, strongest first (without the session layer).
2. At which stage times does `/Ball.radius` have time samples?
3. What does `/Ball.radius` return at stage time 105?

Exercise 15-B. In the same scene, the stage's edit target is the root layer and a script calls `stage.MuteLayer(<identifier of fix.usda>)`, then `stage.Save()`. Is `fix.usda` still listed in `shot.usda` when the file is opened again? What color does `/Ball` show in a new stage?

Chapter review

Summary

- Layer stack order: session → root → `subLayers[0]` (with its own sublayers, depth-first) → `subLayers[1]` → ...
- The first sublayer listed is the strongest; the root's own opinions beat all sublayers.
- `subLayers` is a plain list; edit `layer.subLayerPaths` to reorder. There is no `prepend subLayers`.
- Layer offsets: stage time = layer time × scale + offset; nested offsets compose; different `timeCodesPerSecond` values add an automatic scale.

- The edit target (default: root layer) decides where `Usd` API writes go; it must be in the local layer stack.
- `GetEditTargetForLocalLayer` maps time through sublayer offsets; a bare `Usd.EditTarget(layer)` does not.
- `Usd.EditContext` sets a target temporarily; `GetVariantEditContext()` writes inside the selected variant.
- Muting is per stage, in memory, reversible, never saved; the root layer cannot be muted.
- Multi-user strategy: one department per sublayer, downstream stronger, thin root, session layer for personal tweaks.
- Sublayers for shot-wide shared namespace; references for reusable assets; payloads for deferrable heavy data (Obj 1.3).

If you see... → think...

If you see...	Think...
A nested <code>subLayers</code> inside a sublayer	Depth-first: its children rank right after it
<code>@file@</code> (offset = N; scale = S)	$\text{Stage} = \text{layer} \times S + N$
"Edits land in the wrong file"	Edit target; use <code>Usd.EditContext</code>
"Samples at the wrong time after Set"	Use <code>GetEditTargetForLocalLayer</code> for offset sublayers
"Test the scene without one layer"	<code>stage.MuteLayer(identifier)</code> (not the root)
"Many artists, one shot"	Department sublayers, one writer each, downstream stronger
"Try a change without saving"	Session layer as edit target
"Reuse an asset at several paths"	Reference, not sublayer

Review questions

15-R1 · Obj 1.3 · Single choice A team wants to place the same tree asset 40 times at different paths in a forest. Which arc is most appropriate? A. Sublayers B. References C. Layer muting D. Specializes from the root layer

15-R2 · Obj 1.1 · USDA reading `root.usda` has `subLayers = [@p.usda@, @q.usda@]`, and `q.usda` has `subLayers = [@r.usda@]`. `p.usda` sets nothing on `/Lamp.inputs:intensity`; `q.usda` sets 20; `r.usda` sets 50; `root.usda` sets nothing. What is the composed intensity? A. 50 B. 20 C. 70 D. The schema fallback

15-R3 · Single choice A sublayer is listed as `@anim.usda@` (offset = 24; scale = 0.5). A sample in `anim.usda` is at time 48. At what stage time does it appear? A. 24 B. 48 C. 72 D. 120

15-R4 · Obj 1.1 · Select two. A script must write an override into `shot_fx.usda`, a sublayer of the open stage. Which **two** approaches work? A. `stage.SetEditTarget(fx_layer)` and then `attr.Set(v)` B. with `Usd.EditContext(stage, fx_layer): attr.Set(v)` C. `stage.MuteLayer(fx_layer.identifier)` and then `attr.Set(v)` D. `attr.Set(v)` with no other change

15-R5 · Single choice What is the default edit target of a newly opened stage? A. The session layer B. The strongest sublayer C. The root layer D. None; you must set one

15-R6 · Single choice `anim.usda` is sublayered with `offset = 10`. You call `stage.SetEditTarget(Usd.EditTarget(anim))` and then `attr.Set(1.0, 30)`. At what time is the sample stored in `anim.usda`? A. 20 B. 30 C. 40 D. 3

15-R7 · Select two. Which **two** statements about layer muting are correct? A. A muted layer can still be the edit target. B. Muting is not saved by `stage.Save()`. C. The root layer cannot be muted. D. Muting removes the layer from the file's `subLayers` list.

15-R8 · Obj 1.5 · Single choice In a shot, the lighting department's tweaks keep being hidden by animation's opinions. The sublayer list is `[layout, anim, lighting]`. What is the best structural fix? A. Reorder to `[lighting, anim, layout]` so downstream departments are stronger. B. Ask lighting to author into the session layer. C. Mute the animation layer permanently. D. Move lighting into a payload.

15-R9 · Python reading What does this print?

```
off = Sdf.LayerOffset(offset=10, scale=2)
print(off * 5.0, off.GetInverse() * 30.0)
```

A. 20.0 10.0 B. 15.0 40.0 C. 20.0 20.0 D. 30.0 10.0

15-R10 · Obj 1.5 · Single choice A reviewer wants to try a brighter light during a review without saving anything to shared files. Where should the edit go? A. The root layer B. The lighting department layer C. The session layer D. The asset file of the light

15-R11 · Obj 1.3 · Select two. Which **two** are true of sublayers but not of references? A. They share the root's prim namespace (same paths). B. They are the strongest arc in LIVERPS (local). C. They can target one specific prim in the other file. D. They require `defaultPrim` in the target file.

15-R12 · Single choice A script runs `stage.SetEditTarget(asset_layer)` where `asset_layer` is reached only through a reference on `/World/Chair`. What happens? A. Edits are written into the asset file. B. A `Tf.ErrorException` is raised because the layer is not in the local layer stack. C. The reference is converted to a sublayer. D. Edits are silently discarded.

The code in 15-R9 is a fragment (it omits the import), so it is not run automatically.

Answers

Exercise 15-A — Answer. 1. `shot.usda`, `fix.usda`, `base.usda`, `older.usda` (`older.usda` is `base.usda`'s sublayer, so it comes right after it). 2. The offset applies to the `base.usda` entry, so its samples at 0 and 10 appear at stage times **100 and 110**. 3. **1.5**: stage time 105 is layer time 5, halfway between the samples 1 and 2. The default `radius = 7` in `older.usda` is weaker, and `base.usda` has time samples, so they win. (Time samples vs. defaults across layers is covered in Chapter 21.) Verified by composing the files on USD 26.08.

Exercise 15-B — Answer. Yes, `fix.usda` is still listed: muting never edits files, and `stage.Save()` only saves dirty layers. A new stage opens without the mute, so `/Ball` shows the red `displayColor` from `fix.usda` again.

- **15-R1: B.** References graft one asset to many paths. Sublayers share one namespace and cannot place copies. Review: §15.1, Chapter 16.
- **15-R2: B.** Order is root, p, q, r. p and root are silent, so q (20) wins over r (50). Values are not added. Review: §15.1.
- **15-R3: B.** $48 \times 0.5 + 24 = 48$. Review: §15.2.
- **15-R4: A and B.** Both set the edit target to the fx layer. A muted layer cannot be targeted; with no change, the edit goes into the root layer. Review: §15.3.
- **15-R5: C.** The root layer. Review: §15.3.
- **15-R6: B.** A bare `Usd.EditTarget(layer)` has no time mapping, so the sample is stored at 30. `GetEditTargetForLocalLayer` would have stored it at 20. Review: §15.3.
- **15-R7: B and C.** Muting is stage state only; muted layers leave the layer stack and cannot be targets. Review: §15.4.

- **15-R8: A.** The first sublayer is strongest, so downstream lighting should be listed first. The session layer is not saved; permanent muting and payloads do not fix ordering. Review: §15.5.
- **15-R9: A.** $5 \times 2 + 10 = 20$; $(30 - 10) / 2 = 10$. Review: §15.2.
- **15-R10: C.** The session layer is stronger than the root layer and is not saved by `stage.Save()`. Review: §15.5.
- **15-R11: A and B.** References can target one prim, and `defaultPrim` matters for references without a prim path. Review: §15.1.
- **15-R12: B.** Only layers in the local layer stack are valid edit targets. Review: §15.3.

CHAPTER 16

Chapter 16 — References

Exam domain: Composition (23%) · **Objectives:** 1.3, 1.4 · **Study day:** 5 · **Est. time:** 120 min
Prerequisites: Ch 2 (stage, `defaultPrim`), Ch 3 (layers), Ch 4 (prim paths), Ch 10 (time samples), Ch 14 (composition, list editing), Ch 15 (sublayers, layer offsets on sublayers)

Learning goals

- Author an external reference in USDA and in Python, and predict which prims and values it produces.
- Explain how `defaultPrim` lets you reference a file without naming a prim path, and what happens when it is missing.
- Use internal references to reuse a prim from the same layer stack.
- Place the same animation at different times and speeds with `Sdf.LayerOffset(offset, scale)`, and calculate the resulting times.
- Diagnose why a reference “does nothing”, using `Usd.Stage.GetCompositionErrors()`.
- Compare references with sublayers and choose the right one for a task.

Key terms

Term	One-line definition
Reference	A composition arc that brings a prim (and everything below it) from a layer into a prim on your stage.
Referencing prim	The prim on your stage that holds the reference (the destination).
Target prim	The prim inside the referenced layer that is brought in (the source).
External reference	A reference whose target lives in another file, named by an asset path such as <code>@chair.usda@</code> .
Internal reference	A reference whose target lives in the same layer stack; it has a prim path but no asset path.
<code>defaultPrim</code>	Layer metadata naming the prim that is used when a reference gives no prim path.
Layer offset	A pair (offset, scale) that shifts and stretches the time samples coming through an arc.
Composition error	A problem found while composing (missing file, missing prim, cycle). The stage still opens.
Namespace mapping	The renaming of paths from the target prim’s location to the referencing prim’s location.
Anchoring	Resolving a relative asset path from the folder of the layer that wrote it.

16.1 External references

What is it? An **external reference** tells a prim on your stage: “get your contents from that prim in that other file.” The prim in your stage then shows the referenced prim’s type, properties, and children, as if they were written there.

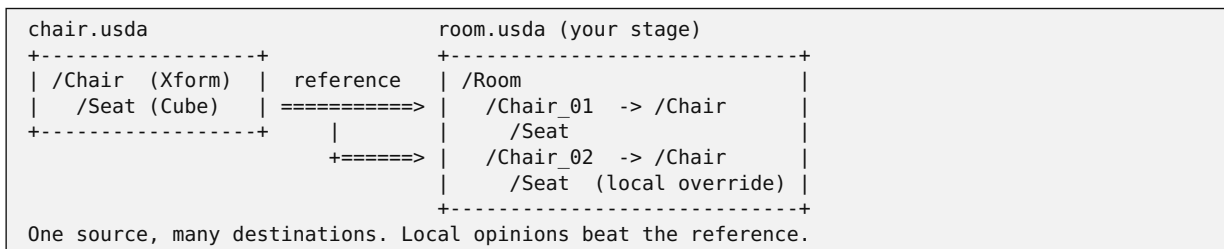
Why do we need it? Real scenes reuse assets. A city has 500 identical street lamps; a film set has 40 copies of one chair. Without references you would copy the chair’s data 40 times. A fix to the chair would then need 40 edits. With references, the chair lives in one file, and every copy follows it.

Beginner explanation. Think of **linking a reusable part from a catalog**. Your room design does not contain a drawing of the chair; it says “chair, catalog item 17, here.” When the catalog item is improved, every room that links it gets the improvement. You can still write a local note on one copy (“this one is red”), and your note wins over the catalog.

Where the analogy breaks: a catalog link usually copies a whole product. A USD reference brings exactly **one prim and its subtree**, and it renames all of its paths to fit your scene.

Technical explanation. - A reference is stored as prim metadata named `references`. Each entry is an `Sdf.Reference(assetPath, primPath, layerOffset, customData)`. - In Python you edit it through `Usd.References`, returned by `prim.GetReferences()`. `AddReference(assetPath, primPath)` adds one entry. - `references` is a **list-edited** field (Chapter 14). `AddReference` writes `prepend` by default, so new references are added at the front of the list of prepended items. Earlier entries in the final list are **stronger**. - The target prim’s whole **layer stack** comes along: if `chair.usda` has sublayers, their opinions are included too. - **Namespace mapping:** the target `/Chair` is mapped to the referencing prim, say `/Room/Chair_01`. So `/Chair/Seat` appears as `/Room/Chair_01/Seat`. Relationship targets inside `/Chair` are remapped the same way. - **Strength:** references are the **R** in LIVERPS (Chapter 14). Local opinions on the referencing prim, from any layer of your stage’s layer stack, are stronger than anything brought in by the reference. That is why overrides work. - Layer metadata of the referenced file (`upAxis`, `metersPerUnit`, `defaultPrim`) does not become stage metadata. Only the stage’s root layer and session layer set stage metadata. - Relative asset paths are **anchored** to the layer that authored them, not to your current working folder.

Mental model.



Simple example.

| Step | What you write | What you see on the stage |

|-----|-----|-----| | 1 | chair.usda defines `/Chair/Seat` with `size = 1` |

— | | 2 | room.usda: `/Room/Chair_01` references chair.usda `/Room/Chair_01/Seat`, `size 1` |

| 3 | room.usda: over on `/Room/Chair_02/Seat`, `size 2` | Chair_02’s seat is `size 2`; Chair_01’s stays `1` |

USDA example. File: chair.usda

```
#usda 1.0
(
    defaultPrim = "Chair"
)

def Xform "Chair"
{
    def Cube "Seat"
    {
        double size = 1
    }
}
```

File: room.usda

```
#usda 1.0

def "Room"
```

```

{
    def "Chair_01" (
        prepend references = @./chair.usda@</Chair>
    )
    {
    }

    def "Chair_02" (
        prepend references = @./chair.usda@</Chair>
    )
    {
        over "Seat"
        {
            double size = 2
        }
    }
}

```

Line-by-line notes:

- `defaultPrim = "Chair"` is layer metadata; Section 16.2 explains why it matters.
- `prepend references = @./chair.usda@</Chair>`: the asset path is between `@...@`, the target prim path between `<...>`.
- `Chair_01` and `Chair_02` have no type of their own. They become `Xform` because the referenced `/Chair` is an `Xform`.
- `over "Seat"` adds an opinion without defining a new prim; it is stronger than the reference, so `size` becomes 2 for `Chair_02` only.

Python example.

```

from pxr import Usd

CHAIR = """#usda 1.0
(
    defaultPrim = "Chair"
)
def Xform "Chair"
{
    def Cube "Seat"
    {
        double size = 1
    }
}
"""

with open("chair.usda", "w") as f:
    f.write(CHAIR)

stage = Usd.Stage.CreateNew("room.usda")
for name in ["Chair_01", "Chair_02"]:
    prim = stage.DefinePrim(f"/Room/{name}")
    prim.GetReferences().AddReference("./chair.usda", "/Chair")

# A local opinion on one copy only.
seat2 = stage.GetPrimAtPath("/Room/Chair_02/Seat")
seat2.GetAttribute("size").Set(2.0)

for prim in stage.Traverse():
    print(prim.GetPath(), prim.GetTypeName())
for name in ["Chair_01", "Chair_02"]:
    seat = stage.GetPrimAtPath(f"/Room/{name}/Seat")
    print(name, "seat size =", seat.GetAttribute("size").Get())
stage.Save()
print(stage.GetRootLayer().ExportToString())

```

Expected output

```

/Room
/Room/Chair_01 Xform
/Room/Chair_01/Seat Cube
/Room/Chair_02 Xform
/Room/Chair_02/Seat Cube

```

```

Chair_01 seat size = 1.0
Chair_02 seat size = 2.0
#usda 1.0

def "Room"
{
    def "Chair_01" (
        prepend references = @./chair.usda@</Chair>
    )
    {
    }

    def "Chair_02" (
        prepend references = @./chair.usda@</Chair>
    )
    {
        over "Seat"
        {
            double size = 2
        }
    }
}

```

Notice that `Set(2.0)` on a referenced attribute did not touch `chair.usda`. USD wrote an `over` into the current edit target, the root layer (Chapter 15).

Real-world use case. In film set dressing, a “set” file references hundreds of prop assets (chairs, cups, books) by asset path. In a factory digital twin, each robot cell references the same robot asset; the robot vendor updates one file, and every cell follows. In AEC, a building model references a window family many times, and local overrides change only the frame color of some.

Common mistakes.

MISTAKE

Editing a referenced prim and expecting the source file to change. Edits go to the current edit target (usually your root layer) as ``over`` opinions. To change the asset itself, open ``chair.usda`` and edit it there.

MISTAKE

Writing a prim path without a leading ``/``, such as ``AddReference("chair.usda", "Chair")``. This raises a ``Tf.ErrorException`` ("must be either empty or an absolute prim path"). Use ``"/Chair"``.

MISTAKE

Assuming the asset path is relative to the folder you run Python from. It is anchored to the folder of the layer that contains the reference.

Exam traps.

TRAP

"A reference copies the asset into the file." False. The reference stores only the link (``@chair.usda@``). The data is composed at load time and stays in the source file.

TRAP

"The referenced file's ``upAxis`` applies to the stage." False. Stage metadata comes from the root (and session) layer only. Unit or axis mismatches must be fixed by the pipeline (Chapter 28).

TRAP

Two references on one prim: the one listed **first** is stronger, not the last one written.

Exam takeaways.

KEY

- A reference = asset path + optional prim path + optional layer offset, stored as list-edited `references` metadata. - Only the target prim and its descendants come in, renamed to the referencing prim's path. - Local opinions beat referenced opinions; that is how per-copy overrides work. - `prim.GetReferences().AddReference(...)` writes `prepend references` into the edit target.

16.2 defaultPrim and references without prim paths

What is it? A reference may leave out the prim path: `@chair.usda@`. USD then uses the referenced layer's **defaultPrim** metadata to decide which prim to bring in.

Why do we need it? The person who uses an asset should not need to know how it is organised inside. With **defaultPrim**, the asset author says “this is my entry point”, and every user writes only the file name. If the author later renames the root prim, they update **defaultPrim**, and no referencing file breaks.

Beginner explanation. A catalog item can contain many sheets: the product, a spare-parts list, notes. The catalog marks one sheet as “**the product**”. If you order item 17 without naming a sheet, you get the marked sheet.

Where the analogy breaks: if no sheet is marked, USD does not guess. You get nothing, plus an error.

Technical explanation. - **defaultPrim** is layer metadata: `defaultPrim = "Chair"` in the layer header. - Python: `stage.SetDefaultPrim(prim)`, `stage.GetDefaultPrim()`, `stage.HasDefaultPrim()`, or on a layer `layer.defaultPrim = "Chair"`. - A reference with an empty prim path (`Sdf.Path()` or simply omitted) targets the **defaultPrim** of the referenced layer. - If the layer has **no** **defaultPrim**, or names a prim that does not exist, composition reports a **Pcp.ErrorUnresolvedPrimPath** error. The referencing prim remains, but receives nothing from the reference. - `AddReference` still returns `True`: it only *authored* the reference. The failure appears when the stage composes. - **defaultPrim** is also used by payloads (Chapter 17) and by tools that “open the asset’s main prim”.

VERSION

Verified on USD 26.08: `defaultPrim` may also hold an absolute path such as `"/Asset/Geo"`, and `Sdf.Layer.GetDefaultPrimAsPath()` returns it as an `Sdf.Path`. Most documentation, asset guidelines, and exam material assume the classic form: the ****name of a root prim****. Use that form.

Mental model.

```
reference has a prim path?
|
yes +----> use that path
|
no  +----> does the layer have defaultPrim?
      |
      yes +----> use /<defaultPrim>
      |
      no  +----> ErrorUnresolvedPrimPath; reference contributes nothing
```

Simple example. | Referenced file | Reference written | Result |

Referenced file	Reference written	Result
<code>defaultPrim = "Lamp", defines /Lamp</code>	<code>@lamp.usda@</code>	<code>/Lamp</code> comes in
<code>no defaultPrim, defines /Lamp</code>	<code>@lamp.usda@</code>	nothing comes in, composition error
<code>no defaultPrim, defines /Lamp</code>	<code>@lamp.usda@</Lamp></code>	<code>/Lamp</code> comes in

USDA example. File: *lamp.usda*

```
#usda 1.0
(
```

```

    defaultPrim = "Lamp"
)

def Xform "Lamp"
{
    def Cylinder "Pole"
    {
    }
}

def Scope "Notes"
{
}

```

File: street.usda

```

#usda 1.0

def "Street"
{
    def "Lamp_01" (
        prepend references = @./lamp.usda@
    )
    {
    }
}

```

Notes: @./lamp.usda@ has no <...> part, so /Lamp (the defaultPrim) is used. /Notes is a sibling of /Lamp and is **not** brought in.

Python example.

```

from pxr import Sdf, Usd

WITH_DEFAULT = """#usda 1.0
(
    defaultPrim = "Lamp"
)
def Xform "Lamp"
{
    def Cylinder "Pole" {}
}
"""
NO_DEFAULT = """#usda 1.0
def Xform "Lamp"
{
    def Cylinder "Pole" {}
}
"""

for name, text in [("good.usda", WITH_DEFAULT), ("nodefault.usda", NO_DEFAULT)]:
    with open(name, "w") as f:
        f.write(text)

stage = Usd.Stage.CreateInMemory()
a = stage.DefinePrim("/Street/Lamp_A")
print("authored:", a.GetReferences().AddReference("good.usda"))
b = stage.DefinePrim("/Street/Lamp_B")
print("authored:", b.GetReferences().AddReference("nodefault.usda"))

for prim in (a, b):
    kids = [c.GetName() for c in prim.GetChildren()]
    print(prim.GetName(), repr(prim.GetTypeName()), kids)
for err in stage.GetCompositionErrors():
    print("error at", err.rootSite.path, "->", err.errorType)

# Fix the asset: give it a defaultPrim. The stage recomposes automatically.
layer = Sdf.Layer.FindOrOpen("nodefault.usda")
layer.defaultPrim = "Lamp"
print("after fix:", [c.GetName() for c in b.GetChildren()],
      "errors:", len(stage.GetCompositionErrors()))

```

Expected output

```

authored: True
authored: True
Lamp_A 'Xform' ['Pole']
Lamp_B '' []
error at /Street/Lamp_B -> Pcp.ErrorType_UnresolvedPrimPath
after fix: ['Pole'] errors: 0

```

USD also prints a warning on the error stream (stderr), for example `Unresolved reference prim path @nodefault.usda@<defaultPrim>`. The expected output above shows only normal output.

Real-world use case. Asset structure guidelines (Chapter 23) require every published asset file to set `defaultPrim` to its root prim. Pipeline validators check this before publishing, because set dressers and layout tools always reference assets by file name only.

Common mistakes.

MISTAKE

Building an asset in Python and forgetting ``stage.SetDefaultPrim(root)`` before ``Save()``. Every reference without a prim path to that file then fails. Fix: always set it when you create the root prim.

MISTAKE

Setting ``defaultPrim`` to a name that does not exist (a typo, or a prim renamed later). The error is the same ``UnresolvedPrimPath``, and the warning names the missing path, e.g. ``@bad.usda@``.

Exam traps.

TRAP

"``AddReference`` returns ``False`` when the file has no ``defaultPrim``." False. It returns ``True``; authoring succeeded. The problem is a composition error, visible through ``stage.GetCompositionErrors()`` and a warning.

TRAP

"Without ``defaultPrim``, USD uses the first root prim." False. USD never guesses.

Exam takeaways.

KEY

- A reference without `` uses the referenced layer's ``defaultPrim``. - Missing or wrong ``defaultPrim`` → ``Pcp.ErrorType_UnresolvedPrimPath``; the prim stays empty. - Authoring still succeeds (``AddReference`` returns ``True``); check ``stage.GetCompositionErrors()``. - Always set ``defaultPrim`` on published assets.

16.3 Internal references

What is it? An **internal reference** points to a prim in the **same layer stack** instead of another file. It has a prim path but no asset path: `references = </Templates/Crate>`.

Why do we need it? Sometimes the thing you want to reuse lives in the same file: a template crate, a light rig, or a set of default attributes. Internal references let many prims share it without creating a separate asset file.

Beginner explanation. It is like a catalog link to **a page in the same binder**: “build this exactly like the crate on page 3.” Edit page 3, and every crate built “like page 3” changes.

Where the analogy breaks: the page-3 crate is also a real prim in the scene. If you do not want it to show up, make it a `class prim` (abstract, Chapter 4) or keep it under a scope you deactivate.

Technical explanation. - Python: `prim.GetReferences().AddInternalReference("/Templates/Crate")`, optionally with an `Sdf.LayerOffset`. - USDA: `prepend references = </Templates/Crate>` (no `@...@` part). `Sdf.Reference.IsInternal()` returns `True` for such entries. - The target is composed from the **whole local layer stack** (root layer plus all sublayers), not only from the layer where the reference is written. - Any prim path may be the target, not only root prims. - It is still a reference arc: **R** in LIVERPS. Local opinions on the referencing prim win. - A prim cannot reference itself or one of its own ancestors. That creates a cycle: `Pcp.ErrorType_ArcCycle`. - Compared with `inherits` (Chapter 19): both reuse a template, but `inherits` are stronger (**I** in LIVERPS) and keep working when the whole asset is referenced elsewhere. Use internal references for “start from this”, `inherits` for “always follow this class”.

Mental model.

```
one layer stack (root.usda + its sublayers)
+-----+
| class "_Crate" (template, abstract) |
|      ^           ^                 |
|      | </_Crate>   | </_Crate>      |
| def "Crate_A"    def "Crate_B"     |
+-----+
Same building, no asset path, still the R arc.
```

Simple example. | Prim | Authored | Composed size | |-----|-----|-----| | class `"_Crate"` | size = 1 | not traversed (abstract) | | `Crate_A` | references = `</_Crate>` | 1 | | `Crate_B` | references = `</_Crate>`, size = 3 | 3 (local wins) |

USDA example.

```
#usda 1.0

class Cube "_Crate"
{
    double size = 1
}

def "Warehouse"
{
    def "Crate_A" (
        prepend references = </_Crate>
    )
    {
    }

    def "Crate_B" (
        prepend references = </_Crate>
    )
    {
        double size = 3
    }
}
```

Notes: `class Cube "_Crate"` is an abstract template; default traversal skips it. `</_Crate>` has no asset path, so it is internal. `Crate_B` overrides `size` locally.

Python example.

```
from pxr import Sdf, Usd

stage = Usd.Stage.CreateInMemory()
tpl = stage.CreateClassPrim("/_Crate")
tpl.SetTypeName("Cube")
tpl.CreateAttribute("size", Sdf.ValueTypeNames.Double).Set(1.0)

for name in ["Crate_A", "Crate_B"]:
```

```

crate = stage.DefinePrim(f"/Warehouse/{name}")
crate.GetReferences().AddInternalReference("/_Crate")

b = stage.GetPrimAtPath("/Warehouse/Crate_B")
b.GetAttribute("size").Set(3.0)

def report(label):
    sizes = [stage.GetPrimAtPath(f"/Warehouse/{n}").GetAttribute("size").Get()
             for n in ["Crate_A", "Crate_B"]]
    print(label, sizes)

print([str(p.GetPath()) for p in stage.Traverse()])
report("before template edit:")
tpl.GetAttribute("size").Set(2.0)      # edit the template once
report("after template edit: ")

ref = b.GetMetadata("references").prependItems[0]
print("internal?", ref.IsInternal(), "asset path:", repr(ref.assetPath))

```

Expected output

```

['/Warehouse', '/Warehouse/Crate_A', '/Warehouse/Crate_B']
before template edit: [1.0, 3.0]
after template edit:  [2.0, 3.0]
internal? True asset path: ''

```

Crate_A followed the template edit. Crate_B kept its local value 3.0, because local beats reference.

Real-world use case. A game level file contains one `class` “pickup” template with a collision shape and score attribute; 200 pickups internally reference it. A lighting file keeps a template light rig and internally references it for each shot camera.

Common mistakes.

MISTAKE

Using a normal ``def`` prim as the template. It renders as an extra object at its own location. Use a ``class`` prim (abstract) for templates you do not want to see.

MISTAKE

Referencing an ancestor, e.g. ``/A/B`` internally referencing ``/A``. This is a cycle (``Pcp.ErrorType_ArcCycle``) and is ignored with a warning.

Exam traps.

TRAP

"Internal references only see opinions from the current layer." False. They see the whole local layer stack at the target path.

TRAP

"An internal reference is an inherit." No. It is a reference (R, weaker than I). The USDA keyword ``references`` versus ``inherits`` tells them apart.

Exam takeaways.

KEY

- Internal reference = prim path only (`` ``), ``AddInternalReference(path)``. - It composes the target from the whole local layer stack. - Still the R arc: local opinions win; weaker than inherits. - Do not reference your own ancestor (cycle error).

16.4 Reference layer offsets: reusing animation at different times

What is it? A **layer offset** on a reference shifts and/or stretches the time samples that come through it. It has two numbers: **offset** (shift, in time codes) and **scale** (speed factor). In Python it is `Sdf.LayerOffset(offset, scale)`.

Why do we need it? Objective 1.4 asks you to build a scene where several elements use the **same animation at different time offsets**: a crowd where people start walking at different frames, or three birds flapping out of sync. Without layer offsets you would need a separate copy of the animation for every start frame.

Beginner explanation. Think of a music track on a timeline in a video editor. You drop the same clip three times: one starts at second 0, one at second 10, and one is slowed to half speed. It is still the same clip file.

Where the analogy breaks: the offset only changes **time samples**. A default (non-animated) value is not affected.

Technical explanation. - Authoring: `AddReference(assetPath, primPath, Sdf.LayerOffset(offset, scale))`, or `AddReference(assetPath, Sdf.LayerOffset(...))` when you use `defaultPrim`. `AddInternalReference(path, Sdf.LayerOffset(...))` works too. Payloads accept layer offsets the same way. - USDA:

`@anim.usda@ (offset = 10; scale = 2)` after the reference. - **The formula** (maps a time in the referenced layer to a time on the stage):

```
stageTime = offset + scale * layerTime
layerTime = (stageTime - offset) / scale
```

- `Sdf.LayerOffset` defaults to `(0, 1)`, the identity. You can apply it as `lo * layerTime` and invert it with `lo.GetInverse()`.
- `scale` must be positive. A zero or negative scale gives `Pcp.ErrorType_InvalidReferenceOffset`, and USD uses no offset instead.
- **Nesting**: offsets multiply through chains of arcs. A reference with offset 10 to a file that references with offset 5 places layer time 0 at stage time 15.
- **Automatic scaling**: if the referenced layer's `timeCodesPerSecond` differs from the stage's, USD adds a scale. Verified: a 48-tcps layer referenced into a 24-tcps stage maps samples 0 and 48 to 0 and 24.
- Before the first sample and after the last, values are **held** (Chapter 10), so a shifted clip “waits” at its first pose.
- Sublayers take layer offsets too (Chapter 15). The math is the same.

Mental model.

```
layer time:  0 ----- 10 ----- 20          (anim.usda: up and down)
Ball_A (0,1): 0 ----- 10 ----- 20
Ball_B (10,1):      10 ----- 20 ----- 30
Ball_C (0,2): 0 ----- 20 ----- 40

stageTime = offset + scale * layerTime
```

Simple example. The animation peaks at layer time 10. Where is the peak on the stage?

Layer offset	Calculation	Peak at stage time
(0, 1)	$0 + 1 \times 10$	10
(10, 1)	$10 + 1 \times 10$	20
(0, 2)	$0 + 2 \times 10$	20
(5, 2)	$5 + 2 \times 10$	25

USDA example. File: *bounce.usda*

```
#usda 1.0
(
  defaultPrim = "Ball"
  startTimeCode = 0
  endTimeCode = 20
)

def Xform "Ball"
```

```
{
    double3 xformOp:translate.timeSamples = {
        0: (0, 0, 0),
        10: (0, 5, 0),
        20: (0, 0, 0),
    }
    uniform token[] xformOpOrder = ["xformOp:translate"]
}
```

File: shot.usda

```
#usda 1.0
(
    startTimeCode = 0
    endTimeCode = 40
)

def "Ball_A" (
    prepend references = @./bounce.usda@
)
{
}

def "Ball_B" (
    prepend references = @./bounce.usda@ (offset = 10)
)
{
}

def "Ball_C" (
    prepend references = @./bounce.usda@ (scale = 2)
)
{
}
```

Notes: (offset = 10) starts Ball_B's bounce 10 frames later. (scale = 2) makes Ball_C's bounce last twice as long. When both are present, they are separated by ;, e.g. (offset = 5; scale = 2).

Python example.

```
from pxr import Sdf, Usd

BOUNCE = """#usda 1.0
(
    defaultPrim = "Ball"
)
def Xform "Ball"
{
    double3 xformOp:translate.timeSamples = {
        0: (0, 0, 0),
        10: (0, 5, 0),
        20: (0, 0, 0),
    }
    uniform token[] xformOpOrder = ["xformOp:translate"]
}
"""

with open("bounce.usda", "w") as f:
    f.write(BOUNCE)

stage = Usd.Stage.CreateInMemory()
offsets = {"Ball_A": Sdf.LayerOffset(),           # identity (0, 1)
           "Ball_B": Sdf.LayerOffset(10),         # start 10 frames later
           "Ball_C": Sdf.LayerOffset(0, 2)}       # half speed
for name, lo in offsets.items():
    stage.DefinePrim("/") + name).GetReferences().AddReference("bounce.usda", lo)

frames = [0, 5, 10, 15, 20, 30, 40]
print("frame ", frames)
for name in offsets:
    attr = stage.GetPrimAtPath("/") + name).GetAttribute("xformOp:translate")
    heights = [attr.Get(t)[1] for t in frames]
    print(name, heights, "samples at", attr.GetTimeSamples())
```

```

lo = Sdf.LayerOffset(5, 2)
print("layer t=10 -> stage t =", lo * 10)
print("stage t=25 -> layer t =", lo.GetInverse() * 25)
print(stage.GetRootLayer().ExportToString().split("\n", 1)[1].strip())

```

Expected output

```

frame [0, 5, 10, 15, 20, 30, 40]
Ball_A [0.0, 2.5, 5.0, 2.5, 0.0, 0.0, 0.0] samples at [0.0, 10.0, 20.0]
Ball_B [0.0, 0.0, 0.0, 2.5, 5.0, 0.0, 0.0] samples at [10.0, 20.0, 30.0]
Ball_C [0.0, 1.25, 2.5, 3.75, 5.0, 2.5, 0.0] samples at [0.0, 20.0, 40.0]
layer t=10 -> stage t = 25.0
stage t=25 -> layer t = 10.0
def "Ball_A" (
    prepend references = @bounce.usda@
)
{
}

def "Ball_B" (
    prepend references = @bounce.usda@ (offset = 10)
)
{
}

def "Ball_C" (
    prepend references = @bounce.usda@ (scale = 2)
)
{
}

```

Check one value by hand: Ball_C at stage frame 15 → layer time $(15 - 0) / 2 = 7.5$ → height $0.75 \times 5 = 3.75$. Ball_B at frame 5 → layer time -5 , before the first sample, so the first value (0) is held.

Real-world use case. Crowd animation: one walk-cycle asset is referenced by 1,000 extras, each with a different offset so they do not move in lockstep. In games cinematics, the same flag-waving animation is reused on many flags with offsets. In manufacturing simulation, the same robot-arm motion is replayed on several cells, each starting when its part arrives.

Common mistakes.

MISTAKE

Getting the direction wrong. `offset = 10` makes the animation happen **later** on the stage (layer frame 0 shows at stage frame 10), not earlier.

MISTAKE

Expecting `scale = 2` to make the animation faster. Scale stretches layer time: `2` means twice as long, i.e. half speed. Use `0.5` for double speed.

MISTAKE

Adding an offset to a reference whose attribute only has a default value. Nothing changes, because offsets affect time samples only.

Exam traps.

TRAP

Formula look-alikes. The correct mapping is `stage = offset + scale × layer`. A distractor might show `stage = (layer + offset) × scale`; with offset 5 and scale 2 that gives 30 instead of 25 for layer time 10.

TRAP

"Layer offsets can only be set on sublayers." False. References, internal references, and payloads each carry their own `layerOffset`.

Exam takeaways.**KEY**

- `Sdf.LayerOffset(offset, scale)`; default `(0, 1)`. - `stageTime = offset + scale × layerTime`. Positive offset = later; scale 2 = slower. - Offsets affect time samples, not default values; outside the samples, values are held. - Each reference has its own offset, so one animation file can drive many prims at different times.

16.5 Why adding a reference may fail

What is it? A list of the common reasons why a reference you wrote does not bring in the content you expect, and how to see the cause.

Why do we need it? A failed reference rarely crashes anything. The stage opens, the prim exists, and it is simply empty or missing parts. If you do not know where USD reports these problems, you can lose hours. The exam's reading list names "reasons adding a reference may fail" and "Pcp composition errors" directly.

Beginner explanation. Your catalog link can break in a few ways: the catalog is not on the shelf (missing file), the item number does not exist (wrong prim path), nobody marked "the product" (no `defaultPrim`), or the link points back at itself (a loop). The shop still opens; that one shelf is just empty, and a note is pinned on the notice board (the error list).

Where the analogy breaks: some mistakes are refused immediately at authoring time (badly written prim paths), not later.

Technical explanation. Two moments can fail:

1. **Authoring time** (when you call `AddReference`). A prim path that is not empty or not an absolute prim path (`"Chair"`, `"/Chair.size"`, `"/Chair{v=a}"`) raises `Tf.ErrorException` and nothing is written.
2. **Composition time** (when the stage composes). Problems become **composition errors**: they are printed as warnings and collected in `stage.GetCompositionErrors()`. Each error object has `errorType` and `rootSite.path` (the prim where it happened). For one prim, `prim.GetPrimIndex().localErrors` lists them.

Cause	What you see (verified on 26.08)
File cannot be found or opened (wrong path, wrong anchoring, parse error)	<code>Pcp.ErrorType_InvalidAssetPath</code>
No <code>defaultPrim</code> and no prim path	<code>Pcp.ErrorType_UnresolvedPrimPath</code> (<code><defaultPrim></code> in message)
Prim path or <code>defaultPrim</code> names a prim that does not exist	<code>Pcp.ErrorType_UnresolvedPrimPath</code>
Prim references itself or an ancestor	<code>Pcp.ErrorType_ArcCycle</code>
Layer offset scale ≤ 0	<code>Pcp.ErrorType_InvalidReferenceOffset</code> ; USD uses no offset
Relationship in the asset targets a prim outside the referenced prim	No composition error; target is dropped with a warning when you call <code>GetTargets()</code>

NOTE

Other causes exist but are not reference-specific: the referencing prim is inactive (Chapter 4), the layer is muted (Chapter 15), or a stronger opinion hides the value (Chapter 21).

VERSION

Verified on USD 26.08: `Usd.Stage.GetCompositionErrors()` returns `Pcp` error objects; `AddReference` returns `True` even when composition later fails.

Mental model.

```
AddReference(...)
|
+-- bad prim path syntax? --> Tf.ErrorException now (nothing authored)
|
v authored OK (returns True)
stage composes
|
+-- file? -----> no --> InvalidAssetPath
+-- prim / default? -> no --> UnresolvedPrimPath
+-- loops back? ----> yes --> ArcCycle
+-- scale <= 0? ----> yes --> InvalidReferenceOffset
v
content appears (check: stage.GetCompositionErrors() == [])
```

Simple example. You reference `@props/cup.usda@` from `shots/s01.usda`. The file is at `props/cup.usda` next to `shots/`. USD anchors to `shots/` and looks for `shots/props/cup.usda`, which does not exist: `InvalidAssetPath`. The fix is `@../props/cup.usda@`.

USDA example. This layer parses, but three of its four references fail when composed (assume `good.usda` exists and has `defaultPrim`, `nodefault.usda` exists without one, and `missing.usda` does not exist).

```
#usda 1.0

def "Broken"
{
    def "MissingFile" (
        prepend references = @./missing.usda@
    )
    {
    }

    def "NoDefaultPrim" (
        prepend references = @./nodefault.usda@
    )
    {
    }

    def "WrongPath" (
        prepend references = @./good.usda@</Nope>
    )
    {
    }

    def "Works" (
        prepend references = @./good.usda@
    )
    {
    }
}
```

Python example.

```
from pxr import Sdf, Tf, Usd

files = {
    "good.usda": '#usda 1.0\n(\n    defaultPrim = "Chair"\n)\n'\n                'def Xform "Chair"\n{\n    def Cube "Seat" {}\n}\n',
    "nodefault.usda": '#usda 1.0\ndef Xform "Chair"\n{\n}\n',
}
```

```

for name, text in files.items():
    with open(name, "w") as f:
        f.write(text)

stage = Usd.Stage.CreateInMemory()
def ref(path, *args):
    stage.DefinePrim(path).GetReferences().AddReference(*args)

ref("/MissingFile", "missing.usda")
ref("/NoDefaultPrim", "nodefault.usda")
ref("/WrongPath", "good.usda", "/Nope")
ref("/BadScale", Sdf.Reference("good.usda", Sdf.Path(), Sdf.LayerOffset(0, -1)))
ref("/Works", "good.usda")
cyc = stage.DefinePrim("/Loop/Child")
cyc.GetReferences().AddInternalReference("/Loop")

for err in sorted(stage.GetCompositionErrors(), key=lambda e: str(e.rootSite.path)):
    print(f"{str(err.rootSite.path):15} {err.errorType}")

works = stage.GetPrimAtPath("/Works")
print("Works children:", [c.GetName() for c in works.GetChildren()])

try:
    ref("/Relative", "good.usda", "Chair")      # not an absolute prim path
except Tf.ErrorException:
    print("authoring refused: prim path must be absolute")

```

Expected output

```

/BadScale      Pcp.ErrorType_InvalidReferenceOffset
/Loop/Child    Pcp.ErrorType_ArcCycle
/MissingFile   Pcp.ErrorType_InvalidAssetPath
/NoDefaultPrim Pcp.ErrorType_UnresolvedPrimPath
/WrongPath     Pcp.ErrorType_UnresolvedPrimPath
Works children: ['Seat']
authoring refused: prim path must be absolute

```

The errors are sorted by path here so the output is stable; `GetCompositionErrors()` itself does not promise an order. You will also see matching warnings on `stderr`. Note that `/BadScale` still gets its content; only the offset is ignored.

The next script shows the relationship case, which produces **no** composition error:

```

from pxr import Usd

ASSET = """#usda 1.0
(
    defaultPrim = "Chair"
)
def Xform "Chair"
{
    rel look = </Looks/Wood>
    rel seat = </Chair/Seat>
    def Cube "Seat" {}
}
def Scope "Looks"
{
    def "Wood" {}
}
"""

with open("asset.usda", "w") as f:
    f.write(ASSET)

stage = Usd.Stage.CreateInMemory()
chair = stage.DefinePrim("/World/Chair_01")
chair.GetReferences().AddReference("asset.usda")
print("seat targets:", chair.GetRelationship("seat").GetTargets())
print("look targets:", chair.GetRelationship("look").GetTargets())
print("composition errors:", len(stage.GetCompositionErrors()))

```

Expected output


```
seat targets: [Sdf.Path('/World/Chair_01/Seat')]
look targets: []
composition errors: 0
```

seat points inside the referenced prim, so it was remapped. look points to /Looks, a sibling that the reference does not bring in, so the target is dropped (stderr warning: “refers to a path outside the scope of the reference”). This is why material assets keep their Looks scope **inside** the root prim (Chapter 23).

Real-world use case. A layout artist reports “the hero car is invisible in shot 12.” The pipeline TD opens the shot in Python, prints `stage.GetCompositionErrors()`, and sees `InvalidAssetPath` for `@../assets/car.usda@`: the shot file had moved one folder deeper. Many studios run this check automatically at publish time.

Common mistakes.

MISTAKE

Checking only the return value of ``AddReference``. It reports authoring success, not composition success. Check ``stage.GetCompositionErrors()`` after authoring.

MISTAKE

Keeping materials in a root scope beside the asset's root prim. Bindings to them break when the asset is referenced. Put ``Looks`` under the ``defaultPrim``.

MISTAKE

Ignoring stderr. USD's warnings name the prim, the asset path, and the reason.

Exam traps.

TRAP

"A missing referenced file stops the stage from opening." False. The stage opens; that arc contributes nothing, and an ``InvalidAssetPath`` error is recorded.

TRAP

"Relationship targets outside the referenced prim are remapped to the new location." False. Only targets inside the referenced subtree are remapped; outside targets are ignored.

TRAP

Wrong prim path and missing ``defaultPrim`` are the **same** error type (``UnresolvedPrimPath``), not two different ones.

Exam takeaways.

KEY

- Bad prim-path syntax fails at authoring; everything else fails at composition and the stage still opens.
- Missing file → ``InvalidAssetPath``; missing prim or ``defaultPrim`` → ``UnresolvedPrimPath``; loops → ``ArcCycle``.
- Use ``stage.GetCompositionErrors()`` (or ``prim.GetPrimIndex().localErrors``) and read stderr.
- Relationship targets outside the referenced prim are dropped silently from the result.

16.6 References vs. sublayers

What is it? A comparison of the two most-used ways to bring another file into a scene. A **sublayer** adds a whole layer to your layer stack (Chapter 15). A **reference** brings one prim's subtree from a layer into one prim of your stage.

Why do we need it? Objective 1.3 asks you to compare referencing, payloads, and sublayers and choose among them. Picking the wrong one causes real bugs: duplicated prims at the wrong paths, or overrides that cannot win.

Beginner explanation. A **sublayer** is another transparent sheet stacked on the projector: it covers the whole picture, at the same positions. A **reference** is a sticker of one catalog part that you can place anywhere on your sheet, as many times as you like.

Where the analogy breaks: a sublayer's sheet sits *in your own stack*, so it counts as local (L). A sticker's contents are always weaker than anything written on your sheets.

Technical explanation. | Question | Sublayer | Reference | |-----|-----|-----| | What comes in? | Every prim in the layer, at the **same paths** | One target prim and its subtree, **renamed** to the referencing prim | | Where is it authored? | Layer metadata: `subLayers = [@a.usda@]` | Prim metadata: `references = @a.usda@</X>` | | Python | `layer.subLayerPaths.append(...)` | `prim.GetReferences().AddReference(...)` | | Strength | Part of the local layer stack (**L**), ordered by position in `subLayers` | **R** arc: weaker than every local opinion | | Use many times at different paths? | No | Yes | | Layer offset? | Yes (per sublayer) | Yes (per reference) | | Can target a prim? | No | Yes (or `defaultPrim`) | | Typical use | Departments' work layers on one shot; overrides for a whole scene | Assets placed into sets and shots |

Both are always loaded. A payload (Chapter 17) is like a reference that can be left unloaded, and is weaker (**P** comes after **R**).

Mental model.

```
SUBLAYER: same paths, whole layer, inside your stack (L)
root.usda [ /Set/Table /Set/Lamp ]
+ base.usda[ /Set/Table /Set/Rug ] -> /Set/Table, /Set/Lamp, /Set/Rug

REFERENCE: one prim, new path, separate stack (R)
/Set/Rug_01 --R--> rug.usda</Rug> -> /Set/Rug_01/...
/Set/Rug_02 --R--> rug.usda</Rug> -> /Set/Rug_02/...
```

Simple example. The animation department writes `anim.usda` with overrides for every character in the shot. Lighting writes `light.usda`. The shot file **sublayers** both. Each character is **referenced** into the shot from its asset file. Both arcs are used, each for its own job.

USDA example. File: `shot.usda`

```
#usda 1.0
(
    subLayers = [
        @./shot_anim.usda@,
        @./shot_layout.usda@
    ]
)

def Xform "Set"
{
    def "Rug_01" (
        prepend references = @./rug.usda@
    )
    {
    }

    def "Rug_02" (
        prepend references = @./rug.usda@
    )
    {
    }
}
```

Notes: `subLayers` is in the layer header and lists whole layers (strongest first). `references` is on individual prims; the same rug is used twice at two paths.

Python example. This script puts the same prim path in a sublayer and in a reference, then shows which wins and what each brings in.

```

from pxr import Usd

files = {
    "base.usda": """"#usda 1.0
def Sphere "Ball"
{
    double radius = 3
}
def Xform "Extra" {}
""",
    "ball_asset.usda": """"#usda 1.0
(
    defaultPrim = "Ball"
)
def Sphere "Ball"
{
    double radius = 1
    custom string madeBy = "asset"
}
def Xform "Unused" {}
""",
}
for name, text in files.items():
    with open(name, "w") as f:
        f.write(text)

stage = Usd.Stage.CreateInMemory()
stage.GetRootLayer().subLayerPaths.append("base.usda")      # whole layer
ball = stage.OverridePrim("/Ball")
ball.GetReferences().AddReference("ball_asset.usda")        # one prim

print("prims:", [str(p.GetPath()) for p in stage.Traverse()])
print("radius:", ball.GetAttribute("radius").Get())
print("madeBy:", ball.GetAttribute("madeBy").Get())
stack = ball.GetAttribute("radius").GetPropertyStack()
print("radius opinions, strongest first:",
      [spec.layer.identifier.split("/")[1] for spec in stack])

```

Expected output

```

prims: ['/Ball', '/Extra']
radius: 3.0
madeBy: asset
radius opinions, strongest first: ['base.usda', 'ball_asset.usda']

```

What to notice:

- The sublayer brought **both** of its root prims (`/Ball`, `/Extra`) at their own paths.
- The reference brought only `/Ball`'s content; its sibling `/Unused` did not appear.
- `radius` is 3: the sublayer's opinion is local (L) and beats the referenced one (R).
- `madeBy` exists only in the reference, so it comes through.

Real-world use case. In a VFX shot, the shot file sublayers department layers (layout, animation, FX, lighting) so each team edits its own file at the same prim paths (Chapter 15). Inside the layout layer, every prop and character is a reference to a published asset. In a digital twin, a plant file sublayers a "live sensor overrides" layer and references each machine's asset.

Common mistakes.

MISTAKE

Sublayering an asset file to "add a chair." The chair appears at its own root path (e.g. ``/Chair``), and you cannot place a second copy. Reference it instead.

MISTAKE

Referencing a department's work layer into a shot. Its opinions become weaker than every local opinion, so lighting tweaks may "not work." Sublayer department layers instead.

Exam traps.**TRAP**

"Sublayers and references have the same strength if they are in the same file." False. Sublayer content is local (L); referenced content is R and always weaker than any local opinion.

TRAP

"You can sublayer a single prim." False. Sublayers are whole layers; only references and payloads can target a prim.

Exam takeaways.**KEY**

- Sublayer = whole layer, same paths, local strength (L). - Reference = one prim subtree, new path, R strength, reusable many times. - Shots sublayer department layers and reference assets. - Payload = reference you can unload, and slightly weaker (Chapter 17).

Chapter lab(s)

- **Lab 13 — External and internal references, defaultPrim (★☆☆).** You build a prop asset with and without defaultPrim, reference it into a room, add internal references to a class template, and read the composition errors.
- **Lab 14 — Same animation at different time offsets (★★☆).** You reference one animated asset into a crowd of prims with different Sdf.LayerOffset values and verify the times by sampling the stage (Obj 1.4).

Both labs are in Part X (python-labs/).

USDA reading exercises

Exercise 16-A. Given these two files, what is /Shot/Bird_2's wingAngle at stage time 20?

File: *flap.usda*

```
#usda 1.0
(
    defaultPrim = "Bird"
)

def Xform "Bird"
{
    double wingAngle.timeSamples = {
        0: 0,
        10: 90,
    }
}
```

File: *shot.usda*

```
#usda 1.0

def "Shot"
{
    def "Bird_2" (
        prepend references = @./flap.usda@ (offset = 5; scale = 2)
    )
    {
    }
}
```

Exercise 16-B. prop.usda contains only def Xform "Prop" {} and no layer metadata. A shot writes def "P" (prepend references = @./prop.usda@) {}. What is /P's type, what error is recorded, and give two ways to fix it.

Exercise 16-C. In this layer, what is the composed size of `/Crate_B`, and which arc supplied it?

```
#usda 1.0

class Cube "_Crate"
{
    double size = 4
}

def "Crate_B" (
    prepend references = </_Crate>
)
{
}
```

Answers

16-A — 67.5. Layer time = $(20 - 5) / 2 = 7.5$. Linear interpolation between 0 and 90: $0.75 \times 90 = 67.5$.

16-B — Type is empty (''); error `Pcp.ErrorType_UnresolvedPrimPath`. Fix 1: add `defaultPrim = "Prop"` to `prop.usda`. Fix 2: write the prim path explicitly, `@./prop.usda</Prop>`.

16-C — 4, from the internal reference to `/_Crate` (an R arc). `Crate_B` is a `Cube` because it receives the template's type too.

Chapter review

Summary

- A reference brings one target prim and its subtree into a referencing prim, renaming paths.
- Authoring: `prim.GetReferences().AddReference(assetPath, primPath, layerOffset)`; it writes `prepend references`.
- Without a prim path, the referenced layer's `defaultPrim` is used; without one, `UnresolvedPrimPath`.
- Internal references (`</Path>`, `AddInternalReference`) reuse prims from the same layer stack.
- References are the R arc: weaker than all local opinions, stronger than payloads.
- `Sdf.LayerOffset(offset, scale): stageTime = offset + scale × layerTime`.
- Different offsets on several references let one animation play at different times (Obj 1.4).
- Composition failures do not stop the stage; read `stage.GetCompositionErrors()` and `stderr`.
- Relationship targets outside the referenced subtree are dropped.
- Sublayers = whole layers, same paths, local strength; references = one prim, any path, R strength.

If you see... → think...

If you see...	Think...
<code>@file.usda@</code> with no <code><path></code>	<code>defaultPrim</code> of that file is used
<code></Path></code> with no <code>@...@</code>	Internal reference
<code>(offset = a; scale = b)</code>	<code>stage = a + b × layer</code>
Prim exists but is empty after referencing	<code>GetCompositionErrors()</code> : missing file, missing prim/defaultPrim
"Same asset placed many times"	References (or payloads), not sublayers

If you see...	Think...
"Department layers at the same paths"	Sublayers
Local over vs. referenced value	Local wins (L before R)
Tf.ErrorException from AddReference	Prim path not absolute or not a prim path
Material binding lost after referencing	Target outside the referenced prim

Review questions

R16-01 · Obj 1.3 · Single choice Which statement about a reference is correct? A. It copies the target prim's data into the referencing layer. B. It brings the target prim and its descendants, mapped to the referencing prim's path. C. It brings every root prim of the referenced layer. D. It is stronger than local opinions.

R16-02 · Obj 1.3 · Single choice `asset.usda` defines one root prim, `/Robot`, and has `defaultPrim = "Robot"`. Another file needs `/Robot`. Which reference brings it in? A. `@asset.usda@` · B. `@asset.usda@</Asset>` · C. `</Robot>` · D. `subLayers = [@asset.usda@]`

R16-03 · Obj 1.4 · Single choice A reference has `(offset = 12)`. The asset's first sample is at layer time 0. At which stage time is it? A. `-12` · B. `0` · C. `12` · D. `24`

R16-04 · Obj 1.4 · Single choice Which `Sdf.LayerOffset` plays a referenced animation at double speed, starting at stage time 0? A. `Sdf.LayerOffset(0, 2)` · B. `Sdf.LayerOffset(0, 0.5)` · C. `Sdf.LayerOffset(2)` · D. `Sdf.LayerOffset(0.5)`

R16-05 · Obj 1.4 · Python reading What does this print? (Marked "not run automatically" so the answer is not printed next to the question; run it yourself after answering.)

```
from pxr import Sdf

lo = Sdf.LayerOffset(10, 3)
print(lo * 4)
```

A. `14.0` · B. `22.0` · C. `42.0` · D. `52.0`

R16-06 · Obj 1.3 · Select two. After `prim.GetReferences().AddReference("lamp.usda")`, nothing appears under the prim. Which checks are useful? A. `Print stage.GetCompositionErrors()` · B. Check that `lamp.usda` has a `defaultPrim`. C. Check the return value of `AddReference`. D. Check the stage's `upAxis`.

R16-07 · Obj 1.3 · USDA reading `base.usda` (a sublayer of the root layer) sets `/Car.color = "blue"`. The root layer has `over "Car" ( prepend references = @car.usda@ ) {}`, and `car.usda`'s default prim sets `color = "red"`. What is the composed color? A. `red` · B. `blue` · C. `empty` · D. It is an error

R16-08 · Obj 1.3 · Single choice Which USDA line is an internal reference? A. `prepend references = @./x.usda@</X>` · B. `prepend references = </X>` · C. `prepend payload = </X>` · D. `prepend inherits = </X>`

R16-09 · Obj 1.3 · Single choice The prim `/A/B` adds an internal reference to `/A`. What happens? A. `/A/B` gets a copy of `/A`, including `/A/B` again. B. A `Pcp.ErrorType_ArcCycle` composition error; the arc is ignored. C. `AddInternalReference` raises an exception. D. The stage fails to open.

R16-10 · Obj 1.3 · Select two. Which are true for sublayers but not for references? A. Content keeps its original prim paths. B. Content is weaker than every local opinion. C. Opinions are part of the local layer stack. D. The same file can be placed at many prim paths.

R16-11 · Obj 1.3 · Single choice An asset’s root prim `/Chair` has `rel material = </Looks/Oak>`, where `/Looks` is a root sibling of `/Chair`. After referencing the asset at `/Room/Chair_1`, what does `GetTargets()` on `material` return? A. `[/Looks/Oak]` · B. `[/Room/Chair_1/Looks/Oak]` · C. `[/Room/Looks/Oak]` · D. `[]`

R16-12 · Obj 1.4 · Single choice Three prims reference `walk.usda` with offsets 0, 8, and 16 (scale 1). The walk cycle has samples from 0 to 24. Which is true at stage time 4? A. All three show the pose at layer time 4. B. Prim 1 shows layer time 4; prims 2 and 3 hold their first pose. C. Prims 2 and 3 show layer times 12 and 20. D. Prims 2 and 3 have no value.

Review answers

R16-01 — B (Obj 1.3). The reference is a live link (not A); only the target subtree comes in (not C); R is weaker than L (not D). Review: §16.1.

R16-02 — A (Obj 1.3). No prim path means “use `defaultPrim`”. B names a prim `/Asset` that does not exist (`UnresolvedPrimPath`); C is internal (same layer stack); D is a sublayer and brings all root prims at their own paths. Review: §16.2.

R16-03 — C (Obj 1.4). $12 + 1 \times 0 = 12$. A positive offset moves the animation later. Review: §16.4.

R16-04 — B (Obj 1.4). Scale 0.5 compresses layer time: layer time 10 appears at stage time 5. A is half speed; C and D are offsets with scale 1. Review: §16.4.

R16-05 — B (Obj 1.4). $10 * 4 = 10 + 3 \times 4 = 22$, printed as `22.0`. C would be $(4 + 10) \times 3$; D is not a possible combination of the formula. Review: §16.4.

R16-06 — A, B (Obj 1.3). Composition errors list the cause; a missing `defaultPrim` is the most common one. `AddReference` returns `True` even when composition fails (C), and `upAxis` is unrelated (D). Review: §16.5.

R16-07 — B (Obj 1.3). The sublayer is in the local layer stack (L), so its “blue” beats the referenced “red” (R). Review: §16.6.

R16-08 — B (Obj 1.3). Prim path only, keyword `references`. C is an internal payload; D is an `inherit`. Review: §16.3.

R16-09 — B (Obj 1.3). Verified: the reference is authored, then composition reports `ArcCycle` and ignores it; the stage opens. Review: §16.3, §16.5.

R16-10 — A, C (Obj 1.3). B and D describe references. Review: §16.6.

R16-11 — D (Obj 1.3). The target is outside the referenced subtree, so it is dropped (with a warning). Keep `Looks` under the root prim. Review: §16.5.

R16-12 — B (Obj 1.4). Prim 2 is at layer time $4 - 8 = -4$ and prim 3 at -12 , both before the first sample, so the first pose is held. Review: §16.4.

CHAPTER 17

Chapter 17 — Payloads

Exam domain: Composition (23%) · **Objectives:** 1.3 · **Study day:** 5 · **Est. time:** 90 min
Prerequisites: Ch 2 (opening stages, traversal), Ch 14 (LIVERPS, list editing), Ch 15 (layer stacks), Ch 16 (references, `defaultPrim`, layer offsets)

Learning goals

- Explain what a payload is and how it differs from a reference.
- Open a stage with everything or nothing loaded, and load or unload parts of it with `Load`, `Unload`, and `LoadAndUnload`.
- Describe a stage's load state with `Usd.StageLoadRules` (`AllRule`, `OnlyRule`, `NoneRule`).
- Open only part of a scene's namespace with `Usd.StagePopulationMask` and `Usd.Stage.OpenMasked`.
- Place payloads correctly in LIVERPS and choose between payloads, references, and sublayers.

Key terms

Term	One-line definition
Payload	A composition arc like a reference, whose content is composed only when the prim is loaded .
Loaded / unloaded	Whether a prim's payload content is currently composed into the stage.
Load set	The list of payload prims that are currently loaded (<code>stage.GetLoadSet()</code>).
Initial load set	Whether payloads are loaded when a stage opens: <code>Usd.Stage.LoadAll</code> (default) or <code>Usd.Stage.LoadNone</code> .
Load rules	<code>Usd.StageLoadRules</code> : a set of path → rule entries that fully describes what is loaded.
Population mask	<code>Usd.StagePopulationMask</code> : a set of paths that limits which prims the stage composes at all.
Working set	The part of a large scene a user actually has loaded and is working on.

17.1 What a payload is

What is it? A **payload** is a composition arc that works like a reference (Chapter 16), with one extra power: you can choose **not to load** it. When unloaded, the prim stays on the stage, but the content behind the payload is not read or composed.

Why do we need it? A city scene may hold 20,000 buildings with gigabytes of geometry. Opening all of it to fix one street lamp wastes minutes and memory. Payloads let you open the scene's structure quickly and load only the parts you need.

Beginner explanation. A payload is a **reference you can choose not to load**, like a labelled box in storage. The inventory list (the stage) always shows the box and its label. You only carry the box in and open it when you need what is inside.

Where the analogy breaks: the “box” can also hold opinions that affect a prim that is already present (for example its type or attributes). While unloaded, those opinions are missing too, not only the child prims.

Technical explanation. - Stored as prim metadata payload (USDA keyword is singular: prepend `payload = @geo.usda@`). Each entry is an `Sdf.Payload(assetPath, primPath, layerOffset)`. - Python: `prim.GetPayloads()` returns `Usd.Payloads` with `AddPayload`,

`AddInternalPayload`, `RemovePayload`, `ClearPayloads`, `SetPayloads`. - Like references: an asset path plus optional prim path (otherwise the target layer's `defaultPrim`), an optional `Sdf.LayerOffset`, list-edited, namespace-mapped, and failure reasons are the same (Chapter 16.5). - Unlike references: composed only when the prim is loaded.

`prim.IsLoaded()` tells you the state; `prim.HasAuthoredPayloads()` tells you whether the prim has a payload. - **Traversal trap:** `stage.Traverse()` uses the default predicate, which skips unloaded prims. Use `stage.TraverseAll()` (or `GetPrimAtPath`) to see them. -

Payloads are the **P** in LIVERPS, just after R: weaker than references (Section 17.5).

VERSION

Older code uses ``prim.SetPayload(...)``, ``prim.ClearPayload()``, and ``prim.HasPayload()``. They still exist in USD 26.08, but the list-edited ``prim.GetPayloads()`` API and ``HasAuthoredPayloads()`` are the current form. Verified on USD 26.08.

Mental model.

```
stage (always present)           storage
/City
  /Block_01 [payload] ---- unloaded -- [ block_01_geo.usda ] not read
  /Block_02 [payload] ==== loaded ==== [ block_02_geo.usda ]
  /Buildings ... (composed from the payload)
```

Simple example. | State | `/City/Block_02` exists? | Its children exist? | `IsLoaded()` |
|-----|-----|-----|-----| | Unloaded | yes | no | `False` | | Loaded |
yes | yes | `True` |

USDA example. File: *tree_geo.usda*

```
#usda 1.0
(
  defaultPrim = "Tree"
)

def Xform "Tree"
{
  def Cylinder "Trunk"
  {
  }
}
```

File: *forest.usda*

```
#usda 1.0

def Xform "Forest"
{
  def Xform "TreeA" (
    prepend payload = @./tree_geo.usda@
  )
  {
  }

  def Xform "TreeB" (
    prepend payload = @./tree_geo.usda@
  )
  {
  }
}
```

Notes: `payload` (singular) is the keyword. No `<...>` prim path is given, so `defaultPrim` (Tree) is used. `TreeA` and `TreeB` are typed `Xform` locally, so they keep their type even when unloaded.

Python example.

```
from pxr import Usd

GEO = """#usda 1.0
(
```

```

    defaultPrim = "Tree"
)
def Xform "Tree"
{
    def Cylinder "Trunk" {}
}
"""
with open("tree_geo.usda", "w") as f:
    f.write(GEO)

stage = Usd.Stage.CreateNew("forest.usda")
for name in ["TreeA", "TreeB"]:
    tree = stage.DefinePrim(f"/Forest/{name}", "Xform")
    tree.GetPayloads().AddPayload("./tree_geo.usda")
stage.Save()

closed = Usd.Stage.Open("forest.usda", Usd.Stage.LoadNone)
print("Traverse: ", [str(p.GetPath()) for p in closed.Traverse()])
print("TraverseAll:", [str(p.GetPath()) for p in closed.TraverseAll()])
tree_a = closed.GetPrimAtPath("/Forest/TreeA")
print("TreeA:", tree_a.GetTypeName(), "loaded =", tree_a.IsLoaded(),
      "has payload =", tree_a.HasAuthoredPayloads())

opened = Usd.Stage.Open("forest.usda") # default: LoadAll
trunks = [str(p.GetPath()) for p in opened.Traverse() if p.GetName() == "Trunk"]
print("LoadAll trunks:", trunks)
print(stage.GetRootLayer().ExportToString())

```

Expected output

```

Traverse:  ['/Forest']
TraverseAll: ['/Forest', '/Forest/TreeA', '/Forest/TreeB']
TreeA: Xform loaded = False has payload = True
LoadAll trunks: ['/Forest/TreeA/Trunk', '/Forest/TreeB/Trunk']
#usda 1.0

def "Forest"
{
    def Xform "TreeA" (
        prepend payload = @./tree_geo.usda@
    )
    {
    }

    def Xform "TreeB" (
        prepend payload = @./tree_geo.usda@
    )
    {
    }
}

```

Notice that the same file gives two different stages depending on how it is opened. The file never stores “loaded” or “unloaded”.

Real-world use case. Asset structure guidelines (Chapter 23) put each asset’s heavy geometry behind a payload in the asset’s entry file. Set dressers open a whole film set unloaded, see every asset’s name and transform, and load only the area around the camera. Digital-twin viewers load factory halls on demand as the user walks through them.

Common mistakes.

MISTAKE

Writing `prepend payloads = ...` in USDA. The keyword is `payload` (singular); `references` is plural. Let Python author it if unsure.

MISTAKE

Concluding "the prims are missing" because `stage.Traverse()` does not list them. Unloaded prims are skipped by the default traversal; use `TraverseAll()` or check `IsLoaded()`.

Exam traps.**TRAP**

"An unloaded payload prim does not exist on the stage." False. The prim exists (with its local opinions); only the payload's content is missing.

TRAP

"Payloads are loaded lazily by default." Not in the sense of opening: `Usd.Stage.Open` loads **all** payloads unless you pass `Usd.Stage.LoadNone`.

Exam takeaways.**KEY**

- Payload = reference that can be unloaded; same targeting rules (asset path, prim path or `defaultPrim`, layer offset). - Unloaded prim exists; payload content (children and opinions) does not. - `prim.GetPayloads().AddPayload(...)` writes `prepend payload`. - Default `Traverse()` skips unloaded prims.

17.2 Loading and unloading (Load, Unload, InitialLoadSet)

What is it? The stage methods that decide which payloads are composed: the **initial load set** given to `Usd.Stage.Open`, and `Load`, `Unload`, and `LoadAndUnload` for changes while the stage is open.

Why do we need it? Users move around a large scene. They need to bring in one building, drop another, and do it without reopening the stage. This is **working set management**, named in the exam's reading list.

Beginner explanation. When you arrive at the storage room you either carry in every box (`LoadAll`) or none (`LoadNone`). Later you fetch some boxes and return others.

Where the analogy breaks: loading a box also loads boxes **inside** it, unless you say otherwise (`Usd.LoadWithoutDescendants`).

Technical explanation. - `Usd.Stage.Open(path, load)`: `load` is `Usd.Stage.LoadAll` (default) or `Usd.Stage.LoadNone`, the values of the enum `Usd.Stage.InitialLoadSet`. The same argument exists on `CreateNew`, `CreateInMemory`, and `OpenMasked`. - `stage.Load(path, policy)`: loads `path`, its ancestors, and by default its descendants (`Usd.LoadWithDescendants`). With `Usd.LoadWithoutDescendants`, nested payloads below stay unloaded. Returns the prim. With no path, it loads everything from /. - `stage.Unload(path)`: unloads `path` and all descendants. With no path, unloads everything. - `stage.LoadAndUnload(loadSet, unloadSet)`: both at once, recomposing only once. Use it for many paths. - `prim.Load()` and `prim.Unload()` are shortcuts. - `stage.GetLoadSet()` lists loaded payload prims. `stage.FindLoadable(rootPath)` lists all prims with payloads below a path (loaded or not). - Load state is **stage state**, not scene description. Loading does not write to any layer and is not saved.

Mental model.

<code>Open(LoadNone)</code>	<code>Load("/City/B1")</code>	<code>Unload("/City/B1")</code>
<code>/City</code>	<code>/City</code>	<code>/City</code>
<code>/B1 [unloaded]</code>	<code>/B1 [loaded]</code>	<code>/B1 [unloaded]</code>
	<code>/Room1 [loaded]</code>	
	<code>/Wall</code>	

(Load = this prim + ancestors + descendants, unless `LoadWithoutDescendants`)

Simple example. `city.usda` has `/City/B1` with a payload to a building, and the building has `/Room1` with a payload to a room.

Call	Load set
<code>Open(..., LoadNone)</code>	<code>[]</code>
<code>Load("/City/B1", Usd.LoadWithoutDescendants)</code>	<code>[/City/B1]</code>
<code>Load("/City/B1")</code>	<code>[/City/B1, /City/B1/Room1]</code>
<code>Unload("/City/B1")</code>	<code>[]</code>

USDA example. *File: room.usda*

```
#usda 1.0
(
    defaultPrim = "Room"
)

def Xform "Room"
{
    def Cube "Wall"
    {
    }
}
```

File: bldg.usda

```
#usda 1.0
(
    defaultPrim = "Bldg"
)

def Xform "Bldg"
{
    def Xform "Room1" (
        prepend payload = @./room.usda@
    )
    {
    }
}
```

File: city.usda

```
#usda 1.0

def Xform "City"
{
    def Xform "B1" (
        prepend payload = @./bldg.usda@
    )
    {
    }
}
```

Notes: this is a **nested payload**. `/City/B1/Room1` only exists after `/City/B1` is loaded, because it comes from inside `bldg.usda`.

Python example.

```
from pxr import Usd

files = {
    "room.usda": '#usda 1.0\n(\n    defaultPrim = "Room"\n)\n'\n                'def Xform "Room"\n{\n    def Cube "Wall" {}\n}\n',
    "bldg.usda": '#usda 1.0\n(\n    defaultPrim = "Bldg"\n)\n'\n                'def Xform "Bldg"\n{\n    def Xform "Room1" ( prepend payload = @./room.usda@ )\n    {\n    }\n}\n',
    "city.usda": '#usda 1.0\ndef Xform "City"\n{\n    def Xform "B1" ( prepend payload = @./bldg.usda@ )\n    {\n    }\n}\n'\n}

```

```

for name, text in files.items():
    with open(name, "w") as f:
        f.write(text)

def show(stage, label):
    names = [p.GetName() for p in stage.Traverse()]
    loaded = [str(p) for p in stage.GetLoadSet()]
    print(f"{label:18} prims={names}")
    print(f"{'':18} loadSet={loaded}")

stage = Usd.Stage.Open("city.usda", Usd.Stage.LoadNone)
show(stage, "opened LoadNone")
stage.Load("/City/B1", Usd.Stage.LoadWithoutDescendants)
show(stage, "load B1 only")
print("loadable:", [str(p) for p in stage.FindLoadable()])
stage.Load("/City/B1")
show(stage, "load B1 + nested")
stage.Unload("/City/B1")
show(stage, "unload B1")
stage.LoadAndUnload(["/City/B1"], [])
show(stage, "LoadAndUnload")
print("root layer dirty?", stage.GetRootLayer().dirty)

```

Expected output

```

opened LoadNone      prims=['City']
                      loadSet=[]
load B1 only          prims=['City', 'B1']
                      loadSet=['/City/B1']
loadable: ['/City/B1', '/City/B1/Room1']
load B1 + nested      prims=['City', 'B1', 'Room1', 'Wall']
                      loadSet=['/City/B1', '/City/B1/Room1']
unload B1              prims=['City']
                      loadSet=[]
LoadAndUnload          prims=['City', 'B1', 'Room1', 'Wall']
                      loadSet=['/City/B1', '/City/B1/Room1']
root layer dirty? False

```

FindLoadable found /City/B1/Room1 only after B1 was loaded, because that payload is inside bldg.usda. dirty is False: loading changed no layer.

Real-world use case. A layout tool shows a scene outliner with a “load / unload” checkbox per asset; each click calls Load or Unload. A batch render farm job opens shots with LoadAll. A pipeline script that only edits transforms opens shots with LoadNone, which is much faster.

Common mistakes.

MISTAKE

Calling `Load` in a loop over thousands of paths. Each call recomposes. Collect the paths and call `LoadAndUnload` once.

MISTAKE

Expecting `stage.Save()` to remember what was loaded. It does not. Store working sets in your own tool's settings, or re-apply load rules (Section 17.3).

Exam traps.

TRAP

`Usd.Stage.LoadAll` / `Usd.Stage.LoadNone` (initial load set when opening) are not the same as `Usd.StageLoadRules.LoadAll()` / `LoadNone()` (static constructors that make rule objects). Read which class the question names.

TRAP

“`Load(path)` loads only that prim.” By default it also loads all payloads below it. Only `Usd.LoadWithoutDescendants` stops that.

Exam takeaways.**KEY**

- ``Usd.Stage.Open(path, Usd.Stage.LoadNone)`` opens with nothing loaded; the default is ``LoadAll``.
- ``Load(path)`` loads ancestors and descendants; ``Usd.LoadWithoutDescendants`` limits it.
- Use ``LoadAndUnload`` for batches; ``GetLoadSet`` and ``FindLoadable`` to inspect.
- Load state is never saved in a layer.

17.3 Usd.StageLoadRules

What is it? `Usd.StageLoadRules` is an object that describes the entire load state of a stage as a short list of **path → rule** entries. `Load` and `Unload` edit these rules for you; you can also build rules yourself and apply them with `stage.SetLoadRules(rules)`.

Why do we need it? A list of 10,000 loaded paths is clumsy. Rules are compact (“load nothing, except everything under `/City/Block_7`”), can be stored by a tool, and applied in one step to any stage.

Beginner explanation. Instead of a list of every box you carried in, you write instructions: “No boxes, except **everything** in aisle 7, and **only the top box** in aisle 9.”

Where the analogy breaks: rules apply to future boxes too. If aisle 7 later gets new boxes (new payloads), the “everything” rule loads them automatically.

Technical explanation. - Three rule values: `Usd.StageLoadRules.AllRule` (load this prim and all descendants), `OnlyRule` (load this prim’s payload, not descendants), `NoneRule` (load nothing here or below). - For any path, the **closest ancestor-or-self entry** decides; `rules.GetEffectiveRuleForPath(path)` returns it. - An **empty** rules object (`Usd.StageLoadRules()`) means “load everything”. `Usd.StageLoadRules.LoadAll()` and `LoadNone()` are constructors for the two starting points. - Edit methods: `AddRule(path, rule)`, `LoadWithDescendants(path)`, `LoadWithoutDescendants(path)`, `Unload(path)`, `LoadAndUnload(...)`, `SetRules(...)`, `Minimize()`. Query: `IsLoaded(path)`, `IsLoadedWithAllDescendants(path)`, `GetRules()`. - `LoadWithDescendants/LoadWithoutDescendants/Unload` also remove rules for descendants of `path`, so the new instruction is the only one that applies there. - Stage: `stage.GetLoadRules()` and `stage.SetLoadRules(rules)`.

Mental model.

```
rules: (/ , NoneRule) (/Forest/TreeB , AllRule)
```

path	closest entry	loaded?
/Forest/TreeA	/ -> NoneRule	no
/Forest/TreeB	/Forest/TreeB All	yes
/Forest/TreeB/Leaves	/Forest/TreeB All	yes

Simple example. | Rules | `/A` (payload) | `/A/B` (nested payload) | `/C` (payload) |
|-----|-----|-----|-----| | empty `Usd.StageLoadRules()` | loaded |
loaded | loaded | | `LoadNone()` + `(/A, AllRule)` | loaded | loaded | unloaded | | `LoadNone()`
+ `(/A, OnlyRule)` | loaded | unloaded | unloaded |

USDA example. Load rules are not scene description, so there is no USDA syntax for them. This is the scene they act on in the Python example (`forest3.usda`):

```
#usda 1.0

def Xform "Forest"
{
  def Xform "TreeA" (
    prepend payload = @./tree_geo.usda@
  )
  {
  }
```

```

def Xform "TreeB" (
    prepend payload = @./tree_geo.usda@
)
{
}

def Xform "TreeC" (
    prepend payload = @./tree_geo.usda@
)
{
}
}

```

Python example.

```

from pxr import Usd

GEO = '#usda 1.0\n(\n    defaultPrim = "Tree"\n)\n' \
      'def Xform "Tree"\n{\n    def Cylinder "Trunk" {}\n}\n'
with open("tree_geo.usda", "w") as f:
    f.write(GEO)
stage = Usd.Stage.CreateNew("forest3.usda")
for name in ["TreeA", "TreeB", "TreeC"]:
    stage.DefinePrim(f"/Forest/{name}", "Xform").GetPayloads().AddPayload(
        "@./tree_geo.usda")
stage.Save()

R = Usd.StageLoadRules
rules = R.LoadNone()
rules.AddRule("/Forest/TreeB", R.AllRule)
print(rules)
print("TreeA:", rules.IsLoaded("/Forest/TreeA"),
      "TreeB:", rules.IsLoaded("/Forest/TreeB"),
      "rule for TreeB/Trunk:", rules.GetEffectiveRuleForPath("/Forest/TreeB/Trunk"))

work = Usd.Stage.Open("forest3.usda")      # LoadAll ...
work.SetLoadRules(rules)                   # ... then apply the rules
print("loadSet:", [str(p) for p in work.GetLoadSet()])

work.Load("/Forest/TreeC")                 # Load/Unload edit the rules
work.Unload("/Forest/TreeB")
for path, rule in work.GetLoadRules().GetRules():
    print(" ", path, rule)
print("loadSet:", [str(p) for p in work.GetLoadSet()])
print("empty rules load everything:", R().IsLoaded("/Forest/TreeA"))

```

Expected output

```

UsdStageLoadRules([ (</>, NoneRule) (</Forest/TreeB>, AllRule) ])
TreeA: False TreeB: True rule for TreeB/Trunk: Usd.StageLoadRules.AllRule
loadSet: ['/Forest/TreeB']
  / Usd.StageLoadRules.NoneRule
  /Forest/TreeB Usd.StageLoadRules.NoneRule
  /Forest/TreeC Usd.StageLoadRules.AllRule
loadSet: ['/Forest/TreeC']
empty rules load everything: True

```

`Load("/Forest/TreeC")` added an `AllRule` entry, and `Unload("/Forest/TreeB")` replaced `TreeB's AllRule` with a `NoneRule`. The rules object is the stage's whole load state.

Real-world use case. A layout application saves the artist's working set as load rules in its own session file. Next morning it reopens the shot with `LoadNone` and calls `SetLoadRules` once; the artist sees exactly what they had loaded yesterday, plus any new assets added under an `AllRule` area.

Common mistakes.

MISTAKE

Thinking an empty ``Usd.StageLoadRules()`` loads nothing. Empty means `**load all**`. Start from ``Usd.StageLoadRules.LoadNone()`` to build "nothing except...".

MISTAKE

Using ``OnlyRule`` to load an asset and its nested payloads. ``OnlyRule`` loads that prim's own payload only; use ``AllRule``.

Exam traps.**TRAP**

Rule names look like method names: ``AllRule`` / ``OnlyRule`` / ``NoneRule`` are **values**; ``LoadWithDescendants`` / ``LoadWithoutDescendants`` / ``Unload`` are **methods** that write those values.

TRAP

"Load rules are saved with the layer." False. They belong to the stage object and live only in memory.

Exam takeaways.**KEY**

- ``Usd.StageLoadRules`` = path → ``AllRule`` / ``OnlyRule`` / ``NoneRule``; the closest entry wins. - Empty rules = load everything; ``LoadNone()`` is the "nothing except" starting point. - ``stage.SetLoadRules(rules)`` applies them in one step; ``Load`` / ``Unload`` edit them. - Rules are stage state, not authored data.

17.4 Population masks (OpenMasked)

What is it? A **population mask** (`Usd.StagePopulationMask`) is a set of prim paths. A stage opened with `Usd.Stage.OpenMasked(path, mask)` composes only those prims, their descendants, and their ancestors. Everything else is not on the stage at all.

Why do we need it? Payloads only help where the asset author put them. A mask works on any file: a tool that only needs `/World/Cameras` can skip composing the rest of a huge scene, whether or not it uses payloads.

Beginner explanation. The storage room inventory itself is torn in half: you take only the pages for aisle 7. Boxes in other aisles do not even appear on your list.

Where the analogy breaks: the pages for the *path to* aisle 7 (its ancestors) are kept, so you can still reach it.

Technical explanation. - Build: `Usd.StagePopulationMask(["/World/Cams"])` or `Usd.StagePopulationMask().Add("/World/Cams")`. `Usd.StagePopulationMask.All()` includes everything. - Open: `Usd.Stage.OpenMasked(filePath or layer, mask, load=Usd.Stage.LoadAll)`. Masks and load sets combine: the mask decides which prims exist; load rules decide which payloads are composed inside the mask. - For each mask path, the stage includes the prim, **all its descendants**, and its **ancestors** (but not the ancestors' other children). - Prims outside the mask are absent: `GetPrimAtPath` returns an invalid prim, and traversal does not visit them. - Change later: `stage.SetPopulationMask(mask)`, `stage.GetPopulationMask()`. `stage.ExpandPopulationMask()` grows the mask to include targets of relationships and attribute connections of prims already in it. - Query: `mask.includes(path)` (path is on the stage), `mask.includesSubtree(path)` (path and everything below it are).

Mental model.

file:	/Forest	mask = [/Forest/TreeB]
	/Forest/TreeA	
	/Forest/TreeB	stage: /Forest (ancestor)
	/Forest/TreeB/Trunk	/Forest/TreeB (mask path)
	/Lights	/Forest/TreeB/Trunk (descendant)
		absent: /Forest/TreeA, /Lights

Simple example. | Mask | /Forest | /Forest/TreeA | /Forest/TreeB/Trunk | /Lights |
 |-----|-----|-----|-----| | [/Forest/TreeB] | yes (ancestor) | no |
 yes | no | | [/Forest] | yes | yes | yes | no | | All() | yes | yes | yes | yes |

USDA example. Masks are not scene description; there is no USDA for them. This is the layer used below (forest_rel.usda); note the relationship from TreeA to TreeB:

```
#usda 1.0

def Xform "Forest"
{
    def Xform "TreeA"
    {
        rel neighbor = </Forest/TreeB>
    }

    def Xform "TreeB"
    {
        def Cylinder "Trunk"
        {
        }
    }
}

def Scope "Lights"
{
}
```

Python example.

```
from pxr import Usd

SCENE = """#usda 1.0
def Xform "Forest"
{
    def Xform "TreeA"
    {
        rel neighbor = </Forest/TreeB>
    }
    def Xform "TreeB"
    {
        def Cylinder "Trunk" {}
    }
}
def Scope "Lights" {}
"""

with open("forest_rel.usda", "w") as f:
    f.write(SCENE)

mask = Usd.StagePopulationMask(["/Forest/TreeB"])
stage = Usd.Stage.OpenMasked("forest_rel.usda", mask)
print("masked:", [str(p.GetPath()) for p in stage.Traverse()])
print("TreeA valid?", stage.GetPrimAtPath("/Forest/TreeA").IsValid())
print("Includes /Forest:", mask.includes("/Forest"),
      "IncludesSubtree /Forest:", mask.includesSubtree("/Forest"))

stage.SetPopulationMask(Usd.StagePopulationMask(["/Forest/TreeA"]))
print("TreeA mask:", [str(p.GetPath()) for p in stage.Traverse()])
stage.ExpandPopulationMask() # follow TreeA.neighbor
print("expanded:", stage.GetPopulationMask())
print("now:", [str(p.GetPath()) for p in stage.Traverse()])
```

Expected output

```
masked: ['/Forest', '/Forest/TreeB', '/Forest/TreeB/Trunk']
TreeA valid? False
Includes /Forest: True IncludesSubtree /Forest: False
TreeA mask: ['/Forest', '/Forest/TreeA']
expanded: UsdStagePopulationMask([ /Forest/TreeA /Forest/TreeB ])
now: ['/Forest', '/Forest/TreeA', '/Forest/TreeB', '/Forest/TreeB/Trunk']
```

Real-world use case. A render-farm “camera export” job opens a 50 GB shot with a mask of `/Shot/Cameras` and finishes in seconds. A digital-twin dashboard opens only `/Plant/Line_3` of a whole factory. A review tool opens one character in isolation and then calls `ExpandPopulationMask()` so its materials, found through bindings, come along.

Common mistakes.

MISTAKE

Masking to one prim and finding its material binding target missing. Bindings are relationships to prims that may be outside the mask. Call ``stage.ExpandPopulationMask()`` or add the material paths to the mask.

MISTAKE

Using a mask to save memory but still loading every payload inside it. Combine: ``OpenMasked(path, mask, Usd.Stage.LoadNone)`` and load what you need.

Exam traps.

TRAP

"A population mask is the same as unloading." No. Unloaded prims still exist on the stage; prims outside a mask do not exist at all (invalid prim).

TRAP

"Masking ``/A/B`` also gives ``/A``'s other children." No. Ancestors are included as path holders only; their other children are excluded.

Exam takeaways.

KEY

- ``Usd.Stage.OpenMasked(path, Usd.StagePopulationMask([...]))`` composes only part of the namespace. - Included: mask paths, their descendants, their ancestors (not the ancestors' other children). - Outside the mask = invalid prim, not "unloaded". - Masks work on any scene; payloads only where authored. Combine both for big scenes.

17.5 Payloads vs. references: strength and use cases

What is it? A comparison of payloads and references: where each sits in strength order (LIVERPS), and when to use which.

Why do we need it? Objective 1.3 asks you to compare referencing, payloads, and sublayers. Exam questions test both the strength order (R is stronger than P) and the design choice (which arc to use for heavy data).

Beginner explanation. Both are catalog links. The **reference** is a part bolted to your design: always there. The **payload** is the box in storage: present only when fetched. When both describe the same thing, the bolted-on part's description wins.

Where the analogy breaks: “stronger” is not about loading. A loaded payload is still weaker than a reference on the same prim.

Technical explanation. - LIVERPS: Local, Inherits, VariantSets, rElocates, **References**, **Payloads**, Specializes. On one prim, any reference opinion beats any payload opinion for the same property. - Opinions only in the payload still apply when loaded (nothing stronger competes). - When unloaded, payload opinions disappear, so values may change, not just children. - Choosing:

Need	Use
Always-needed, lightweight data (asset interface, transforms, look selection)	Reference
Heavy data the user may skip (geometry, high-res textures, simulation caches)	Payload

Need	Use
Whole layer at the same paths (department or override layer)	Sublayer
Open only part of any scene, regardless of arcs	Population mask

- Common asset pattern (Chapter 23): the shot **references** an asset's entry file; inside it, the asset root has a **payload** to its geometry. The shot can still unload it, because payloads inside referenced files remain loadable from the stage.

Mental model.

```
strength on one prim (strongest first):
L local layer stack      <- your overrides
I inherits
V variant sets
E relocates
R references    ---+
P payloads      ---+  "R before P": reference beats payload
S specializes
```

R and P: same targeting rules; only P can be unloaded.

Simple example. /X references a.usda (radius = 1) and has a payload to b.usda (radius = 2, tag = "fromPayload").

State	radius	tag
Loaded	1 (reference wins)	"fromPayload"
Unloaded	1	does not exist

USDA example.

```
#usda 1.0

def "X" (
  prepend payload = @./b.usda@
  prepend references = @./a.usda@
)
{
}
```

Notes: the order in which the two fields are written in the file does not matter; strength comes from the arc type (R before P), not from line order.

Python example.

```
from pxxr import Usd

files = {
    "a.usda": '#usda 1.0\n(\n    defaultPrim = "M"\n)\n'\n              'def Sphere "M"\n{\n    double radius = 1\n}\n',
    "b.usda": '#usda 1.0\n(\n    defaultPrim = "M"\n)\n'\n              'def Sphere "M"\n{\n    double radius = 2\n'\n              '    custom string tag = "fromPayload"\n}\n'\n              '\n    prepend payload = @./b.usda@\n'\n              '    prepend references = @./a.usda@%\n)\n%\n)',
}

for name, text in files.items():
    with open(name, "w") as f:
        f.write(text)

def report(stage, label):
    x = stage.GetPrimAtPath("/X")
    tag = x.GetAttribute("tag")
    print(f"{label:9} loaded={x.IsLoaded()} radius={x.GetAttribute('radius').Get()}"
          f" tag={tag.Get() if tag else None}")

stage = Usd.Stage.Open("shot.usda")
report(stage, "loaded")
```

```
query = Usd.PrimCompositionQuery(stage.GetPrimAtPath("/X"))
for arc in query.GetCompositionArcs():
    print("  arc:", arc.GetArcType(), arc.GetTargetLayer().identifier.split("/")[-1])
stage.Unload("/X")
report(stage, "unloaded")
```

Expected output

```
loaded    loaded=True radius=1.0 tag=fromPayload
  arc: Pcp.ArcTypeRoot shot.usda
  arc: Pcp.ArcTypeReference a.usda
  arc: Pcp.ArcTypePayload b.usda
unloaded  loaded=False radius=1.0 tag=None
```

`Usd.PrimCompositionQuery` lists the arcs strongest first: the root layer, then the reference, then the payload (Chapter 22 covers this tool).

Real-world use case. In a games studio's level editor, each building is referenced (so its transform and collision proxy are always present) and its high-detail interior is a payload loaded only when the designer enters it. In a factory digital twin, each machine's metadata (serial number, sensor IDs) is in the referenced interface, and its CAD-derived meshes are behind a payload.

Common mistakes.

MISTAKE

Putting overrides for an asset in a payload and expecting them to beat its reference. They lose (P is weaker than R). Put overrides in the local layer stack.

MISTAKE

Putting everything behind payloads, including the data needed to place and select assets. Unloaded assets then have no transform or `kind`. Keep the interface in a reference (or local opinions) and only the heavy data in the payload.

Exam traps.

TRAP

"Payloads are stronger than references because they are loaded later." Wrong reasoning. Load timing has nothing to do with strength: R beats P.

TRAP

"Payloads replace references in modern USD." No. Both arcs exist and are used together; each has its own job.

Exam takeaways.

KEY

- LIVERPS: R before P — a reference beats a payload on the same prim. - Payload opinions vanish when unloaded; values can change, not only children. - Reference the lightweight interface, payload the heavy data, sublayer whole layers. - Payloads inside referenced files are still loadable from the stage.

Chapter lab(s)

- **Lab 15 — Payloads, load rules, population masks (★★☆).** You build a city of payload-backed blocks, open it with `LoadNone`, load a working set with `Load` and `LoadAndUnload`, save and reapply it as `Usd.StageLoadRules`, and finally open one block alone with `OpenMasked`, timing each approach.

The lab is in Part X (`python-labs/`).

USDA reading exercises

Exercise 17-A. Using `room.usda`, `bldg.usda`, and `city.usda` from Section 17.2, the stage is opened with `Usd.Stage.LoadNone`, then `stage.Load("/City/B1", Usd.LoadWithoutDescendants)` runs. List the prims visited by `stage.Traverse()` and by `stage.TraverseAll()`.

Exercise 17-B. In this layer, `a.usda`'s default prim has `double size = 5` and `b.usda`'s default prim has `double size = 9` and `double depth = 2`. The stage is opened with `LoadAll`. What are `size` and `depth` on `/Crate`? What changes after `stage.Unload("/Crate")`?

```
#usda 1.0

def "Crate" (
    prepend payload = @./b.usda@
    prepend references = @./a.usda@
)
{
}
```

Answers

17-A — `Traverse()`: `/City`, `/City/B1`. `TraverseAll()`: `/City`, `/City/B1`, `/City/B1/Room1`. `Room1` exists (it comes from the loaded `bldg.usda`) but is unloaded, so the default traversal skips it, and `Wall` does not exist yet.

17-B — Loaded: `size = 5` (reference beats payload), `depth = 2` (only the payload has it). After unloading: `size` is still 5; `depth` no longer exists.

Chapter review

Summary

- A payload is a reference that can be left unloaded; keyword `payload`, API `prim.GetPayloads()`.
- Unloaded payload prims exist but have none of the payload's content; `Traverse()` skips them.
- `Usd.Stage.Open(path, Usd.Stage.LoadNone)` opens with nothing loaded; the default is `LoadAll`.
- `Load` / `Unload` / `LoadAndUnload` manage the working set; `Load` includes descendants unless `Usd.LoadWithoutDescendants`.
- `Usd.StageLoadRules` (`AllRule`, `OnlyRule`, `NoneRule`) describes load state compactly; empty rules load all.
- Load state and masks are stage state; nothing is written to layers.
- `Usd.Stage.OpenMasked` with a `Usd.Stage.PopulationMask` composes only mask paths, their descendants and ancestors.
- Prims outside a mask are invalid; `ExpandPopulationMask()` follows relationships.
- In LIVERPS, references are stronger than payloads (R before P).
- Reference the light interface, payload the heavy data, sublayer whole layers.

If you see... → think...

If you see...	Think...
<code>prepend payload = @...@</code>	Payload arc: loadable, weaker than references
"open quickly, load later"	<code>Usd.Stage.Open(path, Usd.Stage.LoadNone) + Load</code>
Prim present but children missing	Unloaded payload (<code>IsLoaded()</code> is <code>False</code>)
<code>Traverse()</code> misses prims that <code>GetPrimAtPath</code> finds	Default predicate skips unloaded prims; use <code>TraverseAll()</code>

If you see...	Think...
AllRule / OnlyRule / NoneRule	Usd.StageLoadRules
"Only this part of the namespace"	Usd.Stage.OpenMasked + Usd.Stage.PopulationMask
GetPrimAtPath returns invalid in a masked stage	Prim is outside the mask
Reference and payload disagree	Reference wins

Review questions

R17-01 · Obj 1.3 · Single choice What is the main difference between a payload and a reference? A. Payloads cannot target a prim path. B. Payloads can be unloaded, so their content is composed only on demand. C. Payloads bring in the whole layer at the same paths. D. Payloads are stronger than local opinions.

R17-02 · Obj 1.3 · Single choice Which call opens a stage with no payloads composed? A. Usd.Stage.Open("s.usda", Usd.Stage.LoadNone) B. Usd.Stage.Open("s.usda", Usd.StageLoadRules.NoneRule) C. Usd.Stage.Open("s.usda", load=False) D. Usd.Stage.OpenMasked("s.usda", Usd.Stage.PopulationMask()) with the default load set

R17-03 · Obj 1.3 · Select two. A stage was opened with LoadNone. /Set/Tree has a payload. Which are true before anything is loaded? A. stage.GetPrimAtPath("/Set/Tree") is valid. B. stage.Traverse() visits /Set/Tree. C. stage.FindLoadable() includes /Set/Tree. D. /Set/Tree's payload children are valid prims.

R17-04 · Obj 1.3 · Single choice /City/B1 has a payload; inside it, /City/B1/Room1 has another. After stage.Load("/City/B1") on a LoadNone stage, what is the load set? A. [] · B. [/City/B1] · C. [/City/B1, /City/B1/Room1] · D. [/City]

R17-05 · Obj 1.3 · Single choice Which Usd.StageLoadRules object loads everything? A. Usd.StageLoadRules() (empty) · B. Usd.StageLoadRules.LoadNone() · C. One entry (/ , OnlyRule) · D. One entry (/ , NoneRule)

R17-06 · Obj 1.3 · Single choice Rules: (/ , NoneRule) (/Set, OnlyRule). /Set has a payload, and /Set/Car (from inside it) has another. What is loaded? A. Nothing · B. /Set only · C. /Set and /Set/Car · D. Everything

R17-07 · Obj 1.3 · Single choice A stage is opened with mask [/World/Cams/Main]. Which prim is on the stage? A. /World/Lights · B. /World/Cams/Alt · C. /World/Cams · D. /Props

R17-08 · Obj 1.3 · Select two. Which statements about population masks are true? A. Prims outside the mask are returned as invalid prims. B. A mask only affects prims that have payloads. C. ExpandPopulationMask() can add relationship targets. D. Masks are written to the root layer on Save().

R17-09 · Obj 1.3 · USDA reading /Lamp has prepend references = @./lamp_a.usda@ (intensity = 10) and prepend payload = @./lamp_b.usda@ (intensity = 50). The stage is fully loaded and has no local intensity. What is intensity? A. 10 · B. 50 · C. 60 · D. no value

R17-10 · Obj 1.3 · Single choice A team wants every asset's transform and kind always available, but its heavy geometry optional. Which structure fits? A. Put everything in a payload. B. Reference the asset interface; put geometry behind a payload inside it. C. Sublayer every asset into the shot. D. Put the geometry in a variant set.

R17-11 · Obj 1.3 · Python reading A forest file has payload prims /Forest/TreeA and /Forest/TreeB. What does this print? (Not run automatically: it needs that forest file; Section 17.1 shows how to create it.)

```
from pxr import Usd

stage = Usd.Stage.Open("forest.usda", Usd.Stage.LoadNone)
```

```
stage.Load("/Forest/TreeA")
print([str(p) for p in stage.GetLoadSet()], stage.GetRootLayer().dirty)
```

A. ['/Forest/TreeA'] False · B. ['/Forest/TreeA'] True · C. ['/Forest/TreeA', '/Forest/TreeB'] False · D. [] False

R17-12 · Obj 1.3 · Single choice Which is true about payloads inside a referenced asset?

A. They are always loaded and cannot be unloaded from the referencing stage. B. They are loadable and unloadable from the referencing stage like any payload. C. They turn into references. D. They are ignored.

Review answers

R17-01 — B (Obj 1.3). Payloads target prims like references (not A), only sublayers bring whole layers (not C), and P is weaker than L (not D). Review: §17.1.

R17-02 — A (Obj 1.3). B passes a rule value where an initial load set is expected; C is not a valid argument; D loads all payloads by default, and an empty mask excludes every prim (verified: the stage is empty). Review: §17.2.

R17-03 — A, C (Obj 1.3). The prim exists and is loadable, but the default traversal skips unloaded prims (B), and its payload children do not exist yet (D). Review: §17.1, §17.2.

R17-04 — C (Obj 1.3). Load defaults to `Usd.LoadWithDescendants`, so the nested payload loads too (verified in §17.2). Review: §17.2.

R17-05 — A (Obj 1.3). Empty rules mean “load all”. B and D load nothing; C loads only the root’s own payload (if any). Review: §17.3.

R17-06 — B (Obj 1.3). `OnlyRule` loads `/Set`’s payload but not payloads below it. Review: §17.3.

R17-07 — C (Obj 1.3). `/World/Cams` is an ancestor of the mask path. Siblings (B) and other branches (A, D) are excluded. Review: §17.4.

R17-08 — A, C (Obj 1.3). Masks apply to all prims (not B) and are stage state (not D). Review: §17.4.

R17-09 — A (Obj 1.3). The reference (R) beats the payload (P). Opinions do not add up (not C). Review: §17.5.

R17-10 — B (Obj 1.3). This is the standard asset pattern. A hides the interface when unloaded; C cannot place assets at new paths; D does not provide on-demand loading. Review: §17.5.

R17-11 — A (Obj 1.3). Only `TreeA` is loaded, and loading writes nothing to the layer, so `dirty` stays `False` (behavior verified in §17.2). Review: §17.2.

R17-12 — B (Obj 1.3). Payloads keep working through references; the stage’s load rules apply to them. Review: §17.5.

CHAPTER 18

Chapter 18 — Variant Sets

Exam domain: Composition (23%) · **Objectives:** 1.7 (also 1.1, 1.6) · **Study day:** 6 · **Est. time:** 100 min **Prerequisites:** Ch 3 (layers), Ch 4 (prims, specifiers), Ch 14 (opinions, strength, LIVERPS first look), Ch 15 (edit targets), Ch 16 (references), Ch 17 (payloads)

Learning goals

- Explain what a variant set, a variant, and a variant selection are, and how they appear in USDA.
- Author opinions inside a variant with `GetVariantEditContext()`, and avoid the “no selection” trap.
- Build nested variant sets and predict which sets exist for a given selection.
- Configure fallback selections with `Usd.Stage.SetGlobalVariantFallbacks()` at the right time.
- Decide when a variant set is the right tool for structuring an asset, and when it is not (Obj 1.7).
- Predict the winning value when direct opinions, variant opinions, and selections compete (Obj 1.1, 1.6).

Key terms

Term	One-line definition
Variant set	A named switch on a prim, such as <code>color</code> or <code>lod</code> , that holds several alternatives.
Variant	One named alternative inside a variant set, such as <code>red</code> or <code>high</code> . It stores opinions.
Variant selection	A prim metadata opinion (<code>variants = { string color = "red" }</code>) naming which variant is active.
Variant opinion	An opinion authored inside a variant. It only counts when that variant is selected.
Variant edit target	An edit target that sends new opinions into one variant instead of onto the prim directly.
Nested variant set	A variant set authored inside a variant of another set.
Fallback selection	The variant USD picks when no selection is authored anywhere.
Direct (local) opinion	An opinion authored on the prim itself in a layer, not inside a variant or another arc.
Variant spec path	The Sdf path of a variant’s contents, written <code>/Chair{color=red}</code> .

18.1 Variant sets and variants

What is it? A **variant set** is a named switch on a prim. Each position of the switch is a **variant**. Each variant holds its own opinions, and only the selected variant contributes to the composed stage.

Why do we need it? Real assets come in versions: a chair in red, blue, or green; a tree at high, medium, or low detail. Without variant sets you would need one file per version and would have to swap references to switch. With a variant set, all versions live in one asset, and anyone downstream switches by changing a single string.

Beginner explanation. Think of a light switch with named positions, or a car radio with preset buttons. The radio contains every station's preset, but only one plays at a time. Pressing another button changes what you hear without rebuilding the radio.

Where the analogy breaks: a radio button only plays sound. A variant can change anything on the prim and its children: attribute values, child prims, references, payloads, even other variant sets.

Technical explanation. - A variant set belongs to one prim. You get it through `prim.GetVariantSets()` (a `Usd.VariantSets` object), then `AddVariantSet(name)` or `GetVariantSet(name)` returns a `Usd.VariantSet`. - `Usd.VariantSet.AddVariant(name)` creates a variant. `GetVariantNames()` lists them (USD 26.08 returns them sorted by name). - The list of set names is prim metadata `variantSets`, list-edited like other lists (Chapter 14). Python authors prepend `variantSets`. - The selection is a separate prim metadata dictionary, `variants`. It maps set name to variant name. - In Sdf terms, a variant's contents live at a **variant spec path** such as `/Chair{color=red}`. Its child prims live below it, for example `/Chair{color=red}Cushion`. - Variant names follow identifier rules with extras: `low-res`, `1k`, and `a|b` are valid; names with `.` or spaces are rejected (checked on 26.08). - Variants are the **V** in LIVERPS: they are stronger than references, payloads, and specializes, and weaker than local opinions and inherits.

Mental model.

```
Prim "Chair"
+-- variant set "color"      (the switch)
|   +-- "blue"   { opinions... }
|   +-- "green"  { opinions... }
|   +-- "red"    { opinions... }  <-- selection: color = "red"
|
Only the selected variant's opinions enter composition.
```

Simple example. | Variant set | Variants | Selection | What you see |

|-----|-----|-----|-----| | color | red, blue, green | blue | The blue chair |

USDA example.

```
#usda 1.0
(
    defaultPrim = "Chair"
)

def Xform "Chair" (
    variants = {
        string color = "blue"
    }
    prepend variantSets = "color"
)
{
    variantSet "color" = {
        "blue" {
            color3f[] primvars:displayColor = [(0, 0, 1)]
        }
        "red" {
            color3f[] primvars:displayColor = [(1, 0, 0)]
        }
    }
}
```

Line by line: - `variants = { string color = "blue" }` is the **selection**. It is prim metadata. - `prepend variantSets = "color"` declares that this prim has a set called `color`. - `variantSet "color" = { ... }` holds the variants. Each `"name" { ... }` block is a variant with its own opinions. - Each variant body must span several lines. A one-line body such as `"red" { int x = 1 }` fails to parse.

Python example.

```

from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
chair = UsdGeom.Xform.Define(stage, "/Chair").GetPrim()

vsets = chair.GetVariantSets()
color = vsets.AddVariantSet("color")
for name in ["red", "blue", "green"]:
    color.AddVariant(name)

print("sets:", vsets.GetNames())
print("variants:", color.GetVariantNames())
print("selection:", repr(color.GetVariantSelection()))
color.SetVariantSelection("blue")
print("selection:", color.GetVariantSelection())
print(stage.GetRootLayer().ExportToString())

```

Expected output

```

sets: ['color']
variants: ['blue', 'green', 'red']
selection: ''
selection: blue
#usda 1.0

def Xform "Chair" (
    variants = {
        string color = "blue"
    }
    prepend variantSets = "color"
)
{
    variantSet "color" = {
        "blue" {

        }
        "green" {

        }
        "red" {

        }
    }
}

```

Notice: the variants were added in the order red, blue, green, but are listed alphabetically. Before you select anything, the selection is the empty string.

Real-world use case. A furniture retailer’s digital catalog stores each sofa once, with a fabric variant set (linen, velvet, leather) and a legs variant set (oak, steel). The web configurator only changes selections; it never copies geometry.

Common mistakes.

MISTAKE

Expecting `GetVariantNames()` to keep the order you added variants. It returns sorted names. If order matters for a UI, store the order yourself (for example in `customData`).

MISTAKE

Writing a variant body on one line in hand-edited USDA (`"red" { int x = 1 }``). The text parser rejects it. Put each opinion on its own line.

MISTAKE

Using a name with a dot, such as `v1.2``, as a variant name. It is rejected. Use `v1_2`` or `v1-2``.

Exam traps.

TRAP

"A variant set is a separate file." No. Variant sets and their variants live inside a layer, on a prim. A variant may **contain** a reference or payload to another file, but the set itself is scene description in the same layer.

TRAP

Confusing ``variantSets`` (the list of set names) with ``variants`` (the selection dictionary). Questions often show both and ask which one switches the look.

Exam takeaways.**KEY**

- Variant set = a named switch on a prim; variant = one position; selection = which position. - ``variantSets`` lists set names; ``variants`` holds selections. Both are prim metadata. - Only the selected variant's opinions compose. Variants are the ****V**** in LIVERPS. - Variant content paths look like ``/Prim{set=variant}``.

18.2 Selections and authoring inside variants

What is it? Authoring inside a variant means writing opinions into one variant instead of onto the prim directly. You do it by selecting the variant and then using a **variant edit target**, usually through `GetVariantEditContext()`.

Why do we need it? Normal authoring writes **direct opinions** on the prim. Those would apply no matter which variant is selected. To make “red” actually mean red, the red color must be stored *inside* the red variant.

Beginner explanation. Imagine a binder with tabbed sections. To write in the “red” section, you first open the binder to the “red” tab, then write. If you forget to open a tab, you write on the binder’s cover, and it shows up whichever tab is open.

Where the analogy breaks: USD does not warn you when you write on the cover. The binder simply ends up with a note on the cover.

Technical explanation. - `vset.SetVariantSelection(name)` authors the selection at the stage’s current edit target (Chapter 15). - `vset.GetVariantEditTarget()` returns a `Usd.EditTarget` that maps prim paths into the selected variant: `/Chair` becomes `/Chair{color=red}`. - `vset.GetVariantEditContext()` returns a Python context manager. Inside the `with` block, the stage’s edit target is that variant target. It is equivalent to `Usd.EditContext(stage, vset.GetVariantEditTarget())`. - The variant edit target always uses the **currently selected** variant. You must select first. - If no variant is selected, the variant edit target is invalid (`IsValid()` is `False`) and `GetVariantEditContext()` leaves the edit target unchanged, so edits land as direct opinions. No error or warning is printed (verified on 26.08). - Selecting a variant name that does not exist is not an error. The selection is authored, but no variant opinions compose. - Attributes you create inside one variant exist only when that variant is selected. Create (not just `Get`) the attribute inside each variant. - `vset.ClearVariantSelection()` removes the authored selection at the current edit target.

Mental model.

Simple example. | Step | Call | Where the opinion lands | |-----|-----|-----| | 1
 | SetVariantSelection("red") | variants metadata on /Chair | | 2 | inside
 GetVariantEditContext(): Set("glossy red") | /Chair{color=red}.paint | | 3 | outside the
 context: Set("primer") | /Chair.paint (direct) |

USDA example.

```
#usda 1.0

def Xform "Chair" (
  variants = {
    string color = "red"
  }
  prepend variantSets = "color"
)
{
  variantSet "color" = {
    "blue" {
      string paint = "matte blue"
    }
    "red" {
      string paint = "glossy red"

      def Sphere "Knob"
      {
      }
    }
  }
}
```

Notes: the red variant adds a child prim Knob. When blue is selected, /Chair/Knob does not exist. Variants can add whole prims, not only values.

Python example.

```
from pxr import Usd, UsdGeom, Sdf

stage = Usd.Stage.CreateInMemory()
chair = UsdGeom.Xform.Define(stage, "/Chair").GetPrim()
color = chair.GetVariantSets().AddVariantSet("color")

paint = {"red": "glossy red", "blue": "matte blue"}
for name, value in paint.items():
    color.AddVariant(name)
    color.SetVariantSelection(name)          # 1. select the variant first
    with color.GetVariantEditContext():      # 2. edits now go inside it
        attr = chair.CreateAttribute("paint", Sdf.ValueTypeNames.String)
        attr.Set(value)

target = color.GetVariantEditTarget()
print("edit target maps /Chair to:", target.MapToSpecPath("/Chair"))
for name in ["red", "blue", "purple"]:
    color.SetVariantSelection(name)
    print(name, "->", chair.GetAttribute("paint").Get())
```

Expected output

```
edit target maps /Chair to: /Chair{color=blue}
red -> glossy red
blue -> matte blue
purple -> None
```

The next script shows the silent mistake: no selection, so the “variant” edit goes onto the prim directly.

```
from pxr import Usd, Sdf

stage = Usd.Stage.CreateInMemory()
chair = stage.DefinePrim("/Chair", "Xform")
color = chair.GetVariantSets().AddVariantSet("color")
color.AddVariant("red")
print("target valid:", color.GetVariantEditTarget().IsValid())
```

```

with color.GetVariantEditContext():          # no selection made!
    chair.CreateAttribute("paint", Sdf.ValueTypeNames.String).Set("red")

layer = stage.GetRootLayer()
print("on /Chair:", list(layer.GetPrimAtPath("/Chair").properties.keys()))
print("in variant:", list(layer.GetPrimAtPath("/Chair{color=red}").properties.keys()))

```

Expected output

```

target valid: False
on /Chair: ['paint']
in variant: []

```

Real-world use case. In a games pipeline, a build script loops over damage variants (intact, dented, wrecked) of a vehicle. For each one it selects the variant, enters the edit context, and references the matching geometry file. Level designers then switch damage states with one selection.

Common mistakes.

MISTAKE

Calling `GetVariantEditContext()` before `SetVariantSelection()`. Edits silently become direct opinions that apply to every variant. Always select first, or check `GetVariantEditTarget().IsValid()`.

MISTAKE

Creating an attribute inside the `red` variant, then calling `chair.GetAttribute("paint").Set(...)` inside the `blue` variant. With `blue` selected the attribute does not exist yet, so `Set` raises an error ("Empty typeName"). Use `CreateAttribute` in every variant.

MISTAKE

Leaving the final selection at whatever the loop set last. After authoring all variants, set the selection you want shipped as the default.

Exam traps.

TRAP

"Selecting a variant that does not exist raises an error." False. The selection is authored and the prim just gets no variant opinions. This is a common cause of "my variant shows nothing" bugs.

TRAP

A question may show `Usd.EditContext(stage, vset.GetVariantEditTarget())`. That is the same as `vset.GetVariantEditContext()`; both are correct.

Exam takeaways.

KEY

- Select first, then author inside `with vset.GetVariantEditContext():`
- The variant edit target maps `/Prim` to `/Prim{set=variant}`
- No selection means edits go onto the prim directly, silently
- An unknown selection is not an error; it just yields no variant opinions.

18.3 Nested variants

What is it? A **nested variant set** is a variant set authored *inside* a variant of another set. For example, each model variant (sedan, truck) can carry its own trim set.

Why do we need it? Some choices only make sense under another choice. A sedan may offer base and sport trims while a truck offers none. Nesting expresses "this option depends on that option" and avoids impossible combinations.

Beginner explanation. A restaurant menu: you first pick “pizza” or “salad”. Only if you pick pizza do you see the “crust” choices. The crust question lives inside the pizza choice. Where the analogy breaks: in USD, nothing stops you from authoring a `trim` selection even when the truck is chosen. The selection is simply ignored because no `trim` set exists there.

Technical explanation. - To nest, select the outer variant, enter its edit context, then call `prim.GetVariantSets().AddVariantSet("trim")`. To author inside an inner variant, select it and enter a second edit context inside the first. - The `Sdf` path of nested content chains the selections: `/Car{model=sedan}{trim=sport}`. - The inner set exists in the composed prim **only** when an outer variant that defines it is selected. `GetVariantSets().GetNames()` changes with the outer selection. - A selection for the inner set can be authored inside the outer variant (a default per model) or directly on the prim. A direct selection is stronger than one authored inside a variant (direct beats variant; see 18.6).

Mental model.

```
/Car
+-- model = sedan ----+-- trim = base   { label = "sedan-base" }
|                      +-- trim = sport { label = "sedan-sport" }
+-- model = truck  ---(no trim set here)
```

Simple example. | `model` | Sets on the prim | Possible `trim` |
|-----|-----|-----| | `sedan` | `model, trim` | `base, sport` | | `truck` | `model` |
none |

USDA example.

```
#usda 1.0

def Xform "Car" (
  variants = {
    string model = "sedan"
  }
  prepend variantSets = "model"
)
{
  variantSet "model" = {
    "sedan" (
      variants = {
        string trim = "base"
      }
      prepend variantSets = "trim"
    ) {
      variantSet "trim" = {
        "base" {
          string label = "sedan-base"
        }
        "sport" {
          string label = "sedan-sport"
        }
      }
    }
    "truck" {
      string label = "truck"
    }
  }
}
```

Notes: the `sedan` variant has its own metadata block (`...`) that declares the nested `trim` set and selects `base` by default. The `truck` variant has no `trim` set at all.

Python example.

```
from pxr import Usd, Sdf

stage = Usd.Stage.CreateInMemory()
car = stage.DefinePrim("/Car", "Xform")
```

```

model = car.GetVariantSets().AddVariantSet("model")

model.AddVariant("sedan")
model.SetVariantSelection("sedan")
with model.GetVariantEditContext():
    trim = car.GetVariantSets().AddVariantSet("trim")
    for t in ["base", "sport"]:
        trim.AddVariant(t)
        trim.SetVariantSelection(t)
        with trim.GetVariantEditContext():
            attr = car.CreateAttribute("label", Sdf.ValueTypeNames.String)
            attr.Set("sedan-" + t)
        trim.SetVariantSelection("base")          # default trim for sedans

model.AddVariant("truck")
model.SetVariantSelection("truck")
with model.GetVariantEditContext():
    car.CreateAttribute("label", Sdf.ValueTypeNames.String).Set("truck")

for m in ["sedan", "truck"]:
    model.SetVariantSelection(m)
    print(m, car.GetVariantSets().GetNames(), car.GetAttribute("label").Get())

model.SetVariantSelection("sedan")
car.GetVariantSet("trim").SetVariantSelection("sport")
print("sedan+sport:", car.GetAttribute("label").Get())
print(stage.GetRootLayer().GetPrimAtPath("/Car{model=sedan}{trim=sport}").path)

```

Expected output

```

sedan ['model', 'trim'] sedan-base
truck ['model'] truck
sedan+sport: sedan-sport
/Car{model=sedan}{trim=sport}

```

Real-world use case. An automotive configurator nests `interior` options inside each `trimLevel` variant, because luxury interiors are only offered on top trims. A digital-twin factory model nests `toolHead` variants inside each `robotModel` variant.

Common mistakes.

MISTAKE

Authoring the inner set while the stage edit target is the plain layer. The ``trim`` set then lives on the prim directly and appears for every model. Create it inside the outer variant's edit context.

MISTAKE

Nesting deeply "just because". Every level multiplies the combinations a reviewer must test. Use independent (sibling) sets when choices really are independent, such as ``color`` and ``size``.

Exam traps.

TRAP

"Nested variant sets are always visible on the prim." No. An inner set appears only when an outer variant that defines it is selected.

TRAP

Sibling sets vs. nested sets: ``color`` and ``size`` on the same prim are **independent** and both always present. Nesting is only for **dependent** choices.

Exam takeaways.

KEY

- Nest by authoring a set inside an outer variant's edit context. - Inner sets exist only under outer variants that define them. - Use nesting for dependent choices, sibling sets for independent ones. - Nested content path: ``/Prim{outer=a}{inner=b}``.

18.4 Fallback selections

What is it? A **fallback selection** is the variant USD chooses for a set when no selection is authored for it in any layer. You register fallbacks for the whole process with `Usd.Stage.SetGlobalVariantFallbacks()`.

Why do we need it? Without any selection, a variant set contributes nothing. An asset might show no geometry at all. Fallbacks let a site or application say “if nobody chose, use `lod = low`”, for example so a lightweight viewer opens heavy scenes quickly.

Beginner explanation. A thermostat that nobody has touched still runs at its factory setting. The factory setting is the fallback. The moment someone turns the dial, their choice wins.

Where the analogy breaks: the USD “factory setting” is not stored in the asset file. It belongs to the program opening the file, so two programs can show different defaults for the same file.

Technical explanation. - `Usd.Stage.SetGlobalVariantFallbacks({"lod": ["medium", "low"]})` takes a dict of set name to an **ordered list** of variant names. USD uses the first name in the list that exists on the prim. - `Usd.Stage.GetGlobalVariantFallbacks()` returns the current dict (empty `{}` by default). - Fallbacks are read **when a stage is opened**. Stages opened before the call keep their old behavior, even after `Reload()`. Set fallbacks first, then open stages. - Any authored selection, in any layer, beats a fallback. `vset.BlockVariantSelection()` authors an empty selection (`string lod = ""`), which hides weaker selections so the fallback applies again. - `vset.GetVariantSelection()` returns the *composed* selection, including a fallback. `prim.GetVariantSets().GetAllVariantSelections()` returns only *authored* selections. - In Python, `vset.HasAuthoredVariantSelection()` returns the authored name, or `None` if nothing is authored (not `True/False`). - Sites can also declare fallbacks in a plugin's `pluginInfo.json` (see Chapter 38, 38.2).

VERSION

Verified on USD 26.08: fallbacks set after opening a stage do not affect that stage; ``BlockVariantSelection()`` exists and authors an empty selection; ``HasAuthoredVariantSelection()`` returns a string or ``None`` in Python.

Mental model.

```

Is a selection authored anywhere (strongest wins)?
|yes                               |no (or blocked with "")
|v                                 |v
use that variant                   global fallback list for this set?
                                   |yes                               |no
                                   |v                                 |v
                                   first listed variant                 no variant applies
                                   that exists on the prim

```

Simple example. | Authored selection | Global fallback for `lod` | Variant used |
|-----|-----|-----| | none | none | none (set contributes nothing)
| | none | ["medium", "low"] | low (no medium on the prim) | | high | ["low"] | high | | ""
(blocked) | ["low"] | low |

USDA example.

```
#usda 1.0
(
```



```

        defaultPrim = "Tree"
    )

    def Xform "Tree" (
        prepend variantSets = "lod"
    )
    {
        variantSet "lod" = {
            "high" {
                int polys = 5000
            }
            "low" {
                int polys = 50
            }
        }
    }
}

```

Notes: there is no `variants = {...}` line, so no selection is authored. What you see depends entirely on the opening program's fallbacks.

Python example.

```

from pxr import Usd

TREE = """"#usda 1.0
def Xform "Tree" (
    prepend variantSets = "lod"
)
{
    variantSet "lod" = {
        "high" {
            int polys = 5000
        }
        "low" {
            int polys = 50
        }
    }
}
"""

with open("tree.usda", "w") as f:
    f.write(TREE)

print("fallbacks:", Usd.Stage.GetGlobalVariantFallbacks())
early = Usd.Stage.Open("tree.usda")
print("no selection:", early.GetPrimAtPath("/Tree").GetAttribute("polys").Get())

Usd.Stage.SetGlobalVariantFallbacks({"lod": ["medium", "low"]})
early.Reload()
print("early stage after setting:", early.GetPrimAtPath("/Tree")
    .GetAttribute("polys").Get())

late = Usd.Stage.Open("tree.usda")
tree = late.GetPrimAtPath("/Tree")
lod = tree.GetVariantSet("lod")
print("stage opened after setting:", tree.GetAttribute("polys").Get())
print("composed selection:", lod.GetVariantSelection())
print("authored selections:", tree.GetVariantSets().GetAllVariantSelections())
print("authored?", lod.HasAuthoredVariantSelection())

lod.SetVariantSelection("high")
print("authored high:", tree.GetAttribute("polys").Get())
lod.BlockVariantSelection()
print("blocked:", tree.GetAttribute("polys").Get(), repr(lod.GetVariantSelection()))

```

Expected output

```

fallbacks: {}
no selection: None
early stage after setting: None
stage opened after setting: 50
composed selection: low
authored selections: {}
authored? None

```

```

authored high: 5000
blocked: 50 'low'

```

Notice: the stage opened *before* `SetGlobalVariantFallbacks` still shows `None`, even after `Reload()`. `medium` does not exist, so the next name in the list, `low`, is used.

Real-world use case. A studio’s review tool sets `{"lod": ["proxy"], "shadingVariant": ["preview"]}` at startup, before opening any stage, so animators get fast scenes. The final render farm process sets no fallbacks and relies on the selections authored in the shot.

Common mistakes.

MISTAKE

Calling `SetGlobalVariantFallbacks()` after the stage is open and wondering why nothing changed. Call it at application startup, before `Usd.Stage.Open()`.

MISTAKE

Using `GetAllVariantSelections()` to find out what is displayed. It lists only authored selections and misses fallbacks. Use `GetVariantSelection()` on each set.

MISTAKE

Testing `if vset.HasAuthoredVariantSelection() == True:`. In Python it returns a name or `None`. Test `is not None`.

Exam traps.

TRAP

"Fallbacks are stored in the asset file." No. They come from the application process (or plugins). An authored `variants = {...}` in the file is a selection, not a fallback.

TRAP

"A fallback beats a weak-layer selection." No. Any authored selection beats a fallback. Only a blocked (empty) selection lets the fallback through.

Exam takeaways.

KEY

- `Usd.Stage.SetGlobalVariantFallbacks({set: [ordered names]})` — set it **before** opening stages. - First existing name in the list wins; any authored selection beats every fallback. - `BlockVariantSelection()` authors `""` so the fallback applies again. - `GetVariantSelection()` = composed (includes fallback); `GetAllVariantSelections()` = authored only.

18.5 When variants are / are not appropriate

What is it? This section is a decision guide (Obj 1.7): which kinds of variation belong in a variant set, and which belong in another tool such as separate assets, payloads, sublayers, inherits, or primvars.

Why do we need it? Variants are easy to add, so people overuse them. Misused variants bloat files, break instancing, multiply test combinations, and confuse downstream users. Choosing the right tool keeps assets small and predictable.

Beginner explanation. A switch is perfect for “one of a few fixed choices” (off, low, high). It is a poor fit for “any value on a dial” (exact temperature) or “a different appliance entirely”. Use a variant like a switch: a small number of named, discrete, mutually exclusive alternatives of the same thing.

Where the analogy breaks: a variant can hold a lot of data, including references and payloads, so “small switch” describes the number of choices, not their size.

Technical explanation. Variants fit when the alternatives are: - **Discrete and named:** lod = high/medium/low, look = clean/dirty, model = sedan/truck. - **Mutually exclusive:** only one per set at a time. - **Versions of the same asset:** same role and interface (same prim path, same purpose). - **Chosen per use, not per frame:** a selection cannot be animated; it is metadata, not a time-sampled attribute.

Variants are a poor fit when: - **The values are continuous or per-instance random** (a color per tree in a forest). Use primvars or attributes on the instance instead (Chapter 11). - **The alternatives must change over time.** Selections cannot be time-sampled; use visibility or attribute animation. - **The choices are independent and many.** 5 colors × 4 sizes × 3 materials as one set is 60 variants; use separate sibling sets, or better, parameters. - **Different departments own the alternatives.** Use separate layers (sublayers, Chapter 15) so people do not edit the same file. - **Instancing matters at scale.** Instances with different selections compose differently, so each distinct selection creates a separate prototype (Chapter 24). Many selections means many prototypes. - **You only want to avoid loading data.** That is the job of payloads (Chapter 17). A variant may contain a payload, but variant sets themselves do not defer loading. - **The alternatives are unrelated assets.** A chair and a lamp should be two assets, not variants.

All variants are stored in the file even when unused, so a set with 30 heavy variants makes the asset file large. Put heavy per-variant data behind references or payloads inside each variant.

Mental model.

Is it a small set of named, exclusive versions of ONE asset, chosen per use (not per frame), owned by one team?	
yes	no
v	v
VARIANT SET	continuous/per-instance -> primvar/attribute
	changes over time -> animated attribute
	defer loading only -> payload
	separate owners -> sublayers
	unrelated things -> separate assets
	shared tweaks to many -> inherits (Ch 19)

Simple example. | Need | Best tool | |-----|-----| | Tree at high/medium/low detail |
Variant set lod | | 10 000 trees, each a slightly different green | Primvar on each instance
| | Light turns on at frame 100 | Animated attribute | | Don't load the city until needed |
Payload | | Lighting and animation teams edit the same shot | Separate sublayers |

USDA example. A good pattern: each lod variant holds only a payload to its own geometry file, so the asset file stays tiny.

```
#usda 1.0
(
  defaultPrim = "Tree"
)

def Xform "Tree" (
  kind = "component"
  variants = {
    string lod = "low"
  }
  prepend variantSets = "lod"
)
{
  variantSet "lod" = {
    "high" (
      prepend payload = @tree_high.usda@
    ) {
    }
    "low" (
      prepend payload = @tree_low.usda@
    ) {
    }
  }
}
```

```

    }
  }
}

```

Notes: arcs such as `payload` and `references` can be authored inside a variant's metadata block. The payload files are not needed to parse this layer.

Python example. Different selections on instanceable prims produce different prototypes (Chapter 24 covers instancing in depth):

```

from pxr import Usd

ROCK = """#usda 1.0
(
  defaultPrim = "Rock"
)
def Xform "Rock" (
  variants = {
    string shape = "round"
  }
  prepend variantSets = "shape"
)
{
  variantSet "shape" = {
    "flat" {
      def Cube "Geo"
      {
      }
    }
    "round" {
      def Sphere "Geo"
      {
      }
    }
  }
}
}
"""

with open("rock.usda", "w") as f:
    f.write(ROCK)

stage = Usd.Stage.CreateInMemory()
for i in range(6):
    rock = stage.DefinePrim(f"/World/Rock{i}")
    rock.GetReferences().AddReference("rock.usda")
    rock.SetInstanceable(True)
    if i % 2:
        rock.GetVariantSet("shape").SetVariantSelection("flat")

print("instances:", 6)
print("prototypes:", len(stage.GetPrototypes()))

```

Expected output

```

instances: 6
prototypes: 2

```

With 2 selections you get 2 prototypes. If every rock had its own variant, instancing would save nothing.

Real-world use case. An AEC firm models a building's window module with a `glazing` variant set (`single`, `double`, `triple`). That is a good fit: few named, exclusive options. The exact tint per window, chosen by a solar study, is stored as a `primvar` instead, because it is continuous and differs per window.

Common mistakes.

MISTAKE

Trying to animate a variant selection over time. Selections are metadata and have no time samples. Animate visibility or attributes instead.

MISTAKE

Packing heavy geometry for 20 variants directly into one file. Every variant is stored and parsed. Reference or payload each variant's data from its own file.

MISTAKE

Giving each of thousands of instances its own variant selection "for variety". It splits instancing into many prototypes. Vary instances with primvars or a few shared selections.

Exam traps.**TRAP**

"Use a variant set so the heavy version is not loaded." Loading control is a ****payload**** feature. A variant only changes *which* opinions compose; combine both if you need both.

TRAP

Look for "per frame", "continuous", "per instance", or "different team" in a scenario. Each of those words usually points away from variants.

Exam takeaways.**KEY**

- Variants: few, named, mutually exclusive versions of one asset, chosen per use. - Not variants: per-frame changes, continuous or per-instance values, separate owners, unrelated assets. - Selections cannot be animated. - Each distinct selection on instances means another prototype. - Variants select; payloads defer loading. Combine them for heavy LODs.

18.6 Variant selection strength surprises

What is it? This section explains how variant opinions and variant selections compete with other opinions. Two separate questions are answered by strength: **which variant is selected**, and **does a variant opinion beat other opinions**.

Why do we need it? The most common variant bug is "I switched the variant and nothing changed". It is almost always a strength problem: a direct opinion somewhere is stronger than the variant's opinion. Knowing the rules lets you diagnose it in seconds (Obj 1.1, 1.6, and 1.8 in Chapter 22).

Beginner explanation. Recall the ladder of opinions: the strongest rung answers. A variant's opinions sit on a rung *below* the direct opinions of the same layer stack (the stack of sublayers from Chapter 15). A sticky note written directly on the prim beats anything in the variant binder, even if the binder sits on a higher sheet of the projector stack.

Where the analogy breaks: when a whole asset is referenced, its entire ladder hangs *below* the referencing layer stack's ladder. Then a variant in the shot beats a direct opinion inside the asset.

Technical explanation. Composition evaluates LIVERPS **per layer stack** (Chapter 14, Chapter 21). Four rules follow:

1. **Direct beats variant within one layer stack.** All direct (L) opinions from every layer of a layer stack are stronger than variant (V) opinions in that layer stack. This holds even when the variant is in a stronger sublayer and the direct opinion is in a weaker sublayer.
2. **The selection is an ordinary opinion.** The variants metadata resolves like any value: the strongest authored selection wins. A shot layer's selection beats the asset's own selection; a stronger sublayer's selection beats a weaker one; the session layer beats them all.

3. A stronger layer stack's variant beats a weaker layer stack's direct opinion.

A variant authored in the shot (on the referencing prim) beats a direct opinion inside the referenced asset, because the asset is a reference (R), evaluated below the shot's L, I, and V.

4. An asset's direct opinion hides its own variant. If the asset author left a direct value on the prim, switching that asset's variant changes nothing for that property.**Mental model.**

STRONGEST		
shot layer stack:	L (direct, all sublayers)	rule 1: L > V
	I	
	V (variants authored in shot)	rule 3: beats asset's L
	R --> asset layer stack:	
	L (asset direct)	rule 4: hides asset V
	I	
	V (asset variants)	
	...	
WEAKEST		
Selection = strongest authored `variants` value		rule 2

Simple example. | Where the opinions are | Winner | |-----|-----| | Shot weak sublayer direct finish = A; shot strong sublayer variant finish = B | A (rule 1) | | Asset selects color = red; shot selects color = blue | blue (rule 2) | | Shot variant tag = S; asset direct tag = D | S (rule 3) | | Asset direct tag = D; asset variant tag = V | D (rule 4) |

USDA example. File: asset.usda

```
#usda 1.0
(
    defaultPrim = "Chair"
)

def Xform "Chair" (
    variants = {
        string color = "red"
    }
    prepend variantSets = "color"
)
{
    string tag = "asset-direct"
    variantSet "color" = {
        "blue" {
            string paint = "blue"
            string tag = "from-blue"
        }
        "red" {
            string paint = "red"
            string tag = "from-red"
        }
    }
}
```

File: strong.usda (the stronger sublayer of the shot)

```
#usda 1.0

over "Chair" (
    variants = {
        string color = "blue"
        string fit = "big"
    }
    prepend variantSets = "fit"
)
{
    variantSet "fit" = {
        "big" {
            string finish = "strong-variant"
            string tag = "shot-variant"
        }
    }
}
```

```

    }
}

```

The shot root layer `shot.usda` sublayers `strong.usda` then `weak.usda`, and its `/Chair` references `asset.usda`. `weak.usda` holds one direct opinion: `string finish = "weak-direct"`.

Python example.

```

from pxr import Usd

FILES = {
    "asset.usda": """#usda 1.0
(
    defaultPrim = "Chair"
)
def Xform "Chair" (
    variants = {
        string color = "red"
    }
    prepend variantSets = "color"
)
{
    string tag = "asset-direct"
    variantSet "color" = {
        "blue" {
            string paint = "blue"
            string tag = "from-blue"
        }
        "red" {
            string paint = "red"
            string tag = "from-red"
        }
    }
}
""",
    "weak.usda": """#usda 1.0
over "Chair" (
    variants = {
        string color = "red"
    }
)
{
    string finish = "weak-direct"
}
""",
    "strong.usda": """#usda 1.0
over "Chair" (
    variants = {
        string color = "blue"
        string fit = "big"
    }
    prepend variantSets = "fit"
)
{
    variantSet "fit" = {
        "big" {
            string finish = "strong-variant"
            string tag = "shot-variant"
        }
    }
}
""",
    "shot.usda": """#usda 1.0
(
    subLayers = [@strong.usda@, @weak.usda@]
)
over "Chair" (
    prepend references = @asset.usda@
)
{
}

```

```

"""
}
for name, text in FILES.items():
    with open(name, "w") as f:
        f.write(text)

shot = Usd.Stage.Open("shot.usda")
chair = shot.GetPrimAtPath("/Chair")
print("rule 2 color selection:", chair.GetVariantSet("color").GetVariantSelection())
print("    paint:", chair.GetAttribute("paint").Get())
print("rule 1 finish:", chair.GetAttribute("finish").Get())
print("rule 3 tag:", chair.GetAttribute("tag").Get())
for spec in chair.GetAttribute("finish").GetPropertyStack():
    print("    finish stack:", spec.layer.identifier.split("/")[-1], spec.path)

asset = Usd.Stage.Open("asset.usda")
achair = asset.GetPrimAtPath("/Chair")
for choice in ["red", "blue"]:
    achair.GetVariantSet("color").SetVariantSelection(choice)
    print(f"rule 4 asset alone, {choice}: tag =", achair.GetAttribute("tag").Get())

```

Expected output

```

rule 2 color selection: blue
    paint: blue
rule 1 finish: weak-direct
rule 3 tag: shot-variant
    finish stack: weak.usda /Chair.finish
    finish stack: strong.usda /Chair{fit=big}.finish
rule 4 asset alone, red: tag = asset-direct
rule 4 asset alone, blue: tag = asset-direct

```

Read the `finish` property stack (strongest first, Chapter 22): the weak sublayer's direct spec comes *before* the strong sublayer's variant spec.

Real-world use case. A layout artist sets `lod = high` in the shot, but one tree stays low-poly. Inspecting the property stack shows the asset author left a direct `points` opinion on the prim, outside the variants (rule 4). The fix is in the asset: move the direct opinion into the variants. Meanwhile, a lighting artist overrides the tree's look by authoring a `look` variant in the shot's layer; it wins over the asset's direct values (rule 3).

Common mistakes.

MISTAKE

Leaving "default" values directly on an asset prim *and* inside its variants. The direct value always wins in that layer stack, so the variant appears broken. Keep switchable properties only inside the variants.

MISTAKE

Putting a variant in a stronger sublayer and expecting it to beat a direct opinion in a weaker sublayer. Within one layer stack, direct beats variant regardless of sublayer order. Author a direct opinion in the stronger sublayer instead.

MISTAKE

Changing the selection in the asset file and expecting the shot to update, when the shot already authors its own selection. The shot's selection is stronger; change it there (or remove it).

Exam traps.

TRAP

"The stronger sublayer always wins." Only when comparing opinions of the same arc type. Direct opinions in any sublayer beat variant opinions in any sublayer of the same layer stack.

TRAP

"Variants are weaker than local opinions, so a shot variant cannot beat an asset's local value." It can. "Local" means local to the *same* layer stack. The referenced asset's local opinions are inside the reference arc, which is weaker than the shot's variants.

TRAP

Selection strength and variant-opinion strength are two separate questions. First find the winning selection, then compare that variant's opinions with the other opinions.

Exam takeaways.**KEY**

- Within one layer stack: direct (L) beats variant (V), whatever the sublayer order. - The selection itself resolves by normal strength; the strongest authored selection wins. - A variant in the referencing layer stack beats direct opinions inside the referenced asset. - An asset's direct opinion hides that asset's own variant opinions for that property. - Debug with ``GetPropertyStack()``: it lists specs strongest first.

Chapter lab(s)

Lab 16 — Variant sets: create, author, select, nest (python-labs/lab16_*.md, Obj 1.7). You build a prop asset with `color` and `lod` variant sets, author inside each variant with edit contexts, nest a `trim` set, then reference the asset into a shot and switch selections from the shot layer. Finally you register a global fallback before opening the stage and confirm which variant appears.

USDA reading exercises

Exercise 18-A. One layer:

```
#usda 1.0

def "Lamp" (
    variants = {
        string power = "eco"
    }
    prepend variantSets = "power"
)
{
    int watts = 60
    variantSet "power" = {
        "bright" {
            int watts = 100
        }
        "eco" {
            int watts = 9
            bool led = 1
        }
    }
}
```

What are the composed values of `watts` and `led`?

Exercise 18-B. Two files. `prop.usda` defines `/Prop` with variant set `size` (`small: double scale = 0.5`; `large: double scale = 2`) and selection `small`. The shot:

```
#usda 1.0

def "Prop" (
    prepend references = @prop.usda@
    variants = {
        string size = "large"
    }
)
{
}
```

What is `scale` on `/Prop` in the shot? What would it be if the shot's `variants` line were `string size = ""`, the process had fallbacks `{"size": ["small"]}`, and the stage were opened after the fallbacks were set?

Chapter review

Summary

- A variant set is a named switch on a prim; each variant stores opinions; only the selected variant composes.
- `variantSets` lists set names; `variants` holds the selection dictionary.
- Author into a variant by selecting it, then using `GetVariantEditContext()`; with no selection, edits silently become direct opinions.
- Nested sets live inside an outer variant and exist only when that variant is selected.
- `Usd.Stage.SetGlobalVariantFallbacks()` must be called before opening a stage; any authored selection beats a fallback.
- `BlockVariantSelection()` authors `""` so fallbacks apply again.
- Use variants for a few named, exclusive versions of one asset; not for animation, continuous or per-instance values, or loading control.
- Within one layer stack, direct opinions beat variant opinions, whatever the sublayer order.
- The strongest authored selection wins; a shot's variant beats direct opinions inside a referenced asset.

If you see... → think...

If you see...	Think...
<code>variants = { string x = "..."} </code>	The selection (prim metadata)
<code>/Prim{set=variant}</code>	Path to a variant's contents
"Switching the variant does nothing"	A direct opinion is stronger (check <code>GetPropertyStack</code>)
Edits made "in a variant" show in every variant	No selection before <code>GetVariantEditContext()</code>
Fallbacks have no effect	They were set after the stage was opened
"Animate the variant"	Not possible; selections are not time-sampled
"Avoid loading heavy data"	Payload (possibly inside a variant)
Thousands of instances, each its own variant	Too many prototypes; use primvars
Nested set "missing"	The outer variant that defines it is not selected

Review questions

R18-01 · Obj 1.7 · Single choice Which situation is the best fit for a variant set? A. A robot arm's joint angle sampled every frame B. A product shipped in `standard`, `pro`, and `max` editions C. A random scale for each of 100 000 pebbles D. Two departments editing the same asset at the same time

R18-02 · Obj 1.1 · Single choice In one layer stack, `strong.usda` (stronger sublayer) authors `radius = 5` inside the selected variant, and `weak.usda` authors a direct `radius = 1` on the same prim. What is `radius`? A. 5, because the stronger sublayer wins B. 1, because direct opinions beat variant opinions in the same layer stack C. 5, because variants are evaluated last D. A composition error is reported

R18-03 · Obj 1.7 · Select two. Which statements about variant selections are true? A. A selection can be time-sampled to switch at frame 50. B. A selection is prim metadata resolved by normal strength ordering. C. A selection naming a variant that does not exist raises an exception. D. A shot layer can select a variant defined inside a referenced asset. E. Selections can only be authored in the asset's root layer.

R18-04 · Obj 1.6 · Python reading

```
vset = prim.GetVariantSets().AddVariantSet("lod")
vset.AddVariant("high")
with vset.GetVariantEditContext():
    prim.CreateAttribute("polys", Sdf.ValueTypeNames.Int).Set(9000)
```

Where is the polys opinion stored? A. In /Prim{lod=high} B. Directly on /Prim C. In the session layer D. Nowhere; the edit raises an error

(This fragment is not run: it assumes an existing prim.)

R18-05 · Obj 1.7 · Single choice A viewer should show lod = proxy for any asset whose files author no lod selection. What should it do? A. Author variants = {lod = "proxy"} into every asset file B. Call Usd.Stage.SetGlobalVariantFallbacks({"lod": ["proxy"]}) before opening stages C. Call SetGlobalVariantFallbacks after opening, then Reload() D. Use BlockVariantSelection() on every prim

R18-06 · Obj 1.1 · Single choice An asset's /Tree has a direct string season = "summer" and also season values inside each look variant. A shot references the asset and selects look = "winter" but authors nothing else. What is season? A. winter B. summer C. Empty, because the opinions conflict D. Whatever the global fallback says

R18-07 · Obj 1.1 · Single choice Same asset as R18-06. The shot instead authors, on its own /Tree, a variant set shotLook with selected variant snow containing season = "snow". What is season? A. summer, because local opinions beat variants B. snow, because the shot's variant is in a stronger layer stack than the asset's direct opinion C. winter D. A composition error

R18-08 · Obj 1.7 · Select two. Which are good reasons *not* to use a variant set? A. The alternatives must change over the course of an animation. B. There are three named, mutually exclusive looks. C. Each of many instances needs a unique value. D. The alternatives are versions of the same asset with the same interface.

R18-09 · Obj 1.7 · Single choice Fallbacks are {"lod": ["medium", "low"]}. A prim has lod variants high and low; the weakest sublayer selects high. What is used? A. medium B. low C. high D. No variant

R18-10 · Obj 1.7 · USDA reading

```
#usda 1.0

def "Bot" (
    variants = {
        string arm = "claw"
    }
    prepend variantSets = "arm"
)
{
    variantSet "arm" = {
        "claw" (
            prepend variantSets = "grip"
        ) {
            variantSet "grip" = {
                "soft" {
                    int force = 1
                }
            }
        }
        "drill" {
            int rpm = 3000
        }
    }
}
```

```

    }
  }
}

```

Which variant sets does /Bot report, and what is force? A. arm and grip; force is 1 B. arm and grip; force has no value C. Only arm; force is 1 D. Only arm; force has no value

R18-11 · Obj 1.2 · Single choice Eight instanceable prims reference the same asset. Four select shape = a, four select shape = b. How many prototypes does the stage create? A. 1 B. 2 C. 4 D. 8

R18-12 · Obj 1.6 · Single choice Which LIVERPS position do variant opinions occupy relative to inherits and references? A. Weaker than references, stronger than payloads B. Stronger than inherits, weaker than local C. Weaker than inherits, stronger than references D. Weakest of all arcs

Answers

USDA reading exercises

18-A. watts = 60, led = 1 (true). The direct watts = 60 beats the eco variant's watts = 9 (direct beats variant in the same layer stack). led has no direct opinion, so the selected eco variant supplies it.

18-B. scale = 2. The shot's selection large is stronger than the asset's small. With string size = "" in the shot, the empty selection is the strongest authored opinion and blocks the asset's small, so no authored selection applies. The fallback small then gives scale = 0.5. (If no fallback were set, there would be no scale at all.)

Review questions

R18-01 — B. A few named, exclusive editions of one product. A is per-frame animation, C is per-instance continuous data (primvars), D is a layering problem (sublayers). Review: §18.5.

R18-02 — B. Within one layer stack, all direct opinions beat all variant opinions. Sublayer order only decides between opinions of the same kind. Review: §18.6.

R18-03 — B, D. Selections are metadata (no time samples), resolved by strength, and can target sets defined in a referenced asset. An unknown name is not an error. Review: §18.2, §18.6.

R18-04 — B. No variant was selected, so the variant edit target is invalid and the edit lands as a direct opinion, silently. Review: §18.2.

R18-05 — B. Fallbacks come from the process and must be set before opening. A changes assets; C does not affect an already-open stage; D only removes selections. Review: §18.4.

R18-06 — B. The asset's direct opinion beats the asset's own variant opinions (rule 4). The shot's selection does switch the variant, but the variant's season is weaker. Review: §18.6.

R18-07 — B. The shot's variant opinion sits in the shot layer stack, which is evaluated before the reference arc that brings in the asset (rule 3). Review: §18.6.

R18-08 — A, C. Selections cannot animate, and per-instance unique values belong in primvars or attributes. B and D describe good variant use. Review: §18.5.

R18-09 — C. Any authored selection, even in the weakest sublayer, beats fallbacks. Review: §18.4.

R18-10 — B. claw is selected, so the nested grip set exists. But no grip selection is authored (and no fallback is given), so the soft variant does not apply and force has no value. Review: §18.3, §18.4.

R18-11 — B. Instances that compose identically share a prototype; two distinct selections give two prototypes. Review: §18.5.

R18-12 — C. LIVERPS: Local, Inherits, Variants, rElocates, References, Payloads, Specializes. Review: §18.1, Chapter 21.

CHAPTER 19

Chapter 19 — Inherits, Classes, and Specializes

Exam domain: Composition (23%) · **Objectives:** 1.1, 1.6 · **Study day:** 6 · **Est. time:** 90 min **Prerequisites:** Ch 4 (specifiers `def`, `over`, `class`), Ch 14 (strength, LIVERPS first look), Ch 15 (layer stacks), Ch 16 (references), Ch 18 (variants)

Learning goals

- Explain what a `class prim` is and why it is abstract.
- Author inherits arcs and use a class to broadcast one edit to many prims.
- Explain specializes as the weakest arc and predict when its opinions win or lose.
- Choose between inherits and specializes with a decision table (Obj 1.1).
- Predict how a class override in a shot reaches every referenced copy of an asset (implied inherits).
- Place inherits (**I**) and specializes (**S**) correctly in LIVERPS (Obj 1.6).

Key terms

Term	One-line definition
class prim	A prim with the <code>class</code> specifier. It is abstract: it holds opinions but is not traversed or rendered by default.
Abstract prim	A prim that is a <code>class</code> or is under one. <code>prim.IsAbstract()</code> returns <code>True</code> .
Inherits arc	A composition arc that makes a prim pick up all opinions from another prim path in the same layer stack. The I in LIVERPS.
Broadcast edit	One edit on a class that changes every prim inheriting from it.
Specializes arc	Like inherits, but its opinions are the weakest of all arcs. The S in LIVERPS.
Base / source prim	The prim an inherits or specializes arc points to.
Implied inherits	When an asset with an inherits arc is referenced, the class path is also looked up in the referencing layer stack, so the shot can override the class.
Direct (local) opinion	An opinion authored on the prim itself in a layer stack, not through any arc.

19.1 class prims

What is it? A `class prim` is a prim written with the `class` specifier instead of `def` or `over`. It is a container of shared opinions that other prims point to. USD treats it as **abstract**: it exists on the stage, but default traversal and renderers skip it.

Why do we need it? You want one place to store settings shared by many prims, for example “all street lamps are 60 W”. If that place were a normal `def` prim, it would also appear in the scene as an extra lamp. A `class prim` holds the settings without becoming visible content.

Beginner explanation. A class is like a style sheet in a word processor: the “Heading” style is not a heading on any page, but every heading follows it.

Where the analogy breaks: a style sheet only formats text, while a class can also contain child prims, relationships, variant sets, and even a prim type.

Technical explanation. - Specifier `Sdf.SpecifierClass`. Create it with `stage.CreateClassPrim(path)` in Python or `class "Name"` in USDA. - A class prim **is defined** (`IsDefined()` is `True`) and **is abstract** (`IsAbstract()` is `True`). Its descendants are abstract too. - `stage.Traverse()` uses the default predicate, which excludes abstract

prims. `stage.TraverseAll()` includes them. The predicate term is `Usd.PrimIsAbstract`. - A class can have a type, for example `class Xform "_class_Lamp"`. A typeless prim that inherits from it then composes the type `Xform` (verified on 26.08). - Naming convention: prefix class names with `_class_` (for example `_class_Tree`) and place them at the root of the layer. This is a convention, not a rule. - Any prim can be the target of an inherits arc, even a `def`. Using `class` just keeps the source out of the rendered scene.

Mental model.

```
Stage
+-- class "_class_Lamp"      abstract: skipped by Traverse(), not rendered
|   +-- Bulb                abstract too (child of a class)
+-- def "LampA"  --inherits--> /_class_Lamp
+-- def "LampB"  --inherits--> /_class_Lamp
```

Simple example. | Prim | Specifier | IsAbstract() | In Traverse()? |

```
|-----|-----|-----|-----| | /_class_Lamp | class | True | No | | /_class_Lamp/
Bulb | def | True | No | | /LampA | def | False | Yes |
```

USDA example.

```
#usda 1.0

class Xform "_class_Lamp"
{
    int watts = 60

    def Sphere "Bulb"
    {
    }
}

def "LampA" (
    prepend inherits = </_class_Lamp>
)
{
}
```

Line by line: - `class Xform "_class_Lamp"` — a typed class prim holding shared opinions. - `def Sphere "Bulb"` inside the class — every inheriting prim gets a `Bulb` child. - `prepend inherits = </_class_Lamp>` — the inherits arc. The target is a **prim path** in angle brackets, not a file path.

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
cls = stage.CreateClassPrim("/_class_Lamp")
UsdGeom.Sphere.Define(stage, "/_class_Lamp/Bulb")

lamp = stage.DefinePrim("/LampA")
lamp.GetInherits().AddInherit("/_class_Lamp")

print("specifier:", cls.GetSpecifier())
print("abstract:", cls.IsAbstract(), "defined:", cls.IsDefined())
print("child abstract:", stage.GetPrimAtPath("/_class_Lamp/Bulb").IsAbstract())
print("Traverse:", [str(p.GetPath()) for p in stage.Traverse()])
print("TraverseAll:", [str(p.GetPath()) for p in stage.TraverseAll()])
print("LampA/Bulb type:", stage.GetPrimAtPath("/LampA/Bulb").GetTypeName())
```

Expected output

```
specifier: Sdf.SpecifierClass
abstract: True defined: True
child abstract: True
Traverse: ['/LampA', '/LampA/Bulb']
TraverseAll: ['/_class_Lamp', '/_class_Lamp/Bulb', '/LampA', '/LampA/Bulb']
LampA/Bulb type: Sphere
```

Real-world use case. A film studio’s set-dressing layer defines `class` `"_class_StreetLamp"`. Hundreds of lamps on a street inherit it. The class never renders, but every lamp gets its bulb child and default settings from it.

Common mistakes.

MISTAKE

Writing the shared source as ``def`` and then seeing an extra copy in the render. Use ``class`` so the source is abstract.

MISTAKE

Looking for a class with ``stage.Traverse()`` and concluding it does not exist. Use ``stage.GetPrimAtPath()`` or ``stage.TraverseAll()``.

Exam traps.

TRAP

"A ``class`` prim is not defined." It *is* defined (``class`` is a defining specifier like ``def``). It is **abstract**, which is a different property. ``over`` is the specifier that does not define a prim.

TRAP

"Inherits can only target ``class`` prims." Any prim path works. ``class`` is a convention that keeps the source out of the scene.

Exam takeaways.

KEY

- ``class`` specifier → abstract prim: defined, but skipped by ``Traverse()`` and renderers. - Descendants of a class are abstract too. - Convention: root-level ``_class_Name``. - Inherits/specializes target prim paths (````), never asset paths.

19.2 Inherits and broadcast edits

What is it? An **inherits arc** makes a prim pick up every opinion of another prim (usually a class). Because the link is live, one edit on the class is a **broadcast edit**: it reaches every inheriting prim.

Why do we need it? Without inherits, changing “all lamps to 60 W” means editing hundreds of prims and hoping you found them all. With inherits, you change the class once. Individual prims can still override the shared value locally.

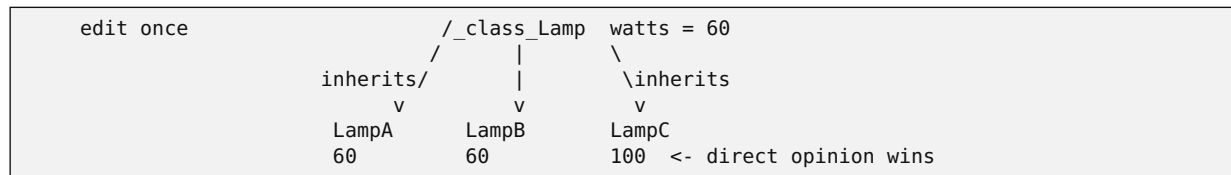
Beginner explanation. A school dress code is the class. Every student follows it, and when the school changes the rule, everyone follows the new rule tomorrow. A student with a doctor’s note (a local opinion) can still wear something different.

Where the analogy breaks: in USD the “note” always wins for that student. There is no way for the class to force a value over a direct opinion in the same layer stack.

Technical explanation. - Python: `prim.GetInherits().AddInherit(path)`; also `RemoveInherit`, `SetInherits`, `ClearInherits`, `GetAllDirectInherits()`. `prim.HasAuthoredInherits()` tells whether any are authored. - USDA: `inherits = </Path>` with list-editing (prepend, append, delete; Chapter 14). Python’s `AddInherit` writes `prepend` by default. - Inherits are **I** in LIVERPS: weaker than local opinions, stronger than variants, references, payloads, and specializes. - So in one layer stack: a prim’s direct opinion beats its class; the class beats the prim’s variant opinions and anything coming through its references. - With several inherits, the first listed is strongest: `inherits = [</A>, </B>]` → `_A` wins where both have opinions. - The target path is resolved in the

same layer stack. If nothing is authored at that path, nothing is inherited, and no composition error is reported. This lets a stronger layer add the class later. - Classes can include child prims; every inheriting prim gets those children (namespace inheritance).

Mental model.



Simple example. | Step | /_class_Lamp.watts | LampA | LampB | LampC (direct 100) |
 |-----|-----|-----|-----|-----| | Start | 40 | 40 | 40 | 100 | | Edit class to
 60 | 60 | 60 | 60 | 100 |

USDA example.

```

#usda 1.0

class "_class_Lamp"
{
    int watts = 60
}

def Xform "LampA" (
    prepend inherits = </_class_Lamp>
)
{
}

def Xform "LampC" (
    prepend inherits = </_class_Lamp>
    variants = {
        string mode = "eco"
    }
    prepend variantSets = "mode"
)
{
    int watts = 100
    variantSet "mode" = {
        "eco" {
            int watts = 9
        }
    }
}
  
```

Notes: `LampA.watts` is 60 (from the class). `LampC.watts` is 100: its direct opinion beats the class, and the class would also beat the `eco` variant's 9 (I is stronger than V).

Python example.

```

from pxr import Usd, Sdf

stage = Usd.Stage.CreateInMemory()
cls = stage.CreateClassPrim("/_class_Lamp")
cls.CreateAttribute("watts", Sdf.ValueTypeNames.Int).Set(40)

lamps = []
for name in ["LampA", "LampB", "LampC"]:
    lamp = stage.DefinePrim(f"/{name}", "Xform")
    lamp.GetInherits().AddInherit("/_class_Lamp")
    lamps.append(lamp)
lamps[2].CreateAttribute("watts", Sdf.ValueTypeNames.Int).Set(100)

def report(label):
    values = [(p.GetName(), p.GetAttribute("watts").Get()) for p in lamps]
    print(label, values)

report("before:")
cls.GetAttribute("watts").Set(60)          # one broadcast edit
  
```

```
report("after: ")
print("LampA inherits:", lamps[0].GetInherits().GetAllDirectInherits())
```

Expected output

```
before: [('LampA', 40), ('LampB', 40), ('LampC', 100)]
after:  [('LampA', 60), ('LampB', 60), ('LampC', 100)]
LampA inherits: [Sdf.Path('/_class_Lamp')]
```

Real-world use case. A crowd of 2 000 soldiers inherits `_class_Soldier`. The art director decides all helmets should be darker. One edit to the class's helmet color updates every soldier, while the hero soldier, who has a direct override, keeps his unique look.

Common mistakes.

MISTAKE

Editing a class and expecting it to beat a prim's own direct opinion. Direct opinions in the same layer stack always win. Remove the direct opinion if the class should control it.

MISTAKE

Writing a file path as the inherit target (``inherits = @lib.usda@``). Inherits take only a prim path. To get a class from another file, reference or sublayer that file so the class is in your layer stack.

MISTAKE

Expecting an error when the class path is misspelled. Nothing is reported; the prim simply inherits nothing. Check ``GetAllDirectInherits()`` and the class path.

Exam traps.

TRAP

"Inherits copy values at authoring time." No. The arc is live; later edits to the class appear immediately in every inheritor.

TRAP

"Variants beat inherits because variants are on the prim itself." No. LIVERPS: I comes before V. A class opinion beats a variant opinion in the same layer stack.

Exam takeaways.

KEY

- Inherits = live link to another prim's opinions; edit the class once, every inheritor updates.
- LIVERPS: L > **I** > V > R > P > S. Direct opinions beat the class; the class beats variants, references, payloads.
- First listed inherit is strongest.
- Missing class path = silently nothing inherited.

19.3 Specializes as weakest fallback

What is it? A **specializes arc** works like inherits: a prim gets opinions from a base prim and receives broadcast edits. The difference is strength: specializes opinions are the **weakest of all**, weaker even than the prim's references and payloads, and weaker than the prim's own opinions wherever they come from.

Why do we need it? Sometimes you want "a default that anything else can override". Example: a material library has a generic `Metal` and a `Gold` that refines it. A shot may adjust `Metal` for every metal, but must never override what makes `Gold` gold. Inherits cannot guarantee that (Section 19.4 shows why); specializes can.

Beginner explanation. A recipe card says “season to taste: 1 tsp salt”. That is a default. Anything the cook writes on their own copy of the recipe wins, and so does anything a guest chef adds. The default only fills in what nobody else decided.

Where the analogy breaks: if the *recipe author* updates the default card, every cook’s copy that did not decide on salt gets the new amount. The fallback is still live, like inherits.

Technical explanation. - Python: `prim.GetSpecializes().AddSpecialize(path);` also `RemoveSpecialize`, `SetSpecializes`, `ClearSpecializes`. `prim.HasAuthoredSpecializes()`. - USDA: `specializes = </Path>` with the usual list-editing forms. - Specializes is **S**, last in LIVERPS. Within one layer stack: `local > inherits > variants > references > payloads > specializes`. - Specializes opinions also stay weaker than everything brought in through the prim’s other arcs, *across* layer stacks. Even when the specialized base is overridden in the strongest shot layer, that override loses to a direct opinion inside a referenced asset. - Still a broadcast: properties that nothing else authors receive the base’s value, including an override of the base made in a stronger layer (Section 19.5). - Like inherits, the target is a prim path in the same layer stack, and a missing target is silently empty.

Mental model.

```

STRONGEST
L local (direct) opinions
I inherits          <- base beats references
V variants
E relocates
R references        <- referenced asset's values
P payloads
S specializes      <- base only fills what nothing above authored
WEAKEST

```

Simple example. `/ViaInherit` and `/ViaSpecialize` both reference an asset with `roughness = 0.2`, and both point to a class with `roughness = 0.9` in the shot layer.

Prim	Arc to class	Result
<code>/ViaInherit</code>	<code>inherits</code>	0.9 (I beats R)
<code>/ViaSpecialize</code>	<code>specializes</code>	0.2 (R beats S)

USDA example. File: `shot.usda` (it references an `asset.usda` whose `/Mat` has `double roughness = 0.2`)

```

#usda 1.0

class "_base"
{
    double roughness = 0.9
}

def "ViaInherit" (
    prepend inherits = </_base>
    prepend references = @asset.usda@
)
{
}

def "ViaSpecialize" (
    prepend specializes = </_base>
    prepend references = @asset.usda@
)
{
}

```

Python example.

```

from pxr import Usd

with open("asset.usda", "w") as f:

```

```

    f.write("#usda 1.0\n")
    (
        defaultPrim = "Mat"
    )
def "Mat"
{
    double roughness = 0.2
}
"""
with open("shot.usda", "w") as f:
    f.write("#usda 1.0\n")
class "_base"
{
    double roughness = 0.9
    double metallic = 1.0
}
def "ViaInherit" (
    prepend inherits = </_base>
    prepend references = @asset.usda@
)
{
}
def "ViaSpecialize" (
    prepend specializes = </_base>
    prepend references = @asset.usda@
)
{
}
"""

stage = Usd.Stage.Open("shot.usda")
for path in ["/ViaInherit", "/ViaSpecialize"]:
    prim = stage.GetPrimAtPath(path)
    print(path, "roughness =", prim.GetAttribute("roughness").Get(),
          "metallic =", prim.GetAttribute("metallic").Get())

query = Usd.PrimCompositionQuery(stage.GetPrimAtPath("/ViaSpecialize"))
for arc in query.GetCompositionArcs():
    # strongest first
    print(" ", arc.GetArcType(), arc.GetTargetNode().path)

```

Expected output

```

/ViaInherit roughness = 0.9 metallic = 1.0
/ViaSpecialize roughness = 0.2 metallic = 1.0
Pcp.ArcTypeRoot /ViaSpecialize
Pcp.ArcTypeReference /Mat
Pcp.ArcTypeSpecialize /_base

```

Notice: `metallic`, which the asset does not author, comes from `_base` in both cases. `Specializes` still fills gaps. The composition query lists arcs strongest first, and the `specialize` arc is last.

Real-world use case. A manufacturing digital twin has a material library: `Steel` is the base, `BrushedSteel` and `PaintedSteel` specialize it. A plant engineer tweaks `Steel.density` for a new supplier, and every steel variant that does not set its own density picks it up. `PaintedSteel`'s own color is never overwritten by changes to `Steel`.

Common mistakes.

MISTAKE

Using `specializes` when the shot must be able to force a value on every copy. `Specializes` loses to the asset's own values. Use `inherits` for "override everything" broadcasts.

MISTAKE

Thinking `specializes` is a one-time copy. It is live: unauthored properties track the base.

MISTAKE

Expecting the specialized base to beat a reference "because the base is in the stronger root layer". Layer position does not rescue specializes; it is always weakest.

Exam traps.**TRAP**

"Specializes is the same as inherits." Same broadcast idea, opposite end of LIVERPS. The difference shows whenever another arc (variant, reference, payload) also has an opinion.

TRAP

"Specializes opinions never apply." They do apply, whenever nothing stronger authors that property.

Exam takeaways.**KEY**

- Specializes = live broadcast like inherits, but the ****weakest**** arc (S, last in LIVERPS). - Loses to local, inherits, variants, references, and payloads — even when the base is in a stronger layer. - Still supplies values nothing else authors. - Typical use: material libraries and "refinement" relationships.

19.4 Inherits vs. specializes: decision table

What is it? A side-by-side comparison to choose the right arc for shared opinions (Obj 1.1: changing the strength of an opinion by choosing the arc that carries it).

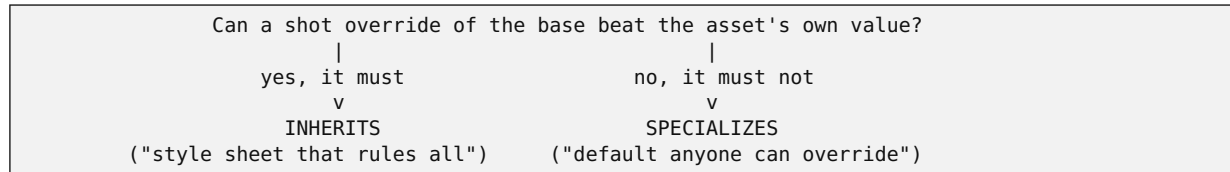
Why do we need it? Both arcs broadcast, so beginners pick at random. The wrong choice either lets a shot override something it must not touch (inherits where specializes was needed) or blocks an override the shot needs (specializes where inherits was needed).

Beginner explanation. Inherits is a **style sheet every instance follows, even after later edits**, and it can overrule what an asset author wrote. Specializes is a **default that anything else can override**.

Where the analogy breaks: both are equally "live"; they differ only in how strong their opinions are.

Technical explanation. The behaviors below were verified on USD 26.08 with an asset tree.usda whose /Tree inherits /_class_Tree and specializes /_spec_Tree, referenced into a shot that overrides both classes.

Question	Inherits	Specializes
LIVERPS position	I (second strongest)	S (weakest)
Live broadcast of later edits?	Yes	Yes
Asset's direct opinion vs. asset's own class	Direct wins	Direct wins
Shot's class override vs. asset's direct opinion	Shot class wins	Asset direct wins
Shot's class override vs. asset's own class	Shot wins	Shot wins
Base vs. the prim's references/payloads (same stack)	Base wins	References/payloads win
Base vs. the prim's variants	Base wins	Variants win
Typical use	Shot-wide or show-wide overrides of all copies	Material libraries, "refinements" of a base

Mental model.

Simple example. | Property on /Tree | Asset authors | Shot class override | Inherits result | Specializes result |

|-----|-----|-----|-----|-----| | leaves / season |
 direct value | new value | shot value | asset value | | bark / height | only in asset's class |
 new value | shot value | shot value |

USDA example. File: tree.usda

```
#usda 1.0
(
    defaultPrim = "Tree"
)

class "_class_Tree"
{
    string bark = "brown-class"
}

class "_spec_Tree"
{
    string height = "tall-spec"
}

def Xform "Tree" (
    prepend inherits = </_class_Tree>
    prepend specializes = </_spec_Tree>
)
{
    string leaves = "green-local"
    string season = "summer-local"
}
```

File: shot.usda

```
#usda 1.0

class "_class_Tree"
{
    string leaves = "autumn-shot"
    string bark = "grey-shot"
}

class "_spec_Tree"
{
    string season = "winter-shot"
    string height = "short-shot"
}

def "TreeA" (
    prepend references = @tree.usda@
)
{
}
```

Python example.

```
from pxr import Usd

FILES = {
    "tree.usda": "" "#usda 1.0
(
    defaultPrim = "Tree"
)
class "_class_Tree"
```

```

{
    string bark = "brown-class"
}
class "_spec_Tree"
{
    string height = "tall-spec"
}
def Xform "Tree" (
    prepend inherits = </_class_Tree>
    prepend specializes = </_spec_Tree>
)
{
    string leaves = "green-local"
    string season = "summer-local"
}
"""
    "shot.usda": """"#usda 1.0
class "_class_Tree"
{
    string leaves = "autumn-shot"
    string bark = "grey-shot"
}
class "_spec_Tree"
{
    string season = "winter-shot"
    string height = "short-shot"
}
def "TreeA" (
    prepend references = @tree.usda@
)
{
}
"""
}
for name, text in FILES.items():
    with open(name, "w") as f:
        f.write(text)

stage = Usd.Stage.Open("shot.usda")          # keep the stage alive
tree = stage.GetPrimAtPath("/TreeA")
rows = [("leaves", "inherits, asset direct"), ("bark", "inherits, asset class"),
        ("season", "specializes, asset direct"), ("height", "specializes, asset class")]
for attr, case in rows:
    print(f"{attr:7}{case:27}-> {tree.GetAttribute(attr).Get()}")

```

Expected output

```

leaves inherits, asset direct    -> autumn-shot
bark  inherits, asset class      -> grey-shot
season specializes, asset direct -> summer-local
height specializes, asset class  -> short-shot

```

Read it row by row: through inherits, the shot beats everything in the asset. Through specializes, the shot beats only the asset's own specialized base, never the asset's direct value.

Real-world use case. Games: every enemy inherits `_class_Enemy` so a balance designer can change all enemies' health from the level layer, even if a prefab author set one. Rendering: every car paint specializes `_spec_CarPaint`, so a look-dev artist's tweak to the base never overwrites the red of `FerrariRed`.

Common mistakes.

MISTAKE

Choosing inherits for a material library. A shot override of the base then clobbers every specific material's authored values. Use specializes.

MISTAKE

Choosing specializes for a "fix all copies" shot override. The asset's own values keep winning. Use inherits.

Exam traps.**TRAP**

In a single layer stack with no other arcs, inherits and specializes give the same result (direct opinions win, the base fills the rest). Exam items that test the difference always involve a reference, payload, or variant. Look for it.

TRAP

Both arcs broadcast. An answer saying "specializes does not receive later edits to the base" is wrong.

Exam takeaways.**KEY**

- Both broadcast; they differ only in strength (I strong, S weakest). - Shot must override all copies → inherits. - Asset-specific values must survive base changes → specializes. - The difference only appears when another arc (variant, reference, payload) also has an opinion.

19.5 Implied inherits across references

What is it? When a prim with an inherits arc is brought into a shot by a reference, USD also looks for the inherited class path in the **shot's** layer stack. This is called an **implied inherit**: the inherit is "implied" into the referencing layer stack. The same happens for specializes.

Why do we need it? An asset is referenced many times. Without implied inherits, the only way to change all copies would be to edit the asset file (affecting every shot) or every copy (tedious). Implied inherits let one shot override the class once and reach every copy *in that shot only*.

Beginner explanation. A franchise restaurant follows the head office's rule book (the asset's class). Each city's manager can post an extra sheet titled with the same rule-book name on the local notice board, and every branch in that city follows it. Other cities are unaffected.

Where the analogy breaks: the "same name" matters literally. The shot's class must be at the exact same path, such as `/_class_Tree`; a different name is not linked.

Technical explanation. - In the asset, `/Tree` inherits `/_class_Tree`. When `/World/Forest/A` references `/Tree`, the reference maps `/Tree` → `/World/Forest/A`. The class path `/_class_Tree` is outside the referenced prim, so it keeps its path, and USD adds an inherit to `/_class_Tree` in the **shot's** layer stack. - That implied arc lives in the shot's layer stack, so its opinions are stronger than anything inside the reference, including the asset's direct opinions (Section 19.4). - It works wherever the asset is placed (`/TreeA`, `/World/Forest/A`, ...): all copies follow the same root-level shot class. - The shot does not need to author any inherits itself. It only authors opinions at the class path; a `class` specifier is the clean way to do it. - `Usd.PrimCompositionQuery` shows the implied arc: `arc.IsImplicit()` returns `True` and the target node's layer stack is the shot's. - `GetAllDirectInherits()` on a copy returns `/_class_Tree`, the path as seen in the shot's namespace.

Mental model.

shot.usda layer stack	tree.usda layer stack
-----	-----
<code>class "_class_Tree" { leaves=autumn } <-. </code>	<code>class "_class_Tree" { leaves=green } </code>


```

      |
      ^
/World/Forest/A --reference--> /Tree --'-----' inherits </_class_Tree>
/World/Forest/B --reference--> /Tree      (implied: also look in the shot)

Strength for A:  shot L > shot I (implied class) > ... > R: asset L > asset I

```

Simple example. | Shot authors | /World/Forest/A.leaves | /World/Forest/B.leaves |
|-----|-----|-----| | nothing | green (asset class) | green | |
class "_class_Tree" { leaves = "autumn" } | autumn | autumn |

USDA example. File: tree.usda

```

#usda 1.0
(
    defaultPrim = "Tree"
)

class "_class_Tree"
{
    string leaves = "green"
}

def Xform "Tree" (
    prepend inherits = </_class_Tree>
)
{
}

```

File: forest.usda

```

#usda 1.0

class "_class_Tree"
{
    string leaves = "autumn"
}

def Xform "World"
{
    def Xform "Forest"
    {
        def "A" (
            prepend references = @tree.usda@
        )
        {
        }

        def "B" (
            prepend references = @tree.usda@
        )
        {
        }
    }
}

```

Notes: forest.usda never writes inherits. Its _class_Tree is picked up by both copies because of the asset's inherits arc.

Python example.

```

from pxr import Usd, Sdf

with open("tree.usda", "w") as f:
    f.write("""#usda 1.0
(
    defaultPrim = "Tree"
)
class "_class_Tree"
{
    string leaves = "green"
}
def Xform "Tree" (
    prepend inherits = </_class_Tree>

```

```

)
{
}
"""

stage = Usd.Stage.CreateInMemory()
copies = []
for name in ["A", "B"]:
    prim = stage.DefinePrim(f"/World/Forest/{name}")
    prim.GetReferences().AddReference("tree.usda")
    copies.append(prim)

def leaves():
    return [p.GetAttribute("leaves").Get() for p in copies]

print("before shot class:", leaves())
shot_class = stage.CreateClassPrim("/_class_Tree") # in the shot's root layer
shot_class.CreateAttribute("leaves", Sdf.ValueTypeNames.String).Set("autumn")
print("after shot class: ", leaves())
print("direct inherits:", copies[0].GetInherits().GetAllDirectInherits())

query = Usd.PrimCompositionQuery(copies[0])
for arc in query.GetCompositionArcs():
    layer = arc.GetTargetNode().layerStack.identifier.rootLayer
    origin = "tree.usda" if layer.identifier.endswith("tree.usda") else "shot"
    print(" ", arc.GetArcType(), arc.GetTargetNode().path, origin,
          "implicit" if arc.IsImplicit() else "")

```

Expected output

```

before shot class: ['green', 'green']
after shot class: ['autumn', 'autumn']
direct inherits: [Sdf.Path('/_class_Tree')]
  Pcp.ArcTypeRoot /World/Forest/A shot
  Pcp.ArcTypeInherit /_class_Tree shot implicit
  Pcp.ArcTypeReference /Tree tree.usda
  Pcp.ArcTypeInherit /_class_Tree tree.usda

```

The implied inherit (`implicit`, in the shot) is listed before the reference: it is stronger than everything in `tree.usda`.

Real-world use case. In a film sequence, a forest of 500 referenced trees must look autumnal in shot 12 only. The shot's lighting layer authors class `"/_class_Tree"` with new leaf colors. All 500 trees change in shot 12; the published tree asset and every other shot are untouched.

Common mistakes.

MISTAKE

Overriding the class at a different path in the shot (for example ``/World/_class_Tree``). Implied inherits use the asset's class path exactly, here ``/_class_Tree``.

MISTAKE

Editing the class inside the published asset to fix one shot. That changes every shot that uses the asset. Override the class in the shot instead.

MISTAKE

Expecting a shot override of a **specialized** base to beat the asset's direct values. Implied specializes exist too, but stay weakest (Section 19.4).

Exam traps.

TRAP

"The shot must add ``inherits`` to every copy to override the class." No. The asset's inherits arc is implied into the shot automatically; the shot only authors the class.

TRAP

"A shot's class override is weaker than the asset's direct opinion, because local beats inherits." "Local" means local to the same layer stack. The implied inherit is in the stronger shot layer stack, so it beats everything inside the reference.

Exam takeaways.**KEY**

- Referencing an asset that inherits `/_class_X`` also links `/_class_X`` in the referencing layer stack. - The shot's class override beats everything inside the reference, including direct opinions. - Same exact path required; the shot need not author ``inherits`` itself. - Implied specializes exist too but remain weakest. - ``PrimCompositionQuery` : `IsImplicit()`` marks implied arcs.

Chapter lab(s)

Lab 17 — Classes and inherits: broadcast edits (python-labs/lab17_*.md, Obj 1.1).

You create a `_class_` prim, make many prims inherit it, broadcast edits, add local overrides, then reference the asset into a shot and override the class there (implied inherits).

Lab 18 — Specializes vs. inherits side by side (python-labs/lab18_*.md, Obj 1.6). You build the same asset twice, once with inherits and once with specializes, reference both into a shot, override the base in the shot, and record which values win and why.

USDA reading exercises

Exercise 19-A. One layer:

```
#usda 1.0

class "_class_Chair"
{
    string fabric = "wool"
    int legs = 4
}

def "Chair1" (
    prepend inherits = </_class_Chair>
    variants = {
        string style = "bar"
    }
    prepend variantSets = "style"
)
{
    string fabric = "leather"
    variantSet "style" = {
        "bar" {
            int legs = 3
            double height = 1.1
        }
    }
}
```

What are `fabric`, `legs`, and `height` on `/Chair1`?

Exercise 19-B. `lib.usda` has class `"_spec_Paint" { string finish = "matte" string coat = "single" }` and def `"RedPaint" (specializes = </_spec_Paint>) { string finish = "gloss" }`. The shot references `lib.usda</RedPaint>` at `/Car/Paint` and authors class `"_spec_Paint" { string finish = "satin" string coat = "double" }`. What are `finish` and `coat` on `/Car/Paint`? What changes if `RedPaint` used `inherits` instead?

Chapter review

Summary

- A `class` prim is defined but abstract: skipped by `Traverse()` and renderers; its children are abstract too.
- Inherits (I) is a live link: edit the class once and every inheritor updates (broadcast).
- Direct opinions beat the class in the same layer stack; the class beats variants, references, payloads, and specializes.
- With several inherits, the first listed is strongest; a missing target is silently empty.
- Specializes (S) also broadcasts but is the weakest arc; it only fills properties nothing else authors.
- A shot override of a specialized base never beats the asset's own values; a shot override of an inherited class does.
- Implied inherits: a shot can override `/_class_X` and reach every referenced copy of an asset that inherits `/_class_X`.
- Implied arcs are reported by `Usd.PrimCompositionQuery` with `IsImplicit() == True`.

If you see... → think...

If you see...	Think...
<code>class "_class_X"</code>	Abstract source of shared opinions
"Change every copy in this shot"	Override the inherited class in the shot (implied inherits)
"Material library", "refinement", "base material"	Specializes
Class edit has no effect on one prim	That prim has a direct opinion
Shot override of a base loses to an asset value	The arc is specializes
Base beats a referenced value	The arc is inherits
<code>inherits = [</_A>, </_B>]</code>	<code>_A</code> wins where both author
Class not found by <code>Traverse()</code>	Abstract; use <code>TraverseAll()</code> or <code>GetPrimAtPath()</code>

Review questions

R19-01 · Obj 1.6 · Single choice Which list orders these arcs from strongest to weakest?

A. Inherits, References, Variants, Specializes B. Inherits, Variants, References, Specializes C. Variants, Inherits, Specializes, References D. References, Inherits, Variants, Specializes

R19-02 · Obj 1.1 · Single choice `/Lamp inherits /_class_Lamp (watts = 60)` and has a direct `watts = 100` in the same layer. The class is changed to `watts = 75`. What is `/Lamp.watts`? A. 60 B. 75 C. 100 D. 175

R19-03 · Obj 1.1 · Single choice A referenced asset `/Car` authors `color = "red"` directly and inherits `/_class_Car`. The shot authors `class "_class_Car" { color = "blue" }`. What is the color of the referenced car in the shot? A. red, because local beats inherits B. blue, because the implied inherit in the shot is stronger than the reference C. red, because the shot never authored inherits D. A composition error

R19-04 · Obj 1.1 · Single choice Same as R19-03, but the asset uses `specializes = </_class_Car>`. What is the color? A. red B. blue C. No value D. Composition error

R19-05 · Obj 1.6 · Select two. Which statements about `class` prims are true? A. `IsDefined()` returns `True`. B. `stage.Traverse()` visits them. C. Their child prims are abstract. D. They can only be targets of inherits, not specializes. E. They must be in a separate file.

R19-06 · Obj 1.1 · USDA reading

```
#usda 1.0

class "_base"
{
    int level = 1
}

def "P" (
    prepend specializes = </_base>
    variants = {
        string v = "a"
    }
    prepend variantSets = "v"
)
{
    variantSet "v" = {
        "a" {
            int level = 2
        }
    }
}
}
```

What is `/P.level`? What would it be with `inherits` instead of `specializes`? A. 1; 1 B. 2; 1 C. 2; 2 D. 1; 2

R19-07 · Obj 1.1 · Single choice A material library must allow look-dev to tweak the generic `Metal` base for a show, without ever overriding values that `Gold` authors itself. Which arc should `Gold` use to `Metal`? A. inherits B. specializes C. references D. sublayer

R19-08 · Obj 1.6 · Select two. Which are true about `specializes`? A. It is the weakest arc in LIVERPS. B. It receives later edits to the base for properties nothing else authors. C. A base override in the strongest layer beats the prim's references. D. It requires an asset path.

R19-09 · Obj 1.1 · Single choice `/Q` has `inherits = </_Missing>`, and nothing is authored at `/_Missing`. What happens? A. A composition error stops the stage from opening B. `/Q` becomes abstract C. Nothing is inherited; no error is reported D. `/Q` becomes inactive

R19-10 · Obj 1.1 · Single choice An asset's `/Tree` inherits `/_class_Tree`. A shot references it at `/World/T1` and wants to override the class for all trees. Which shot path should hold the override? A. `/World/T1/_class_Tree` B. `/World/_class_Tree` C. `/_class_Tree` D. `/Tree/_class_Tree`

R19-11 · Obj 1.6 · Python reading What does this print? (Not run automatically here, so the output does not give the answer away; the answer key shows the verified result.)

```
from pxr import Usd, Sdf

stage = Usd.Stage.CreateInMemory()
for name, size in [("/_A", 1), ("/_B", 2)]:
    cls = stage.CreateClassPrim(name)
    cls.CreateAttribute("size", Sdf.ValueTypeNames.Int).Set(size)
p = stage.DefinePrim("/P")
p.GetInherits().SetInherits(["/_B", "/_A"])
print(p.GetAttribute("size").Get())
```

A. 1 B. 2 C. 3 D. None

Answers

USDA reading exercises

19-A. `fabric = "leather"` (direct beats inherits), `legs = 4` (inherits beat variants, so the class's 4 beats the variant's 3), `height = 1.1` (only the variant authors it).

19-B. `finish = "gloss"`, `coat = "double"`. Through specializes, `RedPaint`'s own `gloss` beats the shot's base override; `coat` is authored only on the base, and the shot's base override is stronger than the library's base. With inherits, `finish` would be `satin` (and `coat` still `double`): the implied class in the shot beats everything inside the reference.

Review questions

R19-01 — B. LIVERPS: Local, Inherits, Variants, rElocates, References, Payloads, Specializes. Review: §19.3.

R19-02 — C. The direct opinion wins in the same layer stack; the class edit reaches only prims without a direct opinion. Review: §19.2.

R19-03 — B. The asset's inherits arc is implied into the shot, and the shot's class is stronger than everything inside the reference. "Local beats inherits" only applies within one layer stack. Review: §19.5.

R19-04 — A. Specializes is always weakest; the asset's direct value wins even over a shot override of the base. Review: §19.4.

R19-05 — A, C. `class` defines an abstract prim; children are abstract; `Traverse()` skips them; they can be targets of either arc; they live in any layer. Review: §19.1.

R19-06 — B. With specializes, the variant (V) beats the base (S): 2. With inherits, the class (I) beats the variant: 1. Review: §19.3, §19.2.

R19-07 — B. Specializes lets base tweaks flow in while `Gold`'s own values always win. Inherits would let a show override of `Metal` clobber `Gold`. Review: §19.4.

R19-08 — A, B. C is false (references beat specializes regardless of layer). D is false (the target is a prim path). Review: §19.3.

R19-09 — C. A missing inherit target simply contributes nothing; verified on 26.08 with no composition errors. Review: §19.2.

R19-10 — C. Implied inherits use the asset's class path unchanged: `/_class_Tree`. Review: §19.5.

R19-11 — B. `SetInherits` writes an explicit list `[_B, /_A]`; the first listed is strongest, so `_B`'s 2 wins. Review: §19.2.

CHAPTER 20

Chapter 20 — Overs and Relocates

Exam domain: Composition (23%) · **Objectives:** 1.8, 6.2 · **Study day:** 6 · **Est. time:** 90 min **Prerequisites:** Ch 4 (specifiers), Ch 14 (LIVERPS, prim index), Ch 15 (layer stacks), Ch 16 (references)

Chapter 4 taught `def`, `over`, and `class`. This chapter puts `over` to work on composed stages: how a shot layer overrides a referenced asset without defining a second copy, and how **relocates** move a prim that arrived through a weaker arc. Both topics sit on the exam’s “why didn’t my opinion take effect?” list (Obj 1.8 / 6.2).

Learning goals

- Choose `over` rather than a typeless `def` when overlaying an existing prim.
- Override referenced children from the shot without editing the asset file.
- Explain why a lone `over` does not appear in `Traverse()`, and why an extra `over` child is not defined.
- Author relocates as layer metadata and predict the new namespace.
- Name the relocate limits that hide opinions (source-path edits, missing destination parent).

Key terms

Term	One-line definition
over	Specifier meaning “overlay”: opinions here, but this spec does not define the prim.
Typeless def	A <code>def</code> with no schema type. It does define the prim.
Composed specifier	The specifier you read on the stage (<code>prim.GetSpecifier()</code>), after all specs combine.
Local override	An opinion in the local layer stack (L) on a prim that also has a reference (R). Local wins.
Relocate	Layer metadata that moves a composed prim from a source path to a destination path.
Relocation source	The old path. Opinions authored there after the relocate are ignored.
Relocation destination	The new path. Author overrides here. Descendants move with the prim.
Sdf.Layer.relocates	Python field: a list of (source, target) <code>Sdf.Path</code> pairs.

20.1 `over` vs. typeless `def`

What is it? An `over` is a prim spec that adds opinions without claiming “this prim exists because of me.” A **typeless `def`** is a `def` with an empty type name. It looks similar in USDA — neither writes `Xform` or `Sphere` — but only the `def` defines the prim.

Why do we need it? Shot layers, department layers, and material layers almost never own the asset. They need to change a color, a pose, or a radius on a prim that some other file `def`’d. If they also `def` that prim, they plant a second existence claim. If they `over` it, they add notes and leave existence to the asset.

Beginner explanation. Think of a **museum label next to a statue**. The statue (`def` in the asset) is the object. The label (`over` in the shot) can say “this one is on loan” without being a second statue. A typeless `def` is like building a blank pedestal and calling it the exhibit: something now exists there even if it has no type.

Where the analogy breaks: a stronger **over** still wins **values**. Specifiers are about existence, not about which number wins (Chapter 4.3, Chapter 14.2).

Technical explanation. Verified on USD 26.08:

Authoring	USDA	Python	IsDefined() alone	In Traverse() alone
Overlay	<code>over</code> <code>"Hero" { ... }</code>	<code>stage.OverridePrim("/Hero")</code>	False	No
Typeless define	<code>def</code> <code>"Hero" { ... }</code>	<code>stage.DefinePrim("/Hero")</code>	True	Yes
Typed define	<code>def Sphere</code> <code>"Hero" { ... }</code>	<code>stage.DefinePrim("/Hero", "Sphere")</code>	True	Yes

- `prim.GetSpecifier()` on the **stage** is the composed specifier. A local **over** on top of a referenced **def** still reports `Sdf.SpecifierDef`, because a **def** somewhere in the prim stack defines it.
- `OverridePrim("/A/B")` authors **over** "A" for the missing parent as well (Chapter 4).
- `DefinePrim` on a path that already exists as a composed prim authors a **def** spec in the edit target. Prefer **over** in overlay layers so you do not accidentally define a prim that should only exist when the reference is present.
- An **over** child that **no** weaker layer defines is a valid prim (`GetPrimAtPath` works) but `IsDefined()` is False and `Traverse()` skips it. A **def** child in the same place is defined and is visited.

The NVIDIA reading list names this contrast as “over vs typeless def”. Exam items love the look-alike USDA.

Mental model.

```
def = plant a flag ("this prim exists")
over = write on a flag that should already be there
class = plant a flag in the templates drawer (abstract)
```

Existence comes from any **def** (or **class**) in the prim stack.
Values come from the strongest opinion, even if that opinion is an **over**.

Simple example. Open a file that contains only `over "Hero" { int hp = 10 }.` `/Hero` is not defined; `Traverse()` is empty. Change **over** to **def** and `/Hero` appears, still with no type name.

USDA example. File: *overlay.usda* — an overlay alone does not define the prim

```
#usda 1.0

over "Hero"
{
    int hp = 10
}
```

File: *typeless.usda* — a typeless **def** does define the prim

```
#usda 1.0

def "Hero"
{
    int hp = 10
}
```

Line notes: both files omit a type name. Only the specifier differs. *overlay.usda* opened as a stage is an empty traversal; *typeless.usda* yields `/Hero`.

Python example.


```

from pxr import Usd

stage = Usd.Stage.CreateInMemory()
hero = stage.OverridePrim("/Hero")
npc = stage.DefinePrim("/Npc")
print("Hero specifier/defined:", hero.GetSpecifier(), hero.IsDefined())
print("Npc specifier/defined:", npc.GetSpecifier(), npc.IsDefined())
print("Traverse:", [str(p.GetPath()) for p in stage.Traverse()])
print("TraverseAll:", [str(p.GetPath()) for p in stage.TraverseAll()])

```

Expected output

```

Hero specifier/defined: Sdf.SpecifierOver False
Npc specifier/defined: Sdf.SpecifierDef True
Traverse: ['/Npc']
TraverseAll: ['/Hero', '/Npc']

```

`TraverseAll()` visits undefined overs. Default `Traverse()` does not. That gap is a common “the prim vanished” report.

Real-world use case. A lighting department’s `shot_lgt.usda` is a stack of over prims: lights that layout already def’d, plus over “Chair_03” to dim a practical lamp. Lighting never defs `/World/Chair_03`. If layout drops that chair, the lighting overs remain in the file but the prim is no longer defined, and the shot does not grow a ghost chair.

Common mistakes.

MISTAKE

Using ``def` "Hero"` in a shot “because it has no type, so it is soft.” A typeless ``def`` still defines ``/Hero``. If the reference is missing you get an empty but defined prim. Use ``over``.

MISTAKE

Opening an overlay-only layer in usdview and concluding the file is empty. The overs are there; nothing defines them. Open the composed stage (asset + overlay) instead.

Exam traps.

TRAP

“``over`` is weaker than ``def``.” False for values. Strength is the layer stack and LIVERPS. Specifier is existence.

TRAP

“Typeless prims are illegal.” They are legal. The FAQ question is over versus **typeless def**, not typed versus typeless.

Exam takeaways.

KEY

- ``over`` overlays; ``def`` defines. Type name is a separate field. - Prefer ``over`` in shot and department layers. - ``Traverse()`` skips undefined overs; ``TraverseAll()`` does not. - Composed ``GetSpecifier()`` can be ``Def`` even when **this** layer wrote ``over``.

20.2 Overriding referenced content

What is it? **Overriding referenced content** means authoring local opinions on a prim that also has a reference (or payload) arc. The local opinions sit at **L** in LIVERPS, so they win over the asset’s opinions at **R** (or **P**). The asset file is not edited.

Why do we need it? You reuse one chair 40 times. Chair 7 must be larger. Chair 12 must hide its back. You do not fork `chair.usda` 40 times. You over the children on those copies in the shot.

Beginner explanation. The catalog page is the reference. A sticky note on *this* copy of the chair is the over. The sticky note wins for that copy only.

Where the analogy breaks: the sticky note is stored in **your** file, at the **composed path** (/Room/Chair_B/Seat), not inside the catalog. USD writes over "Seat" under the referencing prim. You never open chair.usda.

Technical explanation. - Setting an attribute on a composed child authors an over spec in the edit target automatically. You do not have to call OverridePrim first. OverridePrim alone, with no property, may write nothing. - The nested USDA is:

```
def "Chair_B" ( prepend references = @./chair.usda@ )
{
    over "Seat" { double size = 4 }
}
```

- The composed specifier of /Room/Chair_B/Seat stays Sdf.SpecifierDef (the asset defined it). The spec in the *shot* is an over. GetPrimStack() shows both (Chapter 22).
- Local over can change attribute values, metadata (active = false), variant selections (Chapter 18), and can def **new** children that the asset does not have.
- An over child that the asset does not define is **not** defined. Traverse() will not visit it. If you mean to add a new prim, def it.
- active = false on a referenced child hides it from default traversal. The prim is still valid; IsActive() is False. This is a standard way to “remove” a part of an instance-unrelated asset (instanced components have extra rules: Chapter 22.4, Chapter 24).
- A **value block** (double size = None) stops weaker values from showing through. Get() returns None. Schema fallbacks can still make HasValue() True (Cube’s fallback size is 2, for example). Blocking is stronger than “not authored.”
- Strength: anything in the local layer stack, including these overs, is **L**. It beats inherits, variants, relocates, references, payloads, and specializes on that prim (Chapter 21).

EXAM TIP

Obj 1.8 / 6.2: "I set a value and nothing changed." First question: did you author on the composed path in a layer that is actually in the local stack, or did you edit the asset / the relocate source / a muted layer / an instance proxy?

Mental model.

<pre>chair.usda def Xform "Chair" Seat.size = 1 Back.size = 2 Composed /Room/Chair_B: Seat.size = 4 Back exists but is inactive (Traverse skips it)</pre>	<pre>room.usda (local, stronger) def "Chair_B" (references = @chair@) over "Seat" { size = 4 } <-- wins over "Back" (active = false)</pre>
--	---

Simple example. Two referenced chairs. Only B’s seat is enlarged. Only B’s back is deactivated. Chair A still matches the asset.

USDA example. File: chair.usda

```
#usda 1.0
(
    defaultPrim = "Chair"
)

def Xform "Chair"
{
    def Cube "Seat"
```

```

{
    double size = 1
}
def Cube "Back"
{
    double size = 2
}
}

```

File: room.usda

```

#usda 1.0

def Xform "Room"
{
    def "Chair_A" (
        prepend references = @./chair.usda@
    )
    {
    }

    def "Chair_B" (
        prepend references = @./chair.usda@
    )
    {
        over "Seat"
        {
            double size = 4
        }
        over "Back" (
            active = false
        )
        {
        }
    }
}

```

Line notes: `Chair_A` has an empty body — it is a pure reference. `Chair_B` adds two nested overs. `active = false` is prim metadata in the spec's parentheses, not an attribute.

Python example.

```

from pxr import Usd

with open("chair.usda", "w") as f:
    f.write("""#usda 1.0
(
    defaultPrim = "Chair"
)
def Xform "Chair"
{
    def Cube "Seat"
    {
        double size = 1
    }
    def Cube "Back"
    {
        double size = 2
    }
}
""")

stage = Usd.Stage.CreateNew("room.usda")
for name in ("Chair_A", "Chair_B"):
    prim = stage.DefinePrim("/Room/" + name)
    prim.GetReferences().AddReference("./chair.usda")

seat_b = stage.GetPrimAtPath("/Room/Chair_B/Seat")
seat_b.GetAttribute("size").Set(4.0)
stage.GetPrimAtPath("/Room/Chair_B/Back").SetActive(False)

for path in ("/Room/Chair_A/Seat", "/Room/Chair_B/Seat"):
    prim = stage.GetPrimAtPath(path)
    print(path, "size =", prim.GetAttribute("size").Get(),

```

```
"specifier =", prim.GetSpecifier())
print("Traverse:", [str(p.GetPath()) for p in stage.Traverse()])
print(stage.GetRootLayer().ExportToString())
```

Expected output

```
/Room/Chair_A/Seat size = 1.0 specifier = Sdf.SpecifierDef
/Room/Chair_B/Seat size = 4.0 specifier = Sdf.SpecifierDef
Traverse: ['/Room', '/Room/Chair_A', '/Room/Chair_A/Seat', '/Room/Chair_A/Back', '/Room/Chair_B',
'/Room/Chair_B/Seat']
#usda 1.0

def "Room"
{
  def "Chair_A" (
    prepend references = @./chair.usda@
  )
  {
  }

  def "Chair_B" (
    prepend references = @./chair.usda@
  )
  {
    over "Seat"
    {
      double size = 4
    }

    over "Back" (
      active = false
    )
    {
    }
  }
}
```

Set(4.0) authored over "Seat" for you. Chair B's back is gone from Traverse() because it is inactive, not because it was deleted from the asset.

Real-world use case. A digital-twin factory references one robot.usda at twenty stations. Station 3's cell layer overs /Cell/Robot/Gripper to set a different payload mass and deactivates /Cell/Robot/OptionalCamera. Manufacturing still publishes one robot asset.

Common mistakes.

MISTAKE

Editing `chair.usda` to change one copy. Every copy changes. Fix: `over` the child on that copy's referencing prim.

MISTAKE

Adding a new part with `over "Cushion"`. The cushion is not defined, so `Traverse()` and renderers skip it. `def` the new part.

MISTAKE

Calling `OverridePrim("/Room/Chair\_B/Seat")` and saving, then wondering why the layer did not change. With no property authored, USD may write no spec. Set a value (or metadata).

Exam traps.

TRAP

"The composed specifier of an overridden Seat is `Over`." It stays `Def` if the asset defined it. Look at `GetPrimStack()` for the shot's `over` spec.

TRAP

"Local ``over`` is weaker than the reference because the asset ``def``'d the prim." L is stronger than R. The specifier does not change LIVERPS.

Exam takeaways.**KEY**

- Override referenced content with nested ``over`` at the composed path. - ``Set`` on a composed attribute authors that ``over`` for you. - Local L opinions beat the referenced asset. - New prims need ``def``; ``over`` alone will not define them. - Deactivate a child with ``over "Name" (active = false)``, not by editing the asset.

20.3 Relocates: what and why

What is it? A **relocate** is layer metadata that says: “the prim that composition would have put at *source path* now lives at *destination path*.” The prim and its descendants move. The source path becomes invalid. Relocates are the **E** (rElocates) in LIVERPS.

Why do we need it? A referenced rig ships joints under `/Robot/Rig/Arm`. Your animation tools, constraints, or a client contract want `/Bot/Arm`. You must not edit the rig file (another department owns it; many shots reference it). A relocate in *your* layer moves the namespace for this composition only.

Beginner explanation. Think of **relabeling a folder without copying the files**. The catalog still stores `Rig/Arm`. Your scene’s directory listing shows `Arm` next to `Rig`. Everyone who looks at your scene uses the new path.

Where the analogy breaks: the old path is not an alias. `/Bot/Rig/Arm` is **invalid** after the relocate. Relationships that still point at the old path do **not** retarget automatically (Section 20.4).

Technical explanation. - Authoring is **layer** metadata, not prim metadata:

```
(
  relocates = { </Bot/Rig/Arm>: </Bot/Arm> }
)
```

- Python: `layer.relocates` is a **list of pairs**, not a dict. Each pair is `(Sdf.Path(source), Sdf.Path(dest))`. `layer.HasRelocates()` / `layer.ClearRelocates()` exist on USD 26.08. There is no `Usd.Prim` relocate API.
- Relocates apply to prims that arrive through **weaker composition** (typically a reference or payload in this layer stack). They are not a “rename this def I just wrote in the same layer” tool (Section 20.4).
- Descendants move with the prim: `/Bot/Rig/Arm/Hand` becomes `/Bot/Arm/Hand`.
- `Usd.PrimCompositionQuery` lists a `Pcp.ArcTypeRelocate` node on the destination. Target path of that arc is the source path.
- Flattening (`stage.Flatten()`) bakes the **new** namespace. The flattened layer has no relocates metadata and no `Rig/Arm`.
- Strength: E sits between VariantSets and References. Local opinions at the **destination** still beat the relocated content. Chapter 21 puts this in LIVERPS puzzles.

VERSION

Verified on USD 26.08. Relocates as a composition arc landed in the USD 24.x series (authoring work in 24.05; composition in 24.08). ``Usd.PrimCompositionQuery`` reports relocate arcs (26.03+). Older material spells the strength order ****LIVRPS**** and omits E; the other six letters keep the same order.

Mental model.

<pre>robot.usda /Robot/Rig/Arm +---- composed, then moved -----></pre>	<pre>scene.usda relocates { /Bot/Rig/Arm : /Bot/Arm } def "Bot" (references = @robot@) /Bot/Arm (valid) /Bot/Rig (empty of Arm) /Bot/Rig/Arm (invalid)</pre>
---	--

Simple example. Reference a robot. Relocate `/Bot/Rig/Arm` to `/Bot/Arm`. Children of `/Bot` become `Rig` and `Arm`. `Rig` no longer contains `Arm`. `Hand` under `Arm` moved too.

USDA example. *File: robot.usda*

```
#usda 1.0
(
    defaultPrim = "Robot"
)

def Xform "Robot"
{
    def Xform "Rig"
    {
        def Xform "Arm"
        {
            double length = 1
            def Sphere "Hand"
            {
                double radius = 0.2
            }
        }
    }
}
```

File: scene.usda

```
#usda 1.0
(
    relocates = {
        </Bot/Rig/Arm>: </Bot/Arm>
    }
)

def Xform "Bot" (
    prepend references = @./robot.usda@
)
{
}
```

Line notes: `relocates` sits in the **layer** metadata block (the parentheses after `#usda 1.0`), beside `subLayers` and `defaultPrim`. Paths are wrapped in `<...>`. Several entries are separated by commas.

Python example.

```
from pxr import Pcp, Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("robot.usda", """#usda 1.0
(
    defaultPrim = "Robot"
)
def Xform "Robot"
{
    def Xform "Rig"
    {
        def Xform "Arm"
        {
            double length = 1
            def Sphere "Hand"
```

```

        {
            double radius = 0.2
        }
    }
}
"""
write("scene.usda", ""#usda 1.0
(
    relocates = {
        </Bot/Rig/Arm>: </Bot/Arm>
    }
)
def Xform "Bot" (
    prepend references = @./robot.usda@
)
{
}
"""

stage = Usd.Stage.Open("scene.usda")
bot = stage.GetPrimAtPath("/Bot")
print("children of /Bot:", list(bot.GetChildrenNames()))
print("children of /Bot/Rig:",
    list(stage.GetPrimAtPath("/Bot/Rig").GetChildrenNames()))
arm = stage.GetPrimAtPath("/Bot/Arm")
print("/Bot/Arm length =", arm.GetAttribute("length").Get())
hand = stage.GetPrimAtPath("/Bot/Arm/Hand")
print("/Bot/Arm/Hand radius =", hand.GetAttribute("radius").Get())
print("/Bot/Rig/Arm valid:", stage.GetPrimAtPath("/Bot/Rig/Arm").IsValid())
print("layer.HasRelocates():", stage.GetRootLayer().HasRelocates())
print("layer.relocates:", stage.GetRootLayer().relocates)
for arc in Usd.PrimCompositionQuery(arm).GetCompositionArcs():
    print(f"{str(arc.GetArcType()):24}",
        arc.GetTargetLayer().GetDisplayName(), arc.GetTargetPrimPath())

```

Expected output

```

children of /Bot: ['Rig', 'Arm']
children of /Bot/Rig: []
/Bot/Arm length = 1.0
/Bot/Arm/Hand radius = 0.2
/Bot/Rig/Arm valid: False
layer.HasRelocates(): True
layer.relocates: [(Sdf.Path('/Bot/Rig/Arm'), Sdf.Path('/Bot/Arm'))]
Pcp.ArcTypeRoot      scene.usda /Bot/Arm
Pcp.ArcTypeRelocate  scene.usda /Bot/Rig/Arm
Pcp.ArcTypeReference robot.usda /Robot/Rig/Arm

```

The query lists L, then E, then R: local root, relocate, then the referenced spec that supplied `Arm`. Hand moved as a descendant; you did not relocate Hand separately.

Real-world use case. A game character pipeline references a vendor skeleton whose joints sit under `Root/Hips/Spine/...`. The studio's animation layer relocates `.../LeftHand` to `/Hero/Hand_L` so gameplay code and IK setups use a stable, short path. The vendor file is never forked.

Common mistakes.

MISTAKE

Writing ``relocates`` inside a prim's metadata parentheses. It is **layer** metadata at the top of the file.

MISTAKE

Expecting ``/Bot/Rig/Arm`` to keep working as an alias. It is invalid. Update paths, constraints, and relationship targets to ``/Bot/Arm``.

Exam traps.

TRAP

"Relocates are prim metadata like ``references``." They are layer metadata, like ``subLayers``.

TRAP

"E is weaker than R, so the reference wins over the relocate." The relocate `**moves**` the referenced prim; it is not a second value that fights ``length``. Local opinions at the destination still beat the relocated values.

Exam takeaways.**KEY**

- Relocates rename composed namespace without editing the referenced file. - Layer metadata, list of ``(source, dest)`` paths in Python. - Source path becomes invalid; children move with the prim. - E in LIVERPS, between variants and references. - ``PrimCompositionQuery`` shows ``Pcp.ArcTypeRelocate``.

20.4 Relocates authoring and limitations

What is it? This section is the **rules of the road**: how to author relocates in Python, where to put overrides after a move, and the limits that make an opinion disappear (Obj 1.8).

Why do we need it? Most relocate bugs are not "USD forgot the move." They are "I authored at the old path," "the destination's parent does not exist," or "I tried to rename a local def with a relocate." The exam asks you to identify which of those happened.

Beginner explanation. After you relabel the folder, notes stuck to the **old** folder name are thrown away. Notes on the **new** folder name stick. You also cannot relabel a folder that only exists as a sticky note in the same drawer — relocates move content that arrived from somewhere weaker, such as a referenced catalog.

Where the analogy breaks: USD does not always raise. An opinion at the source becomes a composition error (`Pcp.ErrorOpinionAtRelocationSource`) and is ignored. A destination whose parent is missing can fail **silently**: `GetCompositionErrors()` may be empty and the prim simply never appears.

Technical explanation. Verified on USD 26.08:

Authoring in Python. Assign a list of tuples. A dict of strings is not accepted. The next snippet is a fragment (no imports); the full script is in step 8.

```
layer.relocates = [
    (Sdf.Path("/Bot/Rig/Arm"), Sdf.Path("/Bot/Arm")),
]
```

USDA still looks like a map. That spelling difference is an exam favorite.

Overrides go at the destination. Nested over "Arm" { double length = 4 } under /Bot wins. Nested over "Rig" { over "Arm" { ... } } is an opinion at the source. USD warns, records `ErrorOpinionAtRelocationSource`, and keeps the referenced value.

Relocates of local defs in the same layer. If /A/B is def'd in the same layer that relocates /A/B → /A/C, the local spec sits at the source, so it is ignored. /A/C does not receive it. Use a reference (or a weaker sublayer) as the source of the prim you want to move, or just author the prim at the path you want.

Destination parent. Relocating to /Missing/Arm when /Missing does not exist yields an invalid destination. The prim index may still record the mapping; `GetCompositionErrors()` can still be empty. Create the parent (or relocate to a path whose parent already exists).

Relationships. A relationship authored as `rel driver = </Bot/Rig/Arm>` still reports that target after the relocate. USD does not rewrite it to /Bot/Arm. Update targets yourself.

Where in the stack. A relocate in any layer of the **local** layer stack applies. A weaker sublayer can hold the `relocates` map while a stronger layer overs the destination. Payloads relocate the same way as references once they are loaded.

Not a delete tool. An empty destination path `<>` is ill-formed. To hide a prim, deactivate it (`active = false`) or unload a payload (Chapter 17).

Several relocates. One layer may list many pairs. You can move `Arm` and `Head` in the same map.

Mental model.

```
After relocates { /Bot/Rig/Arm : /Bot/Arm }:
```

WRITE HERE	IGNORED (error)
/Bot/Arm.length = 4	/Bot/Rig/Arm.length = 9

Also ignored: a def of `/Bot/Rig/Arm` in the SAME layer as the relocate (that spec is at the source path).

Simple example. The same scene as 20.3, plus two overs: one at `/Bot/Arm` (length 4, wins) and one at `/Bot/Rig/Arm` (length 9, ignored, composition error).

USDA example. *File: scene_bad.usda* (uses `robot.usda` from 20.3)

```
#usda 1.0
(
  relocates = {
    </Bot/Rig/Arm>: </Bot/Arm>
  }
)

def Xform "Bot" (
  prepend references = @./robot.usda@
)
{
  over "Rig"
  {
    over "Arm"
    {
      double length = 9
    }
  }
  over "Arm"
  {
    double length = 4
  }
}
```

Line notes: the `over "Arm"` directly under `Bot` is the destination. The `over "Arm"` under `Rig` is the source and will be ignored.

Python example.

```
from pxr import Sdf, Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("robot.usda", """#usda 1.0
(
  defaultPrim = "Robot"
)
def Xform "Robot"
{
  def Xform "Rig"
  {
    def Xform "Arm"
    {
      double length = 1
    }
  }
}
```

```

    }
}
"""
write("scene_bad.usda", ""#usda 1.0
(
    relocates = {
        </Bot/Rig/Arm>: </Bot/Arm>
    }
)
def Xform "Bot" (
    prepend references = @./robot.usda@
)
{
    over "Rig"
    {
        over "Arm"
        {
            double length = 9
        }
    }
    over "Arm"
    {
        double length = 4
    }
}
"""

stage = Usd.Stage.Open("scene_bad.usda")
print("length at /Bot/Arm:",
      stage.GetPrimAtPath("/Bot/Arm").GetAttribute("length").Get())
print("errors:")
for err in stage.GetCompositionErrors():
    print(" ", type(err).__name__)

layer = Sdf.Layer.CreateAnonymous(".usda")
layer.relocates = [
    (Sdf.Path("/Bot/Rig/Arm"), Sdf.Path("/Bot/Arm")),
]
print("Python relocates:", layer.relocates)
print("HasRelocates:", layer.HasRelocates())
print(layer.ExportToString())
layer.ClearRelocates()
print("after ClearRelocates:", layer.relocates, layer.HasRelocates())

```

Expected output

```

length at /Bot/Arm: 4.0
errors:
  ErrorOpinionAtRelocationSource
Python relocates: [(Sdf.Path('/Bot/Rig/Arm'), Sdf.Path('/Bot/Arm'))]
HasRelocates: True
#usda 1.0
(
    relocates = {
        </Bot/Rig/Arm>: </Bot/Arm>
    }
)

after ClearRelocates: [] False

```

The warning on stderr (not shown above) matches the error: the source-path opinion is ignored. Length 4 from the destination over wins. Assigning `layer.relocates = { "/Bot/Rig/Arm": "/Bot/Arm" }` (a dict of strings) raises `TypeError` on 26.08.

Real-world use case. A manufacturing digital twin references a vendor robot and relocates flange frames to `/Cell/TCP`. The cell's animation layer overs `/Cell/TCP` to key the tool pose. A new artist who keys `/Cell/Robot/Rig/Flange` instead sees no motion and a composition error — the classic Obj 1.8 relocate bug.

Common mistakes.

MISTAKE

Assigning a Python ``dict`` to ``layer.relocates``. Use a list of ``(Sdf.Path, Sdf.Path)`` tuples.

MISTAKE

Relocating a prim ``def`` in the same layer, then wondering why the destination is empty. The ``def`` is an opinion at the source, so it is thrown away. Reference the prim, or author it at the destination to begin with.

MISTAKE

Relocating to ``/NewRoot/Arm`` without defining ``/NewRoot``. The destination prim never appears; check ``GetPrimAtPath`` rather than assuming an error list.

Exam traps.**TRAP**

"The source-path ``over`` is weaker, so the destination ``over`` wins, but both apply." The source-path opinion does not apply at all.

TRAP

"Relationships to the old path follow the relocate." They do not. ``GetTargets()`` still returns the old path.

Exam takeaways.**KEY**

- Python: list of ``(Sdf.Path, Sdf.Path)`` pairs; USDA: a map in layer metadata. - Author overrides at the `**destination**` path. - Source-path opinions → ``ErrorOpinionAtRelocationSource``, ignored. - Destination parent must exist; missing parent can fail silently. - Relationships do not retarget. Relocates do not rename local defs in the same layer.

Chapter lab(s)

There is no dedicated lab number for this chapter. Lab 13 (references) already authors nested overs. Lab 20 (LIVERPS) includes a relocate. Lab 21 (introspection) prints `GetPrimStack` for an over on a referenced child.

USDA reading exercises

Exercise 20-A. `prop.usda` defines `def Xform "Prop" { def Cube "Lid" { double size = 1 } }`. A shot contains:

```
#usda 1.0

def "P" (
  prepend references = @./prop.usda@
)
{
  over "Lid"
  {
    double size = 3
  }
  over "Tag"
  {
    string label = "hot"
  }
}
```

What is `/P/Lid.size`? Is `/P/Tag` defined? Which prims does `Traverse()` visit?

Exercise 20-B. Using `robot.usda` from Section 20.3, the shot's layer metadata is `relocates = { </Hero/Rig/Arm>: </Hero/Arm> }` and the shot defs `/Hero` with a reference to `robot.usda`. An animator sets `/Hero/Rig/Arm.length = 8`. What is `/Hero/Arm.length`, and why?

Answers

20-A. `size = 3` (local over beats the referenced 1). `/P/Tag` is not defined (overlay-only child). `Traverse()` visits `/P` and `/P/Lid` only. Verified by composing the files.

20-B. `length` stays 1 (the referenced value). The animator's opinion is at the relocation **source**, so USD ignores it and records `ErrorOpinionAtRelocationSource`. The animator should set `/Hero/Arm.length`. Review: §20.4.

Chapter review

Summary

- `over` adds opinions without defining; a typeless `def` defines. Prefer `over` in overlay layers.
- `Traverse()` skips undefined overs; `TraverseAll()` lists them.
- Nested `over` under a referencing prim overrides that copy only; the asset is unchanged.
- Set on a composed child authors the nested `over` for you. Composed `GetSpecifier()` can still be `Def`.
- New children need `def`. Deactivate existing children with `active = false`.
- Relocates are layer metadata mapping source → destination; Python uses a list of path pairs.
- After a relocate the source path is invalid; descendants move; overrides go at the destination.
- Source-path opinions are ignored (`ErrorOpinionAtRelocationSource`). Missing destination parents can fail silently.
- Relationships do not follow relocates. Relocates are the E in LIVERPS.

If you see... → think...

If you see...	Think...
<code>over "X"</code> in a shot	Overlay; existence comes from a <code>def</code> elsewhere
Typeless <code>def "X"</code> in a shot	This defines X, even with no type
Prim missing from <code>Traverse()</code> but valid	Undefined <code>over</code> , inactive, abstract, or unloaded
Nested <code>over "Seat"</code> under a reference	Local L override of that copy
<code>relocates = { </old>: </new> }</code>	Layer metadata; E in LIVERPS
<code>/old</code> invalid after a move	Expected; use <code>/new</code>
<code>ErrorOpinionAtRelocationSource</code>	You authored at the old path
Relocate "did nothing"	Missing dest parent, or source was a local <code>def</code>
<code>layer.relocates = { "a": "b" }</code>	<code>TypeError</code> ; use a list of <code>Sdf.Path</code> pairs

Review questions

R20-01 · Obj 1.8 · Single choice What is the difference between `over "Hero"` and `def "Hero"` in a layer opened by itself?

A. None; both define a typeless prim B. `over` does not define; `def` does C. `over` is weaker for values D. `def` is illegal without a type

R20-02 · Obj 1.8 · Single choice A shot `overs` a referenced Cube's `size`. What is the composed specifier of that Cube?

A. `Sdf.SpecifierOver` B. `Sdf.SpecifierDef` C. `Sdf.SpecifierClass` D. Undefined

R20-03 · Obj 1.8 · Single choice Which USDA hides a referenced child without editing the asset?

A. `delete "Back"` B. `over "Back" (active = false) {}` C. `over "Back" { double size = 0 }` D. `class "Back" {}`

R20-04 · Obj 1.8 · Select two. You want a new locator under a referenced lamp. Which authors a defined child?

A. `over "Locator" {}` B. `def Xform "Locator" {}` C. `def "Locator" {}` D. `class "Locator" {}`

R20-05 · Obj 1.8 · Single choice Where must `relocates` be authored?

A. Prim metadata on the moved prim B. Layer metadata of a layer in the local stack C. Attribute `xformOp:relocate` D. Only inside the referenced file

R20-06 · Obj 1.8 · Single choice After relocating `/Bot/Rig/Arm` to `/Bot/Arm`, `GetPrimAtPath("/Bot/Rig/Arm")` returns:

A. A valid alias of `/Bot/Arm` B. An invalid prim C. The `Rig` prim D. A class prim

R20-07 · Obj 6.2 · Single choice Which Python assignment is correct on USD 26.08?

A. `layer.relocates = {"/A": "/B"}` B. `layer.relocates = [(Sdf.Path("/A"), Sdf.Path("/B"))]` C. `prim.GetRelocates().Add("/A", "/B")` D. `layer.SetRelocate("/A", "/B")`

R20-08 · Obj 6.2 · USDA reading `robot.usda` defines `/Robot/Rig/Arm` with `length = 1`. The shot relocates that arm to `/Bot/Arm` and authors `over "Arm" { double length = 4 }` under `/Bot`. What is `/Bot/Arm.length`?

A. 1 · B. 4 · C. 5 · D. no value

R20-09 · Obj 6.2 · Single choice A relationship on `/Bot` still targets `</Bot/Rig/Arm>` after that path was relocated to `/Bot/Arm`. `GetTargets()` returns:

A. `[/Bot/Arm]` automatically B. `[/Bot/Rig/Arm]` C. An empty list D. A composition error and no targets

R20-10 · Obj 1.8 · Select two. Which are valid reasons a local opinion does not show up on a composed prim (Obj 1.8)?

A. It was authored at a relocation source path B. It is an `over` on a child that nothing defs, and you looked at `Traverse()` C. USDA was used instead of USDC D. The opinion is in the session layer

R20-11 · Obj 6.2 · Single choice You relocate `/A/B` to `/A/C` in the same layer that defs `/A/B`. What happens?

A. `/A/C` becomes a copy of `/A/B` B. The local def is ignored (source-path opinion); `/A/C` does not receive it C. USD raises at parse time D. Both paths remain valid aliases

R20-12 · Obj 1.8 · Single choice LIVERPS: `relocates (E)` sit immediately:

A. Before Local B. Between VariantSets and References C. After Specializes D. Between Payloads and Specializes

Review answers

R20-01 — B. Specifier is existence. Review: §20.1.

R20-02 — B. The asset `def`'d the Cube; composed specifier stays Def. Review: §20.2.

R20-03 — B. Deactivate with metadata on an `over`. Review: §20.2.

R20-04 — B and C. Either `def` defines the child (typed or typeless). `over` does not; `class` is abstract. Review: §20.1, §20.2.

R20-05 — B. Layer metadata. Review: §20.3.

R20-06 — B. Source path is invalid. Review: §20.3.

R20-07 — B. List of `Sdf.Path` pairs. No prim API (C, D). Dict of strings fails (A). Review: §20.4.

R20-08 — B. Destination `over` is local and wins. Review: §20.3, §20.4.

R20-09 — B. Targets are not rewritten. Review: §20.4.

R20-10 — A and B. Session-layer opinions *do* take effect (they are strongest local). File format is irrelevant. Review: §20.1, §20.4.

R20-11 — B. Relocates are not a local rename. Review: §20.4.

R20-12 — B. `L I V E R P S`. Review: §20.3.

CHAPTER 21

Chapter 21 — LIVERPS and Value Resolution

Exam domain: Composition (23%) · **Objectives:** 1.1, 1.6, 1.8 · **Study day:** 7 · **Est. time:** 150 min **Prerequisites:** Ch 10 (time samples, layer offsets), Ch 14–20 (every composition arc)

This is the chapter that turns the arcs you already know into a **prediction skill**. You will recite LIVERPS, split strength into “inside a stack” versus “across arcs,” resolve defaults, time samples, clips, and fallbacks, and work ten puzzles the way the exam likes them: given USDA, what is the composed value, and why?

Learning goals

- Recite LIVERPS in strength order and name what each letter does (Obj 1.6).
- Change which opinion wins by moving it in the stack or across arcs (Obj 1.1).
- Resolve an attribute: time samples, clips, default, schema fallback, then weaker sites.
- Apply a layer offset: $\text{stageTime} = \text{offset} + \text{scale} \times \text{sourceTime}$.
- Explain how value clips feed samples, and how an authored default hides them.
- Predict ten mixed-arc compositions and name the winning letter.

Key terms

Term	One-line definition
LIVERPS	Strength order of arcs: Local, Inherits, VariantSets, rElocates, References, Payloads, Specializes
Local (L)	Opinions from the stage’s layer stack (session, root, sublayers)
Value resolution	Picking the composed value of one field at one time
Usd.ResolveInfo	Result of <code>attr.GetResolveInfo(time)</code> : which encoding won
Schema fallback	A schema’s built-in default, used only when nothing authored a value
Property stack	Attribute specs that contribute, strongest first (<code>GetPropertyStack</code>)
Layer offset	<code>(offset, scale)</code> remapping source time into stage time
Value clips	Prim metadata that streams samples from a sequence of files
Winning site	The composition node (arc + layer stack + path) that supplied the value

21.1 LIVERPS spelled out

What is it? LIVERPS is the strength order of composition arcs. When two arcs both have an opinion about the same field, the earlier letter wins.

Why do we need it? A prim can inherit a class, select a variant, relocate a child, reference an asset, load a payload, and specialize a fallback — at once. Without a published order, every DCC would compose a different scene. The exam’s largest domain is this order (Obj 1.6).

Beginner explanation. Read the word as a **queue at a service desk**. Local opinions are at the front. Specializes are at the back. USD asks the queue from the front: the first person who has an answer for this field wins. People who are silent about this field are skipped.

Where the analogy breaks: the queue is rebuilt **per prim** and the question is asked **per field**. Also, following a reference starts a *new* queue inside that asset (nested LIVERPS).

Technical explanation. | Letter | Name | Typical authoring | Stronger than |
 |-----|-----|-----|-----| | **L** | Local | Session, root, subLayers; nested over | I V
 E R P S | | **I** | Inherits | prepend inherits = </_Class> | V E R P S | | **V** | VariantSets |
 variantSet "lod" = { "high" { ... } } | E R P S | | **E** | rElocates | Layer metadata
 relocates = { </old>: </new> } | R P S | | **R** | References | prepend references = @asset@
 | P S | | **P** | Payloads | prepend payload = @heavy@ | S | | **S** | Specializes | prepend
 specializes = </_Defaults> | (nothing) |

- **Local** is not “this one file.” It is the whole **layer stack** of the current node: session layer, root, sublayers in order (Chapter 15).
- Relocates do not usually fight a number against a reference; they **move** the referenced prim. Local opinions at the *destination* are still L and beat the moved content (Chapter 20).
- Arcs **nest**. A referenced asset has its own sublayers, inherits, and payloads. USD walks LIVERPS at the referencing site, then walks LIVERPS again inside the referenced layer stack.
- List-editing of arc lists (prepend / append) changes strength **among arcs of the same letter**, not across letters. Two references: the first prepended is stronger. A reference never beats a local over by being prepended.

VERSION

Verified on USD 26.08. The OpenUSD glossary uses ****LIVERPS****. Older talks and some NVIDIA study material say ****LIVRPS****, which is the same order without relocates. If a question omits E, keep L I V R P S.

EXAM TIP

Composition is 23% of the exam. A question that lists two arcs and two numbers is almost always asking “which letter is earlier?”

Mental model.

STRONGEST			WEAKEST			
L	I	V	E	R	P	S
local	inherits	variants	relocates	references	payloads	specializes
stack	classes	switches	rename	catalog	lazy	defaults

Simple example. A prim references an asset (`radius = 1`) and inherits a class (`radius = 3`). Composed radius is **3** (I before R). Add a local `radius = 8` and it becomes **8** (L before I).

USDA example.

```
#usda 1.0

class Sphere "_Class"
{
    double radius = 3
}

def Sphere "_Defaults"
{
    double radius = 7
}

def "Ball" (
    prepend inherits = </_Class>
    prepend specializes = </_Defaults>
    prepend references = @./asset.usda@
)
{
    double radius = 8
}
```


Line notes: `asset.usda` (not shown) has `radius = 1`. Local 8 wins. Remove the local line and the class's 3 wins. Remove the inherit and the reference's 1 wins. The specializes 7 never wins while any stronger arc has `radius`.

Python example.

```
from pxr import Usd

with open("asset.usda", "w") as f:
    f.write("""#usda 1.0
(
    defaultPrim = "X"
)
def Sphere "X"
{
    double radius = 1
}
""")
with open("scene.usda", "w") as f:
    f.write("""#usda 1.0
class Sphere "_Class"
{
    double radius = 3
}
def Sphere "_Defaults"
{
    double radius = 7
}
def "Ball" (
    prepend inherits = </_Class>
    prepend specializes = </_Defaults>
    prepend references = @./asset.usda@
)
{
    double radius = 8
}
""")

stage = Usd.Stage.Open("scene.usda")
ball = stage.GetPrimAtPath("/Ball")
print("radius =", ball.GetAttribute("radius").Get())
for arc in Usd.PrimCompositionQuery(ball).GetCompositionArcs():
    print(str(arc.GetArcType()))
```

Expected output

```
radius = 8.0
Pcp.ArcTypeRoot
Pcp.ArcTypeInherit
Pcp.ArcTypeReference
Pcp.ArcTypeSpecialize
```

The query lists contributing arcs in strength order. Root is Local. Variant and relocate arcs are absent here. Radius 8 is the local default on that root node.

Real-world use case. A film shot: layout's sublayers are L, a `_class_Hero` inherit tints every hero instance (I), an `outfit` variant picks a costume (V), a rig relocate shortens joint paths (E), each hero references `hero.usda` (R), heavy facial shapes sit in a payload (P), and `_HeroFallback` specializes give shader defaults that any of the above may override (S).

Common mistakes.

MISTAKE

Reciting "references then payloads then variants." Variants (V) are stronger than references (R).

MISTAKE

Treating inherits and specializes as the same because both point at a class-like prim. I is near the top; S is last.

Exam traps.**TRAP**

"Payloads are weaker than references" is a ****strength**** statement. It is not about loading. Unloaded payloads contribute nothing; loaded ones still lose to references on the same field.

TRAP

A local empty ``over "Ball" {}`` does not beat a referenced radius. Local wins only when it ****authors that field****.

Exam takeaways.**KEY**

- LIVERPS: Local, Inherits, VariantSets, rLocates, References, Payloads, Specializes. - Older spelling LIVRPS omits E; the rest is unchanged. - Same-letter lists use prepend/append; letters do not jump the queue. - Nested: each referenced asset has its own LIVERPS walk.

21.2 Strength within a layer stack vs. across arcs

What is it? Strength is a **two-step** rank. First USD ranks layers **inside** the current layer stack. Then it ranks that whole stack against other stacks brought in by arcs.

Why do we need it? Obj 1.1 is “change the strength of an opinion.” You need to know whether to reorder sublayers or to change the arc. Reordering sublayers will not beat a stronger *letter*. Authoring locally will.

Beginner explanation. A layer stack is a **pile of transparent sheets** (Chapter 3). Arcs are **links to other piles**. USD looks at your pile from the top sheet down (session → root → first sublayer → ...). If your pile is silent, it follows a link to another pile, in LIVERPS order, and looks through that pile the same way.

Where the analogy breaks: a link can itself have a pile (the referenced file’s sublayers). You never flatten the piles unless you call `Flatten`.

Technical explanation. Step A — inside one stack (all of this is letter L for the stage’s root node):

1. Session layer (and its sublayers)
2. Root layer
3. `subLayers[0]` and *its* sublayers, depth-first
4. `subLayers[1]` ...

Step B — across arcs: L, then I, then V, then E, then R, then P, then S. Each of those nodes has its own stack; Step A runs there too.

How to make an opinion win (Obj 1.1):

Goal	Move
Beat another department on this shot	Author in a stronger sublayer, or reorder <code>subLayers</code>
Beat the asset without editing it	Author locally (L) on the composed path
	Put the value on an inherited class (I)

Goal	Move
Beat a reference with a shared edit	
Beat a payload	Author a reference opinion, or a local opinion
See who currently wins	<code>attr.GetPropertyStack()</code> (specs, strongest first) and <code>attr.GetResolveInfo()</code>

`GetPropertyStack()` lists `Sdf.AttributeSpec` objects. Silent layers do not appear. The first spec that actually has the encoding you asked for is the winner for that time.

Mental model.

```
ask L stack first: [session] -> [root] -> [sub0] -> [sub1]
                  |           |
                  radius=5    radius=1    --> 5 wins, stop
                  (if session silent, try root, then sublayers)
if L is silent about this field, ask I, then V, then E, then R, then P, then S
```

Simple example. Root authors `radius = 1`. Session authors `radius = 5`. Composed value is 5. The asset is not involved. Property stack: session spec first, then root spec.

USDA example. File: `ball.usda` (root)

```
#usda 1.0

def Sphere "Ball"
{
    double radius = 1
}
```

The session layer is not a file on disk. Python in step 8 authors `radius = 5` there. USDA of the session would look like over `"Ball" { double radius = 5 }`.

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateNew("ball.usda")
sph = UsdGeom.Sphere.Define(stage, "/Ball")
sph.GetRadiusAttr().Set(1.0)
stage.SetEditTarget(stage.GetSessionLayer())
sph.GetRadiusAttr().Set(5.0)
attr = stage.GetPrimAtPath("/Ball").GetAttribute("radius")
print("composed radius =", attr.Get())
print("property stack (strongest first):")
for spec in attr.GetPropertyStack():
    print(" ", spec.layer.GetDisplayName(), "default =", spec.default)
ri = attr.GetResolveInfo()
print("ResolveInfo source:", ri.GetSource())
print("node arcType:", ri.GetNode().arcType)
```

Expected output

```
composed radius = 5.0
property stack (strongest first):
  ball-session.usda default = 5.0
  ball.usda default = 1.0
ResolveInfo source: Usd.ResolveInfoSourceDefault
node arcType: Pcp.ArcTypeRoot
```

Both specs are on the **root** (local) node. The session spec is stronger inside that stack. `GetSource()` says the winning encoding is a default value, not time samples.

Real-world use case. Lighting needs a brighter practical. They do not reorder layout's sublayers (that would reshuffle the whole shot). They author in `shot_lgt.usda`, listed before `shot_lay.usda`, or they author in the session layer for a temporary look. Both are Step A. If the value lives inside the referenced fixture asset, Step A is not enough unless they over the fixture locally (Step B: add an L opinion).

Common mistakes.**MISTAKE**

Reordering sublayers of the **shot** to override a value that lives only inside a **referenced** asset. Sublayers of the shot are L at the shot's paths. The asset's internal layers sit on the R node. Add a local *`over`*, or edit the asset.

MISTAKE

Reading *`GetPropertyStack()`* as "every layer in the stage." Only specs that contribute to ***this*** property are listed.

Exam traps.**TRAP**

"Session layer is an arc." It is the strongest sheet of the ***local*** stack, not a LIVERPS letter.

TRAP

"First-listed sublayer is weakest." First-listed is strongest (Chapter 15).

Exam takeaways.**KEY**

- Two steps: rank the stack, then rank the arc. - Session → root → first sublayer is all Local. - *`GetPropertyStack()`* is strongest-first specs for one property. - To beat an asset, author locally; reordering unrelated sublayers will not.

21.3 Value resolution: defaults, time samples, fallbacks

What is it? **Value resolution** is how USD answers *attr.Get(time)* after composition has ranked the specs. It chooses among time samples, value clips, a default, a schema fallback, or "no value."

Why do we need it? Two specs can both "win" composition and still disagree about *time*: one has a still default, the other a flip-book of samples. The exam mixes this with LIVERPS (Obj 1.8: "I set a default, why does frame 0 ignore it?").

Beginner explanation. Composition picks the **ladder**. Value resolution picks which **writing on that ladder's rungs** counts at this moment. A still photograph (default) is used when you ask for the default time, or when nobody shot a flip-book. A flip-book (samples) is used when you ask for a frame number.

Where the analogy breaks: if a flip-book exists, asking for a frame **never** falls back to the photograph on that same spec (Chapter 10). Frames before the first sample use the **first sample**, not the default.

Technical explanation. Verified on USD 26.08. *attr.GetResolveInfo(time).GetSource()* returns one of:

Source	Meaning
<i>ResolveInfoSourceTimeSamples</i>	Authored samples (interpolated if needed)
<i>ResolveInfoSourceValueClips</i>	Samples streamed from clips (21.5)
<i>ResolveInfoSourceSpline</i>	Spline curve (overview only; Ch 10)
<i>ResolveInfoSourceDefault</i>	Attribute default (<i>TimeCode.Default()</i> or no samples)
<i>ResolveInfoSourceFallback</i>	Schema built-in (<i>Sphere radius</i> is 1)
<i>ResolveInfoSourceNone</i>	No value

Rules that bite on the exam:

- `Get()` with no time uses the default if one is authored; otherwise it can still return a fallback.
- `Get(numeric)` **ignores the default on a spec that has samples**. Pre-first-sample → first sample. After last sample → last sample. Between samples → linear interpolation unless you `stage.SetInterpolationType(Usd.InterpolationTypeHeld)`.
- Schema fallback is **not an opinion**. It is used only when no contributing spec authored a value. `HasAuthoredValue()` is False; `HasValue()` is True.
- A **value block** (`double radius = None`) is an opinion of “no value” that hides weaker opinions. `Get()` returns None.
- Composition strength still comes first. A local default beats referenced samples, because the local spec is a stronger site and it *does* have a value (the default) at numeric times when that spec has no samples of its own.

Mental model.

```
for this field, at this time:
1. walk specs in strength order (stack, then LIVERPS)
2. on the strongest spec that has a value encoding:
   samples (or clips/spline) if asking at a numeric time
   else default
3. if nobody authored anything: schema fallback
```

Simple example. Sphere with fallback only: `Get()` is 1, source Fallback. Set default 3: `Get()` is 3, source Default. Add a sample 8 at time 10: `Get(Default())` is 3, `Get(0)` is 8 (first sample), `Get(10)` is 8.

USDA example.

```
#usda 1.0

def Sphere "HasSamples"
{
    double radius = 2
    double radius.timeSamples = {
        10: 8
    }
}
```

Line notes: both encodings live on one spec. `Get(Usd.TimeCode.Default())` reads = 2. Any numeric `Get` reads the sample dictionary.

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
empty = UsdGeom.Sphere.Define(stage, "/HasFallback").GetRadiusAttr()
authored = UsdGeom.Sphere.Define(stage, "/HasDefault").GetRadiusAttr()
authored.Set(3.0)
sampled = UsdGeom.Sphere.Define(stage, "/HasSamples").GetRadiusAttr()
sampled.Set(2.0)
sampled.Set(8.0, 10)
print("fallback Get() =", empty.Get(), empty.GetResolveInfo().GetSource())
print("default Get() =", authored.Get(),
      authored.GetResolveInfo().GetSource())
print("samples Default() =", sampled.Get(Usd.TimeCode.Default()),
      sampled.GetResolveInfo(Usd.TimeCode.Default()).GetSource())
print("samples Get(0) =", sampled.Get(0),
      sampled.GetResolveInfo(0).GetSource())
print("samples Get(10) =", sampled.Get(10),
      sampled.GetResolveInfo(10).GetSource())
```

Expected output

```
fallback Get() = 1.0 Usd.ResolveInfoSourceFallback
default Get() = 3.0 Usd.ResolveInfoSourceDefault
```

```

samples Default() = 2.0 Usd.ResolveInfoSourceDefault
samples Get(0) = 8.0 Usd.ResolveInfoSourceTimeSamples
samples Get(10) = 8.0 Usd.ResolveInfoSourceTimeSamples

```

`Get(0)` is 8, not 2: numeric queries do not use the default once samples exist.

Real-world use case. A robot's rest pose is the default. Animation caches write samples. A lighter who keys `radius` (or `xformOp:translate`) as a **default** on a shot overlay wonders why the playback still uses the cache: they asked for frames, so samples on a weaker layer can still lose to a *stronger* default, but a default on the *same* spec as the samples is ignored at those frames. Check `GetResolveInfo(frame).GetSource()` before arguing about LIVERPS.

Common mistakes.

MISTAKE

Assuming ``Get(0)`` returns the default. If samples exist, you get the first sample (or interpolated value).

MISTAKE

Calling schema fallback an "opinion." ``HasAuthoredValue()`` is False. You cannot "override the fallback" by reordering sublayers if nobody authored the field — you must author it.

Exam traps.

TRAP

"Defaults always lose to samples, even from a weaker layer." Strength first. A local default beats referenced samples.

TRAP

Sphere ``radius`` is 1 because "someone authored 1." It is often the fallback. Check ``GetResolveInfo().GetSource()``.

Exam takeaways.

KEY

- Strength first, then encoding (samples / clips / default / fallback). - Numeric ``Get`` ignores the default when samples exist on the winning spec. - Fallback is not an opinion. - ``GetResolveInfo(time).GetSource()`` names the encoding.

21.4 Layer offsets in resolution

What is it? A **layer offset** remaps time on a sublayer or a reference (or payload): each source time sample is read as if it happened at a different stage time.

Why do we need it? Obj 1.4: the same animation, played later or faster, without copying samples. Value resolution must use the **remapped** times, or your hold and your mix will disagree with usdview.

Beginner explanation. The clip was filmed with its own clapboard. The shot's clapboard is different. Offset is "start this clip later." Scale is "play it faster or slower."

Where the analogy breaks: USD does not resample onto integer frames unless you ask. Linear interpolation still runs in **stage** time after the mapping.

Technical explanation.

```

stageTime = offset + scale × sourceTime
sourceTime = (stageTime - offset) / scale

```

- USDA, on a reference: `@anim.usda@ (offset = 5; scale = 1)`
- USDA, on a sublayer: `subLayers = [@anim.usda@ (offset = 2; scale = 0.5)]`

- Python: `Sdf.LayerOffset(offset, scale)` on `AddReference`, or `layer.subLayerOffsets` (parallel to `subLayerPaths`).
- Defaults (non-time) are **not** moved. Offsets only affect time samples (and clips/splines).
- Before the first remapped sample, `Get(t)` uses the first sample's **value** (same pre-first rule as Chapter 10).
- `scale = 0.5` plays the source at **half speed** in stage time (source 10 occupies 5 stage units after the offset).

Mental model.

```
source samples:  t=0 -> 0      t=10 -> 10
offset=2, scale=0.5

appear on stage: t=2 -> 0      t=7 -> 10
stage t=4 -> source t=4 -> value 4 (linear)
```

Simple example. Sublayer `anim.usda` with samples 0→0 and 10→10, listed as `@anim.usda@ (offset = 2; scale = 0.5)`. `Get(2)` is 0, `Get(7)` is 10, `Get(4)` is 4.

USDA example. File: `anim.usda`

```
#usda 1.0

def Sphere "Ball"
{
    double radius.timeSamples = {
        0: 0,
        10: 10
    }
}
```

File: `shot.usda`

```
#usda 1.0
(
    subLayers = [
        @./anim.usda@ (offset = 2; scale = 0.5)
    ]
)
```

Line notes: the offset is on the **sublayer entry**, not on the prim. Semicolon separates offset and scale inside those parentheses.

Python example.

```
from pxr import Usd

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("anim.usda", """#usda 1.0
def Sphere "Ball"
{
    double radius.timeSamples = {
        0: 0,
        10: 10
    }
}
""")
write("shot.usda", """#usda 1.0
(
    subLayers = [
        @./anim.usda@ (offset = 2; scale = 0.5)
    ]
)
""")

stage = Usd.Stage.Open("shot.usda")
```

```
attr = stage.GetPrimAtPath("/Ball").GetAttribute("radius")
print("subLayerOffsets =", stage.GetRootLayer().subLayerOffsets)
for t in (0, 2, 4, 7):
    print(f"Get({t}) =", attr.Get(t))
```

Expected output

```
subLayerOffsets = [Sdf.LayerOffset(2, 0.5)]
Get(0) = 0.0
Get(2) = 0.0
Get(4) = 4.0
Get(7) = 10.0
```

Get(0) is still 0: time 0 is before the first remapped sample (which sits at 2), so USD holds the first sample's value.

Real-world use case. A factory-line digital twin reuses `robot_cycle.usda` at three stations. Station B's reference uses `offset = 24` so the cycle starts one second later (at 24 fps). Same samples, different stage times (Obj 1.4, Lab 14).

Common mistakes.

MISTAKE

Using ``stageTime = sourceTime + offset`` and forgetting ``scale``. The full formula multiplies first.

MISTAKE

Expecting ``scale = 2`` to mean "twice as long." ``scale = 2`` maps source 1 to a +2 stage step: the clip plays **faster** (twice the speed) if you think in "stage seconds per source second."

Exam traps.

TRAP

Offsets on sublayers versus references look different in USDA but use the same ``Sdf.LayerOffset`` math.

TRAP

"Offset moves the default pose." Defaults are unvarying; only samples move.

Exam takeaways.

KEY

- ``stageTime = offset + scale × sourceTime``. - Same struct on sublayers and on references.
- Defaults do not shift; samples do. - Pre-first remapped sample still uses the first sample's value.

21.5 Value clips in resolution

What is it? **Value clips** are prim metadata that tell USD to pull time samples from a **sequence of other files** instead of storing every sample on the prim. Resolution can then report `ResolveInfoSourceValueClips`.

Why do we need it? A 10 000-frame cache in one layer is slow to open and huge to version. Clips keep each chunk (or each frame) in its own file and activate them over time. Chapter 10 introduced the idea; this section is how they lose or win during `Get(time)`.

Beginner explanation. Think of a **shelf of tape reels**. Each reel is a small USD file with a few samples. A card on the shelf (clips metadata) says which reel is in the machine at each shot time.

Where the analogy breaks: clips are not a LIVERPS letter. They are a **value encoding** on whatever site already won composition. An authored **default** on that same prim hides the reels.

Technical explanation. Verified on USD 26.08:

- Author with `Usd.ClipsAPI(prim): SetClipAssetPaths, SetClipPrimPath, SetClipActive, SetClipTimes`. USDA stores a clips dictionary on the prim.
- `active` pairs (`stageTime`, `clipIndex`) pick which clip file is current. `times` pairs map stage time to time-inside-the-clip.
- `Get(time)` interpolates across clip boundaries when interpolation is linear (clip1 value 1 at $t=0$, clip2 value 5 at $t=10 \rightarrow t=5$ is 3).
- **An authored default on that attribute blocks clips** at every time, including numeric times. `GetResolveInfo` then says `Default`. Local time samples also beat clips.
- Clips are metadata, not an arc. Strength of the *prim site* still follows LIVERPS. A local default (L) hides clips on that prim; removing the default lets clips show through.

VERSION

Splines: Ts library 24.03 (not yet used by USD attributes); ``UsdAttribute` resolution / `ResolveInfoSourceSpline` 25.05; Get/SetSpline APIs 25.08. The exam's core encodings remain samples, defaults, and clips-at-overview.`

Mental model.

`Get(t)` encodings on the winning spec, strongest first:
 authored time samples > clips > default > schema fallback
 (numeric `t` ignores default when samples exist; default still hides clips)

Simple example. Two clip files, `radius 1` and `5`. Active switches at 0 and 10. `Get(0)=1`, `Get(10)=5`, `Get(5)=3`. Set `radius = 9` on the prim and `Get(5)` becomes 9.

USDA example.

```
#usda 1.0

def Sphere "Ball" (
  clips = {
    dictionary default = {
      double2[] active = [(0, 0), (10, 1)]
      asset[] assetPaths = ["/clip1.usda@", "/clip2.usda@"]
      string primPath = "/Ball"
      double2[] times = [(0, 0), (10, 0)]
    }
  }
)
{
}
```

Line notes: `primPath` is the path **inside each clip file**. `assetPaths` is the ordered reel list. Keep the attribute itself empty if you want the clips to win.

Python example.

```
from pxr import Usd, UsdGeom

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("clip1.usda", """"#usda 1.0
def Sphere "Ball"
{
```

```

    double radius.timeSamples = {
        0: 1
    }
}
"""
write("clip2.usda", ""#usda 1.0
def Sphere "Ball"
{
    double radius.timeSamples = {
        0: 5
    }
}
"""

stage = Usd.Stage.CreateNew("clips.usda")
ball = UsdGeom.Sphere.Define(stage, "/Ball")
api = Usd.ClipsAPI(ball)
api.SetClipAssetPaths(["./clip1.usda", "./clip2.usda"])
api.SetClipPrimPath("/Ball")
api.SetClipActive([(0.0, 0.0), (10.0, 1.0)])
api.SetClipTimes([(0.0, 0.0), (10.0, 0.0)])
attr = ball.GetRadiusAttr()
for t in (0, 5, 10):
    print(f"clip Get({t}) =", attr.Get(t),
          attr.GetResolveInfo(t).GetSource())
attr.Set(9.0)
print("after default 9, Get(5) =", attr.Get(5),
      attr.GetResolveInfo(5).GetSource())

```

Expected output

```

clip Get(0) = 1.0 Usd.ResolveInfoSourceValueClips
clip Get(5) = 3.0 Usd.ResolveInfoSourceValueClips
clip Get(10) = 5.0 Usd.ResolveInfoSourceValueClips
after default 9, Get(5) = 9.0 Usd.ResolveInfoSourceDefault

```

Real-world use case. A VFX cache department publishes `hero_sim.####.usd` as clips on `/Hero`. Lighting never authors a default `xform0p:translate` on that prim; if they did, the whole sim would freeze at the default pose.

Common mistakes.

MISTAKE

Defining the prim with ``double radius = 1`` "so it has a rest pose" ****and**** attaching clips. The rest pose hides the clips. Leave the attribute unauthored, or author samples.

MISTAKE

Thinking clips are weaker than payloads. Clips are not an arc.

Exam traps.

TRAP

"``Get(5)`` with clips interpolates unless a default exists." A default on that attribute stops clip resolution entirely.

TRAP

"Clips are the C in LIVERPS." There is no C. Encodings $\neq$ arcs.

Exam takeaways.

KEY

- Clips stream samples; ``GetResolveInfo`` can say ``ValueClips``. - They are an encoding, not a LIVERPS letter.
- Authored defaults (and local samples) hide clips.
- Leave clip-driven attributes unauthored on the prim.

21.6 Ten worked LIVERPS puzzles

What is it? Ten small compositions. For each one you predict the composed value, then check it. This is the exam format: USDA in, number out, letter as the reason.

Why do we need it? Isolated chapters make each arc look easy. The exam stacks them. Working the set until the letter comes to mind before the number is the study method for Days 6–7.

Beginner explanation. Each puzzle is a **who-speaks-loudest** contest. Write the letters that are present, drop the ones that are silent about the field, and keep the earliest letter's number.

Where the analogy breaks: P6's payload still contributes `extra`, which the reference never mentioned. Weaker arcs can fill fields the stronger arc left blank.

Technical explanation. Use this checklist on every puzzle:

1. Which arcs are present? Write L I V E R P S and cross out missing ones.
2. Which of those authored **this field**?
3. Earliest remaining letter wins.
4. If the winner has samples or clips, apply Chapter 10 / 21.3–21.5 at the requested time.
5. If you authored at a relocation **source**, that opinion does not count (Chapter 20).

Mental model.

```
present: L I V E R P S
has field: I R
winner: I
```

Simple example. The table below is the answer key. Step 8 produces it from files.

#	Contest	Winner	Value
P1	Local 2 vs reference 1	L	2
P2	Sublayer first-listed 5 vs second 9	L (order)	5
P3	Inherit 3 vs reference 1	I	3
P4	Local 8 vs inherit 3	L	8
P5	Variant 10 vs reference 1	V	10
P6	Reference 1 vs payload 9; payload <code>extra</code>	R for radius; P for <code>extra</code>	1 and 1
P7	Specialize 7 vs reference 1	R	1
P8	Dest <code>over</code> 4 on a relocated arm	L at dest	4
P9	Default 2, sample <code>{10: 8}</code> , <code>Get(0)</code>	samples	8
P10	Reference offset 5, samples 0→1 and 10→11	offset math	t=5 → 1, t=10 → 6, t=15 → 11

USDA example. *P1 — local beats reference*

```
#usda 1.0

def "Ball" (
  prepend references = @./asset.usda@
)
{
  double radius = 2
}
```

P7 — reference beats specializes

```
#usda 1.0

def Sphere "_D"
{
    double radius = 7
}

def "Ball" (
    prepend specializes = </_D>
    prepend references = @./asset.usda@
)
{
}
```

asset.usda defines Sphere "X" with radius = 1 and defaultPrim = "X". P1 composes to 2. P7 composes to 1.

Python example.

```
from pxr import Usd, UsdGeom

def write(name, text):
    with open(name, "w") as f:
        f.write(text)

write("asset.usda", ""#usda 1.0
(
    defaultPrim = "X"
)
def Sphere "X"
{
    double radius = 1
}
""")
write("pay.usda", ""#usda 1.0
(
    defaultPrim = "X"
)
def Sphere "X"
{
    double radius = 9
    double extra = 1
}
""")
write("s1.usda", ""#usda 1.0
over "Ball"
{
    double radius = 5
}
""")
write("s2.usda", ""#usda 1.0
def Sphere "Ball"
{
    double radius = 9
}
""")
write("robot.usda", ""#usda 1.0
(
    defaultPrim = "Robot"
)
def Xform "Robot"
{
    def Xform "Rig"
    {
        def Sphere "Arm"
        {
            double radius = 1
        }
    }
}
""")
write("anim.usda", ""#usda 1.0
```

```

(
    defaultPrim = "Ball"
)
def Sphere "Ball"
{
    double radius.timeSamples = {
        0: 1,
        10: 11
    }
}
"""

write("p1.usda", ""#usda 1.0
def "Ball" (
    prepend references = @./asset.usda@
)
{
    double radius = 2
}
"""
st = Usd.Stage.Open("p1.usda")
print("P1 L>R", st.GetPrimAtPath("/Ball").GetAttribute("radius").Get())

write("p2.usda", ""#usda 1.0
(
    subLayers = [ @./s1.usda@, @./s2.usda@ ]
)
"""
st = Usd.Stage.Open("p2.usda")
print("P2 sublayer-first",
    st.GetPrimAtPath("/Ball").GetAttribute("radius").Get())

write("p3.usda", ""#usda 1.0
class Sphere "_C"
{
    double radius = 3
}
def "Ball" (
    prepend inherits = </_C>
    prepend references = @./asset.usda@
)
{
}
"""
st = Usd.Stage.Open("p3.usda")
print("P3 I>R", st.GetPrimAtPath("/Ball").GetAttribute("radius").Get())

write("p4.usda", ""#usda 1.0
class Sphere "_C"
{
    double radius = 3
}
def Sphere "Ball" (
    prepend inherits = </_C>
)
{
    double radius = 8
}
"""
st = Usd.Stage.Open("p4.usda")
print("P4 L>I", st.GetPrimAtPath("/Ball").GetAttribute("radius").Get())

write("p5.usda", ""#usda 1.0
def "Ball" (
    prepend references = @./asset.usda@
    prepend variantSets = "size"
    variants = {
        string size = "big"
    }
)
{
    variantSet "size" = {
        "big" {
            double radius = 10

```

```

    }
    "small" {
        double radius = 0.2
    }
}
"""
st = Usd.Stage.Open("p5.usda")
print("P5 V>R", st.GetPrimAtPath("/Ball").GetAttribute("radius").Get())

write("p6.usda", ""#usda 1.0
def "Ball" (
    prepend references = @./asset.usda@
    prepend payload = @./pay.usda@
)
{
}
"""
st = Usd.Stage.Open("p6.usda")
b = st.GetPrimAtPath("/Ball")
print("P6 R>P", b.GetAttribute("radius").Get(),
      "extra", b.GetAttribute("extra").Get())

write("p7.usda", ""#usda 1.0
def Sphere "_D"
{
    double radius = 7
}
def "Ball" (
    prepend specializes = </_D>
    prepend references = @./asset.usda@
)
{
}
"""
st = Usd.Stage.Open("p7.usda")
print("P7 R>S", st.GetPrimAtPath("/Ball").GetAttribute("radius").Get())

write("p8.usda", ""#usda 1.0
(
    relocates = {
        </Bot/Rig/Arm>: </Bot/Arm>
    }
)
def Xform "Bot" (
    prepend references = @./robot.usda@
)
{
    over "Arm"
    {
        double radius = 4
    }
}
"""
st = Usd.Stage.Open("p8.usda")
print("P8 L-at-dest>E",
      st.GetPrimAtPath("/Bot/Arm").GetAttribute("radius").Get())

st = Usd.Stage.CreateInMemory()
a = UsdGeom.Sphere.Define(st, "/S").GetRadiusAttr()
a.Set(2.0)
a.Set(8.0, 10)
print("P9 Default", a.Get(Usd.TimeCode.Default()),
      "Get(0)", a.Get(0),
      a.GetResolveInfo(0).GetSource())

write("p10.usda", ""#usda 1.0
def "Ball" (
    prepend references = @./anim.usda@ (
        offset = 5
        scale = 1
    )
)
{

```

```

}
"""
st = Usd.Stage.Open("p10.usda")
a = st.GetPrimAtPath("/Ball").GetAttribute("radius")
print("P10 t=5", a.Get(5), "t=10", a.Get(10), "t=15", a.Get(15))

```

Expected output

```

P1 L>R 2.0
P2 sublayer-first 5.0
P3 I>R 3.0
P4 L>I 8.0
P5 V>R 10.0
P6 R>P 1.0 extra 1.0
P7 R>S 1.0
P8 L-at-dest>E 4.0
P9 Default 2.0 Get(0) 8.0 Usd.ResolveInfoSourceTimeSamples
P10 t=5 1.0 t=10 6.0 t=15 11.0

```

P10 check: source 0 appears at stage 5 (value 1). Source 10 appears at stage 15 (value 11). Stage 10 → source 5 → linear halfway from 1 to 11 = 6.

Real-world use case. A lead lighting TD debugs a “wrong roughness” ticket by walking this table: is it a local light-tweak (L), a class broadcast (I), an outfit variant (V), a referenced material (R), or a specialized fallback (S)? Naming the letter tells them which file to open.

Common mistakes.

MISTAKE

Adding the two radii. Composition does not sum. One winner.

MISTAKE

In P6, concluding the payload is unused because radius came from the reference. `extra` still comes from the payload.

Exam traps.

TRAP

P7 looks like "class-like prim, so 7." Specializes is the weakest letter.

TRAP

P9 looks like LIVERPS. It is value resolution on one spec: numeric time ignores the default.

Exam takeaways.

KEY

- Cross out silent letters; the earliest remaining letter wins. - Weaker arcs may still fill other fields. - Relocate puzzles: author at the destination (L). - Not every "wrong value" is LIVERPS; check samples vs default vs offset vs clips.

Chapter lab(s)

Lab 20 (LIVERPS laboratory) is the practice set for this chapter: predict, then run. Lab 14 drills layer offsets. Lab 21 prints stacks and `PrimCompositionQuery`.

USDA reading exercises

Exercise 21-A. `asset.usda` has `radius = 1`. The shot is:

```

#usda 1.0

class Sphere "_C"
{
    double radius = 3
}

```

```
def "Ball" (
  prepend inherits = </_C>
  prepend references = @./asset.usda@
)
{
}
```

What is `/Ball.radius`, and which letter won?

Exercise 21-B. Same asset. A reference uses `(offset = 5; scale = 1)`. Samples in the asset are `{0: 1, 10: 11}`. What is `Get(10)`?

Answers

21-A. 3, letter I. Verified as P3.

21-B. 6. Source time = $10 - 5 = 5$, halfway from 1 to 11. Verified as P10.

Chapter review

Summary

- LIVERPS is strongest-to-weakest: Local, Inherits, VariantSets, rElocates, References, Payloads, Specializes.
- Strength is two steps: rank the layer stack, then rank the arc. Nested assets repeat the walk.
- Change strength (Obj 1.1) by authoring in a stronger site, reordering sublayers, or changing the arc — not by a priority number.
- Value resolution: samples/clips/spline, then default, then schema fallback. Numeric `Get` ignores the default when samples exist.
- Layer offset: `stageTime = offset + scale × sourceTime`.
- Clips are an encoding. An authored default hides them.
- Ten puzzles: L>R, sublayer order, I>R, L>I, V>R, R>P, R>S, L-at-relocate-dest, samples vs default, offset math.

If you see... → think...

If you see...	Think...
Two arcs, two numbers	Earliest LIVERPS letter
"Change who wins"	Stronger layer, reorder sublayers, or stronger arc (Obj 1.1)
<code>GetPropertyStack()</code>	Strongest-first specs for one property
<code>GetResolveInfo().GetSource()</code>	Encoding: samples / clips / default / fallback
<code>Get(0) ≠ default</code>	Samples exist; pre-first uses first sample
Sphere radius 1, nothing authored	Fallback, not an opinion
<code>(offset = a; scale = b)</code>	$stage = a + b \times source$
Clips plus <code>double x = 9</code>	Default hides clips
LIVERPS in a question	Same order, no relocates
Payload and reference both set <code>x</code>	Reference wins; other payload fields may still show

Review questions

R21-01 · Obj 1.6 · Single choice The E in LIVERPS stands for: A. Edits · B. rElocates · C. Extents · D. Export

R21-02 · Obj 1.6 · Single choice Which pair is in the correct strength order (stronger first)? A. Payloads, References · B. Specializes, Inherits C. VariantSets, References · D. References, Local

R21-03 · Obj 1.1 · Single choice The only `radius` opinion lives in a referenced asset.

Fastest way to override it in the shot? A. Mute the root layer B. Local over of `radius` on the referencing prim C. Change `defaultPrim` D. Convert USDA to USDC

R21-04 · Obj 1.1 · Single choice `subLayers = [@a.usda@, @b.usda@]`. a has `radius` 5, b has 9. Composed `radius`? A. 9 · B. 5 · C. 14 · D. fallback 1

R21-05 · Obj 1.8 · Single choice Default 2, samples {10: 8}.

`Get(Usd.TimeCode.Default())` is: A. 8 · B. 2 · C. 5 · D. None

R21-06 · Obj 1.8 · Single choice Same attribute. `Get(0)` is: A. 2 · B. 8 · C. 0 · D. fallback 1

R21-07 · Obj 1.4 · Single choice Reference offset 5, scale 1, source sample at 0 equals 1.

`Get(5)` is: A. 1 · B. 5 · C. no value · D. 0

R21-08 · Obj 1.6 · Select two. A prim references `radius = 1` and payloads `radius = 9` plus `extra = 1`. Which are true? A. `radius` is 1 B. `radius` is 9 C. `extra` is 1 D. Specializes would beat the reference

R21-09 · Obj 1.8 · Single choice Clips would give 3 at t=5. The prim also authors `double radius = 9`. `Get(5)` is: A. 3 · B. 9 · C. 6 · D. 1

R21-10 · Obj 1.6 · Single choice Inherit 3, specialize 7, no other opinions. Composed `radius`? A. 3 · B. 7 · C. 10 · D. fallback

R21-11 · Obj 1.8 · Single choice You relocated `/Bot/Rig/Arm` to `/Bot/Arm` and keyed `/Bot/Rig/Arm.radius`. Why is the value unchanged? A. References beat local B. The opinion is at the relocation source, so it is ignored C. `Radius` is a schema fallback and cannot be authored D. Relocates are weaker than payloads

R21-12 · Obj 1.1 · Single choice `GetResolveInfo().GetNode().arcType` is `Pcp.ArcTypeRoot` and the session spec is first in `GetPropertyStack()`. Who won? A. A reference B. Local stack (session over root) C. Specializes D. Value clips as an arc

Review answers

R21-01 — B. `rElocates`. Review: §21.1.

R21-02 — C. V before R. Review: §21.1.

R21-03 — B. Add an L opinion. Review: §21.2.

R21-04 — B. First-listed sublayer is stronger. Review: §21.2, P2.

R21-05 — B. Default time reads the default. Review: §21.3.

R21-06 — B. Pre-first sample. Review: §21.3, P9.

R21-07 — A. Source 0 maps to stage 5. Review: §21.4, P10.

R21-08 — A and C. R wins `radius`; `extra` from P. Review: P6.

R21-09 — B. Default hides clips. Review: §21.5.

R21-10 — A. I before S. Review: §21.1.

R21-11 — B. Chapter 20 + P8. Review: §21.6.

R21-12 — B. Root node = local; session is strongest in that stack. Review: §21.2.

CHAPTER 22

Chapter 22 — Composition Design and Debugging

Exam domain: Composition (23%) · **Objectives:** 1.2, 1.5, 1.8, 1.9, 1.10, 1.11 · **Study day:** 7 · **Est. time:** 150 min **Prerequisites:** Ch 14–21 (all composition arcs, LIVERPS, value resolution)

Chapters 14–21 taught you each composition tool on its own. This chapter is about using them together, the way a studio or a factory does. You will split one big asset into files that several people can edit at once. You will layer a shot so departments do not overwrite each other. You will turn an internal asset into a clean package for a client. You will change instanced furniture without breaking instancing. Finally, you will learn the tools and the checklist that answer the most common composition question of all: “I authored a value, so why don’t I see it?”

Learning goals

- Split a monolithic asset into geometry, material, and layout workstreams using sublayers and references.
- Design a shot layer stack that lets several departments and artists work at the same time.
- Prepare an internal asset for external delivery with `Usd.Stage.Flatten`, `UsdUtils.FlattenLayerStack`, `UsdUtils.LocalizeAsset`, and `UsdUtils.CreateNewUsdzPackage`.
- Remove or hide properties and child prims of instanced components without editing inside an instance.
- Debug composition with `GetPrimStack`, `GetPropertyStack`, `GetResolveInfo`, `Usd.PrimCompositionQuery`, prim index dumps, and `GetCompositionErrors`.
- Apply a ten-point checklist to explain why an opinion does not take effect.

Key terms

Term	One-line definition
Monolithic asset	An asset whose geometry, materials, and everything else live in one file.
Workstream	One kind of work (modeling, surfacing, layout, animation) done by one team or person.
Workstream layer	A layer that holds only one workstream’s opinions, for example <code>chair_geo.usda</code> .
Entry layer	The file other people reference to use an asset; it pulls in the workstream layers.
Edit target	The layer (and optional variant) that <code>Usd</code> authoring calls write into (Ch 15).
Session layer	An in-memory layer stronger than the root layer, not saved with the stage (Ch 3).
Flatten	Bake a composed result into one layer with no composition arcs to other files.
Localize	Copy an asset and all its dependencies into one folder and rewrite paths to point there.
USDZ package	A single uncompressed zip archive holding layers and textures (Ch 7, Ch 31).
Instance proxy	A read-only prim you see under an instance; it stands for a prim in the prototype (Ch 24).
Prototype	The shared, read-only prim tree that all matching instances use (Ch 24).
Value block	An opinion of <code>None</code> that says “no value here”, hiding weaker values.

Term	One-line definition
Prim stack	The list of prim specs that contribute to a prim, strongest first.
Property stack	The list of property specs that contribute to an attribute or relationship, strongest first.
Prim index	The tree of composition nodes (arcs) Pcp builds for one prim (Ch 14).

22.1 Splitting a monolithic asset into workstreams

What is it? **Splitting a monolithic asset** means moving each kind of work into its own layer. Geometry goes into a geometry layer, materials into a material layer, and an entry layer combines them. Each team edits only its own file.

Why do we need it? If a modeler and a surfacing artist both open and save the same file, the last save wins and the other person’s work is lost. Even with file locking, only one person can work at a time. Splitting the asset lets both work in parallel. It also makes publishing safer: a material change produces a new version of the material layer only, so you know exactly what changed.

Beginner explanation. Think of the transparent sheets on an overhead projector. Instead of drawing the outline and the colors on one sheet, the modeler draws outlines on one sheet and the painter colors on another. Stack them, and you see the full picture. Each person takes away and redraws only their own sheet.

Where the analogy breaks: USD sheets are not pixels. Each sheet holds named opinions (prims and properties). If two sheets set the same property, only the stronger one is used; the weaker one is not mixed in.

Technical explanation. A common split for a component asset has three or four files:

File	Contains	Typical owner
chair.usda (entry layer)	defaultPrim, kind, variant sets, the class it inherits from, sublayer list	Asset lead / pipeline
chair_geo.usda	Meshes, transforms, extents, primvars such as UVs	Modeling
chair_mtl.usda	Material and Shader prims, material:binding relationships	Surfacing
chair_rig.usda (optional)	Rigging or physics data	Rigging

Rules that make the split work:

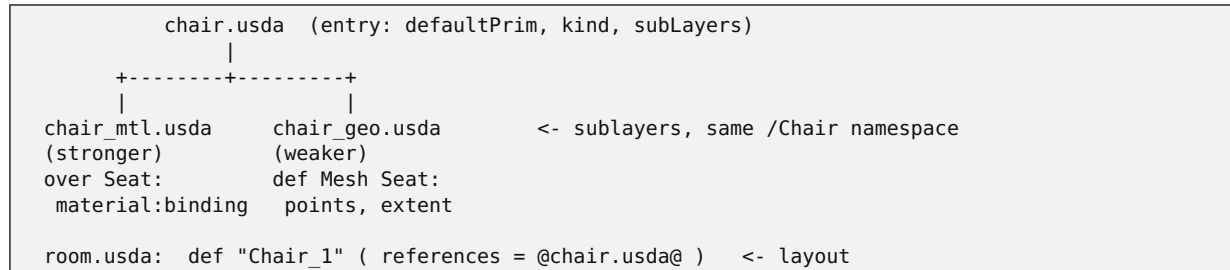
- **All workstream layers describe the same namespace.** Each file writes under / Chair, using `def` for prims it owns and `over` for prims another file defines (Ch 20). That is why **sublayers** are the natural tool inside one asset: sublayers merge layers that share the same paths (Ch 15).
- **Order sets priority.** In `subLayers = [ @./chair_mtl.usda@, @./chair_geo.usda@ ]` the first entry is stronger. Put the layer that “decorates” others (materials) above the layer it decorates (geometry).
- **Avoid overlap.** If two workstream layers set the same property, one silently wins. Agree on ownership: geometry never authors `material:binding`; surfacing never authors `points`.
- **References combine assets; sublayers combine workstreams.** A set or room file *references* chair.usda (Ch 16). The reference brings the whole layer stack of the asset under a new prim path. Layout is a scene-level workstream: it lives in its own layer of the set or shot, and only places references.

- **Payloads** often wrap the heavy workload layers so a scene can skip loading them (Ch 17). The detailed “asset interface” pattern (entry layer → payload → geo/mtl) is in Chapter 23.

In Python you author each workload with an **edit target** (Ch 15):

`stage.SetEditTarget(Usd.EditTarget(geo_layer))` sends every following Usd authoring call into `chair_geo.usda`.

Mental model.



One namespace, many files, one owner per file.

Simple example. | Before (monolithic) | After (split) | |-----|-----| |
`chair.usda` holds `/Chair/Geom/Seat` (mesh) and `/Chair/Looks/Wood` (material) and the binding | `chair_geo.usda` holds the mesh; `chair_mtl.usda` holds the material and the binding; `chair.usda` sublayers both |

The composed stage is identical. Only the file that each opinion lives in has changed.

USDA example. *File: chair.usda* (entry layer)

```

#usda 1.0
(
  defaultPrim = "Chair"
  subLayers = [
    @./chair_mtl.usda@,
    @./chair_geo.usda@
  ]
)

def Xform "Chair" (
  kind = "component"
)
{
}

```

File: chair_geo.usda (modeling owns this)

```

#usda 1.0

over "Chair"
{
  def Xform "Geom"
  {
    def Cube "Seat"
    {
      double size = 1
    }
  }
}

```

File: chair_mtl.usda (surfacing owns this)

```

#usda 1.0

over "Chair"
{
  def Scope "Looks"
  {
    def Material "Wood"
    {

```

```

    }
}

over "Geom"
{
    over "Seat" (
        prepend apiSchemas = ["MaterialBindingAPI"]
    )
    {
        rel material:binding = </Chair/Looks/Wood>
    }
}
}

```

Line notes: the entry layer `defs /Chair` and lists the sublayers, strongest first. `chair_geo.usda` uses `over "Chair"` because it does not own the root prim, then `defs` the seat it does own. `chair_mtl.usda` uses `over` all the way down to the seat it decorates, applies `MaterialBindingAPI` (required before binding), and binds.

Python example. The script builds the split asset from scratch with edit targets, saves it, reopens it, and reports which file each opinion lives in.

```

from pxr import Usd, Sdf, UsdGeom, UsdShade

geo = Sdf.Layer.CreateNew("chair_geo.usda")
mtl = Sdf.Layer.CreateNew("chair_mtl.usda")
stage = Usd.Stage.CreateNew("chair.usda")
root = stage.GetRootLayer()
root.subLayerPaths = ["/chair_mtl.usda", "/chair_geo.usda"]

chair = UsdGeom.Xform.Define(stage, "/Chair")
Usd.ModelAPI(chair.GetPrim()).SetKind("component")
stage.SetDefaultPrim(chair.GetPrim())

stage.SetEditTarget(Usd.EditTarget(geo))          # modeling workstream
UsdGeom.Xform.Define(stage, "/Chair/Geom")
UsdGeom.Cube.Define(stage, "/Chair/Geom/Seat").CreateSizeAttr(1.0)

stage.SetEditTarget(Usd.EditTarget(mtl))          # surfacing workstream
mat = UsdShade.Material.Define(stage, "/Chair/Looks/Wood")
seat = stage.GetPrimAtPath("/Chair/Geom/Seat")
UsdShade.MaterialBindingAPI.Apply(seat).Bind(mat)

for layer in (root, geo, mtl):
    layer.Save()

def authored_paths(layer):
    found = []
    keep = lambda p: p.IsPrimPath() or p.IsPropertyPath()
    layer.Traverse("/", lambda p: found.append(str(p)) if keep(p) else None)
    return sorted(found)

for name in ("chair.usda", "chair_geo.usda", "chair_mtl.usda"):
    print(name)
    for path in authored_paths(Sdf.Layer.FindOrOpen(name)):
        print("    ", path)

reopened = Usd.Stage.Open("chair.usda")
seat = reopened.GetPrimAtPath("/Chair/Geom/Seat")
bound, _ = UsdShade.MaterialBindingAPI(seat).ComputeBoundMaterial()
print("bound material:", bound.GetPath())
print("seat specs from:", [s.layer.GetDisplayName() for s in seat.GetPrimStack()])

```

Expected output

```

chair.usda
  /Chair
chair_geo.usda
  /Chair
    /Chair/Geom
    /Chair/Geom/Seat

```

```

    /Chair/Geom/Seat.size
chair_mtl.usda
  /Chair
    /Chair/Geom
    /Chair/Geom/Seat
    /Chair/Geom/Seat.material:binding
    /Chair/Looks
    /Chair/Looks/Wood
bound material: /Chair/Looks/Wood
seat specs from: ['chair_mtl.usda', 'chair_geo.usda']

```

The entry layer holds only `/Chair` (its kind is metadata on that prim, so it is not a separate path). The seat has two prim specs, one per workstream file. Each team's file contains only its own opinions plus the `overs` needed to reach them.

Real-world use case. A furniture manufacturer builds a digital twin of a showroom. CAD conversion writes `sofa_geo.usda` every night from the engineering data. A visualization artist owns `sofa_mtl.usda` and tweaks fabrics during the day. Because the nightly conversion overwrites only the geometry layer, the artist's material work is never lost, and the next morning's composed sofa has new geometry with the same fabrics.

Common mistakes.

MISTAKE

Using ``def`` for the same prim in two workstream layers with different types (for example ``def Mesh "Seat"`` and ``def Cube "Seat"``). The stronger type wins and the other team's data no longer fits. Fix: only the owning layer ``def``s a prim; other layers use ``over``.

MISTAKE

Putting the workstream layers in as **references** to the same prim path. That works, but it forces each file to have its own ``defaultPrim`` and makes the files look like separate assets. Fix: inside one asset, use sublayers; use references to bring whole assets into a scene.

MISTAKE

Letting two workstreams author the same property (for example both set ``primvars:displayColor``). One wins silently. Fix: write down which layer owns which properties and check with ``GetPropertyStack`` (§22.5).

Exam traps.

TRAP

"Split the asset so modeling and surfacing can work in parallel" → the answer is separate layers combined by sublayers (or by the asset interface), not variant sets. Variants are alternatives you **choose between**; workstreams are parts you **combine**.

TRAP

Sublayer order: the first path in ``subLayers`` is the strongest. A question may list the material layer second and ask why a material opinion loses.

Exam takeaways.

KEY

- One asset, one namespace, several workstream layers combined with ***sublayers***. - The first sublayer is strongest; put "decorating" layers (materials) above geometry. - Only the owning layer ``def``s a prim; others use ``over``. - Assets come into scenes by ***reference*** (or payload); layout is its own layer in the scene.

22.2 Multi-user scene layering

What is it? Multi-user scene layering is a plan for a shot or scene where each department, and sometimes each artist, writes into its own layer. The shot’s root layer sublayers all of them in an agreed order.

Why do we need it? A shot is touched by layout, animation, effects, and lighting, often on the same day. Without separate layers, they would fight over one file. With separate layers, the strength order also encodes **who may override whom**: lighting can fix a small layout problem in its own layer without asking layout to republish.

Beginner explanation. Picture a group project where each person writes on their own transparent sheet, and the sheets are stacked in a fixed order: the teacher’s correction sheet on top, then the final editor, then the writers. Anyone can make a change on their own sheet, and the stacking order decides whose change you see when two people touch the same spot.

Where the analogy breaks: in USD, “seeing” is per property. A stronger sheet hides a weaker one only for the exact properties it sets; everything else on the weaker sheet still shows through.

Technical explanation. A typical shot layer stack, strongest first:

Position	Layer	Why here
(memory)	session layer	Temporary experiments in a tool; never saved (Ch 3)
0	shot.usda (root)	Shot-specific fixes by a supervisor
1	shot_lighting.usda	Lighting is late in the pipeline and may adjust anything
2	shot_fx.usda	Effects
3	shot_anim.usda	Animation overrides layout transforms
4	shot_layout.usda	References the set and characters, places them
5	seq_010.usda	Sequence-wide opinions shared by all shots

Design rules:

- **One writer per layer.** Two people writing one layer at the same time causes lost work. If a department has several artists, give each a sub-layer (for example `shot_anim_jane.usda`) sublayered into the department layer.
- **Downstream departments are stronger.** A department that works later (lighting) usually sits above one that works earlier (layout), so late fixes win without republishing.
- **Shared layers are weaker.** Sequence or project layers go at the bottom so any shot can override them.
- **Use edit targets, not file juggling.** A tool sets `stage.SetEditTarget(Usd.EditTarget(layer))` to the artist’s own layer. All Usd authoring then goes there.
- **Isolate with muting.** `stage.MuteLayer(identifier)` removes a layer’s opinions from the composed result without editing any file (Ch 15). It is ideal for “what did animation change?”.
- **Time offsets per layer.** `Sdf.Layer.subLayerOffsets` can shift a department layer in time (Ch 10, Ch 15).

Mental model.

```
session (memory, not saved)      strongest
shot.usda                        <- supervisor fixes
shot_lighting.usda               <- lighting artist's edit target
shot_fx.usda                     <-
shot_anim.usda                   <- animator's edit target
```

```
shot_layout.usda    <- references set + characters
seq_010.usda        weakest
```

Each artist points the edit target at their own row. The column order decides conflicts.

Simple example. Layout gives the ball radius 1.0. An animator, by accident, sets radius 1.5 in `shot_anim.usda`. Because animation is stronger, the shot shows 1.5. Muting `shot_anim.usda` instantly shows 1.0 again, which tells the supervisor which department changed it.

USDA example. *File: shot.usda*

```
#usda 1.0
(
  subLayers = [
    @./shot_lighting.usda@,
    @./shot_anim.usda@,
    @./shot_layout.usda@,
    @./seq_010.usda@
  ]
  startTimeCode = 1
  endTimeCode = 24
)
```

File: shot_anim.usda

```
#usda 1.0

over "World"
{
  over "Ball"
  {
    double radius = 1.5
    double3 xform0p:translate.timeSamples = {
      1: (0, 0, 0),
      24: (10, 0, 0),
    }
  }
}
```

Line notes: the root layer has no prims at all; it only orders the department layers. The animation layer uses `over` because layout defines `/World/Ball`.

Python example.

```
from pxr import Usd, Sdf, UsdGeom

order = ["shot_lighting.usda", "shot_anim.usda", "shot_layout.usda"]
layers = {name: Sdf.Layer.CreateNew(name) for name in order}
stage = Usd.Stage.CreateNew("shot.usda")
stage.GetRootLayer().subLayerPaths = ["/" + n for n in order]

stage.SetEditTarget(Usd.EditTarget(layers["shot_layout.usda"]))
ball = UsdGeom.Sphere.Define(stage, "/World/Ball")
ball.CreateRadiusAttr(1.0)

stage.SetEditTarget(Usd.EditTarget(layers["shot_anim.usda"]))
ball.GetRadiusAttr().Set(1.5) # the accidental edit

print("layer stack, strongest first:")
for layer in stage.GetLayerStack():
    print("    ", layer.GetDisplayName())
print("radius:", ball.GetRadiusAttr().Get())
winner = ball.GetRadiusAttr().GetPropertyStack()[0].layer
print("winning layer:", winner.GetDisplayName())

stage.MuteLayer(layers["shot_anim.usda"].identifier)
print("anim muted, radius:", ball.GetRadiusAttr().Get())
stage.UnmuteLayer(layers["shot_anim.usda"].identifier)

stage.SetEditTarget(Usd.EditTarget(stage.GetSessionLayer()))
ball.GetRadiusAttr().Set(3.0) # a quick experiment
print("session experiment:", ball.GetRadiusAttr().Get())
```



```
stage.GetSessionLayer().Clear()
print("after clearing session:", ball.GetRadiusAttr().Get())
```

Expected output

```
layer stack, strongest first:
  shot-session.usda
  shot.usda
  shot_lighting.usda
  shot_anim.usda
  shot_layout.usda
radius: 1.5
winning layer: shot_anim.usda
anim muted, radius: 1.0
session experiment: 3.0
after clearing session: 1.5
```

`GetLayerStack()` lists the session layer first because it is the strongest. Muting is non-destructive: unmuting brings the animation opinion back.

Real-world use case. In a game cinematic team, the layout artist publishes `cine_layout.usda` with camera and character placements. Two animators each own a sublayer of `cine_anim.usda`. The lighting artist works above all of them. When the director asks why a prop moved, the lead mutes each animator's layer in turn and sees the prop jump back when the right one is muted.

Common mistakes.

MISTAKE

Leaving the edit target at the root layer, so every department's edits land in ``shot.usda``.
Fix: set the edit target to the department layer at tool start-up, and check ``stage.GetEditTarget().GetLayer()`` before saving.

MISTAKE

Putting a shared sequence layer **above** the shot layers. Then a sequence change silently overrides every shot's fixes. Fix: shared layers go at the bottom.

MISTAKE

Doing experiments in a department layer and forgetting to undo them. Fix: experiment in the session layer, which is never saved with the stage.

Exam traps.

TRAP

"Which approach lets several artists edit the same shot without conflicts?" The answer is separate layers with edit targets, not "lock the file" and not "use variants".

TRAP

Muting a layer does not delete or edit it. A question may offer "mute the layer" as a way to **permanently** remove opinions; it is only a temporary, per-stage view.

Exam takeaways.

KEY

- One writer per layer; combine layers with sublayers in an agreed order. - Later departments are usually stronger; shared sequence layers are weakest. - Edit targets send authoring to the right layer; the session layer is for unsaved experiments. - ``MuteLayer`` isolates a layer's effect without editing files.

22.3 Preparing an asset for external delivery

What is it? **Preparing an asset for delivery** turns a studio-internal asset into a self-contained package that works on someone else’s computer. You remove links to internal servers and tools, collect every needed file into one place, and often bake composition into one file or one `.usdz`.

Why do we need it? Internal assets point to places the client cannot reach: `/studio/library/rigs/...`, custom resolver URIs, in-house scripts. They may also carry private metadata such as artist names or project codes. Send them as-is and the client sees missing references, missing textures, and leaked information.

Beginner explanation. Think of sending a recipe to a friend. Your own recipe card says “use the sauce from the jar on shelf 3” and “see Grandma’s notebook”. Your friend has neither. Before sending, you either write the sauce recipe into the card (flatten) or put the jar and a copy of the notebook in the box with the card (localize/package), and you cross out your private notes.

Where the analogy breaks: flattening in USD is lossy in specific ways (variants not selected are dropped, arcs disappear), and packaging keeps structure but needs every file to be findable when you build the package.

Technical explanation. Four tools, each with a different result. All signatures below were checked on USD 26.08.

Tool	Signature (Python)	Result	Keeps arcs/v
<code>Usd.Stage.Flatten</code>	<code>stage.Flatten(addSourceFileComment=True)</code> → <code>Sdf.Layer</code>	One anonymous layer with the fully composed stage	No arcs, no un variants. Insta via <code>Flattened_Pr</code> prims
<code>UsdUtils.FlattenLayerStack</code>	<code>FlattenLayerStack(stage, tag="")</code> or <code>FlattenLayerStack(stage, resolveAssetPathFn, tag="")</code> → <code>Sdf.Layer</code>	Merges only the root layer stack (sublayers) into one layer	Yes: reference payloads, vari
<code>UsdUtils.LocalizeAsset</code>	<code>LocalizeAsset(assetPath, localizationDirectory, editLayersInPlace=False, processingFunc=None)</code> → <code>bool</code>	Copies the asset and all dependencies into a folder, rewriting paths	Yes, structure unchanged
<code>UsdUtils.CreateNewUsdzPackage</code>	<code>CreateNewUsdzPackage(assetPath, usdzFilePath, firstLayerName="", editLayersInPlace=False)</code> → <code>bool</code>	Same collection, written as one <code>.usdz</code>	Yes

Facts to know:

- **Flatten anchors asset paths.** Both `Flatten` and `FlattenLayerStack` (without a callback) turn relative paths like `@./tex/wood.png@` into absolute paths on *your* disk. That leaks internal paths. Fix it with `FlattenLayerStack(stage, lambda layer, path: path)` (keep paths as authored) or by rewriting afterwards with `UsdUtils.ModifyAssetPaths(layer, fn)`.
- **Inventory first.** `UsdUtils.ComputeAllDependencies(Sdf.AssetId(path))` returns `(layers, assets, unresolvedPaths)`. Unresolved paths are your list of internal dependencies.

- **Remove proprietary dependencies during localization.** `processingFunc(layer, dependencyInfo)` is called for every dependency. Return the `dependencyInfo` to keep it, or an empty `UsdUtils.DependencyInfo()` to remove it from the delivered layer (references are dropped, asset values cleared).
- **Unresolvable paths are not fatal by default.** Without a `processingFunc`, `LocalizeAsset` returns `False`, prints warnings, and still writes layers with remapped placeholders like `@0/publish.py@`. Always check the return value.
- **Private metadata** (`customData`, `assetInfo`, `custom studio: attributes`) is not removed by any of these tools. Clear it yourself, for example `prim.ClearCustomDataByKey("owner")` or `prim.RemoveProperty(...)` on the delivery copy.
- `usdzip` (command line) wraps `CreateNewUsdzPackage`; `usdcat --flatten` wraps `Flatten`. Neither tool ships with `usd-core`.

```
usdzip --asset chair.usda chair.usdz
usdcat --flatten shot.usda -o shot_flat.usda
```

These commands are not installed with `usd-core`; the Python example shows the same operations.

VERSION

Verified on USD 26.08. ``UsdUtils.LocalizeAsset`` was added in 24.03; older material only shows ``CreateNewUsdzPackage`` and manual copying.

Mental model.

```
internal asset --> inventory (ComputeAllDependencies)
|
|-- keep structure --> LocalizeAsset --> folder
|   (processingFunc drops /studio/ deps) |
|   |                                     v
|   |                                     CreateNewUsdzPackage
|   |
|-- bake -----> Flatten (whole stage) or
|                  FlattenLayerStack (sublayers only)
|                  then fix absolute asset paths
always: strip private metadata, re-open the result, check for errors
```

Simple example. `chair.usda` sublayers `chair_geo.usda` and `chair_mtl.usda`, uses texture `./tex/wood.png`, references a rig on `/studio/rigs/`, and has `customData = {owner = "jdoe"}`. Delivery: drop the rig reference, clear `owner`, copy the three layers and the texture into `delivery/`, and zip them as `chair.usdz`.

USDA example. File: `chair.usda` (internal version, before delivery)

```
#usda 1.0
(
    defaultPrim = "Chair"
    subLayers = [
        @./chair_mtl.usda@,
        @./chair_geo.usda@
    ]
)

def Xform "Chair" (
    customData = {
        string owner = "jdoe"
    }
    prepend references = @/studio/rigs/chair_rig.usda@
)
{
    asset studio:publishScript = @/studio/tools/publish.py@
}
```

Line notes: the `references` line and the `studio:publishScript` asset path point to internal servers; `customData.owner` is private. All three must not reach the client.

Python example. The first script inventories the asset, localizes it while dropping internal dependencies, strips private metadata, and packages it.

```
import os
import zipfile
from pxr import Usd, Sdf, UsdUtils

os.makedirs("src/tex", exist_ok=True)
with open("src/tex/wood.png", "wb") as f:
    f.write(b"\x89PNG\r\n\x1a\n") # stand-in texture file
with open("src/chair_geo.usda", "w") as f:
    f.write('#usda 1.0\nover "Chair"\n{\n    def Cube "Seat"\n        {\n    }\n}\n')
with open("src/chair_mtl.usda", "w") as f:
    f.write('#usda 1.0\nover "Chair"\n{\n    def Shader "Tex"\n        {\n        asset inputs:file = @./tex/wood.png@\n    }\n}\n')
with open("src/chair.usda", "w") as f:
    f.write('''#usda 1.0
(
    defaultPrim = "Chair"
    subLayers = [ @./chair_mtl.usda@, @./chair_geo.usda@ ]
)
def Xform "Chair" (
    customData = {string owner = "jdoe"}
    prepend references = @/studio/rigs/chair_rig.usda@
)
{
    asset studio:publishScript = @/studio/tools/publish.py@
}
''')

layers, assets, unresolved = UsdUtils.ComputeAllDependencies(
    Sdf.AssetPath("src/chair.usda"))
print("layers:", sorted(os.path.basename(l.realPath) for l in layers))
print("assets:", [os.path.relpath(a) for a in assets])
print("unresolved:", sorted(unresolved))

def drop_internal(layer, dep):
    if dep.assetPath.startswith("/studio/"):
        return UsdUtils.DependencyInfo() # empty info = remove it
    return dep

ok = UsdUtils.LocalizeAsset(Sdf.AssetPath("src/chair.usda"), "delivery",
                           False, drop_internal)
print("localized:", ok)

stage = Usd.Stage.Open("delivery/chair.usda")
chair = stage.GetPrimAtPath("/Chair")
chair.ClearCustomDataByKey("owner")
chair.RemoveProperty("studio:publishScript")
stage.Save()
print(stage.GetRootLayer().ExportToString())

ok = UsdUtils.CreateNewUsdzPackage(Sdf.AssetPath("delivery/chair.usda"),
                                  "chair.usdz")
print("packaged:", ok, sorted(zipfile.ZipFile("chair.usdz").namelist()))
check = Usd.Stage.Open("chair.usdz")
print("errors:", len(check.GetCompositionErrors()),
      "texture:", check.GetPrimAtPath("/Chair/Tex").GetAttribute("inputs:file").Get())
```

Expected output

```
layers: ['chair.usda', 'chair_geo.usda', 'chair_mtl.usda']
assets: ['src/tex/wood.png']
unresolved: ['/studio/rigs/chair_rig.usda', '/studio/tools/publish.py']
localized: True
#usda 1.0
(
    defaultPrim = "Chair"
    subLayers = [
        @./chair_mtl.usda@,
        @./chair_geo.usda@
```

```

    ]
)

def Xform "Chair"
{
}

packaged: True ['chair.usda', 'chair_geo.usda', 'chair_mtl.usda', 'tex/wood.png']
errors: 0 texture: @./tex/wood.png@

```

The internal reference is gone, the asset-valued attribute was removed, `customData` is empty, and the package opens with no composition errors. Warnings about the unresolvable `/studio/` paths go to standard error during `ComputeAllDependencies`; they are expected.

The second script compares the two flatten functions and shows the absolute-path leak and its fix.

```

import os
from pxr import Usd, UsdUtils

with open("geo.usda", "w") as f:
    f.write('#usda 1.0\nover "Lamp"\n{\n    asset tex = @./tex/metal.png@\n}\n')
with open("lamp.usda", "w") as f:
    f.write('''#usda 1.0
(
    subLayers = [ @./geo.usda@ ]
)
def Xform "Lamp" (
    variants = {string size = "big"}
    prepend variantSets = "size"
)
{
    variantSet "size" = {
        "big" {
            double height = 3
        }
        "small" {
            double height = 1
        }
    }
}
''')
here = os.getcwd()
stage = Usd.Stage.Open("lamp.usda")

flat = stage.Flatten(addSourceFileComment=False)
lamp = flat.GetPrimAtPath("/Lamp")
print("Flatten variant sets:", list(lamp.variantSets.keys()))
print("Flatten height:", lamp.attributes["height"].default)
print("Flatten tex:", lamp.attributes["tex"].default.path.replace(here, "<cwd>"))

stack = UsdUtils.FlattenLayerStack(stage)
lamp = stack.GetPrimAtPath("/Lamp")
print("FlattenLayerStack variant sets:", list(lamp.variantSets.keys()))
print("FlattenLayerStack tex:",
      lamp.attributes["tex"].default.path.replace(here, "<cwd>"))

kept = UsdUtils.FlattenLayerStack(stage, lambda layer, path: path)
print("with callback tex:", kept.GetPrimAtPath("/Lamp").attributes["tex"].default)

UsdUtils.ModifyAssetPaths(flat, lambda p: "./tex/" + os.path.basename(p))
print("after ModifyAssetPaths:", flat.GetPrimAtPath("/Lamp").attributes["tex"].default)

```

Expected output

```

Flatten variant sets: []
Flatten height: 3.0
Flatten tex: <cwd>/tex/metal.png
FlattenLayerStack variant sets: ['size']
FlattenLayerStack tex: <cwd>/tex/metal.png

```

```
with callback tex: @./tex/metal.png@
after ModifyAssetPaths: @./tex/metal.png@
```

Flatten baked the selected variant (height = 3) and dropped the variant set. FlattenLayerStack kept the variant set. Both made the texture path absolute until a callback or ModifyAssetPaths fixed it.

Real-world use case. An automotive supplier sends a seat model to a car maker for a design review in a different tool. The supplier localizes the asset, drops references to its internal fastener library and in-house validation scripts, removes customData with engineer names, and delivers a .usdz. The car maker opens one file with no broken links. For an archival copy with no composition at all, the supplier also stores a flattened .usdc.

Common mistakes.

MISTAKE

Flattening and assuming paths are portable. `Flatten` writes absolute paths from your machine. Fix: rewrite with `UsdUtils.ModifyAssetPaths`, or use `FlattenLayerStack` with a callback that keeps authored paths.

MISTAKE

Ignoring the `bool` returned by `LocalizeAsset` or `CreateNewUsdzPackage`. A `False` means some dependency could not be resolved and was written as a placeholder path. Fix: check the result and handle unresolved paths with a `processingFunc`.

MISTAKE

Flattening an asset whose client needs to switch variants. `Flatten` keeps only the selected variant. Fix: use `FlattenLayerStack` or localization, which keep variant sets.

MISTAKE

Forgetting metadata. None of the four tools removes `customData`, `assetInfo`, or custom attributes. Fix: strip them explicitly on the delivery copy.

Exam traps.

TRAP

`Usd.Stage.Flatten` vs `UsdUtils.FlattenLayerStack`: the first flattens *all* composition (references, variants, inherits); the second merges only the layer stack (sublayers) and keeps other arcs.

TRAP

"Package the asset so it works offline" → localize or USDZ-package it. Flattening alone still leaves texture files outside the layer.

TRAP

USDZ packages are zip archives that must be **uncompressed**; `CreateNewUsdzPackage` handles that. Do not answer "zip the folder with any zip tool" (Ch 31).

Exam takeaways.

KEY

- `Flatten` = whole stage, no arcs, selected variants only; `FlattenLayerStack` = sublayers only, arcs kept.
- `LocalizeAsset(assetPath, dir, editLayersInPlace, processingFunc)` collects files; return an empty `DependencyInfo` to drop a dependency.
- `CreateNewUsdzPackage(assetPath, usdzPath)` makes the single-file deliverable.
- Flattening makes asset paths absolute; fix with `ModifyAssetPaths`. Strip private metadata yourself.

22.4 Removing properties from instanced component prims

What is it? This is the task of hiding or removing an attribute, relationship, or child prim of a component that is **instanced** in an assembly, for example removing a tracking attribute from every chair in a room, or hiding one chair’s cushion.

Why do we need it? Instancing (Ch 24) shares one read-only **prototype** among many instances, which saves memory and load time. But the prims you see under an instance are **instance proxies**, and USD refuses edits on them. You need ways to change instanced content that respect that rule.

Beginner explanation. Instancing is a rubber stamp: one carved stamp, many prints. You cannot erase a line on one print and expect the stamp to change, and you cannot carve the stamp from the print. To change prints you either carve a different stamp (a new prototype through a variant or a different arc), change the stamp’s design sheet that all stamps follow (a class all instances inherit), or draw that one picture by hand (turn off instancing for it).

Where the analogy breaks: USD carves new stamps automatically. Any two instances whose composition arcs differ get different prototypes; you never manage prototypes yourself.

Technical explanation. What fails. Every authoring call on an instance proxy or a prototype raises `Tf.ErrorException` (“authoring to an instance proxy is not allowed” / “authoring to an instancing prototype is not allowed”). And if you write an `over` below an instance root directly into a layer (with `Sdf` or by hand), composition **silently ignores** it: instances only take opinions from arcs on and above the instance root, not from local specs beneath it.

What “remove” means. Composition cannot delete a property that a weaker layer defines. You can:

- **Block** an attribute: an opinion of `None` (`attr.Block()`, or `= None` in USDA). The attribute still exists but has no value.
- **Block** a relationship’s targets: `rel material:binding = None` leaves an empty target list.
- **Deactivate** a child prim: `active = false` hides it and all its descendants.
- **Truly remove** it only by editing the source asset layer (`Sdf.PrimSpec.RemoveProperty`), which changes every user of that asset.

Where you may author. The **instance root** (the prim with `instanceable = true`) is a normal prim. You can add arcs and metadata there. That gives four working approaches:

Approach	Where you author	Affects	Prototypes
A. Class edit	<code>class "_class_Chair"</code> in the assembly layer, for an asset that inherits it	All instances that inherit the class	Shared still
B. Variant	Variant selection on the instance root (asset provides e.g. <code>trim = bare</code>)	Instances with that selection	One per distinct selection
C. Extra arc on instance root	<code>prepend inherits = </_fix></code> (or a reference) on chosen roots, plus a class <code>_fix</code>	Only those instances	One per distinct arc set
D. De-instance	<code>instanceable = false</code> on one instance, then edit its children normally	That one chair	It leaves instancing
E. Fix the source	Edit the component asset itself	Every user everywhere	Shared still

Prefer A, B, or C. D costs memory if done widely. E is right only when the property should never have been there.

Choosing an instancing style (Obj 1.2, brief). No instancing for unique hero assets; **scenegraph instancing** (`instanceable = true`) for tens to thousands of repeated components that need their own prim paths, variants, and per-instance overrides on the root; **point instancing** (`UsdGeom.PointInstancer`, Ch 25) for thousands to millions of copies (rocks, trees, bolts) where per-copy edits are limited to position, orientation, scale, and hiding. Chapter 26 gives the full decision table.

Mental model.

```
/Room/Chair_1 (instanceable = true) <- editable: arcs, metadata, variant selection
|-- Seat      (instance proxy)      <- read-only: Usd edit raises an error,
|-- Cushion   (instance proxy)      layer overs here are ignored
    ^
    | comes from the prototype /__Prototype_1 (read-only)

change instanced content = change the ARCS on the root
(class it inherits, variant it selects, extra inherit) -> USD picks the prototype
```

Simple example. A room has three instanced chairs. Every chair carries `studio:trackingId`, which the client must not see. Chair 3 must also lose its cushion. Plan: block `studio:trackingId` in the room's `_class_Chair` (all chairs), and select the asset's `trim = bare` variant on Chair 3 only.

USDA example. File: *room.usda* (assembly; *chair.usda* inherits `</_class_Chair>` and has a `trim` variant set)

```
#usda 1.0
(
  defaultPrim = "Room"
)

def Xform "Room" (
  kind = "assembly"
)
{
  def "Chair_1" (
    instanceable = true
    prepend references = @./chair.usda@
  )
  {
  }

  def "Chair_3" (
    instanceable = true
    prepend references = @./chair.usda@
    variants = {
      string trim = "bare"
    }
  )
  {
  }
}

class "_class_Chair"
{
  over "Seat"
  {
    custom string studio:trackingId = None
  }
}
```

Line notes: the class `_class_Chair` in the assembly layer is stronger than the asset's own opinions (inherits beat references in LIVERPS) and reaches every chair through the inherit arc the asset already authors. `= None` is a value block. `Chair_3`'s variant selection is authored on the instance root, which is allowed.

Python example.

```

from pxr import Usd, Sdf, Tf

with open("chair.usda", "w") as f:
    f.write('''#usda 1.0
(
    defaultPrim = "Chair"
)
def Xform "Chair" (
    inherits = </_class_Chair>
    variants = {string trim = "full"}
    prepend variantSets = "trim"
)
{
    def Mesh "Seat"
    {
        custom string studio:trackingId = "XYZ-123"
    }
    def Mesh "Cushion"
    {
    }
    variantSet "trim" = {
        "bare" {
            over "Cushion" (active = false)
            {
            }
        }
        "full" {
        }
    }
}
class "_class_Chair"
{
}
''')

with open("room.usda", "w") as f:
    f.write('#usda 1.0\ndef Xform "Room"\n{\n' + "".join(
        f'    def "Chair_{i}" (instanceable = true\n'
        f'        prepend references = @./chair.usda@\n        {{\n        }}\n'
        for i in (1, 2, 3)) + "}\n")

stage = Usd.Stage.Open("room.usda")
proxy_attr = stage.GetPrimAtPath("/Room/Chair_1/Seat").GetAttribute(
    "studio:trackingId")
try:
    proxy_attr.Block()
except Tf.ErrorException:
    print("edit on instance proxy: refused")

# A. Class edit: all chairs, prototypes stay shared.
over_seat = stage.OverridePrim("/_class_Chair/Seat")
over_seat.CreateAttribute("studio:trackingId",
    Sdf.ValueTypeNames.String, custom=True).Block()
print("prototypes after class edit:", len(stage.GetPrototypes()))

# B. Variant on one instance root.
stage.GetPrimAtPath("/Room/Chair_3").GetVariantSet("trim").SetVariantSelection("bare")
print("prototypes after variant:", len(stage.GetPrototypes()))

# D. De-instance one chair, then edit it directly.
stage.GetPrimAtPath("/Room/Chair_2").SetInstanceable(False)
stage.GetPrimAtPath("/Room/Chair_2/Cushion").SetActive(False)

for i in (1, 2, 3):
    root = stage.GetPrimAtPath(f"/Room/Chair_{i}")
    attr = stage.GetPrimAtPath(f"/Room/Chair_{i}/Seat").GetAttribute(
        "studio:trackingId")
    cushion = stage.GetPrimAtPath(f"/Room/Chair_{i}/Cushion")
    print(f"Chair_{i}", "instance" if root.IsInstance() else "plain",
        "| trackingId exists:", attr.IsValid(), "value:", attr.Get(),
        "| cushion active:", cushion.IsActive())
print("prototypes at end:", len(stage.GetPrototypes()))

```

Expected output

```
edit on instance proxy: refused
prototypes after class edit: 1
prototypes after variant: 2
Chair_1 instance | trackingId exists: True value: None | cushion active: True
Chair_2 plain | trackingId exists: True value: None | cushion active: False
Chair_3 instance | trackingId exists: True value: None | cushion active: False
prototypes at end: 2
```

Notice three things. The proxy edit raised an error. The class edit reached every chair and kept one prototype. The blocked attribute still *exists*; it just has no value. That is what “remove” means in composition.

Real-world use case. An architecture firm’s office assembly contains 400 instanced desks. The desk asset carries `studio:costCode` attributes that must not leave the firm, and the client wants the monitor hidden on desks in the meeting rooms. The firm blocks `studio:costCode` in the assembly’s `_class_Desk` (one edit, all desks, one prototype) and selects `monitor = none` on the 20 meeting-room desks (a second prototype). Memory stays close to two desks’ worth.

Common mistakes.**MISTAKE**

Calling ``SetActive(False)`` or ``Block()`` on ``/Room/Chair_1/Seat`` and expecting it to work. It raises ``Tf.ErrorException``. Fix: author on the instance root (variant, inherits) or in a class.

MISTAKE

Writing ``over "Seat" { ... }`` under an instanceable prim in the assembly layer by hand. It parses fine but composition ignores it, with no warning. Fix: move the opinion into a class or variant.

MISTAKE

Turning ``instanceable`` off for hundreds of instances to make edits. Memory and load time grow with every de-instanced copy. Fix: use variants or class edits so instances keep sharing a few prototypes.

Exam traps.**TRAP**

"Edit the prototype returned by ``GetPrototype()``" is never correct; prototypes are read-only.

TRAP

A value block (`` = None``) does not delete the property. ``GetAttribute(...).IsValid()`` stays ``True``; ``Get()`` returns ``None``.

TRAP

Look-alike answer: "Set ``instanceable = false`` on the prototype". Instanceable is authored on instance roots, never on prototypes.

Exam takeaways.**KEY**

- Instance proxies and prototypes are read-only: ``Usd`` edits raise errors; layer overs under the instance root are ignored. - Edit instanced content by changing arcs on the instance root or a class it inherits; USD regroups prototypes automatically. - "Remove" = block (``None``) an attribute or relationship, or deactivate a child prim. - Scenegraph instancing for repeated components; point instancing for massive counts (Ch 24-26).

22.5 Debugging composition

What is it? Debugging composition means asking USD *where* a prim or value comes from: which layers, which arcs, which variant. USD has introspection APIs that answer each of those questions directly.

Why do we need it? A composed stage hides its sources. A value of 2.0 on screen could come from the root layer, a sublayer, a variant inside a reference, or a class. Guessing wastes hours. The introspection APIs show the exact path in seconds.

Beginner explanation. Value resolution is a ladder of opinions where the strongest rung answers (Ch 21). Debugging tools let you look at the whole ladder: who stands on every rung (prim stack and property stack), who answered (resolve info), and how the ladder was built (composition query and prim index).

Where the analogy breaks: the “ladder” is really a tree of arcs (the prim index). The rungs you see in a stack are that tree flattened in strength order.

Technical explanation. | Question | API (verified on 26.08) | Returns |
 |-----|-----|-----| | Which specs make this prim? | `prim.GetPrimStack()` |
 | `[Sdf.PrimSpec]`, strongest first; each has `.layer` and `.path` | | ...with time offsets? |
`prim.GetPrimStackWithLayerOffsets()` | | `[(Sdf.PrimSpec, Sdf.LayerOffset)]` | | Which
 specs set this attribute? | `attr.GetPropertyStack(time=Usd.TimeCode.Default())` |
 | `[Sdf.PropertySpec]`, strongest first | | Where does the value come from? |
`attr.GetResolveInfo(time)` | `Usd.ResolveInfo: GetSource(), GetNode(), ValueIsBlocked(),`
`HasAuthoredValue()` | | Which arcs build this prim? |
`Usd.PrimCompositionQuery(prim).GetCompositionArcs()` | | `[Usd.CompositionArc]` | | Full
 composition tree | `prim.GetPrimIndex().DumpToString()` | | Text listing every node | | What
 failed to compose? | `stage.GetCompositionErrors()` | | `[Pcp error]`, each with `errorType`,
`rootSite`, `str()` |

Details:

- `ResolveInfo.GetSource()` is one of `Usd.ResolveInfoSourceNone`, `Fallback`, `Default`, `TimeSamples`, `ValueClips`, `Spline`. `GetNode()` returns the `Pcp.NodeRef` (arc) that won; `node.arcType`, `node.path`, and `node.layerStack` describe it. There is **no layer accessor** on `ResolveInfo`; for the winning layer use `attr.GetPropertyStack()[0].layer`.
- `Usd.CompositionArc` methods: `GetArcType()` (a `Pcp.ArcType`), `GetTargetNode()`, `GetTargetPrimPath()`, `GetTargetLayer()`, `GetIntroducingLayer()`, `GetIntroducingPrimPath()`, `IsImplicit()`, `IsAncestral()`, `HasSpecs()`. The root arc has no introducing layer (`None`).
- Filter the query with `Usd.PrimCompositionQuery.Filter()`; set `filter.arcTypeFilter = Usd.PrimCompositionQuery.ArcTypeFilter.Reference` (also `Inherit`, `Variant`, `Payload`, `Specialize`, `Relocate`, and combinations). `Usd.PrimCompositionQuery.GetDirectReferences(prim)` is a ready-made query.
- `GetCompositionErrors()` exists on `Usd.Stage` and lists errors such as `Pcp.ErrorType_InvalidAssetPath` (missing file) or `ErrorType_ArcCycle`. Errors are also printed as warnings when the stage opens.
- In `usdview`, the **Layer Stack** and **Composition** tabs show the same data. `usdview` is not part of `usd-core` (Ch 42).

Mental model.

Is the prim there?	<code>stage.GetPrimAtPath(p) -> valid? active? defined?</code>
Which files touch it?	<code>prim.GetPrimStack()</code>
Which arcs bring them?	<code>Usd.PrimCompositionQuery(prim).GetCompositionArcs()</code>
Which spec sets the value?	<code>attr.GetPropertyStack()[0]</code>
What kind of value won?	<code>attr.GetResolveInfo().GetSource()</code>

Anything broken?	<code>stage.GetCompositionErrors()</code>
Show me everything	<code>prim.GetPrimIndex().DumpToString()</code>

Work top to bottom: existence, sources, arcs, winner, errors.

Simple example. `/World/Lamp_1` reads `height = 2.0`, but the lamp asset's `small` variant says `0.5`. The property stack shows three specs: `lighting.usda` (2.0), `lamp.usda /Lamp` (1.0), `lamp.usda /Lamp{size=small}` (0.5). The first one wins: the lighting sublayer of the shot overrides the asset.

USDA example. *File: shot.usda*

```
#usda 1.0
(
  subLayers = [
    @./lighting.usda@
  ]
)

def "World"
{
  def "Lamp_1" (
    prepend references = [@./lamp.usda@, @./missing.usda@]
  )
  {
  }
}
```

File: lighting.usda

```
#usda 1.0

over "World"
{
  over "Lamp_1"
  {
    double height = 2
  }
}
```

Line notes: `Lamp_1` has two references; `missing.usda` does not exist, which will show up as a composition error. `lighting.usda` is a sublayer of the shot, so it is part of the root layer stack.

Python example.

```
import os
import re
from pxr import Usd

files = {
  "lamp.usda": '''#usda 1.0
(
  defaultPrim = "Lamp"
)
def Xform "Lamp" (
  variants = {string size = "small"}
  prepend variantSets = "size"
)
{
  double height = 1
  variantSet "size" = {
    "big" {
      double height = 3
    }
    "small" {
      double height = 0.5
    }
  }
}
'''
  "lighting.usda": '#usda 1.0\nover "World"\n{\n  over "Lamp_1"\n    {\n      double height = 2\n    }\n}\n',
```

```

"shot.usda": '''#usda 1.0
(
    subLayers = [ @./lighting.usda@ ]
)
def "World"
{
    def "Lamp_1" (
        prepend references = [ @./lamp.usda@, @./missing.usda@ ]
    )
    {
    }
}
'''
}
for name, text in files.items():
    with open(name, "w") as f:
        f.write(text)

def short(text):
    text = text.replace(os.getcwd() + "/", "")
    return re.sub(r"anon:0x[0-9a-f]+:", "anon:", text)

stage = Usd.Stage.Open("shot.usda")
prim = stage.GetPrimAtPath("/World/Lamp_1")
attr = prim.GetAttribute("height")

print("value:", attr.Get())
print("prim stack:")
for spec in prim.GetPrimStack():
    print("  ", spec.layer.GetDisplayName(), spec.path)
print("property stack:")
for spec in attr.GetPropertyStack():
    print("  ", spec.layer.GetDisplayName(), spec.path, spec.default)

info = attr.GetResolveInfo()
print("resolve:", info.GetSource(), info.GetNode().arcType, info.GetNode().path)
print("winning layer:", attr.GetPropertyStack()[0].layer.GetDisplayName())

print("arcs:")
for arc in Usd.PrimCompositionQuery(prim).GetCompositionArcs():
    intro = arc.GetIntroducingLayer()
    print("  ", arc.GetArcType(), arc.GetTargetNode().path,
          "introduced in", intro.GetDisplayName() if intro else None)

print("errors:")
for err in stage.GetCompositionErrors():
    print("  ", err.errorType)
    print("  ", short(str(err)).strip().replace(" for ", "\n    for "))

print("prim index (excerpt):")
for line in prim.GetPrimIndex().DumpToString().splitlines():
    if line.startswith("Node") or "Type:" in line[:10] or "Source path" in line:
        print("  ", short(line).strip())

```

Expected output

```

value: 2.0
prim stack:
  shot.usda /World/Lamp_1
  lighting.usda /World/Lamp_1
  lamp.usda /Lamp
  lamp.usda /Lamp{size=small}
property stack:
  lighting.usda /World/Lamp_1.height 2.0
  lamp.usda /Lamp.height 1.0
  lamp.usda /Lamp{size=small}.height 0.5
resolve: Usd.ResolveInfoSourceDefault Pcp.ArcTypeRoot /World/Lamp_1
winning layer: lighting.usda
arcs:
  Pcp.ArcTypeRoot /World/Lamp_1 introduced in None
  Pcp.ArcTypeReference /Lamp introduced in shot.usda
  Pcp.ArcTypeVariant /Lamp{size=small} introduced in lamp.usda

```

```

errors:
  Pcp.ErrorType_InvalidAssetPath
  Could not open asset @missing.usda@
    for reference introduced by @shot.usda@</World/Lamp_1>.
prim index (excerpt):
  Node 0:
    Type:          root
    Source path:   </World/Lamp_1>
  Node 1:
    Type:          reference
    Source path:   </Lamp>
  Node 2:
    Type:          variant
    Source path:   </Lamp{size=small}>

```

Reading it: the winning node is the **root** node (the shot's own layer stack, which includes `lighting.usda`), so local beats the reference and its variant. The missing file appears both as a warning on open and in `GetCompositionErrors()`. Notice that the property stack lists the variant's 0.5 last, even though variants are stronger than references in LIVERPS: the variant lives *inside* the reference, so it is only stronger than other opinions within that referenced asset.

Real-world use case. A robotics team's simulation shows a gripper at the wrong scale. Instead of opening a dozen files, the engineer runs a small script that prints `GetPropertyStack()` for `xformOp:scale`. It shows a stale override in `cell_layout_v3.usda`, a layer someone forgot to remove from the sublayer list. The fix takes a minute.

Common mistakes.

MISTAKE

Reading ``GetResolveInfo().GetNode().layerStack`` and taking its root layer as "the layer that won". The node is an arc; its layer stack may hold many sublayers. Fix: use ``attr.GetPropertyStack()[0].layer`` for the winning layer.

MISTAKE

Looking only at the root layer with ``Sdf`` (the authored view) to find a value. The value may come from a sublayer or arc. Fix: use the ``Usd`` stack APIs, which see the composed result.

MISTAKE

Ignoring warnings at stage open. They are composition errors. Fix: call ``stage.GetCompositionErrors()`` in tests and pipelines and fail on unexpected errors.

Exam traps.

TRAP

``GetPropertyStack`` is on ``Usd.Property`` (attributes and relationships); ``GetPrimStack`` is on ``Usd.Prim``. Do not mix them up.

TRAP

Stacks are ordered ****strongest first****. A question may show a stack and ask which value wins: the first entry with a value, unless it is a block.

TRAP

``GetIntroducingLayer()`` is the layer where the arc is *written*; ``GetTargetLayer()`` is the layer the arc *points to*. For a reference authored in ``shot.usda`` to ``lamp.usda``, they are ``shot.usda`` and ``lamp.usda``.

Exam takeaways.

KEY

- ``GetPrimStack`` (prim) and ``GetPropertyStack`` (property) are strongest first. -
``GetResolveInfo`` gives the source kind and winning node; the winning layer is
``GetPropertyStack()[0].layer``. - ``Usd.PrimCompositionQuery(...).GetCompositionArcs()`` →
``GetArcType()``, ``GetTargetNode()``, ``GetIntroducingLayer()``. -
``GetPrimIndex().DumpToString()`` shows the whole arc tree;
``stage.GetCompositionErrors()`` lists failures.

22.6 Why an opinion does not take effect: checklist

What is it? A ten-point checklist for the most common debugging question in USD: “I authored an opinion, but the composed stage does not show it.” Each point is a real cause with a quick test.

Why do we need it? The causes look the same from outside: the value is just “wrong”. Working through a fixed list, with the tools from §22.5, finds the cause quickly and stops you from “fixing” the wrong layer.

Beginner explanation. When a letter does not arrive, you check a list: right address? posted at all? did someone at the same address throw it away? is the house even occupied? The checklist below does the same for an opinion: right layer? strong enough? right path? is the prim active? is it inside an instance?

Where the analogy breaks: in USD, nothing is lost. Every opinion is still in its layer; it just loses the strength contest or is not reachable. That is why stack tools always find it.

Technical explanation. | # | Cause | Quick test | Fix | |—|-----|-----|-----| | 1 | **Wrong edit target / layer:** the edit went to the session layer or another layer that was not saved | `stage.GetEditTarget().GetLayer()`; after reopening, value is gone | Set the edit target to the intended layer, save that layer | | 2 | **Weaker arc:** your opinion is in a reference/payload/specializes, but a stronger arc (local, inherits, variant) sets it | `GetPropertyStack()` shows a stronger spec first | Author in the stronger site or remove the stronger opinion | | 3 | **Stronger local opinion:** a stronger sublayer in the same layer stack sets it | First spec in `GetPropertyStack()` is from another sublayer | Author in that layer, or reorder sublayers | | 4 | **Variant selection in a stronger layer:** your layer selects big, a stronger one selects small | `prim.GetVariantSets().GetVariantSelection(name)` and the prim stack | Set the selection in the strongest needed layer | | 5 | **Typo in path or name:** over “Wrold” creates a separate, undefined prim; a misspelled attribute is a new attribute | `stage.TraverseAll()` shows stray over prims; `GetPropertyNames()` | Correct the path or name | | 6 | **Over on an inactive prim:** a stronger layer set `active = false` | `prim.IsActive()`; children return invalid prims | Re-activate in a stronger layer, or author elsewhere | | 7 | **Inside an instance:** overs below an instance root are ignored; Usd edits there raise errors | `prim.IsInstanceProxy()` | Use a class, variant, or extra arc on the instance root (§22.4) | | 8 | **Muted layer** | `stage.GetMutedLayers()`, `stage.IsLayerMuted(id)` | Unmute | | 9 | **Time samples vs default:** you set the default but time samples in the same layer win at every time code; or a stronger layer’s default hides weaker samples | `GetResolveInfo(t).GetSource()` | Set time samples, or clear them (`attr.Clear()`) | | 10 | **Layer offset:** your samples are shifted by a sublayer or reference offset | `prim.GetPrimStackWithLayerOffsets()` | Author in the offset’s time frame, or adjust the offset |

Other causes worth one line each: the opinion lives in an **unloaded payload** (`stage.Load(path)`); a **value block** (None) in a stronger layer; the file **failed to load** (check `GetCompositionErrors()`); a **value clip** supplies the value at that time (Ch 21).

Mental model.

1 Is my edit in the file I think?	(edit target, saved?)
2 Is the prim there and live?	(typo? active? loaded? instance proxy?)
3 Is my layer in play?	(muted? in the sublayer list? arc resolved?)
4 Does my opinion win?	(GetPropertyStack: who is above me? stronger sublayer, stronger arc, variant)
5 Is it the right kind of value?	(default vs time samples, layer offset)

Location → existence → participation → strength → time.

Simple example. You set `radius = 5` on `/World/Ball` in `shot_anim.usda`, but the viewport shows 2. `GetPropertyStack()` lists `shot_lighting.usda` first with 2. Cause 3: a stronger sublayer. Talk to lighting, or author the change in their layer.

USDA example. File: *shot.usda* (three causes hiding in one file; Exercise 22-A at the end of this chapter uses it)

```
#usda 1.0
(
  subLayers = [
    @./anim.usda@
  ]
)

def "World"
{
  def Sphere "Ball" (
    active = false
  )
  {
    double radius = 2
  }

  def Sphere "Moon"
  {
    double radius.timeSamples = {
      1: 1,
      24: 4,
    }
    double radius = 9
  }
}

over "Wrold"
{
  over "Ball"
  {
    double radius = 5
  }
}
```

Line notes: `Ball` is inactive, so any `over` on it in `anim.usda` cannot show. `Moon` has both time samples and a default in the same layer; queries at a time code return the samples, not 9. `over "Wrold"` is a typo: it creates a separate undefined prim that `Traverse()` does not visit.

Python example. Each block of the script reproduces one cause and prints the authored value next to the composed one.

```
from pxr import Usd, Sdf

def stage_with(*names):
    stage = Usd.Stage.CreateInMemory()
    subs = [Sdf.Layer.CreateAnonymous(n) for n in names]
    stage.GetRootLayer().subLayerPaths = [l.identifier for l in subs]
    return stage, subs

def ball(stage, value=2.0):
    attr = stage.DefinePrim("/World/Ball").CreateAttribute(
        "radius", Sdf.ValueTypeNames.Double)
    attr.Set(value)
```



```

    return attr

# 1. Wrong edit target: the edit lands in the session layer, which is not saved.
s = Usd.Stage.CreateNew("shot.usda")
a = ball(s)
s.SetEditTarget(Usd.EditTarget(s.GetSessionLayer()))
a.Set(5.0)
s.Save()
reopened = Sdf.Layer.FindOrOpen("shot.usda")
print("1 saved file has:", reopened.GetAttributeAtPath("/World/Ball.radius").default)

# 2. Weaker arc: edit the referenced prim, but the referencing prim is stronger.
s = Usd.Stage.CreateInMemory()
lib = s.DefinePrim("/Lib/Ball")
lib.CreateAttribute("radius", Sdf.ValueTypeNames.Double).Set(1.0)
b = s.DefinePrim("/World/Ball")
b.GetReferences().AddInternalReference("/Lib/Ball")
b.GetAttribute("radius").Set(2.0)
lib.GetAttribute("radius").Set(5.0)
print("2 authored 5 in reference, composed:", b.GetAttribute("radius").Get())

# 3. Stronger local opinion: I author in the weaker sublayer.
s, (strong, weak) = stage_with("strong", "weak")
s.SetEditTarget(Usd.EditTarget(strong))
a = ball(s, 2.0)
s.SetEditTarget(Usd.EditTarget(weak))
a.Set(5.0)
print("3 authored 5 in weak sublayer, composed:", a.Get())

# 4. Variant selection in a stronger layer.
s, (strong, weak) = stage_with("strong", "weak")
p = s.DefinePrim("/World/Lamp")
vset = p.GetVariantSets().AddVariantSet("size")
for name in ("big", "small"):
    vset.AddVariant(name)
s.SetEditTarget(Usd.EditTarget(weak))
vset.SetVariantSelection("big")
s.SetEditTarget(Usd.EditTarget(strong))
vset.SetVariantSelection("small")
print("4 selected big in weak layer, composed:", vset.GetVariantSelection())

# 5. Typo in the path: a separate, undefined prim.
s = Usd.Stage.CreateInMemory()
a = ball(s)
s.OverridePrim("/Wrold/Ball").CreateAttribute(
    "radius", Sdf.ValueTypeNames.Double).Set(5.0)
print("5 typo, composed:", a.Get(),
      "| Traverse:", [str(x.GetPath()) for x in s.Traverse()])

# 6. Over on an inactive prim.
s, (strong, weak) = stage_with("strong", "weak")
s.SetEditTarget(Usd.EditTarget(weak))
s.DefinePrim("/World/Ball/Light").CreateAttribute(
    "intensity", Sdf.ValueTypeNames.Float).Set(5.0)
s.SetEditTarget(Usd.EditTarget(strong))
s.GetPrimAtPath("/World/Ball").SetActive(False)
print("6 child of inactive prim valid:", bool(s.GetPrimAtPath("/World/Ball/Light")))

# 7. Inside an instance: an over below the instance root is ignored.
s = Usd.Stage.CreateInMemory()
s.DefinePrim("/Lib/Chair/Seat").CreateAttribute(
    "h", Sdf.ValueTypeNames.Double).Set(1.0)
c = s.DefinePrim("/World/Chair")
c.GetReferences().AddInternalReference("/Lib/Chair")
c.SetInstanceable(True)
layer = s.GetRootLayer()
seat = Sdf.CreatePrimInLayer(layer, "/World/Chair/Seat")
Sdf.AttributeSpec(seat, "h", Sdf.ValueTypeNames.Double).default = 5.0
print("7 authored 5 under instance, composed:",
      s.GetPrimAtPath("/World/Chair/Seat").GetAttribute("h").Get())

# 8. Muted layer.
s, (anim,) = stage_with("anim")

```

```

s.SetEditTarget(Usd.EditTarget(anim))
a = ball(s, 5.0)
s.MuteLayer(anim.identifier)
print("8 authored 5 in muted layer, attribute valid:", a.IsValid())

# 9. Time samples beat the default in the same layer.
s = Usd.Stage.CreateInMemory()
a = ball(s, 9.0)
a.Set(1.0, 1)
a.Set(4.0, 24)
print("9 default 9, at frame 12:", a.Get(12), a.GetResolveInfo(12).GetSource())

# 10. Layer offset shifts my samples.
s, (anim,) = stage_with("anim")
s.GetRootLayer().subLayerOffsets[0] = Sdf.LayerOffset(100)
s.DefinePrim("/World/Ball")
anim_ball = Sdf.CreatePrimInLayer(anim, "/World/Ball")
spec = Sdf.AttributeSpec(anim_ball, "radius", Sdf.ValueTypeNames.Double)
anim.SetTimeSample(spec.path, 10, 5.0)
anim.SetTimeSample(spec.path, 20, 6.0)
a = s.GetPrimAtPath("/World/Ball").GetAttribute("radius")
print("10 authored at frame 10, samples on stage:", a.GetTimeSamples())

```

Expected output

```

1 saved file has: 2.0
2 authored 5 in reference, composed: 2.0
3 authored 5 in weak sublayer, composed: 2.0
4 selected big in weak layer, composed: small
5 typo, composed: 2.0 | Traverse: ['/World', '/World/Ball']
6 child of inactive prim valid: False
7 authored 5 under instance, composed: 1.0
8 authored 5 in muted layer, attribute valid: False
9 default 9, at frame 12: 2.4347826086956523 Usd.ResolveInfoSourceTimeSamples
10 authored at frame 10, samples on stage: [110.0, 120.0]

```

Cause 7 is the sneakiest: no error, no warning, just an ignored opinion. Cause 8 removes the attribute entirely because the muted layer was the only place it was defined.

VERSION

Verified on USD 26.08: overs authored below an instance root are ignored without a warning, and `Usd` authoring on instance proxies raises `Tf.ErrorException`.

Real-world use case. A VFX facility adds this checklist to its “my change doesn’t show” help page and wraps causes 1, 5, 6, 7, 8, and 10 in an automatic script. An artist pastes a prim path and attribute name; the script prints the edit target, prim validity, active state, instance-proxy state, muted layers, the property stack with layer offsets, and the resolve source. Most tickets close without a technical director.

Common mistakes.

MISTAKE

"Fixing" a value by authoring it in the root layer every time. It works, but it hides the real cause and creates overrides nobody can find later. Fix: find the cause with the checklist, then author in the layer that should own the value.

MISTAKE

Setting `attr.Set(value)` (default) on an animated attribute and expecting the animation to change. Fix: set time samples, or clear them with `attr.Clear()` if you want a static value.

MISTAKE

Typing paths by hand in USDA overs. Fix: copy paths from the stage (`prim.GetPath()`), and check `stage.TraverseAll()` for stray `over` prims.

Exam traps.

TRAP

"The layer is sublayered, so its opinion must win" ignores order: the first sublayer is strongest, and the root layer beats all sublayers.

TRAP

A stronger layer's **default** beats a weaker layer's **time samples** at every time code. Time samples only beat defaults within the same layer. Verified on USD 26.08.

TRAP

A variant selection is itself an opinion with strength. "The variant was authored, so it applies" is wrong if a stronger layer selects another variant.

TRAP

Inactive is not the same as invisible. ``visibility = invisible`` keeps the prim composed and editable; ``active = false`` removes it and its children from the stage.

Exam takeaways.**KEY**

- Check in order: location (edit target, saved), existence (typo, active, loaded, instance), participation (muted, resolved), strength (property stack), time (samples, offsets). - Overs below an instance root are ignored silently; ``Usd`` edits on proxies raise errors. - Stronger default beats weaker samples; within one layer, samples beat the default. - Variant selections and ``active`` are opinions too, with normal strength rules.

Chapter lab(s)

Lab 21 — Composition introspection toolkit (Ch 22, 42; Obj 1.8, 6.2). You build a small shot with sublayers, references, a variant, and a class, then write a reusable `explain(prim_path, attr_name)` function that prints the prim stack, property stack with layer offsets, resolve source, composition arcs, muted layers, and composition errors. You finish by breaking the shot in five checklist ways and letting your tool diagnose each one.

Related: **Lab 28 — Dependencies, flattening, and USDZ packaging** (Obj 1.9) and **Lab 23 — Native instancing and instance proxies** (Obj 2.2, 2.5) practise §22.3 and §22.4 in depth.

USDA reading exercises

Exercise 22-A. Using the `shot.usda` from §22.6 step 7 (with `anim.usda` empty), answer: (a) Which prims does `stage.Traverse()` visit? (b) What does `/World/Moon.radius` return at frame 12 and at the default time? (c) What happens to `over "Ball" { double radius = 5 }` if it is added to `anim.usda` under `over "World"`?

Exercise 22-B. The assembly below references `chair.usda` (which defines `/Chair/Seat` with `float roughness = 0.5` and no inherits). What is `/Room/Chair_1/Seat.roughness`, and what is `/Room/Chair_2/Seat.roughness`?

File: `room.usda`

```
#usda 1.0

def "Room"
{
  def "Chair_1" (
    instanceable = true
    prepend references = @./chair.usda@
  )
  {
    over "Seat"
    {
      float roughness = 0.9
    }
  }
}
```

```

    }
}

def "Chair_2" (
    instanceable = true
    prepend inherits = </_rough>
    prepend references = @./chair.usda@
)
{
}

class "_rough"
{
    over "Seat"
    {
        float roughness = 0.9
    }
}

```

Exercise 22-C. A delivery script runs `UsdUtils.FlattenLayerStack(stage)` on an asset with a `color` variant set and a reference to `/studio/lib/bolt.usda`. Does the result still contain (a) the variant set, (b) the bolt reference, (c) relative texture paths?

Chapter review

Summary

- Split monolithic assets into workstream layers (geo, mtl, rig) that share one namespace and combine as sublayers; only the owner `def`s a prim.
- Bring assets into scenes with references or payloads; layout is a scene-level layer.
- Shot layering: one writer per layer, downstream departments stronger, shared layers weakest; use edit targets, the session layer, and muting.
- `Usd.Stage.Flatten` bakes everything (selected variants only); `UsdUtils.FlattenLayerStack` merges only sublayers and keeps arcs.
- `UsdUtils.LocalizeAsset` and `UsdUtils.CreateNewUsdzPackage` collect dependencies; a `processingFunc` can drop internal ones. Always check the `bool` result.
- Flattening makes asset paths absolute; fix with `ModifyAssetPaths` or a callback. Strip private metadata yourself.
- Instance proxies and prototypes are read-only; change instanced content through the instance root's arcs or an inherited class. "Remove" means block or deactivate.
- Debug with `GetPrimStack`, `GetPropertyStack`, `GetResolveInfo`, `PrimCompositionQuery`, `DumpToString`, `GetCompositionErrors`.
- Ten causes of "my opinion does not show": edit target, weaker arc, stronger sublayer, variant selection, typo, inactive, instance, muted, samples vs default, layer offset.

If you see... → think...

If you see...	Think...
"Several departments edit the same asset/shot"	Separate layers, sublayers, edit targets, one writer per layer
"Deliver to a client / external party"	Localize or USDZ-package; drop internal deps; strip metadata; check result
"Keep variants but merge layers"	<code>UsdUtils.FlattenLayerStack</code>
"Single self-contained layer, no arcs"	<code>Usd.Stage.Flatten</code> (and fix absolute paths)

If you see...	Think...
"Remove a property from instanced components"	Block in a class / variant / extra arc on the instance root; not on proxies
"authoring to an instance proxy is not allowed"	You are editing below an instance root
"Which layer provides this value?"	<code>attr.GetPropertyStack()[0].layer</code>
"Time samples or default?"	<code>attr.GetResolveInfo(t).GetSource()</code>
"Which arcs build this prim?"	<code>Usd.PrimCompositionQuery(prim).GetCompositionArcs()</code>
"Missing file / broken reference"	<code>stage.GetCompositionErrors()</code>

Review questions

Q1 · Obj 1.11 · Single choice A studio wants modeling and surfacing to publish a prop independently. Which layout is most appropriate? A. `prop.usda` with a variant set `dept = {model, surface}` B. `prop.usda` sublayering `prop_mtl.usda` and `prop_geo.usda`, both authoring under `/Prop` C. Two separate assets, `prop_model.usda` and `prop_surface.usda`, each with its own `defaultPrim`, referenced side by side in each shot D. One `prop.usda` where both teams save in turn

Q2 · Obj 1.5 · Single choice In `subLayers = [a.usda, b.usda, c.usda]` on the root layer `shot.usda`, which layer's opinion wins when all four (including `shot.usda`) set the same attribute default? A. `c.usda` B. `a.usda` C. `shot.usda` D. The one saved most recently

Q3 · Obj 1.5 · Select two. Which practices support multi-user shot work? A. Each artist sets the edit target to their own layer B. Everyone writes to the root layer and merges by hand C. Shared sequence layers placed weakest in the sublayer list D. Variant sets named after artists E. Saving experiments into the department layer and reverting later

Q4 · Obj 1.9 · Single choice Which call returns a single layer in which references and variant sets are preserved but sublayers are merged? A. `stage.Flatten()` B. `UsdUtils.FlattenLayerStack(stage)` C. `UsdUtils.LocalizeAsset(Sdf.AssetPath(p), "out")` D. `stage.GetRootLayer().Export("out.usda")`

Q5 · Obj 1.9 · Code reading What does this print if `/studio/tools/x.py` cannot be resolved and no `processingFunc` is passed?

```
ok = UsdUtils.LocalizeAsset(Sdf.AssetPath("asset.usda"), "out")
print(ok)
```

(Not run automatically: it needs an `asset.usda` that references an internal path.) A. True, and the path is removed B. False, and the layer is still written with a placeholder path C. It raises `Tf.ErrorException` and writes nothing D. None

Q6 · Obj 1.9 · Select two. After `Usd.Stage.Flatten()`, which statements are true? A. Unselected variants are gone B. Asset paths are made absolute C. `customData` with artist names is removed D. Instancing is always removed and every instance is copied out

Q7 · Obj 1.10 · Single choice `/Room/Chair_1` is instanceable. Which edit removes the value of `studio:tag` from `Chair_1/Seat` while keeping `Chair_1` instanced? A. `stage.GetPrimAtPath("/Room/Chair_1/Seat").GetAttribute("studio:tag").Block()` B. Author over `"Seat" { custom string studio:tag = None }` under `Chair_1` in the room layer C. Add `prepend inherits = </_fix>` on `Chair_1` and block `studio:tag` under class `"_fix"'s Seat` D. Block it on `Chair_1.GetPrototype()`

Q8 · Obj 1.10 · Single choice After blocking `studio:tag` through a class, what does `attr.IsValid()` return? A. True B. False C. None D. It raises an error

Q9 · Obj 1.8 · Code reading `GetPropertyStack()` for radius returns `[session spec (no value, block), shot.usda spec (2.0), asset.usda spec (1.0)]`. What does `attr.Get()` return? A. 2.0 B. 1.0 C. None D. The fallback value of the schema

Q10 · Obj 1.8 · Single choice An artist adds over "Ball" (`active = true`) in a weak sublayer, but Ball stays inactive. Why? A. `active` cannot be authored in sublayers B. A stronger layer authors `active = false`, and `active` follows normal strength rules C. Inactive prims can only be activated with `SetActive` in Python D. The prim is an instance proxy

Q11 · Obj 1.8 · Select two. A weak layer selects variant `big`, but the prim shows `small`. Which tools confirm why? A. `prim.GetPrimStack()` (shows which variant spec contributes) B. `prim.GetVariantSets().GetVariantSelection("size")` C. `stage.GetCompositionErrors()` D. `UsdUtils.ComputeAllDependencies`

Q12 · Obj 1.8 · Single choice Which `Usd.CompositionArc` method tells you the layer where an arc was authored? A. `GetTargetLayer()` B. `GetIntroducingLayer()` C. `GetArcType()` D. `IsImplicit()`

Q13 · Obj 1.2 · Single choice A forest of 2 million trees from 12 tree models, where per-tree edits are only position, rotation, scale, and hiding. Which instancing style fits? A. No instancing B. Scenegraph instancing with `instanceable = true` on 2 million prims C. `UsdGeom.PointInstancer` with 12 prototypes D. One variant per tree

Q14 · Obj 1.8 · USDA reading In one layer: `double radius = 9` and `double radius.timeSamples = { 1: 1, 24: 4 }`. What does `Get(Usd.TimeCode.Default())` return, and what does `Get(1)` return?

Answers

Q1 — B. Workstream layers under one namespace combined as sublayers. A uses variants for parts that should be combined. C makes two “assets” that every shot must combine. D blocks parallel work. Review: §22.1.

Q2 — C. The root layer is stronger than all its sublayers; among sublayers `a.usda` would be next. Save time never matters. Review: §22.2.

Q3 — A and C. Own edit targets avoid conflicts; shared layers weakest let shots override them. B and E cause lost or leaked work; D misuses variants. Review: §22.2.

Q4 — B. `FlattenLayerStack` merges only the layer stack. A removes arcs and variants. C copies files but does not merge. D exports the root layer alone. Review: §22.3.

Q5 — B. Verified on 26.08: it returns `False`, prints warnings, and writes layers with placeholder paths such as `@0/x.py@`. Review: §22.3.

Q6 — A and B. Flatten keeps only selected variants and anchors asset paths. Metadata stays (C false). Instancing is preserved through `Flattened_Prototype_N` prims (D false). Review: §22.3.

Q7 — C. An extra `inherit` on the instance root plus a class opinion changes instanced content legally. A and D raise errors; B is silently ignored. Review: §22.4.

Q8 — A. A block hides the value; the property still exists. Review: §22.4.

Q9 — C. The strongest opinion is a block, so there is no value. Weaker values are hidden. Review: §22.5, §22.6.

Q10 — B. `active` is ordinary metadata with strength; the stronger `false` wins. Review: §22.6 (cause 6).

Q11 — A and B. The prim stack shows `{size=small}` specs; `GetVariantSelection` shows the winning selection. Errors and dependencies are unrelated. Review: §22.5, §22.6 (cause 4).

Q12 — B. `GetIntroducingLayer()` is where the arc is authored; `GetTargetLayer()` is the layer the arc points to. Review: §22.5.

Q13 — C. Point instancing scales to millions with limited per-copy edits. B creates 2 million prims; A and D do not share data. Review: §22.4 (and Ch 25–26).

Q14 — 9.0 for the default time, 1.0 at frame 1. Time samples win at time codes; the default answers only a default-time query. Review: §22.6 (cause 9).

USDA reading exercise answers

22-A. (a) `/World` and `/World/Moon`. `Ball` is inactive, so `Traverse()` skips it, and `/Wrold` is only an `over` (not defined), so `Traverse()` skips it too. (b) At frame 12: about 2.435 (linear interpolation: $1 + 3 \times 11/23$); at the default time: 9.0. (c) Nothing: `Ball` is inactive in the stronger root layer, so the `over` is never seen (checklist cause 6).

22-B. `Chair_1`: 0.5. The `over "Seat"` under an instance root is ignored (cause 7). `Chair_2`: 0.9. The `inherit` is an arc on the instance root, and `inherits` are stronger than references. The two chairs have different prototypes.

22-C. (a) Yes, `FlattenLayerStack` keeps variant sets. (b) Yes, it keeps references, so the internal bolt reference is still there and must be removed separately. (c) No, it anchors asset paths to absolute paths unless you pass a callback such as `lambda layer, path:` `path`.

Part IV — Content Aggregation

CHAPTER 23

Chapter 23 — Asset Structure and Model Hierarchy

Exam domain: Content Aggregation (10%), also Pipeline Development (14%) ·

Objectives: 2.4, 7.2 · **Study day:** 8 · **Est. time:** 90 min **Prerequisites:** Ch 6 (schemas and model kinds), Ch 15 (sublayers), Ch 16 (references), Ch 17 (payloads), Ch 18 (variant sets), Ch 19 (inherits and classes)

Part III taught you *how* composition arcs work. This part teaches you how to *organize* real content with them. **Content aggregation** means assembling many reusable assets into larger scenes (a room from chairs, a city from buildings) in a way that stays fast and easy to manage. This chapter is about the shape of a single asset and the “map” that lets tools find assets inside a huge scene. Chapter 24 then shows how to make thousands of copies of an asset cheaply.

Learning goals

- Explain the four principles of scalable asset structure: legibility, modularity, performance, navigability.
- Build an asset interface: a small entry layer that payloads binary content layers.
- Apply kinds (`component`, `group`, `assembly`, `subcomponent`) so that the model hierarchy is valid, and predict `IsModel()`, `IsGroup()`, `IsComponent()`.
- Design modular components that are referenced into assemblies and edited through classes.
- Document an asset structure using the ASWF guidelines (geo/mtl scopes, class per asset, no `group` kind on published roots).

Key terms

Term	One-line definition
Asset	A named, versioned, structured container of files (USD layers, textures, ...) meant to be reused.
Entry layer (asset interface layer)	The root layer of an asset: the file other scenes reference or open.
Entry point prim	The prim inside the entry layer that others reference, usually the <code>defaultPrim</code> .
Lofting	Copying cheap, important data (kind, variant sets, inherits, <code>extentsHint</code>) up into the entry layer, above the payload.
Kind	Prim metadata (a token) that labels a prim’s role: <code>component</code> , <code>group</code> , <code>assembly</code> , <code>subcomponent</code> .
Model	A prim whose kind is <code>model</code> or derived from it, <i>and</i> which sits in a valid model hierarchy.
Model hierarchy	The tree of model prims from the root down to the component level; a quick “map” of the scene.
Component	A complete, leaf-level asset (a chair, a house). The model hierarchy stops here.
Assembly	A published asset that aggregates other components or assemblies (a room, a city).
Group	

Term	One-line definition
	A kind for organizing prims <i>inside</i> an assembly so the model hierarchy stays valid.
Subcomponent	A marker for an important prim <i>inside</i> a component (a door, a lid). Not a model.
Scope	A prim type (<code>UsdGeom.Scope</code>) used only to organize children; it has no transform.
ASWF guidelines	Community asset-structure recommendations from the ASWF USD Working Group.

23.1 Principles of scalable asset structure

What is it? **Asset structure** is the set of rules for how an asset is split into files, prims, and names. NVIDIA’s whitepaper *Principles of Scalable Asset Structure in OpenUSD* says a scalable structure should be **legible**, **modular**, **performant**, and **navigable**.

Why do we need it? Without agreed rules, every artist and tool invents its own layout. Scripts break, scenes become slow, and nobody can find the prim they need. A structure lets many people and processes work in parallel on the same asset and lets large scenes stay manageable.

Beginner explanation. Think of a well-run warehouse. Every box has a clear **label** (legible). Boxes are standard sizes so they stack and ship anywhere (modular). Heavy pallets stay in the back until needed (performant). Aisles are numbered so anyone can find item 4-B-12 (navigable).

Where the analogy breaks: a warehouse box is physical and in one place. A USD asset is many layers composed together, and the same asset can appear in thousands of scenes at once.

Technical explanation. The four principles, as questions you ask about every design decision:

Principle	Question to ask	Typical USD tools
Legibility	Do prim, property, and file names say what they are?	Naming conventions, <code>Tf.IsValidIdentifier</code> , <code>assetInfo</code> , <code>displayName</code>
Modularity	Can parts be improved and reused independently?	Separate layers per workstream, <code>defaultPrim</code> , references, anchored paths (<code>./</code>)
Performance	Does the structure keep reading and writing fast?	Payloads, USDC for heavy data, instancing, small entry layers
Navigability	Can people and tools find things by inspection?	Partition scopes (<code>geo</code> , <code>mtl</code>), model hierarchy (kinds), relationships, collections

Further rules from the same source:

- **Workstreams become layers.** Geometry and materials are often separate layers so one can be republished without touching the other.
- **Partition the hierarchy** with Scope prims. Scope has no transform, so it cannot move its children by accident.
- **Naming signals intent.** Capitalized names often mean “public, safe to override”; a leading `_` means “internal”. Names beginning with `__` are reserved by OpenUSD (for example the prototype prims `__Prototype_1` you meet in Chapter 24).
- **Do not key logic off names.** A mesh must not render differently because of its name. Use kinds, schemas, or properties instead.
- **Do not grow layer stacks forever** (for example one sublayer per day). Every layer costs time to resolve and open. Layer stacks are not a version-control system.

Mental model.

SCALABLE ASSET STRUCTURE			
LEGIBLE	MODULAR	PERFORMANT	NAVIGABLE
names say what it is	parts are reusable	heavy data loads late	you can find it

Simple example. | Bad | Better | Principle | |----|-----|-----| | Prim /Mesh_017 | / LargeCardboardBox/geo/box | Legibility | | One 2 GB file with geometry, shading, rig | geo.usdc, mtl.usdc, entry layer | Modularity | | Opening the city loads every building | Buildings behind payloads | Performance | | Materials mixed in with meshes | geo and mtl scopes | Navigability |

USDA example.

```
#usda 1.0
(
    defaultPrim = "LargeCardboardBox"
    metersPerUnit = 0.01
    upAxis = "Y"
)

def Xform "LargeCardboardBox" (
    kind = "component"
)
{
    def Scope "geo"
    {
        def Mesh "box"
        {
        }
    }

    def Scope "mtl"
    {
    }
}
```

- `defaultPrim` names the entry point, so others can reference the file without a prim path (Chapter 16).
- `metersPerUnit` and `upAxis` make units explicit (Chapter 28).
- The name `LargeCardboardBox` is legible; `kind = "component"` makes it navigable; `geo` and `mtl` partition the contents.

Python example. A tiny “principles audit” that checks a few measurable rules.

```
from pxr import Usd, Sdf, Tf

stage = Usd.Stage.CreateInMemory()
stage.GetRootLayer().ImportFromString("""#usda 1.0
(
    defaultPrim = "LargeCardboardBox"
    metersPerUnit = 0.01
    upAxis = "Y"
)

def Xform "LargeCardboardBox" (
    kind = "component"
)
{
    def Scope "geo"
    {
        def Mesh "box"
        {
        }
    }
}
```

```

def Scope "mtl"
{
}
}
"""

root = stage.GetDefaultPrim()
checks = {
    "legible: root name is a valid identifier": Tf.IsValidIdentifier(root.GetName()),
    "modular: defaultPrim is set": stage.HasDefaultPrim(),
    "navigable: root has a kind": bool(Usd.ModelAPI(root).GetKind()),
    "navigable: geo and mtl scopes exist": all(
        root.GetChild(n).GetTypeName() == "Scope" for n in ("geo", "mtl")
    ),
}
for name, ok in checks.items():
    print("PASS" if ok else "FAIL", name)

```

Expected output

```

PASS legible: root name is a valid identifier
PASS modular: defaultPrim is set
PASS navigable: root has a kind
PASS navigable: geo and mtl scopes exist

```

Real-world use case. A robotics company publishes thousands of warehouse props for synthetic-data simulation. Each prop follows the same structure, so one script can scatter, validate, and render any of them. When a texture is fixed in `mtl.usdc`, the geometry file is untouched, so storage deduplication keeps working and only one small file is re-sent to the render farm.

Common mistakes.

MISTAKE

Encoding meaning only in names (e.g. "anything called ``*_proxy`` is hidden"). ****Fix:**** use real data: ``purpose``, ``kind``, or a custom property. Names are hints for humans.

MISTAKE

Adding a new sublayer for every revision (``asset_2026_10_01.usd``, ``asset_2026_10_02.usd``, ...). ****Fix:**** use a versioning system or asset resolver for versions; keep layer stacks short.

MISTAKE

Grouping children under ``Xform`` "folders". ****Fix:**** use ``Scope`` for pure organization; it has no transform to change by accident.

Exam traps.

TRAP

The four principles are ****legibility, modularity, performance, navigability****. Look-alike distractors such as "portability" or "flexibility" are not the named four.

TRAP

"There is one correct asset structure for all pipelines." False. Both NVIDIA and the ASWF stress that structure depends on the clients and collaborators; the principles guide the choice.

Exam takeaways.

KEY

- Scalable = ****legible, modular, performant, navigable****. - Workstreams (geometry, materials, ...) become separate layers. - Partition with ``Scope`` prims; ``_name`` = internal, ``__name`` = reserved by USD. - Never make behavior depend on prim names.

23.2 Asset interfaces (entry layer, geo/mtl/payload layers)

What is it? An **asset interface** is the public “front door” of an asset: a small **entry layer** with one **entry point prim** that others reference. The heavy content sits behind a **payload** in separate content layers (typically geometry and materials).

Why do we need it? If a scene had to open every heavy file just to learn an asset’s name, kind, or variants, opening a city would take minutes. The interface keeps that cheap information on top, so a stage can be opened *unloaded* and still show what each asset is and what options it offers.

Beginner explanation. Think of a product box in a shop. The **label** on the outside (name, size, colors available) is the entry layer. The **contents** inside the sealed box are the payload. You can browse a whole shelf of labels without opening a single box.

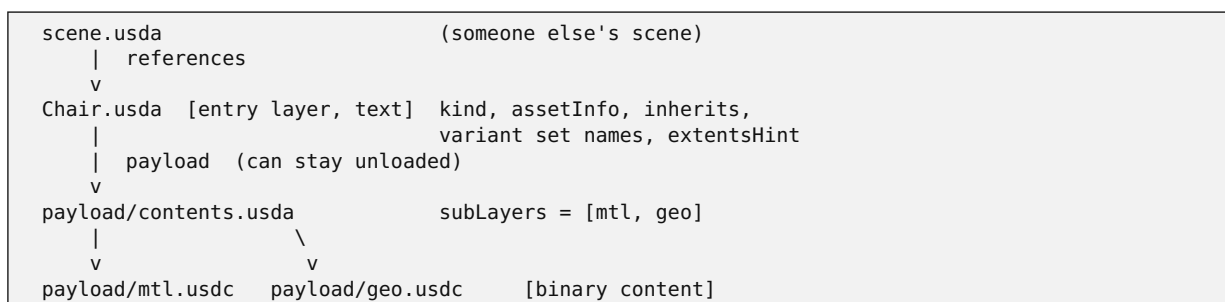
Where the analogy breaks: in USD you can “open the box” (load the payload) and “close it” again at any time, per prim, without losing your edits (Chapter 17).

Technical explanation. NVIDIA’s recommended starting point is: **a referenceable text entry layer → containing a payload → to binary sublayers.**

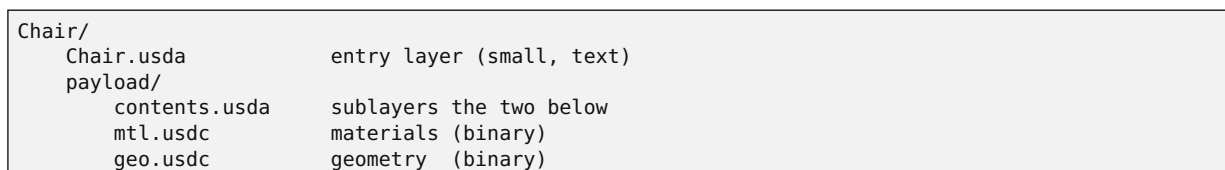
- **Entry layer** (`Chair.usda` or `Chair.usd`): sets `defaultPrim`, and on the entry prim authors `kind`, `assetInfo`, `inherits` to a class, the *names* of variant sets with default selections, and optionally `extentsHint` via `UsdGeom.ModelAPI`. This is called **lofting** the cheap fields above the payload.
- **Payload layer** (`payload/contents.usd`): its job is to sublayer the content layers, e.g. `mtl.usdc` over `geo.usdc`. Keeping this middle layer lets you mute one content layer; a payload’s *target* is treated as a root layer and cannot be muted.
- **Content layers** (`geo.usdc`, `mtl.usdc`): heavy data in binary **Crate** format (USDC), which reads only what composition needs.
- **Anchored paths** (`./payload/contents.usd`) are resolved relative to the layer that contains them, so the asset folder can be moved as a whole.
- This is the **reference-payload pattern**: users *reference* the asset; the payload is internal. They never need to know whether a payload exists.

Why text for the entry layer and binary for the content? Text (USDA) is easy to read and diff, but is always read fully into memory. That is fine for a tiny entry layer. Heavy layers should be USDC. Using the `.usd` extension lets you switch between text and binary later without breaking anyone’s reference paths (Chapter 31).

Mental model.



Simple example.



USDA example. File: `Chair/Chair.usda` (the entry layer)

```
#usda 1.0
(
    defaultPrim = "Chair"
)

def Xform "Chair" (
    kind = "component"
    assetInfo = {
        string name = "Chair"
    }
    prepend payload = @./payload/contents.usda@
)
{
}
```

File: Chair/payload/contents.usda (the payload layer)

```
#usda 1.0
(
    defaultPrim = "Chair"
    subLayers = [
        @./mtl.usdc@,
        @./geo.usdc@
    ]
)
```

- The entry prim carries the cheap, important data. `assetInfo` advertises the asset name.
- `contents.usda` has no prims of its own; it only stacks the content layers. `mtl.usdc` is listed first, so it is stronger: materials may override geometry (for example by adding bindings).
- The content layers are binary, so they are not shown here; Section 23.5 shows what goes in them.

Python example. Build the whole asset on disk, open it **unloaded**, then load it.

```
import os
from pxr import Usd, Sdf, UsdGeom

os.makedirs("Chair/payload", exist_ok=True)

# 1. Binary content layers (USDC): geometry and materials.
geo = Sdf.Layer.CreateNew("Chair/payload/geo.usdc")
geo_stage = Usd.Stage.Open(geo)
geo_stage.SetDefaultPrim(geo_stage.DefinePrim("/Chair", "Xform"))
UsdGeom.Scope.Define(geo_stage, "/Chair/geo")
UsdGeom.Cube.Define(geo_stage, "/Chair/geo/seat")
geo.Save()

mtl = Sdf.Layer.CreateNew("Chair/payload/mtl.usdc")
mtl_stage = Usd.Stage.Open(mtl)
mtl_stage.SetDefaultPrim(mtl_stage.OverridePrim("/Chair"))
UsdGeom.Scope.Define(mtl_stage, "/Chair/mtl")
mtl.Save()

# 2. The payload layer just sublayers the content layers.
contents = Sdf.Layer.CreateNew("Chair/payload/contents.usda")
contents.subLayerPaths.append("./mtl.usdc")
contents.subLayerPaths.append("./geo.usdc")
contents.defaultPrim = "Chair"
contents.Save()

# 3. The interface layer: small, text, lofted metadata + payload.
iface = Sdf.Layer.CreateNew("Chair/Chair.usda")
iface.ImportFromString("""#usda 1.0
(
    defaultPrim = "Chair"
)

def Xform "Chair" (
```

```

    kind = "component"
    assetInfo = {
        string name = "Chair"
    }
    prepend payload = @./payload/contents.usda@
)
{
}
)
)
iface.Save()

stage = Usd.Stage.Open("Chair/Chair.usda", Usd.Stage.LoadNone)
chair = stage.GetPrimAtPath("/Chair")
print("unloaded kind:", Usd.ModelAPI(chair).GetKind())
print("unloaded children:", [c.GetName() for c in chair.GetChildren()])
stage.Load("/Chair")
print("loaded children:", [c.GetName() for c in chair.GetChildren()])
names = sorted(os.path.basename(l.identifier) for l in stage.GetUsedLayers())
print("used layers:", [n for n in names if not n.startswith("anon")])

```

Expected output

```

unloaded kind: component
unloaded children: []
loaded children: ['geo', 'mtl']
used layers: ['Chair.usda', 'contents.usda', 'geo.usdc', 'mtl.usdc']

```

Notice that the kind is readable *before* loading: it was lofted into the entry layer.

Real-world use case. A film studio opens a 20,000-asset city set with `Usd.Stage.LoadNone`. Layout artists see every building's name, kind, variant sets, and bounding box (from `extentsHint`), and load only the three buildings in camera. In manufacturing, a factory digital twin does the same with machines: engineers browse the line unloaded and load one robot cell to inspect it.

Common mistakes.

MISTAKE

Putting variant set *definitions* only inside the payload. When the payload is unloaded, users cannot see or set the variants. **Fix:** loft the variant set names and default selections onto the entry prim (empty variants are fine; the payload supplies the opinions).

MISTAKE

Using absolute file paths inside the asset (``/mnt/proj/chair/geo.usdc``). **Fix:** use anchored relative paths (`./payload/geo.usdc``) so the folder can be moved or delivered.

MISTAKE

Making the entry layer itself heavy (meshes in the text file). **Fix:** keep it tiny; heavy data goes in USDC behind the payload.

Exam traps.

TRAP

"Payloads are always better than references for assets." Not exactly: the recommended pattern is that users **reference** the entry layer and the asset **payloads** its own contents internally (reference-payload pattern).

TRAP

The sublayer order in ``contents.usda`` matters: the **first** listed sublayer is the **strongest** (Chapter 15).

Exam takeaways.

KEY

- Default structure: **text entry layer** → **payload** → **binary sublayers** (``mtl``, ``geo``). - Users **reference** the asset; the asset **payloads** its contents (reference-payload pattern). - **Loft** kind, ``assetInfo``, inherits, variant set names, ``extentsHint`` above the payload. - Use anchored ``./`` paths and the ``.usd`` extension for flexibility.

23.3 Model hierarchy and kinds in practice

What is it? The **model hierarchy** is a simplified, high-level view of a scene made from prims that carry **kind** metadata. Chapter 6 introduced the kinds. Here you learn the rule that makes a kind “count”: a prim is a **model** only if its kind is a model kind *and* every ancestor is a **group** or **assembly** (or a kind derived from them).

Why do we need it? A city scene may hold millions of prims. A tool that wants “all assets” should not visit every mesh and shader. The model hierarchy lets it stop at the **component** level, which is far fewer prims.

Beginner explanation. Think of a postal address: Country → City → Street → House. The post office does not care about the rooms inside a house. The address is the model hierarchy; the house is the component; the rooms are ordinary prims beneath it. If a house is listed with no street (“House 7, somewhere”), the post office cannot find it: a component without a valid chain of groups above it is not a model.

Where the analogy breaks: a real address is checked by people; in USD the rule is enforced automatically by `Usd` when it computes `IsModel()`.

Technical explanation. Built-in kinds (from `Kind.Registry.GetAllKinds()`): **model**, **group**, **assembly**, **component**, **subcomponent**. Their “is-a” relations:

```
model (abstract base; do not author it directly)
|-- group
|   |-- assembly
|   |-- component
subcomponent (NOT a model kind)
```

Rules, verified on USD 26.08:

1. A prim is a **model** (`prim.IsModel()`) if its kind is-a **model** *and* its parent is a **group** (or it is a root prim, whose parent is the pseudo-root).
2. A prim is a **group** (`prim.IsGroup()`) if it is a model and its kind is-a **group** (**group** or **assembly**).
3. `prim.IsComponent()` is true only for a *valid model* whose kind is-a **component**.
4. A component’s descendants are **outside** the model hierarchy. A component inside a component is not a model. Use **subcomponent** for important inner prims.
5. A prim with no kind breaks the chain: any **component** below it is not a model.

APIs:

- `Usd.ModelAPI(prim).GetKind()` / `SetKind("component")` read and write the kind metadata. `prim.GetMetadata("kind")` works too.
- `Usd.ModelAPI(prim).IsKind("component")` checks the kind *and* the hierarchy by default (`Usd.ModelAPI.KindValidationModelHierarchy`). Pass `Usd.ModelAPI.KindValidationNone` to check only the token.
- `Kind.Registry.IsA("assembly", "group")` compares kind tokens (True).
- Traversal predicate `Usd.PrimIsModel` (and `Usd.PrimIsGroup`) prunes traversal: children of a non-matching prim are skipped. There is no `PrimIsComponent` predicate; filter with `prim.IsComponent()`.

NOTE

Custom kinds (for example ``a_location`` derived from ``assembly``) are registered through plugin info and checked with ``Kind.Registry``. Chapter 38 covers them. NVIDIA's whitepaper recommends using custom kinds sparingly.

Mental model.

```
/City          assembly  model, group
/Block_A      group      model, group
/House_1      component  model, component  <-- model hierarchy stops here
/Door         subcomponent (marker only, not a model)
/props        (no kind)  breaks the chain
/Bench        component  NOT a model
```

Simple example. | Prim | Kind | Parent valid group? | IsModel() |

```
|-----|-----|-----|-----| | /Kitchen | assembly | root | True | | /Kitchen/Props |
group | yes | True | | /Kitchen/Props/Kettle | component | yes | True | | /Kitchen/Clutter
| none | — | False | | /Kitchen/Clutter/Mug | component | no | False |
```

USDA example.

```
#usda 1.0

def Xform "Kitchen" (
    kind = "assembly"
)
{
    def Xform "Props" (
        kind = "group"
    )
    {
        def Xform "Kettle" (
            kind = "component"
        )
        {
            def Xform "Lid" (
                kind = "subcomponent"
            )
            {
            }
        }
    }

    def Xform "Clutter"
    {
        def Xform "Mug" (
            kind = "component"
        )
        {
        }
    }
}
}
```

- Kitchen → Props → Kettle is a valid chain: assembly → group → component.
- Clutter has no kind, so Mug is *not* a model even though it says component.
- Lid is a subcomponent: a useful label, but outside the model hierarchy.

Python example.

```
from pxr import Usd

stage = Usd.Stage.CreateInMemory()
stage.GetRootLayer().ImportFromString("""#usda 1.0

def Xform "Kitchen" (
    kind = "assembly"
)
{
    def Xform "Props" (
```



```

        kind = "group"
    )
    {
        def Xform "Kettle" (
            kind = "component"
        )
        {
            def Xform "Lid" (
                kind = "subcomponent"
            )
            {
            }
        }
    }

    def Xform "Clutter"
    {
        def Xform "Mug" (
            kind = "component"
        )
        {
        }
    }
}
"""

print(f"{'path':28} {'kind':13} model group component")
for prim in stage.Traverse():
    kind = Usd.ModelAPI(prim).GetKind() or "-"
    print(f"{'str(prim.GetPath()):28} {'kind':13} {'prim.IsModel():s:5} "
          f"{'prim.IsGroup():s:5} {'prim.IsComponent():s:5}")

print("model traversal:")
for prim in stage.Traverse(Usd.PrimIsModel):
    print(" ", prim.GetPath())

```

Expected output

path	kind	model	group	component
/Kitchen	assembly	True	True	False
/Kitchen/Props	group	True	True	False
/Kitchen/Props/Kettle	component	True	False	True
/Kitchen/Props/Kettle/Lid	subcomponent	False	False	False
/Kitchen/Clutter	-	False	False	False
/Kitchen/Clutter/Mug	component	False	False	False

model traversal:

```

/Kitchen
/Kitchen/Props
/Kitchen/Props/Kettle

```

The `Usd.PrimIsModel` traversal never even visits `Clutter`'s children, which is exactly the pruning that makes the model hierarchy fast.

VERSION

Verified on USD 26.08: a ``component`` whose parent has no kind returns ``False`` for ``IsModel()`` and ``IsComponent()``; a ``component`` at the root of the stage returns ``True``.

Real-world use case. A lighting tool lists “every asset in the shot” by traversing with `Usd.PrimIsModel` and keeping components. On a 5-million-prim set it visits a few thousand prims instead of millions. In AEC (architecture, engineering, construction), a building is an assembly, each floor a group, and each furniture item a component, so a facilities app can count chairs per floor instantly.

Common mistakes.

MISTAKE

Referencing a component under an organizational prim with no kind (``/World/Props/Chair`` where ``World`` and ``Props`` have no kind). ****Fix:**** tag the ancestors ``assembly`/`group``.

MISTAKE

Referencing a component asset *inside* another component and leaving its kind as ``component``. **Fix:** override the inner one to ``subcomponent``, or restructure as an assembly ("package").

MISTAKE

Tagging individual meshes as ``component``. This makes the hierarchy deep and slow, the opposite of its purpose. **Fix:** components are whole products (a chair, a house).

Exam traps.**TRAP**

``subcomponent`` is **not** a model kind. ``prim.IsModel()`` is ``False`` for a subcomponent prim.

TRAP

``assembly`` is a kind of ``group``. ``IsGroup()`` is ``True`` for assemblies. A ``component`` is a model but **not** a group.

TRAP

Kind alone is not enough: ``GetKind()`` can return ``"component"`` while ``IsComponent()`` returns ``False`` because the ancestor chain is broken.

Exam takeaways.**KEY**

- Model = model kind **+** every ancestor is group/assembly (or the prim is at the root). - ``assembly`` is-a ``group``; ``component`` is-a ``model``; ``subcomponent`` is **not** a model. - The hierarchy **stops at components**; use ``subcomponent`` inside. - Traverse with ``Usd.PrimIsModel`` to visit only the model hierarchy.

23.4 Modular, reusable components

What is it? A **modular component** is a self-contained asset (geometry, materials, and its own class) that is published once and **referenced** many times into assemblies, with every use editable through the asset's **class** prim.

Why do we need it? If each room copied the chair's data, fixing the chair would mean editing every room. With references, you fix the chair file once and every scene sees the fix. Classes let an assembly change all chairs (or one chair) without touching the source file.

Beginner explanation. A **reference** is like linking a reusable part from a catalog (Chapter 16). A **class** is like a style sheet every copy follows, even after later edits (Chapter 19). Change the catalog part and every product that links it updates. Change the style sheet and every copy changes its look.

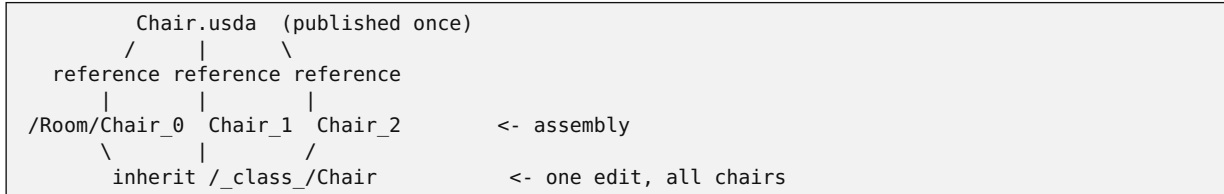
Where the analogy breaks: the "catalog" is composed live. There is no copy step at all; the chair data exists only once on disk.

Technical explanation. What makes a component modular:

- **Self-contained:** everything it needs lives under its root prim, apart from shared material libraries. Relationship targets (like `material:binding`) **must point inside the referenced prim**. A target outside the reference's scope is ignored, with the warning "refers to a path outside the scope of the reference" (verified on USD 26.08).

- **One entry point:** the root prim is `defaultPrim`, is transformable (`Xform`), and has `kind = "component"`.
- **Inherits a class:** the entry prim inherits `/_class_/<AssetName>`. Because the class path is *outside* the referenced prim, it maps to the same path in the **referencing** stage. This is how an assembly can author `/_class_/Chair` and reach every chair (implied inherits, Chapter 19.5).
- **Public interface:** variant sets and primvars on the entry prim are the “knobs” downstream users may set. Prims and properties with a `_` prefix are internal.
- **Assemblies** reference components (and other assemblies), tag organizing prims `group`, and position each reference with transforms on the referencing prim.

Mental model.



Simple example. | Want | Edit where | Affects | |-----|-----|-----| | Fix the seat shape for everyone | `Chair.usda` (source) | Every scene, every chair | | Smaller seats in this room only | `/_class_/Chair` in the room | All chairs in this room | | One chair different | `/Room/Chair_2/...` | Only that chair |

USDA example. *File: Chair.usda*

```
#usda 1.0
(
    defaultPrim = "Chair"
)

class "_class_"
{
    class "Chair"
    {
    }
}

def Xform "Chair" (
    kind = "component"
    prepend inherits = </_class_/Chair>
)
{
    def Scope "geo"
    {
        def Cube "seat"
        {
            double size = 1
        }
    }
}
}
```

File: Room.usda

```
#usda 1.0

over "_class_"
{
    over "Chair"
    {
        over "geo"
        {
            over "seat"
            {
                double size = 0.5
            }
        }
    }
}
```

```

    }
}

def Xform "Room" (
    kind = "assembly"
)
{
    def "Chair_0" (
        prepend references = @./Chair.usda@
    )
    {
    }
}

```

- The class is empty in the source. It contributes nothing until someone writes to it.
- `Room.usda` writes `size = 0.5` into `/_class_/Chair`. Every chair in the room inherits it. Inherits (I) are stronger than references (R) in LIVERPS, so the class beats the source's `size = 1`.

Python example.

```

from pxr import Usd, Sdf, UsdGeom

CHAIR = """#usda 1.0
(
    defaultPrim = "Chair"
)

class "_class_"
{
    class "Chair"
    {
    }
}

def Xform "Chair" (
    kind = "component"
    prepend inherits = </_class_/Chair>
)
{
    def Scope "geo"
    {
        def Cube "seat"
        {
            double size = 1
        }
    }
}
"""

with open("Chair.usda", "w") as f:
    f.write(CHAIR)

stage = Usd.Stage.CreateInMemory()
room = stage.DefinePrim("/Room", "Xform")
Usd.ModelAPI(room).SetKind("assembly")
for i in range(3):
    chair = stage.DefinePrim(f"/Room/Chair_{i}", "Xform")
    chair.GetReferences().AddReference("./Chair.usda")
    UsdGeom.XformCommonAPI(chair).SetTranslate((i * 2.0, 0, 0))

# One edit in the assembly's class reaches every chair.
over = stage.OverridePrim("/_class_/Chair/geo/seat")
over.CreateAttribute("size", Sdf.ValueTypeNames.Double).Set(0.5)
stage.GetPrimAtPath("/Room/Chair_2/geo/seat").GetAttribute("size").Set(2.0)

for i in range(3):
    seat = stage.GetPrimAtPath(f"/Room/Chair_{i}/geo/seat")
    print(f"Chair_{i} seat size:", seat.GetAttribute("size").Get())

```

Expected output

```
Chair_0 seat size: 0.5
Chair_1 seat size: 0.5
Chair_2 seat size: 2.0
```

Chair_2's own local opinion (L) beats the class (I), so it keeps 2.0.

Real-world use case. A game studio publishes one `StreetLamp` component and references it 400 times into a city assembly. When art direction wants warmer lamps at night, a lighting artist edits `/_class_/StreetLamp` in the night-time layer only. Daytime scenes are unaffected, and the source asset is untouched.

Common mistakes.

MISTAKE

Binding a component's mesh to a material outside the component (e.g. `_/Looks/Wood``). After referencing, the binding target is ignored. **Fix:** keep materials under the asset root (`_/Chair/mtl/...``) or reference a library material *into* the asset.

MISTAKE

Editing the source asset to fix a problem that exists only in one scene. **Fix:** edit the class or the referencing prim in that scene's layer.

Exam traps.

TRAP

A class prim (``class`` specifier) is **abstract**: it is not traversed by default and is not rendered. Its only job is to be inherited.

TRAP

"Changing the class affects only instanceable copies." False: class edits affect every prim that inherits it, instanced or not (Chapter 24).

Exam takeaways.

KEY

- Component = self-contained, ``defaultPrim``, ``Xform`` root, ``kind = "component"``, inherits a class. - Relationship targets must stay **inside** the asset root. - Edit the **source** to fix everywhere, the **class** to change all copies in a scene, the **prim** to change one.

23.5 ASWF asset structure guidelines

What is it? The **ASWF guidelines** (*Guidelines for Structuring USD Assets*, from the Academy Software Foundation USD Working Group) are community recommendations for a “minimum viable” USD asset: what a component, assembly, and group are, and how to lay out scopes, classes, files, and folders.

Why do we need it? Objective 7.2 asks you to *document* asset structure guidelines. Shared vocabulary lets artists, tools, and studios talk about assets the same way. You can adopt the ASWF rules as a starting point instead of inventing your own.

Beginner explanation. Think of building codes for houses. They do not design your house, but they say “every bedroom needs a window” and “stairs need a rail”. The ASWF guidelines are the building code for USD assets.

Where the analogy breaks: building codes are law; the ASWF guidelines are recommendations. Studios adapt them.

Technical explanation. Component (the basic published asset):

- `kind = "component"` on an `Xform` root prim that is the root layer's `defaultPrim`.
- Geometry behind a **payload**, so it can be deferred.

- Self-contained (shared material libraries are the usual exception).
- Inherits from at least one **class** prim.
- At least two Scopes: **geo** (visible geometry) and **mtl** (materials and shaders).
Optional: purpose scopes under **geo** (**proxy**, **render**), and scopes such as **fx** or **lgt**.
- Set **purpose** on the scopes, not on every gprim.
- Variant sets live on the **root prim** so they are settable when unloaded or instanced.
Start with one geometry and one material variant set.

Assembly:

- `kind = "assembly"`; references components or other assemblies.
- Intermediate organizing prims use `kind = "group"`.
- Usually **no payload of its own**: the components already have payloads, and nested payloads add overhead.
- Referenced components are often made **instanceable** (Chapter 24) or used as PointInstancer prototypes (Chapter 25).
- Assembly-level materials go in its own **mtl** scope; new geometry created by the assembly goes behind a payload with `component` kind.

Group: used *only* to keep the model hierarchy valid inside assemblies. **Do not put group kind on the root of a published asset.**

Files: a folder named after the asset, a root file named after the asset (`campfire.usd`), plus `payload.usd`, `geo.usd`, `mtl.usd`. The `.usd` extension is recommended for the root so it can switch between text and binary without breaking references. Extra data goes in subfolders (`maps`, `vdb`, ...).

NOTE

The ASWF examples name the class root ``_class_``. The NVIDIA whitepaper notes that names starting with a double underscore are reserved for OpenUSD's own use, so this book uses ``_class_``. Either way, the structure is the same.

Mental model.

COMPONENT		ASSEMBLY	
<code>/apple</code>	(Xform) component	<code>/appleBowl</code>	(Xform) assembly
<code>/geo</code>	(Scope)	<code>/apples</code>	(Xform) group
<code>/proxy</code>	(purpose=proxy)	<code>/apple1</code>	-> ref apple component
<code>/render</code>	(purpose=render)	<code>/apple2</code>	-> ref apple component
<code>/mtl</code>	(Scope)	<code>/bowl</code>	-> ref bowl component
<code>/_class_</code>		<code>/mtl</code>	(Scope) optional
<code>/apple</code>	(class) <- inherited	<code>/_class_/appleBowl</code>	(class)

Simple example. Checklist for documenting a structure (Objective 7.2):

Item	ASWF recommendation
Root prim	Xform, defaultPrim, kind component or assembly
Organizing prims	Scope inside components; group kind inside assemblies
Scopes	geo, mtl (plus fx, lgt, ... as needed)
Class	Root inherits one class
Payload	Components yes; assemblies usually no
Variants	On the root prim
Files	<asset>/<asset>.usd, payload.usd, geo.usd, mtl.usd

USDA example. File: `apple.usda` (ASWF-style component entry layer)

```
#usda 1.0
(
  defaultPrim = "apple"
```

```

        upAxis = "Y"
    )

    class "_class_"
    {
        class "apple"
        {
        }
    }

    def Xform "apple" (
        kind = "component"
        assetInfo = {
            string name = "apple"
        }
        prepend inherits = </_class_/apple>
        prepend payload = @./payload.usd@
    )
    {
    }
}

```

- Root is an `Xform` with `component` kind and is the `defaultPrim`.
- The root inherits its class and payloads the content.
- The `geo` and `mtl` scopes come from the payload's layers.

Python example. A small checker that documents and enforces some ASWF rules. Here it catches a published asset whose root was wrongly tagged `group`.

```

from pxr import Usd

stage = Usd.Stage.CreateInMemory()
stage.GetRootLayer().ImportFromString("""#usda 1.0
(
    defaultPrim = "AppleBowl"
)

class "_class_"
{
    class "AppleBowl"
    {
    }
}

def Xform "AppleBowl" (
    kind = "group"
)
{
    def Xform "apples"
    {
        def Xform "apple1" (
            kind = "component"
        )
        {
        }
    }
}
""")

def aswf_report(stage):
    problems = []
    root = stage.GetDefaultPrim()
    kind = Usd.ModelAPI(root).GetKind()
    if kind not in ("component", "assembly"):
        problems.append(f"root kind is '{kind}', expected component/assembly")
    if not root.GetInherits().GetAllDirectInherits():
        problems.append("root does not inherit from a class")
    for prim in stage.Traverse():
        if Usd.ModelAPI(prim).GetKind() == "component" and not prim.IsModel():
            problems.append(f"{prim.GetPath()} breaks the model hierarchy")
    if kind == "component":
        for scope in ("geo", "mtl"):

```

```

        if not root.GetChild(scope):
            problems.append(f"missing '{scope}' scope")
        return problems

for p in aswf_report(stage):
    print("-", p)

```

Expected output

```

- root kind is 'group', expected component/assembly
- root does not inherit from a class
- /AppleBowl/apples/apple1 breaks the model hierarchy

```

The fix is: tag the root assembly, tag apples as group, and add `prepend inherits = </_class_/AppleBowl>`. Chapter 30 shows how to turn checks like this into real `UsdValidation` validators.

Real-world use case. A studio writes a one-page “asset spec” for vendors based on the ASWF layout. Every delivery is run through a checker like the one above before ingestion. Because vendor assets match the in-house layout, they drop straight into assemblies and instancing works without manual fixes.

Common mistakes.

MISTAKE

Publishing an asset root with ``kind = "group"``. **Fix:** published roots are ``component`` or ``assembly``; ``group`` is only for organizing inside assemblies.

MISTAKE

Putting payloads on assemblies *and* components (nested payloads). **Fix:** payloads on components; assemblies reference them directly.

MISTAKE

Defining variant sets deep inside the asset. **Fix:** put them on the root prim so users can select them even when the asset is instanced or unloaded.

Exam traps.

TRAP

ASWF scope names are `geo`` and `mtl``. NVIDIA's whitepaper examples use ``Geometry`` and ``Materials``. The *idea* (separate geometry and material scopes) is what matters; don't pick an answer just because of the spelling.

TRAP

"Assemblies should payload their components." ASWF says assemblies usually do **not** need payloads because the components already have them.

Exam takeaways.

KEY

- Component: ``Xform`` root, ``component`` kind, ``defaultPrim``, payload, class, ``geo`` + ``mtl`` scopes. - Assembly: ``assembly`` kind, ``group`` for intermediate prims, references components, usually no payload. - Never ``group`` on a published root. Variants on the root prim. - Use ``.usd`` for the root file; folder named after the asset.

Chapter lab(s)

Lab 22 — Build a component asset and an assembly (python-labs/lab22_*.md, Obj 2.4, 7.2). You build an ASWF-style component on disk (entry layer, payload, geo/mtl USD layers, class), reference it several times into an assembly with group organizers, verify the model hierarchy with `IsModel()/IsComponent()`, and open the assembly unloaded to confirm lofted data is visible.

USDA reading exercises

Exercise 23-A. Which prims are valid models? Which are components?

```
#usda 1.0

def Xform "City" (
  kind = "assembly"
)
{
  def Xform "Block_A" (
    kind = "group"
  )
  {
    def Xform "House_1" (
      kind = "component"
    )
    {
      def Xform "Mailbox" (
        kind = "component"
      )
      {
      }
    }
  }

  def Scope "props"
  {
    def Xform "Bench" (
      kind = "component"
    )
    {
    }
  }
}
```

Exercise 23-B. An asset's entry layer contains `kind`, `assetInfo`, and `prepend payload = @./payload/contents.usd@`, while variant sets are defined only in `payload/geo.usdc`. A layout artist opens the scene with `Usd.Stage.LoadNone`. What can the artist see, what can't they see, and how would you fix it?

Answers

- **23-A.** Models: `/City`, `/City/Block_A`, `/City/Block_A/House_1`. Only `House_1` is a component. `Mailbox` is a component inside a component, so it is not a model (it should be subcomponent). `Bench` is not a model because its parent `props` has no `kind`. (Verified by composing in USD 26.08: `IsModel()` is `False` for `Mailbox`, `props`, and `Bench`.)
- **23-B.** They see the entry prim's `kind` and `assetInfo` (lofted), but no variant sets and no children, because those live behind the unloaded payload. Fix: loft the variant set names and default selections onto the entry prim in the entry layer.

Chapter review

Summary

- Scalable asset structure is **legible, modular, performant, navigable** (NVIDIA whitepaper).

- Model workstreams as layers; partition prims with `Scope`; `_` = internal, `__` = reserved.
- Recommended start: **text entry layer** → **payload** → **binary sublayers**; users reference, the asset payloads (reference-payload pattern).
- **Loft** kind, `assetInfo`, inherits, variant set names, and `extentsHint` above the payload.
- Kinds: `assembly` is-a `group` is-a `model`; `component` is-a `model`; `subcomponent` is not a `model`.
- A prim is a `model` only if every ancestor is a `group/assembly`; the hierarchy stops at components.
- `Usd.PrimIsModel` traversal prunes to the `model` hierarchy.
- Components are self-contained; relationship targets outside the asset root are ignored after referencing.
- ASWF: `geo/mtl` scopes, class per asset, payload on components, `group` only inside assemblies, never on published roots.

If you see... → think...

If you see...	Think...
"component not showing in model traversal"	An ancestor lacks <code>group/assembly</code> kind
<code>kind = "component"</code> under another component	Should be <code>subcomponent</code>
"see variants while unloaded"	Loft variant sets onto the entry prim
"which kind for a published root?"	<code>component</code> or <code>assembly</code> , never <code>group</code>
"binding target dropped after reference"	Target was outside the referenced prim
"fix everywhere vs this scene only"	Source asset vs class in the scene layer
"entry layer format"	Small text layer; heavy data in USDC; <code>.usd</code> extension
"legible / modular / performant / navigable"	NVIDIA's four principles

Review questions

R23-1 (Obj 2.4) Which kind should an intermediate organizing prim between an assembly and its components have? A. `component` B. `subcomponent` C. `group` D. `model`

R23-2 (Obj 2.4) Given `/Set` (`assembly`) → `/Set/Table` (`component`) → `/Set/Table/Drawer` (`component`), which is true? A. Both `Table` and `Drawer` are components. B. `Table` is a component; `Drawer` is not a `model`. C. Neither is a `model` because `Set` is an `assembly`. D. `Drawer` is a component; `Table` becomes a `group`.

R23-3 (Obj 7.2) Which describes NVIDIA's suggested starting asset structure? A. A binary entry layer that sublayers text geometry layers B. A referenceable text entry layer containing a payload to binary sublayers C. A single USDZ file per asset D. One layer per prim, combined with references

R23-4 (Obj 7.2) Select two items that should be lofted into an asset's entry layer. A. `Mesh points` B. Variant set names and default selections C. Texture files D. `kind` and `assetInfo`

R23-5 (Obj 2.4) What does this print?

```
print(Usd.ModelAPI(stage.GetPrimAtPath("/Shelf/Lid")).GetKind(),
      stage.GetPrimAtPath("/Shelf/Lid").IsModel())
# /Shelf is kind "component", /Shelf/Lid is kind "subcomponent"
```

A. `subcomponent` True B. `subcomponent` False C. `component` False D. False

R23-6 (Obj 7.2) According to the ASWF guidelines, which statement is correct? A. Assemblies should always add their own payload around referenced components. B. Published asset roots should use the `group` kind. C. A component should have at least `geo` and `mtl` scopes. D. Purpose should be set on every individual mesh.

R23-7 (Obj 2.4) A tool must list all component models in a huge stage as fast as possible. Best approach? A. `stage.Traverse()` and check `GetKind() == "component"` B. `stage.Traverse(Usd.PrimIsModel)` and keep prims where `IsComponent()` is true C. `stage.Traverse(Usd.TraverseInstanceProxies())` D. Search prim names for “component”

R23-8 (Obj 7.2) Why is `.usd` recommended for the entry layer’s extension? A. `.usd` files load faster than `.usdc`. B. It lets the file switch between text and binary without changing references to it. C. Only `.usd` files may contain payloads. D. `.usd` is required for `defaultPrim`.

R23-9 (Obj 2.4) Select two true statements about `Usd.ModelAPI(prim).IsKind("component")`. A. By default it also checks the model hierarchy. B. It returns `True` for assembly prims. C. With `Usd.ModelAPI.KindValidationNone` it checks only the kind token. D. It changes the prim’s kind.

R23-10 (Obj 7.2) Which principle is served by naming a prim `LargeCardboardBox` instead of `Mesh_017`? A. Performance B. Modularity C. Legibility D. Navigability through the model hierarchy

R23-11 (Obj 2.4) A component’s mesh binds a material at `/Looks/Steel`, outside the component’s root. After the component is referenced into an assembly, what happens? A. The binding is remapped to `/Assembly/Looks/Steel`. B. The binding target is ignored, with a warning. C. The stage fails to open. D. The material is copied into the assembly.

R23-12 (Obj 7.2) Where should variant sets live, according to ASWF, and why? A. On the `geo` scope, so they are near the geometry B. On the root prim, so they can be set even when the asset is instanced or unloaded C. In a separate variant layer referenced by every mesh D. In the class prim only

Review answers

- **R23-1: C.** `group` keeps the chain valid. `model` is abstract and should not be authored. Review: §23.3.
- **R23-2: B.** The hierarchy stops at `Table`; a component inside a component is not a model. Review: §23.3.
- **R23-3: B.** Text interface layer → payload → binary sublayers. Review: §23.2.
- **R23-4: B and D.** Cheap, important fields are lofted; heavy data (points, textures) stays behind the payload. Review: §23.2.
- **R23-5: B.** `GetKind` returns the token; subcomponent is not a model kind. Review: §23.3.
- **R23-6: C.** A and B are the opposite of ASWF advice; D: set purpose on scopes. Review: §23.5.
- **R23-7: B.** The `PrimIsModel` predicate prunes below components; A visits every prim and ignores the hierarchy rule. Review: §23.3.
- **R23-8: B.** The extension does not fix the encoding; references keep working after a format switch. Review: §23.2.
- **R23-9: A and C.** It is a query only; assemblies are not components. Review: §23.3.
- **R23-10: C.** Clear names are legibility. Review: §23.1.
- **R23-11: B.** Out-of-scope targets are dropped (“refers to a path outside the scope of the reference”). Review: §23.4.
- **R23-12: B.** Root-level variant sets remain accessible on instance roots and unloaded prims. Review: §23.5.

CHAPTER 24

Chapter 24 — Native Scenegraph Instancing

Exam domain: Content Aggregation (10%) · **Objectives:** 1.10, 2.2, 2.4, 2.5 · **Study day:** 8 · **Est. time:** 120 min **Prerequisites:** Ch 16 (references), Ch 18 (variants), Ch 19 (inherits), Ch 20 (overs), Ch 22.4 (instanced components)

Native instancing is OpenUSD’s “rubber stamp”: many prims share one prototype in memory. The exam asks you to turn it on, find the prototype, explain instance proxies, and change one copy’s look **without** authoring on a proxy (Obj 1.10, 2.5).

Learning goals

- Mark referencing prims `instanceable` and confirm they become instances.
- Find the prototype with `GetPrototype` / `GetPrototypes` (never `GetMaster`).
- Describe instance proxies and why default `Traverse()` skips them.
- Edit a shared look at the source, or uninstance one copy, without writing on a proxy.
- Vary copies with inherits, variants, and primvars on the **instance root**.
- Remove or hide parts of an instanced component using a class or variant, not a nested `over`.

Key terms

Term	One-line definition
Native instancing	Sharing one prototype prim tree among many instanceable prims
instanceable	Metadata on a prim: “I am willing to be an instance”
Instance	A prim for which <code>IsInstance()</code> is True; it has a prototype
Instance root	That prim itself (you may author on it)
Prototype	The hidden, shared prim tree (<code>/__Prototype_N</code>); not saved in the layer
Instance proxy	A descendant you see under an instance; it stands for a prototype prim
GetMaster	Removed name. Use <code>GetPrototype</code> (USD 21.x+)
De-instance	<code>SetInstanceable(False)</code> so that copy is unique again

24.1 instanceable

What is it? `instanceable` is prim metadata. When it is true, and the prim’s composed descendants match other instanceable prims, USD may make those prims **instances** of one shared prototype.

Why do we need it? A stadium of 5 000 identical seats should not compose 5 000 copies of the mesh in memory. Instancing keeps one seat in the prototype and stamps it everywhere. Payloads help load time; instancing helps **memory** and **compose time** once the asset is loaded (Chapter 26).

Beginner explanation. Think of a **rubber stamp**. You carve the chair once (`/Chair`). Each `instanceable` reference is an ink stamp on the page. The ink is shared. Changing the carving changes every stamp. Writing on one *ink print* with a pen is not allowed — that is authoring on a proxy (24.3).

Where the analogy breaks: stamps that select a different variant, or inherit a different class, are carved differently: USD makes **another** prototype (24.5). The source `/Chair` remains in the scene; it is not the prototype path.

Technical explanation. - Author `instanceable = true` in USDA, or `prim.SetInstanceable(True)` in Python. `IsInstanceable()` is the authored/composed flag. `IsInstance()` is True only when USD actually built a prototype for that prim. - Typical pattern: a typeless def that **references** an asset (or internally references a prototype prim) and is instanceable. The referenced subtree is what gets shared. - Matching instances share a prototype when their composition (references, variant selections, inherits, ...) is the same. Different selections → different prototypes. - Default `Traverse()` visits the instance **root** but **not** its proxy descendants. `GetPrimAtPath("/Room/A/Seat")` still works. - Prototypes are **not** written to the layer. Saving the stage does not create `/__Prototype_1` specs.

VERSION

Verified on USD 26.08. Native instancing "master" prims were renamed "prototype" and ``GetMaster`/`GetMasters`` were deprecated in USD 20.11; the deprecated master API was **removed** in 21.08. Use ``GetPrototype()``.

Mental model.

```
/Chair      (source prim; ordinary, editable)
  Seat

/Room/A  instanceable=true, references </Chair>   ---\
/Room/B  instanceable=true, references </Chair>   ---+---> prototype /__Prototype_1
                                                Seat (shared)
```

Simple example. Two instanceable internal references to `/Chair`. Both `IsInstance()` True. `Traverse()` lists `/Chair`, `/Chair/Seat`, `/Room`, `/Room/A`, `/Room/B` — not `/Room/A/Seat`.

USDA example.

```
#usda 1.0

def Xform "Chair"
{
    def Cube "Seat"
    {
    }
}

def Xform "Room"
{
    def "A" (
        instanceable = true
        prepend references = </Chair>
    )
    {
    }

    def "B" (
        instanceable = true
        prepend references = </Chair>
    )
    {
    }
}
```

Line notes: `instanceable = true` is prim metadata next to the reference. The source `/Chair` stays a normal prim. A and B are the instance roots.

Python example.

```
from pxr import Sdf, Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
UsdGeom.Xform.Define(stage, "/Chair")
UsdGeom.Cube.Define(stage, "/Chair/Seat")
a = stage.DefinePrim("/Room/A")
a.GetReferences().AddInternalReference(Sdf.Path("/Chair"))
```

```

a.SetInstanceable(True)
b = stage.DefinePrim("/Room/B")
b.GetReferences().AddInternalReference(Sdf.Path("/Chair"))
b.SetInstanceable(True)
print("A IsInstanceable/IsInstance:", a.IsInstanceable(), a.IsInstance())
print("B IsInstanceable/IsInstance:", b.IsInstanceable(), b.IsInstance())
print("Traverse:", [str(p.GetPath()) for p in stage.Traverse()])
print(stage.GetRootLayer().ExportToString())

```

Expected output

```

A IsInstanceable/IsInstance: True True
B IsInstanceable/IsInstance: True True
Traverse: ['/Chair', '/Chair/Seat', '/Room', '/Room/A', '/Room/B']
#usda 1.0

def Xform "Chair"
{
    def Cube "Seat"
    {
    }
}

def "Room"
{
    def "A" (
        instanceable = true
        prepend references = </Chair>
    )
    {
    }

    def "B" (
        instanceable = true
        prepend references = </Chair>
    )
    {
    }
}

```

Real-world use case. A city block references `streetlamp.usda` two hundred times with `instanceable = true`. Memory holds one lamp prototype. Layout still sees two hundred roots to place.

Common mistakes.

MISTAKE

Setting `instanceable`` on the `**source**`/Chair`` only. The `*referencing*` prims must be `instanceable`.

MISTAKE

Expecting `Traverse()`` to list `/Room/A/Seat``. Use `GetPrimAtPath`` or `Usd.TraverseInstanceProxies()``.

Exam traps.

TRAP

"`instanceable`` copies the mesh into the referencing layer." It shares a prototype; the layer only stores the flag and the reference.

TRAP

"`IsInstanceable`` and `IsInstance`` are the same." The flag can be true on a prim that is not currently an instance (for example it has no sharable composition yet).

Exam takeaways.

KEY

- ``instanceable = true`` on the referencing prim. - ``IsInstance()`` means a prototype exists. - ``Traverse()`` skips proxy children; the source prim remains editable. - Prototypes are not saved into the layer.

24.2 Prototypes (GetPrototype)

What is it? A **prototype** is the shared prim tree USD builds for a set of matching instances. You reach it with `prim.GetPrototype()` or `stage.GetPrototypes()`. Its path looks like `/__Prototype_1`.

Why do we need it? Debugging “why do these two chairs differ?” starts with: do they share a prototype? If not, their composition differs (variant, inherit, extra local child).

Beginner explanation. The prototype is the **carving of the rubber stamp**, stored in a drawer labeled `/__Prototype_1`. You do not ship the drawer as a file. `GetInstances()` on the prototype lists every stamp that uses that carving.

Where the analogy breaks: the carving is composed from the source asset plus the instance roots’ matching arcs. It is not a literal copy of `/Chair`’s path.

Technical explanation. - `instance.GetPrototype()` returns the prototype prim, or an invalid prim if `IsInstance()` is False. - `stage.GetPrototypes()` lists all prototypes on the stage. - `prototype.IsPrototype()` is True. `Usd.Prim.IsPrototypePath(path)` is True for `/__Prototype_1`. - `prototype.GetInstances()` returns the instance roots that share it. - `GetMaster` does **not** exist on USD 26.08. - Editing a prototype prim directly is not how pipelines work; edit the **source** (`/Chair`) or the instance **root**. Prototype paths are runtime.

Mental model.

<code>GetPrototype()</code>	<code>GetInstances()</code>
<code>/Room/A -----></code>	<code>/__Prototype_1 -----> /Room/A, /Room/B</code>
<code>/Room/B -----></code>	

Simple example. A and B from 24.1 share `/__Prototype_1`. `A.GetPrototype() == B.GetPrototype()`.

USDA example. The USDA is the same as Section 24.1. Prototypes never appear in the saved file. The next Python block prints the runtime paths.

Python example.

```
from pxr import Sdf, Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
UsdGeom.Xform.Define(stage, "/Chair")
UsdGeom.Cube.Define(stage, "/Chair/Seat")
for name in ("A", "B"):
    p = stage.DefinePrim("/Room/" + name)
    p.GetReferences().AddInternalReference(Sdf.Path("/Chair"))
    p.SetInstanceable(True)
a = stage.GetPrimAtPath("/Room/A")
b = stage.GetPrimAtPath("/Room/B")
proto = a.GetPrototype()
print("prototype path:", proto.GetPath())
print("IsPrototype:", proto.IsPrototype())
print("GetPrototypes:", [str(p.GetPath()) for p in stage.GetPrototypes()])
print("instances of proto:", [str(p.GetPath()) for p in proto.GetInstances()])
print("A.GetPrototype() is B.GetPrototype():",
      a.GetPrototype() == b.GetPrototype())
print("hasattr GetMaster:", hasattr(a, "GetMaster"))
```

Expected output

```
prototype path: /__Prototype_1
IsPrototype: True
```

```
GetPrototypes: ['/_Prototype_1']
instances of proto: ['/Room/A', '/Room/B']
A.GetPrototype() is B.GetPrototype(): True
hasattr GetMaster: False
```

Real-world use case. A tooling script groups instanceable furniture by `GetPrototype()` to count unique composed variants in a shot: “12 lamp prototypes, 4 800 instances.”

Common mistakes.

MISTAKE

Searching the USDA for ``/_Prototype_1``. It will not be there. Use Python or `usdview`'s instance UI.

MISTAKE

Calling ``GetMaster()``. Use ``GetPrototype()``.

Exam traps.

TRAP

"The prototype is ``/Chair``." ``/Chair`` is the source. The prototype is a runtime prim under ``/_Prototype_N``.

TRAP

"Two instanceable references always share one prototype." Different variant selections split them (24.5).

Exam takeaways.

KEY

- ``GetPrototype()``, ``GetPrototypes()``, ``GetInstances()``. - Prototype paths are runtime, not authored. - Matching composition → shared prototype. - Never write ``GetMaster`` in new code.

24.3 Instance proxies

What is it? An **instance proxy** is a prim at a path *under* an instance root, for example `/Room/A/Seat`. It is a window onto the corresponding prototype prim. `IsInstanceProxy()` is `True`. You can read it; you cannot author on it.

Why do we need it? Tools still need to pick “the seat of chair A” for bounding boxes, selection, and queries. Proxies give a path in instance namespace without duplicating data. Obj 2.5 is about editing **without** treating that path like a normal prim.

Beginner explanation. A proxy is a **photograph of the stamp’s seat**. You can look at it. You cannot scribble on the photograph and expect only that stamp to change. Scribbling is rejected: “authoring to an instance proxy is not allowed.”

Where the analogy breaks: `GetPrimInPrototype()` tells you which prototype prim the photograph shows. Changing the carving (`/Chair/Seat`) changes every photograph.

Technical explanation. - `GetPrimAtPath("/Room/A/Seat")` is valid. `IsInstanceProxy()` `True`. `GetPrimInPrototype()` → `/_Prototype_1/Seat`. - Default `Traverse()` and `TraverseAll()` skip proxies. `stage.Traverse(Usd.TraverseInstanceProxies())` includes them. - `Set`, `OverridePrim`, `CreateAttribute`, `Clear` on a proxy raise `Tf.ErrorException` (USD 26.08: `_ValidateEditPrim`). - The instance **root** (`/Room/A`) is not a proxy. You may author metadata, variants, inherits, and primvars there (24.5). - Reading attributes on a proxy returns the composed prototype values.

Mental model.

<code>/Room/A</code>	instance root	-- author here OK
<code>/Seat</code>	instance proxy	-- read OK, write FORBIDDEN


```

      |
      v
    /__Prototype_1/Seat

```

Simple example. `/Room/A/Seat.size` can be read. Setting it raises. Traversal with `TraverseInstanceProxies` lists the Seat paths.

USDA example. Same setup as 24.1. Proxies are not extra USDA; they exist only on the composed stage.

Python example.

```

from pxr import Sdf, Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
UsdGeom.Xform.Define(stage, "/Chair")
UsdGeom.Cube.Define(stage, "/Chair/Seat")
for name in ("A", "B"):
    p = stage.DefinePrim("/Room/" + name)
    p.GetReferences().AddInternalReference(Sdf.Path("/Chair"))
    p.SetInstanceable(True)
seat = stage.GetPrimAtPath("/Room/A/Seat")
print("Seat IsInstanceProxy:", seat.IsInstanceProxy())
print("Seat GetPrimInPrototype:", seat.GetPrimInPrototype().GetPath())
print("TraverseInstanceProxies:",
      [str(p.GetPath())
       for p in stage.Traverse(Usd.TraverseInstanceProxies())])
try:
    seat.GetAttribute("size").Set(9)
except Exception as err:
    print("set on proxy:", type(err).__name__)

```

Expected output

```

Seat IsInstanceProxy: True
Seat GetPrimInPrototype: /__Prototype_1/Seat
TraverseInstanceProxies: ['/Chair', '/Chair/Seat', '/Room', '/Room/A', '/Room/A/Seat', '/Room/B',
'/Room/B/Seat']
set on proxy: ErrorException

```

Real-world use case. A viewer highlights `/Room/A/Seat` when the artist clicks that mesh. The pick path is a proxy. The exporter must not try to bake a unique `displayColor` onto that path; it writes a primvar on `/Room/A` instead.

Common mistakes.

MISTAKE

Hand-writing ``over "Seat"`` under an instanceable prim in USDA. It may parse, but composition ignores it (Chapter 22.4). Prefer a variant or class.

MISTAKE

Using ``TraverseAll()`` to "see everything including proxies." On 26.08 ``TraverseAll()`` still skipped proxies in the verified example. Use ``TraverseInstanceProxies()``.

Exam traps.

TRAP

"Proxies are invalid prims." They are valid. They are not editable.

TRAP

"The instance root is a proxy." The root is the instance; its descendants are proxies.

Exam takeaways.

KEY

- Proxies are readable paths under an instance; ``IsInstanceProxy()`` is True. - Writes to proxies fail. - ``TraverseInstanceProxies()`` is the walker that includes them. - ``GetPrimInPrototype()`` maps a proxy to the shared prim.

24.4 Editing look without breaking instancing

What is it? You still need to change how instances look. The legal places are: the **source** prim (all copies), the **instance root** (that copy's arcs and primvars), or **de-instancing** that copy with `SetInstanceable(False)`.

Why do we need it? Obj 2.5: vary a copy without destroying sharing for everyone else. Artists who “just over the seat” break the rule in 24.3.

Beginner explanation. Three legal pens:

1. Sharpen the **carving** (`/Chair/Seat`) — every stamp changes.
2. Write on the **stamp handle** (`/Room/B`) — variants, inherits, primvars.
3. Throw away the stamp for one chair (`SetInstanceable(False)`) and draw freely on that copy.

Where the analogy breaks: option 2 may create a **second prototype** (a second carving) for that handle's unique combination. Other chairs keep the first prototype. Sharing is not “broken”; it is split.

Technical explanation. | Edit | Where | Instances | Sharing | |-----|-----|-----|-----| |
Change source `/Chair/Seat.size` | Source prim | All that use that source | Kept | |
Primvar / inherit / variant on `/Room/B` | Instance root | That copy (and any with the same
combo) | Split if combo differs | | `SetInstanceable(False)` then edit `/Room/B/Seat` | No
longer an instance | That copy only | That copy leaves the prototype | | over "Seat" under
`/Room/B` while instanceable | Proxy | Rejected or ignored | Do not do this |

De-instancing is the escape hatch when a copy must be truly unique (a hero smashed chair). It costs memory for that copy.

Mental model.

shared edit:	<code>/Chair/Seat.size = 3</code>	-> A and B both 3
unique edit:	<code>B.instanceable = false</code> <code>/Room/B/Seat.size = 9</code>	-> A stays 3, B is 9

Simple example. Two instances, source size 1. Set source to 3: both 3. De-instance B and set B's seat to 9: A stays 3, B is 9, `B.IsInstance()` is False.

USDA example. After de-instancing B in Python, the layer stores `instanceable = false` (or omits the flag) and a nested over "Seat" that is now legal because B is not an instance.

```
#usda 1.0

def Xform "Chair"
{
  def Cube "Seat"
  {
    double size = 3
  }
}

def "Room"
{
  def "A" (
    instanceable = true
    prepend references = </Chair>
  )
  {
```

```

    }

    def "B" (
        prepend references = </Chair>
    )
    {
        over "Seat"
        {
            double size = 9
        }
    }
}

```

Python example.

```

from pxr import Sdf, Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
UsdGeom.Xform.Define(stage, "/Chair")
UsdGeom.Cube.Define(stage, "/Chair/Seat").GetSizeAttr().Set(1.0)
for name in ("A", "B"):
    p = stage.DefinePrim("/Room/" + name)
    p.GetReferences().AddInternalReference(Sdf.Path("/Chair"))
    p.SetInstanceable(True)
print("shared size A,B:",
      stage.GetPrimAtPath("/Room/A/Seat").GetAttribute("size").Get(),
      stage.GetPrimAtPath("/Room/B/Seat").GetAttribute("size").Get())
stage.GetPrimAtPath("/Chair/Seat").GetAttribute("size").Set(3.0)
print("after source edit A,B:",
      stage.GetPrimAtPath("/Room/A/Seat").GetAttribute("size").Get(),
      stage.GetPrimAtPath("/Room/B/Seat").GetAttribute("size").Get())
stage.GetPrimAtPath("/Room/B").SetInstanceable(False)
stage.GetPrimAtPath("/Room/B/Seat").GetAttribute("size").Set(9.0)
print("after B unique:",
      stage.GetPrimAtPath("/Room/A/Seat").GetAttribute("size").Get(),
      stage.GetPrimAtPath("/Room/B/Seat").GetAttribute("size").Get(),
      "B IsInstance", stage.GetPrimAtPath("/Room/B").IsInstance())

```

Expected output

```

shared size A,B: 1.0 1.0
after source edit A,B: 3.0 3.0
after B unique: 3.0 9.0 B IsInstance False

```

Real-world use case. A set dresser instances 40 chairs. Chair 12 is on fire in this shot. They `SetInstanceable(False)` on that one copy and swap in a burned mesh. The other 39 keep sharing.

Common mistakes.

MISTAKE

De-instancing every copy "so we can edit." You throw away the memory win. Prefer root primvars and variants.

MISTAKE

Editing ``/_Prototype_1/Seat`` in a tool that shows it. Edit ``/Chair/Seat`` instead.

Exam traps.

TRAP

"De-instancing A also de-instances B." Only the prim you call ``SetInstanceable(False)`` on leaves.

TRAP

"Source edits do not affect instances because instances are copies." Native instances are not copies; they share the prototype fed by that source.

Exam takeaways.

KEY

- Shared look: edit the source (or a shared class). - Per-copy look: instance-root arcs and primvars, or de-instance. - De-instance is the unique-hero escape hatch. - Never author on a proxy.

24.5 Overriding instances: inherits, variants, primvars on instance roots

What is it? The instance **root** is a normal prim for composition arcs. You may set variant selections, add inherits, and author primvars there. Those opinions stay on the root; descendants remain proxies.

Why do we need it? This is the intended way to make chair A “bare” and chair B “full,” or to tint one row of seats red, without de-instancing.

Beginner explanation. The stamp **handle** can have a switch (variant), a style tag (inherit), and a color sticker (primvar). Different handle settings may mean a different carving (a second prototype). Stickers on the handle inherit down to the stamped mesh at render time (Chapter 11).

Where the analogy breaks: two roots with the same variant and inherits still share one prototype. Only a **different** combo splits.

Technical explanation. Verified on USD 26.08:

- `GetVariantSet("trim").SetVariantSelection("bare")` on an instance root keeps `IsInstance()` `True`. A and B then have **two** prototypes if B did not select `bare`.
- `GetInherits().AddInherit(/_Look)` on the root is legal. Implied inherits (Chapter 19) still apply.
- Primvars authored on the root (`primvars:displayColor`) keep instancing. `UsdGeom.PrimvarsAPI(proxy).FindPrimvarsWithInheritance()` sees them on descendants. `GetAttribute("primvars:displayColor")` on the proxy itself may be empty; inheritance is a primvar lookup, not a copied attribute spec.
- Authoring a nested `over` child still targets a proxy and fails or is ignored.

Mental model.

```
/Room/A (instance root)
  variants.trim = "bare"      -> prototype 2 (cushion off)
  primvars:displayColor      -> inherited onto proxies
  inherits = </_Look>         -> may split prototype if B lacks it
/Room/A/Seat (proxy)         -> still not editable
```

Simple example. Chair asset with `trim` variants `full` and `bare` (`bare` deactivates `Cushion`). Instance A selects `bare`, B does not. Two prototypes. A’s cushion is inactive; B’s is active. Both remain instances.

USDA example. File: *chair.usda* (trim variants)

```
#usda 1.0
(
  defaultPrim = "Chair"
)

def Xform "Chair" (
  prepend variantSets = "trim"
)
{
  def Cube "Seat"
  {
    double size = 1
  }
  def Cube "Cushion"
  {
```

```

        double size = 0.2
    }
    variantSet "trim" = {
        "full" {
        }
        "bare" {
            over "Cushion" (
                active = false
            )
            {
            }
        }
    }
}

```

File: room.usda (A selects bare)

```

#usda 1.0

def "Room"
{
    def "A" (
        instanceable = true
        prepend references = @./chair.usda@
        variants = {
            string trim = "bare"
        }
    )
    {
    }

    def "B" (
        instanceable = true
        prepend references = @./chair.usda@
    )
    {
    }
}

```

Python example.

```

from pxr import Sdf, Usd, UsdGeom

with open("chair.usda", "w") as f:
    f.write("""#usda 1.0
(
    defaultPrim = "Chair"
)
def Xform "Chair" (
    prepend variantSets = "trim"
)
{
    def Cube "Seat"
    {
        double size = 1
    }
    def Cube "Cushion"
    {
        double size = 0.2
    }
    variantSet "trim" = {
        "full" {
        }
        "bare" {
            over "Cushion" (
                active = false
            )
            {
            }
        }
    }
}
""")

```

```

stage = Usd.Stage.CreateNew("room.usda")
for name in ("A", "B"):
    p = stage.DefinePrim("/Room/" + name)
    p.GetReferences().AddReference("./chair.usda")
    p.SetInstanceable(True)
stage.GetPrimAtPath("/Room/A").GetVariantSet("trim").SetVariantSelection(
    "bare")
print("A IsInstance:", stage.GetPrimAtPath("/Room/A").IsInstance())
print("same prototype:",
      stage.GetPrimAtPath("/Room/A").GetPrototype() ==
      stage.GetPrimAtPath("/Room/B").GetPrototype())
print("A Cushion active:",
      stage.GetPrimAtPath("/Room/A/Cushion").IsActive())
print("B Cushion active:",
      stage.GetPrimAtPath("/Room/B/Cushion").IsActive())
print("num prototypes:", len(stage.GetPrototypes()))

st2 = Usd.Stage.CreateInMemory()
UsdGeom.Xform.Define(st2, "/Chair")
UsdGeom.Cube.Define(st2, "/Chair/Seat")
root = st2.DefinePrim("/Room/A")
root.GetReferences().AddInternalReference(Sdf.Path("/Chair"))
root.SetInstanceable(True)
root.CreateAttribute(
    "primvars:displayColor",
    Sdf.ValueTypeNames.Color3fArray).Set([(0.2, 0.3, 0.4)])
print("still instance:", root.IsInstance())
api = UsdGeom.PrimvarsAPI(st2.GetPrimAtPath("/Room/A/Seat"))
pv = [x for x in api.FindPrimvarsWithInheritance()
      if x.GetPrimvarName() == "displayColor"][0]
c = pv.Get()[0]
print("inherited color:", (round(c[0], 1), round(c[1], 1), round(c[2], 1)))

```

Expected output

```

A IsInstance: True
same prototype: False
A Cushion active: False
B Cushion active: True
num prototypes: 2
still instance: True
inherited color: (0.2, 0.3, 0.4)

```

Real-world use case. A stadium tints home-side seats with `primvars:displayColor` on each instance root and uses a `wear` variant (new / worn) on rows near the tunnels. Most seats still share a handful of prototypes, not 50 000 unique meshes.

Common mistakes.

MISTAKE

Authoring ``primvars:displayColor`` on the proxy mesh. Put it on the instance root and let it inherit.

MISTAKE

Expecting two different variant selections to keep one prototype. Different composition → different prototype. That is still instancing; each group shares internally.

Exam traps.

TRAP

"A variant selection on the root de-instances the prim." It stays an instance; it may join another prototype.

TRAP

"Inherited primvars must be queried with ``GetAttribute`` on the child." Use ``PrimvarsAPI.FindPrimvarsWithInheritance``.

Exam takeaways.

KEY

- Instance-root variants, inherits, and primvars are the supported per-copy knobs. - Different combos split prototypes; that is still instancing. - Primvars inherit to descendants; query with `PrimvarsAPI`. - Nested `over` on a proxy is not a supported knob.

24.6 Removing properties from instanced component prims

What is it? Obj 1.10 / 2.5: hide or strip a property or child on *some* instanced copies (no cushion, no extra bolt) without editing proxies and without exploding instancing for everyone.

Why do we need it? Assemblies instance components, then a shot needs one component without a part. `Clear()` on the proxy fails. A local `over "Cushion" (active = false)` under the instance is the trap Chapter 22.4 already named.

Beginner explanation. You cannot erase ink from one stamp print. You either switch that handle to a “bare” setting (variant / inherited class that deactivates the part) or stop using a stamp for that chair (de-instance) and then deactivate.

Where the analogy breaks: the class or variant is authored **once** and reused. You are not hand-deactivating 400 proxies.

Technical explanation. Legal patterns (verified on 26.08):

1. **Variant on the asset**, selected on the instance root — Section 24.5 `trim=bare`.
2. **Inherited class** with `over "Cushion" (active = false)`. The instance root `prepend inherits = </_HideCushion>`. That copy stays an instance; it gets its own prototype if others do not inherit the class.
3. **De-instance**, then `SetActive(False)` on the now-real child.
4. **Do not instance** that particular copy from the start.

Illegal / ignored: `Clear()` on a proxy attribute; nested `over` under an instanceable prim.

`active = false` hides the child from default traversal; the spec can remain in the asset. That is usually what “remove this part” means in USD. Deleting from the published asset would remove it for every instance.

Mental model.

WANT: chair A with no Cushion, chairs B...Z unchanged and still instanced

DO: `A.inherits = /_HideCushion` or `A.variants.trim = "bare"`
 DON'T: `over "A" { over "Cushion" (active = false) }` while A is instanceable
 DON'T: `GetPrimAtPath("/A/Cushion").Clear()` or `SetActive` on the proxy

Simple example. `_HideCushion` class deactivates `Cushion`. Instance A inherits it; B does not. A's cushion is inactive; B's is not. Two prototypes; both instanceable.

USDA example.

```
#usda 1.0

class "_HideCushion"
{
    over "Cushion" (
        active = false
    )
    {
    }
}

def Xform "Chair"
{
    def Cube "Seat"
    {
    }
}
```

```

    def Cube "Cushion"
    {
    }
}

def "A" (
    instanceable = true
    prepend references = </Chair>
    prepend inherits = </_HideCushion>
)
{
}

def "B" (
    instanceable = true
    prepend references = </Chair>
)
{
}

```

Python example.

```

from pxr import Usd

with open("hide.usda", "w") as f:
    f.write("""#usda 1.0
class "_HideCushion"
{
    over "Cushion" (
        active = false
    )
    {
    }
}
def Xform "Chair"
{
    def Cube "Seat"
    {
    }
    def Cube "Cushion"
    {
    }
}
def "A" (
    instanceable = true
    prepend references = </Chair>
    prepend inherits = </_HideCushion>
)
{
}
def "B" (
    instanceable = true
    prepend references = </Chair>
)
{
}
""")

stage = Usd.Stage.Open("hide.usda")
print("A IsInstance:", stage.GetPrimAtPath("/A").IsInstance())
print("A Cushion active:", stage.GetPrimAtPath("/A/Cushion").IsActive())
print("B Cushion active:", stage.GetPrimAtPath("/B/Cushion").IsActive())
print("same prototype:",
      stage.GetPrimAtPath("/A").GetPrototype() ==
      stage.GetPrimAtPath("/B").GetPrototype())
try:
    stage.GetPrimAtPath("/B/Seat").GetAttribute("size").Clear()
except Exception as err:
    print("Clear on proxy:", type(err).__name__)

```

Expected output

```

A IsInstance: True
A Cushion active: False

```



```
B Cushion active: True
same prototype: False
Clear on proxy: RuntimeException
```

Real-world use case. A digital-twin robot is instanced at twenty stations. Station 7 has no camera. The cell layer inherits `_NoCamera` on that one instance root, which deactivates `OptionalCamera`. The other nineteen keep the camera prototype.

Common mistakes.

MISTAKE

``over "Cushion" (active = false)`` nested under the instanceable prim in the assembly. Composition ignores it. Put the ``over`` in a class or variant the root `**selects**`.

MISTAKE

Editing the published component to delete Cushion. Every instance loses it.

Exam traps.

TRAP

"Splitting the prototype means instancing failed." A still has a prototype; it just is not B's.

TRAP

Obj 1.10 is not "call Clear on the proxy." It is class / variant / de-instance.

Exam takeaways.

KEY

- Remove/hide instanced parts with a root variant or inherited class, or de-instance. - Proxy ``Clear`` / nested ``over`` is the wrong answer to Obj 1.10. - A second prototype for that combo is expected and fine. - Do not delete the part from the published asset unless every copy should lose it.

Chapter lab(s)

Lab 23 (native instancing and instance proxies) belongs with this chapter. Chapter 22.4 is the companion "don't nest overs under instances" drill.

USDA reading exercises

Exercise 24-A. Using the USDA in 24.1, is `/Room/A/Seat` on the stage? Does default `Traverse()` visit it? Can you set its `size`?

Exercise 24-B. Using 24.5's `room.usda`, how many prototypes are there, and is A's cushion active?

Answers

24-A. Yes, it is a valid instance proxy. Default `Traverse()` does not visit it. Setting `size` raises `RuntimeException`. Review: §24.1, §24.3.

24-B. Two prototypes. A's cushion is inactive (`trim=bare`). Review: §24.5.

Chapter review

Summary

- Native instancing: `instanceable = true` on referencing prims; they share a runtime prototype.
- `GetPrototype` / `GetPrototypes` / `GetInstances`. `GetMaster` is gone.
- Instance roots are editable; descendant proxies are not. `Traverse()` skips proxies.
- Shared edits go on the source. Unique heroes `SetInstanceable(False)`.

- Per-copy look: variants, inherits, primvars on the root (may split prototypes).
- Obj 1.10: hide parts with a class or variant on the root, not a proxy over.

If you see... → think...

If you see...	Think...
instanceable = true	Native instancing on that referencing prim
/__Prototype_1	Runtime prototype, not in the USDA
GetMaster	Old name; answer is GetPrototype
IsInstanceProxy	Read-only descendant; do not Set
Traverse() misses /A/Seat	Need TraverseInstanceProxies()
Two instanceable chairs, two prototypes	Different variant/inherit combo
Nested over "Seat" under instance	Wrong; ignored or error (Obj 1.10)
Tint one row of seats	Primvar on each instance root
One smashed chair	SetInstanceable(False)

Review questions

R24-01 · Obj 2.2 · Single choice `IsInstance()` is True when: A. `instanceable` is authored, even with no reference B. USD has built a prototype for that prim C. The prim is a class D. The prim is a payload

R24-02 · Obj 2.2 · Single choice Prototypes are stored: A. As `/__Prototype_N` specs in the root layer B. Only at runtime on the stage C. In `customData` D. As payloads

R24-03 · Obj 2.5 · Single choice Authoring to `/Room/A/Seat` while A is an instance: A. Edits only A B. Edits the asset file C. Is not allowed D. Creates a specialize arc

R24-04 · Obj 2.5 · Single choice `stage.Traverse(Usd.TraverseInstanceProxies())` is for: A. Opening payloads B. Including proxy descendants C. Flattening D. Listing class prims

R24-05 · Obj 2.5 · Select two. Legal ways to change one copy's look: A. Primvar on the instance root B. Variant selection on the instance root C. Set on a proxy attribute D. `GetMaster().GetChild("Seat").Set...`

R24-06 · Obj 2.2 · Single choice A and B are instanceable references to the same asset. A selects `trim=bare`. Result: A. Both de-instance B. Possibly two prototypes; both can stay instances C. USD error D. A loses its reference

R24-07 · Obj 1.10 · Single choice Obj 1.10 "remove properties from instanced component prims" is solved by: A. Nested over under the instance B. Class/variant on the instance root, or de-instance then edit C. `usdcut --strip` D. Muting the session layer

R24-08 · Obj 2.4 · Single choice Native instancing is mainly for: A. Replacing references B. Sharing composed descendants in memory across many copies C. Unloading geometry D. Changing `upAxis`

R24-09 · Obj 2.5 · Single choice After `SetInstanceable(False)` on B, `B.IsInstance()` is: A. True B. False, and `/Room/B/Seat` may be authored C. True, but proxies become editable D. Undefined

R24-10 · Obj 2.2 · Single choice `hasattr(prim, "GetMaster")` on USD 26.08 is: A. True B. False C. True only on prototypes D. True only in C++

R24-11 · Obj 2.5 · Single choice A primvar `displayColor` on the instance root is read on a descendant with: A. `child.GetAttribute("primvars:displayColor").Get()` only B. `UsdGeom.PrimvarsAPI(child).FindPrimvarsWithInheritance()` C. `GetPrototype().GetDisplayColor()` D. `GetMaster()`

R24-12 · Obj 1.10 · Select two. Which keep instancing while hiding a child on one copy?

A. Inherit `_HideChild` on that root B. Select a variant that deactivates the child C. `Clear()` on the proxy D. Delete the child from the published asset (also hides it on every other copy)

Review answers

R24-01 — B. The flag is `IsInstanceable`; `IsInstance` means a prototype exists. Review: §24.1.

R24-02 — B. Review: §24.2.

R24-03 — C. Review: §24.3.

R24-04 — B. Review: §24.3.

R24-05 — A and B. Review: §24.5.

R24-06 — B. Review: §24.5.

R24-07 — B. Review: §24.6.

R24-08 — B. Payloads are C. Review: §24.1.

R24-09 — B. Review: §24.4.

R24-10 — B. Review: §24.2.

R24-11 — B. Review: §24.5.

R24-12 — A and B. D affects every copy. Review: §24.6.

CHAPTER 25

Chapter 25 — Point Instancing

Exam domain: Content Aggregation (10%) · **Objectives:** 2.1, 2.3 (also 2.4) · **Study day:** 8 · **Est. time:** 75 min **Prerequisites:** Ch 5 (properties and relationships), Ch 10 (time samples), Ch 19 (classes), Ch 21 (list editing and value resolution), Ch 24 (native instancing)

In Chapter 24 you met **native instancing** (also called scenegraph instancing): you mark a prim `instanceable = true`, and USD shares one **prototype** among every prim that composes the same way. Native instancing still creates one prim per instance. This chapter covers the other instancing style. A **PointInstancer** stores thousands or millions of copies as a few arrays on a single prim. It is the tool for forests, crowds of rocks, debris, and particle simulations.

Learning goals

- Explain what a `UsdGeom.PointInstancer` is and how it differs from native instancing.
- Read and author the per-instance arrays `protoIndices`, `positions`, `orientations`, and `scales`, and the `prototypes` relationship.
- Add a new prototype to an existing `PointInstancer` without breaking existing instances (Obj 2.1).
- Hide instances efficiently with `invisibleIds` and `inactiveIds`, and compute the result with `ComputeMaskAtTime` (Obj 2.3).
- Use the `ids` attribute to give instances a stable identity that survives reordering.

Key terms

Term	One-line definition
<code>PointInstancer</code>	A prim type (<code>UsdGeom.PointInstancer</code>) that places many copies of prototypes using per-instance arrays
Prototype (point instancing)	A prim (and its subtree) that the <code>PointInstancer</code> copies; targeted by the <code>prototypes</code> relationship
<code>prototypes</code>	Relationship whose ordered targets give each prototype an index 0, 1, 2...
<code>protoIndices</code>	<code>int[]</code> saying which prototype index each instance uses; its length is the instance count
<code>positions</code> / <code>orientations</code> / <code>scales</code>	Per-instance translation (<code>point3f[]</code>), rotation (<code>quath[]</code>), and scale (<code>float3[]</code>)
<code>ids</code>	Optional <code>int64[]</code> giving each instance a permanent ID number
<code>invisibleIds</code>	Animatable <code>int64[]</code> attribute listing instance IDs to hide
<code>inactiveIds</code>	List-editable prim metadata listing instance IDs to prune, for all time
Mask	A list of True/False per instance saying which instances survive hiding/pruning

25.1 `UsdGeom.PointInstancer`

What is it? A **PointInstancer** is a single prim that draws many copies (“instances”) of one or more **prototype** prims. Where each copy goes, how it is rotated and scaled, and which prototype it uses are all stored as arrays on the `PointInstancer` prim.

Why do we need it? Native instancing (Chapter 24) shares the *content* of each instance, but every instance is still a prim on the stage. A forest of one million trees would mean one million prims. Each prim costs memory and composition time (Chapter 26 explains why prim count matters most). A PointInstancer stores one million trees as a handful of arrays on **one** prim, so the stage stays small.

Beginner explanation. Think of the rubber stamp analogy used for all instancing in this book: one prototype, many stamps. With native instancing you buy a separate sheet of paper (a prim) for every stamp. With a PointInstancer you have one sheet of paper and a printed list: “stamp A at (0,0), stamp B at (2,0), stamp A at (4,0)...”. The list is cheap; the paper is the expensive part.

Where the analogy breaks: a stamp on paper cannot be animated. PointInstancer arrays can have time samples, so instances can move, appear, and disappear over time.

Technical explanation. - `UsdGeom.PointInstancer` is a concrete typed schema. It is **Boundable** (it has an extent) and **Xformable**, but it is **not a Gprim**: it has no surface of its own. - Required properties: the `prototypes` relationship, and the `protoIndices` and `positions` attributes. - Instance count = length of `protoIndices` (`GetInstanceCount()` returns it). Every other per-instance array must have the same length. - Prototypes can be any prim subtree, including references to whole assets. The schema docs recommend placing prototypes as children of the PointInstancer, under a `class` prim, so that ordinary traversals (`stage.Traverse()`, `GetChildren()`) skip them and they are not also drawn at their original location. (`over` also works and was the older advice.) - Instances are **not prims**. You cannot select, reference, or override one instance by path. You change instances by editing the arrays. - Prototypes inside a PointInstancer can themselves contain native instances or other PointInstancers (nested instancing).

NOTE

PointInstancer was designed to scale to billions of instances. That is why the transform is split into separate position, orientation (quaternion), and scale arrays instead of one 4×4 matrix per instance: fewer bytes, and you can animate only the arrays that change.

Mental model.

```
PointInstancer "Forest"    (one prim)
+-----+
| rel prototypes  = [ Pine , Oak ]   index: 0  1 |
| protoIndices   = [ 0,      1,      0 ]         |
| positions      = [ (0,0,0), (2,0,0), (4,0,0) ] |
+-----+
          |           |           |
        instance 0   instance 1   instance 2
        Pine @ x=0   Oak @ x=2    Pine @ x=4
```

Simple example. Two prototypes (a cone “Pine” and a cylinder “Oak”) and three instances:

Instance	protoIndices	Prototype	positions
0	0	Pine	(0, 0, 0)
1	1	Oak	(2, 0, 0)
2	0	Pine	(4, 0, 0)

USDA example.

```
#usda 1.0
(
  defaultPrim = "Forest"
)

def PointInstancer "Forest"
{
```

```

point3f[] positions = [(0, 0, 0), (2, 0, 0), (4, 0, 0)]
int[] protoIndices = [0, 1, 0]
rel prototypes = [</Forest/Protos/Pine>, </Forest/Protos/Oak>]

class "Protos"
{
    def Cone "Pine"
    {
    }

    def Cylinder "Oak"
    {
    }
}

```

- `rel prototypes` lists the prototypes in order; Pine is index 0, Oak is index 1.
- `protoIndices` has three entries, so there are three instances.
- class `"Protos"` hides the prototypes from normal traversal, so they are drawn only through the instancer.

Python example.

```

from pxr import Usd, UsdGeom, Sdf

stage = Usd.Stage.CreateInMemory()
forest = UsdGeom.PointInstancer.Define(stage, "/Forest")
protos = stage.DefinePrim("/Forest/Protos")
protos.SetSpecifier(Sdf.SpecifierClass) # keep prototypes out of traversal
UsdGeom.Cone.Define(stage, "/Forest/Protos/Pine")
UsdGeom.Cylinder.Define(stage, "/Forest/Protos/Oak")

forest.CreatePrototypesRel().SetTargets(
    ["/Forest/Protos/Pine", "/Forest/Protos/Oak"])
forest.CreateProtoIndicesAttr([0, 1, 0])
forest.CreatePositionsAttr([(0, 0, 0), (2, 0, 0), (4, 0, 0)])

print("instances:", forest.GetInstanceCount())
print("traversed prims:", [str(p.GetPath()) for p in stage.Traverse()])
print("is a Gprim:", forest.GetPrim().IsA(UsdGeom.Gprim))
print("is Boundable:", forest.GetPrim().IsA(UsdGeom.Boundable))
t = Usd.TimeCode.Default()
for m in forest.ComputeInstanceTransformsAtTime(t, t):
    print("instance at", tuple(m.ExtractTranslation()))

```

Expected output

```

instances: 3
traversed prims: ['/Forest']
is a Gprim: False
is Boundable: True
instance at (0.0, 0.0, 0.0)
instance at (2.0, 0.0, 0.0)
instance at (4.0, 0.0, 0.0)

```

`ComputeInstanceTransformsAtTime(time, baseTime)` returns one `Gf.Matrix4d` per surviving instance (it applies the hiding mask from 25.4 by default). `baseTime` is used when velocities are authored; for static data pass the same time twice.

Real-world use case. An open-world game environment scatters 400,000 rocks, grass clumps, and bushes over terrain with a scattering tool. The tool writes one `PointInstancer` per biome with five to twenty prototypes each. The level loads in seconds because the stage holds a few hundred prims instead of 400,000.

Common mistakes.

MISTAKE

Putting prototypes under a `def` Xform outside the instancer. They are then also drawn once at their own location. Fix: put them under the instancer inside a `class` (or `over`) prim.

MISTAKE

Trying to override one instance by path, e.g. `/Forest/instance\_7`. Instances are not prims. Fix: edit the per-instance arrays, or use native instancing if each copy needs its own prim.

Exam traps.**TRAP**

"A PointInstancer is a Gprim." False. It is Boundable and Xformable, but not a Gprim.

TRAP

"Point instances can be selected and overridden like native instance proxies." False. Native instances (Chapter 24) are prims and have instance proxies; point instances are array entries with no prim path.

Exam takeaways.**KEY**

- One PointInstancer prim = many instances; instances are array entries, not prims. - Required: `prototypes` (rel), `protoIndices` (int[]), `positions` (point3f[]). - Instance count = `len(protoIndices)`. - PointInstancer is Boundable and Xformable, not a Gprim. - Put prototypes under the instancer in a `class` prim.

25.2 prototypes, protoIndices, positions, orientations, scales

What is it? These are the five core properties of a PointInstancer. `prototypes` lists the things to copy. The four arrays say, for each instance, which prototype to use and where, how rotated, and how big.

Why do we need it? You must read and write these arrays correctly to add, move, or edit instances. Exam questions show a USDA snippet and ask which prototype or position an instance gets, or why an instancer draws nothing.

Beginner explanation. Picture a spreadsheet. Each row is one instance. The columns are "prototype number", "position", "rotation", and "scale". USD stores each column as its own array. Row i of the spreadsheet is element i of every array. The `prototypes` relationship is the legend that says "prototype number 0 means Pine, 1 means Oak".

Where the analogy breaks: a spreadsheet tolerates missing cells. A PointInstancer does not. If one column is shorter than `protoIndices`, transform computation fails for the whole instancer.

Technical explanation. | Property | Type | Required | Meaning per instance i |
 |-----|-----|-----|-----|
 | `prototypes` | relationship | yes | Ordered targets; target k is prototype index k |
 | `protoIndices` | int[] | yes | Prototype index of instance i |
 | `positions` | point3f[] | yes | Translation |
 | `orientations` | quath[] | no | Rotation as a unit quaternion (half precision) |
 | `orientationsf` | quatf[] | no | Same, in float precision; if authored non-empty, it is preferred over `orientations` |
 | `scales` | float3[] | no | Non-uniform scale |
 | `velocities`, `accelerations`, `angularVelocities` | vector3f[] | no | Motion between time samples (motion blur) |
 | `ids` | int64[] | no | Stable ID (25.5) |
 | `invisibleIds` | int64[] | no | Hidden IDs (25.4) |

The transform of instance i is built from most-local to least-local:

1. the prototype root prim's own local transform;

2. `scales[i]`;
3. `orientations[i]`;
4. `positions[i]` (plus velocity, if authored);
5. the `PointInstancer` prim's own transform (and its parents').

Rules:

- All per-instance arrays must match `len(protoIndices)`. If they do not, or an index is outside `[0, number of prototypes]`, `ComputeInstanceTransformsAtTime` posts a warning and returns an empty array.
- Optional arrays that are not authored are treated as identity (no rotation, scale 1).
- Pass `UsdGeom.PointInstancer.ExcludeProtoXform` as the third argument of `ComputeInstanceTransformsAtTime` to leave out step 1.
- Primvars on a `PointInstancer` apply per instance, like primvars on a point cloud: a vertex or varying primvar gives element *i* to instance *i*; a constant primvar applies to all instances.

Mental model.

```
instance i: prototypes[ protoIndices[i] ]
              then scale   by scales[i]
              then rotate  by orientations[i]
              then move to positions[i]
              then apply   the instancer's own xform
rule: every per-instance array has len(protoIndices) elements
```

Simple example. A box prototype whose own transform moves it up by 1. One instance has `scales[0] = (2,2,2)` and `positions[0] = (10,0,0)`. The box origin ends up at (10, 2, 0): the prototype's +1 is scaled to +2, then moved to *x* = 10.

USDA example.

```
#usda 1.0

def PointInstancer "Rocks"
{
  quath[] orientations = [(1, 0, 0, 0), (0.7071, 0, 0, 0.7071)]
  point3f[] positions = [(10, 0, 0), (20, 0, 0)]
  int[] protoIndices = [0, 0]
  rel prototypes = </Rocks/Protos/Box>
  float3[] scales = [(2, 2, 2), (1, 1, 1)]

  class "Protos"
  {
    def Cube "Box"
    {
      double3 xformOp:translate = (0, 1, 0)
      uniform token[] xformOpOrder = ["xformOp:translate"]
    }
  }
}
```

- Two instances, both using prototype 0 (Box).
- Quaternions are written (real, i, j, k). (0.7071, 0, 0, 0.7071) is a 90° turn about Z.
- Instance 0 is scaled 2× and placed at *x* = 10; instance 1 is rotated and placed at *x* = 20.

Python example.

```
from pxr import Usd, UsdGeom, Gf, Vt

stage = Usd.Stage.CreateInMemory()
rocks = UsdGeom.PointInstancer.Define(stage, "/Rocks")
box = UsdGeom.Cube.Define(stage, "/Rocks/Protos/Box")
```



```

UsdGeom.XformCommonAPI(box).SetTranslate((0, 1, 0)) # prototype's own xform

rocks.CreatePrototypesRel().SetTargets([box.GetPath()])
rocks.CreateProtoIndicesAttr([0, 0])
rocks.CreatePositionsAttr([(10, 0, 0), (20, 0, 0)])
rocks.CreateScalesAttr([(2, 2, 2), (1, 1, 1)])
turn = Gf.Quat(Gf.Rotation(Gf.Vec3d(0, 0, 1), 90).GetQuat())
rocks.CreateOrientationsAttr(Vt.QuathArray([Gf.Quath(1, 0, 0, 0), turn]))

print(rocks.GetOrientationsAttr().GetTypeName(), rocks.GetScalesAttr().GetTypeName())
t = Usd.TimeCode.Default()
for label, incl in [("include", UsdGeom.PointInstancer.IncludeProtoXform),
                   ("exclude", UsdGeom.PointInstancer.ExcludeProtoXform)]:
    xforms = rocks.ComputeInstanceTransformsAtTime(t, t, incl)
    origins = [m.Transform(Gf.Vec3d(0, 0, 0)) for m in xforms]
    print(label, [tuple(round(v, 2) for v in o) for o in origins])

```

Expected output

```

quath[] float3[]
include [(10.0, 2.0, 0.0), (19.0, 0.0, 0.0)]
exclude [(10.0, 0.0, 0.0), (20.0, 0.0, 0.0)]

```

With the prototype transform included, instance 0's origin is at (10, 2, 0): the prototype's +1 in Y was scaled by 2. Instance 1's +1 in Y was rotated 90° about Z, so it points along -X (19, 0, 0). The tiny rounding comes from half-precision quaternions.

Real-world use case. A VFX crowd department simulates 50,000 birds. Each frame stores `positions`, `orientations`, and `velocities` as time samples, with `protoIndices` choosing one of four bird-flap cycles. Velocities give correct motion blur even though particles are born and die between frames.

Common mistakes.

MISTAKE

Adding an instance to ``protoIndices`` but forgetting ``positions`` (or ``scales``). The arrays no longer match and the whole instancer computes no transforms. Fix: extend every authored per-instance array together.

MISTAKE

Authoring ``orientations`` with ``Gf.Quatf`` values. The attribute type is ``quath[]``; use ``Gf.Quath`` (or author ``orientationsf`` with ``Gf.Quatf``).

Exam traps.

TRAP

Which array sets the instance count? ``protoIndices``, not ``positions`` and not the number of ``prototypes`` targets.

TRAP

Prototype transforms are not ignored: by default the prototype root's own transform is applied first (most locally), and the instance scale affects it.

Exam takeaways.

KEY

- Instance `*i*` uses ``prototypes[protoIndices[i]]``. - Types: ``int[] protoIndices``, ``point3f[] positions``, ``quath[] orientations``, ``float3[] scales``. - All per-instance arrays must match ``len(protoIndices)``. - Transform order: prototype xform → scale → orientation → position → instancer xform.

25.3 Adding a prototype

What is it? Adding a prototype means giving an existing PointInstancer a new kind of object to copy, for example adding a “Bush” to a forest that has only Pine and Oak, and then creating instances that use it (Obj 2.1).

Why do we need it? Set dressers and layout artists often add variety to a scattered set late in production. You want to do this in your own layer, without rewriting the upstream data, and without changing what any existing instance shows.

Beginner explanation. The `prototypes` list is a legend: “0 = Pine, 1 = Oak”. To add Bush, write “2 = Bush” at the **end** of the legend. If you squeeze Bush in at the front, Bush becomes 0, Pine becomes 1, and every existing “0” on the map suddenly means Bush.

Where the analogy breaks: a paper legend is one list. In USD the legend can be assembled from several layers through list editing (Chapter 21), so *where* your edit lands depends on the list-edit operation you author.

Technical explanation. Adding a prototype takes three steps:

1. **Create the prototype prim**, usually under the instancer’s prototype container (`/Forest/Protos/Bush`). It can be a reference to an asset.
2. **Append the target** to the `prototypes` relationship. Its index is its position in the composed target list, so it must go at the end: `index = number of existing prototypes`.
3. **Use the new index**: author instances whose `protoIndices` value is the new index, and extend `positions` (and every other authored per-instance array, including `ids`) to the same new length.

Relationship targets are list-edited (`prepend`, `append`, `delete`, or `explicit`). In Python:

- `rel.AddTarget(path)` defaults to `position=Usd.ListPositionBackOfPrependList`. In a **stronger layer** over an existing list, that writes `prepend rel prototypes`, which puts the new prototype at index 0 and shifts every existing prototype by one.
- Use `rel.AddTarget(path, Usd.ListPositionBackOfAppendList)`, which writes `append rel prototypes`.
- In the **same layer** as an explicit target list, `AddTarget` simply adds to the end of that explicit list.

Arrays such as `protoIndices` and `positions` are **not** list-edited: an opinion in a stronger layer replaces the whole array. To add instances from your own layer, write the complete new arrays (old values + new values).

NOTE

If you only need to **swap** what a prototype looks like (for example, a better Pine model), do not add a prototype at all: change what the existing ``Pine`` prim references. All instances using index 0 update, and no index changes.

Mental model.

composed prototypes	index	prepend Bush (WRONG)	append Bush (RIGHT)
-----	---	-----	-----
Pine	0	Bush	Pine
Oak	1	Pine	Oak
	2	Oak	Bush
<code>protoIndices [0,1]</code> now draw:		Bush, Pine (changed!)	Pine, Oak (unchanged)

Simple example. Before: `prototypes = [Pine, Oak]`, `protoIndices = [0, 1]`, `positions = [(0,0,0), (2,0,0)]`. After adding one Bush instance at `x = 4`: `prototypes = [Pine, Oak, Bush]`, `protoIndices = [0, 1, 2]`, `positions = [(0,0,0), (2,0,0), (4,0,0)]`.

USDA example. File: *shot_layout.usda* (a stronger layer that sublayers the published *forest.usda*; shown alone here)

```
#usda 1.0
(
    subLayers = [@forest.usda@]
)

over "Forest"
{
    point3f[] positions = [(0, 0, 0), (2, 0, 0), (4, 0, 0)]
    int[] protoIndices = [0, 1, 2]
    append rel prototypes = </Forest/Protos/Bush>

    over "Protos"
    {
        def Sphere "Bush"
        {
        }
    }
}
```

- `append rel prototypes` adds Bush **after** Pine and Oak from `forest.usda`, so Bush is index 2.
- `protoIndices` and `positions` are written in full because arrays replace, they do not merge.
- `over "Protos"` adds a child to the existing class "Protos"; the composed specifier stays class.

Python example. The script writes a published forest layer, then adds a Bush from a stronger layer twice: once with the default `AddTarget` (wrong) and once with an explicit `append` (right).

```
from pxr import Usd, UsdGeom, Sdf

base = Usd.Stage.CreateNew("forest.usda")
forest = UsdGeom.PointInstancer.Define(base, "/Forest")
base.DefinePrim("/Forest/Protos").SetSpecifier(Sdf.SpecifierClass)
UsdGeom.Cone.Define(base, "/Forest/Protos/Pine")
UsdGeom.Cylinder.Define(base, "/Forest/Protos/Oak")
forest.CreatePrototypesRel().SetTargets(
    ["/Forest/Protos/Pine", "/Forest/Protos/Oak"])
forest.CreateProtoIndicesAttr([0, 1])
forest.CreatePositionsAttr([(0, 0, 0), (2, 0, 0)])
base.Save()

def add_bush(position):
    stage = Usd.Stage.CreateInMemory()
    stage.GetRootLayer().subLayerPaths.append("forest.usda")
    pi = UsdGeom.PointInstancer(stage.GetPrimAtPath("/Forest"))
    UsdGeom.Sphere.Define(stage, "/Forest/Protos/Bush")
    rel = pi.GetPrototypesRel()
    if position is None:
        rel.AddTarget("/Forest/Protos/Bush")          # default position
    else:
        rel.AddTarget("/Forest/Protos/Bush", position)
    new_index = len(rel.GetTargets()) - 1             # only valid if appended
    pi.GetProtoIndicesAttr().Set(list(pi.GetProtoIndicesAttr().Get()) + [new_index])
    pi.GetPositionsAttr().Set(list(pi.GetPositionsAttr().Get()) + [(4, 0, 0)])
    names = [t.name for t in rel.GetTargets()]
    shown = [names[i] for i in pi.GetProtoIndicesAttr().Get()]
    print(" prototypes:", names)
    print(" instances draw:", shown)

print("default AddTarget (prepend):")
add_bush(None)
print("AddTarget with BackOfAppendList:")
add_bush(Usd.ListPositionBackOfAppendList)
```

Expected output

```
default AddTarget (prepend):
  prototypes: ['Bush', 'Pine', 'Oak']
  instances draw: ['Bush', 'Pine', 'Oak']
AddTarget with BackOfAppendList:
  prototypes: ['Pine', 'Oak', 'Bush']
  instances draw: ['Pine', 'Oak', 'Bush']
```

With the default (prepend) the composed list becomes [Bush, Pine, Oak]: the two original instances silently turn into Bush and Pine, and the “new” instance gets Oak. With append, the originals are unchanged and the new instance is a Bush.

Real-world use case. A digital-twin team scatters 12,000 trees around a factory site from a GIS export. The client asks for hedges along the road. The layout artist appends a Hedge prototype in the site-dressing layer and writes the extended arrays there. The GIS-generated forest layer is never touched, so it can be regenerated later.

Common mistakes.**MISTAKE**

Using `AddTarget(path)` with its default position in a stronger layer. It prepends and re-numbers every existing prototype. Fix: pass `Usd.ListPositionBackOfAppendList`, or author `append` in USDA.

MISTAKE

Appending the target but writing only the new instances into `protoIndices` in the stronger layer. The array replaces the weaker one, so all old instances disappear. Fix: write old + new values.

MISTAKE

Extending `protoIndices` and `positions` but not `ids` (or `scales`, `orientations`) when those are authored. Lengths mismatch and nothing is computed. Fix: extend every authored per-instance array.

Exam traps.**TRAP**

"Add the new prototype path to `protoIndices`." No. Paths go in the `prototypes` relationship; `protoIndices` holds integers.

TRAP

Answers that rebuild the whole PointInstancer or flatten the scene "to add a prototype" are heavy-handed. The minimal correct edit is: create the prim, append the target, use the new index.

Exam takeaways.**KEY**

- New prototype index = its position in the composed `prototypes` list, so **append** it.
- `AddTarget` defaults to prepend; use `Usd.ListPositionBackOfAppendList` across layers.
- Arrays replace, they don't merge: write full `protoIndices`/`positions` in your layer.
- Extend every authored per-instance array to the same length.

VERSION

The default `position` of `Usd.Relationship.AddTarget` (`Usd.ListPositionBackOfPrependList`) and the resulting `prepend` were verified on USD 26.08.

25.4 Hiding instances efficiently (`invisibleIds`, `MakeInvisible`, `DeactivateId`)

What is it? A `PointInstancer` can hide individual instances without deleting them from its arrays. There are two built-in ways: the animatable `invisibleIds` attribute, and the list-editable `inactiveIds` metadata. USD combines both into a **mask** (Obj 2.3).

Why do we need it? Deleting an instance means rewriting every per-instance array (millions of values, maybe over many time samples), and it shifts the index of every later instance. A downstream department (lighting, layout) should not have to rewrite an upstream simulation just to remove a tree that blocks the camera.

Beginner explanation. Imagine a printed seating chart for a stadium. To keep one seat empty, you don't reprint the chart; you put a "Reserved" sticker on it. `invisibleIds` is a sticker list you can change from frame to frame. `inactiveIds` is a sticker list that holds for the whole event, but several people can each add or remove stickers on their own copy, and the lists combine.

Where the analogy breaks: stickers on paper are visible; in USD the hidden instances are simply skipped when transforms, bounds, and drawing are computed.

Technical explanation. | | `invisibleIds` | `inactiveIds` | |-----|-----| | What it is | `int64[]` attribute (default `[]`) | `int64` list-op **metadata** on the prim | | Animatable | Yes (time samples) | No: applies to all time | | Layering | Strongest opinion replaces the whole array | List-edited (prepend/append/delete) across layers | | Hide API | `InvisId(id, time)`, `InvisIds(ids, time)` | `DeactivateId(id)`, `DeactivateIds(ids)` | | Show API | `VisId(id, time)`, `VisIds(ids, time)`, `VisAllIds(time)` | `ActivateId(id)`, `ActivateIds(ids)`, `ActivateAllIds()` | | Good for | Per-frame camera-frustum culling, animated pop-in | Sparse, reversible pruning by downstream departments |

- Values in both lists are **IDs**: values of the `ids` array if authored, otherwise array **indices** (25.5).
- `ComputeMaskAtTime(time)` combines both lists into a list of booleans, one per instance, `True` = survives. If every instance survives, it returns an **empty** list (meaning "no masking").
- `ComputeInstanceTransformsAtTime` and `ComputeExtentAtTime` apply the mask by default, so hidden instances are left out of transforms and bounds. Pass `UsdGeom.PointInstancer.IgnoreMask` as the fourth argument to get all instances.
- Despite the name, `inactiveIds` does not deactivate prims: no prims are removed from the stage. It is only a mask.
- `InvisId` authors into the current edit target. If `invisibleIds` was animated in weaker layers, your stronger opinion at that time hides those weaker samples, so overriding animated `invisibleIds` sparsely is hard. That is why downstream edits prefer `inactiveIds`.

TRAP

``UsdGeom.PointInstancer.MakeInvisible()`` exists, but it is the method inherited from ``UsdGeom.Imageable``. It sets ``visibility = "invisible"`` on the **whole** `PointInstancer` prim and takes only an optional time. It does not hide one instance. The per-instance methods are ``InvisId`/`InvisIds`` (and ``MakeVisible()`` is likewise the whole-prim `Imageable` method).

Mental model.

```
ids:          [ 10  11  12  13 ]
inactiveIds:      11          (list-op metadata, all time)
invisibleIds:      13          (attribute, can animate)
-----
mask:          [ T   F   T   F ] -> 2 instances drawn
no masking at all -> mask = []    (empty means "all pass")
```

Simple example. A forest has four trees with IDs 10–13. Lighting needs tree 11 gone for the whole shot: `DeactivateId(11)`. A camera-culling tool hides tree 13 only from frame 20 on: `InvisId(13, 20)`. The arrays of positions are unchanged.

USDA example.

```
#usda 1.0

def PointInstancer "Forest" (
    inactiveIds = [11]
)
{
    int64[] ids = [10, 11, 12, 13]
    int64[] invisibleIds.timeSamples = {
        1: [],
        20: [13],
    }
    point3f[] positions = [(0, 0, 0), (2, 0, 0), (4, 0, 0), (6, 0, 0)]
    int[] protoIndices = [0, 0, 0, 0]
    rel prototypes = </Forest/Protos/Pine>

    class "Protos"
    {
        def Cone "Pine"
        {
        }
    }
}
```

- `inactiveIds = [11]` is prim metadata in the parentheses after the prim name, not an attribute.
- `invisibleIds` is time-sampled: from frame 20, ID 13 is hidden too.
- A stronger layer could write `delete inactiveIds = [11]` to bring tree 11 back.

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
pi = UsdGeom.PointInstancer.Define(stage, "/Forest")
UsdGeom.Cone.Define(stage, "/Forest/Protos/Pine")
pi.CreatePrototypesRel().SetTargets(["/Forest/Protos/Pine"])
pi.CreateProtoIndicesAttr([0, 0, 0, 0])
pi.CreatePositionsAttr([(0, 0, 0), (2, 0, 0), (4, 0, 0), (6, 0, 0)])
pi.CreateIdsAttr([10, 11, 12, 13])
d = Usd.TimeCode.Default()

print("nothing hidden:", list(pi.ComputeMaskAtTime(d)))
pi.DeactivateId(11) # list-op metadata, all time
pi.InvisId(13, Usd.TimeCode(20)) # animatable attribute
print("inactiveIds:", pi.GetPrim().GetMetadata("inactiveIds"))
print("invisibleIds @20:", list(pi.GetInvisibleIdsAttr().Get(20)))
print("mask @20:", list(pi.ComputeMaskAtTime(Usd.TimeCode(20))))
xf = pi.ComputeInstanceTransformsAtTime(Usd.TimeCode(20), Usd.TimeCode(20))
print("drawn x @20:", [m.ExtractTranslation()[0] for m in xf])
allx = pi.ComputeInstanceTransformsAtTime(
    Usd.TimeCode(20), Usd.TimeCode(20),
    UsdGeom.PointInstancer.IncludeProtoXform, UsdGeom.PointInstancer.IgnoreMask)
print("ignore mask:", len(allx))

pi.MakeInvisible() # Imageable method: whole prim!
print("prim visibility:", pi.GetVisibilityAttr().Get())
pi.MakeVisible()
pi.ActivateAllIds()
pi.VisAllIds(Usd.TimeCode(20))
print("after reset:", list(pi.ComputeMaskAtTime(Usd.TimeCode(20))))
```

Expected output

```
nothing hidden: []
inactiveIds: SdfInt64ListOp(Explicit Items: [11])
```

```
invisibleIds @20: [13]
mask @20: [True, False, True, False]
drawn x @20: [0.0, 4.0]
ignore mask: 4
prim visibility: invisible
after reset: []
```

Notice that `MakeInvisible()` changed the prim's `visibility`, not the instance mask.

The second script shows why `inactiveIds` suits downstream work. Authored from a stronger layer, `DeactivateId` and `ActivateId` write sparse list edits instead of replacing the upstream list.

```
from pxr import Usd, UsdGeom

up = Usd.Stage.CreateNew("sim.usda")
pi = UsdGeom.PointInstancer.Define(up, "/Debris")
pi.CreateProtoIndicesAttr([0, 0, 0, 0])
pi.DeactivateId(1) # upstream prunes index 1
up.Save()

shot = Usd.Stage.CreateInMemory()
shot.GetRootLayer().subLayerPaths.append("sim.usda")
debris = UsdGeom.PointInstancer(shot.GetPrimAtPath("/Debris"))
debris.DeactivateId(2) # downstream prunes index 2
debris.ActivateId(1) # ...and restores index 1
spec = shot.GetRootLayer().GetPrimAtPath("/Debris")
ops = spec.GetInfo("inactiveIds")
print("shot layer deletes:", list(ops.deletedItems))
print("shot layer adds:", list(ops.prependedItems) + list(ops.appendedItems))
print("composed:", list(debris.GetPrim().GetMetadata("inactiveIds").explicitItems))
print("mask:", list(debris.ComputeMaskAtTime(Usd.TimeCode.Default())))
```

Expected output

```
shot layer deletes: [1]
shot layer adds: [2]
composed: [2]
mask: [True, True, False, True]
```

Real-world use case. In a film shot, the camera flies low through a forest. A pipeline tool computes, per frame, which trees are outside the camera frustum and writes them into time-sampled `invisibleIds`. Separately, the director asks to remove three trees blocking a hero moment; the lighting artist uses `DeactivateIds` in the lighting layer. If the trees are needed back, a delete `inactiveIds` in a later layer reverses it without touching the simulation.

Common mistakes.

MISTAKE

Deleting entries from the arrays to "hide" instances. This rewrites everything and shifts later instances. Fix: use ``inactiveIds`` or ``invisibleIds``.

MISTAKE

Calling ``pi.MakeInvisible()`` to hide one instance. It hides the whole instancer. Fix: ``pi.InvisId(id, time)`` or ``pi.DeactivateId(id)``.

MISTAKE

Passing an array index to ``InvisId`` when ``ids`` is authored. The lists hold ID values. Fix: look up the ID with ``pi.GetIdsAttr().Get()[index]``.

Exam traps.

TRAP

"inactiveIds deactivates prims." No. It is a per-instance mask; no prims are deactivated or removed.

TRAP

"invisibleIds can be overridden sparsely in many layers." No; it is an array attribute, so the strongest opinion replaces it (and animated values must be re-authored). ``inactiveIds`` is the list-edited one.

TRAP

An empty result from ``ComputeMaskAtTime`` means **all** instances are visible, not "no instances".

Exam takeaways.**KEY**

- Hide instances with masks, never by deleting array entries. - ``invisibleIds``: attribute, animatable, ``InvisId` / `VisId` / `VisAllIds``. - ``inactiveIds``: list-op metadata, all time, sparse and reversible, ``DeactivateId` / `ActivateId` / `ActivateAllIds``. - ``ComputeMaskAtTime`` combines both; empty list = nothing masked. - ``MakeInvisible()`` on a `PointInstancer` hides the whole prim (`Imageable`).

VERSION

Verified on USD 26.08: ``PointInstancer`` has ``InvisId``, ``InvisIds``, ``VisId``, ``VisIds``, ``VisAllIds``, ``DeactivateId(s)``, ``ActivateId(s)``, ``ActivateAllIds``, and ``ComputeMaskAtTime(time)``. ``MakeInvisible` / `MakeVisible`` are inherited from ``UsdGeom.Imageable``. Some tutorials loosely say "MakeInvisible" when they mean ``InvisId``; the exam objective's intent is per-instance hiding.

25.5 ids and stable identity

What is it? `ids` is an optional `int64[]` attribute that gives each instance a permanent ID number. When it is authored, `invisibleIds` and `inactiveIds` refer to these IDs instead of array positions.

Why do we need it? Array positions are not stable. A re-run of a scatter tool might sort trees differently, and in simulations dead particles' slots are reused by new ones. Without IDs, "hide index 2" may hide a different tree after the upstream data changes. IDs let hiding decisions, and tracking for motion blur, follow the same object.

Beginner explanation. Seat numbers versus names. "Hide the person in seat 3" breaks when everyone changes seats. "Hide the person named Ana" still works. `ids` gives every instance a name tag.

Where the analogy breaks: the "name" is a 64-bit integer, and USD does not check that IDs are unique. Keeping them unique is the authoring tool's job.

Technical explanation. - `ids` has the same length as `protoIndices` and can be time-sampled (simulations where instances are born and die). - If `ids` is **not** authored, the values in `invisibleIds` and `inactiveIds` are interpreted as indices into `protoIndices`. - If `ids` **is** authored, those values are matched against `ids`; values that match no ID have no effect. - A simulator that increments a counter at each birth produces perfect IDs. - When you add instances (25.3), give them new IDs that no existing instance uses.

Mental model.

	without ids	with ids
order v1:	idx 0 1 2 (A B C)	id 7 8 9 (A B C)
hide "1"	-> B hidden	hide 8 -> B hidden
order v2:	idx 0 1 2 (C A B)	id 9 7 8 (C A B)
hide "1"	-> A hidden (wrong!)	hide 8 -> B hidden (still right)

Simple example. Trees A, B, C have IDs 7, 8, 9. You hide ID 8 (tree B). Upstream re-sorts the arrays to C, A, B, with `ids` re-sorted to 9, 7, 8. ID 8 is still tree B, so B stays hidden.

USDA example.

```
#usda 1.0

def PointInstancer "Particles" (
    inactiveIds = [8]
)
{
    int64[] ids = [9, 7, 8]
    point3f[] positions = [(4, 0, 0), (0, 0, 0), (2, 0, 0)]
    int[] protoIndices = [0, 0, 0]
    rel prototypes = </Particles/Protos/Drop>

    class "Protos"
    {
        def Sphere "Drop"
        {
        }
    }
}
```

- ID 8 is at array index 2 here, at position (2, 0, 0). That is the instance pruned.
- Without `ids`, `inactiveIds = [8]` would refer to index 8, which does not exist, so nothing would be hidden.

Python example.

```
from pxr import Usd, UsdGeom

def build(order, use_ids):
    stage = Usd.Stage.CreateInMemory()
    pi = UsdGeom.PointInstancer.Define(stage, "/Trees")
    UsdGeom.Cone.Define(stage, "/Trees/Protos/Pine")
    pi.CreatePrototypesRel().SetTargets(["/Trees/Protos/Pine"])
    tree_x = {"A": 0, "B": 2, "C": 4}
    tree_id = {"A": 7, "B": 8, "C": 9}
    pi.CreateProtoIndicesAttr([0] * len(order))
    pi.CreatePositionsAttr([(tree_x[n], 0, 0) for n in order])
    if use_ids:
        pi.CreateIdsAttr([tree_id[n] for n in order])
        pi.DeactivateId(8) # "tree B" by ID
    else:
        pi.DeactivateId(1) # "tree B" by index in the v1 order
    mask = pi.ComputeMaskAtTime(Usd.TimeCode.Default())
    return [n for n, keep in zip(order, mask) if not keep]

for use_ids in (False, True):
    v1 = build(["A", "B", "C"], use_ids)
    v2 = build(["C", "A", "B"], use_ids)
    print("ids" if use_ids else "no ids", "hidden v1:", v1, "hidden v2:", v2)
```

Expected output

```
no ids hidden v1: ['B'] hidden v2: ['A']
ids hidden v1: ['B'] hidden v2: ['B']
```

Without `ids`, the same edit hides a different tree once the upstream order changes. With `ids`, the hidden tree is always B.

Real-world use case. An FX artist re-simulates a debris explosion after notes. The shot's lighting layer had deactivated three debris pieces that flew into the lens. Because the simulator writes stable `ids`, the same three pieces stay hidden in the new simulation, and nobody has to redo the lighting fix.

Common mistakes.**MISTAKE**

Authoring ``ids`` with a different length than ``protoIndices``. Fix: one ID per instance, extended whenever instances are added.

MISTAKE

Adding ``ids`` to an existing instancer that already uses index-based ``invisibleIds``/``inactiveIds``. The old values now mean IDs, so different (or no) instances are hidden. Fix: convert the masks to ID values when you add ``ids``.

Exam traps.**TRAP**

"``invisibleIds`` always holds array indices." Only when ``ids`` is not authored. With ``ids``, it holds ID values.

TRAP

USD does not enforce unique IDs. A question asking "what guarantees uniqueness?" is pointing at the authoring tool, not at USD.

Exam takeaways.**KEY**

- ``ids`` (`int64[]``, optional, animatable) = stable instance identity. - No ``ids`` → masks use indices; ``ids`` authored → masks use ID values. - Use ``ids`` whenever upstream order can change or slots are reused. - Extend ``ids`` along with every other array when you add instances.

Chapter lab(s)

Lab 24 — PointInstancer: add prototypes, hide instances (Obj 2.1, 2.3; in `python-labs/`). You build a forest `PointInstancer` in a published layer, then from a shot layer you append a new prototype and instances, hide trees with `inactiveIds` and time-sampled `invisibleIds`, and check the result with `ComputeMaskAtTime` and `ComputeInstanceTransformsAtTime`. The stretch task re-sorts the upstream arrays and shows that `ids` keeps your hiding decisions stable.

USDA reading exercises

Exercise 25-A. Read this layer. Which prototype does each instance draw, and which instances survive at time 5?

```
#usda 1.0

def PointInstancer "Set" (
    inactiveIds = [2]
)
{
    point3f[] positions = [(0, 0, 0), (1, 0, 0), (2, 0, 0), (3, 0, 0)]
    int[] protoIndices = [1, 0, 1, 2]
    rel prototypes = [</Set/P/Chair>, </Set/P/Table>, </Set/P/Lamp>]
    int64[] invisibleIds.timeSamples = {
        0: [3],
        10: [],
    }
}

class "P"
{
    def Cube "Chair"
    {
    }

    def Cube "Table"
    {
    }

    def Cube "Lamp"
    {
    }
}
```

```
}
}
```

Exercise 25-B. A stronger layer contains only the following opinion over the layer above. What is the composed `prototypes` order, and which prototype does the instance at index 3 draw now?

```
over "Set"
{
    prepend rel prototypes = </Set/P/Sofa>
}
```

This fragment is not a complete layer (it is only the override part), so it is not parsed on its own.

Answers

- **25-A.** Instances 0–3 draw Table, Chair, Table, Lamp (`prototypes[1]`, `[0]`, `[1]`, `[2]`). No `ids` is authored, so the masks use indices. Index 2 is inactive for all time. At time 5, `invisibleIds` holds the sample from time 0, `[3]` (array values are held, not interpolated, between samples), so index 3 is hidden too. Instances 0 and 1 survive: mask `[True, True, False, False]`.
- **25-B.** The composed order is `[Sofa, Chair, Table, Lamp]`. Instance 3 has `protoIndices` value 2, which now means **Table** instead of Lamp. Every instance's prototype shifted: this is the prepend trap from 25.3.

Chapter review

Summary

- A `PointInstancer` stores many instances as arrays on one prim; instances are not prims.
- Required: `prototypes` (rel), `protoIndices`, `positions`. Optional: `orientations` (`quath[]`), `scales`, `velocities`, `ids`, `invisibleIds`.
- Instance count = `len(protoIndices)`; all per-instance arrays must match it.
- Transform order: prototype root xform → scale → orientation → position → instancer xform.
- Keep prototypes under the instancer inside a `class` prim.
- Add a prototype by **appending** its target, then using the new index and extending every array.
- `AddTarget` defaults to prepend; across layers, pass `Usd.ListPositionBackOfAppendList`.
- Hide instances with `invisibleIds` (animatable) or `inactiveIds` (list-edited, all time); `ComputeMaskAtTime` combines them.
- `MakeInvisible()` on a `PointInstancer` hides the whole prim.
- `ids` makes hiding follow objects, not array positions.

If you see... → think...

If you see...	Think...
"Millions of copies, minimal prim count"	<code>PointInstancer</code>
"Add a new prototype"	Append to <code>prototypes</code> , new index = old count, extend arrays
prepend <code>rel prototypes</code> in a stronger layer	All existing indices shift by one
"Hide some instances efficiently"	<code>invisibleIds</code> / <code>inactiveIds</code> , not deleting array entries
"Hide per frame / animated"	<code>invisibleIds (InvisId(id, time))</code>

If you see...	Think...
"Sparse, reversible, downstream"	<code>inactiveIds</code> (<code>DeactivateId</code> , <code>delete inactiveIds</code>)
<code>MakeInvisible()</code> on the instancer	Whole prim hidden (<code>Imageable</code> visibility)
Empty mask from <code>ComputeMaskAtTime</code>	Nothing masked: all instances visible
"Instance identity survives reordering"	<code>ids</code> attribute
Instancer draws nothing after an edit	Array length mismatch or bad prototype index

Review questions

CA-R25-01 · Obj 2.1 · Medium · Single choice A layout artist adds a `Bush` prototype to a published `PointInstancer` from a stronger layer and wants existing instances unchanged. Which authored opinion is correct? A. `prepend rel prototypes = </Forest/Protos/Bush>` B. `append rel prototypes = </Forest/Protos/Bush>` C. `int[] protoIndices = [</Forest/Protos/Bush>]` D. `rel prototypes = </Forest/Protos/Bush>`

CA-R25-02 · Obj 2.3 · Medium · Single choice Which call hides only instance ID 42 from frame 100 onward? A. `pi.MakeInvisible(Usd.TimeCode(100))` B. `pi.DeactivateId(42)` C. `pi.InvisId(42, Usd.TimeCode(100))` D. `pi.GetPrim().SetActive(False)`

CA-R25-03 · Obj 2.3 · Easy · Select two Which are valid ways to hide individual point instances without editing positions? Select two. A. Author `invisibleIds` B. Author `inactiveIds` C. Set `instanceable = false` D. Set `visibility = "invisible"` on the prototype E. Remove the target from prototypes

CA-R25-04 · Obj 2.1 · Medium · Code reading prototypes composes to `[A, B]` and `protoIndices = [0, 1, 1]`. A stronger layer authors `append rel prototypes = <C>` and `int[] protoIndices = [2]`. How many instances are drawn, and what is the problem? A. 4 instances, no problem B. 1 instance; the stronger array replaced the old one, and positions (length 3) no longer matches, so nothing is computed C. 3 instances; arrays merge like list ops D. 1 instance of C

CA-R25-05 · Obj 2.3 · Medium · Single choice Upstream simulation data has animated `invisibleIds`. A downstream layer must hide one more instance sparsely, so it is easy to undo later. What should it author? A. A new full `invisibleIds` time-sample set B. `inactiveIds` via `DeactivateId` C. `visibility = "invisible"` on the instancer D. A shorter positions array

CA-R25-06 · Obj 2.4 · Easy · Single choice Which property determines a `PointInstancer`'s instance count? A. `positions` B. `prototypes` C. `protoIndices` D. `ids`

CA-R25-07 · Obj 2.3 · Hard · Code reading `ids = [5, 6, 7]`, `invisibleIds = [0]`, `inactiveIds = [7]`. What does `ComputeMaskAtTime` return? A. `[False, True, False]` B. `[True, True, False]` C. `[]` D. `[False, True, True]`

CA-R25-08 · Obj 2.4 · Medium · Select two Which statements about `UsdGeom.PointInstancer` are true? Select two. A. It is a `Gprim` B. It is `Boundable` C. Each instance has its own prim path D. Its prototypes can be any prim subtree, including referenced assets E. It requires `instanceable = true` on prototypes

CA-R25-09 · Obj 2.4 · Medium · Single choice A prototype's root has `xformOp:translate = (0, 1, 0)`. An instance has `scales = (3, 3, 3)` and `positions = (5, 0, 0)`. Using default arguments, where is the prototype's origin placed? A. (5, 1, 0) B. (5, 3, 0) C. (15, 3, 0) D. (5, 0, 0)

CA-R25-10 · Obj 2.3 · Medium · Single choice What is the main advantage of `inactiveIds` over removing entries from the per-instance arrays? A. It reduces file size B. It is non-destructive and list-editable, and it keeps every other instance's index C. It makes instances load faster D. It works only when `ids` is authored

Answers and explanations

- **CA-R25-01 — B.** Append keeps Pine/Oak at indices 0/1. A shifts every index; C puts a path in an int array; D replaces the list in the stronger layer, so the old prototypes are gone. Review: §25.3.
- **CA-R25-02 — C.** InvisId is the animatable per-instance hide. A hides the whole instancer; B hides for all time; D deactivates the prim. Review: §25.4.
- **CA-R25-03 — A, B.** Both are per-instance masks. D hides the prototype (all its instances); E breaks indices; C is native instancing. Review: §25.4.
- **CA-R25-04 — B.** Arrays are not list-edited; the stronger [2] replaces [0, 1, 1]. With positions still length 3, the lengths mismatch and no transforms are computed. Review: §25.3.
- **CA-R25-05 — B.** Overriding animated invisibleIds requires re-authoring the whole array; inactiveIds is sparse and list-edited. Review: §25.4.
- **CA-R25-06 — C.** GetInstanceCount() returns len(protoIndices). Review: §25.2.
- **CA-R25-07 — B.** With ids authored, the 0 in invisibleIds matches no ID, so it hides nothing. ID 7 is at index 2 and is inactive. A wrongly treats 0 as an index. Review: §25.5.
- **CA-R25-08 — B, D.** It is Boundable, not a Gprim; instances have no prim paths; instanceable is native instancing. Review: §25.1.
- **CA-R25-09 — B.** Prototype xform first: (0, 1, 0) scaled by 3 is (0, 3, 0), then translated to (5, 3, 0). Review: §25.2.
- **CA-R25-10 — B.** Masking is non-destructive and never re-indexes; inactiveIds also composes across layers. It works with or without ids. Review: §25.4.

CHAPTER 26

Chapter 26 — Large Scene Optimization

Exam domain: Content Aggregation (10%), also Composition (23%) · **Objectives:** 1.2, 2.4 · **Study day:** 8 · **Est. time:** 70 min **Prerequisites:** Ch 6 (kinds), Ch 13 (extent), Ch 17 (payloads), Ch 18 (variant sets), Ch 23 (asset structure), Ch 24 (native instancing), Ch 25 (point instancing)

A **large scene** is a stage with thousands to millions of objects, such as a city, a forest, a factory, or a stadium crowd. This chapter puts the tools from Part IV together: when to use native instancing, point instancing, or no instancing; how payloads keep a scene light; how draw modes and extents hints let viewers show stand-ins; and how variants give you levels of detail. It ends with a decision table and the official “Maximizing USD Performance” guidance as it applies to aggregation. Chapter 44 revisits performance from the debugging side.

Learning goals

- Choose between no instancing, native instancing, and point instancing for data at different scales (Obj 1.2).
- Structure assets with payloads so a large scene opens fast and loads only what a task needs (Obj 2.4).
- Author `UsdGeom.ModelAPI` draw modes and `extentsHint`, and compute them with `ComputeModelDrawMode` and `ComputeExtentsHint`.
- Build a level-of-detail (LOD) variant set and explain how it interacts with instancing.
- Apply the official performance guidance: prefer crate files, watch layer count, minimize prim count, use transformable gprims, and instance at a higher granularity.

Key terms

Term	One-line definition
Prim count	Number of prims populated on the stage; the main driver of open time and memory
Native instancing	<code>instanceable = true</code> ; instances are prims that share a prototype (Chapter 24)
Point instancing	A <code>PointInstancer</code> prim that stores instances as arrays (Chapter 25)
Working set	The subset of payloads loaded for the current task
Asset interface	Light top-level file of an asset: kind, <code>assetInfo</code> , variant sets, extents hint, and a payload to the heavy data
Draw mode	<code>model:drawMode</code> : a cheap stand-in (origin axes, bounding box, or textured cards) a viewer may draw instead of full geometry
Extents hint	<code>extentsHint</code> : a cached bounding box for a whole model, one box per purpose
LOD	Level of detail: lighter versions of an asset for distant or background use
Crate	The binary <code>.usdc</code> format (also the default for <code>.usd</code> files)
Transformable gprim	A geometric prim (Mesh, Cube...) that carries its own transform, so it needs no parent Xform

26.1 Choosing an instancing style at different scales

What is it? It is the decision of how to represent repeated objects: as ordinary unique prims (no instancing), as native instances (`instanceable = true`), or as entries of a `PointInstancer`. The right answer depends mainly on **how many** copies there are and **how much each copy must differ** (Obj 1.2).

Why do we need it? Each style trades editability against cost. Unique prims are fully editable but expensive. Point instances are extremely cheap but are not prims, so you cannot edit one by path. Choosing wrongly gives you a city that will not open, or a hero prop nobody can art-direct.

Beginner explanation. Three ways to furnish a stadium. **No instancing:** build every seat by hand; each can be unique, and it takes forever. **Native instancing:** order seats from one catalog design; each seat still has its own spot in the inventory (a prim), and you can swap a whole seat for a different catalog design. **Point instancing:** print a seating chart that says “seat design A at these 50,000 coordinates”; there is one document and no per-seat inventory entries.

Where the analogy breaks: the styles mix. A native instance’s prototype can contain a `PointInstancer`, and a `PointInstancer`’s prototype can be a natively instanced asset.

Technical explanation. | | No instancing | Native instancing | Point instancing |
 |-----|-----|-----| | Typical count | 1 – tens | tens – tens of thousands
 | thousands – millions (and more) | | Prims per copy | Whole subtree | 1 (the instance
 prim); subtree shared in a prototype | 0 (array entries on one prim) | | Per-copy edits |
 Anything | On the instance prim: transform, visibility, variant selections, inherited
 primvars, material binding on the root; inside it only via composition arcs | Only via
 arrays and masks: position, orientation, scale, prototype index, per-instance primvars,
 hiding | | Addressable by path | Yes | Yes (instance proxies are read-only) | No | | Copies
 can differ in structure | Yes | Yes, via different variant selections → separate prototypes |
 Only by choosing a different prototype | | Good for | Hero assets, things that will be
 edited | Buildings, vehicles, props in a set | Vegetation, rocks, debris, particles, crowds of
 simple agents |

Rules of thumb:

- If a copy needs **unique internal edits**, do not instance it (or de-instance just that one: `instanceable = false`).
- If copies need **individual identity in the scenegraph** (selectable, referenced by path, own variant selection), use **native** instancing.
- If you have **very many** copies that differ only in transform and maybe a primvar, use a **PointInstancer**.
- Instance at the **highest sensible granularity**: instance the whole asset, not each leaf mesh (26.5 explains why).
- Native instances that compose identically share one prototype. Different variant selections create different prototypes (`len(stage.GetPrototypes())` grows).

Mental model.

how many copies?		do copies need unique internal edits?	
few	very many	yes -> NO INSTANCING	
		no -> need own prim/path/variant?	
v	v	yes -> NATIVE INSTANCING	
NATIVE or NONE	POINT INSTANCER	no, and huge count	
		-> POINT INSTANCER	

Simple example. | Content | Count | Choice | |-----|-----|-----| | The hero spaceship | 1
 | No instancing | | Parked cars in a city block (some get damage variants) | 300 | Native
 instancing | | Grass clumps on a hillside | 2,000,000 | PointInstancer |

USDA example. The same three trees written in the two instancing styles. (`tree.usda` is a placeholder path; the layer parses on its own.)

```
#usda 1.0

def Xform "Native"
{
  def "Tree_01" (instanceable = true
    prepend references = @tree.usda@)
  {
    double3 xformOp:translate = (0, 0, 0)
    uniform token[] xformOpOrder = ["xformOp:translate"]
  }

  def "Tree_02" (instanceable = true
    prepend references = @tree.usda@)
  {
    double3 xformOp:translate = (5, 0, 0)
    uniform token[] xformOpOrder = ["xformOp:translate"]
  }
}

def PointInstancer "Points"
{
  point3f[] positions = [(0, 0, 0), (5, 0, 0)]
  int[] protoIndices = [0, 0]
  rel prototypes = </Points/Protos/Tree>

  class "Protos"
  {
    def "Tree" (prepend references = @tree.usda@)
    {
    }
  }
}
```

- Native: one prim per tree; each carries its own transform and could carry its own variant selection.
- Point: one prim for all trees; the transforms are array entries.

Python example. The script builds 1,000 copies of a 4-prim tree asset in each of the three styles and counts the prims each stage populates.

```
from pxr import Usd, UsdGeom, Sdf

N = 1000
asset = Usd.Stage.CreateNew("tree.usda")
root = UsdGeom.Xform.Define(asset, "/Tree")
asset.SetDefaultPrim(root.GetPrim())
for part in ("Trunk", "Crown", "Roots"):
    UsdGeom.Cube.Define(asset, f"/Tree/{part}")
asset.Save()

def count(stage):
    scene = len(list(stage.Traverse()))
    protos = sum(len(list(Usd.PrimRange(p))) for p in stage.GetPrototypes())
    return scene, protos

def build(style):
    stage = Usd.Stage.CreateInMemory()
    UsdGeom.Xform.Define(stage, "/World")
    if style == "point":
        pi = UsdGeom.PointInstancer.Define(stage, "/World/Trees")
        proto = stage.DefinePrim("/World/Trees/Protos/Tree")
        stage.GetPrimAtPath("/World/Trees/Protos").SetSpecifier(Sdf.SpecifierClass)
        proto.GetReferences().AddReference("tree.usda")
        pi.CreatePrototypesRel().SetTargets([proto.GetPath()])
        pi.CreateProtoIndicesAttr([0] * N)
        pi.CreatePositionsAttr([(i * 3.0, 0, 0) for i in range(N)])
        return stage
    for i in range(N):
```



```

    prim = stage.DefinePrim(f"/World/Tree_{i:04d}")
    prim.GetReferences().AddReference("tree.usda")
    prim.SetInstanceable(style == "native")
    return stage

for style in ("none", "native", "point"):
    scene, protos = count(build(style))
    print(f"{style:6s} scene prims: {scene:5d}  prototype prims: {protos}")

```

Expected output

```

none  scene prims:  4001  prototype prims:  0
native scene prims: 1001  prototype prims:  4
point  scene prims:    2  prototype prims:  0

```

No instancing populates 4 prims per tree. Native instancing populates 1 prim per tree plus one shared 4-prim prototype. The PointInstancer populates 2 prims in total (its prototypes sit under a class, so `Traverse()` skips them).

Real-world use case. An architecture (AEC) firm models a 40-floor office tower. The lobby's sculpture is unique (no instancing). The 3,000 identical desks are native instances, so facilities staff can select a desk and switch its `config` variant. The 120,000 ceiling tiles and bolts are PointInstancers per floor.

Common mistakes.

MISTAKE

Using PointInstancer for objects that each need a different variant selection or must be selectable by path. Fix: use native instancing for those.

MISTAKE

Instancing every leaf mesh of an asset individually. Prim count does not drop. Fix: make the asset root instanceable.

MISTAKE

Giving each native instance a slightly different variant selection "for variety". Each unique combination creates another prototype, eroding the savings. Fix: limit variety to a few combinations, or vary only things authored on the instance root (transform, primvars, binding).

Exam traps.

TRAP

"Native instancing has zero per-instance prims." No: each native instance is still one prim. Only point instancing removes per-copy prims.

TRAP

"Point instancing is always better because it is cheaper." Not when copies need identity, per-copy variant selections, or path-based overrides.

Exam takeaways.

KEY

- Few and unique → no instancing; many with identity → native; huge counts → PointInstancer. - Native: 1 prim per copy. Point: 0 prims per copy. - Different variant selections on native instances → more prototypes. - Instance at the asset level, not per leaf mesh.

26.2 Payloads for scalability

What is it? A **payload** is a composition arc you can choose not to load (Chapter 17). For large scenes, each asset's heavy geometry sits behind a payload, so a stage can open **unloaded**. You then load only the **working set** a task needs.

Why do we need it? A city with 10,000 assets may take minutes to open fully. Many tasks, such as layout, checking what is in the scene, or editing transforms, need only the model hierarchy. With payloads, the scene opens in seconds, and each user loads only their part.

Beginner explanation. Payloads are lazy-loaded boxes in storage. Each box has a label on the outside (name, kind, size, available variants). You can arrange the boxes on the warehouse floor plan by reading labels alone, and open just the boxes you need to work inside.

Where the analogy breaks: anything written *inside* the box (inside the payload layer) is invisible until you load it. Information you want while unloaded, like the bounding box, must be written on the label (the asset interface layer).

Technical explanation. - Open unloaded: `Usd.Stage.Open(path, Usd.Stage.LoadNone)`. Load parts later: `stage.Load(path)`, `stage.Unload(path)`, or `stage.LoadAndUnload(loadSet, unloadSet)`. Inspect with `stage.GetLoadSet()` and `stage.FindLoadable()`. - Unloaded prims exist, but their payload contents do not. `prim.IsLoaded()` is `False`. - `stage.Traverse()` uses the default predicate, which **includes `IsLoaded`**, so it skips unloaded payload prims. Use `stage.TraverseAll()` or `Usd.PrimRange.Stage(stage, Usd.PrimIsActive & Usd.PrimIsDefined)` to see the unloaded model hierarchy. - Recommended asset structure (Chapter 23 and Maximizing USD Performance): a small `.usda` **asset interface** file containing `kind`, `assetInfo`, variant sets, and an `extentsHint` (the “rest bounding box”), plus a payload to binary `.usdc` files holding geometry and shading. - Copying information from inside the payload up onto the interface prim is sometimes called **lofting**. `extentsHint` is the classic example (26.3). - Population masks (`Usd.Stage.OpenMasked`, Chapter 17) are a complementary tool: they limit which *paths* are composed at all.

Mental model.

```
city.usda (assembly, opened LoadNone)
+-- Block_A ref -> building.usda [kind, assetInfo, extentsHint] <- always there
|                               payload -> building_geo.usdc      <- loaded on demand
+-- Block_B ref -> building.usda ...
Open fast with LoadNone -> see hierarchy + bounds
stage.Load("/City/Block_A") -> pull in only that geometry
```

Simple example. A city of three buildings. Opened with `LoadNone`, the stage shows `/City/B0`, `/City/B1`, `/City/B2` and their kinds, but no meshes. A layout artist loads only `B1` to fix its roof.

USDA example. File: *building.usda* (asset interface)

```
#usda 1.0
(
    defaultPrim = "Building"
)

def Xform "Building" (
    prepend apiSchemas = ["GeomModelAPI"]
    assetInfo = {
        string name = "Building"
    }
    kind = "component"
    prepend payload = @./building_geo.usdc@
)
{
```

```
float3[] extentsHint = [(-5, 0, -5), (5, 30, 5)]
}
```

- prepend payload points at the heavy binary file.
- kind, assetInfo, and extentsHint live in the interface, so they are available when the payload is unloaded.

Python example.

```
from pxr import Usd, UsdGeom, Kind, Sdf

geo = Usd.Stage.CreateNew("building_geo.usdc")
geo.SetDefaultPrim(UsdGeom.Xform.Define(geo, "/Building").GetPrim())
UsdGeom.Cube.Define(geo, "/Building/Body").CreateSizeAttr(10.0)
geo.Save()

iface = Usd.Stage.CreateNew("building.usda")
b = UsdGeom.Xform.Define(iface, "/Building")
iface.SetDefaultPrim(b.GetPrim())
Usd.ModelAPI(b.GetPrim()).SetKind(Kind.Tokens.component)
b.GetPrim().GetPayloads().AddPayload("./building_geo.usdc")
iface.Save()

city = Usd.Stage.CreateNew("city.usda")
UsdGeom.Xform.Define(city, "/City")
for i in range(3):
    p = UsdGeom.Xform.Define(city, f"/City/B{i}")
    p.GetPrim().GetReferences().AddReference("building.usda")
    p.AddTranslateOp().Set((i * 20.0, 0, 0))
city.Save()

def files(stage):
    return sorted(Sdf.Layer.GetDisplayNameFromIdentifier(l.identifier)
                  for l in stage.GetUsedLayers() if not l.anonymous)

stage = Usd.Stage.Open("city.usda", Usd.Stage.LoadNone)
print("Traverse: ", [p.GetName() for p in stage.Traverse()])
print("TraverseAll:", [p.GetName() for p in stage.TraverseAll()])
print("loadable:", [str(p) for p in stage.FindLoadable()])
print("layers unloaded:", files(stage))
stage.Load("/City/B1")
print("loaded set:", [str(p) for p in stage.GetLoadSet()])
b1 = stage.GetPrimAtPath("/City/B1")
print("B1 children:", [c.GetName() for c in b1.GetChildren()])
print("layers after load:", files(stage))
```

Expected output

```
Traverse:  ['City']
TraverseAll: ['City', 'B0', 'B1', 'B2']
loadable: ['/City/B0', '/City/B1', '/City/B2']
layers unloaded: ['building.usda', 'city.usda']
loaded set: ['/City/B1']
B1 children: ['Body']
layers after load: ['building.usda', 'building_geo.usdc', 'city.usda']
```

Notice that `Traverse()` stops at unloaded prims, while `TraverseAll()` shows the model hierarchy. The heavy `.usdc` layer is opened only after a load.

Real-world use case. A manufacturing digital twin of a car plant has 9,000 robot and conveyor assets. Each is an interface file plus a payload. A process engineer opens the plant unloaded in two seconds, finds line 4 by its `assetInfo`, and loads only the 60 assets on that line for a reach study.

Common mistakes.

MISTAKE

Putting ``kind``, ``assetInfo``, or ``extentsHint`` inside the payloaded layer. They vanish when unloaded. Fix: author them in the interface layer, on the prim that holds the payload.

MISTAKE

Using `stage.Traverse()` on an unloaded stage and concluding "the scene is empty". Fix: `stage.TraverseAll()` or a custom predicate without `isLoading`.

Exam traps.**TRAP**

"Payloads are weaker than references, so they load later." Strength (LIVERPS) and loading are separate ideas. Payload strength is about which opinion wins; the load/unload choice is about whether the arc is composed.

TRAP

"Unloaded prims are removed from the stage." No; the prim holding the payload remains with its interface opinions. Only the payload's contents are missing.

Exam takeaways.**KEY**

- Heavy data behind payloads; open with `LoadNone`; `Load` the working set. - Interface layer = kind, assetInfo, variants, extentsHint, payload arc. - `Traverse()` skips unloaded prims; `TraverseAll()` does not. - Payload strength (LIVERPS) and loading are unrelated.

26.3 Draw modes (UsdGeom.ModelAPI) and extents hints

What is it? `UsdGeom.ModelAPI` is an applied API schema (written `GeomModelAPI` in USDA) for **model** prims (prims with a kind). It provides **draw modes**, cheap stand-ins a viewer can draw instead of full geometry, and **extentsHint**, a cached bounding box for the whole model.

Why do we need it? Drawing 10,000 detailed buildings far from the camera wastes time. A box or a few textured cards look almost the same at a distance. Bounds are needed for framing, culling, and stand-ins. Computing them by visiting every mesh is slow, and impossible if the payload is unloaded. `extentsHint` caches them on the model.

Beginner explanation. Theatre backdrops. Distant buildings on stage are painted flat boards (cards), and furniture placeholders are taped outlines on the floor (bounds). The tape outline is also written on the label of each crate (`extentsHint`), so stagehands know how big the crate is without opening it.

Where the analogy breaks: a draw mode is only a **request** to the viewer or renderer (through Hydra/UsdImaging). The real geometry is still in the scene and still renders in tools that ignore draw modes.

Technical explanation. Draw mode attributes (all uniform):

Attribute	Type	Fallback	Values / meaning
<code>model:drawMode</code>	token	inherited	origin (axes), bounds (box), cards (textured quads), default (full geometry), inherited (use the parent's)
<code>model:applyDrawMode</code>	bool	false	Whether this prim actually switches to the resolved draw mode
<code>model:drawModeColor</code>	float3	(0.18, 0.18, 0.18)	Color of the stand-in
<code>model:cardGeometry</code>	token	cross	cross, box, or fromTexture

Attribute	Type	Fallback	Values / meaning
<code>model:cardTextureXPos ... ZNeg</code>	asset	—	One texture per card face

- `ComputeModelDrawMode(parentDrawMode="")` resolves `inherited` by walking up to the nearest ancestor with an authored value; the root fallback is `default`.
- The stand-in is applied at a prim where `model:applyDrawMode` is true **or** (per the 26.08 schema documentation) where the prim has kind `component` and `applyDrawMode` is not authored. Below that prim, the full geometry is replaced.
- Draw modes are interpreted by imaging (UsdImaging/Hydra, e.g. `usdview`). `usd-core` has no imaging, so in Python you can author and resolve them, not see them.

Extents hint:

- `extentsHint` is a `float3[]` attribute on the model prim. It holds pairs `[min, max]`, one pair per purpose in the order `default`, `render`, `proxy`, `guide`. Trailing purposes with no geometry are dropped; an empty purpose in the middle is stored as an inverted (empty) box.
- `ComputeExtentsHint(bboxCache)` computes the value; `SetExtentsHint(extents, time)` authors it; `GetExtentsHint(time)` reads it.
- `UsdGeom.BBoxCache(time, purposes, useExtentsHint=True)` uses authored hints instead of visiting children. This is fast and works on unloaded payloads, but the hint is a **cache**: if geometry changes and nobody recomputes the hint, the bounds are stale.
- `extentsHint` is for models. A single gprim's own bounds live in `extent` (Chapter 13).

VERSION

Verified on USD 26.08. The installed schema documentation says the draw mode is applied where ``model:applyDrawMode`` is true, *or* on ``component`-kind` prims where ``applyDrawMode`` is not authored. Older documentation and tutorials say that ``applyDrawMode`` must be set to true. Setting it explicitly works in both cases. This is imaging behavior that cannot be observed with ``usd-core``; only the schema text was checked.

Mental model.

```
/City (assembly)          model:drawMode = "bounds"      <- sets the request
/City/B0 (component)      drawMode: inherited -> bounds    applyDrawMode = true
    Body, Roof ...        replaced by a box drawn from B0's bounds
/City/B1 (component)      drawMode = "default"           <- opts out: full geometry
extentsHint on B0: [dflt min, dflt max, render min, render max, proxy min, proxy max]
```

Simple example. A city assembly sets `model:drawMode = "cards"`. Every building inherits `cards` and shows as textured cards. The hero building next to the camera authors `model:drawMode = "default"` and renders fully.

USDA example.

```
#usda 1.0

def Xform "City" (
  prepend apiSchemas = ["GeomModelAPI"]
  kind = "assembly"
)
{
  uniform token model:drawMode = "cards"

  def Xform "Tower" (
    prepend apiSchemas = ["GeomModelAPI"]
    kind = "component"
  )
```

```

{
    float3[] extentsHint = [(-5, 0, -5), (5, 40, 5)]
    uniform bool model:applyDrawMode = 1
    uniform asset model:cardTextureXPos = @tower_xpos.png@
    uniform token model:cardGeometry = "box"
}

def Xform "Hero" (
    prepend apiSchemas = ["GeomModelAPI"]
    kind = "component"
)
{
    uniform token model:drawMode = "default"
}
}

```

- `City` requests `cards` for everything below it.
- `Tower` inherits `cards`, applies it, and uses `box` card geometry with a texture on its +X face.
- `Hero` overrides to `default`, so it shows full geometry.

Python example.

```

from pxr import Usd, UsdGeom, Kind

stage = Usd.Stage.CreateInMemory()
city = UsdGeom.Xform.Define(stage, "/City")
Usd.ModelAPI(city.GetPrim()).SetKind(Kind.Tokens.assembly)
tower = UsdGeom.Xform.Define(stage, "/City/Tower")
Usd.ModelAPI(tower.GetPrim()).SetKind(Kind.Tokens.component)
UsdGeom.Cube.Define(stage, "/City/Tower/Body").CreateSizeAttr(4.0)
proxy = UsdGeom.Cube.Define(stage, "/City/Tower/Proxy")
proxy.CreatePurposeAttr(UsdGeom.Tokens.proxy)
proxy.AddTranslateOp().Set((0, 10, 0))

tm = UsdGeom.ModelAPI.Apply(tower.GetPrim())
print("fallback drawMode:", tm.ComputeModelDrawMode())
print("fallback applyDrawMode:", tm.GetModelApplyDrawModeAttr().Get())
UsdGeom.ModelAPI.Apply(city.GetPrim()).CreateModelDrawModeAttr("bounds")
tm.CreateModelApplyDrawModeAttr(True)
print("inherited from City:", tm.ComputeModelDrawMode())

cache = UsdGeom.BBoxCache(Usd.TimeCode.Default(),
                          UsdGeom.Imageable.GetOrderedPurposeTokens())
hint = tm.ComputeExtentsHint(cache)
tm.SetExtentsHint(hint)
print("purposes:", list(UsdGeom.Imageable.GetOrderedPurposeTokens()))
print("extentsHint pairs:", len(hint) // 2)
print("default box:", hint[0], hint[1])
print("proxy box:  ", hint[4], hint[5])

stage.GetPrimAtPath("/City/Tower/Body").GetAttribute("size").Set(8.0)
for use_hint in (False, True):
    c = UsdGeom.BBoxCache(Usd.TimeCode.Default(), ["default"],
                          useExtentsHint=use_hint)
    r = c.ComputeWorldBound(tower.GetPrim()).ComputeAlignedRange()
    print("useExtentsHint", use_hint, "->", r.GetMin(), r.GetMax())

```

Expected output

```

fallback drawMode: default
fallback applyDrawMode: False
inherited from City: bounds
purposes: ['default', 'render', 'proxy', 'guide']
extentsHint pairs: 3
default box: (-2, -2, -2) (2, 2, 2)
proxy box:   (-1, 9, -1) (1, 11, 1)
useExtentsHint False -> (-4, -4, -4) (4, 4, 4)
useExtentsHint True  -> (-2, -2, -2) (2, 2, 2)

```

The middle pair (render) is an inverted “empty” box because nothing has `render` purpose. After the body grows to size 8, the cache that trusts the hint still reports the old size-4 box: a stale hint. Recompute hints whenever you publish changed geometry.

Real-world use case. In an animated feature’s city set, the layout department sets `model:drawMode = "cards"` on the set’s assembly. Artists scrub through shots in usdview at interactive speed, seeing card stand-ins textured from pre-rendered views. The few buildings near camera are switched to `default`. Framing in every tool uses each building’s `extentsHint`, even with payloads unloaded.

Common mistakes.

MISTAKE

Forgetting to apply the schema: ``UsdGeom.ModelAPI(prim)`` without ``Apply`` gives you the API object but does not add ``GeomModelAPI`` to ``apiSchemas``. Fix: ``UsdGeom.ModelAPI.Apply(prim)`` when authoring.

MISTAKE

Publishing new geometry without recomputing ``extentsHint``. Bounds-based culling then clips the asset. Fix: recompute with ``ComputeExtentsHint`` in the publish step.

MISTAKE

Expecting draw modes to make a final render cheaper in every renderer. They are a viewer/imaging feature. Fix: use LOD variants (26.4) for renderer-independent savings.

Exam traps.

TRAP

``extentsHint`` vs ``extent``: ``extent`` is on a Boundable gprim (its own geometry); ``extentsHint`` is on a model and covers the whole subtree, per purpose.

TRAP

``model:drawMode`` fallback is ``inherited``, not ``default``. The **resolved** mode with nothing authored anywhere is ``default``.

TRAP

``bounds`` draws a box; ``origin`` draws only axes at the prim's origin; ``cards`` draws textured quads. Don't mix them up.

Exam takeaways.

KEY

- ``UsdGeom.ModelAPI.Apply(prim)`` → ``model:drawMode`` (``origin``/``bounds``/``cards``/``default``/``inherited``), ``model:applyDrawMode``, card attributes. - ``ComputeModelDrawMode()`` resolves inheritance; with nothing authored it returns ``default``. - ``extentsHint`` = per-purpose model bounds; ``ComputeExtentsHint(bboxCache)``, ``SetExtentsHint``, ``GetExtentsHint``. - Hints are caches: recompute when geometry changes.

26.4 LOD with variants

What is it? Level of detail (LOD) means providing lighter versions of an asset (for example high, medium, low) and choosing one per use. In USD you usually author LOD as a **variant set**, often named `lod`, with one variant per detail level (Chapter 18).

Why do we need it? A building 2 km from the camera does not need 2 million polygons. Swapping to a 2,000-polygon version saves memory and render time in every renderer. Draw modes (26.3) only help viewers that support them.

Beginner explanation. A variant set is a switch with named positions. The `lod` switch has positions `high`, `medium`, and `low`. Flip it per asset instance: hero buildings on `high`, background on `low`.

Where the analogy breaks: with payloads, each switch position can point at a different payload file, so the unused detail levels are never even opened.

Technical explanation. - Author one variant per level; inside each, put a different payload or reference (e.g. `building_high.usdc`, `building_low.usdc`), or different geometry. - Select per prim:

```
prim.GetVariantSets().GetVariantSet("lod").SetVariantSelection("low"), or variants = { string lod = "low" }
```

in USDA. A selection in a stronger layer (shot, layout) wins over the asset's default. - Variant **fallbacks** (Chapter 18) can set a pipeline-wide default when nothing is selected. - With **native instancing**, each distinct LOD selection produces a separate prototype. Two LOD levels in use → two prototypes. This is fine; just keep the number of combinations small. - With **point instancing**, give each LOD its own prototype (one prototype per LOD level) and choose via `protoIndices`, or put a `PointInstancer` per LOD band. - LOD by distance at render time is a renderer feature; USD itself stores the choices, it does not switch automatically by camera distance.

Mental model.

```
Building (variantSet "lod")
{lod=high} payload -> building_high.usdc (2M polys)
{lod=medium} payload -> building_med.usdc (200k)
{lod=low} payload -> building_low.usdc (2k)
instance near camera: lod=high instance far away: lod=low
native instances: one prototype per distinct selection in use
```

Simple example. A street has 100 identical native-instanced houses. The 10 closest select `lod=high`; the other 90 select `lod=low`. The stage has 2 prototypes and loads only the two files in use.

USD example.

```
#usda 1.0
(
  defaultPrim = "House"
)

def Xform "House" (
  kind = "component"
  variants = {
    string lod = "high"
  }
  prepend variantSets = "lod"
)
{
  variantSet "lod" = {
    "high" {
      def Sphere "Body"
      {
        double radius = 1
      }
    }
    "low" {
      def Cube "Body"
      {
        double size = 2
      }
    }
  }
}
}
```

- The asset defaults to `high`. A shot layer can author `variants = { string lod = "low" }` on a referencing prim.

- Here the variants hold inline geometry for brevity; production assets usually put a payload in each variant.

Python example.

```
from pxr import Usd, UsdGeom

asset = Usd.Stage.CreateNew("house.usda")
house = UsdGeom.Xform.Define(asset, "/House").GetPrim()
asset.SetDefaultPrim(house)
lod = house.GetVariantSets().AddVariantSet("lod")
for name, parts in (("high", 6), ("low", 1)):
    lod.AddVariant(name)
    lod.SetVariantSelection(name)
    with lod.GetVariantEditContext():
        for i in range(parts):
            UsdGeom.Cube.Define(asset, f"/House/Part{i}")
lod.SetVariantSelection("high")
asset.Save()

street = Usd.Stage.CreateInMemory()
for i in range(100):
    p = street.DefinePrim(f"/Street/House_{i:03d}")
    p.GetReferences().AddReference("house.usda")
    p.SetInstanceable(True)
    if i >= 10:
        # far houses use low LOD
        p.GetVariantSets().GetVariantSet("lod").SetVariantSelection("low")

protos = street.GetPrototypes()
print("prototypes:", len(protos))
for proto in sorted(protos, key=lambda p: len(p.GetChildren())):
    users = [i for i in proto.GetInstances()]
    print(" parts:", len(proto.GetChildren()), "instances:", len(users))
print("scene prims:", len(list(street.Traverse())))
```

Expected output

```
prototypes: 2
 parts: 1 instances: 90
 parts: 6 instances: 10
scene prims: 101
```

Real-world use case. A driving-simulator team (digital twin of a city for autonomous-vehicle testing) publishes every building with lod variants. The sensor simulation near the ego vehicle selects high; buildings beyond 300 m use low. Their scene layer writes only variant selections, so new LODs from the asset team appear automatically.

Common mistakes.

MISTAKE

Putting all LODs in one layer without payloads. The heavy `high` data is opened even when `low` is selected. Fix: put each level behind its own payload (or reference to a separate `.usdc` file).

MISTAKE

Inconsistent prim names across LOD variants (e.g. materials bound to `/House/HighBody`). Overrides from shots then miss on some LODs. Fix: keep the same hierarchy and names in every level.

Exam traps.

TRAP

"USD switches LOD automatically by camera distance." No. USD stores variants; the pipeline or renderer chooses.

TRAP

"Variant selections break instancing." They don't. Instances with the same selection still share a prototype; you get one prototype per distinct selection.

Exam takeaways.**KEY**

- LOD = a variant set (``lod``) with one variant per level, ideally each with its own payload. - Select per instance; stronger layers override the asset default. - Native instances: one prototype per distinct LOD selection. - USD does not choose LOD by distance; your pipeline or renderer does.

26.5 Aggregation decision table

What is it? **Content aggregation** is combining many assets into larger scenes: components into assemblies, assemblies into sets, sets into shots or worlds. This section collects the choices from Part IV into one decision table, and adds the official **Maximizing USD Performance** guidance that shapes them.

Why do we need it? Exam questions in this domain are usually scenarios: “a city with 50,000 buildings must open in under a minute” or “an artist must recolor one instance”. You need a fast way to map the scenario to the right tool.

Beginner explanation. Think of packing for a move. Big furniture you need right away goes in the truck whole (unique prims). Boxes of identical dishes are labelled and stacked (native instances). A bag of 10,000 screws is one bag with a count, not 10,000 entries on your list (PointInstancer). Rooms you won't use for a month stay sealed in storage (unloaded payloads).

Where the analogy breaks: in USD, you can change the packing later without moving anything. Load rules, variant selections, and masks are just opinions in a layer.

Technical explanation. Decision table

Need	Tool	Chapter
Reuse one asset in many places	Reference the asset interface	16, 23
Open huge scenes fast; load per task	Payloads + <code>LoadNone</code> + <code>Load</code>	17, 26.2
Restrict which paths are composed at all	Population mask (<code>OpenMasked</code>)	17
Many identical copies, each a selectable prim	Native instancing	24
Recolor/retransform one native instance without breaking instancing	Opinions on the instance root (primvars, binding, xform)	24
Millions of copies, transform-only variation	PointInstancer	25
Hide some point instances	<code>invisibleIds</code> / <code>inactiveIds</code>	25
Swap detail level	<code>lod</code> variant set	18, 26.4
Fast viewer stand-ins	<code>model:drawMode</code> + <code>extentsHint</code>	26.3
Organize assets into a navigable tree	Kinds and model hierarchy (component / assembly / group)	6, 23

Maximizing USD Performance (official guidance), applied to aggregation

- **Prefer crate files.** Use binary `.usdc` for anything bigger than a few small definitions. A new layer named `*.usd` is crate by default. USDA is fine for small interface or assembly files that mostly reference other files.

- **Monitor layer count.** Open cost grows with the number of files. Asset structures tend to grow from three or four layers to ten or more, multiplied by every unique asset. Collapse layers at publish time where workflows allow. Count with `stage.GetUsedLayers()`. Do not respond by flattening whole sets into one file; that throws away referencing.
- **Minimize prim count.** Cost scales with prims far more than with properties, because every prim must be indexed for composition.
- **Use transformable gprims.** A Mesh (any gprim) has its own `xformOps`. Don't wrap each mesh in an `Xform` just to move it. The docs report 40–50% fewer prims in typical scenes.
- **Instance at higher granularity.** Instancing each leaf gprim gives zero prim-count reduction and adds an arc per leaf. Instance whole assets (and add a level on top if you already instance finer).
- **Prefer property namespaces (:)** for organizing data, instead of extra prims.
- **Package assets with payloads** so that scenes can open with only the model hierarchy.
- The same page also recommends building or running USD with a multithreading-friendly allocator such as `jemalloc` (a build issue, covered in Chapter 44).

Mental model.

SCALE ->	1	100	10,000	1,000,000+
style	unique	native instances		PointInstancer
loading	always	payloads, load the working set		
viewing	full	LOD variants / drawMode + extentsHint		
files	few USDA	interface .usda + payload .usdc (crate)		
prims	minimize: transformable gprims, instance whole assets			

Simple example. A theme-park digital twin: unique hero rides (no instancing, payloaded), 4,000 benches and lamps (native instances of payloaded assets with `lod`), 600,000 plants (PointInstancers per zone), and a `bounds` draw mode on distant zones in the review viewer.

USDA example. An assembly that combines the patterns (the referenced files are placeholders; the layer parses on its own):

```
#usda 1.0
(
    defaultPrim = "Park"
)

def Xform "Park" (
    prepend apiSchemas = ["GeomModelAPI"]
    kind = "assembly"
)
{
    uniform token model:drawMode = "bounds"

    def "Ride" (prepend references = @ride.usda@)
    {
        uniform token model:drawMode = "default"
    }

    def "Bench_001" (instanceable = true
        prepend references = @bench.usda@
        variants = {
            string lod = "low"
        })
    {
    }

    def PointInstancer "Plants"
    {
        point3f[] positions = [(0, 0, 0), (1, 0, 0)]
        int[] protoIndices = [0, 0]
        rel prototypes = </Park/Plants/Protos/Shrub>
```

```

class "Protos"
{
    def "Shrub" (prepend references = @shrub.usda@)
    {
    }
}
}

```

Python example. Two of the prim-count guidelines, measured: transformable gprims versus Xform-wrapped meshes, and instancing at the asset level versus at each leaf. The script also checks that a new `.usd` layer is crate.

```

from pxr import Usd, UsdGeom, Sdf

leg = Usd.Stage.CreateNew("leg.usd")          # .usd -> crate by default
leg.SetDefaultPrim(UsdGeom.Cube.Define(leg, "/Leg").GetPrim())
leg.Save()
print("leg.usd header:", open("leg.usd", "rb").read(8))

asset = Usd.Stage.CreateNew("chair.usd")
asset.SetDefaultPrim(UsdGeom.Xform.Define(asset, "/Chair").GetPrim())
for i in range(4):
    asset.DefinePrim(f"/Chair/Leg{i}").GetReferences().AddReference("leg.usd")
asset.Save()

wrapped = Usd.Stage.CreateInMemory()
direct = Usd.Stage.CreateInMemory()
for i in range(100):
    UsdGeom.Xform.Define(wrapped, f"/Rock{i}").AddTranslateOp().Set((i, 0, 0))
    UsdGeom.Cube.Define(wrapped, f"/Rock{i}/Geo")
    UsdGeom.Cube.Define(direct, f"/Rock{i}").AddTranslateOp().Set((i, 0, 0))
print("Xform+Cube prims:", len(list(wrapped.Traverse())))
print("transformable gprim prims:", len(list(direct.Traverse())))

def room(granularity):
    stage = Usd.Stage.CreateInMemory()
    for i in range(50):
        chair = stage.DefinePrim(f"/Room/Chair{i}")
        chair.GetReferences().AddReference("chair.usd")
        if granularity == "asset":
            chair.SetInstanceable(True)
        else:
            for part in chair.GetChildren():
                # instance each leg gprim
                part.SetInstanceable(True)
    return len(list(stage.Traverse())), len(stage.GetPrototypes())

print("leaf-level instancing (prims, prototypes):", room("leaf"))
print("asset-level instancing (prims, prototypes):", room("asset"))
chair_stage = Usd.Stage.Open("chair.usd")
used = [l for l in chair_stage.GetUsedLayers() if not l.anonymous]
print("layers used by chair.usd:", sorted(l.GetDisplayName() for l in used))

```

Expected output

```

leg.usd header: b'PXR-USDC'
Xform+Cube prims: 200
transformable gprim prims: 100
leaf-level instancing (prims, prototypes): (251, 1)
asset-level instancing (prims, prototypes): (51, 1)
layers used by chair.usd: ['chair.usd', 'leg.usd']

```

Leaf-level instancing leaves the prim count where it was; asset-level instancing cuts it to one prim per chair.

Real-world use case. A games studio's open-world city originally published each building as eleven layers with an Xform above every mesh. After a publish step that collapsed layers to an interface `.usda` plus one `.usdc` payload, removed redundant Xforms, and instanced buildings at the asset level, the world's open time in the editor dropped from minutes to seconds.

Common mistakes.**MISTAKE**

"Optimizing" by flattening the whole set into one giant file. You lose referencing, instancing structure, and collaborative layering. Fix: collapse layers per asset at publish; keep references between assets.

MISTAKE

Storing large geometry in `.usda``. It opens slower and uses more memory. Fix: `.usdc`` for heavy data; keep USDA for small interface/assembly files.

MISTAKE

Wrapping every mesh in an ``Xform``. Fix: put ``xformOp`s` on the mesh itself.

Exam traps.**TRAP**

"Property count is the main cost." The guidance says cost scales with `**prim**` count much more than property count.

TRAP

"Instancing each mesh gives the biggest savings." Gprim-level instancing gives no prim-count reduction; instance at a higher granularity.

TRAP

"Monitor layer count" does not mean "flatten everything". It means keep the number of published layers per asset small.

Exam takeaways.**KEY**

- Scale picks the style: unique → native → PointInstancer. - Payloads + ``LoadNone`` for big scenes; LOD variants and draw modes for viewing cost. - Prefer crate (``usdc``; ``usd`` defaults to crate); keep layer count per asset low (``GetUsedLayers``). - Minimize prims: transformable gprims, instance at asset level, use property namespaces.

Chapter lab(s)

There is no lab dedicated to this chapter in the lab table. Practice its ideas with **Lab 15 — Payloads, load rules, population masks** (Ch 17), **Lab 22 — Build a component asset and an assembly** (Ch 23), **Lab 23 — Native instancing and instance proxies** (Ch 24), and **Lab 24 — PointInstancer: add prototypes, hide instances** (Ch 25). As an extension, add an `extentsHint` to the Lab 22 component and open the assembly with `LoadNone`.

USDA reading exercises

Exercise 26-A. What does `ComputeModelDrawMode()` return for `/Set/Tree` and for `/Set/Hut`?

```
#usda 1.0

def Xform "Set" (
  prepend apiSchemas = ["GeomModelAPI"]
  kind = "assembly"
)
{
  uniform token model:drawMode = "origin"

  def Xform "Tree" (
    prepend apiSchemas = ["GeomModelAPI"]
```

```

        kind = "component"
    )
    {
    }

    def Xform "Hut" (
        prepend apiSchemas = ["GeomModelAPI"]
        kind = "component"
    )
    {
        uniform token model:drawMode = "bounds"
    }
}

```

Exercise 26-B. This scene has three native instances of `rock.usda`. How many prototypes will the stage have? (`rock.usda` has a `size` variant set with variants `big` and `small`.)

```

#usda 1.0

def Xform "Quarry"
{
    def "RockA" (instanceable = true
        prepend references = @rock.usda@
        variants = {
            string size = "big"
        })
    {
    }

    def "RockB" (instanceable = true
        prepend references = @rock.usda@
        variants = {
            string size = "small"
        })
    {
    }

    def "RockC" (instanceable = true
        prepend references = @rock.usda@
        variants = {
            string size = "big"
        })
    {
    }
}

```

Answers

- **26-A.** `/Set/Tree` returns `origin`: its `drawMode` is not authored (fallback inherited), so it takes the nearest ancestor value. `/Set/Hut` returns `bounds`, its own authored value.
- **26-B.** Two prototypes: `RockA` and `RockC` compose identically (`size=big`) and share one; `RockB` (`size=small`) gets another.

Chapter review

Summary

- Pick the instancing style by scale and editability: unique prims, native instances, or a `PointInstancer`.
- Native instancing keeps one prim per copy; point instancing has none. Distinct variant selections create extra prototypes.
- Put heavy data behind payloads; open with `LoadNone`; load the working set.
- `Traverse()` skips unloaded prims; use `TraverseAll()` to see the model hierarchy.
- The asset interface holds `kind`, `assetInfo`, `variants`, `extentsHint`, and the payload.

- `UsdGeom.ModelAPI: model:drawMode` (`origin`, `bounds`, `cards`, `default`, `inherited`), `model:applyDrawMode`, card attributes.
- `extentsHint` is a per-purpose cached bound; recompute it at publish.
- LOD = a lod variant set, ideally with a payload per level.
- Performance guidance: crate files, few layers per asset, few prims (transformable gprims, asset-level instancing, property namespaces).

If you see... → think...

If you see...	Think...
"Millions of copies"	<code>PointInstancer</code>
"Each copy selectable, own variant"	Native instancing
"Scene must open quickly; work on a part"	<code>Payloads</code> , <code>LoadNone</code> , <code>Load</code>
"Bounds while unloaded"	<code>extentsHint</code> in the interface layer
"Stand-in in the viewer"	<code>model:drawMode</code> (<code>bounds</code> , <code>cards</code> , <code>origin</code>)
<code>inherited</code> draw mode	Resolve from nearest ancestor; root → <code>default</code>
"Lighter version far from camera"	<code>lod</code> variant set
"Open time and memory too high"	Prim count first, then layer count, then format
An <code>Xform</code> above every mesh	Use transformable gprims
Instancing at the mesh level	Instance at the asset level

Review questions

CA-R26-01 · Obj 1.2 · Medium · Single choice A forest of 3 million trees varies only in position, rotation, scale, and one of 6 species. Which representation is best? A. 3 million native instances B. One `PointInstancer` with 6 prototypes C. 3 million unique prims D. A variant set with 3 million variants

CA-R26-02 · Obj 1.2 · Medium · Single choice Which requirement rules out point instancing for a set of 40 street cars? A. The cars share one asset B. Each car must be selectable by path and have its own `damage` variant selection C. The cars must be positioned in world space D. The cars must render with motion blur

CA-R26-03 · Obj 2.4 · Medium · Select two A scene of 10,000 payloaded assets opens too slowly. Which actions help directly? Select two. A. Open the stage with `Usd.Stage.LoadNone` and load only the working set B. Move `extentsHint` into the payloaded layer C. Store heavy geometry in `.usdc` instead of `.usda` D. Wrap each mesh in its own `Xform` E. Instance each leaf mesh instead of each asset

CA-R26-04 · Obj 2.4 · Easy · Single choice Which `model:drawMode` value draws textured quads? A. `origin` B. `bounds` C. `cards` D. `default`

CA-R26-05 · Obj 2.4 · Medium · Code reading

```
cache = UsdGeom.BBoxCache(Usd.TimeCode.Default(), ["default"], useExtentsHint=True)
box = cache.ComputeWorldBound(stage.GetPrimAtPath("/City/Tower"))
```

The Tower's geometry was enlarged yesterday, but its `extentsHint` was authored last month. What does `box` report? A. The new, larger bounds B. The old bounds from the hint C. An error, because hints are deprecated D. An empty box

(This snippet needs an existing stage, so it is not run.)

CA-R26-06 · Obj 2.4 · Medium · Single choice Where does `UsdGeom.ModelAPI.ComputeExtentsHint` take its bounds from, and what does it return? A. From the prim's extent only; one box B. From a `UsdGeom.BBoxCache` passed in; a `float3` array with one min/max pair per purpose C. From the payload file on disk; a `Gf.Range3d` D. From `model:drawMode`; a token

CA-R26-07 · Obj 1.2 · Hard · Single choice 200 native instances of a lamp: 120 select lod=low, 80 select lod=high, and 10 of the high ones also author primvars:displayColor on the instance root. How many prototypes are there? A. 1 B. 2 C. 3 D. 12

CA-R26-08 · Obj 2.4 · Easy · Single choice According to the Maximizing USD Performance guidance, what does “leverage transformable gprims” mean? A. Animate every gprim B. Put transform ops directly on the gprim instead of adding a parent Xform only to move it C. Convert gprims to PointInstancers D. Use xformOp:transform matrices only

CA-R26-09 · Obj 2.4 · Medium · Select two Which statements about stage.Traverse() on a stage opened with LoadNone are true? Select two. A. It skips prims whose payloads are unloaded B. It loads payloads as it visits them C. stage.TraverseAll() includes the unloaded model prims D. It raises an error E. It visits the contents of the payloads

CA-R26-10 · Obj 2.4 · Medium · Single choice A distant building should cost less to render in every renderer, not just in usdview. Which is the most direct choice? A. model:drawMode = "bounds" B. Select lod=low from a lod variant set C. Increase extentsHint D. Set model:applyDrawMode = false

Answers and explanations

- **CA-R26-01 — B.** Huge count, transform and prototype choice only: a PointInstancer with one prototype per species. A and C create millions of prims; D misuses variants. Review: §26.1.
- **CA-R26-02 — B.** Point instances have no paths and no per-instance variant selections. A, C, D are all possible with a PointInstancer (it supports velocities for motion blur). Review: §26.1.
- **CA-R26-03 — A, C.** Loading less and using crate both cut cost. B hides bounds while unloaded; D and E add or keep prims. Review: §26.2, §26.5.
- **CA-R26-04 — C.** cards = textured quads; bounds = box; origin = axes; default = full geometry. Review: §26.3.
- **CA-R26-05 — B.** With useExtentsHint=True, the cache trusts the authored hint, which is stale (verified by the §26.3 Python example). Review: §26.3.
- **CA-R26-06 — B.** ComputeExtentsHint(bboxCache) returns a Vt.Vec3fArray of min/max pairs in purpose order: default, render, proxy, guide. Review: §26.3.
- **CA-R26-07 — B.** Primvars and other opinions on the instance root do not change the prototype; only the two LOD selections differ. Review: §26.4 and Chapter 24.
- **CA-R26-08 — B.** Gprims are Xformable; extra Xform parents just add prims. Review: §26.5.
- **CA-R26-09 — A, C.** The default predicate includes IsLoaded, so unloaded prims are skipped; TraverseAll() uses no such filter. Nothing is loaded automatically. Review: §26.2.
- **CA-R26-10 — B.** LOD variants change the actual scene data. Draw modes are a viewer/imaging stand-in; extentsHint is only a bound. Review: §26.4.

Part V — Data Exchange

CHAPTER 27

Chapter 27 — Data Exchange Concepts

Exam domain: Data Exchange (15%) · **Objectives:** 4.1, 4.2, 4.4 (also 7.1) · **Study day:** 9 · **Est. time:** 100 min **Prerequisites:** Ch 2 (stage metadata), Ch 5 (properties), Ch 11 (primvars), Ch 12 (metadata), Ch 13 (extents); Ch 40 (UsdShade) is previewed here and taught fully later

Learning goals

- Explain the difference between importing, exporting, and converting, and name the three ways foreign data reaches USD.
- Describe the extraction → transformation → validation pipeline and what happens in each stage.
- Write a conceptual data mapping document using a reusable template table.
- Map USD geometry and `UsdPreviewSurface` materials to glTF, and list what glTF cannot carry.
- Map UsdShade concepts to MaterialX concepts.
- Design a round-trip pipeline that does not silently lose data.

Key terms

Term	One-line definition
Data exchange	Moving scene data between USD and another tool or file format
DCC	“Digital content creation” application, such as Maya, Houdini, Blender, or a CAD tool
Export	Writing data from a tool’s own format into USD
Import	Reading USD into a tool’s own in-memory format
Conversion	Translating one file into another file, usually by a standalone script, with no DCC open
Translator	General name for an exporter, importer, or converter
Fidelity	How much of the original meaning survives the translation
Lossy / lossless	A translation that drops or changes information / one that keeps everything
Data model	The set of concepts a format can express (meshes, materials, layers, ...)
Data mapping document	A table that says, for every source concept, what it becomes in the target and what is lost
Round trip	USD → DCC → USD (or the reverse) with no unintended changes
glTF	Khronos “GL Transmission Format” 2.0: a compact JSON + binary format for real-time 3D delivery
MaterialX	An open standard (ASWF) for describing material and shading networks
File format plugin	A plugin (<code>SdfFileFormat</code>) that lets USD open a non-USD file directly as a layer

27.1 Import, export, conversion

What is it? **Data exchange** is moving scene data between USD and something else.

Export writes a tool’s data into USD. **Import** reads USD into a tool. **Conversion** translates a file into another file with a standalone program.

Why do we need it? Every tool has its own **data model**: Maya thinks in nodes, a game engine thinks in GPU buffers, a CAD tool thinks in exact curved surfaces. Without a common format, every pair of tools needs its own translator. USD is designed to be the common hub, but data still has to be translated into and out of it.

Beginner explanation. Picture an international conference with one shared language. Each delegate needs one interpreter to and from the shared language, not one interpreter per other delegate. USD is the shared language, and each tool’s exporter and importer is its interpreter.

Where the analogy breaks: human languages can describe almost anything. 3D formats cannot. If glTF has no word for “variant set”, the interpreter has no way to say it, so the information is dropped. That is **loss**.

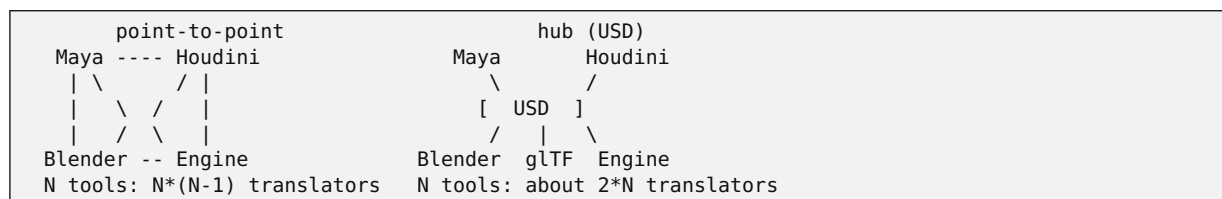
Technical explanation. Foreign data reaches USD in three ways:

1. **DCC exporter / importer.** Code inside the DCC walks the DCC’s scene and calls the `Usd/Sdf` API to write layers (export), or opens a `Usd.Stage` and builds DCC objects (import). Examples: the Maya USD plug-in, Houdini Solaris, Blender’s built-in USD import/export.
2. **Standalone converter.** A script reads file A and writes file B, for example OBJ → USDA or USD → glTF. Chapter 29 writes one.
3. **File format plugin.** A C++ plugin derived from `SdfFileFormat` teaches `Sdf.Layer.FindOrOpen` to read a foreign extension directly, so `Usd.Stage.Open("model.abc")` works. Alembic (`usdAlec`) and MaterialX (`usdMtlx`) plugins ship with full USD builds, but not with `usd-core`. Chapter 38 covers writing one.

Directions are named from the DCC’s point of view: you “export to USD” and “import from USD”. Converting “USD to glTF” is an export from USD’s point of view.

Fidelity is measured, not assumed. A good translator reports what it could not carry, rather than dropping it silently.

Mental model.



Simple example. | Situation | Kind | Who does it | |-----|-----|-----| | An artist clicks “File → Export USD” in a DCC | Export | DCC exporter | | A web team turns `chair.usd` into `chair.glTF` on a build server | Conversion | Standalone script | | `Usd.Stage.Open("chair.abc")` works in a full USD build | Neither: direct read | File format plugin |

USDA example. A converted file should record where it came from. Layer-level `customLayerData` is a free-form dictionary for this.

```
#usda 1.0
(
  customLayerData = {
    string sourceFile = "chair.obj"
    string translator = "obj2usd 1.2"
  }
  defaultPrim = "Chair"
  metersPerUnit = 1
  upAxis = "Y"
)

def Xform "Chair" (
```

```

    kind = "component"
)
{
    def Mesh "Geom"
    {
        int[] faceVertexCounts = [3]
        int[] faceVertexIndices = [0, 1, 2]
        point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
    }
}

```

- `customLayerData` holds provenance: the source file and the translator version. USD does not interpret it.
- `metersPerUnit` and `upAxis` record the converter's unit and axis decisions (Chapter 28).
- `defaultPrim` lets other files reference this one without a prim path (Chapter 16).

Python example. `usd-core` knows only USD's own formats. This script lists them, shows that `.obj` has no reader, and converts USDA to USDC with `Sdf.Layer.Export`, the Python equivalent of `usdcat -o`.

```

from pxr import Sdf, Usd, UsdGeom

print("formats:", sorted(Sdf.FileFormat.FindAllFileFormatExtensions()))
print("obj reader:", Sdf.FileFormat.FindByExtension("obj"))

stage = Usd.Stage.CreateNew("chair.usda")
UsdGeom.Cube.Define(stage, "/Chair")
stage.GetRootLayer().customLayerData = {"sourceFile": "chair.obj"}
stage.Save()

layer = Sdf.Layer.FindOrOpen("chair.usda")
layer.Export("chair.usdc")
crate = Sdf.Layer.FindOrOpen("chair.usdc")
print("usdc format:", crate.GetFileFormat().formatId)
print("provenance kept:", crate.customLayerData)

```

Expected output

```

formats: ['usd', 'usda', 'usdc', 'usdz']
obj reader: None
usdc format: usdc
provenance kept: {'sourceFile': 'chair.obj'}

```

USD-to-USD conversion (USDA ↔ USDC) is lossless. Conversion to a different data model, such as OBJ or glTF, is where loss happens.

Real-world use case. A furniture retailer receives CAD files from manufacturers. A conversion service tessellates the exact curved surfaces into meshes, writes USD for the internal pipeline, and later converts USD to glTF and USDZ for the website and mobile AR viewers. Each hop is a translator with its own loss report.

Common mistakes.

MISTAKE

Assuming `Usd.Stage.Open("model.obj")` works because "USD reads everything". It works only if a file format plugin for that extension is installed. In usd-core` it fails. Fix: write a converter (Chapter 29) or install the plugin.`

MISTAKE

Writing a converted file with no provenance. Months later nobody knows which source or translator version produced it. Fix: record both in `customLayerData`.`

Exam traps.

TRAP

"Converting USDA to USDC loses data." False. Both encode the same USD data model; only size, speed, and readability differ (Chapter 31).

TRAP

Watch the direction words. "Exporter to USD" writes USD; "importer" reads USD into a DCC. A question about writing a USD file from a DCC is about an exporter.

Exam takeaways.**KEY**

- Export writes USD from a tool; import reads USD into a tool; conversion is file-to-file. - Three routes into USD: DCC plugin, standalone converter, ``SdfFileFormat`` plugin. - ``usd-core`` reads only ``usd``, ``usda``, ``usdc``, ``usdz``. - USDA ↔ USDC is lossless; changing data model is where loss happens.

27.2 Extraction → transformation → validation

What is it? The standard three-stage shape of every data exchange pipeline: **extract** data from the source, **transform** it into the target's data model and conventions, then **validate** the result before anyone uses it.

Why do we need it? Mixing reading, fixing, and writing in one tangled loop makes bugs hard to find and fidelity impossible to measure. Separate stages give you clear places to test, log, and report loss. Without validation, broken files reach downstream departments and fail there, far from the cause.

Beginner explanation. Think of a factory that imports raw fruit and ships jam. First you unload the trucks (extract). Then you wash, cut, and cook (transform). Finally quality control checks every jar before shipping (validate). Nobody cooks on the loading dock.

Where the analogy breaks: in software, "quality control" can send data back to the transform stage automatically, and the three stages may run in milliseconds inside one script.

Technical explanation. | Stage | Input → output | Typical work |

|-----|-----|-----| | **Extraction** | Source file or DCC scene → neutral records | Parse the file, walk the DCC scene, read only what the mapping document lists | |

| **Transformation** | Neutral records → USD-shaped data | Rename

(`Tf.MakeValidIdentifier`, Chapter 28), convert units and axes, triangulate or keep polygons, choose value types (Chapter 9), compute extent (Chapter 13), map materials | |

| **Load / write** | USD-shaped data → layer on disk | Author prims with `Usd/Sdf`, set `defaultPrim`, `upAxis`, `metersPerUnit`, save as USDA or USDC | | **Validation** | Layer → list of problems | `pxr.UsdValidation` validators plus your own checks (Chapter 30) |

Some texts call this ETL (extract, transform, load) with validation added. Keep the neutral records simple (Python dicts or small classes) so you can unit-test each stage separately.

Validation in USD 26.08 uses `pxr.UsdValidation`. For example, the validator `usdGeomValidators:StageMetadataChecker` reports a stage that lacks `upAxis` or `metersPerUnit`.

VERSION

Verified on USD 26.08. Older material validates with ``UsdUtils.ComplianceChecker``, which was removed entirely in 26.08. Use ``pxr.UsdValidation`` (Chapter 30).

Mental model.

```
source --> [ EXTRACT ] --> records --> [ TRANSFORM ] --> USD data
|
```

```

                                v
report <-- [ VALIDATE ] <-- layer <-- [ WRITE ]
      |
      v
errors? +--> fix the transform, run again

```

Simple example. A JSON file says a table is 120 units wide, units "cm", up axis "Z", name "Table Top". Extraction reads those four facts. Transformation produces width 1.2 (meters), name `Table_Top`, `upAxis = "Z"`. Validation confirms the stage declares `upAxis` and `metersPerUnit`.

USDA example. This is the output of the transform and write stages for the example above.

```

#usda 1.0
(
  defaultPrim = "Furniture"
  metersPerUnit = 1
  upAxis = "Z"
)

def Xform "Furniture"
{
  def Cube "Table_Top"
  {
    double size = 1.2
    double3 xformOp:translate = (0, 0, 0.75)
    uniform token[] xformOpOrder = ["xformOp:translate"]
  }
}

```

- The name `Table Top` became `Table_Top`, because spaces are not allowed in prim names.
- 120 cm became 1.2 and 75 cm became 0.75, because the target declares `metersPerUnit = 1`.

Python example. A complete, tiny pipeline with one function per stage.

```

from pxr import Tf, Usd, UsdGeom, UsdValidation

SOURCE = {"units": "cm", "up": "Z", "objects": [
    {"name": "Table Top", "size": 120.0, "pos": [0, 0, 75]},
    {"name": "1st Leg", "size": 5.0, "pos": [50, 30, 37.5]}]}
TO_METERS = {"cm": 0.01, "mm": 0.001, "m": 1.0}

def extract(src):
    return src["objects"], src["units"], src["up"]

def transform(objects, units, up):
    k = TO_METERS[units]
    return up, [{"name": Tf.MakeValidIdentifier(o["name"]),
                  "size": o["size"] * k,
                  "pos": [round(c * k, 4) for c in o["pos"]]} for o in objects]

def write(up, records):
    stage = Usd.Stage.CreateInMemory()
    UsdGeom.SetStageUpAxis(stage, up)
    UsdGeom.SetStageMetersPerUnit(stage, UsdGeom.LinearUnits.meters)
    root = UsdGeom.Xform.Define(stage, "/Furniture")
    stage.SetDefaultPrim(root.GetPrim())
    for r in records:
        cube = UsdGeom.Cube.Define(stage, root.GetPath().AppendChild(r["name"]))
        cube.GetSizeAttr().Set(r["size"])
        cube.AddTranslateOp().Set(tuple(r["pos"]))
    return stage

def validate(stage):
    reg = UsdValidation.ValidationRegistry()
    v = reg.GetOrLoadValidatorByName("usdGeomValidators:StageMetadataChecker")
    problems = [e.GetName() for e in v.Validate(stage)]
    for prim in stage.Traverse():

```

```

        if prim.IsA(UsdGeom.Cube) and UsdGeom.Cube(prim).GetSizeAttr().Get() <= 0:
            problems.append(f"non-positive size on {prim.GetPath()}")
        return problems

up, records = transform(*extract(SOURCE))
for r in records:
    print(r)
stage = write(up, records)
print("validation problems:", validate(stage))

```

Expected output

```

{'name': 'Table_Top', 'size': 1.2, 'pos': [0.0, 0.0, 0.75]}
{'name': '_st_Leg', 'size': 0.05, 'pos': [0.5, 0.3, 0.375]}
validation problems: []

```

Notice `1st Leg` became `_st_Leg`: a prim name cannot start with a digit, so `Tf.MakeValidIdentifier` replaced the `1` with `_`. Chapter 28 covers naming in depth.

Real-world use case. A digital-twin team receives nightly factory-layout exports from a CAD system. The extraction stage reads the CAD export, the transformation stage converts millimeters to meters and sanitizes part names, and the validation stage blocks the publish if any part lacks an extent or the stage lacks units. Failures are emailed to the CAD team with the offending part names.

Common mistakes.

MISTAKE

Converting units inside the extraction code "while you are there". Later nobody can tell whether the records are in source or target units. Fix: extraction keeps source values; only the transformation stage converts.

MISTAKE

Skipping validation because "the exporter is tested". New source data breaks old assumptions. Fix: validate every run, automatically.

Exam traps.

TRAP

Validation comes after writing USD, not before extraction. The order is extract → transform → (write) → validate.

TRAP

An answer that suggests ``UsdUtils.ComplianceChecker`` for a current pipeline is outdated. In 26.08 it no longer exists; the current API is ``UsdValidation``.

Exam takeaways.

KEY

- Extract (read source) → transform (rename, units, axes, types, extents, materials) → write → validate. - Validate every run with ``pxr.UsdValidation`` plus your own checks. - Keep extracted records in source units; convert only in the transform stage. - Validation is covered in full in Chapter 30.

27.3 Conceptual data mapping documents (with template)

What is it? A **conceptual data mapping document** is a table that lists every concept in a source data model, what it becomes in the target data model, how it is transformed, and what is lost.

Why do we need it? Translators are written by one person and maintained by others. Without a written mapping, nobody can answer “where did my smoothing groups go?” or “is this loss a bug or a design decision?”. The mapping is also the specification you test the translator against.

Beginner explanation. It is a bilingual dictionary for two data models. For each word in language A it gives the word in language B, plus notes like “no exact equivalent; closest is...”.

Where the analogy breaks: a dictionary maps words, but a data mapping also maps structure (one source object can become several prims, or several source objects can merge into one prim) and units.

Technical explanation. Use this reusable template. One row per source concept.

Source concept	Source type / units	USD target	USD value type	Transformation rule	Fidelity / loss	Round-trips?
(name in source)	(type, units, space)	(prim type, property, or metadata)	(e.g. <code>point3f[]</code>)	(formula, rename, split)	lossless / lossy / dropped + why	yes / no / partial

Add a header above the table:

- **Source and target versions** (for example “OBJ (Wavefront) → USD 26.08, UsdGeom/UsdShade”).
- **Global conventions:** units, up axis, handedness, naming rule, default prim.
- **Out of scope:** concepts deliberately not translated.

Here is a filled-in example for OBJ → USD.

Source concept	Source type / units	USD target	USD value type	Transformation rule	Fidelity / loss	Round-trips?
<code>o</code> object name	string	<code>Mesh</code> prim name	prim name	<code>Tf.MakeValidIdentifier</code> , de-duplicate	lossy for invalid characters	partial
<code>v</code> vertex	3 floats, file units	<code>points</code>	<code>point3f[]</code>	scale by units factor	lossless	yes
<code>f</code> face	1-based indices	<code>faceVertexCounts</code> , <code>faceVertexIndices</code>	<code>int[]</code>	subtract 1	lossless	yes
<code>vt</code> UV	2 floats	<code>primvars:st</code> (faceVarying, indexed)	<code>texCoord2f[]</code>	per face corner	lossless	yes
<code>vn</code> normal	3 floats	<code>normals</code> or <code>primvars:normals</code> (faceVarying)	<code>normal3f[]</code>	per face corner	lossless	yes
<code>usemtl</code>	material name	<code>Material</code> + <code>GeomSubset</code> binding	relationship	one subset per material	lossless	yes
<code>s</code> smoothing group	int or off	none	—	dropped; normals carry shading	dropped	no
<code>l</code> polyline	indices	<code>BasisCurves</code> (linear)	<code>int[]</code>	optional	dropped in v1	no

The same template works for USD → glTF and UsdShade ↔ MaterialX (section 27.4).

Mental model.

+-----+ rule +-----+	
source item -----> USD item	each row = one arrow
+-----+ +-----+	rows with no arrow = LOSS
	(write them down!)
+--> (nothing) "dropped: why"	

Simple example. Row: “OBJ v (3 floats, centimeters) → Mesh.points (point3f[]), multiply by 0.01, lossless, round-trips.” Anyone reading this row knows the target property, the type, the formula, and the fidelity.

USDA example. The USD side of the OBJ mapping above: a single quad with one material subset.

```
#usda 1.0
(
    defaultPrim = "Crate"
    metersPerUnit = 1
    upAxis = "Y"
)

def Mesh "Crate"
{
    int[] faceVertexCounts = [4]
    int[] faceVertexIndices = [0, 1, 2, 3]
    point3f[] points = [(0, 0, 0), (1, 0, 0), (1, 1, 0), (0, 1, 0)]
    texCoord2f[] primvars:st = [(0, 0), (1, 0), (1, 1), (0, 1)] (
        interpolation = "faceVarying"
    )
    int[] primvars:st:indices = [0, 1, 2, 3]

    def GeomSubset "wood"
    {
        uniform token elementType = "face"
        uniform token familyName = "materialBind"
        int[] indices = [0]
    }
}
```

- `faceVertexIndices` are 0-based: the “subtract 1” rule from the `f` row.
- `primvars:st` with `primvars:st:indices` is the indexed, `faceVarying` target of the `vt` row (Chapter 11).
- The `GeomSubset` with `familyName = "materialBind"` is the target of the `usmtl` row.

Python example. Keep the mapping as data in the translator. The same table can then print the document and report unmapped source concepts, so the document never drifts from the code.

```
MAPPING = {
    "o": ("Mesh prim name", "token", "MakeValidIdentifier", "partial"),
    "v": ("points", "point3f[]", "scale to meters", "yes"),
    "f": ("faceVertexCounts/Indices", "int[]", "1-based -> 0-based", "yes"),
    "vt": ("primvars:st", "texCoord2f[]", "faceVarying, indexed", "yes"),
    "vn": ("normals", "normal3f[]", "faceVarying", "yes"),
    "usmtl": ("GeomSubset + Material", "rel", "subset per material", "yes"),
}

OBJ_TEXT = """o Crate
v 0 0 0
v 1 0 0
v 1 1 0
vt 0 0
s 1
usmtl wood
f 1/1 2/1 3/1
l 1 2
"""
```



```
print("| OBJ | USD target | Type | Rule | Round-trips |")
print("|-----|-----|-----|-----|-----|")
for key, (target, vtype, rule, rt) in MAPPING.items():
    print(f"| {key} | {target} | {vtype} | {rule} | {rt} |")

seen = sorted({line.split()[0] for line in OBJ_TEXT.splitlines() if line})
print("unmapped (loss):", [k for k in seen if k not in MAPPING])
```

Expected output

```
| OBJ | USD target | Type | Rule | Round-trips |
|-----|-----|-----|-----|-----|
| o | Mesh prim name | token | MakeValidIdentifier | partial |
| v | points | point3f[] | scale to meters | yes |
| f | faceVertexCounts/Indices | int[] | 1-based -> 0-based | yes |
| vt | primvars:st | texCoord2f[] | faceVarying, indexed | yes |
| vn | normals | normal3f[] | faceVarying | yes |
| usemtl | GeomSubset + Material | rel | subset per material | yes |
unmapped (loss): ['l', 's']
```

Real-world use case. An automotive company exchanges materials between a CAD visualization tool, a MaterialX-based renderer, and USD. Its mapping document has one row per material parameter, with value ranges and color spaces, and is reviewed by both the CAD and rendering teams before any translator code changes.

Common mistakes.

MISTAKE

Listing only what maps. The rows that do not map are the most important ones. Fix: every source concept gets a row, even if the target column says "dropped".

MISTAKE

Leaving out units and spaces ("color" without saying linear or sRGB, "position" without saying meters). Fix: a "Source type / units" column on every row.

Exam traps.

TRAP

A mapping document is conceptual: it describes concepts and rules, not code. An answer that says "the mapping document is the converter script" is wrong, even though code can be generated from it.

TRAP

"Every source concept must have a USD target." No. Dropping is allowed, but must be documented and justified.

Exam takeaways.

KEY

- One row per source concept: target, type, rule, fidelity, round-trip. - Document dropped concepts and why. - Always state units, spaces, versions, and conventions. - Keep the mapping as data so code and document agree.

27.4 Mapping USD to glTF and MaterialX

What is it? Applying the mapping-document idea to two common partners: **glTF 2.0**, a delivery format for real-time and web viewers, and **MaterialX**, an interchange standard for shading networks.

Why do we need it? Objective 4.1 asks you to convert USD to formats such as glTF "with fidelity". That means knowing which USD concepts have a glTF equivalent, which need transformation, and which are lost, and then checking the result. Objective 4.2 uses MaterialX as its example of a data model you map to USD.

Beginner explanation. USD is a full workshop: layers, composition, variants, any custom attribute. glTF is a shipping box: one flattened, triangulated, ready-to-display result. You can pack the finished product, but not the workshop.

Where the analogy breaks: glTF can hold some “workshop” items through extensions and extras, so loss is not absolute. You choose what to pack.

Technical explanation. glTF basics. A .gltf file is JSON plus binary buffers (or one .glb file). It has scenes → nodes (a transform as translation/rotation/scale or matrix) → meshes → primitives. A primitive has vertex attributes (POSITION, NORMAL, TEXCOORD_0, COLOR_0, ...), optional indices, and one material. Data lives in buffers, sliced by bufferViews, typed by accessors. The POSITION accessor must have min and max. glTF uses meters, +Y up, a right-handed system, and counterclockwise front faces.

USD → glTF mapping.

USD concept	glTF concept	Rule / loss
Composed stage	One scene	Compose first; layers, references, variants are not kept (only the current selections, flattened)
Xform + xformOps	node TRS or matrix	Bake xformOpOrder into one matrix or TRS
Mesh points / counts / indices	POSITION + indices	Triangulate polygons; glTF has no quads or n-gons
faceVarying primvars and normals	Per-vertex attributes	Split vertices so each corner has one value
primvars:st	TEXCOORD_0	Flip v: $v_{\text{gltf}} = 1 - v_{\text{usd}}$ (glTF UV origin is top-left)
primvars:displayColor	COLOR_0	Expand constant/uniform to per-vertex
GeomSubset (materialBind)	Several primitives	One primitive per material
metersPerUnit, upAxis = "Z"	(fixed: meters, Y-up)	Scale by metersPerUnit; rotate Z-up to Y-up
subdivisionScheme	—	Lost; export the base cage or a refined mesh
purpose, kind, customData	— or extras	Lost unless written into extras
Custom attributes	extras, or attribute _NAME	glTF requires custom vertex attribute names to start with _
Time-sampled xforms	animations (channels + samplers)	Time in seconds = timeCode / timeCodesPerSecond; only TRS and morph weights animate in core glTF
Native instancing / PointInstancer	Several nodes reusing one mesh	EXT_mesh_gpu_instancing extension for many copies
UsdLux lights	KHR_lights_punctual	Only point, spot, directional

UsdPreviewSurface → glTF pbrMetallicRoughness.

UsdPreviewSurface input	glTF material field	Note
diffuseColor	baseColorFactor RGB / baseColorTexture	
opacity	baseColorFactor alpha	Set alphaMode to BLEND
opacityThreshold	alphaCutoff	With alphaMode MASK

UsdPreviewSurface input	glTF material field	Note
metallic, roughness	metallicFactor, roughnessFactor	One texture: roughness in G, metallic in B
emissiveColor	emissiveFactor / emissiveTexture	
normal	normalTexture	
occlusion	occlusionTexture	
useSpecularWorkflow = 1	no core field	Extension KHR_materials_specular, or convert
clearcoat, ior	no core field	KHR_materials_clearcoat, KHR_materials_ior
Gprim doubleSided	material doubleSided	Moves from geometry to material

UsdShade ↔ MaterialX. MaterialX describes a material as a graph of typed nodes, each defined by a **nodedef** (for example `ND_standard_surface_surfaceshader`).

MaterialX concept	UsdShade concept
surfacematerial node	UsdShade.Material
nodegraph	UsdShade.NodeGraph
node (e.g. standard_surface, image)	UsdShade.Shader with info:id = nodedef name
input / output	inputs: / outputs: attributes
connection (nodename=, nodegraph=)	connectToSource on the input
shader for the MaterialX render context	outputs:mtlx:surface on the Material
look + materialassign	material binding (MaterialBindingAPI), often per variant
UsdPreviewSurface nodedef in MaterialX	Same node name in both: the safe common subset

In a full USD build the `usdMtlx` file format plugin reads `.mtlx` files as USD layers. It is not in `usd-core`, so the mapping is taught conceptually here.

VERSION

Verified on USD 26.08 with ``usd-core``: ``from pxr import UsdMtlx`` fails, and no glTF reader is registered. Full USD builds can include ``UsdMtlx``; glTF import/export usually comes from DCCs or separate tools.

Mental model.

USD (rich)	glTF (delivery)
layers/variants	--compose+flatten--> one scene
polygons	--triangulate-----> triangles
faceVarying	--split vertices---> per-vertex
any units/axis	--scale/rotate----> meters, +Y up
custom data	--extras or LOST---> extras

Simple example. A USD quad (one 4-sided face) with `diffuseColor = (0.8, 0.2, 0.1)` becomes a glTF primitive with 4 positions, 6 indices (two triangles), and a material with `baseColorFactor = [0.8, 0.2, 0.1, 1.0]`.

USDA example. The source asset for the Python example below.

```
#usda 1.0
(
  defaultPrim = "Tile"
  metersPerUnit = 1
  upAxis = "Y"
```

```

)

def Mesh "Tile" (
    prepend apiSchemas = ["MaterialBindingAPI"]
)
{
    int[] faceVertexCounts = [4]
    int[] faceVertexIndices = [0, 1, 2, 3]
    rel material:binding = </Tile/Mat>
    point3f[] points = [(0, 0, 0), (1, 0, 0), (1, 1, 0), (0, 1, 0)]
    custom int studio:tileId = 7

    def Material "Mat"
    {
        token outputs:surface.connect = </Tile/Mat/PBR.outputs:surface>

        def Shader "PBR"
        {
            uniform token info:id = "UsdPreviewSurface"
            color3f inputs:diffuseColor = (0.8, 0.2, 0.1)
            float inputs:metallic = 0
            float inputs:roughness = 0.4
            token outputs:surface

        }
    }
}

```

- The 4-vertex face must become 2 triangles in glTF.
- `inputs:diffuseColor`, `inputs:metallic`, `inputs:roughness` map to the `pbrMetallicRoughness` fields.
- `studio:tileId` has no glTF home, so the exporter writes it to `extras`.

Python example. No glTF library is installed, so this exporter uses only the Python standard library. It triangulates, packs a binary buffer, maps the material, writes `tile.gltf`, then reads the file back and checks fidelity.

```

import base64, json, struct
from pxr import Gf, Sdf, Usd, UsdGeom, UsdShade

stage = Usd.Stage.CreateInMemory()
UsdGeom.SetStageUpAxis(stage, UsdGeom.Tokens.y)
UsdGeom.SetStageMetersPerUnit(stage, UsdGeom.LinearUnits.meters)
mesh = UsdGeom.Mesh.Define(stage, "/Tile")
mesh.CreatePointsAttr([(0, 0, 0), (1, 0, 0), (1, 1, 0), (0, 1, 0)])
mesh.CreateFaceVertexCountsAttr([4])
mesh.CreateFaceVertexIndicesAttr([0, 1, 2, 3])
mesh.GetPrim().CreateAttribute("studio:tileId", Sdf.ValueTypeNames.Int).Set(7)
mat = UsdShade.Material.Define(stage, "/Tile/Mat")
pbr = UsdShade.Shader.Define(stage, "/Tile/Mat/PBR")
pbr.CreateIdAttr("UsdPreviewSurface")
pbr.CreateInput("diffuseColor", Sdf.ValueTypeNames.Color3f).Set((0.8, 0.2, 0.1))
pbr.CreateInput("metallic", Sdf.ValueTypeNames.Float).Set(0.0)
pbr.CreateInput("roughness", Sdf.ValueTypeNames.Float).Set(0.4)
mat.CreateSurfaceOutput().ConnectToSource(pbr.ConnectableAPI(), "surface")
UsdShade.MaterialBindingAPI.Apply(mesh.GetPrim()).Bind(mat)

# --- transform: units, triangulation, material lookup ---
assert UsdGeom.GetStageUpAxis(stage) == "Y" # glTF is +Y up
k = UsdGeom.GetStageMetersPerUnit(stage) # glTF is meters
pts = [Gf.Vec3f(p) * k for p in mesh.GetPointsAttr().Get()]
tris, start = [], 0
idx = mesh.GetFaceVertexIndicesAttr().Get()
for n in mesh.GetFaceVertexCountsAttr().Get(): # fan triangulation
    for i in range(1, n - 1):
        tris += [idx[start], idx[start + i], idx[start + i + 1]]
    start += n
bound, _ = UsdShade.MaterialBindingAPI(mesh.GetPrim()).ComputeBoundMaterial()
surf = bound.ComputeSurfaceSource()[0]
get = lambda name: surf.GetInput(name).Get()
r4 = lambda xs: [round(float(x), 4) for x in xs]

```

```
# --- write glTF JSON + base64 buffer ---
pos = b"".join(struct.pack("<3f", *p) for p in pts)
ind = struct.pack(f"<{len(tris)}H", *tris)
blob = pos + ind
gltf = {
    "asset": {"version": "2.0", "generator": "ch27-sketch"},
    "scene": 0, "scenes": [{"nodes": [0]}],
    "nodes": [{"name": "Tile", "mesh": 0,
        "extras": {"studio:tileId": mesh.GetPrim()
            .GetAttribute("studio:tileId").Get()}}],
    "meshes": [{"primitives": [{"attributes": {"POSITION": 0,
        "indices": 1, "material": 0}}]},
    "materials": [{"name": bound.GetPrim().GetName(), "pbrMetallicRoughness": {
        "baseColorFactor": r4(get("diffuseColor")) + [1.0],
        "metallicFactor": r4([get("metallic"))][0],
        "roughnessFactor": r4([get("roughness"))][0]}]},
    "buffers": [{"byteLength": len(blob), "uri":
        "data:application/octet-stream;base64,"
        + base64.b64encode(blob).decode()}],
    "bufferViews": [
        {"buffer": 0, "byteOffset": 0, "byteLength": len(pos), "target": 34962},
        {"buffer": 0, "byteOffset": len(pos), "byteLength": len(ind),
        "target": 34963}],
    "accessors": [
        {"bufferView": 0, "componentType": 5126, "count": len(pts),
        "type": "VEC3", "min": r4(min(p[a] for p in pts) for a in range(3)),
        "max": r4(max(p[a] for p in pts) for a in range(3))},
        {"bufferView": 1, "componentType": 5123, "count": len(tris),
        "type": "SCALAR"}],
}
with open("tile.gltf", "w") as f:
    json.dump(gltf, f, indent=1)

# --- fidelity check: read the file back and compare with USD ---
back = json.load(open("tile.gltf"))
raw = base64.b64decode(back["buffers"][0]["uri"].split(",")[1])
n_pts = back["accessors"][0]["count"]
xyz = struct.unpack(f"<{3 * n_pts}f", raw[:12 * n_pts])
ext = UsdGeom.PointBased.ComputeExtent(mesh.GetPointsAttr().Get())
print("triangles:", back["accessors"][1]["count"] // 3)
print("points match:", [tuple(Gf.Vec3f(xyz[i:i + 3])) for i in range(0, 12, 3)]
    == [tuple(p) for p in pts])
print("bounds match extent:", back["accessors"][0]["min"] == r4(ext[0]),
    back["accessors"][0]["max"] == r4(ext[1]))
print("material:", back["materials"][0])
print("extras:", back["nodes"][0]["extras"])
```

Expected output

```
triangles: 2
points match: True
bounds match extent: True True
material: {'name': 'Mat', 'pbrMetallicRoughness': {'baseColorFactor': [0.8, 0.2, 0.1, 1.0],
'metallicFactor': 0.0, 'roughnessFactor': 0.4}}
extras: {'studio:tileId': 7}
```

The fidelity check compares counts, positions, and bounds against USD, which is what “ensure fidelity” means in practice. Production converters also render both files and compare images.

Real-world use case. An e-commerce site authors products in USD with a `color` variant set. Its build server exports one glTF per variant selection (`chair_red.glb`, `chair_blue.glb`), because glTF cannot hold variants. Each export runs a fidelity check on triangle counts and bounds, and a turntable render comparison.

Common mistakes.

MISTAKE

Exporting a Z-up, centimeter USD stage to glTF without conversion. The model appears 100× too large and lying on its back. Fix: scale by ``metersPerUnit`` and rotate Z-up to Y-up during transformation.

MISTAKE

Copying ``primvars:st`` straight into ``TEXCOORD_0``. Textures appear upside down. Fix: ``v_glTF = 1 - v_usd``.

MISTAKE

Exporting faceVarying normals as if they were per-vertex. Shading breaks at hard edges. Fix: split vertices so each corner has its own normal.

Exam traps.**TRAP**

"glTF supports variants and layers, so a USD → glTF export is lossless." Core glTF has no layers, references, or variant sets. You export one composed, flattened result per variant choice.

TRAP

glTF puts metallic and roughness in one texture (roughness in G, metallic in B). UsdPreviewSurface has separate inputs. A converter must pack or split channels.

TRAP

MaterialX's ``nodedef`` name maps to the UsdShade Shader's ``info:id``, not to the prim name.

Exam takeaways.**KEY**

- glTF: meters, +Y up, right-handed, triangles, per-vertex attributes, one material per primitive. - USD → glTF: compose and flatten, triangulate, split faceVarying data, flip v, convert units and axis. - Lost unless you use ``extras`` or extensions: layers, variants, custom data, subdivision, most animated attributes. - UsdPreviewSurface maps closely to glTF ``pbrMetallicRoughness``. - MaterialX node → ``UsdShade.Shader`` with ``info:id`` = `nodedef``; `nodegraph`` → ``NodeGraph``.

27.5 Round-trip pipelines with a DCC

What is it? A **round trip** takes USD into a DCC, lets an artist edit it, and brings the result back to USD with only the intended changes.

Why do we need it? Real assets move between departments many times. If each trip drops custom data, renames prims, or flattens composition, the asset degrades a little more each time, and downstream overrides that target the old prim paths stop working.

Beginner explanation. Lending your annotated notebook to a friend who only copies the parts they understand into a fresh notebook. You get back their fixes, but your margin notes are gone. A better friend writes their fixes on sticky notes and hands back your original notebook with the notes on top.

Where the analogy breaks: in USD the “sticky notes” are an override layer, and composition applies them automatically and precisely, attribute by attribute.

Technical explanation. Typical round-trip losses:

- **Unknown data dropped.** The importer ignores attributes and metadata it does not understand.
- **Composition flattened.** References, payloads, and variants become plain local data.
- **Identity changed.** Prims are renamed or re-parented, so paths change.
- **Topology changed.** Meshes are retriangulated or point order is changed, which breaks primvars and downstream overrides.
- **Conventions changed.** Units, up axis, or interpolation are altered.

Strategies, best first:

1. **Sparse override export.** The DCC writes only what the artist changed, as `over` opinions in a new layer that sits above the original (by sublayer or as a stronger layer in the shot). The original asset is never rewritten. This is how USD-native DCCs such as Houdini Solaris and the Maya USD plug-in can work.
2. **Preserve what you cannot interpret.** The importer stores unknown attributes and metadata, and the exporter writes them back unchanged.
3. **Stable identity.** Keep the USD prim path as the DCC object's identity; never rename on import.
4. **Automated diff test.** Import, export with no edits, and compare: the result must equal the input (Chapter 30 for validation, Chapter 35 for DCC integration).

Mental model.

Strategy A: full rewrite	Strategy B: sparse override layer
chair.usda --import--> DCC	chair_edits.usda (over: only the edit)
	subLayers
chair.usda <--export-- (rewritten)	chair.usda (untouched original)
unknown data LOST	unknown data KEPT

Simple example. The asset has `size = 2`, a gray `displayColor`, and `customData` with an asset ID. The DCC understands only `size` and `color`. The artist changes the color to red. Strategy A writes a new file with `size` and red color, and the asset ID is gone. Strategy B writes a layer that contains only the red color; the asset ID survives in the original underneath.

USDA example. The override layer written by strategy B.

File: *chair_edits.usda*

```
#usda 1.0
(
  subLayers = [
    @chair.usda@
  ]
)

over "Chair"
{
  color3f[] primvars:displayColor = [(0.8, 0.1, 0.1)]
}
```

- `subLayers` puts the original under this layer, so the original's opinions still apply where this layer is silent.
- `over` adds an opinion without defining the prim; the type and everything else come from `chair.usda`.

Python example. Both strategies side by side, then a check of what survived.

```
from pxr import Sdf, Usd, UsdGeom

stage = Usd.Stage.CreateNew("chair.usda")
cube = UsdGeom.Cube.Define(stage, "/Chair")
cube.GetSizeAttr().Set(2.0)
cube.GetDisplayColorAttr().Set([(0.5, 0.5, 0.5)])
cube.GetPrim().SetCustomDataByKey("assetId", "CH-001")
stage.SetDefaultPrim(cube.GetPrim())
stage.Save()

# Import: the "DCC" keeps only what it understands, then the artist edits.
dcc = {"size": cube.GetSizeAttr().Get(),
      "color": tuple(cube.GetDisplayColorAttr().Get()[0])}
dcc["color"] = (0.8, 0.1, 0.1)

# Strategy A: rewrite the whole asset from the DCC's data.
```

```

full = Usd.Stage.CreateNew("chair_full.usda")
c = UsdGeom.Cube.Define(full, "/Chair")
c.GetSizeAttr().Set(dcc["size"])
c.GetDisplayColorAttr().Set([dcc["color"]])
full.Save()

# Strategy B: write only changed values into an override layer.
edits = Sdf.Layer.CreateNew("chair_edits.usda")
edits.subLayerPaths.append("chair.usda")
over_stage = Usd.Stage.Open(edits)
chair = UsdGeom.Cube(over_stage.GetPrimAtPath("/Chair"))
if tuple(chair.GetDisplayColorAttr().Get()[0]) != dcc["color"]:
    chair.GetDisplayColorAttr().Set([dcc["color"]])
if chair.GetSizeAttr().Get() != dcc["size"]:
    chair.GetSizeAttr().Set(dcc["size"])
edits.Save()

for name in ["chair_full.usda", "chair_edits.usda"]:
    result = Usd.Stage.Open(name)
    prim = result.GetPrimAtPath("/Chair")
    color = UsdGeom.Cube(prim).GetDisplayColorAttr().Get()[0]
    print(name, "color:", color, "assetId:", prim.GetCustomDataByKey("assetId"))
specs = []
edits.Traverse("/", lambda path: specs.append(str(path)))
print("specs in edits layer:", sorted(specs))
original = Sdf.Layer.FindOrOpen("chair.usda")
print("original color spec:",
      original.GetAttributeAtPath("/Chair.primvars:displayColor").default)

```

Expected output

```

chair_full.usda color: (0.8, 0.1, 0.1) assetId: None
chair_edits.usda color: (0.8, 0.1, 0.1) assetId: CH-001
specs in edits layer: ['/', '/Chair', '/Chair.primvars:displayColor']
original color spec: [(0.5, 0.5, 0.5)]

```

Strategy B carried the edit and kept the asset ID. Strategy A lost it silently. The edits layer holds only the pseudo-root `/`, an `over` for `/Chair`, and the one changed attribute. The original file still says gray.

NOTE

The script reads ``GetCustomDataByKey("assetId")`` rather than ``GetCustomData()``. On USD 26.08, ``GetCustomData()`` on a typed prim such as ``Cube`` also returns a ``userDocBrief`` entry that comes from the schema definition, not from your layers.

Real-world use case. In a film pipeline, layout artists open the shot in a DCC, nudge a few props, and publish. The publish writes a `layout.usda` layer with only `over` opinions for the moved props' `xformOp:translate`. The set-dressing layer and every asset file below stay untouched, and other departments' layers keep composing on top.

Common mistakes.

MISTAKE

Testing a round trip only with your own simple test file. Real assets have custom attributes, variants, and references. Fix: run the "import, export unchanged, diff" test on real published assets.

MISTAKE

Renaming prims on import to match DCC naming rules. Every downstream override that targets the old path stops applying. Fix: keep the USD path as the identity, and use display names in the DCC if needed.

Exam traps.

TRAP

"The safest round trip re-exports the entire asset." Usually wrong. Writing sparse overrides into a new layer is safer, because the original stays untouched and unknown data survives.

TRAP

An override layer uses ``over``, not ``def``. With ``def``, the layer would also be able to define a prim that does not exist below, hiding a path typo.

Exam takeaways.**KEY**

- Round trip = USD → DCC → USD with only intended changes. - Best practice: export sparse ``over`` opinions into a new layer; never rewrite the original. - Preserve unknown data, keep prim paths stable, avoid topology changes. - Prove fidelity with an automated "import, export unchanged, diff" test.

Chapter lab(s)

This chapter has no dedicated lab. Practice its ideas in **Lab 25 — Write a JSON/OBJ → USD converter** (Chapter 29), which follows the extract → transform → validate structure, and **Lab 37 — Capstone: a small multi-asset pipeline end to end**, which includes an export and validation step. Lab 27 (Write an asset validator, Chapter 30) completes the validation stage.

USDA reading exercise(s)

Exercise 27-A. A converter produced this layer. Name two things a glTF export of it must change, and one thing it will lose.

```
#usda 1.0
(
    metersPerUnit = 0.01
    upAxis = "Z"
)

def Mesh "Panel" (
    customData = {
        string partNumber = "P-77"
    }
)
{
    int[] faceVertexCounts = [4]
    int[] faceVertexIndices = [0, 1, 2, 3]
    point3f[] points = [(0, 0, 0), (100, 0, 0), (100, 0, 100), (0, 0, 100)]
    uniform token subdivisionScheme = "catmullClark"
}
```

Exercise 27-B. In the two files below, what is the composed size of `/Chair`, and which file did the round trip write?

File: *chair.usda*

```
#usda 1.0
(
    defaultPrim = "Chair"
)

def Cube "Chair"
{
    double size = 2
}
```

File: *chair_edits.usda*

```
#usda 1.0
(
    subLayers = [
```

```

    @chair.usda@
  ]
)

over "Chair"
{
    double size = 2.5
}

```

Chapter review

Summary

- Data exchange = export (tool → USD), import (USD → tool), conversion (file → file).
- Foreign data reaches USD through DCC plugins, standalone converters, or `SdfFileFormat` plugins; `usd-core` reads only USD formats.
- Every translator follows extract → transform → write → validate.
- A mapping document has one row per source concept, including dropped ones, with units and rules.
- glTF is meters, +Y up, triangles, per-vertex attributes; USD composition and custom data do not survive without `extras`.
- `UsdPreviewSurface` maps closely to glTF PBR; MaterialX nodes map to `UsdShade.Shader` with `info:id = nodedef`.
- Fidelity is proved by checks (counts, bounds, values, renders), not assumed.
- Safe round trips write sparse over opinions into a new layer and keep prim paths stable.

If you see... → think...

If you see...	Think...
“convert USD to glTF with fidelity”	Flatten, triangulate, split <code>faceVarying</code> , flip <code>v</code> , meters/Y-up, then check counts and bounds
“document the mapping to MaterialX”	Template table: concept, type, target, rule, loss; Shader <code>info:id = nodedef</code>
“data lost after round trip”	Full rewrite dropped unknown data; use sparse override layer
“open .obj/.abc directly with <code>Usd.Stage.Open</code> ”	Needs an <code>SdfFileFormat</code> plugin
“stage has no upAxis” in a pipeline check	<code>usdGeomValidators:StageMetadataChecker</code> (<code>UsdValidation</code>)
“custom attribute in glTF”	<code>extras</code> , or vertex attribute named <code>_NAME</code>
“USDA → USDC loses data?”	No; same data model

Review questions

DE-R27-01 · Obj 4.1 · Single choice A USD stage has `metersPerUnit = 0.01` and `upAxis = "Z"`. What must a glTF exporter do to its point positions? A. Nothing; glTF reads `metersPerUnit`. B. Multiply by 0.01 and rotate from Z-up to Y-up. C. Multiply by 100 and keep Z-up. D. Only flip the winding order.

DE-R27-02 · Obj 4.1 · Select two. Which steps are required to export a USD mesh with `faceVarying` UVs to core glTF? A. Split vertices so each corner has a single UV. B. Convert the mesh to a `BasisCurves`. C. Flip the `v` coordinate. D. Store UVs in `extras`.

DE-R27-03 · Obj 4.2 · Single choice In a USD ↔ MaterialX mapping document, a MaterialX nodegraph maps to: A. `UsdShade.Material`. B. `UsdShade.NodeGraph`. C. A `GeomSubset`. D. A variant set.

DE-R27-04 · Obj 4.2 · Single choice Which mapping-document column records “OBJ 1 polylines are dropped in version 1”? A. USD value type. B. Transformation rule. C. Fidelity / loss. D. Source type / units.

DE-R27-05 · Obj 4.4 · Single choice A DCC re-exports a whole asset after an artist edits one color. Downstream, a variant set and a custom attribute are missing. What is the best fix? A. Export as USDC instead of USDA. B. Export only the changed color as an `over` in a layer above the original. C. Add `defaultPrim` to the exported file. D. Run `Tf.MakeValidIdentifier` on every prim name.

DE-R27-06 · Obj 4.1 · Single choice What does this print?

```
counts = [4, 4, 3]
print(sum(n - 2 for n in counts))
```

A. 3 B. 5 C. 8 D. 11

DE-R27-07 · Obj 4.4 · USDA reading In Exercise 27-B, a teammate deletes `chair_edits.usda`. What is the composed size when opening `chair.usda`? A. 2 B. 2.5 C. 1 (the Cube fallback) D. The prim no longer exists.

DE-R27-08 · Obj 4.1 · Select two. Which `UsdPreviewSurface` inputs map directly to core `glTF pbrMetallicRoughness` fields? A. `roughness` B. `clearcoat` C. `diffuseColor` D. `ior`

DE-R27-09 · Obj 4.1 · Single choice In the pipeline order `extract` → `transform` → `write` → `validate`, where does `Tf.MakeValidIdentifier` belong? A. Extract. B. Transform. C. Write. D. Validate.

DE-R27-10 · Obj 4.1 · Single choice Why is a `.usda` → `.usdc` conversion not a fidelity risk? A. USDC compresses only textures. B. Both encode the same USD data model. C. USDC stores only the composed result. D. USDA cannot store time samples.

Answers

Exercise 27-A. Must change: scale points by 0.01 (`glTF` is meters) and rotate Z-up to Y-up; triangulate the quad into 2 triangles. Lost: `subdivisionScheme = "catmullClark"` (`glTF` has no subdivision) and `customData` (unless written to `extras`).

Exercise 27-B. `size = 2.5`. `chair_edits.usda` is the round-trip output; its `over` is stronger than the sublayered original. Verified by composing the two files.

DE-R27-01 — B. `glTF` has fixed conventions (meters, +Y up); the exporter must convert. A is wrong: `glTF` has no `metersPerUnit`. Review: §27.4, Chapter 28.

DE-R27-02 — A, C. `glTF` attributes are per-vertex, and its UV origin is top-left. Curves and `extras` are not how UVs are carried. Review: §27.4.

DE-R27-03 — B. `nodegraph` ↔ `UsdShade.NodeGraph`; `surfacematerial` ↔ `Material`. Review: §27.4.

DE-R27-04 — C. Dropped concepts and their reason go in the fidelity column. Review: §27.3.

DE-R27-05 — B. A sparse override keeps the original (with its variants and custom data) untouched. Formats, default prim, and names do not restore lost data. Review: §27.5.

DE-R27-06 — B. A polygon with n vertices fan-triangulates into $n - 2$ triangles: $2 + 2 + 1 = 5$. D (11) is the vertex count, not the triangle count. Review: §27.4.

DE-R27-07 — A. `chair.usda` alone authors `size = 2`. Review: §27.5.

DE-R27-08 — A, C. `roughness` → `roughnessFactor`, `diffuseColor` → `baseColorFactor`. `clearcoat` and `ior` need extensions. Review: §27.4.

DE-R27-09 — B. Renaming to the target’s naming rules is a transformation. Review: §27.2.

DE-R27-10 — B. USDA and USDC are two encodings of the same data. Review: §27.1, Chapter 31.

CHAPTER 28

Chapter 28 — Units, Axes, and Naming

Exam domain: Data Exchange (15%) · **Objectives:** 4.7 (supports 4.1, 7.1) · **Study day:** 9 · **Est. time:** 75 min **Prerequisites:** Ch 2 (stage metadata), Ch 4 (prim paths), Ch 16 (references), Ch 27 (extract → transform → validate); xformOps are introduced in Ch 13 and taught fully in Ch 39

Learning goals

- Read and write a stage's linear unit with `UsdGeom.SetStageMetersPerUnit` / `GetStageMetersPerUnit` and the `UsdGeom.LinearUnits` constants.
- Read and write a stage's up axis with `UsdGeom.SetStageUpAxis` / `GetStageUpAxis`.
- Explain why USD does not convert units or axes when composing, and add the corrective transform yourself.
- Turn arbitrary source names into valid, unique prim names with `Tf.MakeValidIdentifier`.
- Explain mesh orientation (`rightHanded` / `leftHanded`) and convert data from a left-handed source.

Key terms

Term	One-line definition
Linear unit	The real-world length of one scene unit
metersPerUnit	Layer metadata on the root layer: how many meters one scene unit represents
UsdGeom.LinearUnits	Named constants for common <code>metersPerUnit</code> values (<code>centimeters = 0.01</code> , ...)
upAxis	Layer metadata on the root layer: which axis points “up”, “Y” or “Z”
Fallback	The value USD reports when nothing is authored
Corrective transform	A scale or rotation you add where an asset is referenced, to match the consuming stage
Identifier	A name that is legal as a prim name or property-name part
Handedness	Whether a coordinate system (or a face's winding) follows the right-hand or left-hand rule
Winding order	The order in which a face lists its vertices; it decides which side is the front
orientation	A <code>Gprim</code> attribute that says how to read winding order: <code>rightHanded</code> (fallback) or <code>leftHanded</code>

28.1 `metersPerUnit` and `UsdGeom.LinearUnits`

What is it? `metersPerUnit` is stage metadata that says how long one scene unit is, in meters. `UsdGeom.LinearUnits` gives names to common values, such as `UsdGeom.LinearUnits.centimeters` (0.01).

Why do we need it? A point at `(0, 100, 0)` is meaningless until you know the unit. In centimeters it is 1 m up; in meters it is 100 m up. Without declared units, an importer has to guess, and a guess is how a chair ends up the size of a building.

Beginner explanation. It is the scale bar printed on a map: “1 cm = 1 km”. The map is the same drawing either way; the scale bar tells you how to read it.

Where the analogy breaks: a map reader converts automatically in their head. USD never converts for you (section 28.3). It only records the scale bar.

Technical explanation. - `metersPerUnit` is a `double` in the root layer’s metadata block. Only the stage’s **root layer** (and session layer) counts. Values in sublayers or referenced layers are ignored for the stage. - Read it with `UsdGeom.GetStageMetersPerUnit(stage)`. If unauthored, it returns the **fallback 0.01** (centimeters). Use

`UsdGeom.StageHasAuthoredMetersPerUnit(stage)` to tell “authored 0.01” from “nothing authored”. - Write it with `UsdGeom.SetStageMetersPerUnit(stage, value)`. - Compare with `UsdGeom.LinearUnitsAre(a, b)`, which uses a small tolerance, instead of `==` on floats. - `UsdGeom.LinearUnits` constants: `nanometers` 1e-9, `micrometers` 1e-6, `millimeters` 0.001, `centimeters` 0.01, `decimeters` 0.1, `meters` 1.0, `kilometers` 1000, `lightYears`, `inches` 0.0254, `feet` 0.3048, `yards` 0.9144, `miles` 1609.344. - Converting a length from stage A to stage B: `value_B = value_A * mpu_A / mpu_B`.

VERSION

Verified on USD 26.08: an unauthored stage reports ``metersPerUnit` 0.01`, and a ``metersPerUnit`` authored only in a sublayer is not seen by ``GetStageMetersPerUnit``.

Mental model.

root layer metadata	every length in the stage
+-----+	+-----+
metersPerUnit = mpu	--> meters = value * mpu
+-----+	+-----+
(nothing authored -> mpu = 0.01, centimeters)	

Simple example. | `metersPerUnit` | A cube with size = 2 is... |

|-----|-----| | 0.01 | 2 cm | | 1.0 | 2 m | | 0.0254 | 2 inches (5.08 cm) |

USDA example.

```
#usda 1.0
(
  defaultPrim = "Bolt"
  metersPerUnit = 0.001
  upAxis = "Z"
)

def Cylinder "Bolt"
{
  double height = 40
  double radius = 4
}
```

- `metersPerUnit = 0.001` means millimeters: the bolt is 40 mm long and 4 mm in radius.
- The metadata lives in the layer’s header block, not on any prim.

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
print("authored:", UsdGeom.StageHasAuthoredMetersPerUnit(stage))
print("fallback mpu:", UsdGeom.GetStageMetersPerUnit(stage))

UsdGeom.SetStageMetersPerUnit(stage, UsdGeom.LinearUnits.millimeters)
mpu = UsdGeom.GetStageMetersPerUnit(stage)
print("authored:", UsdGeom.StageHasAuthoredMetersPerUnit(stage), "mpu:", mpu)
print("is mm:", UsdGeom.LinearUnitsAre(mpu, UsdGeom.LinearUnits.millimeters))
print(stage.GetRootLayer().ExportToString().strip())

def convert(value, from_mpu, to_mpu):
    return value * from_mpu / to_mpu

print("250 mm in cm:", convert(250, mpu, UsdGeom.LinearUnits.centimeters))
print("6 ft in m:", round(convert(6, UsdGeom.LinearUnits.feet, 1.0), 4))
```

Expected output

```

authored: False
fallback mpu: 0.01
authored: True mpu: 0.001
is mm: True
#usda 1.0
(
    metersPerUnit = 0.001
)
250 mm in cm: 25.0
6 ft in m: 1.8288

```

Real-world use case. A manufacturing digital twin combines CAD parts in millimeters with a factory building in meters. Every part file declares `metersPerUnit = 0.001`, the factory stage declares `1`, and the assembly tool reads both values to compute the scale for each referenced part.

Common mistakes.

MISTAKE

Not authoring ``metersPerUnit`` and assuming the reader will "know". The fallback is centimeters, which is wrong for most CAD and game data. Fix: always author it explicitly.

MISTAKE

Comparing units with ``==`` (``mpu == 0.01``). Floats from different tools differ slightly. Fix: ``UsdGeom.LinearUnitsAre(mpu, UsdGeom.LinearUnits.centimeters)``.

Exam traps.

TRAP

"The default ``metersPerUnit`` is 1 (meters)." No. The fallback when nothing is authored is 0.01 (centimeters).

TRAP

``metersPerUnit`` is layer metadata, not a prim attribute. An answer that sets it with ``prim.CreateAttribute("metersPerUnit", ...)`` is wrong.

Exam takeaways.

KEY

- ``metersPerUnit`` = meters per scene unit, on the root layer; fallback 0.01. -
- ``UsdGeom.SetStageMetersPerUnit`` / ``GetStageMetersPerUnit`` /
- ``StageHasAuthoredMetersPerUnit``. - Use ``UsdGeom.LinearUnits.*`` constants and
- ``LinearUnitsAre`` to compare. - Convert: ``value_B = value_A * mpu_A / mpu_B``.

28.2 upAxis

What is it? `upAxis` is stage metadata that says which axis points up: "Y" or "Z".

Why do we need it? Different tools disagree. Maya and most game engines are Y-up; Blender, 3ds Max, Unreal, and most CAD tools are Z-up. If an importer reads Z-up data as Y-up, the model lies on its back.

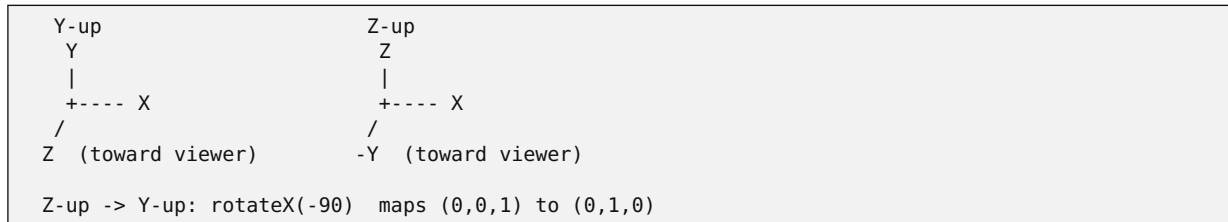
Beginner explanation. It is the "this side up" arrow printed on a shipping box. The contents are the same; the arrow tells you how to stand the box.

Where the analogy breaks: USD prints the arrow but never turns the box for you. Whoever combines data must rotate it.

Technical explanation. - Write with `UsdGeom.SetStageUpAxis(stage, UsdGeom.Tokens.y)` or `UsdGeom.Tokens.z`. Any other value (for example "X") raises `Tf.ErrorException`. - Read with `UsdGeom.GetStageUpAxis(stage)`. If unauthored, it returns the fallback from `UsdGeom.GetFallbackUpAxis()`, which is "Y" in a default installation. - Like `metersPerUnit`, only the root layer's value counts. - To bring Z-up content into a Y-up stage, rotate -90°

around X (`xformOp:rotateX = -90`). To bring Y-up content into a Z-up stage, rotate $+90^\circ$ around X. - `upAxis` is a declaration for tools (cameras, viewers, importers). It does not move any geometry.

Mental model.



Simple example. A tree modeled Z-up has its trunk along +Z. In a Y-up stage, without correction, the trunk points toward the camera. Adding `rotateX = -90` on the referencing prim stands it up along +Y.

USDA example.

```
#usda 1.0
(
  upAxis = "Z"
)

def Xform "Tree"
{
  def Cylinder "Trunk"
  {
    uniform token axis = "Z"
    double height = 5
  }
}
```

- `upAxis = "Z"` declares the stage Z-up.
- The Cylinder's own `axis` attribute is unrelated: it only says which way that cylinder's height runs. Stage `upAxis` never changes it.

Python example.

```
from pxr import Gf, Tf, Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
print("fallback:", UsdGeom.GetStageUpAxis(stage), UsdGeom.GetFallbackUpAxis())
UsdGeom.SetStageUpAxis(stage, UsdGeom.Tokens.z)
print("authored:", UsdGeom.GetStageUpAxis(stage))
try:
    UsdGeom.SetStageUpAxis(stage, "X")
except Tf.ErrorException:
    print("'X' rejected, still:", UsdGeom.GetStageUpAxis(stage))

z_to_y = Gf.Matrix4d().SetRotate(Gf.Rotation(Gf.Vec3d(1, 0, 0), -90))
up = z_to_y.TransformDir(Gf.Vec3d(0, 0, 1))
print("Z-up 'up' after rotateX(-90):", tuple(round(c, 6) + 0.0 for c in up))
```

Expected output

```
fallback: Y Y
authored: Z
'X' rejected, still: Z
Z-up 'up' after rotateX(-90): (0.0, 1.0, 0.0)
```

The `+ 0.0` turns a possible `-0.0` into `0.0` so the printout is tidy.

Real-world use case. A game studio's level editor is Y-up, while its environment artists model in a Z-up DCC. The exporter authors `upAxis = "Z"` honestly, and the level-assembly script adds `rotateX = -90` on every Z-up asset it references, based on reading each asset's `upAxis`.

Common mistakes.**MISTAKE**

Changing ``upAxis`` on an existing asset to "fix" it in a viewer. The geometry does not move; you just made the declaration lie. Fix: leave the declaration truthful and rotate where the asset is consumed.

MISTAKE

Passing ``"x"`` or ``"X"`` for an X-up source. USD supports only Y and Z. Fix: rotate X-up data into Y or Z during the transform stage.

Exam traps.**TRAP**

"``SetStageUpAxis`` rotates the geometry." No. It writes metadata only.

TRAP

Confusing stage ``upAxis`` with a ``Cylinder``/``Capsule``/``Cone`` prim's ``axis`` attribute. They are separate.

Exam takeaways.**KEY**

- ``upAxis`` is ``"Y"`` or ``"Z"`` only; fallback ``"Y"``. - ``UsdGeom.SetStageUpAxis`` / ``GetStageUpAxis`` / ``GetFallbackUpAxis``. - It is a declaration; it moves nothing. - Z-up → Y-up: ``rotateX = -90``.

28.3 USD does not auto-convert units (why)

What is it? When you reference, payload, or sublayer a file whose `metersPerUnit` or `upAxis` differs from your stage's, USD composes the values **unchanged**. No scale or rotation is added.

Why do we need it? You need to know this because it is a classic source of "the asset is 100× too big" bugs. The consumer of an asset is responsible for adding a **corrective transform**.

Beginner explanation. Two recipes, one in grams and one in ounces, are stapled into one cookbook. The stapler does not rewrite the numbers. The cook has to notice and convert.

Where the analogy breaks: in USD you fix it once, with one scale on the referencing prim, and every number below it is converted by the transform hierarchy.

Technical explanation. Why USD does not convert:

- **Stage metrics are root-layer metadata.** Composition arcs bring in prims and their opinions, not another layer's stage-level metadata. The referenced file's `metersPerUnit` is simply not part of the composed stage.
- **No single right place.** A conversion could go on the referencing prim, on each child, or into the points. USD will not guess and silently change authored data.
- **Predictability and speed.** Composed values are exactly what was authored, which keeps value resolution fast and debuggable.

What you do instead (part of the transform stage of Chapter 27):

1. Read the asset's units and axis: open the asset (`Usd.Stage.Open`) and call `GetStageMetersPerUnit` / `GetStageUpAxis`.
2. Compute the scale factor: `asset_mpu / stage_mpu`. Meters into a centimeter stage: `1 / 0.01 = 100`.

3. Add `xformOps` on the **referencing prim**: a uniform `xformOp:scale` and, if the axes differ, `xformOp:rotateX` (−90 for Z-up into Y-up).
4. Validate: a pipeline check (Chapter 30) can compare each referenced asset's metrics with the stage's.

Mental model.

```
shot.usda (cm, Y-up)          chair.usda (m, Z-up)
/World/Chair --references--> /Chair      (Xform, no ops)
  scale = 100    <-- you add this /Chair/Geom Cube size = 1
  rotateX = -90  <-- and this
USD itself adds NOTHING. Values arrive as authored: size = 1.
```

Simple example. `chair_m.usda` (meters) has a cube with `size = 1` (1 m). Referenced into `shot.usda` (centimeters), the composed `size` is still 1, which the shot reads as 1 cm. With `scale = 100` on `/World/Chair`, the world-space size is 100 cm.

USDA example. File: `shot.usda` (the corrected consumer; it references the chair asset)

```
#usda 1.0
(
  metersPerUnit = 0.01
  upAxis = "Y"
)

def Xform "World"
{
  def "Chair" (
    prepend references = @chair_m.usda@
  )
  {
    float xformOp:rotateX = -90
    float3 xformOp:scale = (100, 100, 100)
    uniform token[] xformOpOrder = ["xformOp:rotateX", "xformOp:scale"]
  }
}
```

- `def "Chair"` has no type, so the prim takes whatever type the asset's root has (here `Xform`). A local type such as `def Xform` would be a stronger opinion and would replace a `Cube` or `Mesh` root type.
- The reference brings `/Chair` and its child `Geom` from `chair_m.usda` with their values untouched.
- `xformOp:scale = 100` converts meters to centimeters; `rotateX = -90` converts Z-up to Y-up.
- The corrective ops live on the referencing prim, so the asset file stays untouched and reusable.

Python example. This script writes a meters, Z-up asset, references it into a centimeters, Y-up stage, shows that nothing is converted, then adds the corrective ops.

```
from pxr import Usd, UsdGeom

asset = Usd.Stage.CreateNew("chair_m.usda")
UsdGeom.SetStageMetersPerUnit(asset, UsdGeom.LinearUnits.meters)
UsdGeom.SetStageUpAxis(asset, UsdGeom.Tokens.z)
root = UsdGeom.Xform.Define(asset, "/Chair")
cube = UsdGeom.Cube.Define(asset, "/Chair/Geom")
cube.GetSizeAttr().Set(1.0)
cube.AddTranslateOp().Set((0, 0, 0.5)) # sits on the ground, Z-up
asset.SetDefaultPrim(root.GetPrim())
asset.Save()

shot = Usd.Stage.CreateInMemory()
UsdGeom.SetStageMetersPerUnit(shot, UsdGeom.LinearUnits.centimeters)
UsdGeom.SetStageUpAxis(shot, UsdGeom.Tokens.y)
UsdGeom.Xform.Define(shot, "/World")
prim = shot.DefinePrim("/World/Chair") # typeless: type comes from asset
```

```

prim.GetReferences().AddReference("chair_m.usda")
chair = UsdGeom.Xformable(prim)

def world_range(stage, path):
    cache = UsdGeom.BBoxCache(Usd.TimeCode.Default(), ["default"])
    r = cache.ComputeWorldBound(stage.GetPrimAtPath(path)).ComputeAlignedRange()
    fix = lambda v: tuple(round(c, 3) + 0.0 for c in v)
    return fix(r.GetMin()), fix(r.GetMax())

geom = UsdGeom.Cube(shot.GetPrimAtPath("/World/Chair/Geom"))
print("composed size:", geom.GetSizeAttr().Get())
print("before:", world_range(shot, "/World/Chair"))

src = Usd.Stage.Open("chair_m.usda")
factor = UsdGeom.GetStageMetersPerUnit(src) / UsdGeom.GetStageMetersPerUnit(shot)
if UsdGeom.GetStageUpAxis(src) != UsdGeom.GetStageUpAxis(shot):
    chair.AddRotateXOp().Set(-90.0)
chair.AddScaleOp().Set((factor, factor, factor))
print("factor:", factor)
print("after: ", world_range(shot, "/World/Chair"))

```

Expected output

```

composed size: 1.0
before: ((-0.5, -0.5, 0.0), (0.5, 0.5, 1.0))
factor: 100.0
after: ((-50.0, 0.0, -50.0), (50.0, 100.0, 50.0))

```

Before correction, the chair is 1 unit (1 cm) tall along Z. After, it is 100 cm tall along +Y, standing on the ground.

VERSION

Verified on USD 26.08: referencing a `metersPerUnit = 1`, `upAxis = "Z"` asset into a `0.01`, `Y` stage leaves all values unchanged and emits no warning.

Real-world use case. A VFX studio's sets are in centimeters, Y-up. It licenses a vehicle library authored in meters, Z-up. Its set-dressing tool reads each library asset's metrics when referencing it and authors a scale and rotateX on the reference prim. A validator flags any reference to a mismatched asset that lacks the correction.

Common mistakes.

MISTAKE

Fixing units by editing the asset's points. Every other consumer that already corrected for meters now gets the wrong size. Fix: correct on the referencing prim.

MISTAKE

Adding corrective ops on a referencing prim whose asset root already has its own `xformOpOrder`. Your local `xformOpOrder` is a stronger opinion and replaces the asset's whole list, so the asset's own ops (for example its translate) stop applying. Verified on USD 26.08. Fix: keep asset roots free of xformOps (put them on children, as in the example), or put the correction on a parent "wrapper" Xform above the referencing prim.

MISTAKE

Creating the referencing prim with `UsdGeom.Xform.Define` when the asset's root is a `Cube` or `Mesh`. The local `Xform` type is stronger than the referenced type, so the geometry disappears (empty bounds). Fix: use a typeless `stage.DefinePrim(path)` and wrap it with `UsdGeom.Xformable` to add the corrective ops.

MISTAKE

Reading the asset's units from the composed stage. The composed stage reports its own root layer's metrics, not the asset's. Fix: open the asset file separately and read its metrics.

Exam traps.

TRAP

"USD automatically rescales references whose ``metersPerUnit`` differs." False. Values compose exactly as authored.

TRAP

Inverted factor. Meters into centimeters is $\times 100$ (``asset_mpu / stage_mpu = 1 / 0.01``), not $\times 0.01$.

Exam takeaways.**KEY**

- References, payloads, and sublayers never convert units or axes. - Corrective scale = ``asset_mpu / stage_mpu``; Z-up into Y-up = ``rotateX -90``. - Put corrections on the referencing prim; never edit the asset. - Read the asset's metrics by opening the asset itself.

28.4 Valid prim names (`Tf.MakeValidIdentifier`)

What is it? Prim names must be valid **identifiers**. `Tf.MakeValidIdentifier(name)` turns any string into a valid ASCII identifier by replacing illegal characters with `_`.

Why do we need it? Source formats allow names like `"Door - Left (2)"` or `"1st floor"`. USD rejects these as prim names, so an exporter that passes them straight through crashes or produces invalid paths.

Beginner explanation. It is like a filing clerk who renames folders to fit the filing system's rules: no spaces, no punctuation, don't start with a number.

Where the analogy breaks: the clerk might rename two different folders to the same name. `Tf.MakeValidIdentifier` can do that too, so you must de-duplicate yourself.

Technical explanation. - **ASCII rule:** first character a letter or `_`, then letters, digits, or `_`. `Tf.IsValidIdentifier(name)` checks exactly this. - **UTF-8 rule (USD 24.03 and later):** prim names may use Unicode letters (first character from the Unicode `XID_Start` class or `_`, then `XID_Continue` characters). `Sdf.Path.IsValidIdentifier("café")` is `True`. Digits, spaces, `-`, and `.` are still not allowed where ASCII forbids them. -

`Tf.MakeValidIdentifier` produces ASCII only. It replaces each invalid byte with `_`, so `"café"` becomes `"caf_"` (é is two bytes in UTF-8), and a leading digit becomes `_`. An empty string becomes `"_"`. - **Collisions:** `"my chair"`, `"my-chair"`, and `"my.chair"` all become `"my_chair"`. An exporter must add suffixes to keep sibling names unique, and should store the original name (for example in `customData` or `displayName` metadata) for the round trip. - **Property names** follow the same rule per part, with `:` separating namespaces (`primvars:st`). Check with `Sdf.Path.IsValidNamespacedIdentifier`. - Defining a prim with an invalid name (`stage.DefinePrim("/my chair")`) raises `Tf.ErrorException`.

VERSION

Verified on USD 26.08. UTF-8 prim and property names have been allowed since USD 24.03. ``Tf.IsValidIdentifier`` and ``Tf.MakeValidIdentifier`` remain ASCII-only; ``Sdf.Path.IsValidIdentifier`` applies the UTF-8 rule. Older USD versions reject all non-ASCII names.

Mental model.

```
"1st floor" -> MakeValidIdentifier -> "_st_floor"
"my-chair"  ->                    -> "my_chair"  \ collision!
"my chair"  ->                    -> "my_chair"  / add _1, _2 ...
"café"      ->                    -> "caf_"      (ASCII only)
```

Simple example. | Source name | Valid as is? (Sdf.Path.IsValidIdentifier) |
 Tf.MakeValidIdentifier | |-----|-----|-----|
 Chair_01 | yes | Chair_01 | | my chair | no | my_chair | | 1st_floor | no | _st_floor | |
 café | yes (UTF-8, 24.03+) | caf__ |

USDA example.

```
#usda 1.0

def Xform "Building"
{
  def Xform "_st_floor" (
    customData = {
      string sourceName = "1st floor"
    }
  )
  {
  }

  def Xform "café"
  {
  }
}
```

- `_st_floor` is the sanitized name; `customData` keeps the original for round trips and UIs.
- `café` is a legal UTF-8 prim name in USD 24.03 and later.

Python example.

```
from pxr import Sdf, Tf, Usd

for name in ["Chair_01", "my chair", "1st_floor", "door-left", "café", ""]:
    print(repr(name), Sdf.Path.IsValidIdentifier(name),
          Tf.IsValidIdentifier(name), repr(Tf.MakeValidIdentifier(name)))

def unique_names(raw_names):
    used, result = set(), []
    for raw in raw_names:
        base = name = Tf.MakeValidIdentifier(raw)
        i = 1
        while name in used:
            name, i = f"{base}_{i}", i + 1
        used.add(name)
        result.append(name)
    return result

print(unique_names(["my chair", "my-chair", "my.chair"]))

stage = Usd.Stage.CreateInMemory()
print(stage.DefinePrim("/café").GetName())
try:
    stage.DefinePrim("/my chair")
except Tf.ErrorException:
    print("/my chair" rejected)
```

Expected output

```
'Chair_01' True True 'Chair_01'
'my chair' False False 'my_chair'
'1st_floor' False False '_st_floor'
'door-left' False False 'door_left'
'café' True False 'caf__'
'' False False '_'
['my_chair', 'my_chair_1', 'my_chair_2']
café
'/my chair' rejected
```

The invalid path also prints a warning to standard error; only standard output is shown here.

Real-world use case. An architecture (AEC) exporter reads BIM element names such as "Wall - Exterior (Type 2)". It sanitizes them with `Tf.MakeValidIdentifier`, de-duplicates siblings, and stores the original name and the BIM element ID in `customData`, so a round trip back to the BIM tool can match elements by ID, not by name.

Common mistakes.

MISTAKE

Sanitizing without de-duplicating. Two source objects get the same prim path, and the second silently merges into the first. Fix: add suffixes and track used names per parent.

MISTAKE

Using `Tf.IsValidIdentifier`` to reject UTF-8 names in a 24.03+ pipeline. It is ASCII-only. Fix: use `Sdf.Path.IsValidIdentifier`` if you want to allow UTF-8, or sanitize deliberately if downstream tools need ASCII.

Exam traps.

TRAP

"`Tf.MakeValidIdentifier`` guarantees unique names." No. It only makes names legal.

TRAP

"Prim names must be ASCII." Outdated since USD 24.03; UTF-8 names are valid. Older tools in a pipeline may still reject them.

Exam takeaways.

KEY

- Identifier: letter or ``_`` first, then letters, digits, ``_``; UTF-8 letters allowed since 24.03. - `Tf.MakeValidIdentifier`` makes an ASCII-legal name; it does not make it unique. - `Sdf.Path.IsValidIdentifier`` = UTF-8-aware check; `Tf.IsValidIdentifier`` = ASCII-only check.
- Keep the original name in `customData`` for round trips.

28.5 Coordinate-system handedness

What is it? Handedness says whether a coordinate system follows the right-hand rule or the left-hand rule. USD uses a right-handed coordinate system. On meshes, the `orientation` attribute (`rightHanded` or `leftHanded`) says how to read each face's **winding order**, which decides which side of the face is the front.

Why do we need it? Some sources are left-handed (Unity is left-handed Y-up, Unreal is left-handed Z-up, and many CAD tools vary). Bringing their data in without conversion mirrors the model, and faces may point inward, so back-face culling hides them or lighting looks inverted.

Beginner explanation. Curl the fingers of your right hand in the order a face lists its corners; your thumb points out of the front. That is `rightHanded`. Do the same with your left hand and the thumb points the other way: `leftHanded`.

Where the analogy breaks: USD's whole coordinate system is always right-handed. `orientation` changes only how a mesh's winding is read, not the axes.

Technical explanation. - `orientation` is a uniform token on `UsdGeom.Gprim` (so on `Mesh` and other `gprims`). Fallback: `rightHanded`. - With `rightHanded`, a face whose vertices go counterclockwise when viewed from the front faces you. With `leftHanded`, the front is the other side. Normal direction and back-face culling follow from this. - Setting `orientation = "leftHanded"` gives the same visual result as reversing every face's vertex order. -

Converting a left-handed source (a coordinate-system change): mirror one axis (for example negate Z on points and normals), which flips winding, then either reverse each face's vertex order or set `orientation = "leftHanded"`. Reversing is friendlier to tools

that ignore `orientation`. - A transform with a negative determinant (an odd number of negative scale components) mirrors geometry and also flips the apparent facing direction in world space.

Mental model.

v2	v2
\ 0,1,2 counterclock-	\ same face read leftHanded,
\ wise, rightHanded:	\ or indices 0,2,1:
\ front faces +Z	\ front faces -Z
v0--v1	v0--v1

Simple example. Triangle points $(0,0,0)$, $(1,0,0)$, $(0,1,0)$, indices $[0, 1, 2]$. Right-handed: normal $(0, 0, 1)$. Left-handed, or indices $[0, 2, 1]$: normal $(0, 0, -1)$.

USDA example.

```
#usda 1.0

def Mesh "FromLeftHandedTool"
{
    int[] faceVertexCounts = [3]
    int[] faceVertexIndices = [0, 1, 2]
    uniform token orientation = "leftHanded"
    point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
}
```

- `orientation = "leftHanded"` tells renderers to read the winding the other way, so this triangle faces $-Z$.
- Leaving it out means `rightHanded`, and the triangle would face $+Z$.

Python example. USD has no “compute face normal” helper, so this computes it with the right-hand rule (`Gf.Cross`) and applies `orientation`.

```
from pxr import Gf, Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/Tri")
pts = [Gf.Vec3f(0, 0, 0), Gf.Vec3f(1, 0, 0), Gf.Vec3f(0, 1, 0)]
mesh.CreatePointsAttr(pts)
mesh.CreateFaceVertexCountsAttr([3])
mesh.CreateFaceVertexIndicesAttr([0, 1, 2])

def face_normal(points, idx, orientation):
    a, b, c = (points[i] for i in idx)
    n = Gf.Cross(b - a, c - a).GetNormalized()
    n = n if orientation == UsdGeom.Tokens.rightHanded else -n
    return tuple(round(x, 3) + 0.0 for x in n)

orient = mesh.GetOrientationAttr()
print("fallback:", orient.Get(), face_normal(pts, [0, 1, 2], orient.Get()))
orient.Set(UsdGeom.Tokens.leftHanded)
print("leftHanded:", face_normal(pts, [0, 1, 2], orient.Get()))
print("reversed winding, rightHanded:", face_normal(pts, [0, 2, 1], "rightHanded"))

# Import from a left-handed tool: mirror Z, then reverse winding.
src = [Gf.Vec3f(0, 0, 0), Gf.Vec3f(1, 0, 0), Gf.Vec3f(0, 0, 1)]
mirrored = [Gf.Vec3f(p[0], p[1], -p[2]) for p in src]
print("mirrored + reversed:", face_normal(mirrored, [0, 2, 1], "rightHanded"))
```

Expected output

```
fallback: rightHanded (0.0, 0.0, 1.0)
leftHanded: (0.0, 0.0, -1.0)
reversed winding, rightHanded: (0.0, 0.0, -1.0)
mirrored + reversed: (0.0, -1.0, 0.0)
```

In the source tool’s left-handed space, that triangle faced $-Y$. After mirroring Z and reversing winding, it still faces $-Y$ in USD: the facing direction survived the conversion.

Real-world use case. A robotics company simulates in USD but receives environment meshes from a left-handed game engine. Its importer mirrors Z, reverses face winding, and authors `upAxis` and `metersPerUnit`, so collision normals in the physics simulation point outward.

Common mistakes.

MISTAKE

Mirroring points from a left-handed source but not fixing the winding. All faces now point inward. Fix: reverse vertex order per face, or set ``orientation = "leftHanded"``.

MISTAKE

Flipping normals to fix inside-out shading while leaving winding wrong. Culling still hides the faces. Fix: correct the winding (or ``orientation``) first, then recompute normals.

Exam traps.

TRAP

"USD supports left-handed coordinate systems via ``orientation``." No. USD's coordinate system is right-handed; ``orientation`` only says how to read a `gprim`'s face winding.

TRAP

The fallback for ``orientation`` is ``rightHanded``, not `unset` or ``leftHanded``.

Exam takeaways.

KEY

- USD's coordinate system is right-handed. - ``orientation`` on `gprims`: ``rightHanded`` (fallback) or ``leftHanded``; it controls which side is front. - ``leftHanded`` = reversed winding. - Left-handed source: mirror one axis, then fix winding.

Chapter lab(s)

Lab 26 — Units and up-axis handling (★★☆, Obj 4.7). You build assets in different units and axes, reference them into one stage, confirm that USD changes nothing, and write a helper that adds the corrective `scale` and `rotateX` on each reference. The lab ends with a check that every referenced asset's world bounds match its real-world size.

USDA reading exercise(s)

Exercise 28-A. `lamp.usda` is referenced into `room.usda`. What is the lamp's world-space height in `room.usda` units, and how tall is it in meters?

File: *lamp.usda*

```
#usda 1.0
(
    defaultPrim = "Lamp"
    metersPerUnit = 1
    upAxis = "Y"
)

def Cube "Lamp"
{
    double size = 0.5
}
```

File: *room.usda*

```
#usda 1.0
(
    metersPerUnit = 0.01
    upAxis = "Y"
)
```



```
def Xform "Room"
{
    def "Lamp" (
        prepend references = @lamp.usda@
    )
    {
    }
}
```

Exercise 28-B. Which of these prim names fail in USD 26.08: `wall_A`, `wall A`, `Wänd`, `3D_Text`?

Chapter review

Summary

- `metersPerUnit` and `upAxis` are root-layer metadata; fallbacks are 0.01 and "Y".
- Use `UsdGeom.SetStageMetersPerUnit/GetStageMetersPerUnit`, `UsdGeom.LinearUnits`, `LinearUnitsAre`.
- `upAxis` accepts only "Y" or "Z"; it is a declaration and moves nothing.
- Composition never converts units or axes; the consumer adds `scale = asset_mpu / stage_mpu` and `rotateX` on the referencing prim.
- Metrics in sublayers and referenced layers are ignored by the stage.
- `Tf.MakeValidIdentifier` makes ASCII-legal names; de-duplicate yourself; keep the original name.
- UTF-8 prim names are legal since USD 24.03 (`Sdf.Path.IsValidIdentifier`).
- USD is right-handed; gprim orientation (`rightHanded` fallback) decides the front side of faces.

If you see... → think...

If you see...	Think...
Referenced asset 100× too big or small	Unit mismatch; add <code>scale = asset_mpu / stage_mpu</code> on the reference prim
Asset lying on its back	Z-up vs Y-up; <code>rotateX ±90</code>
<code>GetStageMetersPerUnit</code> returns 0.01 unexpectedly	Nothing authored on the root layer (fallback)
<code>SetStageUpAxis(stage, "X")</code>	Error: only Y or Z
Name with spaces, dashes, leading digit	<code>Tf.MakeValidIdentifier</code> + de-duplication
Non-ASCII name rejected by <code>Tf.IsValidIdentifier</code>	ASCII-only check; <code>Sdf.Path.IsValidIdentifier</code> allows UTF-8 (24.03+)
Faces inside out after import	Winding / <code>orientation</code> ; mirror + reverse for left-handed sources

Review questions

DE-R28-01 · Obj 4.7 · Single choice What does `UsdGeom.GetStageMetersPerUnit` return for a new stage with no metadata? A. 1.0 B. 0.01 C. 0.001 D. 0.0

DE-R28-02 · Obj 4.7 · Single choice A meters asset is referenced into a millimeter stage. What uniform scale corrects it on the referencing prim? A. 0.001 B. 1 C. 100 D. 1000

DE-R28-03 · Obj 4.7 · Select two. Which statements about composition and units are true? A. References preserve authored values exactly. B. USD rescales payloads automatically. C. Only the root layer's `metersPerUnit` affects `GetStageMetersPerUnit`. D. A sublayer's `upAxis` overrides the root layer's.

DE-R28-04 · Obj 4.7 · Single choice What happens when you call `UsdGeom.SetStageUpAxis(stage, "X")` in USD 26.08? A. The stage becomes X-up. B. A `Tf.ErrorException` is raised and the value is not changed. C. The value is silently changed to "Y". D. The geometry is rotated.

DE-R28-05 · Obj 4.7 · Single choice What does this print?

```
from pxr import Tf
print(Tf.MakeValidIdentifier("Door Left-2"))
```

A. Door Left-2 B. DoorLeft2 C. Door_Left_2 D. _oor_Left_2

DE-R28-06 · Obj 4.7 · Single choice An exporter sanitizes "Pipe A" and "Pipe-A" and writes both under the same parent. What goes wrong? A. Nothing; `MakeValidIdentifier` adds suffixes. B. Both map to `Pipe_A`, so the second merges into the first. C. USD rejects the layer on save. D. The prims become instances.

DE-R28-07 · Obj 4.7 · Single choice Which check accepts the prim name `Tür` in USD 26.08? A. `Tf.IsValidIdentifier` B. `Sdf.Path.IsValidIdentifier` C. Both D. Neither

DE-R28-08 · Obj 4.7 · USDA reading A mesh has `uniform token orientation = "leftHanded"` and indices `[0, 1, 2]` with points `(0,0,0)`, `(1,0,0)`, `(0,1,0)`. Which way does the front face point? A. +Z B. -Z C. +X D. +Y

DE-R28-09 · Obj 4.7 · Select two. Which fixes correct a Z-up, meters asset referenced into a Y-up, centimeters stage? A. `xformOp:rotateX = -90` on the referencing prim B. Change the asset's `upAxis` to "Y" C. `xformOp:scale = (100, 100, 100)` on the referencing prim D. `UsdGeom.SetStageMetersPerUnit(stage, 1.0)` on the asset

DE-R28-10 · Obj 4.7 · Single choice What is the value of `UsdGeom.LinearUnits.inches`? A. 0.0254 B. 2.54 C. 0.3048 D. 25.4

Answers

Exercise 28-A. The composed `size` is 0.5 (unchanged by the reference), so the lamp is **0.5 room units** tall, which `room.usda` reads as 0.5 cm. It was meant to be 0.5 m. A `scale = 100` on `/Room/Lamp` fixes it (50 units = 50 cm = 0.5 m). Verified by composing the files on USD 26.08.

Exercise 28-B. `wall A` (space) and `3D_Text` (leading digit) fail. `wall_A` is ASCII-valid; `Wand` is a valid UTF-8 name (24.03+).

DE-R28-01 — B. Unauthored → fallback 0.01 (centimeters). Review: §28.1.

DE-R28-02 — D. `asset_mpu / stage_mpu = 1 / 0.001 = 1000`. A is the inverse. Review: §28.3.

DE-R28-03 — A, C. USD never rescales (B false), and sublayer metrics are ignored (D false). Review: §28.1, §28.3.

DE-R28-04 — B. Only "Y" and "Z" are allowed; the call raises and nothing changes. Metadata never rotates geometry. Review: §28.2.

DE-R28-05 — C. Space and dash each become `_`. Letters and digits are kept. Review: §28.4.

DE-R28-06 — B. `MakeValidIdentifier` does not ensure uniqueness; same path means one prim receives both sets of opinions. Review: §28.4.

DE-R28-07 — B. `Sdf.Path.IsValidIdentifier` applies the UTF-8 rule; `Tf.IsValidIdentifier` is ASCII-only. Review: §28.4.

DE-R28-08 — B. Counterclockwise in XY would face +Z when right-handed; `leftHanded` flips it to -Z. Review: §28.5.

DE-R28-09 — A, C. Corrections go on the consumer's referencing prim. B and D change the asset's declarations without moving data, and other consumers break. Review: §28.3.

DE-R28-10 — A. One inch is 0.0254 m. C is feet. Review: §28.1.

CHAPTER 29

Chapter 29 — Writing Exporters, Converters, and Importers

Exam domain: Data Exchange (15%) · **Objectives:** 4.5, 4.7, 4.8 · **Study day:** 9 · **Est. time:** 120 min **Prerequisites:** Ch 9 (types), Ch 10 (time samples), Ch 13 (extents), Ch 27 (extract → transform → validate), Ch 28 (units, names)

Chapter 27 taught the ideas. This chapter writes the code: a converter you can run, materials and animation, nonstandard data (Obj 4.5), how a DCC importer should walk a stage (Obj 4.8), and **exporter hooks** that enforce structure before you save (Obj 4.7). Lab 25 is the longer version of 29.2.

Learning goals

- Structure an exporter as extract → transform → write → hook → validate.
- Convert a simple JSON (or OBJ) mesh to USD with units, names, kind, extent, and defaultPrim (Obj 4.7).
- Bind a `UsdPreviewSurface` with `MaterialBindingAPI`, and author animation as time samples.
- Carry nonstandard fields as namespaced custom attributes or `customData` until a schema exists (Obj 4.5).
- Design a DCC importer that reads the composed stage and round-trips extra data (Obj 4.8).
- Register post-write hooks that set `defaultPrim`, compute extents, and fail loudly.

Key terms

Term	One-line definition
Exporter	Code in a DCC (or a script) that writes the DCC’s scene into USD
Converter	A standalone program that reads file A and writes USD (or the reverse)
Importer	Code that reads USD and builds the DCC’s own objects
Hook	A small function the exporter calls at a known moment (usually after write, before save)
Neutral record	A dict or struct that is neither DCC-native nor USD-native
Custom attribute	A property not declared by a schema; USDA writes the <code>custom</code> keyword
Applied API schema	A schema you <code>Apply</code> so a prim legally holds extra properties (e.g. <code>MaterialBindingAPI</code>)
UsdPreviewSurface	The standard preview material shader id for interchange
Round trip	Export then import (or the reverse) with no unintended loss

29.1 Exporter structure

What is it? An **exporter** is a program with five jobs in a fixed order: **extract** the source scene, **transform** it into USD-shaped records, **write** prims, run **hooks** that enforce pipeline rules, then **validate**.

Why do we need it? A single 400-line “walk Maya and Set() everything” function cannot be tested, cannot report loss, and cannot be reused for a second DCC. Splitting the jobs lets you unit-test the math without opening Maya, and it gives the exam a vocabulary for Obj 4.7.

Beginner explanation. Think of a **shipping dock**. Extract is packing the warehouse inventory onto carts. Transform is labeling each carton in the customer's language (meters, legal names). Write is loading the truck (the layer). Hooks are the dock checklist (default prim? extents?). Validate is the weigh-station.

Where the analogy breaks: the dock can send a carton back to labeling. Validation errors should fix the transform, not get pasted over in the USDA by hand.

Technical explanation. | Step | Input | Output | Typical calls |
 |-----|-----|-----|-----| | Extract | DCC scene or file | Neutral records | DCC APIs, or
 json.load / an OBJ parser | | Transform | Records + mapping doc | USD-shaped records |
 Tf.MakeValidIdentifier, unit scale, axis, triangulation policy | | Write | USD-shaped
 records | Usd.Stage | Define, Create*Attr, SetDefaultPrim | | Hook | Stage | Same stage,
 rules applied | defaultPrim, kind, ComputeExtentFromPlugins, strip secrets | | Validate |
 Stage | Problem list | pxr.UsdValidation (Chapter 30) plus your checks |

Keep DCC imports inside **extract** only. Transform and write should run in a unit test with a fake list of records. That is how you prove Obj 4.7 without a license for the DCC.

Write through Usd for schema help (UsdGeom.Mesh.Define), or through Sdf plus Sdf.ChangeBlock when you author tens of thousands of prims (Chapters 8 and 34).

Mental model.

```

DCC / file
  | extract
  v
records --transform--> USD records --write--> stage
                                      |
                                      hooks + validate
                                      v
                                      layer.usd
  
```

Simple example. A fake DCC has two nodes. Extract yields two dicts. Transform fixes names. Write makes two Cubes. A hook sets defaultPrim if you forgot it.

USDA example. What a well-structured exporter should always leave on disk: stage metadata, a defaultPrim, and a model kind.

```

#usda 1.0
(
    defaultPrim = "Set"
    metersPerUnit = 1
    upAxis = "Y"
)

def Xform "Set" (
    kind = "assembly"
)
{
    def Cube "Box"
    {
        double size = 1
    }
}
  
```

Python example.

```

from pxr import Tf, Usd, UsdGeom

DCC_NODES = [
    {"name": "Box One", "size": 1.0},
    {"name": "2nd box", "size": 0.5},
]

def extract(nodes):
    return list(nodes)
  
```

```

def transform(records):
    return [{"name": Tf.MakeValidIdentifier(r["name"]),
            "size": r["size"]} for r in records]

def write(records):
    stage = Usd.Stage.CreateInMemory()
    UsdGeom.SetStageUpAxis(stage, "Y")
    UsdGeom.SetStageMetersPerUnit(stage, 1)
    root = UsdGeom.Xform.Define(stage, "/Set")
    root.GetPrim().SetMetadata("kind", "assembly")
    for r in records:
        cube = UsdGeom.Cube.Define(
            stage, root.GetPath().AppendChild(r["name"]))
        cube.GetSizeAttr().Set(r["size"])
    return stage

def hook_default_prim(stage):
    if not stage.GetDefaultPrim():
        stage.SetDefaultPrim(stage.GetPrimAtPath("/Set"))
    return str(stage.GetDefaultPrim().GetPath())

records = transform(extract(DCC_NODES))
stage = write(records)
print("children:", [c.GetName() for c in
                    stage.GetPrimAtPath("/Set").GetChildren()])
print("defaultPrim:", hook_default_prim(stage))
print("steps: extract -> transform -> write -> hook -> validate")

```

Expected output

```

children: ['Box_One', '_nd_box']
defaultPrim: /Set
steps: extract -> transform -> write -> hook -> validate

```

`Tf.MakeValidIdentifier("2nd box")` becomes `_nd_box` (a leading digit is replaced; Chapter 28).

Real-world use case. A CAD exporter at a factory: `extract` walks the CAD assembly, `transform` converts millimeters to meters and names, `write` authors component Xforms, `hooks` stamp `kind = component` and extents, `validate` runs `StageMetadataChecker` on a build server overnight.

Common mistakes.

MISTAKE

Calling DCC APIs from ``write()``. Tests then require the DCC. Keep DCC I/O in ``extract``.

MISTAKE

Saving before hooks. You ship files with no ``defaultPrim``. Call hooks, then ``Save()``.

Exam traps.

TRAP

"The exporter is the mapping document." The document is the spec; the exporter is the program (Chapter 27).

TRAP

"Validate, then write." If you validate an empty stage you always pass. Write, hook, then validate.

Exam takeaways.

KEY

- Five steps: extract, transform, write, hook, validate. - Keep DCC I/O in extract. - Hooks run after write, before save. - `MakeValidIdentifier` belongs in transform, not in a later cleanup by hand.

29.2 Converting a simple format (JSON/OBJ) to USD

What is it? A **converter** reads a non-USD file and writes a USD layer. This section converts a tiny JSON mesh (and shows the same math for OBJ `v / f` lines) into a Mesh component.

Why do we need it? Obj 4.7 is literally “write an exporter or converter to USD.” The exam expects you to know the mesh fields (`points`, `faceVertexCounts`, `faceVertexIndices`), units, names, `kind`, `extent`, and `defaultPrim` — not a production OBJ parser.

Beginner explanation. JSON here is a **packing list**: name, units, points, faces. The converter is a clerk who rewrites the list in USD’s form: meters, legal names, a Mesh prim, a bounding box.

Where the analogy breaks: OBJ face indices are **1-based**. USD’s `faceVertexIndices` are **0-based**. Forgetting to subtract 1 is the classic converter bug.

Technical explanation. Minimum viable mesh converter (verified on USD 26.08):

1. Scale points by the source unit (`cm` → × 0.01 if the stage is meters).
2. `Tf.MakeValidIdentifier` on the object name.
3. `UsdGeom.SetStageUpAxis / SetStageMetersPerUnit`.
4. `UsdGeom.Mesh.Define`; set `points`, `faceVertexCounts`, `faceVertexIndices`.
5. `subdivisionScheme = none` for a hard-surface crate (so the mesh is not Catmull-Clark subdivided).
6. `ComputeExtentFromPlugins` then `CreateExtentAttr` (it does not author by itself, Chapter 13).
7. `SetDefaultPrim`, `kind = component`.
8. Carry extra source fields as custom attributes (29.4).

OBJ: `o` → prim name, `v` → `points`, `f` → faces (`int(token.split("/")[0]) - 1` to drop optional `vt/vn` and convert to 0-based). Ignore `l` (lines) and `s` (smoothing) unless your mapping document says otherwise (Chapter 27).

Mental model.

JSON/OBJ		USD
<code>name "Crate Box"</code>	->	<code>/Crate_Box</code> (identifier)
<code>units cm, pt 100</code>	->	<code>points in meters (1.0)</code>
<code>faces [[0,1,2,3]]</code>	->	<code>counts [4], indices [0,1,2,3]</code>
<code>OBJ f 1 2 3</code>	->	<code>indices [0,1,2]</code> (minus one)

Simple example. A 100 cm × 50 cm quad becomes a 1 m × 0.5 m Mesh named `/Crate_Box/Geo`. Point `(100, 0, 0)` in cm becomes `(1, 0, 0)` in meters.

USDA example.

```
#usda 1.0
(
  defaultPrim = "Crate_Box"
  metersPerUnit = 1
  upAxis = "Y"
)

def Xform "Crate_Box" (
  kind = "component"
)
{
```

```

def Mesh "Geo"
{
    float3[] extent = [(0, 0, 0), (1, 0.5, 0)]
    int[] faceVertexCounts = [4]
    int[] faceVertexIndices = [0, 1, 2, 3]
    point3f[] points = [(0, 0, 0), (1, 0, 0), (1, 0.5, 0), (0, 0.5, 0)]
    uniform token subdivisionScheme = "none"
    custom string factory:partId = "CR-7"
}
}

```

Python example.

```

from pxr import Gf, Sdf, Tf, Usd, UsdGeom

SRC = {
    "name": "Crate Box",
    "units": "cm",
    "up": "Y",
    "points": [[0, 0, 0], [100, 0, 0], [100, 50, 0], [0, 50, 0]],
    "faces": [[0, 1, 2, 3]],
    "partId": "CR-7",
}
T0_M = {"cm": 0.01, "m": 1.0, "mm": 0.001}

def json_to_usd(src):
    k = T0_M[src["units"]]
    name = Tf.MakeValidIdentifier(src["name"])
    points = [Gf.Vec3f(p[0] * k, p[1] * k, p[2] * k) for p in src["points"]]
    counts = [len(f) for f in src["faces"]]
    indices = [i for f in src["faces"] for i in f]
    stage = Usd.Stage.CreateInMemory()
    UsdGeom.SetStageUpAxis(stage, src["up"])
    UsdGeom.SetStageMetersPerUnit(stage, 1)
    root = UsdGeom.Xform.Define(stage, "/" + name)
    stage.SetDefaultPrim(root.GetPrim())
    root.GetPrim().SetMetadata("kind", "component")
    mesh = UsdGeom.Mesh.Define(stage, root.GetPath().AppendChild("Geo"))
    mesh.CreatePointsAttr(points)
    mesh.CreateFaceVertexCountsAttr(counts)
    mesh.CreateFaceVertexIndicesAttr(indices)
    mesh.CreateSubdivisionSchemeAttr("none")
    ext = UsdGeom.Boundable.ComputeExtentFromPlugins(
        mesh, Usd.TimeCode.Default())
    mesh.CreateExtentAttr(ext)
    mesh.GetPrim().CreateAttribute(
        "factory:partId", Sdf.ValueTypeNames.String).Set(src["partId"])
    return stage

stage = json_to_usd(SRC)
mesh = UsdGeom.Mesh(stage.GetPrimAtPath("/Crate_Box/Geo"))
print("defaultPrim:", stage.GetDefaultPrim().GetName())
print("kind:", stage.GetDefaultPrim().GetMetadata("kind"))
print("up/units:", UsdGeom.GetStageUpAxis(stage),
      UsdGeom.GetStageMetersPerUnit(stage))
print("points[1]:", tuple(mesh.GetPointsAttr().Get()[1]))
print("counts:", list(mesh.GetFaceVertexCountsAttr().Get()))
print("extent:", [tuple(v) for v in mesh.GetExtentAttr().Get()])
print("partId:", mesh.GetPrim().GetAttribute("factory:partId").Get())
print("name was:", SRC["name"], "->", Tf.MakeValidIdentifier(SRC["name"]))

OBJ = """o Tiny
v 0 0 0
v 1 0 0
v 0 1 0
f 1 2 3
"""

def parse_obj(text):
    name, pts, faces = "Mesh", [], []
    for line in text.splitlines():

```



```

bits = line.split()
if not bits:
    continue
if bits[0] == "o":
    name = Tf.MakeValidIdentifier(bits[1])
elif bits[0] == "v":
    pts.append(Gf.Vec3f(*map(float, bits[1:4])))
elif bits[0] == "f":
    faces.append([int(b.split("/")[0]) - 1 for b in bits[1:]])
return name, pts, faces

n, pts, faces = parse_obj(OBJ)
print("OBJ name:", n, "nPoints:", len(pts), "first face:", faces[0])

```

Expected output

```

defaultPrim: Crate_Box
kind: component
up/units: Y 1.0
points[1]: (1.0, 0.0, 0.0)
counts: [4]
extent: [(0.0, 0.0, 0.0), (1.0, 0.5, 0.0)]
partId: CR-7
name was: Crate Box -> Crate_Box
OBJ name: Tiny nPoints: 3 first face: [0, 1, 2]

```

The OBJ face `f 1 2 3` became `[0, 1, 2]`. Lab 25 asks you to wire `parse_obj` into `json_to_usd`-style authoring.

Real-world use case. A catalog team publishes furniture as OBJ from a 1990s CAD tool. A nightly converter builds `chair_geo.usda` with meters, Y-up, `kind = component`, and extents so `usdview` framing works on day one.

Common mistakes.

MISTAKE

Leaving OBJ indices 1-based. Faces then point at the wrong vertices (or crash). Always subtract 1.

MISTAKE

Forgetting ``extent``. The mesh is valid but viewers frame nothing (Chapter 13).

Exam traps.

TRAP

"``ComputeExtentFromPlugins`` writes the file." It returns an array. You ``Set``.

TRAP

"JSON converters do not need ``metersPerUnit``." Every USD layer you publish should declare units and up axis (Chapter 28).

Exam takeaways.

KEY

- Mesh trio: ``points``, ``faceVertexCounts``, ``faceVertexIndices``. - OBJ faces minus one; JSON faces already 0-based. - Always: identifier, units, upAxis, defaultPrim, kind, extent. - Extra source fields: namespaced custom attributes.

29.3 Materials and animation mapping

What is it? After geometry, a converter maps **materials** (usually to `UsdPreviewSurface`) and **animation** (to attribute time samples or `xformOp` samples).

Why do we need it? A crate with no material and a robot with no motion fail interchange reviews. Obj 4.1/4.2 already mapped PreviewSurface to glTF; this section authors the USD side.

Beginner explanation. A material is a **paint recipe**. UsdPreviewSurface is the recipe card every DCC is supposed to understand. Animation is a **flip-book** of translates (or any attribute) at time codes, not a Maya animation curve node.

Where the analogy breaks: DCC curves have tangents. USD time samples are just values at times (splines exist, Chapter 10, but samples are the interchange core). You lose Bezier handles unless you store them as extra data (29.4).

Technical explanation. Material (verified 26.08):

1. `UsdShade.Material.Define(stage, path)`
2. `UsdShade.Shader.Define` with `CreateIdAttr("UsdPreviewSurface")`
3. `CreateInput("diffuseColor", Color3f)` (and roughness, metallic, opacity as needed)
4. `CreateOutput("surface")` on the shader;
`material.CreateSurfaceOutput().ConnectToSource(shader.ConnectableAPI(), "surface")`
5. `UsdShade.MaterialBindingAPI.Apply(meshPrim) then .Bind(material)`. Skipping `Apply` still often binds, but validators require the applied schema.

VERSION

Verified on USD 26.08. Since USD 22.11, validators expect `MaterialBindingAPI` to be applied before binding. Always `Apply`.

Animation:

- `xformable.AddTranslateOp().Set(value, time)` authors samples.
- Linear interpolation is the stage default (Chapter 10): samples at 1 and 11 yield the midpoint at time 6.
- Do not put the rest pose only as a default if the DCC also has a key at the start frame; pick one encoding and document it.

Mental model.

```
/Prop/Looks/Paint      Material
  Preview              Shader id=UsdPreviewSurface
/Prop/Geo --Bind-->    Paint  (MaterialBindingAPI applied)

translate.timeSamples { 1: (0,0,0), 11: (2,0,0) }
```

Simple example. A red PreviewSurface bound to a quad. The parent Xform moves from x=0 at time 1 to x=2 at time 11.

USDA example.

```
#usda 1.0

def Xform "Prop"
{
  double3 xformOp:translate.timeSamples = {
    1: (0, 0, 0),
    11: (2, 0, 0)
  }
  uniform token[] xformOpOrder = ["xformOp:translate"]

  def Mesh "Geo" (
    prepend apiSchemas = ["MaterialBindingAPI"]
  )
  {
    rel material:binding = </Prop/Looks/Paint>
  }
}
```

```

def "Looks"
{
    def Material "Paint"
    {
        token outputs:surface.connect = </Prop/Looks/Paint/Preview.outputs:surface>
        def Shader "Preview"
        {
            uniform token info:id = "UsdPreviewSurface"
            color3f inputs:diffuseColor = (0.8, 0.1, 0.1)
            token outputs:surface
        }
    }
}

```

Python example.

```

from pxr import Gf, Sdf, Usd, UsdGeom, UsdShade

st = Usd.Stage.CreateInMemory()
UsdGeom.SetStageUpAxis(st, "Y")
UsdGeom.SetStageMetersPerUnit(st, 1)
root = UsdGeom.Xform.Define(st, "/Prop")
st.SetDefaultPrim(root.GetPrim())
mesh = UsdGeom.Mesh.Define(st, "/Prop/Geo")
mesh.CreatePointsAttr([Gf.Vec3f(0, 0, 0), Gf.Vec3f(1, 0, 0),
                      Gf.Vec3f(1, 1, 0), Gf.Vec3f(0, 1, 0)])
mesh.CreateFaceVertexCountsAttr([4])
mesh.CreateFaceVertexIndicesAttr([0, 1, 2, 3])
op = UsdGeom.Xformable(root).AddTranslateOp()
op.Set(Gf.Vec3d(0, 0, 0), 1)
op.Set(Gf.Vec3d(2, 0, 0), 11)
mat = UsdShade.Material.Define(st, "/Prop/Looks/Paint")
sh = UsdShade.Shader.Define(st, "/Prop/Looks/Paint/Preview")
sh.CreateIdAttr("UsdPreviewSurface")
sh.CreateInput("diffuseColor", Sdf.ValueTypeNames.Color3f).Set(
    (0.8, 0.1, 0.1))
sh.CreateOutput("surface", Sdf.ValueTypeNames.Token)
mat.CreateSurfaceOutput().ConnectToSource(sh.ConnectableAPI(), "surface")
UsdShade.MaterialBindingAPI.Apply(mesh.GetPrim()).Bind(mat)
bound = UsdShade.MaterialBindingAPI(mesh.GetPrim()).ComputeBoundMaterial()[0]
print("material:", bound.GetPath())
print("shader id:", sh.GetIdAttr().Get())
print("translate @1:", op.Get(1), "@11:", op.Get(11), "@6:", op.Get(6))
print("has MaterialBindingAPI:",
      mesh.GetPrim().HasAPI(UsdShade.MaterialBindingAPI))

```

Expected output

```

material: /Prop/Looks/Paint
shader id: UsdPreviewSurface
translate @1: (0, 0, 0) @11: (2, 0, 0) @6: (1, 0, 0)
has MaterialBindingAPI: True

```

Time 6 is 1.0 because linear interpolation is the default.

Real-world use case. A game exporter writes `PreviewSurface` for the engine's debug view and also writes a custom `game:shaderId` (29.4) for the real engine shader. Animation of the root Xform is samples at 30 fps; blend-shape weights become `primvars` samples.

Common mistakes.

MISTAKE

Binding without `MaterialBindingAPI.Apply`. Looks fine in a quick test; `usdchecker` / `UsdValidation` fails.

MISTAKE

Authoring animation only as a default translate, then wondering why the shot does not play. Playback queries numeric times.

Exam traps.**TRAP**

"`info:id = UsdPreviewSurface` goes on the Material prim." It goes on the ****Shader****.

TRAP

"Connections use relationship `rel`." Shader connections are `outputs:surface.connect` on the Material.

Exam takeaways.**KEY**

- PreviewSurface lives on a Shader under a Material. - Always `MaterialBindingAPI.Apply` before `Bind`. - Animation interchange = time samples. - Curve tangents are extra data, not USD samples.

29.4 Nonstandard data via custom attributes or schemas

What is it? Source scenes have fields USD has no built-in schema for: a factory part number, a CAD document GUID, a proprietary flag. Obj 4.5 says to **use USD schemas** to carry them. Until you generate a schema (Chapter 37), you author **namespaced custom attributes** and/or `customData`, then graduate to an API schema.

Why do we need it? If you drop those fields, the round trip fails and downstream tools guess. If you stuff them into random un-namespaced names, they collide with future schemas. A namespace (`factory:partId`) is the contract.

Beginner explanation. A custom attribute is a **sticky note** on the prim: "part id PN-42." A schema is a **printed form** with official boxes. Start with sticky notes in a named pad (`factory:`). When the factory standardizes, print the form (`usdGenSchema`, Chapter 37).

Where the analogy breaks: sticky notes still round-trip. They are real properties. They just have no generated C++/Python accessors and no validator until you register a schema.

Technical explanation. Verified on USD 26.08:

- `prim.CreateAttribute("factory:partId", Sdf.ValueTypeNames.String)` authors custom string `factory:partId`. `IsCustom()` is `True`. `GetNamespace()` is `"factory"`.
- `prim.SetCustomDataByKey("sourceApp", "AcmeCAD 12")` stores dictionary metadata, not a typed attribute. Good for provenance; bad for values you will animate or bind.
- **Applied API schemas** (like `MaterialBindingAPI`) are already "using a schema during import/export." That is Obj 4.5 for bindings.
- When several DCCs must share the same extra fields, write `schema.usda` and generate a codeless or codeful API schema (Chapter 37). The exporter then calls `FactoryAPI.Apply(prim).CreatePartIdAttr().Set(...)`.
- Do not invent a second `points` attribute. If a built-in schema already has the concept, use it.

Decision table:

Need	Mechanism
One pipeline-local string	Namespaced custom attribute
Provenance blob	<code>customData</code> / <code>assetInfo</code> (Chapter 12)
Official interchange field	API or IsA schema (Chapter 37)
Material / collection / geom subset	Existing applied API schemas

Mental model.

```
CAD "PartNumber" --> custom factory:partId      (today)
                  --> FactoryAPI.partId         (when the schema ships)

CAD "native node id" --> customData["sourceApp"] (not an attribute)
```

Simple example. `/Part` gets `factory:partId = "PN-42"` and `customData sourceApp = "AcmeCAD 12"`. `IsCustom()` is `True`. `HasAPI(MaterialBindingAPI)` is `False` until you apply it.

USDA example.

```
#usda 1.0

def Xform "Part" (
  customData = {
    string sourceApp = "AcmeCAD 12"
  }
)
{
  custom string factory:partId = "PN-42"
}
```

Line notes: the `custom` keyword marks a non-schema attribute. `customData` is metadata, not a property you `GetAttribute`.

Python example.

```
from pxr import Sdf, Usd, UsdGeom, UsdShade

stage = Usd.Stage.CreateInMemory()
prim = UsdGeom.Xform.Define(stage, "/Part").GetPrim()
attr = prim.CreateAttribute("factory:partId", Sdf.ValueTypeNames.String)
attr.Set("PN-42")
prim.SetCustomDataByKey("sourceApp", "AcmeCAD 12")
print("IsCustom:", attr.IsCustom())
print("name:", attr.GetName(), "namespace:", attr.GetNamespace())
print("value:", attr.Get())
print("sourceApp:", prim.GetCustomDataByKey("sourceApp"))
print("HasAPI MaterialBindingAPI:",
      prim.HasAPI(UsdShade.MaterialBindingAPI))
```

Expected output

```
IsCustom: True
name: factory:partId namespace: factory
value: PN-42
sourceApp: AcmeCAD 12
HasAPI MaterialBindingAPI: False
```

Xform also stores a schema `userDocBrief` inside `customData` if you dump the whole dict; read a key, do not print the whole bag.

Real-world use case. An automotive CAD exporter writes `vehicle:vin` as a custom string on the assembly root on day one. Six months later the studio publishes a `VehicleAPI` schema; the exporter switches to `VehicleAPI.Apply` and the same files remain readable because the attribute name was planned as `vehicle:vin`.

Common mistakes.**MISTAKE**

``CreateAttribute("points", ...)`` on a Mesh for a second point cloud. You collide with ``UsdGeom.Mesh.points``. Use a namespace.

MISTAKE

Putting the part number only in the prim name (``Bolt_PN42``). Names then cannot change without breaking references.

Exam traps.

TRAP

"Custom attributes are illegal in USD." They are legal; ``custom`` is the keyword.

TRAP

"Obj 4.5 requires ``usdGenSchema`` on the exam machine." You must **know** when to use a schema versus a custom attribute. ``usd-core`` does not ship ``usdGenSchema`` (V-004).

Exam takeaways.**KEY**

- Namespace extra fields (``factory:``). ``IsCustom()`` will be True. - ``customData`` is metadata; attributes are for typed values. - Applied APIs (MaterialBindingAPI) already satisfy "use a schema." - Graduate sticky notes to ``schema.usda`` when several tools share them (Ch 37).

29.5 Importer design in a DCC

What is it? An **importer** reads a USD stage and builds DCC-native objects (Maya nodes, Houdini SOPs, engine actors). Obj 4.8 is "write or extend a USD importer in a DCC." You will not open Maya here; you will design the walk and prove it by converting USD **back** into records.

Why do we need it? Export without import is a one-way dump. A DCC that cannot round-trip custom attributes, time samples, or references will silently destroy the pipeline the next time an artist hits Save.

Beginner explanation. If the exporter is a clerk packing boxes, the importer is the clerk **unpacking** them onto the DCC's shelves. The shelf layout is the DCC data model. Some USD boxes (variants, payloads) have no shelf; you document the loss (Chapter 27) or store the box unopened (`customData`).

Where the analogy breaks: you usually unpack the **composed** stage (`Usd.Stage`), not each layer. Composition has already mixed opinions. A high-end importer may also read specs (`Sdf`) to round-trip layer structure; that is an extension, not the default.

Technical explanation. Design checklist for a DCC importer (Obj 4.8):

1. **Open** the stage (`Usd.Stage.Open`). Decide `LoadAll` vs `LoadNone` (Chapter 17).
2. **Walk** `Traverse()` for defined, active, non-abstract prims. Use `TraverseInstanceProxies()` only if the DCC needs unique nodes per instance (usually it should instance instead, Chapter 24).
3. **Map types:** `IsA(UsdGeom.Mesh)` → DCC mesh, `IsA(UsdGeom.Xform)` → group, `IsA(UsdShade.Material)` → DCC material, unknown → group + custom attrs.
4. **Read data at the right time:** `attr.Get(time)` for the DCC current frame; `GetTimeSamples()` to build curves.
5. **Materials:** `MaterialBindingAPI.ComputeBoundMaterial()`.
6. **Extra fields:** read namespaced custom attributes and `customData` back into DCC extra attributes so the next export restores them.
7. **References:** either create DCC instances / references, or flatten (lossy). Do not explode every reference into unique copies by accident.
8. **Report loss** (lights the DCC cannot draw, variants not represented, etc.).

Extending an existing importer usually means adding a type case ("when `IsA(MySchema)`, build this DCC node") or a post-hook that copies custom namespaces. You rarely rewrite the walker.

Mental model.

```

stage.Traverse()
|   IsA Mesh?  -->  DCC mesh + points/faces
|   IsA Xform? -->  DCC group + xformOps at time
|   Has factory:* --> DCC extra attributes
|   unknown    --> group + dump properties to extras

```

Simple example. Open the crate stage from 29.2. The importer emits one record: `path /Crate_Box/Geo`, 4 points, `partId CR-7`, and the stage `upAxis`.

USDA example. The importer’s *input* is the crate USDA from 29.2. No new syntax. What it must not drop: `factory:partId`, `upAxis`, `kind`.

Python example.

```

from pxr import Gf, Sdf, Tf, Usd, UsdGeom

SRC = {
    "name": "Crate Box", "units": "cm", "up": "Y",
    "points": [[0, 0, 0], [100, 0, 0], [100, 50, 0], [0, 50, 0]],
    "faces": [[0, 1, 2, 3]], "partId": "CR-7",
}
TO_M = {"cm": 0.01, "m": 1.0, "mm": 0.001}

def json_to_usd(src):
    k = TO_M[src["units"]]
    name = Tf.MakeValidIdentifier(src["name"])
    points = [Gf.Vec3f(p[0] * k, p[1] * k, p[2] * k) for p in src["points"]]
    stage = Usd.Stage.CreateInMemory()
    UsdGeom.SetStageUpAxis(stage, src["up"])
    UsdGeom.SetStageMetersPerUnit(stage, 1)
    root = UsdGeom.Xform.Define(stage, "/" + name)
    stage.SetDefaultPrim(root.GetPrim())
    mesh = UsdGeom.Mesh.Define(stage, root.GetPath().AppendChild("Geo"))
    mesh.CreatePointsAttr(points)
    mesh.CreateFaceVertexCountsAttr([len(f) for f in src["faces"]])
    mesh.CreateFaceVertexIndicesAttr([i for f in src["faces"] for i in f])
    mesh.GetPrim().CreateAttribute(
        "factory:partId", Sdf.ValueTypeNames.String).Set(src["partId"])
    return stage

def import_meshes(stage):
    records = []
    for prim in stage.Traverse():
        if prim.IsA(UsdGeom.Mesh):
            mesh = UsdGeom.Mesh(prim)
            records.append({
                "path": str(prim.GetPath()),
                "nPoints": len(mesh.GetPointsAttr().Get()),
                "partId": prim.GetAttribute("factory:partId").Get(),
            })
    return records

st = json_to_usd(SRC)
print("imported meshes:", import_meshes(st))
print("upAxis:", UsdGeom.GetStageUpAxis(st))

```

Expected output

```

imported meshes: [{'path': '/Crate_Box/Geo', 'nPoints': 4, 'partId': 'CR-7'}]
upAxis: Y

```

`partId` survived the round trip because the exporter stored it and the importer asked for it.

Real-world use case. A DCC vendor extends its USD importer with a case for `UsdLux.RectLight` that used to become an empty locator. After the extension, lights round-trip; the mapping document’s “`RectLight` → dropped” row is updated to “`RectLight` → area light.”

Common mistakes.**MISTAKE**

Walking only the root layer (`Sdf`) and ignoring references. You import an empty assembly. Walk the composed stage unless you are a layer editor.

MISTAKE

Turning every native instance into a unique DCC mesh. Memory explodes. Prefer DCC instancing from the prototype.

Exam traps.**TRAP**

"Importers must flatten." Flattening is a choice. It loses composition (Chapter 22).

TRAP

"Unknown prim types should error and abort." Better: create a group, copy extra properties, report a warning, continue.

Exam takeaways.**KEY**

- Importers walk the composed stage; map `IsA` types to DCC nodes. - Round-trip custom namespaces and `customData`. - Keep references/instances as shares, not unique copies, by default. - Extending = new type case + extras, not a rewrite.

29.6 Exporter hooks

What is it? An **exporter hook** is a function the exporter calls at a fixed time — almost always after prims are written and before `Save()` — to enforce pipeline structure: `defaultPrim`, `kind`, `extents`, `naming`, `stripping` internal data.

Why do we need it? Every exporter forgets something. Hooks make the rules **one list** that CAD, DCC, and the JSON converter all run. Chapter 34 returns to hooks in a full pipeline (`notices`, `flatten`). Here you write two hooks and see them fix a stage.

Beginner explanation. Hooks are the **pre-flight checklist**: doors closed, extents filled, default prim set. The plane (the file) does not taxi until the list is green.

Where the analogy breaks: a hook that *silently* drops unknown attributes is a data-loss bug, not a checklist. Hooks should author missing structure or **fail**, not invent geometry.

Technical explanation. Good hooks (verified pattern):

Hook	Does	Fails when
<code>hook_default_prim</code>	<code>SetDefaultPrim</code> to the first defined root if missing	No defined root prim
<code>hook_extents</code>	<code>ComputeExtentFromPlugins</code> + <code>Set</code> on every <code>Boundable</code>	Extent compute returns empty and the prim should be visible
<code>hook_kind</code>	<code>Set component</code> on asset roots that have no kind	Conflicting kind already authored (do not clobber; warn)
<code>hook_strip_internal</code>	Remove <code>debug:*</code> attributes	— (log how many you stripped)

Properties of a hook:

- **Idempotent:** running twice does not change a good stage.
- **No DCC dependency:** it takes a `Usd.Stage`.
- **Ordered:** extents after meshes exist; `defaultPrim` after roots exist.

- Chapter 34 adds change notices (`Tf.Notice`) for live DCC exporters; a batch converter just calls the list.

Mental model.

```
write() --> [defaultPrim] --> [extents] --> [kind] --> validate --> Save
```

Simple example. A stage with `/Crate/Geo` as a Mesh and **no** `defaultPrim`. `hook_default_prim` sets `/Crate`. `hook_extents` authors the triangle's extent.

USDA example. *After hooks, the layer has the missing structure. Before hooks, `/Crate` had no `defaultPrim` and the mesh had no extent.*

```
#usda 1.0
(
    defaultPrim = "Crate"
)

def "Crate"
{
    def Mesh "Geo"
    {
        float3[] extent = [(0, 0, 0), (1, 1, 0)]
        int[] faceVertexCounts = [3]
        int[] faceVertexIndices = [0, 1, 2]
        point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
    }
}
```

Python example.

```
from pxr import Gf, Sdf, Usd, UsdGeom

def hook_default_prim(stage):
    if not stage.GetDefaultPrim():
        roots = [p for p in stage.GetPseudoRoot().GetChildren()
                  if p.GetSpecifier() == Sdf.SpecifierDef]
        if roots:
            stage.SetDefaultPrim(roots[0])
    return bool(stage.GetDefaultPrim())

def hook_extents(stage):
    n = 0
    for prim in stage.Traverse():
        if prim.IsA(UsdGeom.Boundable):
            b = UsdGeom.Boundable(prim)
            ext = UsdGeom.Boundable.ComputeExtentFromPlugins(
                b, Usd.TimeCode.Default())
            if ext:
                b.CreateExtentAttr().Set(ext)
                n += 1
    return n

st = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(st, "/Crate/Geo")
mesh.CreatePointsAttr(
    [Gf.Vec3f(0, 0, 0), Gf.Vec3f(1, 0, 0), Gf.Vec3f(0, 1, 0)])
mesh.CreateFaceVertexCountsAttr([3])
mesh.CreateFaceVertexIndicesAttr([0, 1, 2])
print("defaultPrim before:", bool(st.GetDefaultPrim()))
print("hook default_prim:", hook_default_prim(st),
      "now:", st.GetDefaultPrim().GetPath())
print("extents authored:", hook_extents(st))
print("extent:", [tuple(v) for v in mesh.GetExtentAttr().Get()])
```

Expected output

```
defaultPrim before: False
hook_default_prim: True now: /Crate
```

```
extents authored: 1
extent: [(0.0, 0.0, 0.0), (1.0, 1.0, 0.0)]
```

Define("/Crate/Geo") created a typeless /Crate parent; the hook picked that defined root.

Real-world use case. A studio's "USD export" button in four DCCs all import `studio_export_hooks` and run the same list. When the pipeline adds "every component must have `assetInfo['identifier']`", one hook update fixes all four exporters.

Common mistakes.

MISTAKE

A hook that overwrites an authored ``kind`` with ``component``. Assemblies then become components. Warn, do not clobber.

MISTAKE

Computing extents before points are set. The hook returns empty; you skip the prim. Order the hook list after geometry authoring.

Exam traps.

TRAP

"Hooks are Tf.Notice listeners." Notices are Chapter 34. A hook can be an ordinary function called from the exporter.

TRAP

"Validation is a hook that deletes prims until `usdchecker` is quiet." Validation **reports**. Hooks **author structure**.

Exam takeaways.

KEY

- Hooks enforce structure after write, before save. - Typical: ``defaultPrim``, extents, kind, strip internals. - Idempotent, DCC-free, no silent data invention. - Shared hook lists keep many exporters consistent (Ch 34 continues).

Chapter lab(s)

Lab 25 (JSON/OBJ → USD converter) expands 29.2 into a file on disk with validation. Lab 26 covers units if your converter's scale is wrong. Lab 34 in Chapter 40 binds `PreviewSurface` more fully.

USDA reading exercises

Exercise 29-A. An OBJ face line is `f 4/1/1 5/2/1 6/3/1`. What three integers go into `faceVertexIndices` for that face?

Exercise 29-B. A converter writes a Mesh but no `defaultPrim`, no `extent`, and binds a material without applying `MaterialBindingAPI`. Name the hook or call that fixes each of the three problems.

Answers

29-A. 4, 5, 6 minus one → `[3, 4, 5]`. The `/vt/vn` parts are ignored for this field.

29-B. `hook_default_prim / SetDefaultPrim`; `hook_extents / ComputeExtentFromPlugins` then `Set`; `MaterialBindingAPI.Apply` then `Bind`.

Chapter review

Summary

- Exporter = extract → transform → write → hook → validate. DCC I/O stays in extract.
- Converter (Obj 4.7): scale units, `MakeValidIdentifier`, Mesh fields, `kind`, `defaultPrim`, extent. OBJ faces are 1-based.
- Materials: Shader `UsdPreviewSurface` + `MaterialBindingAPI.Apply` + `Bind`. Animation: time samples.
- Nonstandard data (Obj 4.5): namespaced custom attributes, then a real schema (Ch 37). Applied APIs already count.
- Importer (Obj 4.8): walk the composed stage, map `IsA`, round-trip extras, do not flatten by default.
- Hooks: idempotent, DCC-free, after write; they author structure, they do not invent geometry.

If you see... → think...

If you see...	Think...
Obj 4.7	Write a converter: mesh + metadata + hooks
f 1 2 3	Indices <code>[0, 1, 2]</code>
No <code>defaultPrim</code>	Write step or <code>hook_default_prim</code>
Empty framing in usdview	Missing <code>extent</code> hook
<code>MaterialBindingAPI</code>	<code>Apply</code> then <code>Bind</code>
CAD GUID	<code>factory:guid</code> custom attr (Obj 4.5)
Obj 4.8	Walk composed stage; extend by type case
"Exporter hook"	Function after write, before save
<code>ComputeExtentFromPlugins</code>	Returns values; you <code>Set</code>

Review questions

R29-01 · Obj 4.7 · Single choice The transform step of an exporter should: A. Call Maya's mesh API B. Convert records (units, names) with no DCC C. Open usdview D. Flatten the stage

R29-02 · Obj 4.7 · Single choice JSON point `(100,0,0)` in cm, stage in meters. Stored `points` value? A. `(100, 0, 0)` B. `(1, 0, 0)` C. `(0.1, 0, 0)` D. `(10, 0, 0)`

R29-03 · Obj 4.7 · Single choice `subdivisionScheme = none` on a crate mesh means: A. The file is invalid B. Do not Catmull-Clark subdivide C. No faces D. No extent

R29-04 · Obj 4.7 · Select two. Required Mesh topology attributes: A. `points` B. `faceVertexCounts` and `faceVertexIndices` C. `clipTimes` D. `info:id`

R29-05 · Obj 4.7 · Single choice `PreviewSurface info:id` is authored on: A. The Mesh B. The Shader C. The stage metadata D. `customData` only

R29-06 · Obj 4.5 · Single choice `CreateAttribute("factory:partId", String)` makes: A. An illegal property B. A custom namespaced attribute C. A relationship D. A variant

R29-07 · Obj 4.5 · Single choice `MaterialBindingAPI.Apply` during export is: A. Optional decoration B. Using a schema for interchange (Obj 4.5) and required by validators C. Deprecated in 26.08 D. Only for payloads

R29-08 · Obj 4.8 · Single choice A DCC importer that ignores references and reads only the root layer will: A. See the full composed assembly B. Miss referenced geometry C. Auto-load payloads as meshes D. Convert glTF

R29-09 · Obj 4.8 · Select two. A round-trip importer should restore: A. `factory:*` custom attributes B. `upAxis / metersPerUnit` C. `/__Prototype_1` as a saved prim D. Session-layer mute state

R29-10 · Obj 4.7 · Single choice `hook_default_prim` found no defined root. It should: A. Invent `/World` B. Fail/report, not save a useless file C. Set `defaultPrim` to `/__Prototype_1` D. Mute the root layer

R29-11 · Obj 4.7 · Single choice Time samples at 1 and 11, `Get(6)` linear, values 0 and 2 on X: A. 0 B. 1 C. 2 D. Default only

R29-12 · Obj 4.5 · Single choice When several DCCs must share the same extra fields officially: A. Keep inventing new un-namespaced names B. Publish an API/IsA schema (Chapter 37) C. Put the fields in the prim path D. Store them in `extent`

Review answers

R29-01 — B. Review: §29.1.

R29-02 — B. $\times 0.01$. Review: §29.2.

R29-03 — B. Review: §29.2.

R29-04 — A and B. Review: §29.2.

R29-05 — B. Review: §29.3.

R29-06 — B. Review: §29.4.

R29-07 — B. Review: §29.3, §29.4.

R29-08 — B. Review: §29.5.

R29-09 — A and B. Prototypes are runtime (C). Review: §29.5, Ch 24.

R29-10 — B. Review: §29.6.

R29-11 — B. Review: §29.3.

R29-12 — B. Review: §29.4, Ch 37.

CHAPTER 30

Chapter 30 — Validating USD Assets

Exam domain: Data Exchange (15%) · **Objectives:** 4.6 · **Study day:** 9 · **Est. time:** 90 min
Prerequisites: Ch 2 (stage metadata), Ch 13 (extents), Ch 27 (validate step), Ch 29 (exporter hooks)

Obj 4.6 is “write a validator for the integrity of an OpenUSD asset exported from a DCC.” This chapter shows why, how `usdchecker` fits, the **UsdValidation** framework on USD 26.08, what happened to `ComplianceChecker`, and a pipeline checker you can actually run with `usd-core`.

Learning goals

- Explain why export without validation ships broken `defaultPrim`, units, and extents.
- Describe `usdchecker` and its Python equivalent on `usd-core`.
- Load built-in validators by name and read `ValidationError` fields.
- State that `UsdUtils.ComplianceChecker` is gone in 26.08.
- Write a small integrity checker for `defaultPrim`, `upAxis`, `metersPerUnit`, and extents (Obj 4.6).

Key terms

Term	One-line definition
Validator	A named check that inspects a stage (or prims/layers) and returns errors
UsdValidation	The <code>pxr</code> module for validators, errors, and fixers (USD 24.08+)
ValidationRegistry	Catalog of installed validators and suites
ValidationError	One finding: name, type (Error/Warn/Info), message, sites
ValidationContext	A bundle of validators you run together
usdchecker	CLI that runs <code>UsdValidation</code> on a file (full USD builds; not in <code>usd-core</code>)
ComplianceChecker	Legacy Python class removed in 26.08
Integrity checker	Studio code that asks “is this published asset usable?”

30.1 Why validate

What is it? Validation is running automated checks on a USD asset after export (or before publish) so missing metadata, broken references, and illegal bindings are caught before another department opens the file.

Why do we need it? Exporters forget `upAxis`, `metersPerUnit`, `defaultPrim`, and extent (Chapters 28–29). Those files “open” but frame wrong, cannot be referenced, or fail in `usdview`. A human spot-check does not scale. Obj 4.6 wants a **written** checker, not a hope.

Beginner explanation. Validation is the **weigh-station after the truck is loaded**. The crate is already on the truck (the layer exists). The station asks: do you have a destination (`defaultPrim`)? Did you write the scale (`metersPerUnit`)? Is the box labeled up (`upAxis`)?

Where the analogy breaks: a weigh-station that silently throws cargo overboard is a bad hook (Chapter 29). Validators **report**. You fix the exporter, then run them again.

Technical explanation. Typical integrity questions:

Question	Built-in validator (26.08)
	<code>usdGeomValidators:StageMetadataChecker</code>

Question	Built-in validator (26.08)
upAxis and metersPerUnit authored?	
defaultPrim valid?	usdValidation:StageMetadataChecker
References resolve?	usdUtilsValidators:MissingReferenceValidator
Material bind applied MaterialBindingAPI?	usdShadeValidators:MaterialBindingApiAppliedValidator

A DCC exporter should run these (and your studio checks) in the **validate** step after hooks (Chapter 29). Fail the publish if `GetType()` is `Error`.

Mental model.

```
export --> hooks --> [ validators ] --errors--> fix exporter
|
clean --> publish
```

Simple example. A stage with only `def Sphere "Ball"` produces three errors: missing meters, missing up axis, missing defaultPrim. After setting those three, the same validators return nothing.

USDA example. *File: bad.usda — opens, but fails integrity*

```
#usda 1.0

def Sphere "Ball"
{
    double radius = 1
}
```

File: good.usda

```
#usda 1.0
(
    defaultPrim = "Ball"
    metersPerUnit = 1
    upAxis = "Y"
)

def Sphere "Ball"
{
    double radius = 1
}
```

Python example.

```
from pxr import Usd, UsdGeom, UsdValidation

reg = UsdValidation.ValidationRegistry()
geom = reg.GetOrLoadValidatorByName(
    "usdGeomValidators:StageMetadataChecker")
core = reg.GetOrLoadValidatorByName(
    "usdValidation:StageMetadataChecker")

stage = Usd.Stage.CreateInMemory()
UsdGeom.Sphere.Define(stage, "/Ball")
print("before:")
for validator in (geom, core):
    for err in validator.Validate(stage):
        print(err.GetName(), err.GetType())

UsdGeom.SetStageUpAxis(stage, "Y")
UsdGeom.SetStageMetersPerUnit(stage, 1)
stage.SetDefaultPrim(stage.GetPrimAtPath("/Ball"))
print("after geom/core counts:",
      len(list(geom.Validate(stage))),
      len(list(core.Validate(stage))))
```

Expected output

```
before:
MissingMetersPerUnitMetadata UsdValidation.ValidationErrorType.Error
MissingUpAxisMetadata UsdValidation.ValidationErrorType.Error
MissingDefaultPrim UsdValidation.ValidationErrorType.Error
after geom/core counts: 0 0
```

Do not print `GetMessage()` in tests that use anonymous layers: the message embeds the layer identifier, which changes.

Real-world use case. A game studio’s publish server rejects any component that fails `StageMetadataChecker` or has `UnresolvableDependency`. Artists see the error names in the DCC export log, not a late-night “the chair is two kilometers tall” ticket.

Common mistakes.

MISTAKE

Validating only in usdview by eye. Automation is the objective.

MISTAKE

Treating warnings as leftover noise. Read ``GetType()`: `Error`` fails the publish; ``Warn` / `Info`` may still be policy.

Exam traps.

TRAP

"If ``Stage.Open`` succeeded, the asset is valid." Open succeeds with missing metadata and with unresolved references (composition errors).

TRAP

"One `StageMetadataChecker` does everything." There are two: `geom (units/up)` and `usdValidation (defaultPrim)`.

Exam takeaways.

KEY

- Validation reports integrity; it does not open-or-fail the stage. - Two `StageMetadata` checkers: `units/up` vs `defaultPrim`. - Run validators after exporter hooks. - Obj 4.6 = write (or run) these checks on DCC exports.

30.2 usdchecker

What is it? `usdchecker` is the command-line tool that runs `UsdValidation` on a USD file and prints errors. Full OpenUSD builds ship it. **`usd-core` does not.**

Why do we need it? Build servers and exam reading lists name `usdchecker`. You must know the command, what it uses internally, and the Python stand-in for this book’s environment.

Beginner explanation. `usdchecker` is the **same weigh-station as a CLI**: you pass a file, it prints `Error/Warn` lines. Under the hood since USD 26.03 it calls `UsdValidation`, not the old `ComplianceChecker`.

Where the analogy breaks: flags and output format differ across USD versions. Learn the *framework* (30.3), not a screenshot of one CLI banner.

Technical explanation.

```
usdchecker asset.usda
```

- Exit code is non-zero when errors are found (typical; treat that as “the build failed”).
- Since **26.03**, `usdchecker` is built on **`UsdValidation`**.

- On `usd-core` 26.8: no `usdchecker` binary. Equivalent:

```
ValidationContext([v1, v2, ...]).Validate(stage)
```

VERSION

Verified on USD 26.08. ``usdchecker`` switched to `UsdValidation` in 26.03. Older study material still says it wraps ``UsdUtils.ComplianceChecker`` — that path is gone (30.4).

Mental model.

```
full USD:      usdchecker file.usda --> UsdValidation
usd-core:      ValidationContext.Validate(stage) --> same errors
```

Simple example. `usdchecker good.usda` is quiet. `usdchecker bad.usda` prints `MissingUpAxisMetadata` and friends. The Python block below is the `usd-core` stand-in.

USDA example. Use `good.usda` / `bad.usda` from 30.1. The CLI takes those files. The checker script parses USDA; it does not run the CLI.

Python example. `usdchecker` itself is not runnable with `usd-core`. This is the equivalent:

```
from pxr import Usd, UsdGeom, UsdValidation

reg = UsdValidation.ValidationRegistry()
ctx = UsdValidation.ValidationContext([
    reg.GetOrLoadValidatorByName(
        "usdGeomValidators:StageMetadataChecker"),
    reg.GetOrLoadValidatorByName(
        "usdValidation:StageMetadataChecker"),
])

stage = Usd.Stage.CreateInMemory()
UsdGeom.Sphere.Define(stage, "/Ball")
print("usdchecker-equivalent errors:")
for err in ctx.Validate(stage):
    print(err.GetName())
```

Expected output

```
usdchecker-equivalent errors:
MissingMetersPerUnitMetadata
MissingUpAxisMetadata
MissingDefaultPrim
```

The CLI invocation (not run here) looks like:

```
usdchecker bad.usda
```

Real-world use case. CI runs `usdchecker` on every published `*_component.usda`. Developers without the CLI run the same two validators in a `pytest` using `usd-core`.

Common mistakes.

MISTAKE

Assuming ``pip install usd-core`` gave you ``usdchecker``. It did not (F5, V-001).

MISTAKE

Grepping CLI help from a 24.x blog and expecting `ComplianceChecker` flags.

Exam traps.

TRAP

"``usdchecker`` is a Hydra tool." It is a validation CLI.

TRAP

"You cannot validate without `usdchecker`." Python `UsdValidation` is the API.

Exam takeaways.

KEY

- ``usdchecker file.usda`` on full builds. - 26.03+: `UsdValidation` under the hood. - ``usd-core``: ``ValidationContext.Validate``. - Same error names either way.

30.3 The UsdValidation framework

What is it? `pxr.UsdValidation` is the library: a **registry** of validators, **errors** they return, optional **fixers**, and a **context** that runs several validators at once.

Why do we need it? Obj 4.6 on a modern USD means this API, not a home-grown print loop only. Built-in checks already cover metadata, missing references, and material-binding schema application.

Beginner explanation. The registry is a **phone book** of inspectors. You look up `"usdGeomValidators:StageMetadataChecker"`, ask it to `Validate(stage)`, and get a list of tickets (`ValidationError`). A context is several inspectors walking the site together.

Where the analogy breaks: some validators are time-dependent (`isTimeDependent` on metadata). Most asset-integrity checks are not.

Technical explanation. Verified on USD 26.08 (`usd-core` ships **28** validators):

Class	Role
<code>ValidationRegistry</code>	<code>GetOrLoadValidatorByName</code> , <code>GetAllValidatorMetadata</code> , <code>HasValidator</code>
<code>Validator</code>	<code>Validate(stage) → errors</code> ; <code>GetMetadata()</code>
<code>ValidationError</code>	<code>GetName()</code> , <code>GetMessage()</code> , <code>GetType()</code> , <code>GetErrorAsString()</code> , <code>GetSites()</code> , <code>GetIdentifier()</code>
<code>ValidationErrorType</code>	<code>Error</code> , <code>Warn</code> , <code>Info</code> , <code>None</code>
<code>ValidationContext</code>	<code>ValidationContext([v1, v2]).Validate(stage)</code>
<code>ValidationFixer</code>	Optional auto-fix attached to an error (<code>GetFixers</code>)
<code>ValidatorSuite</code>	Named group; <code>GetOrLoadAllValidatorSuites()</code> may be empty on <code>usd-core</code>

Names are `plugin:ValidatorName`. Useful ones for exporters:

- `usdGeomValidators:StageMetadataChecker`
- `usdValidation:StageMetadataChecker`
- `usdUtilsValidators:MissingReferenceValidator` → `UnresolvableDependency`
- `usdShadeValidators:MaterialBindingApiAppliedValidator`

`RegisterPrimValidator` / `RegisterStageValidator` exist for **plugins**. Passing a Python `lambda` does **not** match the C++ signature on 26.08. Studio checkers in Python are ordinary functions (30.5) unless you ship a C++/plugin validator.

VERSION

`UsdValidation` first phase / registry in 24.08; Python bindings for validator types in 24.11; separate ``pxr.UsdValidation`` library (and ``ValidationContext`` Python wrappings) in 25.02. This book verifies 26.08.

Mental model.

```
Registry.GetOrLoadValidatorByName("plugin:Name")
      |
      v
    validator.Validate(stage)
      |
      v
```

```
[ ValidationError, ... ]
    GetName / GetType / GetMessage
```

Simple example. Ask the registry how many validators exist and whether the two metadata checkers and the missing-reference checker are present. Run a context on a complete Sphere: no errors.

USDA example. No new syntax. Validators read the composed stage produced from any USDA in this chapter.

Python example.

```
from pxr import Usd, UsdGeom, UsdValidation

reg = UsdValidation.ValidationRegistry()
names = sorted(m.name for m in reg.GetAllValidatorMetadata())
print("nValidators:", len(names))
for needle in (
    "StageMetadataChecker",
    "MissingReferenceValidator",
    "MaterialBindingApiAppliedValidator"):
    print(needle, ":", [n for n in names if needle in n])

geom = reg.GetOrLoadValidatorByName(
    "usdGeomValidators:StageMetadataChecker")
core = reg.GetOrLoadValidatorByName(
    "usdValidation:StageMetadataChecker")
stage = Usd.Stage.CreateInMemory()
UsdGeom.Sphere.Define(stage, "/Ball")
UsdGeom.SetStageUpAxis(stage, "Y")
UsdGeom.SetStageMetersPerUnit(stage, 1)
stage.SetDefaultPrim(stage.GetPrimAtPath("/Ball"))
ctx = UsdValidation.ValidationContext([geom, core])
print("context errors:", [e.GetName() for e in ctx.Validate(stage)])
print("ErrorType values:",
      [v.name for v in UsdValidation.ValidationErrorType.allValues])
```

Expected output

```
nValidators: 28
StageMetadataChecker : ['usdGeomValidators:StageMetadataChecker',
                        'usdValidation:StageMetadataChecker']
MissingReferenceValidator : ['usdUtilsValidators:MissingReferenceValidator']
MaterialBindingApiAppliedValidator : ['usdShadeValidators:MaterialBindingApiAppliedValidator']
context errors: []
ErrorType values: ['None_', 'Error', 'Warn', 'Info']
```

Real-world use case. A pipeline registers extra C++ validators under `studioValidators:KindChecker` via `plugInfo.json` (Chapter 36). The same `ValidationContext` list in CI includes both Pixar names and studio names.

Common mistakes.

MISTAKE

Hard-coding ``GetMessage()`` in tests. Messages include layer identifiers. Assert ``GetName()`` and ``GetType()``.

MISTAKE

Expecting suites to be populated on ``usd-core``. ``GetOrLoadAllValidatorSuites()`` returned none here; load validators by name.

Exam traps.

TRAP

"``Validate`` writes a report layer." It returns error objects. Nothing is authored.

TRAP

"`MaterialBindingApiAppliedValidator` is a geom validator." It is ``usdShadeValidators:``.

Exam takeaways.**KEY**

- Registry → named validator → `Validate` → errors. - Context runs several validators. - Assert `GetName()`, not the layer path in the message. - Python cannot `Register\*Validator(lambda)` on 26.08; use a function (30.5) or a plugin.

30.4 Legacy: `UsdUtils.ComplianceChecker`

What is it? `UsdUtils.ComplianceChecker` was the old Python class `usdchecker` used before `UsdValidation`. **It is not in USD 26.08.**

Why do we need it? Exam reading lists and older NVIDIA material still mention it. You must recognize the name, know it is gone, and pick `UsdValidation` instead — not invent a call that will `AttributeError`.

Beginner explanation. `ComplianceChecker` is a **retired inspector**. The office directory still lists the name. The person has left. Ask for `UsdValidation`.

Where the analogy breaks: some studios still run USD 24.x in production. Their docs are not wrong *for that version*. This book and the 26.08 runtime do not have the class.

Technical explanation. | Release | Fact | |-----|-----| | 26.03 | `usdchecker` built on `UsdValidation`; `ComplianceChecker` dropped from the CLI | | 26.05 | `ComplianceChecker` warns when used | | **26.08** | **Removed entirely** from `UsdUtils` |

The next snippet is a fragment (no import); the runnable check is in step 8.

```
hasattr(UsdUtils, "ComplianceChecker") # False on 26.08
```

If a question shows `UsdUtils.ComplianceChecker()`, the current-correct action is “use `UsdValidation.ValidationRegistry`.”

VERSION

Verified on USD 26.08: `hasattr(UsdUtils, "ComplianceChecker")` is False. Do not use it in runnable examples.

Mental model.

```
24.x usdchecker --> ComplianceChecker
26.03 usdchecker --> UsdValidation
26.08 Python --> UsdValidation only (no ComplianceChecker)
```

Simple example. `from pxr import UsdUtils` then `hasattr(UsdUtils, "ComplianceChecker")` prints False.

USDA example. None. This is an API-history fact, not a file format.

Python example.

```
from pxr import UsdUtils

print("ComplianceChecker:", hasattr(UsdUtils, "ComplianceChecker"))
```

Expected output

```
ComplianceChecker: False
```

Real-world use case. A studio upgrading from 24.11 to 26.08 deletes `ComplianceChecker` imports from their publish script and loops `ValidationRegistry().GetOrLoadValidatorByName(...)` instead. The error *names* change; the policy (fail on missing defaultPrim) stays.

Common mistakes.

MISTAKE

Copy-pasting `ComplianceChecker()` from a 2024 blog into a 26.08 environment. It raises `AttributeError`.

Exam traps.**TRAP**

A distractor that still says "call `ComplianceChecker`." That is the legacy answer. Prefer `UsdValidation`.

TRAP

"Removed means validation is gone." The replacement is `UsdValidation`.

Exam takeaways.**KEY**

- `ComplianceChecker` is removed in 26.08. - Replacement: `UsdValidation`. - Older material is a version trap, not current code.

30.5 Writing your own validator

What is it? A **studio integrity checker** is a function you write that inspects a stage and returns problem strings (or `ValidationErrors` if you ship a plugin). Obj 4.6 is satisfied by a real checker that a DCC exporter can call.

Why do we need it? Built-ins do not know that *your* components must have `kind = component` and an authored `extent`. That is pipeline policy. You write it.

Beginner explanation. Built-in validators are **city fire codes**. Your function is the **factory's own checklist**: every shipped crate has a default lid, a stamped size, and a kind label.

Where the analogy breaks: you may later wrap the same rules in a C++ `UsdValidation` plugin so `usdchecker` runs them. Until then, a Python function in the exporter's `validate` step is enough and is what `usd-core` can run.

Technical explanation. A practical Obj 4.6 checker (verified):

1. `defaultPrim` is set.
2. Root layer has `upAxis` and `metersPerUnit` (`pseudoRoot.HasInfo`).
3. Every Boundable with `kind == component` has an authored `extent`.

Call it after hooks (Chapter 29.6). Non-empty list → fail the export.

You can add: legal identifiers, `factory:partId` present, no `UnresolvableDependency` from `MissingReferenceValidator`, `MaterialBindingAPI` applied when a `material:binding` exists.

Plugin path (not runnable as a lambda on 26.08): implement a C++/Python plugin, `RegisterStageValidator` with the metadata the C++ signature expects, ship `plugInfo.json` (Chapters 36–37).

Mental model.

```
pipeline_check(stage) -> ["missing defaultPrim", "no extent on /Crate", ...]
empty list           -> publish
```

Simple example. A Mesh `/Crate` with `kind = component` and nothing else fails four ways: `defaultPrim`, `upAxis`, `metersPerUnit`, `extent`. An Xform root with metadata and `defaultPrim` passes (Xform is not Boundable, so no `extent` demand).

USDA example. *The failing input:*

```
#usda 1.0
```

```
def Mesh "Crate" (
    kind = "component"
)
{
}
```

Python example.

```
from pxr import Usd, UsdGeom

def pipeline_check(stage):
    problems = []
    if not stage.GetDefaultPrim():
        problems.append("missing defaultPrim")
    root = stage.GetRootLayer()
    if not root.pseudoRoot.HasInfo("upAxis"):
        problems.append("missing upAxis")
    if not root.pseudoRoot.HasInfo("metersPerUnit"):
        problems.append("missing metersPerUnit")
    for prim in stage.Traverse():
        if (prim.GetMetadata("kind") == "component"
            and prim.IsA(UsdGeom.Boundable)):
            ext = UsdGeom.Boundable(prim).GetExtentAttr()
            if not ext.HasAuthoredValue():
                problems.append(f"no extent on {prim.GetPath()}")
    return problems

bad = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(bad, "/Crate")
mesh.GetPrim().SetMetadata("kind", "component")
print("bad:", pipeline_check(bad))

good = Usd.Stage.CreateInMemory()
UsdGeom.SetStageUpAxis(good, "Y")
UsdGeom.SetStageMetersPerUnit(good, 1)
root = UsdGeom.Xform.Define(good, "/Crate")
good.SetDefaultPrim(root.GetPrim())
root.GetPrim().SetMetadata("kind", "component")
print("good:", pipeline_check(good))
```

Expected output

```
bad: ['missing defaultPrim', 'missing upAxis', 'missing metersPerUnit', 'no extent on /Crate']
good: []
```

This is a validator you can attach to a DCC export. Lab 27 expands it.

Real-world use case. A CAD exporter calls `pipeline_check` plus `MissingReferenceValidator`. The CAD GUI lists the four strings above next to the export button. Artists learn to fill metadata because the button stays red otherwise.

Common mistakes.

MISTAKE

Demanding `extent` on Xforms. Xform is not Boundable. Check ``IsA(UsdGeom.Boundable)``.

MISTAKE

Using schema fallback (``HasValue()``) as "authored extent." Fallback is not an authored opinion. Use ``HasAuthoredValue()``.

Exam traps.

TRAP

"Obj 4.6 requires `usdGenSchema`." That is custom schemas (Ch 37). A checker function is a validator.

TRAP

"You must `RegisterPrimValidator` from Python for the exam answer to count." The objective is integrity of the asset. A function that returns problems counts; plugins are the production wrap.

Exam takeaways.**KEY**

- Obj 4.6: a real function (or plugin) that flags unusable exports. - Check `defaultPrim`, units, upAxis, extents on Boundable components. - Combine with built-in UsdValidation names. - Python lambdas cannot `Register\*Validator` on 26.08; plugins or plain functions.

Chapter lab(s)

Lab 27 (asset validator) is this chapter's lab: wrap `pipeline_check` plus `MissingReferenceValidator` around the Lab 25 converter output.

USDA reading exercises

Exercise 30-A. `bad.usda` from 30.1 is opened and both `StageMetadata` checkers run. List the three `GetName()` values.

Exercise 30-B. A `Mesh /Crate` has `kind = component`, `defaultPrim`, `upAxis`, and `metersPerUnit`, but no extent. What does `pipeline_check` report?

Answers

30-A. `MissingMetersPerUnitMetadata`, `MissingUpAxisMetadata`, `MissingDefaultPrim`.

30-B. `['no extent on /Crate']`.

Chapter review**Summary**

- Validate after export so missing metadata and broken refs never publish.
- `usdchecker` = CLI on full builds; `UsdValidation` is the library; `usd-core` has no CLI.
- Two `StageMetadata` checkers (`geom units/up`, `usdValidation defaultPrim`).
- `ValidationRegistry`, `Validator.Validate`, `ValidationContext`, error `GetName/GetType`.
- `ComplianceChecker` removed in 26.08.
- Write `pipeline_check` (Obj 4.6); add built-in validators for refs and bindings.

If you see... → think...

If you see...	Think...
Obj 4.6	Integrity checker + <code>UsdValidation</code>
<code>usdchecker</code>	CLI; 26.03+ <code>UsdValidation</code> ; missing in <code>usd-core</code>
<code>ComplianceChecker</code>	Legacy; gone in 26.08
<code>MissingDefaultPrim</code>	<code>usdValidation:StageMetadataChecker</code>
<code>MissingUpAxisMetadata</code>	<code>usdGeomValidators:StageMetadataChecker</code>
<code>UnresolvableDependency</code>	<code>usdUtilsValidators:MissingReferenceValidator</code>
<code>MaterialBindingApiAppliedValidator</code>	Bindings need <code>Apply</code>
Anonymous layer in <code>GetMessage()</code>	Assert <code>GetName()</code> instead

Review questions

R30-01 · Obj 4.6 · Single choice `Stage.Open` of a file with no `upAxis`: A. Always raises B. Succeeds; validation reports `MissingUpAxisMetadata` C. Creates a payload D. Sets Y automatically and errors

R30-02 · Obj 4.6 · Select two. USD 26.08 metadata checkers: A. `usdGeomValidators:StageMetadataChecker` B. `usdValidation:StageMetadataChecker` C. `UsdUtils.ComplianceChecker` D. `GetMaster`

R30-03 · Obj 4.6 · Single choice `usd-core` includes: A. `usdchecker` binary B. `pxr.UsdValidation` C. `ComplianceChecker` D. `usdview`

R30-04 · Obj 4.6 · Single choice `ValidationContext([v1, v2]).Validate(stage)` returns: A. A new layer B. A sequence of `ValidationError` C. True only D. A prototype

R30-05 · Obj 4.6 · Single choice Missing-reference built-in name: A. `usdUtilsValidators:MissingReferenceValidator` B. `usdGeomValidators:EncapsulationChecker` C. `Kind.Registry` D. `Ar.Resolver`

R30-06 · Obj 4.6 · Single choice `ComplianceChecker` in 26.08: A. Required B. Removed C. Renamed `GetPrototype` D. A suite with 28 validators

R30-07 · Obj 4.6 · Select two. A studio `pipeline_check` should flag: A. Missing `defaultPrim` B. Component Boundable without authored extent C. Session-layer existence D. UTF-8 in comments

R30-08 · Obj 4.6 · Single choice `HasAuthoredValue()` on extent vs `HasValue()`: A. The same B. Authored is the integrity check; `HasValue` may be fallback C. Authored is always True on Mesh D. Neither exists

R30-09 · Obj 4.6 · Single choice Registering a Python `lambda` with `RegisterStageValidator` on 26.08: A. Always works B. Does not match the C++ signature C. Replaces `usdchecker` D. Enables Hydra

R30-10 · Obj 4.6 · Single choice When to run validation in an exporter? A. After write and hooks, before publish B. Before extract C. Only on the DCC undo stack D. Instead of `Save`

R30-11 · Obj 4.6 · Single choice `GetType()` for the metadata misses in 30.1: A. Warn B. Error C. Info D. None_

R30-12 · Obj 4.6 · Single choice `usdchecker` since 26.03 uses: A. `ComplianceChecker` B. `UsdValidation` C. `Sdf.ChangeBlock` D. `PointInstancer`

Review answers

R30-01 — B. Review: §30.1.

R30-02 — A and B. Review: §30.3.

R30-03 — B. Review: §30.2, §30.3.

R30-04 — B. Review: §30.3.

R30-05 — A. Review: §30.3.

R30-06 — B. Review: §30.4.

R30-07 — A and B. Review: §30.5.

R30-08 — B. Review: §30.5.

R30-09 — B. Review: §30.3, §30.5.

R30-10 — A. Review: §30.1, §30.5.

R30-11 — B. Review: §30.1.

R30-12 — B. Review: §30.2.

CHAPTER 31

Chapter 31 — File Formats in Depth

Exam domain: Data Exchange (15%) · **Objectives:** 4.3, 7.3 · **Study day:** 9 · **Est. time:** 75 min **Prerequisites:** Ch 7 (USDA / USDC / .usd / USDZ first look)

Chapter 7 named the four encodings. This chapter is the **decision** chapter the exam wants: when USDA vs USDC (Obj 4.3 / 7.3), the USDZ packaging rules, how to tell what a .usd file really is, and the usdcat / usdzip commands with usd-core stand-ins.

Learning goals

- Choose USDA or USDC from performance, readability, diff/merge, and archival needs.
- Explain why a tiny interface file can be *larger* as crate than as text.
- Package a USDZ: uncompressed zip, first file is the root layer, allowed media.
- Detect USDA vs crate vs zip from magic bytes, even when the extension is .usd.
- Convert encodings with Sdf.Layer.Export and UsdUtils.CreateNewUsdzPackage.

Key terms

Term	One-line definition
USDA	Text layer (#usda 1.0); format id usda
USDC / crate	Binary layer (PXR-USDC); format id usdc
.usd	Extension that may hold either encoding; CreateNew writes crate; format id usd
USDZ	Uncompressed zip package of USD layer(s) plus allowed media
Magic bytes	The first bytes of a file that identify the encoding
usdcat	CLI to convert/print layers (not in usd-core); Python: Sdf.Layer.Export
usdzip	CLI to build .usdz (not in usd-core); Python: CreateNewUsdzPackage
ARKit USDZ	Package aimed at Apple viewers; often stores the root as .usdc

31.1 USDA vs. USDC trade-offs

What is it? **USDA** is the text encoding of one layer. **USDC** (crate) is the binary encoding of one layer. Same scene data; different on-disk form. Obj 4.3 and 7.3 ask you to pick one and say why.

Why do we need it? A million-point mesh in USDA is slow to parse and painful in git. A two-line payload interface in USDC is *bigger* than the text and cannot be reviewed in a pull request. The wrong choice is a pipeline tax you pay every day.

Beginner explanation. USDA is **source code**. USDC is a **compiled object file**. You edit and merge source. You ship objects to the renderer. You keep both: a tiny USDA entry layer that payloads a USDC geometry layer (Chapter 23).

Where the analogy breaks: crate is not compiled from USDA by a separate compiler you run. Export("file.usdc") rewrites the same Sdf.Layer. Nothing is lost except comments/layout (crate does not store USDA comments).

Technical explanation. Verified on USD 26.08:

Need	Prefer	Why
Read, diff, merge, code review	USDA	Line-oriented text; git works
	USDC	Faster load; smaller for large layers

Need	Prefer	Why
Heavy geometry, time samples, publish		
Tiny interface (defaultPrim + payload)	USDA	Crate has a fixed header ; a 30-byte USDA was 582 bytes as USDC
Archival of a published mesh	USDC	Compact; not meant to be hand-edited
Archival of a human-authored layout	USDA	You can still read it in 20 years in a text editor
Mixed asset	USDA entry + USDC payload	Interface stays reviewable; geo stays fast

Numbers from the example below (same prims, two encodings):

- Empty Xform: `tiny.usda` **30** bytes, `tiny.usdc` **582** bytes (crate overhead wins).
- 200 spheres: `many.usda` **12119** bytes, `many.usdc` **3171** bytes (crate $\sim 4\times$ smaller).

Other rules:

- `ExportToString()` always emits USDA text, even from a crate layer.
- `CreateNew("x.usdc") / Export("x.usdc")` → format id `usdc`, magic `PXR-USDC`.
- Comments and original USDA whitespace do not survive a round trip through crate.
- Neither encoding is encryption. Crate is not a zip (USDZ is).

Mental model.

```

tiny layer:  USDA 30 B  <  USDC 582 B    (header overhead)
fat layer:   USDA 12 KB >  USDC  3 KB    (binary packing)

typical asset:
  chair.usda    (interface, variants, payload)  keep text
  payload --> chair_geo.usdc                   keep crate

```

Simple example. A published chair: `chair.usda` is 40 lines of metadata. `chair_geo.usdc` holds the mesh. Reviewers diff the USDA. `usdview` loads the crate quickly.

USDA example. The *text* encoding of the tiny layer used in step 8:

```

#usda 1.0

def Xform "W"
{
}

```

That is the entire 30-byte file (plus newline). The crate of the same spec is hundreds of bytes of header.

Python example.

```

from pxr import Usd, UsdGeom
import os

tiny = Usd.Stage.CreateNew("tiny.usda")
UsdGeom.Xform.Define(tiny, "/W")
tiny.GetRootLayer().Save()
tiny.GetRootLayer().Export("tiny.usdc")

many = Usd.Stage.CreateNew("many.usda")
UsdGeom.Xform.Define(many, "/R")
for i in range(200):
    UsdGeom.Sphere.Define(many, f"/R/S{i}").GetRadiusAttr().Set(1.0)
many.GetRootLayer().Save()
many.GetRootLayer().Export("many.usdc")

print("tiny.usda", os.path.getsize("tiny.usda"),
      "tiny.usdc", os.path.getsize("tiny.usdc"))

```

```
print("many.usda", os.path.getsize("many.usda"),
      "many.usdc", os.path.getsize("many.usdc"))
```

Expected output

```
tiny.usda 30 tiny.usdc 582
many.usda 12119 many.usdc 3171
```

For a 30-byte interface, crate is the wrong default. For 200 gprims, crate is the right publish encoding.

Real-world use case. A VFX show: layout and lighting department layers stay USDA so leads can review in GitLab. Animation caches and hero meshes publish as USDC. The shot root is USDA that sublayers both.

Common mistakes.

MISTAKE

Publishing every layer as USDC "for speed," including 8-line payload interfaces. You lose diffs and **increase** size. Keep interfaces USDA.

MISTAKE

Editing USDC in a text editor. You will corrupt the crate. Convert to USDA, edit, convert back if you must.

Exam traps.

TRAP

"USDC is always smaller." False for tiny layers. The table in step 4 is the answer.

TRAP

"USDA cannot hold time samples." It can; it is just verbose.

Exam takeaways.

KEY

- USDA = readable/mergeable; USDC = fast/compact for **large** layers. - Tiny files: USDA can be smaller than crate. - Typical asset: USDA interface, USDC payload. - Comments do not survive crate.

31.2 USDZ packaging rules and usdzip

What is it? USDZ is an **uncompressed, unencrypted zip** of one or more USD layers plus allowed media (images, audio). The **first file in the archive** is the default layer you get when you `Usd.Stage.Open("file.usdz")`.

Why do we need it? Web, AR, and vendor delivery want **one file**. A folder of USDA + PNGs gets lost in email. USDZ is that package, with extra rules so USD can memory-map contents (no DEFLATE).

Beginner explanation. USDZ is a **folder in a box**. The box is zip, but the files inside are stored, not squeezed. The first item in the box is the front door (the root layer). Textures sit beside it.

Where the analogy breaks: you cannot zip an arbitrary folder with the OS GUI and call it USDZ. Compression, absolute paths, nested `.usdz`, and disallowed types (for example Alembic) violate the spec. Use `usdzip` / `CreateNewUsdzPackage`.

Technical explanation. Verified on USD 26.08:

- `UsdUtils.CreateNewUsdzPackage("root.usda", "out.usdz")` → True on success.
- `zipfile` lists `['root.usda', 'tex.png']` when the layer has an `asset` attribute pointing at `./tex.png`.

- `compress_type` is **0** (`ZIP_STORED`). Not DEFLATE.
- `CreateNewARKitUsdzPackage` builds a sibling package for Apple viewers; the **first** entry was `root.usdc` (crate), not `root.usda`.
- `ExtractUsdzPackage(usdz, dir, verbose=False, force=False)` unpacks. Pass `force=True` if the directory already exists; otherwise it can return `False`.
- Opening `out.usdz` gives `GetFileFormat().formatId == "usdz"`. Asset paths inside still look like `./tex.png` (resolved inside the package).

Spec rules the exam expects (OpenUSD USDZ spec; packaging tool enforces most of them):

- No zip compression; 64-byte alignment of file data (mmap).
- Relative paths only; no `...`
- Allowed: `.usd` / `.usda` / `.usdc`, images (png, jpeg, exr, avif, ...), some audio. Not a free-for-all.
- One default layer: the first USD file in the zip.

`usdzip` (full toolset, `.norun` here):

```
usdzip --asset root.usda out.usdz
```

Mental model.

```
out.usdz  (zip, STORE only)
[0] root.usda    <-- Stage.Open uses this
[1] tex.png
```

Simple example. A component with one PNG. `CreateNewUsdzPackage` puts the USDA first and the PNG second, both uncompressed. `ExtractUsdzPackage` restores both files.

USDA example. *File: root.usda* (the layer that gets packed)

```
#usda 1.0
(
    defaultPrim = "Asset"
)

def Xform "Asset"
{
    asset tex = @./tex.png@
}
```

Python example.

```
from pxr import Sdf, Usd, UsdGeom, UsdUtils
import os, zipfile

stage = Usd.Stage.CreateNew("root.usda")
UsdGeom.Xform.Define(stage, "/Asset")
stage.SetDefaultPrim(stage.GetPrimAtPath("/Asset"))
open("tex.png", "wb").write(
    bytes([137, 80, 78, 71, 13, 10, 26, 10]) + b"\x00" * 8)
stage.GetPrimAtPath("/Asset").CreateAttribute(
    "tex", Sdf.ValueTypeNames.Asset).Set(Sdf.AssetPath("./tex.png"))
stage.GetRootLayer().Save()

print("usdzip-equivalent:",
      UsdUtils.CreateNewUsdzPackage("root.usda", "out.usdz"))
print("contents:", zipfile.ZipFile("out.usdz").namelist())
print("compress:",
      [i.compress_type for i in zipfile.ZipFile("out.usdz").infolist()])
print("arkit:",
      UsdUtils.CreateNewARKitUsdzPackage("root.usda", "arkit.usdz"))
print("arkit first:", zipfile.ZipFile("arkit.usdz").namelist()[0])
os.makedirs("unpacked", exist_ok=True)
print("extract:",
      UsdUtils.ExtractUsdzPackage("out.usdz", "unpacked", False, True))
print("unpacked:", sorted(os.listdir("unpacked")))
```

Expected output

```
usdzip-equivalent: True
contents: ['root.usda', 'tex.png']
compress: [0, 0]
arkit: True
arkit first: root.usdc
extract: True
unpacked: ['root.usda', 'tex.png']
```

`compress: [0, 0]` is the exam fact: stored, not deflated. ARKit's first file is crate.

Real-world use case. A manufacturer emails `pump.usdz` to a client. The client opens it on an iPad (ARKit package) and in `usdview` (generic package). Same textures; ARKit edition uses a crate root.

Common mistakes.**MISTAKE**

``zip -r scene.usdz folder`` with default compression. USD may fail to mmap. Use ``CreateNewUsdzPackage``.

MISTAKE

Forgetting ``defaultPrim`` on the packed root. Referencing the USDZ then fails the same way a bare USDA would (Chapter 16).

MISTAKE

``ExtractUsdzPackage`` returning ``False`` because the output dir exists and ``force`` is False.

Exam traps.**TRAP**

"USDZ is compressed USDC." USDC is one layer. USDZ is a zip of files, uncompressed.

TRAP

"The ``root.usda`` inside a USDZ is always the default layer." The **first archive member** that is a USD layer is. ARKit packages may put ``root.usdc`` first.

Exam takeaways.**KEY**

- USDZ = uncompressed zip package, not a crate. - First USD member is the root layer. - Python: ``CreateNewUsdzPackage`` / ``ExtractUsdzPackage`` (``force=True`` if needed). - ARKit helper often crates the root.

31.3 Format detection for `.usd`

What is it? The `.usd` extension **does not tell you the encoding**. The file may be crate or text. You detect by **magic bytes** (and then `GetFileFormat().formatId`).

Why do we need it? The exam and the USD FAQ ask "what file format is my `.usd` file?" Pipelines that assume `.usd` means crate will file a hand-written text `.usd` and panic. Detection is a two-byte (well, nine-byte) look.

Beginner explanation. `.usd` is a **box labeled "USD"** that might hold a letter (USDA) or a brick (USDC). Open the box: if you see `#usda 1.0`, it is a letter; if you see `PXR-USDC`, it is a brick; if you see `PK`, it is actually a zip (usually `.usdz`).

Where the analogy breaks: `GetFileFormat().formatId` for a `.usd` file is often `usd`, not `usda` or `usdc`, even when the bytes are crate or text. The **id follows the extension**; the **magic follows the bytes**. Report both.

Technical explanation. Verified on USD 26.08:

File	Magic (first bytes)	formatId
tiny.usda	#usda 1.0	usda
tiny.usdc	PXR-USDC\x00	usdc
CreateNew("scene.usd")	PXR-USDC\x00	usd
USDA bytes saved as text.usd	#usda 1.0	usd
.usdz	PK (zip local header)	usdz

- `Usd.Stage.CreateNew("scene.usd")` writes **crate** contents. That is the default.
- Copying USDA text into a `.usd` name still **opens**; prims are there; `formatId` stays `usd`.
- Detection function: read 12 bytes; test `startswith(b"#usda")`, `b"PXR-USDC"`, `b"PK"`.

VERSION

Verified on USD 26.08. Default `.usd` = crate` has been the rule for current OpenUSD; always detect, never trust the extension alone.

Mental model.

```
extension = nickname
magic     = actual encoding
formatId  = plugin that claimed the extension
```

Simple example. `scene.usd` from `CreateNew` is crate (PXR-USDC, id `usd`). `text.usd` copied from a `.usda` is text (`#usda 1.0`, id still `usd`).

USDA example. A `.usd` file *can* contain exactly this text — the extension does not forbid it:

```
#usda 1.0

def Xform "W"
{
}
```

If you only `ls *.usd`, you cannot know. Read the first line.

Python example.

```
from pxr import Sdf, Usd, UsdGeom

def detect(path):
    with open(path, "rb") as f:
        b = f.read(12)
    if b.startswith(b"#usda"):
        kind = "usda-text"
    elif b.startswith(b"PXR-USDC"):
        kind = "usdc-crate"
    elif b.startswith(b"PK"):
        kind = "zip-usdz"
    else:
        kind = "unknown"
    return kind, b[:9]

tiny = Usd.Stage.CreateNew("tiny.usda")
UsdGeom.Xform.Define(tiny, "/W")
tiny.GetRootLayer().Save()
tiny.GetRootLayer().Export("tiny.usdc")
st = Usd.Stage.CreateNew("scene.usd")
UsdGeom.Xform.Define(st, "/W")
st.GetRootLayer().Save()
open("text.usd", "wb").write(open("tiny.usda", "rb").read())

for p in ("tiny.usda", "tiny.usdc", "scene.usd", "text.usd"):
    kind, magic = detect(p)
```

```
layer = Sdf.Layer.FindOrOpen(p)
print(p, kind, "formatId=", layer.GetFileFormat().formatId,
      "magic=", magic)
```

Expected output

```
tiny.usda usda-text formatId= usda magic= b'#usda 1.0'
tiny.usdc usdc-crate formatId= usdc magic= b'PXR-USDC\x00'
scene.usd usdc-crate formatId= usd magic= b'PXR-USDC\x00'
text.usd usda-text formatId= usd magic= b'#usda 1.0'
```

`text.usd` is the exam trap: extension `.usd`, id `usd`, **contents USDA**.

Real-world use case. A ingest tool rejects “binary” uploads into git. It must detect magic, not extension: a `.usd` that starts with `#usda` is allowed in the source repo; a `.usd` that starts with `PXR-USDC` goes to the publish store.

Common mistakes.

MISTAKE

``if path.endswith(".usd"): treat_as_crate()`. Fail on hand-written .usd`.`

MISTAKE

Using ``formatId == "usdc"`` to mean crate. A crate `.usd`` reports ``usd``.

Exam traps.

TRAP

`"`.usd` means USDA because the letters are USD ASCII."` Default new `.usd`` is crate.

TRAP

`"`formatId` is always the encoding name."` For `.usd`` it is the **\*\*generic\*\*** id ``usd``.

Exam takeaways.

KEY

- Detect `.usd`` by magic: ``#usda`` vs ``PXR-USDC`` vs ``PK``. - ``CreateNew("*.usd")`` → crate; format id ``usd``. - ``formatId`` follows the extension plugin; magic follows the bytes. - FAQ item: "What file format is my `.usd` file?"

31.4 Conversion tools (`usdcat`, `usdzip`)

What is it? `usdcat` prints or converts a layer (USDA ↔ USDC, optional flatten). `usdzip` builds a `.usdz`. Neither ships with `usd-core`. Python equivalents are `Sdf.Layer.Export / ExportToString` and `UsdUtils.CreateNewUsdzPackage`.

Why do we need it? Obj 4.3 includes converting encodings. Exam questions show CLI flags. This book must also give you a command that runs in the lab `venv`.

Beginner explanation. `usdcat` is **cat for USD**: dump as text, or save as another encoding. `usdzip` is **zip for USD**, with the USDZ rules already applied.

Where the analogy breaks: `usdcat --flatten` is not `cat`. Flatten **bakes composition** into one layer (Chapter 34). Converting USDA→USDC does **not** flatten.

Technical explanation. Full toolset (`.norun` on `usd-core`):

```
usdcat in.usda -o out.usdc
usdcat in.usdc           # print USDA to stdout
usdcat --flatten shot.usda -o flat.usda
usdzip --asset root.usda out.usdz
```

Python stand-ins (verified 26.08):

CLI	Python
<code>usdcat in.usda -o out.usdc</code>	<code>Sdf.Layer.FindOrOpen("in.usda").Export("out.usdc")</code>
<code>usdcat in.usdc (print)</code>	<code>print(layer.ExportToString())</code>
<code>usdzip --asset a out.usdz</code>	<code>UsdUtils.CreateNewUsdzPackage("a", "out.usdz")</code>
<code>unpack</code>	<code>UsdUtils.ExtractUsdzPackage(usdz, dir, verbose, force)</code>

Export picks the encoding from the **destination extension**. `Export("x.usdc")` writes crate (magic PXR-USDC). Flatten is a different API (`Usd.Stage.Flatten`, Chapter 34).

Mental model.

<code>usdcat</code>	<code>= Layer.Export / ExportToString</code>	(encoding)
<code>usdzip</code>	<code>= CreateNewUsdzPackage</code>	(package)
<code>flatten</code>	<code>= Stage.Flatten</code>	(composition; Ch 34)

Simple example. `tiny.usda` exported to `fromcat.usdc` starts with PXR-USDC. Packaging is Section 31.2.

USDA example. No new syntax. Input is any layer from 31.1. Output crate is binary; you check magic, not USDA.

Python example.

```
from pxr import Sdf, Usd, UsdGeom

tiny = Usd.Stage.CreateNew("tiny.usda")
UsdGeom.Xform.Define(tiny, "/W")
tiny.GetRootLayer().Save()
print("usdcat-equivalent export:",
      Sdf.Layer.FindOrOpen("tiny.usda").Export("fromcat.usdc"))
print("fromcat magic:", open("fromcat.usdc", "rb").read(9))
```

Expected output

```
usdcat-equivalent export: True
fromcat magic: b'PXR-USDC\x00'
```

CLI (not run; `usd-core` has no `usdcat`):

```
usdcat tiny.usda -o fromcat.usdc
usdzip --asset root.usda out.usdz
```

Real-world use case. A publish job: `usdcat --flatten` is **not** used (that would smash the asset). Instead `Export` of `chair_geo.usda` → `chair_geo.usdc`, then `CreateNewUsdzPackage` for the client drop.

Common mistakes.

MISTAKE

Using ``usdcat --flatten`` when you only wanted crate. You baked away references.

MISTAKE

``layer.Export("out.usd")`` and assuming USDA. Destination ``.usd`` is crate by default, like ``CreateNew``.

Exam traps.

TRAP

"``usdcat`` is the only way to convert." ``Sdf.Layer.Export`` does it in-process.

TRAP

"``usdzip`` compresses with gzip." It stores, matching 31.2.

Exam takeaways.

KEY

- ``usdc`at -o` ↔ `Sdf.Layer.Export`; stdout` ↔ `ExportToString` . - `usdzip` ↔ `CreateNewUsdzPackage` . - Flatten is a different operation (Ch 34). - Destination `usd` / `usdc` → crate magic.`

Chapter lab(s)

Lab 28 (dependencies, flatten, USDZ) uses these APIs with `LocalizeAsset`. This chapter is the encoding/package theory that lab needs.

USDA reading exercises

Exercise 31-A. `tiny.usda` is 30 bytes and `tiny.usdc` is 582. A shot's lighting layer is 80 lines of overs. Which encoding, and why?

Exercise 31-B. `layout.usd` starts with `#usda 1.0` and `cache.usd` starts with `PXR-USDC`. What is each, and what is `CreateNew("x.usd")`?

Answers

31-A. USDA: it is an interface/department layer for review and merge; crate overhead would dominate and kill diffs. Review: §31.1.

31-B. `layout.usd` is text contents; `cache.usd` is crate; `CreateNew("x.usd")` writes crate with format id `usd`. Review: §31.3.

Chapter review**Summary**

- USDA for humans and tiny interfaces; USDC for large published geometry. Crate is not always smaller.
- Typical asset: USDA entry + USDC payload.
- USDZ is an uncompressed zip; first USD file is the root; `compress_type 0`.
- ARKit packager often crates the root (`root.usdc` first).
- `.usd` encoding = magic bytes, not the extension. Default new `.usd` is crate; `formatId` is `usd`.
- `usdc`at / usdzip` missing from usd-core`; use Export` and CreateNewUsdzPackage` . Flatten ≠ convert.`

If you see... → think...

If you see...	Think...
Obj 4.3 / 7.3	USDA vs USDC table, including tiny-file overhead
PXR-USDC	Crate / USDC
<code>#usda 1.0</code>	Text, even if the name is <code>.usd</code>
PK	Zip → probably USDZ
<code>formatId usd</code>	Generic <code>.usd</code> plugin; still inspect magic
<code>usdzip</code>	<code>CreateNewUsdzPackage</code> ; STORE not DEFLATE
<code>usdc`at -o</code>	<code>Layer.Export` ; not flatten</code>
<code>usdc`at --flatten</code>	Composition bake (Ch 34), not encoding

Review questions

R31-01 · Obj 4.3 · Single choice Best encoding for a 20-line payload interface? A. USDC always B. USDA C. USDZ of Alembic D. Encrypted crate

R31-02 · Obj 7.3 · Single choice Best encoding for a 2 million point cache? A. USDA in git B. USDC C. Comments in crate D. Nested USDZ of USDA copies

R31-03 · Obj 4.3 · Select two. True about crate: A. Magic PXR-USDC B. Drops USDA comments C. Always smaller than USDA D. Is a zip of USDA

R31-04 · Obj 4.3 · Single choice USDZ compression in the worked example: A. DEFLATE B. Type 0 (stored) C. gzip D. Brotli

R31-05 · Obj 4.3 · Single choice Default layer of a USDZ is: A. Any file named `default.usda` B. The first USD archive member C. Always `tex.png` D. The session layer

R31-06 · Obj 4.3 · Single choice `CreateNew("a.usd")` magic: A. `#usda 1.0` B. PXR-USDC C. PK D. empty

R31-07 · Obj 4.3 · Single choice `formatId` of that `a.usd`: A. `usda` B. `usdc` C. `usd` D. `usdz`

R31-08 · Obj 4.3 · Select two. Detect encoding of `.usd` by: A. Extension only B. Magic bytes C. `GetFileFormat().formatId` plus magic D. File mtime

R31-09 · Obj 4.3 · Single choice `usdc` in `usda -o out.usdc` Python stand-in: A. `Stage.Flatten` B. `Sdf.Layer.FindOrOpen("in.usda").Export("out.usdc")` C. `CreateNewUsdzPackage` D. `ComplianceChecker`

R31-10 · Obj 4.3 · Single choice `usdzip` on `usd-core`: A. On PATH B. Use `UsdUtils.CreateNewUsdzPackage` C. `zip -9` D. `usdview --zip`

R31-11 · Obj 4.3 · Single choice `ExtractUsdzPackage` returned `False` because the dir existed. Fix: A. Flatten first B. Pass `force=True` C. Rename to `.usdc` D. Mute the root

R31-12 · Obj 7.3 · Single choice `usdc` `--flatten` vs `Export` to `.usdc`: A. The same B. Flatten bakes composition; `Export` only changes encoding C. Both delete payloads D. Both require `usdview`

Review answers

R31-01 — **B**. Review: §31.1.

R31-02 — **B**. Review: §31.1.

R31-03 — **A and B**. Tiny files can be larger as crate (not C); zip is USDZ (not D). Review: §31.1.

R31-04 — **B**. Review: §31.2.

R31-05 — **B**. Review: §31.2.

R31-06 — **B**. Review: §31.3.

R31-07 — **C**. Review: §31.3.

R31-08 — **B and C**. Review: §31.3.

R31-09 — **B**. Review: §31.4.

R31-10 — **B**. Review: §31.4.

R31-11 — **B**. Review: §31.2.

R31-12 — **B**. Review: §31.4, Ch 34.

Part VI — Pipeline Development

CHAPTER 32

Chapter 32 — Pipeline Architecture and Asset Management

Exam domain: Pipeline Development (14%) · **Objectives:** 7.2 · **Study day:** 10 · **Est. time:** 100 min **Prerequisites:** Ch 15 (sublayers, edit targets), Ch 22 (workstreams, delivery), Ch 23 (asset structure, kinds)

Chapter 23 taught the *shape of one asset*. This chapter is the *studio around it*: how departments, folders, versions, and documents turn those assets into a working **pipeline**. Obj 7.2 is “document asset structure guidelines” — you must be able to draw, name, and explain the rules, not only author a chair.

Learning goals

- Define a USD pipeline as people, folders, publish steps, and composition — not a single file.
- Choose a directory layout and naming scheme that stay stable when versions change.
- Version by **published folders**, not by renaming prims or stacking daily sublayers.
- Inventory an asset with `UsdUtils.ComputeAllDependencies` and contrast it with `ExtractExternalReferences`.
- Diagram layer stacks, asset trees, and ownership so another team can follow them.
- Split collaboration so each department owns one layer and downstream work is stronger.

Key terms

Term	One-line definition
Pipeline	The agreed path from work-in-progress to published, reusable USD
Work file	A layer artists still edit; not what other shots should reference
Publish	Copy or write an immutable snapshot other assets may reference
Pin	A reference (or payload) that points at one published version
Dependency	A layer or file this layer needs (sublayer, reference, payload, texture)
Unresolved path	A dependency <code>ComputeAllDependencies</code> could not find on disk
Asset identifier	Stable name in <code>assetInfo</code> (for example "chair"), independent of version
Guideline document	The written rules and diagrams Obj 7.2 asks you to produce

32.1 What a USD pipeline is

What is it? A **USD pipeline** is the studio’s contract for how 3D work is authored, published, referenced, and handed to the next department. The files are USD. The pipeline is the **rules plus the tools that enforce them**.

Why do we need it? USD will compose whatever you give it. It will not stop two artists from saving the same layer, or a shot from pointing at an artist’s desktop. Without a pipeline, composition still “works” — until a texture goes missing on the farm and nobody knows which chair version the shot meant.

Beginner explanation. Think of a restaurant. The recipes (USD layers) matter, but so do the stations (departments), the walk-in fridge labeled by date (published versions), and the ticket that says which plate is for table 12 (the shot file). A brilliant recipe with no ticket system is not a restaurant.

Where the analogy breaks: plates leave the kitchen once. A published chair stays on disk and is referenced by a thousand shots at once. Publishing is a snapshot, not a one-time serving.

Technical explanation. A small USD pipeline has five moving parts:

Part	What it is	Typical USD tool
Workstreams	One kind of work per layer	Sublayers inside an asset (Ch 22)
Departments	One team per shot layer	Shot sublayer stack, edit targets (Ch 15)
Publish	Immutable copy other people may use	Copy to a versioned folder; never edit in place
Consume	Scenes reference published assets	references / payload with relative paths
Check	Catch missing files and broken metadata	ComputeAllDependencies, UsdValidation (Ch 30)

USD does **not** provide: file locking, a database of versions, or a GUI called “the pipeline.” Those are studio tools. The exam tests whether you can *design* the USD side so those tools have something sensible to call.

Two composition facts that drive the design (already proven in Part III):

- **One writer per layer.** Two saves to the same file overwrite each other.
- **Downstream is stronger.** Later departments sit higher in `subLayers` so they can fix without asking earlier departments to republish.

The shot’s **root layer** is often thin: stage metadata (`defaultPrim`, `upAxis`, `metersPerUnit`) plus the `subLayers` list. Nobody “owns” the root as a dump of opinions.

Mental model.

artist work folders ----- assets/chair/work/ shots/seq010/shot020/work	publish ----- assets/chair/v002/ (immutable)	consume ----- <--- shots/.../layout.usda references that path
shot.usda (thin root) subLayers, strongest first: lighting.usda lighting artist (edit target) anim.usda animator layout.usda layout (places references)		

Simple example. Layout publishes `assets/chair/v002/chair.usda`. The shot’s `layout.usda` references that file. Animation never opens the chair work file; it only authors an over on `/World/Chair_A` in `anim.usda`.

USDA example. File: `shot.usda` — thin root. Department files are listed strongest first.

```
#usda 1.0
(
  defaultPrim = "World"
  metersPerUnit = 1
  upAxis = "Y"
  subLayers = [
    @./lighting.usda@,
    @./anim.usda@,
    @./layout.usda@
  ]
)

def Xform "World"
```

```
{
}
```

- `subLayers` — first listed is strongest (lighting can override anim).
- `./` — paths are anchored to this file, so the shot folder can move as a unit.
- `World` — the shot's namespace root; assets are referenced *under* it in `layout.usda`, not in this file.

Python example.

```
from pxr import Usd, Sdf, UsdGeom
import os

shot = Usd.Stage.CreateNew("shot.usda")
UsdGeom.Xform.Define(shot, "/World")
shot.SetDefaultPrim(shot.GetPrimAtPath("/World"))
shot.SetMetadata("metersPerUnit", 1)
UsdGeom.SetStageUpAxis(shot, UsdGeom.Tokens.y)
for name in ("lighting.usda", "anim.usda", "layout.usda"):
    Sdf.Layer.CreateNew(name).Save()
shot.GetRootLayer().subLayerPaths = [
    "./lighting.usda", "./anim.usda", "./layout.usda"]
shot.GetRootLayer().Save()

stage = Usd.Stage.Open("shot.usda")
print("root:", os.path.basename(stage.GetRootLayer().identifier))
print("stack:")
for layer in stage.GetLayerStack():
    if layer.anonymous:
        print(" session (anonymous, not saved)")
    else:
        print(" ", os.path.basename(layer.identifier))
print("subLayerPaths:", list(stage.GetRootLayer().subLayerPaths))
```

Expected output

```
root: shot.usda
stack:
 session (anonymous, not saved)
shot.usda
lighting.usda
anim.usda
layout.usda
subLayerPaths: ['./lighting.usda', './anim.usda', './layout.usda']
```

`GetLayerStack()` always starts with the **session layer** (anonymous, strongest, not a file). Then the root, then sublayers in list order. That order *is* the pipeline's strength order.

Real-world use case. A digital-twin factory: CAD publishes `cell/v014/cell.usda`. Simulation references that version. Lighting for a marketing still is a stronger sublayer on a *shot* that also references `cell/v014`. CAD can publish `v015` the next day without silently changing yesterday's still, because the still is **pinned**.

Common mistakes.

MISTAKE

Calling "the USD file" the pipeline. One layer is data. The pipeline is work files, publish, pins, and checks.

MISTAKE

Putting layout references in the shot root. Then every department's save rewrites the same file. Fix: references live in `layout.usda`; the root only lists sublayers.

MISTAKE

Using the session layer as a publish destination. It is never saved with the stage (Ch 3).

Exam traps.

TRAP

"USD includes a version database." It does not. Versioning is folders + pins (and optionally a resolver, Ch 33).

TRAP

"First sublayer is weakest." First listed is ****strongest**** (Ch 3, Ch 15).

TRAP

Forgetting the session layer is in ``GetLayerStack()``. The first identifier is often anonymous, not ``shot.usda``.

Exam takeaways.**KEY**

- A pipeline is work → publish → pin → check, with one writer per layer. - Downstream departments sit higher in ``subLayers`` (stronger). - The shot root is metadata + the sublayer list, not a dumping ground. - Obj 7.2: you must be able to ****document**** that structure, not only build it.

32.2 Directory structures and naming conventions

What is it? A **directory structure** is where files live on disk. A **naming convention** is how files, prims, and `assetInfo` identifiers are spelled. Together they are the part of the guideline document that a new hire can follow without asking.

Why do we need it? Relative asset paths (`@./tex/wood.png@`) only work if everyone puts textures in the same place relative to the layer. Scripts that glob `*/chair.usda` only work if every asset uses that name. Inconsistent folders become unresolved paths (Section 32.4).

Beginner explanation. A library that shelves books by color instead of by call number still has books. Nobody can find the same book twice. USD folders are the call numbers. Prim names are the titles on the spine.

Where the analogy breaks: one USD “book” is a *folder* of layers plus textures, not one file. The entry layer is the card in the catalog; the payload is the pages in the stacks (Ch 23).

Technical explanation. A layout that scales from one chair to a feature film:

```
show/
  assets/
    chair/
      work/           artists edit here (not referenced by shots)
      v001/           published, immutable
      v002/
        chair.usda    entry layer (what others reference)
        payload/      heavy layers, often USDC
        tex/          textures belonging to this version
  shots/
    seq010/
      shot020/
        shot.usda     thin root
        layout.usda
        anim.usda
        lighting.usda
```

Rules that belong in the guideline document:

Rule	Why
One published asset = one version folder	The pin is a directory, not a filename trick
Entry file named after the asset (<code>chair.usda</code>)	Tools can find it without a database

Rule	Why
Relative paths only (./, ../)	The tree can be copied or localized (Ch 22)
Prim names are stable (Chair, not Chair_v002)	References keep working; GetModelNameFromRootLayer stays Chair
assetInfo["identifier"] is the dictionary name (chair)	Resolvers and catalogs key off this, not the folder (Ch 12, Ch 33)
Files use valid identifiers where they become prims	Tf.MakeValidIdentifier / Sdf.Path.IsValidIdentifier (Ch 28)
Do not key behavior off names	Use kind, schemas, or properties (Ch 23)

UsdUtils.GetModelNameFromRootLayer(layer) returns the layer's defaultPrim if set, otherwise the name of the (first) root prim. Keep defaultPrim equal to the public prim name, and never bake a version into that name.

EXAM TIP

NVIDIA's four principles (legible, modular, performant, navigable) still apply to folders. Clear names are **legibility**. A version folder you can swap is **modularity**.

Mental model.

```

identifier "chair"    <-- stays the same forever
prim /Chair          <-- stays the same forever
file ../v002/chair.usda <-- version lives HERE
shot references that file <-- pin

```

Simple example. Wrong: Hero_v003.usda whose defaultPrim is Hero_v003. A shot that referenced Hero_v002 cannot switch versions without renaming the prim in every downstream layer.

Right: assets/hero/v003/hero.usda with defaultPrim = "Hero". The shot retargets the asset path only.

USDA example. File: assets/chair/v002/chair.usda

```

#usda 1.0
(
    defaultPrim = "Chair"
    metersPerUnit = 1
    upAxis = "Y"
)

def Xform "Chair" (
    kind = "component"
    assetInfo = {
        string identifier = "chair"
        string name = "DiningChair"
    }
)
{
    asset tex = @./tex/wood.png@
}

```

- defaultPrim = "Chair" — what GetModelNameFromRootLayer returns.
- identifier = "chair" — catalog key; not chair_v002.
- @./tex/wood.png@ — texture lives *with this version*, not in a shared mutable dump.

Python example.

```

from pxr import Sdf, UsdUtils
import os

os.makedirs("assets/chair/v001", exist_ok=True)
open("assets/chair/v001/chair.usda", "w").write("""#usda 1.0
(

```

```

    defaultPrim = "Chair"
)
def Xform "Chair" (
    kind = "component"
    assetInfo = {
        string identifier = "chair"
    }
)
{
    asset tex = @./wood.png@
}
)
)
open("assets/chair/v001/wood.png", "wb").write(b"x")

layer = Sdf.Layer.FindOrOpen("assets/chair/v001/chair.usda")
print("model name:", UsdUtils.GetModelNameFromRootLayer(layer))
print("identifier:",
    layer.GetPrimAtPath("/Chair").assetInfo["identifier"])
tex = layer.GetAttributeAtPath("/Chair.tex").default
print("tex path:", tex.path)
print("relative:", not os.path.isabs(tex.path))

```

Expected output

```

model name: Chair
identifier: chair
tex path: ./wood.png
relative: True

```

Real-world use case. A games studio keeps source under `content/props/barrel/` and published cooked USD under `cooked/props/barrel/v012/`. Engine imports pin `cooked/.../v012`. Artists never reference `content/` from a level file, so a half-saved work file cannot ship.

Common mistakes.

MISTAKE

Absolute paths (`@/mnt/show/assets/chair/wood.png@`). The shot dies when the mount changes. Fix: `./` relative to the layer.

MISTAKE

Sharing one `tex/` folder across versions. Publishing v002 then overwrites v001's textures. Fix: textures live inside the version folder.

MISTAKE

Matching file names to user login (`jdoe_chair.usda`) as the published entry. The identifier and entry name belong to the `**asset**`, not the artist.

Exam traps.

TRAP

"`GetModelNameFromRootLayer` reads `assetInfo['name']`." It uses `defaultPrim` (else a root prim name). `assetInfo` is separate.

TRAP

"Prim names must include the version so you can tell them apart in usdview." That breaks every pin. Use layer comments, `assetInfo`, or the folder name.

TRAP

"`.usd` in the folder name is required." The convention is the `*entry file*` often uses `.usd` so encoding can switch (Ch 23, Ch 31); the folder is still `chair/v002/`.

Exam takeaways.

KEY

- Version lives in the **folder**; prim name and ``identifier`` stay stable. - ``GetModelNameFromRootLayer`` → ``defaultPrim`` (else root prim name). - Relative ``./`` paths; textures travel with the version. - Write these rules down — that writing *is* Obj 7.2.

32.3 Versioning and publishing

What is it? Publishing writes an immutable snapshot of an asset into a versioned location. **Versioning** is how you name those snapshots and how consumers **pin** one of them. Layer stacks are not a version-control system.

Why do we need it? If shots reference a work file, yesterday's lighting render and today's modeling WIP become the same path. If you add `chair_monday.usda` as a new sublayer every day, the stack grows forever and composition slows (Ch 23: do not grow layer stacks as history).

Beginner explanation. A bakery puts each day's bread on a dated shelf. Tuesday's sandwich ticket says "use Tuesday's loaf," not "use whatever is on the counter." Publishing is taking the loaf *off* the counter and putting it on the dated shelf. The counter can keep changing.

Where the analogy breaks: git (or Perforce) may still version the work files. USD publish is an additional, **pipeline-visible** snapshot that other USD files pin. Source control history is not a composition arc.

Technical explanation. Three layers of "version" people confuse:

Mechanism	What it versions	Use it for
Git / Perforce / etc.	Bytes of work files	Artist history, diffs of USDA
Published folder <code>v001</code> , <code>v002</code>	Immutable USD + textures	What shots pin
Layer stack	Strength of opinions	Departments, not history

Publish checklist (document this):

1. Work is done in `.../work/` (or a user sandbox).
2. A publisher copies or exports into `.../vNNN/` (new folder, never overwrite `vNNN` that shots already pin).
3. Run validation (Ch 30) and `ComputeAllDependencies` (Section 32.4); fail if unresolved.
4. Shots that *should* upgrade retarget their reference; others stay pinned.

Retargeting is an asset-path change on the consuming prim:

```
prim.GetReferences().ClearReferences()
prim.GetReferences().AddReference("./assets/chair/v002/chair.usda")
```

The prim path (`/Set/C`) and the asset's internal `/Chair` name do not change. That is why you never put `v002` in the prim name.

VERSION

Verified on USD 26.08. USD still has no built-in "current version" pointer. Some studios add a `.../chair/default`` symlink or an Ar resolver (Ch 33) that maps ``chair`` → ``v002``. The pin is then the identifier plus resolver context, not a raw folder. Teach the folder pin first; it is what the files themselves show.

Mental model.

```
work/chair.usda  --publish-->  v001/chair.usda  <-- shot A (stays)
                  \-->        v002/chair.usda  <-- shot B (upgraded)
```


NOT: shot.usda sublayers v001, then v002, then v003 (that is not versioning)

Simple example. Shot A shipped with chair v001. Modeling publishes v002 with a taller seat (custom double height = 1.2). Shot A still opens v001 until layout retargets. Shot B, started after the publish, pins v002 from the start.

USDA example. File: *set.usda* (consumer, pinned to v001)

```
#usda 1.0
(
    defaultPrim = "Set"
)

def Xform "Set"
{
    def "C" (
        prepend references = @./assets/chair/v001/chair.usda@
    )
    {
    }
}
```

The version is in the **asset path**. The prim is still "C". Inside the asset, `defaultPrim` is still "Chair".

Python example.

```
from pxr import Usd, Sdf, UsdUtils
import os

def write(path, text):
    folder = os.path.dirname(path)
    if folder:
        os.makedirs(folder, exist_ok=True)
    with open(path, "w") as f:
        f.write(text)

write("assets/chair/v001/chair.usda", """"#usda 1.0
(
    defaultPrim = "Chair"
)
def Xform "Chair" (
    kind = "component"
)
{
}
""")
write("assets/chair/v002/chair.usda", """"#usda 1.0
(
    defaultPrim = "Chair"
)
def Xform "Chair" (
    kind = "component"
)
{
    custom double height = 1.2
}
""")
write("set.usda", """"#usda 1.0
(
    defaultPrim = "Set"
)
def Xform "Set"
{
    def "C" (
        prepend references = @./assets/chair/v001/chair.usda@
    )
    {
    }
}
```

```

"""
stage = Usd.Stage.Open("set.usda")
prim = stage.GetPrimAtPath("/Set/C")
print("v001 has height:", prim.HasAttribute("height"))
print("v001 kind:", Usd.ModelAPI(prim).GetKind())
print("model name v001:", UsdUtils.GetModelNameFromRootLayer(
    Sdf.Layer.FindOrOpen("assets/chair/v001/chair.usda")))

prim.GetReferences().ClearReferences()
prim.GetReferences().AddReference("./assets/chair/v002/chair.usda")
stage.Save()

stage2 = Usd.Stage.Open("set.usda")
prim2 = stage2.GetPrimAtPath("/Set/C")
print("v002 height:", prim2.GetAttribute("height").Get())
print("model name v002:", UsdUtils.GetModelNameFromRootLayer(
    Sdf.Layer.FindOrOpen("assets/chair/v002/chair.usda")))
print("prim name still:", prim2.GetName())

```

Expected output

```

v001 has height: False
v001 kind: component
model name v001: Chair
v002 height: 1.2
model name v002: Chair
prim name still: C

```

The consuming prim stayed C. The asset's model name stayed Chair. Only the **path** of the reference changed.

Real-world use case. A film shot is locked for lighting. Modeling must still publish v018 of a hero prop for another sequence. Lighting's pin on v017 does not move. A producer later asks to "pick up v018 in shot 120"; layout retargets that one shot and lighting re-renders it.

Common mistakes.

MISTAKE

Overwriting `v002/` in place because "it's the same version number." Anyone already pinning `v002` silently changes. Fix: new number, or a policy that `v002` is frozen once any shot pins it.

MISTAKE

Using sublayers as history (`monday.usda`, `tuesday.usda`). Every layer costs composition time. Fix: source control for history; one current published layer per workstream.

MISTAKE

Referencing `assets/chair/work/chair.usda` from a shot "just for now." That path will still be there on the farm.

Exam traps.

TRAP

"Add a stronger sublayer to upgrade an asset." A sublayer of the **shot** cannot replace which file a **reference** points at. You change the reference (or use a resolver).

TRAP

"`ClearReferences` deletes the prim." It removes the arc. The `def "C"` spec remains; you then `AddReference` the new path.

TRAP

"Published USDC cannot be pinned because it isn't text." Encoding is independent of publish. Pin the entry layer, whatever its encoding (Ch 31).

Exam takeaways.**KEY**

- Publish = new immutable folder; shots pin that folder. - Do not use layer stacks as version history. - Retarget the asset path; keep prim names and `defaultPrim` stable. - Never let shots reference work files.

32.4 Dependencies (UsdUtils.ComputeAllDependencies)

What is it? A **dependency** is any other file a layer needs in order to compose or render: sublayers, references, payloads, clips, and asset-valued attributes (textures).

`UsdUtils.ComputeAllDependencies` walks that graph from one starting file and returns everything it can find — plus a list of what it cannot.

Why do we need it? A publisher that copies `chair.usda` but forgets `tex/wood.png` produces a package that opens and then renders black. Inventory the graph **before** you copy, flatten, or zip (Ch 22, Ch 31).

Beginner explanation. Before you email a spreadsheet that says `=VLOOKUP` into another workbook, you attach that workbook too. `ComputeAllDependencies` is “list every workbook and image this spreadsheet pulls in, and list the `#REF!` links.”

Where the analogy breaks: USD follows composition arcs **recursively** (a shot’s layout file’s chair’s payload’s texture). Spreadsheet attach-dialogs often stay one hop.

Technical explanation. Verified on USD 26.08:

```
UsdUtils.ComputeAllDependencies(assetPath)
-> (layers, assets, unresolved)
```

`assetPath` may be a string or an `Sdf.AssetId`. Both work.

Return slot	Type	Contains
layers	list of <code>Sdf.Layer</code>	Every USD layer opened while walking
assets	list of <code>str</code>	Non-layer files that resolved (textures, ...)
unresolved	list of <code>str</code>	Paths that did not resolve; often absolute

The walk follows sublayers, references, **and payloads** (payloads are still dependencies even if a stage might later unload them). Missing files produce Tf warnings on stderr and appear in `unresolved`. The call still returns; it does not raise.

A related one-layer helper:

```
UsdUtils.ExtractExternalReferences(filePath)
-> (sublayers, referencesAndAssets, payloads)
```

Differences that exam answers mix up:

	ExtractExternalReferences	ComputeAllDependencies
Scope	One file, authored strings	Recursive , resolved files
Assets	Texture paths sit in the second list with references	Separate <code>assets</code> list
Missing	Still listed as authored <code>@./gone.jpg@</code>	Listed in <code>unresolved</code> (often absolutized)
Recurse into sublayers?	No	Yes

`UsdUtils.LocalizeAsset` and `CreateNewUsdzPackage` (Ch 22, Ch 31) use this same inventory internally. This section is the inventory; those chapters are the copy/zip.

VERSION

``UsdUtils.LocalizeAsset`` exists since 24.03. ``ComputeAllDependencies`` is older. On 26.08 both are present.

Mental model.

<code>ExtractExternalReferences(file)</code>	<code>ComputeAllDependencies(file)</code>
-----	-----
what THIS file writes	the whole snowball
three authored-path buckets	layers assets missing

Simple example. `asset.usda` payloads `payload.usda`, references `seat.usda`, has `@./tex/wood.png@` (exists) and `@./gone.jpg@` (missing). Compute: three layers, one asset `wood.png`, one unresolved `gone.jpg`. Extract on `asset.usda` alone: empty sublayers, refs+assets `./seat.usda` + textures, payloads `./payload.usda`.

USDA example. *File: `asset.usda`*

```
#usda 1.0
(
    defaultPrim = "Chair"
)

def Xform "Chair" (
    kind = "component"
    prepend payload = @./payload.usda@
)
{
    asset tex = @./tex/wood.png@
    asset missing = @./gone.jpg@
    def "Seat" (
        prepend references = @./seat.usda@
    )
    {
    }
}
```

Three different dependency *kinds* on one prim: payload, asset attributes, child reference. `ExtractExternalReferences` puts them in different buckets; `ComputeAllDependencies` flattens them into layers vs assets vs missing.

Python example. Stderr will show a “Failed to resolve” warning for `gone.jpg`. That is expected. The checker compares stdout only.

```
from pxr import UsdUtils
import os

os.makedirs("tex", exist_ok=True)
open("tex/wood.png", "wb").write(b"PNG")
open("seat.usda", "w").write("""#usda 1.0
(
    defaultPrim = "Seat"
)
def Cube "Seat" {}
""")
open("payload.usda", "w").write("""#usda 1.0
(
    defaultPrim = "Extra"
)
def Sphere "Extra" {}
""")
open("asset.usda", "w").write("""#usda 1.0
(
    defaultPrim = "Chair"
)
def Xform "Chair" (
    kind = "component"
```

```

    prepend payload = @./payload.usda@
)
{
    asset tex = @./tex/wood.png@
    asset missing = @./gone.jpg@
    def "Seat" (prepend references = @./seat.usda@) {}
}
"""

layers, assets, unresolved = UsdUtils.ComputeAllDependencies("asset.usda")
print("layers:", sorted(os.path.basename(l.realPath) for l in layers))
print("assets:", sorted(os.path.basename(a) for a in assets))
print("unresolved:", sorted(os.path.basename(u) for u in unresolved))

subs, refs, pays = UsdUtils.ExtractExternalReferences("asset.usda")
print("extract sublayers:", subs)
print("extract refs+assets:", sorted(refs))
print("extract payloads:", pays)

```

Expected output

```

layers: ['asset.usda', 'payload.usda', 'seat.usda']
assets: ['wood.png']
unresolved: ['gone.jpg']
extract sublayers: []
extract refs+assets: ['./gone.jpg', './seat.usda', './tex/wood.png']
extract payloads: ['./payload.usda']

```

Read the extract line twice: `./gone.jpg` and `./tex/wood.png` sit next to `./seat.usda`. The second bucket is **not** “references only.”

Real-world use case. A CI job runs `ComputeAllDependencies` on every published entry layer. If `unresolved` is non-empty, the publish is rejected. A second job compares the `assets` list to the files actually copied into `vnnn/` so a texture cannot be “forgotten” on a network share.

Common mistakes.

MISTAKE

Treating `ExtractExternalReferences` as a complete ship list. It does not walk into `layout.usda` to find the chair. Fix: `ComputeAllDependencies` on the shot or the asset entry.

MISTAKE

Assuming `unresolved` paths look like the authored `@./gone.jpg@`. They are often absolutized. Compare with `os.path.basename` or normalize before matching.

MISTAKE

Ignoring `stderr` warnings because the function returned a tuple. Unresolved items are a failed publish, not a footnote.

Exam traps.

TRAP

`ComputeAllDependencies` returns a dict `{layers: ...}`. It returns a **3-tuple**.

TRAP

“Payloads are omitted because they might be unloaded.” They are still dependencies. `Compute` includes them; `extract` puts them in the **third** list.

TRAP

“The second extract list is only USD references.” Asset-valued attributes land there too.

Exam takeaways.

KEY

- ``ComputeAllDependencies`` → ``(layers, assets, unresolved)``; recursive. -
``ExtractExternalReferences`` → ``(sublayers, refs+assets, payloads)``; one file. - Payloads
 count. Textures are assets (compute) or mixed into refs (extract). - A non-empty
``unresolved`` list fails a publish.

32.5 Diagramming and documenting asset structure

What is it? Documenting asset structure (Obj 7.2) means producing diagrams and short written rules another person can follow: folder tree, layer stack, prim tree with kinds, and an ownership table. The document is part of the pipeline, not a slide you make after the exam.

Why do we need it? A perfect on-disk layout that exists only in one TD's head cannot be validated, onboarded, or examined. NVIDIA's objective uses the verb **document**. The exam will show a diagram or a USDA snippet and ask whether it matches the guidelines (Ch 23's ASWF/NVIDIA rules plus this chapter's folders and pins).

Beginner explanation. A fire-escape plan on the wall is not the building. You still need it so a guest can get out. Asset-structure diagrams are the fire-escape plan for chairs, shots, and departments.

Where the analogy breaks: the diagram must match the **files**, or CI will disagree with the wiki. Generate trees from the stage when you can (step 8).

Technical explanation. Four diagrams belong in every guideline pack:

A. Folder tree (Section 32.2) — where versions and textures live.

B. Layer-stack strip — strongest at the top, one owner per box:

```
session (not saved)
shot.usda           supervisor / metadata
lighting.usda       lighting
anim.usda           animation
layout.usda         layout (references published assets)
```

C. Asset composition — entry → payload → workstream layers (Ch 23):

```
chair.usda (text entry: defaultPrim, kind, assetInfo, payload)
  payload --> contents.usdc
               subLayers: geo.usdc , mtl.usdc
```

D. Model hierarchy — kinds only, so navigability is visible:

```
Room      assembly
Furniture group
Chair     component
Seat      (no kind; inside component)
```

Plus a **table of ownership** (who may author which properties). Geometry owns `points`; surfacing owns `bindings`; nobody else writes those names. Overlap is a silent LIVERPS fight (Ch 21).

Minimum written rules to print next to the diagrams (from Ch 23, restated as a checklist):

- Published roots are `component` or `assembly`, never `group`.
- Loft kind, `assetInfo`, variant set names above the payload.
- Reference published entries; payload heavy content.
- Relative paths; one writer per layer; downstream stronger.

Mental model.

```
guideline document
1. pictures (folders, stacks, kinds)
```

2. tables (owners, pins, names)
3. checks (ComputeAllDependencies, validators)

If it is not in those three, it is folklore.

Simple example. A one-page PDF titled “Chair v002” with: folder tree, the entry USDA’s metadata block, a kind tree, and “layout references assets/chair/v002/chair.usda; do not reference work/.”

USDA example. The model-hierarchy diagram in step 4 is this layer:

```
#usda 1.0
(
    defaultPrim = "Room"
)

def Xform "Room" (
    kind = "assembly"
)
{
    def Xform "Furniture" (
        kind = "group"
    )
    {
        def Xform "Chair" (
            kind = "component"
        )
        {
            def Cube "Seat"
            {
            }
        }
    }
}
```

A reviewer can check the diagram against kind metadata without opening usdview.

Python example.

```
from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
room = UsdGeom.Xform.Define(stage, "/Room")
Usd.ModelAPI(room.GetPrim()).SetKind("assembly")
furn = UsdGeom.Xform.Define(stage, "/Room/Furniture")
Usd.ModelAPI(furn.GetPrim()).SetKind("group")
chair = UsdGeom.Xform.Define(stage, "/Room/Furniture/Chair")
Usd.ModelAPI(chair.GetPrim()).SetKind("component")
UsdGeom.Cube.Define(stage, "/Room/Furniture/Chair/Seat")

def tree(prim, indent=0):
    kind = Usd.ModelAPI(prim).GetKind() or "-"
    print(" " * indent + prim.GetName() + " kind=" + kind)
    for child in prim.GetChildren():
        tree(child, indent + 1)

tree(stage.GetPrimAtPath("/Room"))
```

Expected output

```
Room kind=assembly
  Furniture kind=group
    Chair kind=component
      Seat kind=-
```

This printout is diagram D. Keep it in the publish log next to the folder tree.

Real-world use case. An AEC firm hands contractors a two-page “USD package spec”: folder names, required `assetInfo` keys, the layer-stack order for a building, and a screenshot of the kind tree. Incoming files are rejected when `ComputeAllDependencies` or the kind tree disagrees with the spec.

Common mistakes.

MISTAKE

Documenting only prims and skipping folders. Pins live in paths (Section 32.3).

MISTAKE

Drawing a layer stack with layout on top "because layout comes first in the schedule." Time-order of work and strength-order of layers are different. Downstream (lighting) is stronger.

MISTAKE

A wiki diagram that still shows ``kind = component`` under another component. The files must match ASWF (Ch 23): that child should be ``subcomponent``.

Exam traps.

TRAP

Obj 7.2 is satisfied by "we use USD." The objective is ****guidelines****: names, folders, kinds, ownership, pins.

TRAP

A diagram that puts ``group`` on a published root. ASWF: published roots are ``component`` or ``assembly``.

TRAP

"Seat kind=- means the cube is invalid." No kind on a `gprim` inside a component is normal.

Exam takeaways.

KEY

- Obj 7.2 = write diagrams + ownership rules that match the files. - Four pictures: folders, layer stack, asset payload, kind tree. - Generate the kind tree from the stage so the wiki cannot drift. - Strength order $\neq$ production schedule order.

32.6 Collaboration across departments

What is it? **Collaboration** in a USD pipeline means each department writes to **its own layer**, through an **edit target**, and hands off by **publishing** — not by emailing a merged file. The composed shot is the stack; the people never share a save button.

Why do we need it? USD has no merge tool for two writers of one crate file. Parallel work is parallel **files**. Chapters 15 and 22 taught the mechanics. Here you connect them to publish and to the guideline document.

Beginner explanation. A newspaper: reporters file separate stories (layers). The night editor's page is a stronger sheet that can still override a headline. Nobody types into someone else's story file.

Where the analogy breaks: USD overrides are per property, not per article. Animation can set translate on `/World/Prop` while layout still owns the reference that put `Prop` there.

Technical explanation. Handoff pattern that belongs in the guidelines:

Moment	Who	Where they write	What they publish
Place assets	Layout	<code>layout.usda</code>	References to published asset versions

Moment	Who	Where they write	What they publish
Move them	Animation	anim.usda	over opinions (xforms, time samples)
Light	Lighting	lighting.usda	Lights, stronger xform fixes if needed
Lock a version	Pipeline	new vNNN/ of an asset	Immutable folder; shots retarget if asked

Rules:

- **Edit target = department layer.**
stage.SetEditTarget(Usd.EditTarget(anim_layer)) so AddTranslateOp cannot accidentally hit layout.usda.
- **def once, over elsewhere.** Layout defs /World/Prop. Animation authors over "Prop" in its file (Ch 20).
- **Assets are referenced, not sublayered, into the shot.** Sublayers merge namespaces at the same paths; a chair asset should appear under /World/Chair_A, which is a reference (Ch 16 vs Ch 15).
- **Mute to review.** MuteLayer on anim.usda answers "what did animation change?" without deleting files (Ch 15).
- **Do not publish another department's layer.** Animation's publish is anim.usda (or a shot-level package), not a new chair vNNN unless modeling actually changed the chair.

Inspect **who won** with GetPropertyStack() (Ch 22): the strongest spec's layer should be the department you expect.

Mental model.

```
layout.usda    def /World/Prop + reference to published chair
anim.usda      over /World/Prop { translate }
lighting.usda  over /World/Prop { ... optional fixes }
```

Save buttons: three files. Composed result: one stage.

Simple example. Layout defines /World/Prop. Animation's edit target is anim.usda. After both save, xformOp:translate's property stack lists anim.usda first. Layout still has the def.

USDA example. File: anim.usda (what animation publishes for the shot)

```
#usda 1.0

over "World"
{
    over "Prop"
    {
        double3 xformOp:translate = (3, 0, 0)
        uniform token[] xformOpOrder = ["xformOp:translate"]
    }
}
```

No def, no reference, no texture path. Animation does not take ownership of placement or the asset pin.

Python example.

```
from pxr import Usd, Sdf, UsdGeom
import os

layout = Sdf.Layer.CreateNew("layout.usda")
anim = Sdf.Layer.CreateNew("anim.usda")
shot = Usd.Stage.CreateNew("shot.usda")
UsdGeom.Xform.Define(shot, "/World")
shot.GetRootLayer().subLayerPaths = ["/anim.usda", "/layout.usda"]

shot.SetEditTarget(Usd.EditTarget(layout))
UsdGeom.Xform.Define(shot, "/World/Prop")
```

```

shot.SetEditTarget(Usd.EditTarget(anim))
UsdGeom.Xform.Get(shot, "/World/Prop").AddTranslateOp().Set((3, 0, 0))

shot.GetRootLayer().Save()
layout.Save()
anim.Save()

attr = shot.GetPrimAtPath("/World/Prop").GetAttribute(
    "xformOp:translate")
print("translate stack:",
      [os.path.basename(s.layer.identifier)
       for s in attr.GetPropertyStack()])
print("layout defines Prop?",
      layout.GetPrimAtPath("/World/Prop").specifier == Sdf.SpecifierDef)
print("anim is over?",
      anim.GetPrimAtPath("/World/Prop").specifier == Sdf.SpecifierOver)
print(anim.ExportToString())

```

Expected output

```

translate stack: ['anim.usda']
layout defines Prop? True
anim is over? True
#usda 1.0

over "World"
{
    over "Prop"
    {
        double3 xformOp:translate = (3, 0, 0)
        uniform token[] xformOpOrder = ["xformOp:translate"]
    }
}

```

Animation's file is only overs. The stack for translate has a single spec, in `anim.usda`. Layout still owns the def.

Real-world use case. A live-events team builds a stage: scenic layout pins published set pieces; animation publishes a 90-second `anim.usda` of LED-wall xforms; lighting publishes last. When the director asks "show me the empty set," the operator mutes `anim.usda`. No file is rewritten.

Common mistakes.

MISTAKE

Leaving the edit target on the session layer (the default in some tools) and wondering why Save does not keep the animation. Fix: set the department layer before authoring.

MISTAKE

Animation adding a second reference to the chair "to be safe." Now two arcs fight. Fix: only layout holds the pin; animation overs the instance.

MISTAKE

Lighting editing ``chair/v002/geo.usdc`` to hide a bolt. That mutates a published asset for every shot. Fix: an ``over`` in ``lighting.usda`` or a shot-specific stronger layer.

Exam traps.

TRAP

"Collaboration means ``Flatten`` at the end of each day." Flattening destroys arcs (Ch 22). Daily handoff is `**saving each department layer**`.

TRAP

"Stronger department must ``def`` the prim." Stronger files usually ``over``. The first owner ``def`s`.

TRAP

``GetPropertyStack`` empty means the attribute has no value. It can also mean you queried a prim that does not exist, or a name that was never authored (fallback vs authored — Ch 5).

Exam takeaways.**KEY**

- One department, one layer, one edit target; ``def`` once, ``over`` downstream. - Shots reference published assets; they do not sublayer them. - Handoff is saving (and publishing) layers, not flattening daily. - ``GetPropertyStack`` names the department that won a property.

Chapter lab(s)

- **Lab 22** (Chapter 23) builds a component and an assembly — the asset side of these guidelines.
- **Lab 28** (Chapter 31) runs `ComputeAllDependencies`, `localize`, and `USDZ` — the publish-inventory side.
- **Lab 37** (capstone) is a miniature pipeline: work → publish → pin → validate.

USDA reading exercises

Exercise 32-A. A shot root lists `subLayers = [ @./anim.usda@, @./layout.usda@ ]`. Layout references `@../.../assets/lamp/work/lamp.usda@`. Name two guideline violations.

Exercise 32-B. `ExtractExternalReferences("shot.usda")` returns `(['./anim.usda', './layout.usda'], [], [])`. Does that prove the shot has no textures? What call would you run instead?

Answers

- **32-A.** (1) The shot pins a **work** file, not a published version folder. (2) If lighting exists, it is missing from the stack — and even without lighting, work-file pins are the serious break. (Relative `./` on department layers is fine.)
- **32-B.** No. `Extract` does not recurse, so textures on the lamp never appear. Run `ComputeAllDependencies("shot.usda")` and inspect `assets` and `unresolved`.

Chapter review**Summary**

- A USD pipeline is work → publish → pin → check, with one writer per layer.
- Version lives in **folders**; prim names and `assetInfo` identifiers stay stable.
- `GetModelNameFromRootLayer` reads `defaultPrim`.
- `ComputeAllDependencies` → (`layers`, `assets`, `unresolved`) (recursive).
- `ExtractExternalReferences` → (`sublayers`, `refs+assets`, `payloads`) (one file).
- Obj 7.2 is **documentation**: folders, stacks, kinds, ownership.
- Downstream departments are stronger; they `over`, they do not edit published assets.
- Layer stacks are not version control; git is not a composition arc.

If you see... → think...

If you see...	Think...
"document asset structure"	Obj 7.2: diagrams + naming + pins + owners
<code>Hero_v003</code> as <code>defaultPrim</code>	Version in the wrong place
<code>ComputeAllDependencies</code> three items	<code>layers</code> , <code>assets</code> , <code>unresolved</code>

If you see...	Think...
extract's second list includes <code>.png</code>	refs and asset attributes
shot references <code>.../work/</code>	unpublished pin
two artists, one <code>.usdc</code>	missing department layers
daily <code>monday.usda</code> sublayer	stacks used as history
lighting below layout in the list	strength order inverted

Review questions

R32-1 (Obj 7.2) Which statement belongs in an asset-structure guideline document? A. "USD will lock the file when an artist has it open." B. "Published roots use `component` or `assembly`; version is a folder; shots pin that folder." C. "All departments write the shot root so Save is centralized." D. "Put `v003` in the prim name so `usdview` labels stay unique."

R32-2 (Obj 7.2) `UsdUtils.GetModelNameFromRootLayer(layer)` returns `"Hero_v003"`. What probably went wrong? A. The layer has no `defaultPrim` and the root prim was named with a version B. `ComputeAllDependencies` failed C. The identifier in `assetInfo` is `"hero"` D. The file is `USDC`

R32-3 (Obj 7.2) Select two correct facts about `ComputeAllDependencies`. A. It returns a 3-tuple `(layers, assets, unresolved)` B. It follows payloads C. It never lists textures D. It raises if any path is missing

R32-4 (Obj 7.2) `ExtractExternalReferences("shot.usda")` does not list `chair.usda`, but the composed shot shows a chair. Why? A. References are illegal on sublayers B. Extract does not recurse; the reference is in `layout.usda` C. The chair is only in the session layer D. `chair.usda` is `USDC` so extract ignores it

R32-5 (Obj 7.2) Best pin for a locked lighting shot? A. `assets/chair/work/chair.usda` B. `assets/chair/v017/chair.usda` C. A sublayer of every historical `v001...v017` D. `/Chair_v017` as the prim name, any folder

R32-6 (Obj 7.2) In `GetLayerStack()` of an opened shot, what is first? A. The weakest sublayer B. The anonymous session layer C. `layout.usda` D. The published chair entry

R32-7 (Obj 7.2) Select two collaboration rules. A. Animation authors `over` in `anim.usda` via that layer's edit target B. Lighting may `overwrite` `v017/geo.usdc` to hide a bolt for one shot C. Layout holds the asset reference D. Daily `Flatten` is the handoff

R32-8 (Obj 7.2) A texture `@./tex/wood.png@` exists. Where does `ComputeAllDependencies` put it? A. `layers` B. `assets` C. `unresolved` D. It is omitted because it is not `USD`

R32-9 (Obj 7.2) Why are relative `./` paths part of the guideline? A. Crate files forbid absolute paths B. The tree can move, localize, or package without rewriting mounts C. `defaultPrim` requires them D. `ExtractExternalReferences` cannot see absolute paths

R32-10 (Obj 7.2) Lighting is drawn *below* layout on a stack diagram labeled "strongest at top." The diagram claims lighting can override layout translates. The claim is: A. True, because lighting works later B. False; below means weaker C. True, because payloads are always weaker than sublayers D. True only if lighting uses `def` instead of `over`

R32-11 (Obj 7.2) `assetInfo['identifier'] = "chair"` should change when? A. Every `publish(chair_v002, then chair_v003)` B. Almost never; it is the catalog key C. Whenever `defaultPrim` changes to match the folder D. Only for `USDZ` delivery

R32-12 (Obj 7.2) Which pair is the right split? A. Version control (`git`) = history of work files; publish folders = what shots pin B. Version control = composition arcs; publish = `MuteLayer` C. Both are `subLayers` lists D. Publish folders replace the need for `kind`

Review answers

- **R32-1: B.** That sentence is a guideline. A is false (USD has no lock). C and D violate one-writer and stable names. Review: §32.1–32.3, §32.5.
- **R32-2: A.** The helper reads `defaultPrim` or the root prim name. Review: §32.2.
- **R32-3: A and B.** Textures go in `assets`; missing paths list, they do not raise. Review: §32.4.
- **R32-4: B.** One-file extract. Review: §32.4.
- **R32-5: B.** Immutable version folder. Review: §32.3.
- **R32-6: B.** Session is strongest and first. Review: §32.1.
- **R32-7: A and C.** Do not mutate published assets; do not flatten as handoff. Review: §32.6.
- **R32-8: B.** Non-layer resolved files. Review: §32.4.
- **R32-9: B.** Portability of the pin. Review: §32.2.
- **R32-10: B.** First listed / higher drawn = stronger. Review: §32.1, §32.5.
- **R32-11: B.** Identifier is stable; version is the folder. Review: §32.2.
- **R32-12: A.** Two different jobs. Review: §32.3.

CHAPTER 33

Chapter 33 — Asset Resolution

Exam domain: Pipeline Development (14%), also Customizing USD (6%) and Debugging (11%) · **Objectives:** 3.5, 6.3, 7.5, 7.7 · **Study day:** 10 · **Est. time:** 90 min
Prerequisites: Ch 9.8 (asset values), Ch 16 (anchored references), Ch 32 (pins, `ComputeAllDependencies`)

An **asset path** is a string USD must turn into a real file (or package member). **Ar** (Asset Resolution) is the library that does that turn. Obj 7.7 is “validate asset paths are formatted correctly.” Obj 3.5 / 7.5 are “integrate a custom resolver.” Obj 6.3 is “resolve issues related to asset management” — the debugging face of the same facts.

Learning goals

- Distinguish authored asset paths (`Sdf.AssetPath`, USDA `@...@`) from **resolved** filesystem paths.
- Anchor a relative path to the layer that wrote it (`CreateIdentifier`, `Sdf.Layer.ComputeAbsolutePath`).
- Use `Ar.DefaultResolverContext` and `Ar.ResolverContextBinder` / `Usd.Stage.Open(..., pathResolverContext=)` for search paths.
- Validate format and resolvability (Obj 7.7) without calling `Resolve` on a bare `./` path from the wrong working directory.
- Explain how a custom `ArResolver` plugin is installed and selected (Obj 3.5 / 7.5) without inventing APIs.

Key terms

Term	One-line definition
Asset path	Authored string naming a dependency (<code>Sdf.AssetPath</code> , USDA <code>@./tex.png@</code>)
Resolved path	What <code>Ar</code> turned that string into (<code>Ar.ResolvedPath</code>); empty if missing
Ar	USD’s asset-resolution library (<code>pxr.Ar</code>)
Resolver	The plugin that implements <code>Resolve</code> (<code>Ar.GetResolver()</code>)
Default resolver	Filesystem resolver shipped with USD (<code>Ar.DefaultResolver</code>)
Anchor	Interpret <code>./</code> / <code>../</code> relative to the layer file , not your shell cwd
Search path	Directories the default resolver tries for context-dependent paths
Context-dependent path	A path that uses the resolver context (no <code>./</code> , not absolute)
Resolver context	Per-stage (or bound) extra info, e.g. search directories
URI scheme	Prefix such as <code>s3:</code> ; needs a resolver that registered that scheme

33.1 Asset paths and Ar

What is it? An **asset path** is a value whose job is to name another file. **Ar** is the USD library that **resolves** that name to bytes on disk (or inside a package). `pxr.Ar` sits under `Sdf`, `Pcp`, and `Usd`: composition cannot open a reference until `Ar` succeeds.

Why do we need it? `@./tex/wood.png@` is not a Python path and not a prim path. If you treat it as either, textures go missing and references fail with `Pcp.ErrorInvalidAssetPath` (Obj 6.3).

Beginner explanation. A library catalog card says “Shelf B-12, *Wood Chair*.” The card is the **asset path**. Walking to the shelf and picking up the book is **resolution**. Ar is the librarian. A custom resolver (Section 33.5) is a librarian who also understands warehouse codes such as `s3://...`.

Where the analogy breaks: the card can be relative to *which catalog you opened* (the layer), not relative to where you are standing (the process `cwd`). That surprise is Section 33.2.

Technical explanation. Verified on USD 26.08:

Piece	Type / API	Role
Authored value	<code>Sdf.AssetPath</code> , USDA <code>@path@</code>	Stored on attributes, references, payloads, sublayers
Resolver singleton	<code>Ar.GetResolver()</code>	Process-wide resolver object
Underlying plugin	<code>Ar.GetUnderlyingResolver()</code>	Usually <code>DefaultResolver</code> in <code>usd-core</code>
Success / failure	<code>Ar.ResolvedPath</code>	<code>bool(resolved)</code> is <code>False</code> when empty
Package member	<code>pkg.usdz[foo.usda]</code>	<code>Ar.IsPackageRelativePath</code> , <code>JoinPackageRelativePath</code>

`Sdf.AssetPath` fields (26.08):

Field	Meaning
<code>path / authoredPath</code>	What was written
<code>resolvedPath</code>	Filled after composition Get() if Ar found the file
<code>evaluatedPath</code>	Used with asset-path expressions; empty in simple cases

USDA uses `@...@` for asset paths and `</...>` for **prim** paths. Mixing them is an exam favorite.

`Ar.GetRegisteredURISchemes()` is **empty** with stock `usd-core` (filesystem only). `s3://` is not understood until a URI resolver plugin is loaded.

VERSION

Verified on USD 26.08. Older snippets call ``resolver.AnchorRelativePath``. That method is **not** on ``Ar.Resolver`` in 26.08. Use ``CreateIdentifier`` with an anchor, or ``Sdf.Layer.ComputeAbsolutePath``.

Mental model.

USDA	<code>@./tex/wood.png@</code>	authored asset path
	↓	
	<code>Ar.GetResolver().Resolve(...)</code>	(needs a correct identifier)
	↓	
	<code>Ar.ResolvedPath("/abs/.../wood.png")</code>	or <code>Ar.ResolvedPath()</code> (empty)

Simple example. asset `tex = @./tex/wood.png@` on `/Chair`. After `Usd.Stage.Open` of that layer, `attr.Get().resolvedPath` is the absolute file. `attr.Get().path` is still `./tex/wood.png`.

USDA example.

```
#usda 1.0
(
  defaultPrim = "Chair"
)

def Xform "Chair"
```

```
{
    asset tex = @./tex/wood.png@
}
```

- `@./tex/wood.png@` — asset path (file).
- `</Chair>` would be a prim path, not a file. Do not write `asset tex = </Chair>`.

Python example.

```
from pxr import Ar, Sdf, Usd
import os

os.makedirs("tex", exist_ok=True)
open("tex/wood.png", "wb").write(b"PNG")

stage = Usd.Stage.CreateNew("chair.usda")
prim = stage.DefinePrim("/Chair")
stage.SetDefaultPrim(prim)
attr = prim.CreateAttribute("tex", Sdf.ValueTypeNames.Asset)
attr.Set(Sdf.AssetPath("./tex/wood.png"))
stage.Save()

ap = attr.Get()
print("Get:", ap)
print("type:", type(ap).__name__)
print("path:", ap.path)
print("resolved nonempty:", bool(ap.resolvedPath))
print("resolver:", type(Ar.GetResolver()).__name__)
print("underlying:", type(Ar.GetUnderlyingResolver()).__name__)
print("URI schemes:", Ar.GetRegisteredURISchemes())
print("empty ResolvedPath bool:", bool(Ar.GetResolver().Resolve("nope.png")))
print("package?", Ar.IsPackageRelativePath("a.usdz[b.usda]"))
print("join:", Ar.JoinPackageRelativePath("a.usdz", "b.usda"))
```

Expected output

```
Get: @./tex/wood.png@
type: AssetPath
path: ./tex/wood.png
resolved nonempty: True
resolver: Resolver
underlying: DefaultResolver
URI schemes: []
empty ResolvedPath bool: False
package? True
join: a.usdz[b.usda]
```

`GetResolver()` returns the facade type `Resolver`. The plugin doing filesystem work is `DefaultResolver`.

Real-world use case. A shot references `@./sets/kitchen.usda@` and a shader points at `@./tex/tile.png@`. `usdview` opens because `Ar` resolved both. A farm machine without the `tex/` folder still opens the USD (the layer parses) but `resolvedPath` is empty and the render is black — an **asset management** bug (Obj 6.3), not a Mesh bug.

Common mistakes.

MISTAKE

Storing a texture as ``string`` instead of ``asset``. Strings do not resolve. Fix: ``Sdf.ValueTypeNames.Asset``.

MISTAKE

Writing ``asset tex = ``. Angle brackets are prim paths.

MISTAKE

Assuming ``s3://bucket/x.png`` works with ``usd-core`` out of the box. ``GetRegisteredURISchemes()`` is ``[]``. You need a URI resolver plugin (Section 33.5).

Exam traps.**TRAP**

"`Ar.GetResolver()` returns `DefaultResolver`. It returns `Resolver`. Ask `GetUnderlyingResolver()` for the plugin class.

TRAP

"`resolvedPath` is always filled on `Sdf.AssetPath('x.png')` you just constructed." Only composition `**Get()` (or the two-argument constructor) fills it.

TRAP

Calling `AnchorRelativePath` in 26.08. The method is gone from `Ar.Resolver`.

Exam takeaways.**KEY**

- `@...@` = asset path; `` = prim path; `Ar.ResolvedPath` = lookup result. - `bool(empty\_resolved)` is `False`; that is how you test "missing." - Stock `usd-core` has no URI schemes; filesystem `DefaultResolver` only. - Do not call `AnchorRelativePath` on 26.08.

33.2 Ar.GetResolver(), resolve, and anchoring

What is it? **Resolve** asks the resolver for a `ResolvedPath`. **Anchoring** means a relative asset path is interpreted against the **layer that authored it**, not against the process current working directory.

Why do we need it? `Ar.GetResolver().Resolve("./tex/wood.png")` from a script whose cwd is the show root returns empty, even though the texture sits next to `assets/chair.usda`. Composition still finds it, because Pcp **anchors first**. If you copy the naive `Resolve` into a validator, you fail good files (Obj 7.7 / 6.3).

Beginner explanation. A note in a cookbook says "see the sauce recipe in `./sauces/pesto.md`." `./` means "the folder this cookbook lives in," not "whatever kitchen you are standing in." Anchoring opens the cookbook's folder first, then walks `sauces/pesto.md`.

Where the analogy breaks: if the path has **no** `./` (`pesto.md` or `sauces/pesto.md`), the default resolver may instead walk a **search path** (Section 33.3). That is a different rule, not "also the cookbook folder" in Python `Resolve`.

Technical explanation. Verified on USD 26.08:

1. Naive Resolve uses cwd / search path, not the layer.

```
Ar.GetResolver().Resolve("./tex/wood.png")
```

This looks for `tex/wood.png` under the process cwd. If your script ran in `show/` and the layer lives in `show/assets/`, the result is empty.

2. Anchor, then resolve. Two equivalent tools:

API	What it does
<code>layer.ComputeAbsolutePath("./tex/wood.png")</code>	Join the path to the layer's directory; returns a string (even if the file is missing)
<code>resolver.CreateIdentifier("./tex/wood.png", Ar.ResolvedPath(layer.realPath))</code>	Build the identifier Ar expects, anchored at that layer

`ComputeAbsolutePath` does **not** check existence. `Resolve` on that absolute string does.

3. Composition already anchored. After `Usd.Stage.Open("assets/chair.usda")`, `attr.Get().resolvedPath` is filled when the file exists. Prefer that field in validators that already have a stage.

4. `CreateIdentifier("./tex/wood.png")` with no anchor strips `./` and yields `tex/wood.png`, which is then **context-dependent** (search path), not layer-relative. Always pass the anchor `ResolvedPath` when you mean “relative to this layer.”

5. `GetExtension(path)` returns the suffix without the dot (`usda`, `PNG`). Case follows the string you passed.

Mental model.

```
WRONG: Resolve(authored "./tex/wood.png")      <- cwd
RIGHT: ComputeAbsolutePath -> Resolve(abs)
RIGHT: CreateIdentifier(path, ResolvedPath(layer.realPath)) -> Resolve
RIGHT: stage Get() -> asset.resolvedPath
```

Simple example. Layer `assets/chair.usda` authors `@./tex/wood.png@`. File exists at `assets/tex/wood.png`. From `cwd` `show/:` naive `Resolve` is false; anchored `Resolve` is true; composed `Get().resolvedPath` is nonempty.

USDA example. File: `assets/chair.usda`

```
#usda 1.0

def Xform "Chair"
{
    asset tex = @./tex/wood.png@
}
```

The `./` is the signal: “anchor me to this file’s directory.”

Python example.

```
from pxr import Ar, Sdf, Usd
import os

os.makedirs("assets/tex", exist_ok=True)
open("assets/tex/wood.png", "wb").write(b"PNG")
open("assets/chair.usda", "w").write("""#usda 1.0
def Xform "Chair"
{
    asset tex = @./tex/wood.png@
}
""")

r = Ar.GetResolver()
print("naive Resolve ./tex/wood.png:",
      bool(r.Resolve("./tex/wood.png")))

layer = Sdf.Layer.FindOrOpen("assets/chair.usda")
abs_path = layer.ComputeAbsolutePath("./tex/wood.png")
print("ComputeAbsolutePath is file:", os.path.isfile(abs_path))
print("Resolve abs:", bool(r.Resolve(abs_path)))

ident = r.CreateIdentifier(
    "./tex/wood.png", Ar.ResolvedPath(layer.realPath))
print("CreateIdentifier then Resolve:", bool(r.Resolve(ident)))
print("GetExtension:", r.GetExtension(ident))

no_anchor = r.CreateIdentifier("./tex/wood.png")
print("CreateIdentifier no anchor:", no_anchor)

stage = Usd.Stage.Open("assets/chair.usda")
ap = stage.GetPrimAtPath("/Chair").GetAttribute("tex").Get()
print("composed resolved nonempty:", bool(ap.resolvedPath))
```

Expected output

```
naive Resolve ./tex/wood.png: False
ComputeAbsolutePath is file: True
```

```
Resolve abs: True
CreateIdentifier then Resolve: True
GetExtension: png
CreateIdentifier no anchor: tex/wood.png
composed resolved nonempty: True
```

The first line is the bug. The next lines are the fix. The last line is what a stage-based validator should read.

Real-world use case. A CI validator ran `Resolve(attr.Get().path)` from the repo root and failed every asset. Switching to `bool(attr.Get().resolvedPath)` (after `Stage.Open` of the entry layer) matched usdview: good files pass, missing textures fail.

Common mistakes.

MISTAKE

Validating with ``os.path.exists(ap.path)`` on ``./tex/wood.png``. That is cwd-relative in Python too. Fix: ``ComputeAbsolutePath`` or ``resolvedPath``.

MISTAKE

``CreateIdentifier("./x.png")`` without the anchor ``ResolvedPath``. You just converted an anchored path into a search-path path.

MISTAKE

Treating ``ComputeAbsolutePath`` as "the file exists." It always concatenates. Then ``Resolve`` or ``os.path.isfile``.

Exam traps.

TRAP

"Relative paths resolve from the process cwd, as in Python ``open``." For **composed** USD they resolve from the **layer**. For raw ``Ar.Resolve`` they do not — unless you anchored.

TRAP

"``resolvedPath`` on `Get()` is cwd-based." It is composition-based (anchored).

TRAP

``CreateIdentifier`` with a ``s3://`` URL on the default resolver. 26.08 collapsed ``s3://bucket/x`` to ``s3:/bucket/x`` in a probe. URI paths need a URI resolver, not the default filesystem identifier logic.

Exam takeaways.

KEY

- Raw ``Resolve("./...")`` is cwd-relative. Composition is layer-relative. - Anchor with ``CreateIdentifier(path, ResolvedPath(layer.realPath))`` or ``ComputeAbsolutePath``. - Prefer ``Get().resolvedPath`` when you already opened the stage. - ``CreateIdentifier`` without an anchor drops ``./`` and changes the meaning.

33.3 Resolver contexts and search paths

What is it? A **resolver context** is extra data the resolver uses for this lookup (or this stage). For `DefaultResolver`, that data is a **search path**: a list of directories to try for **context-dependent** asset paths. You bind it with `Ar.ResolverContextBinder` or pass it to `Usd.Stage.Open(..., pathResolverContext=)`.

Why do we need it? Studios want to write `@chair.usda@` (no `./`) and have Ar find the published chair via a configured list of asset roots — the same idea as Chapter 32’s pin, but looked up by **name** instead of a long relative path. Without a context, `@chair.usda@` only succeeds if that file sits in the cwd (or next to the layer, when composition anchors it as a file-relative identifier).

Beginner explanation. Search path is PATH for assets. `chair.usda` is like the command `python`: the shell walks PATH. `./chair.usda` is like `./python`: only the current folder (here: the layer’s folder), never PATH.

Where the analogy breaks: USD search paths do **not** override a file already in cwd for some lookups, and they never apply to paths that `IsContextDependentPath` says are false (`./`, `../`, absolute).

Technical explanation. Verified on USD 26.08:

Context-dependent or not (`resolver.IsContextDependentPath(s)`):

Path	Context-dependent?	Meaning
<code>chair.usda</code>	True	Walk search path (and cwd rules)
<code>tex/wood.png</code>	True	Same
<code>./chair.usda</code>	False	Anchored; not a search
<code>/abs/chair.usda</code>	False	Absolute filesystem

Build and bind a context:

```
ctx = Ar.DefaultResolverContext(["/show/assets"])
rc = Ar.ResolverContext(ctx)
with Ar.ResolverContextBinder(rc):
    Ar.GetResolver().Resolve("chair.usda")
# or:
Usd.Stage.Open("shot.usda", pathResolverContext=rc)
```

`DefaultResolverContext(["searchA"])` stores **absolute** directories (`GetSearchPath()`). Relative inputs are expanded.

`resolver.GetCurrentContext()` inside the binder is non-empty; after the `with` it is empty again. `Usd.Stage.Open(..., pathResolverContext=)` **stores** the context on the stage (`stage.GetPathResolverContext()`), so later resolves for that stage keep using it.

SetDefaultSearchPath on `Ar.DefaultResolver` is process-global. Prefer a per-stage context so tests and tools do not leak search paths.

A shot that writes `@chair.usda@` (no `./`) **fails** to open the reference without a search path if `chair.usda` is not next to the shot. The local def "C" still exists;

`GetCompositionErrors()` contains `ErrorInvalidAssetPath` (Obj 6.3). The same shot **succeeds** when opened with `DefaultResolverContext` pointing at the assets folder.

Mental model.

```
@./chair.usda@    -> layer folder only      (pin by relative path, Ch 32)
@chair.usda@      -> search path / context (pin by lookup)
@/mnt/show/...@   -> absolute; brittle
```

Simple example. `searchA/tex/wood.png` exists. `Unbound Resolve("tex/wood.png")` is empty. Bound to `DefaultResolverContext(["searchA"])`, `Resolve` finds `searchA/tex/wood.png`. `Resolve("./tex/wood.png")` stays empty — `./` is not a search.

USDA example. File: `shot.usda` — lookup-style reference, not a relative pin.

```
#usda 1.0

def Xform "W"
{
    def "C" (
        prepend references = @chair.usda@
```

```

    )
    {
    }
}

```

This is legal USDA. It is not a complete pipeline pin until a resolver context (or a `./` path) says **which** `chair.usda`.

Python example.

```

from pxr import Ar, Usd
import os

os.makedirs("searchA/tex", exist_ok=True)
open("searchA/tex/wood.png", "wb").write(b"A")
os.makedirs("assets", exist_ok=True)
open("assets/chair.usda", "w").write("""#usda 1.0
(
    defaultPrim = "Chair"
)
def Xform "Chair" {}
""")
open("shot.usda", "w").write("""#usda 1.0
def Xform "W"
{
    def "C" (
        prepend references = @chair.usda@
    )
    {
    }
}
""")

r = Ar.GetResolver()
print("unbound tex/wood.png:", bool(r.Resolve("tex/wood.png")))
print("dependent tex/wood.png:", r.IsContextDependentPath("tex/wood.png"))
print("dependent ./tex/wood.png:", r.IsContextDependentPath("./tex/wood.png"))

ctx = Ar.DefaultResolverContext([os.path.abspath("searchA")])
with Ar.ResolverContextBinder(Ar.ResolverContext(ctx)):
    rp = r.Resolve("tex/wood.png")
    print("bound tex/wood.png:", bool(rp), os.path.relpath(str(rp)))
    print("bound ./tex/wood.png:", bool(r.Resolve("./tex/wood.png")))

print("after unbind:", bool(r.Resolve("tex/wood.png")))

rc_assets = Ar.ResolverContext(
    Ar.DefaultResolverContext([os.path.abspath("assets")]))
ok = Usd.Stage.Open("shot.usda", pathResolverContext=rc_assets)
bad = Usd.Stage.Open("shot.usda")
print("with context errors:", len(ok.GetCompositionErrors()),
      "type:", ok.GetPrimAtPath("/W/C").GetTypeName())
print("no context errors:", len(bad.GetCompositionErrors()),
      "err:", type(bad.GetCompositionErrors()[0]).__name__)

```

Expected output

```

unbound tex/wood.png: False
dependent tex/wood.png: True
dependent ./tex/wood.png: False
bound tex/wood.png: True searchA/tex/wood.png
bound ./tex/wood.png: False
after unbind: False
with context errors: 0 type: Xform
no context errors: 1 err: ErrorInvalidAssetPath

```

Search paths find `tex/wood.png`. They do **not** find `./tex/wood.png`. The shot's `@chair.usda@` needs the assets directory in the **stage** context.

Real-world use case. A factory digital twin configures every `Usd.Stage.Open` with `DefaultResolverContext([os.environ["SHOW_ASSETS"]])`. Artists write `@robot.usda@`. IT moves the asset root between sites; shots do not change. Chapter 32's folder pins

(`@../.../assets/robot/v014/robot.usda@`) are the other school: no search path, explicit relative path. Both are valid; mixing them without documenting which one you use is how Obj 6.3 bugs start.

Common mistakes.

MISTAKE

Expecting `@./chair.usda@`` to use the search path. ``./`` disables it.

MISTAKE

Setting ``SetDefaultSearchPath`` in a library and forgetting to clear it. Later tests resolve the wrong chair. Fix: ``ResolverContextBinder`` or ``pathResolverContext``.

MISTAKE

Assuming a failed reference deletes the prim. The local ``def "C"`` remains; check ``GetCompositionErrors()``.

Exam traps.

TRAP

"Search path applies to every relative path." Only **context-dependent** paths.

TRAP

"Open with a context mutates the process resolver forever." The binder is scoped; the stage argument sticks to **that stage**.

TRAP

Printing ``GetLayerStack()[0]`` to debug a missing reference. The session layer is first (Ch 32). Look at composition errors and ``resolvedPath``.

Exam takeaways.

KEY

- Context-dependent = no ``./`` and not absolute; those paths use search paths. - ``DefaultResolverContext`` + ``ResolverContextBinder`` or ``pathResolverContext``. - Failed lookup: prim may still exist; errors are ``ErrorInvalidAssetPath``. - Document whether the pipeline uses ``./`` pins or search-path names (Obj 7.2 + 7.5).

33.4 Validating asset paths

What is it? **Validating an asset path** (Obj 7.7) means checking two things: the string is **formatted** so Ar can understand it, and it **resolves** to a real resource (or is reported as unresolved). Format without resolvability is a style check; resolvability without format still fails on the farm.

Why do we need it? A publish that ships `chair.usda` and forgets `tex/wood.png` is the bug Chapter 32 already inventoried with `ComputeAllDependencies`. Obj 7.7 adds the **per-path** rules: slashes, empty values, `./` vs search-path style, no Windows backslashes, no fake URI schemes.

Beginner explanation. A postal validator checks that the address has a street and a zip (format) and that the house exists (resolve). `@./tex/wood.png@` is the address. `resolvedPath` or `ComputeAllDependencies`'s `unresolved` list is "the house exists."

Where the analogy breaks: an empty `Sdf.AssetPath` is a blank envelope. It parses as a value but must fail validation.

Technical explanation. A practical checklist (document it next to Chapter 32's guidelines):

Check	Pass	Fail
Non-empty	<code>ap.path</code> is not <code>""</code>	<code>Sdf.AssetPath()</code>
Slashes	Forward <code>/</code> only	<code>tex\wood.png</code> (Resolve was <code>False</code> on 26.08 Linux)
Scheme	Empty or a registered URI scheme	<code>s3://...</code> with <code>GetRegisteredURISchemes() == []</code>
Anchored relative	Starts with <code>./</code> or <code>../</code> when it is a pin	Bare <code>chair.usda</code> if the pipeline forbids search-path names
Exists	<code>bool(ap.resolvedPath)</code> after <code>Open</code> , or <code>bool(Resolve(ComputeAbsolutePath(...)))</code> , or not in <code>ComputeAllDependencies unresolved</code>	Empty <code>ResolvedPath</code>

Do not call `Resolve(ap.path)` on a `./` string from a random cwd — Section 33.2.

Package paths: `a.usdz[b.usda]` is formatted correctly when `Ar.IsPackageRelativePath` is `true`. Existence then means the package opens and the inner member exists (`UsdUtils.ExtractUsdzPackage` or `Stage.Open` on the `.usdz`, Ch 31).

`ComputeAllDependencies` remains the recursive **ship list**. Per-attribute `resolvedPath` is the **local** check you can print next to a prim in a DCC.

Mental model.

```
format --> can Ar even parse this string?
resolve --> does the file (or package member) exist *from this layer*?
graph --> ComputeAllDependencies unresolved == []
```

All three belong in a publisher.

Simple example. `/Chair.tex` is `@./tex/wood.png@` and the file exists → pass. `/`

`Chair.missing` is `@./nope.png@` → format pass, resolve fail. `/Chair.win` is `@tex\wood.png@` → format fail.

USDA example.

```
#usda 1.0

def Xform "Chair"
{
    asset tex = @./tex/wood.png@
    asset missing = @./nope.png@
}
```

Both paths are well-formed USDA. Only `tex` should pass a resolvability check.

Python example.

```
from pxr import Ar, Sdf, Usd, UsdUtils
import os

os.makedirs("tex", exist_ok=True)
open("tex/wood.png", "wb").write(b"PNG")
open("chair.usda", "w").write("""#usda 1.0
def Xform "Chair"
{
    asset tex = @./tex/wood.png@
    asset missing = @./nope.png@
}
""")

stage = Usd.Stage.Open("chair.usda")
layer = stage.GetRootLayer()
r = Ar.GetResolver()
```

```

def format_ok(ap):
    p = ap.path
    if not p or "\\\" in p:
        return False
    if "://" in p:
        scheme = p.split(":", 1)[0]
        return scheme in Ar.GetRegisteredURISchemes()
    return True

def exists_for_layer(ap):
    if ap.resolvedPath:
        return True
    abs_path = layer.ComputeAbsolutePath(ap.path)
    return bool(r.Resolve(abs_path))

for name in ("tex", "missing"):
    ap = stage.GetPrimAtPath("/Chair").GetAttribute(name).Get()
    print(name, "format", format_ok(ap), "exists", exists_for_layer(ap))

print("backslash format", format_ok(Sdf.AssetPath("tex\\wood.png")))
print("empty format", format_ok(Sdf.AssetPath()))
print("URI format", format_ok(Sdf.AssetPath("s3://bucket/x.png")))

layers, assets, unresolved = UsdUtils.ComputeAllDependencies("chair.usda")
print("unresolved names:", sorted(os.path.basename(u) for u in unresolved))
print("assets names:", sorted(os.path.basename(a) for a in assets))

```

Expected output

```

tex format True exists True
missing format True exists False
backslash format False
empty format False
URI format False
unresolved names: ['nope.png']
assets names: ['wood.png']

```

`s3://` fails format here because no URI scheme is registered — which is the honest `usd-core` result.

Real-world use case. A DCC export hook (Ch 29, Ch 34) runs this pair of checks on every `asset` attribute plus `ComputeAllDependencies` on the root layer. The export button stays red until `unresolved` is empty. Lighting never discovers a missing UDIM on the farm.

Common mistakes.

MISTAKE

Checking only ``format_ok`` (syntax). ``@./nope.png@`` is pretty and still broken.

MISTAKE

Checking only ``os.path.isfile`` on authored strings. You skipped `Ar` (packages, URI plugins, search paths).

MISTAKE

Treating a URI as valid because it contains ``://``. Validity is **registered scheme** + that plugin's resolve.

Exam traps.

TRAP

"Obj 7.7 is ``ComputeAllDependencies``." That API inventories. 7.7 is **format + resolve** of the path strings themselves. Use both.

TRAP

"Backslashes are fine if Python ``open`` on Windows accepts them." USD asset paths use ``/``.

TRAP

Empty ``resolvedPath`` after ``Sdf.AssetPath("./x.png")`` ****construction**** means the file is missing. No — you never composed. Open the stage first.

Exam takeaways.**KEY**

- Obj 7.7 = format (non-empty, ``/``, registered scheme) + resolve (anchored). - Never ``Resolve("./...")`` from a random cwd. - Pair per-attribute checks with ``ComputeAllDependencies`` unresolved. - URI schemes are invalid until a plugin registers them.

33.5 Custom resolvers (overview)

What is it? A **custom resolver** is an Ar plugin that replaces (or sits beside) `DefaultResolver` so asset paths can mean something other than “a file on this disk”: a URI (`s3://`), a database id (`chair?v=17`), or a studio alias (`chair` → today’s published folder). Obj 3.5 and 7.5: **integrate** one — load it, select it, pass a context into `Stage.Open`.

Why do we need it? Chapter 32’s relative pins work until you have ten sites, two clouds, and identifiers that should not encode folder layout. The default resolver cannot talk to S3 or a production database. A plugin can.

Beginner explanation. The default librarian only walks the building’s shelves (filesystem). A custom librarian also calls the warehouse and understands catalog numbers. You still hand them the same card (`@chair@`). You **install** the new librarian (plugin path) and **introduce** them (`SetPreferredResolver` / URI scheme) so every department uses the same one.

Where the analogy breaks: you rarely write that librarian in Python with `usd-core`. Production resolvers are **C++** `ArResolver` subclasses loaded from `plugInfo.json`. This section is the integration map, not a compiler tutorial (`usd-core` has no `usdGenSchema` / plugin build — Ch 35, Ch 36).

Technical explanation. What you install (conceptual):

1. A shared library implementing `ArResolver` (or a URI-scheme resolver).
2. A `plugInfo.json` that registers the type with base `ArResolver` (plugin system: Chapter 36).
3. The directory of that `plugInfo.json` on `PXR_PLUGINPATH_NAME`.

Illustrative `plugInfo.json` (not loaded by this book’s `usd-core`; do not expect it to change `GetUnderlyingResolver()`):

```
{
  "Plugins": [
    {
      "Type": "resource",
      "Name": "StudioResolver",
      "Info": {
        "Types": {
          "StudioResolver": {
            "bases": ["ArResolver"]
          }
        }
      }
    }
  ]
}
```

What you call from Python once the plugin exists:

Step	API
See filesystem plugin today	<code>type(Ar.GetUnderlyingResolver()).__name__</code> → <code>DefaultResolver</code>
URI schemes the process knows	<code>Ar.GetRegisteredURISchemes()</code>
Prefer a named resolver	<code>Ar.SetPreferredResolver("StudioResolver")</code>
Per-stage search / studio context	<code>Usd.Stage.Open(path, pathResolverContext=rc)</code>
Bind for a block of Resolve calls	<code>with Ar.ResolverContextBinder(rc):</code>

`SetPreferredResolver("NoSuchResolver")` on 26.08 **did not switch** the underlying type; it stayed `DefaultResolver`. Integrating a resolver means the plugin must actually load. Typo names fail closed.

URI vs search-path custom resolvers:

Style	Authored path	Typical plugin
Search / identifier	<code>@chair@</code> or <code>@chair.usda@</code>	Replaces default lookup
URI	<code>@s3://bucket/chair.usda@</code>	Registers scheme <code>s3</code>

Obj 7.5 in a pipeline document: “We load `StudioResolver` via `PXR_PLUGINPATH_NAME`; shots Open with a context that names the show; artists write `@identifier@`, never `/mnt/...`”

Custom resolvers are completed as a plugin topic in Chapter 38. Here you only need the Ar-facing contract: **same** `Resolve` / `CreateIdentifier` / `context` APIs, different backing store.

Mental model.

usd-core today:	<code>GetUnderlyingResolver() == DefaultResolver</code> <code>GetRegisteredURISchemes() == []</code>
studio install:	<code>PXR_PLUGINPATH_NAME -> plugInfo.json -> StudioResolver</code> <code>Stage.Open(..., pathResolverContext=studioContext)</code>
Python still calls	<code>Ar.GetResolver().Resolve(...)</code> -- the facade does not change

Simple example. IT ships `StudioResolver.so` + `plugInfo.json`. A shot opener always passes a context that includes `show=film24`. Artists author `@chair@`. The plugin returns `/prod/film24/assets/chair/v022/chair.usda`. Validators still check `bool(resolvedPath)` (Section 33.4); they do not parse the identifier.

USDA example. The authored layer does not mention the plugin. That is the point: USDA stays `@chair@` or `@s3://...@`.

<pre>#usda 1.0 def Xform "W" { def "C" (prepend references = @chair@) { } }</pre>
--

Without the studio resolver and context, this is an `ErrorInvalidAssetPath`. With them, it is a pin.

Python example. This block shows the **stock** process — what you measure before a plugin is on the path — plus the integration calls that exist even when the plugin does not.

```
from pxr import Ar

print("underlying:", type(Ar.GetUnderlyingResolver().__name__))
print("schemes:", Ar.GetRegisteredURISchemes())
print("has SetPreferredResolver:", hasattr(Ar, "SetPreferredResolver"))
print("has DefaultResolverContext:", hasattr(Ar, "DefaultResolverContext"))
print("has Binder:", hasattr(Ar, "ResolverContextBinder"))

Ar.SetPreferredResolver("NoSuchResolver")
print("after missing name:", type(Ar.GetUnderlyingResolver().__name__))
```

Expected output

```
underlying: DefaultResolver
schemes: []
has SetPreferredResolver: True
has DefaultResolverContext: True
has Binder: True
after missing name: DefaultResolver
```

The APIs exist. A fake type name does not install a resolver. That is the integration lesson: **plugin load first**, then prefer, then Open with a context.

Real-world use case. A cloud renderer has no `/mnt/show`. The studio resolver maps `@chair@` to an object-store URL, `OpenAsset` streams bytes, and `CreateIdentifierForNewAsset` writes publishes back. Farm and artist's laptop run the **same** plugin so identifiers match (Obj 6.3: "it worked on my machine" is an Ar context mismatch until then).

Common mistakes.

MISTAKE

Subclassing `Ar.Resolver`` in a notebook and expecting USD to use it. The process loads C++ plugins from `pluginInfo.json``.

MISTAKE

Setting `PXR_PLUGINPATH_NAME`` after the first `Ar.GetResolver()`` call and assuming types refresh. Load plugins before opening stages (Ch 36).

MISTAKE

Shipping shots that still contain `/mnt/show/...`` after adopting a resolver. Old absolute pins bypass the plugin.

Exam traps.

TRAP

"Custom resolvers are written in USDA." They are plugins. USDA only stores the ****identifiers**** the plugin understands.

TRAP

"`SetPreferredResolver`` always raises on a bad name." On 26.08 it left `DefaultResolver`` in place.

TRAP

Obj 7.5 is satisfied by calling `Resolve``. Integrating means plugin + context + documented identifier style.

Exam takeaways.

KEY

- Custom resolvers are **plugins** (``ArResolver`` + ``plugInfo.json`` + ``PXR_PLUGINPATH_NAME``). - Python keeps calling ``Ar.GetResolver()``; the underlying type changes after a real load. - Integrate = load plugin, prefer it, Open with a context, document identifier style. - ``usd-core`` here: ``DefaultResolver``, no URI schemes — know what "before" looks like.

Chapter lab(s)

Lab 29 (asset resolution) is this chapter's lab: anchor vs naive Resolve, a search-path Stage.Open, and a format/exists validator. Lab 28's `ComputeAllDependencies` list is the recursive half of Obj 7.7.

USDA reading exercises

Exercise 33-A. A shader in `assets/lamp/lamp.usda` has asset `inputs:file = @./tex/metal.png@`. A validator in the show root runs `Ar.GetResolver().Resolve("./tex/metal.png")` and fails. `usdview` shows the texture. Why, and what should the validator call?

Exercise 33-B. `shot.usda` contains `prepend references = @chair.usda@` (no `./`). Opening the shot with no context yields `ErrorInvalidAssetPath`. Opening with `DefaultResolverContext([os.path.abspath("assets")])` succeeds. What kind of path is `@chair.usda@`, and what would `@./chair.usda@` have done instead?

Answers

- **33-A.** Naive Resolve is cwd-relative. The file lives next to the **layer**. Use `Get().resolvedPath` after opening that layer, or `Resolve(layer.ComputeAbsolutePath("./tex/metal.png"))`.
- **33-B.** `@chair.usda@` is **context-dependent** (search path). `@./chair.usda@` is anchored to the shot folder and would **not** use `assets/` even with that context; it would look for `shot-folder/chair.usda`.

Chapter review**Summary**

- Ar turns authored `@...@` into `ResolvedPath`; empty is missing (bool is False).
- Stock `usd-core`: `GetUnderlyingResolver()` is `DefaultResolver`; URI schemes `[]`.
- Anchor `./` to the layer (`CreateIdentifier` + `ResolvedPath(layer.realPath)`, or `ComputeAbsolutePath`).
- Context-dependent paths (`chair.usda`) use `DefaultResolverContext` / `pathResolverContext`.
- Obj 7.7: format (`/`, non-empty, registered scheme) and anchored resolve.
- Obj 3.5 / 7.5: load a plugin, prefer it, Open with a context — not a Python subclass in a notebook.
- Obj 6.3: missing refs are `ErrorInvalidAssetPath`; the local prim may still exist.

If you see... → think...

If you see...	Think...
<code>ErrorInvalidAssetPath</code>	Ar did not find the identifier (cwd vs layer vs search path)
<code>Resolve("./x")</code> False, <code>usdview</code> OK	Missing anchor
<code>@chair.usda@</code> vs <code>@./chair.usda@</code>	Search path vs layer pin
<code>s3://</code> on <code>usd-core</code>	No URI plugin

If you see...	Think...
<code>AnchorRelativePath</code>	Removed from <code>Ar.Resolver</code> in 26.08
<code>SetPreferredResolver</code> no effect	Plugin did not load
<code>resolvedPath</code> empty on a fresh <code>AssetPath()</code>	You never composed

Review questions

R33-1 (Obj 7.7) Best first test that a composed texture attribute found a file? A. `bool(attr.Get().resolvedPath)` B. `os.path.exists(attr.Get().path)` from the repo root C. `attr.GetTypeName() == "string"` D. `Ar.GetRegisteredURISchemes()`

R33-2 (Obj 6.3) `Stage.Open("shot.usda")` leaves `/W/C` but `GetCompositionErrors()` lists `ErrorInvalidAssetPath` for `@chair.usda@`. What happened? A. The spec def "C" exists locally; the reference did not resolve B. The prim was deleted C. `chair.usda` is USDC so `Ar` ignores it D. The session layer swallowed the reference

R33-3 (Obj 7.5) Select two identifiers that use a search-path context on `DefaultResolver`. A. `chair.usda` B. `./chair.usda` C. `tex/wood.png` D. `/abs/chair.usda`

R33-4 (Obj 7.7) Why is `tex\\wood.png` a format failure in this book's validator? A. USDA cannot store asset attributes B. Asset paths should use forward slashes; `Resolve` was `False` with backslashes C. Backslashes are reserved for package paths D. `DefaultResolver` treats `\\` as a URI scheme

R33-5 (Obj 3.5) Custom resolvers are shipped as: A. A class prim in the shot B. An `Ar` plugin (`plugInfo.json` + library) on `PXR_PLUGINPATH_NAME` C. A variant named `resolver` D. A USDA `#resolver` directive

R33-6 (Obj 7.7) `CreateIdentifier("./tex/wood.png")` with **no** anchor returns: A. `./tex/wood.png` unchanged B. `tex/wood.png` (now context-dependent) C. An `Ar.ResolvedPath` to the texture D. A URI scheme `tex`

R33-7 (Obj 7.5) `Usd.Stage.Open(path, pathResolverContext=rc)` is used to: A. Mute sublayers B. Give **that stage** a search path / studio context C. Convert USDA to USDC D. Register a URI scheme named `rc`

R33-8 (Obj 6.3) Select two correct debugging moves for a missing texture. A. Print `attr.Get().path` and `attr.Get().resolvedPath` B. Compare naive `Resolve(path)` from `cwd` with `ComputeAbsolutePath` C. Set `kind = component` on the shader D. Flatten the stage so `Ar` is no longer needed

R33-9 (Obj 7.7) `Ar.GetRegisteredURISchemes()` is `[]`. `@s3://bucket/a.png@` should: A. Pass format because it has `://` B. Fail format / resolve until a URI resolver plugin loads C. Be rewritten automatically to a filesystem path D. Be treated as `./s3://bucket/a.png`

R33-10 (Obj 3.5) `SetPreferredResolver("NoSuchResolver")` on 26.08 `usd-core`: A. Raises `Tf.ErrorException` B. Leaves `DefaultResolver` as the underlying resolver C. Registers URI scheme `nosuch` D. Deletes `plugInfo.json`

R33-11 (Obj 7.7) `layer.ComputeAbsolutePath("./nope.png")` when the file is missing: A. Raises B. Returns an absolute string anyway; `Resolve` of that string is empty C. Returns `Ar.ResolvedPath()` D. Creates the file

R33-12 (Obj 7.5) Select two statements that belong in a pipeline guideline after adopting a studio resolver. A. Artists author identifiers the plugin understands, not `/mnt/show/...` B. Every `Stage.Open` in tools passes the studio `pathResolverContext` C. Asset attributes must change to string D. `AnchorRelativePath` is required in every validator

Review answers

- **R33-1: A.** Composed `resolvedPath` is already anchored. Review: §33.2, §33.4.

- **R33-2: A.** Local def survives; the arc failed. Review: §33.3.
- **R33-3: A and C.** `./` and absolute are not context-dependent. Review: §33.3.
- **R33-4: B.** Forward slashes. Review: §33.4.
- **R33-5: B.** Plugin integration. Review: §33.5.
- **R33-6: B.** Strips `./`. Review: §33.2.
- **R33-7: B.** Per-stage context. Review: §33.3.
- **R33-8: A and B.** Kind and flatten do not fix lookup. Review: §33.2, §33.4.
- **R33-9: B.** No schemes registered. Review: §33.1, §33.5.
- **R33-10: B.** Fail closed. Review: §33.5.
- **R33-11: B.** Join $\neq$ exist. Review: §33.2.
- **R33-12: A and B.** Keep `asset` type; `AnchorRelativePath` is gone. Review: §33.5.

CHAPTER 34

Chapter 34 — Exporter Hooks, Flattening, and Change Processing

Exam domain: Pipeline Development (14%), also Composition (23%) and Debugging (11%) · **Objectives:** 1.9, 6.1 · **Study day:** 10 · **Est. time:** 95 min **Prerequisites:** Ch 8 (Usd vs Sdf, ChangeBlock intro), Ch 22 (delivery: flatten / localize), Ch 29 (exporter hooks first look), Ch 32–33 (publish, Ar)

Chapter 29 called a **hook** after writing prims. Chapter 22 showed **Flatten**, **FlattenLayerStack**, and **LocalizeAsset** for delivery (Obj 1.9). This chapter is the pipeline version: hooks that *fix* structure, the flatten **decision**, stripping proprietary dependencies, **notices** so a DCC can stay live, and `Sdf.ChangeBlock` as the Obj 6.1 bottleneck tool with proof.

Learning goals

- Write exporter hooks that enforce `defaultPrim`, `kind`, and other pipeline rules (and *apply* the fix).
- Choose `Usd.Stage.Flatten`, `UsdUtils.FlattenLayerStack`, or a plain layer `Export` for a delivery.
- Remove studio-only metadata and rewrite leaked absolute asset paths (`ModifyAssetPaths`).
- Register `Usd.Notice.ObjectsChanged` and revoke the listener.
- Use `Sdf.ChangeBlock` so many Sdf edits fire **one** notice (Obj 6.1).

Key terms

Term	One-line definition
Exporter hook	A function the exporter runs after write, before Save, to enforce structure
Flatten	Bake the <i>composed stage</i> into one layer; arcs and unselected variants go away
FlattenLayerStack	Merge only the root sublayers into one layer; references stay
Export (one layer)	Write that layer's specs only; no composition
Proprietary dependency	A path, plugin, or <code>customData</code> key an outside party must not receive
Notice	A callback USD sends when the stage or a layer changes (<code>Tf.Notice</code>)
Resync	A notice that the prim index at a path must be rebuilt
Change block	Batches edits so listeners run once when the block ends

34.1 Exporter hooks to enforce pipeline structure

What is it? An **exporter hook** is a function you call at a fixed point in export — after prims exist, before `Save()` — that **checks and repairs** pipeline structure: `defaultPrim`, `kind`, relative asset paths, extents (Ch 13), naming (Ch 28). Chapter 29 introduced the idea. Here the hook is a **list of structural guarantees** the publisher will not skip.

Why do we need it? Artists and DCC exporters forget metadata. A chair without `defaultPrim` cannot be referenced by path-less `@chair.usda@` (Ch 16). A publish without `kind` disappears from model traversal (Ch 23). Hooks make the rules one function every exporter calls (Obj 1.9 starts with a well-formed internal asset).

Beginner explanation. A hotel checkout checklist: key returned, minibar billed, door locked. The guest already packed. The checklist does not pack the suitcase; it refuses to let you leave until the room matches the house rules.

Where the analogy breaks: a hook may **repair** (set defaultPrim when there is exactly one root) instead of only refusing. Document which checks auto-fix and which fail the publish.

Technical explanation. A pipeline hook has a stable signature, for example hook(stage) -> list[str] of fixes applied (and a separate validator from Chapter 30 that reports remaining errors).

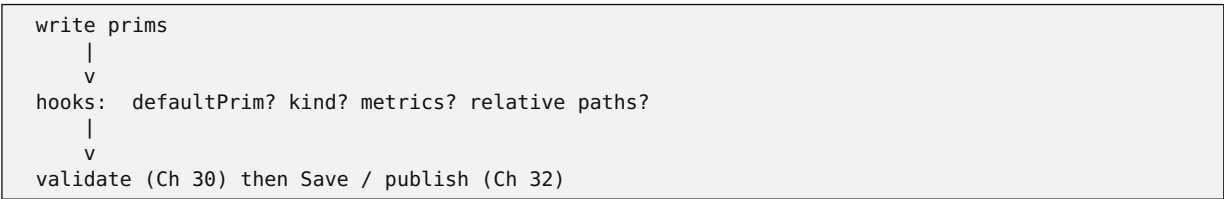
Typical structural fixes:

Rule	Repair if safe	Else fail
defaultPrim	One root prim → SetDefaultPrim	Zero or many roots
kind on the default prim	Missing → component or studio default	Already wrong
Stage metrics	Missing upAxis / metersPerUnit → studio defaults	Mismatch with referenced assets (Ch 28)
Asset paths	Rewrite /mnt/show/... to ./... if the file is inside the asset	Path outside the tree

Hooks run on the **edit target / root layer** you are about to publish, not on the session layer.

Call order (repeat of Ch 29, now pipeline-complete): extract → transform → write → **hooks** → validate → Save. Notices (Section 34.4) are for *live* DCC sync; batch converters just call the list.

Mental model.



Simple example. DCC export creates /Chair and nothing else, with no metadata. The hook sets defaultPrim = Chair and kind = component. Save then produces a referenceable entry layer.

USDA example. After the hook, the published layer’s metadata block looks like this:

```
#usda 1.0
(
  defaultPrim = "Chair"
  metersPerUnit = 1
  upAxis = "Y"
)

def Xform "Chair" (
  kind = "component"
)
{
}
```

That block is what other shots pin (Ch 32). The hook’s job is to make sure it exists even when the DCC GUI never showed a “defaultPrim” field.

Python example.

```
from pxr import Usd, UsdGeom

def pipeline_hooks(stage):
```



```

fixes = []
if not stage.HasDefaultPrim():
    roots = stage.GetPseudoRoot().GetChildren()
    if len(roots) == 1:
        stage.SetDefaultPrim(roots[0])
        fixes.append("defaultPrim")
prim = stage.GetDefaultPrim()
if prim and not Usd.ModelAPI(prim).GetKind():
    Usd.ModelAPI(prim).SetKind("component")
    fixes.append("kind")
return fixes

stage = Usd.Stage.CreateInMemory()
UsdGeom.Xform.Define(stage, "/Chair")
print("before defaultPrim", stage.HasDefaultPrim())
print("before kind", bool(Usd.ModelAPI(
    stage.GetPrimAtPath("/Chair")).GetKind()))
print("fixes", pipeline_hooks(stage))
print("after defaultPrim", stage.GetDefaultPrim().GetName())
print("after kind", Usd.ModelAPI(stage.GetDefaultPrim()).GetKind())
print("fallback meters", stage.GetMetadata("metersPerUnit"))
print("fallback upAxis", UsdGeom.GetStageUpAxis(stage))

```

Expected output

```

before defaultPrim False
before kind False
fixes ['defaultPrim', 'kind']
after defaultPrim Chair
after kind component
fallback meters 0.01
fallback upAxis Y

```

A fresh stage already has **fallback** `metersPerUnit = 0.01` and `upAxis = Y` (Ch 2, Ch 28). `GetMetadata` is therefore not `None`. A metrics hook must check **authored** metadata (`HasAuthoredMetadata`) before overwriting studio units. After this hook, the entry is at least pin-ready (`defaultPrim + kind`).

Real-world use case. Four DCCs (CAD, Maya, Blender, a custom tool) each `import studio_export_hooks` and run `pipeline_hooks(stage)` before `Save`. When the studio adds “identifier in `assetInfo`,” one function changes; every exporter inherits it.

Common mistakes.

MISTAKE

Running hooks on the session layer. Nothing is saved. Fix: `edit target` = the layer you will publish.

MISTAKE

Auto-setting `kind = component` on an assembly root. The hook must know the asset class. Fail instead of guessing when the type is ambiguous.

MISTAKE

Skipping validation after a hook “because we just fixed it.” Hooks can miss unresolved textures (Ch 33).

Exam traps.

TRAP

“Hooks are `Tf.Notice`` listeners.” They can be ordinary functions. Notices are Section 34.4, for live updates.

TRAP

“`HasDefaultPrim()` is `True` on a fresh `CreateInMemory`` stage.” It is `False` until you set one.

TRAP

Setting ``defaultPrim`` when two roots exist. A hook that picks the first prim silently is a pipeline bug.

Exam takeaways.**KEY**

- Hooks enforce pipeline structure after write, before Save. - Safe repairs: one-root ``defaultPrim``, known ``kind``, studio metrics. - Same hook list for every exporter; validate afterwards. - Hooks are functions; notices are a different tool.

34.2 Flatten VS FlattenLayerStack vs layer export

What is it? Three ways to “write one file” that are **not** interchangeable. **Export** writes one layer’s specs. **FlattenLayerStack** merges the root layer’s **sublayers**. **Flatten** bakes the composed **stage** (Obj 1.9).

Why do we need it? Sending `entry.usda` to a client who does not have your sublayers or your `ref.usda` gives them an incomplete scene. Sending a full Flatten may drop variants they still need. Picking the wrong tool is the usual delivery bug.

Beginner explanation. Export is photocopying **one** notebook page. FlattenLayerStack is taping the department pages of *this* binder into one sheet, while keeping “see also book X” footnotes (references). Flatten is rewriting the whole story as if there were never footnotes.

Where the analogy breaks: Flatten also turns `@./tex/t.png@` into an **absolute** path on your disk (Ch 22). You must rewrite paths before the client opens it.

Technical explanation. Verified on USD 26.08:

Tool	Input	Arcs	Sublayers	Asset paths
<code>root.Export / ExportToString</code>	One layer	Unchanged	Listed, not merged	Authored
<code>UsdUtils.FlattenLayerStack(stage)</code>	Root layer stack	References / payloads kept	Merged away (subLayerPaths empty)	Often absolutized
<code>stage.Flatten()</code>	Whole composed stage	Baked away	Baked away	Often absolutized

Facts:

- Flatten of a referenced `Cube` child produces a local `Cube` spec with **no** references = (`typeName` is `Cube`).
- FlattenLayerStack of the same child **keeps** prepend references. The spec’s own `typeName` may be **empty**; the `Cube` type still comes from the referenced file.
- Neither tool strips `customData` (Section 34.3).
- `usdcats --flatten` is the CLI for Flatten; not installed with `usd-core`.

VERSION

Same APIs on 26.08 as in Chapter 22. This chapter is the **choice** table; Chapter 22 has the `localize/USDZ` package path.

Mental model.

```

need the composed picture, no arcs      -> Flatten
need one file of THIS stack, keep refs -> FlattenLayerStack
need exactly what this layer authored  -> Export

```

Simple example. `entry.usda` sublayers `geo.usda` (a texture) and defines `/L/Bolt` as a reference to `ref.usda`. Export of `entry` has no texture attribute. `FlattenLayerStack` has the texture and still references `Bolt`. `Flatten` has the texture and a local Cube named `Bolt`.

USDA example. *File: `entry.usda`*

```

#usda 1.0
(
    defaultPrim = "L"
    subLayers = [
        @./geo.usda@
    ]
)

def Xform "L"
{
    def "Bolt" (
        prepend references = @./ref.usda@
    )
    {
    }
}

```

The texture lives in `geo.usda` as `over "L" { asset tex = @./tex/t.png@ }`. `Flatten` tools pull it up; Export of `entry.usda` does not.

Python example.

```

from pxr import Usd, UsdUtils
import os

os.makedirs("tex", exist_ok=True)
open("tex/t.png", "wb").write(b"x")
open("geo.usda", "w").write("""#usda 1.0
over "L"
{
    asset tex = @./tex/t.png@
}
""")
open("ref.usda", "w").write("""#usda 1.0
(
    defaultPrim = "Bolt"
)
def Cube "Bolt" {}
""")
open("entry.usda", "w").write("""#usda 1.0
(
    defaultPrim = "L"
    subLayers = [@./geo.usda@]
)
def Xform "L"
{
    def "Bolt" (prepend references = @./ref.usda@) {}
}
""")

stage = Usd.Stage.Open("entry.usda")
root = stage.GetRootLayer()
print("Export has tex:", root.GetAttributeAtPath("/L.tex") is not None)
print("Export has subLayers:", bool(list(root.subLayerPaths)))

flat = stage.Flatten()
bolt = flat.GetPrimAtPath("/L/Bolt")
print("Flatten Bolt type:", bolt.typeName)
print("Flatten Bolt has ref:",
      bool(bolt.referenceList.prependedItems))
print("Flatten tex abs:",
      os.path.isabs(flat.GetAttributeAtPath("/L.tex").default.path))

```

```
ls = UsdUtils.FlattenLayerStack(stage)
bolt2 = ls.GetPrimAtPath("/L/Bolt")
print("FLS Bolt type:", bolt2.typeName or "-")
print("FLS Bolt has ref:",
      bool(bolt2.referenceList.prependedItems))
print("FLS subLayers empty:", list(ls.subLayerPaths) == [])
print("FLS tex abs:",
      os.path.isabs(ls.GetAttributeAtPath("/L.tex").default.path))
```

Expected output

```
Export has tex: False
Export has subLayers: True
Flatten Bolt type: Cube
Flatten Bolt has ref: False
Flatten tex abs: True
FLS Bolt type: -
FLS Bolt has ref: True
FLS subLayers empty: True
FLS tex abs: True
```

Export missed the texture (it lives on a sublayer). Flatten baked the Cube. FlattenLayerStack kept the reference and still leaked an absolute texture path.

Real-world use case. A client can open USD but cannot reach your asset resolver. You FlattenLayerStack the entry (one file of your workstreams), then ModifyAssetPaths (next section), then LocalizeAsset so ref.usda and t.png travel in the folder. You do **not** Flatten if they still need to swap the Bolt asset later.

Common mistakes.

MISTAKE

Emailing `entry.usda` Export and wondering why the texture is gone. Fix: FlattenLayerStack or send the sublayers.

MISTAKE

Flattening to "simplify" a shot that must keep variants. Unselected variants disappear.

MISTAKE

Shipping Flatten output with `/home/you/.../tex/t.png`. Rewrite paths.

Exam traps.

TRAP

"`FlattenLayerStack` removes references." It removes ****sublayers****. References stay.

TRAP

"`Flatten` and `Export` are the same if the stage has one layer." Export still will not compose references ***into*** that layer.

TRAP

"Absolute paths in Flatten mean the client can find the files." They point at ***your*** disk.

Exam takeaways.

KEY

- Export = one layer. FlattenLayerStack = merge sublayers, keep refs. Flatten = bake the stage. - Both flatten flavors absolutize asset paths. - Obj 1.9: pick the tool from what the ***receiver*** can open and override.

34.3 Removing proprietary dependencies

What is it? **Proprietary dependencies** are files, URI schemes, plugins, and metadata an outside party should not receive: `/studio/rigs/...` references, `customData` owner names, internal asset attributes, resolver identifiers they cannot load. Removing them is the rest of Obj 1.9 after you pick a flatten tool.

Why do we need it? Flatten and FlattenLayerStack **keep** `customData` and they **leak absolute paths**. A client with your `owner = "jdoe"` field and `@/mnt/show/tex/t.png@` has a broken, confidential package.

Beginner explanation. Before you send a cake recipe, you white-out “Mom’s secret shelf in *our* pantry” and write “1 tsp vanilla” instead. You also take your name off the card.

Where the analogy breaks: USD will not white-out for you. `LocalizeAsset`’s `processingFunc` (Ch 22) can drop arcs; `ModifyAssetPaths` rewrites strings; `ClearCustomDataByKey` is manual.

Technical explanation. Three complementary cleanups (26.08):

Job	API
Rewrite every asset path on a layer	<code>UsdUtils.ModifyAssetPaths(layer, fn)</code> — <code>fn</code> takes the authored path, returns the new string
Drop internal refs while copying a tree	<code>UsdUtils.LocalizeAsset(..., processingFunc)</code> returning empty <code>UsdUtils.DependencyInfo()</code> (Ch 22)
Strip private metadata	<code>prim.ClearCustomDataByKey("owner")</code> , <code>prim.ClearAssetInfo()</code> , <code>RemoveProperty</code>

`ModifyAssetPaths` is the smallest tool for the flatten leak: after FlattenLayerStack, map absolute texture paths back to `./tex/t.png` if you will ship that relative file beside the layer.

Do not leave `@s3://...@` in a package for a client with no URI resolver (Ch 33).

Mental model.

```
flatten or localize
|
+-- ModifyAssetPaths   (no /mnt, no s3 unless they have the plugin)
+-- drop internal arcs (processingFunc / ClearReferences)
+-- clear customData / assetInfo secrets
+-- reopen and ComputeAllDependencies (unresolved == [])
```

Simple example. FlattenLayerStack wrote `@/tmp/.../tex/t.png@`.

`ModifyAssetPaths(layer, lambda p: "./tex/t.png")` makes the attribute portable.

`ClearCustomDataByKey("owner")` removes the artist name.

USDA example. *Before cleanup* (what FlattenLayerStack emitted in spirit):

```
#usda 1.0
(
  defaultPrim = "L"
)

def Xform "L" (
  customData = {
    string owner = "jdoe"
  }
)
{
  asset tex = @/tmp/show/tex/t.png@
}
```

Both the `owner` key and the `/tmp/...` path must not ship.

Python example.

```

from pxr import Sdf, Usd, UsdUtils

layer = Sdf.Layer.CreateAnonymous(".usda")
layer.ImportFromString("""#usda 1.0
(
  defaultPrim = "L"
)
def Xform "L" (
  customData = { string owner = "jdoe" }
)
{
  asset tex = @mnt/show/tex/t.png@
}
""")

print("before tex:", layer.GetAttributeAtPath("/L.tex").default)
print("before owner:", dict(layer.GetPrimAtPath("/L").customData))

UsdUtils.ModifyAssetPaths(layer, lambda p: "./tex/t.png")
stage = Usd.Stage.Open(layer)
stage.GetPrimAtPath("/L").ClearCustomDataByKey("owner")
print("after tex:", layer.GetAttributeAtPath("/L.tex").default)
print("after owner:", dict(layer.GetPrimAtPath("/L").customData))

```

Expected output

```

before tex: @mnt/show/tex/t.png@
before owner: {'owner': 'jdoe'}
after tex: @./tex/t.png@
after owner: {}

```

`ClearCustomDataByKey` is a **Usd.Prim** method (Ch 22). `Sdf.PrimSpec` has `customData` you can assign; it does not have `ClearCustomDataKey`.

Real-world use case. A vendor delivery: `FlattenLayerStack` the internal entry, `processingFunc` drops `@/studio/...@`, `ModifyAssetPaths` forces `./`, `customData` wiped, then `CreateNewUsdzPackage` (Ch 31). The vendor opens the USDZ on a laptop with no studio resolver.

Common mistakes.

MISTAKE

Flattening and calling it "sanitized." Secrets in ``customData`` survived.

MISTAKE

``ModifyAssetPaths`` that returns `````` for every path. You just deleted every texture link. Return a real relative path or handle that file in ``processingFunc``.

MISTAKE

Cleaning the in-memory Flatten layer and forgetting to ``Export`` it. The original ``entry.usda`` is unchanged.

Exam traps.

TRAP

```LocalizeAsset`` strips ``customData``. It copies files; metadata stays (Ch 22).

#### TRAP

"Absolute paths are required in Flatten output." They are a side effect you are expected to fix.

### Exam takeaways.

**KEY**

- Flatten is not a confidentiality tool. - `ModifyAssetPaths` fixes leaked absolute paths. - Clear `customData` / internal arcs yourself (or via `processingFunc`). - Re-open and inventory (`ComputeAllDependencies`) before you ship.

**34.4 Change notifications (Tf.Notice, Usd.Notice)**

**What is it?** A **notice** is USD's callback: something changed, here is what.

`Tf.Notice.Register(Usd.Notice.ObjectsChanged, callback, stage)` runs your function on that stage. Live DCC exporters and UIs use this instead of polling `Traverse()` every frame.

**Why do we need it?** A DCC that re-exports the whole stage on every translate is slow and fights the user. A listener that hears `/World/Prop.xformOp:translate` can write **only** that opinion into `anim.usda`. Pipeline round-trips (Ch 35) sit on this.

**Beginner explanation.** A doorbell, not a security camera you stare at. USD rings `ObjectsChanged` when prims or properties change. You look through the peephole (`GetResyncedPaths`, `GetChangedInfoOnlyPaths`) and decide whether to open the door (update the DCC, dirty a hook).

*Where the analogy breaks:* one user action can ring **more than once** (creating an attribute then setting it). Batch with `ChangeBlock` (Section 34.5) so you get one ring.

**Technical explanation.** Verified on USD 26.08. Notice types on `Usd.Notice`:

Type	When
<code>ObjectsChanged</code>	Prims/properties authored or resynced
<code>StageContentsChanged</code>	Broader "the stage changed"
<code>StageEditTargetChanged</code>	<code>SetEditTarget</code>
<code>LayerMutingChanged</code>	<code>MuteLayer</code> / unmute
<code>StageNotice</code>	Base type

`ObjectsChanged` payload:

- `GetResyncedPaths()` — prim index rebuilt (new prim, type change, composition change).
- `GetChangedInfoOnlyPaths()` — value/metadata change, no resync.

Register:

```
listener = Tf.Notice.Register(
 Usd.Notice.ObjectsChanged, callback, stage)
callback(notice, sender)
listener.Revoke()
```

The callback is `fn(notice, sender)`. `sender` is the stage you passed. **Revoke on the listener object** (`listener.Revoke()`). There is no `Tf.Notice.Revoke(key)` on 26.08.

Creating `/W` resynced `['/W']`. Creating and setting attribute `a` produced **two** notices in a probe: one resync of `/W.a`, one info-only `/W.a`. Do not assume one Python call equals one notice.

**Mental model.**

```
Td.Notice.Register(ObjectsChanged, fn, stage)
 ^
 | DefinePrim -> resync paths
 | Set value -> info-only (sometimes plus resync on create)
 |
fn reads paths, updates DCC or flags "run hooks"
listener.Revoke() when the tool closes
```

**Simple example.** A USD outliner in a DCC registers `ObjectsChanged`. On resync of `/W`, it adds a row. On info-only `/W.a`, it refreshes that cell only.

**USDA example.** Notices are runtime. This is the layer the listener is watching; nothing in USDA registers a callback.

```
#usda 1.0

def Xform "W"
{
 double a = 1
}
```

### Python example.

```
from pxr import Tf, Usd, Sdf, UsdGeom

hits = []

def on_changed(notice, sender):
 hits.append((
 [str(p) for p in notice.GetResyncdPaths()],
 [str(p) for p in notice.GetChangedInfoOnlyPaths()],
))

stage = Usd.Stage.CreateInMemory()
listener = Tf.Notice.Register(
 Usd.Notice.ObjectsChanged, on_changed, stage)

UsdGeom.Xform.Define(stage, "/W")
print("after define", hits)
hits.clear()

stage.GetPrimAtPath("/W").CreateAttribute(
 "a", Sdf.ValueTypeNames.Double).Set(1.0)
print("n after set", len(hits))
for h in hits:
 print(" resync", h[0], "info", h[1])

print("types:", sorted(
 n for n in dir(Usd.Notice) if n[0].isupper()))
listener.Revoke()
```

### Expected output

```
after define [['/W'], []]
n after set 2
resync ['/W.a'] info []
resync [] info ['/W.a']
types: ['LayerMutingChanged', 'ObjectsChanged', 'StageContentsChanged', 'StageEditTargetChanged', 'StageNotice']
```

Define = one resync. CreateAttribute + Set = two notices. Revoke so the listener does not outlive the tool.

**Real-world use case.** A DCC “USD layer” mode registers `ObjectsChanged` on the opened shot. Translating a prop writes into the animation edit target. The listener marks that layer dirty in the UI. Closing the scene calls `listener.Revoke()` so a deleted stage is not pinged.

### Common mistakes.

#### MISTAKE

```
`Tf.Notice.Revoke(listener)` — that API is not on `Tf.Notice` in 26.08. Call
`listener.Revoke()`.
```



**MISTAKE**

Doing heavy work (full flatten, full validate) in the callback. You will hitch. Set a dirty flag; run hooks on idle.

**MISTAKE**

Registering globally when you meant one stage. Pass the stage as the third argument.

**Exam traps.****TRAP**

"One `Set()` is one notice." Attribute create + set was two.

**TRAP**

"`GetChangedInfoOnlyPaths` includes new prims." New prims are **\*\*resync\*\*** paths.

**TRAP**

Notices persist after the stage is destroyed if you forget `Revoke`.

**Exam takeaways.****KEY**

- `Tf.Notice.Register(Usd.Notice.ObjectsChanged, fn, stage)` - Resync vs info-only paths; one call can send two notices. - `listener.Revoke()` — not `Tf.Notice.Revoke`. - Use notices for live DCC; use hooks for batch export.

**34.5 Sdf.ChangeBlock for performance**

**What is it?** Sdf.ChangeBlock is a context manager that **holds** scene-description notices until the block ends, then delivers **one** combined update. Obj 6.1: identify when that removes a bottleneck.

**Why do we need it?** Each Sdf spec you create can resync the stage and ping every ObjectsChanged listener (Section 34.4). A thousand spheres without a block means a thousand UI refreshes. With a block, the outliner updates once.

**Beginner explanation.** A change block is “pause email notifications while I file 200 messages, then send one digest.”

*Where the analogy breaks:* nested blocks are reference-counted; the digest goes out when the **outermost** with exits. And Usd.Stage.DefinePrim / some schema Define calls may **fail inside** a block because the stage has not resynced the parent yet. Author **Sdf specs** inside the block (Ch 8).

**Technical explanation.** Verified on USD 26.08:

```
with Sdf.ChangeBlock():
 Sdf.PrimSpec(parent_spec, "S0", Sdf.SpecifierDef, "Sphere")
 ...
one ObjectsChanged with all new paths
```

Probe: three Sdf.PrimSpec children **inside** a block → **1** notice listing /W/S0, /W/S1, /W/S2. The same three **outside** → **3** notices.

Obj 6.1 answer: ChangeBlock helps when **many small Sdf (or Usd) edits** would each trigger indexing and listeners. It does **not** make Save() or disk fsync faster. It does **not** skip LIVERPS; it delays *notification* of composition.

UsdGeom.Sphere.Define(stage, "/W/S0") inside a ChangeBlock **raised** Failed to define UsdPrim in a 26.08 probe even though /W already existed. Prefer Sdf authoring inside the block; let the stage catch up at the end.

**Mental model.**

```
without: edit -> notice -> edit -> notice -> edit -> notice
with: [edit edit edit] -> one notice
```

**Simple example.** A procedural city exporter writes 10,000 building Xforms. Wrap the Sdf loop in `ChangeBlock`. The DCC outliner (registered in Section 34.4) redraws once.

**USDA example.** `ChangeBlock` does not appear in USDA. The result after the block is ordinary specs:

```
#usda 1.0

def Xform "W"
{
 def Sphere "S0" {}
 def Sphere "S1" {}
 def Sphere "S2" {}
}
```

### Python example.

```
from pxr import Tf, Usd, Sdf, UsdGeom

hits = []

def on_changed(notice, sender):
 hits.append([str(p) for p in notice.GetResyncedPaths()])

stage = Usd.Stage.CreateInMemory()
layer = stage.GetRootLayer()
UsdGeom.Xform.Define(stage, "/W")
parent = layer.GetPrimAtPath("/W")

listener = Tf.Notice.Register(
 Usd.Notice.ObjectsChanged, on_changed, stage)
hits.clear()

with Sdf.ChangeBlock():
 for i in range(3):
 Sdf.PrimSpec(parent, f"S{i}", Sdf.SpecifierDef, "Sphere")
print("inside block notices", len(hits), hits)

hits.clear()
for i in range(3, 6):
 Sdf.PrimSpec(parent, f"S{i}", Sdf.SpecifierDef, "Sphere")
print("outside block notices", len(hits), hits)
listener.Revoke()
```

### Expected output

```
inside block notices 1 [['/W/S0', '/W/S1', '/W/S2']]
outside block notices 3 [['/W/S3'], ['/W/S4'], ['/W/S5']]
```

That 1-versus-3 count is Obj 6.1. The bottleneck is “too many notices / resyncs,” not “USD cannot write spheres.”

**Real-world use case.** A CAD tessellator used to call `UsdGeom.Mesh.Define` per part while a Hydra view listened to `ObjectsChanged`. The view froze. Moving the inner loop to `Sdf.PrimSpec` + `ChangeBlock` dropped export from minutes to seconds; the view refreshed once (Ch 8’s micro-benchmark, Ch 44 continues the story).

### Common mistakes.

#### MISTAKE

Wrapping `DefinePrim` of new children inside `ChangeBlock` and catching unexplained failures. Fix: author `Sdf.PrimSpec` in the block.

**MISTAKE**

Believing ChangeBlock writes USDC faster. It batches **notices**, not I/O.

**MISTAKE**

Leaving a ChangeBlock open across user interaction. The UI sees a stale stage until the ``with`` ends.

**Exam traps.****TRAP**

"ChangeBlock skips composition / LIVERPS." It delays **notification**.

**TRAP**

"Always use ChangeBlock." One ``Set()`` does not need it. Obj 6.1 is **\*when\***: many edits, live listeners.

**TRAP**

Counting Python loop iterations instead of notices. The exam cares that listeners ran once.

**Exam takeaways.****KEY**

- Obj 6.1: ChangeBlock = many edits, one notice / one resync. - Author **Sdf** inside the block; schema ``Define`` may fail until flush. - It is not a disk optimization and not a substitute for hooks. - Nested blocks flush on the outermost exit.

**Chapter lab(s)**

**Lab 30** (notices and change blocks) is this chapter: register `ObjectsChanged`, prove 1-vs-N notices with ChangeBlock, run a `flatten + ModifyAssetPaths` delivery. Lab 28 covers `localize/USDZ`. Lab 25/29 cover the exporter that should call `pipeline_hooks`.

**USDA reading exercises**

**Exercise 34-A.** A delivery layer still contains `prepend references = @./bolt.usda@` and `asset tex = @/home/td/tex/a.png@`. Which flatten tool was probably used, and what two cleanups remain?

**Exercise 34-B.** An exporter calls `UsdGeom.Sphere.Define` 500 times inside with `Sdf.ChangeBlock()`: and raises `Failed to define UsdPrim`. What should it do instead?

**Answers**

- **34-A.** `FlattenLayerStack` (references survived). Still need `ModifyAssetPaths` to a relative texture and to ship `bolt.usda` (or `Flatten` / `localize` it).
- **34-B.** Author `Sdf.PrimSpec` (and attribute specs) inside the block; let Usd see the prims after the block ends. Or `Define` outside a block.

**Chapter review****Summary**

- Hooks repair structure (`defaultPrim`, `kind`, `metrics`) before `Save`.
- Export vs `FlattenLayerStack` vs `Flatten`: one layer / merge sublayers keep refs / bake stage.
- Both flatten tools leak absolute asset paths; `ModifyAssetPaths` and `customData` cleanup are part of Obj 1.9.
- `Tf.Notice.Register(ObjectsChanged, fn, stage); listener.Revoke()`.
- `ChangeBlock`: 3 Sdf edits → 1 notice (Obj 6.1); do not `DefinePrim` inside blindly.

**If you see... → think...**

If you see...	Think...
Missing <code>defaultPrim</code> on export	Hook did not run
Client missing texture, file was a sublayer	They got Export, not FlattenLayerStack
Client still needs to retarget Bolt	Do not Flatten; FlattenLayerStack + send Bolt
@/mnt/show/...@ in a package	ModifyAssetPaths
<code>owner = "jdoe"</code> after Flatten	Strip <code>customData</code>
UI hitches every prim	ChangeBlock + Sdf
Failed to define <code>UsdPrim</code> in a block	Use Sdf specs
<code>Tf.Notice.Revoke</code>	Wrong; <code>listener.Revoke()</code>

**Review questions**

**R34-1** (Obj 1.9) Best description of `stage.Flatten()`? A. Merge sublayers, keep references B. Bake the composed stage into one layer; arcs go away C. Write only the root layer's specs D. Zip a USDZ

**R34-2** (Obj 1.9) Select two FlattenLayerStack results from this chapter's example. A. `/L/ Bolt` still has a reference B. `tex` is typically an absolute path C. `subLayers` remain pointing at `geo.usda` D. `customData` is automatically cleared

**R34-3** (Obj 1.9) `UsdUtils.ModifyAssetPaths` is for: A. Muting layers B. Rewriting asset-path strings on a layer C. Registering notices D. Setting `kind`

**R34-4** (Obj 6.1) Three Sdf specs inside one ChangeBlock produced how many `ObjectsChanged` notices here? A. 1 B. 3 C. 0 D. 6

**R34-5** (Obj 6.1) Select two reasons ChangeBlock is the right diagnosis. A. A live listener redraws on every notice B. You are creating thousands of specs in a loop C. `Save()` to NFS is slow D. LIVERPS is in the wrong order

**R34-6** (Obj 1.9) A hook should set `defaultPrim` when: A. Always, picking `GetPseudoRoot()` B. There is exactly one root prim and none is set C. After Flatten only D. Never; only artists may set it

**R34-7** (Obj 6.1) How do you unregister a notice listener on 26.08? A. `listener.Revoke()` B. `Tf.Notice.Revoke(listener)` C. `stage.Close()` D. Delete the callback function

**R34-8** (Obj 1.9) Export of `entry.usda` did not include `tex`. Why? A. `tex` was authored on a **sublayer** B. Asset attributes cannot export C. Flatten is required to store asset types D. `defaultPrim` was missing

**R34-9** (Obj 1.9) Select two proprietary items to strip before a client package. A. `customData` `owner` B. Internal `/studio/` references C. `defaultPrim` D. `kind = component`

**R34-10** (Obj 6.1) `UsdGeom.Sphere.Define` inside `Sdf.ChangeBlock` in a 26.08 probe: A. Is always the recommended pattern B. May raise Failed to define `UsdPrim`; use `Sdf.PrimSpec` instead C. Skips LIVERPS D. Writes USDC directly

**R34-11** (Obj 1.9) After Flatten, a referenced Cube child is: A. Still a reference B. A local typed spec with no reference list C. Deleted D. Moved to the session layer

**R34-12** (Obj 6.1) `GetResyncdPaths()` vs `GetChangedInfoOnlyPaths()`: A. Resync = prim index rebuilt; info-only = value/metadata without resync B. They are aliases C. Info-only is only for sublayers D. Resync paths are always empty inside ChangeBlock

**Review answers**

- **R34-1: B.** Review: §34.2.
- **R34-2: A and B.** Sublayers merge away; metadata stays. Review: §34.2–34.3.

- **R34-3: B.** Review: §34.3.
- **R34-4: A.** Review: §34.5.
- **R34-5: A and B.** Review: §34.5.
- **R34-6: B.** Review: §34.1.
- **R34-7: A.** Review: §34.4.
- **R34-8: A.** Review: §34.2.
- **R34-9: A and B.** Keep identifier metadata the client needs. Review: §34.3.
- **R34-10: B.** Review: §34.5.
- **R34-11: B.** Review: §34.2.
- **R34-12: A.** Review: §34.4.

CHAPTER 35

# Chapter 35 — Build Configurations and DCC Integration

**Exam domain:** Pipeline Development (14%), also Customizing USD (6%) · **Objectives:** 3.1, 3.2, 7.4, 7.8 · **Study day:** 10 · **Est. time:** 80 min **Prerequisites:** Ch 1 (the USD stack), Ch 27 / 29 (exchange and DCC importers), Ch 33 (resolvers), Ch 34 (hooks, notices)

This chapter is how USD **gets onto a machine** and **into a DCC**. Obj 3.1 / 3.2 are “build a plugin against a given USD” and “build USD from scratch.” Obj 7.4 / 7.8 are round-trip pipelines and writing a DCC importer. `usd-core` in this book’s venv is a **wheel**, not a source build — we teach the official build map, then run what 26.08 Python actually exposes (`Usd.GetVersion`, `UsdPhysics`).

## Learning goals

- Describe `build_usd.py` / CMake as the way you compile USD and optional components.
- Match a plugin’s USD version to the DCC’s USD version (Obj 3.1).
- Design a DCC round-trip that does not drop custom data, units, or payloads (Obj 7.4, 7.8).
- Apply `UsdPhysics` APIs (`RigidBodyAPI`, `CollisionAPI`, `PhysicsScene`, `kilogramsPerUnit`) in a pipeline.

## Key terms

Term	One-line definition
<code>build_usd.py</code>	Pixar’s Python wrapper around CMake that compiles OpenUSD
Monolithic vs shared	One fat library vs many <code>.so/.dll</code> plugins
ABI	Binary interface; plugins built against USD <i>N</i> must not load in USD <i>M</i>
DCC	Digital content creation app (Maya, Houdini, 3ds Max, Blender, CAD, ...)
Round-trip	Export to USD and import back without silent data loss
<code>UsdPhysics</code>	Schema domain for rigid bodies, collisions, joints, mass
<code>kilogramsPerUnit</code>	Stage metadata: how many kilograms one USD mass unit is

## 35.1 Building USD (`build_usd.py`, CMake options)

**What is it? Building USD** means compiling the C++ libraries, Python bindings, plugins, and (optionally) tools such as `usdview` from the OpenUSD source. Pixar ships `build_scripts/build_usd.py`. Under it, **CMake** selects which domains and third-party libraries you get.

**Why do we need it?** The `usd-core` wheel is enough to *author* layers. It does **not** include `usdview`, `usdcat`, Hydra imaging, `MaterialX` (`UsdMtlx`), or `usdGenSchema`. A studio that needs those — or a patched USD — builds from source (Obj 3.2). A plugin author must build **against the same tree** the DCC loads (Obj 3.1).

**Beginner explanation.** Buying a ready-made bicycle (`usd-core` / a DCC’s bundled USD) vs welding your own frame from a kit (`build_usd.py`). The kit lets you add lights, a different wheel size, or a custom rack (Hydra, `MaterialX`, your resolver). You cannot bolt a rack built for a 2024 frame onto a 2026 bike without checking the mounts (ABI).

Where the analogy breaks: you do not rebuild USD to use it in this book. You rebuild when you need a component the wheel omitted, or when you ship a C++ plugin.

**Technical explanation.** Official flow (not runnable in this venv; commands are illustrative):

```
python build_scripts/build_usd.py /opt/usd \
 --python --usdview \
 --build-args USD,"-DPXR_BUILD_USD_PHYSICS=ON"
```

Typical CMake knobs (names you should recognize, not memorize every flag):

Area	What it controls
PXR_BUILD_USD_PHYSICS	UsdPhysics schemas
PXR_BUILD_IMAGING / PXR_BUILD_USD_IMAGING	Hydra, usdview
PXR_BUILD_MATERIALX_PLUGIN	MaterialX file format / UsdMtlx
Python bindings	Whether <code>from pxr import Usd</code> works for that install
Monolithic (PXR_BUILD_MONOLITHIC)	One library vs many plugin libraries

This book's **verified** install:

- Package: `usd-core 26.8` → API `Usd.GetVersion() == (0, 26, 8)`
- Present: `UsdPhysics`, `UsdShade`, `UsdLux`, `UsdSkel`, `UsdValidation`, `Trace`
- Absent: `UsdImaging`, `UsdMtlx`, CLI tools (`usdcats`, `usdview`, `usdGenSchema`)

#### VERSION

`Usd.GetVersion()` returns `*(0, 26, 8)*` for OpenUSD 26.08. The leading `0` is the historical major field. Do not write a check that compares `GetMajorVersion() == 26`.

`build_usd.py` also downloads or uses system TBB, Boost, OpenSubdiv, etc. Obj 3.2 “custom dependencies” means: you point CMake at *your* TBB/Python, not a mystery copy from another DCC.

#### Mental model.

```
source --build_usd.py / CMake--> libusd, plugins, optional usdview
usd-core wheel -----> Python pxr.* only (this book)

GetVersion() (0, 26, 8) == OpenUSD 26.08
```

**Simple example.** A farm image has no GUI. You build USD with imaging **off** and Python **on**. Artists’ workstations build with `--usdview`. Both must still be 26.08 if they share plugin `.so` files.

**USDA example.** Build flags are not stored in USDA. Version metadata you *can* put on a layer for the pipeline:

```
#usda 1.0
(
 customLayerData = {
 string usdVersion = "26.08"
 string generator = "studio_export 4.2"
 }
)
```

`customLayerData` is layer metadata (Ch 12). It does not change `GetVersion()` of the running library.

#### Python example.

```
from pxr import UsdImaging # ModuleNotFoundError on usd-core
```

That import is expected to **fail** on `usd-core`. The runnable check:

```
from pxr import Usd, UsdPhysics
import pkgutil
import pxr

print("GetVersion", Usd.GetVersion())
print("major,minor,patch", Usd.GetMajorVersion(),
 Usd.GetMinorVersion(), Usd.GetPatchVersion())
mods = sorted(m.name for m in pkgutil.iter_modules(pxr.__path__))
print("has UsdPhysics", "UsdPhysics" in mods)
print("has UsdImaging", "UsdImaging" in mods)
print("has UsdMtlx", "UsdMtlx" in mods)
print("RigidBodyAPI", hasattr(UsdPhysics, "RigidBodyAPI"))
```

### Expected output

```
GetVersion (0, 26, 8)
major,minor,patch 0 26 8
has UsdPhysics True
has UsdImaging False
has UsdMtlx False
RigidBodyAPI True
```

**Real-world use case.** A studio's IT runs `build_usd.py` once per USD upgrade, with a locked TBB and Python 3.11 matching Maya's. The same prefix is used to compile the studio resolver (Ch 33) and schema plugin (Ch 37). DCC plugin builds take `-I/opt/usd/` include from that prefix — never from a second stray USD.

### Common mistakes.

#### MISTAKE

Mixing ``usd-core`` from pip with a DCC's ``pxr`` on ``PYTHONPATH``. Two USDs in one process. Fix: one prefix.

#### MISTAKE

Reading ``GetMajorVersion() == 26``. It is ``0``; minor is ``26``.

#### MISTAKE

Assuming ``usdview`` appears after ``pip install usd-core``. Use NVIDIA's Windows/Linux prebuilts (Chapter F5) or build with imaging.

### Exam traps.

#### TRAP

"Obj 3.2 is ``pip install``." Building from scratch is CMake/``build_usd.py`` and *\*your\** dependencies.

#### TRAP

"``usd-core`` includes Hydra." ``UsdImaging`` is missing here.

#### TRAP

Version string ``"26.8"`` vs ``"26.08"`` vs tuple ``(0, 26, 8)``. Know all three labels.

### Exam takeaways.

#### KEY

- Source builds use ``build_usd.py`` / CMake; ``usd-core`` is a Python subset. - ``GetVersion()`` → ``(0, 26, 8)`` for OpenUSD 26.08. - Imaging, MaterialX, CLI tools, ``usdGenSchema`` need a full build. - Obj 3.2: you choose the third-party libraries CMake links.



## 35.2 Matching plugin builds to USD versions

**What is it?** A **plugin** (resolver, schema, file format, Hydra delegate) is a shared library compiled against **one** USD ABI. **Matching** means the DCC, the farm, and the plugin were all built with that same USD version and the same C++ compiler/C++ standard (Obj 3.1).

**Why do we need it?** Loading a 25.11 schema plugin into 26.08 usdview is a crash or a silent miss, not a friendly error. Pipelines die at “it worked in Maya.”

**Beginner explanation.** A trailer hitch rated for one car year. The plugin is the trailer. USD is the car. Same-year hitch (headers + libusd) or the trailer falls off.

*Where the analogy breaks:* Python-only tools that use `usd-core` via `from pxr import Usd` are not C++ plugins. They still must not import a second `pxr`. Codeless schemas (Ch 37) are XML/USDA and are gentler, but C++ schema generated code is not.

**Technical explanation.** Checklist studios write next to Chapter 32’s guidelines:

Check	Why
Same <code>Usd.GetVersion()</code> in DCC and plugin host	ABI
Same build flavor (monolithic vs shared)	Link errors
<code>PXR_PLUGINPATH_NAME</code> points at <b>this</b> USD’s plugin dir plus studio plugins	Wrong <code>plugInfo.json</code> = missing types
One <code>PYTHONPATH</code> / <code>PATH</code> / <code>LD_LIBRARY_PATH</code> prefix	Two libusd in one process
Rebuild plugins on every USD bump	Obj 3.1 “against a given version”

`Plug.Registry().GetAllPlugins()` lists loaded plugin names (`usdPhysics`, `usdGeom`, ...). A studio plugin that did not load will not appear. Set `PXR_PLUGINPATH_NAME` **before** the first `from pxr import ...` in some hosts; in others the DCC loads USD first — then you append via that host’s plugin path UI.

Python:

```
Conceptual: run inside the DCC's Python, not a second venv
from pxr import Usd
assert Usd.GetVersion() == (0, 26, 8)
```

Do not copy Maya’s `pxr` into this book’s `.venv`.

### Mental model.

```
Maya 26.08 <---- plugin.so built with 26.08 headers
farm 26.08 <---- same .so
usdview 25.11 --X-- that .so (wrong year)
```

**Simple example.** IT ships `StudioResolver.so` in `/opt/studio/usd/26.08/plugin`. Maya, the farm, and `usdview` all use `/opt/usd/26.08`. After upgrading to 26.11 they rebuild the resolver; they do not reuse the 26.08 `.so`.

**USDA example.** USDA cannot record ABI. A publish may still stamp the generator:

```
#usda 1.0
(
 customLayerData = {
 string usdVersion = "26.08"
 string pluginSet = "studio-36"
 }
)
```

The farm’s opener refuses layers whose `usdVersion` is newer than the host.

### Python example.

```
from pxr import Plug, Usd

print("version", Usd.GetVersion())
names = sorted(p.name for p in Plug.Registry().GetAllPlugins())
```

```
print("n plugins", len(names))
print("has usdPhysics", "usdPhysics" in names)
print("has usdGeom", "usdGeom" in names)
```

### Expected output

```
version (0, 26, 8)
n plugins 27
has usdPhysics True
has usdGeom True
```

If `usdPhysics` were missing here, either the wheel omitted it (it does not) or `PXR_PLUGINPATH_NAME` hid the default plugins. That is the matching debug for Obj 3.1.

**Real-world use case.** Houdini ships a USD. The studio also builds `usdview`. They **pin Houdini's USD** for Solaris plugins and a **separate** 26.08 prefix for standalone tools, and they never copy `.so` files between those two prefixes.

### Common mistakes.

#### MISTAKE

Dropping a plugin into ``PXR_PLUGINPATH_NAME`` built with a different ``_GLIBCXX`` / `MSVC`. Load fails with an unreadable dynamic-linker error.

#### MISTAKE

Using ``LD_PRELOAD`` to force one `libusd` over another. You now have two heaps. Don't.

### Exam traps.

#### TRAP

"Any USD plugin loads in any USD because USDA is text." C++ plugins are binary.

#### TRAP

"Codeless schemas need a matching ABI too." Codeless still need a USD that understands the schema file format; they do not ship a `.so`` of generated accessors (Ch 37).

### Exam takeaways.

#### KEY

- One USD version per process; plugins rebuilt per bump. - ``GetVersion()`` in the DCC must match the plugin's build. - ``PXR_PLUGINPATH_NAME`` loads *\*this\** tree's ``pluginInfo.json``. - Count ``GetAllPlugins()`` when a type "does not exist."

## 35.3 DCC round-trips (importer / exporter integration)

**What is it?** A **round-trip** is export from a DCC to USD and import back (or USD → DCC → USD) so the scene is still the same in the ways the pipeline promised (Obj 7.4, 7.8). Chapter 27 gave the data-mapping document. Chapter 29 gave importer shape. Here you **wire** those into the DCC: same hooks, same resolver context, same version.

**Why do we need it?** A DCC that drops payloads, secretly Flattens, or rewrites `@./tex.png@` to `@C:\Users\...\@` on Save destroys the pipeline. Round-trip tests catch that before artists notice on the farm.

**Beginner explanation.** A bilingual meeting with a translator both ways. If the joke survives the return trip, the translator is good. If the punchline becomes a cube, you need a new importer.

*Where the analogy breaks:* some loss is **documented** (NURBS → tessellated mesh). Round-trip success is "nothing *undocumented* disappeared," not "CAD b-reps came back."

**Technical explanation.** Integration checklist (pin this next to Obj 7.4):

Step	Implementation
Open	<code>Usd.Stage.Open(path, pathResolverContext=studio_ctx)</code> (Ch 33)
Edit target	Department layer, not session, not Flatten (Ch 15, Ch 32)
Export	extract → transform → write → <b>hooks</b> (Ch 34) → validate (Ch 30) → Save
Import	Read composed Usd; do not require every arc to become a native DCC node
Custom data	Preserve custom attributes and <code>customData</code> (Ch 29)
Units	Honor <code>metersPerUnit</code> , <code>upAxis</code> , <code>kilogramsPerUnit</code> ; do not silently bake
Paths	Keep <code>./</code> pins; do not absolutize on Save (Ch 34 <code>ModifyAssetPaths</code> )
Notices	Optional live sync via <code>ObjectsChanged</code> (Ch 34)

Round-trip **test** (automate in CI): write a tiny stage with a payload, a reference, a custom attribute, a texture asset, and physics APIs → DCC import → DCC export → compare `ComputeAllDependencies` and attribute `Get()` values.

Obj 7.8 (write/extend an importer): the importer is the DCC-side reader that applies this checklist, not a second exporter.

### Mental model.

```

DCC memory --export/hooks--> published USD --import--> DCC memory
 ^ |
 +----- same resolver, version, mapping doc -----+

```

**Simple example.** Maya export writes `/Chair` with custom float `studio:weight = 4`. Maya import must recreate that attribute, not drop unknown names. A round-trip test asserts `Get() == 4`.

**USDA example.** The file the DCC must round-trip without flattening:

```

#usda 1.0
(
 defaultPrim = "Chair"
 metersPerUnit = 1
 upAxis = "Y"
)

def Xform "Chair" (
 kind = "component"
 prepend payload = @./payload.usdc@
)
{
 custom float studio:weight = 4
}

```

If the DCC Save comes back with no payload and inlined mesh points, the round-trip failed even if the chair still *looks* right.

**Python example.** A stand-in for “DCC Save then Open”: Export and reopen must keep the custom attribute and the payload arc.

```

from pxr import Usd, UsdGeom, Sdf

payload = Usd.Stage.CreateNew("payload.usda")
UsdGeom.Cube.Define(payload, "/Chair")
payload.SetDefaultPrim(payload.GetPrimAtPath("/Chair"))
payload.Save()

stage = Usd.Stage.CreateNew("chair.usda")
prim = UsdGeom.Xform.Define(stage, "/Chair").GetPrim()
stage.SetDefaultPrim(prim)
prim.GetPayloads().AddPayload("./payload.usda")
prim.CreateAttribute(
 "studio:weight", Sdf.ValueTypeNames.Float, custom=True).Set(4.0)
stage.Save()

```

```
again = Usd.Stage.Open("chair.usda")
p = again.GetPrimAtPath("/Chair")
items = again.GetRootLayer().GetPrimAtPath("/Chair").payloadList.prependedItems
print("weight", p.GetAttribute("studio:weight").Get())
print("payloads", [str(i.assetPath) for i in items])
print("HasAuthoredPayloads", p.HasAuthoredPayloads())
print("composed type", p.GetTypeName())
```

### Expected output

```
weight 4.0
payloads ['./payload.usda']
HasAuthoredPayloads True
composed type Xform
```

Local `Xform` is stronger than the payload's `Cube` type (LIVERPS), so the composed type stays `Xform`. The test that must not regress: **weight** and the **payload arc** survive a reopen. If a DCC Flattened, `HasAuthoredPayloads` would be `False` and `payloads` would be empty.

**Real-world use case.** A CAD round-trip: CAD → USD (tessellated mesh + `studio:cadId`) → CAD importer looks up `studio:cadId` and reattaches the parametric body. Without that custom attribute, CAD can only import a dumb mesh (Ch 29).

### Common mistakes.

#### MISTAKE

Import that `FlattenLayerStacks` for "simplicity." You just deleted the pipeline.

#### MISTAKE

Export from the DCC's current working file path using absolute Windows paths.

#### MISTAKE

A different USD version in the DCC than on the farm (Section 35.2).

### Exam traps.

#### TRAP

"Round-trip means the USDA text diffs empty." Binary crate, timestamp metadata, and ``doc`` comments will differ. Compare composed values and arcs.

#### TRAP

`Obj 7.8` is ``Stage.Open``. An importer maps USD into **\*\*DCC nodes\*\*** (or a USD-native layer mode). Opening a stage is necessary but not sufficient.

### Exam takeaways.

#### KEY

- Round-trip = mapping doc + hooks + same USD/resolver + tests on arcs and custom data.
- Do not Flatten as a default Save. - Importer (`Obj 7.8`) maps USD into the DCC; it must not invent a second data model silently. - Compare composed values, not USDA bytes.

## 35.4 Physics and other domain schemas in pipelines

**What is it? Domain schemas** beyond `UsdGeom` travel through the same pipeline: `UsdPhysics` (rigid bodies, collisions, scenes, mass), `UsdShade`, `UsdLux`, `UsdSkel`. A pipeline that only round-trips meshes will drop `apiSchemas = ["PhysicsRigidBodyAPI"]` and the simulation dies (`Obj 7.4` applied to physics).

**Why do we need it?** Games, robotics, and digital twins need mass and collision in the **same** asset the renderer uses. If export strips unknown APIs, you fork "render USD" and "physics USD" and they drift.

**Beginner explanation.** The chair's *look* is the mesh and material. The chair's *behavior* when you knock it is RigidBodyAPI + CollisionAPI + a PhysicsScene gravity. Both are USD prims/API schemas. The pipeline must Save both.

*Where the analogy breaks:* UsdPhysics does not run a solver. A DCC or engine reads the schemas and simulates. USD stores the **intent**.

**Technical explanation.** Verified on USD 26.08 (usd-core includes UsdPhysics):

API	Role
UsdPhysics.RigidBodyAPI.Apply(prim)	Marks a prim as a simulated body
UsdPhysics.CollisionAPI.Apply(prim)	Marks it as colliding
UsdPhysics.MassAPI	Mass / density
UsdPhysics.Scene.Define(stage, path)	Gravity and scene settings
UsdPhysics.SetStageKilogramsPerUnit	Mass analog of metersPerUnit

Authored USDA uses prepend apiSchemas = ["PhysicsRigidBodyAPI", "PhysicsCollisionAPI"]. Exporters must **Apply** the API (same rule as MaterialBindingAPI in Ch 29). Listing the token by hand without Apply is fragile.

Defaults: GetStageKilogramsPerUnit returns **1.0** even when not authored; StageHasAuthoredKilogramsPerUnit tells you whether someone wrote it. Gravity on a new PhysicsScene starts unset-looking (magnitude was -inf until we authored it). **Always author gravity** if you care.

Other domains follow the same pipeline rule: if the DCC has no native node, store a custom attribute or keep the USD API schema — do not delete it on import (Ch 29 nonstandard data).

### Mental model.

```
Cube "Box"
 apiSchemas: PhysicsRigidBodyAPI + PhysicsCollisionAPI
 PhysicsScene
 gravityDirection / gravityMagnitude
 stage metadata
 metersPerUnit + kilogramsPerUnit
```

**Simple example.** A robotics cell: each link is a mesh with RigidBodyAPI + CollisionAPI; /PhysicsScene has Earth gravity; kilogramsPerUnit = 1. The engine reads those APIs; usdview still draws the mesh.

### USDA example.

```
#usda 1.0
(
 kilogramsPerUnit = 1
)

def Cube "Box" (
 prepend apiSchemas = ["PhysicsRigidBodyAPI", "PhysicsCollisionAPI"]
)
{
}

def PhysicsScene "PhysicsScene"
{
 vector3f physics:gravityDirection = (0, -1, 0)
 float physics:gravityMagnitude = 10
}
```

### Python example.

```
from pxr import Usd, UsdGeom, UsdPhysics

stage = Usd.Stage.CreateInMemory()
```

```

box = UsdGeom.Cube.Define(stage, "/Box")
UsdPhysics.RigidBodyAPI.Apply(box.GetPrim())
UsdPhysics.CollisionAPI.Apply(box.GetPrim())
print("has RB", box.GetPrim().HasAPI(UsdPhysics.RigidBodyAPI))
print("has Col", box.GetPrim().HasAPI(UsdPhysics.CollisionAPI))

print("kg authored?", UsdPhysics.StageHasAuthoredKilogramsPerUnit(stage))
UsdPhysics.SetStageKilogramsPerUnit(stage, 1)
print("kg", UsdPhysics.GetStageKilogramsPerUnit(stage),
 "authored?", UsdPhysics.StageHasAuthoredKilogramsPerUnit(stage))

scene = UsdPhysics.Scene.Define(stage, "/PhysicsScene")
scene.CreateGravityDirectionAttr().Set((0, -1, 0))
scene.CreateGravityMagnitudeAttr().Set(10)
print("g dir", tuple(scene.GetGravityDirectionAttr().Get()))
print("g mag", scene.GetGravityMagnitudeAttr().Get())
print("apiSchemas",
 list(box.GetPrim().GetAppliedSchemas()))

```

### Expected output

```

has RB True
has Col True
kg authored? False
kg 1.0 authored? True
g dir (0.0, -1.0, 0.0)
g mag 10.0
apiSchemas ['PhysicsRigidBodyAPI', 'PhysicsCollisionAPI']

```

**Real-world use case.** A digital twin of a conveyor: USD carries kinematics (UsdPhysics joints) and the factory mesh. The PLC simulator subscribes to joint schemas; the marketing render ignores them. One published `cell/v014` serves both because the exporter **Apply**d the APIs instead of putting mass in a side-car JSON.

### Common mistakes.

#### MISTAKE

Import that strips ``apiSchemas`` it does not understand. Physics vanishes; the mesh still looks fine.

#### MISTAKE

Assuming ``GetStageKilogramsPerUnit`` going ``1.0`` means someone authored it. Check ``StageHasAuthoredKilogramsPerUnit``.

#### MISTAKE

Forgetting ``PhysicsScene`` gravity. Bodies may not fall; that is data, not a solver bug.

### Exam traps.

#### TRAP

"`UsdPhysics` is not in usd-core." It **is**, on 26.08. ``UsdImaging`` is not.

#### TRAP

"Apply is optional if USDA lists ``apiSchemas``." Use ``Apply`` in Python; validators and ``HasAPI`` expect the schema applied (same story as materials).

#### TRAP

Mass uses ``metersPerUnit``. Mass uses **not** ``kilogramsPerUnit``.

### Exam takeaways.

**KEY**

- `UsdPhysics` ships in 26.08 `usd-core`; Apply `RigidBodyAPI` / `CollisionAPI`. - `kilogramsPerUnit` is authored separately from meters; fallback get is `1.0`. - Author gravity on `PhysicsScene`. - Domain schemas are first-class pipeline data, not optional extras to strip.

**Chapter lab(s)**

No dedicated lab number beyond **Lab 37** (capstone pipeline) and **Lab 25/29** (exporter/importer). When you write those labs, include a `UsdPhysics` Apply and a round-trip assert on `studio:weight` plus payloads.

**USDA reading exercises**

**Exercise 35-A.** A DCC plugin was compiled with USD 25.11 headers. `usdview` reports `GetVersion() == (0, 26, 8)`. The plugin's types do not appear in `GetAllPlugins()`. Why?

**Exercise 35-B.** `GetStageKilogramsPerUnit(stage)` prints 1.0 on a brand-new stage. Did someone author mass units?

**Answers**

- **35-A.** ABI / version mismatch (Obj 3.1). Rebuild the plugin against 26.08 and load it from that prefix's `PXR_PLUGINPATH_NAME`.
- **35-B.** No. Fallback is 1.0. Ask `StageHasAuthoredKilogramsPerUnit`.

**Chapter review****Summary**

- Build USD with `build_usd.py` / CMake when you need imaging, MaterialX, tools, or custom deps (Obj 3.2).
- `GetVersion()` is (0, 26, 8) here — not major 26.
- Plugins match that ABI; one `libusd` per process (Obj 3.1).
- DCC round-trip: same resolver, hooks, custom data, no surprise Flatten (Obj 7.4 / 7.8).
- `UsdPhysics` APIs and `kilogramsPerUnit` travel with the asset.

**If you see... → think...**

If you see...	Think...
<code>GetMajorVersion() == 0</code>	Normal for 26.08
<code>UsdImaging</code> import fails	<code>usd-core</code> wheel
Plugin types missing	Version / <code>PXR_PLUGINPATH_NAME</code>
Save inlined a payload	Flattened round-trip
Physics missing after DCC	<code>apiSchemas</code> stripped
<code>kg == 1.0</code>	Could be fallback

**Review questions**

**R35-1** (Obj 3.2) `build_usd.py` is: A. A pip wrapper for `usd-core` B. The OpenUSD source-build entry that drives CMake C. A Hydra delegate D. A USDZ packer

**R35-2** (Obj 3.1) Select two requirements for a C++ resolver plugin. A. Built against the host USD version B. Discoverable via `pluginInfo.json` / `PXR_PLUGINPATH_NAME` C. Listed in `defaultPrim` D. Flattened into every shot

**R35-3** (Obj 7.4) A round-trip test should fail if: A. USDA whitespace changed B. payloads prepend disappeared and meshes were inlined C. Crate vs USDA encoding changed by policy D. doc metadata was added by Flatten

**R35-4** (Obj 7.8) An importer's job is: A. Only Stage.Open B. Map USD into the DCC (or keep USD-native layers) without silent loss C. Always FlattenLayerStack first D. Delete unknown API schemas

**R35-5** (Obj 3.1) Usd.GetVersion() in this venv: A. (0, 26, 8) B. (26, 8, 0) C. (26, 0, 8) D. (8, 26, 0)

**R35-6** (Obj 7.4) Select two physics round-trip must-keeps. A. PhysicsRigidBodyAPI on the body B. Authored PhysicsScene gravity C. UsdImaging plugin D. GetMaster()

**R35-7** (Obj 3.2) usd-core 26.8 includes: A. UsdPhysics and UsdShade, not UsdImaging B. usdview C. usdGenSchema D. MaterialX UsdMtlx

**R35-8** (Obj 7.4) StageHasAuthoredKilogramsPerUnit is False and GetStageKilogramsPerUnit is 1.0. Meaning? A. Someone authored 1.0 B. Fallback; mass units were not authored C. Physics is disabled D. metersPerUnit is 1.0

**R35-9** (Obj 7.8) Select two exporter/importer integration rules. A. Open with the studio pathResolverContext B. Run pipeline hooks before Save C. Use the session layer as the published file D. Absolutize all asset paths for portability

**R35-10** (Obj 3.1) Plug.Registry().GetAllPlugins() missing usdPhysics on a full USD build usually means: A. The cube has no extent B. Plugin search path / build omitted physics C. kind is wrong D. LIVERPS failed

**R35-11** (Obj 7.4) RigidBodyAPI.Apply authors: A. prepend apiSchemas = ["PhysicsRigidBodyAPI"] B. A new sublayer C. A Hydra material D. defaultPrim

**R35-12** (Obj 3.2) "Build USD with custom dependencies" means: A. Edit USDA subLayers B. Point CMake at your TBB/Python/OpenSubdiv instead of random copies C. pip install numpy D. CreateNewUsdzPackage

### Review answers

- **R35-1: B.** Review: §35.1.
- **R35-2: A and B.** Review: §35.2.
- **R35-3: B.** Bytes and comments are allowed to change. Review: §35.3.
- **R35-4: B.** Review: §35.3.
- **R35-5: A.** Review: §35.1.
- **R35-6: A and B.** Review: §35.4.
- **R35-7: A.** Review: §35.1.
- **R35-8: B.** Review: §35.4.
- **R35-9: A and B.** Review: §35.3.
- **R35-10: B.** Review: §35.2.
- **R35-11: A.** Review: §35.4.
- **R35-12: B.** Review: §35.1.



## Part VII — Customizing USD

### CHAPTER 36

## Chapter 36 — The Plugin System

**Exam domain:** Customizing USD (6%) · **Objectives:** 3.1 · **Study day:** 11 · **Est. time:** 70 min **Prerequisites:** Ch 1 (the USD stack), Ch 6 (schemas), Ch 33 (resolvers), Ch 35 (ABI / `GetVersion`)

USD is not one binary that already knows every schema, file format, and resolver. Those are **plugins**. Obj 3.1 is “build a USD plug-in against a given version of USD” — Chapter 35 covered the *build*. This chapter covers the *discovery* file (`pluginInfo.json`), the registry (`Plug.Registry`), the search path (`PXR_PLUGINPATH_NAME`), and the four plugin *jobs* you must name on the exam.

### Learning goals

- Read a `pluginInfo.json`: `Name`, `Type`, `LibraryPath`, `Info.Types`, `bases`.
- List and look up plugins with `Plug.Registry`.
- Register a plugin at runtime (`RegisterPlugins`) and explain `PXR_PLUGINPATH_NAME`.
- Match a plugin job to its base type: `schema`, `SdfFileFormat`, `ArResolver`, `Hydra/Sdr`.

### Key terms

Term	One-line definition
<b>Plugin</b>	A library (or resource pack) USD discovers and may load
<b>pluginInfo.json</b>	JSON catalog that names the plugin and the <b>types</b> it provides
<b>Plug.Registry</b>	Process-wide catalog of discovered plugins
<b>PXR_PLUGINPATH_NAME</b>	Env var: extra directories in which to look for <code>pluginInfo.json</code>
<b>Type</b>	A C++/Tf type registered in <code>Info.Types</code> (e.g. <code>UsdGeomCube</code> )
<b>bases</b>	Parent type names in <code>pluginInfo.json</code> ( <code>SdfFileFormat</code> , <code>ArResolver</code> , ...)
<b>Resource plugin</b>	No shared library; only files under <code>ResourcePath</code>
<b>Library plugin</b>	Has (or had) a <code>LibraryPath</code> to a <code>.so/.dll</code>

### 36.1 Plugins and `pluginInfo.json`

**What is it?** A **plugin** is a unit USD can discover: usually a shared library plus a JSON file named `pluginInfo.json`. The JSON lists the plugin’s **name**, **type** (`library` or `resource`), where its files live, and which **Tf types** it provides (`Info.Types`).

**Why do we need it?** Without a catalog, USD would have to hard-code every schema and format. Adding `Cube`, `.usda`, or a studio resolver would mean editing USD’s core. `pluginInfo.json` lets Pixar *and you* add types by dropping a folder on the plugin path (Obj 3.1).

**Beginner explanation.** A food-truck license taped to the window: name of the truck, what it is allowed to sell (tacos = `UsdGeomCube`, coffee = `SdfUsdaFileFormat`), and where the kitchen is (`LibraryPath` / `ResourcePath`). The city (USD) does not cook; it only reads licenses and then lets that truck operate.

*Where the analogy breaks:* some “trucks” in `usd-core` have an empty `LibraryPath` because the code is already linked into the Python package. The JSON still **declares the types**. Discovery and linking are separate.

**Technical explanation.** Verified on USD 26.08. A library plugin looks like this (trimmed from `sdf` in `usd-core`):

```
{
 "Plugins": [
 {
 "Type": "library",
 "Name": "sdf",
 "LibraryPath": "",
 "ResourcePath": "resources",
 "Root": "..",
 "Info": {
 "Types": {
 "SdfUsdaFileFormat": {
 "bases": ["SdfFileFormat"],
 "extensions": ["usda"],
 "formatId": "usda",
 "primary": true
 }
 }
 }
 }
]
}
```

Field	Meaning
Plugins	Array; one file may list several
Type	library (code) or resource (files only)
Name	Registry key ( <code>usdGeom</code> , <code>sdf</code> , <code>ar</code> )
LibraryPath	Path to <code>.so/.dll</code> ; empty in this wheel
ResourcePath / Root	Where extra files (and this JSON) live
Info.Types	Map of Tf type name → metadata
bases	What the type <b>is</b> (schema base, <code>SdfFileFormat</code> , <code>ArResolver</code> , ...)

A **root** `plugInfo.json` can be only includes:

```
{ "Includes": ["*/resources/"] }
```

That is how `usd-core` discovers `usdGeom`, `sdf`, `ar`, ... under `pxr/plugininfo/`. `usdGenSchema` emits the per-schema JSON (you will see # Portions of this file auto-generated by `usdGenSchema`).

Schema entries add `schemaKind` (`concreteTyped`, `singleApplyAPI`, ...) and `alias` (the USDA type name `Cube`). File-format entries add `extensions` and `formatId`. Resolver entries list `bases`: `["ArResolver"]`.

### VERSION

Verified on USD 26.08 `usd-core`. `LibraryPath` is empty for built-in plugins in the wheel; a studio C++ plugin you build (Ch 35) has a real library path.

### Mental model.

```
folder/
 plugInfo.json <-- catalog (required)
 libStudio.so <-- optional (library plugins)
 resources/ <-- optional extra files

Info.Types["UsdGeomCube"].bases = ["UsdGeomGprim"]
Info.Types["SdfUsdaFileFormat"].bases = ["SdfFileFormat"]
```

**Simple example.** `def Cube "Box"` works because the `usdGeom` plugin's `plugInfo.json` registers `UsdGeomCube` with `alias Cube`. No extra import is required for the *file* to parse; Python still from `pxr` import `UsdGeom` to *define* one.

**USDA example.** The layer below is legal **because** a plugin declared the `Cube` schema. The `plugInfo.json` itself is JSON, not USDA.

```
#usda 1.0
(
 defaultPrim = "Box"
)

def Cube "Box"
{
 double size = 2
}
```

- `Cube` — alias from `usdGeom's Info.Types.UsdGeomCube`.
- If that plugin is missing, this file still *might* parse as a typed prim, but `IsA(UsdGeom.Cube)` and schema accessors will not behave as a built-in `Cube`.

### Python example.

```
from pxr import Plug, Tf, Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
prim = UsdGeom.Cube.Define(stage, "/Box").GetPrim()
print("typeName", prim.GetTypeName())
print("IsA Cube", prim.IsA(UsdGeom.Cube))

reg = Plug.Registry()
plug = reg.GetPluginWithName("usdGeom")
print("plugin", plug.name)
t = Tf.Type.FindByName("UsdGeomCube")
print("Tf type valid", bool(t))
print("DeclaresType Cube", plug.DeclaresType(t))
print("schemaKind",
 reg.GetStringFromPluginMetaData(t, "schemaKind"))
```

### Expected output

```
typeName Cube
IsA Cube True
plugin usdGeom
Tf type valid True
DeclaresType Cube True
schemaKind concreteTyped
```

`GetStringFromPluginMetaData` reads the JSON field `schemaKind` for that type. `concreteTyped` is the `IsA` schema you author with `def Cube`.

**Real-world use case.** A games studio adds `StudioDestructibleAPI`. `usdGenSchema` writes a `plugInfo.json` whose `Types` entry has `bases: ["UsdAPISchemaBase"]` and `schemaKind: singleApplyAPI`. DCC and farm both find that file (Section 36.2). Artists then `def Mesh "Crate"` plus `apiSchemas = ["DestructibleAPI"]` without a USD source rebuild.

### Common mistakes.

#### MISTAKE

Editing ``plugInfo.json`` ``Name`` to ``usdGeom2`` and leaving the library named ``usdGeom``. The registry key and the library must match what you built.

#### MISTAKE

Putting JSON next to the ``.so`` but naming it ``plugin.json``. USD looks for `**`plugInfo.json`**`.

#### MISTAKE

Assuming an empty ``LibraryPath`` means the plugin is broken. In ``usd-core`` it is normal.

### Exam traps.

**TRAP**

"`plugInfo.json` is USDA." It is JSON. USDA *uses* the types it declares.

**TRAP**

"`bases` is the prim path." It is the **Tf type** parent (`SdfFileFormat`, `UsdGeomGprim`, `ArResolver`).

**TRAP**

"Every plugin must ship a `.so`." Resource plugins (Type: resource) have none.

**Exam takeaways.****KEY**

- `plugInfo.json` catalogs `Name`, `Type` (library/resource), and `Info.Types`. - `bases` says what kind of plugin type you have. - Built-in `Cube` exists because `usdGeom` registered `UsdGeomCube`. - Obj 3.1: you ship this JSON with a library built against a **matched** USD.

**36.2 Plug.Registry and PXR\_PLUGINPATH\_NAME**

**What is it?** **Plug.Registry** is the process-wide list of plugins USD has **discovered**.

**PXR\_PLUGINPATH\_NAME** is the environment variable that adds extra directories to search for `plugInfo.json`. **RegisterPlugins(path)** does the same search at runtime (useful in tests and tools).

**Why do we need it?** A studio plugin sitting in `/opt/studio/plugin` is invisible until that folder is on the search path. "My type does not exist" is usually a path problem, not a C++ problem (Obj 3.1, Obj 6.3).

**Beginner explanation.** **PXR\_PLUGINPATH\_NAME** is **PATH** for licenses. The registry is the clipboard of every license the city has already read. `RegisterPlugins("/opt/studio/plugin")` is handing the clerk one more folder *now*.

*Where the analogy breaks:* some hosts read the env var only at **startup**. Setting it after `from pxr import Usd` can be too late. `RegisterPlugins` is the explicit clerk.

**Technical explanation.** Verified on USD 26.08:

Call	Result
<code>Plug.Registry()</code>	The singleton registry
<code>GetAllPlugins()</code>	List of <code>Plug.Plugin</code>
<code>GetPluginWithName("usdGeom")</code>	One plugin or None
<code>GetPluginForType(Tf.Type)</code>	Which plugin owns that type
<code>FindTypeByName("UsdGeomCube")</code>	<code>Tf.Type</code>
<code>RegisterPlugins("/path/to/dir")</code>	Discovers <code>plugInfo.json</code> there; returns the new <code>Plugin</code> list
<code>plugin.Load()</code>	Load the library (True if ok)
<code>plugin.FindPluginResource("hello.txt")</code>	File under <code>ResourcePath</code>
<code>plugin.DeclaresType(tfType)</code>	Whether this plugin's JSON lists that type

On this wheel, `GetAllPlugins()` returns **27** names, including `usdGeom`, `sdf`, `ar`, `usdPhysics`, `usdShade`, `usdHydra`, and several `*Validators`. `plugin.path` may be empty for built-ins; `resourcePath` points at `pxr/plugininfo/<name>/resources`.

PXR\_PLUGINPATH\_NAME is a **colon-separated** list of directories on Linux (semicolon on Windows). Each directory should contain a `pluginInfo.json` (or an includes-style root). `Tf.GetEnvSetting("PXR_PLUGINPATH_NAME")` returned **None** in this venv — the variable is unset; built-ins still load from the wheel's `pluginInfo` tree.

`RegisterPlugins` with a **resource** plugin (no `.so`) is enough to prove discovery. A C++ schema/resolver still needs `Load()` and a matching ABI (Ch 35).

### Mental model.

```

startup: wheel pluginInfo/ + $PXR_PLUGINPATH_NAME
 \ /
 v v
 Plug.Registry.GetAllPlugins()
 ^
tests/tools: RegisterPlugins("/opt/studio/plugin")

```

**Simple example.** You create `demo_plugin/pluginInfo.json` with `"Name": "BookDemoPlugin"` and `Type: resource`. After `RegisterPlugins("demo_plugin")`, `GetPluginWithName("BookDemoPlugin")` is not `None`, and `FindPluginResource("hello.txt")` returns that file.

**USDA example.** Discovery is not authored in USDA. A layer can *depend* on a plugin type; it cannot load the plugin.

```

#usda 1.0

def Sphere "Ball"
{
 double radius = 1
}

```

If `usdGeom` were not in the registry, `Sphere` would not be a first-class schema in this process. The file might still parse as a prim with that type name; Python `UsdGeom.Sphere` would not.

### Python example.

```

from pxr import Plug
import os

reg = Plug.Registry()
names = sorted(p.name for p in reg.GetAllPlugins())
print("\n built-in", len(names))
print("has usdGeom", "usdGeom" in names)
print("has BookDemoPlugin", "BookDemoPlugin" in names)

os.makedirs("demo_plugin/resources", exist_ok=True)
open("demo_plugin/pluginInfo.json", "w").write("""{
 "Plugins": [
 {
 "Type": "resource",
 "Name": "BookDemoPlugin",
 "Root": ".",
 "ResourcePath": "resources",
 "Info": { "Types": {} }
 }
]
}
""")
open("demo_plugin/resources/hello.txt", "w").write("hi\n")

new = reg.RegisterPlugins(os.path.abspath("demo_plugin"))
print("registered", [p.name for p in new])
p = reg.GetPluginWithName("BookDemoPlugin")
print("isResource", p.isResource, "isPythonModule", p.isPythonModule)
hello = p.FindPluginResource("hello.txt")
print("resource exists", os.path.isfile(hello))
print("basename", os.path.basename(hello))
print("has demo after",
 "BookDemoPlugin" in [x.name for x in reg.GetAllPlugins()])

```

**Expected output**

```
n built-in 27
has usdGeom True
has BookDemoPlugin False
registered ['BookDemoPlugin']
isResource True isPythonModule False
resource exists True
basename hello.txt
has demo after True
```

A resource plugin needs no compiler. A production resolver/schema plugin still needs the `.so` from Chapter 35, then this same `RegisterPlugins` or `PXR_PLUGINPATH_NAME`.

**Real-world use case.** The farm launcher exports `PXR_PLUGINPATH_NAME=/opt/studio/usd/26.08/plugin`. Every job — `usdview`, Maya, a Python publisher — sees `StudioResolver` and `StudioDestructibleAPI`. A TD who runs a laptop Python without that env var “cannot find” the custom type; `GetAllPlugins()` is the debug print (Obj 3.1).

**Common mistakes.****MISTAKE**

Setting ``PXR_PLUGINPATH_NAME`` to the `.so`` file. It must be the **directory** that contains `plugInfo.json``.

**MISTAKE**

Pointing the env var at ``/opt/studio/plugin/resources``. The JSON is usually one level up, or the includes root is the parent.

**MISTAKE**

Forgetting ``RegisterPlugins`` returns the **new** plugins only. ``GetAllPlugins()`` is the full list.

**Exam traps.****TRAP**

``GetAllPlugins()`` loads every `.so``. Discovery  $\neq$  load. ``isLoading`` can be False until ``Load()`` or first use.

**TRAP**

``PXR_PLUGINPATH_NAME`` replaces built-ins." It **adds** search directories. This env var still has 27 built-ins with the var unset.

**TRAP**

``RegisterPlugins`` with a path that has no `plugInfo.json`` silently giving you ``StudioResolver``. It returns an empty list.

**Exam takeaways.****KEY**

- Registry = discovered plugins; ``GetAllPlugins`` / ``GetPluginWithName`` / ``GetPluginForType``.
- ``PXR_PLUGINPATH_NAME`` adds JSON directories at startup.
- ``RegisterPlugins(dir)`` is the runtime equivalent.
- Discovery  $\neq$  load; resource plugins prove the path without a compiler.

**36.3 Plugin types (schemas, file formats, resolvers, Hydra)**

**What is it?** Four plugin **jobs** show up on the exam. They all use `plugInfo.json`, but they declare **different bases**:

Job	Typical bases	What it adds
<b>Schema</b>	UsdTyped / UsdGeomGprim / UsdAPISchemaBase	Prim types and APIs (Cube, MaterialBindingAPI)
<b>File format</b>	SdfFileFormat	.usda / .usdc / .usd / .usdz (or a studio format)
<b>Resolver</b>	ArResolver (or ArPackageResolver)	How @path@ becomes bytes (Ch 33)
<b>Hydra / Sdr</b>	Hydra scene index, SdrDiscoveryPlugin, ...	Rendering / shader discovery

Obj 3.1 is the *build*. Naming the job is how you choose the base class (Ch 37–38 fill each one in).

**Why do we need it?** “Write a plugin” is not one API. A file-format plugin will never fix a missing texture (that is Ar). A resolver will never add `def Door`. The exam asks you to pick the **type of plugin**.

**Beginner explanation.** Four departments in city hall: zoning (schemas — what a building *is*), archives (file formats — how it is *filed*), address office (resolvers — how you *find* it), and the fire-inspection renderer (Hydra — how you *see* it). Same license paper (`pluginInfo.json`); different department stamps (bases).

*Where the analogy breaks:* one JSON file can declare several types (the `sdf` plugin declares four file formats *and* `Sdf_UsdzResolver`).

**Technical explanation.** Verified on USD 26.08 `usd-core`:

**Schemas.** `usdGeom` declares `UsdGeomCube` with `schemaKind: concreteTyped` and alias `Cube`. `GetPluginForType(Tf.Type.FindByName("UsdGeomCube")).name` is `usdGeom`. Custom schemas: Chapter 37.

**File formats.** `Sdf.FileFormat.FindAllFileFormatExtensions()` → `['usd', 'usda', 'usdc', 'usdz']`. `FindByExtension("usda") / FindById("usda")` return the format object. `GetPluginForType(Tf.Type.FindByName("SdfUsdaFileFormat")).name` is `sdf`. Custom formats: Chapter 38.

**Resolvers.** `GetPluginForType(Tf.Type.FindByName("ArDefaultResolver")).name` is `ar`. `Sdf_UsdzResolver` is an `ArPackageResolver` for `@foo.usdz[bar.usda]` (Ch 31, Ch 33). Custom resolvers: Chapters 33 and 38.

**Hydra.** This wheel still registers plugin `usdHydra` (shader discovery / generative-procedural API). It does **not** ship `UsdImaging` or a render window. Scene-index plugins (Obj 3.7) are Chapter 38; they need a full imaging build (Ch 35).

Decision table:

You need to...	Plugin job	bases
Add <code>def Door</code> with schema properties	Schema	UsdTyped or UsdAPISchemaBase
Open <code>.mygeo</code> as a USD layer	File format	SdfFileFormat
Map <code>@chair@</code> to a database	Resolver	ArResolver
Generate Hydra prims at render	Scene index (Hydra)	Hydra scene-index base (Ch 38)

### Mental model.

schema	--> what a prim is (Cube, PhysicsRigidBodyAPI)
file format	--> how a file is parsed (.usda, .usdc, .usdz)
resolver	--> how @path@ is found
Hydra	--> how a renderer sees it

**Simple example.** Someone wants `.glTF` to `Usd.Stage.Open` without a converter. That is a **file format** plugin (`SdfFileFormat`), not a resolver (the path already points at a file) and not a schema (`glTF` already has meshes). A converter (Ch 27) is the other option if you do not want a plugin.

**USDA example.** Four jobs, one tiny stage: a schema type, a format implied by the file, an asset path for `Ar`, no `Hydra` in the file.

```
#usda 1.0
(
 defaultPrim = "Box"
)

def Cube "Box"
{
 asset preview = @./preview.png@
}
```

- `Cube` — schema plugin `usdGeom`.
- `.usda` — file-format plugin `sdf`.
- `@./preview.png@` — resolver plugin `ar` (default filesystem).
- Drawing the cube — `Hydra` / `UsdImaging` (not in `usd-core`).

### Python example.

```
from pxr import Plug, Tf, Sdf

reg = Plug.Registry()

def owner(type_name):
 t = Tf.Type.FindByName(type_name)
 plug = reg.GetPluginForType(t)
 return plug.name if plug else None

print("Cube owner", owner("UsdGeomCube"))
print("USDA format owner", owner("SdfUsdaFileFormat"))
print("default resolver owner", owner("ArDefaultResolver"))
print("extensions", sorted(Sdf.FileFormat.FindAllFileFormatExtensions()))
print("usda by ext",
 Sdf.FileFormat.FindByExtension("usda").formatId)
print("usdc by id", Sdf.FileFormat.FindById("usdc").formatId)

g = reg.GetPluginWithName("usdGeom")
print("usdGeom declares Material",
 g.DeclaresType(Tf.Type.FindByName("UsdShadeMaterial")))
print("has usdHydra",
 "usdHydra" in [p.name for p in reg.GetAllPlugins()])
print("has usdPhysics",
 "usdPhysics" in [p.name for p in reg.GetAllPlugins()])
```

### Expected output

```
Cube owner usdGeom
USDA format owner sdf
default resolver owner ar
extensions ['usd', 'usda', 'usdc', 'usdz']
usda by ext usda
usdc by id usdc
usdGeom declares Material False
has usdHydra True
has usdPhysics True
```

`UsdShadeMaterial` lives in plugin `usdShade`, not `usdGeom`. That is `DeclaresType` earning its keep.



**Real-world use case.** A CAD vendor ships: (1) a **file format** plugin so `.sldprt` opens as a stage, (2) a **schema** for CAD identifiers, (3) nothing in Ar if paths are ordinary files. A cloud studio instead ships only an **ArResolver** so `@part/bolt@` hits object storage, and they convert CAD offline (Ch 27). Pick the job from the problem, then build against 26.08 (Ch 35).

### Common mistakes.

#### MISTAKE

Writing a resolver to "add a Door prim." Resolvers find files; schemas define prims.

#### MISTAKE

Writing a file-format plugin to "search `/opt/assets/`." That is Ar search paths / a custom resolver (Ch 33).

#### MISTAKE

Expecting `usdHydra` in this wheel to open usdview. Discovery of a Hydra *\*schema\** is not an imager.

### Exam traps.

#### TRAP

"All plugins subclass `UsdTyped`." Only typed schemas do.

#### TRAP

"`.usd` is a fourth encoding plugin besides USDA/USDC/USDZ." `.usd` is `SdfUsdFileFormat` (`formatId`usd``); it *\*contains\** USDA or crate (Ch 31).

#### TRAP

Obj 3.7 (scene index) is satisfied by `plugInfo.json` for `usdGeom`. Scene indices are a Hydra plugin family, Chapter 38.

### Exam takeaways.

#### KEY

- Four jobs: schema, file format, resolver, Hydra — distinguished by `bases`. - Built-in extensions: `usd`, `usda`, `usdc`, `usdz` from plugin `sdf`. - `GetPluginForType` tells you who owns a Tf type. - Obj 3.1: build *\*that\** job's plugin against the host USD version.

## Chapter lab(s)

**Lab 31** (codeless custom schema, Chapter 37) writes a `plugInfo.json` and a `schema.usda`. This chapter is the registry theory that lab needs. Lab 32 covers custom kinds and fallbacks (Chapter 38).

## USDA reading exercises

**Exercise 36-A.** A `plugInfo.json` type has `"bases": ["SdfFileFormat"], "extensions": ["abc"], "formatId": "abc"`. What can a user do after this plugin loads? What can they *not* do with this plugin alone?

**Exercise 36-B.** `GetAllPlugins()` on a farm box does not list `StudioResolver`. `plugInfo.json` exists at `/opt/studio/plugin/pluginInfo.json`. Give two configuration checks before you rebuild C++.

## Answers

- **36-A.** They can `Stage.Open("foo.abc")` if the plugin's reader understands Alembic-or-whatever bytes. They cannot resolve `@hero@` from a database (needs `ArResolver`) and they do not get a new `def Door` schema.
- **36-B.** (1) Is `/opt/studio/plugin` on `PXR_PLUGINPATH_NAME` (or passed to `RegisterPlugins`)? (2) Was the env var set **before** USD started? Also confirm the JSON `Name` matches what you query.

## Chapter review

### Summary

- Plugins are discovered via `pluginInfo.json`; `Info.Types` + bases declare the job.
- `Plug.Registry` lists them; `RegisterPlugins` / `PXR_PLUGINPATH_NAME` add more.
- This wheel: 27 built-ins; `LibraryPath` often empty; resource plugins need no `.so`.
- Schema → `usdGeom` / `usdPhysics` / ...; formats → `sdf`; resolver → `ar`; Hydra → `usdHydra` (no `UsdImaging` here).
- Obj 3.1 = matching ABI (Ch 35) **plus** this discovery story.

### If you see... → think...

If you see...	Think...
<code>pluginInfo.json</code>	Plugin catalog, not USDA
<code>bases: ["SdfFileFormat"]</code>	New file extension
<code>bases: ["ArResolver"]</code>	Custom <code>@path@</code> lookup
<code>schemaKind: concreteTyped</code>	<code>def Cube</code> -style <code>IsA</code> schema
Type missing in DCC	<code>PXR_PLUGINPATH_NAME</code> / version / <code>GetAllPlugins</code>
<code>RegisterPlugins</code>	Runtime discovery
Empty <code>LibraryPath</code> in <code>usd-core</code>	Normal for wheel built-ins

### Review questions

**R36-1** (Obj 3.1) `pluginInfo.json` Type for a plugin that only ships files, no `.so`, is: A. `library` B. `resource` C. `usda` D. `component`

**R36-2** (Obj 3.1) Select two `Plug.Registry` queries. A. `GetAllPlugins()` B. `GetPluginWithName("usdGeom")` C. `Usd.Stage.Flatten` D. `ComputeAllDependencies`

**R36-3** (Obj 3.1) `GetStringFromPluginMetaData(UsdGeomCube, "schemaKind")` in this venv: A. `concreteTyped` B. `singleApplyAPI` C. `component` D. `usdc`

**R36-4** (Obj 3.1) `PXR_PLUGINPATH_NAME` should list: A. Shared-library file names B. Directories containing `pluginInfo.json` C. USDA layer identifiers D. Prim paths

**R36-5** (Obj 3.1) Select two plugin jobs and their owners here. A. File formats — `sdf` B. Default resolver — `ar` C. Cube schema — `sdf` D. USDA format — `usdGeom`

**R36-6** (Obj 3.1) `RegisterPlugins` on a folder with a valid resource `pluginInfo.json`: A. Raises unless a `.so` exists B. Returns a list of newly discovered `Plugin` objects C. Replaces the 27 built-ins D. Sets `defaultPrim`

**R36-7** (Obj 3.1) `usdGeom.DeclaresType(UsdShadeMaterial)` is `False` because: A. Materials are illegal B. That type is declared by plugin `usdShade` C. `pluginInfo.json` cannot list API schemas D. `usd-core` has no `Shade`

**R36-8** (Obj 3.1) You want `@part/bolt@` to hit a database. `bases` should include: A. `SdfFileFormat` B. `ArResolver` C. `UsdGeomGprim` D. `SdrDiscoveryPlugin`

**R36-9** (Obj 3.1) Select two true statements about this wheel. A.

FindAllFileFormatExtensions() is used, usda, usdc, usdz B. usdHydra appears in GetAllPlugins() C. UsdImaging imports successfully D. LibraryPath empty means discovery failed

**R36-10** (Obj 3.1) Discovery vs load: A. They are the same B. Discovery reads JSON; Load() brings in the library C. Load() reads USDA D. Discovery requires Hydra

**R36-11** (Obj 3.1) A root pluginInfo.json containing only "Includes": ["\*/resources/"]: A. Is invalid B. Points the registry at nested plugin JSON files C. Registers a Cube D. Sets PXR\_PLUGINPATH\_NAME

**R36-12** (Obj 3.1) Building a plugin “against a given version” (Obj 3.1) requires: A. Matching USD headers/libs (Ch 35) and a pluginInfo.json the host can discover B. Only renaming the JSON Name to 26.08 C. Flattening the plugin into the shot D. A URI scheme in every asset path

### Review answers

- **R36-1: B.** Review: §36.1, §36.2.
- **R36-2: A and B.** Review: §36.2.
- **R36-3: A.** Review: §36.1.
- **R36-4: B.** Review: §36.2.
- **R36-5: A and B.** Review: §36.3.
- **R36-6: B.** Review: §36.2.
- **R36-7: B.** Review: §36.3.
- **R36-8: B.** Review: §36.3.
- **R36-9: A and B.** Review: §36.2–36.3, Ch 35.
- **R36-10: B.** Review: §36.2.
- **R36-11: B.** Review: §36.1.
- **R36-12: A.** Review: §36.1, Ch 35.

## CHAPTER 37

## Chapter 37 — Custom Schemas

**Exam domain:** Customizing USD (6%), also Data Exchange (15%) · **Objectives:** 3.4, 3.6, 4.5 · **Study day:** 11 · **Est. time:** 85 min **Prerequisites:** Ch 6 (typed vs API schemas), Ch 29 (custom attributes), Ch 36 (plugInfo.json)

A **custom schema** is a contract *you* add: a new typed prim (def Door) or an API you Apply (LabelAPI). Obj 3.4 / 3.6 / 4.5 all ask when to use a schema instead of a one-off custom attribute, how schema.usda and usdGenSchema fit, and which **base class** to inherit. This book's usd-core has **no** usdGenSchema; you still must read schema.usda and use Usd.SchemaRegistry on the built-ins.

## Learning goals

- Decide when a custom schema is worth it versus a namespaced custom attribute.
- Read and write a minimal schema.usda (class, inherits, GLOBAL).
- Contrast **codeless** vs **codeful** output of usdGenSchema.
- Choose UsdTyped (IsA) vs UsdAPISchemaBase (Apply).
- Carry nonstandard DCC fields through a schema or a documented custom attribute (Obj 4.5).

## Key terms

Term	One-line definition
schema.usda	The USDA source file usdGenSchema reads to generate code and plugInfo.json
usdGenSchema	Tool that compiles schema.usda into C++/Python wrappers and plugin metadata
Codeless schema	Schema used at runtime from generated USDA + plugInfo.json, <b>no</b> C++ types
Codeful schema	Schema that generates C++ classes you compile against a USD version
UsdTyped	Base for IsA / typed schemas (def Cube)
UsdAPISchemaBase	Base for API schemas you Apply
Usd.SchemaRegistry	Query concrete vs abstract vs applied vs multiple-apply
skipCodeGeneration	customData flag in schema.usda for a codeless type

## 37.1 When to create a schema

**What is it?** Creating a schema means publishing a **named contract** (typed prim or API) that many tools will honor. It is not the same as adding one custom float studio:weight on one chair.

**Why do we need it?** Custom attributes are fast (Ch 29). They do not give you Apply, HasAPI, generated accessors, or usdchecker-style validators that know the field. When three DCCs and a farm all need partId, a schema stops the name from drifting (part\_id vs partId vs pn).

**Beginner explanation.** A sticky note on one box is a custom attribute. A printed form every warehouse uses is a schema. Print the form when the sticky note is copied so often that people misspell it.

*Where the analogy breaks:* a schema still *stores* ordinary USD properties. You are not inventing a new file format (that is Ch 38). You are naming a bundle of properties and (for typed schemas) a type name.

**Technical explanation.** Use a **custom attribute** (or `customData`) when:

- One exporter, one shot, or a temporary experiment
- The name is already namespaced (`studio:cadId`) and documented
- You do not need `HasAPI` / generated `CreatePartIdAttr`

Use a **custom schema** when:

- Several tools must create, validate, and query the same fields (Obj 3.4, 3.6)
- You want `def Door` in USDA (typed / `IsA`) or `apiSchemas = ["LabelAPI"]` (API)
- You will write a validator that looks for that schema (Ch 30)
- Import/export of a DCC family should round-trip through `Apply` (Obj 4.5)

Cost: `schema.usda`, `usdGenSchema`, a plugin (Ch 36), and — if codeful — a rebuild against each USD version (Ch 35). That cost is why NVIDIA still expects you to *start* with custom attributes.

### Mental model.

one tool, one field	--> custom <code>studio:weight</code>
many tools, a bundle	--> schema (API or typed)
new encoding on disk	--> file-format plugin (Ch 38), not a schema

**Simple example.** Factory CAD: first week, custom string `studio:partId`. Six months later every DCC and the MES need it → `LabelAPI` with `studio:partId`, `usdGenSchema`, plugin on `PXR_PLUGINPATH_NAME`.

**USDA example.** The *before* (no schema). Legal, portable, and what Ch 29 taught.

```
#usda 1.0
(
 defaultPrim = "Chair"
)

def Xform "Chair" (
 kind = "component"
)
{
 custom string studio:partId = "PN-42"
 custom float studio:weight = 4
}
```

### Python example.

```
from pxr import Usd, UsdGeom, Sdf

stage = Usd.Stage.CreateInMemory()
prim = UsdGeom.Xform.Define(stage, "/Chair").GetPrim()
attr = prim.CreateAttribute(
 "studio:partId", Sdf.ValueTypeNames.String, custom=True)
attr.Set("PN-42")
print("custom", attr.IsCustom())
print("value", attr.Get())
print("HasAPI LabelAPI", prim.HasAPI(Usd.APISchemaBase))
print("typeName", prim.GetTypeName())
```

### Expected output

```
custom True
value PN-42
HasAPI LabelAPI False
typeName Xform
```

`HasAPI(Usd.APISchemaBase)` is not a substitute for a real `LabelAPI`. The attribute works; nothing in the registry yet knows `LabelAPI`.

**Real-world use case.** A robotics team stored joint limits as `customData` for a year. When two engines needed the same limits, they moved to `UsdPhysics` (already a schema) plus a thin `StudioJointAPI` for extra fields. The `customData` leftovers are stripped on publish (Ch 34).

### Common mistakes.

#### MISTAKE

Creating a typed schema ``def Weight "W"`` for a single float. An API or a custom attribute is enough.

#### MISTAKE

A new schema for every DCC version. Namespace the attribute until the contract is stable.

#### MISTAKE

Using a schema plugin as a file format. Schemas do not parse ``.sldprt``.

### Exam traps.

#### TRAP

"Obj 4.5 always requires ``usdGenSchema`` on the exam machine." You must *\*choose\** schema vs custom attribute. ``usd-core`` does not ship the generator.

#### TRAP

"Custom attributes are illegal in production." They are the right first step.

### Exam takeaways.

#### KEY

- Custom attribute first; schema when many tools share a bundle. - Schema  $\neq$  file format  $\neq$  resolver. - Obj 4.5 is the *\*choice\**, not a requirement to run ``usdGenSchema`` in ``usd-core``.

## 37.2 `schema.usda`

**What is it?** `schema.usda` is an USDA file that *describes* schemas for code generation. It is not a shot. It uses `class` prims, `inherits`, and an over `"GLOBAL"` block with `libraryName` / `libraryPath`.

**Why do we need it?** One text file is the source of truth for property names, fallbacks, and inheritance. `usdGenSchema` reads it so C++, Python, and `plugInfo.json` cannot drift.

**Beginner explanation.** `schema.usda` is the blank form template. `usdGenSchema` is the print shop. Shots are filled-in copies (`def Cube "Box"`). You do not hand the blank form to lighting as a set file.

*Where the analogy breaks:* `class Cube "Cube"` in the *official* `usdGeom/schema.usda` is the generator's class, not a LIVERPS class you inherit in a shot (Ch 19). Same USDA keyword; different pipeline role.

**Technical explanation.** Verified pattern (Pixar `usdGeom/schema.usda`, USD 26.08):

- Layer opens with `#usda 1.0` and often sublayers `usd/schema.usda` (for `Typed`, `Gprim`, ...).
- over `"GLOBAL"` holds `customData.libraryName` and `libraryPath`.
- Each schema is a **class** prim: `class Cube "Cube" ( inherits = </Gprim> )`.
- Properties inside the class become schema attributes (`double size = 2.0`).
- doc metadata becomes C++ / Python documentation.
- Extra `customData` can set `implementsComputeExtent`, `apiSchemaType`, `skipCodeGeneration`.

A **minimal** file that still **parses** as USDA (this is teaching-sized, not a full Pixar library): Typed schema class inherits `</Typed>` (the `UsdTyped` prim in `usd/schema.usda`). API schema class inherits `</APISchemaBase>`.

Parsing `schema.usda` with `Sdf` does **not** register the schema. Registration needs `usdGenSchema` output in a plugin (Section 37.3).

### Mental model.

```
schema.usda --usdGenSchema--> generatedSchema.usda
 pluginInfo.json
 C++ (if codeful)
shots use: def Cube "Box" or apiSchemas = ["LabelAPI"]
```

**Simple example.** Studio `schema.usda` defines class `"Door"` with `bool sliding` and `float width`. After generation, artists write `def Door "Hall"` in sets.

**USDA example.** File: `schema.usda` (minimal, parses as USDA; not installed as a plugin here)

```
#usda 1.0

over "GLOBAL" (
 customData = {
 string libraryName = "studio"
 string libraryPath = "studio"
 }
)
{
}

class "Door" (
 inherits = </Typed>
 doc = "A studio door prim."
)
{
 bool sliding = 0
 float width = 0.9
}

class "LabelAPI" (
 inherits = </APISchemaBase>
 doc = "A single-apply API for a part label."
)
{
 string studio:partId = ""
}
```

- `GLOBAL` — library identity for the generator.
- class `"Door"` + `inherits = </Typed>` — IsA schema (Section 37.4).
- class `"LabelAPI"` + `</APISchemaBase>` — API schema.
- Fallbacks (`sliding = 0`, `width = 0.9`) become schema defaults, like Cube's `size = 2`.

### Python example.

```
from pxr import Sdf

text = """#usda 1.0

over "GLOBAL" (
 customData = {
 string libraryName = "studio"
 string libraryPath = "studio"
 }
)
{
}

class "Door" (
```

```

 inherits = </Typed>
)
 {
 bool sliding = 0
 float width = 0.9
 }

class "LabelAPI" (
 inherits = </APISchemaBase>
)
{
 string studio:partId = ""
}
"""
layer = Sdf.Layer.CreateAnonymous(".usda")
print("parsed", layer.ImportFromString(text))
for spec in layer.rootPrims:
 inh = [str(p) for p in spec.inheritPathList.GetAddedOrExplicitItems()]
 print(spec.name, str(spec.specifier), "inherits", inh)

```

### Expected output

```

parsed True
GLOBAL Sdf.SpecifierOver inherits []
Door Sdf.SpecifierClass inherits ['/Typed']
LabelAPI Sdf.SpecifierClass inherits ['/APISchemaBase']

```

Sdf sees classes and inherits. The **SchemaRegistry** still does not know `Door` until this file is generated into a plugin.

**Real-world use case.** Pixar's own `usdGeom/schema.usda` is thousands of lines. `class Cube "Cube" ( inherits = </Gprim> )` with `double size = 2.0` is why `FindConcretePrimDefinition("Cube")` reports fallback size 2 (next sections).

### Common mistakes.

#### MISTAKE

Referencing ``schema.usda`` from a shot as a sublayer. It is generator input, not scene data.

#### MISTAKE

Using ``def Door`` in ``schema.usda`` instead of ``class``. The generator looks for schema classes.

#### MISTAKE

Forgetting ``over "GLOBAL" `libraryName``. ``usdGenSchema`` needs the library identity.

### Exam traps.

#### TRAP

"``inherits = `` in `schema.usda` is the same as a shot ``inherits`` class for broadcast edits." Shot inherits are LIVERPS (Ch 19). Schema inherits are the **C++ / registry** base.

#### TRAP

"If Sdf parses ``schema.usda``, ``def Door`` works in `usdview`." Parse  $\neq$  register.

### Exam takeaways.

#### KEY

- ``schema.usda``: ``GLOBAL`` + ``class`` + ``inherits`` (``/Typed`` or ``/APISchemaBase``). - Sdf can parse it; the registry ignores it until ``usdGenSchema`` + plugin. - Do not sublayer ``schema.usda`` into shots.



### 37.3 usdGenSchema; codeful vs. codeless schemas

**What is it?** `usdGenSchema` reads `schema.usda` and writes generated files: `generatedSchema.usda`, `plugInfo.json`, and (unless told not to) C++/Python wrappers. A **codeless** schema skips those wrappers (`skipCodeGeneration`) and is used from the generated USDA at runtime. A **codeful** schema compiles C++ classes (`UsdStudioDoor`) against a USD version.

**Why do we need it?** Hand-writing `plugInfo.json` Types for every property is how names drift. The generator is the official bridge from `schema.usda` to the plugin system (Ch 36). Codeless lets you ship a schema **without** a C++ rebuild on every farm image; codeful gives typed Python/C++ APIs.

**Beginner explanation.** Codeless is a PDF form everyone fills in a viewer that already knows “PDF.” Codeful is a native app compiled for that OS. Both came from the same InDesign file (`schema.usda`).

*Where the analogy breaks:* codeless schemas still need a USD that can load the plugin’s `generatedSchema.usda`. They are not “just JSON.”

**Technical explanation.**

```
usdGenSchema schema.usda
```

Not installed with `usd-core` (`shutil.which` is `None` below). A full OpenUSD build provides it (Ch 35). Typical outputs:

Output	Role
<code>generatedSchema.usda</code>	Runtime schema definitions the plugin loads
<code>plugInfo.json</code>	Info.Types with <code>schemaKind</code> , <code>bases</code> , <code>aliases</code>
<code>door.cpp</code> / wrap files	Codeful only; compile with matching USD
Tokens header	Attribute name tokens

**Codeless:** on the `class` (or library), `customData` includes `skipCodeGeneration = true` (token/bool per official docs). Tools talk to the prim through generic `Usd.Prim / CreateAttribute`, or a thin Python helper. Faster to ship; weaker compile-time API.

**Codeful:** generate + compile `UsdTyped / UsdAPISchemaBase` subclasses.

`UsdStudioDoor.Define(stage, path)` looks like `UsdGeom.Cube.Define`. Must rebuild when USD’s ABI changes (Obj 3.1, Ch 35).

Both still need `PXR_PLUGINPATH_NAME / RegisterPlugins`.

#### VERSION

Verified: ``usdGenSchema`` is **not** on ``PATH`` in this ``usd-core` 26.8` venv. Teach the command as ``norun``. Built-in schemas (``Cube``, ``MaterialBindingAPI``) were generated by Pixar when *they* built USD.

#### Mental model.

```

schema.usda
 |
 v
usdGenSchema
 |
 |-- always: generatedSchema.usda + plugInfo.json
 |-- codeful: C++/Python classes (compile)
 |-- codeless: skipCodeGeneration, no compile

```

**Simple example.** A small studio ships `LabelAPI` as **codeless** so every DCC Python can Apply via the generic API once the plugin path is set. The engine team later promotes it to **codeful** for C++ accessors.

**USDA example.** Codeless marker on the class (still ordinary USDA `customData`):

```
#usda 1.0

class "LabelAPI" (
 inherits = </APISchemaBase>
 customData = {
 token skipCodeGeneration = "true"
 token apiSchemaType = "singleApply"
 }
)
{
 string studio:partId = ""
}
```

`usdGenSchema` would emit plugin metadata instead of `LabelAPI.cpp`.

### Python example.

```
import shutil
from pxr import Usd, UsdGeom

print("usdGenSchema", shutil.which("usdGenSchema"))
sr = Usd.SchemaRegistry()
print("Cube registered",
 sr.FindConcretePrimDefinition("Cube") is not None)
print("Cube size fallback",
 sr.FindConcretePrimDefinition("Cube").GetAttributeFallbackValue("size"))
print("Door registered",
 sr.FindConcretePrimDefinition("Door") is not None)
```

### Expected output

```
usdGenSchema None
Cube registered True
Cube size fallback 2.0
Door registered False
```

Pixar's generated Cube is in the wheel. Your Door from Section 37.2 is **not**, because we never ran `usdGenSchema` or loaded a plugin.

**Real-world use case.** A games company checks `generatedSchema.usda` and `plugInfo.json` into git (codeless) so QA machines without a C++ toolchain still load `DestructibleAPI`. The runtime engine uses a **codeful** copy compiled into the game for speed.

### Common mistakes.

#### MISTAKE

Editing ``generatedSchema.usda`` by hand and losing the edits on the next ``usdGenSchema``. Edit ``schema.usda``.

#### MISTAKE

Shipping codeful ``so`` built for 25.11 into 26.08 `usdview` (Ch 35).

#### MISTAKE

Expecting ``which usdGenSchema`` to work after ``pip install usd-core``.

### Exam traps.

#### TRAP

"Codeless means no ``plugInfo.json``." Codeless still needs the JSON and generated USDA.

#### TRAP

"Codeful schemas do not need a plugin path." They still discover via ``plugInfo.json``.

### Exam takeaways.

**KEY**

- `usdGenSchema` turns `schema.usda` into plugin files (+ C++ if codeful). - Codeless = no generated C++; still a plugin. Codeful = compile against USD. - This venv: generator absent; built-in Cube fallback `size` is `2.0`.

**37.4 IsA (UsdTyped) vs. API (UsdAPISchemaBase) base classes**

**What is it?** When you author `schema.usda`, you pick a **base class**. **UsdTyped** (schema inherit `</Typed>`) makes an IsA type: `def Door`. **UsdAPISchemaBase** (inherit `</APISchemaBase>`) makes an API you **Apply** onto an existing typed prim. This is the exam's favorite custom-schema fork (Obj 3.4, 3.6).

**Why do we need it?** Physics properties on a mesh should not force `def PhysicsMesh`. They **Apply** onto `Mesh / Cube` (`UsdPhysics.RigidBodyAPI`, Ch 35). A brand-new thing that is a door gets `def Door` from `UsdTyped`.

**Beginner explanation.** `UsdTyped` is a new species. `UsdAPISchemaBase` is a collar you buckle onto any animal. Collars compose (`RigidBodyAPI` + `CollisionAPI` + `MaterialBindingAPI` on one `Cube`). Species is one `typeName`.

*Where the analogy breaks:* abstract typed schemas (`Gprim`) are species that you never instantiate; they only exist so `Cube/Sphere` inherit them. `IsAbstract` is `True`, `IsConcrete` is `False`.

**Technical explanation.** `Usd.SchemaRegistry` queries (verified 26.08):

Question	Cube	Gprim	GeomModelAPI	CollectionAPI	MaterialBindi
<code>IsTyped</code>	<code>True</code>	<code>True</code>	<code>False</code>	<code>False</code>	<code>False</code>
<code>IsConcrete</code>	<code>True</code>	<code>False</code>	<code>False</code>	<code>False</code>	<code>False</code>
<code>IsAbstract</code>	<code>False</code>	<code>True</code>	—	—	—
<code>IsAppliedAPISchema</code>	<code>False</code>	<code>False</code>	<code>True</code>	<code>True</code>	<code>True</code>
<code>IsMultipleApplyAPISchema</code>	<code>False</code>	<code>False</code>	<code>False</code>	<code>True</code>	<code>False</code>
<code>GetSchemaKind</code>	<code>ConcreteTyped</code>	<code>abstract typed</code>	<code>SingleApplyAPI</code>	<code>MultipleApplyAPI</code>	<code>SingleApplyA</code>

Python:

- Typed: `UsdGeom.Cube.Define` → `prim.IsA(UsdGeom.Cube)`
- Single-apply API: `UsdShade.MaterialBindingAPI.Apply(prim)` → `HasAPI(...)` (must `Apply`; Ch 29)
- Multiple-apply: `Usd.CollectionAPI.Apply(prim, "include")` → `CollectionAPI:include`

**Rule of thumb for custom schemas:** extra fields on existing geometry → **API** (`UsdAPISchemaBase`). A new first-class object in the outliner → **typed** (`UsdTyped`). NVIDIA's physics-on-geometry example is the API fork.

`FindSchemaInfo(UsdGeom.Cube).kind` is `ConcreteTyped`; `.identifier` / `.family` are `Cube`.

**Mental model.**

`typeName`: one IsA (`UsdTyped`, `concrete`)  
`apiSchemas`: zero or more applied APIs (`UsdAPISchemaBase`)

**Simple example.** `Door` as typed: artists `def Door "Hall"`. `LabelAPI` as API: `def Mesh "Crate" plus LabelAPI.Apply`. Putting `partId` on a new `def PartIdPrim` would pollute the model hierarchy.

**USDA example.**

```
#usda 1.0

def Mesh "Crate" (
 prepend apiSchemas = ["MaterialBindingAPI"]
)
{
}

def Cube "Box"
{
 double size = 2
}
```

Mesh *is-a* Mesh (typed). MaterialBindingAPI is applied (API). Cube is concrete typed; you never def Gprim.

### Python example.

```
from pxr import Usd, UsdGeom, UsdShade, Tf

sr = Usd.SchemaRegistry()
cube_t = UsdGeom.Cube
gprim_t = Tf.Type.FindByName("UsdGeomGprim")
print("Cube IsConcrete", sr.IsConcrete(cube_t),
 "IsTyped", sr.IsTyped(cube_t))
print("Gprim IsConcrete", sr.IsConcrete(gprim_t),
 "IsAbstract", sr.IsAbstract(gprim_t),
 "IsTyped", sr.IsTyped(gprim_t))
print("ModelAPI applied", sr.IsAppliedAPISchema(UsdGeom.ModelAPI),
 "multi", sr.IsMultipleApplyAPISchema(UsdGeom.ModelAPI))
print("Collection multi", sr.IsMultipleApplyAPISchema(Usd.CollectionAPI))
print("Binding applied",
 sr.IsAppliedAPISchema(UsdShade.MaterialBindingAPI))
info = sr.FindSchemaInfo(UsdGeom.Cube)
print("Cube kind", info.kind, "family", info.family)

stage = Usd.Stage.CreateInMemory()
prim = UsdGeom.Mesh.Define(stage, "/M").GetPrim()
UsdShade.MaterialBindingAPI.Apply(prim)
print("IsA Mesh", prim.IsA(UsdGeom.Mesh),
 "IsA Cube", prim.IsA(UsdGeom.Cube))
print("HasAPI binding", prim.HasAPI(UsdShade.MaterialBindingAPI))
print("applied", list(prim.GetAppliedSchemas()))
```

### Expected output

```
Cube IsConcrete True IsTyped True
Gprim IsConcrete False IsAbstract True IsTyped True
ModelAPI applied True multi False
Collection multi True
Binding applied True
Cube kind ConcreteTyped family Cube
IsA Mesh True IsA Cube False
HasAPI binding True
applied ['MaterialBindingAPI']
```

**Real-world use case.** USD Physics: bodies stay Cube/Mesh (typed geom) plus RigidBodyAPI / CollisionAPI (applied). That is the pattern the exam wants copied for “add friction to existing meshes.”

### Common mistakes.

#### MISTAKE

Inheriting `UsdTyped` for "has a part id." You just forced a new type name on every labeled mesh.

#### MISTAKE

Forgetting `Apply` on an API schema. `HasAPI` stays False (Ch 29, Ch 40).

**MISTAKE**

```
`def Gprim "Thing"`. Gprim is abstract.
```

**Exam traps.****TRAP**

```
"`IsA` tests API schemas." `IsA` is typed inheritance. APIs use `HasAPI`.
```

**TRAP**

```
"CollectionAPI is single-apply because it is an API." It is **multiple-apply**.
```

**TRAP**

```
"Concrete means 'has C++ code'." Concrete means instantiable typed schema (`def Cube`).
Codeless typed schemas can still be concrete.
```

**Exam takeaways.****KEY**

- New species → `UsdTyped` / `def Door`. Extra collar → `UsdAPISchemaBase` / `Apply`. -  
`IsA` vs `HasAPI`; concrete vs abstract; single vs multiple apply. - Physics-on-geometry is  
the API pattern.

**37.5 Schemas for nonstandard import/export data**

**What is it?** Obj 4.5 / 3.6: when a DCC has data USD does not model (CAD id, PLC tag, proprietary shader knob), you either **map it to a custom attribute** or **wrap it in a schema** so round-trips do not invent a new name each time.

**Why do we need it?** Chapter 29 showed `studio:partId` as a custom attribute. That is correct until the importer must *find* every labeled part: then `HasAPI(LabelAPI)` is faster and safer than string-matching attribute names.

**Beginner explanation.** The factory stamp on a crate is either a Sharpie (custom attribute) or an official barcode format (schema). Shipping with a Sharpie works. A barcode reader in every warehouse wants the official format.

*Where the analogy breaks:* both still live on the same USD prim. The schema does not move the data to a side-car file.

**Technical explanation.** Exporter policy (document it, Ch 32):

Data	Mechanism
One-off / unstable	custom namespaced attribute ( <code>studio:...</code> )
Stable bundle, many tools	API schema <code>Apply</code> + generated properties
New outliner object	Typed schema
Whole other file type	File-format plugin (Ch 38), not a schema

Round-trip test (Ch 35): after DCC import/export, `HasAPI` and the attribute `Get()` still match. Flatten must not be the default Save (Ch 34) or the API application can be baked in a way that loses the plugin dependency — still keep `apiSchemas` in the layer.

You can mix: `def Mesh` (built-in typed) + `LabelAPI` (custom API) + `custom float studio:debug` (ephemeral).

**Mental model.**

```
DCC extra fields
|
+-- unstable --> custom studio:*
+-- stable --> Apply(LabelAPI) / def Door
+-- file --> converter or SdfFileFormat
```

**Simple example.** CAD `cadId` is stable → `LabelAPI.studio:partId`. A debug tessellation count that changes every export → `custom int studio:tessDebug`.

**USDA example.** Mesh (built-in typed) plus an applied built-in API, plus a custom attribute still waiting to become `LabelAPI`:

```
#usda 1.0

def Mesh "Crate" (
 prepend apiSchemas = ["MaterialBindingAPI"]
)
{
 custom string studio:partId = "PN-42"
}
```

After the studio ships `LabelAPI`, the same file grows `LabelAPI` in `apiSchemas` and `studio:partId` becomes a **schema** property (often no longer marked `custom`).

### Python example.

```
from pxr import Usd, UsdGeom, UsdShade, Sdf

stage = Usd.Stage.CreateInMemory()
prim = UsdGeom.Mesh.Define(stage, "/Crate").GetPrim()
UsdShade.MaterialBindingAPI.Apply(prim)
prim.CreateAttribute(
 "studio:partId", Sdf.ValueTypeNames.String, custom=True).Set("PN-42")

print("type", prim.GetTypeName())
print("applied", list(prim.GetAppliedSchemas()))
print("partId custom",
 prim.GetAttribute("studio:partId").IsCustom())
print("partId", prim.GetAttribute("studio:partId").Get())

Simulate a DCC reopen from a layer string.
text = stage.GetRootLayer().ExportToString()
again = Usd.Stage.CreateInMemory()
again.GetRootLayer().ImportFromString(text)
p2 = again.GetPrimAtPath("/Crate")
print("reopen type", p2.GetTypeName())
print("reopen applied", list(p2.GetAppliedSchemas()))
print("reopen partId", p2.GetAttribute("studio:partId").Get())
```

### Expected output

```
type Mesh
applied ['MaterialBindingAPI']
partId custom True
partId PN-42
reopen type Mesh
reopen applied ['MaterialBindingAPI']
reopen partId PN-42
```

If a DCC drops unknown `custom` attributes or `apiSchemas`, this reopen test fails. That is the Obj 4.5 regression.

**Real-world use case.** A PLC exporter writes `studio:tag` on every motor. When the digital-twin standard froze, they generated `FactoryTagAPI` (codeless) and the exporter switched from `CreateAttribute` to `Apply` + schema property. Old files still open: the importer accepts *either* the custom attribute or the API until the cut-over date.

### Common mistakes.

#### MISTAKE

Deleting unknown ``custom`` attributes on import "to keep the scene clean." You just broke the round-trip (Ch 29, Ch 35).

#### MISTAKE

Putting CAD ids in prim names. Names are identifiers, not data (Ch 28, Ch 32).

**MISTAKE**

A typed schema per CAD part instance. One Mesh + API per part, or a schema on the component root.

**Exam traps.****TRAP**

"Nonstandard data must be a new file format." Only if the `*bytes*` are not USD. Fields inside USD are attributes/schemas.

**TRAP**

"``custom`` keyword is required on schema properties." Generated schema properties are usually ordinary (non-custom) attributes.

**Exam takeaways.****KEY**

- Nonstandard `*fields*` → attribute or schema; nonstandard `*files*` → format plugin. - Round-trip: keep ``apiSchemas`` and unknown ``studio:*`` customs. - Promote custom attributes to an API when the contract freezes.

**Chapter lab(s)**

**Lab 31** — Codeless custom schema: write `schema.usda` + `plugInfo.json` structure (even if this venv cannot run `usdGenSchema`), and round-trip a `studio:partId` attribute as in §37.5.

**USDA reading exercises**

**Exercise 37-A.** A schema file contains `class "Bolt" ( inherits = </APISchemaBase> )` and someone authors `def Bolt "B" in a shot`. What went wrong?

**Exercise 37-B.** `FindConcretePrimDefinition("Door")` is `None` after `Sdf` parsed the `schema.usda` from §37.2. Why?

**Answers**

- **37-A.** `APISchemaBase` is for **Apply**, not `def Bolt`. Use `inherits = </Typed>` for a typed `def Bolt`, or `def Mesh + BoltAPI.Apply`.
- **37-B.** `Parse` ≠ `register`. Need `usdGenSchema` output loaded as a plugin.

**Chapter review****Summary**

- Custom attribute first; schema when many tools share a bundle (Obj 3.4, 4.5).
- `schema.usda`: `GLOBAL`, `class`, `inherits /Typed` or `/APISchemaBase`.
- `usdGenSchema` is absent in `usd-core`; codeless vs codeful is still exam material.
- `UsdTyped = IsA`; `UsdAPISchemaBase = Apply`; `Gprim` is abstract; `Collection` is multiple-apply.
- Round-trip nonstandard data: keep APIs and `studio:*` customs.

**If you see... → think...**

If you see...	Think...
Physics on a Mesh	API schema, not a new type
<code>def Door</code>	<code>UsdTyped</code> / concrete
<code>apiSchemas = ["LabelAPI"]</code>	<code>UsdAPISchemaBase</code> + <code>Apply</code>

If you see...	Think...
usdGenSchema on usd-core	Not installed
schema.usda sublayered in a shot	Wrong role
Cube size fallback 2	PrimDefinition
One-off CAD debug field	custom attribute

### Review questions

**R37-1** (Obj 3.4) Extra friction on existing meshes. Inherit: A. UsdTyped B.

UsdAPISchemaBase C. SdfFileFormat D. UsdGeomGprim as a def Gprim

**R37-2** (Obj 3.4) Select two schema.usda facts. A. Typed schemas inherits = </Typed> B. API schemas inherits = </APISchemaBase> C. Shot defaultPrim belongs in GLOBAL D. def is preferred over class for generator input

**R37-3** (Obj 3.6) Codeless schemas: A. Need no plugInfo.json B. Skip C++ generation; still load via a plugin C. Cannot be applied D. Are file formats

**R37-4** (Obj 4.5) Select two round-trip keepers. A. MaterialBindingAPI in apiSchemas B. custom string studio:partId C. The session layer D. Absolute /mnt texture paths you meant to strip

**R37-5** (Obj 3.4) IsConcrete(Gprim) is False because: A. Gprim is an abstract typed schema B. Gprim is multiple-apply C. usd-core omitted geom D. Gprim is an API

**R37-6** (Obj 3.4) prim.IsA(UsdGeom.Cube) vs prim.HasAPI(UsdShade.MaterialBindingAPI): A. They are aliases B. IsA is typed; HasAPI is applied APIs C. HasAPI is only for kinds D. IsA tests plugInfo.json names

**R37-7** (Obj 3.6) usdGenSchema in this venv prints from shutil.which: A. /usr/bin/usdGenSchema B. None C. usdc D. True

**R37-8** (Obj 4.5) Best first encoding for an experimental DCC knob? A. Codeful typed schema rebuilt weekly B. Namespaced custom attribute C. New .knob file format plugin D. Prim rename

**R37-9** (Obj 3.4) Select two true 26.08 registry facts. A. CollectionAPI is multiple-apply B. MaterialBindingAPI is applied (single) C. Cube is abstract D. ModelAPI is typed concrete

**R37-10** (Obj 3.6) Editing generatedSchema.usda instead of schema.usda: A. Is the official workflow B. Will be overwritten the next usdGenSchema C. Registers Door without a plugin D. Sets skipCodeGeneration automatically

**R37-11** (Obj 4.5) After promoting studio:partId to LabelAPI, old files with only the custom attribute: A. Must be discarded B. Can be accepted by an importer that reads either form during cut-over C. Automatically gain HasAPI D. Become .usdz only

**R37-12** (Obj 3.4) FindSchemaInfo(UsdGeom.Cube).kind is: A. ConcreteTyped B. SingleApplyAPI C. component D. usdc

### Review answers

- **R37-1: B.** Review: §37.4.
- **R37-2: A and B.** Review: §37.2.
- **R37-3: B.** Review: §37.3.
- **R37-4: A and B.** Review: §37.5.
- **R37-5: A.** Review: §37.4.
- **R37-6: B.** Review: §37.4.
- **R37-7: B.** Review: §37.3.
- **R37-8: B.** Review: §37.1.
- **R37-9: A and B.** Review: §37.4.
- **R37-10: B.** Review: §37.3.



- **R37-11: B.** Review: §37.5.
- **R37-12: A.** Review: §37.4.

## CHAPTER 38

## Chapter 38 — Kinds, Variant Fallbacks, File Formats, Resolvers, Scene Indices

**Exam domain:** Customizing USD (6%) · **Objectives:** 3.3, 3.5, 3.7, 3.8 · **Study day:** 11 · **Est. time:** 80 min **Prerequisites:** Ch 6 / 23 (kinds), Ch 18 (variants), Ch 33 (Ar), Ch 36 (plugInfo), Ch 37 (schemas)

Chapter 36 named the four plugin jobs. This chapter fills the remaining **customizing** knobs: **custom kinds** (Obj 3.3), **variant fallbacks**, **file-format plugins**, **custom resolvers** (Obj 3.5, 3.8), and **Hydra scene indices** (Obj 3.7). `usd-core` can prove kinds, fallbacks, and file-format queries. C++ resolvers and scene indices stay conceptual, with verified Python around the edges.

### Learning goals

- Query the built-in kind tree with `Kind.Registry` and explain how a custom kind is *declared* (Obj 3.3).
- Set process-wide variant fallbacks and know they lose to authored selections.
- Identify a file-format plugin (`SdfFileFormat`, `extensions`, `IsPackage`).
- Restate how a custom `ArResolver` is integrated (Obj 3.5) and what Obj 3.8 adds.
- Describe a Hydra scene-index plugin at concept level (Obj 3.7).

### Key terms

Term	One-line definition
<b>Kind registry</b>	Catalog of kind tokens and their <code>baseKind</code> ( <code>Kind.Registry</code> )
<b>Custom kind</b>	A studio token (e.g. <code>prop</code> ) whose base is a built-in kind
<b>Variant fallback</b>	Default variant choice used when the prim has <b>no</b> authored selection
<b>File-format plugin</b>	<code>SdfFileFormat</code> subclass registered for an extension
<b>Package format</b>	A format that contains other files (USDZ); <code>IsPackage()</code>
<b>Scene index</b>	Hydra graph node that can generate or filter render prims
<b>Procedural resolver</b>	Resolver that can produce bytes / prims without a normal file (Obj 3.8)

### 38.1 Custom model kinds (`Kind.Registry`)

**What is it?** **Kinds** label a prim’s role in the model hierarchy (`component`, `assembly`, `group`, `model`, `subcomponent`). A **custom kind** is an extra token you add to that catalog, always with a **base kind** so `IsA("prop", "component")` can still be true (Obj 3.3).

**Why do we need it?** `component` is too coarse for a studio that wants to filter “only props” in a layout tool. A custom kind `prop` with `baseKind = component` keeps model-hierarchy rules (Ch 23) and gives tools a finer label.

**Beginner explanation.** Built-in kinds are the Linnaean ranks: animal / mammal / dog. A custom kind is “terrier”: still a dog (`component`), just a studio nickname. Layout can say “select all terriers” without breaking `IsComponent()`.

*Where the analogy breaks:* you do **not** invent a rank *beside* `model` that is not based on `model/component/group` if you still want a valid model hierarchy. `subcomponent` is already “inside a component, not a model.”

**Technical explanation.** Verified on USD 26.08:

Call	Result here
<code>Kind.Registry.GetAllKinds()</code>	assembly, component, group, model, subcomponent
<code>GetBaseKind("component")</code>	model
<code>GetBaseKind("assembly")</code>	group
<code>GetBaseKind("group")</code>	model
<code>GetBaseKind("model")</code>	empty
<code>GetBaseKind("subcomponent")</code>	empty
<code>IsA("component", "model")</code>	True
<code>IsA("assembly", "model")</code>	True
<code>IsA("subcomponent", "model")</code>	<b>False</b>
<code>HasKind("door")</code>	False until a plugin declares it

**There is no `Kind.Registry.Register` in this Python.** Custom kinds are declared in `plugInfo.json`:

```
"Info": {
 "Kinds": {
 "prop": { "baseKind": "component" }
 }
}
```

A resource plugin registered at runtime in a 26.08 probe **did not** make `HasKind("prop")` True. Treat custom kinds as **startup** plugin data (often on the kind library's JSON), not as a live `Register` call.

Authoring still uses prim metadata: `Usd.ModelAPI(prim).SetKind("prop")` once the token exists. If the token is unknown, you can still *set the string*, but `IsA` / model queries will not treat it as a component.

### Mental model.

```
model
 component <-- custom "prop" should base here
 group
 assembly
 subcomponent (not a model; inside a component)
```

**Simple example.** Studio `plugInfo.json` adds `prop` → `baseKind: component`. Sets use `kind = "prop"` on furniture. `PrimIsModel` and `IsComponent()` still work because `IsA("prop", "component")` is designed to be True **after** the plugin loads.

### USDA example.

```
#usda 1.0

def Xform "Chair" (
 kind = "component"
)
{
 def Xform "Seat" (
 kind = "subcomponent"
)
 {
 }
}
```

`Chair` is a model/component. `Seat` is not a model. A custom `prop` would replace `"component"` on `Chair` only after the kind plugin exists.

### Python example.

```
from pxr import Kind

print("all", sorted(Kind.Registry.GetAllKinds()))
```

```

print("HasKind door", Kind.Registry.HasKind("door"))
print("base component", Kind.Registry.GetBaseKind("component"))
print("base assembly", Kind.Registry.GetBaseKind("assembly"))
print("base group", Kind.Registry.GetBaseKind("group"))
print("base model", repr(Kind.Registry.GetBaseKind("model")))
print("base subcomponent",
 repr(Kind.Registry.GetBaseKind("subcomponent")))
print("component IsA model", Kind.Registry.IsA("component", "model"))
print("assembly IsA model", Kind.Registry.IsA("assembly", "model"))
print("subcomponent IsA model",
 Kind.Registry.IsA("subcomponent", "model"))
print("component IsA group", Kind.Registry.IsA("component", "group"))
print("IsComponent component", Kind.Registry.IsComponent("component"))
print("IsModel assembly", Kind.Registry.IsModel("assembly"))
print("has Register", hasattr(Kind.Registry, "Register"))

```

### Expected output

```

all ['assembly', 'component', 'group', 'model', 'subcomponent']
HasKind door False
base component model
base assembly group
base group model
base model ''
base subcomponent ''
component IsA model True
assembly IsA model True
subcomponent IsA model False
component IsA group False
IsComponent component True
IsModel assembly True
has Register False

```

**Real-world use case.** A layout tool’s “props only” filter uses `Kind.Registry.IsA(kind, "prop")` once `prop` is registered. Until IT ships the kind plugin, the same tool falls back to `IsComponent()` and over-selects characters.

### Common mistakes.

#### MISTAKE

Inventing ``kind = "hero"`` with no ``baseKind``. Model hierarchy tools ignore it. Fix: ``baseKind`` ``component`` or ``assembly``.

#### MISTAKE

``Kind.Registry.Register("prop")`` in Python 26.08. The method `**does not exist**`. Use ``plugInfo.json``.

#### MISTAKE

``kind = "subcomponent"`` on a published root. Published roots are ``component`` or ``assembly`` (Ch 23).

### Exam traps.

#### TRAP

"`subcomponent` IsA `model`." False. The hierarchy `**stops**` at components.

#### TRAP

"`assembly` bases on `component`." It bases on `**`group`**`, which bases on ``model``.

#### TRAP

"Custom kinds are schemas." Kinds are tokens in the kind registry, not ``UsdTyped`` classes.

### Exam takeaways.

**KEY**

- Built-in tree: component→model, assembly→group→model, subcomponent not a model. - Custom kinds: `plugInfo.json` `Kinds` + `baseKind`; no Python `Register` here. - Obj 3.3: add a kind only when a tool must filter finer than component/assembly.

## 38.2 Variant fallback selections

**What is it?** A **variant fallback** is a process-wide default: “if nobody authored a selection for variant set `lod`, pick `low`.” You set it with `Usd.Stage.SetGlobalVariantFallbacks`. Plugins can also ship fallbacks in `plugInfo.json`. An **authored** `variants = { string lod = "high" }` always wins.

**Why do we need it?** A games runtime wants every asset’s `lod` set to `low` unless a shot pinned `high`. You should not edit a thousand entry layers. Fallbacks fill the gap when the selection is **missing** (Ch 18).

**Beginner explanation.** A thermostat schedule: “if nobody touched the dial, use 18 °C.” If a guest set 22 °C, the schedule does not override them. Fallbacks are the schedule; authored `variants` are the dial.

*Where the analogy breaks:* fallbacks are **per process**, not per stage file. `SetGlobalVariantFallbacks` affects later `Stage.Open` calls in that process.

**Technical explanation.** Verified on USD 26.08:

```
Usd.Stage.GetGlobalVariantFallbacks() -> dict (default {})
Usd.Stage.SetGlobalVariantFallbacks({"lod": ["low"]})
```

The value is a **list** of names (tried in order). With no authored selection, opening a layer that has `variantSet "lod" = { "high": Sphere, "low": Cube }` composed **Cube** and `GetVariantSelection("lod")` was `"low"`. With authored `variants = { string lod = "high" }` **and** fallback `low`, the child was still **Sphere**.

Without fallback *and* without authored selection, the variant set exists but **no child** /A/G composed.

`plugInfo.json` can declare plugin variant fallbacks (studio default `lod=low` for every DCC that loads the plugin). Python `SetGlobalVariantFallbacks` is the runtime equivalent for a player or test.

Reset to `{}` when the test ends so you do not leak defaults into the next Open in the same process.

### Mental model.

```
authored variants { lod = high } --> always wins
else global / plugin fallback --> used
else --> no selection (children in variants missing)
```

**Simple example.** A batch renderer sets `SetGlobalVariantFallbacks({"lod": ["render"]})` then opens every shot. Artists who pinned `lod=proxy` in the shot keep proxy; everyone else gets render.

**USDA example.** File: `asset.usda` — **no** authored selection:

```
#usda 1.0

def Xform "A" (
 prepend variantSets = "lod"
)
{
 variantSet "lod" = {
 "high" {
 def Sphere "G"
 {
 }
 }
 }
}
```

```

 "low" {
 def Cube "G"
 {
 }
 }
 }
}

```

Without a fallback, `/A/G` does not exist. With fallback `low`, `/A/G` is a Cube.

### Python example.

```

from pxr import Usd

open("asset.usda", "w").write("""#usda 1.0
def Xform "A" (
 prepend variantSets = "lod"
)
{
 variantSet "lod" = {
 "high" {
 def Sphere "G" {}
 }
 "low" {
 def Cube "G" {}
 }
 }
}
""")

open("pinned.usda", "w").write("""#usda 1.0
def Xform "A" (
 variants = {
 string lod = "high"
 }
 prepend variantSets = "lod"
)
{
 variantSet "lod" = {
 "high" {
 def Sphere "G" {}
 }
 "low" {
 def Cube "G" {}
 }
 }
}
""")

Usd.Stage.SetGlobalVariantFallbacks({})
s = Usd.Stage.Open("asset.usda")
print("no fb child", bool(s.GetPrimAtPath("/A/G")))

Usd.Stage.SetGlobalVariantFallbacks({"lod": ["low"]})
s2 = Usd.Stage.Open("asset.usda")
print("fb type", s2.GetPrimAtPath("/A/G").GetTypeName())
print("fb sel",
 s2.GetPrimAtPath("/A").GetVariantSet("lod").GetVariantSelection())

s3 = Usd.Stage.Open("pinned.usda")
print("pinned type", s3.GetPrimAtPath("/A/G").GetTypeName())
Usd.Stage.SetGlobalVariantFallbacks({})
print("reset", Usd.Stage.GetGlobalVariantFallbacks())

```

### Expected output

```

no fb child False
fb type Cube
fb sel low
pinned type Sphere
reset {}

```

**Real-world use case.** usdview and a farm player load the same `prop` plugin that sets fallback `drawMode=cards` for huge sets. A lighting shot that authored `drawMode=default` still sees full geometry.

**Common mistakes.**

**MISTAKE**

Expecting fallbacks to override shot selections. Authored wins.

**MISTAKE**

Setting fallbacks after ``Stage.Open``. Open again (or open after the set).

**MISTAKE**

``SetGlobalVariantFallbacks({"lod": "low"})`` if the API wants a **list**. Use ``["low"]``.

**Exam traps.**

**TRAP**

"Fallbacks are layer metadata." They are process (or plugin) defaults, not ``defaultPrim``.

**TRAP**

"Missing selection uses the first variant in the USDA file." Not guaranteed. Without fallback, the child may be **absent**.

**Exam takeaways.**

**KEY**

- Fallbacks fill **missing** selections; authored ``variants`` win. - ``SetGlobalVariantFallbacks({set: [names]})``; reset to ``{}``. - No selection and no fallback can mean the variant's prims are gone.

### 38.3 File format plugins (`SdfFileFormat`)

**What is it?** A **file-format plugin** teaches Sdf how to read (and maybe write) a file extension. Built-ins: `usda`, `usdc`, `usd`, `usdz`. A studio format (`.mygeo`, `.gltf` as USD) subclasses `SdfFileFormat` and lists extensions / `formatId` in `plugInfo.json` (Ch 36).

**Why do we need it?** `Stage.Open("hero.abc")` only works if some plugin claimed `abc`. Otherwise you convert offline (Ch 27) instead of opening in place.

**Beginner explanation.** Windows "Open with..." for `.usda`. The format plugin is the app registered for that extension. USDZ is a **package** app: it also knows there are files *inside*.

*Where the analogy breaks:* `.usd` is one extension that may hold USDA or crate (Ch 31). The plugin still has `formatId usd`.

**Technical explanation.** Verified on USD 26.08:

Query	Result
<code>Sdf.FileFormat.FindAllFileFormatExtensions()</code>	<code>usd</code> , <code>usda</code> , <code>usdc</code> , <code>usdz</code>
<code>FindByExtension("usda").formatId</code>	<code>usda</code>
<code>FindByExtension("usdz").IsPackage()</code>	<code>True</code>
<code>FindByExtension("usdz").SupportsWriting()</code>	<b>False</b>
<code>FindByExtension("usda").SupportsWriting()</code>	<code>True</code>
Owner plugin of <code>SdfUsdaFileFormat</code>	<code>sdf</code>

`plugInfo.json` fields you already saw: `bases: ["SdfFileFormat"]`, `extensions`, `formatId`, `primary`. USDZ also sets `supportsWriting: false` in JSON — matching `SupportsWriting()`.

Obj 3.8 sometimes pairs a format with a **package resolver** (`ArPackageResolver`, as `SdfUsdzResolver` does for `.usdz`).

You cannot implement `SdfFileFormat` in this venv's pure Python. You *can* query the registry and know which bases to put in JSON.

### Mental model.

```
.usda --> SdfUsdaFileFormat (text, writable)
.usdc --> SdfUsdcFileFormat (crate)
.usd --> SdfUsdFileFormat (either encoding)
.usdz --> SdfUsdzFileFormat (package, not writable as a layer)
.mygeo --> your SdfFileFormat plugin
```

**Simple example.** A CAD vendor registers `.sldprt` as a format plugin so `Stage.Open` tessellates on the fly. Pipelines that prefer a frozen mesh still run the Ch 29 converter instead.

**USDA example.** USDA is itself a format. Opening this file used `SdfUsdaFileFormat`:

```
#usda 1.0
(
 defaultPrim = "Box"
)

def Cube "Box"
{
 double size = 1
}
```

### Python example.

```
from pxr import Sdf, Plug, Tf

print("extensions", sorted(Sdf.FileFormat.FindAllFileFormatExtensions()))
usda = Sdf.FileFormat.FindByExtension("usda")
usdz = Sdf.FileFormat.FindByExtension("usdz")
print("usda id", usda.formatId, "write", usda.SupportsWriting())
print("usdz id", usdz.formatId, "package", usdz.IsPackage(),
 "write", usdz.SupportsWriting())
print("abc", Sdf.FileFormat.FindByExtension("abc"))
t = Tf.Type.FindByName("SdfUsdaFileFormat")
print("usda plugin", Plug.Registry().GetPluginForType(t).name)
```

### Expected output

```
extensions ['usd', 'usda', 'usdc', 'usdz']
usda id usda write True
usdz id usdz package True write False
abc None
usda plugin sdf
```

**Real-world use case.** A VFX house opens `.abc` through a format plugin during layout, then publishes `.usdc` so lighting never depends on the Alembic plugin on the farm.

### Common mistakes.

#### MISTAKE

Expecting ``Stage.Save()`` on a USDZ layer. ``SupportsWriting`` is False; rebuild the package (Ch 31).

#### MISTAKE

Writing a resolver because you want a new extension. Extension → format plugin. Search path → resolver.

### Exam traps.



**TRAP**

"USDZ is just crate with a different extension." It is a **\*\*package\*\*** format (``IsPackage`` True).

**TRAP**

"``FindByExtension('abc')`` raises." It returns ``None``.

**Exam takeaways.****KEY**

- File-format plugin = ``SdfFileFormat` + `extensions` / `formatId``. - Built-ins: four extensions; USDZ is a non-writable package. - ``FindByExtension`` returns ``None`` for unknown suffixes.

**38.4 Custom asset resolvers (ArResolver)**

**What is it?** A **custom resolver** is an `ArResolver` plugin that changes how `@path@` becomes bytes (Obj 3.5). Obj 3.8 asks for a resolver that can **generate in-memory primitives** — a procedural lookup that does not need a file on disk.

**Why do we need it?** Chapter 33 taught search paths and `DefaultResolver`. Studios still need `s3://`, database ids, or a resolver that *synthesizes* a cube when you open `@procedural:box@`.

**Beginner explanation.** The default librarian walks shelves. A custom librarian can phone a warehouse (Obj 3.5) or **print a new pamphlet on the spot** (Obj 3.8) when the card says `procedural:box`.

*Where the analogy breaks:* generating prims still has to look like a USD layer (or Hydra prims) to the rest of the stack. You do not `def Cube` from inside `Resolve` without a `format/` layer to parse.

**Technical explanation.** Integration recap (Ch 33, Ch 36), verified APIs still:

- `plugInfo.json`: `"bases": ["ArResolver"]` (or `ArPackageResolver` for packages)
- `PXR_PLUGINPATH_NAME` / `RegisterPlugins`
- `Ar.SetPreferredResolver("StudioResolver")` only after the type **loads** (fake names stay `DefaultResolver`)
- `Usd.Stage.Open(path, pathResolverContext=rc)`
- `GetRegisteredURISchemes()` is `[]` on this wheel until a URI plugin registers

Obj 3.8 (generate in-memory primitives): typical design is `Resolve` → `OpenAsset` returns a memory buffer of USDA/USDC that `Sdf` then parses, **or** a Hydra scene-index / procedural that emits render prims (Section 38.5). Both are C++ plugins. `usd-core` Python cannot subclass `Ar.Resolver` usefully.

Package relative paths (`a.usdz[b.usda]`) already use `ArPackageResolver` (`Sdf_UsdzResolver`).

**Mental model.**

```
Obj 3.5 Resolve("chair") --> /prod/show/assets/chair/v022/chair.usda
Obj 3.8 Resolve("procedural:box") --> in-memory USDA of a Cube
```

**Simple example.** `@s3://bucket/chair.usda@` needs a URI resolver (scheme `s3`). `@box@` that never touches disk needs Obj 3.8-style `OpenAsset` memory.

**USDA example.** The layer does not mention the plugin. Identifiers are just asset paths:

```
#usda 1.0

def "C" (
 prepend references = @chair@
```

```
)
{
}
```

Without the studio resolver and context, this is `ErrorInvalidAssetPath` (Ch 33).

### Python example.

```
from pxr import Ar, Plug, Tf

print("underlying", type(Ar.GetUnderlyingResolver().__name__))
print("schemes", Ar.GetRegisteredURISchemes())
t = Tf.Type.FindByName("ArDefaultResolver")
print("plugin", Plug.Registry().GetPluginForType(t).name)
print("package path",
 Ar.IsPackageRelativePath("a.usdz[b.usda]"))
Ar.SetPreferredResolver("NoSuchResolver")
print("after fake prefer", type(Ar.GetUnderlyingResolver().__name__))
```

### Expected output

```
underlying DefaultResolver
schemes []
plugin ar
package path True
after fake prefer DefaultResolver
```

Same fail-closed behavior as Chapter 33: a name that did not load does not switch the resolver.

**Real-world use case.** A cloud renderer maps `@asset/chair@` to an object-store GET (Obj 3.5). A lookdev tool maps `@proc:uvgrid@` to an in-memory Mesh of a calibration grid (Obj 3.8) so artists never check a file in.

### Common mistakes.

#### MISTAKE

Subclassing ``Ar.Resolver`` in a notebook. Load a C++ plugin (Ch 36).

#### MISTAKE

Using a resolver to add ``def Door``. That is a schema (Ch 37).

### Exam traps.

#### TRAP

Obj 3.8 is ``CreateInMemory()``. That API makes an empty stage, not a resolver that *\*generates\** assets from an identifier.

#### TRAP

Obj 3.5 is satisfied by ``Resolve("./x")`` from cwd. Integration means plugin + context + documented identifiers.

### Exam takeaways.

#### KEY

- Custom resolvers: ``ArResolver`` + `plugInfo` + context (Obj 3.5). - Obj 3.8: identifier → in-memory layer or Hydra prims, still a plugin. - Fake ``SetPreferredResolver`` stays ``DefaultResolver``.

## 38.5 Hydra scene index plugins (concepts)

**What is it?** Hydra is USD's imaging architecture: a renderer-agnostic scene, then a **render delegate** (Storm, Karma, ...). A **scene index** is a node in Hydra 2.0's scene graph that can filter, instance, or **generate** renderable prims. Obj 3.7: write a scene-index plugin that generates geometry as Hydra prims.

**Why do we need it?** Some geometry should not live in USD layers at all: a procedural ocean, a CAD tessellation at render time, a point-cloud display. A scene index can create Hydra meshes when the renderer asks, without `def Mesh` in the USDA.

**Beginner explanation.** USD is the script. Hydra is the stage lighting and cameras of the *theatre*. A scene-index plugin is a technician who *builds extra scenery during the show* that was never in the script.

Where the analogy breaks: `usdview` is that theatre, and this wheel has **no** `UsdImaging`. You can register `usdHydra` schema types and still have no window. Obj 3.7 assumes a USD build with imaging (Ch 35).

**Technical explanation.** What this venv **can** prove:

- Plugin `usdHydra` is in `GetAllPlugins()` (shader discovery / Hydra API schemas).
- `import pxr.UsdImaging` fails on `usd-core`.

What a scene-index plugin **is** (exam-level, not runnable here):

- A C++ class implementing the Hydra scene-index API (filtering or generating Hd prims).
- Discovered via `plugInfo.json` like every other plugin (`bases` pointing at the scene-index type).
- Inserted into the imaging graph for `usdview` / a DCC viewport / a batch Hydra render.

Obj 3.7 vs 3.8: scene index generates **Hydra** prims (render). Procedural **resolver** generates **USD** bytes/layers (composition). You can use both: resolver provides a tiny overlay layer; scene index densifies it at render.

You do not author scene indices in USDA. You might author a typed prim the index *looks for* (`def DistantLight`, or a custom procedural schema).

#### VERSION

``usd-core` 26.8: no Hydra runtime. Full `build_usd.py --usdview` (Ch 35) is required to *run* Obj 3.7.`

#### Mental model.

```
USD stage --> Hydra scene index chain --> render delegate
 ^
 | your plugin can insert/generate prims here
```

**Simple example.** A “card” scene index turns unloaded payloads into bounding-box meshes for the viewport. The USDA still has payloads; Hydra never tessellates until you load (related to draw modes, Ch 26).

**USDA example.** The file the index might watch — still ordinary USD:

```
#usda 1.0

def Cube "ProxyCard"
{
 double size = 1
}
```

A generating index could *invent* this Cube at render without the layer existing. That is the point of Obj 3.7.

#### Python example.

```
from pxr import Plug

names = [p.name for p in Plug.Registry().GetAllPlugins()]
print("has usdHydra", "usdHydra" in names)
```

```
print("has usdGeom", "usdGeom" in names)
try:
 import pxr.UsdImaging # noqa: F401
 print("UsdImaging", True)
except ModuleNotFoundError:
 print("UsdImaging", False)
```

### Expected output

```
has usdHydra True
has usdGeom True
UsdImaging False
```

The Hydra *schema plugin* is present. The imaging *module* is not. Obj 3.7 needs the latter in a real DCC/usdview.

**Real-world use case.** A digital-twin viewport uses a scene index to draw millions of instanced bolts from a table, while the USD layer only stores the table. The Hydra delegate never sees a million `def Cube` specs.

### Common mistakes.

#### MISTAKE

Thinking ``usdHydra`` in ``GetAllPlugins()`` means usdview works. It does not on ``usd-core``.

#### MISTAKE

Implementing Obj 3.7 as ``UsdGeom.Cube.Define`` in a Python loop. That writes USD, not a Hydra scene index.

### Exam traps.

#### TRAP

"Scene index = variant set." Unrelated. Variants are composition; scene indices are imaging.

#### TRAP

Obj 3.7 is ``plugInfo.json`` for ``usdGeom``. Geom schemas are not scene indices.

### Exam takeaways.

#### KEY

- Scene index = Hydra 2.0 plugin that can generate/filter render prims (Obj 3.7). - Needs an imaging build; ``usd-core`` has ``usdHydra`` JSON but no ``UsdImaging``. - Resolver (USD bytes) vs scene index (Hydra prims) — Obj 3.8 vs 3.7.

## Chapter lab(s)

**Lab 32** — Custom kinds & fallbacks: print the `Kind.Registry` table and a fallback vs authored-selection pair. Resolvers stay Lab 29; Hydra stays conceptual until you have usdview.

## USDA reading exercises

**Exercise 38-A.** A prim has `kind = "prop"` but `Kind.Registry.HasKind("prop")` is False. `IsComponent()`? What should IT ship?

**Exercise 38-B.** `variants` is not authored. Fallback is `lod=["low"]`. Variant set has `high` (Sphere) and `low` (Cube). What is `/A/G`?

### Answers

- **38-A.** Unknown kind is just a string; it is **not** a component in the registry. Ship `plugInfo.json Kinds.prop.baseKind = component` at process start.
- **38-B.** A Cube named G (fallback applied).

## Chapter review

### Summary

- Kinds: five built-ins; custom via `plugInfo Kinds + baseKind`; no Python `Register`.
- Fallbacks fill missing variant selections; authored wins; process-wide.
- File formats: `SdfFileFormat`; USDZ is a non-writable package.
- Resolvers: Obj 3.5 lookup, Obj 3.8 in-memory generation; still plugins.
- Scene indices: Obj 3.7 Hydra generation; not in `usd-core` imaging.

### If you see... → think...

If you see...	Think...
<code>GetBaseKind("assembly")</code>	group
<code>subcomponent IsA model</code>	False
<code>Kind.Registry.Register</code>	Missing in 26.08 Python
<code>Fallback vs variants =</code>	Authored wins
<code>IsPackage</code>	USDZ
<code>SupportsWriting False on usdz</code>	Package via <code>CreateNewUsdzPackage</code>
Obj 3.7	Hydra scene index, not <code>def Mesh</code>
Obj 3.8	Resolver that synthesizes bytes/prims

### Review questions

**R38-1** (Obj 3.3) `GetBaseKind("component")` is: A. group B. model C. assembly D. subcomponent

**R38-2** (Obj 3.3) Select two true kind facts. A. `IsA("assembly", "model")` is True B. `IsA("subcomponent", "model")` is False C. `HasKind("prop")` is True in stock `usd-core` D. Python `Kind.Registry.Register` exists on 26.08

**R38-3** (Obj 3.3) Custom kind `prop` should declare: A. `baseKind: component` in `plugInfo.json` B. `inherits = </Typed>` C. `formatId: prop` D. `SdfFileFormat`

**R38-4** No authored selection, fallback `lod=["low"]`, variants `high=Sphere low=Cube`. /A/ G is: A. Sphere B. Cube C. Missing D. Both

**R38-5** Select two fallback facts. A. Authored variants beat fallbacks B. `SetGlobalVariantFallbacks` is process-wide C. Fallbacks are stronger than shot layers D. Fallbacks require Hydra

**R38-6** (Obj 3.5) Custom path lookup plugin bases: A. `ArResolver` B. `UsdTyped` C. `SdfFileFormat` D. `UsdAPISchemaBase`

**R38-7** `FindByExtension("usdz")`: A. `IsPackage True`, `SupportsWriting False` B. Same as `USDA` C. Raises D. `formatId usda`

**R38-8** (Obj 3.7) Scene-index plugins generate: A. Kind tokens B. Hydra render prims C. `plugInfo.json Includes` D. `defaultPrim`

**R38-9** (Obj 3.8) Select two legal designs. A. Resolver `OpenAsset` returns in-memory `USDA` B. Scene index emits a mesh at render C. `Kind.Registry.Register` in a notebook D. `GetMajorVersion() == 26`

**R38-10** This `venv`: A. `UsdImaging` imports; `usdHydra` missing B. `usdHydra` present; `UsdImaging` missing C. Both present D. Both missing

**R38-11** Unknown extension `.abc`: A. `FindByExtension` returns `None` B. Raises `Tf.ErrorException` C. Opens as `USDA` D. Uses `DefaultResolver` to parse Alembic

**R38-12** (Obj 3.3) Published asset root kind: A. subcomponent B. component or assembly (custom kinds should still IsA those) C. model authored as the token model D. Empty so kinds stay flexible

**Review answers**

- **R38-1: B.** Review: §38.1.
- **R38-2: A and B.** Review: §38.1.
- **R38-3: A.** Review: §38.1.
- **R38-4: B.** Review: §38.2.
- **R38-5: A and B.** Review: §38.2.
- **R38-6: A.** Review: §38.4.
- **R38-7: A.** Review: §38.3.
- **R38-8: B.** Review: §38.5.
- **R38-9: A and B.** Review: §38.4–38.5.
- **R38-10: B.** Review: §38.5.
- **R38-11: A.** Review: §38.3.
- **R38-12: B.** Review: §38.1, Ch 23.

## Part VIII — Visualization

### CHAPTER 39

## Chapter 39 — UsdGeom: Geometry, Transforms, Cameras

**Exam domain:** Visualization (8%) · **Objectives:** 5.1, 8.1 · **Study day:** 11 · **Est. time:** 100 min **Prerequisites:** Ch 2 (stages and prims), Ch 5 (attributes), Ch 6 (schemas), Ch 11 (primvars), Ch 13 (extents)

Everything you see in a USD viewport comes from the **UsdGeom** module: the shapes, where they sit, whether they are drawn, and the camera you look through. This chapter teaches the parts of UsdGeom the exam expects you to read and write fluently. Primvars were taught in Chapter 11 and extents in Chapter 13; here we only connect them to meshes.

### Learning goals

- Author transforms with `xformOps` and predict the result from `xformOpOrder`.
- Compute local and world transforms with `UsdGeom.XformCache`, including `resetXformStack`.
- Build a polygon mesh from `points`, `faceVertexCounts`, and `faceVertexIndices`, and choose the right `subdivisionScheme`.
- Explain how `normals` and `primvars:normals` interact.
- Split a mesh into `GeomSubsets` for per-face materials.
- Control drawing with `visibility` and `purpose`, and describe a camera with `UsdGeom.Camera`.

### Key terms

Term	One-line definition
<b>Xformable</b>	Schema base class for every prim that can carry a transform ( <code>UsdGeom.Xformable</code> ).
<b>Xform</b>	A concrete prim type ( <code>def Xform</code> ) that only groups and transforms its children.
<b>xformOp</b>	One transform step stored as an attribute named <code>xformOp:&lt;type&gt;[:&lt;suffix&gt;]</code> .
<b>xformOpOrder</b>	A <code>uniform token[]</code> attribute listing which <code>xformOps</code> apply and in which order.
<b>Local transform</b>	The matrix built from a prim's own <code>xformOps</code> (relative to its parent).
<b>World transform</b>	Local transform combined with every ancestor's transform (local-to-world).
<b>resetXformStack</b>	A marker in <code>xformOpOrder</code> meaning "ignore my parents' transforms".
<b>Gprim</b>	A "geometric primitive": a drawable shape such as <code>Mesh</code> , <code>Sphere</code> , <code>Cube</code> .
<b>Mesh</b>	A <code>Gprim</code> made of polygons (faces) built from points.
<b>Winding order</b>	The order a face lists its points; with <code>orientation</code> it decides the front side.
<b>Subdivision</b>	Smoothing a coarse mesh into a limit surface (default scheme <code>catmullClark</code> ).
<b>GeomSubset</b>	A child prim that names a group of faces (or points, edges...) of its parent.
<b>Imageable</b>	Schema base class that adds <code>visibility</code> and <code>purpose</code> to anything drawable.
<b>Purpose</b>	A category (default, <code>render</code> , <code>proxy</code> , <code>guide</code> ) that renderers include or skip.

Term	One-line definition
Camera	A UsdGeom.Camera prim; converts to the math class Gf.Camera.

### 39.1 xform and xformOps, xformOpOrder

**What is it?** An **xformOp** is one step of a transform, such as “translate by (10, 0, 0)” or “rotate 90° about Z”. A prim stores each step as an attribute and lists the active steps, in order, in its **xformOpOrder** attribute.

**Why do we need it?** Artists think in separate steps: move, then rotate, then scale, maybe around a pivot. Storing one opaque 4×4 matrix would lose those steps, make animation curves hard to edit, and make pivots impossible to keep. xformOps keep each step separate, typed, and animatable on its own.

**Beginner explanation.** Think of a recipe card. Each line is one instruction (“add salt”, “stir”, “bake”). The card’s line order matters, and a line that is not on the card is never done, even if the salt is sitting on the counter. In USD the xformOp attributes are the ingredients on the counter, and xformOpOrder is the recipe card.

*Where the analogy breaks:* a recipe is read top to bottom; for a point, USD applies the **last** op on the card first (see the technical explanation).

**Technical explanation.** - Any prim whose schema derives from **UsdGeom.Xformable** can carry xformOps: Xform, Mesh, Sphere, Camera, lights, and so on. Scope is not Xformable, so it cannot. - Op attributes are named xformOp:<opType> with an optional suffix: xformOp:translate, xformOp:rotateXYZ, xformOp:scale, xformOp:translate:pivot, xformOp:transform (a full matrix), xformOp:orient (a quaternion). - Python adds them with AddTranslateOp(), AddRotateXYZOp(), AddRotateZOp(), AddScaleOp(), AddOrientOp(), AddTransformOp(), ... Each call creates the attribute **and** appends its name to xformOpOrder. - Default precisions (verified): translate is double3, rotate is float/float3, scale is float3. - **Only ops listed in xformOpOrder count.** An xformOp:\* attribute not in the list is ignored. - **Order semantics.** For xformOpOrder = [A, B, C] the local matrix is the product in which **C is applied to a point first, then B, then A**. So the standard ["xformOp:translate", "xformOp:rotateXYZ", "xformOp:scale"] means *scale, then rotate, then translate* — the familiar “TRS”. - In matrix terms USD uses row vectors:  $p' = p \cdot M_C \cdot M_B \cdot M_A$ . - An op name prefixed with !invert! applies the inverse of that op (used for pivots). - Adding the same op type and suffix twice raises an error; use a suffix for a second op of the same type.

#### NOTE

``UsdGeom.XformCommonAPI`` is a helper that reads and writes the common “translate, pivot, rotate, scale, inverse pivot” stack in one call. DCC exporters often use it so that every app can understand the result.

#### Mental model.

```
xformOpOrder = [translate , rotateZ , scale]
 ^ ^ ^
 | | |
 applied 3rd applied 2nd applied 1st <- to each point
point → scale → rotate → translate → position in parent space
```

Read the list **right to left** to follow a point.

**Simple example.** Point (1, 0, 0). Ops: translate (10, 0, 0), rotateZ 90°, scale 2. - Scale first: (2, 0, 0). Rotate 90° about Z: (0, 2, 0). Translate: **(10, 2, 0)**. - Reverse the list (scale, rotate, translate): translate first gives (11, 0, 0), rotate gives (0, 11, 0), scale gives **(0, 22, 0)**.

**USDA example.**



```
#usda 1.0
(
 defaultPrim = "Robot"
)

def Xform "Robot"
{
 double3 xformOp:translate = (10, 0, 0)
 float xformOp:rotateZ = 90
 float3 xformOp:scale = (2, 2, 2)
 double3 xformOp:translate:pivot = (0, 1, 0)
 float3 xformOp:rotateXYZ = (45, 0, 0)
 uniform token[] xformOpOrder = [
 "xformOp:translate",
 "xformOp:translate:pivot",
 "xformOp:rotateZ",
 "!invert!xformOp:translate:pivot",
 "xformOp:scale",
]
}
```

- Lines with `xformOp:` are the ingredients; `xformOpOrder` is the recipe.
- `xformOp:translate:pivot` plus its `!invert!` twin rotate the robot around (0, 1, 0) instead of its origin.
- `xformOp:rotateXYZ` is authored but **not listed**, so it has no effect.
- `xformOpOrder` is `uniform`: it cannot be animated (the `op` values can).

**Python example.** The script builds both orders from §6 and transforms the same point.

```
from pxr import Usd, UsdGeom, Gf

stage = Usd.Stage.CreateInMemory()

def make(path, order):
 xf = UsdGeom.Xform.Define(stage, path)
 for op in order:
 if op == "T":
 xf.AddTranslateOp().Set(Gf.Vec3d(10, 0, 0))
 elif op == "R":
 xf.AddRotateZOp().Set(90)
 else:
 xf.AddScaleOp().Set(Gf.Vec3f(2, 2, 2))
 return xf

for path, order in [("/TRS", "TRS"), ("/SRT", "SRT")]:
 xf = make(path, order)
 m = xf.GetLocalTransformation()
 p = m.Transform(Gf.Vec3d(1, 0, 0))
 names = [str(n) for n in xf.GetXformOpOrderAttr().Get()]
 print(path, names)
 print(" (1,0,0) ->", tuple(round(v, 6) + 0.0 for v in p))

trs = stage.GetPrimAtPath("/TRS")
print("precisions:", [op.GetPrecision().name for op in
 UsdGeom.Xformable(trs).GetOrderedXformOps()])
```

### Expected output

```
/TRS ['xformOp:translate', 'xformOp:rotateZ', 'xformOp:scale']
(1,0,0) -> (10.0, 2.0, 0.0)
/SRT ['xformOp:scale', 'xformOp:rotateZ', 'xformOp:translate']
(1,0,0) -> (0.0, 22.0, 0.0)
precisions: ['PrecisionDouble', 'PrecisionFloat', 'PrecisionFloat']
```

The output proves the rule: in the first prim the scale (last in the list) hit the point first.

**Real-world use case.** In film, a rigged prop exported from Maya keeps translate, rotate-pivot, rotate, and scale as separate ops, so a layout artist in another app can nudge only the translate in a stronger layer (Chapter 15) without baking the whole matrix. In robotics digital twins, each joint is often an Xform with a single animated rotateZ, which keeps joint curves readable.

### Common mistakes.

#### MISTAKE

Creating ``xformOp:translate`` with ``CreateAttribute`` and forgetting ``xformOpOrder``. Nothing moves. Fix: use ``AddTranslateOp()``, which updates the order for you, or set ``xformOpOrder`` yourself.

#### MISTAKE

Calling ``AddTranslateOp()`` twice on the same prim. The second call raises an error ("already exists in xformOpOrder"). Fix: reuse ``GetTranslateOp()``, or add a second op with a suffix: ``AddTranslateOp(opSuffix="pivot")``.

#### MISTAKE

Overriding ``xformOpOrder`` in a stronger layer with a shorter list. The ops you dropped silently stop applying. Fix: compare the composed ``xformOpOrder`` with the authored op attributes when a prim "jumps".

### Exam traps.

#### TRAP

"The first op in ``xformOpOrder`` is applied first." False for a point: the **last** listed op is applied to the point first.

#### TRAP

An ``xformOp:*`` attribute that exists but is not in ``xformOpOrder`` does nothing. Questions show it in USDA to tempt you.

#### TRAP

``Scope`` is not Xformable. A ``Scope`` cannot carry xformOps; use ``Xform`` when you need a transform.

### Exam takeaways.

#### KEY

- Ops are attributes ``xformOp[:suffix]``; ``xformOpOrder`` decides which apply and in what order.
- Follow a point **right to left** through ``xformOpOrder``; "translate, rotate, scale" = scale first.
- Unlisted ops are ignored; ``!invert!`` applies an op's inverse (pivots).
- ``Add*Op()`` creates the attribute and appends to the order; duplicates need a suffix.

## 39.2 XformCache, local vs. world transforms

**What is it?** A prim's **local transform** places it relative to its parent. Its **world transform** (local-to-world) places it in the stage's root space by combining its local transform with all ancestors'. **UsdGeom.XformCache** computes world transforms and remembers the results for reuse.

**Why do we need it?** A renderer, exporter, or physics tool needs world positions. Computing each prim's world matrix from scratch re-walks every ancestor, which is slow for thousands of prims sharing parents. The cache computes each ancestor once per time code.

**Beginner explanation.** Your seat number on a train tells you where you are *in the carriage* (local). To know where you are *in the country* (world), you also need where the carriage is on the train and where the train is on the track. If 200 passengers ask, the conductor writes the train's position down once instead of re-measuring it 200 times. That note is the cache.

*Where the analogy breaks:* the cache stores one time code at a time; ask for another time and you must call `SetTime()`.

**Technical explanation.** - `UsdGeom.Xformable(prim).GetLocalTransformation(time)` returns the local `Gf.Matrix4d`. -

`UsdGeom.Imageable(prim).ComputeLocalToWorldTransform(time)` computes the world matrix directly (no caching). - `cache = UsdGeom.XformCache(time)` then: -

`cache.GetLocalToWorldTransform(prim) → Gf.Matrix4d`. -

`cache.GetLocalTransformation(prim) → a tuple (matrix, resetsXformStack)`. -

`cache.GetParentToWorldTransform(prim), cache.ComputeRelativeTransform(prim, ancestor)`. - `cache.SetTime(t)` switches time; `cache.Clear()` empties it (do this after editing transforms). - With row vectors, `world = local × parentWorld`. - **resetXformStack:**

if `xformOpOrder` starts with `"!resetXformStack!"`, the prim ignores all ancestor transforms; its local transform is its world transform. Python:

`xformable.SetResetXformStack(True)`; query with `GetResetXformStack()`.

### Mental model.

<code>/World</code>	<code>local W</code>	} → <code>world(Wheel) = Wheel · C · W</code>
<code>/Car</code>	<code>local C</code>	
<code>/Wheel</code>	<code>local Wh</code>	
<code>/Sticker</code>	<code>["!resetXformStack!", ...]</code>	→ <code>world(Sticker) = Sticker</code>

**Simple example.** `/Parent` translates (5, 0, 0). `/Parent/Child` translates (0, 1, 0). Child's world position is (5, 1, 0). Add `!resetXformStack!` to the child: world position becomes (0, 1, 0).

### USDA example.

```
#usda 1.0

def Xform "Parent"
{
 double3 xformOp:translate = (5, 0, 0)
 uniform token[] xformOpOrder = ["xformOp:translate"]

 def Xform "Child"
 {
 double3 xformOp:translate = (0, 1, 0)
 uniform token[] xformOpOrder = ["xformOp:translate"]
 }

 def Xform "Hud"
 {
 double3 xformOp:translate = (0, 1, 0)
 uniform token[] xformOpOrder = ["!resetXformStack!", "xformOp:translate"]
 }
}
```

- `Child` inherits the parent's (5, 0, 0) shift; world (5, 1, 0).
- `Hud` lists `!resetXformStack!` first, so it is placed at (0, 1, 0) in world space even though it sits under `Parent`.

### Python example.

```
from pxr import Usd, UsdGeom, Gf

stage = Usd.Stage.CreateInMemory()
parent = UsdGeom.Xform.Define(stage, "/Parent")
op = parent.AddTranslateOp()
```

```

op.Set(Gf.Vec3d(0, 0, 0), 1)
op.Set(Gf.Vec3d(10, 0, 0), 10)
child = UsdGeom.Xform.Define(stage, "/Parent/Child")
child.AddTranslateOp().Set(Gf.Vec3d(0, 1, 0))
hud = UsdGeom.Xform.Define(stage, "/Parent/Hud")
hud.AddTranslateOp().Set(Gf.Vec3d(0, 1, 0))
hud.SetResetXformStack(True)

cache = UsdGeom.XformCache(Usd.TimeCode(10))
for prim in (child.GetPrim(), hud.GetPrim()):
 world = cache.GetLocalToWorldTransform(prim)
 local, resets = cache.GetLocalTransformation(prim)
 print(prim.GetName(), "world", world.ExtractTranslation(),
 "local", local.ExtractTranslation(), "resets", resets)

cache.SetTime(Usd.TimeCode(1))
print("t=1 child world", cache.GetLocalToWorldTransform(child.GetPrim())
 .ExtractTranslation())
direct = child.ComputeLocalToWorldTransform(Usd.TimeCode(10))
print("direct t=10", direct.ExtractTranslation())
print("Hud order", [str(t) for t in hud.GetXformOpOrderAttr().Get()])

```

### Expected output

```

Child world (10, 1, 0) local (0, 1, 0) resets False
Hud world (0, 1, 0) local (0, 1, 0) resets True
t=1 child world (0, 1, 0)
direct t=10 (10, 1, 0)
Hud order ['!resetXformStack!', 'xformOp:translate']

```

**Real-world use case.** A game-engine exporter walks 50,000 prims and writes world matrices for each mesh. Using one `XformCache` per frame means each shared parent (a building, a level) is evaluated once. HUD elements, camera-attached props, or simulation results already in world space use `resetXformStack` so they ignore the rig they are parented under.

### Common mistakes.

#### MISTAKE

Reading ``GetLocalTransformation()`` and treating it as the world position. Fix: use ``XformCache.GetLocalToWorldTransform()`` or ``ComputeLocalToWorldTransform()``.

#### MISTAKE

Editing `xformOps` and then reading the same cache again. The cache may hold the old matrix. Fix: call ``cache.Clear()`` after edits, or make a new cache.

#### MISTAKE

Expecting ``XformCache.GetLocalTransformation()`` to return a matrix. It returns ``(matrix, resetsXformStack)``. Fix: unpack the tuple.

### Exam traps.

#### TRAP

``!resetXformStack!`` must be the **first** entry of ``xformOpOrder``. It is a marker, not an attribute.

#### TRAP

"XformCache is thread-safe and shared across time codes." It holds one time; switch with ``SetTime()``. Answers that say it caches all times are wrong.

### Exam takeaways.

**KEY**

- Local = own ops; world = local  $\times$  parent world (row vectors). -  
``UsdGeom.XformCache(time)` + `GetLocalToWorldTransform(prim)`; `SetTime()`` for other frames, ``Clear()`` after edits. - ``"!resetXformStack!"`` first in ``xformOpOrder``  $\rightarrow$  ignore ancestors (``SetResetXformStack(True)``).

### 39.3 Meshes: points, faceVertexCounts, faceVertexIndices, orientation, subdivision

**What is it?** A `UsdGeom.Mesh` is a surface made of polygons. Three arrays define it: `points` (vertex positions), `faceVertexCounts` (how many corners each face has), and `faceVertexIndices` (which points make each face, listed face after face).

**Why do we need it?** Almost every asset (characters, cars, buildings) is a polygon mesh. Storing shared points once and referring to them by index keeps files small and lets neighbouring faces share corners.

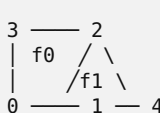
**Beginner explanation.** Think of a connect-the-dots puzzle. `points` is the list of numbered dots. `faceVertexIndices` is the long list of dot numbers you connect. `faceVertexCounts` tells you where one shape ends and the next starts: “the first 4 numbers are shape 1, the next 3 are shape 2”.

*Where the analogy breaks:* the direction you connect the dots (clockwise or counter-clockwise) also decides which side of the face is the front.

**Technical explanation.** - `point3f[] points` — positions in local space. - `int[] faceVertexCounts` — one entry per face; the sum must equal `len(faceVertexIndices)`. - `int[] faceVertexIndices` — indices into points; each must be in `[0, len(points))`. - `uniform token orientation` — default **rightHanded**: a face whose points go **counter-clockwise when you look at it** faces you (normal points toward you). `leftHanded` flips this. - `uniform token subdivisionScheme` — default **catmullClark** (verified). Allowed: `catmullClark`, `loop`, `bilinear`, `none`. The default means renderers treat the mesh as a **smooth subdivision surface**. For hard-surface or game/CAD meshes that are already final polygons, set **none**. - `bool doubleSided` (default false) — whether the back side is drawn. - `extent` must match points (see Chapter 13); after changing points, recompute with `UsdGeom.Boundable.ComputeExtentFromPlugins`. - Helpers: `mesh.GetFaceCount()`, `UsdGeom.Mesh.ValidateTopology(indices, counts, numPoints)  $\rightarrow$  (ok, reason)`.

#### Mental model.

```
points: 0(0,0,0) 1(1,0,0) 2(1,1,0) 3(0,1,0) 4(2,0,0)
faceVertexCounts: [4 , 3]
faceVertexIndices: [0 1 2 3 | 1 4 2]
 face 0 face 1
```



viewed from +Z, 0 $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 3 is counter-clockwise  
 $\rightarrow$  with rightHanded, the normal points to +Z (toward you)

**Simple example.** A single square: 4 points, `faceVertexCounts = [4]`, `faceVertexIndices = [0, 1, 2, 3]`. A square plus a triangle sharing an edge needs only 5 points, not 7, because points 1 and 2 are shared.

#### USDA example.

```
#usda 1.0
(
 upAxis = "Y"
)

def Mesh "Panel"
{
 float3[] extent = [(0, 0, 0), (2, 1, 0)]
```

```

int[] faceVertexCounts = [4, 3]
int[] faceVertexIndices = [0, 1, 2, 3, 1, 4, 2]
point3f[] points = [(0, 0, 0), (1, 0, 0), (1, 1, 0), (0, 1, 0), (2, 0, 0)]
uniform token orientation = "rightHanded"
uniform token subdivisionScheme = "none"
color3f[] primvars:displayColor = [(0.8, 0.8, 0.8)]
}

```

- Sum of counts (4 + 3) = 7 = number of indices.
- `orientation` is written out here only for clarity; `rightHanded` is the fallback.
- `subdivisionScheme = "none"` keeps the flat polygons; remove that line and a renderer would smooth the panel.
- `primvars:displayColor` gives a viewport color without a material (Chapter 11.7).

### Python example.

```

from pxr import Usd, UsdGeom, Gf

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/Panel")
print("fallbacks:", mesh.GetOrientationAttr().Get(),
 mesh.GetSubdivisionSchemeAttr().Get(), mesh.GetDoubleSidedAttr().Get())

points = [(0, 0, 0), (1, 0, 0), (1, 1, 0), (0, 1, 0), (2, 0, 0)]
counts = [4, 3]
indices = [0, 1, 2, 3, 1, 4, 2]
mesh.CreatePointsAttr(points)
mesh.CreateFaceVertexCountsAttr(counts)
mesh.CreateFaceVertexIndicesAttr(indices)
mesh.CreateSubdivisionSchemeAttr(UsdGeom.Tokens.none)

ok, why = UsdGeom.Mesh.ValidateTopology(indices, counts, len(points))
print("faces:", mesh.GetFaceCount(), "valid:", ok)
bad_ok, bad_why = UsdGeom.Mesh.ValidateTopology([0, 1, 9], [3], len(points))
print("bad:", bad_ok, bad_why)

extent = UsdGeom.Boundable.ComputeExtentFromPlugins(mesh, Usd.TimeCode.Default())
mesh.CreateExtentAttr(extent)
print("extent:", list(mesh.GetExtentAttr().Get()))
print("subdivisionScheme:", mesh.GetSubdivisionSchemeAttr().Get())

```

### Expected output

```

fallbacks: rightHanded catmullClark False
faces: 2 valid: True
bad: False Out of range face vertex index 9: Vertex must be in the range [0,5).
extent: [Gf.Vec3f(0.0, 0.0, 0.0), Gf.Vec3f(2.0, 1.0, 0.0)]
subdivisionScheme: none

```

#### VERSION

Verified on USD 26.08: the fallback of ``subdivisionScheme`` is ``catmullClark``, ``orientation`` is ``rightHanded``, and ``doubleSided`` is ``false``.

**Real-world use case.** A CAD-to-USD converter for a factory digital twin writes tessellated triangles. If it forgets `subdivisionScheme = "none"`, Storm and production renderers smooth every sharp machined edge into a blob, and render time explodes. Character assets for film, by contrast, keep `catmullClark` on purpose: the low-resolution cage is light to store and smooths at render time.

### Common mistakes.

#### MISTAKE

Exporting triangulated game/CAD meshes with the default subdivision scheme. Edges look melted. Fix: author ``subdivisionScheme = "none"``.

**MISTAKE**

``faceVertexCounts`` that do not add up to ``len(faceVertexIndices)``, or an index  $\geq$  ``len(points)``. Renderers drop or garble the mesh. Fix: run ``UsdGeom.Mesh.ValidateTopology``.

**MISTAKE**

Mirroring a mesh (negative scale or flipped winding) and getting inside-out shading. Fix: reverse the winding, or set ``orientation = "leftHanded"``.

**MISTAKE**

Editing ``points`` without updating ``extent``. The mesh gets culled at the wrong times (Chapter 13).

**Exam traps.****TRAP**

"The default ``subdivisionScheme`` is ``none``." It is ``catmullClark``. An unconfigured mesh is a smooth surface.

**TRAP**

``faceVertexCounts`` is per **face**; ``faceVertexIndices`` is per **face-vertex** (corner). Don't confuse the lengths.

**TRAP**

Right-handed means counter-clockwise winding **as seen from the front**. A question may describe clockwise winding and ask which way the normal points.

**Exam takeaways.****KEY**

- ``points`` + ``faceVertexCounts`` (per face) + ``faceVertexIndices`` (per corner); sums must match.
- ``orientation`` fallback ``rightHanded``: counter-clockwise from the front faces you.
- ``subdivisionScheme`` fallback ``catmullClark`` → smooth; use ``none`` for final polygons.
- Keep ``extent`` in sync with ``points``; validate topology with ``Mesh.ValidateTopology``.

## 39.4 Normals

**What is it?** A **normal** is a direction perpendicular to the surface at a point. Renderers use it to compute lighting. On a mesh you can store normals in the `normals` attribute or as the primvar `primvars:normals`.

**Why do we need it?** Without authored normals, a renderer computes them from the polygons, which gives faceted or smooth shading by its own rules. Authored normals let an exporter keep the exact hard and soft edges an artist chose in the DCC.

**Beginner explanation.** Imagine tiny flagpoles stuck into the surface. Light is judged by the angle between the light and each flagpole. If two faces share a corner but each has its own flagpole there, the edge looks sharp; if they share one flagpole, the edge looks smooth.

*Where the analogy breaks:* the flagpoles are not geometry; they never change the shape, only the shading.

**Technical explanation.** - `normal3f[] normals` is defined on `UsdGeom.PointBased`. Its length depends on its **interpolation** metadata (`SetNormalsInterpolation`), which defaults to `vertex` (one per point). `faceVarying` (one per face-corner) is how hard edges are stored. - `normals` is **not** a primvar, although it has interpolation. You can instead author `primvars:normals` (a real primvar, so it can be **indexed**, see Chapter 11.4). - **Precedence:**

if both `normals` and `primvars:normals` are authored, **`primvars:normals` wins** (UsdGeomPointBased documentation). - Do **not** author normals on a mesh that is subdivided (`subdivisionScheme` not `none`): subdivision computes its own normals and ignores yours. Set `subdivisionScheme = "none"` when you export normals. - Lighting expects normals that point outward on the front side defined by `orientation`.

### Mental model.

```
Which normals does a renderer use?
subdivisionScheme != "none" → computed by subdivision (yours ignored)
else primvars:normals authored? → use primvars:normals
else normals authored? → use normals
else → renderer computes them
```

**Simple example.** A cube exported with hard edges has 8 points but 24 face-corners. Store 24 normals with `faceVarying` interpolation (each face's corners point straight out), or 6 unique normals in an indexed `primvars:normals` with 24 indices.

### USDA example.

```
#usda 1.0

def Mesh "Quad"
{
 int[] faceVertexCounts = [4]
 int[] faceVertexIndices = [0, 1, 2, 3]
 point3f[] points = [(0, 0, 0), (1, 0, 0), (1, 1, 0), (0, 1, 0)]
 uniform token subdivisionScheme = "none"
 normal3f[] normals = [(0, 0, -1), (0, 0, -1), (0, 0, -1), (0, 0, -1)] (
 interpolation = "vertex"
)
 normal3f[] primvars:normals = [(0, 0, 1)] (
 interpolation = "faceVarying"
)
 int[] primvars:normals:indices = [0, 0, 0, 0]
}
```

- Both forms are authored. The `primvars:normals` version (+Z, indexed, `faceVarying`) is the one renderers use.
- The stale `normals` (−Z) would shade the quad as if it faced away; it is ignored here.
- `subdivisionScheme = "none"` so the normals are honored at all.

### Python example.

```
from pxr import Usd, UsdGeom, Sdf

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/Quad")
mesh.CreatePointsAttr([(0, 0, 0), (1, 0, 0), (1, 1, 0), (0, 1, 0)])
mesh.CreateFaceVertexCountsAttr([4])
mesh.CreateFaceVertexIndicesAttr([0, 1, 2, 3])
mesh.CreateSubdivisionSchemeAttr("none")

print("default normals interpolation:", mesh.GetNormalsInterpolation())
mesh.CreateNormalsAttr([(0, 0, -1)] * 4)

pv = UsdGeom.PrimvarsAPI(mesh).CreatePrimvar(
 "normals", Sdf.ValueTypeNames.Normal3fArray, UsdGeom.Tokens.faceVarying)
pv.Set([(0, 0, 1)])
pv.SetIndices([0, 0, 0, 0])

api = UsdGeom.PrimvarsAPI(mesh)
if api.HasPrimvar("normals"):
 used = api.GetPrimvar("normals")
 print("using primvars:normals",
 [tuple(n) for n in used.ComputeFlattened()])
else:
 print("using normals", list(mesh.GetNormalsAttr().Get()))
```



```
print("is 'normals' a primvar?",
 UsdGeom.Primvar.IsPrimvar(mesh.GetNormalsAttr()))
```

### Expected output

```
default normals interpolation: vertex
using primvars:normals [(0.0, 0.0, 1.0), (0.0, 0.0, 1.0), (0.0, 0.0, 1.0), (0.0, 0.0, 1.0)]
is 'normals' a primvar? False
```

The `if` reproduces the documented rule (USD itself has no “effective normals” query; renderers apply it in Hydra). `ComputeFlattened()` expands the indexed primvar to one value per corner.

**Real-world use case.** A game-engine import pipeline reads `primvars:normals` (indexed, `faceVarying`) so hard edges survive and the data stays compact. A film studio exporting subdivision characters omits normals entirely, because the renderer derives them from the limit surface.

### Common mistakes.

#### MISTAKE

Fixing shading by editing ``normals`` while an old ``primvars:normals`` is still authored. Nothing changes. Fix: edit or block ``primvars:normals``, which has precedence.

#### MISTAKE

Authoring normals on a ``catmullClark`` mesh and wondering why they are ignored. Fix: set ``subdivisionScheme = "none"``.

#### MISTAKE

Writing one normal per point (``vertex``) for a hard-edged cube. Edges look soft. Fix: use ``faceVarying``.

### Exam traps.

#### TRAP

"``normals`` is a primvar because it has interpolation." No: only ``primvars:``-namespaced attributes are primvars. ``UsdGeom.Primvar.IsPrimvar(normalsAttr)`` is ``False``.

#### TRAP

When both are present, the answer "``normals`` wins because it is the schema attribute" is wrong; ``primvars:normals`` wins.

### Exam takeaways.

#### KEY

- ``normals`` (PointBased attribute, not a primvar) vs. ``primvars:normals`` (real, indexable primvar). - Both authored → `*`primvars:normals` wins*`. - Normals are only meaningful when ``subdivisionScheme = "none"``. - Hard edges = ``faceVarying`` normals.

## 39.5 GeomSubset

**What is it?** A `GeomSubset` is a child prim of a geometry prim that lists the indices of some of its elements (usually faces). It lets you name parts of one mesh, for example “the glass faces” of a car body.

**Why do we need it?** Exporters often merge many parts into one mesh for speed, but different faces need different materials. Without subsets you would have to split the mesh into many prims.

**Beginner explanation.** Think of a colouring book page with numbered regions. The page is the mesh. A subset is a sticky note saying “regions 3, 4, and 9 are sky”: you can now colour all sky regions blue without cutting the page apart.

*Where the analogy breaks:* subsets of one **family** can be required not to overlap (one region cannot be both “sky” and “sea”), depending on the family type.

**Technical explanation.** - Prim type `GeomSubset` (`UsdGeom.Subset`), defined as a **direct child** of the geometry prim. - Attributes: `uniform token elementType` (fallback `face`; also `point`, `edge`, `segment`, `tetrahedron` in 26.08), `int[] indices`, `uniform token familyName`. - A **family** is the set of subsets sharing a `familyName`. The parent stores the family type as `uniform token subsetFamily:<familyName>:familyType`: `unrestricted` (default), `nonOverlapping`, or `partition` (every element in exactly one subset). - Create: `UsdGeom.Subset.CreateGeomSubset(geom, name, elementType, indices, familyName="", familyType="")`. Query: `UsdGeom.Subset.GetGeomSubsets(geom, elementType, familyName)`, `GetUnassignedIndices(subsets, elementCount)`, `ValidateFamily(geom, elementType, familyName)`. - For materials the family name is **materialBind**. `UsdShade.MaterialBindingAPI(mesh).CreateMaterialBindSubset(name, indices)` creates such a subset and sets the family type to `nonOverlapping` (verified). Binding to subsets is covered in Chapter 40.6.

### Mental model.

```
Mesh "Body" (faces 0..5) subsetFamily:materialBind:familyType = "nonOverlapping"
├─ GeomSubset "paint" familyName="materialBind" indices=[0,1,2,3]
└─ GeomSubset "glass" familyName="materialBind" indices=[4,5]
```

**Simple example.** A mesh with 2 faces. Subset `left` = face 0, subset `right` = face 1, both in family `materialBind`. Unassigned faces: none.

### USDA example.

```
#usda 1.0

def Mesh "Body"
{
 int[] faceVertexCounts = [4, 4]
 int[] faceVertexIndices = [0, 1, 4, 3, 1, 2, 5, 4]
 point3f[] points = [(0, 0, 0), (1, 0, 0), (2, 0, 0),
 (0, 1, 0), (1, 1, 0), (2, 1, 0)]
 uniform token subdivisionScheme = "none"
 uniform token subsetFamily:materialBind:familyType = "nonOverlapping"

 def GeomSubset "paint"
 {
 uniform token elementType = "face"
 uniform token familyName = "materialBind"
 int[] indices = [0]
 }

 def GeomSubset "glass"
 {
 uniform token elementType = "face"
 uniform token familyName = "materialBind"
 int[] indices = [1]
 }
}
```

### Python example.

```
from pxr import Usd, UsdGeom, UsdShade

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/Body")
mesh.CreatePointsAttr([(0, 0, 0), (1, 0, 0), (2, 0, 0),
 (0, 1, 0), (1, 1, 0), (2, 1, 0)])
mesh.CreateFaceVertexCountsAttr([4, 4])
mesh.CreateFaceVertexIndicesAttr([0, 1, 4, 3, 1, 2, 5, 4])
```

```

paint = UsdGeom.Subset.CreateGeomSubset(
 mesh, "paint", UsdGeom.Tokens.face, [0], "materialBind")
print("after CreateGeomSubset:",
 UsdGeom.Subset.GetFamilyType(mesh, "materialBind"))

glass = UsdShade.MaterialBindingAPI.Apply(mesh.GetPrim()).CreateMaterialBindSubset(
 "glass", [1])
print("after CreateMaterialBindSubset:",
 UsdGeom.Subset.GetFamilyType(mesh, "materialBind"))

subsets = UsdGeom.Subset.GetGeomSubsets(mesh, "face", "materialBind")
for s in subsets:
 print(s.GetPrim().GetName(), s.GetElementTypeAttr().Get(),
 list(s.GetIndicesAttr().Get()))
print("unassigned:", list(UsdGeom.Subset.GetUnassignedIndices(subsets, 2)))
print("valid family:", UsdGeom.Subset.ValidateFamily(mesh, "face", "materialBind"))

```

### Expected output

```

after CreateGeomSubset: unrestricted
after CreateMaterialBindSubset: nonOverlapping
paint face [0]
glass face [1]
unassigned: []
valid family: (True, '')

```

**Real-world use case.** An automotive configurator exports each car body as one mesh with `materialBind` subsets for paint, chrome, glass, and rubber. Swapping the paint material changes one binding, and the mesh stays a single draw call.

### Common mistakes.

#### MISTAKE

Putting the `GeomSubset` somewhere other than directly under its mesh. It is not found. Fix: make it a direct child.

#### MISTAKE

Misspelling the family (``"material"`, `"materialBinding"``). Material binding to subsets ignores it. Fix: use ``"materialBind"`, or `CreateMaterialBindSubset`.`

#### MISTAKE

Leaving faces outside every subset and expecting them to be unshaded. They get the mesh's own binding. Check with ``GetUnassignedIndices``.

### Exam traps.

#### TRAP

The family name for material subsets is ``materialBind``, not the material's name.

#### TRAP

The fallback ``elementType`` is ``face``. The family type lives on the **parent** mesh (``subsetFamily::familyType``), not on the subset.

### Exam takeaways.

#### KEY

- ``GeomSubset`` = direct child; ``elementType`` (fallback ``face``), ``indices``, ``familyName``.
- Material subsets: ``familyName`` = `"materialBind"`; family type stored on the mesh.
- Create with ``UsdGeom.Subset.CreateGeomSubset`` or ``MaterialBindingAPI.CreateMaterialBindSubset`` (sets ``nonOverlapping``).

## 39.6 Visibility and purpose

**What is it?** `visibility` hides a prim and everything under it. `purpose` labels a prim as `default`, `render`, `proxy`, or `guide`, so a renderer or viewer can choose which categories to draw. Both come from `UsdGeom.Imageable`.

**Why do we need it?** Scenes contain things that should not always appear: hidden props, light-weight stand-ins for the viewport, heavy final geometry, and helper curves for riggers. Deleting them would destroy data; `visibility` and `purpose` switch them off non-destructively.

**Beginner explanation.** Visibility is a light switch for a whole room: switch off the room and every lamp in it goes dark. Purpose is a label on each lamp: “work light”, “display lamp”, “night light”. The building manager decides which labels are on tonight.

*Where the analogy breaks:* a lamp inside a dark room cannot be switched on by itself in USD; `MakeVisible()` has to change the room too.

**Technical explanation.** - token `visibility` — `inherited` (fallback) or `invisible`.

**Animatable.** `invisible` on a prim hides all its descendants regardless of their own opinion. `ComputeVisibility(time)` returns the effective value. - `MakeInvisible()` authors `invisible`. `MakeVisible()` on a prim under an invisible ancestor sets the ancestor to `inherited` and **makes the siblings invisible** to keep them hidden (verified). - uniform token `purpose` — `default` (fallback), `render` (final quality, for renders), `proxy` (light stand-in, for interactive viewports), `guide` (helpers, usually never rendered). **Not animatable.** - Purpose **inherits**: a prim with no authored purpose takes its nearest ancestor’s authored purpose. `ComputePurpose()` returns the effective value; `GetOrderedPurposeTokens()` lists all four. - `proxyPrim` relationship on a `render` prim can point to its `proxy` stand-in. - Viewers typically show `default` + `proxy`; final renders show `default` + `render`. `UsdGeom.BBoxCache` takes the purposes to include (Chapter 44.5).

### Mental model.

```
/Tree (default)
├─ Hi_Res purpose = "render" ← final frames only
├─ Lo_Res purpose = "proxy" ← viewport only
└─ Pivot purpose = "guide" ← helpers, off by default
visibility = "invisible" on /Tree → nothing under /Tree is drawn
```

**Simple example.** Tree with a render mesh and a proxy mesh. In `usdview` (`proxy` on) you see the light mesh; in the final render you see the full one. Set `/Tree` `invisible`: neither appears.

### USDA example.

```
#usda 1.0

def Xform "Tree"
{
 def Mesh "Hi_Res"
 {
 uniform token purpose = "render"
 rel proxyPrim = </Tree/Lo_Res>
 }

 def Mesh "Lo_Res"
 {
 uniform token purpose = "proxy"
 }

 def Xform "Helpers"
 {
 uniform token purpose = "guide"

 def BasisCurves "PivotAxis"
 {
 }
 }
}
```

```

 }

 def Mesh "Leaves"
 {
 token visibility = "invisible"
 }
}

```

- `PivotAxis` has no purpose of its own but inherits `guide` from `Helpers`.
- `Leaves` is hidden; `visibility` could also be time-sampled to make leaves fall.

### Python example.

```

from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
tree = UsdGeom.Xform.Define(stage, "/Tree")
helpers = UsdGeom.Xform.Define(stage, "/Tree/Helpers")
axis = UsdGeom.BasisCurves.Define(stage, "/Tree/Helpers/Axis")
a = UsdGeom.Mesh.Define(stage, "/Tree/A")
b = UsdGeom.Mesh.Define(stage, "/Tree/B")

print("fallbacks:", tree.GetVisibilityAttr().Get(), tree.GetPurposeAttr().Get())
helpers.CreatePurposeAttr(UsdGeom.Tokens.guide)
print("Axis purpose (inherited):", axis.ComputePurpose())
print("all purposes:", list(tree.GetOrderedPurposeTokens()))

tree.MakeInvisible()
print("A effective:", a.ComputeVisibility(),
 "| A authored:", a.GetVisibilityAttr().Get())

a.MakeVisible()
print("after A.MakeVisible():")
for x in (tree, a, b):
 print(" ", x.GetPath(), "authored", x.GetVisibilityAttr().Get(),
 "effective", x.ComputeVisibility())

```

### Expected output

```

fallbacks: inherited default
Axis purpose (inherited): guide
all purposes: ['default', 'render', 'proxy', 'guide']
A effective: invisible | A authored: inherited
after A.MakeVisible():
 /Tree authored inherited effective inherited
 /Tree/A authored inherited effective inherited
 /Tree/B authored invisible effective invisible

```

`MakeVisible()` on A had to flip `/Tree` back to `inherited`, then hid sibling B so it stays invisible.

**Real-world use case.** A forest of 10,000 trees uses `proxy` cards in the layout viewport and `render` meshes on the render farm; artists never wait for full trees to draw. Rigging controls and collision shapes are `guide`, so they never leak into final frames. Shot animators animate `visibility` to pop props in and out.

### Common mistakes.

#### MISTAKE

Setting a child to `"visible"`. That token does not exist; the values are `"inherited"` and `"invisible"`. Fix: use `MakeVisible()`.

#### MISTAKE

Trying to time-sample `purpose`. It is `"uniform"`. Fix: animate `visibility` instead, or use variants.

**MISTAKE**

Tagging the root of an asset ``proxy`` and wondering why the final render is empty. Purpose inherits to every child. Fix: tag only the proxy branch.

**Exam traps.****TRAP**

``visibility`` is animatable; ``purpose`` is not.

**TRAP**

A child cannot be "more visible" than an invisible parent. Effective visibility is invisible if any ancestor is invisible.

**TRAP**

Purpose ``default`` means "always drawn", not "unknown". Both render and viewport include it.

**Exam takeaways.****KEY**

- ``visibility``: ``inherited`` | ``invisible``; animatable; invisible ancestor hides all descendants. - ``purpose``: ``default`` | ``render`` | ``proxy`` | ``guide``; uniform; inherits down namespace. - ``MakeVisible()`` edits ancestors and siblings to reveal one prim. - Viewports usually draw default + proxy; renders draw default + render.

## 39.7 Cameras

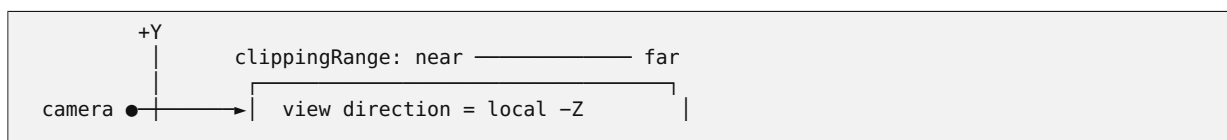
**What is it?** A `UsdGeom.Camera` prim describes a physical camera: lens (focal length), film back (apertures), clipping planes, and depth-of-field settings. Its transform places it in the scene. `camera.GetCamera(time)` converts it to `Gf.Camera`, a math object for computing fields of view and frustums.

**Why do we need it?** Layout, animation, and lighting must all render from the same shot camera. Storing it as scene description means every application and renderer uses the exact same lens and framing.

**Beginner explanation.** A camera prim is the spec sheet of a real camera taped to a tripod. The spec sheet says "35 mm lens, full-frame sensor"; the tripod's position is the prim's `xformOps`.

*Where the analogy breaks:* USD measures lens and sensor in **tenths of a scene unit**, not always millimetres.

**Technical explanation.** - Attributes and fallbacks (verified on 26.08): `projection = "perspective"` (or `orthographic`), `focalLength = 50`, `horizontalAperture = 20.955`, `verticalAperture = 15.2908`, `clippingRange = (1, 1000000)` (near, far in scene units), `focusDistance = 0`, `fStop = 0` (0 means no depth of field). - Units: `focalLength` and apertures are in **tenths of a scene unit**. With `metersPerUnit = 0.01` (centimetres), that is millimetres, matching real lenses. - In its own space, a camera looks down **-Z** with **+Y** up. Point it with normal `xformOps`. - `UsdGeom.Camera(prim).GetCamera(time) → Gf.Camera` with `focalLength`, `horizontalAperture`, `clippingRange`, `transform`, `projection`, `frustum`, and `GetFieldOfView(Gf.Camera.FOVHorizontal)`. - `camera.SetFromCamera(gfCamera, time)` writes a `Gf.Camera` back. - Camera attributes are animatable (focus pulls, zooms).

**Mental model.**

$$\text{FOV} \approx 2 \cdot \text{atan}(\text{horizontalAperture} / (2 \cdot \text{focalLength}))$$

**Simple example.** A 35 mm lens on the default 20.955 aperture gives a horizontal field of view of about 33.3°. Moving the camera to  $z = 10$  and leaving no rotation makes it look back at the origin.

### USDA example.

```
#usda 1.0
(
 metersPerUnit = 0.01
 upAxis = "Y"
)

def Camera "ShotCam"
{
 float focalLength = 35
 float horizontalAperture = 36
 float verticalAperture = 24
 float2 clippingRange = (0.1, 10000)
 float focusDistance = 500
 float fStop = 2.8
 token projection = "perspective"
 double3 xformOp:translate = (0, 150, 800)
 uniform token[] xformOpOrder = ["xformOp:translate"]
}
```

- With centimetre units, 35 and 36 read as 35 mm and 36 mm (full-frame).
- `fStop = 2.8` with `focusDistance = 500` (5 m) enables depth of field.
- The camera sits 1.5 m up and 8 m back and looks down  $-Z$  toward the origin.

### Python example.

```
from pxr import Usd, UsdGeom, Gf

stage = Usd.Stage.CreateInMemory()
cam = UsdGeom.Camera.Define(stage, "/ShotCam")
print("fallback focal/hAperture/clip:", cam.GetFocalLengthAttr().Get(),
 round(cam.GetHorizontalApertureAttr().Get(), 3),
 cam.GetClippingRangeAttr().Get())

cam.AddTranslateOp().Set(Gf.Vec3d(0, 0, 10))
cam.GetFocalLengthAttr().Set(35.0)
cam.GetClippingRangeAttr().Set(Gf.Vec2f(0.1, 1000))

gf = cam.GetCamera(Usd.TimeCode.Default())
print(type(gf).__name__, gf.projection)
print("focal", gf.focalLength, "clip", gf.clippingRange)
print("hFOV", round(gf.GetFieldOfView(Gf.Camera.FOVHorizontal), 2))
print("position", gf.transform.ExtractTranslation())
print("view dir", Gf.Vec3d(*[round(v, 6) + 0.0 for v in
 gf.frustum.ComputeViewDirection()])))
```

### Expected output

```
fallback focal/hAperture/clip: 50.0 20.955 (1, 1000000)
Camera Gf.Camera.Perspective
focal 35.0 clip [0.1...1000]
hFOV 33.33
position (0, 0, 10)
view dir (0, 0, -1)
```

**Real-world use case.** In film, layout publishes `ShotCam` to a camera layer; animation, lighting, and comp all reference it, so a focal-length change propagates to every department. In manufacturing, inspection cameras in a digital twin are stored as `Camera` prims so simulated images match the real cameras' lenses.

### Common mistakes.

**MISTAKE**

Setting `focalLength = 0.035`` because "35 mm is 0.035 m". Units are tenths of a scene unit. Fix: in a centimetre scene use 35.

**MISTAKE**

A tiny object disappears close to the camera. The near clip (fallback 1 unit) is too far. Fix: lower `clippingRange[0]``.

**MISTAKE**

Rotating the camera expecting it to look down +Z. It looks down local -Z.

**Exam traps.****TRAP**

`GetCamera()` returns a `Gf.Camera``, not a `UsdGeom.Camera``. Questions may ask which type gives `GetFieldOfView``.

**TRAP**

`clippingRange`` is in scene units; focal length and aperture are in tenths of a scene unit. Mixing them is a classic distractor.

**Exam takeaways.****KEY**

- `UsdGeom.Camera``: `focalLength`` (50), `horizontalAperture`` (20.955), `clippingRange`` (1, 1e6), `projection``. - Lens/aperture units = tenths of a scene unit; clipping = scene units.  
- Looks down local -Z, +Y up; placed with `xformOps`. - `GetCamera(time)`` → `Gf.Camera`` (FOV, frustum).

**Chapter lab(s)**

**Lab 33 — Xforms, XformCache, and cameras** (★★☆, Obj 8.x). You build a small rig of nested Xforms, predict world positions from `xformOpOrder` before checking them with `XformCache`, add a `resetXformStack` child, and create a shot camera whose field of view you compute through `Gf.Camera`.

**Lab 11 — Build a mesh from scratch and compute its extent** (Ch 13, 39; Obj 5.6) also exercises §39.3: `author points/faceVertexCounts/faceVertexIndices`, then keep `extent` in sync.

**USDA reading exercise(s)**

**Exercise 39-A.** Where does the point (1, 0, 0) of `Arm` end up in world space?

```
#usda 1.0

def Xform "Base"
{
 double3 xformOp:translate = (0, 5, 0)
 uniform token[] xformOpOrder = ["xformOp:translate"]

 def Xform "Arm"
 {
 double3 xformOp:translate = (2, 0, 0)
 float xformOp:rotateZ = 90
 float3 xformOp:scale = (3, 3, 3)
 uniform token[] xformOpOrder = ["xformOp:translate", "xformOp:rotateZ"]
 }
}
```

**Exercise 39-B.** Which normals does a renderer use for `Wall`, and why might your edit to `normals` have had no effect?



```
#usda 1.0

def Mesh "Wall"
{
 int[] faceVertexCounts = [4]
 int[] faceVertexIndices = [0, 1, 2, 3]
 point3f[] points = [(0, 0, 0), (1, 0, 0), (1, 1, 0), (0, 1, 0)]
 normal3f[] normals = [(0, 0, 1), (0, 0, 1), (0, 0, 1), (0, 0, 1)]
}
```

**Exercise 39-C.** Is `/Set/Lamp/Bulb` drawn in a final render? In a viewport showing proxy?

```
#usda 1.0

def Xform "Set"
{
 def Xform "Lamp"
 {
 uniform token purpose = "proxy"

 def Mesh "Bulb"
 {
 token visibility = "inherited"
 }
 }
}
```

## Chapter review

### Summary

- `xformOps` are attributes; `xformOpOrder` chooses which apply. Read it right to left for a point.
- `XformCache(time).GetLocalToWorldTransform(prim)` gives world matrices efficiently; `!resetXformStack!` ignores ancestors.
- A Mesh = `points` + `faceVertexCounts` (per face) + `faceVertexIndices` (per corner).
- Fallbacks: `orientation = rightHanded`, `subdivisionScheme = catmullClark`, `doubleSided = false`.
- `primvars:normals` beats `normals`; `normals` only matter when `subdivisionScheme = "none"`.
- `GeomSubsets` name faces of one mesh; `familyName = "materialBind"` for materials.
- `visibility` (`inherited/invisible`) is animatable and prunes descendants; `purpose` is uniform and inherited.
- Cameras: `lens/aperture` in tenths of a scene unit, look down `-Z`, `GetCamera()` → `Gf.Camera`.

### If you see... → think...

If you see...	Think...
An <code>xformOp:*</code> attribute not in <code>xformOpOrder</code>	It has no effect
<code>["xformOp:translate", "xformOp:rotateXYZ", "xformOp:scale"]</code>	Scale first, rotate, then translate
<code>"!resetXformStack!"</code>	Ignore parent transforms
Many world matrices needed	<code>UsdGeom.XformCache</code>
Melted/rounded hard-surface mesh	<code>subdivisionScheme</code> still <code>catmullClark</code> → set <code>none</code>
Normals edits ignored	<code>primvars:normals</code> authored, or mesh is subdivided
<code>familyName = "materialBind"</code>	Per-face material binding via <code>GeomSubset</code>

If you see...	Think...
A child shows inherited but is hidden	An ancestor is invisible
purpose = "proxy"	Viewport-only stand-in
GetCamera()	Returns Gf.Camera

## Review questions

**Q39.1** · Obj 8.1 · Medium · Single choice A prim's `xformOpOrder` is `["xformOp:rotateZ", "xformOp:translate"]` with `rotateZ = 90` and `translate = (1, 0, 0)`. Where does the origin (0, 0, 0) go? A. (1, 0, 0) · B. (0, 1, 0) · C. (0, 0, 0) · D. (-1, 0, 0)

**Q39.2** · Obj 8.1 · Easy · Single choice Which Python call computes a prim's world matrix with caching? A. `UsdGeom.Xformable.GetLocalTransformation` · B. `UsdGeom.XformCache().GetLocalToWorldTransform` · C. `Gf.Matrix4d.GetInverse` · D. `Usd.Prim.GetTransform`

**Q39.3** · Obj 8.1 · Medium · Select two. Which statements about `resetXformStack` are true? A. It is written as the first entry of `xformOpOrder` · B. It is a boolean attribute named `resetXformStack` · C. It makes the prim ignore ancestor transforms · D. It resets the prim's own ops to identity

**Q39.4** · Obj 5.5 · Medium · Single choice An imported CAD bracket looks soft and rounded, though its triangles are correct. What is the most likely cause? A. `orientation = "leftHanded"` · B. Missing `extent` · C. `subdivisionScheme` is `unauthored` · D. `doubleSided = false`

**Q39.5** · Obj 8.1 · Easy · Single choice `faceVertexCounts = [4, 4, 3]`. How long must `faceVertexIndices` be? A. 3 · B. 4 · C. 11 · D. 12

**Q39.6** · Obj 5.5 · Medium · Single choice A mesh authors `normals` (pointing +Y) and `primvars:normals` (pointing -Y) and has `subdivisionScheme = "none"`. Which way does lighting treat the surface as facing? A. +Y · B. -Y · C. Average of both · D. Computed from winding, both ignored

**Q39.7** · Obj 8.3 · Medium · Select two. Which are required for a `GeomSubset` to receive its own material binding? A. `familyName = "materialBind"` · B. The subset is a direct child of the mesh · C. `elementType = "point"` · D. The subset is referenced from a material layer

**Q39.8** · Obj 5.5 · Medium · Single choice `/Car` is invisible. You call `UsdGeom.Imageable(wheel).MakeVisible()` on `/Car/Wheel`. Which edit does USD make? A. Authors `visibility = "visible"` on `Wheel` · B. Sets `/Car` to `inherited` and makes `Wheel`'s siblings `invisible` · C. Nothing, an error is raised · D. Removes `/Car` from the stage

**Q39.9** · Obj 5.5 · Easy · Single choice Which attribute can be time-sampled? A. `purpose` · B. `xformOpOrder` · C. `visibility` · D. `subdivisionScheme`

**Q39.10** · Obj 8.1 · Easy · Single choice What does `UsdGeom.Camera.GetCamera(time)` return? A. `UsdGeom.Camera` · B. `Gf.Frustum` · C. `Gf.Camera` · D. `Gf.Matrix4d`

**Q39.11** · Obj 8.1 · Medium · USDA reading What is the world position of `/A/B`'s origin?

```
#usda 1.0

def Xform "A"
{
 double3 xformOp:translate = (0, 0, 4)
 uniform token[] xformOpOrder = ["xformOp:translate"]

 def Xform "B"
 {
 double3 xformOp:translate = (1, 0, 0)
 double3 xformOp:translate:extra = (9, 9, 9)
 uniform token[] xformOpOrder = ["xformOp:translate"]
 }
}
```

A. (1, 0, 4) · B. (10, 9, 13) · C. (1, 0, 0) · D. (0, 0, 4)

**Q39.12** · Obj 8.1 · Easy · Single choice In a scene with `metersPerUnit = 0.01`, which `focalLength` value describes a 50 mm lens? A. 0.05 · B. 5 · C. 50 · D. 500

### Answers

**Q39.1 — B.** Translate is last, so it applies first: (1, 0, 0); rotating 90° about Z gives (0, 1, 0). A ignores the rotation. Review: §39.1.

**Q39.2 — B.** `XformCache` stores ancestor results. A gives the local matrix; C and D are not world-transform APIs. Review: §39.2.

**Q39.3 — A, C.** The marker `!resetXformStack!` goes first in `xformOpOrder` and discards ancestors. There is no such attribute (B); the prim's own ops still apply (D). Review: §39.2.

**Q39.4 — C.** The fallback is `catmullClark`, which smooths the mesh. Orientation affects facing, extent affects culling, `doubleSided` affects back faces. Review: §39.3.

**Q39.5 — C.**  $4 + 4 + 3 = 11$ . Review: §39.3.

**Q39.6 — B.** `primvars:normals` has precedence over `normals`. Review: §39.4.

**Q39.7 — A, B.** Material subsets use family `materialBind` and must be direct children. `point` subsets do not carry face materials; no reference is needed. Review: §39.5 and §40.6.

**Q39.8 — B.** `MakeVisible()` makes ancestors `inherited` and hides siblings to preserve their look. "visible" is not a valid token. Review: §39.6.

**Q39.9 — C.** `visibility` is a varying token. The others are `uniform`. Review: §39.6.

**Q39.10 — C.** Review: §39.7.

**Q39.11 — A.** Only `xformOp:translate` is listed, so B is at (1, 0, 0) in A's space, (1, 0, 4) in the world. The `:extra` op is ignored. Review: §39.1-39.2.

**Q39.12 — C.** Units are tenths of a scene unit; with centimetres that is millimetres. Review: §39.7.

### USDA exercise answers

**39-A — (2, 6, 0).** Scale is authored but not listed, so it is ignored. `RotateZ` applies first: (1, 0, 0) → (0, 1, 0). Translate (2, 0, 0) → (2, 1, 0). Parent Base adds (0, 5, 0) → **(2, 6, 0)**.

**39-B — Normals computed by subdivision, not the authored ones.**

`subdivisionScheme` is unauthored, so it is `catmullClark`; the mesh is subdivided and its normals come from subdivision. Set `subdivisionScheme = "none"` to make `normals` count (and check no `primvars:normals` exists, which would win).

**39-C — Final render: no. Proxy viewport: yes.** `Bulb` has no purpose of its own and inherits `proxy` from `Lamp`. Renders skip proxy; viewports showing proxy draw it. Its `visibility` is `inherited` and no ancestor is invisible.

## CHAPTER 40

# Chapter 40 — UsdShade: Materials and Shaders

**Exam domain:** Visualization (8%) · **Objectives:** 8.2, 8.3, 8.4 · **Study day:** 11 · **Est. time:** 110 min **Prerequisites:** Ch 5 (properties), Ch 6 (schemas / API schemas), Ch 11 (primvars), Ch 29 (exporter PreviewSurface sketch), Ch 39 (meshes, GeomSubset)

A mesh without a material is a grey shape. **UsdShade** is the schema domain that describes *what it looks like*: a **Material** prim, the **Shader** nodes inside it, the connections between their inputs and outputs, and the **binding** that assigns that material to geometry. This chapter is the whole Visualization domain’s material half: assign PreviewSurface (Obj 8.2), bind it to a mesh (Obj 8.3), and drive diffuse color from a primvar (Obj 8.4).

Chapter 29 showed a one-shader bind as part of an exporter. Here you learn the network, the purposes, the strength, and the public knobs you expose for downstream overrides.

## Learning goals

- Author a Material with nested Shader and NodeGraph prims, and explain why a Material is a NodeGraph.
- Connect shader outputs to inputs, including the Material’s `outputs:surface` terminal.
- Build a UsdPreviewSurface shader with the inputs Hydra preview renderers understand.
- Wire UsdUVTexture and UsdPrimvarReader\_float2 so a texture follows mesh UVs.
- Build the Obj 8.4 network: UsdPrimvarReader\_float3 reading displayColor into diffuseColor.
- Apply MaterialBindingAPI, bind with purpose and strength, and resolve the bound material.
- Expose Material interface inputs so a shot layer can recolor an asset without editing the shader graph.

## Key terms

Term	One-line definition
<b>Material</b>	A UsdShade.Material prim: the container you bind to geometry.
<b>Shader</b>	A UsdShade.Shader prim: one node in the graph, identified by <code>info:id</code> .
<b>NodeGraph</b>	A container for connected shaders; Material derives from it.
<b>info:id</b>	Uniform token on a Shader naming the implementation (UsdPreviewSurface, UsdUVTexture, ...).
<b>Connectable</b>	A prim whose inputs and outputs can be wired (ConnectableAPI).
<b>Connection</b>	An attribute with <code>.connect</code> targeting another input or output.
<b>UsdPreviewSurface</b>	The standard interchange shader id for preview / lookdev.
<b>Binding</b>	The relationship from geometry to a Material ( <code>material:binding</code> ).
<b>MaterialBindingAPI</b>	Applied API schema that makes a prim a legal binding target.
<b>Purpose</b>	Which binding to use: all-purpose <code>'</code> , <code>preview</code> , or <code>full</code> .
<b>bindMaterialAs</b>	Strength metadata: <code>strongerThanDescendants</code> or <code>weakerThanDescendants</code> .
<b>Interface input</b>	A Material-level input that shaders connect <i>to</i> , so shots can override one knob.

## 40.1 Material, Shader, NodeGraph

**What is it?** **UsdShade** describes shading as a small scene graph of its own. A **Material** is the named look you bind to geometry. A **Shader** is one node in that look (a preview surface, a texture, a primvar reader). A **NodeGraph** groups nodes. A Material is a NodeGraph, so it can own inputs, outputs, and child shaders.

**Why do we need it?** Geometry (Chapter 39) answers “where is the surface?” Shading answers “how does it respond to light?” Without a Material, every DCC invents its own look encoding and interchange dies. UsdShade is the shared encoding.

**Beginner explanation.** Think of a recipe card in a binder. The **Material** is the card you pull (“Wood”). The **Shader** nodes are the steps on the card (“mix brown paint”, “read the grain photo”). A **NodeGraph** is a sub-recipe you reuse (“make wood grain”) that is not itself the card you hand the cook.

*Where the analogy breaks:* the Material card is also a sub-recipe (it is a NodeGraph). You do not keep a separate empty binder for the card.

**Technical explanation.** - Concrete typed schemas: `UsdShade.Material`, `UsdShade.Shader`, `UsdShade.NodeGraph`. Define with `UsdShade.Material.Define(stage, path)` (same pattern as `UsdGeom.Mesh.Define`). - **Material IsA NodeGraph is True.** Shader is *not* a NodeGraph. NodeGraph is *not* a Material. Verified on USD 26.08. - A Shader’s implementation is **not** its prim type. The prim type is always `Shader`. The implementation is the uniform token `info:id`, set with `shader.CreateIdAttr("UsdPreviewSurface")`. - Conventional layout: `/Looks/<MaterialName>/<ShaderName>`. `/Looks` is often a Scope (no transform). The Material is what you bind; the shaders stay *inside* it so the look ships as one prim tree. - Shaders and materials are connectable (`UsdShade.ConnectableAPI`). That is how §40.2 wires them. - **MaterialX** (`UsdMtlx`) is a separate plugin. `usd-core` 26.8 does not ship it. The exam’s interchange shader is `PreviewSurface`, not `MaterialX`.

### VERSION

Verified on USD 26.08 (`usd-core` 26.8). ``UsdMtlx`` is absent. Do not write ``from pxr import UsdMtlx`` in runnable examples.

### Mental model.

```
Scope "Looks"
├─ Material "Wood" <- this is what you BIND (also a NodeGraph)
│ ├── Shader "Preview" info:id = UsdPreviewSurface
│ ├── Shader "DiffuseTex" info:id = UsdUVTexture
│ └─ NodeGraph "Utility" <- nested graph, not bindable by itself
```

Bind **Wood**, never the Preview shader.

**Simple example.** Three prims: `/Looks/Wood` (Material), `/Looks/Wood/Preview` (Shader), `/Looks/Wood/Utility` (NodeGraph). Types: Material, Shader, NodeGraph. Wood IsA NodeGraph.

### USDA example.

```
#usda 1.0
(
 defaultPrim = "Looks"
)

def Scope "Looks"
{
 def Material "Wood"
 {
 def Shader "Preview"
 {
 uniform token info:id = "UsdPreviewSurface"
 }

 def NodeGraph "Utility"
```

```

 {
 }
}

```

- `def Material "Wood"` is the bind target.
- `info:id` lives on the **Shader**, not on the Material.
- `Utility` is a nested graph; nothing binds to it.

### Python example.

```

from pxr import Usd, UsdShade

stage = Usd.Stage.CreateInMemory()
mat = UsdShade.Material.Define(stage, "/Looks/Wood")
shader = UsdShade.Shader.Define(stage, "/Looks/Wood/Preview")
graph = UsdShade.NodeGraph.Define(stage, "/Looks/Wood/Utility")

print("Material type:", mat.GetPrim().GetTypeName())
print("Shader type:", shader.GetPrim().GetTypeName())
print("NodeGraph type:", graph.GetPrim().GetTypeName())
print("Material IsA NodeGraph:", mat.GetPrim().IsA(UsdShade.NodeGraph))
print("Shader IsA NodeGraph:", shader.GetPrim().IsA(UsdShade.NodeGraph))
print("NodeGraph IsA Material:", graph.GetPrim().IsA(UsdShade.Material))

```

### Expected output

```

Material type: Material
Shader type: Shader
NodeGraph type: NodeGraph
Material IsA NodeGraph: True
Shader IsA NodeGraph: False
NodeGraph IsA Material: False

```

The IsA line is the fact to memorize: Material is a NodeGraph; the reverse is false.

**Real-world use case.** A furniture catalog publishes each wood species as `/Looks/Oak`, `/Looks/Walnut`, each a Material with a PreviewSurface child. A set-dressing USD references the catalog and binds `/Looks/Oak` onto table meshes. Artists never bind the Preview shader; if they did, a later extra texture node inside the Material would be skipped.

### Common mistakes.

#### MISTAKE

Putting `info:id = "UsdPreviewSurface"` on the Material prim. Hydra looks for a `*Shader*` with that id, connected to `outputs:surface``. Fix: author a child Shader.

#### MISTAKE

Binding `/Looks/Wood/Preview`` instead of `/Looks/Wood``. The relationship can point at a Shader, but binding APIs and validators expect a `**Material**`.

#### MISTAKE

Treating NodeGraph as a Material. `IsA(UsdShade.Material)`` on a NodeGraph is False; you cannot bind it as a look.

### Exam traps.

#### TRAP

`"`UsdPreviewSurface` is a prim type."` It is an `info:id` **value**`. The prim type is ``Shader``.

#### TRAP

"A Material is not connectable; only Shaders are." Material IsA NodeGraph, so it has inputs and outputs. That is how §40.2 and §40.7 work.

**TRAP**

Questions that offer `UsdMtlx` as the only way to preview. PreviewSurface is the required interchange path; MaterialX is optional and missing from `usd-core`.

**Exam takeaways.****KEY**

- Bind a **Material**. Put **Shader** children inside it. `info:id` names the shader implementation. - Material IsA NodeGraph (True); Shader IsA NodeGraph (False). - `UsdPreviewSurface` is an id string, not a schema type. - `usd-core` has no UsdMtlx; PreviewSurface is the runnable interchange shader.

## 40.2 Inputs, outputs, connections

**What is it?** A shader node has **inputs** (values it consumes) and **outputs** (values it produces). A **connection** wires one output (or a NodeGraph/Material interface input) to another input. USDA writes this as `inputs:foo.connect = </Path.outputs:bar>`.

**Why do we need it?** A PreviewSurface that always uses a constant brown cannot show a wood photo or a per-face paint color. Connections are how you build a **network**: reader → texture → surface, and surface → the Material terminal that Hydra actually draws.

**Beginner explanation.** Think of patch cables on a small analog synth. Each module has jacks in (inputs) and jacks out (outputs). A cable from Texture's `rgb` into Preview's `diffuseColor` is a connection. The Material's `outputs:surface` is the speaker output: if that cable is unplugged, you hear (see) nothing.

*Where the analogy breaks:* USD will let you author a constant *and* a cable on the same input. The cable wins for shading; the constant remains in the file (see the Python example).

**Technical explanation.** - Create with `shader.CreateInput("diffuseColor", Sdf.ValueTypeNames.Color3f)` and `shader.CreateOutput("surface", Sdf.ValueTypeNames.Token)`. Property names in USDA are `inputs:diffuseColor` and `outputs:surface`. - Connect: `dest.ConnectToSource(sourceOutput)` or `dest.ConnectToSource(connectableAPI, "surface")`. Query: `HasConnectedSource()`, `GetConnectedSources()` → (list of `ConnectionSourceInfo`, `invalidPaths`). - The Material terminal Hydra preview uses is **outputs:surface**, created with `material.CreateSurfaceOutput()`. Also: `CreateDisplacementOutput()` → `outputs:displacement`, `CreateVolumeOutput()` → `outputs:volume`. A render-context suffix is allowed: `CreateSurfaceOutput("ri")` → `outputs:ri:surface`. - `material.ComputeSurfaceSource()` returns (shader, outputName, attributeType) for the connected surface shader. Unpacking is required; it is a tuple, not a Shader. - A connected input can still hold an authored **value**. `Get()` returns the value; `HasConnectedSource()` is True. Renderers follow the connection. - Connections compose like other attributes: a stronger layer can reconnect or disconnect.

**Mental model.**

```
Material.outputs:surface <— connect — Shader.outputs:surface
 |
 inputs:diffuseColor (value and/or connect)
```

If `outputs:surface` on the Material is not connected, the look has no surface terminal.

**Simple example.** Shader Preview has `inputs:diffuseColor = (0.6, 0.4, 0.2)` and `outputs:surface`. Material Wood connects its `outputs:surface` to Preview's `outputs:surface`. `ComputeSurfaceSource` → `/Looks/Wood/Preview`.

**USD example.**

```
#usda 1.0

def Material "Wood"
{
 token outputs:surface.connect = </Wood/Preview.outputs:surface>

 def Shader "Preview"
 {
 uniform token info:id = "UsdPreviewSurface"
 color3f inputs:diffuseColor = (0.6, 0.4, 0.2)
 token outputs:surface
 }
}
```

- `.connect` is authored on the **destination**.
- The target path is `prim + .outputs:surface` (or `.inputs:name` for interface inputs in §40.7).
- `outputs:surface` on the Shader can be declared with no value; the connection only needs the name to exist.

### Python example.

```
from pxr import Usd, UsdShade, Sdf

stage = Usd.Stage.CreateInMemory()
mat = UsdShade.Material.Define(stage, "/Looks/Wood")
ps = UsdShade.Shader.Define(stage, "/Looks/Wood/Preview")
ps.CreateIdAttr("UsdPreviewSurface")
diff = ps.CreateInput("diffuseColor", Sdf.ValueTypeNames.Color3f)
diff.Set((0.6, 0.4, 0.2))
surf = ps.CreateOutput("surface", Sdf.ValueTypeNames.Token)
mat_out = mat.CreateSurfaceOutput()
mat_out.ConnectToSource(surf)

print("shader id:", ps.GetIdAttr().Get())
print("has connect:", mat_out.HasConnectedSource())
shader, name, _ = mat.ComputeSurfaceSource()
print("surface source:", shader.GetPath(), name)
cs, invalid = mat_out.GetConnectedSources()
print("connected count:", len(cs), "invalid:", invalid)
for line in stage.GetRootLayer().ExportToString().splitlines():
 if "connect" in line or "diffuseColor" in line:
 print(line.strip())
r = UsdShade.Shader.Define(stage, "/Looks/Wood/Reader")
r.CreateOutput("result", Sdf.ValueTypeNames.Float3)
diff.ConnectToSource(r.GetOutput("result"))
print("after connect Get():", diff.Get())
print("HasConnectedSource:", diff.HasConnectedSource())
```

### Expected output

```
shader id: UsdPreviewSurface
has connect: True
surface source: /Looks/Wood/Preview surface
connected count: 1 invalid: []
token outputs:surface.connect = </Looks/Wood/Preview.outputs:surface>
color3f inputs:diffuseColor = (0.6, 0.4, 0.2)
after connect Get(): (0.6, 0.4, 0.2)
HasConnectedSource: True
```

After the extra connect, USDA would also show `inputs:diffuseColor.connect`. `Get()` still returns the leftover `(0.6, 0.4, 0.2)`.

**Real-world use case.** A lookdev artist in `usdview` (or any Hydra viewport) sees brown wood because `outputs:surface` is connected. A lighting TD then reconnects `diffuseColor` to a texture in a stronger shot layer without replacing the Material prim: composition edits the `.connect` target (Chapter 15 edit targets).

### Common mistakes.



**MISTAKE**

Calling ``ComputeSurfaceSource()`` and treating the return as a Shader. It is a **tuple** `(shader, name, type)``. Unpacking, then ``shader.GetPath()``.

**MISTAKE**

Connecting the Shader *to* the Material terminal but never creating ``outputs:surface`` on the Shader. ``ConnectToSource`` needs a real output.

**MISTAKE**

Reading ``Get()`` on a connected input and assuming there is no connection. Leftover values are common after a DCC connect.

**Exam traps.****TRAP**

USDA shows both ``inputs:diffuseColor = (1, 0, 0)`` *and* ``connect``. The connection is what the renderer uses; the value is leftover.

**TRAP**

"Connect the Material's surface output to the *input* of the Shader." The Material terminal connects to the Shader's **output** ``surface``.

**TRAP**

``outputs:ri:surface`` is a RenderMan-context terminal. ``CreateSurfaceOutput()`` with no argument is the universal ``outputs:surface`` that PreviewSurface uses.

**Exam takeaways.****KEY**

- Inputs are ``inputs:``, outputs ``outputs:``; ``connect`` is authored on the destination. - Material ``outputs:surface`` must connect to a Shader ``outputs:surface``. - ``ComputeSurfaceSource()`` → unpack a tuple. - Authored value + connect can coexist; the connect wins for shading.

**40.3 UsdPreviewSurface**

**What is it?** **UsdPreviewSurface** is the standard, DCC-neutral shader implementation for preview and interchange. You author it as a Shader whose `info:id` is the string `"UsdPreviewSurface"`, then set physically based inputs such as `diffuseColor`, `metallic`, `roughness`, and `opacity`.

**Why do we need it?** Maya's Standard Surface, Blender's Principled BSDF, and glTF's `pbrMetallicRoughness` all describe "a painted surface," but their node graphs do not travel. PreviewSurface is the common dialect so `usdview`, Storm, and other Hydra delegates can draw *something correct enough* everywhere. Obj 8.2 is exactly this: assign PreviewSurface materials for asset visualization.

**Beginner explanation.** Think of a restaurant's "house salad" that every location must be able to make from a short ingredient list. PreviewSurface is that house salad: not every garnish from the Michelin kitchen, but lettuce, dressing, salt — `diffuseColor`, `roughness`, `metallic`, `opacity` — so any cook (renderer) can plate it.

*Where the analogy breaks:* production renderers (RenderMan, Arnold, MaterialX graphs) often live *beside* PreviewSurface, connected to `outputs:full` or a render-context terminal. Preview is the interchange plate, not the à-la-carte menu.

**Technical explanation.** - Author: `UsdShade.Shader.Define` →

`CreateIdAttr("UsdPreviewSurface")` → `CreateInput` / `CreateOutput("surface", Token)` → connect to the Material's `CreateSurfaceOutput()`. - Common inputs (types you will author in this book): - `diffuseColor` — `color3f` (the “paint” color; Obj 8.4 often replaces this with a connection) - `emissiveColor` — `color3f` - `metallic` — `float` (0 dielectric, 1 metal) - `roughness` — `float` (0 mirror, 1 chalk) - `opacity` — `float` (1 opaque) - `opacityThreshold` — `float` (cutout) - `ior` — `float` - `normal` — `normal3f` (usually connected to a texture's `rgb`) - `displacement` — `float` - `occlusion` — `float` - `useSpecularWorkflow` — `int` (0 = metallic workflow, 1 = specular) - Outputs used in preview: `surface` (token).

Displacement/volume terminals exist on the Material for other networks. -

PreviewSurface does **not** read mesh primvars by itself. To use UVs or `displayColor` you insert reader/texture shaders (§40.4, §40.5). - Hydra Storm / `usdview` consume this network. `usd-core` has no Hydra; you author and inspect the graph in Python, you do not rasterize it here.

**NOTE**

glTF's metallic-roughness maps onto PreviewSurface (``metallic``, ``roughness``, ``diffuseColor`` ← `baseColor`). That mapping is why exporters in Chapter 29 wrote this shader.

**Mental model.**

```
Material
└─ Shader info:id = "UsdPreviewSurface"
 inputs: diffuseColor, metallic, roughness, opacity, ...
 outputs: surface —connect→ Material.outputs:surface
```

If `info:id` is missing or misspelled, the renderer has no implementation to run.

**Simple example.** A red plastic: `diffuseColor (0.8, 0.1, 0.1)`, `metallic 0`, `roughness 0.5`, `opacity 1`. That is enough for Obj 8.2 visualization.

**USDA example.**

```
#usda 1.0

def Material "Paint"
{
 token outputs:surface.connect = </Paint/Preview.outputs:surface>

 def Shader "Preview"
 {
 uniform token info:id = "UsdPreviewSurface"
 color3f inputs:diffuseColor = (0.8, 0.1, 0.1)
 float inputs:metallic = 0
 float inputs:roughness = 0.5
 float inputs:opacity = 1
 token outputs:surface
 }
}
```

- Four inputs plus `info:id` and the surface terminal: a complete preview look.
- `metallic = 0` and `opacity = 1` are often left unauthored (fallbacks exist in the spec); authoring them makes the intent obvious for exam reading.

**Python example.**

```
from pxr import Usd, UsdShade, Sdf

stage = Usd.Stage.CreateInMemory()
mat = UsdShade.Material.Define(stage, "/Looks/Paint")
sh = UsdShade.Shader.Define(stage, "/Looks/Paint/Preview")
sh.CreateIdAttr("UsdPreviewSurface")
sh.CreateInput("diffuseColor", Sdf.ValueTypeNames.Color3f).Set(
 (0.8, 0.1, 0.1))
sh.CreateInput("metallic", Sdf.ValueTypeNames.Float).Set(0.0)
```

```

sh.CreateInput("roughness", Sdf.ValueTypeNames.Float).Set(0.5)
sh.CreateInput("opacity", Sdf.ValueTypeNames.Float).Set(1.0)
sh.CreateOutput("surface", Sdf.ValueTypeNames.Token)
mat.CreateSurfaceOutput().ConnectToSource(
 sh.ConnectableAPI(), "surface")
print("id:", sh.GetIdAttr().Get())
for name in ("diffuseColor", "metallic", "roughness", "opacity"):
 i = sh.GetInput(name)
 print(name, i.GetTypeName(), i.Get())

```

### Expected output

```

id: UsdPreviewSurface
diffuseColor color3f (0.8, 0.1, 0.1)
metallic float 0.0
roughness float 0.5
opacity float 1.0

```

`ConnectToSource(sh.ConnectableAPI(), "surface")` is the overload that names the output as a string; §40.2 used the Output object. Both work.

**Real-world use case.** A CAD exporter (Chapter 29) cannot ship SolidWorks appearances. It writes one PreviewSurface per unique color/metal pair so Storm and usdview show the assembly in roughly the right colors during design review. The high-end Arnold network, if any, lives in a sibling render-context output the CAD tool never writes.

### Common mistakes.

#### MISTAKE

Setting `info:id` to `"PreviewSurface"` or `"previewSurface"`. The id is exactly `UsdPreviewSurface`.

#### MISTAKE

Filling `diffuseColor` and forgetting to connect `outputs:surface` to the Material. usdview shows default grey; the shader is an orphan.

#### MISTAKE

Using `float3` for `diffuseColor`. The interchange type is `color3f`. Connections from a `float3` texture `rgb` still work in practice, but author the Preview input as `color3f`.

### Exam traps.

#### TRAP

Options that put PreviewSurface inputs on the Material prim. Inputs on the Material are *interface* knobs (§40.7), not the shader implementation.

#### TRAP

"Metallic workflow means you must omit `diffuseColor`." You still author `diffuseColor` (base color); `metallic` blends dielectric vs metal.

#### TRAP

Obj 8.2 vs 8.4: assigning PreviewSurface with a constant `diffuseColor` satisfies 8.2. Reading color from a primvar is a *different* objective (8.4) and needs a reader node.

### Exam takeaways.

#### KEY

- Obj 8.2 = Shader `info:id = UsdPreviewSurface` under a Material, surface output connected.
- Core inputs: `diffuseColor`, `metallic`, `roughness`, `opacity`.
- PreviewSurface does not magically see mesh primvars; wire readers/textures.
- Exact id string: `UsdPreviewSurface`.

## 40.4 Textures: UsdUVTexture, UsdPrimvarReader\_float2

**What is it?** **UsdUVTexture** is a Shader that samples an image file.

**UsdPrimvarReader\_float2** is a Shader that reads a 2-float primvar (almost always **st**, the UV coordinates) from the bound geometry and feeds it to the texture's **st** input.

**Why do we need it?** A constant **diffuseColor** paints the whole mesh one color. Real assets have wood grain, logos, and rust. Those live in image files, and the image must stick to the surface using UVs stored as a primvar (Chapter 11).

**Beginner explanation.** Think of wrapping patterned paper around a gift box. The **photo** is the texture file (**inputs:file**). The **crease pattern** telling you which part of the paper hits which face is the UV primvar **st**. The **reader** is the person looking up the crease pattern; the **UVTexture** is the person laying the photo down.

*Where the analogy breaks:* the reader does not copy the primvar into the Material. It samples it *on the geometry being shaded*, so one Material can wrap many meshes that each have their own **st**.

**Technical explanation.** - **UsdPrimvarReader\_float2**:

**CreateIdAttr("UsdPrimvarReader\_float2")**. Input **varname** is a token naming the primvar **without** the primvars: prefix ("st", not "primvars:st"). Output result is float2. -

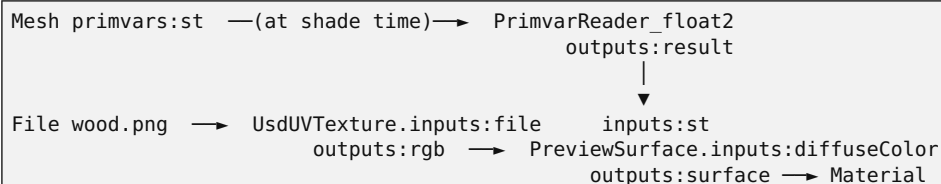
**UsdUVTexture**: **CreateIdAttr("UsdUVTexture")**. Inputs: **file** (asset, e.g. `./wood.png`), **st** (float2, connected from the reader). Outputs: **rgb** (float3), plus **r**, **g**, **b**, **a**. Connect **rgb** to **PreviewSurface diffuseColor**. - Other UVTexture inputs you will see in USDA: **wrapS** / **wrapT** (repeat, clamp, mirror), **fallback**, **scale**, **bias**. - Asset paths follow Chapter 33: `./wood.png` is layer-relative when written as an asset-valued input. Naive

`Ar.GetResolver().Resolve("./wood.png")` is still cwd; the composed stage uses the layer as anchor. - `usd-core` will author the graph; it will not open the PNG. Missing files are an asset-resolution problem, not a UsdShade schema error.

### VERSION

Verified on USD 26.08. There is no Python class `UsdShade.UVTexture`; it is a Shader id, like `PreviewSurface`.

### Mental model.



**Simple example.** `varname = "st" → UVTexture file = @./wood.png@ → rgb into diffuseColor`. Three shaders, one Material.

### USDA example.

```

#usda 1.0

def Material "Wood"
{
 token outputs:surface.connect = </Wood/Preview.outputs:surface>

 def Shader "StReader"
 {
 uniform token info:id = "UsdPrimvarReader_float2"
 token inputs:varname = "st"
 float2 outputs:result
 }

 def Shader "DiffuseTex"
 {
 uniform token info:id = "UsdUVTexture"
 }
 }

```

```

 asset inputs:file = @./wood.png@
 float2 inputs:st.connect = </Wood/StReader.outputs:result>
 float3 outputs:rgb
}

def Shader "Preview"
{
 uniform token info:id = "UsdPreviewSurface"
 color3f inputs:diffuseColor.connect = </Wood/DiffuseTex.outputs:rgb>
 token outputs:surface
}
}

```

- Reader id is `UsdPrimvarReader_float2` (the 2 matches UV's two floats).
- varname is `"st"`, matching `primvars:st` on the mesh.
- Texture file uses asset syntax `@./wood.png@`.

### Python example.

```

from pxr import Usd, UsdShade, Sdf

stage = Usd.Stage.CreateInMemory()
mat = UsdShade.Material.Define(stage, "/Looks/Wood")
reader = UsdShade.Shader.Define(stage, "/Looks/Wood/StReader")
reader.CreateIdAttr("UsdPrimvarReader_float2")
reader.CreateInput("varname", Sdf.ValueTypeNames.Token).Set("st")
reader.CreateOutput("result", Sdf.ValueTypeNames.Float2)

tex = UsdShade.Shader.Define(stage, "/Looks/Wood/DiffuseTex")
tex.CreateIdAttr("UsdUVTexture")
tex.CreateInput("file", Sdf.ValueTypeNames.Asset).Set("./wood.png")
tex.CreateInput("st", Sdf.ValueTypeNames.Float2).ConnectToSource(
 reader.GetOutput("result"))
tex.CreateOutput("rgb", Sdf.ValueTypeNames.Float3)

ps = UsdShade.Shader.Define(stage, "/Looks/Wood/Preview")
ps.CreateIdAttr("UsdPreviewSurface")
ps.CreateInput("diffuseColor", Sdf.ValueTypeNames.Color3f).ConnectToSource(
 tex.GetOutput("rgb"))
ps.CreateOutput("surface", Sdf.ValueTypeNames.Token)
mat.CreateSurfaceOutput().ConnectToSource(ps.GetOutput("surface"))

print("reader id:", reader.GetIdAttr().Get())
print("tex id:", tex.GetIdAttr().Get())
print("st connected:", tex.GetInput("st").HasConnectedSource())
print("file:", tex.GetInput("file").Get())
for line in stage.GetRootLayer().ExportToString().splitlines():
 if any(k in line for k in ("varname", "file", "connect")):
 print(line.strip())

```

### Expected output

```

reader id: UsdPrimvarReader_float2
tex id: UsdUVTexture
st connected: True
file: @./wood.png@
token outputs:surface.connect = </Looks/Wood/Preview.outputs:surface>
asset inputs:varname = "st"
asset inputs:file = @./wood.png@
float2 inputs:st.connect = </Looks/Wood/StReader.outputs:result>
color3f inputs:diffuseColor.connect = </Looks/Wood/DiffuseTex.outputs:rgb>

```

`Get()` on an asset input prints the USDA `@path@` form.

**Real-world use case.** A games exporter writes one 2K albedo PNG next to `wood.usda` and this three-node graph. The engine's USD importer either samples the same `PreviewSurface` in the editor or reads `inputs:file` to feed its own sampler. Variant sets (Chapter 18) can swap `file` between `wood_2k.png` and `wood_512.png` without rebuilding the graph.

### Common mistakes.

**MISTAKE**

``varname = "primvars:st"``. The reader wants the primvar `**name**`st``. The ``primvars:`` prefix is the attribute namespace on the mesh, not the varname.

**MISTAKE**

Using ``UsdPrimvarReader_float3`` for UVs. UVs are two floats; the id must be ``UsdPrimvarReader_float2``.

**MISTAKE**

Connecting the texture ``file`` input to the reader. ``file`` is the image; ``st`` is the UV. Mix them up and the texture has no coordinates (or tries to open a path named after a float2).

**Exam traps.****TRAP**

USDA with ``varname = "UVMap"`` while the mesh authors ``primvars:st``. The names must match.

**TRAP**

"UsdUVTexture is a typed schema you ``Apply``." It is a Shader `**id**`, like `PreviewSurface`.

**TRAP**

A question that omits the reader and sets ``inputs:st = (0, 0)`` on the texture. That samples one texel for the whole mesh.

**Exam takeaways.****KEY**

- Texture network: ``UsdPrimvarReader_float2`` (``varname=st``) → ``UsdUVTexture`` (``file``, ``st``) → `Preview`diffuseColor``. - ``varname`` has no ``primvars:`` prefix. - Both nodes are Shaders with ``info:id``; there is no UVTexture prim type. - ``inputs:file`` is an ``asset`` (``@./wood.png@``).

## 40.5 Reading diffuse color from a primvar

**What is it?** Objective 8.4 asks you to create a `UsdPreviewSurface` network whose **diffuse color comes from a primvar** on the mesh — usually `displayColor`. The extra node is `UsdPrimvarReader_float3`: it reads three floats (a color) and connects to `inputs:diffuseColor`.

**Why do we need it?** Point clouds, CAD faces, and debug visualizations often carry a color per point or per mesh already (`primvars:displayColor`, Chapter 11 / 39). Baking that into a constant on the shader would freeze the color. A reader keeps one Material and lets each mesh (or each vertex) supply its own color.

**Beginner explanation.** Think of a paint-by-numbers kit that already printed the numbers on the canvas. The Material is “use the printed number as the paint color.” The reader is the instruction “look at the `displayColor` box on this mesh.” Different canvases, same instruction card.

*Where the analogy breaks:* interpolation matters. A constant primvar paints the whole prim one color; vertex interpolates; `faceVarying` follows corners. The reader does not change interpolation — the primvar does (Chapter 11).

**Technical explanation.** - Mesh side: author `primvars:displayColor` as `color3f[]`. Helpers: `UsdGeom.Mesh.CreateDisplayColorPrimvar(interpolation)` or `UsdGeom.PrimvarsAPI(mesh).CreatePrimvar("displayColor", ...)`. The primvar **name** is `displayColor`. - Shader side: Shader id `UsdPrimvarReader_float3` (three floats, not two).

`inputs:varname = "displayColor"`. Output `result (float3)` connects to `PreviewSurface` `inputs:diffuseColor`. - Then bind the Material to the mesh (§40.6). The reader samples the primvar **on the geometry being shaded**, so one Material can serve many colored meshes. - `displayOpacity` is the usual companion (`float / UsdPrimvarReader_float` into `opacity`). Obj 8.4 names diffuse color specifically. - This is **not** the texture path. If the exam stem says “reads diffuse color from a primvar,” look for `PrimvarReader_float3`, not `UsdUVTexture`.

#### EXAM TIP

Obj 8.4 is a named, testable network. Memorize: Mesh ``primvars:displayColor`` + Shader ``UsdPrimvarReader_float3`` + ``varname = displayColor`` + connect to Preview ``diffuseColor`` + bind.

#### Mental model.

```
Mesh
 primvars:displayColor = [(0.2, 0.5, 0.9)]
 rel material:binding → Material "FromPrimvar"
 |
 Shader ColorReader
 id = UsdPrimvarReader_float3
 varname = "displayColor"
 result → Preview.diffuseColor
```

**Simple example.** One mesh, constant blue-ish `displayColor`, one Material, two shaders (reader + Preview), binding applied. `ComputeBoundMaterial` → that Material. Reader `varname` → `displayColor`.

#### USDA example.

```
#usda 1.0
(
 defaultPrim = "World"
)

def Xform "World"
{
 def Mesh "Board" (
 prepend apiSchemas = ["MaterialBindingAPI"]
)
 {
 rel material:binding = </Looks/FromPrimvar>
 color3f[] primvars:displayColor = [(0.2, 0.5, 0.9)] (
 interpolation = "constant"
)
 }
}

def Scope "Looks"
{
 def Material "FromPrimvar"
 {
 token outputs:surface.connect = </Looks/FromPrimvar/Preview.outputs:surface>

 def Shader "ColorReader"
 {
 uniform token info:id = "UsdPrimvarReader_float3"
 token inputs:varname = "displayColor"
 float3 outputs:result
 }

 def Shader "Preview"
 {
 uniform token info:id = "UsdPreviewSurface"
 color3f inputs:diffuseColor.connect = </Looks/FromPrimvar/ColorReader.outputs:result>
 token outputs:surface
 }
 }
}
```

- `varname` matches the primvar name, not `primvars:displayColor`.
- Binding and the applied API schema are on the **mesh** (§40.6).

### Python example.

```
from pxr import Usd, UsdGeom, UsdShade, Sdf, Gf, Vt

stage = Usd.Stage.CreateInMemory()
mesh = UsdGeom.Mesh.Define(stage, "/World/Board")
UsdGeom.PrimvarsAPI(mesh).CreatePrimvar(
 "displayColor", Sdf.ValueTypeNames.Color3fArray,
 UsdGeom.Tokens.constant
).Set(Vt.Vector3f([Gf.Vector3f(0.2, 0.5, 0.9)]))

mat = UsdShade.Material.Define(stage, "/Looks/FromPrimvar")
reader = UsdShade.Shader.Define(stage, "/Looks/FromPrimvar/ColorReader")
reader.CreateIdAttr("UsdPrimvarReader_float3")
reader.CreateInput("varname", Sdf.ValueTypeNames.Token).Set(
 "displayColor")
reader.CreateOutput("result", Sdf.ValueTypeNames.Float3)
ps = UsdShade.Shader.Define(stage, "/Looks/FromPrimvar/Preview")
ps.CreateIdAttr("UsdPreviewSurface")
ps.CreateInput("diffuseColor", Sdf.ValueTypeNames.Color3f).ConnectToSource(
 reader.GetOutput("result"))
ps.CreateOutput("surface", Sdf.ValueTypeNames.Token)
mat.CreateSurfaceOutput().ConnectToSource(ps.GetOutput("surface"))
UsdShade.MaterialBindingAPI.Apply(mesh.GetPrim()).Bind(mat)
bound = UsdShade.MaterialBindingAPI(mesh.GetPrim()).ComputeBoundMaterial()[0]
print("reader id:", reader.GetIdAttr().Get())
print("varname:", reader.GetInput("varname").Get())
print("diffuse connected:", ps.GetInput("diffuseColor").HasConnectedSource())
print("bound:", bound.GetPath())
print("primvar:", UsdGeom.PrimvarsAPI(mesh).GetPrimvar("displayColor").Get())
```

### Expected output

```
reader id: UsdPrimvarReader_float3
varname: displayColor
diffuse connected: True
bound: /Looks/FromPrimvar
primvar: [(0.2, 0.5, 0.9)]
```

This block is the Obj 8.4 recipe in runnable form.

**Real-world use case.** A digital-twin importer colors each machine by status: running = green, fault = red, stored on the mesh as `displayColor` and updated every tick. One Material (`FromPrimvar`) is bound to every machine. Changing a primvar is cheaper than swapping materials, and `usdview`'s default purpose still shows the colors.

### Common mistakes.

#### MISTAKE

Using ``UsdPrimvarReader_float2`` for color. Color is three floats; UVs are two. Wrong id → wrong output type → failed connection.

#### MISTAKE

``varname = "primvars:displayColor"``. Same trap as UVs: drop the namespace.

#### MISTAKE

Setting Preview ``diffuseColor`` to a constant \*and\* forgetting the reader. That is Obj 8.2, not 8.4. The exam stem's "from a primvar" is the tell.

### Exam traps.



**TRAP**

A mesh with ``displayColor`` authored as a **non-primvar** attribute ``color3f displayColor``. Readers look up **primvars**. Use ``primvars:displayColor`` / ``CreateDisplayColorPrimvar``.

**TRAP**

Binding the ColorReader Shader instead of the Material.

**TRAP**

Connecting the reader to ``emissiveColor`` and leaving ``diffuseColor`` black. Obj 8.4 says **diffuse**.

**Exam takeaways.****KEY**

- Obj 8.4 = ``UsdPrimvarReader_float3`` + ``varname=displayColor`` → Preview ``diffuseColor``. - Mesh authors ``primvars:displayColor``; varname omits ``primvars:``. - float3 = color, float2 = UVs. Do not swap the ids. - Still bind a Material (next section); the reader is not the bind target.

## 40.6 Material binding (**MaterialBindingAPI**, strength, purpose, collections)

**What is it? Binding** assigns a Material to geometry. The relationship is `rel material:binding`. The applied API schema **MaterialBindingAPI** is what makes that relationship legal. Bindings have **purpose** (``` / `preview` / `full`) and **strength** (`strongerThanDescendants` / `weakerThanDescendants`). You can also bind via **collections** and **GeomSubsets**.

**Why do we need it?** A Material sitting under `/Looks` does nothing until geometry points at it. Without a standard relationship, every DCC would invent `rel myApp:shader`. `material:binding` is the contract. Obj 8.3 is this contract.

**Beginner explanation.** Think of a gallery label stuck on a sculpture: “this piece uses the Wood look.” The sculpture is the mesh; the label is `material:binding`; the look is the Material. A **purpose** is a different label for different lighting (“preview” for the lobby snapshot, “full” for the catalog shoot). **Strength** says whether a child’s label beats the parent’s.

*Where the analogy breaks:* you can stick a label that says “none” (`rel material:binding = None`) and still inherit a parent’s label. Unbinding *this* prim is not unbinding the ancestor.

**Technical explanation.** - **Always** `UsdShade.MaterialBindingAPI.Apply(prim)` then `.Bind(material)`. Skipping `Apply` still writes `rel material:binding` and `Bind` returns `True`, but `HasAPI(MaterialBindingAPI)` stays `False` and validators fail.

**VERSION**

Verified on USD 26.08. Since USD 22.11, validators expect ``MaterialBindingAPI`` applied. Always ``Apply``.

- Direct bind USDA: `rel material:binding = </Looks/Wood>`. Purpose-specific: `rel material:binding:preview` and `rel material:binding:full`. `GetMaterialPurposes()` returns `['', 'preview', 'full']`. The empty string is **all-purpose** (`UsdShade.Tokens.allPurpose`).
- Resolve: `api.ComputeBoundMaterial()` → `(Material, Relationship)`. Pass `UsdShade.Tokens.preview` to resolve the preview purpose (falls back to all-purpose if no preview bind exists).

- **Strength.** Fallback (unauthored) is `strongerThanDescendants`: a parent's bind beats a child's. Pass `bindingStrength=UsdShade.Tokens.weakerThanDescendants` to let children win. Strength is metadata `bindMaterialAs` on the relationship, authored only when you set a non-default.
- **UnbindDirectBinding()** authors `rel material:binding = None` (empty targets). `ComputeBoundMaterial` then returns an invalid Material *if nothing else binds* (parent/collection). It does not remove the applied API schema.
- **Collections.** `api.Bind(collectionAPI, material)` writes `rel material:binding:collection:<name>` with **two targets**: the collection, then the Material. Include paths on the collection name the bound prims.
- **GeomSubsets.** `api.CreateMaterialBindSubset(name, Vt.IntArray([...]))` creates a child `GeomSubset` with `familyName = "materialBind"` and parent `subsetFamily:materialBind:familyType = "nonOverlapping"`. Bind the subset prim the same way: Apply + Bind. Family name must be `materialBind` (Chapter 39.5).
- Bindings inherit down the prim tree until a stronger descendant bind wins.

### Mental model.

```
purpose: all-purpose (') preview full
 material:binding :preview :full

strength: strongerThanDescendants → parent beats child
 weakerThanDescendants → child beats parent

collection bind rel targets: [<prim.collection:name> , <Material>]
```

**Simple example.** Mesh `/World/Board/Top` Apply + Bind Wood. `ComputeBoundMaterial` → `/Looks/Wood`, rel name `material:binding`. Bind WoodPreview with purpose `preview` and weaker strength → rel `material:binding:preview`. Parent Board weaker-binds Wood, child Inlay binds Inlay → child wins; flip parent to stronger → parent Wood wins.

### USDA example.

```
#usda 1.0

def Xform "World" (
 prepend apiSchemas = ["MaterialBindingAPI", "CollectionAPI:woodbits"]
)
{
 prepend rel collection:woodbits:includes = </World/Board>
 rel material:binding:collection:woodbits = [
 </World.collection:woodbits>,
 </Looks/Wood>,
]

 def Mesh "Board" (
 prepend apiSchemas = ["MaterialBindingAPI"]
)
 {
 rel material:binding = </Looks/Wood>
 rel material:binding:preview = </Looks/WoodPreview> (
 bindMaterialAs = "weakerThanDescendants"
)
 }
}

def Scope "Looks"
{
 def Material "Wood" {}
 def Material "WoodPreview" {}
}
```

- Direct bind has no `bindMaterialAs` → fallback stronger.
- Preview bind authors the metadata on the **relationship**.
- Collection rel has **two** targets, collection first.

**Python example.**

```

from pxr import Usd, UsdGeom, UsdShade, Vt

stage = Usd.Stage.CreateInMemory()
board = UsdGeom.Xform.Define(stage, "/World/Board")
top = UsdGeom.Mesh.Define(stage, "/World/Board/Top")
inlay = UsdGeom.Mesh.Define(stage, "/World/Board/Inlay")
loose = UsdGeom.Mesh.Define(stage, "/World/Loose")

wood = UsdShade.Material.Define(stage, "/Looks/Wood")
inlay_mat = UsdShade.Material.Define(stage, "/Looks/Inlay")
preview = UsdShade.Material.Define(stage, "/Looks/WoodPreview")

print("HasAPI before:", top.GetPrim().HasAPI(UsdShade.MaterialBindingAPI))
top_api = UsdShade.MaterialBindingAPI.Apply(top.GetPrim())
print("HasAPI after Apply:",
 top.GetPrim().HasAPI(UsdShade.MaterialBindingAPI))
top_api.Bind(wood)
bound, rel = top_api.ComputeBoundMaterial()
print("direct bind:", bound.GetPath(), rel.GetName())

top_api.Bind(
 preview,
 bindingStrength=UsdShade.Tokens.weakerThanDescendants,
 materialPurpose=UsdShade.Tokens.preview,
)
print("purposes:", UsdShade.MaterialBindingAPI.GetMaterialPurposes())
pbound, prel = top_api.ComputeBoundMaterial(UsdShade.Tokens.preview)
print("preview bind:", pbound.GetPath(), prel.GetName())

board_api = UsdShade.MaterialBindingAPI.Apply(board.GetPrim())
inlay_api = UsdShade.MaterialBindingAPI.Apply(inlay.GetPrim())
inlay_api.Bind(inlay_mat)
board_api.Bind(
 wood, bindingStrength=UsdShade.Tokens.weakerThanDescendants)
print("inlay when parent weaker:",
 inlay_api.ComputeBoundMaterial()[0].GetPath())
board_api.Bind(
 wood, bindingStrength=UsdShade.Tokens.strongerThanDescendants)
print("inlay when parent stronger:",
 inlay_api.ComputeBoundMaterial()[0].GetPath())

loose_api = UsdShade.MaterialBindingAPI.Apply(loose.GetPrim())
loose_api.Bind(wood)
loose_api.UnbindDirectBinding()
ubound, _ = loose_api.ComputeBoundMaterial()
print("after UnbindDirectBinding valid:", bool(ubound))
print("unbound rel targets:",
 list(loose_api.GetDirectBindingRel().GetTargets()))

sub = top_api.CreateMaterialBindSubset("faces", Vt.IntArray([0, 1]))
print("subset family:", sub.GetFamilyNameAttr().Get())

```

**Expected output**

```

HasAPI before: False
HasAPI after Apply: True
direct bind: /Looks/Wood material:binding
purposes: ['', 'preview', 'full']
preview bind: /Looks/WoodPreview material:binding:preview
inlay when parent weaker: /Looks/Inlay
inlay when parent stronger: /Looks/Wood
after UnbindDirectBinding valid: False
unbound rel targets: []
subset family: materialBind

```

Default Bind(wood) does not disturb the later preview-purpose bind: they are different relationships.

**Real-world use case.** A car body mesh binds a full-fidelity Material (`purpose=full`, paint flakes, clearcoat) and a cheap PreviewSurface (`purpose=preview`) for `usdview`. Body panels are `GeomSubsets` `paint` / `glass` / `chrome`, each with `familyName = materialBind` and its own bind. The assembly root binds a weaker “clay” Material so any unpainted child still has a look.

### Common mistakes.

#### MISTAKE

``Bind`` without ``Apply``. The rel exists, ``HasAPI`` is False, `UsdValidation` fails. Fix: ``Apply`` first.

#### MISTAKE

Expecting ``UnbindDirectBinding`` to clear inherited parent binds. It only authors ``None`` on `**this**` prim.

#### MISTAKE

Creating a `GeomSubset` with empty ``familyName`` and binding it. Material families must be `**`materialBind`**`. Use ``CreateMaterialBindSubset``.

### Exam traps.

#### TRAP

"Parent bind always loses to the child." Only if the parent is ``weakerThanDescendants``. Default / unauthored is `**stronger**`.

#### TRAP

Collection binding relationship has one target, the Material. It has `**two**`: collection, then Material.

#### TRAP

``GetMaterialPurposes()`` includes ``all``. The all-purpose token is the `**empty string**` ````.

#### TRAP

``purpose`` on ``UsdGeom.Imageable`` (``proxy`` / ``render``) is a different concept from material-binding purpose (``preview`` / ``full``). Do not mix them (Chapter 39.6 vs this section).

### Exam takeaways.

#### KEY

- Obj 8.3: ``Apply`` then ``Bind``; ``ComputeBoundMaterial`` to resolve. - Purposes: ```` (all), ``preview``, ``full``. Strength lives in ``bindMaterialAs``. - Default strength is `strongerThanDescendants` (parent wins). - Collection bind rel = ``[collection, material]``. Subsets use family ``materialBind``. - ``UnbindDirectBinding`` → ``rel = None``; inherited binds can remain.

## 40.7 Exposing material parameters for overrides

**What is it?** A Material can publish **interface inputs** — public knobs such as `inputs:paint` — that internal shaders connect *to*. A shot or assembly layer then overrides `inputs:paint` on the Material without diving into `PreviewSurface`, textures, or connections.

**Why do we need it?** If every lookdev edit retargets `/Looks/Paint/`

`Preview.inputs.diffuseColor`, referencing assets (Chapter 16) and variant sets become brittle: rename the shader child and every override breaks. Interface inputs are the stable API of the look, the same idea as exposing one color on a `MaterialX` public node or a DCC “published parameter.”

**Beginner explanation.** Think of a blender with a single “strength” dial on the outside. Inside, that dial’s cable runs to the motor. You do not open the housing to change speed. The Material’s `inputs:paint` is the dial; the Shader’s `diffuseColor` is the motor, connected to the dial.

*Where the analogy breaks:* USD still lets you override the internal shader input in a stronger layer. The interface is a *convention for stability*, not a lock.

**Technical explanation.** - Create on the Material: `mat.CreateInput("paint", Sdf.ValueTypeNames.Color3f).Set((0.8, 0.2, 0.1))`. USDA: `color3f inputs:paint = (0.8, 0.2, 0.1)` on the **Material** prim. - Connect the *shader* input to that interface: `shader.GetInput("diffuseColor").ConnectToSource(paintInput)`. USDA: `color3f inputs:diffuseColor.connect = </Looks/Paint.inputs:paint>`. - Notice the reverse of §40.2’s terminal: there, the Material **output** connected to a Shader **output**. Here, a Shader **input** connects to a Material **input**. - `GetConnectedSources()` on the shader input reports `source.GetPath() == /Looks/Paint` and `sourceName == paint` (the connectable is the Material prim). - Downstream: over the Material (or a reference to the asset) and `Set` the interface input. Internal connections stay. This is the composition-friendly override path (Chapters 16, 20). - Name interface inputs for *artists* (`paint`, `wear`, `roughness`), not for the shader implementation (`diffuseColor`), so you can retarget the internal graph later.

### Mental model.

```
Shot layer: over Material "Paint" { inputs:paint = green }

Asset: Material "Paint"
 inputs:paint = red ← public knob
 outputs:surface.connect → Preview.outputs:surface
 Shader "Preview"
 inputs:diffuseColor.connect → Material.inputs:paint
```

Override the knob, not the child shader.

**Simple example.** Material `/Looks/Paint` has `inputs:paint = (0.8, 0.2, 0.1)`. Preview `diffuseColor` connects to it. A shot over sets `inputs:paint = (0, 1, 0)`. The network is unchanged; the color is green.

### USDA example.

```
#usda 1.0

def Material "Paint"
{
 color3f inputs:paint = (0.8, 0.2, 0.1)
 token outputs:surface.connect = </Paint/Preview.outputs:surface>

 def Shader "Preview"
 {
 uniform token info:id = "UsdPreviewSurface"
 color3f inputs:diffuseColor.connect = </Paint.inputs:paint>
 token outputs:surface
 }
}
```

- Interface input sits on the Material, sibling to `outputs:surface`.
- Shader input `.connect` points at `.inputs:paint`, not at another shader.

### Python example.

```

from pxr import Usd, UsdShade, Sdf

stage = Usd.Stage.CreateInMemory()
mat = UsdShade.Material.Define(stage, "/Looks/Paint")
sh = UsdShade.Shader.Define(stage, "/Looks/Paint/Preview")
sh.CreateIdAttr("UsdPreviewSurface")
paint = mat.CreateInput("paint", Sdf.ValueTypeNames.Color3f)
paint.Set((0.8, 0.2, 0.1))
diff = sh.CreateInput("diffuseColor", Sdf.ValueTypeNames.Color3f)
diff.ConnectToSource(paint)
sh.CreateOutput("surface", Sdf.ValueTypeNames.Token)
mat.CreateSurfaceOutput().ConnectToSource(sh.GetOutput("surface"))

print("interface:", paint.GetFullName(), paint.Get())
print("shader connected:", diff.HasConnectedSource())
src, _ = diff.GetConnectedSources()
print("connected from:", src[0].source.GetPath(), src[0].sourceName)
for line in stage.GetRootLayer().ExportToString().splitlines():
 if "paint" in line or "diffuseColor" in line or "connect" in line:
 print(line.strip())

```

### Expected output

```

interface: inputs:paint (0.8, 0.2, 0.1)
shader connected: True
connected from: /Looks/Paint paint
color3f inputs:paint = (0.8, 0.2, 0.1)
token outputs:surface.connect = </Looks/Paint/Preview.outputs:surface>
color3f inputs:diffuseColor.connect = </Looks/Paint.inputs:paint>

```

GetFullName() includes the inputs: namespace; sourceName is just paint.

**Real-world use case.** A hero car publishes inputs:paint, inputs:roughness, and inputs:flake. The lookdev graph inside /Looks/CarPaint has eight shaders. Lighting publishes a shot layer with over "CarPaint" { color3f inputs:paint = (0.02, 0.09, 0.28) } for a night-blue livery. The internal graph can be rebuilt next week; the shot still only knows paint.

### Common mistakes.

#### MISTAKE

Connecting the Material input \*to\* the Shader input ( paint.connect = ). The public knob would then \*follow\* the shader, which is the opposite of an override API. Shader input connects to the Material input.

#### MISTAKE

Exposing twenty internal texture scale inputs. Publish the three the shot actually changes; keep the rest private.

#### MISTAKE

Naming the interface `diffuseColor` and then switching the internal graph to a texture mix. Shots that overrode `diffuseColor` now fight the texture. Use a stable public name ( paint ).

### Exam traps.

#### TRAP

USDA with `inputs:paint` on the Shader. That is just another shader input, not an interface. Interface inputs live on the \*\*Material\*\* (or NodeGraph).

#### TRAP

"You must flatten the Material to override a color." An `over` on the interface input is enough (Chapter 20).

**TRAP**

``ConnectToSource(paint)`` reports source path ``/Looks/Paint/Preview``. No: the connectable is the **Material** prim ``/Looks/Paint``.

**Exam takeaways.****KEY**

- Material ``CreateInput`` = public knob; shader inputs connect *\*to\** it. - USDA:  
``Shader.inputs:diffuseColor.connect = ``. - Override the interface in a stronger layer; leave the graph intact. - Name knobs for artists, not for the current shader child.

**Chapter lab(s)**

**Lab 34 — UsdPreviewSurface + primvar-driven color + binding (★★☆, Obj 8.2–8.4).** You author a PreviewSurface Material, bind it with `MaterialBindingAPI`, add a `UsdPrimvarReader_float3` so `displayColor` drives `diffuseColor`, then expose `inputs:paint` as an interface override. Stretch: UVTexture + `st` reader, and a `materialBind` `GeomSubset`.

Chapter 29's exporter sketch is the warm-up; this lab is the complete network.

**USDA reading exercise(s)**

**Exercise 40-A.** A lookdev USDA is supposed to show red plastic in `usdview`. Why is the mesh still grey?

```
#usda 1.0

def Mesh "Tile" (
 prepend apiSchemas = ["MaterialBindingAPI"]
)
{
 rel material:binding = </Looks/Paint>
}

def Scope "Looks"
{
 def Material "Paint"
 {
 uniform token info:id = "UsdPreviewSurface"
 color3f inputs:diffuseColor = (0.8, 0.1, 0.1)
 token outputs:surface
 }
}
```

**Exercise 40-B.** What Material does `/Set/Knob` resolve to for all-purpose binding, and why?

```
#usda 1.0

def Xform "Set" (
 prepend apiSchemas = ["MaterialBindingAPI"]
)
{
 rel material:binding = </Looks/Clay> (
 bindMaterialAs = "weakerThanDescendants"
)

 def Mesh "Knob" (
 prepend apiSchemas = ["MaterialBindingAPI"]
)
 {
 rel material:binding = </Looks/Chrome>
 }
}

def Scope "Looks"
{

```

```

def Material "Clay" {}
def Material "Chrome" {}
}

```

**Exercise 40-C.** Does this network satisfy Obj 8.4 (diffuse from a primvar)? If not, what is wrong?

```

#usda 1.0

def Mesh "Flag"
{
 color3f[] primvars:displayColor = [(0, 1, 0)] (
 interpolation = "constant"
)
}

def Material "FromPrimvar"
{
 token outputs:surface.connect = </FromPrimvar/Preview.outputs:surface>

 def Shader "ColorReader"
 {
 uniform token info:id = "UsdPrimvarReader_float2"
 token inputs:varname = "primvars:displayColor"
 float2 outputs:result
 }

 def Shader "Preview"
 {
 uniform token info:id = "UsdPreviewSurface"
 color3f inputs:diffuseColor.connect = </FromPrimvar/ColorReader.outputs:result>
 token outputs:surface
 }
}

```

## Chapter review

### Summary

- Bind **Materials**. Shaders live inside them; `info:id` names the implementation (`UsdPreviewSurface`, `UsdUVTexture`, `UsdPrimvarReader_float2` / `_float3`).
- Material IsA `NodeGraph`; connect `outputs:surface` to the surface shader.
- Obj 8.2: `PreviewSurface` with `diffuseColor` / `metallic` / `roughness` / `opacity`.
- Textures: reader `varname=st` → `UVTexture` file + `st` → `rgb` → `diffuseColor`.
- Obj 8.4: `UsdPrimvarReader_float3`, `varname=displayColor`, mesh `primvars:displayColor`.
- Obj 8.3: `MaterialBindingAPI.Apply` then `Bind`. Purposes `''` / `preview` / `full`. Default strength parent-wins.
- Interface inputs on the Material are the public override API.

### If you see... → think...

If you see...	Think...
<code>info:id</code> on a Material	Wrong place; belongs on a Shader
<code>UsdPreviewSurface</code> as a prim type	It is an id string on type <code>Shader</code>
Mesh grey in <code>usdview</code> , Material exists	Missing bind, missing <code>Apply</code> , or <code>outputs:surface</code> not connected
Obj 8.4 / "from a primvar"	<code>UsdPrimvarReader_float3</code> + <code>varname=displayColor</code>
<code>varname = "primvars:st"</code>	Drop the namespace; use <code>"st"</code>
<code>HasAPI(MaterialBindingAPI) False</code>	Forgot <code>Apply</code> (rel may still exist)
Parent bind vs child bind	Read <code>bindMaterialAs</code> ; default is stronger (parent wins)
<code>material:binding:collection:X</code>	Two targets: collection, then Material



If you see...	Think...
<code>familyName = "materialBind"</code>	Per-face materials via <code>GeomSubset</code>
<code>inputs:paint</code> on the Material	Interface knob; shaders connect to it
Value <i>and</i> <code>.connect</code> on one input	Connection wins for shading
<code>purpose = "proxy"</code> on the mesh	Imageable purpose (Ch 39), not material purpose

### Review questions

**Q40.1** · Obj 8.2 · Easy · Single choice Where is uniform token `info:id = "UsdPreviewSurface"` authored? A. On the Material prim · B. On a Shader prim · C. On the bound Mesh · D. As layer metadata

**Q40.2** · Obj 8.2 · Easy · Single choice `UsdShade.Material.Define(...).GetPrim().IsA(UsdShade.NodeGraph)` prints: A. False · B. True · C. An error · D. None

**Q40.3** · Obj 8.2 · Medium · Single choice `ComputeSurfaceSource()` returns: A. A Shader · B. A Material · C. A tuple (shader, name, type) · D. A relationship

**Q40.4** · Obj 8.4 · Medium · Single choice Which Shader `info:id` reads mesh `primvars:displayColor` into Preview `diffuseColor`? A. `UsdPrimvarReader_float2` · B. `UsdUVTexture` · C. `UsdPrimvarReader_float3` · D. `UsdPreviewSurface`

**Q40.5** · Obj 8.4 · Medium · Single choice The reader's varname for that primvar must be: A. `primvars:displayColor` · B. `displayColor` · C. `diffuseColor` · D. `st`

**Q40.6** · Obj 8.3 · Easy · Select two. A correct mesh bind includes: A. `MaterialBindingAPI.Apply` · B. `.Bind(material)` · C. `info:id` on the mesh · D. `Kind.Register`

**Q40.7** · Obj 8.3 · Medium · Single choice Parent bind is unauthored strength; child binds a different Material. Who wins? A. Child · B. Parent · C. Neither; the mesh is grey · D. The stronger purpose

**Q40.8** · Obj 8.3 · Medium · Single choice `GetMaterialPurposes()` includes which all-purpose token? A. "all" · B. "default" · C. "" · D. "universal"

**Q40.9** · Obj 8.3 · Medium · Select two. A collection material bind relationship's targets are: A. The collection path · B. The Material path · C. Every included mesh path · D. The Shader path

**Q40.10** · Obj 8.2 · Medium · USDA reading What color does shading use for `diffuseColor`?

```
#usda 1.0

def Shader "Preview"
{
 uniform token info:id = "UsdPreviewSurface"
 color3f inputs:diffuseColor = (1, 0, 0)
 color3f inputs:diffuseColor.connect = </Paint/Reader.outputs:result>
}
```

A. Red (1, 0, 0) · B. The connected reader result · C. Average of both · D. Preview fallback grey

**Q40.11** · Obj 8.2 · Easy · Single choice Typical `UVTexture` network uses which reader id and varname? A. `UsdPrimvarReader_float3`, `st` · B. `UsdPrimvarReader_float2`, `st` · C. `UsdPrimvarReader_float2`, `primvars:st` · D. `UsdUVTexture`, `uv`

**Q40.12** · Obj 8.3 · Medium · Single choice `UnbindDirectBinding()` on a mesh with no ancestor bind authors: A. Removal of `MaterialBindingAPI` · B. `rel material:binding = None` · C. Deletion of the Material · D. `rel material:binding:preview = None` only

**Q40.13** · Obj 8.2 · Medium · Single choice A shot should recolor an asset without editing `/Looks/Paint/Preview`. What should the asset have authored? A. A variant set on the Shader · B. Material interface `inputs:paint` that Preview connects to · C. `displayColor` on every mesh · D. `bindMaterialAs` on the mesh

### Answers

**Q40.1** — **B**. `info:id` identifies a Shader implementation. Review: §40.1, §40.3.

**Q40.2** — **B**. Material derives from `NodeGraph`. Review: §40.1.

**Q40.3** — **C**. Unpack the tuple; do not call `.GetPath()` on the return value itself. Review: §40.2.

**Q40.4** — **C**. `float3` = color. `float2` is UVs; `UVTexture` is images; `PreviewSurface` is the lighting model. Review: §40.5.

**Q40.5** — **B**. `varname` is the primvar name without `primvars:`. Review: §40.5.

**Q40.6** — **A, B**. Apply then Bind. Meshes do not carry `info:id`. Review: §40.6.

**Q40.7** — **B**. Unauthored / default strength is `strongerThanDescendants`. Review: §40.6.

**Q40.8** — **C**. Empty string is all-purpose. Review: §40.6.

**Q40.9** — **A, B**. Two targets, collection then Material; not the included prims and not a Shader. Review: §40.6.

**Q40.10** — **B**. Connection wins; `(1, 0, 0)` is leftover. Review: §40.2.

**Q40.11** — **B**. `UsdPrimvarReader_float2` + `varname=st`. Review: §40.4.

**Q40.12** — **B**. The applied schema stays; the direct rel is authored `None`. Review: §40.6.

**Q40.13** — **B**. Interface inputs are the public override API. Review: §40.7.

### USDA exercise answers

**40-A — The Material has no Shader, and info:id is on the Material.** Hydra runs a Shader with `info:id = UsdPreviewSurface` whose `outputs:surface` is connected to the Material terminal. Move `info:id` and `diffuseColor` onto a child Shader, declare `outputs:surface` there, and connect token `outputs:surface.connect = </Looks/Paint/Preview.outputs:surface>` on the Material. The bind and Apply are already correct.

**40-B — /Looks/Chrome.** The parent is explicitly `weakerThanDescendants`, so the child's direct bind wins. If `bindMaterialAs` were omitted, fallback `stronger` would have resolved to Clay.

**40-C — No.** Three problems: (1) reader id is `float2` but color needs `UsdPrimvarReader_float3`; (2) `varname` must be `displayColor`, not `primvars:displayColor`; (3) the mesh never binds the Material (`MaterialBindingAPI` + `rel material:binding`). Any one of these fails Obj 8.4.

## CHAPTER 41

## Chapter 41 — UsdLux and Rendering

**Exam domain:** Visualization (8%) · **Objectives:** 8.x (UsdLux / Hydra) · **Study day:** 11 · **Est. time:** 80 min **Prerequisites:** Ch 6 (schema domains), Ch 13 / 39 (Xformable, purpose, cameras), Ch 40 (UsdShade inputs: namespace)

Geometry (Chapter 39) and materials (Chapter 40) still need **light**. **UsdLux** is the schema domain for lights. Typed prims such as `DistantLight` and `RectLight` carry **LightAPI** automatically. Optional **ShadowAPI** and **ShapingAPI** add shadows and spot cones. This chapter also sketches **Hydra**, render delegates, and `usdview` — concepts you must recognize even though `usd-core` cannot draw a pixel.

Official numbered objectives stop at 8.4 (primvars and `PreviewSurface`). **UsdLux** and **Hydra** appear in the study-guide reading list and in Visualization questions that name light types, `inputs:intensity`, and `usdview`. Treat them as exam material.

### Learning goals

- Define the six common light types and predict intensity fallbacks (Distant vs local).
- Explain built-in **LightAPI** versus applied **ShadowAPI** / **ShapingAPI**.
- Author `inputs:intensity`, `inputs:exposure`, `inputs:color`, and color temperature.
- Describe Hydra, render delegates, and `usdview`, and know what `usd-core` cannot do.

### Key terms

Term	One-line definition
<b>UsdLux</b>	Schema domain for lights, light filters, and light linking.
<b>LightAPI</b>	Applied API (built into typed lights) that owns <code>inputs:intensity</code> , <code>exposure</code> , <code>color</code> , ...
<b>DistantLight</b>	Infinite parallel rays (the sun); default intensity <b>50000</b> .
<b>Sphere / Rect / Disk / Cylinder</b>	Local area lights with size attributes.
<b>DomeLight</b>	Image-based environment light ( <code>inputs:texture:file</code> ).
<b>ShadowAPI</b>	Applied API for shadow on/off, color, distance, falloff.
<b>ShapingAPI</b>	Applied API for spot cones ( <code>shaping:cone:angle</code> ) and IES focus.
<b>Exposure</b>	Power-of-two scale: rendered intensity $\propto$ intensity $\times 2^{\text{exposure}}$ .
<b>Color temperature</b>	Kelvin blackbody tint, used only when <code>enableColorTemperature</code> is true.
<b>Hydra</b>	USD's imaging framework: scene $\rightarrow$ render delegate $\rightarrow$ pixels.
<b>Render delegate</b>	A Hydra backend (Storm, RenderMan, Arnold, ...).
<b>usdview</b>	The reference USD viewer (not shipped in <code>usd-core</code> ).

### 41.1 Light types (Distant, Sphere, Rect, Disk, Cylinder, Dome)

**What is it?** UsdLux provides **typed light prims**. The six you must recognize: **DistantLight** (sun), **SphereLight**, **RectLight**, **DiskLight**, **CylinderLight**, and **DomeLight** (environment / HDRI). Each Defines like a Mesh:  
`UsdLux.RectLight.Define(stage, "/Lights/Key").`

**Why do we need it?** A PreviewSurface with roughness 0.1 still looks flat without a light. Exporters that drop lights produce “grey clay” in usdview even when materials are perfect. Shared light types are how DCCs exchange a key / fill / rim / dome.

**Beginner explanation.** Think of a film set. The **sun** through a window is Distant (parallel rays, very far). A **softbox** is Rect. A **bare bulb** is Sphere. A **snooted circular light** is Disk. A **fluorescent tube** is Cylinder. The **painted cyclorama / HDRI ball** is Dome.

*Where the analogy breaks:* USD lights do not consume watts from a generator. Intensity is a schema number; each render delegate maps it to its own photometric units.

**Technical explanation.** - All six are **Xformable** and **Imageable** (Chapter 39): they take xformOps, visibility, and purpose. They emit along local **-Z**, like cameras. - **LightAPI is built in.** HasAPI(UsdLux.LightAPI) is True after Define even though USDA shows no apiSchemas. Built-in companions: CollectionAPI:lightLink and CollectionAPI:shadowLink (includeRoot fallback True = affect everything). - **Boundable vs not.** Sphere, Rect, Disk, Cylinder (and Portal) are BoundableLightBase — they have size and an extent. Distant and Dome are NonboundableLightBase — infinite / environmental, no mesh-like extent. -

**Intensity fallbacks (verified 26.08):** DistantLight **50000**; every local type and Dome **1**. Distant also has inputs:angle (angular diameter in degrees, fallback **~0.53**, the real sun). - Size fallbacks: Sphere / Disk inputs:radius **0.5**; Rect inputs:width / height **1**; Cylinder inputs:length **1**, radius **0.5**. Sphere has treatAsPoint; Cylinder has treatAsLine (both fallback False). - Dome: inputs:texture:file (asset), inputs:texture:format (automatic, latlong, angular, mirroredBall). **DomeLight\_1** adds poleAxis (fallback scene) so the HDRI follows stage upAxis. Prefer **\_1** in new files; this book shows DomeLight because it is the name you will still see. - **MeshLightAPI** applied to a Mesh turns geometry into a light (replaces the older GeometryLight typed prim). PortalLight is a window that admits dome light.

#### VERSION

Verified on USD 26.08. Light attributes live in the `**inputs:**` namespace (``inputs:intensity``) since USD 21.02 so they can connect in a shading graph. Old files used bare ``intensity``. Always author ``inputs:``.

#### Mental model.

sun / directional	DistantLight	intensity fallback 50000	non-boundable
point-like / bulb	SphereLight	radius 0.5	boundable
softbox	RectLight	width × height	boundable
circular / snoot	DiskLight	radius	boundable
tube	CylinderLight	length × radius	boundable
HDRI / sky	DomeLight	texture:file	non-boundable

**Simple example.** /Lights/Distant with no authored intensity still reports `Get() == 50000`. /Lights/Sphere reports 1. That difference is a favorite exam trap.

#### USDA example.

```
#usda 1.0
(
 defaultPrim = "Lights"
 upAxis = "Y"
)

def Xform "Lights"
{
 def DistantLight "Sun"
 {
 float inputs:intensity = 50000
 float inputs:angle = 0.53
 float xformOp:rotateX = -90
 uniform token[] xformOpOrder = ["xformOp:rotateX"]
 }
}
```

```

def RectLight "Key"
{
 float inputs:width = 2
 float inputs:height = 1
 float inputs:intensity = 8
 double3 xformOp:translate = (2, 3, 4)
 uniform token[] xformOpOrder = ["xformOp:translate"]
}

def DomeLight "Sky"
{
 asset inputs:texture:file = @./sky.hdr@
 token inputs:texture:format = "latlong"
}
}

```

- Empty `def DistantLight "Sun" {}` would still *behave* at intensity 50000; authoring the value makes the intent readable.
- `rotateX = -90` on a Y-up stage aims the Distant –Z axis downward (sun from above). Same convention as cameras (Chapter 39.7).

### Python example.

```

from pxr import Usd, UsdLux

stage = Usd.Stage.CreateInMemory()
makers = [
 ("/Lights/Distant", UsdLux.DistantLight),
 ("/Lights/Sphere", UsdLux.SphereLight),
 ("/Lights/Rect", UsdLux.RectLight),
 ("/Lights/Disk", UsdLux.DiskLight),
 ("/Lights/Cylinder", UsdLux.CylinderLight),
 ("/Lights/Dome", UsdLux.DomeLight),
]
for path, cls in makers:
 prim = cls.Define(stage, path).GetPrim()
 api = UsdLux.LightAPI(prim)
 print(f"{prim.GetName()} {prim.GetTypeName()} "
 f"I={api.GetIntensityAttr().Get()} "
 f"boundable={prim.IsA(UsdLux.BoundableLightBase)}")
sun = UsdLux.DistantLight(stage.GetPrimAtPath("/Lights/Distant"))
print("sun angle:", round(sun.GetAngleAttr().Get(), 2))
print("sphere radius:",
 UsdLux.SphereLight(stage.GetPrimAtPath("/Lights/Sphere"))
 .GetRadiusAttr().Get())
print("rect size:",
 UsdLux.RectLight(stage.GetPrimAtPath("/Lights/Rect"))
 .GetWidthAttr().Get(),
 UsdLux.RectLight(stage.GetPrimAtPath("/Lights/Rect"))
 .GetHeightAttr().Get())
print("applied on Distant:",
 list(stage.GetPrimAtPath("/Lights/Distant").GetAppliedSchemas()))

```

### Expected output

```

Distant DistantLight I=50000.0 boundable=False
Sphere SphereLight I=1.0 boundable=True
Rect RectLight I=1.0 boundable=True
Disk DiskLight I=1.0 boundable=True
Cylinder CylinderLight I=1.0 boundable=True
Dome DomeLight I=1.0 boundable=False
sun angle: 0.53
sphere radius: 0.5
rect size: 1.0 1.0
applied on Distant: ['LightAPI', 'CollectionAPI:lightLink', 'CollectionAPI:shadowLink']

```

USDA for that Distant prim is still `def DistantLight "Distant" {}` — the applied schemas are **built in**, not authored.

**Real-world use case.** A product-shot USD ships `RectLight` "Key" and "Fill" plus `DomeLight` "Studio" with a lat-long HDRI. The manufacturing digital twin uses one `DistantLight` for the sun (intensity left at the 50000 fallback) and a `DomeLight` for sky. Games often import only the Dome as IBL and ignore local UsdLux, which is why the mapping table in Chapter 27 mentioned `KHR_lights_punctual`.

### Common mistakes.

#### MISTAKE

Setting `Distant`inputs:intensity = 1`` because "that's the default for lights." You just made the sun 50 000× dimmer than the schema fallback. Use `~50000` or compensate with exposure.

#### MISTAKE

Importing lights from ``UsdGeom``. There is no ``UsdGeom.DistantLight``. ``from pxr import UsdLux``.

#### MISTAKE

Forgetting `xformOps`. A `RectLight` at the origin, emitting `-Z`, may light the back of the set. Place it like a camera.

### Exam traps.

#### TRAP

"All lights default intensity 1." `DistantLight` is `**50000**`.

#### TRAP

USDA shows no ``apiSchemas`` on a `DistantLight`, so ``HasAPI(LightAPI)`` is `False`. It is `**True**` (built-in).

#### TRAP

`DomeLight`texture:file`` is a `Shader`UsdUVTexture``. No: it is an `**attribute on the DomeLight**` (``asset inputs:texture:file``).

### Exam takeaways.

#### KEY

- Six types: `Distant`, `Sphere`, `Rect`, `Disk`, `Cylinder`, `Dome`. Domain = `**UsdLux**`. - `Distant` fallback intensity `**50000**`; others `**1**`. `Distant`angle``  $\approx 0.53^\circ$ . - Typed lights already have `LightAPI` + `lightLink/shadowLink` collections. - Emit along local `-Z`; place with `xformOps`. ``inputs:`` namespace since 21.02.

## 41.2 LightAPI, ShadowAPI, ShapingAPI

**What is it?** `LightAPI` is the common applied schema on every UsdLux light: intensity, exposure, color, normalize, diffuse/specular weights, color temperature. **ShadowAPI** and **ShapingAPI** are *optional* applied APIs. Shadow controls whether the light casts shadows. Shaping turns a `Disk/Rect/Sphere` into a **spot** (`shaping:cone:angle`) or applies an IES profile.

**Why do we need it?** Not every light needs a cone or a shadow toggle. Keeping those properties off the typed schema means a `Distant` sun is a short prim, and a theatrical spot is the same `DiskLight` plus two applied APIs. That is the same Apply pattern as `MaterialBindingAPI` (Chapter 40) — with one twist: `LightAPI` is already there.

**Beginner explanation.** `LightAPI` is the dimmer and gel that every lamp ships with. `ShadowAPI` is the optional barn-door / flag kit ("this lamp casts a shadow"). `ShapingAPI` is the snoot or cookie that squeezes the beam into a cone.

Where the analogy breaks: applying ShapingAPI does not change the prim **type**. It is still a DiskLight. Validators look at apiSchemas, not at a SpotLight type (USD has none).

**Technical explanation.** - **LightAPI** attributes (all inputs:): intensity, exposure, color, enableColorTemperature, colorTemperature, normalize, diffuse, specular. Also light:shaderId and light:materialSyncMode (advanced). - Typed Define already applies LightAPI. Calling LightAPI.Apply again is legal but redundant; it authors prepend apiSchemas = ["LightAPI"] that was already built in. - **ShadowAPI.Apply(prim)** then set inputs:shadow:enable (fallback **True**), inputs:shadow:color (fallback black), plus distance / falloff / falloffGamma. Until you Apply, HasAPI(ShadowAPI) is False even though the light may still cast shadows in a renderer using its own defaults. -

**ShapingAPI.Apply(prim)** then inputs:shaping:cone:angle (fallback **90**, i.e. a hemisphere), inputs:shaping:cone:softness, inputs:shaping:focus, inputs:shaping:focusTint, IES file / angle scale. A 30° cone is a spot. -

**MeshLightAPI.Apply(mesh)** makes a Mesh emit; it brings LightAPI (and the link collections) with it. USDA shows prepend apiSchemas = ["MeshLightAPI"] only — LightAPI is built into that API. - **Light linking:** CollectionAPI:lightLink / shadowLink on the light. includeRoot True means “all geometry.” Exclude paths to keep a light off a character. LightListAPI on a model caches a list of lights for the renderer. - **LightFilter** is a typed prim for extra filters (barn doors, blockers) in the Lux family; know the name, do not memorize every filter.

#### VERSION

Verified on USD 26.08. Contrast with Chapter 40: `MaterialBindingAPI` is **not** built into Mesh — you must `Apply`. `LightAPI` **is** built into `DistantLight` / `RectLight` / ... You must `Apply` only ShadowAPI, ShapingAPI, and MeshLightAPI.

#### Mental model.

```
DiskLight "Spot"
 built-in: LightAPI, CollectionAPI:lightLink, CollectionAPI:shadowLink
 authored: prepend apiSchemas = ["ShadowAPI", "ShapingAPI"]
 inputs:shadow:enable
 inputs:shaping:cone:angle = 30
```

Type = DiskLight. Behavior = spot, because of applied APIs.

**Simple example.** Define /Lights/Spot as DiskLight. HasAPI LightAPI True, HasAPI ShadowAPI False. Apply Shadow + Shaping, set cone 30. USDA shows those two APIs in apiSchemas, not LightAPI.

#### USDA example.

```
#usda 1.0

def DiskLight "Spot" (
 prepend apiSchemas = ["ShadowAPI", "ShapingAPI"]
)
{
 float inputs:intensity = 20
 bool inputs:shadow:enable = 1
 float inputs:shaping:cone:angle = 30
 float inputs:shaping:cone:softness = 0.2
 double3 xform0p:translate = (0, 3, 0)
 float xform0p:rotateX = -90
 uniform token[] xform0pOrder = ["xform0p:translate", "xform0p:rotateX"]
}
```

- apiSchemas lists **only** the extra APIs you applied.
- Cone angle 30 + rotateX -90 = a spotlight from above on a Y-up stage.

#### Python example.

```

from pxr import Usd, UsdLux

stage = Usd.Stage.CreateInMemory()
spot = UsdLux.DiskLight.Define(stage, "/Lights/Spot")
print("HasAPI LightAPI:", spot.GetPrim().HasAPI(UsdLux.LightAPI))
print("HasAPI Shadow before:",
 spot.GetPrim().HasAPI(UsdLux.ShadowAPI))
UsdLux.ShadowAPI.Apply(spot.GetPrim())
UsdLux.ShapingAPI.Apply(spot.GetPrim())
sh = UsdLux.ShadowAPI(spot.GetPrim())
sg = UsdLux.ShapingAPI(spot.GetPrim())
print("HasAPI Shadow:", spot.GetPrim().HasAPI(UsdLux.ShadowAPI))
print("HasAPI Shaping:", spot.GetPrim().HasAPI(UsdLux.ShapingAPI))
print("shadow enable fallback:", sh.GetShadowEnableAttr().Get())
print("cone angle fallback:", sg.GetShapingConeAngleAttr().Get())
sh.GetShadowEnableAttr().Set(True)
sg.GetShapingConeAngleAttr().Set(30.0)
sg.GetShapingConeSoftnessAttr().Set(0.2)
print("applied:", list(spot.GetPrim().GetAppliedSchemas()))
for line in stage.GetRootLayer().ExportToString().splitlines():
 if "apiSchemas" in line or "shadow" in line or "shaping" in line:
 print(line.strip())

```

### Expected output

```

HasAPI LightAPI: True
HasAPI Shadow before: False
HasAPI Shadow: True
HasAPI Shaping: True
shadow enable fallback: True
cone angle fallback: 90.0
applied: ['LightAPI', 'CollectionAPI:lightLink', 'CollectionAPI:shadowLink', 'ShadowAPI',
'ShapingAPI']
prepend apiSchemas = ["ShadowAPI", "ShapingAPI"]
bool inputs:shadow:enable = 1
float inputs:shaping:cone:angle = 30
float inputs:shaping:cone:softness = 0.2

```

GetAppliedSchemas() lists built-ins first; USDA apiSchemas lists only what you Applied.

**Real-world use case.** A stage play USD uses DiskLights with ShapingAPI cones parented to moving Xforms (follow spots). ShadowAPI stays enabled for the key, disabled on fill (inputs:shadow:enable = 0) so fill does not double the character's shadow. Light-link collections exclude the theatrical curtain so the key does not light the drape.

### Common mistakes.

#### MISTAKE

Searching for `UsdLux.SpotLight`. There is none. Disk/Rect/Sphere + **\*\*ShapingAPI\*\*** is a spot.

#### MISTAKE

Assuming ShadowAPI is built in because shadows appear in Storm. `HasAPI(ShadowAPI)` is False until `Apply`. Author it when you need an explicit off switch.

#### MISTAKE

Applying LightAPI on every Mesh "to be safe." That does not make a mesh a light. Use **\*\*MeshLightAPI\*\*** (or a typed light prim).

### Exam traps.

#### TRAP

"You must `LightAPI.Apply` before setting intensity, like MaterialBindingAPI." Intensity works without that `Apply` on typed lights; LightAPI is built in.



**TRAP**

Cone fallback 30°. It is **\*\*90\*\*** (hemisphere). 30 is a value *\*you\** author for a spot.

**TRAP**

``bool inputs:shadow:enable = 1`` in USDA is False because it is not ``true``. In USDA, ``1`` is true for bools.

**Exam takeaways.****KEY**

- LightAPI is built into typed lights; ShadowAPI and ShapingAPI are opt-in ``Apply``. - No SpotLight type: ShapingAPI cone on Disk/Rect/Sphere. - USDA ``apiSchemas`` shows extras; ``GetAppliedSchemas()`` also lists built-ins. - MeshLightAPI on a Mesh; lightLink/shadowLink collections for linking.

**41.3 Intensity, exposure, color temperature**

**What is it?** Every LightAPI light has **intensity** (linear brightness), **exposure** (photographic stops, a power of two), **color** (an RGB gel), and optional **color temperature** in Kelvin. Rendered brightness scales as **intensity × 2<sup>exposure</sup>**. Temperature applies only when **enableColorTemperature** is true.

**Why do we need it?** Lighting TDs think in stops (“plus one stop”) while TDs debugging a converter think in a single float. Exposure lets the artist add a stop without rewriting intensity. Color temperature matches real lamps (2700 K tungsten, 6500 K daylight) without picking RGB by eye.

**Beginner explanation.** Intensity is the dimmer’s linear scale. Exposure is the camera’s ISO/stop knob: each +1 doubles the light (2<sup>exposure</sup>). Color is a gel taped on the lamp. Color temperature is “set the gel from a Kelvin chart,” but the lamp ignores the chart until you flip **enableColorTemperature**.

*Where the analogy breaks:* USD does not guarantee nits or lumens. `normalize` (fallback False) asks the renderer to keep brightness independent of the light’s area; delegates differ. Always check the delegate’s UsdLux notes.

**Technical explanation.** - `inputs:intensity` (float). Fallback **1** on local lights / Dome, **50000** on Distant (§41.1). - `inputs:exposure` (float, fallback **0**). Formula used by UsdLux: **final ∝ intensity × 2<sup>exposure</sup>**. Exposure 2 with intensity 3 → scale 3 × 4 = 12. - `inputs:color` (color3f, fallback white (1, 1, 1)). - `inputs:enableColorTemperature` (bool, fallback **False**). `inputs:colorTemperature` (float, fallback **6500**, daylight). When enabled, the renderer tints using a blackbody curve. Python helper: `UsdLux.BlackbodyTemperatureAsRgb(kelvin) → Gf.Vec3f` (verified: 2700 K ≈ (1.9353, 0.8036, 0.1917)). - `inputs:normalize` (bool, fallback False): when True, intensity is per-area so scaling a RectLight up does not make it brighter. - `inputs:diffuse` / `inputs:specular` (float, fallback 1): weights for those lobes. Set specular 0 for a fill that should not sparkle in the eye. - These are connectable **inputs** (same inputs: idea as UsdShade). A stronger layer can override them; a shader could theoretically connect to them.

**Mental model.**

```
emitted ~ color × intensity × 2exposure × (blackbody if enableColorTemperature)

2700 K tungsten enableColorTemperature = true
6500 K daylight fallback temperature (ignored until enable is true)
```

**Simple example.** Sphere bulb: intensity 3, exposure 2, color (1, 0.9, 0.8), `enableColorTemperature true`, 2700 K. USDA authors all five inputs:.

**USDA example.**

```
#usda 1.0

def SphereLight "Bulb"
{
 color3f inputs:color = (1, 0.9, 0.8)
 float inputs:colorTemperature = 2700
 bool inputs:enableColorTemperature = 1
 float inputs:exposure = 2
 float inputs:intensity = 3
 bool inputs:normalize = 0
 float inputs:radius = 0.5
}
```

- Without `enableColorTemperature = 1`, the 2700 value is authored but unused.
- `exposure = 2` is two stops up ( $\times 4$ ), not “plus 2 intensity.”

### Python example.

```
from pxr import Usd, UsdLux

stage = Usd.Stage.CreateInMemory()
bulb = UsdLux.SphereLight.Define(stage, "/Lights/Bulb")
api = UsdLux.LightAPI(bulb.GetPrim())
api.GetIntensityAttr().Set(3.0)
api.GetExposureAttr().Set(2.0)
api.GetColorAttr().Set((1.0, 0.9, 0.8))
api.GetEnableColorTemperatureAttr().Set(True)
api.GetColorTemperatureAttr().Set(2700.0)
rgb = UsdLux.BlackbodyTemperatureAsRgb(2700)
print("intensity:", api.GetIntensityAttr().Get())
print("exposure:", api.GetExposureAttr().Get())
print("2**exposure:", 2 ** api.GetExposureAttr().Get())
print("color:", api.GetColorAttr().Get())
print("enableColorTemperature:",
 api.GetEnableColorTemperatureAttr().Get())
print("colorTemperature:", api.GetColorTemperatureAttr().Get())
print("2700K rgb:", tuple(round(c, 4) for c in rgb))
print("normalize fallback:", api.GetNormalizeAttr().Get())
for line in stage.GetRootLayer().ExportToString().splitlines():
 if "inputs:" in line:
 print(line.strip())
```

### Expected output

```
intensity: 3.0
exposure: 2.0
2**exposure: 4.0
color: (1, 0.9, 0.8)
enableColorTemperature: True
colorTemperature: 2700.0
2700K rgb: (1.9353, 0.8036, 0.1917)
normalize fallback: False
color3f inputs:color = (1, 0.9, 0.8)
float inputs:colorTemperature = 2700
bool inputs:enableColorTemperature = 1
float inputs:exposure = 2
float inputs:intensity = 3
```

The USDA dump matches §7 (plus the unauthored radius fallback, which is not written).

**Real-world use case.** A practical tungsten lamp on a film set is a `SphereLight` at 2700 K with `enableColorTemperature` on. The gaffer adds one stop in the shot layer by setting `inputs:exposure = 1` rather than multiplying intensity, so the published asset’s intensity stays a round number. Daylight plates keep 6500 K and usually leave enable off, relying on `inputs:color` from the HDRI dome instead.

### Common mistakes.

**MISTAKE**

Authoring `colorTemperature = 2700`` and leaving `enableColorTemperature`` false. The light stays white (or whatever `inputs:color`` is). Flip the bool.

**MISTAKE**

Treating exposure as additive intensity (`intensity + exposure``). It is **stops**: `2^exposure``.

**MISTAKE**

Using `BlackbodyTemperatureAsRgb`` as the authored `inputs:color`` *and* enabling temperature, double-tinting. Pick one: Kelvin+enable, or a gel RGB.

**Exam traps.****TRAP**

Fallback color temperature 2700 K. It is **6500**. 2700 is a value you author for tungsten.

**TRAP**

"Color temperature replaces intensity." It tints; intensity/exposure still scale brightness.

**TRAP**

`inputs:intensity`` vs old `intensity``. Exam USDA that shows bare `intensity`` is pre-21.02; the current name is `inputs:intensity``.

**Exam takeaways.****KEY**

- Brightness  $\propto \text{intensity} \times 2^{\text{exposure}}$ . Exposure 0 is a no-op. - Distant intensity fallback 50000; local 1; exposure fallback 0. - Color temperature is ignored until `enableColorTemperature`` is true (fallback temp 6500). - `inputs:`` names; `normalize`` keeps intensity per-area when True.

**41.4 Hydra, render delegates, usdview (concepts)**

**What is it?** **Hydra** is OpenUSD's imaging architecture: it takes a composed stage and asks a **render delegate** to produce pixels. **usdview** is the reference application that hosts Hydra. A **render delegate** is a plug-in backend — Storm (OpenGL rasterizer), RenderMan, Arnold, Cycles, and others. **UsdRender.Settings** is the schema for render settings (resolution, products) stored in the stage.

**Why do we need it?** Authoring PreviewSurface and UsdLux is pointless if you cannot name the thing that *draws* them. Exam questions say "in usdview" or "the Storm delegate" or "Hydra." You must know the pipeline even when your pip package cannot open a window.

**Beginner explanation.** Think of a mixing desk (the stage) plugged into different speakers. Hydra is the amp rack. Storm is the cheap rehearsal speaker (fast, GL). RenderMan is the concert PA (slow, beautiful). usdview is the room where you pick which speaker is plugged in. Switching delegates does not rewrite the USD; it changes who interprets materials and lights.

*Where the analogy breaks:* some delegates ignore parts of UsdLux or PreviewSurface, or only read purpose = render. The file is the same; the picture is not.

**Technical explanation.** - **Pipeline:** composed Usd.Stage → Hydra **scene index** (Chapter 38: the modern imaging filter graph) → **render delegate** → framebuffer or EXR. Older path: Hydra primary adapter / UsdImaging hydra plugin. usd-core 26.8 ships **neither** UsdImaging nor Hd. - **Storm** (HdStorm) is the default raster delegate in usdview: good for

layout, purpose, PreviewSurface, simple UsdLux. It is not a production path tracer. - **usdview** features you will see named: viewport, **Metal/GL**, **purpose** filters (proxy / render / guide, Chapter 39.6), **Lights** on/off, Hydra renderer dropdown, Layer Stack / Composition tabs (Chapter 22). Command: `usdview shot.usda` (not runnable here). - **Material purpose** (Chapter 40.6 `preview / full`) is how `usdview`'s preview pass picks PreviewSurface while a film delegate may bind `full`. - **UsdRender.Settings**, `RenderProduct`, `RenderVar`: describe cameras, resolution, and AOVs inside USD so `usdrecord / batch` renderers do not need a side-car XML. Present in `usd-core`. - **usdrecord** renders frames through Hydra from the command line. Same limitation: not in `usd-core`. - Changing a light's intensity is a stage edit; Hydra notices via the same change pipeline as Chapter 34 (`ObjectsChanged`). You do not restart `usdview` to see a new intensity.

#### VERSION

Verified on USD 26.08. ``from pxr import UsdImaging`` and ``from pxr import Hd`` raise ``ImportError`` in this book's ``.venv``. Teach Hydra as concepts; keep examples that `*run*` limited to schema authoring and ``UsdRender.Settings``.

#### Mental model.

```
.usda / .usdc
 → Usd.Stage (composition, LIVERPS)
 → Hydra scene index
 → render delegate Storm | Prman | Arnold | ...
 → pixels in usdview / usdrecord / DCC viewport
```

No delegate, no picture. The stage does not know about pixels.

**Simple example.** `usdview set.usda` opens Storm, draws meshes with PreviewSurface, uses UsdLux Distant + Dome. Switching the Hydra dropdown to a RenderMan delegate still reads the same prims; shadows and Dome importance sampling change.

#### USDA example.

```
#usda 1.0
(
 defaultPrim = "World"
)

def Xform "World"
{
 def Mesh "Ground" (
 prepend apiSchemas = ["MaterialBindingAPI"]
)
 {
 rel material:binding = </Looks/Clay>
 }

 def DistantLight "Sun"
 {
 float inputs:intensity = 50000
 }
}

def Scope "Looks"
{
 def Material "Clay" {}
}

def Scope "Render"
{
 def RenderSettings "Settings"
 {
 }
}
```

- Geometry, a light, a (stub) material bind, and a RenderSettings prim: the four prims a Hydra app looks for.
- `RenderSettings` is authored data; it does not draw by itself.

**Python example.** This block checks what `usd-core` actually imports and authors a `RenderSettings` prim.

```
print("UsdImaging imported:", end=" ")
try:
 from pxr import UsdImaging
 print(True)
except ImportError:
 print(False)
print("Hd imported:", end=" ")
try:
 from pxr import Hd
 print(True)
except ImportError:
 print(False)

from pxr import Usd, UsdRender

stage = Usd.Stage.CreateInMemory()
settings = UsdRender.Settings.Define(stage, "/Render/Settings")
print("render settings:", settings.GetPrim().GetTypeName())
```

### Expected output

```
UsdImaging imported: False
Hd imported: False
render settings: RenderSettings
```

Usdview itself is a command-line tool, not a Python import. Mark it `.norun` when you quote it:

```
Not in usd-core; requires a full OpenUSD / NVIDIA inventory build.
usdview shot.usda
usdrecord --camera /Shot/Cam --frames 1001:1100 shot.usda out.#.exr
```

**Real-world use case.** A layout artist in usdview / Storm places `RectLights` and checks proxy purpose. Lighting opens the same USD in a DCC with the `RenderMan` delegate, which reads `purpose = render meshes`, `material:binding:full`, and `DomeLight` importance samples. Batch `usdrecord` uses `RenderSettings` for resolution. Nobody converts the file between those apps.

### Common mistakes.

#### MISTAKE

Expecting ``pip install usd-core`` to provide `usdview`. It does not (Chapter F5). On Windows and Linux, NVIDIA's OpenUSD prebuilts include `usdview`; otherwise build with `imaging`.

#### MISTAKE

Assuming Storm matches the beauty render. Storm is a raster preview. Lookdev sign-off needs the production delegate.

#### MISTAKE

Putting renderer-specific attributes only on ``RenderSettings`` and omitting `UsdLux`. Delegates still need the light prims.

### Exam traps.

#### TRAP

"Hydra is a renderer." Hydra is the `**framework**`; Storm / Prman / Arnold are `**delegates**`.

**TRAP**

Mixing Imageable `purpose` (`proxy`/`render`/`guide`) with material-binding purpose (`preview`/`full`). Both affect usdview; they are different schemas (Ch 39.6 vs Ch 40.6).

**TRAP**

`UsdImaging` is part of `usd-core` because UsdGeom is. Geom is data; imaging is the optional view plugin.

**Exam takeaways.****KEY**

- Hydra = imaging framework; a **render delegate** draws pixels. - usdview hosts Hydra (Storm by default); not shipped in `usd-core`. - Imageable purpose and material purpose both affect what you see; they are not the same. - `UsdRender.Settings` is authorable in `usd-core`; `UsdImaging` / `Hd` are not.

**Chapter lab(s)**

**Lab 35 — Lights with UsdLux** (★★☆, Obj 8.x). You define a Distant sun (leave or author intensity 50000), a Rect key with xformOps, a Disk spot with ShapingAPI + ShadowAPI, and a Dome with `texture:file`. You set exposure vs intensity and enable 2700 K on a Sphere practical. Stretch: MeshLightAPI on a panel, lightLink exclude.

**USDA reading exercise(s)**

**Exercise 41-A.** A set is nearly black in usdview despite a sun prim. Why, given this USDA?

```
#usda 1.0

def DistantLight "Sun"
{
 float inputs:intensity = 1
 float inputs:exposure = 0
}
```

**Exercise 41-B.** Is `/Lights/Spot` a spot, and does it have LightAPI?

```
#usda 1.0

def DiskLight "Spot" (
 prepend apiSchemas = ["ShapingAPI"]
)
{
 float inputs:shaping:cone:angle = 25
}
```

**Exercise 41-C.** Will this bulb render as tungsten orange?

```
#usda 1.0

def SphereLight "Practical"
{
 float inputs:colorTemperature = 2700
 float inputs:intensity = 5
}
```

**Chapter review****Summary**

- UsdLux typed lights: Distant, Sphere, Rect, Disk, Cylinder, Dome. Emit -Z. Distant intensity fallback 50000; others 1.
- LightAPI is built in; Apply ShadowAPI / ShapingAPI / MeshLightAPI when you need them. No SpotLight type.

- Brightness  $\propto$  intensity  $\times 2^{\text{exposure}}$ . Color temperature needs `enableColorTemperature`.
- Hydra delegates draw; `usdview/Storm/UsdImaging` are not in `usd-core`. `UsdRender.Settings` is.

### If you see... → think...

If you see...	Think...
Dim sun, <code>inputs:intensity = 1</code>	Distant fallback is 50000
<code>SpotLight</code> as a prim type	Does not exist; <code>ShapingAPI</code> on <code>Disk/Rect</code>
Empty <code>def DistantLight "Sun" {}</code>	Still HasAPI <code>LightAPI</code> ; intensity 50000
<code>prepend apiSchemas = ["ShadowAPI"]</code>	Extra API; <code>LightAPI</code> is built-in already
<code>colorTemperature</code> but still white	<code>enableColorTemperature</code> is false
<code>exposure = 2</code>	$\times 4$ , not $+2$
<code>inputs:intensity</code> vs <code>intensity</code>	Post-21.02 namespace
<code>usdview / Storm / Hydra</code>	Framework vs delegate vs app; not in <code>usd-core</code>
<code>purpose = proxy</code> vs <code>material:binding:preview</code>	Imageable vs material purpose
Mesh that emits	<code>MeshLightAPI</code> , not <code>LightAPI</code> alone

### Review questions

- Q41.1** · Obj 8.x · Easy · Single choice Which module defines `RectLight`? A. `UsdGeom` · B. `UsdShade` · C. `UsdLux` · D. `UsdRender`
- Q41.2** · Obj 8.x · Easy · Single choice `Unauthored DistantLight intensity Get()` returns: A. 1.0 · B. 0.0 · C. 50000.0 · D. None
- Q41.3** · Obj 8.x · Medium · Select two. Which lights are *not* `BoundableLightBase`? A. `DistantLight` · B. `RectLight` · C. `DomeLight` · D. `SphereLight`
- Q41.4** · Obj 8.x · Medium · Single choice How do you author a spotlight in `UsdLux`? A. `UsdLux.SpotLight.Define` · B. `Disk/Rect/Sphere + ShapingAPI cone` · C. `UsdGeom.Cone + LightAPI` · D. `DomeLight` with `angle`
- Q41.5** · Obj 8.x · Medium · Single choice `def DistantLight "Sun" {}` — `HasAPI(UsdLux.LightAPI)` is: A. False until `LightAPI.Apply` · B. True (built-in) · C. True only if intensity is authored · D. An error
- Q41.6** · Obj 8.x · Easy · Single choice Intensity 1, exposure 2. Relative brightness vs intensity 1 exposure 0: A. Same · B.  $\times 2$  · C.  $\times 4$  · D.  $+2$
- Q41.7** · Obj 8.x · Medium · Single choice `inputs:colorTemperature = 2700` with no other temperature attrs. Effect? A. Tungsten tint · B. None; enable is false · C. Replaces intensity with 2700 · D. Blackbody is multiplied twice
- Q41.8** · Obj 8.x · Easy · Single choice `UsdLux` attributes such as intensity are named: A. `intensity` · B. `inputs:intensity` · C. `light:intensity` · D. `outputs:intensity`
- Q41.9** · Obj 8.x · Medium · Select two. Which imports fail on `usd-core 26.8`? A. `from pxr import UsdLux` · B. `from pxr import UsdImaging` · C. `from pxr import Hd` · D. `from pxr import UsdRender`
- Q41.10** · Obj 8.x · Medium · Single choice `Storm` is: A. A path tracer · B. Hydra's GL raster render delegate · C. A `UsdShade` shader id · D. A file format
- Q41.11** · Obj 8.x · Easy · Single choice Lights emit in which local direction (same as cameras)? A.  $+Y$  · B.  $+Z$  · C.  $-Z$  · D.  $-X$

**Q41.12** · Obj 8.x · Medium · USDA reading What is the cone angle of /Lights/Key?

```
#usda 1.0

def RectLight "Key"
{
 float inputs:intensity = 10
}
```

A. 90 · B. 30 · C. Unauthored; no ShapingAPI, so not a cone light · D. 0.53

### Answers

**Q41.1 — C.** UsdLux is the light domain. Review: §41.1.

**Q41.2 — C.** Distant fallback is 50000. Review: §41.1, §41.3.

**Q41.3 — A, C.** Distant and Dome are NonboundableLightBase. Review: §41.1.

**Q41.4 — B.** There is no SpotLight type. Review: §41.2.

**Q41.5 — B.** Built-in applied schema. Review: §41.1–41.2.

**Q41.6 — C.**  $2^2 = 4$ . Review: §41.3.

**Q41.7 — B.** enableColorTemperature fallback is False. Review: §41.3.

**Q41.8 — B.** Since 21.02. Review: §41.1, §41.3.

**Q41.9 — B, C.** UsdLux and UsdRender import; imaging / Hd do not. Review: §41.4.

**Q41.10 — B.** usdview's default delegate. Review: §41.4.

**Q41.11 — C.** Local –Z. Review: §41.1.

**Q41.12 — C.** No ShapingAPI, so no cone. 90 is the *fallback if* ShapingAPI is applied; 0.53 is Distant angle. Review: §41.2.

### USDA exercise answers

**41-A — Intensity 1 on a DistantLight is ~50 000× below the schema fallback.**

Storm/usdview will look almost unlit next to a default sun. Author ~50000 or raise exposure (~15–16 stops to compensate — not practical). The prim type is correct; the number is wrong.

**41-B — Yes it is a spot (cone 25 via ShapingAPI), and yes it has LightAPI.** LightAPI is built into DiskLight; USDA does not list it. ShadowAPI is not applied.

**41-C — No.** enableColorTemperature is missing (fallback false), so 2700 K is unused. The bulb is white at intensity 5. Add `bool inputs:enableColorTemperature = 1`.



## Part IX — Debugging and Performance

### CHAPTER 42

## Chapter 42 — Stage Introspection and Composition Debugging

**Exam domain:** Debugging & Troubleshooting (11%) · **Objectives:** 6.2, 6.3, 6.4 · **Study day:** 12 · **Est. time:** 110 min **Prerequisites:** Ch 15 (layer stacks, edit targets), Ch 16–18 (references, payloads, variants), Ch 21 (LIVERPS), Ch 22.5–22.6 (composition stacks), Ch 33 (asset resolution), Ch 39–41 (what “looks wrong” means)

When a composed stage is wrong, the data is almost never gone. It is in a layer you are not looking at, on a path you did not type, behind a payload you did not load, or weaker than another opinion. This chapter is a **symptom playbook**: inspect the stack, find missing prims, see who won, repair broken arcs and asset paths, and explain visual bugs. Chapter 22 taught the APIs (`GetPrimStack`, `GetPropertyStack`, `PrimCompositionQuery`). Here you *use* them against the bugs the exam names in Obj 6.2, 6.3, and 6.4.

Diagnostics (`TfDebug`, `Trace`) wait until Chapter 43. Performance knobs wait until Chapter 44.

### Learning goals

- Print a layer stack, name the edit target, and mute a layer to test who is contributing.
- Distinguish invalid, inactive, undefined, unloaded, and instance-proxy prims.
- Read a property stack and resolve-info to explain why an opinion did not win (Obj 6.2).
- Diagnose missing references, missing `defaultPrim`, bad variant selections, and composition errors.
- Find unresolved asset paths with `ComputeAllDependencies` (Obj 6.3).
- Map unexpected pictures to purpose, visibility, extents, subdivision, units, and bindings (Obj 6.4).
- Follow a fixed debugging order instead of guessing.

### Key terms

Term	One-line definition
<b>Layer stack</b>	The session layer (optional) + root layer + sublayers, strongest first.
<b>Edit target</b>	The layer (or variant site) new <code>Usd</code> edits write into.
<b>Muted layer</b>	A layer the stage is told to ignore ( <code>MuteLayer</code> ).
<b>Invalid prim</b>	<code>GetPrimAtPath</code> returned a handle with <code>IsValid() == False</code> (no such path).
<b>Inactive prim</b>	Exists but <code>active = false</code> ; children are pruned from composition.
<b>Undefined prim</b>	An <code>over</code> with no <code>def</code> anywhere; <code>IsDefined()</code> is <code>False</code> .
<b>Unloaded prim</b>	A payload (or ancestor payload) not loaded; children are missing.
<b>Property stack</b>	The list of specs for one property, strongest first.
<b>Composition error</b>	A <code>Pcp</code> error captured on the stage ( <code>GetCompositionErrors()</code> ).
<b>Unresolved asset</b>	A path <code>ComputeAllDependencies</code> could not turn into a file.

## 42.1 Inspecting layers and layer stacks

**What is it?** A composed value is the winner among **layers** in a **layer stack**.

Introspection starts by listing those layers, the **edit target** (where your Python writes), and whether any layer is **muted**.

**Why do we need it?** If you do not know which files are in play, you will “fix” the asset while the shot’s stronger sublayer keeps winning, or you will author into the session layer and lose the edit on reopen. Obj 6.2 often starts with “which layer?”

**Beginner explanation.** Think of a stack of transparent sheets on a projector (Chapter 3). Debugging is flipping through the sheets: which sheet is on top (session), which is the cardboard backing (root), which overlays you slipped in (sublayers, first-listed stronger), and whether someone taped a “skip me” note on a sheet (mute).

*Where the analogy breaks:* a reference is not another sheet in *this* stack. It is a stack of its own, reached by an arc. `GetLayerStack()` does not list referenced files; `GetPrimStack()` / `GetUsedLayers()` do.

**Technical explanation.** - `stage.GetLayerStack(includeSessionLayers=True)` returns `[session, root, sublayer0, sublayer1, ...]` **strongest first**. `False` drops the session layer. The session layer’s display name is `<rootDisplay>-session.usda` (verified: `root.usda` → `root-session.usda`). - **Root is stronger than every sublayer.** First-listed sublayer is stronger than later ones. Session is stronger than the root. Same rule as Chapter 15. - `stage.GetEditTarget().GetLayer()` is where Usd APIs author. Default is the root layer. `SetEditTarget(stage.GetSessionLayer())` makes session edits: they win, and they vanish if you do not export the session layer. - `stage.MuteLayer(identifier)` / `UnmuteLayer` / `GetMutedLayers()` / `IsLayerMuted`. Identifier is the resolved layer id (use `os.path.abspath("b.usda")` for a file you opened relatively). Muting removes that layer’s opinions; composition recomputes. - `stage.GetUsedLayers()` is every layer the composed scene actually opened (root, session, sublayers, **and** referenced/payload layers). Broader than `GetLayerStack()`. - `GetDisplayName()` is the short name for logs; identifier may be an absolute path. Print display names in this book so output stays portable. - usdview’s **Layer Stack** tab is this list. `usd-core` has no usdview; Python is the stand-in.

### Mental model.

```
strongest session (includeSessionLayers=True)
 root (always; default edit target)
 sublayer[0] (first in subLayers = stronger)
 sublayer[1]
weakest ...

MuteLayer(id) → that sheet is pulled out of the projector
GetUsedLayers → this stack PLUS referenced / payload files
```

**Simple example.** Root lists sublayers `[b.usda, a.usda]`. `a` defs a Sphere radius 1; `b` overs radius 2. Composed radius is **2**. Mute `b` → **1**. Session sets `9` → **9**, winning layer `root-session.usda`.

**USDA example.** File: `a.usda`

```
#usda 1.0

def Xform "World"
{
 def Sphere "Ball"
 {
 double radius = 1
 }
}
```

File: `b.usda`

```
#usda 1.0
```

```

over "World"
{
 over "Ball"
 {
 double radius = 2
 }
}

```

*File: root.usda*

```

#usda 1.0
(
 subLayers = [
 @./b.usda@,
 @./a.usda@
]
)

```

- `b` is listed first, so it beats `a`. The root file itself authors nothing on `radius`.
- Session is not a file on disk until you export it.

### Python example.

```

import os
from pxr import Usd

files = {
 "a.usda": """#usda 1.0
def Xform "World"
{
 def Sphere "Ball"
 {
 double radius = 1
 }
}
""",
 "b.usda": """#usda 1.0
over "World"
{
 over "Ball"
 {
 double radius = 2
 }
}
""",
 "root.usda": """#usda 1.0
(
 subLayers = [
 @./b.usda@,
 @./a.usda@
]
)
""",
}
for name, text in files.items():
 with open(name, "w") as f:
 f.write(text)

stage = Usd.Stage.Open("root.usda")
print("edit target:", stage.GetEditTarget().GetLayer().GetDisplayName())
print("with session:")
for lyr in stage.GetLayerStack(includeSessionLayers=True):
 print(" ", lyr.GetDisplayName())
print("no session:")
for lyr in stage.GetLayerStack(includeSessionLayers=False):
 print(" ", lyr.GetDisplayName())
ball = stage.GetPrimAtPath("/World/Ball").GetAttribute("radius")
print("radius:", ball.Get())
ident_b = os.path.abspath("b.usda")
stage.MuteLayer(ident_b)
print("muted:", [os.path.basename(x) for x in stage.GetMutedLayers()])
print("radius muted b:", ball.Get())

```

```
stage.UnmuteLayer(ident_b)
print("radius unmuted:", ball.Get())
stage.SetEditTarget(stage.GetSessionLayer())
ball.Set(9)
print("radius session:", ball.Get())
print("winning layer:", ball.GetPropertyStack()[0].layer.GetDisplayName())
```

### Expected output

```
edit target: root.usda
with session:
 root-session.usda
 root.usda
 b.usda
 a.usda
no session:
 root.usda
 b.usda
 a.usda
radius: 2.0
muted: ['b.usda']
radius muted b: 1.0
radius unmuted: 2.0
radius session: 9.0
winning layer: root-session.usda
```

Muting is a *test*, not a fix. If muting `b` “repairs” the value, you found the layer that was winning.

**Real-world use case.** A lighting TD swears they set radius in `shot_light.usda`. You print the layer stack and see `shot_anim.usda` listed *before* it (stronger). Mute anim: the light value appears. The fix is a conversation about sublayer order, not another override in a weaker file.

### Common mistakes.

#### MISTAKE

Calling ``GetLayerStack()`` and wondering where the referenced asset is. Use ``GetUsedLayers()`` or ``prim.GetPrimStack()``.

#### MISTAKE

Authoring on the default edit target (root) when you meant a sublayer. Check ``GetEditTarget()`` before every debug ``Set``.

#### MISTAKE

Muting with the USDA token ``@./b.usda@`` as the id. ``MuteLayer`` wants the resolved identifier (``abspath`` of the opened file).

### Exam traps.

#### TRAP

"The first sublayer is weakest." First-listed sublayer is **\*\*strongest among sublayers\*\***. The root still beats all of them.

#### TRAP

Session layer is not in ``GetLayerStack()`` by default. Pass ``includeSessionLayers=True`` (and it is the default in 26.08 if you omit the flag — this book always passes it so the call is readable).

#### TRAP

Muting a layer deletes it from disk. Mute is per-stage and in-memory.

### Exam takeaways.

**KEY**

- Stack = session > root > sublayers (first-listed stronger). - Print ``GetLayerStack`` and ``GetEditTarget`` before chasing a value. - ``MuteLayer`` is a diagnostic: if the bug vanishes, you found the winning layer. - Referenced files are not in this stack.

## 42.2 Missing prims

**What is it?** A “missing prim” is several different states that look the same in a viewport: **invalid path**, **inactive**, **undefined over**, **unloaded payload**, or **instance proxy skipped by Traverse()**. You distinguish them with `IsValid`, `IsActive`, `IsDefined`, `IsLoaded`, `IsInstanceProxy`, and `Traverse` vs `TraverseAll`.

**Why do we need it?** Guessing “the reference is broken” when the prim is only inactive wastes an hour. Each state has a different fix. Obj 6.2 includes “the prim I authored is not on the stage.”

**Beginner explanation.** You look for a coworker at their desk. They might have the wrong office number (invalid path), be on leave (inactive), be a nameplate with no person yet (over / undefined), be in a locked archive (unloaded payload), or be a photocopied extra that the lobby directory does not list (Traverse skips instance proxies).

*Where the analogy breaks:* an inactive prim’s *children* become invalid handles, not merely inactive. You cannot walk into the locked office’s closet.

**Technical explanation.** - `GetPrimAtPath("/World/Nope")` **always returns a handle**. Ask `IsValid()`. False means no spec composed at that path. - **Inactive:** `prim.SetActive(False)` / metadata `active = false`. `IsValid()` True, `IsActive()` False, `IsDefined()` True. `Traverse()` skips it and its descendants. `TraverseAll()` includes it. Children: `GetPrimAtPath` on a child returns **invalid** (not composed). - **Undefined over:** specifier `Sdf.SpecifierOver`, `IsDefined()` False, empty type name. `Traverse()` skips it; `TraverseAll()` includes it. Usually a typo (over "Wrold"). - **Payload vs reference, LoadNone:** `Usd.Stage.Open(path, Usd.Stage.LoadNone)` **still loads references**. Payloads stay unloaded: `IsLoaded()` False, `GetTypeName()` '', children invalid. `stage.Load("/World/Lazy")` populates them. `Traverse()` skips unloaded payloads; `TraverseAll()` includes the payload root but not its unloaded children. - **Instancing:** `Traverse()` visits instance roots, not descendants. `GetPrimAtPath("/World/A/S")` is valid with `IsInstanceProxy()` True. Edits below the instance root do not stick (Chapter 24). - Also check: wrong defaultPrim (prim exists *inside* the asset at /Only, not at the referencing path); muted layer that held the def; composition error (next section).

### Mental model.

```
handle = GetPrimAtPath(p)
├── not IsValid → path never composed (typo, child of inactive,
│ unloaded payload child, muted def)
├── IsValid, not Active → deactivated; children invalid
├── IsValid, not Defined → stray over
├── IsValid, not Loaded → payload (or ancestor) unloaded
└── IsInstanceProxy → live view of a prototype; Traverse skipped it
```

**Simple example.** Stage opened with `LoadNone`. `/World/Loaded` (reference) is an Xform with `/Geo`. `/World/Lazy` (payload) is defined, unloaded, no `/Geo`. `/World/Nope` is inactive; `/World/Nope/Hidden` is invalid. `/World/TypoBall` is an undefined over.

### USDA example.

```
#usda 1.0

def Xform "World"
{
 def "Loaded" (
 prepend references = @./asset.usda@
```

```

)
 {
 }

 def "Lazy" (
 prepend payload = @./asset.usda@
)
 {
 }

 def "Nope" (
 active = false
)
 {
 def Sphere "Hidden"
 {
 }
 }

 over "TypoBall"
 {
 double radius = 4
 }
}

```

- `Loaded` vs `Lazy` is the reference/payload split. Same target file; different load rules.
- `TypoBall` has no matching `def` anywhere.

### Python example.

```

from pxr import Usd

asset = ""
#usda 1.0
(
 defaultPrim = "Prop"
)
def Xform "Prop"
{
 def Cube "Geo"
 {
 double size = 1
 }
}
"""
shot = ""
#usda 1.0
def Xform "World"
{
 def "Loaded" (
 prepend references = @./asset.usda@
)
 {
 }
 def "Lazy" (
 prepend payload = @./asset.usda@
)
 {
 }
 def "Nope" (
 active = false
)
 {
 def Sphere "Hidden" {}
 }
 over "TypoBall"
 {
 double radius = 4
 }
}
"""
open("asset.usda", "w").write(asset)
open("shot.usda", "w").write(shot)

```

```

st = Usd.Stage.Open("shot.usda", Usd.Stage.LoadNone)
print("missing valid:", st.GetPrimAtPath("/World/Missing").IsValid())
nope = st.GetPrimAtPath("/World/Nope")
print("Nope valid/active/defined:",
 nope.IsValid(), nope.IsActive(), nope.IsDefined())
hid = st.GetPrimAtPath("/World/Nope/Hidden")
print("Hidden valid:", hid.IsValid())
lazy = st.GetPrimAtPath("/World/Lazy")
print("Lazy loaded/type:", lazy.IsLoaded(), repr(lazy.GetTypeName()))
loaded = st.GetPrimAtPath("/World/Loaded")
print("Loaded loaded/type:", loaded.IsLoaded(), repr(loaded.GetTypeName()))
print("Loaded/Geo:", st.GetPrimAtPath("/World/Loaded/Geo").IsValid())
print("Lazy/Geo unloaded:", st.GetPrimAtPath("/World/Lazy/Geo").IsValid())
print("traverse:", [str(p.GetPath()) for p in st.Traverse()])
print("traverseAll:", [str(p.GetPath()) for p in st.TraverseAll()])
typo = st.GetPrimAtPath("/World/TypoBall")
print("TypoBall specifier:", typo.GetSpecifier(),
 "defined:", typo.IsDefined(), "type:", repr(typo.GetTypeName()))
st.Load("/World/Lazy")
print("Lazy after Load type:",
 repr(st.GetPrimAtPath("/World/Lazy").GetName()),
 "Geo:", st.GetPrimAtPath("/World/Lazy/Geo").IsValid())

```

### Expected output

```

missing valid: False
Nope valid/active/defined: True False True
Hidden valid: False
Lazy loaded/type: False ''
Loaded loaded/type: True 'Xform'
Loaded/Geo: True
Lazy/Geo unloaded: False
traverse: ['/World', '/World/Loaded', '/World/Loaded/Geo']
traverseAll: ['/World', '/World/Loaded', '/World/Loaded/Geo', '/World/Lazy', '/World/Nope', '/World/TypoBall']
TypoBall specifier: Sdf.SpecifierOver defined: False type: ''
Lazy after Load type: 'Xform' Geo: True

```

LoadNone did **not** unload the reference. Only the payload stayed empty.

**Real-world use case.** A shot opens with LoadNone for speed (Chapter 17, 44). Layout cannot see set dressing parented under payloads and files a “missing assets” ticket. TraverseAll still lists the payload roots; IsLoaded() is the tell. Loading that branch restores /Geo.

### Common mistakes.

#### MISTAKE

Treating `GetPrimAtPath` returning an object as "the prim exists." Always `IsValid()`.

#### MISTAKE

Opening with `LoadNone` to "load nothing" and panicking that references vanished. They did not.

#### MISTAKE

Using `Traverse()` to hunt instance internals or inactive prims. Use `GetPrimAtPath` + `TraverseAll`.

### Exam traps.

#### TRAP

Child of an inactive prim is inactive. It is **\*\*invalid\*\*** (not composed).

#### TRAP

`over "Ball"` with no `def` is a composed prim you can `GetAttribute` on — but `IsDefined()` is False and `Traverse()` skips it.

**TRAP**

``IsInstanceProxy()`` True means the prim is missing. It is present; ``Traverse()`` just does not yield it.

**Exam takeaways.****KEY**

- Valid / active / defined / loaded / proxy are five different questions. - Inactive children are invalid. Undefined overs are ``SpecifierOver``. - ``LoadNone`` still loads references. Payloads need ``Load``. - ``Traverse`` is a filtered walk; ``TraverseAll`` + ``GetPrimAtPath`` for debugging.

## 42.3 Wrong opinions winning

**What is it?** The prim is there, the attribute exists, and the value is still “wrong.” A **stronger opinion** won: stronger layer, stronger LIVERPS arc, a **value block**, or **time samples vs Default**. `GetPropertyStack()` lists every spec strongest-first; `GetResolveInfo()` names the kind of value that won (Obj 6.2).

**Why do we need it?** Without the stack you will add a fourth override instead of deleting or moving the one that already wins. Chapter 22.6 is the ten-point checklist; this section is the measurement.

**Beginner explanation.** Several people shout a number. The property stack is the attendance list in speaking order. The first name is who the stage heard. Resolve-info is *what kind* of sentence they used: a default, a time sample, a clip, a block (“nobody speak”).

*Where the analogy breaks:* a quieter person (weaker spec) is still on the list. You can read their number; it just does not compose.

**Technical explanation.** - `attr.GetPropertyStack()` → `[Sdf.PropertySpec, ...]`, strongest first. Each spec has `.layer`, `.path`, `.default`. The winning layer is `[0].layer`. - Strength inside one stack: **session > root > earlier sublayers > later sublayers** for *local* specs, then weaker **arcs** (references, payloads, specializes) unless a stronger arc (inherits, variant, relocates) applies — LIVERPS, Chapter 21. - `attr.GetResolveInfo(time)`: `GetSource()` is `Usd.ResolveInfoSourceDefault`, `TimeSamples`, `ValueClips`, `Spline`, `Fallback`, or `None`. `ValueIsBlocked()` is True when a stronger spec authored `None` (a **block**). `Get()` then returns `None`. - **Default vs samples:** `Get()` with no time uses `Usd.TimeCode.Default()`. An authored default can still come back at Default even when samples exist. `Get(1)` at a numeric time uses samples (`ResolveInfoSourceTimeSamples`). Viewports usually play numeric time — `Get()` in a debug print can disagree with the picture. - `GetPrimStack()` is the same idea for the prim itself (who contributed specs, including empty overs). - Full arc list: `Usd.PrimCompositionQuery(prim).GetCompositionArcs()` (Chapter 22.5). Filter with `GetDirectReferences(prim)` when you only care about references.

**VERSION**

Verified on USD 26.08. There is no ``ResolveInfo`` method that returns a layer; use ``GetPropertyStack()[0].layer``. ``GetMaster`` does not exist; instancing uses ``GetPrototype``.

**Mental model.**

`GetPropertyStack()[0]` = who won (layer + path + default)  
`GetResolveInfo().source` = how they won (default / samples / clips / block)

If your spec is on the list but not [0], you lost a strength fight.  
 If your spec is not on the list, you authored the wrong path or layer.



**Simple example.** Root local `height = 1`, stronger-than-reference lighting sublayer `height = 2`, referenced asset `height = 0.5`. Stack: shot (1), light (2), asset (0.5). Winner: **1** (root beats sublayers *and* the reference). Moon has default 9 and samples 1 / 4: `Get()` is 9, `Get(1)` is 1. Blocked sphere: `Get()` is None, `ValueIsBlocked` True.

**USDA example.** File: *shot.usda* (root) plus a lighting sublayer and a referenced asset:

```
#usda 1.0
(
 subLayers = [
 @./light.usda@
]
)

def Xform "World"
{
 def "Lamp_1" (
 prepend references = @./refsrc.usda@
)
 {
 double height = 1
 }

 def Sphere "Moon"
 {
 double radius = 9
 double radius.timeSamples = {
 1: 1,
 24: 4,
 }
 }

 def Sphere "Blocked"
 {
 double radius = None
 }
}
```

- Root local `1` is stronger than `light.usda`'s `2` and the reference's `0.5`.
- `radius = None` is a block, not "radius zero."

### Python example.

```
from pxr import Usd

open("refsrc.usda", "w").write("""#usda 1.0
(
 defaultPrim = "Lamp"
)
def Xform "Lamp"
{
 double height = 0.5
}
""")
open("light.usda", "w").write("""#usda 1.0
over "World"
{
 over "Lamp_1"
 {
 double height = 2
 }
}
""")
open("shot.usda", "w").write("""#usda 1.0
(
 subLayers = [
 @./light.usda@
]
)
def Xform "World"
{
```

```

def "Lamp_1" (
 prepend references = @./refsrc.usda@
)
{
 double height = 1
}
def Sphere "Moon"
{
 double radius = 9
 double radius.timeSamples = {
 1: 1,
 24: 4,
 }
}
def Sphere "Blocked"
{
 double radius = None
}
}
"""

so = Usd.Stage.Open("shot.usda")
h = so.GetPrimAtPath("/World/Lamp_1").GetAttribute("height")
print("height:", h.Get())
print("property stack:")
for spec in h.GetPropertyStack():
 print(" ", spec.layer.GetDisplayName(), spec.path, spec.default)
print("resolve source:", h.GetResolveInfo().GetSource())
print("winning layer:", h.GetPropertyStack()[0].layer.GetDisplayName())
moon = so.GetPrimAtPath("/World/Moon").GetAttribute("radius")
print("moon Get():", moon.Get())
print("moon Get(1):", moon.Get(1),
 "src", moon.GetResolveInfo(1).GetSource())
print("moon Default:", moon.Get(Usd.TimeCode.Default()),
 "src", moon.GetResolveInfo(Usd.TimeCode.Default()).GetSource())
blk = so.GetPrimAtPath("/World/Blocked").GetAttribute("radius")
print("blocked Get:", blk.Get(),
 "ValueIsBlocked", blk.GetResolveInfo().ValueIsBlocked())

```

### Expected output

```

height: 1.0
property stack:
 shot.usda /World/Lamp_1.height 1.0
 light.usda /World/Lamp_1.height 2.0
 refsrc.usda /Lamp.height 0.5
resolve source: Usd.ResolveInfoSourceDefault
winning layer: shot.usda
moon Get(): 9.0
moon Get(1): 1.0 src Usd.ResolveInfoSourceTimeSamples
moon Default: 9.0 src Usd.ResolveInfoSourceDefault
blocked Get: None ValueIsBlocked True

```

If you expected lighting's 2 to win, you swapped root vs sublayer strength. Root local wins.

**Real-world use case.** Anim publishes `radius` samples on a ball. A debug script prints `Get()` and reports 9 all day while usdview at frame 1 shows 1. The TD “fixes” the default, which still never plays. Reading `GetResolveInfo(1).GetSource()` would have said `TimeSamples` in one call.

### Common mistakes.

#### MISTAKE

Comparing ``Get()`` to the viewport without passing the viewport's time code.

#### MISTAKE

Deleting the referenced asset's value because you see it on the stack. It is weaker; leave the asset, change the winner or remove the stronger spec.

**MISTAKE**

Reading ``radius = None`` as "use the fallback 1." It **\*\*blocks\*\***; weaker radii never apply.

**Exam traps.****TRAP**

"A sublayer always beats the root." The root beats sublayers. First-listed sublayer beats later sublayers.

**TRAP**

Local reference opinion vs sublayer: both can be "local" in LIVERPS if they sit in the root layer stack. Order in the stack decides.

**TRAP**

``Get()`` ignoring samples "because samples always win." At ``TimeCode.Default()``, an authored default still resolves as ``SourceDefault`` (verified above).

**Exam takeaways.****KEY**

- Obj 6.2: ``GetPropertyStack()[0]`` is the winner; the rest lost. - Root > sublayers > referenced opinions (unless a stronger arc). - Blocks (``None``) are winners that contribute no number. - Pass the same time code the viewport uses.

## 42.4 Broken references, payloads, variants

**What is it?** Arcs fail in three common ways: **the file cannot be opened**

(`InvalidAssetPath`), **the file opens but the prim path is missing** (`UnresolvedPrimPath`, often no `defaultPrim`), or **the variant selection names a set member that does not exist** so the variant opinions never apply.

**Why do we need it?** A referencing prim still *exists* (`IsDefined()` True) when the asset is missing — it is just an empty Xform with no type. The viewport looks like a missing model. `GetCompositionErrors()` is the difference between "USD ate my mesh" and "typo in the path" (Obj 6.2 / 6.3).

**Beginner explanation.** A library request can fail because the building is gone (missing file), the building is there but the catalog has no default shelf (`defaultPrim` missing and you did not pass `</Only>`), or you asked for the "tiny" edition of a book that only comes in small/big (bad variant selection).

*Where the analogy breaks:* USD still creates the referencing prim. You get an empty shelf label, not a crash.

**Technical explanation.** - `stage.GetCompositionErrors()` → Pcp errors. Opening the stage also prints them as **warnings** on stderr. Types you will see: -

`Pcp.ErrorType_InvalidAssetPath` — file not found / unreadable (`@./missing.usda@`). -

`Pcp.ErrorType_UnresolvedPrimPath` — file found, target prim not (message says `<defaultPrim>` when the reference omitted a prim path and the layer has no default). - A failed reference: `prim.IsDefined()` True, `GetTypeName()` '', no children from the asset. -

**Fix for no defaultPrim:** add ( `defaultPrim = "Only"` ) on the asset, or reference `@./nodef.usda@</Only>` with an explicit prim path (Chapter 16). - **Variants:**

`prim.GetVariantSets().HasVariantSet("size")` can be True (the set exists on the asset) while `GetVariantSelection("size")` is "tiny" and no "tiny" block exists. Attributes inside the real variants (h) then do not compose. Fallbacks (Chapter 38) only apply when **no** selection is authored. An authored wrong name blocks the fallback. - Payloads use the same error types when the file is missing; when the file is fine but **unloaded**, there is

often **no** composition error — go back to §42.2 `IsLoaded()`. -

`Usd.PrimCompositionQuery.GetDirectReferences(prim)` lists reference arcs that *did* bind, with `GetTargetLayer()`.

### Mental model.

```
empty referencing prim
├─ GetCompositionErrors()
│ ├── InvalidAssetPath → fix the @path@ (typo, cwd, resolver)
│ └── UnresolvedPrimPath → defaultPrim or explicit </Prim>
└─ no error, IsLoaded False → payload not loaded
variant set exists, attribute missing
└─ selection string not in the set (authored "tiny")
```

**Simple example.** `BadRef` → missing file → `InvalidAssetPath`, type `''`. `NoDefault` → `nodef.usda` has `def Sphere "Only"` but no `defaultPrim` → `UnresolvedPrimPath`, `/NoDefault/Only` invalid. Same file with `@./nodef.usda@</Only>` becomes a `Sphere`. `Var` selects `tiny`; `h` is missing.

### USDA example.

```
#usda 1.0

def Xform "World"
{
 def "BadRef" (
 prepend references = @./missing.usda@
)
 {
 }

 def "NoDefault" (
 prepend references = @./nodef.usda@
)
 {
 }

 def "Var" (
 prepend references = @./var.usda@
 variants = {
 string size = "tiny"
 }
)
 {
 }
}
```

- All three prims compose as empty containers until you fix the arc or the selection.
- `var.usda` (not shown) has `defaultPrim` and variant names `small` / `big` only.

### Python example.

```
from pxr import Usd

open("nodef.usda", "w").write("""#usda 1.0
def Sphere "Only"
{
 double radius = 1
}
""")
open("var.usda", "w").write("""#usda 1.0
(
 defaultPrim = "P"
)
def Xform "P" (
 prepend variantSets = "size"
)
{
 variantSet "size" = {
 "small" {
 double h = 1
 }
 }
}
```

```

 }
 "big" {
 double h = 3
 }
}
"""
open("shot.usda", "w").write("""#usda 1.0
def Xform "World"
{
 def "BadRef" (
 prepend references = @./missing.usda@
)
 {
 }
 def "NoDefault" (
 prepend references = @./nodef.usda@
)
 {
 }
 def "Var" (
 prepend references = @./var.usda@
 variants = {
 string size = "tiny"
 }
)
 {
 }
}
"""
open("ok.usda", "w").write("""#usda 1.0
def "X" (
 prepend references = @./nodef.usda@</Only>
)
{
}
"""

sb = Usd.Stage.Open("shot.usda")
print("error types:")
for err in sb.GetCompositionErrors():
 print(" ", err.errorType)
bad = sb.GetPrimAtPath("/World/BadRef")
print("BadRef defined/type:", bad.IsDefined(), repr(bad.GetTypeName()))
print("NoDefault/Only:", sb.GetPrimAtPath("/World/NoDefault/Only").IsValid())
var = sb.GetPrimAtPath("/World/Var")
print("selection:", var.GetVariantSets().GetVariantSelection("size"))
print("has h:", var.HasAttribute("h"), "Get h:", var.GetAttribute("h").Get())
ok = Usd.Stage.Open("ok.usda")
print("explicit path type:", ok.GetPrimAtPath("/X").GetTypeName(),
 "radius:", ok.GetPrimAtPath("/X").GetAttribute("radius").Get())

```

### Expected output

```

error types:
 Pcp.ErrorType_UnresolvedPrimPath
 Pcp.ErrorType_InvalidAssetPath
BadRef defined/type: True ''
NoDefault/Only: False
selection: tiny
has h: False Get h: None
explicit path type: Sphere radius: 1.0

```

Stderr also warns while the stage opens; the exam-facing API is `GetCompositionErrors()`.

**Real-world use case.** A set-dressing USD references `chair.usda` without a prim path. The chair file's `defaultPrim` was removed in a “cleanup.” Every chair in the shot becomes an empty Xform. `UnresolvedPrimPath ... <defaultPrim>` is the signature. Either restore `defaultPrim` or switch to `@chair.usda@</Chair>`.

**Common mistakes.**

**MISTAKE**

Checking only `IsValid()` on the referencing prim. It is valid. Check `**type`, `children`, and `composition errors**`.

**MISTAKE**

Authoring `variants = { string size = "tiny" }` to "clear" a selection. That `*is*` a selection; it will not fall back to `small`.

**MISTAKE**

Treating an unloaded payload as `InvalidAssetPath`. No error; `IsLoaded()` is `False`.

**Exam traps.****TRAP**

Missing `defaultPrim` is `InvalidAssetPath`. It is `**UnresolvedPrimPath**` (file opened).  
Missing `*file*` is `InvalidAssetPath`.

**TRAP**

"Empty type name means the prim is invalid." Defined, valid, untyped.

**TRAP**

Variant fallbacks rescue an authored wrong name. Fallbacks apply only when the selection is `**missing**`.

**Exam takeaways.****KEY**

- Failed arcs leave an empty but valid prim. Read `GetCompositionErrors()`. - Missing file  $\neq$  missing `defaultPrim` (`InvalidAssetPath` vs `UnresolvedPrimPath`). - Explicit `@file@` bypasses `defaultPrim`. - Wrong authored variant name is not a missing selection; fallbacks will not fire.

## 42.5 Asset-path problems

**What is it?** Even when composition succeeds, **asset-valued attributes** (`inputs:file`, `assetInfo`, `media`) can point at files the resolver cannot find. Obj 6.3: resolve issues related to asset management. The pipeline tools are `UsdUtils.ComputeAllDependencies`, `ExtractExternalReferences`, and anchored `ComputeAbsolutePath` (Chapter 33).

**Why do we need it?** `usdview` shows a grey `PreviewSurface` because `wood.png` never resolved; Hydra does not always scream. A packaging job that only copies `.usda` leaves textures behind. Computing dependencies *before* publish is the debug step that belongs in the exporter (Chapter 32).

**Beginner explanation.** A recipe says "add the packet marked `@./tex.png@`." Composition found the recipe book. Asset resolution walks to the pantry. `ComputeAllDependencies` is the clipboard of everything the recipe needs; **unresolved** is the list of packets that were not on the shelf.

*Where the analogy breaks:* `Ar.GetResolver().Resolve("./tex.png")` uses the **process current directory**, not the layer that authored the path. The pantry key is the layer's location (`ComputeAbsolutePath / CreateIdentifier`), not `cwd` — except in these scripts, `cwd` is the layer directory, which hides the bug.

**Technical explanation.** - `UsdUtils.ComputeAllDependencies(layerPath) → (layers, assets, unresolved)` **recursive**. Layers are `Sdf.Layer` handles; assets are resolved file paths; unresolved are identifier strings that failed. - `UsdUtils.ExtractExternalReferences(layerPath) → (sublayers, refs_and_assets,`

payloads) for **one file**, no recursion. Useful to see what *this* layer wrote, including a broken `@missing.png@`. - `Sdf.Layer.ComputeAbsolutePath("./tex.png")` anchors to that layer. Naive `Ar.GetResolver().Resolve("./tex.png")` is `cwd` (Chapter 33). A path **without** `./` is context-dependent: it may resolve only with a resolver context. - After composition, `attribute.Get()` on an asset input returns `Sdf.AssetPath.str(val)` looks like `@./tex.png@`. `val.resolvedPath` is empty when resolution failed. - Common breaks: wrong relative path (`@textures/wood.png@` vs `@./wood.png@`), published USDA moved without its sidecar files, resolver search path not set, Windows vs POSIX separators, URI the default resolver does not handle.

#### VERSION

Verified on USD 26.08. ``Ar.AnchorRelativePath`` is gone; use ``CreateIdentifier`` + ``ComputeAbsolutePath``.

#### Mental model.

```
ComputeAllDependencies(root)
 layers = every USD file pulled in
 assets = textures / audio / ... that resolved
 unresolved = the bug list ← start here
```

`ExtractExternalReferences(one layer)` = what that file spelled, even if broken

**Simple example.** `shot.usda` references `look.usda`, which has asset inputs: `file = @./tex.png@` next to a real `tex.png`. Dependencies: layers `look+shot`, assets `tex.png`, unresolved empty. `look2.usda` points at `missing.png`: unresolved `['missing.png']`. `Resolve("missing.png")` is `False`.

#### USDA example.

```
#usda 1.0
(
 defaultPrim = "Wood"
)

def Material "Wood"
{
 def Shader "Tex"
 {
 uniform token info:id = "UsdUVTexture"
 asset inputs:file = @missing.png@
 }
}
```

- Composition of this Material succeeds. The *texture* does not.
- `@missing.png@` has no `./`; default resolver will not treat it as next-to-the-layer unless a context says so.

#### Python example.

```
import os
from pxr import Usd, Sdf, UsdUtils, Ar

open("tex.png", "w").write("x")
open("look.usda", "w").write("""#usda 1.0
(
 defaultPrim = "Wood"
)

def Material "Wood"
{
 def Shader "Tex"
 {
 uniform token info:id = "UsdUVTexture"
 asset inputs:file = @./tex.png@
 }
}
""")
```

```

open("look2.usda", "w").write("""#usda 1.0
(
 defaultPrim = "Wood"
)
def Material "Wood"
{
 def Shader "Tex"
 {
 uniform token info:id = "UsdUVTexture"
 asset inputs:file = @missing.png@
 }
}
""")
open("shot.usda", "w").write("""#usda 1.0
def "Looks" (
 prepend references = @./look.usda@
)
{
}
""")

layers, assets, unresolved = UsdUtils.ComputeAllDependencies("shot.usda")
print("layers:", sorted(os.path.basename(l.identifier) for l in layers))
print("assets:", sorted(os.path.basename(a) for a in assets))
print("unresolved:", list(unresolved))
_, _, u2 = UsdUtils.ComputeAllDependencies("look2.usda")
print("look2 unresolved:", list(u2))
print("extract look2:", UsdUtils.ExtractExternalReferences("look2.usda"))
r = Ar.GetResolver()
print("missing.png resolved:", bool(r.Resolve("missing.png")))
print("./tex.png resolved:", bool(r.Resolve("./tex.png")))
layer = Sdf.Layer.FindOrOpen("look.usda")
print("anchor ./tex.png:",
 os.path.basename(layer.ComputeAbsolutePath("./tex.png")))
st = Usd.Stage.Open("shot.usda")
val = st.GetPrimAtPath("/Looks/Tex").GetAttribute("inputs:file").Get()
print("authored file:", val)
print("resolvedPath nonempty:", bool(str(val.resolvedPath)))

```

### Expected output

```

layers: ['look.usda', 'shot.usda']
assets: ['tex.png']
unresolved: []
look2 unresolved: ['missing.png']
extract look2: ([], ['missing.png'], [])
missing.png resolved: False
./tex.png resolved: True
anchor ./tex.png: tex.png
authored file: @./tex.png@
resolvedPath nonempty: True

```

`ExtractExternalReferences` lists `missing.png` even though it does not resolve; `ComputeAllDependencies` puts it in **unresolved**.

**Real-world use case.** A lookdev package ships `wood.usda` + `wood.png`. Someone copies only the USDA into the shot tree. Storm shows grey. `ComputeAllDependencies` on the published layer lists `wood.png` under unresolved. The fix is packaging (Chapter 31 USDZ / `LocalizeAsset`), not a new shader.

### Common mistakes.

#### MISTAKE

Calling ``Resolve("./tex.png")`` from a unittest whose cwd is not the layer directory, then "fixing" the USDA. Anchor with ``ComputeAbsolutePath`` on the layer.

#### MISTAKE

Treating ``ExtractExternalReferences`` as recursive. It is one file.



**MISTAKE**

Checking only composition errors for a missing texture. Textures are not composition arcs.

**Exam traps.****TRAP**

``ComputeAllDependencies`` third tuple element is "all asset paths." It is **unresolved** only. Successful textures are in the second element.

**TRAP**

``@./x@`` and ``@x@`` always resolve the same. Only ``./`` (or an absolute path) is clearly file-relative for the default resolver.

**TRAP**

Empty ``resolvedPath`` means the attribute is unauthored. It can be authored **and** unresolved.

**Exam takeaways.****KEY**

- Obj 6.3: ``ComputeAllDependencies`` → ``(layers, assets, unresolved)``. - ``ExtractExternalReferences`` is one-file and includes broken paths. - Anchor relatives to the **layer**, not cwd (``ComputeAbsolutePath``). - Missing textures rarely show up as composition errors.

## 42.6 Unexpected visual results

**What is it?** The stage is composed, assets resolve, and the picture is still wrong. Obj 6.4: **unexpected visual results** come from geometry, imaging purpose, units, materials, and lights — not from LIVERPS. You measure them with `UsdGeom` / `UsdShade` / `UsdLux` queries.

**Why do we need it?** A “broken reference” ticket that is actually `subdivisionScheme = catmullClark` on a CAD mesh, or `purpose = proxy` on the set, trains you to pick the wrong API. The exam mixes composition stems with visual stems; this section is the visual half.

**Beginner explanation.** You developed the photograph (composition) and the print looks odd. Maybe the paper is the wrong size (units / `upAxis`), the subject is marked “lobby snapshot only” (purpose), the statue is still in the crate (extent / visibility), the clay was never painted (material bind), or the sculptor left the smoothing on (subdivision).

*Where the analogy breaks:* Hydra may skip drawing without a USD error.

`GetCompositionErrors()` can be empty while the mesh is invisible.

**Technical explanation.** Checklist (each item is a prior chapter; here you only *query*):

Symptom	Query	Typical cause
Melted / rounded CAD	<code>UsdGeom.Mesh.GetSubdivisionSchemeAttr().Get()</code>	Unauthored → fallback <b>catmullClark</b> . Set "none". Ch 39.
Flickers / disappears when framed	<code>extent HasAuthoredValue()</code>	Missing or stale extent. Ch 13.
Missing in beauty, present in preview	<code>Imageable.ComputePurpose()</code>	Inherited proxy / guide. Ch 39.6.
	<code>ComputeVisibility()</code>	

Symptom	Query	Typical cause
Whole group gone		Ancestor invisible. Token "visible" is invalid; authored values are inherited / invisible.
Grey clay	HasAPI(MaterialBindingAPI), ComputeBoundMaterial()	No bind, or PreviewSurface not connected. Ch 40.
Wrong scale	UsdGeom.GetStageMetersPerUnit	0.01 vs 1. Ch 28.
On its side	UsdGeom.GetStageUpAxis	Y vs Z. Ch 28.
Wrong color	primvar displayColor vs shader diffuseColor leftover .connect	Ch 40.2 / 40.5.
Transform ignored	op not in xformOp0Order; instance proxy	Ch 39.1, 24.
Dim scene	DistantLight intensity 1 (fallback is 50000)	Ch 41.

ComputePurpose() inherits. A child with no authored purpose under purpose = "proxy" computes **proxy** — beauty renderers skip it.

ComputeVisibility() on a drawing prim often returns **inherited**, which still draws. **invisible** means skipped. Do not expect the string "visible".

usd-core cannot rasterize; these queries are what you can prove without usdview.

### Mental model.

```
empty GetCompositionErrors()
├─ purpose proxy/guide → not a composition bug
├─ visibility invisible → ancestor or self
├─ scheme catmullClark → CAD looks soft
├─ no extent → culling / selection weirdness
├─ no bound material → grey
└─ units / upAxis → size and orientation
```

**Simple example.** Y-up studio file referenced into a Z-up shot: upAxis Z, metersPerUnit 1. Mesh Soft has no extent, scheme catmullClark, inherits purpose proxy, visibility inherited, no MaterialBindingAPI. Mesh Hidden is invisible with an authored extent.

### USDA example.

```
#usda 1.0
(
 upAxis = "Z"
 metersPerUnit = 1
)

def Xform "Set"
{
 uniform token purpose = "proxy"

 def Mesh "Soft"
 {
 int[] faceVertexCounts = [3]
 int[] faceVertexIndices = [0, 1, 2]
 point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
 token visibility = "inherited"
 }

 def Mesh "Hidden"
 {
 token visibility = "invisible"
 int[] faceVertexCounts = [3]
 int[] faceVertexIndices = [0, 1, 2]
 }
}
```

```

 point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
 float3[] extent = [(-1, -1, -1), (1, 1, 1)]
 }
}

```

- `Soft` will not appear in a render-purpose viewport: computed purpose is `proxy`.
- Unauthored `subdivisionScheme` is still `catmullClark` when you `Get()`.

### Python example.

```

from pxr import Usd, UsdGeom, UsdShade

open("vis.usda", "w").write("""#usda 1.0
(
 upAxis = "Z"
 metersPerUnit = 1
)

def Xform "Set"
{
 uniform token purpose = "proxy"

 def Mesh "Soft"
 {
 int[] faceVertexCounts = [3]
 int[] faceVertexIndices = [0, 1, 2]
 point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
 token visibility = "inherited"
 }

 def Mesh "Hidden"
 {
 token visibility = "invisible"
 int[] faceVertexCounts = [3]
 int[] faceVertexIndices = [0, 1, 2]
 point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
 float3[] extent = [(-1, -1, -1), (1, 1, 1)]
 }
}
""")

sv = Usd.Stage.Open("vis.usda")
print("upAxis:", UsdGeom.GetStageUpAxis(sv))
print("metersPerUnit:", UsdGeom.GetStageMetersPerUnit(sv))
soft = UsdGeom.Mesh(sv.GetPrimAtPath("/Set/Soft"))
print("scheme:", soft.GetSubdivisionSchemeAttr().Get())
print("extent authored:", soft.GetExtentAttr().HasAuthoredValue())
print("purpose Set:",
 UsdGeom.Imageable(sv.GetPrimAtPath("/Set")).GetPurposeAttr().Get())
print("ComputePurpose Soft:",
 UsdGeom.Imageable(soft.GetPrim()).ComputePurpose())
print("ComputeVisibility Soft:",
 UsdGeom.Imageable(soft.GetPrim()).ComputeVisibility())
print("ComputeVisibility Hidden:",
 UsdGeom.Imageable(sv.GetPrimAtPath("/Set/Hidden")).ComputeVisibility())
print("HasAPI MaterialBindingAPI:",
 soft.GetPrim().HasAPI(UsdShade.MaterialBindingAPI))
print("bound valid:",
 bool(UsdShade.MaterialBindingAPI(soft.GetPrim()).ComputeBoundMaterial()[0]))

```

### Expected output

```

upAxis: Z
metersPerUnit: 1.0
scheme: catmullClark
extent authored: False
purpose Set: proxy
ComputePurpose Soft: proxy
ComputeVisibility Soft: inherited
ComputeVisibility Hidden: invisible
HasAPI MaterialBindingAPI: False
bound valid: False

```

Five visual bugs, zero composition errors.

**Real-world use case.** A CAD-to-USD exporter (Chapter 29) forgets `subdivisionScheme = none` and `extent`. Storm shows a melted bracket that vanishes when the camera dollies. Composition stacks look perfect. The visual checklist catches both in two `Get()` calls.

### Common mistakes.

#### MISTAKE

Debugging a proxy-purpose asset in a render-purpose viewport. Compute purpose before recomposing.

#### MISTAKE

Looking for ``visibility = "visible"``. The drawing value is `**`inherited`**`.

#### MISTAKE

Blaming LIVERPS for DistantLight intensity 1. That is a UsdLux fallback (Chapter 41), not a weaker layer.

### Exam traps.

#### TRAP

Unauthored ``subdivisionScheme`` means no subdivision. Fallback is `**catmullClark**`.

#### TRAP

Imageable ``purpose`` (``proxy``) vs material ``purpose`` (``preview``). Different schemas (Ch 39 vs 40).

#### TRAP

``ComputeBoundMaterial`` invalid means the Material prim is missing. It often means `**no bind**` on this mesh.

### Exam takeaways.

#### KEY

- Obj 6.4: empty composition errors do not mean the picture is right. - Query purpose, visibility, scheme, extent, bind, metersPerUnit, upAxis. - ``inherited`` visibility draws; ``proxy`` purpose often does not (in beauty). - CAD + default subdivision is the classic "melted mesh."

## 42.7 A systematic debugging procedure

**What is it?** A **fixed order** of questions so you do not skip from "grey mesh" to rewriting the asset. Each step is a section of this chapter. Stop at the first failed check; that is the bug class.

**Why do we need it?** Random `usdcat` and extra `overs` create new bugs. Obj 6.2–6.4 span composition, assets, and imaging. One procedure covers all three without mixing the tools.

**Beginner explanation.** A medic does not start with surgery. Airway, breathing, circulation — then the interesting injury. USD debugging is: **can I find the prim, is it live, did arcs work, who won the value, do files exist, why is the picture wrong?**

*Where the analogy breaks:* later steps can still be wrong after an early pass. If you fix a missing payload and the mesh is still melted, continue down the list.

**Technical explanation. The eight-step order (verified APIs):**

1. **Layer stack and edit target.** `GetLayerStack(includeSessionLayers=True)`, `GetEditTarget()`, `GetMutedLayers()`. Are you looking at / writing the layer you think?
2. **Prim identity.** `GetPrimAtPath` → `IsValid`, `IsActive`, `IsDefined`, `IsLoaded`, `IsInstanceProxy`. `Traverse` vs `TraverseAll`.
3. **Composition errors.** `GetCompositionErrors()`. `InvalidAssetPath` vs `UnresolvedPrimPath` vs `none`.
4. **Arcs.** `PrimCompositionQuery` / `GetDirectReferences`. Variant `GetVariantSelection`. Payload `IsLoaded`.
5. **Opinion winner.** `GetPropertyStack()`, `GetResolveInfo(time)` with the **viewport time**. Check blocks and `Default` vs `samples`.
6. **Asset paths.** `ComputeAllDependencies` unresolved list; `resolvedPath` on asset attributes.
7. **Visual queries.** Purpose, visibility, subdivisionScheme, extent, material bind, `metersPerUnit`, `upAxis`, light intensity.
8. **Only then** flatten / export a snapshot (`Flatten`, Chapter 34) if you need a single-file repro. Flattening is a diagnostic copy, not a fix.

Write the answers down. The exam loves stems that skip a step (e.g. flatten first, which bakes the wrong winner).

**Mental model.**

```
1 stack/target → 2 prim flags → 3 Pcp errors → 4 arcs/variants
 → 5 property stack + time → 6 unresolved assets → 7 visual
 → 8 freeze a flatten only for a bug file
```

Never start at 8. Never skip 2 because `Traverse()` “didn’t list it.”

**Simple example.** Grey proxy set from §42.6. Steps 1–6 pass (no errors, prims valid). Step 7: `ComputePurpose` is proxy, no bind, scheme `catmullClark`. Three visual findings, zero composition work.

**USDA example.** The same `vis.usda` as §42.6. The procedure does not need a new file format; it needs discipline on that file.

```
#usda 1.0
(
 upAxis = "Z"
 metersPerUnit = 1
)

def Xform "Set"
{
 uniform token purpose = "proxy"

 def Mesh "Soft"
 {
 int[] faceVertexCounts = [3]
 int[] faceVertexIndices = [0, 1, 2]
 point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
 }
}
```

**Python example.**

```
from pxr import Usd, UsdGeom, UsdShade

open("vis.usda", "w").write("""#usda 1.0
(
 upAxis = "Z"
 metersPerUnit = 1
)
```

```
def Xform "Set"
{
 uniform token purpose = "proxy"

 def Mesh "Soft"
 {
 int[] faceVertexCounts = [3]
 int[] faceVertexIndices = [0, 1, 2]
 point3f[] points = [(0, 0, 0), (1, 0, 0), (0, 1, 0)]
 token visibility = "inherited"
 }
}
"""
sv = Usd.Stage.Open("vis.usda")
prim = sv.GetPrimAtPath("/Set/Soft")
print("1 edit target:", sv.GetEditTarget().GetLayer().GetDisplayName())
print("1 muted:", sv.GetMutedLayers())
print("2 valid/active/defined/loaded:",
 prim.IsValid(), prim.IsActive(), prim.IsDefined(), prim.IsLoaded())
print("3 errors:", len(sv.GetCompositionErrors()))
print("4 variant sets:", prim.GetVariantSets().GetNames())
print("5 has points:", prim.GetAttribute("points").IsValid())
print("6 unresolved skip (no assets in this file)")
img = UsdGeom.Imageable(prim)
mesh = UsdGeom.Mesh(prim)
print("7 purpose:", img.ComputePurpose())
print("7 visibility:", img.ComputeVisibility())
print("7 scheme:", mesh.GetSubdivisionSchemeAttr().Get())
print("7 extent authored:", mesh.GetExtentAttr().HasAuthoredValue())
print("7 bound:",
 bool(UsdShade.MaterialBindingAPI(prim).ComputeBoundMaterial()[0]))
print("7 upAxis/meters:", UsdGeom.GetStageUpAxis(sv),
 UsdGeom.GetStageMetersPerUnit(sv))
```

### Expected output

```
1 edit target: vis.usda
1 muted: []
2 valid/active/defined/loaded: True True True True
3 errors: 0
4 variant sets: []
5 has points: True
6 unresolved skip (no assets in this file)
7 purpose: proxy
7 visibility: inherited
7 scheme: catmullClark
7 extent authored: False
7 bound: False
7 upAxis/meters: Z 1.0
```

Stop at step 7: purpose, subdivision, extent, bind, units. Do not rebuild the mesh.

**Real-world use case.** A new hire dumps `FlattenLayerStack` into Slack for every ticket. Half the time the flatten *hides* the weaker asset opinion they were supposed to keep. The eight-step list in the studio wiki (this section) cuts those tickets to a stack dump and a purpose print.

### Common mistakes.

#### MISTAKE

Flattening first. You lose the evidence of *\*which layer\** won.

#### MISTAKE

Skipping step 2 because the outliner (Traverse) is empty. Payloads, inactives, and proxies live off that list.

#### MISTAKE

Mixing step 7 into step 5 ("the value is wrong" when the value is fine and purpose is proxy).

**Exam traps.****TRAP**

Stems that offer "open usdview Composition tab" as the only fix. Know the Python equivalents; usdview is not in `usd-core`.

**TRAP**

"Add a stronger `over` in the session layer" as a universal repair. That hides the real layer bug and will not survive reopen.

**TRAP**

One failed check means skip the rest forever. Fix it, then continue if the symptom remains.

**Exam takeaways.****KEY**

- Order: stack → prim flags → Pcp errors → arcs → property stack → assets → visual → flatten last. - First failed check names the bug class (6.2 vs 6.3 vs 6.4). - Python stand-ins exist for every usdview tab you will see named on the exam. - Extra overs are not a debugging strategy.

**Chapter lab(s)**

**Lab 21 — Composition introspection toolkit** (★★☆, Obj 1.8, 6.2; Ch 22 and 42). You plant a wrong-winner, a missing reference, and a muted sublayer, then recover each with `GetLayerStack`, `GetPropertyStack`, and `GetCompositionErrors`.

Chapter 33's Lab 29 (asset-path validation) covers §42.5. Lab 36 (Ch 43) adds `TfDebug / Trace` once the *scene* bug is understood.

**USDA reading exercise(s)**

**Exercise 42-A.** `radius` on `/World/Ball` is 4 in a debug `Get()`, but the artist edited `anim.usda` to 7. Why?

```
#usda 1.0
(
 subLayers = [
 @./anim.usda@
]
)

def Xform "World"
{
 def Sphere "Ball"
 {
 double radius = 4
 }
}
```

**Exercise 42-B.** `/World/Set` is an empty Xform. `set.usda` on disk contains `def Xform "Set" { def Mesh "Wall" {} }` and has no `defaultPrim`. What error type, and how do you fix it without renaming prims?

```
#usda 1.0

def "World"
{
 def "Set" (
 prepend references = @./set.usda@
)
 {
 }
}
```

**Exercise 42-C.** Beauty render is empty; Storm’s “proxy” purpose view shows the set. Composition errors: none. Name the most likely authored opinion.

```
#usda 1.0

def Xform "World"
{
 uniform token purpose = "proxy"

 def Mesh "Hall"
 {
 int[] faceVertexCounts = [4]
 int[] faceVertexIndices = [0, 1, 2, 3]
 point3f[] points = [(0, 0, 0), (1, 0, 0), (1, 0, 1), (0, 0, 1)]
 }
}
```

## Chapter review

### Summary

- Inspect session > root > sublayers; mute to test; watch the edit target.
- Missing prim = invalid / inactive / undefined / unloaded / proxy — five tests.
- `GetPropertyStack()[0]` is Obj 6.2. Blocks and Default-vs-samples fool `Get()`.
- `InvalidAssetPath` vs `UnresolvedPrimPath`; wrong variant names are not fallbacks.
- `ComputeAllDependencies` unresolved list is Obj 6.3.
- Visual bugs (Obj 6.4) are purpose, visibility, scheme, extent, bind, units, lights.
- Procedure order beats random overs; flatten last.

### If you see... → think...

If you see...	Think...
Value changes after mute	That layer was the winner
Session display name *-session.usda	Edit target / unsaved debug edit
LoadNone but references still there	Payloads are the ones unloaded
Child of active = false	Invalid, not inactive
SpecifierOver, IsDefined False	Typo over, no def
Stack[0] not your layer	Strength (root vs sublayer vs arc)
Get() ≠ viewport	Pass the numeric time; samples vs Default
radius = None	Value block
InvalidAssetPath	Missing file
UnresolvedPrimPath / <defaultPrim>	Add defaultPrim or </Prim>
Variant selection tiny	Authored wrong name; no fallback
unresolved: ['wood.png']	Asset path, not a composition error
Empty beauty, proxy view OK	ComputePurpose → proxy
Melted CAD	subdivisionScheme fallback catmullClark

### Review questions

**Q42.1** · Obj 6.2 · Easy · Single choice Among sublayers @./b.usda@, @./a.usda@, who wins? A. a · B. b · C. Neither; the root must author the value · D. The session layer always

**Q42.2** · Obj 6.2 · Easy · Single choice `GetLayerStack(includeSessionLayers=True)` lists the session layer: A. Last (weakest) · B. First (strongest) · C. Not at all · D. Only if saved to disk



**Q42.3** · Obj 6.2 · Medium · Single choice `Usd.Stage.Open(path, Usd.Stage.LoadNone)` loads: A. Nothing · B. References only, not payloads · C. Payloads only · D. Only the root layer's defs

**Q42.4** · Obj 6.2 · Medium · Select two. An inactive prim `/A` with child `/A/B`: A. `/A.IsValid()` True · B. `/A/B.IsValid()` True · C. `/A/B.IsValid()` False · D. `Traverse()` yields `/A/B`

**Q42.5** · Obj 6.2 · Medium · Single choice Property stack: shot 1, light 2, asset 0.5. Composed height? A. 0.5 · B. 2 · C. 1 · D. Average

**Q42.6** · Obj 6.2 · Medium · Single choice `GetResolveInfo().ValueIsBlocked()` True. `Get()` returns: A. Schema fallback · B. Next stack spec's value · C. None · D. 0

**Q42.7** · Obj 6.2 · Medium · Single choice Missing referenced file. `errorType` is: A. `UnresolvedPrimPath` · B. `InvalidAssetPath` · C. `ArcCycle` · D. None; the prim is invalid

**Q42.8** · Obj 6.2 · Medium · Single choice Referenced file opens, no `defaultPrim`, no prim path in the arc. `errorType` is: A. `InvalidAssetPath` · B. `UnresolvedPrimPath` · C. None · D. `LayerOffset`

**Q42.9** · Obj 6.3 · Easy · Single choice `ComputeAllDependencies` returns `(layers, assets, unresolved)`. Missing textures appear in: A. `layers` · B. `assets` · C. `unresolved` · D. `GetCompositionErrors()`

**Q42.10** · Obj 6.4 · Easy · Single choice `Unauthored` `subdivisionScheme` `Get()` is: A. none · B. `catmullClark` · C. `loop` · D. None

**Q42.11** · Obj 6.4 · Medium · Select two. Beauty render empty, proxy view full, no Pcp errors. Likely queries: A. `ComputePurpose()` · B. `GetCompositionErrors()` will be nonempty · C. `Inherited purpose = proxy` · D. `MuteLayer` of the root

**Q42.12** · Obj 6.4 · Easy · Single choice `ComputeVisibility()` on a drawing mesh usually prints: A. `visible` · B. `inherited` · C. `draw` · D. `True`

**Q42.13** · Obj 6.2 · Medium · Single choice Where should flatten sit in the debugging procedure? A. Step 1, to simplify · B. After muting every layer · C. Last, as a repro snapshot · D. Never; it is illegal

## Answers

**Q42.1 — B.** First-listed sublayer is stronger. Review: §42.1.

**Q42.2 — B.** Session is strongest. Review: §42.1.

**Q42.3 — B.** References still load. Review: §42.2.

**Q42.4 — A, C.** Parent valid+inactive; child not composed. Review: §42.2.

**Q42.5 — C.** First spec wins. Review: §42.3.

**Q42.6 — C.** A block contributes `None`. Review: §42.3.

**Q42.7 — B.** File cannot open. The referencing prim is still defined. Review: §42.4.

**Q42.8 — B.** `<defaultPrim>` in the message. Review: §42.4.

**Q42.9 — C.** Successful files are `assets`; failures are `unresolved`. Review: §42.5.

**Q42.10 — B.** Review: §42.6 / Ch 39.

**Q42.11 — A, C.** Composition is fine; imaging purpose is not. Review: §42.6.

**Q42.12 — B.** `"visible"` is not the authored token. Review: §42.6.

**Q42.13 — C.** Flatten last so you do not erase the stack evidence. Review: §42.7.

**USDA exercise answers****42-A — The root layer's local radius = 4 is stronger than the anim.usda sublayer.**

First-listed sublayer still loses to the root. Move the 4 into anim, remove it from the root, or reorder so a *stronger* layer than the root holds the 7 (session / a new stronger sublayer above the root — which is impossible for a sublayer; the 4 must leave the root).

**42-B — Pcp.ErrorType\_UnresolvedPrimPath.** Fix: ( defaultPrim = "Set" ) on set.usda, or prepend references = @./set.usda@</Set>.

**42-C — uniform token purpose = "proxy" on /World, inherited by Hall.**

ComputePurpose() is proxy. Beauty skips it; a proxy-purpose view draws it. Not a missing reference.

## CHAPTER 43

## Chapter 43 — Diagnostics and Profiling Tools

**Exam domain:** Debugging & Troubleshooting (11%) · **Objectives:** 6.5 · **Study day:** 12 · **Est. time:** 90 min **Prerequisites:** Ch 42 (scene debugging), Ch 8 / 34 (notices, ChangeBlock), Ch 22 (composition errors)

Chapter 42 asked *what is wrong with the stage*. This chapter asks *how USD talks while it works*: **status**, **warnings**, **errors**, **TfDebug** symbols, **diagnostic delegates** that capture those messages in Python, **Trace** timing, **TfMallocTag** memory, and **Pcp composition errors** as objects you can assert on. That is Obj 6.5 in one sitting.

`usd-core` 26.8 includes all of these Python modules. It still has no `usdview` profiler UI.

### Learning goals

- Emit and catch `Tf.Status`, `Tf.Warn`, and `Tf.ErrorException`.
- List, enable, and disable `Tf.Debug` symbols (and know `TF_DEBUG`).
- Capture warnings with `UsdUtils.CoalescingDiagnosticDelegate`.
- Turn `Trace.Collector` on, mark events, and update the global reporter.
- Initialize `Tf.MallocTag` and read `GetTotalBytes` (including the 0-byte `usd-core` case).
- Read `stage.GetCompositionErrors()` as the structured form of open-time warnings.

### Key terms

Term	One-line definition
<b>Tf</b>	The C++/Python foundation library: diagnostics, debug bits, errors.
<b>Status</b>	An informational diagnostic ( <code>Tf.Status</code> ).
<b>Warning</b>	A non-fatal diagnostic ( <code>Tf.Warn</code> ); composition failures often arrive as warnings.
<b>ErrorException</b>	Python exception raised for USD coding/runtime errors.
<b>TfDebug symbol</b>	A named switch ( <code>USD_CHANGES</code> , <code>PCP_PRIM_INDEX</code> , ...) that prints extra logs when enabled.
<b>Diagnostic delegate</b>	An object that <i>receives</i> diagnostics instead of (or as well as) printing them.
<b>Coalescing</b>	Grouping identical diagnostics so a storm of the same warning is one item.
<b>Trace</b>	USD's instrumentation timer ( <code>Trace.Collector</code> / <code>Trace.Reporter</code> ).
<b>MallocTag</b>	Optional allocator tagging so you can see which subsystem allocated bytes.
<b>Pcp error</b>	A composition-error object ( <code>errorType</code> , <code>rootSite</code> ) on the stage.

### 43.1 Tf errors, warnings, status

**What is it?** USD's messaging API lives on **Tf**. **Tf.Status(msg)** is info, **Tf.Warn(msg)** is a warning, **Tf.RaiseCodingError(msg)** raises **Tf.ErrorException**. Failed layer parses and many API misuses raise `ErrorException` automatically.

**Why do we need it?** Print statements in your exporter are not what Hydra or `usdchecker` emit. Matching the exam (and production logs) means recognizing `Warning: in ... --` lines and catching `ErrorException` instead of a bare `Exception` when you want USD-only failures.

**Beginner explanation.** Status is a polite note. Warning is a yellow flag you can keep driving through. `ErrorException` is the referee stopping play. All three share a log format: Kind: in <function> at line N of <file> -- message.

*Where the analogy breaks:* composition “errors” (missing references) are usually **warnings** at open time, not `ErrorException`. The stage still opens. Chapter 42 / §43.6 collect them as `Pcp` objects.

**Technical explanation.** - `Tf.Status("...")` / `Tf.Warn("...")` write to **stderr**, not `stdout`. `check_code_blocks.py` will not see them; you still emit them so a human (or a delegate, §43.3) does. - `Tf.RaiseCodingError("...")` and `Tf.RaiseRuntimeError("...")` raise **`Tf.ErrorException`**. The message includes Python coding error: ... Catch `Tf.ErrorException`, not `RuntimeError`. - `Sdf.Layer.ImportFromString("not usda")` and similar parse failures also raise `ErrorException`. - `Tf.Fatal` / fatal diagnostic types abort; do not call them in libraries. Types you will see on captured diagnostics: `TF_DIAGNOSTIC_WARNING_TYPE`, `TF_DIAGNOSTIC_STATUS_TYPE`, `TF_DIAGNOSTIC_CODING_ERROR_TYPE`, ... - `Tf.Error` is a class for posted C++ errors; Python pipelines usually see `ErrorException` or a delegate item, not a manual `Tf.Error()` call.

### Mental model.

stderr:	Status	Warn	(keep going)
Python:	raise	<code>Tf.ErrorException</code>	(catch me)
fatal:	abort		(do not use in tools)

**Simple example.** `Tf.Status("ch43-status")` and `Tf.Warn("ch43-warn")` print on `stderr`. `RaiseCodingError("ch43-boom")` is caught as `ErrorException` containing `ch43-boom`.

**USDA example.** There is no USDA for a `Tf` call. A **bad** USDA string is what triggers `ErrorException` from the parser:

```
#usda 1.0

def Sphere "Ok"
{
 double radius = 1
}
```

That file is valid. The Python example deliberately feeds `"not usda"` to `ImportFromString` to show the exception path.

### Python example.

```
from pxr import Tf, Sdf

print("emitting Status and Warn on stderr")
Tf.Status("ch43-status")
Tf.Warn("ch43-warn")
try:
 Tf.RaiseCodingError("ch43-boom")
except Tf.ErrorException as err:
 print("caught:", type(err).__name__)
 print("has boom:", "ch43-boom" in str(err))
layer = Sdf.Layer.CreateAnonymous(".usda")
try:
 layer.ImportFromString("not usda")
 print("import ok")
except Tf.ErrorException:
 print("import raised ErrorException")
```

### Expected output

```
emitting Status and Warn on stderr
caught: ErrorException
has boom: True
import raised ErrorException
```

Stderr also shows `Status: ... ch43-status` and `Warning: ... ch43-warn` plus the coding-error dump. Those lines are not part of stdout.

**Real-world use case.** A batch importer wraps each `Stage.Open` and each `ImportFromString` in `except Tf.ErrorException`. Corrupt files fail that file, not the whole farm job. Status lines (`imported 12 shots`) stay on stderr for the farm log.

### Common mistakes.

#### MISTAKE

Catching ``Exception`` around ``Stage.Open`` and swallowing composition warnings. Open `*succeeds*`; warnings are not exceptions. Use a delegate or ``GetCompositionErrors()``.

#### MISTAKE

Looking for Tf messages on stdout. They are on `**stderr**`.

#### MISTAKE

Calling ``Tf.Fatal`` from an exporter to "fail loudly." It aborts the DCC.

### Exam traps.

#### TRAP

"A missing reference raises ``ErrorException``." It warns; the stage opens; Pcp lists the error.

#### TRAP

``Tf.Warn`` returns a bool you must check. It returns `None`; it `*emits*`.

#### TRAP

``ErrorException`` is a ``RuntimeError`` subclass you can catch as ``RuntimeError`` only — catch `**`Tf.ErrorException`**` to avoid eating unrelated bugs.

### Exam takeaways.

#### KEY

- Status/Warn → stderr; coding/runtime problems → ``Tf.ErrorException``. - Missing references are warnings, not exceptions. - Catch ``Tf.ErrorException`` around parse/open of untrusted files. - Fatal diagnostics abort; do not use them in pipeline tools.

## 43.2 TfDebug and TF\_DEBUG

**What is it?** `TfDebug` is a set of **named switches**. When `USD_CHANGES` is enabled, USD prints change-processing chatter. Python:

`Tf.Debug.SetDebugSymbolsByName("USD_CHANGES", True)`. The environment variable `TF_DEBUG` enables the same names before the process starts (`TF_DEBUG=USD_CHANGES`).

**Why do we need it?** Stack dumps (§42) tell you *who won*. Debug symbols tell you *what USD did internally* (recompositions, layer loads, prim indexing). Obj 6.5 names `TfDebug` explicitly.

**Beginner explanation.** Think of extra verbose stickers on factory machines. Off, the floor is quiet. Flip `USD_CHANGES`, and the change-processing machine narrates every notice. `TF_DEBUG` is the wall switch you flip before entering the building (process start). `SetDebugSymbolsByName` is the switch on the machine after you are inside.

*Where the analogy breaks:* enabling a symbol can flood stderr. Enable one name, reproduce, disable.

**Technical explanation.** - `Tf.Debug.GetDebugSymbolNames()` → list of strings (58 names in `usd-core 26.8`, including `USD_CHANGES`, `PCP_PRIM_INDEX`, `PCP_CHANGES`, `SDF_ASSET`, ...). - `Tf.Debug.GetDebugSymbolDescription("USD_CHANGES")` → "USD change processing"

(verified). - `Tf.Debug.IsDebugSymbolNameEnabled(name)` → bool, default **False**. - `Tf.Debug.SetDebugSymbolsByName("USD_CHANGES", True)` → `['USD_CHANGES']` (the names it actually flipped). Pass `False` to turn off. Unknown names yield an empty list, not an exception. - **TF\_DEBUG**: colon- or space-separated names, or glob-like patterns depending on build. Set **before** launching Python. Changing the env var *after* import does not retroactively enable symbols. - `Tf.Debug.SetOutputFile` can redirect debug prints. Default is `stderr`. - These prints are *not* composition errors and are *not* captured by `GetCompositionErrors()`. A delegate (§43.3) may see them as status/warnings depending on how the symbol logs.

#### VERSION

Verified on USD 26.08. Symbol *\*names\** are stable enough for the exam (``USD_CHANGES``, ``PCP_PRIM_INDEX``); the exact count (58 here) can grow. Do not memorize the count.

#### Mental model.

```
TF_DEBUG=USD_CHANGES python job.py # before process
Tf.Debug.SetDebugSymbolsByName(name, True) # after import
Tf.Debug.IsDebugSymbolNameEnabled(name) # query
```

**Simple example.** `USD_CHANGES` starts off. `SetDebugSymbolsByName("USD_CHANGES", True)` returns `['USD_CHANGES']` and `IsDebugSymbolNameEnabled` becomes `True`. Turning it off returns to `False`.

**USDA example.** Debug symbols are process state, not USD files. A tiny layer you might watch under `USD_CHANGES`:

```
#usda 1.0

def Xform "World"
{
 def Sphere "Ball"
 {
 double radius = 1
 }
}
```

Enabling `USD_CHANGES` then setting `radius` prints change chatter on `stderr` (not shown; not deterministic enough to snapshot).

#### Python example.

```
from pxr import Tf

print("USD_CHANGES enabled:",
 Tf.Debug.IsDebugSymbolNameEnabled("USD_CHANGES"))
print("description:",
 Tf.Debug.GetDebugSymbolDescription("USD_CHANGES"))
got = Tf.Debug.SetDebugSymbolsByName("USD_CHANGES", True)
print("set returned:", got)
print("USD_CHANGES enabled:",
 Tf.Debug.IsDebugSymbolNameEnabled("USD_CHANGES"))
Tf.Debug.SetDebugSymbolsByName("USD_CHANGES", False)
print("after off:", Tf.Debug.IsDebugSymbolNameEnabled("USD_CHANGES"))
print("has USD_CHANGES:",
 "USD_CHANGES" in Tf.Debug.GetDebugSymbolNames())
```

#### Expected output

```
USD_CHANGES enabled: False
description: USD change processing
set returned: ['USD_CHANGES']
USD_CHANGES enabled: True
after off: False
has USD_CHANGES: True
```

Always turn symbols off when the capture is done so later tests stay quiet.

**Real-world use case.** A recomposition loop makes usdview hitch. A TD relaunches with `TF_DEBUG=PCP_PRIM_INDEX,USD_CHANGES`, repeats the edit, and sees prim indexes rebuilt on every keystroke because a script authors in a `ChangeBlock`-less loop (Chapter 34). The fix is batching, not a faster machine.

### Common mistakes.

#### MISTAKE

Setting ``os.environ["TF_DEBUG"]`` after ``from pxr import Usd``. Too late for many symbols. Set it in the shell, or use ``SetDebugSymbolsByName``.

#### MISTAKE

Leaving ``USD_CHANGES`` on in a farm job. Logs explode; disk fills.

#### MISTAKE

Treating debug prints as ``GetCompositionErrors()``. Different channels.

### Exam traps.

#### TRAP

"``TF_DEBUG`` is a prim metadata flag." It is a **process environment variable**.

#### TRAP

``SetDebugSymbolsByName`` raises on unknown names. It returns an **empty list**.

#### TRAP

``USD_CHANGES`` is on by default in usd-core. It is **off**.

### Exam takeaways.

#### KEY

- Obj 6.5: `TfDebug` names are strings; default off; ``SetDebugSymbolsByName`` / ``TF_DEBUG``. - ``USD_CHANGES`` = change processing chatter. - Unknown names → empty list. Turn symbols off after use. - Debug logs ≠ Pcp composition errors.

## 43.3 Diagnostic delegates (`UsdUtils.CoalescingDiagnosticDelegate`)

**What is it?** A **diagnostic delegate** is a Python object that **receives** Tf diagnostics. `UsdUtils.CoalescingDiagnosticDelegate()` (no arguments) installs itself while it lives, then lets you `TakeUncoalescedDiagnostics()` (every message) or `TakeCoalescedDiagnostics()` (grouped duplicates).

**Why do we need it?** Farm tests cannot read a human's stderr. A missing reference would otherwise be a warning you never asserted on. The delegate turns that warning into a list of objects with `commentary`, `diagnosticCode`, and `sourceFileName`. Obj 6.5 names diagnostic delegates.

**Beginner explanation.** A delegate is a tape recorder plugged into the factory PA. USD still shouts; you also get a cassette. Uncoalesced is every shout. Coalesced is "the same shout 400 times, here is one copy plus a count."

*Where the analogy breaks:* constructing the delegate is plugging it in. There is no separate `Register` call. Let it go out of scope (or keep a reference until you `Take*`) so you do not leak it.

**Technical explanation.** - `delegate = UsdUtils.CoalescingDiagnosticDelegate()` starts capturing immediately. - `TakeUncoalescedDiagnostics()` → list of `_DiagnosticBase: .commentary` (message text), `.diagnosticCode` (`Tf.TF_DIAGNOSTIC_WARNING_TYPE, ...`), `.diagnosticCodeString`

(`"TF_DIAGNOSTIC_WARNING_TYPE"`), `.sourceFileName` (e.g. `stage.cpp`), `.sourceLineNumber`, `.sourceFunction`. Taking **clears** the queue. - **TakeCoalescedDiagnostics()** groups identical messages. After a take, the other queue may be empty — call the `Take*` that matches how you want to assert, not both unless you know which one drains first. - Helpers: `DumpUncoalescedDiagnostics()`, `DumpCoalescedDiagnosticsToStdout()`, `DumpCoalescedDiagnosticsToStderr()`. - Opening a stage with `@./missing.usda@` yields one uncoalesced item, warning type, commentary containing `missing.usda` (verified). - Delegates do **not** replace `GetCompositionErrors()`. Use both: delegate for the log line, `Pcp` for the typed error. - `Tf.DiagnosticTrap` exists as a context helper in some bindings; in 26.8's Python it has no extra public methods — prefer the coalescing delegate.

#### VERSION

Verified on USD 26.08. ``UsdUtils.CoalescingDiagnosticDelegate`` is present. There is no ``UsdUtils.DiagnosticDelegate`` base you subclass in this wheel; use the coalescing class.

#### Mental model.

```
delegate = CoalescingDiagnosticDelegate() # plug in
Stage.Open(...) # warnings recorded
items = delegate.TakeUncoalescedDiagnostics()
 items[0].diagnosticCodeString == TF_DIAGNOSTIC_WARNING_TYPE
 "missing.usda" in items[0].commentary
```

**Simple example.** Open `shot.usda` that references `missing.usda`.

`TakeUncoalescedDiagnostics()` has length 1, code string `TF_DIAGNOSTIC_WARNING_TYPE`, commentary mentions `missing.usda`.

#### USDA example.

```
#usda 1.0

def "A" (
 prepend references = @./missing.usda@
)
{
}
```

- Valid USDA. The *target file* is what is missing.
- Open emits a warning the delegate records.

#### Python example.

```
import os
from pxr import Tf, Usd, UsdUtils

open("shot.usda", "w").write("""#usda 1.0
def "A" (
 prepend references = @./missing.usda@
)
{
}
""")

delegate = UsdUtils.CoalescingDiagnosticDelegate()
stage = Usd.Stage.Open("shot.usda")
items = delegate.TakeUncoalescedDiagnostics()
print("n:", len(items))
print("code:", items[0].diagnosticCodeString)
print("warning:",
 items[0].diagnosticCode == Tf.TF_DIAGNOSTIC_WARNING_TYPE)
print("mentions missing:", "missing.usda" in items[0].commentary)
print("source file:", os.path.basename(items[0].sourceFileName))
print("pcp errors:", len(stage.GetCompositionErrors()))
```

#### Expected output



```
n: 1
code: TF_DIAGNOSTIC_WARNING_TYPE
warning: True
mentions missing: True
source file: stage.cpp
pcp errors: 1
```

Same incident, two APIs: one log line, one Pcp error.

**Real-world use case.** CI opens every published asset under a delegate and fails the build if any item is a warning mentioning `Could not open asset`. That catches broken references even when the publisher forgot `GetCompositionErrors()`.

### Common mistakes.

#### MISTAKE

Creating the delegate `*after* `Stage.Open``. You missed the warning.

#### MISTAKE

Calling both ``TakeCoalesced`` and ``TakeUncoalesced`` and expecting both to be full. The first take drains.

#### MISTAKE

Asserting on ``commentary ==`` the full string. It contains absolute paths. Assert ``in`` (`"missing.usda" in commentary``).

### Exam traps.

#### TRAP

The delegate class is ``Tf.CoalescingDiagnosticDelegate``. It is `**`UsdUtils.CoalescingDiagnosticDelegate`**`.

#### TRAP

Captured missing-reference diagnostics are ``TF_DIAGNOSTIC_CODING_ERROR_TYPE``. They are `**warnings**`.

#### TRAP

You must ``Apply`` the delegate on a prim. It is process-global while the object lives.

### Exam takeaways.

#### KEY

- ``UsdUtils.CoalescingDiagnosticDelegate()`` starts capturing on construct. - `Uncoalesced` = every message; `coalesced` = grouped. `Take` clears the queue. - Missing refs: warning delegate item `*and*` a Pcp error. - Match substrings in ``commentary``, not full paths.

## 43.4 Trace collector and reports

**What is it?** Trace is USD's built-in timer. `Trace.Collector()` is the global collector: set `enabled = True`, `BeginEvent("name") / EndEvent("name")`, then `Trace.Reporter.globalReporter.UpdateTraceTrees()`. `ReportTimes()` prints an aggregate table (stderr/stdout depending on the call).

**Why do we need it?** Obj 6.5 lists Trace. When a flatten is slow, you want *which* USD function ate the seconds, not a generic Python profiler that cannot see C++ (Pcp, crate reads). Trace is that C++-aware timeline.

**Beginner explanation.** `BeginEvent / EndEvent` are start/stop buttons labelled with a string (`"ch43"`). The collector is off by default so production does not pay for the stopwatch. The global reporter is the printout you ask for after the race.

*Where the analogy breaks:* timings are **not deterministic**. Never snapshot milliseconds in a test. Snapshot that the collector toggled and that your event name was used.

**Technical explanation.** - `c = Trace.Collector()` returns the **global** collector (`GetLabel()` → "TraceRegistry global collector"). `c.enabled` default **False**. `c.pythonTracingEnabled` can also wrap Python functions (noisier). - `c.BeginEvent("ch43")` / `c.EndEvent("ch43")` must pair. Mismatches drop events. - `Trace.Reporter.globalReporter` (`GetLabel()` → "Trace global reporter"). Call **UpdateTraceTrees()** before reading. `ReportTimes()` prints "Total time for each key" with millisecond rows (including your "ch43"). `ReportChromeTracingToFile(path)` writes a Chrome-tracing JSON for `chrome://tracing`. - `c.Clear()` / `reporter.ClearTree()` reset. Leave `enabled = False` when done. - Trace does not require a stage. You can time any Python + USD mix.

#### VERSION

Verified on USD 26.08. ``Trace.Collector``, ``Trace.Reporter.globalReporter``, ``BeginEvent`` / ``EndEvent``, ``ReportTimes``, ``ReportChromeTracingToFile`` are present.

#### Mental model.

```
enabled False → no cost
enabled True → BeginEvent("name") ... work ... EndEvent("name")
 → globalReporter.UpdateTraceTrees()
 → ReportTimes() / Chrome JSON
```

**Simple example.** Default `enabled False`. Turn on, `BeginEvent("ch43")`, `sum(range(1000))` (= 499500), `EndEvent`, turn off. Reporter label is `Trace global reporter`.

**USDA example.** Trace is not authored in USDA. Any stage you time is fine:

```
#usda 1.0

def Sphere "Ball"
{
 double radius = 1
}
```

Wrap `Usd.Stage.Open` / `Flatten` with `Begin/End` in real jobs.

#### Python example.

```
from pxr import Trace

c = Trace.Collector()
print("enabled default:", c.enabled)
c.enabled = True
c.BeginEvent("ch43")
total = sum(range(1000))
c.EndEvent("ch43")
print("enabled after on:", c.enabled)
c.enabled = False
print("enabled after off:", c.enabled)
print("sum:", total)
print("collector label:", c.GetLabel())
rep = Trace.Reporter.globalReporter
rep.UpdateTraceTrees()
print("reporter label:", rep.GetLabel())
```

#### Expected output

```
enabled default: False
enabled after on: True
enabled after off: False
sum: 499500
collector label: TraceRegistry global collector
reporter label: Trace global reporter
```

`ReportTimes()` would print a `ch43` row with a tiny millisecond value; we omit it because the number changes.

**Real-world use case.** A nightly flatten of a city set jumps from 4 s to 40 s. Trace around `UsdUtils.FlattenLayerStack` vs `stage.Export` (Chapter 34) shows `Export` spending its time in asset-path remapping. The TD switches the publish path to `FlattenLayerStack` for that show.

### Common mistakes.

#### MISTAKE

Reading the reporter without ``UpdateTraceTrees()``. Empty report.

#### MISTAKE

Leaving ``enabled = True`` in production. Every ``GetAttribute`` pays.

#### MISTAKE

Asserting ``ReportTimes`` milliseconds in CI. They jitter.

### Exam traps.

#### TRAP

``Trace.Collector()`` constructs a *\*new\** collector each time. It is the *\*\*global\*\** collector (same object).

#### TRAP

`Trace` is part of ``UsdUtils``. It is *\*\*`from pxr import Trace`\*\**.

#### TRAP

Collector is on by default so you always get timings. Default is *\*\*False\*\**.

### Exam takeaways.

#### KEY

- ``Trace.Collector().enabled`` default `False`; ``BeginEvent`` / ``EndEvent`` pair. - ``Trace.Reporter.globalReporter.UpdateTraceTrees()`` then report. - Chrome JSON via ``ReportChromeTracingToFile``. - Time values jitter; API names do not.

## 43.5 TfMallocTag

**What is it?** `Tf.MallocTag` tags heap allocations with a subsystem name so you can ask **who allocated how many bytes**. `Initialize()`, `IsInitialized()`, `GetTotalBytes()`, `GetMaxTotalBytes()`, `GetCallTree().GetPrettyPrintString()`.

**Why do we need it?** Obj 6.5 lists `TfMallocTag` next to `Trace`: time vs memory. A stage that is “slow” is sometimes just “huge.” `MallocTag` is how USD-aware builds answer that without a generic heap profiler.

**Beginner explanation.** Every dollar spent in a store gets a sticker: “produce,” “hardware.” At the end you add up stickers. `MallocTag` is those stickers on `malloc`. `GetPrettyPrintString()` is the receipt.

*Where the analogy breaks:* the stickers are compiled into USD. The **pip usd-core wheel may initialize the API and still count 0 bytes** because tagging was not compiled in. The exam still expects the API names.

**Technical explanation.** - `Tf.MallocTag.IsInitialized()` starts **False**. `Initialize()` returns **False** on this wheel but `IsInitialized()` becomes **True** (verified 26.08). Treat “initialized” as “the module is ready,” not “bytes will be nonzero.” - `GetTotalBytes()` / `GetMaxTotalBytes()` → **0** here. A studio *debug/tagging* build of OpenUSD returns real numbers after you open a large stage. - `GetCallTree()` → `CallTree` with `GetPrettyPrintString()`, `GetCallSites()`, `GetRoot()`, `Report`. The pretty string starts with

Malloc Tag Report and Total bytes = 0 on this wheel. - SetDebugMatchList / SetCapturedMallocStacksMatchList filter tags (advanced). - Use MallocTag for **which USD subsystem** grew (Pcp vs Imaging vs Crate). Use OS tools (ps, heaptrack) for the whole process.

#### VERSION

Verified on USD 26.08 `usd-core`: `Initialize` returns False, `IsInitialized` True, `GetTotalBytes` 0. Full OpenUSD builds with malloc tagging report nonzero bytes. Teach the API; do not promise nonzero counts from pip.

#### Mental model.

```
MallocTag.Initialize() → IsInitialized True (even if bytes stay 0)
GetTotalBytes() → 0 on usd-core; nonzero on tagged builds
GetCallTree().GetPrettyPrintString() → "Malloc Tag Report"
```

**Simple example.** Call Initialize, print IsInitialized True, GetTotalBytes 0, pretty-print starts with the report banner.

**USDA example.** Memory tagging does not live in USDA. Opening a large layer is the usual workload you wrap:

```
#usda 1.0

def Xform "World"
{
 def Sphere "Ball"
 {
 double radius = 1
 }
}
```

#### Python example.

```
from pxr import Tf

print("before:", Tf.MallocTag.IsInitialized())
result = Tf.MallocTag.Initialize()
print("Initialize returned:", result)
print("after:", Tf.MallocTag.IsInitialized())
print("total bytes:", Tf.MallocTag.GetTotalBytes())
print("max bytes:", Tf.MallocTag.GetMaxTotalBytes())
text = Tf.MallocTag.GetCallTree().GetPrettyPrintString()
print("report banner:", "Malloc Tag Report" in text)
print("report has Total bytes:", "Total bytes" in text)
```

#### Expected output

```
before: False
Initialize returned: False
after: True
total bytes: 0
max bytes: 0
report banner: True
report has Total bytes: True
```

Initialize returned: False with after: True is the usd-core pattern, not a bug in your script.

**Real-world use case.** A lighting DCC build (tagged) shows Pcp indexes at 4 GB after loading a city. Instancing (Chapter 24) and payloads (Chapter 17) drop the tag. The same script on usd-core still “works”; it just prints 0, so TDs run it on the DCC’s interpreter for real numbers.

#### Common mistakes.

##### MISTAKE

Treating `Initialize() == False` as fatal. Check `IsInitialized()` too.

**MISTAKE**

Comparing `GetTotalBytes()` across pip and a studio build. Different compilation.

**MISTAKE**

Using `MallocTag` to find a Python list leak. It tags `**USD's**` allocator, not all of CPython.

**Exam traps.****TRAP**

`MallocTag` is in ``UsdUtils``. It is `**`Tf.MallocTag`**`.

**TRAP**

"If total bytes is 0, Initialize failed and you cannot call `GetCallTree`." You can; the report says 0.

**TRAP**

`Trace` and `MallocTag` are the same tool. `Trace = **time**`; `MallocTag = **bytes**`.

**Exam takeaways.****KEY**

- ``Tf.MallocTag.Initialize` / `IsInitialized` / `GetTotalBytes` / `GetCallTree` . - usd-core may report **0 bytes**; studio tagged builds do not. - Trace = time, MallocTag = memory. Obj 6.5 expects both names. - `Initialize() == False` can still leave `IsInitialized` True.`

## 43.6 Pcp composition errors

**What is it?** **Pcp** (the composition engine) records **typed errors** on the stage: `stage.GetCompositionErrors()`. Each item has **errorType** (enum) and **rootSite** (where it was introduced). This is the structured twin of the warning in §43.1 / §43.3 and of the playbook in §42.4.

**Why do we need it?** Stderr text is for humans. Tests and Obj 6.5 want an object: `Pcp.ErrorType_InvalidAssetPath` vs `UnresolvedPrimPath`. You already used the API in Chapter 42; here you connect it to diagnostics (same warning, two APIs) and to the other Obj 6.5 tools.

**Beginner explanation.** The factory PA shouted “could not open missing.usda” (warning). The incident log binder has a form with a **type code** and a **site** (Pcp error). Delegates copy the shout; `GetCompositionErrors` copies the form.

*Where the analogy breaks:* not every warning is a Pcp error (`Tf.Warn` from your own code is not). Not every Pcp error stays a warning if a future version promotes it.

**Technical explanation.** - `stage.GetCompositionErrors()` → list of Pcp error objects. Public fields in 26.08: **errorType**, **rootSite**. `str(err)` is the message (contains `@missing.usda@`, paths). - Types you must recognize: - `Pcp.ErrorType_InvalidAssetPath` — file cannot be opened. - `Pcp.ErrorType_UnresolvedPrimPath` — file opened, prim path / defaultPrim failed. - `Pcp.ErrorType_ArcCycle` — circular composition (mentioned in Ch 22). - `rootSite` prints like `@shot.usda@, @anon:...:shot-session.usda@` — the layer stack plus the introducing prim. Do not snapshot the anon id; check `"</A>"` in `str(err.rootSite)` or similar. - Errors are filled when the stage **opens / recomposes**. They sit on the stage until composition changes. Opening under `LoadNone` still reports missing *reference* files (references load); missing *payload* files report when that payload is loaded. - Pipeline pattern: `if stage.GetCompositionErrors(): fail_publish()`. Combine with a delegate if you also care about non-Pcp warnings.

**Mental model.**

```

Stage.Open
 stderr warning → CoalescingDiagnosticDelegate (text)
 Pcp error object → GetCompositionErrors() (type + site)

```

Same missing file, both channels.

**Simple example.** `shot.usda` references `missing.usda`. `Errors` length 1, `errorType` is `InvalidAssetPath`, `str(err)` contains `missing.usda`, `rootSite` mentions `</A>`.

**USDA example.**

```

#usda 1.0

def "A" (
 prepend references = @./missing.usda@
)
{
}

```

Identical to §43.3's file on purpose: one incident, two APIs.

**Python example.**

```

from pxr import Usd, Pcp

open("shot.usda", "w").write("""#usda 1.0
def "A" (
 prepend references = @./missing.usda@
)
{
}
""")
stage = Usd.Stage.Open("shot.usda")
errs = stage.GetCompositionErrors()
print("n:", len(errs))
print("type:", errs[0].errorType)
print("is InvalidAssetPath:",
 errs[0].errorType == Pcp.ErrorType_InvalidAssetPath)
print("mentions missing:", "missing.usda" in str(errs[0]))
print("site has /A:", "" in str(errs[0].rootSite))

```

**Expected output**

```

n: 1
type: Pcp.ErrorType_InvalidAssetPath
is InvalidAssetPath: True
mentions missing: True
site has /A: True

```

Import **Pcp** to compare enum values; `str(err.errorType)` also contains `InvalidAssetPath`.

**Real-world use case.** A publisher fails the job when *any* `GetCompositionErrors()` item is `InvalidAssetPath` or `UnresolvedPrimPath`, and emails `str(err)` plus the layer display name. Lighting still uses the delegate when they care about `Tf.Warn` from their own shaders.

**Common mistakes.**

#### MISTAKE

Parsing `stderr` instead of ``GetCompositionErrors()`` in tests. Fragile paths, locale, and line numbers.

#### MISTAKE

Assuming ``rootSite`` is a ``Sdf.Path``. It is a `Pcp` site; ``str()`` it.

#### MISTAKE

Clearing errors with ``Tf.Status``. Only a recomposition that fixes the arc clears them.

**Exam traps.**

**TRAP**

``GetCompositionErrors`` lives on ``Pcp.Cache``. You call it on `**`Usd.Stage`**`.

**TRAP**

``InvalidAssetPath`` means the referencing prim is invalid. The prim is defined; the `*asset*` is missing.

**TRAP**

Pcp errors are TfDebug symbols. They are `**objects**`; TfDebug is optional chatter `*about*` composition.

**Exam takeaways.****KEY**

- Pcp errors = typed composition failures on the stage (``errorType``, ``rootSite``). - Same missing file also warns (delegate). Tests should use the enum. - ``InvalidAssetPath``  $\neq$  ``UnresolvedPrimPath``; both leave a defined empty prim. - Obj 6.5 ties this to TfDebug, delegates, Trace, and MallocTag — five tools, five jobs.

**Chapter lab(s)**

**Lab 36 — Debugging toolkit: diagnostics delegate, TfDebug, Trace (★★★, Obj 6.5).** You capture a missing-reference warning with `CoalescingDiagnosticDelegate`, `assert GetCompositionErrors()[0].errorType`, toggle `USD_CHANGES`, and wrap a flatten in `Trace.Collector` Begin/End events. Stretch: `MallocTag.GetPrettyPrintString()` banner check.

Chapter 42's Lab 21 remains the composition-stack lab; this lab is the *instrumentation* lab.

**USDA reading exercise(s)**

**Exercise 43-A.** This file opens. Does Python raise `Tf.ErrorException`? What do you call to see the failure in-process?

```
#usda 1.0

def "Chair" (
 prepend references = @./does_not_exist.usda@
)
{
}
```

**Exercise 43-B.** A TD sets `os.environ["TF_DEBUG"] = "USD_CHANGES"` on the line *after* from `pxr import Usd`, then authors an attribute, and sees no extra logs. Why?

**Exercise 43-C.** `MallocTag.GetTotalBytes()` is 0 after loading a city set in the book's `.venv`. Is Initialize broken?

**Chapter review****Summary**

- Status/Warn → `stderr`; `RaiseCodingError` → `Tf.ErrorException`. Missing refs warn, they do not raise.
- TfDebug: named symbols, default off, `SetDebugSymbolsByName` / `TF_DEBUG`.
- `UsdUtils.CoalescingDiagnosticDelegate` captures diagnostics; take uncoalesced or coalesced.
- Trace: global collector enabled, Begin/End, `Reporter.globalReporter`.
- MallocTag: Initialize / `GetTotalBytes` / `GetCallTree`; `usd-core` often stays at 0 bytes.

- `Pcp: GetCompositionErrors()` for typed failures (`InvalidAssetPath`, `UnresolvedPrimPath`).

### If you see... → think...

If you see...	Think...
Warning: in <code>_ReportErrors</code> ... Could not open asset	Delegate + <code>GetCompositionErrors</code>
<code>Tf.ErrorException</code> / Python coding error	Parse or <code>RaiseCodingError</code> , not a missing ref
<code>TF_DEBUG=USD_CHANGES</code>	Process env, set before launch
<code>SetDebugSymbolsByName</code> → []	Unknown symbol name
<code>TakeUncoalescedDiagnostics</code> empty	Delegate created too late, or already taken
<code>diagnosticCodeString</code> WARNING	Missing ref is a warning
Trace report empty	Forgot <code>UpdateTraceTrees</code> or enabled
<code>GetTotalBytes()</code> == 0	usd-core without tagging, not necessarily an empty stage
<code>Pcp.ErrorType_UnresolvedPrimPath</code>	File opened; prim path / <code>defaultPrim</code> failed

### Review questions

- Q43.1** · Obj 6.5 · Easy · Single choice `Tf.Warn("x")` writes to: A. `stdout` · B. `stderr` · C. the session layer · D. `GetCompositionErrors()`
- Q43.2** · Obj 6.5 · Easy · Single choice `Tf.RaiseCodingError("x")` raises: A. `RuntimeError` · B. `Tf.ErrorException` · C. `Sdf.Error` · D. nothing; it warns
- Q43.3** · Obj 6.5 · Medium · Single choice `Stage.Open` of USDA that references a missing file: A. Raises `ErrorException` · B. Returns `None` · C. Returns a stage and warns · D. Hangs until the file appears
- Q43.4** · Obj 6.5 · Easy · Single choice Which call enables `USD_CHANGES` in an already-running interpreter? A. `os.environ["TF_DEBUG"] = "USD_CHANGES"` · B. `Tf.Debug.SetDebugSymbolsByName("USD_CHANGES", True)` · C. `stage.SetDebug("USD_CHANGES")` · D. `UsdUtils.EnableDebug()`
- Q43.5** · Obj 6.5 · Easy · Single choice `GetDebugSymbolDescription("USD_CHANGES")` is: A. "USD change processing" · B. "composition" · C. "" · D. `None` until enabled
- Q43.6** · Obj 6.5 · Medium · Select two. `UsdUtils.CoalescingDiagnosticDelegate`: A. Starts capturing on construction · B. Must be `Apply`'d on the root prim · C. `TakeUncoalescedDiagnostics` returns each message · D. Lives in `Tf`
- Q43.7** · Obj 6.5 · Medium · Single choice A captured missing-reference diagnostic's `diagnosticCodeString` is: A. `TF_DIAGNOSTIC_CODING_ERROR_TYPE` · B. `TF_DIAGNOSTIC_WARNING_TYPE` · C. `TF_DIAGNOSTIC_STATUS_TYPE` · D. `PCP_ERROR`
- Q43.8** · Obj 6.5 · Easy · Single choice Trace lives in: A. `UsdUtils` · B. `Tf` · C. `Trace` · D. `Usd`
- Q43.9** · Obj 6.5 · Medium · Single choice `DefaultTrace.Collector().enabled` is: A. `True` · B. `False` · C. `None` until `Initialize` · D. `True` in `usdview` only
- Q43.10** · Obj 6.5 · Easy · Single choice `Tf.MallocTag` measures: A. Time · B. Bytes allocated (when tagging is compiled in) · C. Prim count · D. Layer count
- Q43.11** · Obj 6.5 · Medium · Single choice `usd-core 26.8` after `MallocTag.Initialize()`: `GetTotalBytes()` is typically: A. The size of the crate file · B. 0 · C. Always an exception · D. -1



**Q43.12** · Obj 6.5 · Medium · Select two. A 26.08 Pcp error object has: A. `errorType` · B. `GetPropertyStack` · C. `rootSite` · D. `TfDebugSymbol`

**Q43.13** · Obj 6.5 · Medium · Single choice Missing file vs missing `defaultPrim`: A. Both `InvalidAssetPath` · B. `InvalidAssetPath` vs `UnresolvedPrimPath` · C. Both raise `ErrorException` · D. Both are `TfDebug` symbols

### Answers

**Q43.1** — B. Review: §43.1.

**Q43.2** — B. Review: §43.1.

**Q43.3** — C. Review: §43.1, §43.6.

**Q43.4** — B. Env var after import is too late. Review: §43.2.

**Q43.5** — A. Review: §43.2.

**Q43.6** — A, C. Module is `UsdUtils`, not `Tf`; no `Apply`. Review: §43.3.

**Q43.7** — B. Review: §43.3.

**Q43.8** — C. `from pxr import Trace`. Review: §43.4.

**Q43.9** — B. Review: §43.4.

**Q43.10** — B. Review: §43.5.

**Q43.11** — B. Tagging not in the pip wheel. Review: §43.5.

**Q43.12** — A, C. Review: §43.6.

**Q43.13** — B. Review: §43.6 / §42.4.

### USDA exercise answers

**43-A — No `ErrorException`.** The stage opens. Call `GetCompositionErrors()` (and optionally a `CoalescingDiagnosticDelegate` constructed *before* `Open`). `errorType` is `InvalidAssetPath`.

**43-B — `TF_DEBUG` must be set before the process (or before `pxr` is imported) for many symbols.** Use `Tf.Debug.SetDebugSymbolsByName("USD_CHANGES", True)` after import instead.

**43-C — No.** On `usd-core`, `Initialize` can return `False`, `IsInitialized` `True`, and bytes stay 0 because `malloc` tagging was not compiled in. Run a studio/debug build for real counts.

## CHAPTER 44

# Chapter 44 — Performance: Load and Render Times

**Exam domain:** Debugging & Troubleshooting (11%) · **Objectives:** 6.1, 6.4 · **Study day:** 12 · **Est. time:** 90 min **Prerequisites:** Ch 17 (payloads), Ch 24–26 (instancing), Ch 31 (USDA/USDC), Ch 34 (ChangeBlock, notices), Ch 39 (extent, purpose), Ch 42–43 (measure before you guess)

A “slow USD” ticket is usually one of five things: **too much I/O** (ASCII USDA, no payloads), **too many prims in memory** (no instancing, no mask), **too many notices** (edits outside a ChangeBlock), **too much to draw** (no purpose/extent/drawMode), or **recomposing the world** on every keystroke. Obj 6.1 is ChangeBlock and batched edits. Obj 6.4 is the visual side of speed (culling, purpose, extents). This chapter is the checklist you run after Chapter 42 found that the *data* is correct.

Do not optimize from folklore. Trace (Chapter 43) times a step; then pick the knob below that matches the step.

## Learning goals

- Name the usual load-time vs render-time costs and which file/stage APIs address each.
- Open with `LoadNone`, `Load`, `FindLoadable`, and `OpenMasked`.
- Explain why native instancing cuts memory and why `Traverse()` looks “too small.”
- Batch edits in `Sdf.ChangeBlock` and count `ObjectsChanged` notices (Obj 6.1).
- Author extent, purpose, and `model:drawMode` so Hydra can skip work (Obj 6.4).
- Walk a performance checklist without flattening first.

## Key terms

Term	One-line definition
<b>USDC / crate</b>	Binary USD; faster and smaller than USDA for big geometry.
<b>Payload</b>	A lazy reference; unloaded until <code>Load</code> .
<b>Load rules</b>	Which payloads are in memory ( <code>LoadAll</code> / <code>LoadNone</code> / per-path).
<b>Population mask</b>	Which <i>prim paths</i> exist on the stage at all ( <code>OpenMasked</code> ).
<b>Native instancing</b>	Many instance roots share one prototype in memory.
<b>ChangeBlock</b>	Batches scene description edits so notices fire once (Obj 6.1).
<b>Extent</b>	Axis-aligned box Hydra uses to cull.
<b>Purpose</b>	<code>default</code> / <code>render</code> / <code>proxy</code> / <code>guide</code> — what a renderer includes.
<b>Draw mode</b>	Model-level Hydra shortcut: <code>bounds</code> , <code>cards</code> , <code>origin</code> , ...

## 44.1 What makes stages slow

**What is it?** **Load time** is opening layers, composing prims, and resolving assets. **Edit time** is how long a `Set` takes (notices, recomp). **Render time** is how long Hydra spends on the composed result. A slow *session* can be any mix; the fix is different for each.

**Why do we need it?** People “optimize” by flattening (which *increases* memory) or by converting to USDA (which *increases* parse time). Naming the bucket first is Obj 6.1/6.4 hygiene.

**Beginner explanation.** Opening a warehouse (load), rearranging boxes (edit), photographing the floor (render). A crate (USDC) is a packed pallet — faster to fork-lift than a pile of labelled sticky notes (USDA). Payloads are rooms you do not unlock. Instancing is one chair prototype stamped a thousand times.

*Where the analogy breaks:* flattening is photocopying every room into one giant hall. It can make *render* of a one-file delivery simpler and still make *load* of the working scene worse. Flatten is a publish tool (Chapter 34), not a speed ritual.

**Technical explanation.** Typical costs (what to measure with Trace):

Bucket	Cost	First knobs
Parse / I/O	USDA text parse, many tiny files	USDC; fewer layers; <code>usdcat --flatten</code> only at deliver
Composition	Huge prim indexes, many arcs	Payloads, masks, simpler graphs (Ch 22)
Memory	Duplicate meshes	Instancing, payloads, masks
Notices / Python	Thousands of <code>ObjectsChanged</code>	<code>Sdf.ChangeBlock</code> (Obj 6.1)
Asset resolve	Thousands of <code>Resolve</code>	Package / USDZ; search-path hygiene (Ch 33)
Draw	Every mesh, every frame	<code>purpose</code> , <code>extent</code> , draw modes, vis

- `Sdf.FileFormat.FindByExtension("usda").formatId` is `usda`; `"usdc"` is `usdc`. The `"usd"` id is a wrapper: `CreateNew("x.usd")` still has `formatId usd`, but `GetUnderlyingFormatForLayer` is `usdc` and the file starts with magic PXR-USDC (Chapter 31).
- Session layer + many sublayers is fine; **hundreds of sublayers** that each delta one prim is not (composition cost).
- `GetUsedLayers()` size is a cheap “how many files did I actually open?” probe.

### Mental model.

LOAD	parse USDC/USDA → compose (Pcp) → resolve assets
EDIT	author spec → notices → maybe recompose
DRAW	traverse imageable prims → cull extent/purpose → shade

Ask which bar is long (Trace) before turning a knob.

**Simple example.** A 20 MB USDA mesh file takes longer to `Stage.Open` than the same data as USDC. The composed prim count is identical. Switching format fixes **load**, not **draw**.

**USDA example.** A working-scene pattern that stays fast to open: payloads for heavy geo, USDA only for the shot list.

```
#usda 1.0
(
 defaultPrim = "World"
)

def Xform "World"
{
 def "City" (
 prepend payload = @./city.usdc@
)
 {
 }

 def Camera "ShotCam"
 {
 }
```

```
}
}
```

- The shot file is tiny USDA (diffable). The city is crate, behind a payload.

### Python example.

```
from pxr import Sdf

print("usda id:", Sdf.FileFormat.FindByExtension("usda").formatId)
print("usdc id:", Sdf.FileFormat.FindByExtension("usdc").formatId)
print("usd id:", Sdf.FileFormat.FindByExtension("usd").formatId)
layer = Sdf.Layer.CreateNew("tmp.usd")
print("CreateNew .usd formatId:", layer.GetFileFormat().formatId)
usd_fmt = Sdf.FileFormat.FindByExtension("usd")
print("underlying:", usd_fmt.GetUnderlyingFormatForLayer(layer))
layer.Save()
with open("tmp.usd", "rb") as fh:
 print("magic:", fh.read(8))
```

### Expected output

```
usda id: usda
usdc id: usdc
usd id: usd
CreateNew .usd formatId: usd
underlying: usdc
magic: b'PXR-USDC'
```

.usd without a format hint is **crate** on disk (magic PXR-USDC). The format *id* stays usd; ask `GetUnderlyingFormatForLayer` or read the bytes. That is a load-time win and a diff-time loss — pick on purpose (Chapter 31).

**Real-world use case.** A games exporter wrote every prop as USDA “so we can git diff.” Opening a level took 40 s. Switching published geometry to USDC (keeping shot layers USDA) cut open to 6 s. Git still diffs the shot; geometry diffs happen in DCC, not in text.

### Common mistakes.

#### MISTAKE

Flattening the working scene to “make it faster.” You pay composition *once* at flatten, then carry a giant file forever.

#### MISTAKE

Measuring `ExportToString`` time and calling it load time. That is ASCII serialize, the opposite direction.

#### MISTAKE

Optimizing draw when Trace says `Sdf.Layer.Open`` is 90% of the hitch.

### Exam traps.

#### TRAP

USDA is always faster because it is smaller to think about. Binary crate is faster to **parse**.

#### TRAP

`.usd`` means USDA. Default **crate** (`PXR-USDC``); `formatId` is still the wrapper `usd``.

#### TRAP

“More sublayers always slower than one file.” A few workstream sublayers are the design (Ch 22). Hundreds of tiny deltas are the smell.

### Exam takeaways.

**KEY**

- Split load / edit / draw before changing files. - USDC for heavy geometry; USDA for small authored shots. - ``.usd`` wraps crate (`formatId` `usd``, underlying `usdc``). - Trace the slow bar; then pick the matching section below.

**44.2 USDC, payloads, masks, load rules**

**What is it?** Three *stage population* knobs: **payloads** (lazy arcs), **load rules** (`LoadAll` / `LoadNone` / `Load(path)`), and **population masks** (`OpenMasked`) that omit prims entirely. USDC is the file-format partner: payload targets should be crate.

**Why do we need it?** Opening a city to edit one shop should not compose every brick. `LoadNone` keeps payload *roots* so you can pick which to load. A mask is stricter: prims outside the mask **do not exist** on that stage.

**Beginner explanation.** Payloads are locked rooms; load rules are which keys you brought; a population mask is a floor plan that omits whole wings from the building. References are open doors — they always load (Chapter 42.2).

*Where the analogy breaks:* `FindLoadable()` lists payload paths, not references. Masking `/World/A` also drops `/World/B` and referenced `/World/C`.

**Technical explanation.** - `Usd.Stage.Open(path)` defaults to `LoadAll`. `Open(path, Usd.Stage.LoadNone)` leaves payloads unloaded; **references still compose** (verified). - `stage.FindLoadable()` → payload prim paths (`/World/A`, `/World/B`). `stage.Load("/World/A")` / `Unload` / `LoadAndUnload`. - `stage.GetLoadRules()` → `Usd.StageLoadRules`. `GetEffectiveRuleForPath` after an explicit `Load` is `AllRule`. Other rules: `NoneRule`, `OnlyRule`, `LoadWithDescendants` / `WithoutDescendants`. - **`Usd.Stage.OpenMasked(path, mask)`** with `Usd.StagePopulationMask()` + `mask.Add("/World/A")`. `TraverseAll` is `/World`, `/World/A`, and descendants. `/World/B` is **invalid**. This is not unload — the prims were never populated. - Payload targets as **USDC** give you lazy *and* fast parse when you do load them. - Masks are for tools (a sim that only needs `/World/Chars`). Load rules are for interactive working sets (`usdview` load/unload).

**VERSION**

Verified on USD 26.08. `Open(path, mask)` is not a valid overload; use `**OpenMasked**`. `Open(path, Usd.Stage.LoadNone)` is valid.

**Mental model.**

<code>LoadNone</code>	A, B exist, unloaded	C (reference) fully there
<code>Load(A)</code>	A populated	B still unloaded
<code>OpenMasked{A}</code>	only <code>/World/A</code> tree	B and C are not on the stage

**Simple example.** Shot with payloads A, B and reference C. `LoadNone`: A/B unloaded, C has `/Hi`. `FindLoadable` = A, B. `Load(A)` brings `/World/A/Hi`. `OpenMasked {/World/A}`: B invalid, C invalid.

**USDA example.**

```
#usda 1.0

def Xform "World"
{
 def "A" (
 prepend payload = @./geo.usdc@
)
 {
 }

 def "B" (
 prepend payload = @./geo.usdc@
)
 {
 }
}
```

```

 }

 def "C" (
 prepend references = @./geo.usdc@
)
 {
 }
}

```

- Same target file; payload vs reference decides *when* it costs memory.

### Python example.

```

from pxr import Usd

open("geo.usda", "w").write("""#usda 1.0
(
 defaultPrim = "P"
)
def Xform "P"
{
 def Mesh "Hi"
 {
 }
 def Mesh "Lo"
 {
 }
}
""")
open("shot.usda", "w").write("""#usda 1.0
def Xform "World"
{
 def "A" (
 prepend payload = @./geo.usda@
)
 {
 }
 def "B" (
 prepend payload = @./geo.usda@
)
 {
 }
 def "C" (
 prepend references = @./geo.usda@
)
 {
 }
}
""")

stn = Usd.Stage.Open("shot.usda", Usd.Stage.LoadNone)
print("LoadNone traverseAll:",
 [str(p.GetPath()) for p in stn.TraverseAll()])
print("FindLoadable:", sorted(str(p) for p in stn.FindLoadable()))
stn.Load("/World/A")
print("after Load A/Hi:", stn.GetPrimAtPath("/World/A/Hi").IsValid(),
 "B loaded:", stn.GetPrimAtPath("/World/B").IsLoaded())
print("rule A:", stn.GetLoadRules().GetEffectiveRuleForPath("/World/A"))

mask = Usd.Stage.PopulationMask()
mask.Add("/World/A")
stm = Usd.Stage.OpenMasked("shot.usda", mask)
print("masked traverseAll:",
 [str(p.GetPath()) for p in stm.TraverseAll()])
print("masked B valid:", stm.GetPrimAtPath("/World/B").IsValid())

```

### Expected output

```

LoadNone traverseAll: ['/World', '/World/A', '/World/B', '/World/C', '/World/C/Hi', '/World/C/Lo']
FindLoadable: ['/World/A', '/World/B']
after Load A/Hi: True B loaded: False
rule A: Usd.StageLoadRules.AllRule

```

```
masked traverseAll: ['/World', '/World/A', '/World/A/Hi', '/World/A/Lo']
masked B valid: False
```

C's children appear under LoadNone because C is a **reference**. Masking /World/A drops B and C.

**Real-world use case.** A layout DCC opens the shot with LoadNone, then loads only /World/Set/Building\_12 when the artist selects it. A crowd sim uses OpenMasked on /World/Agents so character caches never even compose the set.

### Common mistakes.

#### MISTAKE

Using references for hero buildings you wanted lazy. Switch those arcs to **\*\*payloads\*\***.

#### MISTAKE

``Open(path, mask)`` — that overload does not exist. **\*\*`OpenMasked`\*\***.

#### MISTAKE

Expecting ``FindLoadable()`` to list referenced prims. It lists **\*\*payloads\*\***.

### Exam traps.

#### TRAP

``LoadNone`` unloads references. It does not.

#### TRAP

Masked-out prims are unloaded payloads. They are **\*\*not on the stage\*\*** (``IsValid` False`).

#### TRAP

``Load("/World/A")`` loads B too. It does not; rules are per path (with descendant policy).

### Exam takeaways.

#### KEY

- Payloads + ``LoadNone`` / ``Load`` = working set. References always compose. - ``OpenMasked`` + ``StagePopulationMask`` omits prims entirely. - ``FindLoadable`` = payload paths. ``GetLoadRules`` for the policy. - Put heavy payload targets in USDC.

## 44.3 Instancing for memory and speed

**What is it? Native scenegraph instancing** (`instanceable = true` on a referencing prim) stores descendant specs **once** on a **prototype**. A thousand chairs share /Prototype\_1. Draw and memory scale with unique prototypes, not with instance count (Chapter 24).

**Why do we need it?** Duplicating a 50k-face mesh 2 000 times without instancing is the classic “stage is 12 GB and usdview crawls” bug. Point instancers (Chapter 25) are the next step for tens of thousands of cheap copies.

**Beginner explanation.** A rubber stamp vs 2 000 carved copies. The stamp is the prototype. `Traverse()` visits stamp handles, not every carved groove — that is why the list looks short, not why the chairs are missing (Chapter 42.2).

*Where the analogy breaks:* instance roots can still have local overs (translate each chair). Overs **below** the root are ignored. That is a correctness constraint, not a perf bug.

**Technical explanation.** - Author `instanceable = true` on the prim that **references** (or inherits) the shared asset. `prim.IsInstance()` True; descendants `IsInstanceProxy()` True. - `prim.GetPrototype().GetPath()` → `/__Prototype_1` (name assigned by the stage). `GetMaster` does not exist (26.08). - `Traverse()`: instance roots yes, interior proxies no.

Hydra still *draws* the proxies. - Cost: unique prototypes × size. 2 000 identical refs → ~1 prototype. 2 000 slightly different meshes (unique local topology) → no sharing. - PointInstancer: one prototype relationship + per-instance arrays (positions, scales, ...). Even cheaper when you do not need a prim per copy. - Instancing does not replace payloads. Instance a payload root: many stamps, still lazy.

#### VERSION

Verified on USD 26.08: `GetPrototype()`, not `GetMaster()`.

#### Mental model.

```
/World/A instanceable ┌
/World/B instanceable ┤ /__Prototype_1 /S
Traverse: /Proto /Proto/S /World /World/A /World/B
 (no /World/A/S)
GetPrimAtPath(/World/A/S) still valid (proxy)
```

**Simple example.** Two instanceable refs of `/Proto`. Traverse has five prims, not seven. Prototype path `/__Prototype_1`.

#### USDA example.

```
#usda 1.0

def Xform "Proto"
{
 def Sphere "S"
 {
 }
}

def Xform "World"
{
 def "A" (
 instanceable = true
 prepend references = </Proto>
)
 {
 }

 def "B" (
 instanceable = true
 prepend references = </Proto>
)
 {
 }
}
```

#### Python example.

```
from pxr import Usd

open("inst.usda", "w").write("""#usda 1.0
def Xform "Proto"
{
 def Sphere "S"
 {
 }
}
def Xform "World"
{
 def "A" (
 instanceable = true
 prepend references = </Proto>
)
 {
 }
 def "B" (
 instanceable = true
 prepend references = </Proto>
)
}
```



```

 {
 }
}
"""
si = Usd.Stage.Open("inst.usda")
print("traverse:", [str(p.GetPath()) for p in si.Traverse()])
print("A instance:", si.GetPrimAtPath("/World/A").IsInstance())
print("A/S proxy:", si.GetPrimAtPath("/World/A/S").IsInstanceProxy())
print("prototype:",
 si.GetPrimAtPath("/World/A").GetPrototype().GetPath())

```

### Expected output

```

traverse: ['/Proto', '/Proto/S', '/World', '/World/A', '/World/B']
A instance: True
A/S proxy: True
prototype: /__Prototype_1

```

Five traverse hits, two drawn spheres. That is the memory win.

**Real-world use case.** A stadium of identical seats: one `seat.usdc` payload, instanceable refs per row, or one `PointInstancer` for the bowl. Layout still translates instance roots. Lighting binds on the prototype (or uses collection binds) rather than 40 000 unique materials.

### Common mistakes.

#### MISTAKE

Setting ``instanceable`` on the prototype asset's *\*root in its own file\** only, and referencing without ``instanceable`` on the *\*shot prim\**. The shot prim must be instanceable.

#### MISTAKE

Unique ``displayColor`` local on every proxy child. That de-instances or is ignored. Color at the instance root / primvar on the prototype.

#### MISTAKE

Counting ``Traverse()`` length as "draw prims." Proxies are drawn, not traversed.

### Exam traps.

#### TRAP

``GetMaster()``. *\*\*`GetPrototype()`*.

#### TRAP

Instancing is a payload. Different knobs; they compose.

#### TRAP

`PointInstancer`` is native instancing. Related, different schema (Ch 25).

### Exam takeaways.

#### KEY

- Native instancing: one prototype, many roots; ``GetPrototype`` not ``GetMaster``. - ``Traverse`` skips proxies; Hydra does not. - Instanceable flag belongs on the copies. - `PointInstancer`` when you do not need a prim per copy.

## 44.4 `Sdf.ChangeBlock`` and batched edits

**What is it?** `Sdf.ChangeBlock`` is a context manager that **holds USD notices** until the block exits. A hundred `Set`s` inside one block send **one** `ObjectsChanged`` instead of hundreds. That is Obj 6.1.

**Why do we need it?** Every notice can trigger Hydra sync, Python listeners, and recomposition. An importer that sets 50 000 points one attribute at a time without a block stutters for minutes; the same writes in a block finish in a beat (Chapter 34).

**Beginner explanation.** A change block is “hold your applause until the end of the scene.” USD still writes every spec immediately into the layer; it just does not *announce* each one.

*Where the analogy breaks:* `Usd.DefinePrim` of a **child that needs a live parent** can fail inside a block (Chapter 34). Author with `Sdf.PrimSpec` or `Define` parents *before* the block. Attribute Sets on existing prims are the happy path.

**Technical explanation.** - with `Sdf.ChangeBlock()`: ... edits ... Notices (`Usd.Notice.ObjectsChanged`) flush on exit. - Verified pattern: three `CreateAttribute+Set` pairs **without** a block → **6** notices; the same **with** a block → **1**. - Register: `key = Tf.Notice.Register(Usd.Notice.ObjectsChanged, callback, stage)` and `key.Revoke()` (not `Tf.Notice.Revoke`). - Nested blocks coalesce to the outermost exit. - `ChangeBlock` is **not** a transaction: no automatic undo. Failed Python in the block still leaves specs; notices still fire on exit. - Combine with USDC writes: author in-memory, one `Export`. Do not `Export` per prim.

#### VERSION

Verified on USD 26.08. ``listener.Revoke()` / `key.Revoke()``; ``DefinePrim`` of new children inside a block may fail — use `Sdf` specs.

#### Mental model.

without block:	Set	Set	Set	→	notice	notice	notice
with block:	Set	Set	Set	→	.....	notice	

**Simple example.** Six attribute creates/sets → 6 notices. Wrap in `ChangeBlock` → 1.

**USDA example.** USDA is the *result*, not the block. After a blocked author of three ints:

```
#usda 1.0

def Xform "W"
{
 int a = 1
 int b = 2
 int c = 3
}
```

The block never appears in the file.

#### Python example.

```
from pxr import Tf, Usd, Sdf, UsdGeom

class Counter:
 def __init__(self):
 self.n = 0

 def __call__(self, notice, sender):
 self.n += 1

stage = Usd.Stage.CreateInMemory()
key_holder = []
lis = Counter()
key = Tf.Notice.Register(Usd.Notice.ObjectsChanged, lis, stage)
prim = UsdGeom.Xform.Define(stage, "/W").GetPrim()
lis.n = 0
prim.CreateAttribute("a", Sdf.ValueTypeNames.Int).Set(1)
prim.CreateAttribute("b", Sdf.ValueTypeNames.Int).Set(2)
prim.CreateAttribute("c", Sdf.ValueTypeNames.Int).Set(3)
print("without block:", lis.n)
lis.n = 0
```

```
with Sdf.ChangeBlock():
 prim.CreateAttribute("d", Sdf.ValueTypeNames.Int).Set(4)
 prim.CreateAttribute("e", Sdf.ValueTypeNames.Int).Set(5)
 prim.CreateAttribute("f", Sdf.ValueTypeNames.Int).Set(6)
print("with block:", lis.n)
key.Revoke()
```

### Expected output

```
without block: 6
with block: 1
```

Obj 6.1 in two numbers.

**Real-world use case.** An Alembic→USD converter used to set each of 200 000 points time samples without a block; usdview's listener hung the DCC. Wrapping the sample loop in `ChangeBlock` dropped the hitch to one refresh.

### Common mistakes.

#### MISTAKE

Defining new child prims with ``Usd.DefinePrim`` inside the block. Prefer ``Sdf.PrimSpec`` on the target layer.

#### MISTAKE

Forgetting ``key.Revoke()``. Listeners leak; later tests count extra notices.

#### MISTAKE

Nested ``Export`` inside the block. Flush the block first, then write the file.

### Exam traps.

#### TRAP

`ChangeBlock` delays *\*writes\** to the layer. It delays *\*\*notices\*\**; specs are already in the layer.

#### TRAP

``Tf.Notice.Revoke(key)``. Call *\*\*`key.Revoke()`*.

#### TRAP

One ``Set`` always one notice. Not inside a block; also some ``Set``'s send two (Ch 34).

### Exam takeaways.

#### KEY

- Obj 6.1: ``with Sdf.ChangeBlock():`` around bulk edits. - Verified: 6 notices → 1 for three create+set pairs. - ``key.Revoke()``; avoid ``Usd.DefinePrim`` of new children inside the block.
- Specs write immediately; applause waits.

## 44.5 Render-time factors (extent, purpose, draw modes)

**What is it?** Hydra's cheap rejections: **extent** (frustum cull), **purpose** (do not even consider guide/proxy in a beauty pass), **visibility**, and **model draw modes** (bounds / cards / origin) that replace a component's full mesh with a box or camera-facing cards. Obj 6.4 includes these as causes of both *wrong* pictures and *slow* pictures.

**Why do we need it?** A correct 12-million-face set that is all `purpose = default` with empty extents forces Storm to bound and shade everything. Authoring extents and a proxy purpose is often faster than instancing.

**Beginner explanation.** Extent is the moving-box on a truck so the dock can skip opening it. Purpose is a sticker: proxy for the lobby snapshot, render for the catalog, guide for riggers. Draw mode cards is a cardboard cut-out of the truck when you only need the silhouette.

*Where the analogy breaks:* unauthored extent is not “infinite.” Many delegates compute a box (slow) or skip culling (also slow). Unauthored subdivisionScheme is a *look* bug (melted CAD) that also costs tessellation.

**Technical explanation.** - **Extent:** `UsdGeom.Boundable extent (float3[] min/max)`. `HasAuthoredValue()` False means every consumer may recompute. Keep it in sync when you edit points (Ch 13). `UsdGeom.ModelAPI.GetExtentsHint / SetExtentsHint` stores a *model-level* hint for draw modes. - **Purpose:** `Imageable.ComputePurpose()`. Beauty: include default + render. Layout: often proxy. Guides hidden in both. Inherited (Ch 39.6, 42.6). - **Visibility:** invisible prunes descendants. Prefer purpose for “this is a stand-in,” visibility for animated hide. - **Draw modes** (`UsdGeom.ModelAPI` on a model kind): `model:drawMode` fallback **inherited**. Values: default (full geo), bounds, origin, cards. `model:applyDrawMode` True to actually switch. `ComputeModelDrawMode()` resolves inheritance. Cards use `model:cardTextureXPos` etc. - Apply `ModelAPI` (or rely on built-in for some schemas) on **component** models (kind = component). - Lights: too many unbound shadow-casting `DiskLights` (Ch 41) is a render-time cost; `inputs:shadow:enable = 0` on fills.

#### VERSION

```
Verified on USD 26.08. `GetModelDrawModeAttr().Get()` unauthored → `**inherited`.
`CreateModelDrawModeAttr("cards")` + `CreateModelApplyDrawModeAttr(True)` →
`ComputeModelDrawMode()` `**cards`.
```

#### Mental model.

```
per gprim: extent? purpose? visibility?
per model: drawMode bounds/cards/origin applyDrawMode
Hydra: skip → cheap bounds → cards → full mesh
```

**Simple example.** Component /Asset with `ModelAPI`, `drawMode = cards`, `applyDrawMode = true`. `ComputeModelDrawMode` returns `cards`. Unauthored draw mode is `inherited`.

#### USDA example.

```
#usda 1.0

def Xform "Asset" (
 kind = "component"
 prepend apiSchemas = ["GeomModelAPI"]
)
{
 uniform token model:drawMode = "cards"
 uniform bool model:applyDrawMode = 1
 float3[] extentsHint = [(-1, 0, -1), (1, 2, 1)]

 def Mesh "Hi"
 {
 uniform token purpose = "render"
 float3[] extent = [(-1, 0, -1), (1, 2, 1)]
 }

 def Mesh "Lo"
 {
 uniform token purpose = "proxy"
 float3[] extent = [(-1, 0, -1), (1, 2, 1)]
 }
}
```

- Layout cameras that include proxy draw Lo (or cards). Beauty uses Hi.
- `GeomModelAPI` is the schema name in USDA for `UsdGeom.ModelAPI`.

**Python example.**

```

from pxr import Usd, UsdGeom

stage = Usd.Stage.CreateInMemory()
xf = UsdGeom.Xform.Define(stage, "/Asset")
xf.GetPrim().SetMetadata("kind", "component")
api = UsdGeom.ModelAPI.Apply(xf.GetPrim())
print("fallback drawMode:", api.GetModelDrawModeAttr().Get())
api.CreateModelDrawModeAttr("cards")
api.CreateModelApplyDrawModeAttr(True)
print("mode:", api.GetModelDrawModeAttr().Get())
print("apply:", api.GetModelApplyDrawModeAttr().Get())
print("ComputeModelDrawMode:", api.ComputeModelDrawMode())
print("HasAPI:", xf.GetPrim().HasAPI(UsdGeom.ModelAPI))

```

**Expected output**

```

fallback drawMode: inherited
mode: cards
apply: True
ComputeModelDrawMode: cards
HasAPI: True

```

**Real-world use case.** A city layout set ships proxy box meshes plus render hi meshes, extents on both, and component drawMode cards for helicopter shots. Storm stays interactive; the beauty delegate never sees the boxes.

**Common mistakes.****MISTAKE**

Setting `drawMode` without `applyDrawMode`. Hydra keeps drawing full geo.

**MISTAKE**

Stale extent after a points edit. Culls wrong (objects pop) — Obj 6.4.

**MISTAKE**

Putting `purpose = render` on the only mesh, then wondering why layout cameras that exclude render are empty.

**Exam traps.****TRAP**

Unauthored `model:drawMode` is `default`. It is **inherited**.

**TRAP**

Draw mode is a Mesh attribute. It is **ModelAPI** on the model root.

**TRAP**

Imageable purpose vs material purpose vs draw mode — three switches (Ch 39, 40, this section).

**Exam takeaways.****KEY**

- Obj 6.4 speed: extent, purpose, visibility, draw modes. - ModelAPI `drawMode` fallback `inherited`; set `applyDrawMode`. - Proxy/render pairs beat sending hi meshes to layout. - Stale extent is both a wrong picture and a slow picture.

## 44.6 Maximizing USD performance checklist

**What is it?** A **fixed order** of performance questions, parallel to Chapter 42.7’s correctness procedure. You measure (Trace / MallocTag / prim counts), then walk the list. Pixar’s “Maximizing USD Performance” essay is the spirit; the items below are the APIs this book verified.

**Why do we need it?** Random instancing of a 12-prim shot wastes time. The checklist stops you from applying §44.3 when §44.2 was the bug (you referenced instead of payloaded).

**Beginner explanation.** Same medic order as 42.7, but the vital signs are **time and bytes**: how many files, how many loaded payloads, how many notices per edit, how many imageable prims Hydra will visit.

*Where the analogy breaks:* a scene can be correct *and* slow. Chapter 42 passing does not skip this list.

**Technical explanation. Checklist (stop when the measured bar drops):**

1. **Measure.** `Trace.Collector` around `Stage.Open`, around your edit loop, around export. `MallocTag` on a tagged build. `len(list(stage.Traverse()))` vs `TraverseAll`.
2. **Format.** Heavy layers USDC; shot/work USDA. `.usd` is crate unless you say otherwise.
3. **Working set.** Payloads for heavy roots; `LoadNone` + `Load` / `FindLoadable`. Masks for tools that only need a subtree (`OpenMasked`).
4. **Instancing / PointInstancer.** Shared geometry gets `instanceable` copies or a `PointInstancer`.
5. **Edit batching.** `Sdf.ChangeBlock` around importer loops (Obj 6.1). Revoke listeners.
6. **Assets.** `ComputeAllDependencies` unresolved empty; avoid cwd-relative resolves in inner loops (Ch 33).
7. **Draw.** Extents authored; purpose split; ModelAPI draw modes for layout; lights’ shadow flags; `subdivisionScheme = none` on CAD (also Obj 6.4 correctness).
8. **Do not flatten the working scene.** Flatten/USDZ at *publish* (Ch 31, 34).

usdview’s **Memory / Hydra** debug (not in `usd-core`) visualizes 3–4 and 7. Python above is the exam-safe stand-in.

**Mental model.**

Trace which bar
Open slow → USDC, payloads, masks, fewer layers
Edit slow → ChangeBlock, fewer listeners
Draw slow → extent, purpose, instance, drawMode
RAM high → instance, payloads, masks (MallocTag if tagged)

**Simple example.** `LoadNone` shot: `TraverseAll` lists payload roots + referenced geo; `FindLoadable` is the working-set menu. That one print tells you whether anyone used payloads.

**USDA example.** A “fast by construction” shot:

```
#usda 1.0
(
 defaultPrim = "World"
)

def Xform "World"
{
 def "Set" (
 prepend payload = @./set.usdc@
)
 {
 }
}
```

```

def "Hero" (
 instanceable = true
 prepend payload = @./hero.usdc@
)
{
}
}

```

- Crate payloads, instanceable hero, tiny USDA shot.

### Python example.

```

from pxr import Usd, Sdf

open("geo.usda", "w").write("""#usda 1.0
(
 defaultPrim = "P"
)
def Xform "P"
{
 def Mesh "Hi"
 {
 }
}
""")
open("shot.usda", "w").write("""#usda 1.0
def Xform "World"
{
 def "Set" (
 prepend payload = @./geo.usda@
)
 {
 }
 def "Hero" (
 instanceable = true
 prepend payload = @./geo.usda@
)
 {
 }
}
""")
st = Usd.Stage.Open("shot.usda", Usd.Stage.LoadNone)
print("format usd default:",
 Sdf.FileFormat.FindByExtension("usd").formatId)
print("traverseAll:", [str(p.GetPath()) for p in st.TraverseAll()])
print("loadable:", sorted(str(p) for p in st.FindLoadable()))
print("Hero instanceable:",
 st.GetPrimAtPath("/World/Hero").GetMetadata("instanceable"))
print("Set loaded:", st.GetPrimAtPath("/World/Set").IsLoaded())

```

### Expected output

```

format usd default: usd
traverseAll: ['/World', '/World/Set', '/World/Hero']
loadable: ['/World/Hero', '/World/Set']
Hero instanceable: True
Set loaded: False

```

Both heavy prims are payloads and unloaded; Hero is ready to instance when loaded. That is a passing score on steps 2–4 before anyone hits Render.

**Real-world use case.** A studio wiki page titled “Maximizing USD Performance” is this checklist plus two commands: `TF_DEBUG=USD_CHANGES` for surprise recomposes, and `Trace around Open`. New shows copy the shot template in §7 rather than debating crate vs USDA on Slack.

### Common mistakes.

#### MISTAKE

Instanting first on a scene whose Open is 90% USDA parse. Format/payloads first.

**MISTAKE**

Population-masking the artist's working stage so they cannot see the set. Masks are for **tools**; load rules are for **people**.

**MISTAKE**

Publishing flattened crates **and** expecting payloads to still be lazy. Flatten already pulled them in.

**Exam traps.****TRAP**

The performance essay's first item is flatten. For a **working** scene it is near the last.

**TRAP**

``LoadNone`` as a render optimization for beauty. Beauty must ``Load`` (or ``LoadAll``) the payloads it needs; masks/purpose then cut draw.

**TRAP**

`ChangeBlock` speeds ``Stage.Open``. It speeds **edits**, not open.

**Exam takeaways.****KEY**

- Measure (Trace) → format → payloads/mask → instance → `ChangeBlock` → draw. - Working set = load rules; tools = masks; publish = flatten/USDZ. - Obj 6.1 is `ChangeBlock`; Obj 6.4 is extent/purpose/draw/subdivision. - Do not flatten to "go faster" while still authoring.

**Chapter lab(s)**

**Lab 30 — Change notices and change-block performance** (★★☆, Obj 6.1, Ch 34 and 44). You count `ObjectsChanged` with and without `Sdf.ChangeBlock`.

**Lab 23 — Native instancing** and **Lab 15 — payloads / load rules** cover §44.3 and §44.2. There is no separate "perf" lab; the checklist is the exercise.

**USDA reading exercise(s)**

**Exercise 44-A.** A DCC opens this shot with `LoadNone` to be fast. The artist cannot see `/World/Hall/Wall`. Why, and is the arc type the problem?

```
#usda 1.0

def Xform "World"
{
 def "Hall" (
 prepend references = @./hall.usdc@
)
 {
 }
}
```

**Exercise 44-B.** An importer sets `points` 10 000 times. `usdview` freezes until it finishes. Which Obj 6.1 tool, and what must wrap the loop?

**Exercise 44-C.** Layout is slow; beauty is fine. Hi meshes have `purpose = default`, no proxy, no extents, `model:drawMode` `unauthored`. Name three authors that would speed layout without changing beauty.



## Chapter review

### Summary

- Load vs edit vs draw. USDC for heavy data; USDA for small shots; `.usd` defaults crate.
- Payloads + `LoadNone/Load/FindLoadable`; `OpenMasked` omits prims; references always load.
- Instancing: `GetPrototype`, `Traverse` skips proxies.
- `ChangeBlock`: 6 notices → 1 for three create+set pairs (Obj 6.1).
- Draw: extent, purpose, `ModelAPI` cards/bounds + `applyDrawMode` (Obj 6.4).
- Checklist: measure, then format, working set, instance, batch, assets, draw; flatten at publish.

### If you see... → think...

If you see...	Think...
Open slow, tiny prim count	USDA parse / too many files
<code>.usd</code>	Wrapper id <code>usd</code> ; disk magic <b>PXR-USDC</b>
<code>LoadNone</code> still heavy	You used <b>references</b>
<code>FindLoadable</code> empty	No payloads
<code>Open(path, mask)</code> <code>TypeError</code>	<b><code>OpenMasked</code></b>
Masked prim <code>IsValid</code> <code>False</code>	Not unloaded — <b>absent</b>
RAM huge, many identical refs	Missing <code>instanceable</code>
<code>GetMaster</code>	<b><code>GetPrototype</code></b>
Edit hitch, Open fine	<code>ChangeBlock</code> / notices
Layout slow, beauty OK	purpose / <code>drawMode</code> / extents
<code>drawMode</code> ignored	Forgot <code>applyDrawMode</code>
Flatten to go faster	Publish tool, not a working-set tool

### Review questions

**Q44.1** · Obj 6.1 · Easy · Single choice `CreateNew("shot.usd")` writes crate bytes. `FindByExtension("usd").formatId` is still: A. `usda` · B. `usdc` · C. `usd` · D. None

**Q44.2** · Obj 6.1 · Easy · Single choice `Stage.Open(path, Usd.Stage.LoadNone)` still fully composes: A. Payloads · B. References · C. Nothing · D. Only the session layer

**Q44.3** · Obj 6.1 · Medium · Single choice Which call opens a stage that *omits* prims not under `/World/A`? A. `Open(path, LoadNone)` then `Unload` · B. `OpenMasked(path, mask)` with `mask.Add("/World/A")` · C. `Open(path, mask)` · D. `MuteLayer`

**Q44.4** · Obj 6.1 · Easy · Single choice `FindLoadable()` lists: A. Every prim · B. Payload paths · C. Referenced paths · D. Muted layers

**Q44.5** · Obj 6.1 · Medium · Single choice Native instance prototype accessor in 26.08: A. `GetMaster()` · B. `GetPrototype()` · C. `GetInstance()` · D. `GetProxy()`

**Q44.6** · Obj 6.1 · Medium · Single choice Three `CreateAttribute+Set` pairs: notices without vs with `ChangeBlock` (this book's measurement): A. 3 and 3 · B. 6 and 1 · C. 1 and 1 · D. 6 and 6

**Q44.7** · Obj 6.1 · Easy · Single choice Unregister an `ObjectsChanged` listener with: A. `Tf.Notice.Revoke(key)` · B. `key.Revoke()` · C. `del key` · D. `stage.Unsubscribe(key)`

**Q44.8** · Obj 6.4 · Easy · Single choice Unauthored `model:drawMode` `Get()` is: A. `default` · B. `cards` · C. `inherited` · D. None

**Q44.9** · Obj 6.4 · Medium · Select two. Layout-only speedups that should not change beauty: A. `purpose = proxy` stand-in meshes · B. Deleting render meshes from the asset · C. `model:drawMode = cards` with `applyDrawMode` · D. Flattening the lighting layer into geometry

**Q44.10** · Obj 6.4 · Easy · Single choice Hydra frustum culling wants authored: A. `kind` · B. `extent` · C. `info:id` · D. `defaultPrim`

**Q44.11** · Obj 6.1 · Medium · Single choice `ChangeBlock` speeds up: A. `Stage.Open` of USDA · B. Bulk **edits** (fewer notices) · C. Texture resolve · D. Variant selection fallbacks

**Q44.12** · Obj 6.1 · Medium · Single choice A working-scene performance checklist puts flatten: A. First · B. Instead of USDC · C. At **publish**, not while authoring · D. Never in any pipeline

### Answers

**Q44.1** — C. The wrapper id is `usd`; `GetUnderlyingFormatForLayer` is `usdc` and the magic is `PXR-USDC`. Review: §44.1.

**Q44.2** — B. Review: §44.2 / §42.2.

**Q44.3** — B. `Open(path, mask)` is not an overload. Review: §44.2.

**Q44.4** — B. Review: §44.2.

**Q44.5** — B. Review: §44.3.

**Q44.6** — B. Review: §44.4.

**Q44.7** — B. Review: §44.4 / Ch 34.

**Q44.8** — C. Review: §44.5.

**Q44.9** — A, C. Deleting hi meshes changes beauty; flatten is unrelated. Review: §44.5–44.6.

**Q44.10** — B. Review: §44.5.

**Q44.11** — B. Review: §44.4.

**Q44.12** — C. Review: §44.6.

### USDA exercise answers

**44-A** — `hall` is a *reference*, so `LoadNone` still composes `hall.usdc`. The artist should see `Wall` if the asset has `defaultPrim` (or an explicit prim path). If they wanted laziness, the arc should be a **payload**. The format `.usdc` is fine.

**44-B** — with `Sdf.ChangeBlock()`: **around the loop** (Obj 6.1). Optionally revoke extra listeners. `ChangeBlock` delays notices, not the `points` writes.

**44-C** — (1) `purpose = proxy` (or a sibling proxy mesh), (2) authored extent, (3) `ModelAPI drawMode bounds` or `cards with applyDrawMode`. Do not delete hi meshes; beauty still needs them at `purpose = render / default`.

## Part X — Python Labs

---

### LAB 01

## Lab 01 — Install and verify `usd-core`

**Domain / objectives:** Foundations · **Chapter:** F5 · **Time:** 15 min · **Difficulty:** ★☆☆

### Goal

Prove that the book's Python environment can import OpenUSD 26.08 and that the pip wheel has **no** `usdcats` / `usdview` on `PATH`.

### Background

The study environment uses `usd-core 26.8`, which is OpenUSD **26.08**. `Usd.GetVersion()` returns `(0, 26, 8)` — the leading month zero is dropped. Schema domains (`UsdGeom`, `UsdShade`, `UsdLux`) import; command-line tools and Hydra do **not** ship with this wheel (Ch F5, Ch 31, Ch 41).

### Steps

1. Activate `.venv` and run the script with `.venv/bin/python python-labs/lab01_verify_usdcore.py`.
2. Confirm the version tuple is `(0, 26, 8)`.
3. Confirm `usdcats` and `usdview` print `False`. Later labs use Python stand-ins (`ExportToString`, `UsdValidation`).

Run `python-labs/lab01_verify_usdcore.py` (script is in the repo; output below).

### Expected output

```
USD version: (0, 26, 8)
has UsdGeom: True
has UsdShade: True
has UsdLux: True
usdcats on PATH: False
usdview on PATH: False
```

### Check your understanding

1. Why is the pip version 26.8 but the release 26.08?
2. Does a missing `usdcats` mean USD is broken?
3. Which module defines `DistantLight`?

**Answers** 1. The package drops the month's leading zero; `GetVersion()` is `(0, 26, 8)`. 2. No — `usd-core` never ships CLI tools. Use Python equivalents. 3. **UsdLux**, not `UsdGeom`.

### Stretch challenge

Import `UsdImaging` and catch `ImportError`. (It should fail on this wheel.) Solution: `from pxr import UsdImaging` raises `ImportError`; that is expected (Ch 41.4).

### LAB 02

## Lab 02 — Your first stage: Xform + Sphere, save as USDA

**Domain / objectives:** Foundations · **Chapter:** 2 · **Time:** 20 min · **Difficulty:** ★☆☆

## Goal

Create a stage, define `/World` (Xform) and `/World/Ball` (Sphere), set a radius, mark `defaultPrim`, and save USDA.

## Background

A **stage** is the live composed scene (`Usd.Stage`). `CreateNew("hello.usda")` makes a file-backed root layer. `UsdGeom.Xform.Define / Sphere.Define` author typed prims.

`SetDefaultPrim` writes layer metadata so later references can omit a prim path (Ch 2, Ch 16).

The script `chdirs` to a temp directory so it does not clutter your repo. `ExportToString()` is the Python stand-in for `usdcat`.

## Steps

1. Run the script. Read the USDA it prints.
2. Note `double radius = 2` (you set `2.0`; USDA drops the trailing zero).
3. `file exists: True` means `Save()` wrote `hello.usda` in the temp dir.

Run `python-labs/lab02_first_stage.py` (script is in the repo; output below).

## Expected output

```
#usda 1.0
(
 defaultPrim = "World"
)

def Xform "World"
{
 def Sphere "Ball"
 {
 double radius = 2
 }
}

defaultPrim: World
radius: 2.0
file exists: True
```

## Check your understanding

1. What prim type is `/World`?
2. Why set `defaultPrim`?
3. `Get()` prints `2.0` but USDA shows `2`. Is the value different?

**Answers** 1. **Xform** — a grouping transform, not geometry. 2. So a reference to this file without `</World>` still finds the asset root. 3. No. Python `Get()` is `2.0`; USDA omits `.0` for whole doubles.

## Stretch challenge

Also define `/World/Ground` as a `Cube` with `size = 4`. Print its type name. Solution:

`UsdGeom.Cube.Define(stage, "/World/Ground").GetSizeAttr().Set(4)` then `GetPrim().GetTypeName()` → `Cube`.

---

## LAB 03

# Lab 03 — Open and traverse a stage

**Domain / objectives:** Data Modeling 5.4 · **Chapter:** 2 · **Time:** 20 min · **Difficulty:** ★☆☆

## Goal

Write a USDA file, `Stage.Open` it, walk prims with `Traverse()`, and look up paths with `GetPrimAtPath` (including a missing path).

## Background

`Traverse()` yields defined, active, loaded prims in hierarchy order. `GetPrimAtPath` always returns a handle — you must ask `IsValid()`. A missing path is **not** an exception (Ch 2, Ch 42.2). This lab recreates `hello.usda` itself so it does not depend on Lab 02's temp directory.

## Steps

1. Run the script.
2. Confirm two prims: `/World Xform`, `/World/Ball Sphere`.
3. Confirm `/Nope` is invalid.

Run `python-labs/lab03_open_traverse.py` (script is in the repo; output below).

## Expected output

```
root ends: hello.usda
prims:
 /World Xform
 /World/Ball Sphere
Ball valid: True type: Sphere
missing valid: False
```

## Check your understanding

1. Does `GetPrimAtPath("/Nope")` raise?
2. What does `Traverse()` skip that `TraverseAll()` would include? (Preview of Ch 42.)
3. Why `basename` the identifier?

**Answers** 1. **No**. It returns an invalid prim (`IsValid()` `False`). 2. Inactive, undefined overs, unloaded payloads, instance interiors. 3. The identifier may be an absolute path; `display/basename` stays portable.

## Stretch challenge

Print `opened.GetPrimAtPath("/World").GetChildren()` names. Solution:  
`[c.GetName() for c in ...GetChildren()] → ['Ball']`.

## LAB 04

# Lab 04 — Root layer, session layer, export to string

**Domain / objectives:** Composition 1.3 · **Chapter:** 3 · **Time:** 20 min · **Difficulty:** ★☆☆

## Goal

See that the **session layer** is strongest, is the first `GetLayerStack` entry, and is *not* the default edit target. An override there wins while the root still holds the weaker value.

## Background

Every stage has a **root layer** (the document) and a **session layer** (anonymous scratch, strongest). Default `GetEditTarget()` is the root. `SetEditTarget(session)` makes subsequent `Set` calls write on the session layer. Reopening the file would lose those edits unless you export the session yourself (Ch 3, Ch 15, Ch 42.1).

## Steps

1. Run the script. Two layers: session first, then root.
2. After targeting the session, composed radius is 9.
3. `root` still has: 1.0 — Sdf on the root layer still sees the original default. The session won composition, it did not rewrite the file.

Run `python-labs/lab04_root_session.py` (script is in the repo; output below).

### Expected output

```
n layers: 2
first is session: True
second is root: True
edit target is root: True
radius: 9.0
winning layer is session: True
root still has: 1.0
```

## Check your understanding

1. Is the session layer in `GetLayerStack(includeSessionLayers=False)`?
2. Why is the default edit target the root, not the session?
3. Would `Save()` on the root persist radius 9?

**Answers** 1. **No.** Pass `True` (and it is the default if you omit the flag — this lab passes it explicitly). 2. So ordinary `Set` calls go into the document you will save. 3. **No.** 9 lives on the session layer. The root still has 1.

## Stretch challenge

`print(stage.GetSessionLayer().ExportToString())` and find `double radius = 9`. Solution: the session layer USDA contains an `over "Ball"` (or a `def`) with radius 9.

## LAB 05

# Lab 05 — Build a sublayer stack and see who wins

**Domain / objectives:** Composition 1.1 · **Chapter:** 3, 15 · **Time:** 25 min · **Difficulty:**

★☆☆

## Goal

Author a root with two sublayers, predict the composed `radius`, then **mute** the winner to confirm which file was contributing.

## Background

`subLayers` on the root are weaker than the root itself but ordered **first-listed = stronger** among themselves (Ch 15). Muting (`MuteLayer`) pulls a layer out of composition without deleting the file — a debugging tool (Ch 42.1). Use **multiline** USDA; `def Sphere "Ball" { double radius = 1 }` on one line fails to parse.

## Steps

1. `b.usda` is listed first, so it beats `a.usda`. Radius **2**.
2. Mute `b` → radius **1** from `a`.
3. The root file itself authors nothing on `radius`.

Run `python-labs/lab05_sublayer_stack.py` (script is in the repo; output below).

### Expected output

```
stack:
 root.usda
 b.usda
 a.usda
radius: 2.0
muted radius: 1.0
```

## Check your understanding

1. Why does `b` beat `a`?
2. Does the root beat `b` if it also authored radius 4?
3. Does muting delete `b.usda`?

**Answers** 1. It is **first-listed**, so stronger among sublayers. 2. **Yes**. Root is stronger than every sublayer. 3. **No**. Mute is per-stage; the file stays on disk.

## Stretch challenge

Unmute `b` and print radius again. Solution:

```
stage.UnmuteLayer(os.path.abspath("b.usda")) then Get() → 2.0.
```

### LAB 06

## Lab 06 — Attributes and value types

**Domain / objectives:** Data Modeling 5.2 · **Chapter:** 5, 9 · **Time:** 25 min · **Difficulty:**

★☆☆

### Goal

Create attributes with four `Sdf.ValueTypeNames` (`Int`, `Float`, `Color3f`, `Point3f`), set values, and list **authored** property names (not the whole schema).

### Background

Attributes are typed. `color3f` and `point3f` are both three floats but mean different things to Hydra (a colour vs a position). `GetPropertyNames()` on an `Xform` also lists schema leftovers (`visibility`, `xformOpOrder`, ...). `GetAuthoredPropertyNames()` is the list you wrote (Ch 5, Ch 9).

### Steps

1. Run the script. Match each type name to the value.
2. `authored` is exactly the four names you created, sorted.

Run `python-labs/lab06_attributes_types.py` (script is in the repo; output below).

### Expected output

```
count int 3
mass float 1.5
tint color3f (0.2, 0.4, 0.8)
pos point3f (1, 2, 3)
authored: ['count', 'mass', 'pos', 'tint']
```

## Check your understanding

1. Why not store the colour as `Point3f`?
2. Does `GetPropertyNames()` equal `GetAuthoredPropertyNames()` on this `Xform`?
3. What namespace would a `primvar` use? (Ch 11)

**Answers** 1. Downstream tools treat `color3f` as a colour (gamma, Hydra) and `point3f` as a position. 2. **No**. The schema adds `visibility`, `purpose`, `xformOpOrder`, `proxyPrim`. 3. `primvars: (primvars:displayColor, ...)`.

### Stretch challenge

Create `Sdf.ValueTypeNames.Token` mode set to `"run"`. Print `GetTypeName()`. Solution: `token`.

## LAB 07

# Lab 07 — Relationships and targets

**Domain / objectives:** Data Modeling 5.4 · **Chapter:** 5 · **Time:** 20 min · **Difficulty:** ★☆☆

### Goal

Author a **relationship** `looks` on `/A` targeting `/B`. Prove it is a relationship, not an attribute.

### Background

Relationships store **prim or property paths**, not numeric values. Material binding (`rel material:binding`, Ch 40) is the exam-facing relationship. `HasRelationship` / `HasAttribute` tell the two property kinds apart. `GetTargets()` returns `Sdf.Path` objects — print `str(t)` for a stable string (Ch 5).

### Steps

1. Run the script.
2. Targets list `['/B']`.
3. `HasAttribute looks` is `False` — the name is a relationship.

Run `python-labs/lab07_relationships.py` (script is in the repo; output below).

### Expected output

```
targets: ['/B']
HasRelationship: True
HasAttribute looks: False
```

### Check your understanding

1. Can a relationship hold 2.0?
2. What API adds a second target?
3. Material binding uses which property kind?

**Answers** 1. **No**. Targets are paths. Use an attribute for numbers. 2. `AddTarget` again (list-op prepend/append on the spec). 3. A **relationship** (`material:binding`).

### Stretch challenge

`rel.AddTarget("/A")` then print targets. Solution: `['/B', '/A']` (order is authoring order unless you list-edit).

## LAB 08

# Lab 08 — Metadata, customData, assetInfo, stage metadata

**Domain / objectives:** Data Modeling 5.3, Pipeline 7.6 · **Chapter:** 12 · **Time:** 25 min · **Difficulty:** ★☆☆



## Goal

Read the **CreateInMemory** fallbacks for `upAxis` and `metersPerUnit`, then author Z-up metres, `defaultPrim`, a `customData` key, and an `assetInfo` identifier.

## Background

Stage metadata lives on the **root layer**. Fresh **CreateInMemory** stages already have `upAxis = Y` and `metersPerUnit = 0.01` (centimetres) — verified on USD 26.08 (Ch 2, Ch 28). `customData` is a dictionary on the prim (not a typed attribute). `GetCustomData()` may also include schema `userDocBrief`; this lab reads **only** the key we set (`GetCustomDataByKey`).

## Steps

1. Run the script. Defaults are Y and 0.01.
2. After the setters: Z and 1.0, `defaultPrim World`.
3. `dept / asset id` are the keys you authored.

Run `python-labs/lab08_metadata.py` (script is in the repo; output below).

### Expected output

```
default upAxis: Y
default meters: 0.01
upAxis: Z
meters: 1.0
defaultPrim: World
dept: layout
asset id: world
```

## Check your understanding

1. What are **CreateInMemory** fallbacks for `upAxis` and `metersPerUnit`?
2. Is `customData` an attribute you `CreateAttribute`?
3. Why not print all of `GetCustomData()` in this lab?

**Answers** 1. **Y** and **0.01**. 2. **No**. It is prim metadata (`SetCustomDataByKey`). 3. The schema may inject `userDocBrief`; key lookup is stable.

## Stretch challenge

Set `timeCodesPerSecond` to 24 with `stage.SetTimeCodesPerSecond(24)` and print `GetTimeCodesPerSecond()`. Solution: 24.0.

### LAB 09

## Lab 09 — Authoring with Sdf specs + Sdf.ChangeBlock

**Domain / objectives:** Debugging 6.1 · **Chapter:** 8 · **Time:** 30 min · **Difficulty:** ★★☆

## Goal

Count `ObjectsChanged` notices for three `Usd CreateAttribute+Set` calls (**6** notices), then author two more attributes through **Sdf.AttributeSpec** inside **Sdf.ChangeBlock** (**1** notice). Revoke the listener.

## Background

Usd APIs compose as they write. Sdf APIs write the layer spec directly (Ch 8). **Sdf.ChangeBlock delays notices**, not the writes (Ch 34, Ch 44.4). Three create+set pairs without a block sent **6** notices in USD 26.08. `Usd.DefinePrim` of *new children* inside a block can fail — this lab only adds attributes on an existing prim, via Sdf, which is the safe pattern.

Unregister with `key.Revoke()`, not `Tf.Notice.Revoke`.

## Steps

1. Run the script. `without block: 6`.
2. `with block: 1` even though two attributes were authored.
3. `d via Usd: 4` — the stage sees the Sdf write after the block exits.

Run `python-labs/lab09_sdf_changeblock.py` (script is in the repo; output below).

### Expected output

```
without block: 6
with block: 1
d via Usd: 4
spec typeName: Xform
revoked
```

## Check your understanding

1. Does `ChangeBlock` delay the spec writes until `exit`?
2. How do you unregister the listener?
3. Why 6 notices for 3 attributes without a block?

**Answers** 1. **No**. Specs write immediately; **notices** wait. 2. `key.Revoke()`. 3. Each `CreateAttribute` and each `Set` can emit a notice ( $2 \times 3 = 6$ ).

## Stretch challenge

Move the `Sdf.AttributeSpec` lines *out* of the `with block` and compare the notice count. Solution: you will see more than 1 (typically 4 for two create+default pairs).

## LAB 10

# Lab 10 — Time samples, `Get()` vs `Get(t)`, linear vs held

**Domain / objectives:** Data Modeling 5.2 · Composition 1.4 · **Chapter:** 10 · **Time:** 25 min  
· **Difficulty:** ★★☆

## Goal

Author two translate samples, prove that `Get()` with no time is `None` when there is no default, then compare **linear** interpolation at the midpoint with **held**.

## Background

A still photograph is the **default** (`Get()` / `TimeCode.Default()`). A flip-book is **time samples**. Numeric queries **ignore the default**; they only look at samples (Ch 10). Translate ops have **no schema fallback**, so `Get()` is `None` until you author a default. Stage interpolation starts as **linear** (`Usd.InterpolationTypeLinear`). Held keeps the **previous** sample.

Print `Gf.Vec3d` as `tuple(...)` so the output is plain numbers, not a type name.

## Steps

1. Run the script. `Get(): None` — samples are not a default.
2. `Get(1)` is the first key `(0.0, 0.0, 0.0)`.
3. Midpoint `Get(6)` under linear is `(5.0, 0.0, 0.0)`.
4. After `SetInterpolationType(Held)`, `Get(6)` stays `(0.0, 0.0, 0.0)`.
5. `GetTimeSamples()` lists `[1.0, 11.0]`.

Run `python-labs/lab10_time_samples.py` (script is in the repo; output below).

## Expected output

```
Get(): None
Get(1): (0.0, 0.0, 0.0)
Get(6) linear: (5.0, 0.0, 0.0)
interp: Usd.InterpolationTypeLinear
Get(6) held: (0.0, 0.0, 0.0)
samples: [1.0, 11.0]
```

## Check your understanding

1. Why is `Get()` `None` instead of `(0, 0, 0)`?
2. Held at `t=6` uses which key?
3. What is the default interpolation type?

**Answers** 1. There is **no authored default** and `translate` has **no fallback**. Samples are only for numeric times. 2. The **previous** sample (`t=1` → origin). Not the next key. 3. **Linear** (`Usd.InterpolationTypeLinear`).

## Stretch challenge

Author a default with `attr.Set(Gf.Vec3d(9, 9, 9))` (no time) and print `Get()` again. Solution: `Get()` becomes `(9, 9, 9)`; `Get(1)` and `Get(6)` are unchanged — numeric queries still ignore the default.

### LAB 11

## Lab 11 — Mesh topology, `subdivisionScheme`, and extent

**Domain / objectives:** Data Modeling 5.5, 5.6 · **Chapter:** 13, 39 · **Time:** 30 min ·

**Difficulty:** ★★☆

### Goal

Build a triangle Mesh, see that **extent is not automatic**, compute it with `ComputeExtentFromPlugins`, Set it, and author `subdivisionScheme = none` so the polygons stay flat.

### Background

A Mesh needs `points`, `faceVertexCounts`, and `faceVertexIndices` (Ch 39). **extent** is a local-space AABB (`float3[]` of length 2). Changing points does **not** update it; you compute and author it (Obj 5.6, Ch 13). `ComputeExtentFromPlugins` **returns** values — it does not write. The `subdivisionScheme` fallback is **catmullClark** (smooth). Hard-surface / CAD meshes must set **none**.

### Steps

1. Run the script. Fallback scheme is `catmullClark` even though you never authored it.
2. After `CreateSubdivisionSchemeAttr(UsdGeom.Tokens.none)`, it is `none`.
3. `extent` authored?: `False` and `Get()` is `None`.
4. `computed` is `[(0.0, 0.0, 0.0), (1.0, 1.0, 0.0)]` — min/max of the three points.
5. After `Set`, `extent` after matches the computed box.

Run `python-labs/lab11_mesh_extent.py` (script is in the repo; output below).

### Expected output

```
scheme fallback: catmullClark
scheme authored: none
extent authored?: False
extent before: None
```

```
computed: [(0.0, 0.0, 0.0), (1.0, 1.0, 0.0)]
extent after: [(0.0, 0.0, 0.0), (1.0, 1.0, 0.0)]
```

## Check your understanding

1. Does `ComputeExtentFromPlugins` author extent?
2. What is the unauthored `subdivisionScheme`?
3. Is `extent` in world space or local space?

**Answers** 1. **No**. It returns a `Vt.Vec3fArray`. You must `Set`. 2. `catmullClark`, not `none`. 3. **Local**. Parent xforms are applied by `BBoxCache`, not stored in `extent`.

## Stretch challenge

Move one point to `(4, 0, 0)` and recompute. Solution: computed max x becomes `4.0`; the previously authored extent is stale until you `Set` again.

### LAB 12

## Lab 12 — Indexed vertex displayColor primvar

**Domain / objectives:** Data Modeling 5.1 · Visualization 8.1 · **Chapter:** 11 · **Time:** 25 min · **Difficulty:** ★★☆

### Goal

Author a **vertex** `displayColor` primvar with **two unique colors** and **four indices** `[0, 1, 0, 1]` (paint-by-number). Print values as tuples, not `Gf.Vec3f`.

### Background

A **primvar** is a `primvars:` attribute with interpolation metadata (Ch 11). Use `UsdGeom.PrimvarsAPI.CreatePrimvar` so interpolation and indices are set correctly. **Indexing** stores a palette plus `primvars:foo:indices`; it is independent of interpolation. `vertex` interpolation means one value (or one index) per point.

### Steps

1. Run the script. Interpolation is `vertex`.
2. `indexed: True` after `SetIndices`.
3. Indices are `[0, 1, 0, 1]` — four corners of a quad sharing two colors.
4. Values are the palette `[(1.0, 0.0, 0.0), (0.0, 1.0, 0.0)]` (red, green).

Run `python-labs/lab12_primvars.py` (script is in the repo; output below).

### Expected output

```
interpolation: vertex
indexed: True
indices: [0, 1, 0, 1]
values: [(1.0, 0.0, 0.0), (0.0, 1.0, 0.0)]
```

## Check your understanding

1. Does indexing change the interpolation?
2. For vertex interpolation on a 4-point mesh, what must have length 4 — values or indices?
3. Why print `tuple(v)` instead of `v`?

**Answers** 1. **No**. Indexing is independent. Interpolation still describes how values (or indices) attach to topology. 2. **Indices**. Values are the unique palette (here length 2). 3. `Get()` returns `Gf.Vec3f`; `tuple` keeps Expected output stable and readable.

## Stretch challenge

Call `pv.SetInterpolation(UsdGeom.Tokens.constant)` and think about length. Solution: constant wants **one** value (or one index). A four-index array with constant interpolation is a length mismatch — a classic Obj 5.5 unexpected-visuals bug.

### LAB 13

## Lab 13 — defaultPrim, explicit paths, UnresolvedPrimPath, internal refs

**Domain / objectives:** Composition 1.3 · **Chapter:** 16 · **Time:** 30 min · **Difficulty:** ★★☆☆

### Goal

Reference four ways: **defaultPrim**, a file **with no defaultPrim**, an **explicit prim path**, and an **internal reference**. Prove that `AddReference` returning `True` is not composition success.

### Background

A reference without `<prim path>` uses the target layer's **defaultPrim** (Ch 16). USD does **not** guess the first root prim. Missing or wrong `defaultPrim` →

`Pcp.ErrorType_UnresolvedPrimPath`. The referencing prim stays; it just receives nothing. `AddInternalReference` targets a prim path on this stage (no asset path). Sibling prims in the referenced file (here `/Skip`) are **not** brought in.

A warning is also printed on stderr for `/B`. Expected output below is **stdout only**.

### Steps

1. `/A` via `defaultPrim` is a Cube of size 2.0. `AddReference` is `True`.
2. `/B` has no `defaultPrim`: type `''`, size `None`, still authored: `True`.
3. `/C` names `/Only` and gets size 1.0. `/Skip` is **not** on the stage.
4. `/D` internally references `/_Tmpl` and becomes a Sphere of radius 4.0.
5. `GetCompositionErrors()` reports `UnresolvedPrimPath` at `/B`.

Run `python-labs/lab13_references.py` (script is in the repo; output below).

### Expected output

```
A authored: True
B authored: True
C authored: True
D authored: True
A type/size: Cube 2.0
B type/size: '' None
C type/size: Cube 1.0
Skip on stage?: False
D type/radius: Sphere 4.0
error at /B -> Pcp.ErrorType_UnresolvedPrimPath
```

### Check your understanding

1. Does `AddReference` return `False` when `defaultPrim` is missing?
2. Without `defaultPrim` and without a prim path, what does USD pick?
3. Why is `/Skip` not on the stage?

**Answers** 1. **No**. It returns `True` (authoring succeeded). Check `GetCompositionErrors()`. 2. **Nothing**. USD never guesses the first root prim. 3. The reference targeted `/Only` only. Siblings of the target prim are not composed in.

## Stretch challenge

Set `layer.defaultPrim = "Box"` on `nodefault.usda` and re-check `/B`. Solution: type becomes `Cube`, size `9.0`, and the error list is empty (Ch 16.2).

### LAB 14

## Lab 14 — Time-offset references (`Sdf.LayerOffset`)

**Domain / objectives:** Composition 1.4 · **Chapter:** 10, 16 · **Time:** 25 min · **Difficulty:** ★★☆☆

### Goal

Reference the **same** animated clip twice: identity offset on `/A`, **offset 10** on `/B`. Confirm composed sample times follow `stageTime = offset + scale * layerTime`.

### Background

A **layer offset** remaps time on a reference (or payload, or sublayer) without copying the clip (Obj 1.4, Ch 10/16). `Sdf.LayerOffset(offset, scale)` defaults to `(0, 1)`. The formula:

```
stageTime = offset + scale * layerTime
```

`lo * layerTime` applies it. Defaults (non-animated values) are **not** shifted — only samples. Before the first sample and after the last, values are held.

### Steps

1. `/A` (identity) has samples at `[0.0, 10.0]`, matching the clip.
2. `/B` (offset 10) has samples at `[10.0, 20.0]`.
3. `/A` at 0 and 10 is origin then `(10, 0, 0)`.
4. `/B` at 10 and 20 is the **same poses** — the clip started 10 frames later.
5. `LayerOffset(10) * 0` is `10.0`; `* 10` is `20.0`.

Run `python-labs/lab14_time_offset_refs.py` (script is in the repo; output below).

### Expected output

```
A samples: [0.0, 10.0]
B samples: [10.0, 20.0]
A at 0: (0.0, 0.0, 0.0)
A at 10: (10.0, 0.0, 0.0)
B at 10: (0.0, 0.0, 0.0)
B at 20: (10.0, 0.0, 0.0)
offset*0: 10.0
offset*10: 20.0
```

### Check your understanding

1. Write the formula that maps layer time to stage time.
2. Does a layer offset move a **default** (non-sampled) value?
3. `/B` at stage time 0 — which pose, and why?

**Answers** 1. `stageTime = offset + scale * layerTime`. 2. **No**. Only time samples (and clips) are remapped. 3. The **first** pose `(0, 0, 0)`, held, because stage 0 is before `/B`'s first sample at 10.

### Stretch challenge

Add `/C` with `Sdf.LayerOffset(0, 2)` (scale 2) and print its sample times. Solution: `[0.0, 20.0]` — the clip lasts twice as long (`0 + 2 * 10 = 20`).

**LAB 15****Lab 15 — Payloads, LoadNone, and OpenMasked**

**Domain / objectives:** Composition 1.5 · Content Aggregation 2.x · **Chapter:** 17 · **Time:** 30 min · **Difficulty:** ★★☆☆

**Goal**

Open a city with **payloads** on `/World/A` and `/World/B` and a **reference** on `/World/C`. Prove **LoadNone still loads references**, then **Load** one payload, then **OpenMasked** so `/World/B` does not exist at all.

**Background**

A **payload** is a reference you can leave unloaded (Ch 17). `Usd.Stage.Open(path)` defaults to **LoadAll**. Pass **Usd.Stage.LoadNone** to open with no payloads composed. **References are not payloads** — `LoadNone` does not hide `/World/C/Hi`. `FindLoadable()` lists prims that have payloads. A **population mask** (`Usd.Stage.OpenMasked`, not `Open(path, mask)`) decides which prims **exist**; load rules decide which payloads inside the mask are composed. Default `OpenMasked` still uses `LoadAll`, so `/World/A/Hi` appears.

**Steps**

1. `LoadNone C/Hi: True` — the reference is always composed.
2. `LoadNone A/Hi: False` — the payload is not loaded.
3. `loadable` is `['/World/A', '/World/B']` (not C). `load set` is empty.
4. `Load("/World/A")` brings A/Hi only; B/Hi stays missing.
5. `OpenMasked` with `/World/A`: A exists, B and C do not. A/Hi is True because the mask used the default load set.

Run `python-labs/lab15_payloads_masks.py` (script is in the repo; output below).

**Expected output**

```
LoadNone C/Hi: True
LoadNone A/Hi: False
loadable: ['/World/A', '/World/B']
load set: []
after Load A, A/Hi: True
after Load A, B/Hi: False
masked A: True
masked B: False
masked C: False
masked A/Hi (payload LoadAll): True
```

**Check your understanding**

1. Does `Open(..., LoadNone)` skip references?
2. Is `Usd.Stage.Open(path, mask)` how you apply a population mask?
3. Why is A/Hi True on the masked stage?

**Answers** 1. **No**. `LoadNone` only skips **payloads**. `/World/C/Hi` is still there. 2. **No**. Use `Usd.Stage.OpenMasked(path, mask)`. 3. `OpenMasked` defaults to **LoadAll**, so payloads **inside the mask** load. Combine with `LoadNone` if you want them off.

**Stretch challenge**

Reopen with `Usd.Stage.OpenMasked("city.usda", mask, Usd.Stage.LoadNone)` and print A/Hi. Solution: `False` — the prim `/World/A` exists (mask) but its payload is unloaded.

**LAB 16****Lab 16 — Variant sets and** `GetVariantEditContext`**Domain / objectives:** Composition 1.7 · **Chapter:** 18 · **Time:** 25 min · **Difficulty:** ★★☆**Goal**

Add a `size` variant set with `small` (height 1) and `big` (height 3). Select first, then author inside `GetVariantEditContext()`. List names with the **correct APIs**.

**Background**

A variant set is opinions behind a named switch on one prim (Ch 18). **Select the variant, then** enter with `vset.GetVariantEditContext()`. If you skip selection, the edit target is invalid and opinions land as **direct** opinions that apply to every variant — no error.

`GetNames()` is on `Usd.VariantSets` (the list of set names). `GetVariantNames()` is on `Usd.VariantSet` (the variants inside one set). `GetNames()` does not exist on `VariantSet`.

**Steps**

1. set names: ['size'] from `GetVariantSets().GetNames()`.
2. variant names: ['big', 'small'] from `GetVariantNames()` (sorted, not insertion order).
3. Selection `small` → height 1.0.
4. Selection `big` → height 3.0. The last `SetVariantSelection` leaves selection `big`.

Run `python-labs/lab16_variant_sets.py` (script is in the repo; output below).

**Expected output**

```
set names: ['size']
variant names: ['big', 'small']
small height: 1.0
big height: 3.0
selection: big
```

**Check your understanding**

1. What happens if you call `GetVariantEditContext()` with no selection?
2. Which object has `GetNames()` — `VariantSet` or `VariantSets`?
3. Why is variant names ['big', 'small'] even though `small` was added first?

**Answers** 1. Edits become **direct opinions** on the prim (every variant). No warning. 2. `VariantSets` (plural). On `VariantSet` use `GetVariantNames()`. 3. `GetVariantNames()` returns them **sorted**, not in insertion order.

**Stretch challenge**

Add a third variant `medium` **without** calling `SetVariantSelection` first, then author height 2 inside the context. Solution: height 2 becomes a direct opinion and shows up for `small` and `big` too (Ch 18 trap).

**LAB 17****Lab 17 — Classes, inherits, and a local override**

**Domain / objectives:** Composition 1.1, 1.6 · **Chapter:** 19 · **Time:** 25 min · **Difficulty:** ★★☆



## Goal

Create a **class** prim, inherit it on /A and /B, **broadcast** one edit, then author a **local** value on /A that /B does not pick up. Print inherit paths as strings.

## Background

A **class** (CreateClassPrim) is abstract: `IsAbstract()` is True and **Traverse()** **skips it**; **TraverseAll()** includes it (Ch 19). An **inherits** arc is a live link — later edits to the class reach every inheritor. **Local** opinions beat inherits (L before I in LIVERPS). `GetAllDirectInherits()` returns `Sdf.Path` objects; print `str(p)` for stable output.

## Steps

1. Both start at size 2.0 from /\_Look.
2. One `Set(5)` on the class broadcasts: A and B become 5.0.
3. Local `Set(9)` on /A only: A is 9.0, B stays 5.0.
4. Direct inherits: `['_Look']`.
5. /A is not abstract; /\_Look is. `Traverse` lists A, B; `TraverseAll` lists \_Look, A, B.

Run `python-labs/lab17_classes_inherits.py` (script is in the repo; output below).

### Expected output

```
before A/B: 2.0 2.0
after broadcast A/B: 5.0 5.0
local A, B stays: 9.0 5.0
direct inherits: ['/_Look']
A abstract?: False class abstract?: True
Traverse: ['A', 'B']
TraverseAll: ['_Look', 'A', 'B']
```

## Check your understanding

1. Does editing the class copy values once, or stay live?
2. Why is /\_Look missing from `Traverse()`?
3. Can a local opinion on /A be weaker than the class?

**Answers** 1. **Live**. The next `Set` on the class updates every inheritor that has no stronger opinion. 2. It is **abstract**. Default traversal skips `Usd.PrimIsAbstract`. 3. **No**. Local (L) beats inherits (I).

## Stretch challenge

`a.GetAttribute("size").Clear()` after the local 9. Solution: /A returns to 5.0 (the class) because the local opinion is gone; /B is unchanged.

## LAB 18

# Lab 18 — Specializes vs inherits vs references

**Domain / objectives:** Composition 1.1, 1.6 · **Chapter:** 19 · **Time:** 30 min · **Difficulty:** ★★★

## Goal

Give two prims the same referenced asset (`roughness = 0.2`) and the same class (`roughness = 0.9`, `metallic = 1.0`). One **inherits**, one **specializes**. Then prove **local beats both**, and `Clear()` restores the LIVERPS results.

## Background

**Inherits (I)** beat **references (R)**. **Specializes (S)** lose to everything above them, including references (Ch 19, LIVERPS). Both arcs still **fill gaps**: `metallic` is only on the class, so both prims get `1.0`. `HasAuthoredInherits()` / `HasAuthoredSpecializes()` report whether the arc is authored, not which value won. A **local** `Set(0.5)` wins on both. `Clear()` removes that local opinion and composition re-resolves.

## Steps

1. Inherit: roughness **0.9** (class beats asset), metallic **1.0**.
2. Specialize: roughness **0.2** (asset beats class), metallic **1.0** (gap fill).
3. Both flags are `True`.
4. Local `0.5` on both wins (L before I and S).
5. After `Clear()`: inherit back to **0.9**, specialize back to **0.2**.

Run `python-labs/lab18_specializes_vs_inherits.py` (script is in the repo; output below).

## Expected output

```
inherit roughness/metallic: 0.9 1.0
specialize roughness/metallic: 0.2 1.0
HasAuthoredInherits: True
HasAuthoredSpecializes: True
local on both: 0.5 0.5
after Clear inherit/specialize: 0.9 0.2
```

## Check your understanding

1. Why is inherit roughness 0.9 but specialize roughness 0.2?
2. Why is metallic 1.0 on **both** prims?
3. Does `HasAuthoredSpecializes()` mean the class value won?

**Answers** 1. **I beats R; R beats S**. Same class, same asset, different arc strength. 2. The asset does not author `metallic`. Both arcs **fill gaps** from the class. 3. **No**. It only means a specializes arc is authored. The referenced 0.2 still wins.

## Stretch challenge

Print `Usd.PrimCompositionQuery` arcs on `/ViaSpecialize` (strongest first). Solution: `Root`, then `Reference /Mat`, then `Specialize /_base` — S last (Ch 19.3).

## LAB 19

# Lab 19 — Edit targets, `EditContext`, and offset mapping

**Domain / objectives:** Composition 1.1, 1.5 · **Chapter:** 15 · **Time:** 30 min · **Difficulty:** ★★☆☆

## Goal

See the **default edit target** (root layer), write a sample through `GetEditTargetForLocalLayer` so offset math is applied, write another through a **bare** `Usd.EditTarget`, park a comment in the **session layer** with `Usd.EditContext`, and fail to target a layer that is **not in the local stack**.

## Background

`Usd` writes go to the **edit target**, which starts as the root layer (Ch 15). `GetEditTargetForLocalLayer(layer)` includes that sublayer's **layer offset**, so `Set(value, stage_time)` stores at **layer time**:  $(20 - 10) / 1 = 10$ . A bare `Usd.EditTarget(layer)` stores the stage time unchanged (sample at 20). with `Usd.EditContext(stage, target):`

sets the target for the block and **restores** it. The target layer must be in the **local layer stack** (session, root, or a sublayer). A referenced-only or anonymous layer raises `Tf.ErrorException`.

## Steps

1. Default target display name is `shot.usda`.
2. `anim` samples: `{10.0: 1.0, 20.0: 2.0}` — mapped time then unmapped time.
3. After the `with` block, the target is `shot.usda` again. The comment lives on the **session** prim spec.
4. Targeting an anonymous layer prints `not in stack: True`.

Run `python-labs/lab19_edit_targets.py` (script is in the repo; output below).

## Expected output

```
default target: shot.usda
anim samples: {10.0: 1.0, 20.0: 2.0}
restored: shot.usda
session comment: try bigger
not in stack: True
```

## Check your understanding

1. At what layer time is a sample stored if you `Set(..., 20)` through `GetEditTargetForLocalLayer` on a sublayer with offset 10?
2. Does `EditContext` leave the session layer as the target after the block?
3. Can you `SetEditTarget` a layer that is only reached through a reference?

**Answers** 1. **10.**  $(\text{stageTime} - \text{offset}) / \text{scale} = (20 - 10) / 1$ . 2. **No.** It restores the previous target (`shot.usda` here). 3. **No.** The layer must be in the local layer stack.

## Stretch challenge

Mute `anim.usda` and try `SetEditTarget(anim)` again. Solution: a muted layer is not in the stack, so you get the same `Tf.ErrorException` (Ch 15.3).

## LAB 20

# Lab 20 — LIVERPS laboratory

**Domain / objectives:** Composition 1.6 · **Chapter:** 21 · **Time:** 30 min · **Difficulty:** ★★★

## Goal

Hold a referenced asset at **radius 1** constant, then beat or lose to it with **Local**, **Inherits**, **VariantSets**, and **Specializes**. Recite the winning **letter** before you look at the number.

## Background

**LIVERPS** is strongest-to-weakest: Local, Inherits, VariantSets, rElocates, References, Payloads, Specializes (Ch 21). This lab skips E (relocates, Ch 20) and P (payloads, Lab 15) so each contest is one letter versus **R**. Weaker arcs can still **fill gaps**; here every opinion is about radius, so there is no gap. Predict, then run.

```
present: L I V E R P S
has field: (whichever you authored)
winner: earliest remaining letter
```

## Steps

1. **L vs R:** local 2 beats referenced 1 → 2.0.

2. **I vs R**: class inherit 3 beats referenced 1 → 3.0.
3. **V vs R**: variant 10 (authored inside `GetVariantEditContext`) beats 1 → 10.0.
4. **S vs R**: specialize 7 **loses** to referenced 1 → 1.0.

Run `python-labs/lab20_liverps.py` (script is in the repo; output below).

### Expected output

```
L vs R (local 2, ref 1): 2.0
I vs R (inherit 3, ref 1): 3.0
V vs R (variant 10, ref 1): 10.0
S vs R (specialize 7, ref 1): 1.0
```

### Check your understanding

1. Recite LIVERPS.
2. Why does specialize 7 lose to reference 1, while inherit 3 wins?
3. Is a session-layer opinion a LIVERPS letter?

**Answers** 1. **Local, Inherits, VariantSets, rElocates, References, Payloads, Specializes**. 2. **I is before R; S is after R**. Same asset, different arc strength. 3. **No**. The session layer is the strongest **sheet of L** (the local stack), not its own letter.

### Stretch challenge

Add a payload-backed prim `/PvsR` with payload radius 9 and a reference radius 1 on the same prim. Solution: composed radius is **1** (R before P). A field authored **only** on the payload would still show through (Ch 21 P6).

## LAB 21

# Lab 21 — Composition introspection toolkit

**Domain / objectives:** Composition 1.8 · Debugging 6.2 · **Chapter:** 22, 42 · **Time:** 30 min · **Difficulty:** ★★★

### Goal

Build `/Lamp` with a **local** height 2, an **inherit** of 9, and a **reference** of 1. Print the **prim stack**, **property stack**, **ResolveInfo**, and **PrimCompositionQuery** arcs (strongest first) to prove local won.

### Background

When a composed value is “wrong,” the data is usually still on the stage (Ch 22, Ch 42). `GetPrimStack()` lists prim specs strongest first. `GetPropertyStack()` lists property specs (with `.default` here). `GetResolveInfo()` names the encoding (`ResolveInfoSourceDefault` vs time samples) and the winning **arc type**.

`Usd.PrimCompositionQuery(prim).GetCompositionArcs()` lists arcs strongest first: Root, then Inherit, then Reference in this lab. Print target paths with `str(...)`.

### Steps

1. Composed value: 2.0 — local wins.
2. Prim stack: `shot.usda /Lamp`, then the class spec `shot.usda /_Look`, then `lamp.usda /Lamp`.
3. Property stack defaults: 2.0, 9.0, 1.0 — same order.
4. Resolve source is **Default** (not a time sample) on **Root** (the local site).
5. Arcs: Root `/Lamp`, Inherit `/_Look`, Reference `/Lamp`. errors: 0.

Run `python-labs/lab21_composition_introspection.py` (script is in the repo; output below).

### Expected output

```
value: 2.0
prim stack:
 shot.usda /Lamp
 shot.usda /_Look
 lamp.usda /Lamp
property stack:
 shot.usda /Lamp.height 2.0
 shot.usda /_Look.height 9.0
 lamp.usda /Lamp.height 1.0
resolve: Usd.ResolveInfoSourceDefault Pcp.ArcTypeRoot
arcs:
 Pcp.ArcTypeRoot /Lamp
 Pcp.ArcTypeInherit /_Look
 Pcp.ArcTypeReference /Lamp
errors: 0
```

### Check your understanding

1. Which stack entry won `height`, and how do you know?
2. Does `Pcp.ArcTypeRoot` mean “no composition”?
3. Why print `str(arc.GetTargetNode().path)`?

**Answers** 1. `shot.usda /Lamp.height 2.0` — first in the property stack. `ResolveInfo` source Default on Root agrees. 2. **No**. Root is the prim’s own site in this layer stack (the L in LIVERPS), including local inherits listed next. 3. The path object prints as `Sdf.Path('/Lamp')` otherwise; `str` keeps Expected output stable.

### Stretch challenge

`prim.GetAttribute("height").Clear()` and reprint value + winning stack entry. Solution: value becomes `9.0` and the first property spec is `shot.usda /_Look.height` (inherit, I before R).

---

## LAB 22

# Lab 22 — Component, assembly, group, and a broken model chain

**Domain / objectives:** Content Aggregation 2.4 · Pipeline 7.2 · **Chapter:** 23 · **Time:** 30 min · **Difficulty:** ★★☆

## Goal

Publish a **component** chair, reference it under an **assembly** → **group** chain, then put another **component** under a prim with **no kind**. Prove `GetKind()` can say “component” while `IsComponent()` is **False**.

## Background

A prim is a **model** only if its kind is a model kind **and** every ancestor is **group** or **assembly** (or it is the stage root) (Ch 23). **assembly is-a group** (`Kind.Registry.IsA`). The hierarchy **stops at components**. `subcomponent` and ordinary geo (here `Seat`) are not models. A **component** under an un-kinded parent is **not** a model — the classic “chair missing from the asset list” bug. `Usd.PrimIsModel` traversal prunes non-models, so it never even visits `Clutter`’s children.

## Steps

1. `assembly is-a group`: True.
2. `/Kitchen assembly`: model+group. `/Kitchen/Props group`: model+group.
3. Referenced `/Kitchen/Props/Chair_0` is a valid **component** (Xform, kind component).
4. `/Kitchen/Clutter/Mug` has kind `component` but `IsModel/IsComponent` are **False**.
5. Model traversal lists only Kitchen, Props, Chair\_0 — not Mug, not Seat.

Run `python-labs/lab22_component_assembly.py` (script is in the repo; output below).

### Expected output

```
assembly is-a group: True
path kind model group component
/Kitchen assembly True True False
/Kitchen/Props group True True False
/Kitchen/Props/Chair_0 component True False True
/Kitchen/Props/Chair_0/Seat - False False False
/Kitchen/Clutter - False False False
/Kitchen/Clutter/Mug component False False False
model traversal:
 /Kitchen
 /Kitchen/Props
 /Kitchen/Props/Chair_0
Chair_0 type: Xform kind: component
```

## Check your understanding

1. Why is Mug not a model even though `GetKind()` is `component`?
2. Is `group` a legal kind for a **published root**?
3. Does `assembly` count as a group?

**Answers** 1. **Clutter has no kind**, so the ancestor chain is broken. 2. **No**. Published roots are `component` or `assembly`. `group` is only for organizers inside assemblies. 3. **Yes**. `Kind.Registry.IsA("assembly", "group")` is True, so `IsGroup()` is True.

## Stretch challenge

`Usd.ModelAPI(stage.GetPrimAtPath("/Kitchen/Clutter")).SetKind("group")` and reprint Mug. Solution: Mug becomes `IsModel/IsComponent True` — the chain is `assembly → group → component`.

## LAB 23

# Lab 23 — Native instantancing: prototypes, proxies, de-instance

**Domain / objectives:** Content Aggregation 2.2 · **Chapter:** 24 · **Time:** 30 min ·

**Difficulty:** ★★☆☆

## Goal

Mark two internal references **instanceable**, prove they share one `GetPrototype()`, that **GetMaster is gone**, that default `Traverse()` **skips instance children**, and that `SetInstanceable(False)` lets one copy become unique.

## Background

**Native instantancing** shares one prototype prim tree among matching instanceable prims (Ch 24). Author `instanceable` on the **referencing** prims, not only on the source. `IsInstanceable()` is the flag; `IsInstance()` means USD actually built a prototype.

Descendants of an instance are **instance proxies**: `GetPrimAtPath` finds them, `Traverse()` does not. Source edits reach every instance. Nested over under an instanceable prim is ignored — de-instance that copy instead.

#### VERSION

Verified on USD 26.08. ``GetMaster()`` was renamed ``GetPrototype()`` in USD 21.x and is absent from this wheel.

## Steps

1. A and B are both instanceable **and** instances.
2. `GetMaster` exists: `False`. One prototype `/__Prototype_1`; A and B share it.
3. `Traverse` lists `/Room/A` and `/Room/B` but **not** `/Room/A/Seat`.
4. The proxy `Seat` is valid; source `Set(3)` updates **both**.
5. De-instance B and set size 9: A stays 3, B is 9 and `IsInstance` `False`.

Run `python-labs/lab23_native_instancing.py` (script is in the repo; output below).

## Expected output

```
A IsInstanceable/IsInstance: True True
B IsInstanceable/IsInstance: True True
GetMaster exists: False
prototypes: ['/__Prototype_1']
A proto is B proto: True
Traverse: ['/Chair', '/Chair/Seat', '/Room', '/Room/A', '/Room/B']
proxy Seat valid: True IsInstanceProxy: True
shared size: 1.0
after source A,B: 3.0 3.0
after B unique A,B: 3.0 9.0 B IsInstance False
```

## Check your understanding

1. Where do you author `instanceable = true`?
2. Why is `/Room/A/Seat` missing from `Traverse()`?
3. Does de-instancing B also de-instance A?

**Answers** 1. On each **referencing** prim that should share a prototype — not only on `/Chair`. 2. Default traversal **does not walk into instances**. Use `GetPrimAtPath` or `Usd.TraverseInstanceProxies`. 3. **No**. Only B leaves the prototype. A stays an instance at size 3.

## Stretch challenge

While B is still instanceable, try `Set(9)` on `/Room/B/Seat`. Solution: the write is ignored or does not stick on the proxy; de-instance first (Ch 24.4).

### LAB 24

## Lab 24 — PointInstancer: append a prototype, hide one instance

**Domain / objectives:** Content Aggregation 2.1, 2.3 · **Chapter:** 25 · **Time:** 30 min · **Difficulty:** ★★★

### Goal

Build a forest PointInstancer, **append** a third prototype without scrambling indices, confirm it is **Boundable but not a Gprim**, then **InvisId** one instance and see transforms skip it while `GetInstanceCount()` **stays 3**.

## Background

A **PointInstancer** stores copies as arrays on **one** prim (Ch 25). Required: `prototypes rel`, `protoIndices`, `positions`. Instance count = `len(protoIndices)`. Put prototypes under a **class** child so `Traverse()` skips them. **AddTarget prepends by default** and re-numbers every existing prototype — pass **Usd.ListPositionBackOfAppendList**. `InvisId(id, time)` hides one instance (not `MakeInvisible()`, which hides the whole prim). Hidden instances drop out of `ComputeInstanceTransformsAtTime` via the mask; they are not deleted.

## Steps

1. Targets after append: Pine, Oak, Bush (Bush last).
2. Three instances; `Traverse` is only `/Forest` (class Protos hidden).
3. Not a Gprim; is Boundable. Translations at 0, 2, 4.
4. `InvisId(1, Default) → mask [True, False, True]`, `invisibleIds [1]`.
5. Transforms remain at 0 and 4. Count is still 3.

Run `python-labs/lab24_point_instancer.py` (script is in the repo; output below).

## Expected output

```
targets: ['Pine', 'Oak', 'Bush']
instances: 3
traversed: ['/Forest']
is Gprim: False
is Boundable: True
at (0.0, 0.0, 0.0)
at (2.0, 0.0, 0.0)
at (4.0, 0.0, 0.0)
mask: [True, False, True]
invisibleIds: [1]
transforms after hide:
at (0.0, 0.0, 0.0)
at (4.0, 0.0, 0.0)
count still: 3
```

## Check your understanding

1. Which array sets the instance count?
2. Why pass `ListPositionBackOfAppendList`?
3. Does `MakeInvisible()` hide instance 1?

**Answers** 1. `protoIndices`, not `positions` and not the number of prototype targets. 2. Default **AddTarget prepends**, so index 0 becomes the new prim and every old instance draws the wrong prototype. 3. **No**. That is Imageable visibility on the **whole** `PointInstancer`. Use `InvisId(id, time)`.

## Stretch challenge

Call `rel.AddTarget("/Forest/Protos/Bush")` with **no** list position on a stronger sublayer that already has Pine, Oak. Solution: composed targets start with Bush; existing `protoIndices [0, 1]` now draw Bush and Pine (Ch 25.3 prepend trap).

---

## LAB 25

# Lab 25 — JSON/OBJ → USD converter

**Domain / objectives:** Data Exchange 4.5, 4.7, 4.8 · **Chapter:** 29 · **Time:** 35 min ·

**Difficulty:** ★★★



## Goal

Convert a JSON crate (centimeters, a space in the name) into a **meters, Y-up, kind=component** Mesh with **extent** and a custom `factory:partId`. Parse a tiny OBJ and prove face indices are **0-based**.

## Background

A converter's transform stage maps DCC records onto USD schemas (Ch 27–29).

**Tf.MakeValidIdentifier** turns "Crate Box" into `Crate_Box`. Scale source units into meters ( $\text{cm} \rightarrow \times 0.01$ ) and author **metersPerUnit = 1**. Always **ComputeExtentFromPlugins then Set**. OBJ f 1 2 3 is 1-based — subtract 1. Custom namespaced attributes (`factory:partId`) carry data that has no USD schema (Obj 4.5). Set `subdivisionScheme = none` for tessellated CAD.

## Steps

1. `defaultPrim` is `Crate_Box`; `kind` `component`; `up/units` `Y 1.0`.
2. Point `[100,0,0]` cm became `(1.0, 0.0, 0.0)` meters.
3. Extent matches the scaled quad; `partId` is `CR-7`.
4. OBJ object `Tiny`, 3 points, first face `[0, 1, 2]` (not `1 2 3`).

Run `python-labs/lab25_json_obj_converter.py` (script is in the repo; output below).

## Expected output

```
defaultPrim: Crate_Box
kind: component
up/units: Y 1.0
points[1]: (1.0, 0.0, 0.0)
counts: [4]
extent: [(0.0, 0.0, 0.0), (1.0, 0.5, 0.0)]
partId: CR-7
name was: Crate Box -> Crate_Box
OBJ name: Tiny nPoints: 3 first face: [0, 1, 2]
```

## Check your understanding

1. Why is `points[1]` `(1,0,0)` if the JSON said `[100,0,0]`?
2. What does `f 1 2 3` become, and why?
3. Does `ComputeExtentFromPlugins` write extent for you?

**Answers** 1. Source units were **cm**; the converter scales by **0.01** into meters. 2.

`[0, 1, 2]`. OBJ vertex indices are 1-based; USD is 0-based. 3. **No**. It returns an array. You `CreateExtentAttr / Set`.

## Stretch challenge

Wire `parse_obj` into `json_to_usd` by building a `src` dict from `n`, `pts`, `faces` with `"units": "m"`. Solution: you get `/Tiny/Geo` with three points and `faceVertexCounts = [3]`.

## LAB 26

# Lab 26 — Units and up-axis: USD does not convert

**Domain / objectives:** Data Exchange 4.7 · **Chapter:** 28 · **Time:** 30 min · **Difficulty:**

★★☆

## Goal

Reference a **meters, Z-up** chair into a **centimeters, Y-up** shot. Prove composed `size` stays **1.0**, then add `rotateX(-90)` and `scale 100` on the referencing prim so world bounds match `1 m = 100 cm`.

## Background

Stage metrics (`upAxis`, `metersPerUnit`) live on the **root layer**. Composition arcs do **not** copy them (Ch 28). CreateInMemory fallback is **Y** and **0.01**. Factor = `asset_mpu / shot_mpu` (meters into cm → **100**). Read metrics by **opening the asset file**, not from the composed stage. Use a **typeless** `DefinePrim` so the asset's Xform type comes through; `Xform.Define` on the reference would replace a Cube/Mesh root. `+ 0.0` after `round` clears `-0.0`.

## Steps

1. Fallback metrics: `Y 0.01`.
2. Composed Cube size is still `1.0`. Shot stays `Y 0.01`; asset is `Z 1.0`.
3. Before correction, world range is ~1 unit tall on **Z**.
4. `factor: 100.0`. After rotate+scale, height is 100 cm on **+Y**.

Run `python-labs/lab26_units_upaxis.py` (script is in the repo; output below).

### Expected output

```
fallback: Y 0.01
composed size: 1.0
shot metrics: Y 0.01
before: ((-0.5, -0.5, 0.0), (0.5, 0.5, 1.0))
asset metrics: Z 1.0
factor: 100.0
after: ((-50.0, 0.0, -50.0), (50.0, 100.0, 50.0))
```

## Check your understanding

1. Does referencing a meters asset into a centimeters stage rescale `size`?
2. Why open `chair_m.usda` separately to read units?
3. Why `DefinePrim` instead of `UsdGeom.Xform.Define` for `/World/Chair`?

**Answers** 1. **No**. Composed `size` stays `1.0`. You add the scale. 2. The composed stage reports **its own** root-layer metrics, not the asset's. 3. A local Xform type would **replace** a Cube/Mesh asset root. Typeless lets the asset's type come through; wrap with `UsdGeom.Xformable` to add ops.

## Stretch challenge

Invert the factor (`shot_mpu / asset_mpu`). Solution: scale 0.01; the chair becomes 1 cm tall in a cm stage — the classic “tiny imported hero” bug.

### LAB 27

## Lab 27 — Asset validator (UsdValidation)

**Domain / objectives:** Data Exchange 4.6 · **Chapter:** 30 · **Time:** 25 min · **Difficulty:**

★★☆

### Goal

Run built-in **StageMetadataChecker** validators on an empty-metrics stage, then author **upAxis**, **metersPerUnit**, and **defaultPrim** until both checkers return **0** errors. Prove **UsdUtils.ComplianceChecker** is gone.

## Background

Obj 4.6 is “write a validator for the integrity of an exported asset.” On USD 26.08 the library is `pxr.UsdValidation` (Ch 30). `usdchecker` is a CLI on full builds; **usd-core has no CLI**. `UsdUtils.ComplianceChecker` was removed in 26.08. Two metadata checkers:

`usdGeomValidators:StageMetadataChecker` (up axis, meters) and `usdValidation:StageMetadataChecker` (defaultPrim). Do not print `GetMessage()` here — it embeds the anonymous layer id.

#### VERSION

Verified on USD 26.08. ``hasattr(UsdUtils, "ComplianceChecker")`` is False.

## Steps

1. `ComplianceChecker: False`.
2. Before authoring metrics: three Errors (missing meters, missing upAxis, missing defaultPrim).
3. After `SetStageUpAxis`, `SetStageMetersPerUnit`, `SetDefaultPrim`: both checkers return **0**.

Run `python-labs/lab27_asset_validator.py` (script is in the repo; output below).

## Expected output

```
ComplianceChecker: False
before:
MissingMetersPerUnitMetadata UsdValidation.ValidationErrorMessage.Error
MissingUpAxisMetadata UsdValidation.ValidationErrorMessage.Error
MissingDefaultPrim UsdValidation.ValidationErrorMessage.Error
after geom/core counts: 0 0
```

## Check your understanding

1. What replaced `UsdUtils.ComplianceChecker`?
2. Does missing `usdchecker` on PATH mean you cannot validate?
3. Which checker reports `MissingDefaultPrim`?

**Answers** 1. `pxr.UsdValidation` (`ValidationRegistry`, `ValidationContext`). 2. **No**. Call validators from Python. The CLI is only on full OpenUSD builds. 3.

`usdValidation:StageMetadataChecker`. Geom checker covers upAxis and metersPerUnit.

## Stretch challenge

Skip `SetDefaultPrim` and reprint core's error names. Solution: `MissingDefaultPrim` remains; geom count is 0 (axis and meters are set).

### LAB 28

## Lab 28 — Dependencies, Flatten vs FlattenLayerStack, USDZ

**Domain / objectives:** Pipeline 7.2 · Data Exchange 4.3 · Composition 1.9 · **Chapter:** 31, 32, 34 · **Time:** 35 min · **Difficulty:** ★★★

## Goal

Inventory an entry with `ComputeAllDependencies` (3-tuple), contrast **Export / Flatten / FlattenLayerStack**, then pack a USDZ whose zip method is **STORE (0)** — the `usdzip` stand-in on `usd-core`.

## Background

`ComputeAllDependencies(path)` returns (`layers`, `assets`, `unresolved`) and walks recursively (Ch 32). `ExtractExternalReferences` is one-file only. **Export** writes one layer (the texture lives on a sublayer, so Export misses it). **FlattenLayerStack** merges sublayers and **keeps references**. **Flatten** bakes the composed stage (Bolt becomes a local Cube;

no ref). Both flatten tools often **absolutize** asset paths. `CreateNewUsdzPackage` is the Python `usdzip`; USDZ files are **uncompressed** zip members (`compress_type == 0`). A warning on stderr for `gone.jpg` is expected.

## Steps

1. Layers: `entry`, `geo`, `ref`. Assets: `wood.png`. Unresolved: `gone.jpg`.
2. Export of the root has **no** `tex`. Flatten Bolt type `Cube`, no ref. FLS Bolt type empty, **has** ref, `subLayers` empty.
3. USDZ succeeds; contents `root.usda` then `tex.png`; both stored (0).

Run `python-labs/lab28_deps_flatten_usdz.py` (script is in the repo; output below).

## Expected output

```
layers: ['entry.usda', 'geo.usda', 'ref.usda']
assets: ['wood.png']
unresolved: ['gone.jpg']
Export has tex: False
Flatten Bolt type: Cube
Flatten Bolt has ref: False
FLS Bolt type: -
FLS Bolt has ref: True
FLS subLayers empty: True
usdz: True
contents: ['root.usda', 'tex.png']
compress: [0, 0]
```

## Check your understanding

1. What three lists does `ComputeAllDependencies` return?
2. Does `FlattenLayerStack` remove references?
3. Why is `compress: [0, 0]` the exam fact?

**Answers** 1. (`layers`, `assets`, `unresolved`) — a 3-tuple, not a dict. 2. **No**. It removes **sublayers**. References stay. Flatten bakes them away. 3. USDZ members are **stored**, not deflated, so USD can mmap them.

## Stretch challenge

`UsdUtils.CreateNewARKitUsdzPackage("root.usda", "arkit.usdz")` and print the first zip name. Solution: `root.usdc` (crate), not `USDA` (Ch 31).

---

## LAB 29

# Lab 29 — Asset resolution: anchor, search path, stage context

**Domain / objectives:** Pipeline 7.5, 7.7 · Debugging 6.3 · **Chapter:** 33 · **Time:** 30 min · **Difficulty:** ★★★

## Goal

Prove naive `Resolve("./tex/wood.png")` is **False** from the wrong cwd, that **anchoring** finds the file, that **search paths** only help **context-dependent** paths (no `./`), and that `Stage.Open(..., pathResolverContext=)` is how `@chair.usda@` resolves.

## Background

`Ar.GetResolver()` is the **Resolver** facade; `GetUnderlyingResolver()` is **DefaultResolver** on `usd-core` (Ch 33). **AnchorRelativePath is gone** in 26.08 — use `CreateIdentifier(path, ResolvedPath(layer.realPath))` or `ComputeAbsolutePath`. Composition anchors to the **layer**; raw `Resolve("./...")` uses `cwd`. `./` paths are **not** search-path lookups. `@chair.usda@` without `./` is context-dependent: Open without a context → `ErrorInvalidAssetPath`.

## Steps

1. Facade `Resolver`, plugin `DefaultResolver`, no `AnchorRelativePath`.
2. Naive `./tex/wood.png` `False`; absolute `/` `CreateIdentifier` `True`.
3. No-anchor `CreateIdentifier` strips to `tex/wood.png` (now search-path).
4. Bound search finds `tex/wood.png`, **not** `./tex/wood.png`. Unbind restores `False`.
5. Shot with context: 0 errors, type `Xform`. Without: 1 `ErrorInvalidAssetPath`.

Run `python-labs/lab29_asset_resolution.py` (script is in the repo; output below).

### Expected output

```
resolver: Resolver
underlying: DefaultResolver
AnchorRelativePath: False
naive ./tex/wood.png: False
Resolve abs: True
CreateIdentifier then Resolve: True
no-anchor ident: tex/wood.png
dependent tex/wood.png: True
dependent ./tex/wood.png: False
unbound tex/wood.png: False
bound tex/wood.png: True
bound ./tex/wood.png: False
after unbind: False
with ctx errors/type: 0 Xform
no ctx errors: 1 ErrorInvalidAssetPath
```

## Check your understanding

1. Does `GetResolver()` return `DefaultResolver`?
2. Why is naive `Resolve("./tex/wood.png")` `False` even though the PNG exists?
3. Do search paths apply to `@./tex/wood.png@`?

**Answers** 1. **No**. It returns **Resolver**. Ask `GetUnderlyingResolver()`. 2. Raw `Resolve` uses `cwd`, not the layer that authored `./`. Anchor first. 3. **No**. `./` is layer-relative, not context-dependent.

## Stretch challenge

Print `hasattr(Ar.Resolver, "AnchorRelativePath")`. Solution: **False** on USD 26.08 (Ch 33 VERSION).

## LAB 30

# Lab 30 — `ObjectsChanged`, `ChangeBlock`, and the `DefinePrim` trap

**Domain / objectives:** Debugging 6.1 · **Chapter:** 34, 44 · **Time:** 30 min · **Difficulty:**

★★☆

## Goal

Split **resync** vs **info-only** notices, batch three `Sdf.PrimSpec` children into **one** `ObjectsChanged`, prove three outside the block, and catch `UsdGeom.Sphere.Define` inside `ChangeBlock`.

## Background

Lab 09 counted 6 vs 1 for attribute create+set. This lab names the **payload**: `GetResyncedPaths()` vs `GetChangedInfoOnlyPaths()` (Ch 34). `CreateAttribute` is a resync of `/W.a`; `Set` is info-only. `Sdf.ChangeBlock` **delays notices**, not writes. Author **Sdf specs** inside the block — `Define` of a **new child** raised `Tf.ErrorException` on USD 26.08. Unregister with `listener.Revoke()`.

## Steps

1. Define `/W` → one resync `['/W']`, empty info list.
2. `CreateAttribute+Set` → **2** notices: resync `/W.a`, then info `/W.a`.
3. Three `Sdf.PrimSpec` inside a block → **1** notice with all three paths.
4. Three more outside → **3** notices.
5. `Sphere.Define` inside a block → `Tf.ErrorException`. Then `Revoke`.

Run `python-labs/lab30_notices_changeblock.py` (script is in the repo; output below).

## Expected output

```
after define [['/W'], []]
n after create+set 2
 resync ['/W.a'] info []
 resync [] info ['/W.a']
inside block 1 [['/W/S0', '/W/S1', '/W/S2']]
outside block 3 [['/W/S3'], ['/W/S4'], ['/W/S5']]
Define inside block: Tf.ErrorException
revoked
```

## Check your understanding

1. Does `ChangeBlock` delay the spec writes until `exit`?
2. How do you unregister the listener?
3. Should a CAD exporter call `UsdGeom.Mesh.Define` 10,000 times inside a `ChangeBlock`?

**Answers** 1. **No**. Specs write immediately; **notices** wait. 2. `listener.Revoke()`, not `Tf.Notice.Revoke`. 3. **No**. Prefer `Sdf.PrimSpec` (or `Define` **outside** the block). New `Define` children can fail inside the block.

## Stretch challenge

Print `sorted(n for n in dir(Usd.Notice) if n[0].isupper())`. Solution includes `ObjectsChanged`, `LayerMutingChanged`, `StageContentsChanged`, `StageEditTargetChanged`, `StageNotice` (Ch 34.4).

---

### LAB 31

## Lab 31 — Codeless custom schema (`schema.usda` vs registry)

**Domain / objectives:** Customizing USD 3.4, 3.6 · **Chapter:** 37 · **Time:** 25 min ·

**Difficulty:** ★★☆☆

## Goal

Show `usdGenSchema` is **missing** from `usd-core`, that **Cube is registered** (fallback size 2), that **MaterialBindingAPI is an applied API**, and that **parsing `schema.usda` does not register Door**.

## Background

A custom schema starts as `schema.usda`: over "GLOBAL" with `libraryName / skipCodeGeneration`, plus `class` prims that inherit `</Typed>` or `</APISchemaBase>` (Ch 37). `usdGenSchema` turns that into plugin files. **Parse  $\neq$  register**. Codeless still needs generated USDA + `plugInfo.json`; it only skips C++ wrappers. This wheel already contains Pixar's Cube. `FindConcretePrimDefinition("Door")` stays `None` until a plugin loads.

### VERSION

Verified on USD 26.08: ``shutil.which("usdGenSchema")`` is ``None``.

## Steps

1. `usdGenSchema` `None`. Cube registered; size fallback 2.0; `IsConcrete` `True`.
2. `MaterialBindingAPI` is an applied API schema.
3. Door is **not** registered.
4. Sdf parses GLOBAL and Door; `skipCodeGeneration` is `True`; Door **still** unregistered.

Run `python-labs/lab31_codeless_schema.py` (script is in the repo; output below).

### Expected output

```
usdGenSchema None
Cube registered True
Cube size fallback 2.0
IsConcrete Cube True
IsAppliedAPISchema MaterialBindingAPI True
Door registered False
parsed classes: ['GLOBAL', 'Door']
skipCodeGeneration True
Door still unregistered True
```

## Check your understanding

1. If Sdf parses `schema.usda`, can you `def Door` in `usdview`?
2. Does codeless mean no `plugInfo.json`?
3. What is Cube's fallback size?

**Answers** 1. **No**. Parse  $\neq$  register. You need `usdGenSchema` output loaded as a plugin. 2. **No**. Codeless skips **C++ wrappers**. JSON + generated USDA still ship. 3. **2.0** from the built-in concrete definition.

## Stretch challenge

`sr.IsTyped("Cube")` and `sr.IsTyped("MaterialBindingAPI")`. Solution: Cube `True` (IsA type); `MaterialBindingAPI` `False` (applied API, Ch 37.4).

### LAB 32

## Lab 32 — Kind registry and an unregistered custom kind

**Domain / objectives:** Customizing USD 3.3 · **Chapter:** 38 · **Time:** 20 min · **Difficulty:**

★★☆

## Goal

Print the built-in **kind tree**, prove **Kind.Registry.Register** **does not exist**, and show that **SetKind("door")** still authors the token but **IsComponent()** **becomes False**.

## Background

Built-in kinds: `model`, `group`, `assembly`, `component`, `subcomponent` (Ch 6, 23, 38). `assembly` is-a `group` is-a `model`. `component` is-a `model` but **not** a `group`. `subcomponent` is **not** a `model`. Custom kinds are declared in **plugInfo.json**, not Python. `GetKind()` returns whatever token you authored; `IsComponent()` / `IsModel()` consult the registry **and** the hierarchy.

## Steps

1. `GetAllKinds()` lists the five built-ins. `HasKind("door")` is False.
2. Bases: `component`→`model`, `assembly`→`group`, `group`→`model`, `model`→`''`.
3. `component IsA model True`; `subcomponent IsA model False`; `component IsA group False`.
4. `has Register False`.
5. `Chair` with `component` is `IsComponent True`. After `SetKind("door")`, `GetKind` is `door` but `IsComponent` is **False**.

Run `python-labs/lab32_kinds_fallbacks.py` (script is in the repo; output below).

## Expected output

```
all ['assembly', 'component', 'group', 'model', 'subcomponent']
HasKind door False
base component model
base assembly group
base group model
base model ''
component IsA model True
subcomponent IsA model False
component IsA group False
has Register False
Chair IsComponent True
kind door still GetKind door
Chair IsComponent with door False
```

## Check your understanding

1. How do you register a custom kind prop on USD 26.08 Python?
2. Is `subcomponent` a model kind?
3. Can `GetKind()` return a token the registry does not know?

**Answers** 1. You **cannot** call `Kind.Registry.Register` — it does not exist. Ship **plugInfo.json**. 2. **No**. `IsA("subcomponent", "model")` is False. 3. **Yes**. Metadata stores the string; `IsModel/IsComponent` then fail closed.

## Stretch challenge

`Kind.Registry.IsModel("assembly")` and `IsComponent("assembly")`. Solution: True and False — `assembly` is a `group/model`, not a `component`.

## LAB 33

# Lab 33 — Xform ops, XformCache, and Gf.Camera

**Domain / objectives:** Visualization 8.x · **Chapter:** 39 · **Time:** 30 min · **Difficulty:** ★★☆



## Goal

Author a translate with `AddTranslateOp` (which fills `xformOpOrder`), catch a **second** `AddTranslateOp`, compose parent+child with `XformCache`, and convert a `UsdGeom.Camera` to `Gf.Camera` for horizontal FOV.

## Background

`AddTranslateOp()` creates `xformOp:translate` **and** appends it to `xformOpOrder`. Creating the attribute alone does not move anything (Ch 39). A second `AddTranslateOp()` without a suffix **raises**. World transforms include ancestors: Lid at local `(0,2,0)` under Box `(1,0,0)` is world `(1,2,0)`. `GetCamera(time)` returns a `Gf.Camera`, not another `UsdGeom` schema. Fallback focal **50**, horizontal aperture **20.955**. Focal 35 → hFOV **33.33** degrees.

## Steps

1. `xformOpOrder` is `['xformOp:translate']`.
2. Second `AddTranslateOp()` → `Tf.ErrorException`.
3. World Box `(1,0,0)`; Lid `(1,2,0)`.
4. Fallback 50 / 20.955. After focal 35: type Camera, Perspective, hFOV 33.33, position `(0,0,10)`.

Run `python-labs/lab33_xforms_cameras.py` (script is in the repo; output below).

### Expected output

```
order: ['xformOp:translate']
second AddTranslateOp: Tf.ErrorException
world Box: (1.0, 0.0, 0.0)
world Lid: (1.0, 2.0, 0.0)
fallback focal/hAperture: 50.0 20.955
Camera Gf.Camera.Perspective
hFOV 33.33
position (0.0, 0.0, 10.0)
```

## Check your understanding

1. Why use `AddTranslateOp` instead of `CreateAttribute("xformOp:translate")`?
2. What does `UsdGeom.Camera.GetCamera(t)` return?
3. Does a Camera look down +Z or -Z?

**Answers** 1. It also appends `xformOpOrder`. Without the order, nothing moves. 2. A `Gf.Camera` (math: FOV, frustum), not a `UsdGeom.Camera`. 3. **-Z** in local camera space (OpenUSD camera convention).

## Stretch challenge

`xf.AddTranslateOp(opSuffix="pivot")` after the first translate. Solution: order becomes `['xformOp:translate', 'xformOp:translate:pivot']` — suffixes allow a second translate (Ch 39).

## LAB 34

# Lab 34 — PreviewSurface and MaterialBindingAPI

**Domain / objectives:** Visualization 8.2, 8.3 · **Chapter:** 40 · **Time:** 30 min · **Difficulty:** ★★☆☆

## Goal

Author a **Material** with a child **Shader** whose `info:id` is `UsdPreviewSurface`, connect `outputs:surface`, and prove **Bind without Apply writes the relationship but HasAPI stays False**. Then `Apply + Bind` and compute the bound material.

## Background

UsdPreviewSurface is an `info:id string`, not a prim type (Ch 40). Hydra draws a Shader connected to the Material terminal. **MaterialBindingAPI.Apply(prim) then .Bind(material)** is required. Skipping Apply still writes `rel material:binding` and Bind can succeed, but validators fail HasAPI. `ComputeSurfaceSource()` returns a **tuple** (length 3 on 26.08), not a single shader.

### VERSION

Verified on USD 26.08. Bind without Apply writes the rel; ``HasAPI`` is False until Apply.

## Steps

1. Bind without Apply: HasAPI False, has rel True.
2. After Apply: HasAPI True; bound path `/Looks/Paint`.
3. Shader id UsdPreviewSurface. Surface source tuple length 3.

Run `python-labs/lab34_preview_binding.py` (script is in the repo; output below).

### Expected output

```
HasAPI without Apply: False
has rel: True
HasAPI after Apply: True
bound: /Looks/Paint
shader id: UsdPreviewSurface
surface source len: 3
```

## Check your understanding

1. Where is `info:id = "UsdPreviewSurface"` authored?
2. Is Bind-without-Apply a no-op?
3. What does `ComputeSurfaceSource()` return?

**Answers** 1. On the **Shader** child, not on the Material prim. 2. **No**. It still writes `material:binding`. HasAPI stays False until **Apply**. 3. A **tuple** (length 3 here), not a single shader object.

## Stretch challenge

Connect `UsdPrimvarReader_float3 (varname = displayColor)` to `inputs:diffuseColor`.  
Solution: Obj 8.4 — PreviewSurface does not read mesh primvars by itself (Ch 40.5).

### LAB 35

## Lab 35 — UsdLux lights, LightAPI, intensity $\times 2^{\text{exposure}}$

**Domain / objectives:** Visualization 8.x · **Chapter:** 41 · **Time:** 25 min · **Difficulty:** ★★☆☆

### Goal

Compare **DistantLight intensity fallback 50000** with **SphereLight 1**, prove **LightAPI is built-in** (no Apply), and scale a bulb as **intensity  $\times 2^{\text{exposure}}$** .

## Background

UsdLux typed lights carry **LightAPI automatically** — unlike `MaterialBindingAPI` on a Mesh (Ch 41). Distant (sun) fallback **50000**; local lights and Dome **1**. Distant is **not** Boundable; Sphere is. Brightness  $\propto$  **intensity  $\times 2^{\text{exposure}}$** . Color temperature is ignored until **enableColorTemperature** is true (fallback False). Attribute names are **inputs:intensity** since 21.02.

## Steps

1. Distant I=50000.0; Sphere I=1.0.
2. HasAPI(LightAPI) True; applied schemas include LightAPI and collection APIs.
3. Distant not boundable; Sphere is. Sun angle  $\approx 0.53^\circ$ .
4. Intensity 3, exposure 2  $\rightarrow$  scale **12.0**. Color-temp enable fallback False.

Run `python-labs/lab35_usdlux.py` (script is in the repo; output below).

### Expected output

```
distant I: 50000.0
sphere I: 1.0
HasAPI LightAPI: True
applied: ['LightAPI', 'CollectionAPI:lightLink', 'CollectionAPI:shadowLink']
distant boundable: False
sphere boundable: True
sun angle: 0.53
bulb I/E: 3.0 2.0
scale intensity*2**exposure: 12.0
enableColorTemp fallback: False
```

## Check your understanding

1. Unauthenticated Distant `GetIntensityAttr().Get()`?
2. Must you `LightAPI.Apply` before setting intensity on a `DistantLight`?
3. Exposure 2 means plus 2 intensity, or  $\times 4$ ?

**Answers** 1. **50000.0**, not 1. 2. **No**. `LightAPI` is **built into** typed lights. Apply `ShadowAPI/ ShapingAPI` only. 3.  **$\times 4$**  ( $2^2$ ). Stops, not additive.

## Stretch challenge

`UsdLux.ShapingAPI.Apply(bulb.GetPrim())` and print `HasAPI`. Solution: True after `Apply` — `ShapingAPI` is optional, unlike `LightAPI` (Ch 41.2).

### LAB 36

## Lab 36 — Debugging toolkit: delegate, Trace, MallocTag

**Domain / objectives:** Debugging 6.5 · **Chapter:** 43 · **Time:** 30 min · **Difficulty:** ★★★

### Goal

Capture a missing-reference **warning** with `UsdUtils.CoalescingDiagnosticDelegate`, pair it with `GetCompositionErrors()`, prove **Trace starts disabled**, and show `Tf.MallocTag` **reports 0 bytes** on this usd-core wheel.

### Background

Construct the delegate **before** `Stage.Open` or you miss the warning (Ch 43). The class is `UsdUtils.CoalescingDiagnosticDelegate`, not `Tf.CoalescingDiagnosticDelegate`. `sourceFileName` may be a full path — **basename** it. `Trace = time`; `MallocTag = bytes`. `MallocTag.Initialize()` can return False while `IsInitialized()` becomes True; total bytes stay **0** on the pip wheel.

#### VERSION

Verified on USD 26.08. ``Tf.MallocTag.GetTotalBytes()`` is 0 after `Initialize` on usd-core.

## Steps

1. One uncoalesced item; code `TF_DIAGNOSTIC_WARNING_TYPE`; commentary mentions `missing.usda`; source `stage.cpp`.
2. One Pcp composition error as well.
3. Trace enabled default False; collector label is the global collector.
4. MallocTag: before False, Initialize returned False, after True, **total bytes 0**.

Run `python-labs/lab36_debugging_toolkit.py` (script is in the repo; output below).

### Expected output

```
n: 1
code: TF_DIAGNOSTIC_WARNING_TYPE
warning: True
mentions missing: True
source file: stage.cpp
pcp errors: 1
trace enabled default: False
collector label: TraceRegistry global collector
malloc before: False
Initialize returned: False
malloc after: True
total bytes: 0
```

## Check your understanding

1. When must you construct the coalescing delegate?
2. Trace vs MallocTag — which measures time?
3. Does `Initialize()` returning False mean MallocTag is broken?

**Answers** 1. **Before** `Stage.Open`. After misses the warning. 2. **Trace** is time. MallocTag is bytes. 3. **No** on this wheel. `IsInitialized()` is True; bytes stay 0 because the pip allocator is untagged.

## Stretch challenge

Call `TakeUncoalescedDiagnostics()` a second time. Solution: empty — the first take **drains** the queue (Ch 43.3).

---

### LAB 37

## Lab 37 — Capstone: publish a component, assemble, validate, inventory

**Domain / objectives:** Content Aggregation 2.4 · Data Exchange 4.6 · Pipeline 7.2 ·

**Chapter:** 23, 30, 32 · **Time:** 35 min · **Difficulty:** ★★★

### Goal

Publish a **component** chair with stage metrics and `defaultPrim`, pass **UsdValidation** metadata checkers, reference it into an **assembly** → **group** kitchen, and ship-check with `ComputeAllDependencies`. Print `Usd.GetVersion()`.

### Background

This lab stitches Labs 22, 27, and 28 (Ch 23, 30, 32). A published component needs **defaultPrim**, **upAxis**, **metersPerUnit**, and **kind**. An assembly uses **group** organizers so referenced chairs stay valid models. `ComputeAllDependencies` on the kitchen lists both layers and an empty unresolved list — the publish gate.

## Steps

1. Chair validators report **0** errors; kind component.
2. assembly is-a group: True.
3. /Kitchen/Props/Chair\_0 is a valid component Xform.
4. Layers chair.usda + kitchen.usda; unresolved [].
5. Version (0, 26, 8).

Run `python-labs/lab37_capstone.py` (script is in the repo; output below).

### Expected output

```
chair errors: 0
chair kind: component
assembly is-a group: True
Chair_0 IsComponent: True
Chair_0 type: Xform
layers: ['chair.usda', 'kitchen.usda']
unresolved: []
USD version: (0, 26, 8)
```

## Check your understanding

1. Why would Chair\_0 fail IsComponent if Props had no kind?
2. What does a non-empty unresolved list mean at publish time?
3. Is `Usd.GetVersion()` (26, 8) or (0, 26, 8)?

**Answers** 1. The model hierarchy **breaks** without a group/assembly ancestor (Lab 22). 2. A dependency file is **missing** — fail the publish (Lab 28). 3. **(0, 26, 8)**. The leading month zero is dropped in the pip name 26.8.

## Stretch challenge

Drop `upAxis` from `chair.usda` and re-run the geom checker. Solution: `MissingUpAxisMetadata` Error (Lab 27); the kitchen would still compose the chair.

## USDA Reading Workbook

---

### USDA reading exercises

**Original exercises.** Not NVIDIA exam content. Each item shows one or more USDA files, then a composed-value / existence question. Answers are at the end. Verified by opening the layers with usd-core 26.8.

Target: 30. This file currently holds **U-001-U-030** (complete).

One-line `{ double x = 1 }` bodies **do not parse** — every file here is multiline.

---

#### U-001 · Sublayer strength · Obj 1.5

File: `strong.usda`

```
#usda 1.0
over "Ball"
{
 double radius = 3
}
```

File: `weak.usda`

```
#usda 1.0
over "Ball"
{
 double radius = 7
}
```

File: `root.usda`

```
#usda 1.0
(
 subLayers = [
 @./strong.usda@,
 @./weak.usda@
]
)
def Sphere "Ball"
{
}
```

**Question:** What is composed `/Ball.radius`?

---

#### U-002 · Inherit vs reference · Obj 1.1

File: `asset.usda`

```
#usda 1.0
(
 defaultPrim = "X"
)
def Sphere "X"
{
 double radius = 1
}
```

File: `shot.usda`

```
#usda 1.0
class "_I"
{
 double radius = 3
}

def "Ball" (
 prepend inherits = </_I>
 prepend references = @asset.usda@
)
{
}
```

**Question:** What is composed `/Ball.radius`?

---

**U-003 • Local vs variant • Obj 1.8**

```
#usda 1.0
def "Asset" (
 prepend variantSets = "size"
 variants = {
 string size = "big"
 }
)
{
 double height = 1
 variantSet "size" = {
 "big" {
 double height = 3
 }
 }
}
```

**Question:** Selection is `big`. Composed height?

---

**U-004 • over alone • Obj 1.8**

```
#usda 1.0
over "Hero"
{
 int hp = 10
}
```

**Question:** Is `/Hero` defined? Does `Traverse()` list it?

---

**U-005 • Missing defaultPrim • Obj 1.3**

File: `nodefault.usda`

```
#usda 1.0
def Cube "Box"
{
 double size = 9
}
```

Python: `prim.GetReferences().AddReference("nodefault.usda")` (no explicit path).

**Question:** Composed type name of `prim`, and composition error family?

---

**U-006 • Layer offset samples • Obj 1.4**

File: `anim.usda`

```
#usda 1.0
(
 defaultPrim = "Ball"
)
def Xform "Ball"
{
 double tx.timeSamples = {
 0: 0,
 10: 10,
 }
}
```

File: `shot.usda`

```
#usda 1.0
def "B" (
 prepend references = @anim.usda@ (
 offset = 10
)
)
{
}
```

**Question:** Composed `tx` time-sample times on `/B`?

---

**U-007 • LoadNone • Obj 1.3**File: `hi.usda`

```
#usda 1.0
(
 defaultPrim = "Root"
)
def Xform "Root"
{
 def Cube "Hi"
 {
 }
}
```

File: `city.usda`

```
#usda 1.0
def Xform "W"
{
 def Xform "A" (
 prepend payload = @./hi.usda@
)
 {
 }
 def Xform "C" (
 prepend references = @./hi.usda@
)
 {
 }
}
```

Opened with `Usd.Stage.Open("city.usda", Usd.Stage.LoadNone)`.**Question:** Does `/W/C/Hi` exist? Does `/W/A/Hi` exist?**U-008 • Reference vs specialize • Obj 1.6**

```
#usda 1.0
def Sphere "Asset"
{
 double radius = 1
}

def "_S"
{
 double radius = 7
}

def "Ball" (
 prepend references = </Asset>
 prepend specializes = </_S>
)
{
}
```

**Question:** Composed `/Ball.radius`?**U-009 • Mute • Obj 1.8 / 6.2**Same files as U-001. After `stage.MuteLayer(strongIdentifier):`**Question:** Composed `/Ball.radius`?**U-010 • Inherit broadcast • Obj 1.1**

```
#usda 1.0
class "_Look"
{
 float roughness = 0.2
}

def "A" (
 prepend inherits = </_Look>
)
```



```
{
 float roughness = 0.9
}

def "B" (
 prepend inherits = </_Look>
)
{
}
```

**Question:** Composed /A.roughness and /B.roughness?

### U-011 · Prepend vs append references · Obj 1.1

File: r1.usda — Sphere radius = 2 with defaultPrim = "X".

File: r2.usda — Sphere radius = 8 with defaultPrim = "X".

```
#usda 1.0
def "P" (
 prepend references = @r1.usda@
 append references = @r2.usda@
)
{
}
```

**Question:** Composed /P.radius?

### U-012 · Inherit vs variant · Obj 1.1 / 1.6

```
#usda 1.0
class "_I"
{
 double radius = 3
}

def Sphere "Ball" (
 prepend inherits = </_I>
 prepend variantSets = "look"
 variants = {
 string look = "hi"
 }
)
{
 variantSet "look" = {
 "hi" {
 double radius = 10
 }
 }
}
```

**Question:** Composed /Ball.radius?

### U-013 · Three sublayers · Obj 1.5

l1/l2/l3 author radius 1, 2, 3. Root:

```
#usda 1.0
(
 subLayers = [
 @./l1.usda@,
 @./l2.usda@,
 @./l3.usda@
]
)
def Sphere "Ball"
{
}
```

**Question:** Composed radius?

### U-014 · Explicit reference path · Obj 1.3

File: ex.usda has def Cube "Only" { double size = 1 } and no defaultPrim.

```
#usda 1.0
def "P" (
 prepend references = @ex.usda@</Only>
)
{
}
```

**Question:** Composed type and size of /P?

### U-015 • Stronger default vs weaker samples • Obj 1.8

Stronger sublayer: `double radius = 5.`

Weaker sublayer: `samples { 0: 0, 10: 10 }.`

**Question:** `Get()`, `Get(10)`, and `GetTimeSamples()`?

### U-016 • Nested over under instanceable • Obj 1.10 / 2.5

File: `chair.usda` — `defaultPrim = "Chair", Cube /Chair/Seat size = 1.`

```
#usda 1.0
def Xform "Room"
{
 def "A" (
 prepend references = @chair.usda@
 instanceable = true
)
 {
 over "Seat"
 {
 double size = 9
 }
 }
}
```

**Question:** Composed `/Room/A/Seat.size`? Does default `Traverse()` list `/Room/A/Seat`?

### U-017 • Variant with no selection • Obj 1.7

```
#usda 1.0
def "Asset" (
 prepend variantSets = "size"
)
{
 variantSet "size" = {
 "big" {
 double height = 3
 }
 "small" {
 double height = 1
 }
 }
}
```

**Question:** Composed `height` with no selection authored?

### U-018 • Layer offset with scale • Obj 1.4

Asset samples at layer times 0 and 10. Reference `Sdf.LayerOffset(10, 2)` (offset 10, scale 2).

**Question:** Composed sample times?

### U-019 • delete list-op • Obj 1.1

Weaker layer: `prepend references = [@r1.usda@, @r2.usda@]` (radii 2 and 8).

Stronger: `delete references = [@r1.usda@].`

**Question:** Composed radius?

### U-020 • Relationship list-edit • Obj 1.6

Weaker: `rel looks = </MatA>.` Stronger sublayer: `prepend rel looks = </MatB>.`

**Question:** Composed `looks` targets (strongest first)?

**U-021 • Implied inherits • Obj 1.1**

Asset Char inherits /\_Look with roughness = 0.2. Shot:

```
#usda 1.0
class "_Look"
{
 float roughness = 0.9
}

def "Hero" (
 prepend references = @char.usda@
)
{
}
```

**Question:** Composed /Hero.roughness?

**U-022 • Relocates • Obj 1.8**

Asset /Bot/Rig/Arm Sphere radius = 1. Shot:

```
#usda 1.0
(
 relocates = {
 </Bot/Rig/Arm>: </Bot/Arm>
 }
)
def "Bot" (
 prepend references = @./bot.usda@
)
{
 over "Rig"
 {
 over "Arm"
 {
 double radius = 9
 }
 }
 over "Arm"
 {
 double radius = 3
 }
}
```

**Question:** Does /Bot/Rig/Arm exist? /Bot/Arm.radius?

**U-023 • Broken model chain • Obj 2.4**

```
#usda 1.0
def Xform "Kitchen" (
 kind = "assembly"
)
{
 def Xform "Clutter"
 {
 def Xform "Mug" (
 kind = "component"
)
 {
 }
 }
 def Xform "Props" (
 kind = "group"
)
 {
 def Xform "Chair" (
 kind = "component"
)
 {
 }
 }
}
```

**Question:** `Mug.GetKind()` / `IsComponent()`? `Chair.IsComponent()`?

---

### U-024 • Explicit references list • Obj 1.1

Weaker prepends `r1` then `r2`. Stronger authors `references = [@r2.usda@]` (explicit, no prepend).

**Question:** Composed radius (`r1=2`, `r2=8`)?

---

### U-025 • Bind without Apply (USDA) • Obj 8.3

```
#usda 1.0
def Mesh "Board"
{
 rel material:binding = </Looks/Paint>
}

def Xform "Looks"
{
 def Material "Paint"
 {
 }
}
```

**Question:** `HasRelationship("material:binding")`? `HasAPI(MaterialBindingAPI)`?

---

### U-026 • Local vs inherit • Obj 1.1

```
#usda 1.0
class "_I"
{
 double radius = 3
}

def Sphere "Ball" (
 prepend inherits = </_I>
)
{
 double radius = 1
}
```

**Question:** Composed radius?

---

### U-027 • Schema fallback in USDA • Obj 5.2

```
#usda 1.0
def Sphere "S"
{
}
```

**Question:** Composed `radius` (nothing authored)?

---

### U-028 • defaultPrim present • Obj 1.3

Contrast U-005. File `box.usda`:

```
#usda 1.0
(
 defaultPrim = "Box"
)
def Cube "Box"
{
 double size = 9
}
```

`AddReference("box.usda")` with no path.

**Question:** Type and `size`?

---

### U-029 • Imageable purpose • Obj 5.5 / 6.4

```
#usda 1.0
def Mesh "M"
{
}
```

```
uniform token purpose = "guide"
}
```

**Question:** Authored purpose? Why might it vanish in the default usdview pass?

### U-030 • Typeless def vs over • Obj 1.8

File A is U-004 (over "Hero"). File B:

```
#usda 1.0
def "Hero"
{
 int hp = 10
}
```

**Question:** For B, is /Hero defined and in Traverse()? Type name?

## Answers

**U-001 — 3.** First listed sublayer (strong.usda) wins over weak (7) and over the unauthored Sphere fallback. Resolution: local stack, first sublayer. Review: §15.1.

**U-002 — 3.** Inherits beat references (I before R). Review: §21.2, lab 18.

**U-003 — 1.** Local height = 1 is L; variant 3 is V; L beats V even when size = big. Review: §18.6.

**U-004 — Not defined; Traverse empty.** over overlays; it does not define. GetPrimAtPath still returns a handle. Review: §20.1.

**U-005 — Type '';** Pcp.ErrorType\_UnresolvedPrimPath. AddReference returns True. Missing defaultPrim is not InvalidAssetPath (that is a missing file). Review: §16.2, lab 14.

**U-006 — [10.0, 20.0].** stageTime = offset + scale \* layerTime with offset 10, scale 1. Review: §16.4.

**U-007 — /W/C/Hi True (reference still loads); /W/A/Hi False (payload unloaded).** Review: §17.2, lab 16.

**U-008 — 1.** References beat specializes. Review: §19.3.

**U-009 — 7.** Muting strong.usda drops it from the stack; weak 7 remains. Review: §15.4.

**U-010 — A 0.9, B 0.2.** Local on A does not change B; B still inherits the class. Review: §19.2, lab 18.

**U-011 — 2.** Prepend (r1) is stronger than append (r2). Review: §14.4.

**U-012 — 3.** Inherits beat the selected variant's 10. Review: §21.2.

**U-013 — 1.** First listed sublayer wins. Review: §15.1.

**U-014 — Cube, size 1.** Explicit </Only> does not need defaultPrim. Review: §16.2.

**U-015 — Get()/Get(10) = 5; samples [].** Stronger default wins at every time. Review: §21.3.

**U-016 — size 1.0; Traverse is ['/Room', '/Room/A'] (no Seat).** Nested over on a proxy is ignored; proxies are skipped. Review: lab 23.

**U-017 — None (attribute not composed).** No selection → variant opinions off. Review: §18.2.

**U-018 — [10.0, 30.0].** 10 + 2 \* {0, 10}. Review: §16.4.

**U-019 — 8.** Delete removes r1; r2 remains. Review: §14.4.

**U-020 — ['/MatB', '/MatA'].** List-ops compose; prepend is stronger. Review: §14.4.

**U-021 — ~0.9.** Shot class "\_Look" contributes via implied inherits. Review: §19.5.

**U-022 — /Bot/Rig/Arm does not exist; /Bot/Arm.radius is 3.** Source-path over (9) is ignored (Pcp warning). Review: §20.3.

**U-023 — Mug kind component but IsComponent/IsModel False; Chair True/True.** Un-kindred Clutter breaks the model chain. Review: lab 22.

**U-024 — 8.** Explicit list replaces weaker preprends. Review: §14.4.

**U-025 — Rel True; HasAPI False.** Bind-without-Apply in USDA. Review: lab 34.

**U-026 — 1.** Local beats inherit. Review: §19.2.

**U-027 — 1.0.** Sphere schema fallback. Review: §13.1.

**U-028 — Cube, size 9.** defaultPrim supplies the target. Review: §16.2.

**U-029 — guide.** Default purpose mask omits guide/proxy. Review: §13.5.

**U-030 — Defined, in Traverse, type '' (typeless).** def without a type still defines. Review: §20.1.

---

*USDA reading exercises complete: U-001-U-030 (target 30).*

## Question bank

### Question Bank — Fundamentals (FUN)

**Original practice questions.** Not actual NVIDIA exam content. Not blueprint-weighted; they cover Part I (Ch 1-7) so a complete beginner can sit the domain banks. Verified against **USD 26.08** (usd-core 26.8).

Target: 30. This file holds **FUN-001-FUN-030** (domain complete).

Work the stems first. Answers are at the end.

**FUN-001** · Foundations · Difficulty: Easy · Type: Single choice

OpenUSD is best described as:

A. Only a file extension `.usd` B. A **scene description** system (composition, schemas, APIs) that *includes* file formats C. A renderer that replaced Hydra D. A Pixar-only closed format

**FUN-002** · Foundations · Difficulty: Easy · Type: Single choice

Python imports live in which package?

A. `usd` B. `pxr` (from `pxr import Usd`) C. `openusd` D. `nvidia`

**FUN-003** · Foundations · Difficulty: Medium · Type: Single choice

Stack order: Tf, Gf, Vt, Ar, Sdf, Pcp, Usd. Which layer is **scene composition** of opinions?

A. Tf B. Sdf C. **Pcp** D. Hydra

**FUN-004** · Foundations · Difficulty: Easy · Type: Single choice

A **stage** is:

A. One USDA file on disk only B. The live **composed** scene (`Usd.Stage`) built from layers and arcs C. A Hydra render delegate D. `plugInfo.json`

**FUN-005** · Foundations · Difficulty: Easy · Type: Single choice

`Usd.GetVersion()` on this book's environment?

A. (26, 8, 0) B. (0, 26, 8) C. (24, 11) D. "usd-core"

**FUN-006** · Foundations · Difficulty: Easy · Type: Multiple select Select two.

usd-core 26.8 **does** include / **does not** include:

A. Does: `Usd`, `UsdGeom`, `UsdShade`, `UsdLux` Python modules B. Does not: `usdcat` / `usdview` on PATH C. Does: Hydra `UsdImaging` in the wheel D. Does: `UsdMtlx`

**FUN-007** · Foundations · Difficulty: Medium · Type: Single choice

`CreateInMemory()` vs `CreateNew("a.usda")`?

A. Both always write disk immediately B. In-memory has no path until Export/Save-as; `CreateNew` makes a **file-backed** root layer C. `CreateNew` cannot save D. In-memory cannot Define prims

**FUN-008** · Foundations · Difficulty: Easy · Type: Single choice

Fresh `CreateInMemory upAxis` and `metersPerUnit` fallbacks?

A. Z and 1.0 B. **Y and 0.01** C. None and None D. Y and 1.0

**FUN-009** · Foundations · Difficulty: Easy · Type: Single choice

Python stand-in for `usdcat` on usd-core?

A. There is none B. `stage.GetRootLayer().ExportToString()` / `Export` C. `print(stage)` D. `GetMaster()`

**FUN-010** · Foundations · Difficulty: Medium · Type: Single choice

A **layer** (`Sdf.Layer`) vs the stage?

A. Identical objects B. A layer is one **opinion container** (file or anonymous); the stage **composes** many layers C. Layers cannot hold prims D. Only USDC is a layer

**FUN-011** · Foundations · Difficulty: Medium · Type: Single choice

Session layer vs root layer: composed `radius 9` vs `1`, then `Save()` the root. Root file contains?

A. 9 B. **1** — session is not written into the root C. Both averaged D. Save fails

**FUN-012** · Foundations · Difficulty: Medium · Type: Single choice

`subLayers = [@b.usda@, @a.usda@]` with b radius 5 and a radius 9. Composed?

A. 9 (last listed) B. **5** (first listed is stronger) C. 7 D. None

**FUN-013** · Foundations · Difficulty: Easy · Type: Single choice

A **prim** is:

A. Only a mesh B. A **namespace object** on the stage (`Xform`, `Sphere`, `typeless`, ...) C. An attribute D. A layer offset

**FUN-014** · Foundations · Difficulty: Easy · Type: Single choice

Specifiers `def`, `over`, `class` mean:

A. The same B. **Define / overlay** (does not by itself define) / **abstract** class C. File formats D. Kinds

**FUN-015** · Foundations · Difficulty: Medium · Type: Single choice

Opened alone, over `"Hero" { int hp = 10 }` — `Traverse()`?

A. Lists `/Hero` B. **Empty** — `over` alone is not defined C. Raises D. Lists a Cube

**FUN-016** · Foundations · Difficulty: Easy · Type: Single choice

Attribute vs relationship?

A. Both store `double` values B. Attributes store **typed values**; relationships store **prim/property targets** C. Relationships are only for materials D. Attributes cannot have defaults

**FUN-017** · Foundations · Difficulty: Easy · Type: Single choice

`primvars:`, `xformOp:`, `inputs:` are:

A. Composition arcs B. **Property namespaces** C. File extensions D. Kinds

**FUN-018** · Foundations · Difficulty: Easy · Type: Single choice

Five built-in model kinds?

A. `door`, `window`, `wall`, `roof`, `floor` B. **model**, **group**, **assembly**, **component**, **subcomponent** C. `Mesh`, `Xform`, `Camera`, `Light`, `Material` D. `usda`, `usdc`, `usd`, `usdz`, `abc`

**FUN-019** · Foundations · Difficulty: Medium · Type: Single choice

Typed schema vs API schema?

A. No difference B. Typed is **IsA** (Cube); API is **applied** (`MaterialBindingAPI.Apply`) C. API replaces kinds D. Typed schemas cannot have attributes

**FUN-020** · Foundations · Difficulty: Easy · Type: Single choice

`.usda` vs `.usdc` vs `.usd` vs `.usdz`?

A. All crate binary B. Text / crate binary / **either** (detect magic) / **zip package** C. All require Hydra D. `.usd` is always text

**FUN-021** · Foundations · Difficulty: Medium · Type: Single choice

`CreateNew("scene.usd")` magic and `formatId`?

A. `#usda`, `usda` B. **PXR-USDC**, **usd** C. `PXR-USDC`, `usdc` D. `PK`, `usdz`

**FUN-022** · Foundations · Difficulty: Easy · Type: Single choice

USDA files must start with:

A. `<?xml` B. **#usda 1.0** C. `PXR-USDC` D. `{`

**FUN-023** · Foundations · Difficulty: Medium · Type: Single choice

`GetPrimAtPath("/NoSuch")` — `bool(prim)`?

A. True B. **False** (invalid handle; `GetPrimAtPath` does not raise) C. Raises `KeyError` D. Defines the prim

**FUN-024** · Foundations · Difficulty: Easy · Type: Single choice

`defaultPrim` is stored on:

A. Each attribute B. **The layer** (stage metadata) C. `Hydra` D. `Kind.Registry`

**FUN-025** · Foundations · Difficulty: Medium · Type: Single choice

Anonymous layer (`CreateAnonymous`) in the local stack?

A. Always the root B. A layer **without a file path**; not automatically in the stack until you `sublayer/set` it C. The same as session D. Cannot hold prims

**FUN-026** · Foundations · Difficulty: Easy · Type: Single choice

`Traverse()` vs `GetPrimAtPath`?

A. Identical B. `Traverse` **walks defined prims** (skipping some overs/class/instance proxies); `GetPrimAtPath` is a **direct lookup** C. `GetPrimAtPath` walks `Hydra` D. `Traverse` cannot see Spheres

**FUN-027** · Foundations · Difficulty: Medium · Type: Multiple select Select two.

Opinions (first look, Ch 3):

A. Strongest opinion for a scalar field **wins** (no averaging) B. First listed sublayer is stronger than later ones C. Weaker layers delete stronger files D. Session is weaker than the root

**FUN-028** · Foundations · Difficulty: Easy · Type: Single choice

Pixar opened USD; the alliance for open standards is often abbreviated:

A. NVIDIA-only B. **AOUSD** (Alliance for OpenUSD) C. `glTF` D. `USDZ Inc.`

**FUN-029** · Foundations · Difficulty: Medium · Type: Single choice

Hydra's role in the stack?

A. Composition of layers B. **Imaging / render-delegate** architecture (not in `usd-core`) C. Asset resolution D. `Kind registry`

**FUN-030** · Foundations · Difficulty: Easy · Type: Single choice



UsdGeom.Sphere.Define(stage, "/World/Ball") then GetRadiusAttr().Set(2) — lab 02  
 USDA shows:

A. float radius = 2.0 B. **double radius = 2**  
 (USDA may drop .0) C. int radius = 2 D.  
 Radius cannot be authored

---

## Answers

**FUN-001 — Answer: B.** Scene description, not “just a file.” Review: §1.1.

**FUN-002 — Answer: B.** pxr. Review: §1.5.

**FUN-003 — Answer: C.** Pcp = prim cache population / composition. Review: §1.4.

**FUN-004 — Answer: B.** Stage = composed view. Review: §2.1.

**FUN-005 — Answer: B.** (0, 26, 8). Review: lab 01.

**FUN-006 — Answer: A, B.** Schema modules yes; CLI/Hydra/Mtlx no. Review: lab 01, F5.

**FUN-007 — Answer: B.** File-backed vs memory. Review: §2.2.

**FUN-008 — Answer: B.** Y, 0.01. Review: lab 08, §28.

**FUN-009 — Answer: B.** ExportToString. Review: F5, lab 02.

**FUN-010 — Answer: B.** Layer vs stage. Review: §3.1.

**FUN-011 — Answer: B.** Session not in root Save. Review: lab 04.

**FUN-012 — Answer: B.** First listed wins. Review: lab 05.

**FUN-013 — Answer: B.** Prim = namespace object. Review: §4.1.

**FUN-014 — Answer: B.** def/over/class. Review: §4.3.

**FUN-015 — Answer: B.** over alone not defined. Review: §20.1.

**FUN-016 — Answer: B.** Value vs targets. Review: §5.2-5.3.

**FUN-017 — Answer: B.** Namespaces. Review: §5.5.

**FUN-018 — Answer: B.** Five kinds. Review: §6.5, lab 32.

**FUN-019 — Answer: B.** IsA vs Apply. Review: §6.2-6.3.

**FUN-020 — Answer: B.** Four encodings. Review: §7.1-7.4.

**FUN-021 — Answer: B.** Crate magic, formatId usd. Review: §31.3.

**FUN-022 — Answer: B.** #usda 1.0. Review: §7.5.

**FUN-023 — Answer: B.** Invalid handle. Review: §4.2.

**FUN-024 — Answer: B.** Layer metadata. Review: §2.4.

**FUN-025 — Answer: B.** Anonymous not auto-stacked. Review: §3.5, lab 19.

**FUN-026 — Answer: B.** Walk vs lookup. Review: §2.3.

**FUN-027 — Answer: A, B.** Strongest wins; first sublayer. Session is strongest local. Review: §3.6.

**FUN-028 — Answer: B.** AOUSD. Review: §1.2.

**FUN-029 — Answer: B.** Imaging. Review: §1.4, §41.

**FUN-030 — Answer: B.** double radius = 2. Review: lab 02.

---

*Fundamentals domain complete: FUN-001–FUN-030 (target 30). Question bank exam+FUN total 430.*

## Question Bank — Composition (COMP)

**Original practice questions.** Not actual NVIDIA exam content. Mapped to NCP-OUUSD study-guide objectives 1.1–1.11 (Composition, 23%).

Verified against **USD 26.08** (usd-core 26.8). Domain target: 92 questions. This file currently holds **COMP-001–COMP-092** (domain complete).

Work the stems first. Answers and explanations are grouped at the end.

**COMP-001** · Obj 1.6 · Difficulty: Easy · Type: Single choice

What is the strength order of composition arcs, strongest to weakest, as spelled by LIVERPS?

A. Local, Inherits, VariantSets, rElocates, References, Payloads, Specializes B. Local, Inherits, VariantSets, References, Payloads, Specializes, rElocates C. Local, VariantSets, Inherits, rElocates, References, Payloads, Specializes D. Inherits, Local, VariantSets, References, Payloads, Specializes, rElocates

**COMP-002** · Obj 1.6 · Difficulty: Easy · Type: Single choice

In LIVERPS, what does the letter **E** stand for?

A. Expression arcs (value clips) B. Edit targets C. rElocates D. Extents

**COMP-003** · Obj 1.6 · Difficulty: Medium · Type: Multiple select Select two.

Which statements about LIVERPS are true?

A. A local opinion in the stage's layer stack always beats a referenced opinion on the same field. B. Specializes beat references, which is why they are a safe place to put hero overrides. C. Inherits beat variant opinions on the same field. D. Value clips are the "C" in LIVERPS and sit between variants and relocates.

**COMP-004** · Obj 1.1 · Difficulty: Easy · Type: Single choice

A referenced asset authors `radius = 1`. You need the composed value on this shot to be 9 without editing the asset file. Which change is strongest and sufficient?

A. Add a `class` prim that specializes the sphere with `radius = 9`. B. Author `double radius = 9` as a local opinion on the referencing prim (or a stronger local sublayer). C. Mute the root layer so the reference can win. D. Set `instanceable = true` on the referencing prim.

**COMP-005** · Obj 1.1 · Difficulty: Medium · Type: USDA-reading

File: `asset.usda`

```
#usda 1.0
(
 defaultPrim = "X"
)
def Sphere "X"
{
 double radius = 1
}
```

File: `shot.usda`

```
#usda 1.0
class "_I"
{
 double radius = 3
}

def "Ball" (
 prepend inherits = </ I>
 prepend references = @asset.usda@
)
{
}
```

What is the composed value of `/Ball.radius`?

A. 1 (the reference wins because it defines the Sphere type) B. 3 (inherits beat references) C. None (the two arcs cancel) D. 2 (USD averages conflicting numeric opinions)

**COMP-006** · Obj 1.1 · Difficulty: Medium · Type: Single choice

A prim both **references** an asset that authors `radius = 1` and **specializes** a class that authors `radius = 7`. There is no local `radius`. What is the composed value?

A. 7, because specializes are a "base class" and win like inherits B. 1, because references beat specializes C. None, because R and S conflict D. 7, but only if the specialize is listed before the reference in USDA

**COMP-007** · Obj 1.1 · Difficulty: Medium · Type: Multiple select Select two.

A prim has these list-edits on `references`:

```
#usda 1.0
def "P" (
 prepend references = @r1.usda@
 append references = @r2.usda@
)
{
}
```

`r1.usda` authors `radius = 2`; `r2.usda` authors `radius = 8`. Which statements are true?

A. The composed `radius` is 2. B. The composed `radius` is 8 because append is "later, therefore stronger." C. Prepended references are stronger than appended references. D. USD rejects a prim that both prepends and appends the same list field.

**COMP-008** · Obj 1.1 · Difficulty: Hard · Type: USDA-reading

```
#usda 1.0
class "_I"
{
 double radius = 3
}

def Sphere "Ball" (
 prepend inherits = </_I>
 prepend variantSets = "look"
 variants = {
 string look = "hi"
 }
)
{
 variantSet "look" = {
 "hi" {
 double radius = 10
 }
 }
}
```

What is the composed value of `/Ball.radius`?

- A. 10 (the selected variant is the “current look,” so it wins) B. 3 (inherits beat variant opinions) C. 1 (Sphere schema fallback) D. None (inherit and variant block each other)

**COMP-009** · Obj 1.3 · Difficulty: Easy · Type: Single choice

Which statement best distinguishes a **sublayer** from a **reference**?

- A. A sublayer contributes opinions in the *same* namespace as its parent layer; a reference brings another layer’s contents in at a prim path (often renaming the target to that prim). B. A sublayer is always weaker than a reference. C. A reference can only point at `.usdc` files; a sublayer can only point at `.usda` files. D. Sublayers load lazily like payloads; references always load.

**COMP-010** · Obj 1.3 · Difficulty: Medium · Type: Multiple select Select two.

Compared with a reference, a **payload** is the better choice when you need which of the following?

- A. The introduced opinions to beat local shot overrides B. The ability to omit the target’s descendants from the stage until Load (or a load-all open) C. Weaker-than-reference composition strength, so local and referenced opinions still win D. A replacement for `defaultPrim` (payloads do not use `defaultPrim`)

**COMP-011** · Obj 1.3 · Difficulty: Medium · Type: USDA-reading

```
#usda 1.0
def Xform "W"
{
 def Xform "A" (
```

```
 prepend payload = @./hi.usda@
)
 {
 }
 def Xform "C" (
 prepend references = @./hi.usda@
)
 {
 }
}
```

`hi.usda` has `defaultPrim = "Root"` and defines `/Root/Hi`. The stage is opened with `Usd.Stage.Open(path, Usd.Stage.LoadNone)`. Which statement is true?

- A. Neither `/W/A/Hi` nor `/W/C/Hi` exists, because `LoadNone` blocks every composition arc. B. `/W/C/Hi` exists (True); `/W/A/Hi` does not (False). References still load; payloads do not. C. Both `/W/A/Hi` and `/W/C/Hi` exist; `LoadNone` only skips textures. D. `/W/A/Hi` exists and `/W/C/Hi` does not, because payloads are stronger than references.

**COMP-012** · Obj 1.3 · Difficulty: Medium · Type: USDA-reading

```
#usda 1.0
def Cube "Box"
{
 double size = 9
}
```

Python:

`prim.GetReferences().AddReference("nodefault.usda")` returns `True`. The referencing prim has no explicit target path. What is the composed type name of the referencing prim, and which composition error is reported?

- A. Type `Cube`, error `none` — USD infers the first root prim B. Type `''` (empty), error `Pcp.ErrorType_UnresolvedPrimPath` C. Type `''`, error `Pcp.ErrorType_InvalidAssetPath` D. `AddReference` actually returns `False` and authors nothing

**COMP-013** · Obj 1.3 · Difficulty: Medium · Type: Multiple select Select two.

A lighting TD needs a shot that (1) stacks department layers that all edit `/World` in place, and (2) brings in a published chair asset under `/World/Chair_0`. Which pair of arcs matches those two jobs?

- A. Sublayers for the department stack B. A reference (or payload) for the chair asset C. An inherit for the department stack, because inherits share namespace D. A specialize for the chair, because specializes copy assets into a new path

**COMP-014** · Obj 1.4 · Difficulty: Easy · Type: Single choice

A reference is authored with `Sdf.LayerOffset(offset=10, scale=1)`. How does a **layer time** `t` in the referenced file map to **stage time**?

- A. `stageTime = t / 10` B. `stageTime = offset + scale * layerTime → 10 + t` C. `stageTime = layerTime` (offsets apply only to sublayers, never to references) D. `stageTime = t - 10`

**COMP-015** · Obj 1.4 · Difficulty: Medium · Type: USDA-reading

File: `anim.usda`

```
#usda 1.0
(
 defaultPrim = "Ball"
)
def Xform "Ball"
{
 double3 xformOp:translate.timeSamples = {
 0: (0, 0, 0),
 10: (10, 0, 0),
 }
 uniform token[] xformOpOrder = ["xformOp:translate"]
}
```

File: `shot.usda`

```
#usda 1.0
def "B" (
 prepend references = @anim.usda@ (
 offset = 10
)
)
{
}
```

What are the composed time-sample times on / B.xformOp:translate?

- A. `[0.0, 10.0]` (offsets do not move sample times) B. `[10.0, 20.0]` C. `[-10.0, 0.0]` D. `[]` (the offset strips samples and leaves only the default)

**COMP-016** · Obj 1.4 · Difficulty: Medium · Type: Single choice

The referenced asset authors both a **default** `xformOp:translate = (100, 0, 0)` and time samples at layer times 0 and 10. The reference uses `Sdf.LayerOffset(10)`. What does `attr.Get()` (no time argument) return on the referencing prim?

- A. `(0, 0, 0)` — the first sample, shifted to stage time 10, then read at default B. `(100, 0, 0)` — defaults are not shifted by a layer offset C. None — `Get()` never sees a default when samples exist D. `(10, 0, 0)` — USD adds the offset to each vector component

**COMP-017** · Obj 1.4 · Difficulty: Medium · Type: Single choice

A sublayer is included as `@anim.usda@ (offset = 10)`. You Set a time sample at stage time 20. Where is the sample stored in `anim.usda` if the edit target was created with `stage.GetEditTargetForLocalLayer(anim)` versus a bare `Usd.EditTarget(anim)`?

- A. Both store the sample at 20. B. `GetEditTargetForLocalLayer` stores at **10**; the bare `EditTarget` stores at **20**. C. `GetEditTargetForLocalLayer` stores at **20**; the bare `EditTarget` stores at **10**. D. Neither can author samples into a sublayer that has an offset.

**COMP-018** · Obj 1.5 · Difficulty: Easy · Type: Single choice

On a multi-user shot, layout, animation, and lighting each need to author opinions without overwriting each other's files. What is the standard OpenUSD pattern?

- A. One root layer with a `subLayers` list, strongest department first, each department owning one layer file B. One root layer that **specializes** each department file C. Native instancing of the shot so each TD gets a prototype D. Flattening the shot at the start of every session so everyone edits a copy

**COMP-019** · Obj 1.5 · Difficulty: Medium · Type: Single choice

You `SetEditTarget` to a freshly created anonymous layer that is **not** in the stage's local layer stack, then `DefinePrim("/Ghost")`. What happens?

- A. The prim appears on the composed stage because any edit target is automatically sublayered. B. `SetEditTarget` raises `Tf.ErrorException` (not in the local `LayerStack`); the prim is not authored on the stage. C. The prim is written to the session layer as a fallback. D. The prim is written to disk next to the root layer as `Ghost.usda`.

**COMP-020** · Obj 1.5 · Difficulty: Medium · Type: Single choice

A shot's default edit target is the root layer. Animation should land in `anim.usda`, which is a **sublayer** of that root (not a referenced asset). What should the TD do?

- A. `stage.SetEditTarget(stage.GetEditTargetForLocalLayer(anim))` (or an equivalent local-stack target) B. `stage.SetEditTarget` on the referenced chair's root layer, because chairs contain the animation C. Author in the session layer and hope `Save()` writes it into `anim.usda` D. Call `stage.Flatten()` first so there is only one layer to target

**COMP-021** · Obj 1.5 · Difficulty: Medium · Type: Multiple select Select two.

Which practices support a multi-user OpenUSD shot?

A. List stronger, later-arriving departments **first** in `subLayers` (first listed is stronger). B. Let every TD `Save()` into the same `shot.usda` root so Git can merge USDA. C. Keep the session layer for personal, unsaved experiments; do not publish it as the show file. D. Put each department's work on a referenced prim at a unique path such as `/layout` vs `/anim` so they never share `/World`.

**COMP-022** · Obj 1.7 · Difficulty: Easy · Type: Single choice

When are **variant sets** the appropriate structure for an asset?

A. When the asset has a small set of named, mutually exclusive options the consumer will pick (size, look, LODs) B. When you need to instance 50,000 identical trees C. When two departments must edit `/World.tx` on the same shot at the same time D. When you want opinions that always beat local shot overrides

**COMP-023** · Obj 1.7 · Difficulty: Medium · Type: Multiple select Select two.

In which situations are variants **not** the right tool?

A. Encoding a continuous slider (any float mass between 0 and 100) as hundreds of named variants B. Switching a robot between a handful of published looks (`clean`, `dirty`, `damaged`) C. Replacing composition itself: putting every shot's animation clips inside the character's variant set D. Choosing among two geometry LODs on a component (`full`, `proxy`)

**COMP-024** · Obj 1.7 · Difficulty: Medium · Type: USDA-reading

A Python author does this (no variant is selected first):

```
vs =
prim.GetVariantSets().AddVariantSet("size")
vs.AddVariant("big")
with vs.GetVariantEditContext():
 prim.GetAttribute("height").Set(3)
```

What is authored?

A. `height = 3` inside variant `big` B. `height = 3` as a **direct local** opinion on the prim; variant `big` stays empty C. Nothing; `GetVariantEditContext` raises if there is no selection D. A new reference to an anonymous layer that holds `height = 3`

**COMP-025** · Obj 1.7 · Difficulty: Medium · Type: Single choice

A facility wants a "hero" chair that is a completely different mesh and material graph, published by a different team, swapped in at shot time. Why is a **reference** (or payload) a better swap than a variant on the hero chair itself?

A. Variants cannot store meshes; they only store tokens. B. A different published asset is a different composition *source*; variants switch options *inside* one asset's layer stack. Cross-asset swaps belong on the referencing (assembly/shot) side. C. References always beat variants, so the variant would never be visible. D. Payloads are the only arc allowed to introduce materials.

**COMP-026** · Obj 1.8 · Difficulty: Medium · Type: Multiple select Select two.

A stronger sublayer `strong.usda` authors `radius = 3`; a weaker sublayer authors `radius = 7`. After `stage.MuteLayer(strongIdentifier)`, the composed radius becomes `7`. Which statements explain related "my opinion vanished" cases?

A. Muting a layer removes it from the local stack, so its opinions no longer contribute. B. A muted layer's opinions still win if the prim is **instanceable**. C. Authoring on a layer that is **not** the current edit target (for example the root, while the target is a weaker sublayer) will not put the opinion where you think it went. D. Muting always deletes the USDA file from disk.

**COMP-027** · Obj 1.8 · Difficulty: Medium · Type: Single choice

`/Room/A` references a chair and is `instanceable = true`. `/Room/A/Seat` is an instance proxy. A TD calls `stage.OverridePrim("/Room/A/Seat")` to set `size = 9`. What happens?

A. The override wins locally; only this instance's seat becomes size 9. B. The call raises `Tf.ErrorException` (authoring to an instance proxy is not allowed); the nested opinion is not authored. C. USD silently writes the opinion onto the shared prototype, changing every instance. D. USD converts the prim to a `PointInstancer` automatically.

**COMP-028** · Obj 1.8 · Difficulty: Medium · Type: USDA-reading

```
#usda 1.0
def "Asset" (
 prepend variantSets = "size"
 variants = {
 string size = "big"
 }
)
{
 double height = 1
 variantSet "size" = {
```

```

 "big" {
 double height = 3
 }
}

```

The variant selection is **big**. What is the composed **height**, and why might a TD think the variant “did not take effect”?

- A. 3 — variants beat local opinions B. 1 — the local opinion is **L** and beats the variant opinion **V** C. None — selection and local opinion cancel D. 2 — USD blends local and variant numeric values

**COMP-029** · Obj 1.8 · Difficulty: Medium · Type: Single choice

A layer authors `relocates = { </Bot/Rig/Arm>: </Bot/Arm> }`. A TD then authors `over "Rig" { over "Arm" { double radius = 9 } }` on `/Bot` in that same local stack, intending to edit the arm. The composed `/Bot/Arm.radius` stays at the referenced value. Why?

- A. Relocates delete the prim, so no opinion can ever apply. B. After a relocate, opinions at the **source** path no longer apply to the destination; author `over "Arm"` at `/Bot/Arm` (or relocate after the edit is in the source asset). C. Relocates only work on payloads, and this prim was referenced. D. `radius` is a schema fallback and cannot be overridden.

**COMP-030** · Obj 1.9 · Difficulty: Easy · Type: Single choice

You must hand a vendor a single layer with **no composition arcs**. Which API is the right primary tool?

- A. `stage.Flatten()` — bakes references, payloads, inherits, variants, and sublayers into one layer B. `UsdUtils.FlattenLayerStack(stage)` — this also bakes references away C. `rootLayer.Export("out.usda")` — export always flattens D. `stage.Export()` to USDZ — USDZ forbids arcs by definition

**COMP-031** · Obj 1.9 · Difficulty: Medium · Type: Multiple select Select two.

`shot.usda` sublayers `bolt.usda` (a Cube named **Bolt**) and also **references** that same bolt at `/Refd`. After each operation, which descriptions match?

- A. `stage.Flatten()` produces a Cube `/Refd` with `size` copied in and **no** `references` field; `/Bolt` from the sublayer is also present as a Cube. B. `UsdUtils.FlattenLayerStack(stage)` **keeps** prepend references on `/Refd`, bakes the sublayer so `/Bolt` is local, and clears `subLayers`. C. Both APIs remove every

reference and every sublayer identically. D. `FlattenLayerStack` is the only way to include the session layer.

**COMP-032** · Obj 1.10 · Difficulty: Easy · Type: Single choice

Where do you author `instanceable = true` so many assembly prims share one prototype?

- A. On each **referencing** prim in the assembly (the instance root), not inside the component asset’s default prim as a required setting B. Only on the component’s `defaultPrim` in the published asset; assemblies must not set it C. On every descendant mesh (`Seat`, `Leg`), never on the component root D. On the stage metadata block next to `metersPerUnit`

**COMP-033** · Obj 1.10 · Difficulty: Medium · Type: Single choice

An assembly instances many chairs. You must remove or override a property on **one** chair’s nested `Seat` without changing the others. Which approach is valid?

- A. `OverridePrim` the nested proxy path while the prim stays instanceable B. Set `instanceable = false` on that one instance root (or de-instance), then author the nested override; the sibling instances keep the shared prototype C. Call `prim.GetMaster()` and `Clear()` the property on the master D. Mute the assembly root layer

**COMP-034** · Obj 1.10 · Difficulty: Medium · Type: Single choice

On USD 26.08, two instanceable prims that reference the same chair share a prototype. How do you obtain that prototype prim, and what is **not** available?

- A. `prim.GetPrototype()` returns a prim under `/_Prototype_N`; `GetMaster()` is **not** a `Usd.Prim` method on this version B. `prim.GetMaster()` is the only supported name; `GetPrototype()` was removed in 26.08 C. Both names exist and return different prims D. Prototypes are only accessible from `UsdGeom.PointInstancer`

**COMP-035** · Obj 1.11 · Difficulty: Easy · Type: Single choice

A monolithic `Robot.usda` holds geometry, materials, and a payload of high-resolution parts in one file, and two teams block on it. What is the first composition-friendly split?

- A. Separate published layers (interface/payload, geometry, materials) referenced or sublayered from a thin asset root, so each workstream owns a file B. Copy the file per artist and flatten daily

C. Convert the whole robot to a `PointInstancer`  
 D. Replace all `defs` with `class` so nobody's edits are concrete

**COMP-036** · Obj 1.11 · Difficulty: Medium · Type: Multiple select Select two.

When splitting a show into collaborative workstreams, which choices match OpenUSD practice?

A. Put **payloads on heavy component contents** so shots can open with proxies or remain unloaded B. Put every department's animation samples in the **component asset** variant set so shots stay empty C. Use a shot **sublayer stack** (layout, anim, lighting) so `/World` is shared and strength is explicit D. Author relocates in every component so department layers never need matching paths

**COMP-037** · Obj 1.2 · Difficulty: Easy · Type: Single choice

You need tens of thousands of similar trees with per-tree position, scale, and a cheap hide. Which instancing style fits?

A. Native `instanceable` references, one prim per tree B. A `UsdGeom.PointInstancer` with prototypes, `protoIndices`, and `positions` C. Unique `def` copies of the full tree asset for every tree D. One inherit arc from a class `_Tree` with no instancing

**COMP-038** · Obj 1.2 · Difficulty: Medium · Type: Single choice

A kitchen set has 40 hero chairs that must stay native instances for memory, plus **one** chair the director wants re-modeled at the `Seat` child. What do you do?

A. `PointInstancer` for all 41 chairs so `invisibleIds` can hide the hero B. Keep 40 `instanceable` references; set `instanceable = false` (or a unique reference) only on the hero chair so nested edits are legal C. Edit the shared prototype; accept that all 41 seats change D. Switch the set to payloads only — payloads disable instancing automatically

**COMP-039** · Obj 1.2 · Difficulty: Medium · Type: Multiple select Select two.

Choose an appropriate instancing style for each scale.

A. A handful of unique hero props that need arbitrary nested edits → ordinary references (or de-instanced prims), not native instances B. Hundreds of identical published chairs in a dining hall → native `instanceable` component references C. Millions of scattered pebbles → native `instanceable` Xforms, one prim each, so

Hydra can name them D. Native instances for the pebbles, because `PointInstancer` cannot instance more than 256 points

**COMP-040** · Obj 1.3 · Difficulty: Hard · Type: Single choice

You want a stage that contains `/World/A` but must not populate `/World/B` at all (not even as an unloaded payload prim). Which API is correct on USD 26.08?

A. `Usd.Stage.Open(path, mask=["/World/A"])` B. `Usd.Stage.OpenMasked(path, Usd.Stage.PopulationMask([Sdf.Path("/World/A")]))` C. `Usd.Stage.Open(path, Usd.Stage.LoadNone)` — this is equivalent to a population mask D. `stage.MuteLayer` on the layer that defines `/World/B`

**COMP-041** · Obj 1.1 · Difficulty: Medium · Type: Single choice

A weaker layer prepends `references = [@r1.usda@, @r2.usda@]` (`r1` has `radius = 2`, `r2` has `radius = 8`). A stronger sublayer authors `delete references = [@r1.usda@]`. What is the composed `radius`?

A. 2 (delete cannot remove a prepended arc) B. 8 (`r1` is removed; `r2` remains and is now the strongest remaining reference) C. None (delete empties the whole list) D. 5 (USD averages the remaining opinions)

**COMP-042** · Obj 1.8 · Difficulty: Hard · Type: Single choice

A stronger sublayer authors only a **default** `radius = 5`. A weaker sublayer authors **radius time samples** at 0 and 10. What does `Get()`, `Get(0)`, and `GetTimeSamples()` return?

A. `Get() = 5`, `Get(0) = 0`, samples `[0, 10]` — samples always show through B. `Get() = 5`, `Get(0) = 5`, `Get(10) = 5`, samples `[]` — the stronger default wins at every time C. `Get() = None` because samples exist on some layer D. Composition error; a field cannot mix a default and samples across layers

**COMP-043** · Obj 1.4 · Difficulty: Medium · Type: Single choice

A reference uses `Sdf.LayerOffset(10, 2)` (offset 10, scale 2). The asset has samples at layer times 0 and 10. What are the composed sample times?

A. `[0.0, 10.0]` B. `[10.0, 20.0]` (scale is ignored on references) C. `[10.0, 30.0]` because `stageTime = 10 + 2 * layerTime` D. `[5.0, 10.0]`

**COMP-044** · Obj 1.3 · Difficulty: Medium · Type: Single choice

On an in-memory stage you `DefinePrim("/Asset/Geo")`, set `sz = 4`, then `DefinePrim("/Inst")` and `AddInternalReference("/Asset/Geo")`. What is `/Inst.sz`?

A. None — internal references only work in files on disk B. 4 C. 0 — internal references copy the prim spec but drop attributes D. The call raises `Tf.ErrorException` because source and destination share a stage

**COMP-045** · Obj 1.3 · Difficulty: Medium · Type: Single choice

A stage has payloads on `/W/A` and `/W/B` and a **reference** on `/W/C`. What does `stage.FindLoadable()` return?

A. `['/W/A', '/W/B', '/W/C']` B. `['/W/A', '/W/B']` — only payload sites are loadable C. `['/W/C']` — references are the loadable arcs D. `[]` until you call `LoadNone`

**COMP-046** · Obj 1.3 · Difficulty: Medium · Type: Multiple select Select two.

A stage is opened with `LoadNone`. `/W/A` has a payload to an asset that defines `/Root/Hi`. Which operations are true?

A. Before `Load`, `/W/A/Hi` does not exist and `/W/A.IsLoaded()` is `False`. B. `stage.Load("/W/A")` makes `/W/A/Hi` exist; `Unload("/W/A")` removes it again. C. `Load("/W/A")` is ignored because `LoadNone` locks the stage forever. D. `FindLoadable()` includes referenced prims as well as payloads.

**COMP-047** · Obj 1.1 · Difficulty: Medium · Type: Single choice

Class `/_Look` authors `roughness = 0.2`. Both `/A` and `/B` inherit it. You then set `/A.roughness = 0.9` locally. What are the composed values?

A. A 0.9, B 0.9 (inherits always stay in sync) B. A 0.9, B 0.2 (local on A does not change B) C. A 0.2, B 0.2 (inherits beat local) D. A 0.9, B None

**COMP-048** · Obj 1.1 · Difficulty: Medium · Type: Single choice

Continuing COMP-047: after A's local 0.9, you change the **class** to `roughness = 0.5`. What is composed?

A. A 0.5, B 0.5 (class edits beat local) B. A 0.9, B 0.5 (A keeps its local opinion; B follows the class) C. A 0.9, B 0.2 (the class is now ignored) D. Both None until you re-`AddInherit`

**COMP-049** · Obj 1.7 · Difficulty: Easy · Type: Single choice

You add variants `big` and `small` to variant set `size`. How do you list the variant names, and what is printed?

A. `variantSet.GetNames()` → `['big', 'small']` B. `variantSet.GetVariantNames()` → `['big', 'small']` (sorted); `GetNames()` lives on `Usd.VariantSets`, not on `Usd.VariantSet` C. `prim.GetVariantSets().GetVariantNames()` D. Variant names are only visible in USDA, not in Python

**COMP-050** · Obj 1.8 · Difficulty: Medium · Type: Single choice

`stage.OverridePrim("/NoSuch")` on an empty stage. Which description of `/NoSuch` is correct?

A. The prim exists as a handle, `IsDefined()` is `False`, specifier is `Sdf.SpecifierOver` B. USD raises because `over` cannot create a prim C. `IsDefined()` is `True` because any authored spec defines the prim D. The prim is automatically converted to `def Xform`

**COMP-051** · Obj 1.5 · Difficulty: Easy · Type: Single choice

A root lists `subLayers = [@l1@, @l2@, @l3@]` where those layers author `radius 1, 2, and 3`. What is composed?

A. 3 (last listed wins, like CSS) B. 1 (first listed is strongest) C. 2 (the middle layer is the "active" edit target) D. None (three opinions cancel)

**COMP-052** · Obj 1.3 · Difficulty: Medium · Type: Single choice

`Usd.Stage.OpenMasked("m.usda", mask)` with the mask containing only `/W/A`. The file also defines `/W/B`. What is true?

A. `/W/A` exists, `/W/B` does not B. Both exist; the mask only skips payload loading C. Neither exists until you `Load` D. `OpenMasked` is not in USD 26.08; you must pass `mask=` to `Open`

**COMP-053** · Obj 1.1 · Difficulty: Hard · Type: Single choice

A published character **inherits** `/_Look` inside the asset (`roughness = 0.2`). A shot **references** that character at `/Hero`, then defines a shot-level class `"_Look"` with `roughness = 0.9`. What is `/Hero.roughness`?

A. 0.2 — shot classes cannot affect referenced inherits B. 0.9 — implied inherits let the shot's `/_Look` contribute to the referenced prim C. None — two `_Look` classes conflict D. 0.2 unless you also `AddInherit("/_Look")` on `/Hero`

**COMP-054** · Obj 1.5 · Difficulty: Medium · Type: Multiple select Select two.

Root authors `radius = 1`. Session authors `radius = 9`. A weaker sublayer authors `radius = 5`. Which are true?



A. Composed `radius` is 9 (session is the strongest local sheet). B. The root layer spec still stores 1; session opinions are not written into the root. C. Saving the root layer publishes the session value 9 into `shot.usda`. D. Sublayer 5 beats the session because sublayers are “more permanent.”

**COMP-055** · Obj 1.10 · Difficulty: Medium · Type: Single choice

On a **non-instance** prim, `attr.Set(5)` then `attr.Block()`. What does `attr.Get()` return?

A. 5 (block only hides the USDA line) B. None (a block is a strong opinion that hides weaker values, including the previous default) C. Schema fallback (for example Sphere radius 1) D. 0

**COMP-056** · Obj 1.6 · Difficulty: Medium · Type: Single choice

Why is “composition is not merging” the right mental model for double `radius`?

A. List-op fields and scalar fields both concatenate every opinion B. For a scalar, the **strongest** opinion replaces weaker ones; USD does not average or blend 1 and 9 C. Weaker opinions are deleted from disk when a stronger one appears D. Composition only runs at flatten time

**COMP-057** · Obj 1.6 · Difficulty: Easy · Type: Single choice

Where do **value clips** sit in LIVERPS?

A. They are the “C” between variants and relocates B. They are not a LIVERPS letter; clip values resolve *inside* the strength of the arc/layer that introduced them C. They always beat local opinions D. They replaced relocates in USD 26.08

**COMP-058** · Obj 1.9 · Difficulty: Medium · Type: Multiple select Select two.

A studio must deliver an internal show asset to an external vendor. Which pair belongs in the delivery checklist?

A. Flatten (or package a USDZ) so the vendor is not chasing unshipped layer paths; localize remaining asset paths B. Include the artist’s **session layer** so they see the same scratch comments C. Strip or provide fallbacks for proprietary schemas/plugins the vendor does not have D. Leave `subLayers` pointing at `/mnt/show/...` absolute paths so the vendor’s resolver “just works”

**COMP-059** · Obj 1.5 · Difficulty: Easy · Type: Single choice

```
stage.SetEditTarget(stage.GetRootLayer())
with Usd.EditContext(stage, stage.GetSessionLay
```

```
er()):
 prim.SetMetadata("comment", "try bigger")
```

After the `with` block, what is the edit target?

A. Still the session layer B. Restored to the root layer C. Cleared (None), and the next `Set` raises D. The referenced asset’s root, because comments always go there

**COMP-060** · Obj 1.8 · Difficulty: Medium · Type: Single choice

You open a layer that contains only over `"Hero" { int hp = 10 }` as a stage. What happens, and what changes if you replace over with a typeless `def`?

A. over: /Hero is not defined and `Traverse()` is empty. Typeless `def`: /Hero is defined (`IsDefined()` True) and appears in `Traverse()`, still with no schema type. B. Both specifiers define the prim the same way; only USDA spelling differs. C. over defines the prim; `def` without a type is a syntax error. D. `Traverse()` includes overs and skips defs.

**COMP-061** · Obj 1.4 · Difficulty: Medium · Type: Single choice

Two crowd characters must play the **same** walk clip, one starting 30 frames later. What is the composition tool?

A. Duplicate the clip file and rewrite every sample time by +30 B. Two references (or sublayers) to the same animation layer with different `Sdf.LayerOffset / USDA (offset = ...)` C. A variant set time with one variant per frame D. `instanceable = true`, which delays time by instance index

**COMP-062** · Obj 1.7 · Difficulty: Medium · Type: Single choice

A component already has variant set `look = {clean, dirty}`. A shot TD also needs a one-off “muddy hero” that will never be reused. Where should that look live?

A. Add a third variant on the published component from the shot file (variants are always global) B. Shot-level opinions (local overs / a shot-only layer) on that one instance; do not grow the published variant set for a one-off C. A specialize of `clean`, because specializes beat local shot work D. A `PointInstancer` prototype named `muddy`

**COMP-063** · Obj 1.8 · Difficulty: Medium · Type: Single choice

`GetPrimStack()` on a prim that is defined in the root and referenced from `r1.usda` prints display names `['shot.usda', 'r1.usda']`. What does that order mean?

A. Weakest first; `r1` wins B. Strongest first; the root/shot spec is stronger than the referenced spec C. Alphabetical D. The stack only lists payloads

**COMP-064** · Obj 1.11 · Difficulty: Medium · Type: Multiple select Select two.

A robot asset is being split for parallel work. Which file split matches common OpenUSD asset structure?

A. `Robot.usda` (interface + defaultPrim + payload/reference to contents), `Robot_payload.usda` / geometry layer, `Robot_mtl.usda` B. One USDA per polygon, referenced individually from the shot C. Shot `subLayers` for layout/anim/lighting that all overlay `/World`, leaving the robot as a referenced component D. Bake every department into the robot's variant set `dept`

**COMP-065** · Obj 1.9 · Difficulty: Medium · Type: Single choice

You need a **package** the vendor can copy as one file, still containing a USDA root plus a texture. Flattening to one layer would drop the texture file. What do you do?

A. `Usd.ZipFile` / USDZ (STORE) that includes `root.usda` and `tex.png`, after making asset paths package-relative B. Email `tex.png` separately and keep absolute `/mnt` paths in the USDA C. `FlattenLayerStack` only — it always embeds PNG bytes D. Rename the USDA to `.usdc` so textures inline automatically

**COMP-066** · Obj 1.10 · Difficulty: Medium · Type: Single choice

Which override is legal **without** de-instanting a native instance?

A. `OverridePrim("/Room/A/Seat")` and set `size` B. An opinion on the **instance root** `/Room/A` (purpose, primvar, variant selection, some metadata) C. Editing `/__Prototype_1/Seat` from the assembly as the supported per-instance API D. `AddReference` on the proxy child `/Room/A/Seat`

**COMP-067** · Obj 1.2 · Difficulty: Medium · Type: Single choice

A stadium of 80,000 identical seats that are never edited individually, plus a broadcast booth that is a unique hero set piece. Best pairing?

A. `PointInstancer` (or native instances) for seats; ordinary reference for the booth B. Native instance every seat **and** the booth so they share one prototype C. 80,000 full `def` copies of the seat asset D. One payload for the entire stadium so seats cannot draw until `LoadAll`

**COMP-068** · Obj 1.11 · Difficulty: Easy · Type: Single choice

Why put a payload on the **contents** of a component rather than on the assembly root of a whole city block?

A. Payloads on the assembly root are illegal B. Consumers can already see the component's interface/proxy and choose to load heavy geometry; a city-sized payload is a coarse all-or-nothing hitch C. Payloads only work one level below `defaultPrim` D. Assemblies cannot contain payloads if they also use sublayers

**COMP-069** · Obj 1.1 · Difficulty: Medium · Type: Single choice

A weaker layer prepends two references. A stronger layer authors an **explicit** references = `[@r2.usda@]` (no prepend/append/delete). What happens?

A. USD concatenates explicit with prepend B. The explicit list **replaces** the weaker list-edits; only `r2` remains C. The explicit list is ignored because prepend is sticky D. Composition error `Pcp.ErrorType_InvalidAssetPath`

**COMP-070** · Obj 1.4 · Difficulty: Easy · Type: Single choice

Layer offset scale is `0.5`, offset is `0`. A sample at layer time 10 appears at which stage time?

A. `20 (layerTime / scale)` B. `5 (offset + scale * layerTime = 5)` C. `10` (scale never applies to samples) D. `0.5`

**COMP-071** · Obj 1.3 · Difficulty: Medium · Type: Multiple select Select two.

Which statements about **internal** vs **external** references are true?

A. `AddInternalReference("/Asset/Geo")` reuses a prim already on this stage (or in this layer stack) at a new path. B. `AddReference("chair.usda")` introduces another layer; it needs `defaultPrim` or an explicit prim path. C. Internal references are always stronger than external ones (they skip LIVERPS). D. Internal references cannot carry attributes, only transforms.

**COMP-072** · Obj 1.6 · Difficulty: Medium · Type: Single choice

`custom rel looks` is a list-op field. A weaker layer prepends `/MatA`; a stronger layer prepends `/MatB`. What is the composed target list (strongest-first typical)?

A. Only `/MatA` (relationships do not list-edit) B. `/MatB` then `/MatA` (list-ops **compose**; they are the exception to "scalar replace") C. `/MatA` then `/MatB` because weaker is "older" D. Empty; two prepends cancel

**COMP-073** · Obj 1.1 · Difficulty: Medium · Type: USDA-reading

```
#usda 1.0
class "_I"
{
 double radius = 3
}

def Sphere "Ball" (
 prepend inherits = </_I>
)
{
 double radius = 1
}
```

What is composed /Ball.radius?

A. 3 (inherits beat local) B. 1 (local beats inherits) C. 2 (average) D. None (local and inherit cancel)

**COMP-074** · Obj 1.3 · Difficulty: Hard · Type: Single choice

You need /World/A present and **loaded**, but /World/B must not appear in traversal at all. LoadNone is not enough because B is a **reference**. What do you use?

A. OpenMasked with a mask of /World/A B. Unload("/World/B") — Unload works on references C. MuteLayer on the root D. SetInstanceable(True) on B

**COMP-075** · Obj 1.5 · Difficulty: Medium · Type: Single choice

Lighting arrives last and must override animation's intensity on /World/Key. How should subLayers be ordered (first listed = strongest)?

A. [anim, lighting, layout] so lighting is in the middle B. [lighting, anim, layout] (or lighting above whoever it must beat) C. [layout, anim, lighting] so lighting is last and therefore strongest D. Order does not matter; lighting always beats anim by department name

**COMP-076** · Obj 1.10 · Difficulty: Medium · Type: Single choice

To hide a **root-level** property opinion on one instanceable chair without de-instancing, which is a supported pattern discussed for assemblies?

A. Nested Block() on /Chair\_0/Seat.size while instanceable B. Author a block or weaker-arc workaround at the **instance root**, or de-instance that chair if the property lives on a descendant proxy C. GetMaster().RemoveProperty("size") D. Delete the prototype prim from /\_\_Prototype\_1

**COMP-077** · Obj 1.8 · Difficulty: Easy · Type: Single choice

Where are relocates authored?

A. As a rel relocates relationship on each moved prim B. As **layer metadata** relocates = { </old>: </new> } (Python: layer.relocates list of path pairs) C. As a variant set named relocates D. Only inside UsdGeom.PointInstancer

**COMP-078** · Obj 1.3 · Difficulty: Easy · Type: Single choice

AddReference("explicit.usda", "/Only") where /Only is a Cube with size = 1 and the file has **no** defaultPrim. Result?

A. UnresolvedPrimPath and empty type B. Type Cube, size 1 — an explicit path does not need defaultPrim C. InvalidAssetPath D. AddReference returns False

**COMP-079** · Obj 1.8 · Difficulty: Medium · Type: Multiple select Select two.

A TD's new radius does not show in usdview. Which are common composition causes (Obj 1.8)?

A. The opinion is on a muted layer, a weaker arc (specialize vs reference), or an instance proxy path B. The opinion is a local default that is losing to a **weaker** layer's time samples (weaker samples always win) C. The edit target was a different layer than the one they inspected; or a variant selection is off; or LIVERPS put an inherit above their variant D. metersPerUnit of 0.01 automatically discards radius

**COMP-080** · Obj 1.1 · Difficulty: Medium · Type: Single choice

A prim has a local roughness = 0.5 over an inherit of 0.9 and a specialize of 0.2. After attr.Clear() of the local opinion, what wins?

A. Specialize 0.2 (Clear jumps to the weakest arc) B. Inherit 0.9 (I beats S; local is gone) C. Schema fallback only D. 0.5 remains because Clear cannot remove local specs

**COMP-081** · Obj 1.3 · Difficulty: Easy · Type: Single choice

Usd.Stage.Open("city.usda") with no load-set argument. The file payloads /W/A. Is /W/A/Hi present?

A. No — the default is LoadNone B. Yes — the default initial load set is Usd.Stage.LoadAll C. Only if A is also referenced D. Only after FindLoadable()

**COMP-082** · Obj 1.3 · Difficulty: Medium · Type: Single choice

AddReference("no\_such\_file.usda") (the file does not exist). Which composition error is reported?

A. `Pcp.ErrorType_UnresolvedPrimPath` B. `Pcp.ErrorType_InvalidAssetPath` C. No error; `AddReference` returns `False` D. `Tf.ErrorException` and the stage fails to open

**COMP-083** · Obj 1.8 · Difficulty: Medium · Type: Single choice

Root authors `r = 1`; session authors `r = 9`. What does `attr.GetPropertyStack()` list first? A. Root, then session (weakest first) B. Session, then root (strongest first) — display names like `...-session.usda`, then the root C. Only the referenced asset D. Property stacks exist only after `Flatten()`

**COMP-084** · Obj 1.4 · Difficulty: Medium · Type: Single choice

A **sublayer** is included as `@anim.usda@ (offset = 10)`. Samples in that file sit at 0 and 10. Composed sample times?

A. `[0.0, 10.0]` — offsets apply only to references, not sublayers B. `[10.0, 20.0]` — the same `stageTime = offset + scale * layerTime` formula C. `[10.0, 30.0]` D. Samples are deleted; only defaults remain

**COMP-085** · Obj 1.6 · Difficulty: Easy · Type: Multiple select Select two.

Which fields **list-edit** (compose by combining edits) rather than scalar-replace?

A. `double radius` B. `references`, `payloads`, `inherits`, `specializes`, `variantSets` C. `rel looks` (relationship targets) D. Layer metadata `doc` concatenates every contributing layer's documentation string

**COMP-086** · Obj 1.5 · Difficulty: Medium · Type: Single choice

Layout owns `layout.usda`, animation owns `anim.usda`. A layout TD accidentally `SetEditTarget` to `anim.usda` and authors `/World/Chair.xformOp:translate`. What went wrong (Obj 1.5 / 1.8)?

A. Nothing — any local sublayer is an equally valid place for layout B. The opinion is in the wrong department file; strength may still make it visible, but the workstream split is broken and anim's next save can collide C. USD silently moves the opinion back to `layout.usda` on Save D. Edit targets can only be the root layer, so the Set was a no-op

**COMP-087** · Obj 1.9 · Difficulty: Medium · Type: Single choice

`UsdUtils.FlattenLayerStack` vs `stage.Flatten()` for a vendor who **must keep** the ability to retarget `chair.usda` after delivery?

A. Use `Flatten()` so references become baked Cubes they can still retarget B. Use `FlattenLayerStack` — sublayers collapse, **references remain** for the vendor to re-point C. Neither keeps references D. Only `root.Export()` keeps references and also bakes sublayers

**COMP-088** · Obj 1.7 · Difficulty: Medium · Type: Single choice

A character has two variant sets, `look` and `lod`, that should combine (`dirty + proxy`). How does OpenUSD model that?

A. One variant set with four packed names (`dirty_proxy, ...`) is required; sets cannot combine B. Two variant sets on the same prim; selections compose independently (`look=dirty, lod=proxy`) C. Nested references, because variant sets cannot coexist D. A `PointInstancer` prototype per combination

**COMP-089** · Obj 1.2 · Difficulty: Hard · Type: Single choice

Native instances of a chair share `/__Prototype_1`. A TD needs a **displayColor** change on every instance without de-instancing. Best composition-friendly option?

A. Edit `/Room/A/Seat.displayColor` on one proxy (illegal) and hope it broadcasts B. Author an inherited class or a component-level primvar/material binding that the instances pick up, or set a primvar on each **instance root** if the shader reads it C. `GetMaster()` and change the mesh once D. Convert all chairs to unique `def` copies

**COMP-090** · Obj 1.11 · Difficulty: Medium · Type: Multiple select Select two.

You are splitting a monolithic `Kitchen.usda` that inlined every chair mesh. Which pair matches Obj 1.11?

A. Publish `Chair.usda` as a component; the kitchen assembly **references** it at `Chair_0`, `Chair_1`, ... B. Keep all meshes in `Kitchen.usda` but add a variant per artist name C. Mark instanceable on the assembly's chair prims if the chairs stay identical D. Move each chair vertex into a separate payload file named after the vertex index

**COMP-091** · Obj 1.8 · Difficulty: Hard · Type: USDA-reading

```
#usda 1.0
class "_I"
{
 double radius = 3
}

def Sphere "Ball" (
 prepend inherits = </_I>
```

```

prepend specializes = </_S>
)
{
}

def "_S"
{
 double radius = 7
}

```

Composed /Ball.radius?

A. 7 (specializes beat inherits) B. 3 (inherits beat specializes) C. 1 (Sphere fallback) D. 5 (average of 3 and 7)

**COMP-092** · Obj 1.1 · Difficulty: Medium · Type: Single choice

You need a **studio-wide fallback** roughness that every asset can beat with a local opinion, but that should **lose** to a published reference. Inherit or specialize?

A. Inherit — it beats references, so assets could not override with a reference B. Specialize — weakest arc; local and references still win, the class is only the fallback C. Either; they have identical strength D. Payload — payloads beat both inherits and specializes

## Answers

**COMP-001 — Answer: A.** LIVERPS is Local, Inherits, VariantSets, rElocates, References, Payloads, Specializes. B drops relocates into last place. C swaps Inherits and VariantSets. D puts Local second. Review: §21.1.

**COMP-002 — Answer: C.** E is rElocates (the glossary spelling LIVERPS). Older LIVRPS material omitted relocates; clips and edit targets are not LIVERPS letters. Review: §21.1.

**COMP-003 — Answer: A, C.** Local beats referenced opinions (A). Inherits beat variants (C). Specializes are *weaker* than references (B is backwards). Value clips are not a LIVERPS letter (D). Review: §21.1–21.2.

**COMP-004 — Answer: B.** A local opinion is L, stronger than the reference (R). Specializes (A) are weaker than the reference, so 7 would lose to 1. Muting the root (C) removes *your* local stack. Instancing (D) does not change radius strength. Review: §14.2, §19.3.

**COMP-005 — Answer: B.** Verified: inherit 3 beats referenced 1. USD does not average (D) or cancel (C). The Sphere type comes along with the reference; it does not make R stronger than I. Review: §19.2, §21.6.

**COMP-006 — Answer: B.** Verified with an internal reference plus specialize: composed radius is 1. Specializes are the weakest arc; listing order in USDA does not invert R vs S. Review: §19.3–19.4.

**COMP-007 — Answer: A, C.** Verified: prepended r1 (radius = 2) beats appended r2 (radius = 8). Prepend is stronger than append (C). USD allows both list-ops on one field (D is false). Review: §14.4.

**COMP-008 — Answer: B.** Verified on USD 26.08: composed radius is 3.0. I beats V even when the variant is selected. Sphere fallback is not used when an opinion exists. Review: §21.2, §21.6.

**COMP-009 — Answer: A.** Sublayers extend the *same* layer stack and namespace. References (and payloads) introduce another prim index node at the referencing path. Sublayers are not weaker than references as a class (they are Local). Format is independent. Sublayers are not lazy; payloads are. Review: §15.1, §16.6.

**COMP-010 — Answer: B, C.** Payloads are the unloadable, weaker sibling of references. They do not beat local opinions (A is the opposite of what you want for shot overrides). They still use defaultPrim when the target path is omitted (D). Review: §17.1, §17.5.

**COMP-011 — Answer: B.** Verified: LoadNone leaves /W/C/Hi loaded (reference) and /W/A/Hi unloaded (payload). LoadNone is not a population mask and does not skip textures as its definition. Payloads are weaker, not stronger, than references. Review: §17.2, lab 16.

**COMP-012 — Answer: B.** Verified: AddReference returns True, composed type is '', error is Pcp.ErrorType\_UnresolvedPrimPath (missing defaultPrim, no explicit path). InvalidAssetPath is the missing-file error. USD does not silently pick the first root prim. Review: §16.2, §16.5.

**COMP-013 — Answer: A, B.** Department layers that all edit /World are sublayers (same namespace). A published chair under a new path is a reference or payload. Inherits broadcast *class* opinions; they are not a department file stack. Specializes do not “copy an asset to a new path”; references do. Review: §15.5, §16.1.

**COMP-014 — Answer: B.** stageTime = offset + scale \* layerTime. The same formula applies to reference offsets and sublayer offsets. Review: §16.4, §21.4.

**COMP-015 — Answer: B.** Verified: composed sample times are [10.0, 20.0]. Layer times 0 and 10 plus offset 10. Review: §16.4, lab 16.

**COMP-016 — Answer: B.** Verified: `Get()` and `Get(Usd.TimeCode.Default())` return `(100, 0, 0)`. Layer offsets remap *time samples*, not the default. `Get()` is default time; it is `None` only when **no** default exists (samples-only). Review: §10.5, §21.3.

**COMP-017 — Answer: B.** Verified in lab 19: `GetEditTargetForLocalLayer` maps stage time 20 through offset 10 → stored at 10. A bare `EditTarget` records 20. Review: §15.2–15.3.

**COMP-018 — Answer: A.** Collaborative shots are a strong-to-weak `subLayers` list with one owner per file. Specializes, instancing, and flattening are the wrong tools for live multi-user authoring. Review: §15.5, §22.2.

**COMP-019 — Answer: B.** Verified: `SetEditTarget` on an anonymous (or referenced-only) layer raises `Tf.ErrorException` because the layer is not in the local `LayerStack`. Review: §15.3, lab 19.

**COMP-020 — Answer: A.** Animation belongs in the anim *sublayer* via a local-stack edit target (preferably `GetEditTargetForLocalLayer` so offsets map). The session layer is not a publish target; flattening destroys the collaborative stack. Review: §15.3, §15.5.

**COMP-021 — Answer: A, C.** First listed sublayer wins. Session is for scratch, not for the show. Sharing one root file (B) invites overwrite races. Putting layout and anim on different prim paths (D) splits the scene instead of overlaying / *World*. Review: §15.1, §15.5, §22.2.

**COMP-022 — Answer: A.** Variants are named, discrete options inside one asset. Instancing, multi-user layering, and beating local opinions are other arcs' jobs. Review: §18.5.

**COMP-023 — Answer: A, C.** Continuous parameters belong on attributes, not variant combinatorics. Shot animation belongs in shot layers (references to clips / time samples), not in the character's variant set. Looks and LODs (B, D) are classic variant uses. Review: §18.5.

**COMP-024 — Answer: B.** Verified: with selection `''`, `GetVariantEditContext()` authors `height = 3` on the prim itself; the big variant block is empty. Always `SetVariantSelection` before the context when you mean to fill a variant. Review: §18.2, lab 17.

**COMP-025 — Answer: B.** Variants switch options packaged *inside* one asset. A separately published hero is a new reference/payload at the assembly. Variants can contain meshes (A is false). Strength is LIVERPS, not "references always beat variants." Review: §18.5, §16.1.

**COMP-026 — Answer: A, C.** Muting drops the layer from the stack (verified: mute strong → composed 7). Wrong edit target is the other common "I wrote it but I don't see it" cause. Instancing does not resurrect muted opinions; mute does not delete files. Review: §15.4, §22.6.

**COMP-027 — Answer: B.** Verified: `OverridePrim` on an instance proxy raises `Tf.ErrorException`. USD does not silently retarget the prototype. Review: §24.3, §22.4, lab 23.

**COMP-028 — Answer: B.** Verified: composed height is 1.0 even with `size = big`, because local beats variant. This is a standard "variant did nothing" debug. Review: §18.6, §21.2.

**COMP-029 — Answer: B.** Relocates move the namespace; opinions left at the old source path do not follow. Author at the destination (or in the asset before relocate). Relocates apply to referenced structure, not payloads only. Review: §20.3–20.4.

**COMP-030 — Answer: A.** `Stage.Flatten()` bakes arcs into one layer (verified: no `references` field; `/Refd` becomes a Cube). `FlattenLayerStack` keeps references. Export of a root layer writes that layer's specs, it does not flatten. USDZ can still describe arcs inside the package. Review: §22.3, §34.2.

**COMP-031 — Answer: A, B.** Verified on the bolt/shot setup: Flatten bakes `/Refd` to Cube and includes `/Bolt`; `FlattenLayerStack` keeps prepend references on `/Refd`, inlines `/Bolt`, `subLayers` empty. They are not identical (C). Session inclusion is a separate Flatten flag/behavior, not unique to `FlattenLayerStack` (D). Review: §34.2, lab 28.

**COMP-032 — Answer: A.** Instancing is opted in at the **use** site (assembly instance roots) so the published component remains a normal asset. Nesting `instanceable` on every gprim is the wrong grain. Review: §24.1, §22.4.

**COMP-033 — Answer: B.** Nested proxy authoring is forbidden (see COMP-027). De-instance that one root, then override `Seat`; other instanceable chairs still share `/__Prototype_N` (verified: sibling size stays 1). `GetMaster()` is gone. Muting the assembly is unrelated. Review: §24.5–24.6, §22.4.

**COMP-034 — Answer: A.** Verified: `GetPrototype()` path is `/__Prototype_1`; `hasattr(prim, "GetMaster")` is `False` on `usd-core 26.8`. `PointInstancer` is a different instancing style. Review: §24.2, lab 23.

**COMP-035 — Answer: A.** Split by workstream into layers connected with sublayers/references/payloads; keep a thin interface. Copy/flatten, PointInstancer, and class-only encoding do not give two teams safe files. Review: §22.1, §23.2.

**COMP-036 — Answer: A, C.** Heavy contents behind payloads; shot departments as ordered sublayers on a shared /World. Do not hide show animation inside the character asset (B). Relocates are for namespace repair, not a substitute for a consistent model hierarchy (D). Review: §22.1–22.2, §17.5.

**COMP-037 — Answer: B.** PointInstancer is the scale for huge, similar, mostly uniform copies with per-instance transforms and cheap hides (invisibleIds). Native instances still cost one prim each; full copies cost full composition. Review: §25.1, §26.1.

**COMP-038 — Answer: B.** Native instances for the crowd of identical chairs; de-instance (or don't instance) the one hero that needs nested modeling. PointInstancer is the wrong grain for hero furniture. Editing the prototype changes every instance. Payloads do not turn instancing off. Review: §26.1, §24.5.

**COMP-039 — Answer: A, B.** Unique heroes → ordinary refs; identical furniture at hundreds → native instances. Millions of pebbles belong on a PointInstancer, not one native instance prim each (C, D are wrong). Review: §26.1.

**COMP-040 — Answer: B.** Population masks use `Usd.Stage.OpenMasked`, not `Open(path, mask=...)`. `LoadNone` still creates payload prims and still loads references (COMP-011). Muting a whole layer is coarser than masking one subtree and may drop unrelated opinions. Review: §17.4, lab 16.

**COMP-041 — Answer: B.** Verified: delete `r1` from `prepend [r1, r2]` leaves `r2`, composed radius 8. Delete removes listed items; it does not empty the list (C) or average (D). Review: §14.4.

**COMP-042 — Answer: B.** Verified: stronger default 5 wins at `Get()`, `Get(0)`, and `Get(10)`; `GetTimeSamples()` is `[]`. Weaker samples do not leak through a stronger default. Review: §21.3.

**COMP-043 — Answer: C.** Verified: `Sdf.LayerOffset(10, 2)` maps layer times 0 and 10 to stage times 10 and 30. `stageTime = offset + scale * layerTime`. Review: §16.4, §21.4.

**COMP-044 — Answer: B.** Verified: `AddInternalReference("/Asset/Geo")` returns `True` and `/Inst.sz` is 4. Internal references are same-stage/same-stack reuse, not disk-only. Review: §16.3.

**COMP-045 — Answer: B.** Verified: `FindLoadable()` returns `['/W/A', '/W/B']` — payload sites only, not the reference at `/W/C`. Review: §17.2, lab 16.

**COMP-046 — Answer: A, B.** Verified: `LoadNone` → `/W/A/Hi` missing, `IsLoaded()` `False`; `Load` brings `Hi` in; `Unload` removes it. `LoadNone` is the initial set, not a lock (C). `FindLoadable` is payloads only (D). Review: §17.2.

**COMP-047 — Answer: B.** Verified: local on A is 0.9; B stays at the class 0.2. Inherits broadcast until a stronger local opinion appears on that prim. Review: §19.2, lab 18.

**COMP-048 — Answer: B.** Verified: after the class moves to 0.5, A stays 0.9 (local) and B becomes 0.5 (still inheriting). Review: §19.2.

**COMP-049 — Answer: B.** Verified: `GetVariantNames()` → `['big', 'small']`; `GetNames()` on the `VariantSet` raises `AttributeError`; `VariantSets.GetNames()` lists set names such as `['size']`. Review: §18.1, lab 17.

**COMP-050 — Answer: A.** Verified: `OverridePrim` creates a handle, `IsDefined()` `False`, specifier `Sdf.SpecifierOver`. `Traverse()` skips it. A typeless `def` would define it. Review: §20.1.

**COMP-051 — Answer: B.** Verified: first listed sublayer wins (1). Last-listed-wins is a CSS myth. Review: §15.1.

**COMP-052 — Answer: A.** Verified: `OpenMasked` with `/W/A` keeps A and drops B. `Open(path, mask=)` is not the API. `LoadNone` is a different switch. Review: §17.4, lab 16.

**COMP-053 — Answer: B.** Verified: shot-level class `"_Look"` with 0.9 changes the referenced `/Hero.roughness` via implied inherits. You do not have to re-`AddInherit` on `/Hero`. Review: §19.5.

**COMP-054 — Answer: A, B.** Verified: composed radius is 9; the root spec still stores 1. Saving the root does not publish the session layer. A weaker sublayer cannot beat the session. Review: §15.3, lab 04.

**COMP-055 — Answer: B.** Verified: after `Block()`, `Get()` is `None` even on `UsdGeom.Sphere` radius (not the schema fallback 1). A block is a strong empty opinion. Review: §21.3, §22.4.

**COMP-056 — Answer: B.** Scalars resolve to one winner. List-ops are the fields that combine. Weaker specs stay on disk. Composition runs when the stage is composed, not only at flatten. Review: §14.6.

**COMP-057 — Answer: B.** Clips are not a LIVERPS letter. They resolve inside the introducing node's strength. Review: §21.1, §21.5.

**COMP-058 — Answer: A, C.** Flatten or USDZ plus localized paths; ship or avoid proprietary schemas. Do not ship the session layer. Do not leave `/mnt/show` paths. Review: §22.3, §34.2–34.3.

**COMP-059 — Answer: B.** Verified in lab 19: `EditContext` restores the previous target (root). Review: §15.3.

**COMP-060 — Answer: A.** Verified: overlay-only `over` → not defined, empty `Traverse()`; typeless `def` → defined and listed. Review: §20.1.

**COMP-061 — Answer: B.** Same animation, different `LayerOffset` on each reference (Obj 1.4). Duplicating files, per-frame variants, and instancing do not offset time. Review: §16.4.

**COMP-062 — Answer: B.** Published variants are for reused, named options. A one-off belongs in the shot. Specializes do not beat local. Review: §18.5, §22.2.

**COMP-063 — Answer: B.** Prim stack is strongest first. Root then referenced asset. Review: §14.5, §22.5.

**COMP-064 — Answer: A, C.** Interface/payload/geo/mtl split plus shot department sublayers. Per-polygon files and stuffing departments into the asset variant set are the wrong grain. Review: §22.1, §23.2.

**COMP-065 — Answer: A.** USDZ STORE can hold `root.usda` plus `tex.png` (lab 28). Flattening does not embed PNG bytes. Review: §22.3, §7.4, lab 28.

**COMP-066 — Answer: B.** Verified: instance-root opinions (e.g. `purpose = guide`) are legal; nested `OverridePrim` on a proxy raises. Do not treat prototype internals as the per-instance API. Review: §24.4–24.5.

**COMP-067 — Answer: A.** Huge identical seats → `PointInstancer` or native instances; unique booth → ordinary reference. Sharing a prototype with the booth would block nested booth edits. Review: §26.1.

**COMP-068 — Answer: B.** Payload the heavy component contents so the interface stays cheap; a city-root payload is too coarse. Payloads on assemblies are legal, just usually the wrong hitch. Review: §17.5, §22.1, §26.2.

**COMP-069 — Answer: B.** An explicit list replaces weaker `prepend/append/delete`. It does not concatenate. Review: §14.4.

**COMP-070 — Answer: B.** `0 + 0.5 * 10 = 5`. Review: §16.4.

**COMP-071 — Answer: A, B.** Internal = path on this stage; external = another layer + `defaultPrim` or explicit path. Internal refs still follow LIVERPS and can carry attributes (verified `sz = 4`). Review: §16.2–16.3.

**COMP-072 — Answer: B.** Verified: session `prepend /MatB` plus root `prepend /MatA` → targets `['/MatB', '/MatA']`. Relationships list-edit. Review: §14.4, §14.6.

**COMP-073 — Answer: B.** Verified: local 1 beats inherit 3. Review: §19.2, §21.2.

**COMP-074 — Answer: A.** References ignore `Unload/LoadNone`. A population mask is the tool that omits `/World/B` entirely. Muting the root drops everything. Review: §17.4.

**COMP-075 — Answer: B.** First listed is strongest, so lighting must appear before anim if it must win. Last-listed-wins is wrong. Review: §15.1, §15.5.

**COMP-076 — Answer: B.** Nested proxy `Block()` is the same illegal authoring as COMP-027. Instance-root opinions or de-instancing are the assembly patterns. `GetMaster` is gone; do not delete prototypes. Review: §22.4, §24.6.

**COMP-077 — Answer: B.** Relocates are layer metadata; Python `layer.relocates` is a list of path pairs. Review: §20.3.

**COMP-078 — Answer: B.** Verified: explicit `/Only` → type `Cube`, size 1, no `defaultPrim` required. Review: §16.2, lab 14.

**COMP-079 — Answer: A, C.** Muted/wrong-arc/proxy (A) and wrong target / variant / LIVERPS (C) are the checklist. Weaker samples do **not** beat a stronger default (COMP-042). `metersPerUnit` does not drop `radius`. Review: §22.6.

**COMP-080 — Answer: B.** Verified: local 0.5 then `Clear()` → inherit 0.9 (I beats S). Review: §19.3–19.4, lab 18.

**COMP-081 — Answer: B.** Verified: default `Open` loads payloads (`/W/A/Hi True`). `LoadNone` is opt-in. Review: §17.2.



**COMP-082 — Answer: B.** Verified: missing file → `Pcp.ErrorType_InvalidAssetPath`. Missing `defaultPrim` is `UnresolvedPrimPath` (COMP-012). `AddReference` still returns `True`. Review: §16.5.

**COMP-083 — Answer: B.** Verified: property stack lists session then root (strongest first). Review: §22.5, lab 21.

**COMP-084 — Answer: B.** Verified: sublayer offset 10 maps samples to `[10.0, 20.0]`. Same formula as reference offsets. Review: §15.2, §16.4.

**COMP-085 — Answer: B, C.** Composition-arc fields and relationships list-edit. `double radius` and most metadata (`doc`) scalar-replace; documentation strings do not concatenate. Review: §14.4, §14.6.

**COMP-086 — Answer: B.** Edit targets write into a specific layer; the opinion can still compose as `Local`, but it lives in the wrong workstream file. USD does not relocate it on Save. Review: §15.3, §15.5.

**COMP-087 — Answer: B.** Verified: `FlattenLayerStack` keeps prepend references; `Flatten` bakes them away. Export of the root writes only that layer's specs (`subLayers` still listed, not baked). Review: §34.2.

**COMP-088 — Answer: B.** Multiple variant sets on one prim combine by independent selections. Packing names into one set is optional, not required. Review: §18.1, §18.3.

**COMP-089 — Answer: B.** Instance-root `primvars` / inherited classes / material binds at a level the prototype reads. Nested proxy edits are illegal; `GetMaster` is gone; unique copies defeat instancing. Review: §24.4–24.5.

**COMP-090 — Answer: A, C.** Extract a component asset and reference it; instance identical copies. Artist-name variants and per-vertex payloads are the wrong split. Review: §22.1, §23.4, lab 22.

**COMP-091 — Answer: B.** Verified: inherit 3 beats specialize 7. Review: §19.3, §21.2.

**COMP-092 — Answer: B.** Specializes are the weakest fallback (R beats S). Inherits would beat the published reference. Review: §19.3–19.4.

---

*Composition domain complete: COMP-001–COMP-092 (target 92).*

## Question Bank — Data Exchange (DE)

**Original practice questions.** Not actual NVIDIA exam content. Mapped to NCP-OUSD study-guide objectives 4.1-4.8 (Data Exchange, 15%).

Verified against **USD 26.08** (usd-core 26.8). Domain target: 60 questions. This file currently holds **DE-001-DE-060** (domain complete).

Work the stems first. Answers and explanations are grouped at the end.

**DE-001** · Obj 4.3 · Difficulty: Easy · Type: Single choice

Which statement best captures the USDA vs USDC trade-off?

A. USDA is binary and faster to load; USDC is text for Git diffs B. USDA is UTF-8 text (readable, mergeable, larger); USDC is crate binary (faster I/O, smaller, not line-diff friendly) C. They are identical encodings with different extensions D. USDC cannot store time samples; USDA can

**DE-002** · Obj 4.3 · Difficulty: Medium · Type: Single choice

Usd.Stage.CreateNew("scene.usd") then Save(). What are the **magic bytes** and GetFileFormat().formatId?

A. Magic #usda 1.0, formatId usda B. Magic PXR-USDC, formatId usdc C. Magic PXR-USDC, formatId usd (id follows the .usd plugin; magic follows crate bytes) D. Magic PK, formatId usdz

**DE-003** · Obj 4.3 · Difficulty: Medium · Type: Multiple select Select two.

A pipeline copies a valid USDA file to text.usd. Which are true on USD 26.08?

A. The file still opens; prims and values are there B. formatId is usd (extension plugin), while magic starts with #usda C. formatId becomes usda because the bytes are text D. USD refuses to open USDA bytes under a .usd name

**DE-004** · Obj 4.3 · Difficulty: Easy · Type: Single choice

You must pick an encoding for a **layout department layer** that artists merge in Git, versus a **published cache** opened thousands of times on the farm. Best pair?

A. Git: USDC; farm: USDA B. Git: USDA; farm: USDC C. Both USDA — binary is never used in film D. Both .usdz — zip is always smallest

**DE-005** · Obj 4.3 · Difficulty: Medium · Type: Single choice

How should an ingest tool decide whether shot.usd is text or crate?

A. Trust the extension: .usd always means crate B. Read magic: #usda → text, PXR-USDC → crate, PK → zip C. Call Sdf.FileFormat.FindByExtension("usd") and assume crate D. formatId == "usdc" is necessary and sufficient for crate .usd files

**DE-006** · Obj 4.7 · Difficulty: Easy · Type: Single choice

Tf.MakeValidIdentifier("Crate Box") returns what?

A. Crate Box (spaces are legal prim names) B. Crate\_Box C. crateBox D. "" (invalid; the call raises)

**DE-007** · Obj 4.7 · Difficulty: Medium · Type: Single choice

Tf.MakeValidIdentifier("2ball") and Tf.IsValidIdentifier("2ball")?

A. "2ball", True B. "\_ball", False for the original string (identifiers cannot start with a digit) C. "ball2", True D. Both calls raise Tf.ErrorException

**DE-008** · Obj 4.7 · Difficulty: Easy · Type: Single choice

Usd.Stage.CreateInMemory() with no metrics authored. What do GetStageUpAxis and GetStageMetersPerUnit return?

A. Y and 1.0 B. Y and 0.01 C. Z and 1.0 D. None and None

**DE-009** · Obj 4.7 · Difficulty: Medium · Type: Single choice

A meters, Z-up chair is **referenced** into a centimeters, Y-up shot. The chair's Cube size is 1 in the asset. Composed size on the shot?

A. 100 — USD multiplies by the metersPerUnit ratio automatically B. 1.0 — composition does not convert units; you author scale (and often rotateX) on the referencing prim C. 0.01 D. None — mixed metrics are a composition error

**DE-010** · Obj 4.7 · Difficulty: Medium · Type: Single choice

Asset metersPerUnit = 1 (meters), shot metersPerUnit = 0.01 (centimeters). Correct scale **factor** to apply on the referencing prim so 1 m becomes 100 shot units?

A. shot / asset = 0.01 B. asset / shot = 100 C. Always 1 because Cube size is already 1 D. 2.54 (inches)

**DE-011** · Obj 4.7 · Difficulty: Medium · Type: Multiple select Select two.

A Z-up meters asset is referenced into a Y-up centimeters shot. Besides the scale factor, which authoring is the usual axis fix?

A. `AddRotateXOp().Set(-90)` on the referencing prim B. Change the **composed** stage's `upAxis` metadata — that rotates every referenced asset C. Read **asset** metrics by opening the asset layer, not by reading the shot's composed `GetStageUpAxis` D. Set `instanceable = true` so USD converts axes

**DE-012** · Obj 4.7 · Difficulty: Medium · Type: Single choice

An OBJ face line is `f 1 2 3`. What should `faceVertexIndices` store?

A. `[1, 2, 3]` (OBJ is already 0-based) B. `[0, 1, 2]` (OBJ is 1-based; USD Mesh indices are 0-based) C. `[3, 2, 1]` (USD requires clockwise winding) D. The converter should skip faces; only `v` lines matter

**DE-013** · Obj 4.7 · Difficulty: Medium · Type: Python-reading

A JSON record has a point `[100, 0, 0]` in **centimeters**. The converter scales into meters and authors `metersPerUnit = 1`. What is `points[1]` if that point is the second vertex?

A. `(100.0, 0.0, 0.0)` with `metersPerUnit = 0.01` B. `(1.0, 0.0, 0.0)` with `metersPerUnit = 1` C. `(0.01, 0.0, 0.0)` with `metersPerUnit = 100` D. Leave centimeters in the points and hope Hydra reads `units: "cm"` from JSON

**DE-014** · Obj 4.7 · Difficulty: Medium · Type: Single choice

After writing Mesh `points`, what must the exporter still do for a complete boundable?

A. Nothing; `extent` updates itself B. `ComputeExtentFromPlugins` (or equivalent) **then Set** on `extent` C. Set `kind = component` — that computes extent D. Export as USDA so `usdview` fills extent

**DE-015** · Obj 4.5 · Difficulty: Easy · Type: Single choice

A CAD `partId` has no `UsdGeom` attribute. How should a first exporter carry it (Obj 4.5)?

A. Drop it; USD forbids unknown fields B. A namespaced custom attribute such as `factory:partId` (`Sdf.ValueTypeNames.String`) C. Store it in the prim **name** only D. Put it in `faceVertexIndices`

**DE-016** · Obj 4.5 · Difficulty: Medium · Type: Multiple select Select two.

When is a **custom schema** (not only a loose attribute) the right import/export tool?

A. Many DCCs and tools must share a typed `Door` prim with known properties and fallbacks B. A single one-off vendor id that will never be queried as a type C. You need `HasAPI / Isa` checks and documentation in `schema.usda` D. You parsed `schema.usda` in memory — that already registers `Door` for `Isa`

**DE-017** · Obj 4.5 · Difficulty: Medium · Type: Single choice

On `usd-core 26.8` you `ImportFromString` a `schema.usda` that defines `class "Door"`. `FindConcretePrimDefinition("Door")`?

A. The `Door` definition — parse registers schemas B. None — parse  $\neq$  register; `usdGenSchema` / a plugin is required C. It raises `Tf.ErrorException` D. It returns `Cube`'s definition as a fallback

**DE-018** · Obj 4.6 · Difficulty: Easy · Type: Single choice

`hasattr(UsdUtils, "ComplianceChecker")` on USD 26.08?

A. True — that is still the public validator B. False — use `pxr.UsdValidation (ValidationRegistry)` C. True only if `usdchecker` is on `PATH` D. `ComplianceChecker` moved to `UsdGeom`

**DE-019** · Obj 4.6 · Difficulty: Medium · Type: Single choice

A stage is missing `upAxis`, `metersPerUnit`, and `defaultPrim`.

`usdGeomValidators:StageMetadataChecker` vs `usdValidation:StageMetadataChecker` report which names?

A. `Geom: MissingDefaultPrim`; core: missing meters and `upAxis` B. `Geom: MissingMetersPerUnitMetadata` and `MissingUpAxisMetadata`; core: `MissingDefaultPrim` C. Both checkers report all three D. Neither runs without the `usdchecker` CLI

**DE-020** · Obj 4.6 · Difficulty: Medium · Type: Multiple select Select two.

You are writing a **publish validator** for DCC exports (Obj 4.6). Which checks belong?

A. Stage metadata: `defaultPrim`, `upAxis`, `metersPerUnit` B. `UsdUtils.ComputeAllDependencies` unresolved list is empty (or listed as known) C. Every prim's `GetMaster()` is non-null D. Reject USDA files; only USDC can be valid

**DE-021** · Obj 4.6 · Difficulty: Easy · Type: Single choice

usd-core has no `usdchecker` binary. Can you still validate?

A. No — Obj 4.6 requires the CLI B. Yes — call `UsdValidation` validators from Python C. Only by flattening first D. Only inside `usdview`

**DE-022** · Obj 4.1 · Difficulty: Medium · Type: Single choice

USD Mesh faces may be quads. Core glTF mesh primitives expect what?

A. Quads are legal in core glTF 2.0 B. Triangles — the converter must **triangulate** n-gons C. Only points (no indices) D. USD `faceVertexCounts` copied unchanged

**DE-023** · Obj 4.1 · Difficulty: Medium · Type: Single choice

USD `primvars:st` (UV) → glTF `TEXCOORD_0`. Which extra transform is the usual fidelity rule?

A. `u_glTF = 1 - u_usd` (flip U) B. `v_glTF = 1 - v_usd` (glTF UV origin is top-left) C. Scale UVs by `metersPerUnit` D. Drop UVs; glTF has no texcoords

**DE-024** · Obj 4.1 · Difficulty: Medium · Type: Multiple select Select two.

Converting a composed USD shot to glTF 2.0 **without extensions**, which USD concepts are typically **lost** (must be documented in the mapping)?

A. Variant sets, payloads, and layer stacks as first-class composition B. Triangle mesh positions after triangulation C. Native USD composition arcs as live, editable arcs in the `.glTF` D. A single baked node tree with TRS

**DE-025** · Obj 4.1 · Difficulty: Easy · Type: Single choice

glTF 2.0 core assumes which units and up axis?

A. Centimeters, Z-up (like many DCCs) B. Meters, +Y up C. Inches, Y-up D. Whatever `metersPerUnit` is in the USD file — glTF reads that metadata

**DE-026** · Obj 4.2 · Difficulty: Medium · Type: Single choice

`UsdPreviewSurface` `metallic` / `roughness` map most directly onto which glTF field group?

A. `KHR_materials_unlit` only B. `pbrMetallicRoughness` (`metallicFactor` / `roughnessFactor` and the packed metal-rough texture) C. `MaterialX ND_standard_surface` with no remainder D. USDZ `compress_type`

**DE-027** · Obj 4.2 · Difficulty: Medium · Type: Single choice

`MaterialX` `nodedef` / node graph corresponds most closely to which `UsdShade` idea?

A. `UsdGeom.Mesh` extents B. A

`UsdShade.Shader` / node-graph network with connections, not a single `PreviewSurface` input block C. `Sdf.LayerOffset` D. `Kind.Registry`

**DE-028** · Obj 4.2 · Difficulty: Easy · Type: Multiple select Select two.

Why write a **conceptual data mapping document** before coding a translator (Obj 4.2)?

A. It records source concept → USD (or glTF/ `MaterialX`) target, transforms, and **known loss** B. NVIDIA requires the document as an exam submission file C. Teams can review mappings (units, color space, UV flip) before code freezes the wrong rule D. Mapping documents replace validators

**DE-029** · Obj 4.4 · Difficulty: Medium · Type: Single choice

A DCC round-trip (USD → DCC → USD) silently drops payloads and custom `factory:partId`. What is the first design fix?

A. Always `Flatten()` on import so the DCC only sees one layer B. Extraction → transformation → validation: preserve composition arcs and namespaced attributes; **diff**

`ComputeAllDependencies` and attribute values in CI C. Convert to OBJ in the middle so loss is obvious D. Store everything in `displayColor`

**DE-030** · Obj 4.4 · Difficulty: Medium · Type: Multiple select Select two.

Healthy DCC **import** of a USD assembly should usually:

A. Keep references/payloads as instances or deferred loads when the DCC can B. Flatten on import “to make Maya simpler,” discarding the assembly structure C. Round-trip custom attributes and applied API schemas the studio cares about D. Rewrite every prim name through a new random identifier

**DE-031** · Obj 4.8 · Difficulty: Medium · Type: Single choice

You are extending a DCC **importer** (Obj 4.8). A referenced chair root is type `Xform`. The importer currently calls `UsdGeom.Xform.Define` on the same path before adding the reference. Why is that a bug?

A. `Xform.Define` is illegal in importers B. Defining a typed `Xform` on the path can **override/replace** the incoming type; use a **typeless** `DefinePrim` then `AddReference` so the asset’s type wins C. You must `Sphere.Define` first D. Importers may only use `Sdf.Layer.Open`

**DE-032** · Obj 4.8 · Difficulty: Medium · Type: Single choice

A DCC importer must bind a `UsdPreviewSurface` material that the exporter wrote with `Bind` but **without** `MaterialBindingAPI.Apply`. On USD 26.08, `HasAPI(MaterialBindingAPI)` is `False` even though the relationship exists. What should the importer do?

A. Ignore the binding; `HasAPI` `False` means there is no material B. Read `material:binding` (or `Apply` then `Bind` on export); do not treat `HasAPI` as the only signal C. Switch to `OpenGL` bind points D. Call `GetMaster()` on the mesh

**DE-033** · Obj 4.7 · Difficulty: Medium · Type: Single choice

Tessellated CAD should set `Mesh subdivisionScheme` to what so Hydra does not Catmull-Clark the tessellation?

A. Leave the fallback (`catmullClark`) B. `"none"` C. `"loop"` D. `"bilinear"` is required for all CAD

**DE-034** · Obj 4.3 · Difficulty: Easy · Type: Single choice

USDZ packaging on this book's verified path uses which zip compression for files?

A. `DEFLATE` (`compress_type` 8) B. `STORE` (`compress_type` 0) C. `LZMA` D. Uncompressed only for `USDA`, `DEFLATE` for `PNG`

**DE-035** · Obj 4.1 · Difficulty: Medium · Type: Single choice

A USD asset is Y-up centimeters. You convert to `glTF` (meters, Y-up). Besides shrinking points by 100×, what else belongs in the fidelity checklist?

A. Nothing; `glTF` reads `metersPerUnit` B. Scale positions (and extents / `POSITION` min/max); keep +Y; triangulate; flip V; report dropped variants C. Rotate -90 about X because `glTF` is Z-up D. Write the `USDA` as extras and skip mesh export

**DE-036** · Obj 4.7 · Difficulty: Easy · Type: Single choice

`UsdGeom.LinearUnits.meters` and `.centimeters` are:

A. 1.0 and 0.01 B. 100 and 1 C. Tokens `"m"` and `"cm"` D. Undefined on `usd-core`

**DE-037** · Obj 4.6 · Difficulty: Medium · Type: Python-reading

```
print(hasattr(UsdUtils, "ComplianceChecker"))
print(shutil.which("usdchecker"))
```

On the book's `usd-core` 26.8 environment, what prints?

A. `True` then a path to `usdchecker` B. `False` then `None` C. `True` then `None` D. `AttributeError`

**DE-038** · Obj 4.4 · Difficulty: Hard · Type: Single choice

Round-trip CI opens the DCC export and calls `UsdUtils.ComputeAllDependencies`. The return value is a 3-tuple. What are the three lists?

A. Prim paths, property names, time samples B. Layers, asset paths, **unresolved** identifiers C. `USDA` files, `USDC` files, `USDZ` files D. References, payloads, sublayers only

**DE-039** · Obj 4.8 · Difficulty: Medium · Type: Single choice

`usd-core` cannot import `UsdMtlx`. A DCC that must round-trip `MaterialX` graphs needs what?

A. Nothing; `PreviewSurface` is `MaterialX` B. A **full OpenUSD build** (or the DCC's `USD`) with the `MaterialX` plugin; mapping documented separately from `PreviewSurface` C. Rename `.mtlx` to `.usda` D. `Tf.MakeValidIdentifier` on the `MaterialX XML`

**DE-040** · Obj 4.7 · Difficulty: Medium · Type: Single choice

Exporter hooks (Obj 4.7 / 4.8) should typically enforce which **stage** fields before publish?

A. `defaultPrim`, `upAxis`, `metersPerUnit`, and (for components) `kind` B. relocates on every layer C. `instanceable = true` on the default prim D. Empty `subLayers` so the file is "simple"

**DE-041** · Obj 4.1 · Difficulty: Medium · Type: Single choice

USD time samples are in **timeCodes**. `glTF` animation input is **seconds**. Conversion rule?

A. Copy `timeCodes` unchanged B. `seconds = timeCode / timeCodesPerSecond` (and author that metadata on the USD stage) C. `seconds = timeCode * metersPerUnit` D. `glTF` cannot animate transforms at all

**DE-042** · Obj 4.1 · Difficulty: Easy · Type: Single choice

USD `doubleSided = true` on a `gprim` maps to which `glTF` material flag?

A. There is no equivalent; drop it B. `material.doubleSided` C. `extras.upAxis` D. `KHR_lights_punctual`

**DE-043** · Obj 4.1 · Difficulty: Medium · Type: Multiple select Select two.

A fidelity checklist for USD → `glTF` should include which **reports of loss** (not silent drops)?

A. Variant selections baked; other variants omitted B. Layer/payload structure flattened to one node tree C. Triangulated positions (this is

success, not loss, if documented as the transform) D. USDA ↔ USDC encoding (this is a different, lossless hop)

**DE-044** · Obj 4.2 · Difficulty: Medium · Type: Single choice

UsdShade Material with a surface output connected to UsdPreviewSurface vs a MaterialX standard\_surface graph: what belongs in the mapping document?

A. They are bitwise identical; no row needed B. PreviewSurface is a small fixed PBR; MaterialX nodedefs are a broader graph — list each input, color space, and what has **no** PreviewSurface peer C. MaterialX is only textures; UsdShade is only lights D. Both are glTF extras only

**DE-045** · Obj 4.2 · Difficulty: Easy · Type: Single choice

A mapping table row should contain at least:

A. Only the USD token name B. Source concept, target concept, transform (units/UV/color), and loss/notes C. Only Python API names D. NVIDIA exam objective IDs

**DE-046** · Obj 4.3 · Difficulty: Medium · Type: Single choice

usd-core has no usdcat. What is the Python equivalent to rewrite a.usda as crate?

A. There is none B. Sdf.Layer.FindOrOpen("a.usda").Export("a.usdc") (encoding follows the destination extension) C. stage.Flatten() — flatten always writes USDC D. Rename the file to .usdc

**DE-047** · Obj 4.3 · Difficulty: Medium · Type: Single choice

For **archival** of a human-readable show bible (comments, layer structure you may need to recover in 20 years), which encoding is the usual pick?

A. USDC only (smaller is always better archives) B. USDA (or USDA + a crate publish), because text remains inspectable without the crate parser C. USDZ STORE only D. OBJ

**DE-048** · Obj 4.4 · Difficulty: Medium · Type: Single choice

A DCC exporter writes a flattened crate and drops the payload that pointed at Chair\_payload.usdc. Round-trip test using ComputeAllDependencies will show what?

A. Nothing — flatten is lossless for dependencies B. The payload layer disappears from the layer list; the test should **fail** if the pipeline required that payload to survive C. Unresolved always includes Chair\_payload.usdc even after flatten D. Flatten adds a fake payload automatically

**DE-049** · Obj 4.4 · Difficulty: Medium · Type: Multiple select Select two.

A round-trip harness (Obj 4.4) should compare which of the following after DCC export?

A. Key attribute Get() values (including custom namespaces) B. ComputeAllDependencies layer/asset/unresolved lists C. GetMaster() paths D. Whether the file still starts with #usda only — crate exports are automatic failures

**DE-050** · Obj 4.5 · Difficulty: Medium · Type: Single choice

customData dictionary vs a typed factory:partId attribute vs a Door schema. Which guideline matches Obj 4.5?

A. Always use customData for anything non-UsdGeom B. Queryable, typed, per-prim payload → attribute or schema; opaque bag of editor notes → customData; shared studio type → schema C. Schemas cannot be used during import/export D. customData is deleted on Flatten

**DE-051** · Obj 4.6 · Difficulty: Medium · Type: Single choice

You write a studio validator that flags meshes with points but no extent. Where does it run in the exchange pipeline?

A. Only inside usdview B. After transformation, **before publish** (validation stage of extract → transform → validate) C. Before extraction, on the DCC native scene only D. It replaces the mapping document

**DE-052** · Obj 4.6 · Difficulty: Hard · Type: Single choice

After authoring upAxis and metersPerUnit but **not** defaultPrim, lab 27's two checkers return which counts?

A. Geom 0, core still has MissingDefaultPrim (1) B. Both still 3 errors C. Core 0, geom still missing defaultPrim D. Both 0 because Sphere.Define sets defaultPrim

**DE-053** · Obj 4.7 · Difficulty: Medium · Type: Single choice

Lab 26 applies rotateX(-90) then scale(factor). Why order matters for a Z-up asset into Y-up cm?

A. Order never matters for xformOps B. xformOpOrder is applied in list order; rotate then scale matches "stand the asset up, then size it in shot units" C. Scale must always be first in USD D. USD ignores xformOpOrder on referencing prims

**DE-054** · Obj 4.7 · Difficulty: Easy · Type: Single choice

`Tf.IsValidIdentifier("Crate_Box")` vs `"Crate_Box"`?

- A. True, True B. True, False C. False, False  
D. False, True

**DE-055** · Obj 4.8 · Difficulty: Medium · Type: Single choice

A DCC importer calls `Usd.Stage.Open(path, Usd.Stage.LoadNone)` and never Loads. Chair assets were published as **payloads**. Artists see empty roots. Why?

- A. `LoadNone` is an illegal argument, so the stage is empty B. Payloads stay unloaded until `Load/LoadAll`; the importer must set a load policy. References would still load under `LoadNone` C. `LoadNone` unloads references and payloads equally D. Empty roots mean `defaultPrim` was missing, not a load-set issue

**DE-056** · Obj 4.8 · Difficulty: Medium · Type: Multiple select Select two.

Extending a DCC importer for animation (Obj 4.8) should:

- A. Read time samples **and** defaults (`Get()` vs `Get(time)`); missing default is not “zero” B. Apply layer offsets when evaluating referenced clips C. Drop all samples and keep one default at frame 0 D. Require USDA; refuse USDC

**DE-057** · Obj 4.1 · Difficulty: Medium · Type: Single choice

glTF custom vertex attributes must be named how, if you park a USD primvar that has no core glTF slot?

- A. Any name B. Prefix `_` (e.g. `_factory_partId` analog at vertex rate) or use `extras` at mesh/node C. USD colon namespaces (`factory:partId`) are legal glTF attribute names D. Only via `MaterialX`

**DE-058** · Obj 4.3 · Difficulty: Easy · Type: Single choice

`Sdf.FileFormat.FindByExtension("usd").formatId` on 26.08?

- A. `usdc` B. `usd` C. `usda` D. None

**DE-059** · Obj 4.7 · Difficulty: Medium · Type: Single choice

Handedness: USD and glTF are both **right-handed**. A converter that also mirrors X “because game engines are left-handed” without a documented axis policy is:

- A. Required for all glTF B. An undocumented extra transform that breaks fidelity unless the mapping says so C. How `upAxis = Z` is implemented D. Performed automatically by `AddReference`

**DE-060** · Obj 4.4 · Difficulty: Medium · Type: Single choice

`UsdPhysics` APIs on a collider mesh survive a DCC that only knows `PreviewSurface` and `Mesh`. Round-trip result?

- A. Physics is core glTF, so it always survives B. Loss unless the importer/exporter maps it (custom schema/API) or you keep a sidecar USD; document it as out of scope C. Flatten restores physics D. `kind = component` encodes physics

## Answers

**DE-001 — Answer: B.** Text vs crate is the core trade-off. They are different encodings; USDC stores the same scene data including samples. Review: §7.1–7.2, §31.1.

**DE-002 — Answer: C.** Verified: `CreateNew("scene.usd")` writes crate magic PXR-USDC but `formatId` is `usd`. Review: §31.3.

**DE-003 — Answer: A, B.** Verified: copied USDA bytes under `.usd` open with radius 2; `formatId` stays `usd`; magic `#usda`. Review: §31.3.

**DE-004 — Answer: B.** Mergeable text for work layers; crate for high-frequency published I/O. Review: §31.1.

**DE-005 — Answer: B.** Extension `.usd` is ambiguous. `FindByExtension("usd")` is `usd`, not the encoding. Crate `.usd` does **not** report `formatId usdc`. Review: §31.3.

**DE-006 — Answer: B.** Verified: `"Crate_Box" → Crate_Box`. Spaces are not valid identifiers. Review: §28.4, lab 25.

**DE-007 — Answer: B.** Verified: `MakeValidIdentifier("2ball")` is `_ball`; `IsValidIdentifier("2ball")` is False. Review: §28.4.

**DE-008 — Answer: B.** Verified: fallback Y, 0.01. Review: §28.1–28.2, lab 26.

**DE-009 — Answer: B.** Verified: composed Cube size stays `1.0`. USD does not auto-convert. Review: §28.3, lab 26.

**DE-010 — Answer: B.** Verified: `factor asset_mpu / shot_mpu = 100`. Review: §28.3, lab 26.

**DE-011 — Answer: A, C.** Z-up into Y-up → rotateX -90 on the **referencing** prim; read the asset file's metrics. Changing shot upAxis does not rotate referenced geometry. Review: §28.2-28.3, lab 26.

**DE-012 — Answer: B.** Verified in lab 25: f 1 2 3 → [0, 1, 2]. Review: §29.2.

**DE-013 — Answer: B.** Verified: 100 cm → (1.0, 0.0, 0.0) meters, metersPerUnit = 1. Review: §29.2, lab 25.

**DE-014 — Answer: B.** Verified: GetExtentAttr() is None until you compute and Set. Review: §13.3, §29.2.

**DE-015 — Answer: B.** Namespaced custom attributes are the lightweight Obj 4.5 path (factory:partId in lab 25). Review: §29.4.

**DE-016 — Answer: A, C.** Shared typed prims need schemas and Isa. A one-off id can stay a custom attribute. Parsing USDA does **not** register Door (D is false). Review: §29.4, §37.1, lab 31.

**DE-017 — Answer: B.** Verified: Door stays unregistered after parse; usdGenSchema is missing on usd-core. Review: §37.1, lab 31.

**DE-018 — Answer: B.** Verified: ComplianceChecker is gone. Review: §30.4, lab 27.

**DE-019 — Answer: B.** Verified lab 27 error names. Review: §30.3.

**DE-020 — Answer: A, B.** Metadata checkers plus unresolved-dependency checks are the integrity baseline. GetMaster() is gone on 26.08 and is not a validator. USDA files can be valid. Review: §30.5, §32.4.

**DE-021 — Answer: B.** Python UsdValidation is enough. Review: §30.2-30.3.

**DE-022 — Answer: B.** Core glTF primitives are triangles. Review: §27.4.

**DE-023 — Answer: B.** V flip is the documented UV convention. Review: §27.4.

**DE-024 — Answer: A, C.** Composition is not a glTF core concept; you bake a node tree (D is what you *keep*, not what you *lose*). Triangle positions after triangulation are the intended geometry carry. Review: §27.4.

**DE-025 — Answer: B.** Meters, +Y. glTF does not read USD stage metadata. Review: §27.4.

**DE-026 — Answer: B.** PreviewSurface metal/rough → glTF PBR metal-rough. Review: §27.4.

**DE-027 — Answer: B.** MaterialX is a nodedef graph; UsdShade Shader/NodeGraph is the cousin, not Mesh extents. Review: §27.4.

**DE-028 — Answer: A, C.** Mapping documents are the Obj 4.2 artifact. They do not replace validators or exam paperwork. Review: §27.3.

**DE-029 — Answer: B.** Round-trip is extract/transform/validate plus automated diffs. Flatten-on-import is a common source of loss. Review: §27.2, §27.5, §35.3.

**DE-030 — Answer: A, C.** Keep structure and custom/API data. Random names and flatten-by-default fight round-trip. Review: §27.5, §29.5.

**DE-031 — Answer: B.** Lab 26: typeless DefinePrim so the asset type comes through. Review: §28.3, §29.5.

**DE-032 — Answer: B.** Verified: Bind without Apply writes the rel, HasAPI False. Review: §40.x, lab 34.

**DE-033 — Answer: B.** Fallback is catmullClark; tessellated CAD wants none. Review: §13, lab 11 / 25.

**DE-034 — Answer: B.** Verified lab 28: compress\_type 0 STORE. Review: §31.2.

**DE-035 — Answer: B.** Scale to meters, keep Y-up, triangulate, flip V, document dropped composition. glTF is not Z-up. Review: §27.4, §28.3.

**DE-036 — Answer: A.** Verified: 1.0 and 0.01. Review: §28.1.

**DE-037 — Answer: B.** Verified: False, None. Review: §30.2, lab 27.

**DE-038 — Answer: B.** Verified lab 28: 3-tuple (layers, assets, unresolved). Review: §32.4.

**DE-039 — Answer: B.** UsdMtlx ImportError on usd-core. Review: §27.1, §35.1.

**DE-040 — Answer: A.** Exporter hooks author pipeline metadata. Review: §29.6, §34.1.

**DE-041 — Answer: B.** glTF sampler input is seconds; divide by timeCodesPerSecond. Core glTF can animate TRS (not “cannot animate”). Review: §27.4.

**DE-042 — Answer: B.** doubleSided is a core glTF material flag. Review: §27.4.

**DE-043 — Answer: A, B.** Variants and layer/payload structure are composition losses. Triangulation is a documented transform of geometry, not a silent concept drop. USDA↔USDC is lossless and not a glTF issue. Review: §27.4, §31.1.

**DE-044 — Answer: B.** PreviewSurface ≠ full MaterialX graph; the mapping table lists gaps. Review: §27.4.

**DE-045 — Answer: B.** That is the Chapter 27 template. Review: §27.3.



**DE-046 — Answer: B.** `Export` picks encoding from the destination extension (verified Ch 31). Flatten is composition, not format. Rename does not transcode. Review: §31.4.

**DE-047 — Answer: B.** Archival readability favors USDA. Review: §31.1.

**DE-048 — Answer: B.** Flatten bakes payloads away; a round-trip that required the payload must fail. Review: §27.5, §34.2.

**DE-049 — Answer: A, B.** Values plus dependency lists. `GetMaster` is gone; crate vs USDA is not an automatic fail. Review: §27.5, §32.4, §35.3.

**DE-050 — Answer: B.** Match the container to how the data will be queried. Review: §29.4, §12.2.

**DE-051 — Answer: B.** Validation is the third pipeline stage, at publish. Review: §27.2, §30.5.

**DE-052 — Answer: A.** Stretch in lab 27: skip `SetDefaultPrim` → core still `MissingDefaultPrim`; `geom` is 0 once axis and meters are set. `Sphere.Define` does not set `defaultPrim`. Review: lab 27, §30.3.

**DE-053 — Answer: B.** Ops apply in `xformOpOrder`; lab 26 authors `rotateX` then scale. Review: §28.3, lab 26, §39.

**DE-054 — Answer: B.** Verified: `Crate_Box` True, `Crate_Box` False. Review: §28.4.

**DE-055 — Answer: B.** Verified: `LoadNone` leaves references loaded and payloads unloaded. Importers that pass `LoadNone` must Load what artists need. Review: §17.2, §29.5.

**DE-056 — Answer: A, B.** Defaults vs samples and layer offsets are composition/time rules importers must honor. Review: §10, §16.4, §29.5.

**DE-057 — Answer: B.** glTF requires `_` prefix on custom vertex attributes; colons are not legal names there. Review: §27.4.

**DE-058 — Answer: B.** Verified: `FindByExtension("usd").formatId` is `usd`. Review: §31.3.

**DE-059 — Answer: B.** Extra mirrors must be in the mapping document. USD/glTF are right-handed; reference does not mirror. Review: §27.4, §28.5.

**DE-060 — Answer: B.** Domain schemas survive only if the DCC maps them or you keep USD as source of truth. Review: §27.5, §35.4.

---

*Data Exchange domain complete: DE-001–DE-060 (target 60).*

## Question Bank — Pipeline Development (PD)

**Original practice questions.** Not actual NVIDIA exam content. Mapped to NCP-OUSD study-guide objectives 7.1–7.8 (Pipeline Development, 14%).

Verified against **USD 26.08** (usd-core 26.8). Domain target: 56 questions. This file currently holds **PD-001-PD-056** (domain complete).

Work the stems first. Answers and explanations are grouped at the end.

---

**PD-001** · Obj 7.2 · Difficulty: Easy · Type: Single choice

In a USD pipeline, what is the difference between a **work** file and a **publish**?

A. Work files are USDC; publishes are USDA B. A work file is still being edited; a publish is an immutable snapshot other assets may **pin** C. Publishes live only in the session layer D. Work files cannot contain references

---

**PD-002** · Obj 7.2 · Difficulty: Medium · Type: Single choice

A shot references `assets/chair/work/chair.usda`. What guideline does that violate?

A. Relative paths are illegal B. Shots should pin a **published version** (for example `assets/chair/v017/chair.usda`), not a work tree that still changes C. Chairs may only be payloads D. Version numbers must appear in the prim name `/Chair_v017`

---

**PD-003** · Obj 7.2 · Difficulty: Medium · Type: Single choice

`defaultPrim = "Hero_v003"` on a published component. What is wrong?

A. `defaultPrim` is illegal on components B. Version belongs in the **folder** (and maybe `assetInfo['version']`); the identifier/`defaultPrim` should stay stable (Hero) C. `v003` must be uppercase D. `defaultPrim` must match the filename only, never the prim

---

**PD-004** · Obj 7.2 · Difficulty: Easy · Type: Single choice

`assetInfo['identifier'] = "chair"` should change when?

A. Every publish (`chair_v002`, then `chair_v003`) B. Almost never — it is the catalog key for the asset C. Whenever the USDA comment changes D. Only when flattening

---

**PD-005** · Obj 7.2 · Difficulty: Medium · Type: Multiple select Select two.

Obj 7.2 asks you to **document** asset structure. Which artifacts belong in that guideline pack?

A. Folder tree, layer-stack diagram (strongest first), prim/kind tree, ownership table B. A private mental model the lead TD keeps C. Pin rules (work vs publish) and naming (`Tf.MakeValidIdentifier`) D. NVIDIA's live exam PDF reprinted as the studio bible

---

**PD-006** · Obj 7.2 · Difficulty: Medium · Type: Single choice

ASWF-style structure for a chair component typically includes what?

A. One monolithic USDA with every shot's animation inside B. A thin interface layer (`defaultPrim`, payload/reference to contents), plus geometry/material layers C. A `PointInstancer` of every vertex D. No `kind` — kinds are only for assemblies

---

**PD-007** · Obj 7.2 · Difficulty: Medium · Type: Single choice

`UsdUtils.ComputeAllDependencies("entry.usda")` returns what?

A. A dict `{layers, assets, unresolved}` B. A **3-tuple** `(layers, assets, unresolved)` walked **recursively** C. Only unresolved paths D. Prim paths in traverse order

---

**PD-008** · Obj 7.2 · Difficulty: Medium · Type: Single choice

`ExtractExternalReferences("shot.usda")` returns `(['./anim.usda', './layout.usda'], [], [])`. Does that prove the shot has no textures?

A. Yes — empty second list means no assets exist anywhere B. No — extract is **one file**; textures on `layout.usda` never appear. Run `ComputeAllDependencies` C. Yes if the shot is USDA D. Extract always recurses into sublayers

---

**PD-009** · Obj 7.2 · Difficulty: Medium · Type: Multiple select Select two.

Which statements about the two inventory APIs are true?

A. Extract: `(sublayers, refs+assets, payloads)` for **one** layer's authored strings B. Compute: recursive `(layers, assets, unresolved)`; payloads **are** followed C. Missing files abort Compute with `Tf.ErrorException` and no tuple D. Extract's second list is USD references only — never `.png`

---

**PD-010** · Obj 7.2 · Difficulty: Easy · Type: Single choice

On a layer-stack diagram labeled “strongest at top,” lighting is drawn **below** layout, but the caption says lighting overrides layout translates. The caption is:

A. True, because lighting works later in the schedule B. False — below means weaker; first-listed / top of the strength diagram wins C. True if lighting uses `def` D. True because payloads beat sublayers

**PD-011** · Obj 7.2 · Difficulty: Medium · Type: Single choice

Two layout artists overwrite the same `shot.usdc` all day. What pipeline split fixes it?

A. Git rebase on USDC B. Department **sublayers** (`layout.usda`, `anim.usda`, ...) with one owner per file; stronger departments listed first C. Native instancing of the shot D. Flatten every hour

**PD-012** · Obj 7.3 · Difficulty: Easy · Type: Single choice

A pipeline stores **WIP department layers** in Git and **farm caches** on a render share. Encoding policy?

A. Git: USDC; farm: USDA B. Git: USDA (diff/merge); farm: USDC (I/O) C. Always USDZ D. Encoding does not matter in a pipeline

**PD-013** · Obj 7.3 · Difficulty: Medium · Type: Single choice

A publish job writes `chair.usd` via `CreateNew`. Reviewers assume it is text because the extension is not `.usdc`. What should the pipeline check?

A. Extension only B. Magic bytes (PXR-USDC vs `#usda`); `formatId` for `.usd` is `usd`, not the encoding name C. `FindByExtension("usd")` equals `usdc` D. Git says “binary” so it must be USDA

**PD-014** · Obj 7.5 · Difficulty: Easy · Type: Single choice

`Ar.GetResolver()` vs `Ar.GetUnderlyingResolver()` on `usd-core 26.8?`

A. Both return `DefaultResolver` B. Facade **Resolver**; underlying **DefaultResolver** C. Facade `DefaultResolver`; underlying `Resolver` D. Both return `None` until a stage is open

**PD-015** · Obj 7.5 · Difficulty: Medium · Type: Single choice

`hasattr(Ar.GetResolver(), "AnchorRelativePath")` on USD 26.08?

A. **True** — that is the supported anchor API B. **False** — use `CreateIdentifier(path, ResolvedPath(layer.realPath))` or

`layer.ComputeAbsolutePath` C. **True** only inside `ResolverContextBinder` D. The method moved to `Sdf.Path`

**PD-016** · Obj 7.7 · Difficulty: Medium · Type: Single choice

From a temp cwd, `r.Resolve("./tex/wood.png")` is **False** even though `assets/tex/wood.png` exists. Why?

A. PNG is not a USD asset B. Raw `Resolve` of `./` uses **cwd**, not the layer; composition anchors to the **layer** C. Search paths always apply to `./` paths D. You must call `AnchorRelativePath`

**PD-017** · Obj 7.7 · Difficulty: Medium · Type: Single choice

`r.CreateIdentifier("./tex/wood.png")` with **no** anchor returns what, and is that path search-path eligible?

A. `./tex/wood.png` still; `./` paths **are** search-path lookups B. `tex/wood.png`; **unanchored** identifiers become search-path (context-dependent); `./` paths are **not** C. An absolute `/mnt/...` path D. **None**

**PD-018** · Obj 7.7 · Difficulty: Medium · Type: Multiple select Select two.

With a `DefaultResolverContext` bound to `searchA/` (which contains `tex/wood.png`):

A. `Resolve("tex/wood.png")` succeeds B. `Resolve("./tex/wood.png")` still fails — `./` is not a search-path lookup C. Both succeed equally D. Unbinding keeps the search path for the rest of the process

**PD-019** · Obj 7.5 · Difficulty: Medium · Type: Single choice

`shot.usda` references `@chair.usda@` (no `./`). `Usd.Stage.Open("shot.usda")` without a context reports `ErrorInvalidAssetPath`. Opening with `pathResolverContext=` pointing at `assets/` succeeds. Lesson?

A. `Open` cannot take a resolver context B. Context-dependent identifiers need `Stage.Open(..., pathResolverContext=)`; do not rely on cwd C. Missing `defaultPrim` always produces `InvalidAssetPath` D. You must convert the reference to a sublayer

**PD-020** · Obj 7.5 · Difficulty: Medium · Type: Multiple select Select two.

A studio custom resolver is missing in the DCC. Which integration steps match Obj 7.5?

A. Ship `ArResolver` plugin + `plugInfo.json` on `PXR_PLUGINPATH_NAME` (before USD initializes) B. Rebuild the plugin against the **same** USD the DCC loads C. `pip install` any

resolver wheel compiled against USD 24 into a USD 26 DCC D. Call `AnchorRelativePath` to register the plugin

**PD-021** · Obj 7.6 · Difficulty: Easy · Type: Single choice

Where should a pipeline store a **typed, queryable** department tag `dept = "layout"` on a prim?

A. In the prim **name** (`layout_Chair`) B. `customData` (dictionary metadata) via `SetCustomDataByKey / USDA customData = { ... }` C. In `faceVertexIndices` D. Only as a filename prefix

**PD-022** · Obj 7.6 · Difficulty: Medium · Type: Single choice

`prim.GetCustomData()` after setting only `dept` may contain extra keys such as `userDocBrief`. How should a validator read the studio key?

A. Assume `GetCustomData()` is exactly `{dept: ...}` B. `GetCustomDataByKey("dept")` (or equivalent key lookup) C. `GetAssetInfo()` — `customData` aliases `assetInfo` D. Parse USDA with regex only

**PD-023** · Obj 7.6 · Difficulty: Medium · Type: Single choice

`assetInfo` vs `customData` in a publish pipeline?

A. They are the same field B. `assetInfo` holds catalog fields (`identifier`, `version`, `name`); `customData` is the open bag for studio keys; neither replaces typed attributes/schemas C. `customData` is stripped on every Save D. `assetInfo` is only legal on materials

**PD-024** · Obj 7.6 · Difficulty: Medium · Type: Multiple select Select two.

Flattening a shot for a vendor **keeps** `customData` owner names and often **absolutizes** `@/mnt/show/tex.png@`. What must the pipeline still do (Obj 7.6 / 1.9)?

A. Clear confidential `customData / assetInfo` keys B. `UsdUtils.ModifyAssetPaths` (or `localize`) so paths are package-relative C. Rely on Flatten to strip metadata automatically D. Leave `/mnt` paths so the vendor's `DefaultResolver` "finds them"

**PD-025** · Obj 7.7 · Difficulty: Easy · Type: Single choice

A publish gate should fail when `ComputeAllDependencies` returns a non-empty **unresolved** list. Why?

A. Unresolved is only a warning for `usdview` B. The package is missing a layer or asset the stage needs; ship is incomplete C. Unresolved always includes the session layer D. Unresolved means the file is USDA

**PD-026** · Obj 7.7 · Difficulty: Medium · Type: Single choice

Authored path is `@./gone.jpg@`. Compute's unresolved entry is often an **absolute** tmp path. How should the gate compare?

A. String-equal against `./gone.jpg` only B. `Normalize / os.path.basename` (or resolve both) before matching C. Ignore unresolved if `stderr` also warned D. Unresolved is never absolutized

**PD-027** · Obj 7.4 · Difficulty: Medium · Type: Single choice

A DCC round-trip CI should treat which event as a **failure** if the pipeline requires live payloads?

A. USDA rewritten as USDC (lossless encoding hop) B. Flatten on export that **bakes away** the payload layer C. `upAxis` still Y D. `ComputeAllDependencies` still lists the payload

**PD-028** · Obj 7.4 · Difficulty: Medium · Type: Multiple select Select two.

A DCC **importer** in a pipeline (Obj 7.4 / 7.8) should:

A. Honor `LoadNone` vs `LoadAll` / masks according to studio policy B. Preserve namespaced custom attributes and `customData` keys the mapping lists C. `Xform.Define` on every referenced root before adding the reference D. Call `GetMaster()` to discover assets

**PD-029** · Obj 7.8 · Difficulty: Medium · Type: Single choice

Extending a DCC importer: chairs are payloads. The plugin uses `Open(..., LoadNone)` for speed and never Loads. Result?

A. Illegal API B. Empty payload contents until Load; references would still be populated C. Payloads and references both stay empty D. Hydra loads them anyway

**PD-030** · Obj 7.8 · Difficulty: Medium · Type: Single choice

The DCC's USD is 26.08; a studio file-format plugin was built against 24.11 and dropped on `PXR_PLUGINPATH_NAME`. Typical outcome?

A. Perfect load; ABI is stable across years B. Plugin fail / missing types — **match plugin builds to the DCC's USD** (Obj 3.1 / 7.8) C. USD silently recompiles the plugin D. Only USDA files fail

**PD-031** · Obj 7.1 · Difficulty: Medium · Type: Single choice

A pipeline converts published USD components to glTF for a web viewer. Fidelity policy should require which documented steps?

A. Copy USDA into .gltf extras and skip meshes B. Triangulate, meters/+Y, UV V-flip, bake or report dropped variants/layers, then **validate** the glTF C. Trust metersPerUnit inside glTF (glTF reads USD metadata) D. Keep quads; core glTF 2.0 stores n-gons

**PD-032** · Obj 7.1 · Difficulty: Easy · Type: Single choice

Which hop is **lossless** and therefore not a fidelity incident?

A. USD Mesh → OBJ B. USDA ↔ USDC C. USD variants → core glTF D. CAD NURBS → USD Mesh

**PD-033** · Obj 7.1 · Difficulty: Medium · Type: Multiple select Select two.

A mapping document for the USD→glTF farm job should list:

A. Known loss (composition arcs, unbaked variants) B. Required transforms (triangulate, V-flip, unit scale) C. That glTF formatId is usd D. That Flatten is illegal in every pipeline

**PD-034** · Obj 7.4 · Difficulty: Medium · Type: Single choice

Exporter **hooks** shared by four DCCs should run when?

A. After the farm already rendered B. Immediately before Save/publish: defaultPrim, units, upAxis, kind, identifier C. Only inside usdview D. Only when exporting USDA

**PD-035** · Obj 7.2 · Difficulty: Medium · Type: Single choice

GetLayerStack() of an opened shot lists what **first**?

A. Weakest sublayer B. The anonymous **session** layer (strongest local sheet) C. layout.usda D. Referenced chair assets

**PD-036** · Obj 7.5 · Difficulty: Easy · Type: Single choice

Artists should author which style of asset path when a studio resolver is in play?

A. Absolute /mnt/show/seq/shot/tex.png in every layer B. Resolver **identifiers** (search-path or URI), never machine mounts C. Windows drive letters only D. file:// with user home directories

**PD-037** · Obj 7.7 · Difficulty: Medium · Type: Single choice

IsContextDependentPath("tex/wood.png") vs ("./tex/wood.png")?

A. Both True B. True for tex/wood.png, False for ./tex/wood.png C. Both False D. ./ is always context-dependent

**PD-038** · Obj 7.6 · Difficulty: Medium · Type: USDA-reading

```
#usda 1.0
def Xform "Chair" (
 assetInfo = {
 string identifier = "chair"
 string version = "3"
 }
 customData = {
 string dept = "model"
 }
)
{
}
```

Which Python reads the catalog key vs the studio tag?

A. Both from GetAttribute("identifier") B. GetAssetInfoByKey("identifier") and GetCustomDataByKey("dept") C. GetAssetInfoByKey("dept") for both D. Layer metadata only — prims cannot hold these dictionaries

**PD-039** · Obj 7.2 · Difficulty: Hard · Type: Single choice

ExtractExternalReferences("shot.usda") does not list chair.usda, but the composed shot shows a chair. Why?

A. References on sublayers are illegal B. Extract does not recurse; the reference is authored on layout.usda C. USDC chairs are invisible to extract D. The chair exists only as a PointInstancer prototype

**PD-040** · Obj 7.3 · Difficulty: Medium · Type: Single choice

usd-core has no usdcats. A pipeline convert job should:

A. Fail — Obj 7.3 requires the CLI B. Sdf.Layer.Export to the destination extension (.usda / .usdc) C. Rename bytes without transcoding D. Always Flatten first

**PD-041** · Obj 7.2 · Difficulty: Easy · Type: Single choice

Pipelines should author which field so UsdUtils.GetModelNameFromRootLayer is a **stable** asset name?

A. The filename stem only (chair.usda → always "chair") B. defaultPrim (the API reads that; without it it may fall back to an arbitrary root prim name) C. assetInfo['name'] only — defaultPrim is ignored D. A session-layer comment

**PD-042** · Obj 7.2 · Difficulty: Medium · Type: Multiple select Select two.

Which pair is the right split for history vs pins?

A. Git (or similar) versions **work** files B. Publish **folders** (`v017/`) are what shots pin C. Daily `monday.usda` sublayers on the shot are the supported history mechanism D. Layer stacks replace git

**PD-043** · Obj 7.5 · Difficulty: Medium · Type: Single choice

Setting `PXR_PLUGINPATH_NAME` **after** the DCC has already imported `pxr` / opened a stage. Expected result?

A. Custom `StudioResolver` appears immediately B. Too late in many hosts — resolver plugins must be visible **before** USD initializes C. USD restarts automatically D. Only Hydra plugins need early path setup

**PD-044** · Obj 7.7 · Difficulty: Medium · Type: Single choice

A validator flags asset paths that contain spaces, backslashes, or unanchored `C:\` drives in published layers. That check is primarily Obj:

A. 8.2 (PreviewSurface) B. 7.7 (asset paths formatted correctly) plus resolver policy (7.5) C. 1.6 (LIVERPS) D. 3.3 (custom kinds)

**PD-045** · Obj 7.4 · Difficulty: Medium · Type: Single choice

Round-trip of `UsdPhysics.CollisionAPI` through a DCC that only maps Mesh + PreviewSurface. Pipeline choice?

A. Physics always survives because it is core USD imaging B. Keep USD as source of truth / sidecar, or extend the importer; document the loss in the mapping C. Flatten restores physics APIs D. Set `kind = component` to encode colliders

**PD-046** · Obj 7.8 · Difficulty: Medium · Type: Multiple select Select two.

When extending a DCC importer you should **not**:

A. Flatten the stage on import “to simplify the node graph” if the studio needs live references B. Typeless `DefinePrim` then `AddReference` so asset types survive C. Drop `factory:partId` because it is not `UsdGeom` D. Honor layer offsets on referenced animation

**PD-047** · Obj 7.2 · Difficulty: Medium · Type: Single choice

A kitchen assembly should use which **kinds** in the documented model hierarchy?

A. Assembly at the kitchen, **component** on each published chair; groups as needed B. Component on every vertex C. No kinds — they slow the farm D. `model` on cameras only

**PD-048** · Obj 7.3 · Difficulty: Easy · Type: Single choice

Why might a pipeline **forbid USDC in Git** for department WIP?

A. USDC cannot store references B. Crate is binary — merges and reviews are painful compared with USDA C. Git cannot store files larger than 1 KB D. USDC is deprecated in 26.08

**PD-049** · Obj 7.5 · Difficulty: Medium · Type: Single choice

`Ar.ResolverContextBinder` is used in lab 29 to:

A. Register a new C++ resolver class B. Temporarily bind a `DefaultResolverContext` (search paths) for `Resolve` calls C. Mute layers D. Flatten the stage

**PD-050** · Obj 7.7 · Difficulty: Hard · Type: Python-reading

Lab 29 prints with `ctx errors/type: 0 Xform` and `no ctx errors: 1`

`ErrorInvalidAssetPath`. The referenced identifier was `@chair.usda@`. What did the context provide?

A. A population mask B. A search path so the identifier resolved under `assets/` C. `LoadAll` D. A session layer

**PD-051** · Obj 7.6 · Difficulty: Easy · Type: Single choice

Pipeline documentation (`assetInfo.identifier`) vs secrets (`customData.owner = "jdoe"`) in a **vendor** delivery:

A. Ship both; Flatten removes owner B. Keep identifier if it is public catalog; **strip** owner and other PII/secrets C. Strip identifier always; keep owner D. `customData` cannot be stripped from Python

**PD-052** · Obj 7.2 · Difficulty: Medium · Type: Single choice

`UsdUtils.LocalizeAsset` / USDZ packaging uses the same inventory as Compute. After localize, unresolved `gone.jpg` should:

A. Be ignored B. Fail the job or be listed as a known omission — do not ship a silent hole C. Turn into a Sphere D. Always be downloaded from NVIDIA

**PD-053** · Obj 7.1 · Difficulty: Medium · Type: Single choice

Farm converts USD → glTF **after** Flatten. Variants are gone. Is that a fidelity bug?

A. Always — Flatten is forbidden before glTF B. Only if the mapping promised live variants; if the job is “delivery bake,” document Flatten as the transform and still validate meshes/UVs/units C. Never — glTF stores variant sets in core D. Only if USDA was used

**PD-054** · Obj 7.4 · Difficulty: Medium · Type: Multiple select Select two.

CI after DCC export should compare:

A. `ComputeAllDependencies` 3-tuple (basenames) B. Selected attribute `Get()` values including custom namespaces C. `GetMaster()` equality D. That `ComplianceChecker` still exists

**PD-055** · Obj 7.8 · Difficulty: Easy · Type: Single choice

usd-core lacks `usdview` and Hydra. A pipeline importer running in a headless convert farm should:

A. Require `usdview` to import B. Use `Usd/Sdf` APIs only; do not assume imaging plugins C. Call `UsdMtlx` on this wheel D. Shell out to `usdcats` on usd-core

**PD-056** · Obj 7.2 · Difficulty: Medium · Type: Single choice

Best pin for a **locked** lighting shot that must not drift when modelers keep working?

A. `assets/chair/work/chair.usda` B. `assets/chair/v017/chair.usda` (immutable publish) C. Sublayer every historical `v001...v017` on the shot D. Rename the prim `/Chair_v017` in the work file

## Answers

**PD-001 — Answer: B.** Work vs immutable publish vs pin is the pipeline loop. Encoding is independent. Review: §32.1–32.3.

**PD-002 — Answer: B.** Pin published versions, not work trees. Version in the prim name is the other common mistake. Review: §32.3.

**PD-003 — Answer: B.** Stable `defaultPrim` / identifier; version in folders. Review: §32.3, §32.2.

**PD-004 — Answer: B.** Identifier is the catalog key. Review: §32.3.

**PD-005 — Answer: A, C.** Obj 7.2 is written guidelines plus diagrams. Review: §32.5, §23.5.

**PD-006 — Answer: B.** Interface + payload/contents layers; kinds on models. Review: §23.2–23.5.

**PD-007 — Answer: B.** Verified 3-tuple, recursive. Not a dict. Review: §32.4, lab 28.

**PD-008 — Answer: B.** Extract is one file. Review: §32.4.

**PD-009 — Answer: A, B.** Missing files warn and list; they do not abort. Extract’s second list includes textures. Review: §32.4.

**PD-010 — Answer: B.** First listed / top of the strength diagram is stronger. Review: §15.1, §32.6.

**PD-011 — Answer: B.** One owner per department layer. Review: §32.6, §15.5.

**PD-012 — Answer: B.** Same trade-off as DE, applied as pipeline policy. Review: §31.1, §7.3.

**PD-013 — Answer: B.** Verified magic vs `formatId usd`. Review: §31.3.

**PD-014 — Answer: B.** Verified lab 29. Review: §33.2.

**PD-015 — Answer: B.** `AnchorRelativePath` is gone on 26.08. Review: §33.2.

**PD-016 — Answer: B.** Naive `./` is cwd; layers anchor. Review: §33.2, lab 29.

**PD-017 — Answer: B.** Verified: no-anchor ident `tex/wood.png`; `./` is not search-path. Review: §33.3, lab 29.

**PD-018 — Answer: A, B.** Verified bound `tex` True, bound `./` False; unbind restores False. Review: lab 29.

**PD-019 — Answer: B.** Verified: with context 0 errors / Xform; without `ErrorInvalidAssetPath`. Review: §33.3, lab 29.

**PD-020 — Answer: A, B.** Plugin path + ABI match. Do not mix USD 24 plugins into 26. Review: §33.5, §35.2.

**PD-021 — Answer: B.** `customData` is the metadata bag (Obj 7.6). Review: §12.2, lab 08.

**PD-022 — Answer: B.** Schema may inject `userDocBrief`. Review: lab 08, §12.2.

**PD-023 — Answer: B.** Catalog vs studio bag vs typed properties. Review: §12.2.

**PD-024 — Answer: A, B.** Flatten does not sanitize. Review: §34.3.

**PD-025 — Answer: B.** Non-empty unresolved fails publish. Review: §32.4.

**PD-026 — Answer: B.** Unresolved is often absolutized. Review: §32.4.

**PD-027 — Answer: B.** Flatten bakes payloads; encoding `USDA↔USDC` is lossless. Review: §34.2, §27.5.

**PD-028 — Answer: A, B.** Load policy + preserve mapped metadata. `Xform.Define` before reference can clobber type; `GetMaster` is gone. Review: §35.3, §29.5.

**PD-029 — Answer: B.** Same LoadNone fact, importer framing. Review: §17.2, §35.3.

**PD-030 — Answer: B.** Match plugin ABI to host USD. Review: §35.2.

**PD-031 — Answer: B.** Fidelity = transform + validate + report loss. Review: §27.4.

**PD-032 — Answer: B.** USDA→USDC lossless. Review: §31.1, §27.1.

**PD-033 — Answer: A, B.** Mapping document contents. Flatten can be a delivery step; it is not universally illegal. Review: §27.3–27.4.

**PD-034 — Answer: B.** Shared hooks before Save. Review: §34.1, §29.6.

**PD-035 — Answer: B.** Session is strongest local. Review: §15.3, §32.6.

**PD-036 — Answer: B.** Identifiers, not mounts. Review: §33.5.

**PD-037 — Answer: B.** Verified lab 29. Review: §33.3.

**PD-038 — Answer: B.** `assetInfo` vs `customData` APIs. USDA metadata is on the spec in parentheses. Review: §12.2.

**PD-039 — Answer: B.** Extract does not walk layout. Review: §32.4.

**PD-040 — Answer: B.** Python Export stand-in for `usdcat`. Flatten is composition. Review: §31.4, §35.1.

**PD-041 — Answer: B.** The API reads `defaultPrim`. Verified: with `defaultPrim = "B"` it returns B; with no default it returned the first root prim (A); an empty layer returns `''`. Author `defaultPrim` for a stable name. Review: §32.3.

**PD-042 — Answer: A, B.** Git for work history; publish folders for pins. Daily sublayers are not version control. Review: §32.3, §32.6.

**PD-043 — Answer: B.** Plugin path must be set before USD init in many hosts. Review: §33.5, §35.2, §36.2.

**PD-044 — Answer: B.** Path format + resolver policy. Review: §33.4.

**PD-045 — Answer: B.** Domain APIs need mapping or a USD sidecar. Review: §35.4, §27.5.

**PD-046 — Answer: A, C.** Flatten-on-import and dropping custom attrs are the anti-patterns; B and D are correct importer behavior (not selected). Review: §29.5, §35.3.

**PD-047 — Answer: A.** Assembly/group/ component model hierarchy. Review: §23.3, lab 22.

**PD-048 — Answer: B.** Crate is not deprecated; it is the wrong Git WIP default. Review: §31.1.

**PD-049 — Answer: B.** Lab 29 binds search-path context. Review: §33.3.

**PD-050 — Answer: B.** Search path `assets/` resolves `@chair.usda@`. Review: lab 29.

**PD-051 — Answer: B.** Identifier can stay; strip secrets. Flatten keeps `customData`. Review: §34.3.

**PD-052 — Answer: B.** Unresolved is a failed or explicit omission, never silent. Review: §32.4, §31.2.

**PD-053 — Answer: B.** Bake vs live is a documented transform. Core glTF has no variant sets. Review: §27.4, §34.2.

**PD-054 — Answer: A, B.** `GetMaster` gone; `ComplianceChecker` gone. Review: §32.4, §35.3, lab 27–28.

**PD-055 — Answer: B.** Headless `Usd/Sdf`; no `usdcat/UsdMtlx` on `usd-core`. Review: §35.1, F5.

**PD-056 — Answer: B.** Pin the published version folder. Review: §32.3.

---

*Pipeline Development domain complete:  
PD-001–PD-056 (target 56).*



## Question Bank — Data Modeling (DM)

**Original practice questions.** Not actual NVIDIA exam content. Mapped to NCP-OUSD study-guide objectives 5.1–5.6 (Data Modeling, 13%).

Verified against **USD 26.08** (usd-core 26.8). Domain target: 52 questions. This file currently holds **DM-001-DM-052** (domain complete).

Work the stems first. Answers and explanations are grouped at the end.

---

**DM-001** · Obj 5.2 · Difficulty: Easy · Type: Single choice

Which `Sdf.ValueTypeNames` type should store Mesh points?

A. `Color3fArray` — RGB and XYZ are interchangeable B. `Point3fArray` (`point3f[]`) — point **role** so transforms treat them as positions C. String of comma-separated numbers D. `IntArray` of packed floats

---

**DM-002** · Obj 5.2 · Difficulty: Easy · Type: Single choice

A texture filename on a shader input should be which value type?

A. `string` B. `asset` (`Sdf.AssetName`) so Ar can resolve it C. `token` D. `matrix4d`

---

**DM-003** · Obj 5.2 · Difficulty: Medium · Type: Single choice

`subdivisionScheme` and `purpose` are closed vocabularies. Preferred type?

A. `string` so any spelling is legal B. `token` (`Sdf.ValueTypeNames.Token`) C. `asset` D. `bool`

---

**DM-004** · Obj 5.2 · Difficulty: Medium · Type: Multiple select Select two.

Which pair of roles is **not** interchangeable even though both are 3-floats?

A. `point3f` (positions; translate with the prim) B. `normal3f` (normals; inverse-transpose) C. They are identical at runtime; roles are comments only D. `color3f` is also a point and should go in `points`

---

**DM-005** · Obj 5.2 · Difficulty: Medium · Type: Single choice

A `translate` op has **only** time samples at 1 and 11, no default. What does `attr.Get()` (no argument) return?

A. The first sample B. `None` C. `(0, 0, 0)` schema fallback D. Linear midpoint

---

**DM-006** · Obj 5.2 · Difficulty: Medium · Type: Python-reading

Samples at `t=1 (0,0,0)` and `t=11 (10,0,0)`. Stage interpolation starts as linear. What is `Get(6)` before and after `SetInterpolationType(Held)?`

A. Linear `(5,0,0)`; held `(0,0,0)` (previous sample) B. Both `(5,0,0)` C. Both `None` D. Linear `None`; held `(10,0,0)`

---

**DM-007** · Obj 5.2 · Difficulty: Easy · Type: Single choice

Default stage interpolation type on a new stage?

A. `Usd.InterpolationTypeHeld` B. `Usd.InterpolationTypeLinear` C. `Spline` D. `None` — you must set it or `Get(t)` raises

---

**DM-008** · Obj 5.1 · Difficulty: Easy · Type: Single choice

A primvar is which kind of property?

A. Any relationship B. An attribute in the `primvars:` namespace with interpolation metadata C. Layer metadata only D. A kind of composition arc

---

**DM-009** · Obj 5.1 · Difficulty: Medium · Type: Single choice

Preferred API to add `displayColor` on a mesh (Obj 5.1)?

A. `CreateAttribute("displayColor", ...)` without a namespace B. `UsdGeom.PrimvarsAPI.CreatePrimvar("displayColor", Color3fArray, interpolation)` C. `SetKind("primvar")` D. `AddReference` to a color layer

---

**DM-010** · Obj 5.1 · Difficulty: Medium · Type: Single choice

Per-face ID integers on a mesh of 10 faces should use which interpolation?

A. `vertex` B. `uniform` C. `constant` with 10 values D. `faceVarying` is required for any per-face data

---

**DM-011** · Obj 5.1 · Difficulty: Medium · Type: Single choice

UV seams (different UVs on a shared point) need which interpolation?

A. `vertex` B. `faceVarying` C. `constant` D. `uniform`

---

**DM-012** · Obj 5.1 · Difficulty: Medium · Type: Python-reading

Lab 12 authors `vertex displayColor` with values red/green and `SetIndices([0,1,0,1])`. What prints?

A. `indexed: False` because vertex cannot be indexed  
 B. `indexed: True`, indices `[0, 1, 0, 1]`, two palette values  
 C. Four color values and no indices  
 D. `CreatePrimvar` raises

**DM-013** · Obj 5.1 · Difficulty: Easy · Type: Single choice

Indexing and interpolation are related how?

A. Indexed means interpolation must be `vertex`  
 B. Independent — indexing is a palette; interpolation is how indices/values map to topology  
 C. Indexed means `constant`  
 D. You cannot index `displayColor`

**DM-014** · Obj 5.1 · Difficulty: Medium · Type: Single choice

`primvars:st` on a Mesh: `GetNamespace()` of that attribute?

A. `""` (empty) B. `primvars` C. `st` D. `inputs`

**DM-015** · Obj 5.1 · Difficulty: Medium · Type: Single choice

A parent Xform authors constant

`primvars:displayColor`. A child Mesh does not.

`PrimvarsAPI(child).FindPrimvarWithInheritance("displayColor")`?

A. Invalid — `primvars` never inherit  
 B. Finds the inherited `primvar` (verified non-null with the parent's value)  
 C. Only works for `st`  
 D. Raises `Tf.ErrorException`

**DM-016** · Obj 5.6 · Difficulty: Easy · Type: Single choice

You change Mesh `points`. What happens to `extent` automatically?

A. It recomputes  
 B. Nothing — you must `ComputeExtentFromPlugins` (or equivalent)  
 then `Set`  
 C. `kind = component` updates it  
 D. Saving USDA fills it

**DM-017** · Obj 5.6 · Difficulty: Medium · Type: Single choice

`ComputeExtentFromPlugins(boundable, time)` return value vs authored `extent`?

A. It writes the attribute for you  
 B. It **returns** a min/max pair; `GetExtentAttr().Set(...)` authors it  
 C. It returns world-space boxes only  
 D. It is removed in 26.08

**DM-018** · Obj 5.6 · Difficulty: Medium · Type: Python-reading

Triangle points `(0,0,0)`, `(1,0,0)`, `(0,1,0)`. After compute-and-set, `extent` is?

A. `[(0,0,0), (1,1,0)]` B. `[(0,0,0), (1,1,1)]` C. None forever  
 D. A 4×4 matrix

**DM-019** · Obj 5.5 · Difficulty: Medium · Type: Single choice

A CAD tessellation looks “melted” in the viewer. Likely cause (Obj 5.5)?

A. `subdivisionScheme` fallback `catmullClark`; set `"none"`  
 B. Missing `customData`  
 C. USDA instead of USDC  
 D. `metersPerUnit = 1`

**DM-020** · Obj 5.5 · Difficulty: Medium · Type: Multiple select Select two.

Which authored mistakes commonly cause **unexpected visuals**?

A. Stale `extent` after moving points (wrong bounds / culling)  
 B. `vertex` length array with `faceVarying` interpolation (UV/color seams wrong)  
 C. `ComputeAllDependencies` returning three lists  
 D. `formatId` of `.usd` being `usd`

**DM-021** · Obj 5.5 · Difficulty: Medium · Type: Single choice

A mesh is in the stage but missing in the default render. `purpose` fallback is `default`. You authored `purpose = guide`. Why?

A. `Purpose` does not affect imaging  
 B. `guide` (and often `proxy`) is omitted from the default `purpose` pass; switch `purpose` or the draw mask  
 C. `guide` deletes the `prim`  
 D. You must Flatten first

**DM-022** · Obj 5.5 · Difficulty: Easy · Type: Single choice

Imageable `visibility` fallback is:

A. `invisible`  
 B. `inherited`  
 C. `visible` as an authored default on every `gprim`  
 D. `None`

**DM-023** · Obj 5.4 · Difficulty: Easy · Type: Single choice

`prim.GetAttribute("points")` vs `prim.GetRelationship("points")` on a Mesh?

A. Both valid  
 B. `GetAttribute` is valid (`HasAttribute True`); `points` is not a relationship  
 C. `Points` are always relationships  
 D. You must use `GetMaster()`

**DM-024** · Obj 5.4 · Difficulty: Medium · Type: Single choice

`prim.GetAttribute("nope").IsValid()` for a missing name?

A. Raises `KeyError`  
 B. `False` (the handle exists but is invalid)  
 C. `True` with value `None`  
 D. Defines the attribute

**DM-025** · Obj 5.4 · Difficulty: Medium · Type: Single choice

`UsdGeom.Mesh.Define` then `GetProperties()` includes names like `accelerations` you never authored. Why?

A. A bug  
 B. **Schema** properties appear on the composed `prim` even without specs; use `HasAuthoredValue` / property stacks to see

what is authored C. Flatten ran implicitly D. `GetProperties` only lists authored specs — so they were authored

**DM-026** · Obj 5.4 · Difficulty: Medium · Type: Multiple select Select two.

Usd (composed) vs Sdf (authored) for Obj 5.4:

A. `Usd.Attribute.Get()` is the composed value  
B. `Sdf.AttributeSpec` on a layer is what that layer authored C. They are always identical D. Sdf cannot create attributes

**DM-027** · Obj 5.4 · Difficulty: Easy · Type: Single choice

`UsdGeom.Tokens.Z` vs `UsdGeom.Tokens.z` for up-axis?

A. `Tokens.Z` exists B. `Tokens.z` is "Z"; `Tokens.Z` is **missing** (AttributeError) C. Both missing; only the string "z" works D. Tokens are integers

**DM-028** · Obj 5.3 · Difficulty: Easy · Type: Single choice

Studio key `dept = "layout"` on a prim belongs in:

A. `customData` B. `faceVertexCounts` C. `metersPerUnit` D. A specialize arc

**DM-029** · Obj 5.3 · Difficulty: Medium · Type: Single choice

`GetCustomData()` after setting one key may also show `userDocBrief`. Read `dept` with:

A. `GetCustomData()["dept"]` only after asserting dict length 1 B. `GetCustomDataByKey("dept")` C. `GetAssetInfo()` D. `GetTypeName()`

**DM-030** · Obj 5.3 · Difficulty: Medium · Type: Single choice

`kind`, `documentation`, `hidden` vs `customData`?

A. All four are unregistered dictionaries B. `kind` / `documentation` / `hidden` are **registered** metadata; `customData` is the open dictionary C. `hidden` is an attribute of type bool named `hidden` D. `kind` is a relationship

**DM-031** · Obj 5.3 · Difficulty: Medium · Type: Multiple select Select two.

Layer metadata (not prim `customData`) includes:

A. `defaultPrim`, `upAxis`, `metersPerUnit`, `subLayers` B. `timeCodesPerSecond`, `startTimeCode` C. `primvars:displayColor` D. `xformOp:translate`

**DM-032** · Obj 5.2 · Difficulty: Medium · Type: Single choice

Cube `size` fallback vs Sphere `radius` fallback (no authored value)?

A. Both 1 B. Cube 2.0, Sphere 1.0 C. Both None D. Cube 1, Sphere 50000

**DM-033** · Obj 5.2 · Difficulty: Easy · Type: Single choice

UVs should use which role array?

A. `color3f[]` B. `texCoord2f[]` (`TexCoord2fArray`) C. `point3f[]` D. `matrix4d[]`

**DM-034** · Obj 5.5 · Difficulty: Hard · Type: Single choice

Hydra culls a mesh that is clearly in front of the camera after you scaled `points` 100× but left `extent` at the old 1-unit box. Diagnosis?

A. Wrong `formatId` B. Stale **extent** used as a bounds hint (Obj 5.5 / 5.6) C. Linear interpolation D. `Get()` returned None so the mesh has no points

**DM-035** · Obj 5.4 · Difficulty: Medium · Type: Single choice

`GetPrimAtPath("/NoSuch")` when that path does not exist. What is safe?

A. `IsDefined()` always returns False B. `bool(prim)` is False and `IsValid()` is False; calling `IsDefined()` on the null prim **raises** `RuntimeError` C. The call to `GetPrimAtPath` itself raises D. The prim is defined but empty

**DM-036** · Obj 5.1 · Difficulty: Medium · Type: Single choice

`elementSize = 3` on a float primvar means?

A. Always 3 vertices B. Three array entries make **one** logical element (e.g. a float3 packed as floats) C. Interpolation is uniform D. Indexed is True

**DM-037** · Obj 5.2 · Difficulty: Medium · Type: Single choice

World transform matrix type in USD schemas is typically:

A. `matrix3f` B. `matrix4d` (`Matrix4d`) C. `quatf` only D. `int[]`

**DM-038** · Obj 5.5 · Difficulty: Medium · Type: Multiple select Select two.

A gprim is `invisible` at the parent and the child never authored visibility. What do you expect, and what API family is this?

A. Child stays invisible via **inherited** visibility (`UsdGeom.Imageable`) B. Child is always visible because Mesh overrides `Imageable` C. Setting `parent visibility = inherited` (the fallback) does not hide; you need `invisible` to hide D. Visibility is a LIVERPS arc

**DM-039** · Obj 5.6 · Difficulty: Easy · Type: Single choice

`extent` is stored in which space?

A. World B. **Local** axis-aligned min/max (`float3[] length 2`) C. Camera D. UV

**DM-040** · Obj 5.2 · Difficulty: Medium · Type: Single choice

`int` vs `int64` vs `float` vs `double` — picking a type for a large frame counter that must match other DCCs exactly?

A. Always `float` (everything is float in USD) B. Prefer the width the source uses (`int64` if the DCC is 64-bit counts); do not silently downcast C. Always `token` D. Store numbers in `string` for safety

**DM-041** · Obj 5.4 · Difficulty: Easy · Type: Single choice

`Sdf.Path("/World/Chair/Seat").GetParentPath()`?

A. `/World` B. `/World/Chair` C. `/Seat` D. `.`

**DM-042** · Obj 5.4 · Difficulty: Medium · Type: Single choice

`GetProperty("points")` on a Mesh returns what class of object?

A. Only `Usd.Relationship` B. A valid `Usd.Attribute` (same as `GetAttribute("points")`) C. None D. `Sdf.Layer`

**DM-043** · Obj 5.2 · Difficulty: Medium · Type: Single choice

`subdivisionScheme` variability vs `points` variability?

A. Both varying B. Scheme is **uniform** (no time samples in the usual schema); `points` are **varying** (can be sampled) C. Scheme is varying; `points` are uniform D. Variability does not exist on Usd attributes

**DM-044** · Obj 5.1 · Difficulty: Easy · Type: Single choice

Whole-object red `displayColor` uses which interpolation and roughly how many values (`elementSize 1`)?

A. `faceVarying`, one per corner B. `constant`, one `color3f` C. `vertex`, one per point even if they match D. `uniform` with zero values

**DM-045** · Obj 5.5 · Difficulty: Medium · Type: Single choice

Parent visibility is `invisible`. Child attribute `Get()` is still inherited. `ComputeVisibility()` on the child?

A. `visible` because `Get()` is inherited B. `invisible` — compute walks the namespace; `Get()` is only the local authored/fallback token C. None D. Raises

**DM-046** · Obj 5.6 · Difficulty: Medium · Type: Single choice

`UsdGeom.BBoxCache` vs the `extent` attribute?

A. They are unrelated B. `BBoxCache` computes world/local bounds; it **uses authored extent** when present as a hint — stale extent still hurts C. `BBoxCache` overwrites extent on disk D. `BBoxCache` exists only in `usdview`

**DM-047** · Obj 5.3 · Difficulty: Easy · Type: Single choice

`hidden` metadata on a prim is meant to:

A. Delete the prim from the layer B. Hint UIs/outliners to hide it; it is **not** the same as `visibility = invisible` C. Unload payloads D. Mute the layer

**DM-048** · Obj 5.2 · Difficulty: Medium · Type: Single choice

Value **clips** vs time samples on the attribute (Obj 5.2 overview)?

A. Clips replace LIVERPS B. Clips are an extra value source for animation intervals; samples/defaults still follow strength — clips are not a LIVERPS letter C. Clips are the only way to animate in 26.08 D. `Get()` with clips always returns the first clip sample even at Default

**DM-049** · Obj 5.5 · Difficulty: Medium · Type: Multiple select Select two.

Mesh `doubleSided` fallback is `False`. Unexpected “black backfaces” can come from:

A. Single-sided mesh with inverted winding / wrong normals B. `doubleSided` still `False` and the camera seeing the back C. `ComputeAllDependencies` unresolved D. `formatId` `usd`

**DM-050** · Obj 5.4 · Difficulty: Medium · Type: Single choice

To list **only authored** attributes on a prim in the current edit target’s layer, which view?

A. `prim.GetAttributes()` — that list is schema+authored mixed B. Walk `Sdf.PrimSpec / GetAuthoredProperties / HasAuthoredValue` filters C. `GetPrototype()` D. `FindLoadable()`

**DM-051** · Obj 5.1 · Difficulty: Medium · Type: Single choice

`displayOpacity` is typically authored as:

A. `float[]` primvar (often with `displayColor`)  
 B. A relationship to a number C. Layer  
`metersPerUnit` D. token `opaque`

**DM-052** · Obj 5.2 · Difficulty: Easy · Type: Single choice

Attribute **splines** (Ts) on USD 26.08 relative to the exam's time-sample core?

A. Splines replaced time samples; `Set` at a time now writes a spline B. Time samples remain the core encoding; splines exist but this book treats them as a version note, not the primary Obj 5.2 tool C. Splines are required for linear interpolation D. `Get()` None is caused by splines always

## Answers

**DM-001 — Answer: B.** Point role matters under transforms. Review: §9.4, §9.9.

**DM-002 — Answer: B.** `asset` is the resolvable type. Review: §9.8.

**DM-003 — Answer: B.** Closed vocabularies are tokens. Review: §9.3.

**DM-004 — Answer: A, B.** Roles differ; C and D are false. (Select two true statements: A and B.) Review: §9.4.

**DM-005 — Answer: B.** Verified lab 10: `Get()` is None without a default. Review: §10.1.

**DM-006 — Answer: A.** Verified: linear midpoint 5; held previous sample 0. Review: §10.4, lab 10.

**DM-007 — Answer: B.** Verified `InterpolationTypeLinear`. Review: §10.4.

**DM-008 — Answer: B.** `primvars: +` interpolation. Review: §11.1.

**DM-009 — Answer: B.** `PrimvarsAPI` sets namespace and interpolation. Review: §11.2.

**DM-010 — Answer: B.** `uniform = per face`. Review: §11.3.

**DM-011 — Answer: B.** `Seams` → `faceVarying`. Review: §11.3.

**DM-012 — Answer: B.** Verified lab 12. Review: §11.4.

**DM-013 — Answer: B.** Indexing  $\perp$  interpolation. Review: §11.4.

**DM-014 — Answer: B.** Verified: `primvars:st` namespace `primvars`. Review: §5.5, §11.1.

**DM-015 — Answer: B.** Verified `FindPrimvarWithInheritance`. Review: §11.6.

**DM-016 — Answer: B.** Extent is authored, not automatic. Review: §13.2, lab 11.

**DM-017 — Answer: B.** Compute returns; `Set` writes. Review: §13.3.

**DM-018 — Answer: A.** Verified `[(0,0,0), (1,1,0)]`. Review: lab 11.

**DM-019 — Answer: A.** Fallback `catmullClark`. Review: §13.5, lab 11.

**DM-020 — Answer: A, B.** Stale extent and wrong interpolation. Review: §13.5, §11.3.

**DM-021 — Answer: B.** Purpose tokens filter draws. Review: §13.1, §13.5.

**DM-022 — Answer: B.** Verified fallback `inherited`. Review: §13.1.

**DM-023 — Answer: B.** `points` is an attribute. Review: §5.2–5.3.

**DM-024 — Answer: B.** Invalid handle, no raise. Review: §4.2, §5.4.

**DM-025 — Answer: B.** Schema property list  $\neq$  authored specs. Review: §6.2, §8.1.

**DM-026 — Answer: A, B.** Two views. Review: §8.1–8.2.

**DM-027 — Answer: B.** Verified: `Tokens.Z` missing; `Tokens.z` is `Z`. Review: lab 08, §28.2.

**DM-028 — Answer: A.** Obj 5.3. Review: §12.2.

**DM-029 — Answer: B.** Avoid extra schema keys. Review: lab 08.

**DM-030 — Answer: B.** Registered vs open dictionary. Review: §12.1–12.3.

**DM-031 — Answer: A, B.** Stage/layer fields. Review: §12.4, §2.4.

**DM-032 — Answer: B.** Verified Cube 2, Sphere 1. Review: §13.1, lab 31.

**DM-033 — Answer: B.** `texCoord2f[]`. Review: §9.4, §11.3.

**DM-034 — Answer: B.** Bounds hint vs true points. Review: §13.2, §13.5.

**DM-035 — Answer: B.** Verified: `bool/IsValid()` are False; `IsDefined()` raises `RuntimeError` on a null prim. Review: §4.1.

**DM-036 — Answer: B.** `elementSize` packs logical elements. Review: §11.5.

**DM-037 — Answer: B.** `matrix4d`. Review: §9.5.

**DM-038 — Answer: A, C.** A hidden parent hides children through inherited visibility. The fallback token `inherited` does not itself hide; author `invisible`. Not a composition arc. Review: §13.1.

**DM-039 — Answer: B.** Local AABB. Review: §13.2.

**DM-040 — Answer: B.** Match source width. Review: §9.2, §9.9.

**DM-041 — Answer: B.** `/World/Chair`. Review: §8.3.

**DM-042 — Answer: B.** Verified

`GetProperty("points")` is a valid Attribute.

Review: §5.1.

**DM-043 — Answer: B.** Verified uniform vs

varying. Review: §5.2, §10.1.

**DM-044 — Answer: B.** constant one color.

Review: §11.3, §11.7.

**DM-045 — Answer: B.** Verified: child `Get()` is

inherited; `ComputeVisibility()` is

invisible. Review: §13.1.

**DM-046 — Answer: B.** Extent is the authored

hint `BBoxCache` can use. Review: §13.4.

**DM-047 — Answer: B.** `hidden`  $\neq$  Imageable

visibility. Review: §12.3.

**DM-048 — Answer: B.** Clips are not a LIVERPS

letter. Review: §10.6, §21.5.

**DM-049 — Answer: A, B.** Winding/sides, not

deps or `formatId`. Review: §13.5.

**DM-050 — Answer: B.** Schema `GetAttributes` is

not “authored only.” Review: §8.1–8.2.

**DM-051 — Answer: A.** Companion primvar to

`displayColor`. Review: §11.7.

**DM-052 — Answer: B.** Samples remain core.

Review: §10.7.

---

*Data Modeling domain complete: DM-001–DM-052 (target 52).*

# Question Bank — Debugging and Troubleshooting (DBG)

**Original practice questions.** Not actual NVIDIA exam content. Mapped to NCP-OUUSD study-guide objectives 6.1–6.5 (Debugging, 11%).

Verified against **USD 26.08** (usd-core 26.8). Domain target: 44 questions. This file holds **DBG-001-DBG-044** (domain complete).

Work the stems first. Answers and explanations are grouped at the end.

---

**DBG-001** · Obj 6.1 · Difficulty: Easy · Type: Single choice

What does `Sdf.ChangeBlock` delay?

A. The writes themselves (nothing hits the layer until exit) B. **Notices** (`ObjectsChanged`); specs are still authored immediately C. Composition until `Flatten()` D. Disk `Save()`

---

**DBG-002** · Obj 6.1 · Difficulty: Medium · Type: Single choice

Three `CreateAttribute + Set` pairs without a block produced how many `ObjectsChanged` notices in lab 09?

A. 3 B. **6** (create is one notice, set is another) C. 1 D. 0

---

**DBG-003** · Obj 6.1 · Difficulty: Medium · Type: Single choice

The same lab's two `Sdf.AttributeSpec` authors **inside** `ChangeBlock` produced how many notices?

A. 6 B. 2 C. **1** D. 0 — `Sdf` cannot notify

---

**DBG-004** · Obj 6.1 · Difficulty: Medium · Type: Single choice

`UsdGeom.Sphere.Define` of a **new child** inside `Sdf.ChangeBlock` on USD 26.08?

A. Always succeeds B. Can raise `Tf.ErrorException`; author `Sdf.PrimSpec` instead C. Silently no-ops D. Converts the block into `Flatten`

---

**DBG-005** · Obj 6.1 · Difficulty: Easy · Type: Multiple select Select two.

When is a `ChangeBlock` the right performance fix (Obj 6.1)?

A. Thousands of spec edits in one burst (importers, converters) B. A single `Set` of one attribute C. Batching `Sdf` child prims so listeners see one coalesced `ObjectsChanged` D. Replacing `MuteLayer`

---

**DBG-006** · Obj 6.1 · Difficulty: Medium · Type: Single choice

`CreateAttribute` then `Set` on `/W.a` produced two notices. Typical split?

A. Both info-only B. First **resync** of `/W.a`, then **info-only** on `/W.a` C. First info, then resync of the stage root D. Notices do not distinguish resync vs info

---

**DBG-007** · Obj 6.1 · Difficulty: Easy · Type: Single choice

Unregister an `ObjectsChanged` listener with:

A. `Tf.Notice.Revoke` (that name is wrong) B. `listener.Revoke()` (the key returned by `Tf.Notice.Register`) C. `del notice` D. `stage.MuteLayer`

---

**DBG-008** · Obj 6.2 · Difficulty: Medium · Type: Single choice

A TD authored `radius = 9` but `usdview` still shows 1. First composition debug step?

A. Reinstall Python B. `GetPropertyStack / GetPrimStack` (strongest first) and check mute, edit target, LIVERPS, variant selection C. `MallocTag.GetTotalBytes()` D. Convert to OBJ

---

**DBG-009** · Obj 6.2 · Difficulty: Medium · Type: Multiple select Select two.

Opinions that **do not take effect** often sit in which traps?

A. Muted layer; weaker arc (specialize vs reference); instance **proxy** nested over B. Weaker time samples beating a **stronger** default (they do not) C. Wrong edit target; local opinion beating a variant; relocate **source** path D. `formatId` used on a `.usd` crate file

---

**DBG-010** · Obj 6.2 · Difficulty: Medium · Type: Single choice

`OverridePrim` on `/Room/A/Seat` while `/Room/A` is instanceable. Debug result?

A. Per-instance size wins B. `Tf.ErrorException`: authoring to an instance proxy is not allowed C. Silent prototype edit D. `GetMaster()` fixes it

---

**DBG-011** · Obj 6.2 · Difficulty: Medium · Type: Single choice

Variant `size=big` authors `height=3`; the prim also has local `height=1`. Composed height is 1. Debug lesson?

A. Variants beat local B. **L beats V**; the “variant did nothing” report is LIVERPS, not a broken variant set C. USD averaged 2 D. Need `OpenMasked`

**DBG-012** · Obj 6.2 · Difficulty: Easy · Type: Single choice

GetPrimStack() order?

A. Weakest first B. **Strongest first** C. Alphabetical D. Payloads only

**DBG-013** · Obj 6.2 · Difficulty: Medium · Type: Single choice

Session layer has radius=9, root has 1. Composed is 9 but Save() of the root still writes 1. Debug?

A. Composition is broken B. Session opinions are **not** in the root file; inspect session / change edit target before Save C. Mute the session to publish 9 into the root automatically D. Flatten is the only Save

**DBG-014** · Obj 6.3 · Difficulty: Medium · Type: Single choice

AddReference("no\_such\_file.usda") composition error type?

A. UnresolvedPrimPath B. InvalidAssetPath (missing file) C. None; returns False D. Tf.MallocTag

**DBG-015** · Obj 6.3 · Difficulty: Medium · Type: Single choice

File exists but has no defaultPrim and the reference omits a prim path. Error type?

A. InvalidAssetPath B. UnresolvedPrimPath C. ErrorInvalidAssetPath from Ar (no such file) D. No error; first root prim is used

**DBG-016** · Obj 6.3 · Difficulty: Medium · Type: Multiple select Select two.

Resolve("./tex/wood.png") is False from the wrong cwd, but the shot still renders the texture. Why can both be true?

A. Composition **anchors** @./tex/...@ to the **layer**, not to process cwd B. Raw Resolve of ./ uses cwd — a common false “missing texture” panic C. AnchorRelativePath is required in 26.08 D. Search paths apply to ./ paths

**DBG-017** · Obj 6.3 · Difficulty: Medium · Type: Single choice

Open("shot.usda") without context: @chair.usda@ (no ./) → ErrorInvalidAssetPath. Fix?

A. Flatten B. Stage.Open(..., pathResolverContext=) with a search path / DefaultResolverContext C. LoadNone D. MuteLayer on the shot

**DBG-018** · Obj 6.3 · Difficulty: Easy · Type: Single choice

ComputeAllDependencies third tuple slot for a missing @./gone.jpg@?

A. layers B. assets (resolved) C. **unresolved** D. payloads only

**DBG-019** · Obj 6.4 · Difficulty: Medium · Type: Single choice

CAD mesh looks subdivided/smooth though points are a tessellation. Debug subdivisionScheme?

A. Authored none already B. Fallback **catmullClark** if unauthored — set none C. Fallback loop D. Scheme is ignored by Hydra

**DBG-020** · Obj 6.4 · Difficulty: Medium · Type: Single choice

Points were scaled 100x; viewer still culls the mesh. Check?

A. MallocTag B. Stale **extent** C. formatId D. GetMaster

**DBG-021** · Obj 6.4 · Difficulty: Medium · Type: Multiple select Select two.

Mesh exists in Traverse but not in the default camera. Likely?

A. purpose = guide or proxy vs default purpose mask B. visibility computed invisible (parent hide) C. ExtractExternalReferences empty second list D. USDA encoding

**DBG-022** · Obj 6.4 · Difficulty: Medium · Type: Single choice

Z-up meters chair in a Y-up cm shot looks on its side and tiny. USD converted units automatically?

A. Yes B. **No** — debug metrics on the **asset file**, then rotateX/scale on the reference C. Yes if referenced D. upAxis on the composed stage rotates children

**DBG-023** · Obj 6.4 · Difficulty: Easy · Type: Single choice

UVs look flipped in a glTF viewer after export. First mapping check?

A. v\_glTF = 1 - v\_usd B. u\_glTF = metersPerUnit C. Drop UVs D. subdivisionScheme

**DBG-024** · Obj 6.5 · Difficulty: Easy · Type: Single choice

Python class that captured the missing-reference **warning** in lab 36?

A. Tf.CoalescingDiagnosticDelegate B. UsdUtils.CoalescingDiagnosticDelegate C. Trace.Collector D. Usd.Notice.ObjectsChanged

**DBG-025** · Obj 6.5 · Difficulty: Medium · Type: Single choice



When must you construct that delegate to see Stage.Open warnings?

A. After Open B. **Before** Stage.Open C. After Flatten D. Only if usdchecker is installed

**DBG-026** · Obj 6.5 · Difficulty: Medium · Type: Single choice

Lab 36 sourceFileName printed stage.cpp. How should you assert in portable tests?

A. Full absolute path equality B. `os.path.basename(...)` — the field may be a full path C. Ignore source entirely; it is always empty D. It is always `usdview.cpp`

**DBG-027** · Obj 6.5 · Difficulty: Easy · Type: Single choice

Trace vs MallocTag?

A. Trace = bytes; MallocTag = seconds B. Trace = **time**; MallocTag = **bytes** C. Both measure FPS D. Both require Hydra

**DBG-028** · Obj 6.5 · Difficulty: Medium · Type: Python-reading

On usd-core 26.8 lab 36:

`MallocTag.Initialize()` returned False, then `IsInitialized()` True, `GetTotalBytes()`?

A. A large positive count B. **0** C. Raises D. None

**DBG-029** · Obj 6.5 · Difficulty: Easy · Type: Single choice

`Trace.Collector()` default enabled?

A. True B. **False** C. None D. Follows `TF_DEBUG`

**DBG-030** · Obj 6.5 · Difficulty: Medium · Type: Single choice

Enable extra change-processing logs after `import pxr`?

A. `TF_DEBUG` only works as a Python function of that name B.

`Tf.Debug.SetDebugSymbolsByName("USD_CHANGES", True)` C. `stage.SetDebug(True)` D. `Usd.GetVersion()`

**DBG-031** · Obj 6.5 · Difficulty: Easy · Type: Single choice

Process-start equivalent of that switch?

A. `USD_LOG=1` B. `TF_DEBUG=USD_CHANGES` C. `PCP=1` D. `HYDRA_DEBUG=1` on usd-core always

**DBG-032** · Obj 6.5 · Difficulty: Medium · Type: Single choice

`SetDebugSymbolsByName("NOT_A_SYMBOL", True)` on 26.08?

A. Raises B. Returns **empty list**; unknown names are ignored C. Creates the symbol D. Crashes

**DBG-033** · Obj 6.5 · Difficulty: Medium · Type: Multiple select Select two.

Missing `@./missing.usda@` reference. Lab 36 saw:

A. Coalescing delegate item with `TF_DIAGNOSTIC_WARNING_TYPE` B. `len(GetCompositionErrors()) == 1` C. `ComplianceChecker` error D. `MallocTag` bytes > 0 as proof of the miss

**DBG-034** · Obj 6.2 · Difficulty: Hard · Type: Single choice

Stronger sublayer default `radius=5`; weaker sublayer has samples. `Get(10)` is 5 and `GetTimeSamples()` is []. Debug?

A. Samples always win B. Stronger **default wins at all times**; weaker samples do not leak C. Need Held interpolation D. Bug in usd-core

**DBG-035** · Obj 6.3 · Difficulty: Medium · Type: Single choice

`ExtractExternalReferences(shot)` lists no textures; `Compute` lists `wood.png`. Debug conclusion?

A. `Compute` is wrong B. `Extract` is **one file**; textures live on a **sublayer** C. PNG cannot be a dependency D. Need `usdcat`

**DBG-036** · Obj 6.1 · Difficulty: Medium · Type: Single choice

Three `Sdf.PrimSpec` children **inside** one `ChangeBlock` vs three **outside**. Notice counts?

A. 3 and 3 B. **1** (all three paths) vs **3** C. 0 and 3 D. 3 and 1

**DBG-037** · Obj 6.4 · Difficulty: Medium · Type: Single choice

`displayColor` interpolation vertex with a faceVarying-length array. Visual symptom?

A. Perfect colors B. Wrong mapping (seams/colors); length must match interpolation C. Stage fails to open D. Extent becomes None

**DBG-038** · Obj 6.5 · Difficulty: Easy · Type: Single choice

`GetDebugSymbolDescription("USD_CHANGES")` (verified book text)?

A. "USD change processing" B. "malloc" C. "" D. Raises

**DBG-039** · Obj 6.2 · Difficulty: Medium · Type: Single choice

Relocate `{</Bot/Rig/Arm>: </Bot/Arm>}`. TD overs `/Bot/Rig/Arm`. Radius unchanged. Debug?

A. Relocates ignore all others B. Author at the **destination**; source-path opinions do not apply  
C. Relocates only work on payloads D. Need ChangeBlock

**DBG-040** · Obj 6.3 · Difficulty: Medium · Type: Single choice

Custom resolver missing in the DCC; identifiers fail. Checklist?

A. `PXR_PLUGINPATH_NAME` + plugin built for **this** USD version, set **before** init B. `AnchorRelativePath` C. `GetMaster()` D. USDA only

**DBG-041** · Obj 6.4 · Difficulty: Medium · Type: Single choice

`OpenMasked` vs `LoadNone` when debugging a “missing” `/World/B` that is a **reference**?

A. They are identical B. `LoadNone` still **loads references**; a population **mask** omits B entirely  
C. `LoadNone` omits references; mask loads them  
D. `MuteLayer` is the only tool

**DBG-042** · Obj 6.5 · Difficulty: Medium · Type: Single choice

`Initialize()` returning `False` while `IsInitialized()` becomes `True` means?

## Answers

**DBG-001 — Answer: B.** Notices delayed; writes happen. Review: §8.5, §34.5, lab 09.

**DBG-002 — Answer: B.** Verified 6. Review: lab 09.

**DBG-003 — Answer: C.** Verified 1. Review: lab 09.

**DBG-004 — Answer: B.** Verified `Tf.ErrorException`. Review: lab 30, §34.5.

**DBG-005 — Answer: A, C.** Burst `Sdf` authoring and coalesced notices. Review: §6.1, §44.4.

**DBG-006 — Answer: B.** Verified lab 30. Review: §34.4.

**DBG-007 — Answer: B.** `listener.Revoke()`. Review: lab 09/30.

**DBG-008 — Answer: B.** Stacks first. Review: §22.6, §42.

**DBG-009 — Answer: A, C.** B is the reverse of COMP-042; D is not a strength bug. Review: §22.6.

**DBG-010 — Answer: B.** Instance proxy. Review: §24.3, lab 23.

**DBG-011 — Answer: B.** L beats V. Review: §18.6, §21.2.

**DBG-012 — Answer: B.** Strongest first. Review: §14.5, lab 21.

A. `MallocTag` is broken; abort B. On this **usd-core wheel**, tagging may not count bytes (`GetTotalBytes()` 0) even though the API is “on” C. Trace is now on D. You must call `Initialize` twice

**DBG-043** · Obj 6.1 · Difficulty: Medium · Type: Multiple select Select two.

Safe inside `Sdf.ChangeBlock` on 26.08?

A. `Sdf.PrimSpec` / `Sdf.AttributeSpec` on an existing parent spec B. `UsdGeom.Sphere.Define` of a brand-new child path C. `Attribute` create+set on an **already defined** prim via `Sdf` (lab 09) D. `stage.DefinePrim` of new nested paths (same trap as `Sphere.Define`)

**DBG-044** · Obj 6.2 · Difficulty: Easy · Type: Single choice

Eight-step “why doesn’t my opinion win?” (Ch 22/42) should include mute, stack, LIVERPS, variant, instance, relocate, edit target, and:

A. `usdview` Layer Stack / property stack (or Python equivalents) B. Reinstalling the OS C. Converting to glTF first D. `GetPrototype` on non-instanceable prims only

**DBG-013 — Answer: B.** Session vs root Save. Review: §15.3, lab 04.

**DBG-014 — Answer: B.** Missing file. Review: §16.5, §42.

**DBG-015 — Answer: B.** Missing defaultPrim. Review: §16.2, lab 14.

**DBG-016 — Answer: A, B.** Layer anchor vs cwd. Review: §33.2, lab 29.

**DBG-017 — Answer: B.** `pathResolverContext`. Review: lab 29.

**DBG-018 — Answer: C.** Third slot unresolved. Review: §32.4, lab 28.

**DBG-019 — Answer: B.** `catmullClark` fallback. Review: lab 11, §13.5.

**DBG-020 — Answer: B.** Stale extent. Review: §13.2, Obj 5.6/6.4.

**DBG-021 — Answer: A, B.** Purpose and visibility. Review: §13.5, §42.

**DBG-022 — Answer: B.** No auto unit convert. Review: lab 26.

**DBG-023 — Answer: A.** UV V-flip. Review: §27.4.

**DBG-024 — Answer: B.** `UsdUtils`, not `Tf`. Review: lab 36, §43.

**DBG-025 — Answer: B.** Construct first. Review: lab 36.

**DBG-026 — Answer: B.** Basename  
sourceFileName. Review: lab 36.

**DBG-027 — Answer: B.** Time vs bytes. Review:  
§43.

**DBG-028 — Answer: B.** Verified 0 bytes.  
Review: lab 36.

**DBG-029 — Answer: B.** Trace default False.  
Review: lab 36.

**DBG-030 — Answer: B.**  
SetDebugSymbolsByName. Review: §43.2.

**DBG-031 — Answer: B.** TF\_DEBUG env.  
Review: §43.2.

**DBG-032 — Answer: B.** Unknown → empty list.  
Review: §43.2.

**DBG-033 — Answer: A, B.** Warning + Pcp  
error. Review: lab 36.

**DBG-034 — Answer: B.** Stronger default hides  
samples. Review: §21.3.

**DBG-035 — Answer: B.** Extract vs Compute.  
Review: §32.4.

**DBG-036 — Answer: B.** 1 vs 3. Review: lab 30.

**DBG-037 — Answer: B.** Interpolation length.  
Review: §11.3.

**DBG-038 — Answer: A.** Verified description.  
Review: §43.2.

**DBG-039 — Answer: B.** Relocate destination.  
Review: §20.3.

**DBG-040 — Answer: A.** Plugin path + ABI.  
Review: §33.5, §35.2.

**DBG-041 — Answer: B.** Mask vs LoadNone.  
Review: §17.4, lab 16.

**DBG-042 — Answer: B.** usd-core MallocTag 0  
bytes. Review: lab 36.

**DBG-043 — Answer: A, C.** Sdf specs OK; Usd  
Define of new children is the trap (B, D).  
Review: labs 09-30.

**DBG-044 — Answer: A.** Introspection UI/API.  
Review: §22.6, §42.

---

*Debugging domain complete: DBG-001-  
DBG-044 (target 44).*

## Question Bank — Content Aggregation (CA)

**Original practice questions.** Not actual NVIDIA exam content. Mapped to NCP-OUUSD study-guide objectives 2.1–2.5 (Content Aggregation, 10%).

Verified against **USD 26.08** (usd-core 26.8). Domain target: 40 questions. This file holds **CA-001-CA-040** (domain complete).

Work the stems first. Answers and explanations are grouped at the end.

---

**CA-001** · Obj 2.4 · Difficulty: Easy · Type: Single choice

Where do you author `instanceable = true` so many chairs share a prototype?

A. Only on every `Seat` mesh inside the published asset B. On each **referencing** prim (instance root) in the assembly C. Stage metadata next to `upAxis` D. On the session layer only

---

**CA-002** · Obj 2.4 · Difficulty: Easy · Type: Single choice

On USD 26.08, the API for the shared native-instance prototype is:

A. `prim.GetMaster()` B. `prim.GetPrototype()` (`GetMaster` is **absent**) C. `UsdGeom.PointInstancer.GetPrototype` D. `Kind.Registry.GetPrototype`

---

**CA-003** · Obj 2.4 · Difficulty: Medium · Type: Single choice

Two instanceable prims reference the same chair. `a.GetPrototype() == b.GetPrototype()` ?

A. False — one prototype per path B. True if their composition (refs/variants/inherits) matches C. True only after Flatten D. Prototypes exist only for `PointInstancer`

---

**CA-004** · Obj 2.4 · Difficulty: Medium · Type: Single choice

Default `Traverse()` on a stage with instanceable `/Room/A` that has `/Room/A/Seat` ?

A. Lists `/Room/A/Seat` B. Lists `/Room/A` (and `/Room/B`) but **skips** instance-proxy children C. Skips the instance roots too D. Raises

---

**CA-005** · Obj 2.4 · Difficulty: Medium · Type: Multiple select Select two.

`IsInstanceable()` vs `IsInstance()` ?

A. `IsInstanceable` is the authored flag B. `IsInstance` means USD actually built a prototype for that prim C. They are always equal D. `IsInstance` is the old name for `GetMaster`

---

**CA-006** · Obj 2.2 · Difficulty: Medium · Type: Single choice

Edit **one** instance's nested `Seat` color **without** de-instancing (Obj 2.2). Legal place?

A. `OverridePrim("/Room/A/Seat")` and set `displayColor` B. A **primvar / inherit / variant** on the **instance root** `/Room/A` (shader must read it) C. `GetMaster().Clear()` D. Mute the assembly

---

**CA-007** · Obj 2.2 · Difficulty: Medium · Type: Single choice

`OverridePrim("/Room/A/Seat")` while A is instanceable. Result on 26.08?

A. Unique color on A only B. `Tf.ErrorException` — authoring to an instance proxy is not allowed C. Silent write to all instances D. Converts A to a `PointInstancer`

---

**CA-008** · Obj 2.2 · Difficulty: Easy · Type: Single choice

The instance **root** `/Room/A` is a proxy?

A. Yes — never author there B. **No** — the root is the instance; **descendants** are proxies; root primvars/variants/inherits are legal C. Only if `kind = component` D. Only in USDA

---

**CA-009** · Obj 2.2 · Difficulty: Medium · Type: Multiple select Select two.

To tint **all** instanced chairs the same without breaking instancing:

A. Author `displayColor` (or material) on the **source** `/Chair` B. Nested over on every `/Room/*/Seat` proxy C. A class they all **inherit** from the instance roots D. `SetInstanceable(False)` on every copy

---

**CA-010** · Obj 2.5 · Difficulty: Medium · Type: Single choice

Remove or override a property on **one** instanced component's nested gprim in an assembly (Obj 2.5)?

A. Block the proxy path while instanceable B. `SetInstanceable(False)` on that instance root, then author the nested override; siblings keep the prototype C. `GetMaster()` and `RemoveProperty` D. Delete `/__Prototype_1`

---

**CA-011** · Obj 2.5 · Difficulty: Medium · Type: Single choice

After de-instancing B and setting `Seat` size 9, lab 23: A's size?

A. Also 9 B. Stays at the prototype/source value (3 in the lab); B is unique C. None D. Both become non-instanceable automatically

**CA-012** · Obj 2.5 · Difficulty: Easy · Type: Single choice

De-instance **every** chair “so we can edit.”  
Pipeline cost?

A. None — instancing is free either way B. You throw away the **memory** win; prefer root primvars/variants unless the copy is a true hero C. Hydra requires it D. PointInstancer appears automatically

**CA-013** · Obj 2.1 · Difficulty: Easy · Type: Single choice

Required PointInstancer properties to instance anything?

A. Only `positions` B. `prototypes` relationship, `protoIndices`, and `positions` C. `kind = assembly` D. `instanceable = true` on each point

**CA-014** · Obj 2.1 · Difficulty: Medium · Type: Single choice

`GetInstanceCount()` equals?

A. `len(positions)` even if `protoIndices` is shorter B. `len(protoIndices)` C. Number of prototype targets D. Traverse prim count

**CA-015** · Obj 2.1 · Difficulty: Medium · Type: Single choice

Adding a third prototype from a **stronger** layer with `rel.AddTarget(path)` and **no** position argument typically authors:

A. `append` (safe, last index) B. **`prepend`** (`ListPositionBackOfPrependList`) — new proto at index **0**, shifting every `protoIndices` meaning C. A new PointInstancer D. Nothing

**CA-016** · Obj 2.1 · Difficulty: Medium · Type: Single choice

Safe append of `/Forest/Protos/Bush` so `Pine=0` and `Oak=1` stay put?

A. Default `AddTarget` B. `AddTarget(path, Usd.ListPositionBackOfAppendList)` C. `SetInstanceable(True)` on Bush D. `InvisId(2)`

**CA-017** · Obj 2.1 · Difficulty: Medium · Type: Python-reading

Lab 24: after append with `BackOfAppendList`, targets names?

A. Bush, Pine, Oak B. Pine, Oak, Bush C. Oak, Pine, Bush D. Empty

**CA-018** · Obj 2.1 · Difficulty: Easy · Type: Single choice

Why put prototypes under a `class` child (`SpecifierClass`)?

A. Class prims cannot hold geometry B.

`Traverse()` **skips** class descendants, so the forest walk is the instancer prim not every pine mesh C. Required by LIVERPS D. PointInstancer forbids `def` prototypes

**CA-019** · Obj 2.3 · Difficulty: Easy · Type: Single choice

Hide **one** PointInstancer instance efficiently (Obj 2.3)?

A. `MakeInvisible()` on the PointInstancer prim (hides **all**) B. `InvisId(id, time)` / `invisibleIds` C. Deactivate the whole `/Forest` prim D. `SetInstanceable(False)`

**CA-020** · Obj 2.3 · Difficulty: Medium · Type: Single choice

`InvisId(1, Default)` then `GetInstanceCount()` ?

A. 2 B. **Still 3** — hide is a mask, not a delete C. 0 D. Raises

**CA-021** · Obj 2.3 · Difficulty: Medium · Type: Single choice

After hiding id 1, `ComputeMaskAtTime` in the lab?

A. `[False, False, False]` B. `[True, False, True]` C. `[1]` D. None

**CA-022** · Obj 2.3 · Difficulty: Medium · Type: Multiple select Select two.

`ComputeInstanceTransformsAtTime` after hiding the middle instance (`positions 0, 2, 4`)?

A. Translations at 0 and 4 remain; the hidden one is omitted from that compute B. Count stays 3 C. Positions array is rewritten to two samples D. `MakeInvisible` was called on `/Forest`

**CA-023** · Obj 2.3 · Difficulty: Medium · Type: Single choice

`InvisId` signature that failed in early drafts?

A. `InvisId(id, time)` — **time is required** B. `InvisId(id)` alone is the documented 26.08 form C. `InvisId(PointInstancer, int)` D. `prim.MakeInvisible(id)`

**CA-024** · Obj 2.4 · Difficulty: Medium · Type: Single choice

PointInstancer `IsA(Gprim)` vs `IsA(Boundable)` ?

A. Both True B. **Not** a Gprim; **is** Boundable C. Is a Gprim; not Boundable D. Neither

**CA-025** · Obj 2.4 · Difficulty: Medium · Type: Single choice

80,000 identical stadium seats vs one unique broadcast booth. Pairing?

A. Native instance or PointInstancer seats; ordinary reference for the booth B. Native-instance the booth with the seats so they share one prototype C. 80,000 full `def` copies D. One payload for the whole stadium is enough instancing

**CA-026** · Obj 2.4 · Difficulty: Easy · Type: Single choice

Native instances vs PointInstancer grain?

A. Native: one prim per copy (proxies, names, variants); PI: arrays on **one** prim for huge crowds B. PI always uses `GetPrototype` C. Native cannot share geometry D. PI requires `instanceable=true` on each point

**CA-027** · Obj 2.4 · Difficulty: Medium · Type: Multiple select Select two.

Matching native instances **split** into two prototypes when:

A. They pick different variant selections B. They inherit different classes / extra local arcs C. They have different prim **names** only (`Chair_0` vs `Chair_1`) D. They sit under the same assembly

**CA-028** · Obj 2.4 · Difficulty: Medium · Type: Single choice

`/Kitchen/Clutter/Mug` has `kind = component` but `IsComponent()` is False. Why?

A. Kind `component` is illegal B. Model hierarchy **broke**: un-kind ancestor; `GetKind` can still say component C. Mug is instanceable D. Need Flatten

**CA-029** · Obj 2.4 · Difficulty: Medium · Type: Single choice

`Usd.PrimIsModel` traversal lists Kitchen, Props, `Chair_0` but not Mug. Reason?

A. Bug B. Non-models are **pruned**; Clutter is not a model so children are not visited C. Mug is USDC D. `Chair_0` is a payload so it cannot be a model

**CA-030** · Obj 2.4 · Difficulty: Easy · Type: Single choice

`Kind.Registry.IsA("assembly", "group")`?

A. False B. **True** — assembly is-a group C. Raises D. Only after `Register`

**CA-031** · Obj 2.1 · Difficulty: Medium · Type: Single choice

After adding Bush you must also:

A. Nothing — `protoIndices` auto-extend B. Author **protoIndices** (and usually **positions**) entries that **index** the new prototype C. Set

`instanceable` on Forest D.

`ComputeExtentFromPlugins` on each tree prim only

**CA-032** · Obj 2.5 · Difficulty: Medium · Type: Single choice

Source `/Chair/Seat` size `Set(3)` while A and B are instances. Both seats read 3. That is:

A. A bug B. **Source edits broadcast** through the prototype — intended C. Only A updates D. Forbidden; you must de-instance first

**CA-033** · Obj 2.2 · Difficulty: Hard · Type: Single choice

Exporter picks `/Room/A/Seat` (proxy) for a unique `displayColor`. Correct rewrite?

A. Write color on the proxy anyway B. Write a primvar on **instance root** `/Room/A` (or de-instance if a nested unique mesh is required) C. Write on `/__Prototype_1/Seat` as the per-instance API D. Convert to PointInstancer `displayColors` always

**CA-034** · Obj 2.3 · Difficulty: Easy · Type: Single choice

`inactiveIds` vs `invisibleIds` (conceptually)?

A. Identical B. Invisible hides for imaging; inactive is a stronger omit (no load/eval of that id) — both are per-id, not `MakeInvisible` on the prim C. `inactiveIds` is LIVERPS D. Only `invisibleIds` exists in OpenUSD

**CA-035** · Obj 2.4 · Difficulty: Medium · Type: Single choice

`stage.GetPrototypes()` in lab 23 listed:

A. `/Chair` only B. `/__Prototype_1` (implicit prototype prims, not the source path) C. `/Room/A/Seat` D. Empty until Flatten

**CA-036** · Obj 2.1 · Difficulty: Medium · Type: Multiple select Select two.

Same-layer explicit `SetTargets([Pine, Oak])` then `AddTarget(Bush)` without position — lab used `BackOfAppendList` because:

A. Stronger-layer default prepend is the exam trap B. Append keeps indices 0,1 stable C. Prepend is always safer D. PointInstancer ignores list-ops

**CA-037** · Obj 2.4 · Difficulty: Medium · Type: Single choice

Different variant on B than A. Shared prototype?

A. Always shared B. Often a **second** prototype — composition differs C. Instancing breaks entirely D. B cannot be instanceable

**CA-038** · Obj 2.5 · Difficulty: Medium · Type: Single choice

Blocking a property on the **instance root** (not a proxy child) while staying instanceable?

A. Illegal always B. Legal for root-level opinions; nested proxy `Block()` is the illegal one C. Only via `GetMaster` D. Only via `PointInstancer` mask

**CA-039** · Obj 2.4 · Difficulty: Easy · Type: Single choice

`GetPrimAtPath("/Room/A/Seat")` on an instance proxy?

A. Invalid always B. Valid handle; `IsInstanceProxy()` True; readable; not in default `Traverse` C. Raises D. Equals `GetPrototype()`

**CA-040** · Obj 2.3 · Difficulty: Medium · Type: Single choice

Hide 50,000 of 2,000,000 PI instances every frame. Prefer?

A. 50,000 native `visibility=invisible` prims  
B. `invisibleIds` / `InvisId` on the instancer (array update) C. Delete `protoIndices` entries each frame D. `MuteLayer` on the shot

## Answers

**CA-001 — Answer: B.** Instanceable at the use site. Review: §24.1, lab 23.

**CA-002 — Answer: B.** Verified `GetMaster` missing. Review: §24.2.

**CA-003 — Answer: B.** Shared proto when composition matches. Review: lab 23.

**CA-004 — Answer: B.** `Traverse` skips proxies. Review: lab 23.

**CA-005 — Answer: A, B.** Flag vs built instance. Review: §24.1.

**CA-006 — Answer: B.** Obj 2.2 legal edit sites. Review: §24.4–24.5.

**CA-007 — Answer: B.** Proxy authoring raises. Review: lab 23, COMP-027.

**CA-008 — Answer: B.** Root vs proxy. Review: §24.3.

**CA-009 — Answer: A, C.** Source or inherited class. Review: §24.4.

**CA-010 — Answer: B.** De-instance one hero. Review: §24.6, Obj 2.5.

**CA-011 — Answer: B.** Verified A stays 3, B 9. Review: lab 23.

**CA-012 — Answer: B.** Don't de-instance the crowd. Review: §24.5.

**CA-013 — Answer: B.** Three required fields. Review: §25.2.

**CA-014 — Answer: B.** Count = `protoIndices` length. Review: lab 24.

**CA-015 — Answer: B.** Default `AddTarget` prepends in a stronger layer. Review: §25.3.

**CA-016 — Answer: B.** `BackOfAppendList`. Review: lab 24.

**CA-017 — Answer: B.** Pine, Oak, Bush. Review: lab 24.

**CA-018 — Answer: B.** Class skipped by `Traverse`. Review: lab 24.

**CA-019 — Answer: B.** `InvisId` / `invisibleIds`. Review: §25.4.

**CA-020 — Answer: B.** Count still 3. Review: lab 24.

**CA-021 — Answer: B.** Mask `[True, False, True]`. Review: lab 24.

**CA-022 — Answer: A, B.** Transforms omit hidden; count unchanged. Review: lab 24.

**CA-023 — Answer: A.** Time argument required. Review: lab 24 notes.

**CA-024 — Answer: B.** Boundable, not `Gprim`. Review: lab 24.

**CA-025 — Answer: A.** Style by scale. Review: §26.1.

**CA-026 — Answer: A.** Prim-per-copy vs arrays. Review: §25.1, §26.1.

**CA-027 — Answer: A, B.** Variant/inherit split prototypes. Names alone don't. Review: §24.2, §24.5.

**CA-028 — Answer: B.** Broken model chain. Review: lab 22.

**CA-029 — Answer: B.** `PrimIsModel` prunes. Review: lab 22.

**CA-030 — Answer: B.** assembly is-a group. Review: lab 22. `Kind.Registry.Register` does not exist in Python 26.08.

**CA-031 — Answer: B.** Indices must reference the new proto. Review: §25.3.

**CA-032 — Answer: B.** Source broadcasts. Review: lab 23.

**CA-033 — Answer: B.** Don't bake unique color on a proxy. Review: §24.4.

**CA-034 — Answer: B.** Two per-id hides. Review: §25.4.

**CA-035 — Answer: B.** `/__Prototype_1`. Review: lab 23.

**CA-036 — Answer: A, B.** Exam trap is prepend. Review: §25.3.

**CA-037 — Answer: B.** Different composition → another prototype. Review: §24.5.

**CA-038 — Answer: B.** Root Block vs proxy Block. Review: §24.6, Obj 2.5.

**CA-039 — Answer: B.** Readable proxy handle. Review: lab 23.

**CA-040 — Answer: B.** Array hide at PI scale. Review: §25.4, §26.1.

---

*Content Aggregation domain complete: CA-001–CA-040 (target 40).*



## Question Bank — Visualization (VIS)

**Original practice questions.** Not actual NVIDIA exam content. Mapped to NCP-OUSD study-guide objectives 8.1–8.4 (Visualization, 8%).

Verified against **USD 26.08** (usd-core 26.8).

Domain target: 32 questions. This file holds

**VIS-001-VIS-032** (domain complete).

Work the stems first. Answers and explanations are grouped at the end.

**VIS-001** · Obj 8.1 · Difficulty: Easy · Type: Single choice

How should you add `displayColor` to a mesh for Hydra/usdview (Obj 8.1)?

A. A relationship named `color` B. `UsdGeom.PrimvarsAPI.CreatePrimvar("displayColor", Color3fArray, interpolation)` C. Layer metadata `displayColor` D. `kind = color`

**VIS-002** · Obj 8.1 · Difficulty: Easy · Type: Single choice

The primvar attribute name on the prim is:

A. `displayColor` B. `primvars:displayColor` C. `inputs:displayColor` D. `info:displayColor`

**VIS-003** · Obj 8.1 · Difficulty: Medium · Type: Single choice

Per-vertex colors on a mesh of N points: interpolation and array length?

A. `constant`, length 1 B. `vertex`, length N (or N indices if indexed) C. `uniform`, length N D. `faceVarying` is required for vertex colors

**VIS-004** · Obj 8.1 · Difficulty: Medium · Type: Multiple select Select two.

Lab 12 indexed `displayColor`:

A. Interpolation `vertex` B. Indices `[0, 1, 0, 1]` and two palette colors C. Indexing forces `constant` D.

`CreateAttribute("displayColor")` without `PrimvarsAPI` is the recommended path

**VIS-005** · Obj 8.1 · Difficulty: Medium · Type: Single choice

UVs for texturing a `PreviewSurface` usually live as:

A. `primvars:st (texCoord2f[])`, interpolation `vertex` or `faceVarying` B. Shader `info:id` C. `points` D. `customData['uv']`

**VIS-006** · Obj 8.2 · Difficulty: Easy · Type: Single choice

`UsdPreviewSurface` is:

A. A typed prim `def PreviewSurface` B. An **info:id** string on a **UsdShade.Shader**, not a prim type C. A Hydra render delegate D. A variant set

**VIS-007** · Obj 8.2 · Difficulty: Medium · Type: Single choice

A drawable `PreviewSurface` material needs which connection?

A. Mesh `points` connected to the shader B. Material **outputs:surface** connected to the shader's `outputs:surface` C. Only `Bind` — no connections D. `outputs:displacement` is required in usdview

**VIS-008** · Obj 8.2 · Difficulty: Easy · Type: Single choice

`ps.CreateIdAttr("UsdPreviewSurface")` writes which attribute?

A. `id` B. `info:id` (value `UsdPreviewSurface`) C. `token purpose` D. `asset file`

**VIS-009** · Obj 8.2 · Difficulty: Medium · Type: Single choice

`mat.ComputeSurfaceSource()` on 26.08 returns:

A. A single `UsdShade.Shader` B. A **tuple of length 3** (`shader`, `outputName`, `sourceType`) C. `None` until `Flatten` D. The bound mesh

**VIS-010** · Obj 8.2 · Difficulty: Medium · Type: Multiple select Select two.

`UsdShade` building blocks for Obj 8.2:

A. `Material` (terminals) B. `Shader` (`nodedef / info:id`) C. `Cube` as the material type D. `PointInstancer` as a required shader

**VIS-011** · Obj 8.3 · Difficulty: Easy · Type: Single choice

Correct bind sequence on USD 26.08?

A. `Bind` only B. **`MaterialBindingAPI.Apply(prim)` then `Bind(material)`** C. `HasAPI` then skip `Bind` D. `SetKind("material")` on the mesh

**VIS-012** · Obj 8.3 · Difficulty: Medium · Type: Single choice

`Bind` **without** `Apply`. What is true?

A. No relationship is written B. `rel material:binding` is written; **`HasAPI` stays `False`** C. `HasAPI` `True` D. `Bind` raises

**VIS-013** · Obj 8.3 · Difficulty: Medium · Type: Python-reading

After `Apply+Bind`, `ComputeBoundMaterial()` `[0].GetPath()` in lab 34?

A. /World/Board B. /Looks/Paint C. /Looks/Paint/Preview D. /\_\_Prototype\_1

**VIS-014** · Obj 8.3 · Difficulty: Medium · Type: Single choice

Binding **purpose** preview vs full?

A. Illegal in OpenUSD B. Separate bindings so a DCC/Hydra preview pass can differ from the full render C. Purpose on Imageable, not on materials D. preview deletes full

**VIS-015** · Obj 8.3 · Difficulty: Medium · Type: Single choice

`weakerThanDescendants` vs `strongerThanDescendants` on a bind?

A. Cosmetic names B. Strength: whether this bind yields to (or overrides) **descendant** binds (collections/subsets) C. LIVERPS letters D. Only legal on lights

**VIS-016** · Obj 8.3 · Difficulty: Medium · Type: Multiple select Select two.

Ways to bind materials besides a direct rel on the gprim:

A. Collection-based `MaterialBindingAPI` collections B. `UsdGeom.Subset` faces with their own bind C. `subLayers` named material D. `kind = material` on the mesh

**VIS-017** · Obj 8.4 · Difficulty: Medium · Type: Single choice

Does `UsdPreviewSurface` **automatically** read mesh `primvars:displayColor` into `diffuseColor`?

A. Yes always B. **No** — you connect a `UsdPrimvarReader_float3` (or float) whose varname is `displayColor` to `inputs:diffuseColor` (Obj 8.4) C. Yes if interpolation is constant D. Yes after Apply

**VIS-018** · Obj 8.4 · Difficulty: Medium · Type: Single choice

`UsdPrimvarReader_float3` varname input type and value for `displayColor`?

A. string `"primvars:displayColor"` B. **token `displayColor`** (the primvar name without requiring the `primvars:` prefix in varname) C. `asset` D. `color3f`

**VIS-019** · Obj 8.4 · Difficulty: Medium · Type: Single choice

Texture lookup for `PreviewSurface` typically chains:

A. `UsdPrimvarReader_float2 (st)` → `UsdUVTexture` → `PreviewSurface diffuseColor`  
B. Mesh points → `UVTexture` C. `UsdLux.DistantLight` → `diffuseColor` D. `ComputeExtentFromPlugins`

**VIS-020** · Obj 8.4 · Difficulty: Easy · Type: Single choice

Reader node `info:id` for UVs?

A. `UsdPreviewSurface` B. `UsdPrimvarReader_float2` C. `UsdUVTexture` D. `UsdPrimvarReader_float3`

**VIS-021** · Obj 8.4 · Difficulty: Medium · Type: Multiple select Select two.

Verified Obj 8.4 network facts:

A. `diffuseColor` `HasConnectedSource()` True after connecting the reader output B. `PreviewSurface info:id` stays `UsdPreviewSurface` C. The reader must be type `Mesh` D. varname must be the full `primvars:displayColor` path as an `asset`

**VIS-022** · Obj 8.2 · Difficulty: Medium · Type: Single choice

`UsdShade.NodeGraph` is for:

A. Replacing Material B. Nesting a reusable subgraph of shaders inside/beside a Material C. Point instancing D. Layer offsets

**VIS-023** · Obj 8.1 · Difficulty: Medium · Type: Single choice

Wrong interpolation on `st` (vertex-length with `faceVarying`) shows up as:

A. A composition error B. **Unexpected UVs / seams** (Obj 5.5/8.1) C. `HasAPI` False D. Missing `defaultPrim`

**VIS-024** · Obj 8.3 · Difficulty: Easy · Type: Single choice

Relationship name for a direct material bind?

A. looks B. `material:binding` C. `outputs:surface` D. `info:id`

**VIS-025** · Obj 8.2 · Difficulty: Medium · Type: Single choice

`PreviewSurface diffuseColor` input is authored with `CreateInput("diffuseColor", Color3f)`. On the prim the attribute is:

A. `diffuseColor` B. `inputs:diffuseColor` C. `outputs:diffuseColor` D. `primvars:diffuseColor`

**VIS-026** · Obj 8.4 · Difficulty: Hard · Type: Single choice

Mesh has constant `displayColor (0.2, 0.5, 0.9)` **and** a bound `PreviewSurface` with unconnected `diffuseColor = (1,0,0)`. What does a typical Hydra `PreviewSurface` draw?

A. Always the primvar (`displayColor` always wins) B. The **shader input** (red) unless you connect a primvar reader — `displayColor` is a

fallback for unshaded / simple modes, not an automatic PreviewSurface input C. Average magenta D. Black until Flatten

**VIS-027** · Obj 8.3 · Difficulty: Medium · Type: Single choice

ComputeBoundMaterial() return (verified)?

A. Only a Material B. A **tuple**: bound Material plus the **relationship** that won C. A Shader D. **None** if Apply was used

**VIS-028** · Obj 8.1 · Difficulty: Easy · Type: Single choice

displayOpacity is typically:

A. A float[] primvar alongside displayColor B. A material bind purpose C. metersPerUnit D. A relationship

**VIS-029** · Obj 8.3 · Difficulty: Medium · Type: Single choice

Binding a material to **some faces** of a mesh?

A. Impossible B. UsdGeom.Subset (or collection bind) + MaterialBindingAPI on the subset C. Split the USDA file D. **visibility = invisible** on indices

**VIS-030** · Obj 8.2 · Difficulty: Easy · Type: Single choice

## Answers

**VIS-001 — Answer: B.** PrimvarsAPI. Review: §11.2, §40, lab 12.

**VIS-002 — Answer: B.** primvars: namespace. Review: §11.1.

**VIS-003 — Answer: B.** vertex ↔ points. Review: §11.3.

**VIS-004 — Answer: A, B.** Lab 12. Review: lab 12.

**VIS-005 — Answer: A.** st texCoord. Review: §11.3, §40.4.

**VIS-006 — Answer: B.** info:id on Shader. Review: §40.3, lab 34.

**VIS-007 — Answer: B.** Material surface terminal. Review: §40.2.

**VIS-008 — Answer: B.** Verified info:id. Review: lab 34.

**VIS-009 — Answer: B.** Tuple length 3. Review: lab 34.

**VIS-010 — Answer: A, B.** Material + Shader. Review: §40.1.

**VIS-011 — Answer: B.** Apply then Bind. Review: lab 34.

**VIS-012 — Answer: B.** Rel without HasAPI. Review: lab 34.

usd-core 26.8 and UsdMtlx / MaterialX graphs in usdview?

A. from pxr import UsdMtlx works on this wheel B. **ImportError** — PreviewSurface is the visualization path this book's environment can author; MaterialX needs a full build C. MaterialX is a primvar interpolation D. UsdPreviewSurface is an alias for MaterialX

**VIS-031** · Obj 8.4 · Difficulty: Medium · Type: Single choice

Connect reader result to PreviewSurface in Python:

A. Set((1,0,0)) on diffuseColor only B. ps.CreateInput("diffuseColor", Color3f).ConnectToSource(reader.GetOutput("result")) C. AddReference the reader D. Bind(reader)

**VIS-032** · Obj 8.1 · Difficulty: Medium · Type: Single choice

GetNamespace() of primvars:st vs Mesh points?

A. Both primvars B. st → primvars; points → **empty** namespace C. Both empty D. points → inputs

**VIS-013 — Answer: B.** /Looks/Paint. Review: lab 34.

**VIS-014 — Answer: B.** Binding purposes. Review: §40.6.

**VIS-015 — Answer: B.** Bind strength vs descendants. Review: §40.6.

**VIS-016 — Answer: A, B.** Collections and GeomSubset. Review: §40.6, §39.5.

**VIS-017 — Answer: B.** Need PrimvarReader (Obj 8.4). Review: §40.5.

**VIS-018 — Answer: B.** Token varname displayColor. Review: §40.5, probe.

**VIS-019 — Answer: A.** UV reader → UVTexture → PS. Review: §40.4.

**VIS-020 — Answer: B.** float2 reader. Review: §40.4.

**VIS-021 — Answer: A, B.** Verified connection + id. Review: probe, §40.5.

**VIS-022 — Answer: B.** NodeGraph subgraph. Review: §40.1.

**VIS-023 — Answer: B.** UV seams. Review: §11.3.

**VIS-024 — Answer: B.** material:binding. Review: lab 34.

**VIS-025 — Answer: B.** `inputs:` namespace.  
Review: §40.2.

**VIS-026 — Answer: B.** Unconnected PS uses its input; `displayColor` is not auto-wired. Review: §40.5, lab 34 stretch.

**VIS-027 — Answer: B.** Tuple material + rel.  
Review: probe, lab 34.

**VIS-028 — Answer: A.** `displayOpacity` primvar.  
Review: §11.7.

**VIS-029 — Answer: B.** `GeomSubset` binds.  
Review: §39.5, §40.6.

**VIS-030 — Answer: B.** No `UsdMtlx` on `usd-core`. Review: §35.1, DE-039.

**VIS-031 — Answer: B.** `ConnectToSource`.  
Review: §40.2, §40.5.

**VIS-032 — Answer: B.** Verified namespaces.  
Review: DM-014, §5.5.

---

*Visualization domain complete: VIS-001-VIS-032 (target 32).*

## Question Bank — Customizing USD (CUST)

**Original practice questions.** Not actual NVIDIA exam content. Mapped to NCP-OUSD study-guide objectives 3.1–3.8 (Customizing USD, 6%).

Verified against **USD 26.08** (usd-core 26.8). Domain target: 24 questions. This file holds **CUST-001-CUST-026** (domain complete; two extra Obj 3.8 items added in the Phase 15 audit).

Work the stems first. Answers and explanations are grouped at the end.

---

**CUST-001** · Obj 3.1 · Difficulty: Easy · Type: Single choice

A studio .so plugin built against USD 24.11 is copied onto a DCC that embeds 26.08. Likely result?

A. Perfect load; ABI is frozen B. Load failure / missing types — **rebuild against the DCC's USD** C. USD recompiles it at Open D. Only USDA plugins fail

---

**CUST-002** · Obj 3.1 · Difficulty: Medium · Type: Multiple select Select two.

To load a studio plugin (Obj 3.1):

A. pluginInfo.json in a directory on PXR\_PLUGINPATH\_NAME B. Set the path **before** USD initializes when the host allows C. Kind.Registry.Register("plugin") D. pip install a wheel built for a different \_GLIBCXX

---

**CUST-003** · Obj 3.1 · Difficulty: Easy · Type: Single choice

Plug.Registry is used to:

A. Flatten stages B. Discover loaded plugins (GetAllPlugins, typed registries) C. Author primvars D. Replace Ar

---

**CUST-004** · Obj 3.2 · Difficulty: Medium · Type: Single choice

Why build OpenUSD from source (build\_usd.py / CMake) instead of usd-core (Obj 3.2)?

A. usd-core cannot author layers B. Need imaging, MaterialX, CLI (usdview, usdcat, usdGenSchema), or custom deps C. Source builds forbid Python D. Crate files require a source build

---

**CUST-005** · Obj 3.2 · Difficulty: Easy · Type: Single choice

Usd.GetVersion() on this book's wheel?

A. (0, 24, 11) B. (0, 26, 8) C. (26, 8, 0) D. None

---

**CUST-006** · Obj 3.3 · Difficulty: Easy · Type: Single choice

hasattr(Kind.Registry, "Register") on 26.08 Python?

A. True B. **False** — custom kinds are **pluginInfo.json**, not a Python Register call C. True after from pxr import Kind D. Register lives on Usd.ModelAPI

---

**CUST-007** · Obj 3.3 · Difficulty: Medium · Type: Single choice

SetKind("door") on a prim that was component.GetKind() and IsComponent()?

A. door, True B. **door, False** — unregistered kind is stored but is not a model/component C. Raises D. Kind resets to component

---

**CUST-008** · Obj 3.3 · Difficulty: Medium · Type: Multiple select Select two.

Built-in kind tree (verified):

A. assembly is-a group is-a model B. component is-a model but **not** a group C. subcomponent is-a model D. HasKind("door") is True on stock usd-core

---

**CUST-009** · Obj 3.3 · Difficulty: Easy · Type: Single choice

When is a **custom kind** appropriate?

A. For every unique mesh name B. When the studio needs a new **model taxonomy** the five built-ins cannot express, declared in plugin metadata C. Instead of component for all furniture D. To hide instance proxies

---

**CUST-010** · Obj 3.4 · Difficulty: Easy · Type: Single choice

shutil.which("usdGenSchema") on usd-core?

A. A path B. **None** C. usdcat D. Raises

---

**CUST-011** · Obj 3.4 · Difficulty: Medium · Type: Single choice

Sdf.Layer.ImportFromString of schema.usda defining class "Door".

FindConcretePrimDefinition("Door")?

A. Door's definition B. **None** — **parse ≠ register** C. Cube's definition D. Raises

---

**CUST-012** · Obj 3.4 · Difficulty: Medium · Type: Single choice

Codeless schema (skipCodeGeneration = 1) still needs:

A. Nothing but the parsed USDA in memory B. Generated schema.usda + **pluginInfo.json loaded as a plugin**; it only skips C++ wrappers C. Hydra SceneIndex D. GetMaster()

**CUST-013** · Obj 3.4 · Difficulty: Medium · Type: Multiple select Select two.

Typed vs applied API schemas:

A. Concrete typed (`UsdTyped / IsA("Cube")`) — Cube is concrete, size fallback 2 B. Applied API (`MaterialBindingAPI`) — `IsAppliedAPISchema` True; must **Apply** C. All APIs are `IsA` types D. Door is registered on `usd-core`

**CUST-014** · Obj 3.6 · Difficulty: Medium · Type: Single choice

Nonstandard CAD factory: `partId` vs a Door schema (Obj 3.6 / 4.5)?

A. Always schema first B. One-off field → namespaced **attribute**; shared typed prim across DCCs → **schema** C. `customData` cannot hold strings D. Schemas cannot be used on import

**CUST-015** · Obj 3.5 · Difficulty: Easy · Type: Single choice

`Ar.GetResolver()` vs `GetUnderlyingResolver()` on `usd-core`?

A. Both `DefaultResolver` B. Facade **Resolver**; underlying **DefaultResolver** C. Both None D. Underlying is `StudioResolver` after pip install `usd-core`

**CUST-016** · Obj 3.5 · Difficulty: Medium · Type: Single choice

Custom resolver integration (Obj 3.5)?

A. `AnchorRelativePath` registers it B. `ArResolver` C++ (or wrapped) plugin + `plugInfo.json` + `PXR_PLUGINPATH_NAME` + Open with a context C. `SetEditTarget` to the resolver D. Python `Kind.Registry.Register("ar")`

**CUST-017** · Obj 3.8 · Difficulty: Medium · Type: Single choice

An `AssetResolver` that **generates in-memory primitives** (Obj 3.8) is conceptually:

A. `DefaultResolver` finding PNG files B. A resolver `Resolve/OpenAsset` that returns **generated** bytes/layers (procedural URI), not only disk files C. `UsdGeom.Mesh.Define` in `usdview` D. `FlattenLayerStack`

**CUST-018** · Obj 3.7 · Difficulty: Medium · Type: Single choice

Hydra **SceneIndex** plugin (Obj 3.7) vs `usd-core`:

A. Fully implemented in the pip wheel (`import pxr.Hd` works) B. **Imaging/Hd absent** from `usd-core` — `SceneIndex` is a Hydra 2.0 filter/source of **render prims**; you need a full/imaging build C. `SceneIndex` replaces `Sdf` D. `SceneIndex` is `Kind.Registry`

**CUST-019** · Obj 3.7 · Difficulty: Easy · Type: Single choice

`SceneIndex` plugins generate:

A. USDA comments B. **Hydra prims / scene index entries** for the render delegate (not a substitute for authoring USD on disk unless that is your design) C. Git commits D. `customData` only

**CUST-020** · Obj 3.1 · Difficulty: Medium · Type: Single choice

`PXR_PLUGINPATH_NAME` pointing at plugins **instead of** the DCC's USD plugin dir so `usdGeom` disappears. Debug?

A. Expected B. Prepending a path that **hides** the default plugins — include **both** the USD prefix and the studio dir C. `MuteLayer` D. Need `LoadNone`

**CUST-021** · Obj 3.6 · Difficulty: Medium · Type: Single choice

`schema.usda class "Door" (inherits = </Typed>)` means Door is:

A. An applied API B. A **typed/IsA** schema (concrete or abstract depending on metadata) C. A file format plugin D. A resolver

**CUST-022** · Obj 3.4 · Difficulty: Easy · Type: Single choice

over "GLOBAL" `customData libraryName` in `schema.usda` is:

A. Ignored B. The schema **library** name `usdGenSchema` / plugins use C. The defaultPrim D. A kind

**CUST-023** · Obj 3.5 · Difficulty: Medium · Type: Multiple select Select two.

`AnchorRelativePath` on `Ar.Resolver` in 26.08:

A. Exists and is required B. **Gone** — use `CreateIdentifier` + `ResolvedPath` or `ComputeAbsolutePath` C. Custom resolvers still implement **their** resolve/anchor rules in C++ D. Python `Register` on `Kind` replaces `Ar`

**CUST-024** · Obj 3.2 · Difficulty: Medium · Type: Single choice

CMake flag conceptually enabling MaterialX in a source build?

A. `PXR_BUILD_MATERIALX_PLUGIN` (book's table) — not present in `usd-core` B. `USD_CORE_ONLY=1` on pip C. `Kind.Registry.Register("mtlx")` D. `stage.Flatten()`

**CUST-025** · Obj 3.8 · Difficulty: Medium · Type: Single choice

A candidate says Obj 3.8 is just `Usd.Stage.CreateInMemory()`. What is wrong?

A. Nothing — `CreateInMemory` is the procedural resolver API B. `CreateInMemory` makes an **empty stage in this process**; Obj 3.8 is an **AssetResolver** (or Hydra procedural) that **generates bytes/prims from an identifier** C. `CreateInMemory` was removed in 26.08 D. You must call `CreateInMemory` inside `Kind.Registry.Register`

**CUST-026** · Obj 3.8 · Difficulty: Medium · Type: Multiple select Select two.

Legal designs for “generate in-memory renderable primitives” from an identifier:

A. `Custom ArResolver.OpenAsset` returns a memory buffer of USDA/USDC that Sdf then parses B. A Hydra scene-index / procedural emits render prims for a URI such as `@proc:box@` C.

`Kind.Registry.Register("proc")` in Python on `usd-core 26.8` D. `stage.Flatten()` on any referenced file

## Answers

**CUST-001 — Answer: B.** ABI/version match. Review: §35.2.

**CUST-002 — Answer: A, B.** `plugInfo` + early path. Review: §36.2.

**CUST-003 — Answer: B.** `Plug.Registry`. Review: §36.2.

**CUST-004 — Answer: B.** Full stack vs wheel. Review: §35.1.

**CUST-005 — Answer: B.** Verified (0, 26, 8). Review: lab 01, §35.

**CUST-006 — Answer: B.** No Python Register. Review: lab 32.

**CUST-007 — Answer: B.** `GetKind` door, `IsComponent` False. Review: lab 32.

**CUST-008 — Answer: A, B.** Tree facts; door not registered; subcomponent not model. Review: lab 32.

**CUST-009 — Answer: B.** Taxonomy plugin. Review: §38, §3.3.

**CUST-010 — Answer: B.** `usdGenSchema` missing. Review: lab 31.

**CUST-011 — Answer: B.** Parse ≠ register. Review: lab 31.

**CUST-012 — Answer: B.** Still a plugin. Review: §37.3.

**CUST-013 — Answer: A, B.** Cube vs `MaterialBindingAPI`. Review: lab 31.

**CUST-014 — Answer: B.** Attribute vs schema. Review: §29.4, §37.5.

**CUST-015 — Answer: B.** Facade vs plugin. Review: lab 29.

**CUST-016 — Answer: B.** Resolver plugin. Review: §33.5.

**CUST-017 — Answer: B.** Procedural `OpenAsset`. Review: §38.

**CUST-018 — Answer: B.** No Hd on `usd-core`. Review: §38, §41.

**CUST-019 — Answer: B.** Hydra prims. Review: §38.

**CUST-020 — Answer: B.** Don't hide default plugins. Review: §35.2, §36.2.

**CUST-021 — Answer: B.** Typed inherit. Review: §37.4.

**CUST-022 — Answer: B.** `libraryName`. Review: lab 31.

**CUST-023 — Answer: B, C.** API gone; C++ resolver still anchors. Review: §33.2.

**CUST-024 — Answer: A.** CMake `MaterialX` plugin. Review: §35.1.

**CUST-025 — Answer: B.** `CreateInMemory` is an empty in-process stage, not a resolver. Review: §38.4.

**CUST-026 — Answer: A, B.** Resolver memory buffer or Hydra procedural; not `Kind.Register` or `Flatten`. Review: §38.4–38.5.

*Customizing USD domain complete: CUST-001–CUST-026 (target 24; +2 Obj 3.8 audit fill).*

## Mock exams

# Mock Exam 1 — NCP-OUSD Practice Paper

**Original practice exam. Not actual NVIDIA exam content.**

Verified against **USD 26.08** (usd-core 26.8). Difficulty: **medium** (paper 1 of 3).

Questions	65
Time	120 minutes
Blueprint mix	COMP 15 · DE 10 · PD 9 · DM 8 · DBG 7 · CA 7 · VIS 5 · CUST 4
Types	Single choice, multiple select (count stated), USDA- or Python-reading
Answers	mock-exams/mock01_answers.md — <b>do not open until you finish</b>

Work in order. Mark one letter unless the stem says **Select two** or **Select three**. USDA in this paper is always multiline (one-line { double x = 1 } does not parse). Python snippets are marked .norun; they are stems, not labs.

This book's readiness bar is its own. NVIDIA does not publish a passing score.

**M1-001** · COMP · Obj 1.6 · Difficulty: Medium · Type: Single choice

Two opinions target the same double reach on /Crane. One is authored in a **specializes** class. The other is authored on the **referenced** asset. No local, inherit, variant, or relocate opinion exists. Which value wins?

A. The specializes class, because class prims are always strongest B. The referenced asset, because references beat specializes in LIVERPS C. Neither — USD averages numeric opinions D. The specializes class, because specializes sit next to local in LIVERPS

**M1-002** · DM · Obj 5.2 · Difficulty: Medium · Type: Python-reading

```
from pxr import Usd, UsdGeom, Gf
stage = Usd.Stage.CreateInMemory()
xf = UsdGeom.Xform.Define(stage, "/T")
attr = xf.AddTranslateOp().GetAttr()
attr.ClearDefault()
attr.Set(Gf.Vec3d(0, 0, 0), 0)
attr.Set(Gf.Vec3d(10, 0, 0), 10)
print(attr.Get())
```

What does print show?

A. (0, 0, 0) B. (10, 0, 0) C. (5, 0, 0) D. None

**M1-003** · DE · Obj 4.3 · Difficulty: Medium · Type: Single choice

Layout artists merge a department layer in Git several times a day. The farm opens the published cache thousands of times per lighting job. Which encoding pair fits that split?

A. Git: USDC crate; farm: USDA text B. Git: USDA text; farm: USDC crate C. Both Git and farm: USDZ, because it is always smallest D. Git: .usdz; farm: USDA so usdcat can run on the farm

**M1-004** · COMP · Obj 1.5 · Difficulty: Medium · Type: USDA-reading

File: setdress.usda

```
#usda 1.0
over "Desk"
{
 double height = 4
}
```



File: `layout.usda`

```
#usda 1.0
over "Desk"
{
 double height = 12
}
```

File: `shot.usda`

```
#usda 1.0
(
 subLayers = [
 @./setdress.usda@,
 @./layout.usda@
]
)
def Xform "Desk"
{
}
```

What is the composed height on `/Desk`?

- A. 12, because `layout.usda` is listed last  
 B. 4, because the first sublayer in the list is strongest  
 C. 8, the mean of the two opinions  
 D. None — two sublayers with the same field is a composition error

**M1-005** · CA · Obj 2.4 · Difficulty: Medium · Type: Multiple select Select two.

On USD 26.08, which statements about native-instance prototypes are true?

- A. `prim.GetPrototype()` returns the shared prototype prim  
 B. `prim.GetMaster()` is the documented 26.08 name  
 C. `GetMaster` **does not exist** on `Usd.Prim` in this version  
 D. `prim.GetInstanceProxy()` is how you fetch the prototype

**M1-006** · PD · Obj 7.2 · Difficulty: Medium · Type: Single choice

`UsdUtils.ExtractExternalReferences(path)` and `UsdUtils.ComputeAllDependencies(path)` both return a 3-tuple. How do those tuples differ?

- A. Extract is (sublayers, references+other assets, payloads) for **one** file; Compute walks the **closure** as (layers, assets, unresolved)  
 B. They are identical aliases  
 C. Extract returns layers; Compute returns only unresolved paths  
 D. Compute is USDA-only; Extract is USDC-only

**M1-007** · VIS · Obj 8.3 · Difficulty: Medium · Type: Single choice

You want `HasAPI(UsdShade.MaterialBindingAPI)` to be true on a mesh and a working binding. What is the correct Python order on USD 26.08?

- A. `Bind(...)` only — Apply is implied  
 B. `UsdShade.MaterialBindingAPI.Apply(prim)` then `MaterialBindingAPI(prim).Bind(material)`  
 C. Author `rel material:binding` in USDA and skip Apply; HasAPI becomes true automatically  
 D. `UsdGeom.Gprim(prim).BindMaterial(material)`

**M1-008** · DBG · Obj 6.1 · Difficulty: Medium · Type: Python-reading

A listener counts `Usd.Notice.ObjectsChanged`. Three `CreateAttribute(...).Set(...)` calls on an existing prim fire **6** notices. The same two attribute specs authored inside with `Sdf.ChangeBlock()`: fire **1** notice. Why use the block?

- A. It disables composition so LIVERPS is skipped  
 B. It coalesces change processing into one notice, which can remove a notice-storm bottleneck  
 C. It makes `DefinePrim` of a brand-new child path always succeed  
 D. It writes USDC instead of USDA

**M1-009** · COMP · Obj 1.1 · Difficulty: Medium · Type: USDA-readingFile: `crane_asset.usda`

```
#usda 1.0
(
 defaultPrim = "Crane"
)
def Xform "Crane"
{
 double reach = 2
}
```

File: crane\_shot.usda

```
#usda 1.0
class "_Reach"
{
 double reach = 7
}

def "Crane" (
 prepend references = @./crane_asset.usda@
 prepend inherits = </_Reach>
)
{
}
```

What is composed **reach** on /Crane?

A. 2 — references beat inherits B. 7 — inherits beat references C. 9 — USD adds the two opinions D. None — inherit and reference cannot share a prim

---

**M1-010** · DM · Obj 5.1 / 8.1 · Difficulty: Medium · Type: Single choice

A mesh has 4 vertices and 2 triangle faces. You author **primvars:displayColor** as **vertex** interpolation. How many color values belong on the primvar array (ignoring indexing)?

A. 1 — one color for the whole mesh B. 2 — one color per face C. 4 — one color per vertex D. 6 — one color per face-vertex corner

---

**M1-011** · DE · Obj 4.3 · Difficulty: Medium · Type: Single choice

Usd.Stage.CreateNew("hero.usd") then Save(). What are the crate **magic bytes** and GetFileFormat().formatId?

A. Magic #usda 1.0, formatId usda B. Magic PXR-USDC, formatId usdc C. Magic PXR-USDC, formatId usd (id follows the .usd plugin; magic follows crate bytes) D. Magic PK, formatId usdz

---

**M1-012** · COMP · Obj 1.3 · Difficulty: Medium · Type: Multiple select Select two.

A set has a cheap stand-in **Hero** and a heavy sculpt **Heavy** that lighting should be able to unload. Which pair of arcs matches that intent?

A. **Hero** uses a **reference** (always composed when the parent is) B. **Heavy** uses a **payload** (deferred; LoadNone leaves it unloaded) C. **Heavy** must be a **sublayer**, because only sublayers can unload D. **Hero** must be a **specialize**, because specializes are optional

---

**M1-013** · PD · Obj 7.2 · Difficulty: Medium · Type: Single choice

In a work / publish / pin pipeline, which statement is true?

A. Artists should overwrite the published USDC in place so Git always has one file B. Work layers stay editable; publish writes an immutable cache; the shot **pins** a versioned identifier C. Pinning means deleting the work layer after each save D. Publish layers must be USDA so the farm can diff them

---

**M1-014** · CUST · Obj 3.3 · Difficulty: Medium · Type: Single choice

A studio wants a new model kind **heroComponent** registered from Python on usd-core 26.8. What happens if they call Kind.Registry.Register("heroComponent", "component")?

A. It succeeds and `HasKind("heroComponent")` becomes true B. The method **does not exist** — kinds are registered through the kind plugin / `plugInfo.json`, not `Kind.Registry.Register` C. It succeeds only inside `Sdf.ChangeBlock` D. It succeeds but only for the current stage

---

**M1-015** · CA · Obj 2.3 · Difficulty: Medium · Type: Single choice

A `UsdGeom.PointInstancer` has `protoIndices = [0, 0, 0]` and then `invisibleIds = [1]`. What is `GetInstanceCount()` after the hide?

A. 2 — hidden ids are removed from the count B. 3 — the count still includes hidden instances; `invisibleIds` is a mask C. 0 — any `invisibleIds` deactivates the instancer D. 1 — only the hidden id is counted

---

**M1-016** · COMP · Obj 1.4 · Difficulty: Medium · Type: USDA-reading

File: `walk.usda`

```
#usda 1.0
(
 defaultPrim = "Clip"
)
def Xform "Clip"
{
 double3 xformOp:translate.timeSamples = {
 0: (0, 0, 0),
 10: (10, 0, 0),
 }
 uniform token[] xformOpOrder = ["xformOp:translate"]
}
```

File: `crowd.usda`

```
#usda 1.0
def "Walker" (
 prepend references = @./walk.usda@ (
 offset = 10.0
 scale = 2.0
)
)
{
}
```

What are the composed sample times on `/Walker.xformOp:translate`?

A. `[0, 10]` — offsets never change sample times B. `[10, 30]` — composed time is `offset + scale * layerTime` C. `[20, 40]` — USD multiplies then adds twice D. `[5, 15]` — USD uses `(offset + layerTime) / scale`

---

**M1-017** · DM · Obj 5.6 · Difficulty: Medium · Type: Multiple select Select two.

An exporter moved every point on a mesh and forgot to touch `extent`. Which pair restores a valid extent?

A. `UsdGeom.Boundable.ComputeExtentFromPlugins(mesh, time)` then `mesh.GetExtentAttr().Set(...)` B. Set extent to `[(0,0,0), (0,0,0)]` and hope Hydra ignores it C. Recompute extent whenever points change; keep the two arrays in sync D. Delete `extent` permanently so USD guesses at render time from kind alone

---

**M1-018** · VIS · Obj 8.2 · Difficulty: Medium · Type: Multiple select Select two.

On a fresh in-memory stage, which intensity fallbacks are true on USD 26.08?

A. `UsdLux.DistantLight` intensity fallback is 50000 B. `UsdLux.SphereLight` intensity fallback is 1 C. Both lights fallback to None until you set intensity D. Distant and Sphere share the same fallback of 1

---

**M1-019** · DBG · Obj 6.3 · Difficulty: Medium · Type: Single choice

A referenced layer path cannot be opened (@./missing.usda@). A referenced prim path is well-formed but the target prim is not in that layer. Which diagnostic pair matches those two failures?

A. Both are `UnresolvedPrimPath` B. Missing file: `InvalidAssetPath`; missing prim in an opened layer: `UnresolvedPrimPath` C. Both are `Tf.MallocTag` D. Missing file: `UnresolvedPrimPath`; missing prim: `InvalidAssetPath`

---

**M1-020** · COMP · Obj 1.7 · Difficulty: Medium · Type: USDA-reading

```
#usda 1.0
def Xform "Crate" (
 prepend variantSets = "fit"
)
{
 variantSet "fit" = {
 "tight" {
 double binWidth = 1
 }
 "loose" {
 double binWidth = 8
 }
 }
}
```

No variant is selected. What does `GetPrimAtPath("/Crate").GetAttribute("binWidth").Get()` return?

A. 1 — first variant wins automatically B. 8 — last variant wins automatically C. None — without a selection the attribute is not composed D. 0 — USD fills a numeric zero

---

**M1-021** · DE · Obj 4.7 · Difficulty: Medium · Type: Multiple select Select two.

You flatten a shot that **references** `bolt.usda` and has a **sublayer** of notes. Which statements match USD 26.08?

A. `stage.Flatten()` **bakes** the reference (the flattened prim no longer holds the reference list) B. `UsdUtils.FlattenLayerStack(stage)` **keeps** references and **drops** sublayers C. `FlattenLayerStack` keeps sublayers and deletes references D. `Flatten()` refuses to run if any payload is unloaded

---

**M1-022** · PD · Obj 7.5 · Difficulty: Medium · Type: Single choice

On `usd-core 26.8`, `type(Ar.GetResolver()).__name__` is `Resolver`. Where is `DefaultResolver`?

A. There is no `DefaultResolver` in 26.08 B. `type(Ar.GetUnderlyingResolver()).__name__` is `DefaultResolver` — `GetResolver()` is the facade C. `Ar.GetResolver()` is already a `DefaultResolver` instance D. You must call `prim.GetPrototype()` to see the resolver

---

**M1-023** · CA · Obj 2.4 · Difficulty: Medium · Type: USDA-reading

```
#usda 1.0
def Xform "Proto"
{
 def Cube "Body"
 {
 }
}

def Xform "Wagon" (
 instanceable = true
 prepend references = </Proto>
)
{
 over "Seat"
 {
```

```
}
}
```

Which statement is true of `/Wagon`?

- A. `/Wagon/Seat` is a composed child; `Traverse` lists it B. `/Wagon` is an instance; `/Wagon/Body` is an instance proxy; `/Wagon/Seat` is **not** composed (nested over of a new child is ignored) C. `instanceable` is ignored unless the prim is a `PointInstancer` D. `GetMaster()` returns `/Proto`

**M1-024** · COMP · Obj 1.8 · Difficulty: Medium · Type: USDA-reading

File: `bot_asset.usda`

```
#usda 1.0
(
 defaultPrim = "Bot"
)
def Xform "Bot"
{
 def Xform "Rig"
 {
 def Cube "Arm"
 {
 double size = 1
 }
 }
}
```

File: `bot_shot.usda`

```
#usda 1.0
(
 relocates = {
 </Bot/Rig/Arm>: </Bot/Arm>
 }
)
def "Bot" (
 prepend references = @./bot_asset.usda@
)
{
 over "Arm"
 {
 double size = 4
 }
 over "Rig"
 {
 over "Arm"
 {
 double size = 99
 }
 }
}
```

After composition, what is `size` on `/Bot/Arm`?

- A. 99 — the source-path over is strongest B. 4 — the **destination** over wins; the source-path opinion is ignored C. 1 — relocates disable all overs D. None — relocates cannot be authored as layer metadata

**M1-025** · DM · Obj 5.4 · Difficulty: Medium · Type: Multiple select Select two.

Z-up tokens on `UsdGeom` in USD 26.08:

- A. `hasattr(UsdGeom.Tokens, "Z")` is True B. `hasattr(UsdGeom.Tokens, "z")` is True C. `UsdGeom.Tokens.Z` does **not** exist (capital Z is a trap) D. `UsdGeom.UpAxis.Z` is the only legal spelling

**M1-026** · VIS · Obj 8.4 · Difficulty: Medium · Type: Single choice

`UsdShade.Material.ComputeSurfaceSource()` on usd-core 26.8 returns a value whose `len(...)` is:

- A. 1 — just the shader B. 2 — shader and output name C. 3 — a 3-tuple D. 0 — it always returns `None`

---

**M1-027** · PD · Obj 7.3 · Difficulty: Medium · Type: Single choice

A coordinator needs one layer that still **points at** published component files, but must not keep department **sublayers**. Which API?

- A. `stage.Flatten()` — it keeps references B. `UsdUtils.FlattenLayerStack(stage)` — sublayers collapse; references remain C. `Sdf.Layer.Export` — it always strips references D. `Usd.Stage.OpenMasked` — it deletes sublayers as a side effect

---

**M1-028** · COMP · Obj 1.3 · Difficulty: Medium · Type: Single choice

`Usd.Stage.Open("set.usda", Usd.Stage.LoadNone)` where `/Hero` **references** `lamp.usda` (defaultPrim `Lamp` with child `Shade`) and `/Heavy` **payloads** the same file. What is in the stage?

- A. Neither `/Hero/Shade` nor `/Heavy/Shade` B. `/Hero/Shade` is composed (references still load); `/Heavy/Shade` is not (payload deferred) C. Both children load, because `LoadNone` only skips sublayers D. The call raises — `LoadNone` cannot be passed to `Open`

---

**M1-029** · DBG · Obj 6.1 · Difficulty: Medium · Type: Single choice

Inside with `Sdf.ChangeBlock()`: you call `stage.DefinePrim("/ExistingParent/BrandNewChild")` where that child spec does not yet exist. What can happen on USD 26.08?

- A. It always succeeds silently B. It can raise `Tf.ErrorException` — defining a **new** child path inside a live `ChangeBlock` is a known trap C. It converts the stage to USDA D. It only fails if the parent is instanceable

---

**M1-030** · DE · Obj 4.3 · Difficulty: Medium · Type: Single choice

`UsdUtils.CreateNewUsdzPackage("root.usda", "out.usdz")` on usd-core 26.8 writes ZIP members with which `compress_type`?

- A. `zipfile.ZIP_DEFLATED` (8) B. `zipfile.ZIP_STORED` (0) — USDZ is store-only C. `zipfile.ZIP_BZIP2` D. Uncompressed only if the file is USDA

---

**M1-031** · CA · Obj 2.1 · Difficulty: Medium · Type: Single choice

A `PointInstancer` already has one prototype and `protoIndices` of all zeros. You `prototypes.AddTarget("/PI/Tree")` (default list position). What is the usual production hazard?

- A. `AddTarget` always appends, so old indices stay valid B. `AddTarget` **prepends** by default, so index 0 now names the new proto and every existing instance switches C. `AddTarget` deletes `protoIndices` D. `PointInstancer` forbids a second prototype

---

**M1-032** · COMP · Obj 1.1 · Difficulty: Medium · Type: USDA-reading

Same `crane_asset.usda` as M1-009 (`reach = 2`). Shot:

```
#usda 1.0
class "_ReachS"
{
 double reach = 7
}

def "Crane" (
 prepend references = @./crane_asset.usda@
 prepend specializes = </_ReachS>
)
```

```
{
}
```

Composed reach?

A. 7 — specializes beat references B. 2 — references beat specializes C. None — specialize plus reference is illegal D. 4.5 — the mean

**M1-033** · DM · Obj 5.2 · Difficulty: Medium · Type: USDA-reading

File: `cached.usda`

```
#usda 1.0
(
 defaultPrim = "P"
)
def Sphere "P"
{
 double radius.timeSamples = {
 0: 1,
 10: 5,
 }
}
```

File: `hero.usda`

```
#usda 1.0
def "P" (
 prepend references = @./cached.usda@
)
{
 double radius = 9
}
```

`attr = stage.GetPrimAtPath("/P").GetAttribute("radius").` What are `attr.Get()` and `list(attr.GetTimeSamples())`?

A. 9.0 and [] — a stronger **default** hides weaker samples B. 9.0 and [0, 10] C. 1.0 and [0, 10] — samples always beat defaults D. None and [0, 10]

**M1-034** · CUST · Obj 3.4 · Difficulty: Medium · Type: Single choice

You `Sdf.Layer.FindOrOpen("schema.usda")` for a codeless schema. Does that **register** the schema with `Usd`?

A. Yes — parsing is registration B. No — parse ≠ register; the plugin must be on `PXR_PLUGINPATH_NAME / Plug.Registry` so generated types exist C. Yes, if the file is named `schema.usda` D. Yes, but only for `UsdAPISchemaBase` types

**M1-035** · PD · Obj 7.6 · Difficulty: Medium · Type: Single choice

A producer must stash a pipeline `assetId` string that renderers should ignore. Best field?

A. `customData` dictionary on the prim or layer (not a schema attribute) B. `primvars:displayColor` C. `kind = "assetId"` D. `upAxis`

**M1-036** · COMP · Obj 1.8 · Difficulty: Medium · Type: Single choice

A department layer authors over `"Lamp" { double radius = 9 }` but the composed stage still shows the referenced 1. Which check is the most useful first step?

A. Assume USD is broken and reinstall `usd-core` B. Inspect **LIVERPS and the layer stack**: a stronger opinion (local in a stronger sublayer, inherit, variant, or a muted target layer) can hide the department over C. Convert the file to USDZ D. Call `GetMaster()` on `/Lamp`

**M1-037** · VIS · Obj 8.4 · Difficulty: Medium · Type: Single choice

A `UsdPreviewSurface` must read **vertex displayColor**. Which reader id and input name are the usual pair?

A. `UsdPrimvarReader_float3` with `inputs:varname = displayColor` B. `UsdUVTexture` with `inputs:varname = st` C. `UsdPrimvarReader_float` with `inputs:varname = points` D. `UsdPreviewSurface` itself reads `displayColor` with no reader

**M1-038** · DBG · Obj 6.5 · Difficulty: Medium · Type: Multiple select Select two.

You need allocator stats and a timeline of USD work on this book's pip `usd-core` 26.8 wheel. Which pair is accurate?

A. `Tf.MallocTag.GetTotalBytes()` often reports **0 bytes** on this wheel — tagging was not compiled in B. `Usd.Trace.Collector / Reporter` can still record a trace of function scopes C. `Tf.MallocTag` always matches `ps RSS` D. Trace requires `UsdImaging` and will not import

**M1-039** · DE · Obj 4.4 · Difficulty: Medium · Type: Multiple select Select two.

A DCC round-trip must keep studio identifiers stable. Which practices belong in the exporter/importer pair?

A. Run names through `Tf.MakeValidIdentifier` on export and persist the original name in `customData` if it changed B. Drop `defaultPrim` so the DCC can pick a root at random C. Author `metersPerUnit` and `upAxis` on the root layer and document that **composition does not auto-convert** units or axis D. Write every mesh as `Sphere` to save bytes

**M1-040** · COMP · Obj 1.1 · Difficulty: Medium · Type: USDA-reading

File: `r1.usda`

```
#usda 1.0
(
 defaultPrim = "Ball"
)
def Sphere "Ball"
{
 double radius = 2
}
```

File: `r2.usda`

```
#usda 1.0
(
 defaultPrim = "Ball"
)
def Sphere "Ball"
{
 double radius = 8
}
```

File: `shot.usda`

```
#usda 1.0
def "Ball" (
 prepend references = @./r1.usda@
 append references = @./r2.usda@
)
{
}
```

Composed `radius` on `/Ball`?

A. 8 — append always beats prepend B. 2 — prepended references are stronger than appended ones C. 5 — the mean D. None — prepend and append cannot combine

**M1-041** · DM · Obj 5.3 · Difficulty: Medium · Type: Single choice

`prim.GetCustomDataByKey("dept")` versus `prim.GetMetadata("documentation")`. Which statement is true?



A. They are the same field B. `customData` is a dictionary of producer key/values; `documentation` / `userDocBrief` are the human-doc metadata fields C. `customData` is only legal on attributes, never prims D. Setting `customData` registers a new schema

**M1-042** · CA · Obj 2.4 · Difficulty: Medium · Type: USDA-reading

```
#usda 1.0
def Xform "Clutter"
{
 def Cube "Mug" (
 kind = "component"
)
 {
 }
}
```

`mug = stage.GetPrimAtPath("/Clutter/Mug")`. What are `Usd.ModelAPI(mug).GetKind()` and `mug.IsComponent()`?

A. `component` and `True` B. `component` and `False` — the parent is not a model, so the component chain is broken C. `assembly` and `True` D. `""` and `True`

**M1-043** · PD · Obj 7.4 · Difficulty: Medium · Type: Single choice

A DCC plugin is loaded against this venv. `Usd.GetVersion()` prints `(0, 26, 8)`. What must the plugin match?

A. Any USD 0.x is ABI-compatible B. The **same** OpenUSD / usd-core ABI the DCC was built against — a 26.8 wheel does not make a 24.11 plugin safe C. Only the Python minor version, not USD D. `Kind.Registry` version tags

**M1-044** · COMP · Obj 1.2 · Difficulty: Medium · Type: Single choice

You need to open only `/World/Hero` and its descendants from a huge stage. Which API is correct on 26.08?

A. `Usd.Stage.Open(path, mask=["/World/Hero"])` B. `Usd.Stage.OpenMasked(path, Usd.Stage.PopulationMask(["/World/Hero"]))` C. `Usd.Stage.Open(path, Usd.Stage.LoadNone)` — that is a population mask D. `prim.GetPrototype()`

**M1-045** · DBG · Obj 6.5 · Difficulty: Medium · Type: Single choice

Composition spews hundreds of similar asset-path warnings during `Open`. Which helper coalesces them for a readable report?

A. `UsdUtils.CoalescingDiagnosticDelegate` installed **before** `Open` B. `Tf.MallocTag.Initialize` after `Open` C. `Kind.Registry.GetAllKinds` D. `UsdGeom.Tokens.Z`

**M1-046** · DE · Obj 4.5 · Difficulty: Medium · Type: USDA-reading

```
#usda 1.0
def Mesh "Bolt"
{
 rel material:binding = </Looks/Steel>
}

def Scope "Looks"
{
 def Material "Steel"
 {
 }
}
```

On `/Bolt`, `HasAPI(UsdShade.MaterialBindingAPI)` and `HasRelationship("material:binding")` are:

A. True, True B. False, True — the rel is authored; the API schema was not applied C. True, False D. Both False — USDA cannot author relationships

**M1-047** · VIS · Obj 8.3 · Difficulty: Medium · Type: Multiple select Select two.

Which statements about material **purpose** and binding strength are true?

A. `preview` purpose is the usual choice for `usdview` / `UsdPreviewSurface` lookdev B. `allPurpose` (empty token) is a fallback when a specific purpose is not bound C. `weakerThanDescendants` makes the ancestor always win over a child bind D. Purpose `physics` is a required `UsdShade` token on every mesh

**M1-048** · CUST · Obj 3.7 · Difficulty: Medium · Type: Single choice

A teammate wants a Hydra **SceneIndex** plugin that emits geometry as Hydra prims, using only this book's pip `usd-core 26.8` environment. What is the blocker?

A. SceneIndex plugins are authored in USDA B. `UsdImaging` / `Hd` are **not** in the `usd-core` wheel — you need an imaging build of OpenUSD C. SceneIndex requires `Kind.Registry.Register` D. SceneIndex works if you `import pxr.UsdShade`

**M1-049** · COMP · Obj 1.5 · Difficulty: Medium · Type: Multiple select Select two.

Two departments must edit the same shot without clobbering each other. Which layering choices support that?

A. Each department owns a **sublayer**; the shot root lists them in strength order B. Everyone authors into the same root layer with no sublayers C. Use **edit targets** so a tool's `Set` writes the intended department layer D. Mute the root layer so no one can save

**M1-050** · DM · Obj 5.2 · Difficulty: Medium · Type: Python-reading

Samples at `t=0` `(0,0,0)` and `t=10` `(10,0,0)`. Interpolation is **linear**, then **held**. What is `attr.Get(4)` in each mode?

A. Linear `(4, 0, 0)`; held `(0, 0, 0)` B. Linear `(0, 0, 0)`; held `(4, 0, 0)` C. Both `(4, 0, 0)` D. Both None

**M1-051** · PD · Obj 7.7 · Difficulty: Medium · Type: Multiple select Select two.

A validator calls `Ar.GetResolver().Resolve("./tex/wood.png")` from a random cwd and gets empty, but the composed stage shows the texture. Why?

A. Naive `Resolve("./...")` uses **process cwd**; composition **anchors** to the layer that authored the path B. `AnchorRelativePath` is the 26.08 replacement and always exists on `Ar.Resolver` C. `Sdf.Layer.ComputeAbsolutePath / CreateIdentifier` with the layer's `ResolvedPath` is the layer-relative check D. Search paths apply to identifiers that start with `./`

**M1-052** · CA · Obj 2.2 · Difficulty: Medium · Type: Single choice

You must tint **one** native instance's mesh without turning `instanceable` off. Which approach is legal?

A. `stage.OverridePrim("/Instance/Mesh")` and set `primvars:displayColor` on the proxy — this raises on 26.08 B. Author the color on the **instance root** (or a stronger inherited opinion that applies to that instance) so the prototype stays shared C. Edit the prototype mesh in place — that tints **every** instance D. Set `instanceable = false` on the prototype prim

**M1-053** · COMP · Obj 1.9 · Difficulty: Medium · Type: Single choice

`UsdUtils.GetModelNameFromRootLayer(layer)` on a layer with **no** `defaultPrim` and two root prims First then Second returns:

A. `''` always B. `'First'` — it falls back to the first root prim C. `'Second'` D. It raises `Tf.ErrorException`

---

**M1-054** · DE · Obj 4.6 · Difficulty: Medium · Type: Multiple select Select two.

Validation on this book's `usd-core 26.8` venv:

A. `from pxr import UsdValidation` works (`ValidationRegistry`, `Validator`, ...) B. `UsdUtils.ComplianceChecker` exists C. `hasattr(UsdUtils, "ComplianceChecker")` is `False` D. The `usdchecker` CLI ships in the pip wheel

---

**M1-055** · DBG · Obj 6.2 · Difficulty: Medium · Type: Single choice

`/Lamp.radius` is `1` from a reference. A **specializes** class on the shot authors `9`. Why does the artist still see `1`?

A. Specializes are stronger than references, so this cannot happen B. References beat specializes — the class opinion is weaker, so the referenced `1` wins C. Specializes only work inside payloads D. Numeric attributes cannot be specialized

---

**M1-056** · CUST · Obj 3.5 · Difficulty: Medium · Type: Multiple select Select two.

A studio needs a **custom Ar resolver** that maps `hero://` URIs. Which statements are true?

A. Implement an Ar resolver plugin, declare it in `pluginInfo.json`, and put it on the plugin path B. `Ar.GetResolver()` remains the process-wide facade after the plugin loads C. `AnchorRelativePath` is required public API on 26.08 D. Resolvers are registered with `Kind.Registry`

---

**M1-057** · COMP · Obj 1.11 · Difficulty: Medium · Type: Multiple select Select two.

A monolithic `hero.usda` (`geo`, `look`, `rig`) must split for parallel workstreams. Which split matches OpenUSD practice?

A. Payload **geometry**, payload or reference **look**, keep a small interface layer with `defaultPrim` B. Put every department into one root layer forever C. Use sublayers inside the asset for department work; publish pinned USDC per stream D. Replace composition with `Flatten()` as the only daily format

---

**M1-058** · DE · Obj 4.1 · Difficulty: Medium · Type: Single choice

Exporting USD to glTF, which fidelity issue is the exporter most responsible for documenting?

A. glTF will run LIVERPS the same way USD does, so composition can be ignored B. Materials, units, and node hierarchy must be **mapped** — glTF has no LIVERPS; `PreviewSurface/USDZ` extras do not always round-trip C. glTF automatically reads USDA variants D. `metersPerUnit` is always `1` after conversion, so no note is needed

---

**M1-059** · PD · Obj 7.2 · Difficulty: Medium · Type: Single choice

What should a shot's published identifier pin?

A. A floating `latest/` directory that mutates every hour B. A **versioned** asset path (or resolver URI) that will still resolve the same bytes next month C. A relative `./temp.usda` in an artist's home directory D. The `usd-core` git commit of whoever last saved

---

**M1-060** · DM · Obj 5.5 · Difficulty: Medium · Type: Python-reading

```
from pxr import Usd, UsdGeom
s = Usd.Stage.CreateInMemory()
print(UsdGeom.GetStageUpAxis(s), UsdGeom.GetStageMetersPerUnit(s))
```

What prints on USD 26.08?

A. Z 1.0 B. Y 0.01 C. Y 1.0 D. Z 0.01

---

**M1-061** · DE · Obj 4.8 · Difficulty: Medium · Type: Single choice

A DCC importer sees `formatId == "usd"` but the file's first bytes are `#usda 1.0`. What happened?

A. The file is corrupt B. The `.usd` plugin owns the **extension**; the **contents** can still be USDA text (or crate) C. `formatId` always matches the magic string D. USD refuses to open USDA bytes under a `.usd` name

---

**M1-062** · CA · Obj 2.5 · Difficulty: Medium · Type: Single choice

An assembly instance-roots several component assets. You need to **remove** a property that lives on an instanced component without breaking instancing for the others. Where can you author that?

A. On the instance **proxy** mesh with `OverridePrim` — always legal B. At the **instance root** (or by de-instancing that one root), not on descendant proxies C. Only by editing every prototype in the crate D. Only with `PointInstancer.invisibleIds`

---

**M1-063** · DBG · Obj 6.4 · Difficulty: Medium · Type: Single choice

A cube is in the stage but missing in the viewport. `imageable:purpose` is `proxy` and `usdview` is showing **render** purpose only. What is the likely cause?

A. `kind` must be `subcomponent` B. Purpose **mismatch** — the `gprim` is excluded from the viewer's purpose mask C. `extent` is always required for purpose to work D. Distant lights hide proxy purpose

---

**M1-064** · DE · Obj 4.7 · Difficulty: Medium · Type: Multiple select Select two.

A first-pass component exporter should author which root-layer facts so downstream composition is predictable?

A. `defaultPrim` B. `metersPerUnit` and `upAxis` C. `Kind.Registry.Register` in the USDA header D. `GetMaster` metadata

---

**M1-065** · PD · Obj 7.8 · Difficulty: Medium · Type: Multiple select Select two.

When extending a DCC USD importer, which hooks belong in the design?

A. Map DCC nodes to typed USD prims / API schemas; preserve unknown properties as custom attributes or `customData` B. Call `stage.Flatten()` on every import so the DCC never sees composition C. Honor payloads vs references (working-set load) instead of always composing the heavy sculpt D. Require `UsdMtlx` because `usd-core` always ships it

---

*End of Mock Exam 1. 65 questions. Check `mock-exams/mock01_answers.md` only after you have finished.*

# Mock Exam 1 — Answer Key

**Original practice exam. Not actual NVIDIA exam content.**

Paper: `mock-exams/mock01_paper.md`. Verified against **USD 26.08** (`usd-core` 26.8). USDA/Python items were re-run before this key was written.

Scoring: 1 point per question (multiple-select must match **all** correct letters). Total **65**. This book's Mock 1 review bar is  $\geq$  **70% overall (46/65)** and no domain under 50%. NVIDIA does not publish a passing score.

## Score sheet

Domain	Q IDs	/n	Your score	%
Composition (COMP)	001, 004, 009, 012, 016, 020, 024, 028, 032, 036, 040, 044, 049, 053, 057	/15		
Data Exchange (DE)	003, 011, 021, 030, 039, 046, 054, 058, 061, 064	/10		
Pipeline Development (PD)	006, 013, 022, 027, 035, 043, 051, 059, 065	/9		
Data Modeling (DM)	002, 010, 017, 025, 033, 041, 050, 060	/8		
Debugging (DBG)	008, 019, 029, 038, 045, 055, 063	/7		
Content Aggregation (CA)	005, 015, 023, 031, 042, 052, 062	/7		
Visualization (VIS)	007, 018, 026, 037, 047	/5		
Customizing USD (CUST)	014, 034, 048, 056	/4		
<b>Total</b>	M1-001-M1-065	<b>/ 65</b>		

Copy the weakest domains into `00_MASTER_PLAN.md` §9 before Mock 2.

## Answers

**M1-001 — Answer: B.** Obj 1.6. LIVERPS: Local, Inherits, VariantSets, rElocates, References, Payloads, **Specializes** (weakest). A referenced `reach` beats a specializes class. A is the class-is-always-strong myth. C: USD does not average. D: specializes are not next to local. Review: Ch 14, Ch 21.

**M1-002 — Answer: D.** Obj 5.2. `Get()` with no time argument reads the **default**. Samples-only attributes have no default, so `Get()` is `None` (not the first sample). A/B/C confuse default with time-sampled evaluation. Review: Ch 10.

**M1-003 — Answer: B.** Obj 4.3. USDA is UTF-8 text (Git-friendly). USDC crate is smaller/faster for the farm. A reverses the pair. C/D: USDZ is a ZIP package, not a department merge format. Review: Ch 7, Ch 31.

**M1-004 — Answer: B.** Obj 1.5. Verified: first sublayer in `subLayers` is strongest, so `height = 4`. Last-listed is weakest (A). No averaging (C). Two overs on the same field is normal composition (D). Review: Ch 15.

**M1-005 — Answer: A, C.** Obj 2.4. `GetPrototype()` is the 26.08 name. `GetMaster` **does not exist** on `Usd.Prim`. B/D invent APIs. Review: Ch 24.

**M1-006 — Answer: A.** Obj 7.2. `Extract` is a **one-file** split (`sublayers`, `refs+assets`, `payloads`). `Compute` walks the **closure** (`layers`, `assets`, `unresolved`). They are not aliases. Review: Ch 32.

**M1-007 — Answer: B.** Obj 8.3. Apply the API schema, then `Bind`. `Bind` without `Apply` can still write the relationship, but `HasAPI` stays false (A/C). There is no `Gprim.BindMaterial` (D). Review: Ch 40.

**M1-008 — Answer: B.** Obj 6.1. Lab 09: three `CreateAttribute().Set()` pairs → **6** notices; two `Sdf` specs inside `ChangeBlock` → **1**. The block coalesces change processing. C is the opposite: `DefinePrim` of a **new** child inside a live block can raise. Review: Ch 8, Ch 34, Ch 44.

**M1-009 — Answer: B.** Obj 1.1. Verified: inherit 7 beats referenced 2. A reverses `LIVERPS`. C/D are invented. Review: Ch 19, Ch 21.

**M1-010 — Answer: C.** Obj 5.1 / 8.1. Vertex interpolation: one value per vertex → 4. Face interpolation would be 2 (B). `FaceVarying` would be 6 corners (D). Constant is 1 (A). Review: Ch 11.

**M1-011 — Answer: C.** Obj 4.3. `CreateNew("hero.usd")` writes crate bytes `PXR-USDC` but `formatId` is `usd` (the `.usd` plugin). B names `formatId` `usdc` (wrong for `.usd`). A is a `.usda` file. D is ZIP. Review: Ch 7, Ch 31.

**M1-012 — Answer: A, B.** Obj 1.3. References compose with the parent; payloads are the deferred working set (`LoadNone` leaves `/Heavy/Shade` out). Sublayers are the shot's layer stack, not a per-prim unload switch (C). Specializes are a weak opinion arc, not "optional geo" (D). Review: Ch 16, Ch 17.

**M1-013 — Answer: B.** Obj 7.2. Work stays editable; publish is an immutable cache; the shot pins a versioned identifier. In-place overwrite of publish (A) and "pin = delete work" (C) break pipelines. Farm caches want `USDC`, not `USDA` diffs (D). Review: Ch 32.

**M1-014 — Answer: B.** Obj 3.3. `Kind.Registry.Register` **does not exist**. Kinds come from the kind plugin / `pluginInfo.json`. Review: Ch 6, Ch 38.

**M1-015 — Answer: B.** Obj 2.3. Verified: `GetInstanceCount()` stays **3**; `invisibleIds` masks drawing, it does not shrink the count. Review: Ch 25.

**M1-016 — Answer: B.** Obj 1.4. Verified: `Sdf.LayerOffset(offset=10, scale=2)` maps layer times 0 and 10 to composed **10 and 30** (`offset + scale * layerTime`). Review: Ch 10, Ch 16.

**M1-017 — Answer: A, C.** Obj 5.6. Compute via `ComputeExtentFromPlugins`, then `Set` on `extent`. Keep points and extent in sync. Zero extent (B) and "delete and hope" (D) produce wrong bounds / warnings. Review: Ch 13.

**M1-018 — Answer: A, B.** Obj 8.2. Verified: Distant intensity fallback **50000**; Sphere **1**. Exposure fallback is `0`. Review: Ch 41.

**M1-019 — Answer: B.** Obj 6.3. Missing asset file → `InvalidAssetPath`. Opened layer but missing prim → `UnresolvedPrimPath`. A/D swap them. Review: Ch 42.

**M1-020 — Answer: C.** Obj 1.7. Verified: no selection → `binWidth.Get()` is `None` (the attribute is not even composed / not valid). Variants do not auto-pick first or last. (Trap: a **Cube** size would still show the schema fallback `2.0` with `HasAuthoredValue()` `False` — this stem used a custom double to avoid that.) Review: Ch 18.

**M1-021 — Answer: A, B.** Obj 4.7. Verified: `Flatten()` bakes references; `FlattenLayerStack` keeps references and drops sublayers. C reverses FLS. D is false. Review: Ch 34.

**M1-022 — Answer: B.** Obj 7.5. `Ar.GetResolver()` is the **Resolver** facade; `Ar.GetUnderlyingResolver()` is **DefaultResolver** on `usd-core`. `GetResolver()` is not itself `DefaultResolver` (C). Review: Ch 33.

**M1-023 — Answer: B.** Obj 2.4. Verified: `/Wagon` is an instance; `/Wagon/Body` is an instance proxy; `/Wagon/Seat` is **not** composed; `OverridePrim` on the proxy raises; `GetMaster` is gone. `Traverse` lists `/Wagon` but not the nested `Seat`. Review: Ch 24.

**M1-024 — Answer: B.** Obj 1.8. Verified: destination `/Bot/Arm` size **4**; the over at the **source** path `/Bot/Rig/Arm` is ignored (composition warning). Relocates are layer metadata. Review: Ch 20.

**M1-025 — Answer: B, C.** Obj 5.4. Verified: `Tokens.z` exists; `Tokens.Z` does **not**. Review: Ch 9, Ch 39.

**M1-026 — Answer: C.** Obj 8.4. Verified: `ComputeSurfaceSource()` is a tuple of **len 3** (even on an empty material). Review: Ch 40.

**M1-027 — Answer: B.** Obj 7.3. `FlattenLayerStack` collapses the **layer stack** (sublayers) and leaves references. `Flatten()` bakes references (A is backwards). Review: Ch 31, Ch 34.

**M1-028 — Answer: B.** Obj 1.3. Verified: `LoadNone` still loads **references** (`/Hero/Shade` true) and defers **payloads** (`/Heavy/Shade` false). Review: Ch 17.

**M1-029 — Answer: B.** Obj 6.1. Verified: `DefinePrim` of a brand-new child inside `Sdf.ChangeBlock` can raise `Tf.ErrorException`. Review: Ch 34, Ch 44.

**M1-030 — Answer: B.** Obj 4.3. Verified: USDZ members use `compress_type 0` (`ZIP_STORED`). Review: Ch 31.

**M1-031 — Answer: B.** Obj 2.1. `Usd.Relationship.AddTarget` prepends by default (`ListPositionFrontOfPrependList`). Existing `protoIndices` of 0 silently retarget. Append a new proto with `Usd.ListPositionBackOfAppendList`. Review: Ch 25.

**M1-032 — Answer: B.** Obj 1.1. Verified: referenced 2 beats specializes 7. M1-009 used **inherits** (which win); this stem uses **specializes** (which lose). Review: Ch 19, Ch 21.

**M1-033 — Answer: A.** Obj 5.2. Verified: stronger local **default** 9.0 hides weaker samples (`GetTimeSamples()` is []). Samples do not automatically beat a stronger default. Review: Ch 10, Ch 21.

**M1-034 — Answer: B.** Obj 3.4. Opening `schema.usda` parses a layer; it does **not** register Usd schema types. `Plugins + Plug.Registry / PXR_PLUGINPATH_NAME` do. Review: Ch 37.

**M1-035 — Answer: A.** Obj 7.6. `customData` is the dictionary for producer keys renderers should ignore. Do not overload `kind`, `upAxis`, or `displayColor`. Review: Ch 12.

**M1-036 — Answer: B.** Obj 1.8. When an over “does nothing,” walk LIVERPS and the layer stack (mute, edit target, stronger local/inherit/variant). Do not reach for removed APIs. Review: Ch 22, Ch 42.

**M1-037 — Answer: A.** Obj 8.4. `UsdPrimvarReader_float3` with `inputs:varname = displayColor` feeds `PreviewSurface` diffuse. `UVTexture` reads `st` for textures, not `displayColor`. Review: Ch 40.

**M1-038 — Answer: A, B.** Obj 6.5. This pip wheel typically reports **0** malloc-tag bytes. `Trace Collector/Reporter` still works. `Trace` does not require `UsdImaging` to import. Review: Ch 43.

**M1-039 — Answer: A, C.** Obj 4.4. Validate identifiers and keep the original name in `customData`. Author `units/upAxis` and document that **referencing does not auto-convert**. Dropping `defaultPrim` (B) causes `UnresolvedPrimPath` on nameless references. Review: Ch 27, Ch 29.

**M1-040 — Answer: B.** Obj 1.1. Verified: prepended `r1` (`radius = 2`) beats appended `r2` (8). Review: Ch 14.

**M1-041 — Answer: B.** Obj 5.3. `customData` is a dictionary; `documentation / userDocBrief` are the human-readable metadata fields. Review: Ch 5, Ch 12.

**M1-042 — Answer: B.** Obj 2.4. Verified: `GetKind()` is component but `IsComponent()` is **False** because `/Clutter` is not a model — the model hierarchy is broken. Review: Ch 6, Ch 23.

**M1-043 — Answer: B.** Obj 7.4. Plugin ABI must match the USD the DCC loaded. `Usd.GetVersion() == (0, 26, 8)` describes this venv, not a free pass for older plugins. Review: Ch 35.

**M1-044 — Answer: B.** Obj 1.2. Use `Usd.Stage.OpenMasked` with a `StagePopulationMask`. `Open(path, mask=...)` is not the API (`LoadNone` is a load rule, not a mask). Review: Ch 17, Ch 44.

**M1-045 — Answer: A.** Obj 6.5. Install `UsdUtils.CoalescingDiagnosticDelegate` **before** `Open` so warnings group. Review: Ch 43.

**M1-046 — Answer: B.** Obj 4.5. Verified: `USDA rel material:binding` without `apiSchemas` → `HasAPI` **False**, `HasRelationship` **True**. Review: Ch 29, Ch 40.

**M1-047 — Answer: A, B.** Obj 8.3. `preview` is the lookdev purpose; empty `allPurpose` is the fallback. `weakerThanDescendants` lets **child** binds win, not the ancestor (C reversed). `physics` is not a required `UsdShade` purpose (D). Review: Ch 40.

**M1-048 — Answer: B.** Obj 3.7. `SceneIndex / Hydra` imaging is not in the `usd-core` wheel (`UsdImaging`, `Hd` absent). Review: Ch 38.

**M1-049 — Answer: A, C.** Obj 1.5. Per-department sublayers plus edit targets. One shared root (B) and muting the root (D) fight collaboration. Review: Ch 15, Ch 22.

**M1-050 — Answer: A.** Obj 5.2. Verified: linear `Get(4) = (4, 0, 0)`; held `Get(4) = (0, 0, 0)` (holds the previous sample). Review: Ch 10.

**M1-051 — Answer: A, C.** Obj 7.7. Naive `Resolve("./...")` is `cwd`; composition anchors to the layer. Use `ComputeAbsolutePath / CreateIdentifier`. `AnchorRelativePath` is **gone** in 26.08 (B). Paths starting `./` are **not** search-path lookups (D). Review: Ch 33.

**M1-052 — Answer: B.** Obj 2.2. Color the **instance root** (or inherit onto that instance). `OverridePrim` on a descendant proxy **raises**. Editing the prototype tints every instance. Review: Ch 24.

**M1-053 — Answer: B.** Obj 1.9. Verified: no `defaultPrim` → `GetModelNameFromRootLayer` returns the **first root prim** (`First`). Empty layer returns `''`. Review: Ch 16, Ch 32.

**M1-054 — Answer: A, C.** Obj 4.6. `UsdValidation` imports. `ComplianceChecker` is **gone**. The pip wheel ships **no** `usdchecker` CLI. Review: Ch 30.

**M1-055 — Answer: B.** Obj 6.2. Same LIVERPS fact as M1-001/M1-032, framed as a debug: specializes lose to references, so the artist still sees 1. Review: Ch 21, Ch 42.

**M1-056 — Answer: A, B.** Obj 3.5. Custom Ar plugin + `plugInfo.json` + plugin path; `GetResolver()` stays the facade. Kinds do not register resolvers. Review: Ch 33, Ch 38.

**M1-057 — Answer: A, C.** Obj 1.11. Split heavy geo behind payloads; department sublayers; pin published USDC. Daily Flatten-only (D) throws composition away. Review: Ch 22.

**M1-058 — Answer: B.** Obj 4.1. `glTF` has no LIVERPS; materials, units, and hierarchy need an explicit mapping. Review: Ch 27.

**M1-059 — Answer: B.** Obj 7.2. Pin a **versioned** identifier that still resolves the same bytes. Floating `latest/` is the classic break. Review: Ch 32.

**M1-060 — Answer: B.** Obj 5.5. Verified: `CreateInMemory` fallbacks are upAxis **Y** and `metersPerUnit` **0.01**. Unexpected small/Y-up assets often start here. Review: Ch 2, Ch 13.

**M1-061 — Answer: B.** Obj 4.8. `.usd` `formatId` follows the **extension plugin**; the bytes may still be USDA text (`#usda 1.0`) or crate (`PXR-USDC`). Review: Ch 7, Ch 31.



**M1-062 — Answer: B.** Obj 2.5. Remove or block properties at the **instance root** (or descendant that one root). Descendant proxy `OverridePrim` is not the edit site. Review: Ch 22, Ch 24.

**M1-063 — Answer: B.** Obj 6.4. Imageable **purpose** filters what a viewer draws. A proxy `gprim` is hidden when the viewer shows only `render`. Review: Ch 13, Ch 42.

**M1-064 — Answer: A, B.** Obj 4.7. Exporters should set `defaultPrim`, `metersPerUnit`, and `upAxis`. Do not invent USDA for `Kind.Registry.Register` or `GetMaster`. Review: Ch 28, Ch 29.

**M1-065 — Answer: A, C.** Obj 7.8. Map to typed prims/API schemas; keep unknowns in custom attributes/`customData`; honor payloads vs references. Do not Flatten as the only import (loses non-destructive round-trip). `UsdMtlx` is not in `usd-core`. Review: Ch 29, Ch 35.

---

### Domain repair (if you missed several)

If weak in	Re-read	Re-lab
COMP	Ch 14-22	Labs 13-21
DE	Ch 27-31	Labs 25-28
PD	Ch 32-35	Labs 29-30
DM	Ch 8-13	Labs 08-12
DBG	Ch 42-44	Labs 09, 30, 36
CA	Ch 23-26	Labs 22-24
VIS	Ch 39-41	Labs 33-35
CUST	Ch 36-38	Lab 31

# Mock Exam 2 — NCP-OUSD Practice Paper

**Original practice exam. Not actual NVIDIA exam content.**

Verified against **USD 26.08** (usd-core 26.8). Difficulty: **medium-hard** (paper 2 of 3). Do not reuse Mock 1 answers by memory — several stems look similar and are not.

Questions	65
Time	120 minutes
Blueprint mix	COMP 15 · DE 10 · PD 9 · DM 8 · DBG 7 · CA 7 · VIS 5 · CUST 4
Answers	mock-exams/mock02_answers.md — <b>do not open until you finish</b>

USDA is always multiline. Python stems are .norun.

**M2-001** · COMP · Obj 1.5 · Difficulty: Hard · Type: USDA-reading

File: paint.usda

```
#usda 1.0
over "Pier"
{
 double length = 6
}
```

File: layout.usda

```
#usda 1.0
over "Pier"
{
 double length = 15
}
```

File: harbor.usda

```
#usda 1.0
(
 subLayers = [
 @./paint.usda@,
 @./layout.usda@
]
)
def Xform "Pier"
{
}
```

Composed length is 6. The TD mutes paint.usda. What is length now?

A. 6 — mute cannot change composed values B. 15 — the remaining weaker sublayer is now strongest C. None — muting any sublayer invalidates the prim D. 21 — USD adds the two opinions once mute “unlocks” addition

**M2-002** · DM · Obj 5.4 · Difficulty: Hard · Type: Python-reading

```
from pxr import Usd
stage = Usd.Stage.CreateInMemory()
p = stage.GetPrimAtPath("/Missing")
print(bool(p), p.IsValid())
p.IsDefined()
```

What happens?

A. Prints True True, then IsDefined() returns False B. Prints False False, then IsDefined() **raises RuntimeError** C. Prints False False, then IsDefined() returns False D. GetPrimAtPath raises before print

**M2-003** · DE · Obj 4.3 · Difficulty: Medium · Type: Multiple select Select two.

On USD 26.08, Tf.MakeValidIdentifier produces:

A. "2ball" → "\_ball" B. "Pier-Dock" → "Pier\_Dock" C. "2ball" is already legal and is returned unchanged D. Hyphens are kept ("Pier-Dock" unchanged)

---

**M2-004** · COMP · Obj 1.1 · Difficulty: Hard · Type: USDA-reading

```
#usda 1.0
def Sphere "Buoy" (
 prepend variantSets = "sea"
 variants = {
 string sea = "storm"
 }
)
{
 double radius = 3
 variantSet "sea" = {
 "storm" {
 double radius = 11
 }
 "calm" {
 double radius = 1
 }
 }
}
```

Composed radius?

A. 11 — the selected variant always beats a local opinion B. 3 — **local** beats VariantSets in LIVERPS C. 1 — USD picks the last variant block D. None — local plus a selection is illegal

---

**M2-005** · CA · Obj 2.1 · Difficulty: Hard · Type: Python-reading

A PointInstancer already has prototype /PI/Bush at index 0. Which call puts /PI/Tree at **index 0** (Bush shifts to 1)?

A. prototypes.AddTarget("/PI/Tree") with no position (observed order on 26.08: Bush stays 0) B. prototypes.AddTarget("/PI/Tree", position=Usd.ListPositionFrontOfPrependList) C. prototypes.AddTarget("/PI/Tree", position=Usd.ListPositionBackOfAppendList) D. prototypes.SetTargets([]) then hope indices rewrite themselves

---

**M2-006** · PD · Obj 7.5 · Difficulty: Hard · Type: Multiple select Select two.

A validator does Ar.GetResolver().Resolve("./maps/rust.png") from a random cwd and gets empty, while usdview shows the texture on a prim in assets/cleat.usda. Which pair is true on 26.08?

A. Composition anchored the asset path to **the layer**; naive Resolve("./...") uses **process cwd** B. AnchorRelativePath is the supported 26.08 replacement on Ar.Resolver C. layer.ComputeAbsolutePath("./maps/rust.png") (or CreateIdentifier + the layer's ResolvedPath) is the layer-relative check D. Identifiers starting with ./ are looked up on the resolver **search path**

---

**M2-007** · VIS · Obj 8.2 · Difficulty: Hard · Type: Python-reading

```
from pxr import Usd, UsdLux
s = Usd.Stage.CreateInMemory()
sun = UsdLux.DistantLight.Define(s, "/Sun")
sun.GetIntensityAttr().Set(2)
sun.GetExposureAttr().Set(3)
print(sun.GetIntensityAttr().Get() * (2 ** sun.GetExposureAttr().Get()))
```

What prints (UsdLux brightness model: intensity × 2<sup>exposure</sup>)?

A. 5 B. 6 C. 16 D. 8

---

**M2-008** · DBG · Obj 6.2 · Difficulty: Hard · Type: Single choice

GetPropertyStack() on /Pier.length returns specs from paint.usda (default 6) then layout.usda (default 15). What does that order mean?

- A. Weakest first — layout wins B. **Strongest first** — paint's 6 is the composed value C. Alphabetical by layer name D. Stacks always include only the root layer

---

**M2-009** · COMP · Obj 1.8 · Difficulty: Hard · Type: USDA-reading

File: skiff\_a.usda

```
#usda 1.0
(
 defaultPrim = "Skiff"
)
def Sphere "Skiff"
{
 double radius = 3
}
```

File: skiff\_b.usda

```
#usda 1.0
(
 defaultPrim = "Skiff"
)
def Sphere "Skiff"
{
 double radius = 11
}
```

File: work.usda

```
#usda 1.0
def "Skiff" (
 prepend references = @./skiff_a.usda@
 append references = @./skiff_b.usda@
)
{
}
```

File: shot.usda

```
#usda 1.0
(
 subLayers = [
 @./work.usda@
]
)
over "Skiff" (
 delete references = @./skiff_a.usda@
)
{
}
```

Composed radius on /Skiff?

- A. 3 — delete cannot remove a prepend B. 11 — the stronger layer's **delete** drops skiff\_a; the remaining append wins C. None — delete of a reference is illegal D. 7 — the mean

---

**M2-010** · DM · Obj 5.1 / 8.1 · Difficulty: Hard · Type: Single choice

Parent /P authors constant primvars:displayColor = [(0, 1, 0)]. Child mesh /P/M does not. How does the child read that primvar?

- A. child.GetAttribute("displayColor") B. UsdGeom.PrimvarsAPI(child).FindPrimvarWithInheritance("displayColor") C. Inheritance of primvars is impossible; copy the array onto every mesh D. UsdShade.MaterialBindingAPI(child).GetDisplayColor()

---

**M2-011** · DE · Obj 4.6 · Difficulty: Medium · Type: Single choice

`UsdUtils.ComputeAllDependencies("entry.usda")` on a layer that sublayers `geo.usda` (asset `@./tex/wood.png@`) and authors `@./gone.jpg@` (missing). The 3-tuple's last element contains:

- A. `wood.png` only B. `gone.jpg` (unresolved) C. Both PNG and JPG as unresolved D. Always `[]` — Compute never reports missing files

---

**M2-012** · COMP · Obj 1.7 · Difficulty: Hard · Type: Python-reading

```
vs = prim.GetVariantSet("look")
print(repr(vs.GetVariantSelection()))
with vs.GetVariantEditContext():
 prim.CreateAttribute("mark", Sdf.ValueTypeNames.Double).Set(7)
```

Selection is still `''`. Where is `mark = 7` authored?

- A. Inside every variant B. Inside `"look"` automatically C. As a **local** opinion on the prim (empty selection → context is not a variant) D. The `with` block raises

---

**M2-013** · PD · Obj 7.2 · Difficulty: Medium · Type: Single choice

`UsdUtils.ExtractExternalReferences("set.usda")` where the file **references** `lamp.usda` and **payloads** `crate.usda` (no sublayers) returns:

- A. (layers, assets, unresolved) B. (`[]`, `['./lamp.usda']`, `['./crate.usda']`) — (sublayers, refs+assets, payloads) C. A flat list of strings D. Only payloads, because Extract ignores references

---

**M2-014** · CUST · Obj 3.4 · Difficulty: Hard · Type: Single choice

You need a **new typed Gprim** (HarborBuoy) versus a **non-destructive API** (HarborTagAPI) that can apply to existing meshes. Which bases?

- A. Both inherit `UsdSchemaBase` only B. Gprim: `UsdTyped` (often via `UsdGeomGprim`); API: `UsdAPISchemaBase` C. Gprim: `UsdAPISchemaBase`; API: `UsdTyped` D. Both must inherit `UsdPhysicsBase`

---

**M2-015** · CA · Obj 2.3 · Difficulty: Medium · Type: Multiple select Select two.

Hiding `PointInstancer` instances efficiently:

- A. Author `invisibleIds` (or activate an invis-id mask) — instance count **stays** the same B. Delete those indices from `protoIndices` every frame C. Deactivate the whole `PointInstancer` prim to hide one id D. `invisibleIds` is a mask; it does not shrink `GetInstanceCount()`

---

**M2-016** · COMP · Obj 1.4 · Difficulty: Hard · Type: Python-reading

A shot sublayers `anim.usda` with `(offset = 10)`. You `SetEditTarget(stage.GetEditTargetForLocalLayer(anim))` then `attr.Set(1.0, 20)`. Where is the sample stored in `anim.usda`?

- A. `t=20` — edit targets ignore offsets B. `t=10` — `GetEditTargetForLocalLayer` maps stage time through the offset (`stageTime - offset` here) C. `t=30` — USD adds offset twice D. The Set raises because offsets forbid authoring

---

**M2-017** · DM · Obj 5.6 · Difficulty: Medium · Type: Single choice

`UsdGeom.Mesh.Define` then `GetSubdivisionSchemeAttr().Get()` before anyone authors the attribute:

- A. None B. `none` C. `catmullClark` (schema fallback) D. `bilinear`

---

**M2-018** · VIS · Obj 8.2 · Difficulty: Hard · Type: Multiple select Select two.

`UsdLux.DistantLight.Define(stage, "/Sun")` on a fresh stage:

A. HasAPI(UsdLux.LightAPI) is **True** (LightAPI is built in on the typed light) B. HasAPI(UsdLux.ShadowAPI) is **False** until ShadowAPI.Apply C. ShapingAPI is also built in (HasAPI True with no Apply) D. DistantLight cannot apply ShadowAPI

**M2-019** · DBG · Obj 6.3 · Difficulty: Hard · Type: Single choice

```
#usda 1.0
def Xform "Alpha"
{
}
def Xform "Beta"
{
}
```

A shot does `prepend references = @./node.usda@` with **no** `defaultPrim` on that file and **no** explicit prim path. The composition error is:

A. `InvalidAssetPath` — the file cannot be opened B. `UnresolvedPrimPath` / unresolved **defaultPrim** on an opened layer C. Silent success; `/Hero` becomes `Alpha` D. `Tf.MallocTag`

**M2-020** · COMP · Obj 1.3 · Difficulty: Hard · Type: Single choice

Same file referenced by `/A` and payloaded by `/B`. `Usd.Stage.Open(path, Usd.Stage.LoadNone)`:

A. `/A` child geo missing; `/B` child geo present B. `/A` child geo **present** (references still load); `/B` child geo **missing** C. Both missing D. `Open(..., LoadNone)` is not valid — must use `OpenMasked`

**M2-021** · DE · Obj 4.7 · Difficulty: Hard · Type: Multiple select Select two.

`stage.Flatten()` of a shot that references `lantern.usda` and sublayers notes:

A. Flattened lantern prim **no longer holds** the reference list (baked) B. Department notes from the sublayer **are baked** into the flattened root C. Flatten **keeps** live references and only drops sublayers D. Flatten refuses if any attribute has time samples

**M2-022** · PD · Obj 7.4 · Difficulty: Medium · Type: Single choice

`stage.SetEditTarget(Sdf.Layer.CreateAnonymous("elsewhere"))` when that layer is **not** in the local layer stack:

A. Silently authors into the anonymous layer B. Raises `Tf.ErrorException` (not in the local LayerStack) C. Redirects to the session layer D. Converts the stage to USDA

**M2-023** · CA · Obj 2.4 · Difficulty: Hard · Type: USDA-reading

```
#usda 1.0
def "Dock"
{
}

over "Jetty"
{
}
```

Which pair matches USD 26.08?

A. `/Dock` is defined (typeless); `/Jetty` `IsDefined()` is **False** but `bool(prim)` can still be **True** B. Both are null prims (`bool False`) C. `/Jetty` is defined because `over` always defines D. `/Dock` fails to parse without a type

**M2-024** · COMP · Obj 1.8 · Difficulty: Hard · Type: USDA-reading

File: `lantern.usda`

```
#usda 1.0
(
 defaultPrim = "Lantern"
```

```

)
def Xform "Lantern"
{
 def Sphere "Globe"
 {
 double radius = 1
 }
}

```

File: `shot.usda`

```

#usda 1.0
def "Hero" (
 prepend references = @./lantern.usda@
)
{
 over "Globe"
 {
 double radius = 4
 }
}

```

Composed `/Hero/Globe.radius`?

- A. 1 — you cannot over a referenced child B. 4 — a local over on the composed child path wins C. None — name mismatch with defaultPrim D. 5 — sum

**M2-025** · DM · Obj 5.2 · Difficulty: Medium · Type: Single choice

A vertex `primvars:displayColor` has 2 color samples and `SetIndices([0, 1, 0, 1])`. `IsIndexed()` is:

- A. False — indices are only for `faceVertexIndices` B. True — four corners share two stored colors C. Illegal — index length must equal face count D. True only for uniform interpolation

**M2-026** · VIS · Obj 8.4 · Difficulty: Hard · Type: Single choice

`UsdPreviewSurface` reads a **UV** texture. Which reader/texture pair is the usual network?

- A. `UsdPrimvarReader_float2 (varname = st) → UsdUVTexture → PreviewSurface diffuseColor` B. `UsdPrimvarReader_float3 (varname = st)` C. `UsdUVTexture` with `varname = displayColor` and no reader D. `UsdMtlx` node, because `usd-core` always includes `MaterialX`

**M2-027** · PD · Obj 7.3 · Difficulty: Medium · Type: Single choice

You need a **single layer** that still **references** published components (not baked), with department sublayers collapsed. API?

- A. `stage.Flatten()` B. `UsdUtils.FlattenLayerStack(stage)` C. `Usd.Stage.OpenMasked` D. `Sdf.ChangeBlock`

**M2-028** · COMP · Obj 1.2 · Difficulty: Medium · Type: Single choice

`Usd.Stage.Open("huge.usda", mask=["/World/Ferry"])` on `usd-core 26.8`:

- A. Opens only `/World/Ferry` B. **Fails / TypeError** — population masks use `OpenMasked + Usd.Stage.PopulationMask`, not an `Open(..., mask=)` kwarg C. Equivalent to `LoadNone` D. Equivalent to muting every other layer

**M2-029** · DBG · Obj 6.1 · Difficulty: Hard · Type: Single choice

You must author a **new** child `/Harbor/Winch` (no spec yet) while a `Sdf.ChangeBlock` is already active. Safest 26.08 practice?

A. `DefinePrim` inside the block — always succeeds B. Close the block (or never open it) before `DefinePrim` of a brand-new path; defining a new child **inside** a live block can raise `Tf.ErrorException` C. Switch to `OverridePrim` inside the block to create defs D. `ChangeBlock` only wraps `MuteLayer`

---

**M2-030** · DE · Obj 4.2 · Difficulty: Medium · Type: Single choice

A mapping doc must relate `UsdPreviewSurface` `diffuseColor` to a `MaterialX` / glTF base color. On this `usd-core` venv, from `pxr` import `UsdMtlx`:

A. Succeeds; `UsdMtlx` is in `usd-core` 26.8 B. **ImportError** — conceptual mapping is still required; the `MaterialX` USD plugin is not in this wheel C. Succeeds only after `Kind.Registry.Register` D. Succeeds and registers `Hydra SceneIndexes`

---

**M2-031** · CA · Obj 2.1 · Difficulty: Hard · Type: Python-reading

```
rel.AddTarget("/PI/Bush")
rel.AddTarget("/PI/Tree", position=Usd.ListPositionBackOfAppendList)
rel.AddTarget("/PI/Rock") # default position
print(list(rel.GetTargets()))
```

On 26.08 this prints:

A. `['/PI/Rock', '/PI/Bush', '/PI/Tree']` B. `['/PI/Bush', '/PI/Rock', '/PI/Tree']` C. `['/PI/Bush', '/PI/Tree', '/PI/Rock']` D. `['/PI/Tree', '/PI/Bush', '/PI/Rock']`

---

**M2-032** · COMP · Obj 1.6 · Difficulty: Hard · Type: Multiple select Select two.

LIVERPS two-step resolution:

A. Strength **inside a layer stack** (sublayer order, list-ops) is decided first for each arc B. Then arcs are compared: Local > Inherits > VariantSets > rElocates > References > Payloads > Specializes C. Specializes beat references D. Value clips are the “C” in LIVERPS

---

**M2-033** · DM · Obj 5.5 · Difficulty: Hard · Type: Single choice

Parent `/H` has `visibility = invisible`. Child cube `/H/C` never authors visibility. `GetVisibilityAttr().Get()` on the child vs `ComputeVisibility()`:

A. Both invisible B. `Get` → inherited; `Compute` → invisible C. `Get` → invisible; `Compute` → inherited D. Both None

---

**M2-034** · CUST · Obj 3.1 · Difficulty: Medium · Type: Single choice

A C++ schema plugin built against USD 24.11 is `dlopen`'d into a process that loaded `usd-core` **26.8**. Likely result?

A. Always fine — Python wheels hide ABI B. **ABI mismatch** — plugins must match the USD the process loaded C. Fine if `plugInfo.json` lists both versions D. Fine if you `MuteLayer` the plugin path

---

**M2-035** · PD · Obj 7.6 · Difficulty: Medium · Type: Single choice

Read a producer key stored as `customData["show"] = "harbor"`:

A. `prim.GetMetadata("show")` B. `prim.GetCustomDataByKey("show")` (or `GetCustomData()["show"]`) C. `prim.GetKind()` D. `Ar.GetResolver().Resolve("show")`

---

**M2-036** · COMP · Obj 1.9 · Difficulty: Hard · Type: Single choice

A layer has **no** `defaultPrim`, root prims `Ferry` then `Barge`. `UsdUtils.GetModelNameFromRootLayer(layer)` returns:

A. `''` B. `'Ferry'` (first root prim fallback) C. `'Barge'` D. Raises

---

**M2-037** · VIS · Obj 8.3 · Difficulty: Hard · Type: Single choice



A parent mesh binds a material `strongerThanDescendants`. A child subset also binds a different material. Who wins on the subset?

A. The child subset — descendants always win B. The **parent** bind — `strongerThanDescendants` beats descendant bindings C. Neither — USD averages shader parameters D. Binding strength is ignored unless `HasAPI` is false

**M2-038** · DBG · Obj 6.5 · Difficulty: Medium · Type: Multiple select Select two.

Live diagnostics while `Open` spews repeated asset warnings:

A. Install `UsdUtils.CoalescingDiagnosticDelegate` **before** `Open` B. `Tf.Debug.SetDebugSymbolsByName("USD_CHANGES", True)` (or `TF_DEBUG=USD_CHANGES`) for change chatter C. `UsdGeom.Tokens.Z` enables diagnostics D. `GetMaster()` dumps the layer stack

**M2-039** · DE · Obj 4.4 · Difficulty: Hard · Type: Multiple select Select two.

Round-tripping a DCC node named "2ball":

A. Export as `Tf.MakeValidIdentifier` → `_ball`; stash "2ball" in `customData` B. Keep the USDA identifier `2ball` — leading digits are legal prim names C. Author `metersPerUnit / upAxis` and document that **references do not auto-convert** D. Skip `defaultPrim` so the importer can guess

**M2-040** · COMP · Obj 1.1 · Difficulty: Hard · Type: USDA-reading

File: `asset.usda` — `def Xform "Winch" { double load = 2 } with defaultPrim = "Winch".`

Shot:

```
#usda 1.0
class "_Load"
{
 double load = 11
}

def "Winch" (
 prepend references = @./asset.usda@
 prepend specializes = </_Load>
)
{
}
```

Composed `load`?

A. 11 — specializes beat references B. 2 — references beat specializes C. None D. 13

**M2-041** · DM · Obj 5.3 · Difficulty: Medium · Type: Single choice

`userDocBrief` vs `customData["assetId"]`:

A. Same field B. `userDocBrief` / documentation are human-facing metadata; `customData` is a producer dictionary C. `userDocBrief` registers a schema D. `customData` can only live on the root layer metadata, never on a prim

**M2-042** · CA · Obj 2.2 · Difficulty: Hard · Type: Single choice

Tint **one** native instance without `SetInstanceable(False)` and without editing the shared prototype mesh:

A. `stage.OverridePrim("/Instance/Mesh")` and set `displayColor` on the proxy B. Author `primvars:displayColor` (or an `inherit`) on the **instance root** C. Edit `/__Prototype_1/Mesh` — that affects only that instance D. Set `kind = "subcomponent"` on the proxy

**M2-043** · PD · Obj 7.7 · Difficulty: Hard · Type: Single choice

PXR\_PLUGINPATH\_NAME is how a pipeline injects plugInfo.json trees. If it is unset, Plug.Registry.GetInstance().RegisterPlugins("/studio/plugin") from Python:

A. Cannot work — the env var is the only mechanism B. Can still register a path at runtime (lab pattern); the env var is the process-wide default search C. Registers kinds only, not file formats D. Silently installs SceneIndex plugins from usd-core

**M2-044** · COMP · Obj 1.7 · Difficulty: Medium · Type: Single choice

prim.GetVariantSets().GetNames() vs prim.GetVariantSet("look").GetVariantNames():

A. They are aliases B. GetNames() → variant **set** names (['look']); GetVariantNames() → names **inside** that set (['green', 'red']) C. GetNames() lists variant values; GetVariantNames() lists set names D. Only GetMaster() lists variants

**M2-045** · DBG · Obj 6.4 · Difficulty: Hard · Type: Single choice

A mesh is present in Traverse but missing in usdview. purpose is guide; the viewer shows **default+render** only. Cause?

A. kind must be group B. **Purpose filter** — guide is excluded from that viewer mask C. Missing extent always culls guide purpose D. DistantLight intensity 50000 hides guides

**M2-046** · DE · Obj 4.8 · Difficulty: Medium · Type: Multiple select Select two.

A file named hero.usd starts with #usda 1.0. Which are true on 26.08?

A. GetFileFormat().formatId is usd (the **extension** plugin) B. The layer still opens; prims and values are there (contents are USDA text) C. formatId becomes usda because the first line is #usda D. USD refuses to open USDA bytes under a .usd name

**M2-047** · VIS · Obj 8.2 · Difficulty: Medium · Type: Multiple select Select two.

Fresh UsdLux.DistantLight color-temperature fallbacks on 26.08:

A. GetEnableColorTemperatureAttr().Get() is False B. GetColorTemperatureAttr().Get() is 6500 C. Enable defaults to True, so Kelvin always affects the light D. Color temperature fallback is None until set

**M2-048** · CUST · Obj 3.8 · Difficulty: Hard · Type: Single choice

A procedural Ar resolver should generate **in-memory** renderable prims from proc:// URIs. That job is:

A. The same as a Hydra SceneIndex (and ships in usd-core) B. An **Ar resolver plugin** (identifier → asset / layer bytes); SceneIndex is a separate imaging-time job and needs an imaging build C. Kind.Registry callbacks D. Sdf.ChangeBlock only

**M2-049** · COMP · Obj 1.5 · Difficulty: Hard · Type: Multiple select Select two.

Authoring into a department layer without clobbering the shot root:

A. stage.SetEditTarget(departmentLayer) when that layer is in the local stack B. Usd.EditContext(stage, departmentLayer) for a scoped change that restores the previous target C. Always CreateNew a second root layer and hope composition finds it D. GetPrototype() switches edit targets

**M2-050** · DM · Obj 5.2 · Difficulty: Hard · Type: Python-reading

Samples at 2 → (0,0,0) and 8 → (6,0,0). Interpolation **linear**. attr.Get(4)?

A. (0, 0, 0) B. (2, 0, 0) C. (6, 0, 0) D. None

**M2-051** · PD · Obj 7.1 · Difficulty: Medium · Type: Single choice

Python stand-in for usdcat layer.usda -o out.usda in this venv (no CLI tools):

A. `os.system("usdcat ...")` — the binary ships with usd-core B.  
`Sdf.Layer.FindOrOpen(...).Export("out.usda")` (or `stage.GetRootLayer().Export()`) C.  
`UsdUtils.CreateNewUsdzPackage` only D. `Kind.Registry.GetAllKinds()`

---

**M2-052** · CA · Obj 2.4 · Difficulty: Medium · Type: Multiple select Select two.

`UsdGeom.PointInstancer` on USD 26.08:

A. It is a `UsdGeomGprim` (a `gprim`) B. It is `UsdGeomBoundable` C. It is **not** a `Gprim` D. It is a `UsdShadeNodeGraph`

---

**M2-053** · COMP · Obj 1.11 · Difficulty: Hard · Type: Multiple select Select two.

Splitting a monolithic `trawler.usda` (geo + look + rig) for parallel work:

A. Payload heavy geo; keep a small interface layer with `defaultPrim` B. Department **sublayers** for look vs rig; publish pinned USDC C. One shared root layer with no arcs so Git never merges D. Replace daily work with `Flatten()` so composition never runs

---

**M2-054** · DE · Obj 4.5 · Difficulty: Hard · Type: Single choice

An exporter writes extra `custom float studio:lod = 2` on a Mesh (no studio schema plugin loaded downstream). Downstream `GetAttribute("studio:lod").Get()` ?

A. Always `None` — unknown namespaces are dropped B. The **custom attribute still composes** as a property; the typed schema is optional for storage C. USD refuses to open the layer D. It is converted to `primvars:studio:lod` automatically

---

**M2-055** · DBG · Obj 6.2 · Difficulty: Hard · Type: Single choice

A department over on `/Hero/Globe.radius` does not show. `GetPropertyStack` lists the over, then a **local** default on a stronger sublayer, then the referenced sample. Why is the over losing?

A. Overs cannot override references B. A **stronger local** opinion in a stronger stack position beats the department over (same LIVERPS step: local layer-stack strength) C. References always beat local D. `GetPropertyStack` is unordered trivia

---

**M2-056** · CUST · Obj 3.2 · Difficulty: Medium · Type: Multiple select Select two.

Building OpenUSD from source for a DCC that needs MaterialX and Hydra:

A. usd-core **pip** is not that build — it has no `UsdMtlx`, `UsdImaging`, or CLI B. Enable the imaging/MaterialX options in the OpenUSD build; match the DCC's compiler/ABI C. `Kind.Registry.Register` replaces `CMake` D. Copying `plugInfo.json` from usd-core enables Hydra

---

**M2-057** · COMP · Obj 1.10 · Difficulty: Hard · Type: Single choice

Remove a property from **one** instanced component in an assembly without de-instancing every copy:

A. `OverridePrim` on the instance-proxy mesh descendant B. Author at the **instance root** (or de-instance **that** root only) C. Mute the assembly root layer D. `GetMaster().Clear()`

---

**M2-058** · DE · Obj 4.1 · Difficulty: Medium · Type: Single choice

glTF export of a USD asset with variants and payloads. The fidelity note should say:

A. glTF evaluates LIVERPS natively B. You must **compose or bake** a chosen working set (variant selections, loaded payloads) — glTF has no payloads/variants C. USDZ and glTF are the same bytes D. `metersPerUnit` is ignored in every converter

---

**M2-059** · PD · Obj 7.8 · Difficulty: Hard · Type: Multiple select Select two.

DCC importer design on 26.08:

A. Map DCC nodes to typed prims / applied API schemas; keep unknowns as custom properties or customData B. Honor **payload vs reference** so the DCC can load a working set C. `import pxr.UsdMtlx` is guaranteed D. Call `GetMaster()` for every instance

---

**M2-060** · DM · Obj 5.5 · Difficulty: Medium · Type: Python-reading

```
from pxr import Usd, UsdGeom
xf = UsdGeom.Xform.Define(Usd.Stage.CreateInMemory(), "/X")
xf.AddTranslateOp().Set((1, 0, 0))
print(list(xf.GetXformOpOrderAttr().Get()))
```

Prints:

A. [] B. ['xformOp:translate'] C. ['translate'] D. None

---

**M2-061** · DE · Obj 4.3 · Difficulty: Medium · Type: Single choice

`UsdUtils.CreateNewUsdzPackage` ZIP `compress_type` on 26.08:

A. ZIP\_DEFLATED (8) B. ZIP\_STORED (0) C. ZIP\_BZIP2 D. Mixed per file type

---

**M2-062** · CA · Obj 2.5 · Difficulty: Hard · Type: Single choice

`UsdGeom.ModelAPI` draw modes (`cards`, `bounds`, ...) apply to:

A. Every gprim, including shapeless overs B. **Model** prims (component/assembly with a valid model hierarchy) as a cheap viz stand-in C. PointInstancer proto indices only D. Materials only

---

**M2-063** · DBG · Obj 6.5 · Difficulty: Medium · Type: Single choice

`Tf.MallocTag.GetTotalBytes()` on this pip wheel often returns 0. Why?

A. USD never allocates B. The wheel was **not built with malloc tagging**; use Trace for timelines instead of trusting bytes C. You must call `GetMaster()` first D. `MallocTag` only works inside `ChangeBlock`

---

**M2-064** · DE · Obj 4.7 · Difficulty: Medium · Type: Multiple select Select two.

A component exporter's root layer should include:

A. `defaultPrim` B. `metersPerUnit` and `upAxis` C. `Kind.Registry.Register` in the USDA header D. A `GetMaster` metadata block

---

**M2-065** · PD · Obj 7.2 · Difficulty: Hard · Type: Single choice

`ComputeAllDependencies` vs `ExtractExternalReferences` — which walks **closure** (sublayers + refs + payloads + texture assets, with unresolved listed separately)?

A. Extract (one file only) B. **ComputeAllDependencies** → (`layers`, `assets`, `unresolved`) C. Both are identical D. Only `FlattenLayerStack`

---

*End of Mock Exam 2. 65 questions. Check `mock-exams/mock02_answers.md` only after you have finished.*

# Mock Exam 2 — Answer Key

**Original practice exam. Not actual NVIDIA exam content.**

Paper: `mock-exams/mock02_paper.md`. Verified against **USD 26.08** (`usd-core` 26.8).

Scoring: 1 point per question (multiple-select must match **all** correct letters). Total **65**.

This book's Mock 2 review bar is  $\geq$  **75% overall (49/65)** and no domain under 60%.

NVIDIA does not publish a passing score.

## Score sheet

Domain	Q IDs	/n	Your score	%
Composition (COMP)	001, 004, 009, 012, 016, 020, 024, 028, 032, 036, 040, 044, 049, 053, 057	/15		
Data Exchange (DE)	003, 011, 021, 030, 039, 046, 054, 058, 061, 064	/10		
Pipeline Development (PD)	006, 013, 022, 027, 035, 043, 051, 059, 065	/9		
Data Modeling (DM)	002, 010, 017, 025, 033, 041, 050, 060	/8		
Debugging (DBG)	008, 019, 029, 038, 045, 055, 063	/7		
Content Aggregation (CA)	005, 015, 023, 031, 042, 052, 062	/7		
Visualization (VIS)	007, 018, 026, 037, 047	/5		
Customizing USD (CUST)	014, 034, 048, 056	/4		
<b>Total</b>	M2-001–M2-065	<b>/ 65</b>		

Copy weakest domains into `00_MASTER_PLAN.md` §9 before Mock 3.

## Answers

**M2-001 — Answer: B.** Obj 1.5. Verified: first sublayer 6; after `MuteLayer` of that identifier, composed `length` is **15**. `Mute` removes that layer from the stack; it does not average or invalidate the prim. Review: Ch 15.

**M2-002 — Answer: B.** Obj 5.4. A missing path is a null prim: `bool/IsValid` are **False**. `IsDefined()` on a null prim **raises `RuntimeError`**. Do not write `if prim.IsDefined()` without `if prim` first. Review: Ch 4, Ch 8.

**M2-003 — Answer: A, B.** Obj 4.3. Verified: `"2ball" → "_ball"; "Pier-Dock" → "Pier_Dock"`. Leading digits get an underscore prefix; hyphens become underscores. Review: Ch 4, Ch 29.

**M2-004 — Answer: B.** Obj 1.1. Verified: local `radius = 3` beats the selected variant's 11. Local is stronger than `VariantSets` in `LIVERPS`. Review: Ch 18, Ch 21.

**M2-005 — Answer: B.** Obj 2.1. Verified: **explicit** `ListPositionFrontOfPrependList` puts `Tree` at index 0. Default `AddTarget` on 26.08 left `Bush` at 0 (`[Bush, Tree]`). `BackOfAppendList` appends (keeps 0). Review: Ch 25.

**M2-006 — Answer: A, C.** Obj 7.5. Naive `Resolve("./...")` is `cwd`; composition anchors to the layer. `ComputeAbsolutePath / CreateIdentifier` is the layer-relative check. `AnchorRelativePath` is **gone**. `./` is not a search-path lookup. Review: Ch 33.

**M2-007 — Answer: C.** Obj 8.2. Verified:  $2 * 2^{**3} = 16$ . Brightness scales as  $\text{intensity} \times 2^{\text{exposure}}$ . Review: Ch 41.

**M2-008 — Answer: B.** Obj 6.2. Verified: `GetPropertyStack` is **strongest first** (paint.usda 6, then layout.usda 15). Review: Ch 22, Ch 42.

**M2-009 — Answer: B.** Obj 1.8. Verified: a **stronger** layer's `delete references = @./skiff_a.usda@` drops the prepend; remaining append `skiff_b` gives **11**. Same-layer `delete+prepend` of the same item does **not** remove it — the delete must win from a stronger list-op. Review: Ch 14, Ch 16.

**M2-010 — Answer: B.** Obj 5.1 / 8.1. Verified: `FindPrimvarWithInheritance("displayColor")` finds `/P.primvars:displayColor`. There is no `GetDisplayColor` on `MaterialBindingAPI`. Review: Ch 11.

**M2-011 — Answer: B.** Obj 4.6. Verified: unresolved list contains `gone.jpg`; `wood.png` is in **assets**, not unresolved. Review: Ch 32, Ch 42.

**M2-012 — Answer: C.** Obj 1.7. Verified: empty selection + `GetVariantEditContext()` authors a **local** custom double mark = 7, not inside `red/green`. Review: Ch 18.

**M2-013 — Answer: B.** Obj 7.2. Extract is one-file (sublayers, refs+assets, payloads). Compute is the closure 3-tuple. Review: Ch 32.

**M2-014 — Answer: B.** Obj 3.4. New `Gprim` → `UsdTyped` (typically via `UsdGeomGprim`). Non-destructive bundled properties → `UsdAPISchemaBase`. There is no `UsdPhysicsBase` for this. Review: Ch 6, Ch 37.

**M2-015 — Answer: A, D.** Obj 2.3. `invisibleIds` masks drawing; `GetInstanceCount()` **does not shrink**. Do not rewrite topology every frame or deactivate the whole instancer to hide one id. Review: Ch 25.

**M2-016 — Answer: B.** Obj 1.4. Verified: `GetEditTargetForLocalLayer` on a sublayer with `offset = 10` stores `Set(..., 20)` at **t=10**. A bare `Usd.EditTarget(anim)` stores 20. Review: Ch 15, Lab 19.

**M2-017 — Answer: C.** Obj 5.6. Verified: Mesh `subdivisionScheme` fallback is `catmullClark`. Review: Ch 13.

**M2-018 — Answer: A, B.** Obj 8.2. Verified: `DistantLight` **HasAPI** `LightAPI` without `Apply`; `ShadowAPI` (and `ShapingAPI`) need `Apply`. Review: Ch 41.

**M2-019 — Answer: B.** Obj 6.3. Verified warning: unresolved reference prim path `...@nodef.usda@<defaultPrim>`. The layer **opens**; `defaultPrim` is missing. That is not `InvalidAssetPath`. Review: Ch 16, Ch 42.

**M2-020 — Answer: B.** Obj 1.3. Verified: `LoadNone` still loads **references**; **payloads** stay unloaded. `OpenMasked` is a different API (population mask). Review: Ch 17.

**M2-021 — Answer: A, B.** Obj 4.7. `Flatten()` **bakes** references and sublayer opinions into one layer. `FlattenLayerStack` is the API that **keeps** references and drops sublayers (C describes FLS, not `Flatten`). Review: Ch 34.

**M2-022 — Answer: B.** Obj 7.4. Verified: anonymous layer not in the local stack → `Tf.ErrorException`. Review: Ch 15, Lab 19.

**M2-023 — Answer: A.** Obj 2.4. Verified: typeless `def "Dock"` is defined, type `''`. over `"Jetty"` with no defining spec: `IsDefined()` **False**, `bool(prim)` **True** (not a null prim). Null missing paths are `bool False`. Review: Ch 20.

**M2-024 — Answer: B.** Obj 1.8. Verified: local over `"Globe" { radius = 4 }` on the referencing prim wins over the asset's 1. Review: Ch 16, Ch 20.

**M2-025 — Answer: B.** Obj 5.2. Verified: indexed primvar — 2 stored colors, 4 indices. Review: Ch 11.

**M2-026 — Answer: A.** Obj 8.4. `UsdPrimvarReader_float2 + st` → `UsdUVTexture` → `PreviewSurface`. `UsdMtlx` is not in `usd-core`. Review: Ch 40.

**M2-027 — Answer: B.** Obj 7.3. FlattenLayerStack collapses sublayers and **keeps references**. Flatten bakes references. Review: Ch 34.

**M2-028 — Answer: B.** Obj 1.2. `Open(..., mask=)` is not the 26.08 API. Use `OpenMasked + StagePopulationMask`. Review: Ch 17, Ch 44.

**M2-029 — Answer: B.** Obj 6.1. `DefinePrim` of a **new** child inside a live `ChangeBlock` can raise `Tf.ErrorException`. Review: Ch 34, Ch 44.

**M2-030 — Answer: B.** Obj 4.2. Verified: `from pxr import UsdMtlx` → **ImportError**. Mapping is still an exam topic; the plugin is not in this wheel. Review: Ch 27, Ch 40.

**M2-031 — Answer: B.** Obj 2.1. Verified: `[Bush, Rock, Tree]`. Bush prepended; Tree **appended**; default `AddTarget(Rock)` landed in the prepend list **after** Bush (not at front). Do not assume default is `FrontOfPrependList` on 26.08. Review: Ch 25.

**M2-032 — Answer: A, B.** Obj 1.6. Two-step: stack/list-ops first, then LIVERPS. Specializes are weakest (C false). Clips are not a LIVERPS letter (D). Review: Ch 21.

**M2-033 — Answer: B.** Obj 5.5. Verified: child `Get()` is **inherited**; `ComputeVisibility()` walks parents → **invisible**. Review: Ch 13.

**M2-034 — Answer: B.** Obj 3.1. Plugin ABI must match the USD the process loaded. Review: Ch 35, Ch 36.

**M2-035 — Answer: B.** Obj 7.6. `GetCustomDataByKey("show")`. `GetMetadata("show")` is not that dictionary. Review: Ch 12.

**M2-036 — Answer: B.** Obj 1.9. Verified: no defaultPrim → first root prim name (Ferry). Empty layer → `''`. Review: Ch 16, Ch 32.

**M2-037 — Answer: B.** Obj 8.3. `strongerThanDescendants` makes this prim's bind beat descendant binds (including subsets). `weakerThanDescendants` is the reverse. Review: Ch 40.

**M2-038 — Answer: A, B.** Obj 6.5. Coalescing delegate **before** Open; `USD_CHANGES` for change debug. `Tokens.Z` and `GetMaster` do not exist as diagnostic tools. Review: Ch 43.

**M2-039 — Answer: A, C.** Obj 4.4. Sanitize identifiers; keep the original in `customData`; author units/upAxis and document no auto-convert. `2ball` is **not** a legal prim name. Review: Ch 29.

**M2-040 — Answer: B.** Obj 1.1. Verified pattern: references beat specializes → composed 2. (Inherits would have won — that was Mock 1's different stem.) Review: Ch 19, Ch 21.

**M2-041 — Answer: B.** Obj 5.3. Documentation metadata vs `customData` dictionary. Review: Ch 12.

**M2-042 — Answer: B.** Obj 2.2. Instance-root primvars/inherits. Proxy `OverridePrim` raises. Editing the prototype tints **every** instance. Review: Ch 24.

**M2-043 — Answer: B.** Obj 7.7. `PXR_PLUGINPATH_NAME` is the env default; `Plug.Registry.RegisterPlugins` still works at runtime (Ch 36 labs). Review: Ch 36.

**M2-044 — Answer: B.** Obj 1.7. Verified: `GetNames()` → `['look']`; `GetVariantNames()` → `['green', 'red']`. Review: Ch 18.

**M2-045 — Answer: B.** Obj 6.4. Imageable purpose `guide` is hidden when the viewer mask is default+render only. Review: Ch 42.

**M2-046 — Answer: A, B.** Obj 4.8. `.usd` formatId follows the **extension**; bytes may still be USDA. formatId does not flip to `usda` just because the magic is `#usda`. Review: Ch 7, Ch 31.

**M2-047 — Answer: A, B.** Obj 8.2. Verified: enable **False**, Kelvin fallback **6500** (ignored until enable). Review: Ch 41.

**M2-048 — Answer: B.** Obj 3.8. Ar resolver  $\neq$  SceneIndex. Imaging plugins need an imaging build; usd-core has neither Hd nor UsdImaging. Review: Ch 38.

**M2-049 — Answer: A, B.** Obj 1.5. SetEditTarget / EditContext on a department layer **in the local stack**. Creating a second root that is not sublayered (C) and GetPrototype() (D) do not switch edit targets. Review: Ch 15.

**M2-050 — Answer: B.** Obj 5.2. Verified: linear from  $t=2$  (0,0,0) to  $t=8$  (6,0,0);  $t=4$  is 1/3 of the span  $\rightarrow$  (2, 0, 0). Review: Ch 10.

**M2-051 — Answer: B.** Obj 7.1. usd-core has **no** usdcat. Sdf.Layer.Export is the stand-in. Review: Ch 31.

**M2-052 — Answer: B, C.** Obj 2.4. Verified: IsA(Boundable) True, IsA(Gprim) **False**. Review: Ch 25.

**M2-053 — Answer: A, B.** Obj 1.11. Payload geo; department sublayers; pin publish. Flatten-as-daily-format throws composition away. Review: Ch 22.

**M2-054 — Answer: B.** Obj 4.5. Custom (namespaced) attributes persist without a downstream schema plugin. Review: Ch 29, Ch 37.

**M2-055 — Answer: B.** Obj 6.2. Same-field local opinions still rank by **layer-stack strength**. A stronger local hides a weaker over even if both are “local.” Review: Ch 21, Ch 42.

**M2-056 — Answer: A, B.** Obj 3.2. pip usd-core  $\neq$  a full imaging/MaterialX build. Review: Ch 35.

**M2-057 — Answer: B.** Obj 1.10. Instance-root (or de-instance that one root). No GetMaster. Review: Ch 24.

**M2-058 — Answer: B.** Obj 4.1. Bake/choose the working set; glTF has no LIVERPS/payloads/variants. Review: Ch 27.

**M2-059 — Answer: A, B.** Obj 7.8. Typed mapping + working-set load rules. No UsdMtlx, no GetMaster. Review: Ch 29, Ch 35.

**M2-060 — Answer: B.** Obj 5.5. Verified: AddTranslateOp authors xformOpOrder = ['xformOp:translate']. Review: Ch 39.

**M2-061 — Answer: B.** Obj 4.3. Verified: USDZ compress\_type **0** (ZIP\_STORED). Review: Ch 31.

**M2-062 — Answer: B.** Obj 2.5. ModelAPI draw modes are for **models** (component/assembly) as cheap viz. Review: Ch 23, Ch 44.

**M2-063 — Answer: B.** Obj 6.5. Malloc tagging often compiled out of the pip wheel (0 bytes). Use Trace. Review: Ch 43.

**M2-064 — Answer: A, B.** Obj 4.7. defaultPrim, metersPerUnit, upAxis. No Registry.Register in USDA, no GetMaster metadata. Review: Ch 28.

**M2-065 — Answer: B.** Obj 7.2. Compute walks the **closure**. Extract is one file. Review: Ch 32.

---

## Domain repair

If weak in	Re-read	Re-lab
COMP	Ch 14-22	Labs 13-21
DE	Ch 27-31	Labs 25-28
PD	Ch 32-35	Labs 29-30
DM	Ch 8-13	Labs 08-12
DBG	Ch 42-44	Labs 09, 30, 36



<b>If weak in</b>	<b>Re-read</b>	<b>Re-lab</b>
CA	Ch 23-26	Labs 22-24
VIS	Ch 39-41	Labs 33-35
CUST	Ch 36-38	Lab 31

# Mock Exam 3 — NCP-OUSD Practice Paper

**Original practice exam. Not actual NVIDIA exam content.**

Verified against **USD 26.08** (usd-core 26.8). Difficulty: **hard** (paper 3 of 3). Heavier USDA-reading and debugging. Mock 1/2 answers will mislead you on several stems.

Questions	65
Time	120 minutes
Blueprint mix	COMP 15 · DE 10 · PD 9 · DM 8 · DBG 7 · CA 7 · VIS 5 · CUST 4
Answers	mock-exams/mock03_answers.md — <b>do not open until you finish</b>

This book's Mock 3 readiness bar is  $\geq 80\%$  (52/65) and  $\geq 70\%$  in every domain. NVIDIA does not publish a passing score.

USDA is always multiline. Python stems are .norun.

**M3-001** · COMP · Obj 1.5 · Difficulty: Hard · Type: USDA-reading

Files s1.usda / s2.usda / s3.usda each over "Anvil" { double mass = N } with N = 1, 2, 3 respectively (multiline overs). Root:

```
#usda 1.0
(
 subLayers = [
 @./s1.usda@,
 @./s2.usda@,
 @./s3.usda@
]
)
def Xform "Anvil"
{
}
```

Composed mass?

A. 3 — last sublayer wins B. 1 — first listed sublayer is strongest C. 2 — the middle layer is the "active" edit target by default D. None — three overs of the same field is an error

**M3-002** · DM · Obj 5.5 · Difficulty: Hard · Type: Python-reading

Samples at 0 → (0,0,0) and 10 → (10,0,0). Both **held** and **linear**. attr.Get(99)?

A. Held (10,0,0); linear None B. Both (10, 0, 0) — after the last sample, both modes hold the last value C. Both (0, 0, 0) D. Linear (99, 0, 0)

**M3-003** · DE · Obj 4.5 · Difficulty: Hard · Type: Single choice

UsdLux on 26.08: DistantLight.GetIntensityAttr().GetName() is:

A. intensity B. inputs:intensity (UsdLux attributes are namespaced) C. inputs:intensity:value D. lux:intensity

**M3-004** · COMP · Obj 1.1 · Difficulty: Hard · Type: USDA-reading

```
#usda 1.0
class "_H"
{
 double heat = 8
}

def Sphere "Forge" (
 prepend inherits = </_H>
 prepend variantSets = "fire"
 variants = {
 string fire = "hot"
 }
)
```

```
{
 variantSet "fire" = {
 "hot" {
 double heat = 2
 }
 "cold" {
 double heat = 0
 }
 }
}
```

Composed heat?

A. 2 — the selected variant beats inherits B. 8 — **Inherits beat VariantSets** C. 0 — last variant block D. 10 — sum

**M3-005** · CA · Obj 2.4 · Difficulty: Hard · Type: USDA-reading

```
#usda 1.0
def Xform "Proto"
{
 def Cube "Body"
 {
 }
}

def Xform "Vise" (
 instanceable = true
 prepend references = </Proto>
)
{
}
```

After `prim.SetInstanceable(False)` on `/Vise`:

A. Still an instance; `GetPrototype()` still returns `/__Prototype_1` B. `IsInstance()` is **False**; `/Vise/Body` is defined and **not** an instance proxy C. `/Vise/Body` disappears D. `SetInstanceable` raises on 26.08

**M3-006** · PD · Obj 7.5 · Difficulty: Hard · Type: Multiple select Select two.

`stage.MuteLayer(s1_identifier)` where `s1` was the strongest sublayer (`mass = 1`, weaker `mass = 2`):

A. Composed `mass` becomes 2 B. `GetLayerStack()` **omits** the muted layer C. `GetLayerStack()` still lists it, only `GetMutedLayers()` changes D. Mute deletes the file on disk

**M3-007** · VIS · Obj 8.4 · Difficulty: Hard · Type: Python-reading

A material's `PreviewSurface` shader is connected with

`CreateSurfaceOutput().ConnectToSource(shader.ConnectableAPI(), "surface").len(material.ComputeSurfaceSource())` and the first tuple element?

A. 1, the Material prim B. 3, and index 0 is the **connected UsdShade.Shader** C. 0, always empty until you flatten D. 2, shader id and material path

**M3-008** · DBG · Obj 6.5 · Difficulty: Hard · Type: Python-reading

```
listener records ObjectsChanged
stage.DefinePrim("/N")
then:
prim.CreateAttribute("f", Sdf.ValueTypeNames.Float).Set(1.0)
```

Resynced paths vs changed-info-only paths:

A. `DefinePrim`: resync `['/N']`; `Set`: info-only `['/N.f']` (no resync) B. Both operations only resync `/` C. `Set` resyncs `/N`; `DefinePrim` is info-only D. Notices never fire for in-memory stages

**M3-009** · COMP · Obj 1.1 · Difficulty: Hard · Type: USDA-reading

cls.usda (stronger sublayer) defines Forge with prepend inherits = </\_H> and class \_H { double heat = 8 }. Weaker sublayer localweak.usda:

```
#usda 1.0
over "Forge"
{
 double heat = 2
}
```

Composed heat?

A. 8 — the stronger sublayer's inherit wins B. 2 — a **local** opinion beats inherits even when it lives in a **weaker** sublayer C. None — mixed arcs on one field fail D. 5

**M3-010** · DM · Obj 5.6 · Difficulty: Hard · Type: Python-reading

Mesh points [(0,0,0), (2,0,0), (0,3,0)], one triangle (faceVertexCounts = [3], indices [0,1,2]). ComputeExtentFromPlugins returns:

A. [(0,0,0), (3,3,3)] B. [(0, 0, 0), (2, 3, 0)] C. None until extent is authored D. [(0,0,0), (2,0,0)] — Z is ignored

**M3-011** · DE · Obj 4.7 · Difficulty: Hard · Type: Single choice

A DCC writes custom float studio:lod = 2 on a Mesh. Downstream has **no** studio schema plugin. GetAttribute("studio:lod").Get()?

A. None — unknown namespaces are stripped B. 2 — custom attributes persist without a schema C. The layer fails to open D. Auto-converted to primvars:studio:lod

**M3-012** · COMP · Obj 1.8 · Difficulty: Hard · Type: USDA-reading

File: rig.usda (defaultPrim Mill) has /Mill/Rig/Arm Cube size = 1. Shot:

```
#usda 1.0
(
 relocates = {
 </World/Mill/Rig/Arm>: </World/Mill/Arm>
 }
)
def "Mill" (
 prepend references = @./rig.usda@
)
{
 over "Arm"
 {
 double size = 7
 }
}
```

What is true?

A. Relocates succeed; /Mill/Rig/Arm is gone; /Mill/Arm size 7 B. Relocates **source path does not match** the composed prim (/Mill/... not /World/Mill/...), so they are ignored; /Mill/Rig/Arm still exists (size 1) and /Mill/Arm is a **sibling over** (size 7) C. The layer fails to parse D. Relocates always prefix /World automatically

**M3-013** · PD · Obj 7.2 · Difficulty: Hard · Type: Single choice

GetPrimStack() on /Anvil in a root-with-two-sublayers stage. Order of layer display names (strongest first) is:

A. Weakest sublayer, then stronger, then root B. **Root, then stronger sublayer, then weaker** (root3.usda, s1.usda, s2.usda) C. Random D. Only the muted layers

**M3-014** · CUST · Obj 3.6 · Difficulty: Hard · Type: Single choice

A studio needs collision properties on existing meshes without a new Gprim type. 26.08:

A. `UsdPhysics.CollisionAPI.Apply(prim)` (API schema) B. `Kind.Registry.Register("collision")` C. Inherit `UsdPhysicsBase` in USDA D. Set `purpose = physics`

**M3-015** · CA · Obj 2.3 · Difficulty: Hard · Type: Multiple select Select two.

A `PointInstancer` has 4 instances; `invisibleIds = [0, 3]`. Which are true?

A. `GetInstanceCount()` is still 4 B. `GetInstanceCount()` becomes 2 C. Ids 0 and 3 are masked from drawing D. `protoIndices` is rewritten to length 2

**M3-016** · COMP · Obj 1.3 · Difficulty: Hard · Type: USDA-reading

```
#usda 1.0
def "Heavy" (
 prepend payload = @./geo.usda@
)
{
 over "Ball"
 {
 double radius = 5
 }
}
```

`geo.usda` defaultPrim G with child Sphere "Ball" radius = 1. Open with **LoadNone**. / Heavy/Ball?

A. Missing (bool False) — payloads do not compose B. **Present** with radius = 5 — the **local over** exists even while the payload is unloaded C. Present with radius = 1 — payload still loads D. Raises

**M3-017** · DM · Obj 5.1 / 8.1 · Difficulty: Hard · Type: Single choice

Vertex `primvars:displayColor` on a parent Xform; child Mesh has none. The call that finds the parent's primvar is:

A. `child.GetAttribute("displayColor")` B. `UsdGeom.PrimvarsAPI(child).FindPrimvarWithInheritance("displayColor")` C. `UsdShade.Material(child).GetDisplayColor()` D. Inheritance of primvars is forbidden

**M3-018** · VIS · Obj 8.2 · Difficulty: Hard · Type: Multiple select Select two.

UsdLux **namespacing** and APIs on a typed `DistantLight`:

A. Intensity is `inputs:intensity` B. `LightAPI` is already applied (HasAPI True) C. `ShadowAPI` is already applied D. Attribute name is bare `intensity` (no `inputs:`)

**M3-019** · DBG · Obj 6.3 · Difficulty: Hard · Type: Single choice

`two.usda` has root prims Alpha (k=1) and Beta (k=2), **no** defaultPrim. Shot: `prepend references = @./two.usda@</Beta>`. Composed `/Hero.k?`

A. None — missing defaultPrim always fails B. 2 — the **explicit prim path** selects Beta C. 1 — first root prim always wins D. `InvalidAssetPath`

**M3-020** · COMP · Obj 1.4 · Difficulty: Hard · Type: Single choice

Two crowd agents reference the same `walk.usda`. Agent A: `offset = 0`. Agent B: `offset = 4`, `scale = 1`. Sample stored at layer time 10 appears on B at composed time:

A. 10 B. 14 (`offset + scale * layerTime`) C. 6 D. 4

**M3-021** · DE · Obj 4.6 · Difficulty: Hard · Type: Multiple select Select two.

`ComputeAllDependencies` on a layer that sublayers `geo.usda` (resolves `wood.png`) and authors missing `@./gone.jpg@`:

A. `wood.png` is in the **assets** list B. `gone.jpg` is in the **unresolved** list C. Both files are unresolved D. `Compute` returns a single flat list of strings

---

**M3-022** · PD · Obj 7.4 · Difficulty: Hard · Type: Single choice

You `Set(1.0, 20)` with `GetEditTargetForLocalLayer(anim)` where `anim` is a sublayer with `offset = 10`. Sample in `anim.usda` is at:

A. 20 B. 10 C. 30 D. The Set is rejected

---

**M3-023** · CA · Obj 2.2 · Difficulty: Hard · Type: Single choice

Color **one** native instance of `/Vise` without de-instancing and without editing `/Prototype_1/Body`:

A. `OverridePrim("/Vise/Body")` then set `displayColor` B. Author `primvars:displayColor` on `/Vise` (instance root) C. Edit the prototype mesh — it only affects this instance D. `GetMaster().GetPrim().SetDisplayColor`

---

**M3-024** · COMP · Obj 1.7 · Difficulty: Hard · Type: USDA-reading

Asset `Tongs` has variant `set size` selected `sm (span = 1)` and variant `lg (span = 8)`. Shot:

```
#usda 1.0
def "Tongs" (
 prepend references = @./varasset.usda@
 variants = {
 string size = "lg"
 }
)
{
}
```

Composed `span`?

A. 1 — asset selection is stronger than the shot B. 8 — the **referencing prim's variant selection** is a stronger local opinion than the asset's selection C. None — two selections cancel D. 9

---

**M3-025** · DM · Obj 5.4 · Difficulty: Hard · Type: Multiple select Select two.

Relationship `/A.r` targets `/B.r`; `/B.r` targets `/C`. On 26.08:

A. `GetTargets()` → `['/B.r']` B. `GetForwardedTargets()` → `['/C']` C. `GetTargets()` already returns `/C` D. Forwarding requires `Kind.Registry`

---

**M3-026** · VIS · Obj 8.3 · Difficulty: Hard · Type: Single choice

A `UsdGeom.Subset` under a Mesh should receive its own material. Where is the binding authored?

A. Only on the Mesh — subsets cannot bind B. On the **Subset** prim (`MaterialBindingAPI Apply + Bind`), typically `weakerThanDescendants` on the parent if the parent also binds C. On the Material, targeting the subset via `GetMaster` D. Only via `PointInstancer invisibleIds`

---

**M3-027** · PD · Obj 7.3 · Difficulty: Hard · Type: Single choice

You must **bake references** (external parties cannot see your component files) but you are fine collapsing sublayers too:

A. `UsdUtils.FlattenLayerStack` — keeps references B. `stage.Flatten()` — bakes references C. `MuteLayer` on every reference D. `OpenMasked`

---

**M3-028** · COMP · Obj 1.9 · Difficulty: Hard · Type: Single choice

Preparing an internal mill asset for an external vendor. Which pair is the usual delivery?

A. Leave live hero:// resolver URIs and unpublished work layers B. **Localize** (or Flatten) so all needed layers/assets resolve, pin versions, set defaultPrim C. Ship only a .py that rebuilds the stage D. Strip metersPerUnit so the vendor's DCC guesses

---

**M3-029** · DBG · Obj 6.2 · Difficulty: Hard · Type: Single choice

GetPropertyStack on heat lists: weaker-sublayer local spec 2, then inherit node 8. Composed value is 2. Why is inherit listed at all?

A. Stacks are a bug B. Weaker opinions **remain in the stack** for debugging; LIVERPS already picked local C. Inherit actually wins; the stack is reverse order D. GetPropertyStack only lists the winner

---

**M3-030** · DE · Obj 4.2 · Difficulty: Hard · Type: Single choice

A mapping table lists UsdPreviewSurface metallic ↔ glTF metallicFactor ↔ a MaterialX metalness input. import pxr.UsdMtlx on this venv:

A. Required for the table to be valid B. **Fails** — the table is still the right artifact; the plugin is not in usd-core C. Registers the mapping automatically D. Converts glTF at import

---

**M3-031** · CA · Obj 2.4 · Difficulty: Hard · Type: Multiple select Select two.

Root Xform, Usd.ModelAPI(prim).SetKind("assembly") (no parent):

A. prim.IsGroup() is True B. prim.IsComponent() is True C. prim.IsComponent() is False — assembly is a group kind D. IsGroup() raises unless kind is exactly "group"

---

**M3-032** · COMP · Obj 1.6 · Difficulty: Hard · Type: Multiple select Select two.

Which LIVERPS comparisons match 26.08?

A. Inherits beat VariantSets (M3-004) B. Local beats Inherits even from a weaker sublayer (M3-009) C. Specializes beat References D. Relocates are not an arc; ignore the E

---

**M3-033** · DM · Obj 5.2 · Difficulty: Hard · Type: Multiple select Select two.

After CreateInMemory, then SetStageUpAxis(..., UsdGeom.Tokens.z) and SetStageMetersPerUnit(..., 1):

A. GetStageUpAxis is Z B. GetStageMetersPerUnit is 1.0 C. In-memory stages ignore Set\* and stay Y / 0.01 D. Tokens.z cannot be passed to SetStageUpAxis

---

**M3-034** · CUST · Obj 3.1 · Difficulty: Hard · Type: Single choice

plugInfo.json for a **codeless** schema plugin must include:

A. Only a Kind.Registry block B. Plugin metadata (SdfMetadata / schema plugin resources) so Plug.Registry can find schema.usda — parsing the USDA by hand is not registration C. A Hydra SceneIndex by default D. usdGenSchema output checked into site-packages of usd-core

---

**M3-035** · PD · Obj 7.6 · Difficulty: Hard · Type: Single choice

Store a non-rendered producer id heatId = "H-44" on /Forge:

A. kind = "H-44" B. customData key heatId C. upAxis D. primvars:displayColor

---

**M3-036** · COMP · Obj 1.2 · Difficulty: Hard · Type: Single choice

A mill yard has 50 identical vises (native instances) plus 20,000 bolts (PointInstancer). Why both styles?

A. They are identical; pick either B. **Native instances** share a prototype prim graph (per-instance root overrides); **PointInstancer** is for huge homogeneous point samples (proto index + transforms, not a full prim per bolt) C. PointInstancer requires `GetMaster` D. Native instances cannot be referenced assets

---

**M3-037** · VIS · Obj 8.3 · Difficulty: Hard · Type: Single choice

Parent Mesh bind `weakerThanDescendants`; child Subset binds a different material. The subset surface uses:

A. The parent material — parent always wins B. The **subset** material — `weakerThanDescendants` lets descendants win C. Neither D. Average of the two `diffuseColors`

---

**M3-038** · DBG · Obj 6.4 · Difficulty: Hard · Type: Single choice

A cube has `extent` still at the default empty/old box after points moved to `(0,0,0)–(2,3,0)`. Viewport bounds look wrong. First fix?

A. `Kind.Registry.GetAllKinds()` B. Recompute with `ComputeExtentFromPlugins` and `Set` on `extent` C. `Set purpose = proxy` D. `MuteLayer` the root

---

**M3-039** · DE · Obj 4.4 · Difficulty: Hard · Type: Multiple select Select two.

Round-trip of identifier `"Ember-2"`:

A. `Tf.MakeValidIdentifier("Ember-2")` → `"Ember_2"`; keep `"Ember-2"` in `customData` B. Hyphens are legal in prim names — skip sanitizing C. Author `defaultPrim` plus units/`upAxis` D. Drop payloads so the DCC always flattens

---

**M3-040** · COMP · Obj 1.1 · Difficulty: Hard · Type: USDA-reading

```
#usda 1.0
class "_Look"
{
 float roughness = 0.9
}

def "A" (
 prepend references = @./ingot.usda@
 prepend inherits = </_Look>
)
{
}

def "B" (
 prepend references = @./ingot.usda@
 prepend inherits = </_Look>
)
{
}
```

`ingot.usda` authors `roughness = 0.2`. Composed roughness on `/A` and `/B`?

A. `0.2` and `0.2` B. `0.9` and `0.9` — inherit **broadcasts** to every inheriting reference C. `0.9` and `0.2` — only the first inheritor wins D. None, None

---

**M3-041** · DM · Obj 5.3 · Difficulty: Hard · Type: Single choice

`GetCustomDataByKey("heatId")` vs `GetMetadata("comment")`:

A. Aliases B. `customData` is a dictionary of producer keys; `comment` is a separate metadata field C. `comment` is only legal on attributes D. Both register schemas

---

**M3-042** · CA · Obj 2.1 · Difficulty: Hard · Type: Single choice

To append a new PointInstancer prototype **without** shifting index 0 on 26.08:



A. `AddTarget(new, position=Usd.ListPositionFrontOfPrependList)` B. `AddTarget(new, position=Usd.ListPositionBackOfAppendList)` C. `AddTarget(new)` is guaranteed to prepend on 26.08 D. `SetTargets` is illegal

---

**M3-043** · PD · Obj 7.7 · Difficulty: Hard · Type: Multiple select Select two.

Validating `@./maps/rust.png@` authored in `assets/cleat.usda`:

A. `layer.ComputeAbsolutePath("./maps/rust.png")` (layer-relative) B. `Ar.GetResolver().Resolve("./maps/rust.png")` from a random cwd is sufficient C. Empty resolve + composed texture present ⇒ you used the **cwd** resolver, not the layer anchor D. `AnchorRelativePath` is required public API in 26.08

---

**M3-044** · COMP · Obj 1.7 · Difficulty: Hard · Type: Single choice

Empty variant selection; with `variantSet.GetVariantEditContext(): attr.Set(7)`. Where does 7 land?

A. Inside the first variant B. **Local** on the prim (not in any variant) C. Inside every variant D. The context raises

---

**M3-045** · DBG · Obj 6.1 · Difficulty: Hard · Type: Single choice

`DefinePrim("/Hearth/Flue")` (new child) inside an already-open `Sdf.ChangeBlock`:

A. Always succeeds B. Can raise `Tf.ErrorException` on 26.08 C. Converts specs to USDA D. Only fails if Flue is instanceable

---

**M3-046** · DE · Obj 4.8 · Difficulty: Hard · Type: Single choice

Importer sees crate magic PXR-USDC on `mill.usd`. `formatId` is:

A. `usda` B. `usd` (extension plugin) — magic is crate, id follows `.usd` C. `usdc` D. `usdz`

---

**M3-047** · VIS · Obj 8.4 · Difficulty: Hard · Type: Single choice

`PreviewSurface diffuseColor` should come from **vertex displayColor** (not a UV texture). Reader:

A. `UsdPrimvarReader_float2 / st` B. `UsdPrimvarReader_float3 / varname = displayColor` C. `UsdUVTexture / displayColor` D. `UsdMtlx standard_surface`

---

**M3-048** · CUST · Obj 3.7 · Difficulty: Hard · Type: Single choice

Shipping a Hydra **SceneIndex** that emits render geometry from this book's pip wheel:

A. Works: `import pxr.UsdImaging` B. **Blocked** — `UsdImaging / Hd` are not in `usd-core`; you need an imaging build C. Works via `Kind.Registry` D. Works via `Sdf.ChangeBlock`

---

**M3-049** · COMP · Obj 1.5 · Difficulty: Hard · Type: Multiple select Select two.

Session vs root: you `SetEditTarget(session)` then `radius.Set(9)` while root has 1.

A. Composed `Get()` is 9 (session is strongest in the local stack) B. Root layer still stores 1 C. Session writes are always discarded on `Save` of the root D. Session cannot hold attribute values

---

**M3-050** · DM · Obj 5.5 · Difficulty: Hard · Type: Single choice

Parent visibility `invisible`; child never authors visibility. `ComputeVisibility()` on the child is `invisible`. `GetVisibilityAttr().Get()` on the child is:

A. `invisible` B. `inherited` C. `None` D. `visible`

---

**M3-051** · PD · Obj 7.1 · Difficulty: Hard · Type: Single choice

Stand-in for `usdzip` in this `venv`:

A. The `usdzip` binary in `.venv/bin` B. `UsdUtils.CreateNewUsdzPackage` (STORE compression) C. `zipfile.ZIP_DEFLATED` of USDA files by hand (spec-legal) D. `usdcats --zip`

---

**M3-052** · CA · Obj 2.5 · Difficulty: Hard · Type: Single choice

Broken model hierarchy: un-kinded `/Clutter` with child `kind = component`. Child `IsComponent()` ?

A. True because `GetKind` is component B. **False** — parent is not a model C. True if the child is a Cube D. Raises

---

**M3-053** · COMP · Obj 1.11 · Difficulty: Hard · Type: Multiple select Select two.

Split `smelter.usda` (geo, look, rig) for three TDs:

A. Payload geo; sublayer look; sublayer rig; small interface with `defaultPrim` B. Pin published USDC per stream C. One root layer, no arcs, daily Flatten only D. Put kinds in `customData` instead of composition

---

**M3-054** · DE · Obj 4.3 · Difficulty: Hard · Type: Single choice

Layout layer merged in Git vs farm cache opened 10k times. Pair?

A. Git USDC / farm USDA B. Git USDA / farm USDC C. Both USDZ D. Git USDZ / farm USDA

---

**M3-055** · DBG · Obj 6.2 · Difficulty: Hard · Type: Single choice

Artist sets shot variant `fire = hot` (`heat = 2`) but composed heat is still 8 from an inherit class. Why?

A. Variants cannot be selected on a referencing prim B. **Inherits beat VariantSets** — the class opinion wins until you override with **local** (or remove the inherit) C. Variant selections never affect doubles D. Inherit is weaker than variants

---

**M3-056** · CUST · Obj 3.5 · Difficulty: Hard · Type: Multiple select Select two.

`hero://mill/anvil` custom resolver:

A. Ar plugin + `plugInfo.json` + plugin path B. `Ar.GetResolver()` stays the process facade C. `AnchorRelativePath` must be implemented (26.08 public API) D. Register the resolver with `Kind.Registry.Register`

---

**M3-057** · COMP · Obj 1.10 · Difficulty: Hard · Type: Single choice

Remove `primvars:displayColor` from **one** instanced vise in an assembly:

A. OverridePrim on `/Vise/Body` B. Author a block/delete at the **instance root**, or de-instance that root C. `GetMaster().Clear()` D. Mute the prototype layer for everyone

---

**M3-058** · DE · Obj 4.1 · Difficulty: Hard · Type: Single choice

Exporting a payload-heavy mill to glTF. Fidelity note:

A. glTF will lazy-load USD payloads B. You must **load the working set and bake** (glTF has no payloads / LIVERPS) C. Keep `@./geo.usda@` paths inside glTF JSON D. `metersPerUnit` is copied as a required glTF extension always

---

**M3-059** · PD · Obj 7.8 · Difficulty: Hard · Type: Multiple select Select two.

DCC importer:

A. Map to typed prims / API schemas; keep unknowns as custom properties B. Honor payload vs reference (working set) C. `from pxr import UsdMtlx` is guaranteed D. `GetMaster()` for instances

---

**M3-060** · DM · Obj 5.2 · Difficulty: Hard · Type: Single choice

One triangle mesh: `faceVertexCounts` length and `faceVertexIndices` length?

- A. Counts 1, indices 3 B. Counts 3, indices 1 C. Counts 3, indices 3 D. Counts 1, indices 1

**M3-061** · DE · Obj 4.3 · Difficulty: Hard · Type: Single choice

USDZ from `CreateNewUsdzPackage` uses which ZIP compression?

- A. DEFLATED (8) B. STORED (0) C. BZIP2 D. Per-member mix

**M3-062** · CA · Obj 2.4 · Difficulty: Hard · Type: Multiple select Select two.

`UsdGeom.PointInstancer`.Define then:

- A. `prim.IsA(UsdGeom.Gprim)` is True B. `prim.IsA(UsdGeom.Gprim)` is False C. `prim.IsA(UsdGeom.Boundable)` is True D. `prim.IsA(UsdGeom.Boundable)` is False

**M3-063** · DBG · Obj 6.5 · Difficulty: Hard · Type: Single choice

Coalescing similar `Open` warnings: install `UsdUtils.CoalescingDiagnosticDelegate`

- A. After `Open` returns B. **Before** `Open` C. Only inside `ChangeBlock` D. Only if `MallocTag` is non-zero

**M3-064** · DE · Obj 4.7 · Difficulty: Hard · Type: Multiple select Select two.

Component exporter root-layer checklist:

- A. `defaultPrim` B. `metersPerUnit` and `upAxis` C. `Kind.Registry.Register` in the USDA header D. `GetMaster` metadata

**M3-065** · PD · Obj 7.2 · Difficulty: Hard · Type: Single choice

`ExtractExternalReferences` vs `ComputeAllDependencies`:

- A. They are aliases B. Extract = **one file** (`sublayers`, `refs+assets`, `payloads`); Compute = **closure** (`layers`, `assets`, `unresolved`) C. Extract walks the closure; Compute is one file D. Only Extract reports unresolved textures

*End of Mock Exam 3. 65 questions. Check `mock-exams/mock03_answers.md` only after you have finished.*

# Mock Exam 3 — Answer Key

**Original practice exam. Not actual NVIDIA exam content.**

Paper: `mock-exams/mock03_paper.md`. Verified against **USD 26.08** (`usd-core` 26.8).

Scoring: 1 point per question (multiple-select must match **all** correct letters). Total **65**.

This book's Mock 3 readiness bar is  $\geq 80\%$  (**52/65**) and  $\geq 70\%$  in every domain.

NVIDIA does not publish a passing score.

## Score sheet

Domain	Q IDs	/n	Your score	%
Composition (COMP)	001, 004, 009, 012, 016, 020, 024, 028, 032, 036, 040, 044, 049, 053, 057	/15		
Data Exchange (DE)	003, 011, 021, 030, 039, 046, 054, 058, 061, 064	/10		
Pipeline Development (PD)	006, 013, 022, 027, 035, 043, 051, 059, 065	/9		
Data Modeling (DM)	002, 010, 017, 025, 033, 041, 050, 060	/8		
Debugging (DBG)	008, 019, 029, 038, 045, 055, 063	/7		
Content Aggregation (CA)	005, 015, 023, 031, 042, 052, 062	/7		
Visualization (VIS)	007, 018, 026, 037, 047	/5		
Customizing USD (CUST)	014, 034, 048, 056	/4		
<b>Total</b>	M3-001-M3-065	<b>/ 65</b>		

Copy weakest domains into `00_MASTER_PLAN.md` §9. If any domain is under 70%, re-read that part before booking the exam.

## Answers

**M3-001 — Answer: B.** Obj 1.5. Verified: three sublayers, first in the list wins (`mass = 1`). Last is weakest. Review: Ch 15.

**M3-002 — Answer: B.** Obj 5.5. Verified: `Get(99)` is `(10, 0, 0)` for **both** held and linear — past the last sample, linear does not extrapolate. Review: Ch 10.

**M3-003 — Answer: B.** Obj 4.5. Verified: `GetIntensityAttr().GetName()` is `inputs:intensity`. Review: Ch 41.

**M3-004 — Answer: B.** Obj 1.1. Verified: inherit `8` beats selected variant `2`. Inherits `> VariantSets`. Review: Ch 19, Ch 21.

**M3-005 — Answer: B.** Obj 2.4. Verified: `SetInstanceable(False)` de-instances; `/Vise/ Body` is defined and not a proxy. Review: Ch 24.

**M3-006 — Answer: A, B.** Obj 7.5. Verified: mute strongest sublayer → composed `2`; `GetLayerStack()` **omits** the muted layer; `GetMutedLayers()` lists it. Review: Ch 15, Ch 42.

**M3-007 — Answer: B.** Obj 8.4. Verified: `ComputeSurfaceSource` length `3`; index `0` is the connected shader (`/M/PBR`). Review: Ch 40.

**M3-008 — Answer: A.** Obj 6.5. Verified: `DefinePrim` resyncs `/N`; attribute `Set` is **changed-info-only** `/N.f`. Review: Ch 34, Lab 30.

**M3-009 — Answer: B.** Obj 1.1. Verified: weaker-sublayer **local** 2 beats stronger-sublayer **inherit** 8. Local is the strongest *arc*, not “the strongest sublayer’s arc.” Review: Ch 21.

**M3-010 — Answer: B.** Obj 5.6. Verified: extent `[(0, 0, 0), (2, 3, 0)]`. Review: Ch 13.

**M3-011 — Answer: B.** Obj 4.5. Custom namespaced attributes persist without a downstream schema plugin. Review: Ch 29, Ch 37.

**M3-012 — Answer: B.** Obj 1.8. Verified: relocates keys must be **composed paths** (`</Mill/Rig/Arm>`). `</World/Mill/...>` is ignored. `/Mill/Rig/Arm` remains size 1; over “Arm” is a sibling at `/Mill/Arm` with 7. Review: Ch 20.

**M3-013 — Answer: B.** Obj 7.2. Verified: `GetPrimStack` strongest first: root, then `s1`, then `s2`. Review: Ch 22, Ch 42.

**M3-014 — Answer: A.** Obj 3.6. Verified: `UsdPhysics.CollisionAPI.Apply` exists on `usd-core` 26.8. There is no `UsdPhysicsBase / Kind.Register` shortcut. Review: Ch 35, Ch 37.

**M3-015 — Answer: A, C.** Obj 2.3. `invisibleIds` masks drawing; count stays 4. Review: Ch 25.

**M3-016 — Answer: B.** Obj 1.3. Verified: with a **local over** of `Ball`, `/Heavy/Ball` exists and `radius = 5` even under `LoadNone`. (Without that over, the payload child is missing.) Review: Ch 17, Ch 20.

**M3-017 — Answer: B.** Obj 5.1 / 8.1. `FindPrimvarWithInheritance`. Review: Ch 11.

**M3-018 — Answer: A, B.** Obj 8.2. Namespaced intensity; `LightAPI` built-in; `ShadowAPI` needs `Apply`. Review: Ch 41.

**M3-019 — Answer: B.** Obj 6.3. Verified: explicit `@file@</Beta>` composes `k = 2` even with no defaultPrim. (A nameless `@file@` would be `UnresolvedPrimPath / defaultPrim`.) Review: Ch 16.

**M3-020 — Answer: B.** Obj 1.4. `composed = offset + scale * layerTime`  $\rightarrow 4 + 1 \cdot 10 = 14$ . Review: Ch 10, Ch 16.

**M3-021 — Answer: A, B.** Obj 4.6. Verified pattern: resolved textures in **assets**; missing files in **unresolved**. Review: Ch 32.

**M3-022 — Answer: B.** Obj 7.4. Verified: `GetEditTargetForLocalLayer` maps stage `t=20` through offset 10  $\rightarrow$  stored at **10**. Review: Ch 15, Lab 19.

**M3-023 — Answer: B.** Obj 2.2. Instance-root primvars. Proxy `OverridePrim` raises. Prototype edits tint everyone. Review: Ch 24.

**M3-024 — Answer: B.** Obj 1.7. Verified: `shot variants = { size = “lg” }` yields `span = 8`, beating the asset’s `sm`. Review: Ch 18.

**M3-025 — Answer: A, B.** Obj 5.4. Verified: `GetTargets()` stays `/B.r`; `GetForwardedTargets()` walks to `/C`. Review: Ch 5.

**M3-026 — Answer: B.** Obj 8.3. Bind on the Subset; parent often `weakerThanDescendants`. Review: Ch 40.

**M3-027 — Answer: B.** Obj 7.3. Flatten **bakes** references (external delivery). `FlattenLayerStack` **keeps** them. Review: Ch 22, Ch 34.

**M3-028 — Answer: B.** Obj 1.9. Localize/flatten, pin, defaultPrim. Do not ship live `hero://` work URIs. Review: Ch 22, Ch 32.

**M3-029 — Answer: B.** Obj 6.2. The stack lists contributing specs **strongest first** for debugging; composition already chose local 2. Review: Ch 42.

**M3-030 — Answer: B.** Obj 4.2. `UsdMtlx` `ImportError` on `usd-core`; the mapping table is still required. Review: Ch 27.

**M3-031 — Answer: A, C.** Obj 2.4. Verified: assembly `IsGroup()` **True**, `IsComponent()` **False**. Review: Ch 6, Ch 23.

**M3-032 — Answer: A, B.** Obj 1.6.  $I > V$ ;  $L > I$  (even weak-stack local). Specializes lose to references. E is relocates. Review: Ch 21.

**M3-033 — Answer: A, B.** Obj 5.2. Verified: after Set, *upAxis Z*, *metersPerUnit 1.0\**. Review: Ch 2, Ch 13.

**M3-034 — Answer: B.** Obj 3.1. Plug registry + `schema.usda` resources; parse  $\neq$  register. Review: Ch 36, Ch 37.

**M3-035 — Answer: B.** Obj 7.6. `customData`. Review: Ch 12.

**M3-036 — Answer: B.** Obj 1.2. Native instances vs PointInstancer scale. Review: Ch 24, Ch 26.

**M3-037 — Answer: B.** Obj 8.3. `weakerThanDescendants`  $\rightarrow$  child/subset bind wins. (Mock 2 asked the inverse strength.) Review: Ch 40.

**M3-038 — Answer: B.** Obj 6.4. Stale extent after points change. Review: Ch 13, Ch 42.

**M3-039 — Answer: A, C.** Obj 4.4. Hyphens become underscores; keep original in `customData`; author defaultPrim and units. Review: Ch 29.

**M3-040 — Answer: B.** Obj 1.1. Verified: inherit broadcasts `0.9` to both `/A` and `/B`. Review: Ch 19.

**M3-041 — Answer: B.** Obj 5.3. `customData` vs comment/documentation fields. Review: Ch 12.

**M3-042 — Answer: B.** Obj 2.1. `BackOfAppendList` keeps index 0. Default `AddTarget` is **not** `FrontOfPrependList` on 26.08. Review: Ch 25.

**M3-043 — Answer: A, C.** Obj 7.7. Layer-relative check; cwd `Resolve("./...")` is the classic false negative. `AnchorRelativePath` is gone. Review: Ch 33.

**M3-044 — Answer: B.** Obj 1.7. Empty `GetVariantEditContext` authors **local**. Review: Ch 18.

**M3-045 — Answer: B.** Obj 6.1. New-child `DefinePrim` inside `ChangeBlock` can raise. Review: Ch 34, Ch 44.

**M3-046 — Answer: B.** Obj 4.8. Crate magic `PXR-USDC`, formatId **usd** for `.usd`. Review: Ch 7, Ch 31.

**M3-047 — Answer: B.** Obj 8.4. `UsdPrimvarReader_float3` + `displayColor`. Review: Ch 40.

**M3-048 — Answer: B.** Obj 3.7. No `UsdImaging/Hd` in `usd-core`. Review: Ch 38.

**M3-049 — Answer: A, B.** Obj 1.5. Verified: session 9 wins; root still has `1`. Review: Ch 2, Ch 15.

**M3-050 — Answer: B.** Obj 5.5. Get  $\rightarrow$  inherited; Compute  $\rightarrow$  invisible. Review: Ch 13.

**M3-051 — Answer: B.** Obj 7.1. No CLI `usdzip`; `CreateNewUsdzPackage` (STORE). Hand-zip with DEFLATE is **not** USDZ-legal. Review: Ch 31.

**M3-052 — Answer: B.** Obj 2.5. Broken model chain  $\rightarrow$  `IsComponent()` **False**. Review: Ch 6, Ch 23.

**M3-053 — Answer: A, B.** Obj 1.11. Payload + department sublayers + pin. Review: Ch 22.

**M3-054 — Answer: B.** Obj 4.3. Git USDA / farm USDC. Review: Ch 31.

**M3-055 — Answer: B.** Obj 6.2. Same fact as M3-004, debug framing: inherit hides the variant until you author **local**. Review: Ch 21, Ch 42.

**M3-056 — Answer: A, B.** Obj 3.5. Ar plugin + facade. No Registry.Register; no AnchorRelativePath. Review: Ch 33, Ch 38.

**M3-057 — Answer: B.** Obj 1.10. Instance root (or de-instance that root). Review: Ch 24.

**M3-058 — Answer: B.** Obj 4.1. Bake the working set for glTF. Review: Ch 27.

**M3-059 — Answer: A, B.** Obj 7.8. Typed mapping + payloads. No UsdMtlx/GetMaster. Review: Ch 35.

**M3-060 — Answer: A.** Obj 5.2. One triangle: counts length **1**, indices length **3**. Review: Ch 11, Ch 13.

**M3-061 — Answer: B.** Obj 4.3. ZIP\_STORED (0). Review: Ch 31.

**M3-062 — Answer: B, C.** Obj 2.4. Verified: not a Gprim; is Boundable. Review: Ch 25.

**M3-063 — Answer: B.** Obj 6.5. Delegate **before** Open. Review: Ch 43.

**M3-064 — Answer: A, B.** Obj 4.7. defaultPrim + units/upAxis. Review: Ch 28.

**M3-065 — Answer: B.** Obj 7.2. Extract = one file; Compute = closure. Review: Ch 32.

## Domain repair

If weak in	Re-read	Re-lab
COMP	Ch 14–22 (especially §21 LIVERPS puzzles)	Labs 13–21
DE	Ch 27–31	Labs 25–28
PD	Ch 32–35	Labs 29–30
DM	Ch 8–13	Labs 08–12
DBG	Ch 42–44	Labs 09, 30, 36
CA	Ch 23–26	Labs 22–24
VIS	Ch 39–41	Labs 33–35
CUST	Ch 36–38	Lab 31

If Mock 3 is under 80% overall or any domain is under 70%, add review days before booking. NVIDIA does not publish a passing score.

## Flashcards

## Flashcards

**Original study cards.** Not NVIDIA exam content. Cut on the table grid: Front / Back.  
Backs stay  $\leq 30$  words. Verified against USD 26.08.

Target:  $\geq 160$ . This file currently holds **FC-001-FC-180** (complete,  $\geq 160$ ).

### Fundamentals (FC-001-FC-020)

#	Front	Back
FC-001	What is OpenUSD besides a file type?	A scene-description system: layers, composition, schemas, APIs. Formats are one piece.
FC-002	Python package name?	<code>pxr</code> — from <code>pxr import Usd</code> .
FC-003	<code>Usd.GetVersion()</code> on this book?	<code>(0, 26, 8)</code> — OpenUSD 26.08 / <code>usd-core</code> 26.8.
FC-004	Stage vs layer?	Stage = composed live scene. Layer = one opinion container (file or anonymous).
FC-005	CreateInMemory fallbacks?	<code>upAxis Y</code> , <code>metersPerUnit 0.01</code> .
FC-006	<code>usdc</code> on <code>usd-core</code> ?	Missing. Use <code>ExportToString()</code> / <code>Layer.Export</code> .
FC-007	First-listed sublayer?	Stronger. Last-listed-wins is a CSS myth.
FC-008	Session vs root Save?	Session opinions are not written into the root file.
FC-009	<code>def</code> / <code>over</code> / <code>class</code> ?	Define / overlay (does not define alone) / abstract class.
FC-010	Attribute vs relationship?	Values vs prim/property targets.
FC-011	<code>primvars:</code> <code>xformOp:</code> <code>inputs:</code> ?	Property namespaces.
FC-012	Five model kinds?	model, group, assembly, component, subcomponent.
FC-013	Typed vs API schema?	IsA (Cube) vs Apply ( <code>MaterialBindingAPI</code> ).
FC-014	<code>.usda</code> / <code>.usdc</code> / <code>.usd</code> / <code>.usdz</code> ?	Text / crate / either (magic) / zip package.
FC-015	<code>CreateNew("x.usd")</code> magic + <code>formatId</code> ?	Magic PXR-USDC; <code>formatId usd</code> .
FC-016	USDA first line?	<code>#usda 1.0</code>
FC-017	<code>GetPrimAtPath</code> missing?	Invalid handle; <code>bool(prim)</code> False; does not raise.
FC-018	Pcp in the stack?	Composition (prim index / opinions).
FC-019	Hydra?	Imaging / render delegates — not in <code>usd-core</code> .
FC-020	<code>defaultPrim</code> lives where?	Layer metadata, not on each prim.

### Composition (FC-021-FC-060)

#	Front	Back
FC-021	LIVERPS?	Local, Inherits, VariantSets, rElocates, References, Payloads, Specializes.
FC-022	LIVERPS letter E?	rElocates. Older talks said LIVRPS.
FC-023	Local vs reference?	Local wins ( $L > R$ ).
FC-024	Inherit vs reference?	$I > R$ .
FC-025	Reference vs specialize?	$R > S$ . Specialize is weakest.
FC-026	Inherit vs variant?	$I > V$ .



#	Front	Back
FC-027	Local vs variant?	$L > V$ — “variant did nothing.”
FC-028	Prepend vs append refs?	Prepend is stronger.
FC-029	Explicit references = [...]?	Replaces weaker list-ops.
FC-030	Sublayer vs reference?	Sublayer: same namespace. Reference: bring in at a path.
FC-031	Payload vs reference?	Payload weaker and unloadable.
FC-032	LoadNone still loads?	References yes; payloads no.
FC-033	Missing defaultPrim error?	UnresolvedPrimPath. Missing file: InvalidAssetPath.
FC-034	Layer offset formula?	$\text{stageTime} = \text{offset} + \text{scale} * \text{layerTime}$ .
FC-035	Offset 10, samples 0 and 10?	Composed times 10 and 20.
FC-036	Defaults and offsets?	Offsets remap samples, not the default.
FC-037	GetEditTargetForLocalLayer offset 10, Set at 20?	Stores at layer time 10. Bare EditTarget stores 20.
FC-038	Anonymous edit target?	Tf.ErrorException — not in local LayerStack.
FC-039	When variants?	Named discrete options (look, LOD). Not continuous sliders.
FC-040	Empty GetVariantEditContext?	Writes <b>local</b> opinions; fill the variant only after SetVariantSelection.
FC-041	GetVariantNames vs GetNames?	Names on VariantSet; GetNames is on VariantSets.
FC-042	Flatten vs FlattenLayerStack?	Flatten bakes arcs. FLS keeps references, bakes sublayers.
FC-043	instanceable where?	On the <b>referencing</b> prim (instance root).
FC-044	Prototype API 26.08?	GetPrototype(). GetMaster is gone.
FC-045	Nested over on instance proxy?	Tf.ErrorException / ignored in USDA. De-instance or author the root.
FC-046	Traverse and instances?	Skips proxy children; roots still listed.
FC-047	PointInstancer hide one?	InvisId(id, time) / invisibleIds. Count unchanged.
FC-048	AddTarget default on stronger layer?	<b>Prepends</b> — shifts protoIndices. Use BackOfAppendList.
FC-049	PI: Gprim or Boundable?	Boundable, not Gprim.
FC-050	Mute a stronger sublayer?	Its opinions vanish; weaker wins.
FC-051	Relocate source over?	Ignored. Author at the <b>destination</b> .
FC-052	Open with a mask?	Use OpenMasked, not Open(path, mask=).
FC-053	Stronger default vs weaker samples?	Default wins at all times; samples [].
FC-054	Get() samples only, no default?	None.
FC-055	Work vs publish vs pin?	Edit work; pin immutable publish folders, not .../work/.
FC-056	ComputeAllDependencies 3-tuple?	(layers, assets, unresolved) recursive.
FC-057	ExtractExternalReferences?	One file: (sublayers, refs+assets, payloads).

#	Front	Back
FC-058	Ar.GetResolver() type?	Facade Resolver. Underlying often DefaultResolver.
FC-059	AnchorRelativePath 26.08?	Gone. CreateIdentifier + anchor, or ComputeAbsolutePath.
FC-060	ChangeBlock delays?	<b>Notices</b> , not writes. New-child DefinePrim can raise.

## Data Modeling (FC-061-FC-080)

#	Front	Back
FC-061	Mesh points type?	point3f[] — point role, not color3f.
FC-062	Texture filename type?	asset (Sdf.AssetPath) so Ar can resolve it.
FC-063	purpose / subdivisionScheme type?	token, not freeform string.
FC-064	Get() with samples only?	None. Defaults and samples are separate.
FC-065	Linear Get(6) samples 1→11 of 0...10?	(5,0,0). Held keeps (0,0,0).
FC-066	Default interpolation?	Usd.InterpolationTypeLinear.
FC-067	What is a primvar?	primvars: attribute plus interpolation metadata.
FC-068	Add displayColor how?	UsdGeom.PrimvarsAPI.CreatePrimvar.
FC-069	Per-face IDs interpolation?	uniform. Seams/UVs: faceVarying.
FC-070	Indexing vs interpolation?	Independent. Indexing is a palette.
FC-071	primvars:st namespace?	primvars. Mesh points namespace is empty.
FC-072	Child inherits parent displayColor?	FindPrimvarWithInheritance.
FC-073	Points changed — extent?	Does not auto-update. Compute then Set.
FC-074	ComputeExtentFromPlugins writes?	No — it <b>returns</b> ; you Set extent.
FC-075	Triangle extent (0,0,0)(1,0,0)(0,1,0)?	[(0,0,0), (1,1,0)].
FC-076	CAD looks “melted”?	Fallback subdivisionScheme is catmullClark. Set none.
FC-077	purpose = guide missing in view?	Default purpose pass omits guide/proxy.
FC-078	Visibility fallback?	inherited. ComputeVisibility walks parents.
FC-079	Cube size / Sphere radius fallbacks?	2.0 and 1.0.
FC-080	GetCustomDataByKey why?	GetCustomData() may inject userDocBrief.

## Data Exchange (FC-081-FC-095)

#	Front	Back
FC-081	USDA vs USDC trade-off?	Text: readable/mergeable. Crate: faster, smaller, not line-diff friendly.
FC-082	Detect .usd encoding?	Magic: #usda, PXR-USDC, or PK. formatId follows extension.
FC-083	MakeValidIdentifier("Crate Box")?	Crate_Box. "2ball" → _ball.

#	Front	Back
FC-084	USD auto-converts units on reference?	<b>No.</b> Scale factor <code>asset_mpu / shot_mpu</code> ; often <code>rotateX(-90)</code> .
FC-085	OBJ f 1 2 3 → indices?	[0, 1, 2] — OBJ is 1-based.
FC-086	100 cm point → meters?	(1,0,0) with <code>metersPerUnit = 1</code> .
FC-087	ComplianceChecker 26.08?	<b>Gone.</b> Use <code>pxr.UsdValidation</code> .
FC-088	Geom vs core StageMetadataChecker?	Geom: <code>upAxis/meters</code> . Core: <code>defaultPrim</code> .
FC-089	glTF faces?	Triangulate. UV: <code>v_gltf = 1 - v_usd</code> . Meters, +Y.
FC-090	PreviewSurface → glTF?	<code>pbrMetallicRoughness</code> .
FC-091	USDA ↔ USDC lossless?	Yes. USD → glTF is lossy (no live variants).
FC-092	Bind without Apply?	Writes <code>material:binding</code> ; <code>HasAPI False</code> .
FC-093	ComputeAllDependencies shape?	3-tuple: layers, assets, unresolved.
FC-094	UsdMtlx on usd-core?	<code>ImportError</code> . Need a full build.
FC-095	USDZ zip method here?	<code>STORE</code> , <code>compress_type 0</code> .

### Content Aggregation (FC-096-FC-110)

#	Front	Back
FC-096	Native instance prototype API?	<code>GetPrototype()</code> → <code>/__Prototype_N</code> . No <code>GetMaster</code> .
FC-097	Where <code>instanceable</code> ?	Referencing prim (instance root), not every mesh.
FC-098	Edit nested Seat on one instance?	De-instance that root, or primvar on the <b>root</b> . Not the proxy.
FC-099	Proxy <code>OverridePrim</code> ?	<code>Tf.ErrorException</code> .
FC-100	Traverse instance children?	Skipped. <code>GetPrimAtPath</code> still finds proxies.
FC-101	PointInstancer required fields?	<code>prototypes</code> , <code>protoIndices</code> , <code>positions</code> . <code>Count = len(indices)</code> .
FC-102	Hide one PI instance?	<code>InvisId(id, time)</code> . Count stays the same.
FC-103	<code>AddTarget</code> default (stronger layer)?	Prepends — shifts indices. Pass <code>BackOfAppendList</code> .
FC-104	PI Gprim?	No. It <b>is</b> Boundable.
FC-105	Prototypes under <code>class</code> ?	Traverse skips class descendants.
FC-106	Mug kind component, <code>IsComponent False</code> ?	Un-kind ancestor broke the model chain.
FC-107	assembly is-a?	<code>group</code> (and thus <code>model</code> ).
FC-108	Source Seat Set(3) on instances?	Broadcasts to every instance. Intended.
FC-109	Unique variant on one instance?	Often a <b>second</b> prototype.
FC-110	80k seats vs one booth?	PI or native instances for seats; ordinary ref for the booth.

### Visualization (FC-111-FC-120)

#	Front	Back
FC-111		No — Shader <code>info:id</code> string.

#	Front	Back
	UsdPreviewSurface is a prim type?	
FC-112	Bind sequence 26.08?	MaterialBindingAPI.Apply then Bind.
FC-113	ComputeSurfaceSource type?	Tuple length 3, not a single shader.
FC-114	PS auto-reads displayColor?	<b>No.</b> Connect UsdPrimvarReader_float3 (varname token displayColor).
FC-115	UV texture chain?	UsdPrimvarReader_float2 (st) → UsdUVTexture → PS.
FC-116	Direct bind relationship name?	material:binding.
FC-117	Shader input namespace?	inputs:diffuseColor.
FC-118	Binding purposes?	preview vs full (and all-purpose).
FC-119	Face-level material?	UsdGeom.Subset + binding API.
FC-120	ComputeBoundMaterial?	Tuple: Material + winning relationship.

## Pipeline Development (FC-121-FC-140)

#	Front	Back
FC-121	Work vs publish?	Work is still edited. Publish is an immutable snapshot shots <b>pin</b> .
FC-122	Pin .../work/chair.usda?	Wrong. Pin .../v017/chair.usda.
FC-123	Hero_v003 as defaultPrim?	Version belongs in the <b>folder</b> . Keep identifier stable.
FC-124	assetInfo['identifier']?	Catalog key — almost never rename per publish.
FC-125	Extract vs Compute?	Extract = one file. Compute = recursive snowball.
FC-126	Extract second list?	References <b>and</b> asset-valued attributes (textures).
FC-127	Non-empty unresolved?	Fail the publish.
FC-128	Unresolved path spelling?	Often absolutized — compare basenames.
FC-129	GetModelNameFromRootLayer?	Reads defaultPrim (falls back to a root prim if unset).
FC-130	Git vs layer stack?	Git versions work files. Stacks are not VCS.
FC-131	./tex Resolve from wrong cwd?	False. Composition anchors to the <b>layer</b> .
FC-132	Search path and ./?	./ is <b>not</b> a search-path lookup.
FC-133	@chair.usda@ no context?	ErrorInvalidAssetPath. Pass pathResolverContext.
FC-134	Custom resolver load?	pluginInfo.json + PXR_PLUGINPATH_NAME before init; match ABI.
FC-135	customData vs assetInfo?	Studio bag vs catalog fields (identifier, version).
FC-136	Flatten strips owner names?	<b>No.</b> Clear customData yourself. Absolutized paths leak.
FC-137	Artist asset paths with a studio resolver?	Identifiers/URIs, not /mnt/show/...
FC-138	Context-dependent tex/wood.png?	True. ./tex/wood.png is False.
FC-139	Exporter hooks when?	Before Save: defaultPrim, units, upAxis, kind, identifier.

#	Front	Back
FC-140	Plugin after import pxr?	Often <b>too late</b> . Set plugin path first.

## Debugging (FC-141-FC-165)

#	Front	Back
FC-141	ChangeBlock 3× create+set?	6 notices without; 1 inside a block (Sdf specs).
FC-142	Sphere.Define new child in block?	Can raise <code>Tf.ErrorException</code> . Use <code>Sdf.PrimSpec</code> .
FC-143	Unregister ObjectsChanged?	<code>listener.Revoke()</code> .
FC-144	Opinion not showing — first tool?	<code>GetPrimStack</code> / <code>GetPropertyStack</code> (strongest first).
FC-145	CreateAttribute then Set notices?	Resync <code>/W.a</code> , then info-only <code>/W.a</code> .
FC-146	Coalescing delegate class?	<code>UsdUtils.CoalescingDiagnosticDelegate</code> — construct <b>before</b> Open.
FC-147	<code>sourceFileName</code> in tests?	<code>os.path.basename</code> — may be a full path ( <code>stage.cpp</code> ).
FC-148	Trace vs MallocTag?	Trace = time. MallocTag = bytes.
FC-149	MallocTag bytes on usd-core?	<code>GetTotalBytes()</code> is <b>0</b> after Initialize.
FC-150	Trace default enabled?	False.
FC-151	Enable USD_CHANGES in-process?	<code>Tf.Debug.SetDebugSymbolsByName("USD_CHANGES", True)</code> .
FC-152	Process-start debug env?	<code>TF_DEBUG=USD_CHANGES</code> .
FC-153	Unknown TfDebug name?	Empty list, no exception.
FC-154	Missing file vs missing prim path?	<code>InvalidAssetPath</code> vs <code>UnresolvedPrimPath</code> .
FC-155	LoadNone vs OpenMasked?	LoadNone still loads <b>refs</b> . Mask omits subtrees.
FC-156	Stale extent visual?	Wrong culling/bounds after scaling points.
FC-157	Z-up asset in Y-up shot?	Sideways/tiny until rotateX + scale. USD does not convert.
FC-158	<code>USD_CHANGES</code> description?	"USD change processing".
FC-159	Initialize() False, IsInitialized True?	Normal on this wheel; still 0 bytes.
FC-160	Eight-step opinion debug?	Mute, stack, LIVERPS, variant, instance, relocate, edit target, usdview LayerStack.
FC-161	Relocate source opinion warning?	Invalid opinion at relocation source — ignored.
FC-162	PI InvisId needs?	<code>(id, time)</code> — time is required.
FC-163	Null prim IsDefined()?	Raises <code>RuntimeError</code> . Use <code>bool(prim)</code> / <code>IsValid()</code> .
FC-164	Tokens.Z?	Missing. Use <code>UsdGeom.Tokens.z</code> → "Z".

#	Front	Back
FC-165	DistantLight intensity fallback?	50000. Sphere light 1. Brightness $\propto$ intensity $\times 2^{\text{exposure}}$ .

### Customizing USD (FC-166-FC-180)

#	Front	Back
FC-166	Plugin vs DCC USD version?	Rebuild the plugin against <b>that</b> USD. ABI mismatch fails.
FC-167	Kind.Registry.Register?	<b>Does not exist</b> in Python 26.08. Use <code>plugInfo.json</code> .
FC-168	SetKind("door")?	GetKind is door; IsComponent becomes False.
FC-169	usdGenSchema on usd-core?	Not on PATH.
FC-170	Parse schema.usda registers Door?	<b>No</b> . Parse $\neq$ register. Needs a plugin.
FC-171	Codeless schema still needs?	Generated USDA + <code>plugInfo.json</code> . Skips C++ wrappers only.
FC-172	Cube registered fallback size?	2.0. IsConcrete True.
FC-173	MaterialBindingAPI schema kind?	Applied API — Apply then Bind.
FC-174	Custom kind when?	New taxonomy the five built-ins cannot express.
FC-175	Typed schema inherit in schema.usda?	<code>inherits = &lt;/Typed&gt;</code> . API: <code>&lt;/APISchemaBase&gt;</code> .
FC-176	GLOBAL libraryName?	Schema library name for usdGenSchema/plugins.
FC-177	SceneIndex on usd-core?	No Hd/UsdImaging. Needs an imaging build.
FC-178	Procedural AssetResolver (Obj 3.8)?	<code>Resolve/OpenAsset</code> returns generated bytes, not only disk files.
FC-179	Hide default plugins with PXR_PLUGINPATH?	Prepend studio <b>and</b> keep the USD prefix.
FC-180	PXR_BUILD_MATERIALX_PLUGIN?	CMake flag for a source build — not in usd-core.

Flashcards complete: FC-001-FC-180 ( $\geq 160$ ).

## Cheat sheets

# Cheat sheet — LIVERPS & value resolution

USD 26.08 (usd-core 26.8) · Obj 1.1, 1.6, 1.8 · Ch 14–22 · Day 7

Older spelling **LIVRPS** is the same order **without relocates**.

## Strength (strongest → weakest)

	Arc	Author	Beats
<b>L</b>	Local	Session, root, subLayers, nested over	I V E R P S
<b>I</b>	Inherits	prepend inherits = </_Class>	V E R P S
<b>V</b>	VariantSets	variantSet "lod" = { "hi" { } }	E R P S
<b>E</b>	rElocates	Layer metadata relocates = { </old>: </new> }	R P S
<b>R</b>	References	prepend references = @asset@	P S
<b>P</b>	Payloads	prepend payload = @heavy@ (keyword is singular)	S
<b>S</b>	Specializes	prepend specializes = </_Defaults>	—

L → I → V → E → R → P → S  
 stack class switch rename catalog lazy defaults

## Two-step rank (Obj 1.1)

**A. Inside one stack** (this is Local at the root node):

session → root → subLayers[0] (and its children) → subLayers[1] ...

**First listed sublayer is strongest** (not last-wins).

**B. Across arcs:** if the current stack is silent on this *field*, walk LIVERPS.

Each referenced/payloaded asset has its **own** nested LIVERPS walk.

Same-letter lists: prepend stronger than append. Letters never jump the queue.

## Verified 26.08 outcomes

Situation	Winner
Inherit 7 vs referenced 2	7 (I > R)
Specialize 7 vs referenced 2	2 (R > S)
Local 3 vs selected variant 11	3 (L > V)
Inherit 8 vs selected variant 2	8 (I > V)
Local 2 in a <b>weaker</b> sublayer vs inherit 8	2 (L still beats I)
Session 9 vs root 1	9
Mute strongest sublayer (6); weaker has 15	15; muted layer <b>drops</b> from GetLayerStack
Stronger <b>default</b> vs weaker time samples	Default; GetTimeSamples() → []
Samples only; Get() (no time)	<b>None</b> (not the first sample)
Relocate dest over vs source-path over	<b>Destination</b> ; source ignored
Relocate keys that are not composed paths	<b>Ignored</b> ; sibling over is not a move
Shot variants = { size = "lg" } vs asset sm	Shot selection (local)

## Value resolution order (one field, one time)

1. Authored **time sample** (or clip sample) at/near that time
2. Else authored **default**
3. Else **schema fallback** (Cube size → 2.0 even with no variant)

4. Else **nothing** (`Get()` → `None`; custom attrs are not valid)

`GetPropertyStack()` / `GetPrimStack()`: contributing specs, **strongest first**.

Silent layers do not appear. Weaker specs still list for debugging.

## Time, offsets, interpolation

`composedTime = offset + scale × layerTime`

`Sdf.LayerOffset(offset, scale)` on a reference or `subLayerOffsets`.

Call	Result (26.08)
Samples 0 and 10; <b>linear</b> <code>Get(4)</code>	<code>(4,0,0)</code>
Same; <b>held</b> <code>Get(4)</code>	<code>(0,0,0)</code> (holds previous)
<code>Get(99)</code> past last sample, linear or held	<b>last sample</b> (no extrapolate)
<code>GetEditTargetForLocalLayer</code> + sublayer <code>offset=10</code> ; <code>Set(..., 20)</code>	Stored at <b>t=10</b>
Bare <code>Usd.EditTarget(layer); Set(..., 20)</code>	Stored at <b>t=20</b>

## Debug “why doesn’t this opinion show?”

1. `GetPropertyStack` / `GetResolveInfo` / `Usd.PrimCompositionQuery`
2. Muted layer? Wrong **edit target**?
3. Stronger **letter** (inherit hiding a variant; reference hiding a specialize)?
4. Stronger **local** in a stronger sublayer?
5. Empty variant selection → opinions not composed (Cube size still shows fallback 2)?
6. Relocate source path mismatch? Instance proxy (`OverridePrim` raises)?
7. `LoadNone` skipped a **payload** (references still load)?

**Make it win:** author **local** on the composed path, or move the layer up the stack — do not expect specializes or variants to beat inherits.



# Cheat sheet — USDA syntax

USD 26.08 · Obj 4.3, 7.3 · Ch 7, 14-20 · Always **multiline** { } — one-line { double r = 1 } **does not parse**.

## File skeleton

```
#usda 1.0
(
 defaultPrim = "World"
 upAxis = "Z"
 metersPerUnit = 0.01
 subLayers = [
 @./layout.usda@,
 @./anim.usda@
]
 relocates = {
 </Bot/Rig/Arm>: </Bot/Arm>
 }
)

def Xform "World" (
 kind = "component"
)
{
 def Sphere "Ball"
 {
 double radius = 1
 color3f[] primvars:displayColor = [(1, 0, 0)]
 rel material:binding = </Looks/Paint>
 }
}
```

First line **must** be `#usda 1.0`. Layer metadata lives in the top ( ... ).  
`defaultPrim` is layer metadata, not a prim field.

## Specifiers

Keyword	Defines?	In Traverse?	Typical use
<code>def</code>	Yes (type may be '')	Yes	Create a prim
<code>over</code>	<b>No</b> alone	Only if something else defines	Overlay opinions
<code>class</code>	Abstract	Usually skipped in world Traverse	Inherit/specialize source

Typeless `def "Dock" → IsDefined() True`, type ''.

`over "Jetty" with no defining spec → IsDefined() False`, `bool(prim) True` (not null).

Missing path → `bool/IsValid` False; `IsDefined()` **raises**.

## Composition arcs (on the prim's ( ... ))

```
def "Hero" (
 prepend references = @./asset.usda@
 prepend payload = @./heavy.usda@
 prepend inherits = </_Look>
 prepend specializes = </_Defaults>
 instanceable = true
 prepend variantSets = "lod"
 variants = {
 string lod = "high"
 }
)
{
}
```

- Payload keyword is **payload**, not `payloads`. Python: `GetPayloads()`.
- `@file@` uses the target's `defaultPrim` unless you write `@file@</Prim>`.
- No `defaultPrim` and no prim path → `UnresolvedPrimPath (<defaultPrim>)`.
- Missing file → `InvalidAssetPath`.

## Variants

```
variantSet "fit" = {
 "tight" {
 double binWidth = 1
 }
 "loose" {
 double binWidth = 8
 }
}
```

No selection → custom attr `Get()` is `None` (not composed).

`GetVariantSets().GetNames()` → set names.

`GetVariantSet("fit").GetVariantNames()` → "tight", "loose".

Empty `GetVariantEditContext()` authors **local**, not inside a variant.

## List editing (same field, one spec)

USDA	Effect
<code>prepend references = @a@</code>	Front (stronger)
<code>append references = @b@</code>	Back (weaker)
<code>delete references = @a@</code>	Drop <b>a</b> from <b>weaker</b> layers
<code>references = [@a@, @b@]</code>	Explicit list (replaces)

Same-spec `delete` + `prepend` of the **same** item does **not** remove it. Delete from a **stronger** layer.

`subLayers` first item is strongest. Time offset: `@anim.usda@` (offset = 10; scale = 2).

## Properties

Kind	USDA	Python
Attribute	<code>double radius = 1</code>	<code>CreateAttribute / schema</code> <code>getter</code>
Time samples	<code>double r.timeSamples = { 0: 1, 10: 5 }</code>	<code>attr.Set(v, time)</code>
Relationship	<code>rel material:binding = &lt;/Mat&gt;</code>	<code>CreateRelationship</code>
Metadata	<code>customData = { string id = "x" }</code>	<code>SetCustomDataByKey</code>
API schema	<code>prepend apiSchemas = ["MaterialBindingAPI"]</code>	<code>MaterialBindingAPI.Apply</code>
Block	(authored none)	<code>attr.Block()</code>

Namespaces: `primvars:`, `xformOp:`, `inputs:` (UsdLux intensity is `inputs:intensity`).

## Paths and assets

Syntax	Meaning
<code>&lt;/World/Ball&gt;</code>	Absolute prim path
<code>.radius</code>	Property on the current prim
<code>@./chair.usda@</code>	Layer-relative asset (composition anchors to <b>this layer</b> )
<code>@chair.usda@</code>	Context-dependent; may need a resolver context

## Traps

- One-line `{ double x = 1 }` parse error — always break braces.
- `CreateNew("x.usd")` is crate (PXR-USDC); `formatId` still `usd`.
- USDA copied to `.usd` still **opens as text**; `formatId` stays `usd`.
- `rel material:binding` without `apiSchemas` → HasAPI **False**, `rel` **True**.
- Nested over "Seat" under `instanceable` is **ignored**.

- Relocates are **layer** metadata; keys must be **composed** paths.

## Cheat sheet — Python API (usd-core 26.8)

Usd.GetVersion() → (0, 26, 8). Package: from pxr import Usd.

**Gone / missing here:** CLI (usdcats, usdchecker, usdzip), UsdImaging, Hd, UsdMtlx, usdGenSchema, GetMaster, Kind.Registry.Register, AnchorRelativePath, UsdUtils.ComplianceChecker.

### Usd — stage and prims

Task	Call
New / open	Stage.CreateInMemory(), CreateNew(path), Open(path)
Mask	<b>OpenMasked(path, StagePopulationMask([...]))</b> — not Open(..., mask=)
Load rules	Open(path, Stage.LoadNone) — <b>refs still load</b> ; payloads deferred
Units	CreateInMemory → upAxis <b>Y</b> , mpu <b>0.01</b>
Set units	UsdGeom.SetStageUpAxis(s, UsdGeom.Tokens.z) · SetStageMetersPerUnit(s, 1)
Traverse	stage.Traverse(); missing path: bool(prim) False; IsDefined() <b>raises</b>
Prototype	prim.GetPrototype() ( <b>not</b> GetMaster)
Edit target	SetEditTarget(layer); GetEditTargetForLocalLayer(layer) maps offsets
Scoped edit	with Usd.EditContext(stage, layer):
Mute	MuteLayer(identifier) — layer <b>omitted</b> from GetLayerStack
Stacks	prim.GetPrimStack(), attr.GetPropertyStack() (strongest first)
Query	Usd.PrimCompositionQuery(prim)

Anonymous layer **not** in the local stack → SetEditTarget raises Tf.ErrorException.

### sdf — layers and specs

Task	Call
Open / export	Layer.FindOrOpen, Export, ExportToString (always USDA text)
Relocates	layer.relocates (layer metadata)
Offset	Sdf.LayerOffset(offset, scale) → offset + scale * layerTime
Batch	with Sdf.ChangeBlock(): — coalesces notices ( <b>6</b> → <b>1</b> in lab 09)
New child in block	DefinePrim of a <b>new</b> path can raise Tf.ErrorException
Types	Sdf.ValueTypeNames.Float, Color3fArray, Asset

### UsdGeom

Task	Call
Define	Xform.Define, Mesh.Define, Cube.Define, Sphere.Define
Extent	Boundable.ComputeExtentFromPlugins(mesh, time) then extent.Set
Primvars	PrimvarsAPI(prim).CreatePrimvar / FindPrimvarWithInheritance
Visibility	GetVisibilityAttr().Get() may be inherited; use ComputeVisibility()
Tokens	UsdGeom.Tokens.z ( <b>no</b> Tokens.Z)
Xform	AddTranslateOp() authors xformOpOrder
Imageable	purpose fallback default; viewer masks hide proxy/guide
PI	PointInstancer is <b>Boundable</b> , <b>not</b> Gprim
Subset	UsdGeom.Subset.Define

Mesh subdivisionScheme fallback: **catmullClark**. Sphere radius fallback 1; Cube size fallback 2.

### UsdShade / UsdLux

Task	Call
Bind	MaterialBindingAPI.Apply(prim) <b>then</b> .Bind(material)
USDA rel only	HasAPI <b>False</b> , HasRelationship True
Surface	material.ComputeSurfaceSource() → tuple <b>len 3</b>
displayColor	UsdPrimvarReader_float3, inputs:varname = displayColor
UV	UsdPrimvarReader_float2 (st) → UsdUVTexture
Distant intensity	fallback <b>50000</b> ; SphereLight <b>1</b> ; name inputs:intensity
Brightness	intensity * 2**exposure
LightAPI	built-in on typed lights (HasAPI True)
ShadowAPI / ShapingAPI	<b>Apply</b> required
Kelvin	enable fallback <b>False</b> ; temp fallback <b>6500</b>

No UsdMtlx in this wheel.

### Gf Vt Tf Kind

Module	Use
Gf	Gf.Vec3d, Gf.Vec3f, Gf.Matrix4d, Gf.Camera
Vt	Vt.Vec3fArray, Vt.IntArray (attribute arrays)
Tf	MakeValidIdentifier("2ball") → <b>_ball</b> ; "Pier-Dock" → <b>Pier_Dock</b>
Tf.Notice	Register(Usd.Notice.ObjectsChanged, cb, stage); listener.Revoke()
Kind.Registry	GetAllKinds, IsA, IsComponent, ... — <b>no Register</b>

Kinds: model → group → assembly / component; subcomponent.

Un-kinded parent → child GetKind() may be component but IsComponent() **False**.

Assembly: IsGroup() True, IsComponent() False.

### Ar UsdUtils UsdValidation

Task	Call
Facade	Ar.GetResolver() → type <b>Resolver</b>
Plugin	Ar.GetUnderlyingResolver() → <b>DefaultResolver</b>
Cwd trap	Resolve("./tex.png") uses <b>cwd</b> ; composition <b>anchors to the layer</b>
Layer path	layer.ComputeAbsolutePath("./tex.png")
Extract (one file)	ExtractExternalReferences → (sublayers, refs+assets, payloads)
Closure	ComputeAllDependencies → (layers, assets, unresolved)
Flatten	stage.Flatten() <b>bakes refs</b> ; FlattenLayerStack <b>keeps refs</b> , drops sublayers
USDZ	CreateNewUsdzPackage — ZIP <b>STORED (0)</b>
Model name	GetModelNameFromRootLayer — defaultPrim, else <b>first root prim</b> , else ''
Coalesce	CoalescingDiagnosticDelegate <b>before</b> Open
Validate	from pxr import UsdValidation — not ComplianceChecker
Localize	UsdUtils.LocalizeAsset

`PXR_PLUGINPATH_NAME` plus `Plug.Registry.RegisterPlugins(path)`.

## Notices / perf (Ch 43-44)

`DefinePrim` → **resync** paths. `Attribute Set` → **changed-info-only**.

`Trace Collector` works. `Tf.MallocTag.GetTotalBytes()` often **0** on this wheel.

# Cheat sheet — List editing

**USD 26.08** · Obj 1.1 · Ch 14.4 · Fields: references, payloads, inherits, specializes, `apiSchemas`, relationship targets, `subLayers` (order is stack strength).

A **list op** is either an **explicit** list or a set of **edits**. Stronger layers edit weaker lists instead of recopying them.

## USDA forms

Form	Meaning
<code>references = [@a@, @b@]</code>	<b>Explicit</b> whole list; weaker opinions discarded
<code>references = None</code>	Explicit <b>empty</b> — strips every weaker item
<code>prepend references = @c@</code>	Insert at <b>front</b> (stronger among same-letter arcs)
<code>append references = @d@</code>	Insert at <b>back</b> (weaker)
<code>delete references = @a@</code>	Remove <b>a</b> contributed by a <b>weaker</b> layer

Do not mix explicit `references =` and `prepend references =` in one spec — the parser keeps the **last** statement.

Legacy `add` / `reorder`: do not author new ones.

## Compose order (26.08)

1. Apply layers **weakest** → **strongest**.
2. Inside one spec: **delete**, then **prepend**, then **append**.
3. Prepend of an item already present **moves** it to the front.
4. Delete of a missing item is a no-op.
5. Same-spec delete + prepend of the **same** `@a@` does **not** drop **a**. Put delete on a **stronger** layer.

**Verified:** weaker `prepend @r1@ (radius=2)` + `append @r2@ (8)` → composed **2**. Stronger-layer `delete @r1@` → **8**.

<code>weakest : prepend [a, b]</code>	→ [a, b]
<code>stronger: delete [b]; append [c]</code>	→ [a, c]
<code>strongest: prepend [n]</code>	→ [n, a, c]    front = strongest arc
OR <code>explicit [x]</code>	→ [x]

`subLayers = [@paint@, @layout@]`: **paint** is strongest (first listed).

## Python (`Usd.ListPosition*`)

Call	Typical list-op
<code>GetReferences().AddReference(path)</code>	Prepend ( <code>FrontOfPrependList</code> ) — Ch 14
<code>RemoveReference(Sdf.Reference(...))</code>	delete
<code>SetReferences([...])</code>	Explicit
<code>ClearReferences()</code>	Clears <b>this layer's</b> opinion
Same pattern	<code>GetInherits</code> , <code>GetSpecializes</code> , <code>GetPayloads</code> , <code>AddTarget</code>

Positions: `FrontOfPrependList` · `BackOfPrependList` · `FrontOfAppendList` · `BackOfAppendList`.

## PointInstancer prototypes (verified 26.08)

Default `AddTarget("/PI/Tree")` after Bush was **[Bush, Tree]** — index 0 stayed Bush.

**Do not assume** default is `FrontOfPrependList` for `AddTarget`.

Goal	Call
New proto at <b>index 0</b> (shifts others)	<code>AddTarget(path, FrontOfPrependList)</code>

Goal	Call
New proto at <b>end</b> (index 0 safe)	AddTarget(path, BackOfAppendList)

protoIndices are integers into this list — shifting index 0 retargets every instance.

## Relationships

AddTarget / RemoveTarget / SetTargets / GetTargets().

GetForwardedTargets() walks rel-to-rel (/A.r → /B.r → /C yields /C).

## Exam moves (Obj 1.1)

Intent	List-op
Shot wins over asset ref	Local over (LIVERPS), not a weaker append
Drop one weaker ref	delete on a <b>stronger</b> layer
Replace the whole set	Explicit list on the shot
Keep asset additions flowing	Prefer prepend/append, not explicit copy



# Cheat sheet — Primvars & interpolation

USD 26.08 · Obj 5.1, 8.1, 8.4 · Ch 11, 40 · Namespace `primvars:<name>` (+ optional `primvars:<name>:indices`).

Factory: `UsdGeom.PrimvarsAPI`. Wrapper: `UsdGeom.Primvar`.

Built-in on Gprims: `primvars:displayColor`, `displayOpacity`.

## Interpolation (array length)

Mesh: **V** vertices, **F** faces, **C** corners (`sum(faceVertexCounts)`).

Token	Length	Maps to
<code>constant</code>	1	Whole prim
<code>uniform</code>	F	One per <b>face</b>
<code>varying</code>	~V	Surface-varying (smooth)
<code>vertex</code>	V	One per <b>point</b>
<code>faceVarying</code>	C	One per <b>face-corner</b> (seams)

Tokens: `UsdGeom.Tokens.constant` / `uniform` / `varying` / `vertex` / `faceVarying`.

USDA:

```
color3f[] primvars:displayColor = [(1, 0, 0)] (
 interpolation = "constant"
)
```

Need	Interpolation
Whole-object color	<code>constant</code>
Per-face ID	<code>uniform</code>
Shared vertex color	<code>vertex</code>
UV seams / hard corners	<code>faceVarying</code>

**Lengths must match.** Vertex-length array + `faceVarying` is a visual bug (Obj 5.5).

One triangle: `faceVertexCounts` len **1**, `faceVertexIndices` len **3**.

Two quads, 4 verts: vertex colors **4**; `faceVarying` st **8**.

## Indexed primvars

Values = palette; indices = which palette slot per element.

```
pv.Set([(1, 0, 0), (0, 1, 0)])
pv.SetIndices([0, 1, 0, 1])
IsIndexed() True – independent of interpolation
```

`elementSize`: entries per logical element (e.g. 3 floats packed).

## Inheritance

A primvar on an **ancestor Xform** applies to descendant meshes unless overridden.

```
UsdGeom.PrimvarsAPI(child).FindPrimvarWithInheritance("displayColor")
finds /Parent.primvars:displayColor – not child.GetAttribute("displayColor")
```

Instance **root** primvars can tint one native instance without de-instancing.

Authoring on an instance-**proxy** descendant (`OverridePrim("/I/Mesh")`) **raises**.

## Shading (Obj 8.4)

Want	Network
Vertex <code>displayColor</code> → PreviewSurface	<code>UsdPrimvarReader_float3</code> , inputs:varname = <code>displayColor</code>
UV texture	

Want	Network
	UsdPrimvarReader_float2 (st) → UsdUVTexture → diffuseColor

Roles: `color3f[]` for color, `texCoord2f[]` for st, `float[]` for scalars.

Do not store sRGB display color as anonymous `float3[]` if you mean a color role.

## Related visual bugs (Obj 5.5 / 5.6)

Symptom	Check
Bounds wrong after moving points	ComputeExtentFromPlugins then Set extent
Hidden in viewer	purpose vs viewer mask (proxy/guide)
Child invisible	Get() is inherited; ComputeVisibility() walks parents
Subdivision surprise	Mesh fallback <b>catmullClark</b>
Tiny / sideways asset	Stage metersPerUnit / upAxis (in-memory: <b>Y, 0.01</b> )

# Cheat sheet — Instancing decisions

USD 26.08 · Obj 1.2, 1.10, 2.1-2.5 · Ch 24-26 · `GetPrototype()` — `GetMaster` does not exist.

## Which style?

	None	Native (instanceable)	PointInstancer
Count	1-tens	tens~10k	thousands-millions
Prims / copy	Whole subtree	1 instance prim; subtree in prototype	<b>0</b> (arrays)
Path to a copy	Yes	Instance root yes; descendants are <b>proxies</b>	<b>No</b>
Per-copy edits	Anything	Root: xform, vis, variants, inherited primvars, root bind	Arrays / masks only
Unique internals	Yes	De-instance that one, or different variant → new prototype	Different proto index

unique internals? yes → NONE (or de-instance that copy)  
 need a prim path / variant per copy? yes → NATIVE  
 huge count, transform + primvar only → POINT INSTANCER  
 instance the ASSET, not each leaf mesh

Native prototype can contain a PointInstancer; PI prototypes can be native instances.

## Native instancing (Ch 24)

Task	Fact (26.08)
Enable	<code>instanceable = true / SetInstanceable(True)</code>
Live instance?	<code>IsInstance()</code> (flag <code>IsInstanceable()</code> is only the authored intent)
Prototype	<code>prim.GetPrototype()</code> → <code>/__Prototype_N</code> (not saved in the layer)
<code>Traverse()</code>	Lists the instance root; <b>skips</b> proxy descendants
<code>GetPrimAtPath("/I/Body")</code>	Valid <b>proxy</b> ; <code>IsInstanceProxy()</code> True
Nested over "Seat"	<b>Ignored</b> — new child not on the prototype
<code>OverridePrim("/I/Body")</code>	<b>Raises</b>
Tint one copy	Primvar / inherit / variant on the <b>instance root</b>
Tint all copies	Edit the prototype source (or shared class)
Remove a property on one	Instance-root block/delete, or <code>SetInstanceable(False)</code>
De-instance	<code>SetInstanceable(False)</code> → Body defined, not a proxy

Different variant selections → **more prototypes** (`len(stage.GetPrototypes())`).

Broken model chain: un-kindred parent + child `kind = component` → `GetKind()` component, `IsComponent()` **False**.

## PointInstancer (Ch 25)

Field	Role
<code>prototypes rel</code>	List of proto prims (list-op!)
<code>protoIndices</code>	int per instance
<code>positions / orientations / scales</code>	Per-instance xform
<code>invisibleIds</code>	Hide by <b>id</b> ; <code>GetInstanceCount()</code> <b>does not shrink</b>

`PointInstancer` is **Boundable**, not **Gprim**.

Adding a proto: use **explicit** `ListPosition`.

`FrontOfPrependList` → new proto is index **0** (all old `0`s retarget).

`BackOfAppendList` → append; index `0` safe.

Default `AddTarget` on 26.08 was **not** front-prepend — **measure**, don't guess.

## Draw modes & working set (Ch 26, 44)

Tool	Use
Payload	Deferred heavy geo; <code>LoadNone</code> skips payloads, <b>not</b> references
Local over of a payload child	Child <b>exists</b> even while unloaded
<code>OpenMasked</code>	Population mask ( <code>StagePopulationMask</code> )
ModelAPI <code>drawMode</code>	<code>cards</code> / <code>bounds</code> / ... on <b>models</b> (valid hierarchy)
<code>extentsHint</code>	Cached model box per purpose

## Exam traps

- Proxy edits. Nested overs. `GetMaster`.
- PI `AddTarget` shifting proto `0`. Hiding via deleting `protoIndices` instead of `invisibleIds`.
- Instancing every bolt as a native prim (use PI). Instancing a hero that needs unique internals (don't).

# Cheat sheet — Debugging toolkit

USD 26.08 · Obj 6.1–6.5 · Ch 8, 22, 33, 34, 42–44

Composition “errors” at Open are often **warnings**; the stage still opens. Catch **Tf.ErrorException**, not a bare **Exception**, for USD API/parse failures.

## Eight-step procedure (Obj 6.2)

1. `bool(prim) / IsValid()` — missing path is a **null** handle (`IsDefined()` **raises**).
2. `IsDefined()`, specifier `def` vs `over` vs `class`.
3. Load rules: `LoadNone` still loads **references**; payloads deferred. Local `over` of a payload child can exist while unloaded.
4. `GetPrimStack()` / `GetPropertyStack()` — **strongest first**; `[0].layer` won.
5. `GetResolveInfo()` — sample vs default vs fallback.
6. `Usd.PrimCompositionQuery` — which arcs contributed.
7. `stage.GetCompositionErrors()` / `Pcp` types (below).
8. Mute, edit target, variant selection, relocates path, instance proxy.

## Asset vs prim path (Obj 6.3)

Symptom	Type	Fix
File not found / unreadable	<code>InvalidAssetPath</code>	<code>@path@</code> , <code>cwd</code> vs <b>layer</b> anchor, resolver context
File opens; target prim missing	<code>UnresolvedPrimPath</code>	<code>defaultPrim</code> or <code>@file@&lt;/Prim&gt;</code>
Variant name not in the set	Selection ignored	<code>GetVariantNames()</code> ; opinions never apply
<code>Resolve("./tex.png")</code> empty; <code>usdview</code> shows it	<code>Cwd</code> vs <code>layer</code>	<code>layer.ComputeAbsolutePath</code>

`ComputeAllDependencies` → (`layers`, `assets`, `unresolved`).

## Visual (Obj 6.4 / 5.5)

Picture	Check
Bounds wrong after points	<code>ComputeExtentFromPlugins</code> then <code>extent.Set</code>
Present in <code>Traverse</code> , missing in viewer	<code>purpose</code> vs viewer mask ( <code>proxy/guide</code> )
Child gone	<code>GetVisibility = inherited</code> ; <code>ComputeVisibility()</code>
Tiny / axis-flipped	<code>metersPerUnit</code> / <code>upAxis</code> (in-memory <b>Y</b> , <b>0.01</b> )
Subdivision surprise	Mesh fallback <b>catmullClark</b>
Material not showing	Apply <code>MaterialBindingAPI</code> then <code>Bind</code> ; <code>purpose/strength</code>
One instance color	Instance <b>root</b> , not proxy ( <code>OverridePrim</code> raises)

## ChangeBlock & notices (Obj 6.1)

Pattern	Result (26.08)
<code>3× CreateAttribute().Set()</code>	<b>6</b> <code>ObjectsChanged</code> notices (lab 09)
Same work in <code>Sdf.ChangeBlock</code>	<b>1</b> notice
<code>DefinePrim</code> of a <b>new</b> child inside a live block	Can raise <code>Tf.ErrorException</code>
<code>DefinePrim("/N")</code>	<b>Resync</b> /N
<code>attr.Set(...)</code>	<b>Changed-info-only</b> /N.f

Anonymous layer not in the local stack → `SetEditTarget` raises.

## Diagnostics (Obj 6.5)

Tool	Use
<code>Tf.Status / Tf.Warn</code>	stderr; keep going
<code>Tf.RaiseCodingError</code>	<code>Tf.ErrorException</code>
<code>Tf.Debug / TF_DEBUG=</code>	e.g. <code>USD_CHANGES</code> , <code>PCP_PRIM_INDEX</code>
<code>UsdUtils.CoalescingDiagnosticDelegate</code>	Install <b>before</b> <code>Open</code>
<code>Usd.Trace.Collector / Reporter</code>	Timelines (works on this wheel)
<code>Tf.MallocTag.GetTotalBytes()</code>	Often <b>0</b> — tagging not in pip wheel
<code>GetCompositionErrors()</code>	Structured Pcp errors

```

stderr: Status / Warn
Python: raise Tf.ErrorException
Open: composition failures → warnings + Pcp objects

```

## Instancing / LIVERPS traps

- `GetPrototype` not `GetMaster`. Nested over ignored.
- Weaker-sublayer **local** still beats inherit. Inherit beats variant. Reference beats specialize.
- Relocate keys must be **composed** paths.
- Empty `GetVariantEditContext` authors **local**.

# Cheat sheet — File formats & CLI tools

USD 26.08 · Obj 4.3, 7.3 · Ch 7, 31 · This venv has **no** CLI binaries.

## Four encodings

Kind	Extension	Magic	formatId	Use
USDA	.usda	#usda 1.0	usda	Git, interfaces, diffs
USDC / crate	.usdc	PXR-USDC	usdc	Farm, heavy geo, publish
USD	.usd	crate <b>or</b> text	<b>usd</b> (follows <b>extension</b> )	Default CreateNew → crate
USDZ	.usdz	PK (zip)	usdz	Package: USD + images + audio

.usd is a **box**: id is `usd`; **bytes** decide crate vs text.

USDA copied to `text.usd` still **opens**; formatId stays `usd`.

Job	Encoding
Department merge in Git	USDA
Farm opens 10k times	USDC
External vendor drop	Localized / flattened + optional USDZ
usdcat --flatten when you meant crate	<b>Wrong</b> — that <b>bakes composition</b>

## USDZ rules

- Uncompressed zip: `compress_type 0` (ZIP\_STORED). DEFLATE is **not** spec-legal.
- `UsdUtils.CreateNewUsdzPackage("root.usda", "out.usdz")`.
- No nested .usdz, no arbitrary GUI zip, no Alembic as a USDZ member.
- Inside: asset paths stay `./tex.png` (resolved in-package).

## CLI (full OpenUSD builds) vs this book

```
usdcat in.usda -o out.usdc
usdcat in.usdc # USDA to stdout
usdcat --flatten shot.usda -o flat.usda
usdzip --asset root.usda out.usdz
usdchecker asset.usda
```

CLI	usd-core stand-in
usdcat -o	<code>Sdf.Layer.FindOrOpen(...).Export("out.usdc")</code>
usdcat print	<code>print(layer.ExportToString())</code>
usdcat --flatten	<code>stage.Flatten()</code> (bakes <b>refs</b> )
Keep refs, drop sublayers	<code>UsdUtils.FlattenLayerStack(stage)</code>
usdzip	<code>CreateNewUsdzPackage(STORE)</code>
usdchecker	<code>UsdValidation(ValidationRegistry, Validator)</code>
usdview	Not in wheel; LayerStack / purpose are still exam concepts

`UsdUtils.ComplianceChecker` is **gone**.

Two metadata checkers: `usdValidation:StageMetadataChecker` (defaultPrim) and `usdGeomValidators:StageMetadataChecker` (units / upAxis).

## Detect encoding

Read first bytes: `#usda` → text; `PXR-USDC` → crate; `PK` → zip.

Then `layer.GetFileFormat().formatId` for the **plugin** (often `usd` on `.usd`).

## Flatten vs convert

Call	Refs	Sublayers
Export to .usdc	kept	kept
Flatten()	<b>baked</b>	baked
FlattenLayerStack()	<b>kept</b>	collapsed

External delivery: localize or Flatten so hero:// work URIs do not leak (Obj 1.9).



# Cheat sheet — Schemas & kinds

USD 26.08 · Obj 3.3, 3.4, 3.6, 4.5, 5.4 · Ch 6, 23, 37, 38

## Typed vs API

	Typed (UsdTyped)	API (UsdAPISchemaBase)
USDA	def Cube "Box"	prepend apiSchemas = ["MaterialBindingAPI"]
Python	UsdGeom.Cube.Define	MaterialBindingAPI.Apply(prim) then use the API
typeName	Changes (Cube)	Unchanged
IsA / HasAPI	prim.IsA(UsdGeom.Cube)	prim.HasAPI(UsdShade.MaterialBindingAPI)
Use	New prim type / Gprim	Properties on <b>existing</b> prims

Physics properties on a Mesh → **API** (UsdPhysics.CollisionAPI.Apply), not a new typed Mesh.

Verified: CollisionAPI Apply works on this wheel.

**Apply then Bind:** USDA rel material:binding without apiSchemas → HasAPI **False**, rel True.

Built-in vs applied lights: DistantLight **HasAPI LightAPI** already; **ShadowAPI** / **ShapingAPI** need Apply.

Intensity name: inputs:intensity.

## Kind hierarchy (model TOC)

model	
group	
assembly	(aggregates models – IsGroup True, IsComponent False)
component	(publishable asset – IsComponent True if chain intact)
subcomponent	(inside a component – NOT a model)

Kind.Registry.IsA("assembly", "group") True.

IsA("subcomponent", "model") **False**.

Kind	Model?	Typical root
assembly	yes	Shot / set
component	yes	Published asset (Chair)
group	yes	Organize <i>inside</i> assemblies — not the published root
subcomponent	<b>no</b>	Wheel, lid, seat

**Broken chain:** un-kinded /Clutter + child kind = component → GetKind() is component, IsComponent() **False**.

Kind.Registry.Register **does not exist**. Custom kinds: kind plugin / plugInfo.json (Obj 3.3).

Python: GetAllKinds, IsA, IsComponent, IsAssembly, IsGroup, IsModel, HasKind.

ASWF: published roots are usually **component** or **assembly**.

## Custom schemas (Obj 3.4 / 4.5)

Path	When
custom float studio:lod = 2	One-off / exporter; <b>persists</b> without a plugin
API / typed schema	Many DCCs, fallbacks, docs, validation

schema.usda is the source. usdGenSchema generates code + plugInfo.json.

**Parse ≠ register.** Sdf.Layer.FindOrOpen("schema.usda") does not install types.

usdGenSchema is **not** on PATH in usd-core.

Codeless: skipCodeGeneration in customData; still needs a plugin on PXR\_PLUGINPATH\_NAME.

Codeful: rebuild against the **same ABI** the process loaded (Usd.GetVersion() → (0, 26, 8) here).

Plugin jobs (Ch 36–38): schema, **kind**, **Ar resolver**, file format, Hydra SceneIndex.

SceneIndex / UsdImaging / Hd / UsdMtlx: **not** in this wheel.

## ModelAPI extras

`UsdGeom.ModelAPI` draw modes (`cards`, `bounds`, ...) and `extentsHint` apply to **models** with a valid hierarchy — not every `gprim` / shapeless `over`.

# Cheat sheet — NCP-OUSD domains (one-pager)

**Original study aid.** Not NVIDIA exam content. Blueprint verified 2026-10-07 (re-checked 2026-10-08): **60-70 Q, 120 min, \$200**, English, Certiverse, valid 2 years. Passing score **not published**. This book's Mock 3 bar:  $\geq 80\%$  and  $\geq 70\%$  **per domain**.

## Weights (of 65)

#	Domain	Wt	Q	Chapters	First recall
1	Composition	23%	15	14-22	LIVERPS two-step; list-ops; offsets
2	Data Exchange	15%	10	27-31	USDA/USDC/USDZ; mapping; validate
3	Pipeline	14%	9	32-35	Work/publish/pin; Ar; Flatten vs FLS
4	Data Modeling	13%	8	8-13	Types, samples, primvars, extent
5	Debugging	11%	7	42-44	Stacks, Pcp errors, ChangeBlock, Trace
6	Content Aggregation	10%	7	23-26	Native vs PI; kinds; payloads
7	Visualization	8%	5	39-41	PreviewSurface, Bind, lights
8	Customizing USD	6%	4	36-38	Plugin ABI, schemas, Ar, SceneIndex

Cross-listed in the study guide (taught once):  $1.10 = 2.5$ ,  $4.3 = 7.3$ ,  $5.1 = 8.1$ ,  $1.8 = 6.2$ ,  $5.3 = 7.6$ ,  $4.8 = 7.8$ ,  $3.5 = 7.5$ .

## 120-minute pace (~1.8 min / Q)

Min	Do
0-5	Skim all stems; mark USDA-reading
5-90	Answer in order; flag $\leq 3$ hard
90-110	Flagged + USDA/Python-reading
110-120	Domain gaps; never leave blank

Composition first if you run short — it is 23%.

## Version landmines (26.08)

GetPrototype not GetMaster · OpenMasked not Open(mask=) · Apply then Bind · inputs:intensity · Distant **50000** / Sphere **1** · Tokens.z · no Kind.Registry.Register · no AnchorRelativePath · no ComplianceChecker / UsdMtlx / CLI in usd-core · USDZ **STORE** · LoadNone still loads **refs** · default AddTarget  $\neq$  guaranteed front-prepend.

## “If you see X, think Y”

Stem	Y
Two numbers, two arcs	Which <b>letter</b> is earlier?
Git vs farm	USDA vs USDC
Why isn't my over showing?	Stack + LIVERPS + mute + edit target
Missing file vs missing prim	InvalidAssetPath vs UnresolvedPrimPath
Hide PI instances	invisibleIds (count unchanged)
Tint one instance	Instance <b>root</b> , not proxy
New PI proto	Explicit ListPosition; watch index 0
Diffuse from mesh color	PrimvarReader_float3 + displayColor
Custom physics on Mesh	UsdAPISchemaBase / CollisionAPI
Plugin in a DCC	<b>Same ABI</b> as loaded USD

## **Day plan (this book)**

Days 1–4 foundations + DM · 5–7 composition · 8 CA · 9 DE · 10 PD · 11 CUST+VIS · 12 Mock 1 · 13 Mock 2 · 14 Mock 3 + cheat sheets only.

# Final certification checklist

**Original study aid.** Not NVIDIA exam content. Tick every objective before you book. Chapters follow this book's TOC. Exam facts: 60-70 questions, **120 minutes**, \$200, English, Certiverse remote proctor, valid 2 years. NVIDIA does **not** publish a passing score. This book: Mock 3  $\geq 80\%$  and  $\geq 70\%$  in every domain.

Verified against study guide v1.1.0 and the NVIDIA certification page (2026-10-07 / 2026-10-08). Recheck logistics the week you sit.

## Exam-day logistics

- ☐ Government photo ID matching the Certiverse registration name
- ☐ System check / proctoring app installed and tested
- ☐ Quiet private room; clear desk; no second monitors or notes unless the current policy allows them
- ☐ Stable internet; charger; 120 uninterrupted minutes
- ☐ Read current NVIDIA + proctor FAQs (policies change)
- ☐ Pace plan: first pass by minute 90; ~1.8 min/question; composition (23%) not left for last
- ☐ Scratch: LIVERPS letters + layer order on USDA stems

## 1. Composition — 23% (Ch 14-22)

- ☐ **1.1** Change the strength of an opinion — Ch 14, 19, 21
- ☐ **1.2** Choose instancing style by scale — Ch 22, 26
- ☐ **1.3** References vs payloads vs sublayers — Ch 3, 15-17
- ☐ **1.4** Same animation, different layer offsets — Ch 10, 16
- ☐ **1.5** Multi-user layering / edit targets — Ch 15, 22
- ☐ **1.6** Explain LIVERPS (and LIVRPS) — Ch 14, 21
- ☐ **1.7** When variants are / are not appropriate — Ch 18
- ☐ **1.8** Why an opinion does not take effect — Ch 20-22, 42
- ☐ **1.9** Prepare an asset for external delivery — Ch 22, 34
- ☐ **1.10** Remove properties on instanced components — Ch 22, 24
- ☐ **1.11** Split a monolithic asset into workstreams — Ch 22

## 2. Content Aggregation — 10% (Ch 23-26)

- ☐ **2.1** Add a PointInstancer prototype — Ch 25
- ☐ **2.2** Tint an instance without breaking instancing — Ch 24
- ☐ **2.3** Hide PI instances efficiently (`invisibleIds`) — Ch 25
- ☐ **2.4** Native instances for large-scene reuse — Ch 23, 24, 26
- ☐ **2.5** Remove properties on instanced components — Ch 24

## 3. Customizing USD — 6% (Ch 36-38)

- ☐ **3.1** Build a plugin against a given USD ABI — Ch 35, 36
- ☐ **3.2** Build USD from source with custom deps — Ch 35
- ☐ **3.3** Custom model kinds (plugin, not `Kind.Registry.Register`) — Ch 6, 38
- ☐ **3.4** Custom schemas (`schema.usda`; `parse`  $\neq$  `register`) — Ch 6, 37
- ☐ **3.5** Custom Ar resolver — Ch 33, 38
- ☐ **3.6** Schemas for nonstandard import/export data — Ch 37
- ☐ **3.7** Hydra SceneIndex (needs imaging build) — Ch 38
- ☐ **3.8** Resolver that generates in-memory prims — Ch 38

## 4. Data Exchange — 15% (Ch 27-31)

- [ ] 4.1 Convert to glTF / common formats; fidelity notes — Ch 27
- [ ] 4.2 Document USD ↔ other data model (e.g. MaterialX) — Ch 27
- [ ] 4.3 USDA vs USDC vs .usd vs USDZ — Ch 7, 31
- [ ] 4.4 DCC round-trip pipeline — Ch 27, 35
- [ ] 4.5 Nonstandard attributes/schemas on export — Ch 29, 37
- [ ] 4.6 Validator (UsdValidation; no ComplianceChecker) — Ch 30
- [ ] 4.7 Write an exporter / converter to USD — Ch 28, 29
- [ ] 4.8 Write or extend a DCC importer — Ch 29, 35

## 5. Data Modeling — 13% (Ch 8-13)

- [ ] 5.1 Add a primvar to a mesh — Ch 11
- [ ] 5.2 Choose value types / time samples — Ch 9, 10
- [ ] 5.3 Custom metadata / customData — Ch 5, 12
- [ ] 5.4 Retrieve prim properties — Ch 2, 4, 5, 8
- [ ] 5.5 Unexpected visual results — Ch 13, 42
- [ ] 5.6 Update extent after points — Ch 13

## 6. Debugging — 11% (Ch 42-44)

- [ ] 6.1 When Sdf.ChangeBlock helps (and the DefinePrim trap) — Ch 8, 34, 44
- [ ] 6.2 Why an opinion does not take effect — Ch 20, 42
- [ ] 6.3 Asset-path / resolver failures — Ch 33, 42
- [ ] 6.4 Unexpected visuals (purpose, extent, units) — Ch 13, 42, 44
- [ ] 6.5 TfDebug, delegates, Trace, MallocTag — Ch 43

## 7. Pipeline Development — 14% (Ch 32-35)

- [ ] 7.1 Convert to common 3D formats with fidelity — Ch 27
- [ ] 7.2 Document asset structure (work / publish / pin) — Ch 23, 32
- [ ] 7.3 USDA vs USDC trade-offs — Ch 7, 31
- [ ] 7.4 DCC round-trip — Ch 35
- [ ] 7.5 Custom resolvers in the pipeline — Ch 33
- [ ] 7.6 Custom metadata — Ch 12
- [ ] 7.7 Validate asset-path formatting — Ch 33
- [ ] 7.8 Extend a DCC importer — Ch 35

## 8. Visualization — 8% (Ch 39-41)

- [ ] 8.1 Add a primvar to a mesh — Ch 11, 39
- [ ] 8.2 Assign UsdPreviewSurface materials — Ch 40
- [ ] 8.3 Bind materials (Apply then Bind) — Ch 40
- [ ] 8.4 PreviewSurface that reads diffuse from a primvar — Ch 40

## Labs & papers

- [ ] Python labs 01-37 executed in .venv
- [ ] USDA workbook U-001-U-030
- [ ] Domain banks (430) — weakest domain reworked
- [ ] Flashcards FC-001-FC-180
- [ ] Mock 1 (≥ 70% overall) · Mock 2 (≥ 75%) · Mock 3 (≥ 80%, no domain < 70%)
- [ ] Cheat sheets on paper for the last evening (no new topics)

**26.08 API recap (tick if you can explain each)**

- ☐ `GetPrototype` / no `GetMaster`
- ☐ `OpenMasked` + `StagePopulationMask`
- ☐ `LoadNone` loads references
- ☐ `Kind.Registry` has no `Register`
- ☐ `Ar.GetResolver()` is a facade; no `AnchorRelativePath`
- ☐ `Flatten` bakes refs; `FlattenLayerStack` keeps them
- ☐ `USDZ ZIP_STORED` (0)
- ☐ `UsdValidation` imports; `ComplianceChecker` does not
- ☐ Distant intensity 50000; `inputs:intensity`
- ☐ `UsdGeom.Tokens.z` (not `.Z`)

## Back matter

---

### Glossary

One-line definitions for the NCP-OUSD study book. Verified against **USD 26.08** (`usd-core 26.8`) unless a note says otherwise. Chapter numbers point at this book's TOC. Older talks that say **LIVRPS** omit relocates; the rest of the order is unchanged.

**Count:** 167 terms.

---

#### A

**ABI** — Binary interface a plugin must match; a 24.11 plugin is not safe in a 26.8 process. *Ch 35.*

**Active** — Whether a prim participates in default traversal and imaging; `active = false` prunes children. *Ch 4.*

**API schema** — Schema you Apply (`UsdAPISchemaBase`); does not change `typeName` (e.g. `MaterialBindingAPI`). *Ch 6, 37.*

**Ar / asset resolution** — How `@path@` becomes bytes (`Ar.GetResolver()` is the **Resolver** facade). *Ch 33.*

**Assembly** — Model kind that aggregates other models; `IsGroup()` `True`, `IsComponent()` `False`. *Ch 6, 23.*

**Asset path** — `@./chair.usda@` (or a URI). Composition **anchors to the layer**; naive `Resolve("./...")` uses `cwd`. *Ch 33.*

**Attribute** — A typed value on a prim (double `radius`). Contrast relationship. *Ch 5.*

#### B

**Binding (material)** — Relationship `material:binding` from geometry to a `Material`. Apply `MaterialBindingAPI` then `Bind`. *Ch 40.*

**bindMaterialAs** — Binding strength: `strongerThanDescendants` or `weakerThanDescendants`. *Ch 40.*

**Block** — Explicit “no value” on an attribute so weaker opinions cannot contribute (`attr.Block()`). *Ch 5, 21.*

#### C

**ChangeBlock** — with `Sdf.ChangeBlock()`: batches edits so `ObjectsChanged` coalesces; new-child `DefinePrim` inside can raise. *Ch 8, 34, 44.*

**Class specifier** — `class prim`; abstract source for inherits/specializes. *Ch 4, 19.*

**Clip (value clip)** — Prim metadata that streams time samples from a sequence of files. *Ch 21.*

**Codeless schema** — Schema used from generated USDA + `plugInfo.json` without compiled C++ types. *Ch 37.*

**CoalescingDiagnosticDelegate** — Groups identical diagnostics; install **before** `Open`. *Ch 43.*

**Collection** — Path set used for light linking or collection-based material binds. *Ch 40, 41.*

**Component** — Publishable model kind (chair, mill). Broken parent chain → `IsComponent()` `False`. *Ch 6, 23.*

**Composition** — Combining layer opinions into one stage by **LIVRPS** and list-ops. *Ch 1, 14.*



**Composition error** — Pcp object on the stage (`GetCompositionErrors()`); often a **warning** at Open, not `ErrorException`. *Ch 42, 43.*

**Connectable** — Shade prim whose inputs/outputs can be wired (`ConnectableAPI`). *Ch 40.*

**Crate** — USDC binary layout; magic `PXR-USDC`. *Ch 7, 31.*

**customData** — Dictionary of producer key/values renderers should ignore. *Ch 12.*

## D

**Default (attribute)** — Untimed authored value. `Get()` with no time reads this — **None if only samples exist**. *Ch 10.*

**defaultPrim** — Layer metadata naming the prim `@file@` references when no path is given. *Ch 16.*

**DefaultResolver** — Usual underlying Ar plugin; `Ar.GetUnderlyingResolver()`, not `GetResolver()`. *Ch 33.*

**def** — Specifier that **defines** a prim (type may be empty). *Ch 4, 20.*

**Diagnostic delegate** — Object that receives Tf status/warnings/errors. *Ch 43.*

**DistantLight** — Infinite parallel rays; intensity fallback **50000**; name `inputs:intensity`. *Ch 41.*

**Draw mode** — `model:drawMode` stand-in (`bounds`, `cards`, ...) on **models**. *Ch 26, 44.*

## E

**Edit target** — Layer (or variant site) that new `Usd` edits write; must be in the local stack. *Ch 15.*

**EditContext** — `with Usd.EditContext(stage, layer):` scoped edit target that restores after. *Ch 15.*

**ElementSize** — How many array entries make one logical primvar element. *Ch 11.*

**ErrorException** — `Tf.ErrorException` raised for USD coding/runtime/parse errors. *Ch 43.*

**Exposure** — Light scale:  $\text{brightness} \propto \text{intensity} \times 2^{\text{exposure}}$ . *Ch 41.*

**Extent** — Axis-aligned box on boundables; recompute with `ComputeExtentFromPlugins` after points change. *Ch 13.*

**extentsHint** — Cached model-level boxes per purpose. *Ch 13, 26.*

**ExtractExternalReferences** — One-file 3-tuple (`sublayers`, `refs+assets`, `payloads`). *Ch 32.*

## F

**FaceVarying** — Primvar interpolation: one value per face-corner (UV seams). *Ch 11.*

**Fallback (schema)** — Built-in default used only when nothing authored a value (`Cube size`  $\rightarrow$  2.0). *Ch 6, 21.*

**File-format plugin** — `SdfFileFormat` registered for an extension. *Ch 7, 38.*

**Flatten** — `stage.Flatten()` bakes **references** (and `sublayers`) into one layer. *Ch 34.*

**FlattenLayerStack** — Collapses **sublayers**, **keeps** references. *Ch 34.*

**formatId** — Plugin id (`usda` / `usdc` / **`usd`** for `.usd` / `.usdz`); follows the **extension**, not always the magic. *Ch 7, 31.*

## G

**Gf** — Graphics foundations: `Vec3d`, `Matrix4d`, `Camera`. *Ch 9, 39.*

**Gprim** — Geometric primitive (`UsdGeomGprim`: Mesh, Cube, Sphere, ...). `PointInstancer` is **not** a Gprim. *Ch 6, 25.*

**Group (kind)** — Model kind used to organize insides of assemblies; not the usual published root. *Ch 6, 23.*

## H

**Held interpolation** — Time samples hold the previous value until the next; no blend. *Ch 10.*

**Hydra** — Imaging framework (scene index → render delegate). **Not** in the `usd-core` wheel. *Ch 38, 41.*

## I

**Imageable** — Schema for vis/purpose on things that can be drawn. *Ch 13, 39.*

**Inactive prim** — `active = false`; children pruned. *Ch 4, 42.*

**Indexed primvar** — Palette values plus an index array (`SetIndices`). Independent of interpolation. *Ch 11.*

**info:id** — Shader implementation token (`UsdPreviewSurface`, `UsdUVTexture`). *Ch 40.*

**Inherit** — Composition arc broadcasting class opinions; **beats variants, refs, payloads, specializes**. *Ch 19, 21.*

**Instance (native)** — Prim with `IsInstance()` True sharing a prototype. *Ch 24.*

**Instance proxy** — Descendant under an instance; `Traverse` skips it; `OverridePrim` **raises**. *Ch 24.*

**Instance root** — The instanceable prim itself — legal place for per-copy primvars/binds. *Ch 24.*

**instanceable** — Authored intent to share a prototype; `IsInstance()` is the live result. *Ch 24.*

**Interface input** — Material-level input shots can override without editing the shader graph. *Ch 40.*

**Interpolation (time)** — linear vs held between samples; past the last sample both **hold**. *Ch 10.*

**Interpolation (primvar)** — `constant` / `uniform` / `varying` / `vertex` / `faceVarying`. *Ch 11.*

**Invalid prim** — `GetPrimAtPath` miss: `bool/IsValid` False; `IsDefined()` **raises**. *Ch 4, 42.*

**InvalidAssetPath** — Composition error: the `@file@` could not be opened. *Ch 16, 42.*

## K

**Kind** — Model-hierarchy token (`component`, `assembly`, ...). `Kind.Registry.Register` **does not exist**. *Ch 6, 38.*

## L

**Layer** — One opinion container (file or anonymous). *Ch 3.*

**Layer offset** — `(offset, scale): composedTime = offset + scale × layerTime`. *Ch 10, 16.*

**Layer stack** — Session → root → sublayers (first listed **strongest**). *Ch 3, 15.*

**LightAPI** — Built into typed `UsdLux` lights (`HasAPI` True without `Apply`). *Ch 41.*

**Linear interpolation** — Blend between adjacent time samples. *Ch 10.*

**List op / list editing** — `prepend` / `append` / `delete` / explicit list on arcs, `apiSchemas`, relationships. *Ch 14.*

**LIVERPS** — Local, Inherits, VariantSets, rElocates, References, Payloads, Specializes (strongest → weakest). *Ch 14, 21.*

**Load rules** — LoadAll / LoadNone / per-path; **references still load** under LoadNone. *Ch 17, 44.*

**Local** — Opinions from the current node's layer stack (session, root, sublayers, nested over). Strongest LIVERPS letter. *Ch 21.*

**Localize** — Copy needed layers/assets into a self-contained tree (UsdUtils.LocalizeAsset). *Ch 22, 32.*

## M

**Magic bytes** — First bytes of a file: #usda 1.0, PXR-USDC, or PK (zip). *Ch 7, 31.*

**MallocTag** — Optional allocator tags; GetTotalBytes() often **0** on this pip wheel. *Ch 43.*

**Material** — UsdShade.Material container you bind to geometry. *Ch 40.*

**MaterialBindingAPI** — Applied API required for a legal bind (Apply then Bind). *Ch 40.*

**Metadata** — Non-attribute fields (documentation, comment, customData, layer upAxis). *Ch 5, 12.*

**metersPerUnit** — Layer/stage linear unit; **not auto-converted** on reference. In-memory fallback **0.01**. *Ch 2, 28.*

**Model hierarchy** — Kind tree used as a table of contents (assembly / component / group). *Ch 6, 23.*

**Muted layer** — MuteLayer(id); omitted from GetLayerStack. *Ch 15, 42.*

## N

**Namespace (prim)** — Parent/child path tree (/World/Ball). *Ch 4.*

**Namespace (property)** — Prefix such as primvars:, xformOp:, inputs:. *Ch 5, 40, 41.*

**Native instancing** — Shared prototype prim tree via instanceable = true. *Ch 24.*

**NodeGraph** — Container for connected shaders; Material derives from it. *Ch 40.*

**Null prim** — Invalid handle from a missing path. *Ch 4.*

## O

**ObjectsChanged** — Notice: **resync** paths (new prims) vs **changed-info-only** (attribute Set). *Ch 34.*

**OpenMasked** — Open with a StagePopulationMask; **not** Open(..., mask=). *Ch 17, 44.*

**Opinion** — One authored value for a field in one layer. *Ch 14.*

**over** — Specifier that overlays opinions; does **not** define a prim by itself. *Ch 4, 20.*

## P

**Package format** — Format that contains other files (USDZ); IsPackage(). *Ch 31, 38.*

**Payload** — Deferred composition arc; USDA keyword **payload** (singular). Weaker than references. *Ch 17.*

**Pcp** — Prim cache / composition engine that builds the prim index. *Ch 1, 22.*

**Pin** — Published identifier that still resolves the same bytes next month. *Ch 32.*

**Plugin** — Library or resource pack discovered via plugInfo.json / PXR\_PLUGINPATH\_NAME. *Ch 36.*

**PointInstancer** — Boundable (not Gprim) that stores instances as arrays. *Ch 25.*

**Population mask** — Which prim paths exist at all (OpenMasked). *Ch 17, 44.*

**Prim** — Named object on the stage (`Usd.Prim`). *Ch 4.*

**Prim path** — Address `/World/Ball` (`Sdf.Path`). *Ch 4.*

**Prim index** — Pcp’s composed graph of nodes (arcs + stacks) for one prim. *Ch 14, 22.*

**Prim stack** — Specs that contribute to a prim, strongest first (`GetPrimStack`). *Ch 22, 42.*

**Primvar** — Geometric variable in `primvars`: (color, st, IDs), often inherited. *Ch 11.*

**Property** — Attribute or relationship on a prim. *Ch 5.*

**Property stack** — Specs for one property, strongest first (`GetPropertyStack`). *Ch 22, 42.*

**Prototype** — Shared hidden tree `/__Prototype_N` for native instances (`GetPrototype`). *Ch 24.*

**protoIndices** — Per-instance ints into a `PointInstancer`’s `prototypes` list. *Ch 25.*

**Purpose (imageable)** — `default / render / proxy / guide`; viewer masks hide some. *Ch 13, 42.*

**Purpose (material)** — `preview, full, or all-purpose` `''`. *Ch 40.*

**pxr** — Python package: `from pxr import Usd`. *Ch 1.*

## R

**Reference** — Composition arc that always composes with the parent (not deferred). *Ch 16.*

**Relationship** — Property whose value is prim/property targets (`rel`). *Ch 5.*

**Relocates** — Layer metadata renaming composed paths; keys must match **composed** paths. *Ch 20.*

**Render delegate** — Hydra backend (`Storm, RenderMan, Arnold, ...`). *Ch 41.*

**ResolveInfo** — `attr.GetResolveInfo(time)`: which encoding won (`sample/default/fallback`). *Ch 21, 42.*

**Resource plugin** — `pluginInfo.json` + files, no shared library. *Ch 36.*

**Root layer** — The layer `Open/CreateNew` used as the stage’s main file. *Ch 2, 3.*

## S

**Scene description** — Data model of prims, properties, layers, and composition — not “just a file type.” *Ch 1.*

**Scene index** — Hydra graph node that generates/filters render prims; needs an imaging build. *Ch 38.*

**Schema** — Named contract for properties (`Cube, Mesh, MaterialBindingAPI`). *Ch 6.*

**schema.usda** — Source `usdGenSchema` reads; **parsing it does not register** types. *Ch 37.*

**Sdf** — Scene description foundations: layers, specs, value types (no composition). *Ch 8.*

**Session layer** — Strongest anonymous stack layer; not written into the root on `Save`. *Ch 2, 15.*

**Shader** — `UsdShade.Shader` node identified by `info.id`. *Ch 40.*

**ShadowAPI / ShapingAPI** — Applied `UsdLux` APIs (must `Apply`). *Ch 41.*

**Specializes** — Weakest `LIVERPS` arc; class of fallbacks that **lose to references**. *Ch 19, 21.*

**Specifier** — `def, over, or class`. *Ch 4.*

**SphereLight** — Local area light; intensity fallback **1**. *Ch 41.*

**Stage** — Composed live scene (`Usd.Stage`). *Ch 2.*

**StageMetadataChecker** — Two validators: core `defaultPrim` and geom units/upAxis. *Ch 30.*

**Sublayer** — Layer included in another layer’s stack (`subLayers`); first listed strongest. *Ch 3, 15.*

**Subcomponent** — Kind for pieces **inside** a component; **not** a model. *Ch 6.*

**Subset** — `UsdGeom.Subset` faces of a mesh; can hold its own material bind. *Ch 39, 40.*

## T

**Tf** — Foundation library: diagnostics, debug symbols, `MakeValidIdentifier`, notices. *Ch 9, 43.*

**TfDebug** — Named log switches (`USD_CHANGES`, `PCP_PRIM_INDEX`) via API or `TF_DEBUG`. *Ch 43.*

**Time sample** — Value authored at a `Usd.TimeCode`. *Ch 10.*

**Trace** — Instrumentation timer (`Trace.Collector` / `Reporter`). *Ch 43.*

**Typed schema** — `UsdTyped` / `IsA` prim type (`def Cube`). *Ch 6, 37.*

**Typeless prim** — `def/over` with empty `typeName`. *Ch 4, 20.*

## U

**Undefined prim** — `over` with no defining `def`; `IsDefined()` `False`, `bool` may still be `True`. *Ch 20, 42.*

**UnresolvedPrimPath** — File opened but target prim missing (often no `defaultPrim`). *Ch 16, 42.*

**upAxis** — Stage/layer up direction; in-memory fallback **Y**; token `UsdGeom.Tokens.z`. *Ch 2, 28.*

**USDA** — Text encoding; first line `#usda 1.0`; Git-friendly. *Ch 7.*

**USDC** — Crate binary; faster/smaller; poor line-diffs. *Ch 7, 31.*

**USD (.usd)** — Extension that may be crate **or** text; `CreateNew` writes crate; `formatId` `usd`. *Ch 7, 31.*

**USDZ** — Zip **package** (`STORE` / `compress_type 0`) of USD + images + audio. *Ch 31.*

**Usd** — Composed scene API (`Usd.Stage`, `Usd.Prim`) built on `Sdf` + `Pcp`. *Ch 8.*

**usdcats** — CLI convert/print; **not** in `usd-core`; stand-in `Layer.Export` / `ExportToString`. *Ch 31.*

**usdchecker** — CLI validator (`UsdValidation` since 26.03); **not** in `usd-core`. *Ch 30.*

**usdGenSchema** — Generates schema code/`plugInfo.json`; **not** on `PATH` in `usd-core`. *Ch 37.*

**UsdGeom** — Geometry schemas (`Xform`, `Mesh`, `Camera`, `Imageable`, `Boundable`). *Ch 39.*

**UsdLux** — Light schemas; attributes under `inputs:`. *Ch 41.*

**UsdPhysics** — Physics APIs (e.g. `CollisionAPI.Apply`); present on this wheel. *Ch 35.*

**UsdPreviewSurface** — Standard lookdev shader id. *Ch 40.*

**UsdShade** — Materials, shaders, connections, bindings. *Ch 40.*

**UsdUtils** — Pipeline helpers: `deps`, `flatten layer stack`, `USDZ`, `localize`, `coalescing delegate`. *Ch 32-34.*

**UsdValidation** — Validator framework (`ValidationRegistry`); `ComplianceChecker` is **gone**. *Ch 30.*

**usdview** — Reference viewer; not in `usd-core`. *Ch 41.*

**usdzip** — CLI packager; stand-in `CreateNewUsdzPackage`. *Ch 31.*

## V

**Value resolution** — Picking the composed value of one field at one time (samples → default → fallback). *Ch 21*.

**Variant set** — Named switch (`lod`, `color`) with named variants. *Ch 18*.

**Variant selection** — Which variant is chosen; empty selection → variant opinions not composed; empty `GetVariantEditContext` authors **local**. *Ch 18*.

**Vt** — Value-type arrays (`Vt_VEC3fArray`). *Ch 9*.

## W

**Warning (Tf)** — Non-fatal diagnostic (`Tf_WARN`); composition failures often arrive this way. *Ch 43*.

**Working set** — Payloads currently loaded for the task. *Ch 17, 26*.

**Work / publish / pin** — Editable work layers; immutable published cache; shot pins a versioned id. *Ch 32*.

## X

**Xform / Xformable** — Transform prim / schema; ops listed in `xformOpOrder`. *Ch 39*.

## Z

**ZIP\_STORED** — Uncompressed zip method (0) required for USDZ. *Ch 31*.

## Objective coverage matrix

Maps every NCP-OUSD study-guide objective (v1.1.0) to this book's chapter, labs, question bank, and a representative mock item. **Original practice material** — not NVIDIA exam content. Some objectives appear in two domains; the book teaches them once.

Obj	Domain	Chapter(s)	Labs	Bank	Mock (ex.)
1.1	COMP	14, 19, 21	18, 20	COMP	M1-009, M3-009
1.2	COMP	22, 26	23–24	COMP, CA	M1-044, M3-036
1.3	COMP	3, 15–17	05, 13, 15	COMP	M1-012, M3-016
1.4	COMP	10, 16	14	COMP	M1-016, M3-020
1.5	COMP	15, 22	05, 19	COMP	M1-004, M3-001
1.6	COMP	14, 21	20	COMP	M1-001, M3-032
1.7	COMP	18	16	COMP	M1-020, M3-024
1.8	COMP / DBG	20–22, 42	21	COMP, DBG	M1-036, M3-012
1.9	COMP	22, 34	28	COMP	M1-053, M3-028
1.10	COMP / CA	22, 24	23	COMP, CA	M1-062, M3-057
1.11	COMP	22	37	COMP	M1-057, M3-053
2.1	CA	25	24	CA	M1-031, M2-005
2.2	CA	24	23	CA	M1-052, M2-042
2.3	CA	25	24	CA	M1-015, M3-015
2.4	CA	23, 24, 26	22–23	CA	M1-005, M3-005
2.5	CA	24	23	CA	M1-062, M2-057
3.1	CUST	35, 36	31	CUST	M1-043, M2-034
3.2	CUST	35	—	CUST	M2-056
3.3	CUST	6, 38	32	CUST	M1-014
3.4	CUST	6, 37	31	CUST	M1-034, M2-014
3.5	CUST / PD	33, 38	29	CUST, PD	M1-056, M3-056
3.6	CUST	37	31	CUST	M3-014
3.7	CUST	38	—	CUST	M1-048, M3-048
3.8	CUST	38	—	CUST-017, 025, 026	M2-048
4.1	DE / PD	27	25	DE	M1-058, M3-058
4.2	DE	27	—	DE	M2-030, M3-030
4.3	DE / PD	7, 31	28	DE, PD	M1-003, M3-054
4.4	DE / PD	27, 35	25	DE	M1-039, M2-039
4.5	DE	29, 37	25	DE	M1-046, M3-011
4.6	DE	30	27	DE	M1-054, M2-011
4.7	DE	28, 29	25	DE	M1-021, M1-064
4.8	DE / PD	29, 35	—	DE, PD	M1-061, M2-046
5.1	DM / VIS	11	12	DM, VIS	M1-010, M2-010
5.2	DM	9, 10	06, 10	DM	M1-002, M3-002
5.3	DM / PD	5, 12	08	DM, PD	M1-041, M2-035
5.4	DM	2, 4, 5, 8	03, 07	DM	M1-025, M2-002

Obj	Domain	Chapter(s)	Labs	Bank	Mock (ex.)
5.5	DM / DBG	13, 42	11	DM, DBG	M1-060, M3-038
5.6	DM	13	11	DM	M1-017, M3-010
6.1	DBG	8, 34, 44	09, 30	DBG	M1-008, M1-029
6.2	DBG	20, 42	21	DBG	M1-055, M3-055
6.3	DBG	33, 42	29	DBG	M1-019, M2-019
6.4	DBG	13, 42, 44	36	DBG	M1-063, M2-045
6.5	DBG	43	36	DBG	M1-038, M3-008
7.1	PD	27	25	PD	M2-051
7.2	PD	23, 32	28, 37	PD	M1-006, M1-013
7.3	PD	7, 31	28	PD	M1-027, M2-027
7.4	PD	35	—	PD	M1-043, M2-022
7.5	PD	33	29	PD	M1-022, M2-006
7.6	PD	12	08	PD	M1-035
7.7	PD	33	29	PD	M1-051, M2-043
7.8	PD	35	—	PD	M1-065, M2-059
8.1	VIS	11, 39	12	VIS	M2-010
8.2	VIS	40	34, 35	VIS	M1-018, M3-018
8.3	VIS	40	34	VIS	M1-007, M3-037
8.4	VIS	40	34	VIS	M1-037, M3-007

Fundamentals (not scored): F1-F5, Ch 1-7; labs 01-04; qbank\_fundamentals.md; flashcards FC-001-020.

USDA workbook U-001-U-030 drills composition, instancing, kinds, and offsets. Flashcards FC-001-FC-180 cover all eight domains. Cheat sheets cs\_\*.md are the last-evening pass.



## References and further reading

Sources used to write this book. **Tier order:** NVIDIA exam materials, then OpenUSD docs/API, then the installed `usd-core` 26.8 runtime, then NVIDIA Learn OpenUSD / Omniverse, then ASWF / AOUSD, then community guides. Third-party “practice exam” banks were not used as sources. URLs accessed 2026-10-07 unless noted. Recheck the certification page before you book.

Short quotes ( $\leq$  two sentences) from these sources are attributed in chapters. Practice questions, mocks, and flashcards are **original**.

### Exam and blueprint

1. NVIDIA. *OpenUSD Development Professional Certification (NCP-OUUSD)*. <https://www.nvidia.com/en-us/learn/certification/openusd-development-professional/> — 60–70 questions, 120 minutes, US\$200, English, Certiverse, two-year validity. Weights re-checked 2026-10-08.
2. NVIDIA. *NVIDIA-Certified Professional: OpenUSD Development Exam Study Guide*, version 1.1.0 (doc 4379350, Oct 2025). PDF. Objectives 1.1–8.4 and reading lists. Sample questions were used **only** to learn format (single, select-N, USDA stems); none are reproduced.

### OpenUSD documentation and API

1. Pixar Animation Studios / AOUSD. *OpenUSD documentation* (release 26.08). <https://openusd.org/release/index.html>
2. *OpenUSD Glossary* (LIVERPS, layer offset, edit target, flatten, primvar, over). <https://openusd.org/release/glossary.html>
3. *USD Tutorials*. [https://openusd.org/release/tut\\_usd\\_tutorials.html](https://openusd.org/release/tut_usd_tutorials.html)
4. *OpenUSD C++ / Python API reference*. <https://openusd.org/release/api/index.html> — `UsdStage`, `UsdPrim`, `composition`, `SdfChangeBlock`, `PointInstancer`, `MaterialBindingAPI`, `UsdLux`, `Ar`, `Kind`, `Plug`.
5. *USD Toolset* (`usdcats`, `usdchecker`, `usdzip`, `usdview`, `usdresolve`, `usdtree`). <https://openusd.org/release/toolset.html> — this book’s pip `usd-core` ships **none** of these binaries; chapters give Python stand-ins.
6. *USD FAQ* (sublayers vs references, over vs typeless def, `.usd` detection). <https://openusd.org/release/usdfaq.html>
7. *USDZ File Format Specification* (v1.3). [https://openusd.org/release/spec\\_usdz.html](https://openusd.org/release/spec_usdz.html) — store-only zip.
8. *UsdPreviewSurface Specification*. [https://openusd.org/release/spec\\_usdpreviewsurface.html](https://openusd.org/release/spec_usdpreviewsurface.html)
9. *Maximizing USD Performance*. <https://openusd.org/release/maxperf.html>
10. *OpenUSD source and CHANGELOG*. <https://github.com/PixarAnimationStudios/OpenUSD> — latest release 26.08 (2026-07-20).
11. *usd-core on PyPI*. <https://pypi.org/project/usd-core/> — this book verified **26.8**.

### NVIDIA Learn OpenUSD and Omniverse

1. NVIDIA. *Learn OpenUSD*. <https://docs.nvidia.com/learn-openusd/latest/index.html> — curriculum that prepares for the certification (composition, kinds, aggregation, data exchange).

2. NVIDIA Omniverse. *USD documentation*. <https://docs.omniverse.nvidia.com/USD/latest/index.html>
3. NVIDIA. *Principles of Scalable Asset Structure in OpenUSD*. <https://docs.omniverse.nvidia.com/USD/latest/learn-openUSD/independent/asset-structure-principles.html>

## Industry guidelines

1. ASWF USD Working Group. *Asset Structure Guidelines*. <https://github.com/USD-wg/assets/blob/main/docs/asset-structure-guidelines.md>
2. Alliance for OpenUSD (AOUSD). <https://aousd.org>

## Community (tier 6 — never the sole source)

1. Cheller's *USD Survival Guide*. <https://lucascheller.github.io/VFX-UsdSurvivalGuide/>

## How this book cites them

In chapters: [S04] OpenUSD Glossary – LIVERPS. Never “see the link instead of an explanation.” Runtime disagreements are resolved by running `usd-core 26.8` (tier 3) and marking `> [!VERSION]`.